

Kairos / FinanceOS — Full Consolidated Architecture and Feature Encyclopedia

Generated and reviewed: 2026-09-21
This is the long-form reference for a finance professional, engineer, reviewer,
or future collaborator. It combines the operational architecture, feature-level
designs, agent contracts, implementation records, decisions and current status.
A section marked proposed , shadow , collecting , or blocked must not
be interpreted as an enabled trading behavior.

Reading guide

Start with the system overview and current-state chapters. Then use the feature encyclopedia for the detailed design of each feature area. Each included document retains its source path and original status language so historical proposals are not silently rewritten as production behavior.

Status vocabulary

- **Active/shipped:** code and required production schema/cron contract verified.
- **Paper-only:** may affect an isolated simulated book, never broker capital.
- **Shadow/measure-only:** records evidence but cannot affect score, sizing or orders.
- **Collecting:** infrastructure runs, but its evidence gate is not mature.
- **Blocked:** a named data, provenance, owner or broker prerequisite is missing.
- **Proposed:** design only; no implementation is implied.

Finance vocabulary

NAV means net asset value, IC means information coefficient, PIT means point-in-time, OOS means out-of-sample, MAE/MFE means maximum adverse/favorable excursion, and OHLCV means open/high/low/close/volume. A signal is a dated research decision, an entry is a buy fill, a position is open inventory, and a lot is an accounting slice of an entry or partial exit.

How to use this document

This book is deliberately comprehensive, but it is not meant to be read from line 1 to the end in one sitting. Use the path that matches your question:

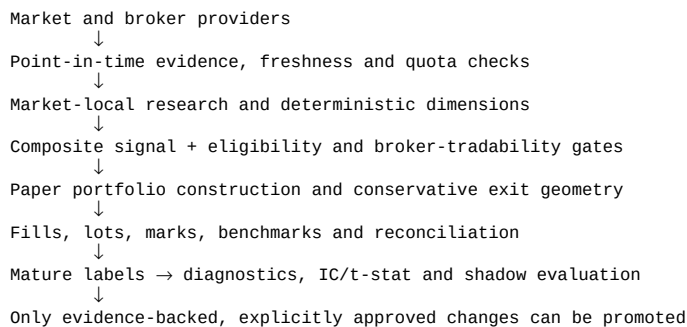
If you are a finance or trading reader

1. Read **Part I** for the one-page system model and the current US, India and crypto books.
2. Read the sections on **Research, Scoring, Portfolio construction, Exit geometry**, and **Benchmarks** in the current-system reference.
3. Use the feature entries in Part II to understand why a proposed change exists, what evidence it records, and what would be required before promotion.

If you are an engineer or reviewer

1. Start with the current-system reference, which is the present-state authority.
2. Follow the data path below from provider evidence to learning.
3. Use the source labels and status vocabulary to distinguish deployed behavior, paper behavior, shadow measurements, and architecture that is only proposed.
4. Treat historical source documents as decision records, not as an additional runtime specification.

The system in one picture

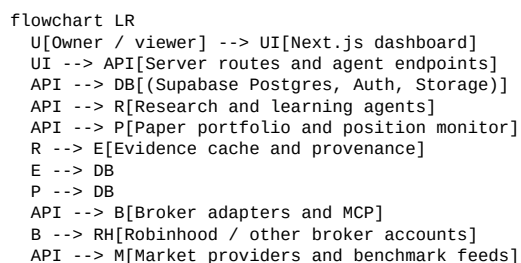


The LLM is an assistant around this pipeline: it can summarize evidence and explain a result, but it is not an unrestricted source of prices, scores, risk limits, targets, stops, or orders.

Visual architecture map

These diagrams are the fastest way to orient yourself. The detailed feature chapters contain additional diagrams for individual agents and workflows.

1. System context



The dashboard is a controlled view of server-side contracts. It does not give a viewer direct database or broker access, and paper execution is separated from live-account permissions.

2. From market data to a trade decision

```
flowchart TD
    A[Provider data] --> B{Fresh, point-in-time,  
quota-safe?}
    B -- no --> X[Record refusal / degraded evidence]
    B -- yes --> C[Market-local dimensions]
    C --> D[Deterministic composite score]
    D --> E{Eligibility, liquidity,  
risk and broker checks}
    E -- no --> Y[No order; retain reason]
    E -- yes --> F[Portfolio sizing and cash checks]
    F --> G[Paper fill or owner-gated live preview]
    G --> H[Lots, marks, exits and reconciliation]
```

Every “no” branch is an explicit outcome. Missing evidence is not silently converted into a bullish score or an order.

3. Exit and learning loop

```
flowchart LR
    S[Open position] --> Q{Fresh completed-session mark}
    Q --> X{Stop, target,  
trail or score invalidation?}
    X -- no --> S
    X -- yes --> L[Partial or full exit]
    L --> T[Closed lot with provenance]
    T --> M{Taint, maturity and  
cohort checks pass?}
    M -- no --> N[Exclude from learning]
    M -- yes --> O[Returns, IC/t-stat,  
benchmark and attribution]
    O --> W[Shadow / upgrade ledger]
    W --> V{Evidence gate passes?}
    V -- no --> W
    V -- yes --> A[Explicit review and promotion]
```

This loop explains why a feature can be collecting for a long time without changing a score, position size, or order.

4. Paper, live and viewer boundaries

```
flowchart TB
    subgraph Research[Shared research and evidence]
        RS[Signals, dimensions, benchmarks, learning]
    end
    RS --> PP[Paper portfolio]
    RS --> PR[Live proposal / approval card]
    PR --> G{Owner live switch +  
broker capability + fresh quote}
    G -- fail --> NO[Blocked; no broker order]
    G -- pass --> LT[Live TraderAgent]
    LT --> BR[Broker account]
    V[Viewer account] --> RO[Read-only allowlist]
    RO -. cannot reach .-> RS
    RO -. cannot reach .-> BR
```

The arrows are intentionally asymmetric: research may inform both paper and live paths, but a viewer cannot use the UI to cross an owner or broker boundary.

What each detailed feature entry tells you

When reading a feature section, look for five questions: **why** the feature exists, **what** it changes, **where** it connects to the application, **how** it is measured, and **what gate** prevents it from silently affecting money-path behavior. A feature can collect data for months without being ready for promotion. That is intentional: measurement is not proof.

Document map

- **Part I — Current system orientation:** the present-state narrative and the current-system reference. Start here.
- **Part II — Detailed feature and operational documents:** the preserved source material, grouped by the original feature or architecture path. Each block is labelled with its source file so it can be traced back to the repository.
- **Status and evidence language:** repeated terms such as **active**, **shadow**, **collecting**, **blocked**, and **proposed** retain their precise meaning from the status vocabulary above.
- **Intentional omissions:** cron tables, environment-variable inventories, database-schema dumps, and coding-convention material are omitted from this friend-facing edition at the owner's request. Their absence is intentional; this document explains behavior and design rather than exposing deployment internals or secrets.

Quick glossary

Agent means a bounded application worker, not an autonomous account owner. **Dimension** means one scored evidence family, such as technical or fundamental. **Shadow** means an isolated counterfactual measurement. **Promotion** means a versioned, reviewed decision to let a measured change affect a downstream path. **Money path** means any code that can create, change, or close a position or move cash. **Taint** means a data-quality condition that disqualifies an outcome from learning or attribution.

Part I — Current system orientation

The canonical current-state explanation is included first. It describes the product purpose, workspaces, data flow, agents, scoring, portfolio construction, exits, benchmarks, learning, brokers, crypto, health and documentation governance.

Part II — Detailed feature and operational documents

Source:
docs/arch/10-current-system-reference.md

Kairos current system reference

Last reviewed: 2026-09-21
This is the friend-facing, current-state guide. It describes what the code and
production contracts do today. A feature marked shadow , collecting , or
blocked is not a live trading policy. Historical feature architecture files
remain design records; this chapter is the status authority for the present.

1. What Kairos is

Kairos is a personal research and paper-trading system for US equities/ETFs, Indian equities/ETFs, and an isolated crypto paper sleeve. It runs deterministic data collection and scoring, records immutable decision evidence, simulates fills, monitors positions, compares portfolios with market-local benchmarks, and measures whether proposed improvements have evidence. LLMs may summarize evidence or help with research prose; they do not choose an unbounded symbol, score, stop, target, or order by themselves.

There are three separate investing books:

Book	Currency	Current execution state
US	USD	Paper trading active; live trading remains owner-gated/off
India	INR	Paper trading active; Zerodha/Kite live path remains owner-gated
Crypto	USD	Isolated paper pool and native research lane; live crypto disabled

Paper books never share NAV, cash, positions, benchmarks, or learning cohorts.

2. Runtime and ownership

The application is Next.js App Router with Supabase Postgres/Auth/pg_cron. Server routes use the server or service Supabase client; owner-only routes use the confirmed owner gate. Cron invokes deployed routes through the Vault-backed kairos_call_agent bridge. Local Windows schedules are not the production source of truth.

The main workspaces are Investing (/dashboard), Property (/property), and Capital (/capital-plan). The System Reference page exposes an owner-only, fixed allowlist of Markdown documents. It is not a repository browser.

3. Research pipeline

3.1 Universe and discovery

Research is bounded. The app combines the configured watchlist, holdings, screener admissions, discovery buckets, ETFs, metals and market-specific research queues. Discovery creates evidence and candidate membership; it does not itself authorize a trade. Broker tradability checks are separate and must be satisfied before a money-path order.

The US discovery lane is bounded and rotates liquid candidates. India uses its market-local universe and exchange symbols. A candidate can be refused for stale data, unsupported instrument type, missing evidence, liquidity, broker availability, or insufficient history. Refusals are recorded rather than silently disappearing.

3.2 Evidence collection

Provider calls are cached and quota-accounted. Alpha Vantage has atomic daily reservation/capacity controls; raw bypasses are not allowed. Price bars are stored in `price_cache` with provider, basis and provenance metadata. Post-close price prewarm runs before PositionMonitor so stop/target evaluation does not rely on the previous session. Evidence shadows exercise the router without changing live score inputs.

US research includes fundamentals, technicals, sentiment, macro and insider inputs where applicable. India uses market-local fundamental, technical and sentiment inputs; US macro is not stamped onto India. ETFs and other instruments have explicit applicability rules instead of fabricated company fundamentals.

3.3 Deterministic score

The equity score is a weighted composite with an availability mask. Unavailable dimensions are excluded according to the shared scorer contract; invalid weights, invalid scores and excluded placeholders fail closed. A high score is a candidate signal, not proof of predictive edge. Current IC/t-stat measurements remain diagnostic because qualifying sessions, mature labels, cohort integrity and overlap-adjusted effective sample size are still insufficient for promotion.

4. Paper trading lifecycle

1. Research writes a session-validated signal.
2. PaperTrader claims eligible signals and checks market controls, kill switches, cash, mandate threshold, open-name/sector limits, duplicate/pyramid rules, broker-compatible symbol policy and fresh pricing.
3. A transactional fill RPC writes the paper event, trade lot, position and cash movement atomically.
4. PositionMonitor refreshes completed-session marks and current evidence. It updates highest price and applies the shared exit ladder.
5. A target touch can be detected from the settled bar high even if the close recovered. Stop precedence remains conservative when both barriers occur.
6. Target handling may sell a partial lot and move the runner protection to breakeven. Trailing protection ratchets upward and never loosens.
7. A score/direction invalidation, stop, target/trail or safety action can close a position. The old unconditional clock time-stop is not the intended current policy; no-clock behavior must still be validated against each market route.
8. Closed lots feed learning and diagnostics only after taint, source, session and label checks.

The historical sizing replay is an offline diagnostic. It compares fixed equal allocation with stop-risk allocation under finite cash, caps, costs, partial exits and supplied marks. It does not alter production sizing or claim a globally optimal percentage. A separate top-up experiment is not yet approved for production.

5. Exit geometry and sizing status

Current production exits preserve the actual entry/position geometry and provenance where available. The recent intraday-high repair prevents a target touch from being lost merely because the closing price fell back below target. This is an execution correctness fix, not evidence that the target formula is optimal.

The sizing replay engine and fingerprinted CLI are implemented and tested. Historical coverage is incomplete: the canonical cache has broad history, but 20 India symbol roots have no matching price history, and partial-lot lineage/entry-stop provenance still require reconciliation. See [features/portfolio-sizing-replay/](#) and its read-only coverage queries.

6. Benchmarks and performance truth

Benchmarks are market-local and session-labelled. US and India collectors run at their own post-close times with bounded retries. A run is complete only when every enabled benchmark reaches that market's expected session; otherwise it is partial or error with provider/session/retry detail. Freshness alerts resolve only after all configured benchmarks advance.

Portfolio-vs-benchmark charts are presentation outputs of these canonical rows, not independent fetches. A stale chart is a data-contract failure, not a reason to silently reuse yesterday's benchmark.

Upgrade Path attribution is append-only and requires a matched baseline and variant, common window/population, frozen versions/hashes, costs, benchmark, independent sessions and exact arithmetic. The ledger currently has no valid measured rows for many paths. "Collecting" and "review-ready" do not mean "proven improvement."

7. Learning, strategies and shadows

LearnerAgent summarizes closed outcomes and generates bounded challengers. Validation uses purged/out-of-sample evidence and does not automatically mutate weights merely because a small sample looks good. Capital rotation, external strategies, sector signals, risk tiers, catalyst scoring, exit alternatives and model comparisons are primarily measure-only shadows until their declared gates pass.

The app must report, per shadow: population, dates, version, number of qualifying sessions, label maturity, overlap-adjusted effective sample size, IC/t-stat or operational result, benchmark comparison, blockers and next safe action. It must never create a winner row from aggregate portfolio P&L or hindsight-selected names.

8. Crypto lane

Crypto research is native and separate from equity scoring. Public Coinbase/Kraken candles and quotes supply research; Robinhood MCP supplies broker capability and execution truth. The approved paper basket is BTC-USD, ETH-USD and SOL-USD, with bounded broker-discovered candidates recorded separately.

Crypto paper entry requires completed data, a native score, fresh two-sided public quote, valid geometry, cash, market controls, a claimed native signal and no open duplicate position. The crypto fill RPC has a separate mandate branch and does not reuse an equity mandate. Crypto live execution remains disabled pending broker preview, protective-order and reconciliation evidence.

9. Live trading safety

Live switches remain off unless explicitly enabled by the owner. Live execution must verify broker account/pair eligibility, fresh executable quote, preview acceptance, idempotency, risk and kill-switch state, and protective-order reconciliation. A signed-in viewer cannot invoke owner money-path routes. Paper and live execution records are tagged and never silently merged.

10. System health and operations

System Health distinguishes failures from informational notices and owner actions. Critical failures include stale/missing market data, quota violations, broker token failure, failed persistence, or an unsafe live position. A healthy collector may still produce refusals; refusals are evidence, not necessarily incidents.

When investigating a problem, verify in this order: expected session, provider provenance, persisted row, route response, cron run, then UI. Never infer that a successful build or rendered card means the money path worked.

11. Status vocabulary

Status	Meaning
Shipped/active	Code and required production contract are present and verified
Paper-only	Can affect isolated simulated books, never broker orders
Shadow/measure-only	Collects evidence; cannot affect score, sizing or trades
Collecting	Infrastructure is live but the minimum evidence gate is not met
Blocked	A named data, provenance, owner or broker prerequisite is missing
Proposed	Architecture only; no implementation should be inferred

12. How to read the rest of the repository

Use docs/arch/ for stable operational chapters, features/*/FEATURE_ARCHITECTURE.md for individual designs, WORK_LOG.md for delivery evidence, PROJECT_DECISIONS.md for owner approvals, and public/agent-diagrams/system-map.json for topology. If an old feature document says “planned” while this chapter says “shipped,” the implementation result and current code win; update the stale feature document in the next documentation pass rather than treating its proposal as runtime behavior.

Source: docs\arch\00-index.md

Kairos Architecture Chapter Index

Last reviewed: 2026-09-21 (current system reference and evidence-status audit)

For a single comprehensive reading path, start with

[KAIROS_CONSOLIDATED_ARCHITECTURE.md](#).

For the complete feature-by-feature encyclopedia, use

[KAIROS_FULL_ARCHITECTURE.md](#).

This directory is the definitive chapter-by-chapter operational architecture. Each chapter is independently updateable: a broker change does not require editing the agent chapter. Use [ARCHITECTURE.md](#) for the documentation hierarchy and [SYSTEM_OVERVIEW.md](#) for a short orientation.

Chapters

File	What it covers	Update when
01-what-is-kairos.md	Core idea, principles, feature map, onboarding diagrams	Product direction or feature areas change
02-tech-stack.md	Runtime stack, providers, adapters, and integrations	A library, provider, adapter, or framework changes
03-agents.md	Agent responsibilities, inputs, outputs, and behavior	An agent changes or is added/removed
04-database-schema.md	Tables, persistence contracts, indexes, and triggers	Any schema migration changes a data contract
05-crons-and-scheduling.md	Vercel and local schedules, operational timing	A cron is added, removed, or rescheduled
06-env-variables.md	Required, optional, and runtime-editable variables	An environment variable changes ownership
07-coding-conventions.md	UI, data access, API, and agent conventions	A project-wide convention changes
08-risk-and-safety.md	Money-path gates, autonomy, kill switches, ledgers	Order flow, autonomy, or safety behavior changes
09-learning-loop.md	Evaluation, learning, validation, and promotion controls	Learning flow, genome, or promotion controls change
10-current-system-reference.md	Friend-facing current behavior, status vocabulary, and shipped-vs-shadow boundaries	Any material runtime, safety, benchmark, crypto, or learning contract changes

Source-of-truth hierarchy

Source	Owns
ARCHITECTURE.md	Architecture portal, principles, documentation ownership, and read paths
SYSTEM_OVERVIEW.md	Concise product and system orientation
docs/arch/	Detailed operational architecture
features/<feature>/FEATURE_ARCHITECTURE.md	Feature micro-architecture: contracts, states, alternatives, rollout, and non-goals
features/<feature>/IMPLEMENTATION_RESULT.md	What actually shipped and approved deviations
PROJECT_DECISIONS.md	Approved decisions, rationale, and reversal cost
public/agent-diagrams/system-map.json	Rendered agent-to-agent topology and topology history

When these sources disagree, reconcile the conflict in the same change. A feature document cannot silently override a shared chapter or approved decision.

The overview and this index change only when the reading path or ownership changes.

The affected chapter and feature implementation result change with the implementation that changes their underlying contract.

Update rules

Change	Update
New or changed agent	03-agents.md; topology JSON/history; 05-crons-and-scheduling.md when scheduled
Scoring dimension, formula, source, weight, applicability, missing-data rule, veto, or cap	03-agents.md plain-English explanation + exact formula contract; 02-tech-stack.md, 04-database-schema.md, and 08-risk-and-safety.md when their ownership changes
New table, column, index, trigger, or RPC	04-database-schema.md
New provider, broker, or runtime integration	02-tech-stack.md
New or changed schedule	05-crons-and-scheduling.md
New environment variable or vault ownership	06-env-variables.md
Money-path, autonomy, or safety-gate change	08-risk-and-safety.md and relevant feature design
Learning, validation, strategy, or promotion change	09-learning-loop.md and relevant feature design
Product scope/principle change	01-what-is-kairos.md and SYSTEM_OVERVIEW.md
Private cross-workspace capital-planning inputs or decisions	04-database-schema.md, 08-risk-and-safety.md, and features/capital-advisor/
Feature implementation	Its feature design and an implementation result; affected shared chapters

Diagram ownership

Agent-to-agent topology lives only in `public/agent-diagrams/system-map.json`, rendered on Dashboard -> Agents -> Agent & Flow Architecture. Do not paste the same edges into chapters. A chapter diagram is appropriate only when it documents a different view, such as a gate sequence, timeline, or contained subsystem. The drift test in `tests/arch-diagram-drift.test.ts` validates the map files and declared overlaps.

In-app access

Dashboard -> Agents -> System Reference exposes a small owner-only allowlist for reading or downloading architecture records. It is not a repository browser and does not replace this documentation structure.

Source: docs\arch\01-what-is-kairos.md

Kairos — What Is This?

Last updated: 2026-09-08

Current runtime/status authority: [10-current-system-reference.md](#).

Update this file when: product direction changes, new feature areas are added, core principles change, or the top-level pitch changes.

1. The core idea

Kairos is an **automated investing research assistant** that behaves like a small, careful hedge-fund team living inside one web app. It doesn't just pick stocks — it **runs a loop**:

Research → Decide → Trade (on paper first) → Watch → Learn → Improve → repeat.

The important word is **loop**. Most stock tools give you a score and stop. Kairos scores a stock, acts on it in a **paper (pretend-money) portfolio**, watches how that trade actually turns out, and then **feeds the outcome back** to make the next decision smarter. Real money is optional, always small, and **never moves without you clicking "yes"**.

2. Three core principles (apply everywhere)

1. **Paper first, real money last.** Every strategy proves itself on pretend money before a cent of real money is at stake.
2. **AI proposes, human disposes.** Agents can **suggest** a new strategy or trade, but a person must approve anything that touches real money or changes the live strategy.
3. **Evidence before belief.** A proposed improvement must beat the current one on held-out historical data before it is allowed to take effect.

3. The big picture

```
flowchart LR
    DATA[Market data\nUS + India] --> RESEARCH[ResearchAgent\nscores each stock]
    MACRO[MacroSentinel\neconomy read] --> RESEARCH
    RESEARCH --> SIGNALS[Signals\nscore per stock]
    SIGNALS --> PAPER[PaperTrader\npretend-money trades]
    SIGNALS -- ".approved by you." --> LIVE[Live order\nreal money, tiny]
    PAPER --> MONITOR[PositionMonitor\nwatches + exits]
    MONITOR --> OUTCOMES[Closed trades\nwin/loss]
    OUTCOMES --> LEARNER[LearnerAgent\nproposes a better strategy]
    LEARNER --> VALIDATE[Validation Engine\ntest on history]
    VALIDATE -- ".owner-approved activation." --> CHAMPION[Champion strategy]
    CHAMPION --> RESEARCH
```

Read it as a wheel: data comes in on the left, becomes decisions, becomes trades, becomes

outcomes, and the outcomes teach a better **Champion strategy** that feeds back into ResearchAgent. That feedback arrow is the whole point.

4. Feature map

Feature area	What it does
Screening & research	Scans US + India universes, scores candidates across 5 dimensions, tracks score trend
Paper trading	Simulated portfolios per market (\$US, ■India) — realistic fills, stops, targets
Live trading	Real orders via Robinhood (US) + Zerodha Kite (India), human-gated
Evolution loop	Turns closed outcomes into challengers, validates and shadows them automatically; owner-approved activation is the separate protected switch to Champion
RAG trade memory	Semantic recall of past setups at scoring time
Performance Truth	Mandate-aware Sharpe/Sortino/alpha/drawdown evaluation ledger
Coaching & briefings	MentorAgent coaching notes + daily email briefings (email-only; no dashboard card)
Property workspace	Owner-recorded property ledger + equity/carrying cost, plus weekly keyless official-source market context (FHFA HPI, FRED, BLS LAUS, HUD FMR). Not an AVM
Capital workspace	Capital allocation profile and decision runs; CapitalRotation shadow evaluates blocked candidates on the investing side
System Health	Funnel of open issues → dashboard card + brief section
Multi-LLM routing	Claude / DeepSeek / Groq / Gemini; per-agent assignments; paper P&L per model
India parity	Full NSE scoring, ■ paper pool, Kite execution, NSE insider+options feeds
Admin & DB cleanup	User management; monthly DB pruning cron

5. What makes this different from a scanner

Ordinary scanner	Kairos
Gives you a score today	Scores + remembers its past accuracy
You decide every trade	Pretend-money paper book tests decisions first
No feedback loop	Closed loop: outcomes teach future scoring
Single model	Multi-LLM routing; Claude vs DeepSeek P&L compared
US-only	US + India (INR ■ pool separate from USD \$)
No safety layers	9+ independent money-safety gates

6. A trade's life, end to end (worked example)

US stock "ACME"

```
flowchart TD
    A[Mon AM: ResearchAgent scores ACME 78\nwrites decision_observation + signal_score_history] --> B{score >= threshold AND long?}
    B -- yes --> C[PaperTrader claims signal\nbuys ACME in $ pool\nrecords expected_price + realized_slip]
```

```

B -- no --> Z[logged only, no trade]
C --> D[PositionMonitor watches daily\nupdates highest_price]
D --> E{exit trigger?}
E -- trailing stop breached --> F[Full close: realized_pnl, outcome=win/loss]
E -- price target hit --> G[Partial: sell half, move stop to breakeven]
E -- score < entry threshold --> F
E -- direction flip --> F
G --> D
F --> H[indexClosedTrade: embed setup, store in trade_memories]
H --> I[Learner training set grows]
I --> J[After 10+ trades on Fridays: LearnerAgent proposes Challenger]

```

Step by step:

- Monday morning:** ResearchAgent scores ACME **78**, writes agent_signals (pending), signal_score_history, decision_observations.
- 10:05 AM:** PaperTrader wakes, claims the signal, checks re-entry cooldown (ACME not held recently), checks pyramid gate (not held at all), buys ACME with 10% of pool NAV. Records expected_price = 102.50, fill at 102.55 (0.05% slip).
- Daily 4:15 PM:** PositionMonitor fetches live ACME price. Updates highest_price. Trailing stop = max(stop_loss, highest_price × anchor), ratcheting up only. There is NO time stop.
- Day 8:** ACME hits the price target at \$123. PositionMonitor sells half (floor(qty/2)), moves stop_loss to avg_cost (\$102.55). The remaining half runs.
- Day 26:** ACME's fresh score falls to 57, below the entry threshold of 60 - it would not be bought today - so PositionMonitor closes the remainder. Had the score stayed healthy, the runner would have kept running until its ratcheting trail was breached. Age alone never closes anything (Decision 74).
- Close:** paper_trades updated with exit_price, pnl_pct, outcome. Cash returned. indexClosedTrade() embeds the setup and stores in trade_memories.
- Friday 5 PM (after 10+ trades):** LearnerAgent reads the closed cohort, proposes a Challenger with slightly higher technical weight.
- Owner activation:** Validation Engine confirms Sharpe 0.72, win_rate 58%. The system keeps learning and collecting challenger evidence regardless; an owner-approved activation then promotes the Challenger to Champion. ResearchAgent's next Monday run uses the new weights.

7. Navigation map

Kairos is THREE workspaces, each with its own layout and left navigation: Investing (app/dashboard/, DashboardShell), Property (app/property/, PropertyShell), and Capital (app/capital-plan/, CapitalShell). All three require auth (Supabase middleware) and are reachable from each other through the workspace switcher in every shell.

7.1 Investing — app/dashboard/

Path	Page name	What's on it
/dashboard	Home	Portfolio summary, live account, macro regime banner, System Health card, agent status

Path	Page name	What's on it
/dashboard/research	Research	Signal list, symbol cards, score breakdown, thesis
/dashboard/learning	Learning	Current champion/baseline, all challengers, historical champions, validation and experiment evidence, plus Performance Truth
/dashboard/agents	Agents	Agent status grid, System Map diagram (Mermaid, from system-map.json), per-agent diagrams
/dashboard/upgrade-path	Upgrade Path	Owner-only live inventory of shadow/paper experiments: evidence counts, provider-call accounting, benefit verdict, blockers, schedule truth, and review ETA
/dashboard/markets	Markets	Macro sentinel gauge, sector TradingView chart, insider trades, breadth
/dashboard/smart-money	Smart Money	Options flow, insider signals, trade queue; 4 tabs; both markets
/dashboard/india	India	NSE score tracker, ■ paper portfolio, Kite live holdings, India order form
/dashboard/live-portfolio	Live Portfolio	All Robinhood account positions, CSV import, trade enrichment, performance chart
/dashboard/journal	Decision Journal	Trade decision log, signal→fill→outcome linking
/dashboard/mentor	Mentor	MentorAgent coaching insights
/dashboard/briefing	Legacy redirect	Redirects to /dashboard. Briefings are email-only since 2026-09-08 — the dashboard card was removed
/dashboard/backtest	Backtest	Validation Engine replay, results
/dashboard/scanner	Scanner	US + India universe scan, NIFTY-100 live fallback
/dashboard/settings	Settings	Account preferences and access, trading controls, AI/provider keys, automation, data routing/capacity, and system maintenance
/dashboard/admin	Legacy redirect	Redirects to Settings → System so old links continue to work
/dashboard/admin/vault	API Vault	Runtime API key management, linked from Settings → System

7.2 Property — app/property/

Separate workspace with its own 236px sidebar (components/property/PropertyShell.tsx).

Owner-recorded ledger plus keyless official-source market context — it is deliberately

NOT an automated per-address valuation.

Path	Page name	What's on it
/property	Overview	Portfolio-level property summary
/property/my-properties	My properties	Owner-recorded property ledger, equity, carrying cost, append-only history snapshots
/property/valuation	Valuation	Stage-one valuation workspace and evidence scopes
/property/markets	Markets	County/market observations from the collected official sources
/property/forecasts	Forecasts	Shadow forecasts + calibration (property_forecasts)
/property/financing	Financing	Financing accounts and terms
/property/opportunities	Opportunities	Candidate screening surface
/property/imports	Evidence imports	Owner evidence import + bulk snapshots
/property/sources	Sources	Source-run health for the collection adapters

7.3 Capital — app/capital-plan/

Path	Page name	What's on it
/capital-plan	Capital plan	Capital allocation profile and decision runs (capital_profiles, capital_decision_runs)

Capital currently has NO sidebar — CapitalShell is a header with the workspace switcher only, because the workspace is a single page. If Capital gains more pages it should adopt the PropertyShell sidebar pattern rather than growing a second navigation idiom.

Source: docs\arch\02-tech-stack.md

Kairos — Tech Stack

Current provider-cache contract: fallback evidence retains the original fetch timestamp, so freshness limits are measured from the real observation rather than from the time a failed request copied it into a same-day cache slot. Failed calls are logged without URLs or credentials. This is an implementation detail supporting the evidence-freshness rules described in the current-system reference; the dated incident history remains in the source audit record.

2026-08-08: **Property market data uses three keyless official US sources**, added as small native adapters rather than any third-party scraping framework or unvetted GitHub package — FHFA HPI (hpi_master.json, quarterly metro price index), FRED MORTGAGE30US (public graph CSV, weekly 30-year rate), and BLS LAUS (keyless public API, monthly metro unemployment). The official HUD FMR adapter is also implemented but remains contract_pending until a free HUD User bearer token is configured server-side. It collects annual Austin/Phoenix **metro affordability references by bedroom count only**, never a property rent estimate, comparable, valuation, forecast, or underwriting input. Required attribution is rendered on the Markets page: *"This product uses FHFA Data but is neither endorsed nor certified by FHFA."* Census ACS remains contract_pending. Redfin bulk files, listing scrapers, and the DealLens/RealVest/HomeHarvest/fred-mcp/rhud packages were reviewed and **rejected** as production runtime dependencies: none improves the security, licensing or provenance boundary over a native adapter against the official release.

Ingestion carries a **per-invocation fetch cache** (beginPropertyCollectionRun() in lib/property/sources.ts). FHFA's multi-megabyte master JSON and FRED's full history were previously downloaded once **per market** with cache: "no-store", costing nine upstream calls for a three-market run; it is now four, and four is correct because BLS legitimately needs one request per metro. The cache is deliberately per-invocation rather than TTL-based — a TTL would serve a stale national file into a later scheduled run.

Property parcel evidence has a separate GitHub Actions implementation lane (.github/workflows/property-evidence.yml), never a Vercel request, but **collection is disabled**. The Maricopa Sales Affidavits and TCAD export contracts are contract_pending: the worker exits before credentials, scopes, or downloads, and the workflow performs parser/self-disable checks only. Historical evidence is retained, but no new parcel or sale record is collected until each source has a documented, permitted machine-use contract. Generic GitHub scrapers remain excluded from the trusted runtime.

Owner-private property values use lib/property/crypto.ts: AES-256-GCM, versioned envelope, authentication tag, fail-closed when PROPERTY_DATA_ENCRYPTION_KEY is absent or is not a base64-encoded 32-byte key. **Local development,**

Vercel, and the Property evidence GitHub Action must carry the SAME key, because the same master also derives domain-separated parcel/event HMACs. A mismatch renders encrypted payloads unreadable and creates unjoinable parcel identities. Plain SHA-256 address/parcel hashes are forbidden.

2026-07-31: **Daily scoring now has a provider-independent completed-session boundary**. Yahoo is the current primary US daily-candle source and the India fallback behind Upstox; after normalization, ResearchAgent removes the current market-local bar until 16:00 ET / 15:30 IST. The separate Yahoo deep-history adapter remains an offline/replay input and does not authorize strategy promotion.

2026-07-31: **Reported fundamentals are event-aware and ADR-safe**. ResearchAgent batch-reads earnings_calendar before provider work: a report in the prior 3 or next 14 days uses a 1-day cache; otherwise or when unknown it uses 7 days. Finnhub issuer profiles use 30 days. Theme Scout validates candidates with a quote, never a full fundamentals fetch. Reviewed exchange-listed ADRs use Yahoo ADS-basis fundamentals only; an unavailable ADR response cannot fall through to a provider that resolves the foreign ordinary share.

2026-07-28: Yahoo daily candles became market-agnostic in lib/data/yahoo-candles.ts (fetchYahooCandles + yahooRange). Measured five-year depth: AAPL 1254 bars and RELIANCE.NS 1239 bars. This is the free keyless deep-history fallback when Massive's entitlement rejects older US history. Range boundaries guarantee the returned window is not shorter than requested.

Last updated: 2026-09-21 (**Python Lane A removed from Vercel** — Functions Storage cap); 2026-09-02 (**reasoning-model budget floor** — every LLM failure in 14 days was truncated reasoning; see "Reasoning models need a budget floor"); 2026-07-22 (Yahoo Finance v8 chart promoted to **primary US candle** source; recency guard added to fetchUsCandles; minCandles default raised 15→60; Massive/EODHD/TwelveData demoted to fallback)

Prior: 2026-07-19 (webull_trade signed sender restored behind database-backed preflight and nine-gate one-shot permits; adapter and activation remain DISABLED; read-only Cloud MCP remains query-only)

Prior: 2026-07-16 (House Stock Watcher congressional feed retired — upstream bucket went private; no licence-clean free replacement qualified)

Update this file when: a new library is added, a provider changes, a new adapter is added, the framework is upgraded, or any layer in the table below changes.

1. Frontend + framework

Layer	Technology	Notes
Framework	Next.js 15 (App Router)	All pages under app/; both RSC and client components
Language	TypeScript (strict)	Centralized types at @/types
Styling	Inline styles + T tokens	No Tailwind, no CSS modules. Each file declares const T = { bg, surface, card, border, text, ... }. Never add external class utilities.
Charts	Recharts	Used for paper P&L, score history, correlation charts
Diagrams	Mermaid v10	v11 webpack-broken; agent flowcharts rendered by components/dashboard/MermaidChart.tsx
TradingView	Advanced Chart widget	Sector ETF charts on Markets page; symbol detail page
Fonts	JetBrains Mono (monospace for numbers/code)	Loaded as system font fallback

2. Backend + data

Layer	Technology	Notes
Database	Supabase (PostgreSQL)	All tables, RLS policies, migrations in supabase/migrations/
Auth	Supabase Auth	Email/password; OWNER_EMAIL gate on all sensitive API routes
Storage	Supabase (not used for large files)	—
Vector store	Supabase pgvector	trade_memories table; 1024-dim Jina/Voyage embeddings; cosine similarity
Server functions	Next.js API routes	All under app/api/; force-dynamic where needed
Cron (cloud)	Vercel Crons	Two crons in vercel.json; hit deployed URL
Cron (local)	Windows Task Scheduler	8 tasks via scripts/run-agents.ps1; PC must be on
Observability	Langfuse	Traces callLLM (single-shot) + runAgentLoop (multi-step tool calls)

3. AI / LLM

Per-flow model selection (2026-07-12). Every agent/flow's model is chosen from

Settings → **AI & keys** (agent_config table), NOT hardcoded. Routes read

getConfiguredModel(svc, agentName, fallback) (lib/agent-model-config.ts) and pass the

result to callLLM({ model }). MODEL_ROUTING in lib/llm-router.ts is now only the

DEFAULT when no model is passed. **Policy: default OFF Claude** — hard-reasoning flows

(research/trade/evaluate/thesis) default to deepseek-v4-pro with thinking explicitly

enabled; cheap flows use deepseek-v4-flash with thinking explicitly disabled. Claude is

opt-in per flow from Settings, never a silent default.

Providers (each key set in Settings → Provider API Keys, vault-first / env-fallback):

Provider	Concrete models	Dispatch in llm-router
DeepSeek	deepseek-v4-flash (fast/non-thinking), deepseek-v4-pro (thinking)	callDeepSeek
Anthropic	claude-haiku-4-5, claude-sonnet-4-6, claude-opus-4-8	callClaude (extended thinking auto-on for research/trade/evaluate/thesis)
Groq	llama-3.3-70b-versatile, llama-3.1-8b-instant, ...	callGroq
Google	gemini-2.5-flash, gemini-2.5-pro	callGemini (added 2026-07-12)
xAI	grok-4-fast, grok-4	callGrok (added 2026-07-12)

Tier aliases (TIER_MODELS): fast→deepseek-v4-flash, reasoning→deepseek-v4-pro,

claude-fast→Haiku, claude-smart→Sonnet. LEGACY_ALIASES transparently rewrites the

retiring deepseek-chat/deepseek-reasoner aliases to the concrete V4 IDs.

SAME_TIER_FALLBACK degrades a single unavailable model to a same-tier sibling (Gemini/Grok

fall back to deepseek-v4-pro) and raises a System Health alert. A missing Anthropic key

falls back to deepseek-v4-flash, not a crash. **No local Claude-CLI/PowerShell path exists anymore** — lib/claude-exec.ts was deleted 2026-07-12; all LLM + data fetches are HTTP.

Adding / changing models

To register a new model: call registerLLMProvider() in lib/llm-router.ts and update the relevant tier alias. A System Health alert fires automatically when any model is deprecated/renamed.

Reasoning models need a budget floor (2026-09-02)

A reasoning model (deepseek-v4-pro, the reasoning tier) emits chain-of-thought into reasoning_content **before a single token of answer**. Give it less than its thinking needs and it returns an *empty* answer with finish_reason=length — having billed the whole budget. A call that cannot succeed is worse than an expensive one: same tokens, no output.

Audit, 14 days of llm_call_log. Every failed LLM call in the system — **213 of them — was this one error**, and every one was on the reasoning tier while every flash flow sat at zero failures:

flow	model	calls	failures	budget
research	pro	561	197 (35%)	1500
macro-read	pro	11	11 (100%)	1500
mentor-evaluate	pro	3	3 (100%)	2000
chat	flash	82	0	2000
holding-risk	flash	60	0	1200

research had been failing a third of its thesis calls for two weeks while logging claude_calls: 0, which reads as "no LLM used" rather than "the call failed".

Why a floor and not another number. The same bug had already been found and fixed three times by raising one flow's constant — research-agent 512→1500, macro-read 600→1500 (its source comment records both, and notes it "silently killed this agent for 4 days"), then mentor-evaluate 2000→16000. macro-read was failing 100% *at the bumped value*. A fifth constant would leave a sixth flow to discover it in production.

REASONING_MIN_TOKENS (16000) is enforced in callLLM **before dispatch**, and applies to every downstream path including the same-tier and auth fallbacks — a fallback that re-sent the original small budget would truncate identically. isReasoningModel() derives from TIER_MODELS["reasoning"] so a renamed tier carries automatically. Flooring raises a warning naming the flow, and that alert states the cheaper alternative rather than only raising cost: a flow wanting three sentences belongs on the **non-thinking tier**, not paying for 16k of reasoning it discards. markets-synthesis (350 tokens) and markets-thesis (400) were reassigned to deepseek-v4-flash on exactly that basis — a 350-token budget can never

be honoured by a model that thinks first.

Separately, `isTruncatedReasoning()` retries a truncated call once with headroom. That predicate is deliberately distinct from `isModelUnavailable()`: the truncation error is worded to avoid the deprecated-model vocabulary so it does not spam the health funnel, and because the only retry path used to be gated on `isModelUnavailable`, the error matched nothing and propagated — while both the error text and the comment beside it promised a retry the control flow prevented.

Outcome (measured 2026-09-09). The floor worked. research ran 116 calls over the six sessions from 2026-09-03 to 2026-09-09 with **zero** truncation failures, against 24/51 on 2026-09-01 immediately before it landed. Any future 30-day failure rate quoted for these flows must be windowed on the fix date or it re-reports cured history as live waste.

DeepSeek cost accounting: off-peak base + peak multiplier (2026-09-09)

PRICING holds DeepSeek's **off-peak**, **cache-miss** rate. `deepSeekPeakMultiplier()` applies the surcharge, and it is the only place the surcharge exists.

model	input (off-peak)	output (off-peak)
deepseek-v4-pro	\$0.66 / 1M	\$1.98 / 1M
deepseek-v4-flash	\$0.22 / 1M	\$0.66 / 1M
deepseek-v4-flash-vision-exp	\$0.22 / 1M	\$0.66 / 1M

Peak is **exactly 2x** off-peak, Monday–Friday **01:00–04:00** and **06:00–10:00 UTC** (effective 2026-08-16 16:00 UTC). All other hours, and all weekend hours, are off-peak. The 04:00–06:00 gap between the two windows is off-peak — merging them into one range silently doubles the ledger for two hours a day.

The defect this fixed. The table carried $[0.14, 0.28]$ for flash and $[0.435, 0.87]$ for pro, understating **output** by 2.4x–4.7x. Output dominates on a thinking model because chain-of-thought bills as completion tokens. Over the 30 days to 2026-09-09 the ledger read **\$1.47** while DeepSeek billed **\$5.65**; repricing the **same logged tokens** at the corrected rates yields \$3.00 all-off-peak to \$6.00 all-peak, bracketing the real bill. The tokens were never missing — the constants were wrong. Two earlier hypotheses were investigated and disproved: the `deepseek-research/learner-agent` Edge Functions do call DeepSeek directly outside `logCall`, but no `cron.job` invokes them (migration 052 unscheduled them), and the truncation failures that log zero tokens were pre-floor history, not ongoing spend.

V4 Pro retirement (2026-09-14). DeepSeek retires V4 Pro and routes its requests to the V4.1 Flash pool, billed at V4.1 Flash prices. Calls keep succeeding — this is a silent **re*pricing*, not a break — so from that date the Pro rate above OVERstates cost. V4.1 Flash has no published API model id yet, so no rate is guessed: `applyDeepSeekPeak` raises a warn

alert instead. On resolving it, update `PRICING` and `TIER_MODELS["reasoning"]`; note `isReasoningModel()` derives from that alias, so the 16000-token floor follows whatever is set there.

4. External brokers

Broker	Market	Auth	Role
Robinhood	US	OAuth PKCE S256 loopback; token in <code>api_key_vault</code>	Read-only account (monitoring) + agentic account (orders only)
Zerodha Kite	India	Daily re-login; SHA256 checksum exchange; <code>KITE_ACCESS_TOKEN</code> in vault	Real NSE/BSE portfolio read + order placement

The same authenticated Kite session also reads Zerodha Coin mutual-fund holdings through `GET /mf/holdings`. Coin is a separate, read-only portfolio sleeve: last-available NAV, units, invested value and P&L are shown on India Live Portfolio, but no mutual fund enters equity research, scoring, stops, targets, ladders, performance history or order execution. A missing Coin NAV withholds combined totals rather than being treated as zero; Coin endpoint failure does not blank valid Kite equity holdings.

Broker adapter layer

Location: `lib/brokers/adapters/`, `lib/brokers/registry.ts`

All broker interactions go through a `BrokerAdapter` interface. Current adapters:

Adapter	File	Market
<code>robinhood-mcp</code>	<code>lib/brokers/adapters/robinhood-mcp.ts</code>	US
<code>robinhood</code>	<code>lib/brokers/adapters/robinhood.ts</code>	US (direct REST, serverless-capable)
<code>kite</code>	<code>lib/brokers/adapters/kite.ts</code>	India
<code>alpaca</code>	<code>lib/brokers/adapters/alpaca.ts</code>	US (future)
<code>webull_trade</code>	<code>lib/brokers/webull-trade/</code>	US — built, DISABLED, transport fixture-tested

The adapter registry (`lib/brokers/registry.ts`) resolves which adapter to use per market and account role. The order placement code never calls a broker API directly — it goes through the registry.

`webull_trade` — signed Webull Trading API (2026-07-18, disabled)

Webull exposes **two unrelated products** and they must never be combined:

Surface	Registry	Role
Cloud MCP (<code>webull</code>)	<code>lib/brokers/mcp-registry.ts</code>	Read-only. Accounts/positions/balances/quotes. No order tools, no write scopes, <code>orderCapable: false</code> . Not in the order registry.
Trading API (<code>webull_trade</code>)	<code>lib/brokers/registry.ts</code>	Signed REST money path. The only Webull order surface.

The inert MCP-based order scaffold (`lib/brokers/adapters/webull.ts`) was **deleted** along with its `webull` order-registry entry — there is no second Webull order adapter.

Module layout (`lib/brokers/webull-trade/`):

File	Purpose
<code>signing.ts</code>	HMAC-SHA1 request signing, implemented directly (no vendor SDK). One auditable canonical request; fresh nonce per request; timestamp-skew guard; constant-time verify.
<code>credentials.ts</code>	Vault-only app-key/app-secret/access-token accessor, provider tag <code>webull_trade</code> , bounded cache. Sandbox and prod use separate env-prefixed vault keys bound to separate hosts derived from the same env, so a sandbox credential can never resolve a prod host.
<code>gates.ts</code>	All 9 gates of the Mandatory Gate Ladder, pure and ordered. Every gate fails closed; unknown/undefined is never "satisfied".
<code>token.ts</code>	Token lifecycle PENDING / NORMAL / INVALID (15 idle days) / EXPIRED (5-min verify window). Fails closed on pending/invalid/expired/unknown/idle; alerts before the 15-day boundary; no keepalive traffic.
<code>preflight.ts</code>	Parses the official POST <code>/openapi/auth/token/check</code> response into the fresh server-confirmed token record required by gate 7.
<code>order.ts</code>	Normalization + boundary rejects. Scope locked to market + limit + GTC STOP_LOSS in CORE. SHORT is rejected even though the API accepts it — a capability is not a permission. Options, trailing, OCO/OTOCO, algos, extended/overnight sessions rejected.
<code>lifecycle.ts</code>	place/query/cancel/reconcile on a stable <code>client_order_id</code> (≤ 32). A timeout, a throw after send, or a 200 with no parseable order id → <code>needs_reconcile</code> : never success, never a blind resubmit.
<code>capabilities.ts</code>	BrokerProtectiveCapabilities declaration (shape per features/hybrid-stop; that shared type is not yet on main, so it is declared locally). Claims only US equity <code>sell_long</code> via GTC stop-market in the regular session.
<code>transport.ts</code>	Sole Webull fetch path. Adds signed headers plus <code>x-access-token</code> , pins host/env, wraps timeout/HTTP errors, and consumes one-shot capability permits. Preflight reads the database flag and cannot reach order endpoints; production order permits require all nine gates.

It places no order today and fails closed on all three of: absent

`strategy_config.webull_trade_orders_enabled` (migration 20260718120000_... written but **NOT applied**), no allowlisted `broker_accounts{broker='webull_trade',market='us',role='trading'}` row, and no `api_key_vault` `provider='webull_trade'` credential. Everything is verified against fixtures with injected transports and fetch stubs only — no live or sandbox Webull call has been made, and the registry adapter still refuses submit/query/cancel before constructing a transport.

Before any live dollar the owner must: confirm the Webull API entitlement; reconcile the canonical signing layout, request paths, and hosts against the current official docs; provision the vault records and the allowlist row; apply the migration; and run a manually-approved sandbox test. Design + open decisions: `features/webull-trading-api/FEATURE_ARCHITECTURE.md`.

5. External data providers

Provider	What it provides	Auth
Alpha Vantage (AV)	Technicals (RSI, EMA, MA), 8 macro indicators; last-resort fundamentals/insider fallback (25/day cap, usually exhausted)	ALPHA_VANTAGE_API_KEY in vault
FRED	Official/free US macro series. MacroSentinel uses its established series; instrument-family measurement additionally reads DFII10 (10-year real yield) and DTWEXBGS (broad dollar) through the shared day cache, and since 2026-09-15 the oil exposure pack reads DCOILWTICO and DCOILBRENTU (EIA spot crude republished by FRED; no separate EIA key). Staleness is per series (DFII10 7d, DTWEXBGS 10d weekly release, crude 10d) and a stale series raises a System Health warning. The values are measure-only and never enter v1 scoring or trading.	FRED_API_KEY in server environment
Finnhub	PRIMARY domestic-US fundamentals — /stock/metric (P/E, net margin, ROE, EPS, revenue growth) + /stock/profile2 (sector), mapped to AV-OVERVIEW shape in lib/data/fundamentals.ts:fetchFinnhubOverview. Metric cache is event-aware (1/7d); profile cache is 30d. Reviewed ADRs bypass Finnhub because it can resolve the foreign underlying. Free 60/min, no daily cap	FINNHUB_API_KEY in Vercel env
Massive Market Data	FALLBACK US candles (was primary; demoted 2026-07-22), stock screener, options, quotes; PRIMARY US insider — /stocks/filings/vx/form-4 open-market P/S transaction scoring (lib/data/massive-insider.ts)	MASSIVE_API_KEY in vault
Yahoo Finance (fundamentals)	SECONDARY domestic US + PRIMARY India + PRIMARY reviewed ADR fundamentals — quoteSummary (crumb handshake) → AV-OVERVIEW shape via fetchIndiaOverview (market-agnostic: margin, ROE, revenue growth, P/E, sector). ADRs fail unavailable if Yahoo is thin; Kairos never mixes a foreign-underlying per-share response with a USD ADS price. Unofficial — cached, paced + fail-soft	None (crumb)
SEC EDGAR (fundamentals)	NOT WIRED — lib/data/sec-fundamentals.ts companyfacts adapter is experimental; a 2026-07-13 spot-check found its raw-tag margin/ROE unreliable (needs the frames API). Left for a future task	None
FMP	Secondary US fundamentals fallback only (its /stable/ratios-ttm+/key-metrics-ttm may be premium-gated, so Finnhub/Yahoo remain ahead of it)	FMP_API_KEY in vault
FinancialDatasets	Fallback US screener discovery (primary until 2026-08-02, now behind Yahoo) and manual Scanner fundamentals through POST /financials/search/screener. Contract pinned in lib/data/financialdatasets-screener.ts. Demoted after the account balance reached \$0.00 on 2026-07-29 and US discovery ran ten days on holdings and watchlist alone with no alarm. NOT used for ResearchAgent scoring fundamentals (Finnhub/Yahoo own that path).	FINANCIAL_DATASETS_API_KEY in vault
Yahoo custom screener	Primary US candidate discovery since 2026-08-02 — POST https://query2.finance.yahoo.com/v1/finance/screener, crumb-authenticated via lib/data/yahoo-crumb.ts, no API key and no account. Undocumented: thresholds are percentages not ratios, and a criterion can be accepted and silently discarded (freecashflow.lasttwelvemonths is in that state today and ships in no bucket). POST /api/validation/screener-contract re-probes every criterion daily with an absurd threshold and raises screener-field-degraded:<field> at critical when one stops filtering. A mandatory NMS/NYQ exchange and 500k volume clause blocks the OTC issues and preferred series that region = us admits. Contract in lib/data/yahoo-screener.ts.	none (keyless)

Provider	What it provides	Auth
Python runtime (GitHub Actions + local)	Free Python libraries usable from the TypeScript codebase, measure-only . Lane A retired from Vercel 2026-09-21 (243.83 MB bundle per deploy pushed team Functions Storage to 17.08 GB / 10 GB cap; nothing called it): <code>ic.py</code> now lives at <code>scripts/python/ic.py</code> , run locally, and <code>vercel.json</code> has no <code>functions</code> block. Historically Lane A (<code>POST /api/py/ic</code>) computed Spearman IC + Newey-West HAC via <code>scipy/statsmodels</code> , gated on the same <code>x-cron-secret/CRON_SECRET</code> contract as the TS crons (<code>hmac.compare_digest</code> , fail-closed when unset). Lane B (deps in <code>scripts/python/requirements.txt</code>): weekly GitHub Actions workflow for work exceeding a serverless timeout, dry-run when secrets are absent. Verified: Lane A reproduces <code>lib/edges/ic.ts</code> 's hand-rolled Newey-West to $\sim 1e-17$. Not wired into any score, eligibility, sizing, exit, promotion or broker path. See <code>features/python-runtime/FEATURE_ARCHITECTURE.md</code> .	none for Lane A; Lane B needs <code>SUPABASE_URL</code> , <code>SUPABASE_SERVICE_ROLE_KEY</code> secrets + <code>LANE_B_TABLE</code> variable
Jina AI	<code>jina-embeddings-v3</code> embeddings (1024-dim, free) + <code>jina-reranker-v2-base-multilingual</code> reranker (free, 1M tokens/month, no CC)	<code>JINA_API_KEY</code> in <code>.env.local</code>
Langfuse	LLM trace/generation observability	<code>LANGFUSE_PUBLIC_KEY</code> , <code>LANGFUSE_SECRET_KEY</code> , <code>LANGFUSE_HOST</code>
Resend	Transactional email (briefings)	<code>RESEND_API_KEY</code>
Stripe	Subscription billing (Pro/Elite tiers)	<code>STRIPE_SECRET_KEY</code> , <code>STRIPE_WEBHOOK_SECRET</code> , <code>NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY</code>
Yahoo Finance	PRIMARY US candles — <code>v8/finance/chart/{SYMBOL}?interval=1d&range=1y</code> (no key, no observed rate limit, 251 adjusted bars, works for US+India); also <code>India.NS price+candles</code> (free, no auth) + fundamentals (cookie+crumb). Recency-guarded in <code>fetchUSCandles</code> : stale source (newest bar > 4 calendar days old) is rejected before fallback chain runs.	None
NSE public JSON	Full equity list (<code>EQUITY_L.csv</code>), insider trades (<code>corporates-pit</code>), option chain, daily FI/DII net cash flows (<code>fiidiiTradeReact</code> , <code>lib/india-macro.ts</code> — India macro input, verified reachable from Vercel 2026-07-12)	Cookie handshake via <code>lib/nse-data.ts</code> / <code>lib/india-macro.ts</code>
GDELT DOC 2.0	US fallback/theme discovery only. The India per-symbol scoring dependency was retired 2026-07-31 after 0/310 usable production observations and traffic-policy throttling.	None (public API)
NSE announcements + Google News RSS	India replacement shadow : official company announcements plus bounded unofficial headline coverage. Stored in canonical evidence cache; no scorer or money-path reader.	None
~~House Stock Watcher~~	DEAD — REMOVED 2026-07-16 . Was congressional stock trade disclosures. Bucket taken private: <code>us-east-2</code> → HTTP 301 PermanentRedirect, <code>us-west-2</code> → HTTP 403 AccessDenied; project unmaintained (sibling Senate Stock Watcher is 403 too). <code>/api/markets/insider-trades</code> no longer fetches — it returns an explicit discontinued state + a System Health alert (<code>markets-congress-source:discontinued</code>), and the Markets panel renders a permanent "Discontinued" note with no retry. No free replacement qualified : Lambda Finance (<code>lambdafin.com/api/congressional/recent</code>) serves live data on HTTP 200 but its ToS forbid automated access ("any data mining, robots, or similar data gathering and extraction tools"; "personal, non-commercial use ... only") and it re-serves FMP data; Finnhub/FMP gate it behind a paid tier (violates free-cloud-only); the official House clerk feed publishes filing metadata only (no ticker/amount/side — trades are per-filing scanned PDFs needing OCR, explicitly out of scope). Restore only if a licence-clean free source appears.	— (retired)
SEC EDGAR	Form 4 CIK lookup + XML → <code>evidence_records</code>	None (public)
StockTwits	Social sentiment for US tickers	(check vault)
WandB	Experiment tracking (optional)	<code>WANDB_API_KEY</code> in <code>.env.local</code>
Robinhood MCP	JSON-RPC tool bridge for Robinhood agentic account	OAuth token in vault

Instrument-family measurement (2026-08-24): `lib/scoring/instrument-family-evidence.ts`

reuses the existing FRED cache and settled price_cache rows for GLD, SLV and GDX. It makes no Alpha Vantage call and introduces no paid provider. Each value stores source, as-of date and freshness status. Provider failure produces missing measurement evidence and cannot block the canonical ResearchAgent decision.

6. Provider adapter layer (added 2026-07-10)

Swap providers by changing one env var. Each adapter implements a shared interface; callers never know which concrete provider runs.

Embedding provider

Env var: EMBEDDING_PROVIDER=jina|openai

Location: lib/providers/embeddings/

Value	Provider	Model	Notes
jina (default)	Jina AI	jina-embeddings-v3 (1024-dim)	Free 1M tokens/month; no CC required
openai	OpenAI	text-embedding-3-small (1536-dim)	Requires OPENAI_API_KEY

Rerank provider

Env var: RERANK_PROVIDER=jina|cohere

Location: lib/providers/rerank/

Value	Provider	Model	Notes
jina (default)	Jina AI	jina-reranker-v2-base-multilingual	Free 1M tokens/month
cohere	Cohere	rerank-english-v3.0	Requires COHERE_API_KEY

Email provider

Env var: EMAIL_PROVIDER=resend|smtp

Location: lib/providers/email/

Value	Provider	Notes
resend (default)	Resend	Requires RESEND_API_KEY; test-mode sends to BRIEFING_TO
smtp	Any SMTP server (nodemailer)	Requires SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS. Implemented 2026-09-15 — it was a non-functional stub before that

SMTP was a stub until 2026-09-15, and the stub was dangerous (lib/providers/email/smtp.ts).

send() only logged "nodemailer not installed" and sendChecked did not exist. invite()

falls back to provider.send(...).then(() => ({ ok: true }))) when sendChecked is absent,

so EMAIL_PROVIDER=smtp would have reported every invitation as sent, written the access

grant, and mailed nothing — the “granted access to someone never told they have it” failure the invite route's checked-send contract exists to prevent, one env var away. It now implements `sendChecked` and `isDeliverable`, and treats an address in nodemailer's rejected list as a failure (`sendMail` resolves when *any* recipient is accepted, so reading only the resolve would turn a refusal into a grant).

Why it exists: Resend's shared `<verified-email-sender>` sender may only mail the Resend account owner, so inviting anyone else returns HTTP 403. SMTP with a Gmail app password (`smtp.gmail.com:465`, 2FA required) needs no verified domain and costs nothing; `EMAIL_FROM` must be the authenticated mailbox or Gmail rewrites the sender.

It is a global transport switch, not per-feature: everything reaching `getEmailProvider()` moves — invitations, access lifecycle notices, the daily user risk email. The newsletter Edge Function calls Resend directly and is unaffected. Bounce tracking degrades: there is no webhook, so an async bounce is never recorded, though a hard SMTP rejection is caught and the `messageId` is retained.

7. Key library files

File	Purpose
<code>lib/llm-router.ts</code>	LLM tier resolution, <code>callLLM()</code> , <code>runAgentLoop()</code> , Langfuse tracing
<code>lib/vault.ts</code>	Read/write runtime keys from <code>api_key_vault</code> table
<code>lib/av-cache.ts</code>	Alpha Vantage day-cache + daily budget guard
<code>lib/research-agent.ts</code>	Core scoring pipeline (deterministic, no LLM)
<code>lib/brokers/registry.ts</code>	Broker adapter registry
<code>lib/market-context.tsx</code>	<code>MarketProvider</code> / <code>useMarket()</code> hook
<code>lib/market-support.ts</code>	Coverage level map per route
<code>lib/india-data.ts</code>	Yahoo Finance + NSE data adapters
<code>lib/nse-data.ts</code>	NSE cookie-handshake adapter
<code>lib/system-health.ts</code>	<code>reportIssue()</code> / <code>resolveIssue()</code> helpers
<code>lib/autonomy.ts</code>	Autonomy ladder check; <code>AUTONOMOUS_LIVE_ENABLED = false</code> (constant)
<code>lib/validators/backtest.ts</code>	Validation Engine (deterministic replay)
<code>lib/evaluation/run-evaluation.ts</code>	Performance Truth Layer evaluation runner
<code>lib/robinhood-mcp-client.ts</code>	Robinhood MCP JSON-RPC client + token refresh CAS

2026-08-08: **Property owner-address resolution.** Owner-entered US addresses

travel only through the owner-gated server route to the official Census Geocoding Services API, then remain in the AES-256-GCM property payload with the returned ZIP/county resolution state. Census geography is not a parcel match, valuation, tax lookup, or insurance quote. Bengaluru address/locality

remains encrypted owner input with no automated resolver.

2026-09-08: **Property address verification (USPS)**. A second, separate

provider answers "is this address real": the official USPS Addresses API v3 (`lib/property/address-verify.ts`), OAuth2 client-credentials via `USPS_CONSUMER_KEY/USPS_CONSUMER_SECRET`, free developer account at `developers.usps.com`. Keyless until configured: with no credential the adapter makes no network call and returns `not_configured` — never a pass and never a false failure. A USPS 200 without a standardized street and ZIP is treated as `not_found` (fail closed). US only; Bengaluru returns `no_validator` because no free Indian address validator has been vetted. An unverified address is flagged on the record, never blocked from saving. Persistent outages open the single System Health key `property-address-verify:usps`.

Source: docs\arch\03-agents.md

Kairos — Agents

Current runtime/status authority: [10-current-system-reference.md](#). This chapter documents responsibilities; it is not permission to enable proposed paths.

2026-09-18: **Short interest wired into the risk-tier shadow** — item #2 (short interest) of the

`achaljhawar/1rok` gap-analysis plan. Yahoo's `defaultKeyStatistics` module (`shortPercentOfFloat`, `shortRatio`) was already fetched for every US symbol via `fetchUsOverview`'s `Finnhub-gap-fill` pass (`lib/india-data.ts`) but never parsed. Added a `BONUS_COPY_ONLY` list in `lib/data/fundamentals.ts` — deliberately separate from the `FILLABLE` gap-trigger list, since `Finnhub` can never supply these fields and adding them there would have silently forced an extra Yahoo call on every US symbol, even ones with full `Finnhub` coverage. Coverage is therefore partial (only when Yahoo is already being called for some other reason), not universal — same honest-unavailable convention as everything else. Folded into `risk-tier`'s existing `specialRisk` component as a squeeze-setup signal (`>20%` of float short + `>5` days to cover). See `features/risk-tier-gate/FEATURE_ARCHITECTURE.md` §5.

2026-09-18: **CatalystScout shadow (measure-only)** — `lib/scoring/catalyst-scout.ts`, item #1 of

the `achaljhawar/1rok` gap-analysis plan (their Catalyst agent's prompt discloses an adaptable scoring shape; theirs is LLM-judged, this is deterministic). Closes the "earnings handled only defensively, never scored for upside" gap. Zero new fetches — built entirely from data already in

scope at the `decision_observations` write: `daysToEarnings`, the FintHub analyst consensus (`lib/data/analyst.ts`, previously logged-only, still not scored live), `insider_score` gated on its own real availability flag, Webull's already-fetched 5-day capital-flow sign, and the existing breakdown veto. Written to `decision_observations.features.catalyst_shadow`; not read by `scoring/sizing/gate/order`. Same coverage-tracked additive-asymmetry discipline as `risk-tier-gate`. See `features/catalyst-scout/FEATURE_ARCHITECTURE.md`.

2026-09-18: **Risk-tier shadow (measure-only)** — `lib/risk/risk-tier.ts`, adapted from

`github.com/achaljhawar/1rok`'s disclosed (LLM-judged there, deterministic here) risk-agent scoring shape: 5-component additive composite (volatility/fragility/concentration/macro/special) off data ResearchAgent already fetches — no new provider, no LLM. Written to `decision_observations.features.risk_tier_shadow` on every scored candidate. Not read by `scoring/sizing/gate/order` — same measure-only discipline as instrument-family evidence and oil exposure. Ships with a stated, unresolved asymmetry (missing evidence contributes zero risk points, not a neutral default — opposite bias from `analyst_score`'s abstain-on-thin-evidence convention), tracked via a coverage field, not silently fixed. See `features/risk-tier-gate/FEATURE_ARCHITECTURE.md`.

2026-09-16: **Crypto Stage 3 — paper trading live (owner-approved evidence-gate override).**

`lib/scoring/instrument-taxonomy.ts` `crypto scoreMode measure_only` → `legacy_v1`. Two new small agents, `CryptoPaperTrader` and `CryptoPositionMonitor` (registry entries below), deliberately NOT threaded into the `equity/India paper-trade/position-monitor` routers. New pool: a third market value, 'crypto', scoped to the 4 paper-ledger tables only (no CHECK constraint on any of them, verified) — see "Crypto paper pool isolation" below. `decision_journal`'s market CHECK widened (blocking bug found by reading `execute_paper_exit`'s SQL body before shipping) and `lib/market-controls.ts`'s `Mkt/norm()` widened (previously coerced any unrecognized market to "us", which would have let a crypto kill-switch trip disable US equity trading). Trading page's Crypto Watch section now shows pool NAV/positions/trades; `/dashboard/symbol/<coin>` gained a technical+macro composite view in place of stock fundamentals. Evidence gate (`MIN_PREDICTIVE_DATES=20`) was NOT met (~12/20 sessions at approval) — owner directed the override explicitly; see `features/robinhood-crypto/FEATURE_ARCHITECTURE.md` and `PROJECT_DECISIONS.md`.

2026-09-15: **ResearchAgent family evidence repaired + oil exposure pack (measure-only).**

`lib/scoring/instrument-family-evidence.ts`: metal price features labelled "20bars" were really ~100-row returns (GLD recorded -7.17% vs real -0.05%); now exactly 20 settled bars, version `instrument-family-features.v2` (v1 rows immutable, evaluate separately). FRED staleness is per series (DFI110 7d, DTWEXBGS 10d, crude 10d) and a stale/missing series raises System Health family-evidence-fred-stale:<series>. New

decision_observations.features.oil_exposure_evidence for a curated US/India oil map (WTI/Brent % change from FRED, settled USO returns); unmapped symbols get nothing and no expected sign is stored. No score, eligibility, paper, live, exit or broker consumer. Details and the read-only oil counterfactual: features/instrument-aware-scoring/FEATURE_ARCHITECTURE.md §11.

2026-09-09: **scoreFundamentals gained REIT awareness (lib/data/scores.ts,

isReitSector).** A REIT's net income is structurally suppressed by mandatory real-estate depreciation, so P/E, profit margin, ROE, and EPS-sign all mean something different for a REIT than a normal company. Verified in production: the one REIT ever scored (0) fell through FINNHUB_INDUSTRY_TO_SECTOR's exact-key crosswalk (Finnhub's raw industry string, e.g. "REIT - Retail", never matches the generic "real estate" key) and landed unscored by accident. Fixing only the crosswalk would have been worse — it would then hit SECTOR_PE_NORM["real estate"] = 30, a tech-level norm that flags a normal ~45-50x REIT P/E as "rich". isReitSector matches by substring ("reit"/"real estate") rather than an exact key, since Finnhub's REIT sub-industry strings vary. Same treatment as the existing ETF branch: an honest neutral 55 baseline instead of scoring on a distorted number. No corrected numeric norm was derived or applied — only n=1 production observation exists, nowhere near enough evidence per this project's Scoring Data-Truth protocol to validate a formula. Revisit once a real FFO/AFFO data source exists. Enabled research on 4 REITs this session (`AMT DLR EQIX INVH`) — they will now score with this honest baseline instead of a distorted one.

2026-09-04 crypto basket Stage 2b: ResearchAgent now always feeds BTC-USD/ETH-USD/SOL-USD via crypto_basket discovery source (parallel to metals_basket). scoreMode="measure_only" — no paper trades, no order path. Evidence accumulates in instrument_family_observations with technical+macro+sentiment features. Trading page shows a "Crypto Watch" section (US-only) with per-coin evidence progress toward MIN_PREDICTIVE_DATES=20. Stage 3 (paper pool) unlocks after 20 qualifying sessions. No market CHECK constraint changed — crypto is market="us", InstrumentFamily="crypto". CRYPTO_SESSION_CUTOFF_UTC=0 (UTC midnight) is the declared aggregation convention, not a real market close. India has no crypto path.

2026-08-24 instrument-family measurement: ResearchAgent now stamps a versioned market/family/exposure classification on every decision, records dated metals drivers in a separate append-only measurement ledger, and writes an uncapped family shadow comparison for structurally special funds. None of these fields changes v1 scoring, paper/live eligibility, sizing, exits, or orders.

2026-08-02 earnings-date validation normalized: all earnings collectors (the earnings_calendar cache, Finnhub/Yahoo-India base, Webull, Robinhood, and fetchIndiaEarningsDate) now route through the one shared parser normalizeRealIsoDate in lib/date-only.ts. Malformed, impossible, or out-of-range provider dates fail closed to no observation instead of being coerced. tradingSessionsBetween keeps its load-bearing validate-before-shortcut ordering. Shadow mode and every score/eligibility/sizing path are unchanged.

2026-08-01 documentation truth audit: reconciled the ResearchAgent screener, five scoring dimensions, provider order, exact formulas, availability rules, weight resolution, and breakdown veto against production code. Added a plain-English report-card explanation beside the exact quantitative contract.

2026-07-31 event-aware/ADR correction: ResearchAgent batch-annotates company symbols from earnings_calendar before fetching fundamentals (1-day cache inside -3/+14 report window; otherwise/unknown 7 days; Finnhub profile 30 days). Theme Scout only proves a ticker with quote data. Reviewed US exchange ADRs persist asset_class='adr', skip structurally inapplicable Form 4 evidence, and use ADS-compatible Yahoo fundamentals without foreign-underlying fallthrough. SKHY is the reviewed Nasdaq SK hynix ADS; SKHYV, HXSCL, and HXSCF are blocked substitutes.

2026-07-31 scoring data-truth correction: ResearchAgent is the sole authoritative scorer; the legacy Supabase research-agent function is a 410 tombstone. Provider sector taxonomies are explicit. Finnhub industries are crosswalked only when unambiguous, unknown sectors omit relative-P/E scoring, and P/E outside (0, 200] is unavailable for valuation. Fundamental, sentiment, macro, and insider inclusion requires explicit availability. A weak close remains evidence, but only ATR-scaled or high-volume declines can hard-veto. See features/scoring-data-truth/FEATURE_ARCHITECTURE.md.

Last updated: 2026-07-22 (contract layer fix: macro.regime_inputs marked US-only in INTENT_CATALOG, INTENT_CLASSIFICATION, and SCORER_FIELD_CONTRACTS — contracts now match the pipeline reality that fetchMacroScore/applicableDimensions/persistObservedResearchEvidence have always enforced; shadow resolver no longer reports 0% India macro starvation; ETF analyst_score capped at 65 in ResearchAgent + archetypes; lane-comparison API added; Monte Carlo in BacktestPage; capital-rotation trigger unblocked — see history below)

Prior: 2026-07-17 (the asset allocator is now market-scoped too — India no longer inherits the US FRED regime for sleeve weights, and macro UNAVAILABLE means NO allocation rather than an untilted config echo; macro_score is US-only — India macro now honestly UNAVAILABLE instead of inheriting the US FRED regime; macro_regime reads are age-bounded (10d) + indicator-backed, fail-safe to UNAVAILABLE never to calm; SEC Form 4 URL fixed — US insider data had never resolved; insider availability now recovered from decision_observations.availability_mask at the smart-money boundary; insider symbol universe unioned across all broker accounts; ResearchAgent holdings: staleness-ordered rotation under the wall-clock budget + fail-loud holdings fetch, both markets; per-flow LLM from Settings; India GDELT news sentiment + live NSE FII/DII macro inputs; low-confidence-research quality alert; Trading Style presets govern the time-stop before a champion is promoted; India SEBI PIT evaluated as the India insider analog and REJECTED on live evidence — 0/34 India symbols clear the US open-market bar at 90d, ~70% of PIT rows are non-open-market (ESOP allotments marked "Buy"); India insider stays honestly UNAVAILABLE, pinned by tests/india-insider-not-wired.test.ts; macro_score is US-only — India macro now honestly UNAVAILABLE instead of inheriting the US FRED regime; macro_regime reads are age-bounded (10d) + indicator-backed, fail-safe to UNAVAILABLE never to calm; SEC Form 4 URL fixed — US insider data had never resolved; insider availability now recovered from decision_observations.availability_mask at the smart-money boundary; insider symbol universe unioned across all broker accounts; ResearchAgent holdings: staleness-ordered rotation under the wall-clock budget + fail-loud holdings fetch, both

markets; per-flow LLM from Settings; India GDELT news sentiment + live NSE FII/DII macro inputs; low-confidence-research quality alert; Trading Style presets govern the time-stop before a champion is promoted; Macro read (Agent Mind Phase 3) documented for the first time and scoped **US-only** — the India read is killed, not faked: BOTH its macro inputs (macro_regime AND the category='macro' learning_priors) are US-only and unmarket-tagged, and the priors were the live leak in prod row id=6, so gating only the regime would not have fixed it; the route now refuses market=india on both verbs before any LLM call or DB write, and MacroReadCard states the gap instead of vanishing. Also fixed the 4-day silent outage of that agent — deepseek-reasoner burned the whole maxTokens: 600 budget on chain-of-thought and returned empty content (finish_reason=length) from 2026-07-13 onward, so nothing was written while both crons reported success; 600 → 1500. Corrected the stale "looks back up to 3 weeks" macro line that contradicted this chapter's own 10-day consumer contract; macro_score is US-only — India macro now honestly UNAVAILABLE instead of inheriting the US FRED regime; macro_regime reads are age-bounded (10d) + indicator-backed, fail-safe to UNAVAILABLE never to calm; SEC Form 4 URL fixed — US insider data had never resolved; insider availability now recovered from decision_observations.availability_mask at the smart-money boundary; insider symbol universe unioned across all broker accounts; ResearchAgent holdings: staleness-ordered rotation under the wall-clock budget + fail-loud holdings fetch, both markets; per-flow LLM from Settings; India GDELT news sentiment + live NSE FII/DII macro inputs; low-confidence-research quality alert; Trading Style presets govern the time-stop before a champion is promoted; CPI Inflation now actually loads — MacroSentinel advertised 8 indicators and shipped 7 on every run since the FRED cutover because it fetched exactly 13 CPI readings and required 13, while FRED writes "." for a missing month (2025-10) that the adapter drops; YoY now aligns on the paired MONTH via fredSeriesDated + observationMonthsBefore rather than array index, since a gap-collapsed array makes vals[12] 13 months back — a wrong number, not just an absent one)

Update this file when: a new agent is added or removed, an agent's schedule changes, an agent's inputs or outputs change, or an agent's key behavior changes.

2026-09-14 (Per-User Broker & Risk Phase 2): UserHoldingRisk — POST
/api/agents/user-holding-risk?market=us|india, cron-gated, weekdays 11:15 UTC (India) / 21:45 UTC (US), 15 minutes after the owner's own holding-risk job so a guest fan-out never delays the owner's book. Input: one connected guest's OWN holdings, fetched with their own read-only credential. Output: user_account_snapshots + user_holding_risk_runs + user_holding_risk_snapshots, all scoped to that user. It reuses computeRiskMetrics / computeCorrelationClusters unchanged but on a **different data budget** — lib/risk/guest-risk.ts imports only the keyless Yahoo candle endpoint, never the metered fallback chain (fetchUsCandles → Massive/EODHD/TwelveData/Alpha Vantage), and no LLM. A held symbol Yahoo cannot serve is recorded as UNCOVERED with its correlation reported *unknown*, never escalated to a paid provider and never treated as zero correlation. Every user it cannot compute is recorded skipped **with a reason** (expired Zerodha token, disconnected, revoked). Advisory-only and read-only: writes no owner table and touches no order path — the guest client has no order method to call.

Adding an agent: create app/api/agents/<name>/route.ts + add cron entry in vercel.json (cloud) or scripts/run-agents.ps1 (local) + update this file (prose/registry only) + update public/agent-diagrams/system-map.json (the node, its edges, and a history entry — this is where the topology change lands) + add or update the per-agent diagram public/agent-diagrams/<agent>.json (same agentId/agentLabel/diagram/nodes/history contract; nodes is an object keyed by node id, never an array).

Agent coordination model

There are zero direct agent-to-agent HTTP calls. All coordination is via shared Supabase tables. Agents write and read a common set of tables; they never invoke each other's HTTP handlers directly.

The topology diagram lives in exactly one place: public/agent-diagrams/system-map.json, rendered at /dashboard/agents → **"System Map"** (the default diagram view). It is the single source of truth for which agent hands what to which, and it carries a history audit trail of

every edge change. This chapter deliberately does **not** redraw it — a second copy of the same edges does not add safety, it just gives a stale claim a second place to hide. (It already did: both copies asserted MACRO --> RESEARCH unqualified long after macro_score became US-only.) tests/arch-diagram-drift.test.ts enforces this.

The table below is the chapter's own altitude: **which tables** carry the coordination, not which arrows exist.

Table-to-agent matrix

Table	Written by	Read by
macro_regime, macro_signals	MacroSentinel	ResearchAgent (US only), Macro read (US only), Dashboard
macro_interpretations	Macro read (US only)	Macro read's own GET → MacroReadCard on /dashboard/markets. Nothing else — no scoring/sizing/gate/order/exit path reads it
agent_signals	ResearchAgent, DeepSeekAgent	PaperTrader, TraderAgent, Dashboard
signal_score_history	ResearchAgent	ResearchAgent (trend), Dashboard charts
decision_observations	ResearchAgent	LearnerAgent, Validation Engine, PerformanceTruth
instrument_family_observations	ResearchAgent	Owner-only family diagnostics; no scoring, paper, live, sizing, exit, learner-promotion, or broker consumer
edge_signals	EdgeScout	ResearchAgent (US candidate admission only ; fresh relative-strength provenance only), EdgeIC, Edge dashboard
paper_positions	PaperTrader	PositionMonitor, LearnerAgent, Dashboard
paper_trades	PaperTrader, PositionMonitor	LearnerAgent, MentorAgent, PerformanceTruth, lane-comparison API (closed trades → strategy lane NAV curves + Monte Carlo)
strategy_versions	LearnerAgent, User	ResearchAgent, Dashboard
trade_memories	PositionMonitor	ResearchAgent (RAG retrieval)
agent_alerts	All reporters	Health-Triage, Dashboard, Briefing
mentor_insights	MentorAgent	LearnerAgent (context), Dashboard
strategy_evaluations	PerformanceTruth/Evaluation	Dashboard
benchmark_scorecard	BenchmarkScorecard route	PerformanceTruth dashboard, future gated learner evidence
rotation_events	PaperTrader capital-rotation shadow evaluator	Dashboard/audit, future rotation validation
validation_experiments, model_artifacts	Validation/Calibration	Promotion gate, Dashboard
broker_orders	Execution Gateway + order sync	Reconciliation, Trading Dashboard (live orders surface on /dashboard/trading — read-only, pending/submitted/filled/error/unknown_needs_reconcile statuses, last 20 rows, market-scoped)
broker_order_events (target)	Execution Gateway + order sync	Audit/reconciliation
trade_proposals (shadow)	AutonomousShadow	Dashboard, owner audit
trade_proposals (live)	AutonomousLive	Dashboard, reconciliation
broker_order_events	AutonomousLive, Execution Gateway	Audit/reconciliation
llm_call_log	All LLM callers	Admin cost view
rag_traces	ResearchAgent (retrieval)	Debug/audit

Agent registry

MacroSentinel — the economist

File: app/api/agents/macro-sentinel/route.ts

Schedule: Mondays 8:00 AM ET (Windows Task Scheduler)

LLM: None — fully deterministic

Inputs:

- 8 Alpha Vantage macro endpoints: Treasury yield 10Y + 2Y, unemployment, real GDP, nonfarm payrolls, CPI, retail sales, federal funds rate, durable goods orders

What it computes:

$$\text{danger_score} = \sum (\text{indicator_value} \times \text{weight} \times \text{direction_sign})$$

Each indicator has a hardcoded `direction_sign` (+1 bad, -1 good) and weight summing to 1.0.

danger_score	regime
0–24	GREEN
25–49	YELLOW
50–74	ORANGE
≥75	RED

Outputs:

- One `macro_regime` row (current regime + score)
- One `macro_signals` row per indicator (raw_value + contribution + direction)

Key behavior: Advisory-only. MacroSentinel never auto-throttles agents or halts trading.

User sees the regime first and decides whether to act.

Scope: US ONLY. Every indicator is a US series (yield curve, Sahm rule, US GDP, nonfarm payrolls, US CPI, US retail sales, fed funds, US durables). `macro_regime` has ****no market column, so its verdict is US-only regardless of which symbol reads it.** It must never score a non-US symbol.** Consumers are responsible for market-gating their own reads — `lib/data/scores.ts` does this for the money path (India → macro UNAVAILABLE). A real India regime would be a separate agent + a market column; `lib/india-macro.ts` (FII/DII) is thesis evidence, NOT a regime.

Consumer contract for `macro_regime` (money path — `lib/data/scores.ts`):

A row may score a symbol only if **all** hold. Otherwise macro is UNAVAILABLE → excluded from the weighted score, weights renormalize. It never falls back to a calm/green default.

Check	Rule	Why
Market	symbol is US	agent is US-only; table has no market column
Verdict	regime != 'unknown'	an unknown row's danger_score is a placeholder 0, not a calm read

Check	Rule	Why
Freshness	week_of within MAX_MACRO_AGE_DAYS = 10	weekly cadence (7d) + 3d cron slack, and strictly < 14 so a fully missed run can never be masked by riding the prior week
Evidence	raw_indicators length >= 3	mirrors MacroSentinel's own classification floor; rejects pre-guard legacy rows that stamped green/danger-0 off zero indicators

signals_triggered = 0 is **NOT** treated as suspect on its own — a genuinely calm week really does trip zero signals, and rejecting those would bias the book bearish. The honest discriminator is the indicator count.

****Second consumer — the asset allocator (lib/allocation/allocator.ts + lib/allocation/regime.ts, 2026-07-17).** computeAllocation(svc, market) maps the regime → sleeve target weights; its equity target may only ever **shrink** that market's gross-equity cap in PaperTrader sizing. It applies the **same four checks as the table above** and imports MAX_MACRO_AGE_DAYS from lib/data/scores.ts rather than re-declaring the bound, so the two money-path consumers cannot drift on what "too old to act on" means.

It previously filtered strategy_sleeves by market but read macro_regime **unscoped**, so the US FRED verdict tilted India's sleeves — the same contamination class as the macro_score defect, and latent only because allocation_enabled defaults off (verified false in prod). India sleeves *do* exist in prod (equity 70 / defensive 20 / cash 10), so this was live-capable, not theoretical. India now returns UNAVAILABLE **without querying** macro_regime **at all**.

Macro UNAVAILABLE → NO allocation (null), not an untilted allocation. The regime is the only input that turns owner-configured sleeve rows into an allocation; emitting the base target_pct with tilt=0 would be byte-identical to a real "macro says neutral" verdict — the neutral-50-and-included fake in a different hat. For India this is permanent until a real India regime is built; whether India should instead run *static* sleeve targets is a product call that is deliberately left to the owner rather than silently made. **Accepted tension, stated not hidden:** because the equity target only ever shrinks the gross cap, null leaves the owner's configured max_gross_exposure_pct in force — *looser* than any regime outcome, including risk_off. That is the documented default (identical to allocation_enabled=false, today's live state), not a fabricated one; tightening on absent evidence would be an unapproved position change justified by nothing. Callers already handle null (it is the shipped-off path). computeAllocationDetailed() returns the same result plus a reason naming the specific rejected row(s).

Known gap (not a regression — pre-existing, reported 2026-07-17): normalizeRegime matches
bull/bear/risk_on/risk_off/aggressive/defensive, but MacroSentinel writes
green/orange/red/unknown. **Every real label therefore normalizes to neutral, so the
regime tilt is dead in prod today.** Fixing the vocabulary would *activate* tilting — a real
behaviour change requiring its own approval — so it was deliberately left alone.

Proposal (not applied — schema change): give macro_regime a market column so the table can
express what it actually is, instead of every consumer having to remember to market-gate its own
read. Until then, "US-only" is a convention enforced consumer-by-consumer, and each new consumer
is a fresh chance to reintroduce this exact bug.

Macro read (Agent Mind Phase 3) — the narrator

File: app/api/agent-mind/macro-read/route.ts (+ pure logic in lib/macro-read.ts)

Schedule: weekdays — kairos-macro-read-us (13:30 UTC). **Cadence:** US only.

Writes: macro_interpretations (one cached row/day/market)

Read by: its own GET → MacroReadCard on /dashboard/markets. **Nothing else.**

Turns the US macro regime + the US book + the system's US macro priors into a plain-English "what this means for your book" narrative.

NARRATIVE ONLY. This is the LLM sibling of macro_score, and it is on **no** money path: macro_interpretations is written here and read back by this route's own GET alone. The deterministic rule stands — no LLM on any scoring/sizing/gate/order/exit path — and this route must never widen its reach.

Scope: US ONLY. The India read is killed, not faked. (2026-07-17)

Both of this read's macro inputs are US-only and **neither is market-tagged**:

Input	Why it is US-only	Market column?
macro_regime	MacroSentinel = 8 US FRED series (see above)	No
learning_priors WHERE category='macro'	US beliefs: Fed funds, DXY, 2Y/10Y curve, ISM/PMI, VIX	No

The priors were the live leak, not just the regime. Prod macro_interpretations id=6
(2026-07-13, market=india) reads: "...no regime-based bias can be assigned to this India book.
The system's high-conviction belief (80%) that rising Fed funds rates comp[resses]..." — against
a 13-position India book, while the regime was already unknown. So market-gating only
macro_regime would not have prevented that sentence. Prior id=8 ("Rising Fed funds rate
environment...", confidence 0.80) did it.

Why killed rather than "honestly scoped": once both US-only inputs are withheld, an India read has **zero** macro evidence left — only the held symbols. An LLM asked for a macro read with no macro input either restates a constant ("we know nothing" — which does not need a deepseek-reasoner call every weekday) or reaches into training knowledge for RBI/rupee/FII colour, inventing exactly what the prompt forbids. So:

- The route **refuses** market=india on both **GET** and **POST** before any LLM call or DB write — zero

spend, zero rows, and the legacy India rows (ids 2/4/6) stay unreachable.

- MacroReadCard renders a **deterministic not-supported note** for India instead of vanishing.
- This also resolves a self-contradiction: the India Markets view already renders a `NotSupportedNote` for `"...macro sentinel..."` while a cron wrote an India read derived from that same US-only sentinel.

Parity means both markets get an **honest** answer, not that both get an LLM call. `lib/india-macro.ts` (NSE FII/DII) is deliberately **not** wired in as a substitute regime — that is a separate build.

US path reuses the same discipline as `lib/data/scores.ts` (unknown-guard, 10-day age bound, `raw_indicators >= 3` floor, fail-safe to `UNAVAILABLE`). When no row qualifies the prompt states the regime is **UNAVAILABLE** and interpolates **no** regime, danger score, summary or indicators — an absent verdict is never described as calm. `MAX_MACRO_AGE_DAYS` is imported from `scores.ts`; the indicator floor is restated in `lib/macro-read.ts` (it is module-private in `scores.ts`) and ****must be kept in sync****.

2026-08-01 correction: the obsolete `kairos-macro-read-india` cron was removed.

India remains without a macro narrative by design until market-local, source-backed exogenous observations exist; the route-level refusal remains defence in depth.

ResearchAgent — the analyst (the brain)

File: `app/api/agents/research/route.ts`, `lib/research-agent.ts`

Schedule: US 09:00 + 14:00 ET and India 09:30 + 12:30 IST on trading weekdays. US slots are DST-safe paired UTC jobs admitted by exact New York local time. Closed-day catch-up is non-executable.

LLM: per-flow from Settings → AI Models (agent_config row `research`, read via `getConfiguredModel`), default `deepseek-v4-pro` with thinking enabled. Writes thesis text only — scores are deterministic. (Was hardcoded Groq Llama pre-2026-07-12.)

Inputs:

1. Account-scoped holdings snapshots. Research may analyze approved holdings, but only holdings verified on the actual order account can authorize a SELL.
2. Watchlist from `watchlist` table
3. Screener candidates from the **Yahoo custom screener** `POST /v1/finance/screener` (US, keyless, primary since 2026-08-02; `FinancialDatasets POST /financials/search/screener` remains the fallback) or NSE universe cache (India) — dual buckets. `FinancialDatasets` is supplemental discovery only: missing credentials, exhausted credits, HTTP failures, and timeouts open a self-healing System Health warning and return no screener candidates; holdings/watchlist/carry-forward research continues, and scoring fundamentals are unaffected:
 - **US fundamental-momentum screen**: revenue growth >15%, earnings growth >10%, gross margin >25%, ROE >15%, market cap >\$2B; ranked by revenue growth. It does **not** use RSI or a moving average.
 - **US value screen**: $0 < P/E < 18$, FCF yield >4%, debt/equity <1, market cap >\$1B; ranked by lowest P/E. It does **not** compare P/E with a sector median or require insider/analyst activity.
 - The two US lists are interleaved before the six-name screener cap, so one bucket cannot silently crowd out the other. India candidates come from the separately scored rotating `india_screen_cache`; the US criteria above are not applied to India.

- **Yahoo screener field contract (2026-08-02).** The Yahoo endpoint is undocumented and a criterion can be accepted by the API and silently discarded — no error, no exception, no failing test, just a bucket quietly wider than its definition. `freecashflow.lasttwelvemonths` is already in that state (an absurd threshold returns the unfiltered count), so the value bucket ships with **no free-cash-flow leg**: $0 < P/E < 18$, $\text{debt/equity} < 1.0$ (expressed as < 100 — Yahoo uses percentages, not ratios), $\text{market cap} > \$1B$. `netincomemargin` is honoured but is earnings-derived, so it is not substituted; cross-checking earnings against earnings would be circular. `POST /api/validation/screener-contract` (cron `kairos-screener-contract`, 11:10 UTC weekdays) re-probes every criterion with an absurd threshold and raises `screener-field-degraded:<field>` at critical when one stops filtering. Each probe reports three distinct states — verified, proven no-op, and **unverifiable** — because collapsing the last two meant a total provider outage returned a clean bill of health, the check reporting loudest exactly when it knew least. When nothing can be probed it raises `screener-contract-unverifiable` instead of reporting no degradation. A mandatory exchange (NMS/NYQ) and volume ($> 500k$) base clause is applied to every query and is not caller-overridable: without it `region = us` admits OTC issues and preferred series (measured: `NLY-PF`, `NLY-PG`, `GGPSF`) that the broker cannot fill.

- **Discovery provenance through the carry-forward queue (2026-08-04).** `research_queue.discovery_source` existed but was never written, and `gatherSymbols` re-added every carried-forward symbol as "watchlist". A screener candidate that overflowed `RESEARCH_CANDIDATE_CAP` (40) therefore came back **relabelled**, so `decision_observations.discovery_source` under-counted screener discoveries and over-counted the watchlist. The "US screener 0 / watchlist 151" reading that motivated `discovery-starved:us` and the deferred-candidate alarm was **partly this attribution bug, not only starvation** — measured 2026-08-04: the Yahoo screener returned 72 momentum / 86 value matches, 19 of the top 22 had never been researched, and four of them (`BBWI`, `LYFT`, `NLY`, `PBR`) were sitting in a 43-deep `research_queue` with `discovery_source` null on every row. Provenance is now carried on all three queue paths (cap overflow, budget tail, failed-candidate retry) and never overwritten with null on a re-defer. Until this shipped, no discovery metric could be trusted, including the two alarms built on one.

- **Discovery-only run (2026-08-04).** `?scope=discovery` restricts the batch to never-held discovery buckets AND exempts `screener_momentum / screener_value / edge_relative_strength` from `RESEARCH_CANDIDATE_CAP` (40), exactly as the metals basket and region ETFs are already exempted. The exemption is required, not cosmetic: `candidateMap` is ordered manual → carry-forward → watchlist → screener, `applyCandidateCarryForward` keeps the top 40, and screener names are last, so they overflow into `research_queue` **before** `gatherSymbols` returns — the run's entry filter never sees them. The 2026-08-04 discovery run scored exactly `SLV/GDX/IAU/INDA/EPI/INDY`, the post-cap appended buckets, and nothing else, which is what exposed this. Bounded by `RESEARCH_SCREENER_MAX` (6) and `RESEARCH_EDGE_DISCOVERY_MAX` (4), so it adds ~10 symbols to a run that carries no holdings. The main run is unchanged: exit re-scoring keeps its full priority and budget and runs on its own cron and budget (`kairos-research-discovery-us` 14:30 UTC, `kairos-research-discovery-india` 05:00 UTC), after each market's main run. Root cause it fixes: candidates are ordered holdings → manual watchlist → carry-forward → watchlist → screener and the wall-clock budget cuts from the tail, so with 54 US holdings and 16 watchlist names against a ~100-symbol batch already deferring ~51, screener candidates were structurally last and never scored — **zero screener-sourced decisions across all of 2026-07, independent of whether discovery itself worked**. Discovery and exit re-scoring were competing for one budget and exits rightly win, so the funnel could never reach the ledger. Holdings are excluded from this run outright, so it cannot touch an exit/SELL path. Alert keys are suffixed `:discovery` so the two runs cannot resolve each other's alerts, and the holdings-deferred resolve is guarded on `!discoveryOnly` — a run carrying no holdings must not clear a shortfall the main run just raised.

- **Deferred-candidate alarm (2026-08-03).** `discovery-starved:us` fires when the screener RETURNS nothing; it cannot see the case where the screener returns candidates and the research wall-clock budget then scores none of them. Both end with zero screener-sourced decisions in the ledger, but only one raised an alert — so a healthy provider and a starved funnel looked identical. The research cron now separately alerts when every screener candidate in a batch was deferred. Kept as a distinct key rather than reusing `discovery-starved:us`: the remedies differ (provider vs throughput), and two writers on one key would fight, one resolving what the other just raised.

- **Discovery coverage alarm (2026-08-02).** `runScreener()` reports the `discovery-starved:us` System Health issue whenever a run yields zero screener candidates — warn on the first dry day, critical once `screenerDrySpellDays()` shows three consecutive research days with no `screener_momentum / screener_value` decision, auto-resolving on the first run that yields one. Days research did not run are absent from `decision_observations` and are not counted, so a weekend cannot escalate the alert. This watches the `*outcome*`, not a specific provider fault: it fires on a funded key returning nothing or a silent contract change, both of which the pre-existing `provider-unavailable:financialdatasets` issue misses. Added after the `FinancialDatasets` balance reached \$0.00 on 2026-07-29 and US discovery ran for ten days on holdings and watchlist alone — every component reporting healthy while research re-scored names it already held.

4. Score trend from `signal_score_history` (last 5 rows per symbol)

5. Champion weights from strategy_versions WHERE is_champion = true AND market = ?
6. Macro regime from the most recent **usable** macro_regime row — **US symbols only**. A row is usable only if it has a real verdict (regime != 'unknown'), is within MAX_MACRO_AGE_DAYS (10) of week_of, and rests on >= 3 real indicators. Otherwise macro is UNAVAILABLE (excluded, never defaulted to calm). India never reads this table. (lib/data/scores.ts, 2026-07-16)
7. RAG memory via retrieveSimilarTrades() (if Jina embeddings are configured — Voyage was replaced by Jina free tier 2026-07)
8. India news/event replacement shadow — NSE corporate announcements + bounded Google News RSS, persisted to the canonical evidence cache but **not read by scoring** (2026-07-31). The old zero-output GDELT dimension is retired.
9. India FII/DII net cash flows — live NSE (lib/india-macro.ts), injected into the India thesis prompt (2026-07-12)

Webull boundary: Webull analyst/extended research is collected by the Evidence Router shadow, not called synchronously by processSymbol. The old inline path could issue up to nine MCP tool calls per US symbol and repeatedly exhausted the per-symbol deadline. Authoritative research may consume Webull only after a separately approved cache-only reader passes Router parity; until then it cannot consume scoring capacity, alter scores/directions, or delay holdings research.

Current production baseline (deterministic_v1) — 5 dimensions:

Plain-English model: a five-subject report card

Think of each stock as receiving a report card. ResearchAgent, not the LLM, is the teacher doing the arithmetic:

Subject	Child-friendly question	Important limitation
Fundamentals	"Is the business healthy and reasonably priced?"	Quarterly company facts move slowly. An analyst target is written in the evidence, but currently earns zero points .
Technicals	"Is the settled price trend healthy, or did the stock just break down?"	Uses completed daily bars, not today's unfinished candle. It describes price behavior; it does not know the business.
Sentiment	"Are enough real news/social observations leaning positive or negative?"	Tiny samples are pulled toward 50. No data means the subject is omitted, not treated as a neutral vote.
Macro	"Is the current US economic weather supportive?"	US only. India does not inherit the Fed/FRED result.
Insider	"Are US insiders spending more money buying than selling in real open-market trades?"	US domestic issuers only; sparse data and ADR/India inapplicability are omitted.

Every available subject receives 0–100. A missing or structurally meaningless subject is removed and the remaining weights are rescaled. Fewer than two usable subjects means **abstain**. The resulting analyst_score is a ranking/eligibility input, not a promised return or a predicted price. The configured LLM may explain the facts, but it cannot change any subject score, weight, direction gate, or order.

Dimension	Source	What it measures
fundamental_score	Finnhub → Yahoo → FMP → AV reserve (domestic US); Yahoo-only ADS basis (reviewed ADR); Yahoo (India)	Sector-relative P/E when taxonomy maps safely, profit margin, ROE, EPS sign, and revenue growth YoY. Analyst target upside is persisted as observational_only and contributes no points. Cache is 1d near earnings, otherwise 7d.
technical_score	Yahoo → Massive → EODHD → Twelve Data → AV reserve (US); Upstox → Yahoo (India), all normalized to completed daily candles	RSI(14) continuous curve, EMA20/50, 20d trend, volume confirmation, and an ATR/high-volume breakdown veto. A weak bottom-quartile close is warning-only. A still-forming daily bar never enters scoring.

Dimension	Source	What it measures
sentiment_score	StockTwits + GDELT first; AV NEWS_SENTIMENT only when both free sources are unusable (US only)	Sample-shrunk US news/social bullishness. StockTwits requires ≥5 tagged messages. When both social and news exist they blend 40%/60%. India: structurally not applicable in the active scorer ; replacement headline/event evidence is shadow-only.
macro_score	macro_regime.danger_score — US ONLY	Macro backdrop from MacroSentinel. India: dimension is UNAVAILABLE and excluded — weights renormalize onto the remaining dimensions (2026-07-16). MacroSentinel is US-only by construction (8 US FRED series) and macro_regime has no market column, so scoring an India symbol from it stamped the US Fed's verdict onto Indian equities. India research still injects a factual FII/DII net-flow line (lib/india-macro.ts, live NSE) into the thesis prompt — narrative grounding only, deliberately NOT wired into macro_score; a real India regime is a separate build. FII/DII is null (line omitted) when NSE geo-throttles Vercel. (2026-07-12)
insider_score	US: Massive Form 4 → SEC EDGAR Form 4 → AV INSIDER_TRANSACTIONS (cascade, resolveInsider, first available:true wins). India: none wired — the dimension is excluded, not scored.	90-day open-market (P/S only) buy/sell value ratio. India's SEBI PIT feed (fetchNseInsider) was evaluated as the analog on 2026-07-17 and rejected : 0/34 live India symbols clear the US ≥3-open-market-txn bar at 90d, and only ~30% of PIT rows are open-market at all (ESOP allotments are marked "Buy"). See the India insider block below. Its agent_signals.insider_score default-fills 50, but decision_observations.availability_mask.insider is false, which is the honest record — do not read the 50 as neutral evidence. (2026-07-17)

Sub-score formulas are deterministic and **fixed** (hand-tuned priors in lib/data/scores.ts + lib/data/technical.ts) — they are NOT agent/genome-mutable. Only the dimension **weights** evolve through an explicitly promoted champion. New candidate features flow through the validation/registry process, not by silently editing these formulas. Volume is scored; analyst-target upside is intentionally observational-only.

Technical calibration shadow (2026-07-21): EdgeScout now measures the exact kairos_technical_score_v1 composite beside bounded macd_atr_12_26_9 and signed_adx_14 challengers. EdgeIC evaluates them independently for US and India and emits market-wide plus sufficiently broad sector diagnostic rows from the same already-fetched candles. This is measure-only: it does not add MACD/ADX to the score, create sector-specific parameters, or change any agent decision. Formula, dataset, segment, and run fingerprints preserve later recalibration history.

US relative-strength discovery (2026-08-01): ResearchAgent may read a fresh completed-session EdgeScout snapshot to add at most four non-ETF US symbols to its candidate batch when both six-month relative strength and 52-week-high proximity are positive within that snapshot. This is **admission only**: the edge values are stored as provenance, never added to analyst_score, thresholds, sizing, exits, or orders. The usual deterministic scoring and portfolio gates remain authoritative. India is not included until an equivalent candidate contract is verified.

The Research Journal renders those immutable admission measurements beside the decision as **Admission evidence**. They explain why the candidate was researched, not why a trade was eligible; the canonical score and downstream terminal state remain separate.

Calibration readiness monitor (2026-07-21): A weekly deterministic monitor

collapses same-window provider revisions, counts only market-wide windows at least five calendar days apart, and publishes progress per edge/market/horizon. Six stable historical windows can request the PIT/walk-forward/cost/FDR validation build; four qualified validation windows can request shadow review. Both are review milestones, not lifecycle promotion or trading permission. The monitor makes no provider call and emits a one-time informational notice plus a warning if collection is stale >10 days.

Weighted composite (availability-masked + renormalized):

$$\text{analyst_score} = \sum (\text{dimension_score} \times \text{effective_weight}[\text{dimension}])$$

Missing/inapplicable dimensions are EXCLUDED and the remaining weights renormalized to sum to 1.0 (lib/scoring/weighted-score.ts); < 2 usable dimensions → abstain (thin evidence), never a low score. Weight source: this market's promoted champion weights_snapshot; if none exists, the selected risk-profile static weights; if the profile is invalid, balanced F.30/T.25/S.20/M.15/I.10. The mandate's strategy tilt is then applied. The legacy signal_weights and learning_priors tables are not live-score fallbacks.

ETF score cap (2026-07-22): ETFs exclude fundamental + insider dimensions; after renorm, technical carries ~62% weight. Low-volatility ETFs (SGOV, VTV, SCHD) were scoring 77–80 — above single-name equities — and the capital-rotation shadow proposed selling PLTR to buy SGOV. A hard cap of ETF_SCORE_CAP = 65 is applied post-computation in both lib/research-agent.ts (main analystScore + challenger shadowScore) and lib/scoring/archetypes.ts (computeArchetypeScore for etf_trend). ETFs can still surface as SELL targets for held positions and can enter the book up to score 65; they cannot displace an equity candidate scoring ≥ 66.

Instrument-family measurement (2026-08-24): the universal cap remains the actionable v1 rule, but it is not treated as proven for every fund. The deterministic instrument-taxonomy.v1 separates bullion, silver, miner funds, metal producers, royalty/streamers, common fund families, known India ETFs, leveraged/inverse funds, and unknowns. GLD/IAU share gold_spot; GDX is gold_miners_fund; KGC is a producer; FNV is a royalty/streamer. ResearchAgent persists this point-in-time classification under decision_observations.features.instrument and appends a measure-only family row. Metals additionally record FRED DFII10, FRED DTWEXBGS, and settled GLD/SLV/GDX relationships with source/as-of/status. The dedicated shadow row family_uncapped_v1:<family> measures cap information loss only. The LLM cannot classify, score, promote, or trade from these fields. The Research Journal displays them separately from the v1 quant audit.

Indicative trade plan (2026-07-20): ResearchAgent now records the latest scoring-candle close in decision_observations.price_at_decision and a versioned native-currency features.trade_plan. Eligible new longs get deterministic mandate-based approximate risk/target levels; rejected and held names do not. This adds no provider call and no LLM number. The plan is audit/presentation context only; a close older than seven calendar days is marked unavailable.

Low-confidence output (2026-07-12): ResearchAgent aggregates per-market evidence availability across a run; when ≥ 50% of scored symbols (min 2) were scored on < 2 of 5 dimensions, it raises a low-confidence-research:<market> System Health alert (warn/data) naming the commonly-missing dimensions, and resolves it when a run recovers. Surfaces *quality* gaps (thin data)

alongside the existing **quota** alerts (provider budget).

Holdings ordering + fail-loud (2026-07-16, bug fix — both markets):

Holdings lead the batch and are exempt from the candidate cap, but they are **not** exempt from the cron's wall-clock budget (RESEARCH_BUDGET_MS, ≤105s). Throughput is ~30 symbols/run; the US book was 56. Because holdings order was **stable** (broker capture order, later alphabetical), the budget decapitated the **same tail every run, permanently** — prod run a4530e8f (07-16) wrote signals for batch slots 1-30 and **zero** for slots 31-56, all holdings. AVGO (slot 55) went unscored 07-13 → 07-16 while Risk Analytics advised trimming it. enqueueDeferred could not rescue them: gatherSymbols rebuilds holdings from the broker snapshot each run and addCandidate drops any symbol already in holdingSet, so a deferred holding re-entered at the same starved index forever.

- **Holdings are staleness-ordered** — least-recently-scored first (orderHoldingsByStaleness, lib/research/holding-symbols.ts), from this market's own agent_signals. The budget's cut now **rotates**, bounding worst-case staleness at $\text{ceil}(n\text{Holdings} / \text{throughput})$ runs instead of never. Applies to US and India identically.
- **fetchHoldings fails loud** — the old catch { return []; } made a broken holdings read indistinguishable from "owns nothing", so research would score new BUYs while blind to every position it might need to SELL. It now raises a research-holdings-fetch:us critical System Health issue and **throws** (no holdings visibility → no run). PostgREST error is checked, not just data.
- **India parity** — a rejected Kite call raises research-holdings-fetch:india-kite (warn, auto-resolves) and continues on the paper book, since a token lapse is expected/recoverable; an India paper_positions DB fault aborts like US.
- **Deferred holdings alert** — any held symbol the budget misses emits a warn cron alert naming the symbols. This is the only signal that the book exceeds one run's throughput, since the deferral queue can't carry holdings.

Known capacity limit (not fixed by ordering): with 56 holdings and ~30/run, **every holding every session** does not fit in one 150s maxDuration invocation. Rotation bounds staleness at ~2 runs; closing it fully needs a capacity decision (raise RESEARCH_PARALLEL, or split holdings into their own cron) — a design change, deliberately not taken here.

Sub-score formula reference (deterministic_v1, exact values)

Each dimension outputs 0–100, clamped. Source of truth: lib/data/scores.ts, lib/data/technicals.ts.

Fundamental (scoreFundamentals) — base 50. At least two real fields among P/E, margin, ROE, EPS, and revenue growth are required before the dimension is marked available. The function emits a display placeholder of 55 for an ETF or missing overview, but that placeholder is excluded from the composite and is not evidence. Additive when available:

Field	Bands → points
P/E (sector-relative)	$\text{ratio} = \text{pe} / \text{SECTOR_PE_NORM}[\text{sector}]$. <0.7 → +18 · <1.0 → +8 · <1.4 → -3 · <2.0 → -12 · ≥2.0 → -22
Profit margin	>0.20 → +20 · >0.10 → +10 · <0 → -20
ROE (TTM)	>0.20 → +15 · >0.10 → +8 · <0 → -10
EPS	>0 → +5 · ≤0 → -10
Rev growth YoY	>0.20 → +15 · >0.10 → +8 · <0 → -10
Analyst target upside (target-price)/price	0 points. Stored as analyst_target_mode='observational_only' for audit/display only.

SECTOR_PE_NORM: technology 30 · communication 20 · health care 25 · consumer disc. 24 · staples 22 · industrials 20 · materials 16 · energy 12 · financials 14 · utilities 18 · real estate 30. Missing/unmapped sector taxonomy omits the P/E component; it does **not** receive a made-up default norm. Nonpositive P/E and P/E >200 are also omitted.

Technical (scoreTechnicals) — base 50; <15 candles → flat 50. Additive:

Signal	Contribution
RSI(14)	continuous interp over anchors (20, -20) (35, -16) (45, -5) (50, +2) (55, +12) (60, +25) (72, +25) (75, +6) (85, -10) (100, -15)
Price vs EMA50	above +15 · below -15
Price vs EMA20	above +10 · below -10
20-day trend (±3% band)	up +10 · down -10
Volume vs 20d avg (direction-confirming)	in-direction $\geq 1.5\times \rightarrow \pm 8 \cdot \geq 1.2\times \rightarrow \pm 4$ (direction from EMA20/trend; neutral context $\rightarrow 0$)

Breakdown veto (detectBreakdownVeto, runs FIRST) — before the additive math, a deterministic crash/meme-reversal check caps the technical score at 20 when the last bar is down ≥ 2.5 ATR, or $\leq -7\%$ on $\geq 1.5\times$ volume. A bottom-quartile close on a down bar is persisted as a **warning only** and does not veto. This fixed the case where a -12% high-volume reversal scored ~100 because RSI fell into the preferred band while price remained above its EMAs.

Sentiment (fetchSocialSentiment + scoreSentiment) — StockTwits and GDELT are fetched first. Fewer than 5 sentiment-tagged StockTwits messages is unavailable. Each available component is shrunk toward 50: social uses neutral prior $K=10$; news uses $K=5$. If both exist, $\text{sentiment} = 0.4 \times \text{social} + 0.6 \times \text{news}$; otherwise the available component wins. AV NEWS_SENTIMENT ((sent+1)×50) is called only when both free sources fail. The final value is excluded unless has_data=true; the label fallback exists for legacy shapes but cannot make a failed current fetch available.

Per-run evidence contract. Every ResearchAgent packet retains the input facts and derived values used for its five dimension scores in research_packets.raw_data._scores.evidence, including the completed-session close and candle date for technicals. signal_score_history retains the immutable score trajectory and links each point back to its packet. The user-visible audit surfaces are Research Journal's **Quant audit** for that run and Score Tracker's **Point Detail** for score-to-score comparisons. This is an explanation record, not a second scorer: displayed facts never alter an already-recorded decision.

The Fundamentals audit page follows the same point-in-time rule. Its all-runs view reads the four stored per-run breakdowns and never joins a historical decision to the latest TTM company facts. Every applicable value that resolved is shown; missing provider evidence is unavailable, structurally meaningless evidence is inapplicable, and neither state is presented as a measured neutral value. The score summary includes insider because the composite has five dimensions. Active and measure-only technical inputs are labeled separately; displaying MACD, ADX, EMA200 or relative strength does not make them part of deterministic_v1.

The Fundamentals action history uses an event read model over the existing ledgers. A closed long paper_trades lot remains order_side='buy', so the UI expands it into a BUY fill at entry and a SELL fill at exit_at; it does not mistake the storage row for a single perpetual BUY. Live proposals, broker-order states, and fills are displayed as distinct stages. Only fills are executions; a pending/rejected/expired proposal is a recorded app decision, not a trade. The table's latest Trade value is market-scoped across actual paper/live fills.

Why column (2026-09-15). Next to Trade, trade_why explains in plain English what the PAPER trader last decided for each market+symbol (bought, not bought, no trade, sold) and why, with the numbers that mattered (score vs threshold, open positions vs cap, sector, remaining room vs the minimum trade size). The single mapper is lib/trading/trade-why.ts; /api/research/universe (latest mode) feeds it, newest decision first: the newest trading-stage chain in pipeline_stage_events (grouped by signal_id), the newest research stage event, the newest decision_observations row when no stage event exists, then the last paper trade

(exit reason included). A decision older than 14 days is prefixed "As of MM-DD"; with no data at all it reads "Not researched yet". Shadow and measurement stages (risk_plan, earnings_risk_shadow, correlation_shadow, capital_rotation) are never shown as the reason. Reads cover only the response's symbols, in chunks of 200, newest first, stopping once every key is found (cap 5,000 rows per source per chunk, 400 per-key observation lookups). A read error leaves the cell blank and does not fail the page. To keep the column complete for future symbols, every skipped.push and filled.push in the paper-trade route now writes a stage event. This was logging only; no gate changed. tests/trade-why.test.ts fails if a new skip path stops logging.

Feature-pack catalog (P0, 2026-08-02). lib/feature-packs/catalog.ts is a typed read model that classifies a decision's inputs as active v1, measure-only, observed-only or inapplicable for its instrument family. Research Journal renders this classification from stored evidence only; it never fetches data or changes a decision. Strategy Library uses the same catalog to label manual Scanner support, shadow-only rules and unsupported conditions. The catalog has no score, paper, live, exit, sizing, broker or feature-registry writer.

Feature registry lifecycle (2026-08-02). Learner-proposed formulas are proposed, quarantined, measure_only, or retired. A deterministic IC screen can only advance a formula to measure_only; it is recorded under decision_observations.features.measured_feature_values and cannot alter a score, eligibility, size, paper/live proposal, exit, or broker order. Any future score use requires the separate market-local replay, shadow, challenger and owner-promotion path in docs/arch/09-learning-loop.md.

Macro (fetchMacroScore) — **US symbols only** (India → UNAVAILABLE, excluded, weights renormalize). 100 - danger_score from the newest macro_regime row that satisfies the full consumer contract above: real verdict (regime != 'unknown') **and** week_of within MAX_MACRO_AGE_DAYS = 10 **and** raw_indicators length >= 3. Nothing qualifies → UNAVAILABLE, never a calm default. (Corrected 2026-07-17: this line previously said "looks back up to 3 weeks", contradicting the 10-day bound documented in the consumer-contract table in this same chapter — the unbounded reach-back was the 2026-07-13 prod bug, not the intended behavior.)

Insider (resolveInsider → Massive / EDGAR / AV) — $10 + \text{buyRatio} \times 80$ where $\text{buyRatio} = \text{buyValue} / (\text{buyValue} + \text{sellValue})$ over 90 days, counting only open-market P/S codes (awards A, exercises M, gifts G, tax-withholding F are other and excluded — they are not conviction trades). Requires ≥3 transactions; <3, no data, ADRs, or fetch-fail → available:false (excluded).

Reviewed ADR contract (2026-07-31): lib/instruments/adrs.ts is the explicit identity registry; suffix guessing is forbidden. ADRs run in the US/USD research and paper pools and are segmented as adr for evidence/learning. Because foreign private issuers generally have no Section 16 Form 4 stream, insider is not fetched and remaining applicable dimensions are renormalized. Live execution recognizes ADR as a non-fund equity but adds no permission: broker review, account allowlist, market controls, kill switches, portfolio gates, and broker acknowledgement all remain mandatory.

Insider is expected to be sparse, and that is correct (EXPECTED_SPARSE_DIMS holds us:insider). Most Form 4 activity is compensation-related, not open-market buying, so an empty insider result is usually a real finding rather than a fault. That is exactly why a *failure* must never render as one — see below.

India insider (SEBI PIT): evaluated 2026-07-17 → DO NOT WIRE. India *does* have an insider analog — fetchNseInsider (lib/nse-data.ts) reads SEBI PIT (Prohibition of Insider Trading) disclosures from NSE /api/corporates-pit. It is built but deliberately **not** connected to insider_score. It failed the data-qualification bar on live evidence measured against the 34 real

India symbols in the book:

Test	Result	Bar
Coverage: any PIT row in 90d (the US window)	2/34 = 6%	—
Coverage: ≥3 open-market txns in 90d (the US bar, MIN_INSIDER_TRANSACTIONS=3)	0/34 = 0%	fails
Widening to 365d	1/10 large caps scorable	fails
Semantics vs US Form 4 (90d open-market P/S buy/sell value ratio)	only ~30% of PIT rows are open-market (Market Purchase 9.7% + Market Sale 19.5%); the rest are ESOP allotments 29.2%, Off Market 22.1%, Gift 5.3%, pledges, amalgamations	fails
Freshness / PIT lag (intimDt – acqfromDt)	p50 2d, p90 34d, max 71d; 83% disclosed after the transaction	usable in principle

A dimension available for **0%** of the universe is not a usable dimension. Worse, PIT's `tdpTransactionType` marks an **ESOP allotment as "Buy"** — so a naive wiring would read routine equity compensation as insider conviction. A same-named field carrying different meaning across markets is worse than an absent one, and `insider_score` is a genome dimension that reaches paper buys. India's insider dimension therefore stays **honestly unavailable**: `applicableDimensions()` omits it for India, the availability mask excludes it, and the weights renormalize onto the remaining dimensions. Pinned by `tests/india-insider-not-wired.test.ts`. Revisit only if SEBI PIT open-market density rises materially — the 0%-at-90d measurement is the gate.

**Two live defects found in the PIT read path while qualifying it (display-only, NOT on the money
path, not fixed here — no caller feeds scoring): (1) <code>fetchNseInsider()</code> with no symbol**
returns <code>{"data": []}</code> unconditionally — NSE's market-wide PIT feed requires <code>from_date/to_date</code> ,
which the function never sends. <code>SmartMoneyPage.tsx</code> calls <code>/api/india/insider</code> with no symbol, so
the India insider tab has never shown a row (same class as the 2026-07-16 EDGAR Form 4 404).
(2) The per-symbol call sends no date window either, so NSE returns the latest ~20 disclosures
regardless of age — RELIANCE's default response spans back to 24-Sep-2021 — while the
route reports available: <code>trades.length > 0</code> , which would present 5-year-old disclosures as
current. Both are cosmetic today precisely because nothing consumes them.

Form 4 URL contract (bug fixed 2026-07-16 — US insider data had never worked): the Form 4 XML must be resolved from `filings.recent.primaryDocument` in the submissions JSON, with the `xsLf345X0N/` prefix **stripped** (`buildForm4XmlUrl, lib/data/edgar-insider.ts`). Two traps: `<accession>.xml` is not a real EDGAR artifact and 404s for every filing; and `primaryDocument` itself points at the XSL **rendering**, which SEC serves as `text/html` — it returns **200** but contains no `<rptOwnerName>`/`<nonDerivativeTransaction>` tags, so it parses to zero transactions (a silent empty, worse than an error). The filename also varies by filer agent (`form4.xml`, `tm2618092-2_4seq1.xml`, `wk-form4_1784149645.xml`), so it must never be hardcoded. The archive path needs the **unpadded** CIK; the zero-padded form 301-redirects.

Availability is recoverable without a schema change. `agent_signals` stores only the score, so an unavailable 50 and a genuinely balanced 50 are identical there. The `available` flag is

already persisted in `decision_observations.availability_mask` (linked by `signal_id`), which is what `/api/markets/smart-money` joins to filter `highInsider`. Note `prod` holds rows that are `available:true` **and** exactly 50 — so "treat any 50 as unavailable" is **not** a valid shortcut; it would misreport real balanced data as missing.

Known gaps (deliberately not yet added — see the hype-catch discussion / IC-gate path): relative-strength vs index, MACD/ATR, 52w-high proximity, EMA200; debt/leverage, FCF yield, EV/EBITDA, revenue **acceleration**, sector-relative margins; per-symbol macro beta; insider role/cluster weighting.

Target v2: asset/setup-specific PIT feature snapshots, comparable-universe rank, structural evidence confidence, contradiction/event gates, and deterministic action. The complete contract is `features/scoring-methodology/FEATURE_ARCHITECTURE.md`; v2 remains non-actionable until its lifecycle and validation gates pass.

LLM role: explanation, risks, catalysts, and a bounded evidence-citing veto only. It never generates score, probability, expected return, direction, weight, size, or lifecycle state.

Screener target: 3 candidates/day (not 5). With \$10k NAV and 10% sizing, max 10 positions. Daily churn of 5+ creates overtrading.

Outputs:

- `agent_signals` row per symbol (score + thesis + recommendation)
- `signal_score_history` row (append-only score history)
- `decision_observations` row (even for skipped/expired candidates)
- `rag_traces` row (if RAG ran)

DeepSeekAgent — the comparison analyst

File: `app/api/agents/deepseek/route.ts`

Schedule: Weekdays 9:00 AM ET (parallel with `ResearchAgent`)

LLM: DeepSeek `deepseek-v4-flash` with thinking disabled

Inputs: Same watchlist and screener pipeline as `ResearchAgent`.

Key behavior: Advisory comparison only. Current code asks DeepSeek for an LLM-generated `analyst_score`; it must be tagged `score_source='llm_advisory'`, remain `status='advisory'`, and be structurally excluded from `PaperTrader` and `TraderAgent`. Future comparison should reuse the deterministic score and compare explanation/veto quality.

Outputs: `agent_signals` rows tagged `agent_label = 'deepseek'`

PaperTrader — the pretend-money trader

File: app/api/agents/paper-trade/route.ts

Schedule: US 10:05 AM ET, India 4:35 PM IST (standalone crons, independent of research)

LLM: None

Inputs:

- agent_signals WHERE status = 'pending' AND created_at is today (market timezone) AND market = ?
- the market's trading_mandates.score_threshold (canonical entry threshold; the legacy global threshold cannot loosen it)
- deterministic score_source and a strategy version currently in paper_active lifecycle
- paper_portfolio for pool cash
- paper_positions for existing open positions

Key behavior:

Signal freshness gate: Only fills signals created today in the market's own timezone (New York for US, Kolkata for India). Older signals are marked expired.

New-entry queue invariant (2026-08-10): Held alpha names are excluded before the per-market top-N entry ranking. Research still writes their fresh deterministic signals and PositionMonitor reads those latest rows without relying on entry-queue status. This prevents held names from consuming every new-entry slot and leaving cash idle while valid, unheld candidates sit below the rank cut.

Claim-and-fill protocol (prevents double-fills):

1. Claims a signal by stamping claim_run_id on the agent_signals row
2. Opens paper position only if it still owns the claim

Position sizing:

- position_size_pct from champion genome (clamped to strategy_config.position_size_pct)
- The approved percentage is applied to the market-local paper NAV, then capped by that pool's available cash and per-order limit. It is never applied to the shrinking cash balance; doing so geometrically undersizes later entries.
- Slippage model: 0.05% above mid
- Records expected_price and realized_slip_pct on every fill

Fill-bound risk plan: PaperTrader ignores research-time absolute levels. It binds stop and target to the fresh actual fill using the current market mandate, or the same-market/horizon MAE/MFE percentiles only after 60 eligible-long labels exist. Learned values are bounded to a 10% maximum stop and 40% maximum target; invalid or thin data falls back to the mandate. Planned-versus-bound percentages are appended to the Research Journal pipeline trail.

Risk gates (added 2026-07-09):

- **Latched controls:** both pause and trading-enabled controls are checked per market; a recovered kill-switch metric cannot silently re-enable entries
- **Name cap:** `trading_mandates.max_open_positions` per market (default 10), enforced again from the canonical DB value inside the row-locked fill RPC. It gates new names only and never liquidates an over-cap book.
- **Re-entry cooldown:** 5-calendar-day block after a position in a symbol closes
- **Pyramid gate:** New BUY only if fill price > existing `avg_cost` (no averaging down)
- **Long-only for new positions:** SELL signals only apply to symbols already held

Capital-rotation shadow (added 2026-07-13; containment restored 2026-08-10): When a candidate cannot be taken as-is — because the book is at its `max_open_names` cap **or** because it lacks cash — PaperTrader calls the deterministic rotation evaluator and writes one `rotation_events` row with the would-be source holding, edge, notional, and gate reasons. This is P0 measurement only. Paper execution is disabled in both market rows after production proved that the P1 executor did not enforce the shadow's economic-readiness result or `rotation_allow_score_only_paper=false`. Live proposals remain disabled.

Until 2026-07-22 the evaluator was reachable **only** from the `insufficient_cash` branch. In practice the name cap binds first — the book exhausts its 10 slots long before it runs out of cash — and the cap check continued before the rotation call, so the evaluator was unreachable and `rotation_events` stayed empty for nine days with shadow enabled. The cap check now sets a flag instead of skipping; the candidate flows through the remaining gates (sector cap, re-entry cooldown, pricing, sizing) and is evaluated for rotation at the funding step. Rotation is slot-for-slot, so this cannot grow the book; a candidate with no viable rotation is still skipped with the same `max_open_names` reason.

Outputs:

- `paper_positions` row (new open position)
- `paper_trades` row (buy leg)
- `paper_order_events` row (submitted + filled events)
- Updates `paper_portfolio.cash` and `paper_portfolio.nav`
- `rotation_events` row when either `max_open_names` or `insufficient_cash` triggers a shadow rotation evaluation

CryptoResearchShadow — the crypto-native evidence collector (2026-09-17)

File: `app/api/agents/crypto-research-shadow/route.ts`

Schedule: `kairos-crypto-native-shadow`, daily 00:15 UTC via Supabase `pg_cron`

LLM: None

This dedicated 24/7 lane prevents broker-listed crypto pairs from competing with US equity holdings for the equity ResearchAgent's bounded wall-clock budget. It reads Robinhood's point-in-time USD-pair inventory, explicit tradability, onboarding and two-sided quotes, then records completed UTC daily-bar trend/structure/volatility evidence for a bounded deterministic subset (the lowest observed spreads). Every unresearched or ineligible pair remains in the run with an explicit refusal rather than vanishing.

It does not write `agent_signals`, create a paper position, infer broker eligibility from a quote, or invent an executable spread from a candle close. A pair can become an **eligible research observation** only with explicit broker tradability, a current quote, 90 completed daily bars and a passing deterministic

score. That state is measure-only—not permission to paper trade, preview, or submit an order. A refusal is successful evidence, not a failed trade.

Outputs: one `crypto_universe_runs` header, one `crypto_universe_members` row for every observed broker USD pair, and idempotent `crypto_geometry_shadows` for the bounded researched subset. These are owner-readable evidence only.

CryptoPaperTrader — legacy paper-entry route (contained, 2026-09-17)

File: `app/api/agents/crypto-paper-trade/route.ts`

Schedule: `kairos-crypto-paper-trade`, daily 00:45 UTC via Supabase `pg_cron`

LLM: None

A deliberately small, separate sibling to PaperTrader — NOT a market branch inside it. PaperTrader's portfolio constructor, correlation shadow, capital rotation, and Kelly/genome sizing are calibrated on equity payoff/volatility distributions and keyed to `market: "us" | "india"` through library functions backed by market-CHECK-constrained tables (`trading_mandates`, `strategy_validation_automation`). Forking that router for a third book was a materially larger, riskier change than Stage 3 paper trading needs.

The route is active for paper-only execution. It uses the existing deterministic crypto research-signal ledger and completed public-candle price adapter; no live broker order is reachable. A missing or stale bar, paused book, active kill switch, insufficient cash, duplicate position, or RPC refusal remains a recorded refusal—not a fictional fill.

Key behavior:

- Flat sizing only — no Kelly, no genome, no portfolio constructor. `min(33%, strategy_config.position_size_pct)` of the crypto pool's own NAV per name.
- One position per coin (3 coins total) — no sector caps, no correlation shadow, no capital rotation.
- Price = latest **exact preceding UTC completed** Coinbase candle, Kraken fallback, Alpha Vantage compatibility fallback (`lib/data/crypto-quotes.ts`). A late provider bar declines the entry rather than filling at stale data.
- Static mandate (`CRYPTO_MANDATE` in the route): stop 15%, target 25%, and a 10-day *research-label horizon* stored with the fill. It is not an exit clock and cannot close a position. Not read from `trading_mandates` (CHECK-constrained to `us/india`).
- Fills via the existing `execute_paper_fill` RPC with `p_market='crypto'` — the RPC is fully generic on market (verified by reading its SQL body), so no RPC change was needed.

Outputs: an auditable fill/skipped response and isolated `crypto_paper_positions/paper_trades`.

This is paper-only; it is not crypto live-trading authorization.

CryptoPositionMonitor — the crypto exit agent (Stage 3, 2026-09-16)

File: app/api/agents/crypto-position-monitor/route.ts

Schedule: kairos-crypto-position-monitor, daily 00:30 UTC via Supabase pg_cron; it evaluates only the immediately preceding, fully closed UTC candle

LLM: None

Mechanical exits only: daily-OHLC-aware stop / target. There is deliberately no time-stop: elapsed time alone is not evidence that a crypto thesis failed. No trailing stop, no partial profit, no benchmark comparison (no crypto analog to VOO/^NSEI) — a deliberately smaller sibling to PositionMonitor's 1200+ lines of equity/India-specific logic, same rationale as CryptoPaperTrader above.

Inputs: All open paper_positions WHERE market='crypto'; the latest exact preceding UTC AV daily OHLC candle per coin. A late/missing bar fails closed rather than using an older bar.

Outputs: Closes via the existing execute_paper_exit RPC (also fully generic on market); writes one paper_performance row per crypto session date (idempotent) as the canonical EOD snapshot for this book, mirroring PositionMonitor's role for us/india.

Crypto paper pool isolation (Stage 3 schema note)

paper_portfolio/paper_positions/paper_trades/paper_performance carry a THIRD market value, 'crypto' — none of the 4 tables have a CHECK constraint on market (verified), so this required no constraint change. Every existing consumer of these tables filters market IN ('us', 'india') explicitly (verified by repo-wide grep — no reader selects across all markets unfiltered), so a 'crypto' row is invisible to every existing equity/India NAV, P&L, learning, or benchmark computation without touching that code. This is scoped to the paper-execution ledger only — decision_journal's market CHECK also had to widen (the execute_paper_exit RPC writes a decision_journal row on every close using the position's own market) — and lib/market-controls.ts's Mkt type/norm() helper, which previously coerced any non-"india" value to "us" (a crypto kill-switch trip would otherwise have disabled US equity trading). Scoring/research keeps crypto tagged market='us' + instrument_family='crypto' (lib/scoring/instrument-taxonomy.ts) — unchanged; the 30+ tables whose market CHECK the original feature doc left at ('us', 'india') for the scoring/session layer were never touched. Migration: supabase/migrations/20260916020000_crypto_paper_pool.sql.

PositionMonitor — the risk watcher

File: app/api/agents/position-monitor/route.ts

Schedule: US 4:15 PM ET, India 6:35 AM ET

LLM: None (exits are rule-based)

Inputs: All open `paper_positions` for the market; current prices.

What it does on each run:

1. Fetch current prices for all open `paper_positions` in the market
2. Update `highest_price` if today's price is a new high
3. Run exit checks (in priority order):

- **No time stop (2026-09-10, Decision 74 supersedes 65).** Age NEVER closes a position. The holding horizon survives only as the review-observation window and the learning label horizon. It closed 140 of 203 paper lots (69%) while the score exit closed zero, and in India was cutting winners short (mean minimum score while held 73.0). A trail is protective, not a guarantee of eventual exit: a position that never makes a new high can remain open, so the time-review shadow measures a separate evidence-driven review rule before any future change.

- **Trailing stop:** `stop_loss = max(original_stop, highest_price × 0.93)` → close if breached

- **Price target:** at target price → **partial profit-taking** (sell half, move stop to

breakeven on remainder; US paper uses six-decimal fractional quantity, India remains

whole-share; full close only when no valid partial leg remains)

- **Score exit (immediate):** a fresh same-market `deterministic_v1` score below the exit threshold with direction still long closes the position at once. The score must be no older than `trading_mandates.max_signal_age_sessions` (default 2). A stale/unavailable score can never close a position; price, stop, target, and hedge exits remain active. Legacy score flags are revalidated and cleared when stale or no longer below the exit threshold.

- **Direction-flip exit (debounced):** a held-long position whose fresh signal flips to short below exit is NOT sold on the first session. It **arms** (staged in `paper_positions.exit_reason` as `direction_flip_armed:<session>`) and only **confirms** the exit once a strictly newer research session still flips; a one-session wobble disarms and the position is held. A flip on a position held fewer than `MIN_FLIP_HOLD_DAYS` (2) market days is ignored. Pure logic in `lib/trading/direction-flip.ts`. This debounce was added 2026-07-24 after 13/22 closed paper trades exited on same-week flips (min 1.3 days held).

4. **NAV drawdown circuit breaker:** if weekly NAV return < -5%, set

`strategy_config.app_paused = true` and fire a critical System Health alert

5. **Benchmark sync:** `upsert paper_performance.bench_nav` with today's VOO (US) / ^NSEI (India) price

On close:

- Call `execute_paper_exit`, which atomically realizes FIFO lots (including a closed slice for a partial lot), updates/deletes the position, credits the correct market pool, and writes the decision journal; any mismatch rolls the whole exit back
- Call `indexClosedTrade()` for RAG (if Voyage embeddings are configured)

TraderAgent — the live order proposer

File: `app/api/agents/trader/route.ts`

Schedule: Weekdays 9:45 AM ET (after research settles)

LLM: None

Inputs: eligible deterministic `agent_signals`. A numeric score alone is not live eligibility.

Current behavior: creates proposals. Manual submission is owner-gated through the hardened Execution Gateway in `app/api/broker/orders/route.ts`.

- **manual (default):** Creates `trade_proposals` rows with `status = 'pending_review'`. Owner reviews and approves/rejects in the dashboard. Send invokes the deterministic Execution Gateway and broker preview/place sequence; no LLM supplies order parameters.

- **auto (future L4):** the old direct-submit branch is rejected. An authenticated worker may call only the shared execution kernel, under deployment flag, expiring owner lease, atomic budget, and a `live_approved` scoring version. Auto BUY is blocked until live protective exits, partial-fill sync, and reconciliation are operational.

Current auto status: disabled. India auto is a separate architecture. Eventual L4 must support verified risk-reducing SELL before autonomous BUY.

Architecture doc: `features/live-auto-trading/FEATURE_ARCHITECTURE.md`

Outputs: `trade_proposals` rows (expire after 30 min in manual mode)

LearnerAgent — the strategy improver

File: `app/api/agents/learner/route.ts` (entry); `app/api/agents/learner-brain/route.ts`

Schedule: Fridays 5:00 PM ET

LLM: Claude Opus 4.8 (upgraded 2026-07-03)

Phase gate: Mutation blocked until 10+ closed trades per market exist.

Inputs (via tool-use loop — 9 tools):

1. `get_closed_trades` — recent `paper_trades` with outcomes
2. `get_signal_weights` — current champion weights
3. `get_strategy_versions` — all challengers + their backtest results
4. `get_decision_observations` — scored decisions (including skipped)
5. `query_trade_decisions` — real historical enriched Robinhood trades by regime/action
6. `propose_challenger` — write a new `strategy_versions` row with new weights + genome
7. `run_validation` — trigger Validation Engine on the proposed challenger
8. `get_mentor_insights` — recent coaching notes
9. `semantic_search_decisions` — pgvector RAG over trade memories (if Voyage embeddings are configured)

What it proposes:

A Challenger `strategy_versions` row containing:

- 5 dimension weights (must sum to 1.0)
- Genome: {`entry_threshold`, `exit_stop_pct`, `exit_target_pct`, `horizon_days`, `position_size_pct`, `sizing_mode`}
- Possibly: a Feature Registry entry (a new formula idea — never runs as code)

Auto-guard: Blocks mutation if last 3 runs have win_rate < 35%.

Governance boundary: Learner/LLM may propose hypotheses, feature specs, and bounded challengers. Deterministic fitting/optimizers produce numeric candidate parameters. Only Vaibhav may promote lifecycle state. Learner cannot activate weights, versions, thresholds, money limits, accounts, orders, or code.

Closed-loop closure (2026-07-05): When user promotes a Challenger to Champion, the promoted weights_snapshot is read by ResearchAgent on its next run.

Per-trade notes: 1-sentence outcome summary per closed trade written to learning_log.

Outputs: strategy_versions (Challenger row), learning_log entries

ThemeScout — the watchlist manager

File: app/api/agents/theme-scout/route.ts

Schedule: Independent weekly pg_cron job, Sunday 8:00 PM ET during daylight time

LLM: Per-flow agent_config; default Groq llama-3.3-70b-versatile

Inputs: Broad GDELT market headlines first. Alpha Vantage NEWS_SENTIMENT is a cached fallback; TOP_GAINERS_LOSERS is used only when neither news source returns usable headlines. Every LLM-suggested candidate must then pass a deterministic Yahoo US quote lookup. That lookup proves the ticker currently resolves to a positive market price; it deliberately does **not** fetch fundamentals, because discovery should not spend scarce fundamentals calls.

Key behavior: Identifies emerging themes (e.g. ai_infrastructure, clean_energy). Adds at most six verified US symbols to owner-scoped watchlist rows tagged by theme, with a seven-day expiry. It is discovery-only: additions enter a later normal ResearchAgent run and must pass the same deterministic scoring and eligibility gates as every other symbol. It is not awaited by ResearchAgent, so a slow scout cannot consume a research run's time or provider budget.

Outputs: New watchlist rows with source = 'llm_theme', theme, reason, auto_added = true, and expires_at; invalid tickers are quarantined.

MentorAgent — the coach

File: app/api/agents/mentor/route.ts

Schedule: After position-monitor + learner runs

LLM: Claude Sonnet 4.6 (c laude - smart)

Inputs: Closed paper_trades + learner_insights + macro context.

Key behavior: Writes plain-English coaching insights to mentor_insights. Three types: pattern (what worked), lesson (what to change), warning (risk concentrations). Advisory only — never touches money, weights, or positions.

****Mentor UI surfaces** (all per-flow model from Settings → AI Models; each response carries a meta: {agent, model, agentKind} for the AI-attribution chip, 2026-07-12):**

- **AI Coach** (/api/agents/mentor-coach, mentor→deepseek-v4-pro) — a TRUE tool-using agent loop (runAgentLoop, ≤10 steps, tools: query_behavior/learning_progress/market_context/read_principles).
- **Ask the Agent** (/api/mentor/ask, mentor-ask→deepseek-v4-flash) — UPGRADED from a fixed recent-rows snapshot to a tool-using retrieval agent (runAgentLoop, ≤8 steps): tools lookup_symbol / recent_activity / list_open_positions / worst_and_best_trades → answers on the EXACT data the question needs (targeted, not a generic LLM guess). SSE-streams the final answer word-by-word after the loop; emits meta as the first + last stream event.
- **Judgment Coach** (/api/mentor/evaluate, mentor-evaluate→deepseek-v4-pro) and ****Market Thesis**** (/api/mentor/thesis, mentor-thesis→deepseek-v4-pro) — single grounded callLLM (no tool loop); evaluate injects the symbol's real research_packet + signal.

Outputs: mentor_insights rows

Health-Triage — the SRE

File: app/api/agents/health-triage/route.ts

Schedule: Every 6h + on-demand from dashboard

LLM: None — operational truth is deterministic

Read-only — can never change config, money limits, weights, orders, or code.

Inputs: Current open agent_alerts plus the latest run per agent/market in the last 48h.

Key behavior: Builds machine-readable structured_issues deterministically. An older failed run disappears from current triage once a newer successful run for the same agent/market exists. The snapshot carries its alert count and timestamp so Home can mark it stale against the live feed.

Dashboard display: SystemHealthCard on dashboard home. Green when clean. Severity-ranked. Deep-link fix hints. Tier-1 safe actions (retry, resolve info/warn) are one-click.

Outputs: Append-only health_triage snapshots and an agent_runs bookkeeping row. It never

creates inferred alerts or modifies trading/configuration state.

AutonomousShadow — the execution dry-run

Files: lib/trading/execution-kernel.ts, lib/trading/autonomous-shadow.ts,
app/api/agents/autonomous-shadow/run/route.ts (owner POST),
app/api/agents/autonomous-shadow/cron/route.ts (CRON_SECRET POST)

Schedule: Weekdays 07:30 UTC (30 min after research cron)

LLM: None — fully deterministic

****No broker calls in PA1.** Purpose: prove the execution kernel fires, gates fire correctly,
and shadow proposals accumulate evidence before live mode is ever enabled.**

Inputs:

- strategy_config live_auto_* policy snapshot
- agent_signals (last 24h, score_source='deterministic_v1', direction=long, score $\geq$ threshold)
- Current broker_orders filled count (open positions)
- Today's trade_proposals with execution_mode='autonomous_shadow' count

Key behavior: For each qualifying signal, creates a trade_proposal row
(execution_mode='autonomous_shadow', auto_run_id, auto_decided_at), then runs it through
evaluateAutonomousExecution() — 9 ordered gates (see lib/trading/execution-kernel.ts).
Updates proposal status to queued_auto (kernel approved) or manual_review_required
(gate failed with named reason). Writes one decision_journal entry per run. Never touches
broker APIs, never calls reserve_live_order_budget, never submits any order.

Gates (in order):

1. AUTONOMOUS_LIVE_ENABLED deployment flag
2. live_auto_enabled DB toggle
3. Lease not expired (live_auto_enabled_until)
4. Direction = long only
5. Score $\geq$ score_threshold
6. evidence_confidence $\geq$ live_auto_min_evidence_confidence (floor 0.6)
7. Open positions < live_auto_max_open_positions
8. Orders today < live_auto_max_orders_per_day
9. Proposed notional $\leq$ live_auto_max_per_order_usd (skipped in PA1 — notional = 0)

In current deployment: gate 1 fires unless AUTONOMOUS_LIVE_ENABLED=true is set in Vercel env.
When false, all proposals land on manual_review_required. Shadow accumulates evidence.

Outputs: trade_proposals rows (shadow), decision_journal run summary

AutonomousLive — the live submitter (PA3)

Files: lib/trading/execution-kernel.ts, lib/trading/autonomous-live.ts,
app/api/agents/autonomous-live/cron/route.ts (CRON_SECRET POST)

Schedule: Weekdays 14:00 UTC (10:00 AM ET, after research at 13:00 UTC)

LLM: None — fully deterministic

Runs ONLY when:

- AUTONOMOUS_LIVE_ENABLED=true in Vercel env AND
- strategy_config.live_auto_enabled=true AND
- live_auto_enabled_until not expired AND
- live_auto_mode_us='autonomous' or live_auto_mode_india='autonomous'

Inputs:

- strategy_config policy + per-market mode columns
- agent_signals (last 24h, score_source='deterministic_v1', direction=long, markets in autonomous mode)
- live_account_snapshots for NAV (account <agentic-account-id>, max age 4h)
- paper_trades for Kelly calibration (last 100 closed)

Key behavior: Same 9-gate kernel as shadow, plus:

1. Checks kill switches (app_paused, security_locked, trading_enabled)
2. Checks live_auto_mode_[market] = 'autonomous' per signal's market
3. Calls computeAutonomousSizing() for approved signals
4. Calls reserve_live_order_budget_v2 RPC (p_execution_actor='autonomous_worker') — atomic
5. Submits to broker:
 - US: rhPlaceMarketOrder() via Robinhood REST API (direct, no MCP — unavailable in serverless)
 - India: placeEquityOrder() via Kite Connect REST
6. Updates broker_orders: status=submitted + broker_order_ref
7. Appends broker_order_events row (actor_kind='autonomous_live')
8. Updates trade_proposals: status=queued_auto or manual_review_required

Per-market mode (migration 141):

- off — market skipped entirely
- manual — no live orders from this agent; owner clicks Approve in dashboard
- autonomous — live orders submitted per above

Safety: approved_by_user=false in broker_orders. Any gate failure = manual_review_required, no order. Budget exceeded (RPC throws) = skip, log. Broker error = unknown_needs_reconcile, budget stays reserved.

Outputs: trade_proposals (autonomous_live), broker_orders, broker_order_events, decision_journal

BriefingAgent — the daily email

File: `app/api/briefing/generate/route.ts`

Schedule: Weekdays 8:00 AM (morning) + 4:30 PM (evening) ET

LLM: Settings-controlled through `getConfiguredModel("briefing")`; default `deepseek-v4-flash`, for editor/outlook prose only.

Inputs: Latest signals, paper positions, NAV, macro regime, open System Health alerts, and the latest complete deterministic Holding Risk snapshots for every live account in the edition's market.

Key behavior: Generates morning and evening briefing emails. Morning: pre-market outlook. Evening: trade recap. Sends via Resend (or configured `EMAIL_PROVIDER`). Includes "Open Issues" band when System Health alerts are present. Its Live Holdings Risk band replays immutable per-account results (actionable/review postures first), exposes freshness/confidence/missing inputs, and never recomputes risk or combines accounts/currencies. LLM prose cannot change a HoldingRisk score or posture.

Outputs: briefings row, newsletters row (on successful Resend send)

Validation Engine

File: `lib/validators/backtest.ts`, `app/api/agents/backtest/route.ts`

Deterministic, no LLM. Replays Challenger vs Champion on the same PIT opportunity set. For v2 it must use purged/embargoed walk-forward folds, train-fold-only preprocessing/calibration, out-of-fold predictions, costs/turnover, and multiple-testing accounting. The existing five-weight replay remains a baseline, not sufficient proof for a new scoring architecture.

Eligibility gates:

- Sharpe ≥ 0.5
- Win rate $\geq 40\%$

Computes: Sharpe, Sortino, max drawdown, win rate, expectancy, alpha vs benchmark. If gates pass, sets `eligibility_passed = true` on the `experiment_runs` row. Promotion is blocked (HTTP 412) unless `eligibility_passed = true`.

Performance Truth Layer

File: lib/evaluation/run-evaluation.ts, /api/agents/evaluation/*

Mandate-aware, deterministic (no LLM), honesty-first evaluation panel on /dashboard/learning.

Evaluation metrics: Sharpe, Sortino, max drawdown, win rate, expectancy, profit factor, alpha vs benchmark, execution slip (mean realized vs 0.05% modeled).

Honesty rules:

- Fewer than 20 trades → shows "too small" instead of a number
- Tainted trades are counted (P&L must not hide them) but labeled as tainted
- health_label summarizes: insufficient_sample → negative_or_zero_edge → promising_but_unvalidated → validation_required

Downside Hedge Controller

File: app/api/agents/downside-hedge/route.ts, lib/trading/downside-hedge.ts

Deterministic US paper-book overlay. It combines MacroSentinel with locally computed SPY/QQQ confirmation and an audited off -> armed -> active -> exit_pending -> cooldown state machine. Generic agents block inverse ETFs; only the paper-only hedge RPC may buy unleveraged SH/PSQ when both settings flags are enabled. No LLM, live-order, broker, rotation, or cross-market path. It ships fully OFF.

P1 gate: Weekly Vercel cron counts closed evaluable trades per market. Fires a System Health info alert when ≥ 20 accumulate.

2026-07-17 Research and Exit Contract Audit

- The watchlist input is filtered by research_enabled=true, unexpired rows, and authoritative market; suffix inference is only a legacy fallback. US and India watchlist rows cannot enter each other's scoring pool.
- Holdings from paper and live snapshots are staleness-ordered and persist agent_signals.is_holding=true. PositionMonitor may only treat a holding-path signal as a reassessment input.
- Held positions receive the same deterministic evidence fetch, five-dimension scoring, availability mask, and market-local weights as candidates. They skip optional LLM narrative and RAG retrieval because those outputs cannot alter an action; this preserves wall-clock capacity for exit coverage and candidates.
- One worker from the existing concurrency pool is reserved for candidates while the remaining workers prioritize staleness-ordered holdings. Concurrency and provider quotas do not increase, but a large book cannot starve all discovery.

- Candidate overflow is carried forward with a stable original defer time and is pruned after six unsuccessful deferrals or seven days. Theme Scout rows expire after seven days, require an owner id, deduplicate per run, and cannot overwrite a durable manual row.
- Sentiment inputs are shrunk toward 50 using source sample size before blending. A five-message 100% bullish split therefore scores 67, not 100.
- Analyst targets are retained as observational evidence only. They do not alter fundamental or composite scores until a validation/promotion decision says so.
- India macro remains unavailable and excluded. It never queries or inherits the US-only macro_regime; effective weights renormalize over genuine India inputs.
- A stored held short below the entry threshold is not an independent exit gate. PositionMonitor still requires the lower entry threshold - hysteresis bound, a fresh score, and holding provenance. Stops and targets remain independent.
- Risk Analytics remains deliberately research-free in its risk formula. The UI may show the latest holding score and age beside the independent risk verdict; research cannot veto a concentration, drawdown, liquidity, or volatility risk.

Earnings-Aware Risk P0 (2026-07-29)

PaperTrader annotates an otherwise-eligible entry after its fill-bound stop/target plan exists and before either capital rotation or `execute_paper_fill`. TraderAgent attaches the same normalized block to `trade_proposals.risk_check_reasons`; its existing Alpha Vantage earnings blackout remains unchanged and authoritative. Policy version 1 is shadow-only: no agent reads the counterfactual verdict to change a score, quantity, stop, target, fill, rotation, or exit. PositionMonitor is deliberately not wired.

US event dates are cross-checked against the PIT calendar, Finnhub, Webull, and Robinhood. India records market-local proximity and always reports options as unavailable. Robinhood research tools are a hardcoded read allowlist disjoint from order/review/cancel tools.

Date validation is one shared parser (2026-08-02). Every earnings collector — the `earnings_calendar` cache, the Finnhub/Yahoo-India base observation, Webull `get_stock_earnings_calendar`, Robinhood `get_earnings_calendar`, and `fetchIndiaEarningsDate` — normalizes its provider value through `normalizeRealIsoDate` in `lib/date-only.ts`. It accepts a YYYY-MM-DD string (with or without a time suffix) or a provider epoch in seconds or milliseconds, and returns `null` for everything else, including calendar days that do not exist (2026-02-30) which JavaScript would otherwise roll forward. Rejection is

fail-closed: the collector yields no observation rather than a coerced date, and a symbol with no usable observation resolves to status: "unknown", never to "no earnings soon". Before this, only the Robinhood collector validated; the cache path and Webull accepted `String( . . . )` as-is and Yahoo India converted an epoch with no range check.

`tradingSessionsBetween` still validates **before** its `from ===` to shortcut and returns `number | null`; callers read `null` as a conflict. That ordering is the defence-in-depth behind the parser and is load-bearing — reordering it silently reopens the fail-closed path. Pinned by `tests/earnings-risk.test.ts` ("treats a self-comparison of a malformed date as unknown, not zero") and `tests/date-only.test.ts`.

Shadow Registry and Upgrade Path (2026-07-29)

`lib/shadows/registry.ts` is the descriptive inventory for every active, paper-active, armed, idle, or disabled evidence program. `/api/upgrade-path` adapts existing truth ledgers into a read-only status model; it does not create a parallel evidence table. `/dashboard/upgrade-path` refreshes that model every minute and reports market scope, schedule state, progress, collection rate, provider-call accounting, benefit evidence, blockers, and the separate activation gate. Since 2026-08-24 it also reports, independently, (a) the first mainline implementation date/commit/scope and why it was merged, and (b) the current production runtime state with evidence proof and "why not next stage." The API includes Vercel environment/build SHA when available so local output is not presented as production truth.

The `DashboardShell` market switch is the only market selector. The API requires `market=us|india` and applies it to every market-bearing ledger query before aggregation. It never combines US and India readiness. A US-only program stays visible in the India view as `not_applicable`.

The registry has no write or activation capability. Estimated days are shown only for a declared target with a non-zero observed rate. Uninstrumented calls are labelled `unmetered` rather than `zero`. Capital rotation is labelled from its live config: it is `paper_active` only while paper execution is truly enabled; after the 2026-08-11 containment it is production shadow measurement with `paper/live` flags false. Any future shadow producer is incomplete until its mainline provenance, ledger, schedule, safety boundary, activation gate, and runtime-state adapter are registered.

Crypto-native evidence sources (2026-09-17)

The crypto research lane uses public, completed UTC daily candles from Coinbase Exchange first and Kraken second. Alpha Vantage DIGITAL_CURRENCY_DAILY is a last-resort compatibility fallback only; a healthy crypto run therefore does not consume the shared Alpha Vantage equity budget. All candle providers are historical research inputs and cannot authorize a paper or live order.

Robinhood MCP is the separate execution-truth source. The native shadow collector reads account onboarding, the broker's USD-pair inventory and explicit tradability fields, plus current two-sided crypto quotes, recording broker receipt time, bid, ask, and spread with the run. It does not call preview, place, cancel, or order-status tools. A quote is not treated as proof of a tradeable pair; only the explicit pair-inventory field can establish broker_tradeable. Webull is not a crypto source: its registered MCP role is read-only US equity fundamentals/analyst research and it has no verified crypto pair, quote, or account-eligibility contract.

ready_for_review means the production measurement floor is reviewable; it is never deployment or promotion authority. Router readiness requires ten distinct fresh passing market sessions. Event-driven programs use an independent run liveness source so a legitimate zero-event window is not mislabeled as dead.

The 2026-08-24 audit is recorded in
features/shadow-registry/PRODUCTION_AUDIT_2026-08-24.md.

Deep-Dive Debate (2026-07-31 correction)

/api/agents/deep-dive is an owner-only, on-demand LLM research surface. Both POST and cached GET use the owner gate before service-role reads. It remains advisory: only deep_analyses is written and no signal, score, paper position, proposal, or broker path consumes its verdict.

The route accepts canonical US class tickers and NSE/BSE .NS/.B0 symbols. US uses Massive/cache quotes plus Alpha Vantage fundamentals; India uses Yahoo INR quotes and Yahoo India fundamentals and never borrows the US macro regime. The configured LLM provider is resolved before its key is checked. The diagram source is public/agent-diagrams/deep-dive.json.

Source: docs\arch\08-risk-and-safety.md

Kairos — Risk & Safety

2026-09-14: **The UI is not a security boundary; RLS is.** `lib/supabase/client.ts` ships `NEXT_PUBLIC_SUPABASE_ANON_KEY` to the browser, so any holder of a valid session can query PostgREST directly, whatever the app renders. Until `20260914140000_viewer_phase0_close_blanket_rls.sql`, 34 tables were reachable by any authenticated session and 7 were writable — including the paper book and the learner's weights. That was never a live breach (only the owner can obtain a session), but it meant the single-email gate in `middleware.ts / requireOwner()` was carrying 100% of the isolation, with RLS contributing nothing. Phase 0 closed it. **Standing rule: before any additional identity is admitted to this system, verify no table grants blanket authenticated access.** Any feature that adds a login must re-run that check; a UI-level permission is not an isolation control.

2026-09-01: **Exit-geometry counterfactual run on all four cohorts. The intuitive fix is refuted; the defect is the STOP, not the target.** Evidence frozen in `docs/audits/2026-09-01-exit-geometry-diagnosis.md`. Nothing changed — the route writes nothing.

The configured +19.2% target fires **zero times across 965 India observations** and 28 times in 900 US h10 observations. Timeouts run 76-97%. The time stop is not competing with the target; it IS the exit policy, which reproduces the live ledger (132 of 179 closed lots, 73.7%, exit on the clock).

Tightening the target lowers mean return in every cohort. US h10: baseline 0.616% -> 0.398% at a 4% target, while win rate RISES 52.0% -> 61.4%. More frequent small wins, worse expectancy. The widest target ranks first and the tightest last in all four cohorts, so "make the target reachable" is measured and wrong.

What beats baseline is a volatility-scaled stop. stop 2.8ATR / target 7.3ATR ranks first in 3 of 4 cohorts, and on US h10 its whole advantage is stop-outs 131 vs 185 (-29%) with timeouts UNCHANGED (706 vs 683). It barely exits on target either.

Margins are thin outside US h10 (+0.295pp): India h10 +0.073pp, India h5 +0.051pp, and the candidate **loses** US h5 by 0.041pp. With 14 configs x 4 cohorts this is a ranking from a search, not a significance test — no CIs, no t-stats, overlapping windows.

Proposed follow-up, awaiting approval: `features/atr-exit-stop/FEATURE_ARCHITECTURE.md` — a single-arm shadow testing one predeclared hypothesis (ATR stop reduces premature stop-outs), Sidak-adjusted for the 14-arm search it came from, with the time stop and target held fixed. Expected verdict for months: `insufficient_evidence` (26 dates at h10 = 2.6 effective observations against a floor of 12).

2026-08-28 (same day): **CORRECTION to the India sizing diagnosis — survivorship materially weakens it.** I published the quartile gradient from CLOSED lots only while listing survivorship as an untested confounder, then tested it. Including the 14 open India positions marked to current price, **8 of them fall in the LARGEST quartile** and are outperforming the closed lots. Q4 mean return moves from -0.55% to **+0.07%**, and the win-rate gradient shallows from 60->38% to **57->43%**.

No longer supported: that the largest positions lose money. **Still supported:** size is uncorrelated with conviction (corr with `analyst_score` -0.128, with cash-at-entry +0.344, 57x notional spread) — measured at ENTRY, so unaffected by survivorship — and win rate still declines with size. The honest claim is that allocation WASTES the edge rather than reversing it.

The profit-factor pair (percent 1.438 vs currency 0.906) inherits the same bias and should be recomputed on matured outcomes before being leaned on. Method note: the confounder should have been tested before the write-up, not listed inside it as future work.

2026-08-28: **India sizing damage diagnosed — cause is cash-path dependence, not volatility.** Full evidence: `docs/audits/2026-08-28-sizing-damage-diagnosis.md`.

A3 reported India percent profit factor 1.438 against currency 0.906. By entry-notional quartile the win rate falls monotonically 60 -> 56 -> 54 -> **38%** as size rises; the smallest quartile earns +5,871 and the largest loses -10,113.

Position size tracks cash available at entry (corr +0.344), not conviction (corr with analyst_score -0.128). Notional spans 57x on a nominally uniform book: allocation is decided by timing, not by the model.

CORRECTED 2026-08-28 (same day): the h10 rank IC +0.105 originally cited here as India's selection edge is the **all-scored** context cohort. Under the measurement-integrity repair (f95c3951) the **eligible-long** cohort — the names that could actually be entered — measures **-0.0083** over 17 dates in India and **-0.0768** over 21 in the US, both below the evidence floor. There is no demonstrated selection edge in the entry cohort, so "allocation throws the edge away" is unsupported; only the sizing mechanism above survives. The US/India split asserted below is also wrong in direction: neither market's eligible cohort ranks.

The volatility budget is a separate, real defect and NOT the cause. Rule 4 has fired zero times in 1,513 constructor events because `maxPortfolioVolPct = 2.0` is unreachable at `DEFAULT_DAILY_VOL = 0.02` — reproducing `estPortfolioVol`, the worst case across every configuration (5 names, 100% gross, all same sector) is **1.649%**. Breaching 2.0% needs per-symbol vol $\geq$ 3.5% daily. The threshold is set above what the model can produce. Fixing it would not fix the sizing damage.

No sizing behaviour changed. This is a diagnosis; changing `position_size_pct`, `maxPortfolioVolPct` or the allocation formula is a separate money-path decision. US is a different problem entirely — both its profit factors are below 1 and its rank IC is -0.012 with a negative quintile spread, so sizing is not its binding constraint.

2026-08-28: **RESOLVED — why capital rotation was disabled on 2026-08-11.** It was never unexplained; the reason was recorded contemporaneously and I had been reading a later, vaguer label.

The disable is migration `20260811033335_disable_unqualified_paper_rotation.sql`, shipped in commit `ba20f4ff`. The filename timestamp is exactly the `rotation_config.updated_at` I had been treating as a mystery. Its own comment:

"Capital rotation may keep measuring, but paper execution must remain off until benchmark-alpha, friction, turnover, correlation and tax readiness are enforced by the execution path."

`features/capital-rotation/FEATURE_ARCHITECTURE.md`, updated in the same commit, gives the mechanism: migration `20260723120000` had enabled both paper rows, **but the executor only rechecked score spread, persistence, cooldown and count caps — it never enforced the P1 readiness result and never read `rotation_allow_score_only_paper`**. Four paper rotations executed through that hole. The same commit added the missing gate to `executeCapitalRotationPaper`.

It was a MISSING GATE, not bad P&L. The four negative rotations were the symptom; the doc says explicitly this "is containment, not a claim that the four-trade result proves rotation lacks edge" — which matches the swap-level analysis on 2026-08-25 that found those rotations neutral-to-positive.

The "unsafe early P1 behavior" phrase in `lib/shadows/registry.ts` was written 2026-08-24 (26216cb3), two weeks after the fact. It is a retrospective summary, not the contemporaneous record, and searching for it is what made this look unexplained. The precise cause was in the migration filename the whole time.

Directly relevant to 2026-08-25. The flag I flipped that day, `rotation_allow_score_only_paper`, IS the gate this commit added to stop score-only execution. My argument was that `score_to_return_mapping_unvalidated` "can only be validated by running it" — the same reasoning that produced the original incident. The flip was reverted for a different reason (a false claim that rotation had never executed); this is the reason it should have been refused on the merits.

The five p1_blockers are the reopening criteria, not incidental noise: `turnover_budget_not_configured`, `exact_tax_lots_unavailable`, `score_to_return_mapping_unvalidated`, `post_swap_gate_unavailable`, `candidate_correlation_unavailable`. They map one-to-one onto the migration's "benchmark-alpha, friction, turnover, correlation and tax readiness". Rotation is re-enableable when those are enforced in the execution path — not before.

2026-08-27: **India alpha provenance loosened to admit `upstox(yahoo_disagreed)` (owner decision).**

CONFIRMED_BENCHMARK_SOURCES.india was ["upstox+yahoo"] — genuine two-vendor agreement only. That rule was written when neither source was trusted over the other. Upstox is now declared authoritative (broker API carrying official exchange data; on a disagreement ITS value is the one stored), so the rule was suppressing alpha on a CORRECT exchange close because the SECONDARY source was wrong. On the 08-19..27 backfill Yahoo disagreed on six of seven sessions by up to 0.67% — and Yahoo was in error every time.

Now ["upstox+yahoo", "upstox(yahoo_disagreed)"].

upstox(unconfirmed) **deliberately stays OUT**, and the distinction is not pedantry: there the second source never resolved the session at all, so nothing corroborates that Upstox returned the right BAR for the right DAY. "The two disagreed and we kept the authoritative one" is a different and stronger claim than "only one source answered". Both directions are mutation-pinned — reverting the loosening fails, and over-loosening to admit upstox(unconfirmed) also fails.

Backfilled bench_return_pct across 38 India rows that now qualify. First measurable since-inception alpha for both books, on settled exchange data:

market	sessions	NAV	benchmark	alpha
india	39	-0.79%	-1.39% (NIFTY 50)	+0.60pp
us	38	+1.85%	+2.60% (VOO)	-0.74pp

India has been BEATING its benchmark since inception and this was invisible until now — the alpha was withheld by provenance rules, not absent from the book. Both figures start 2026-07-06, the first session with a benchmark on both sides.

2026-08-27: **The 8 NULL benchmark rows: 7 filled, 1 correctly refused.**

I had left these alone on the grounds that filling India from a single source would manufacture provenance. Checking rather than assuming resolved it: **Upstox and Yahoo agree to the paisa on all four India sessions**, so upstox+yahoo is earned two-source provenance, not a label applied to one opinion.

market	date	value	source
india	07-06	24430.35	upstox+yahoo (exact agreement)
india	07-07	24398.70	upstox+yahoo (exact agreement)
india	07-08	23882.05	upstox+yahoo (exact agreement)
india	08-17	24287.65	upstox+yahoo (exact agreement)
us	07-06	690.62	yahoo(settled)
us	07-07	687.08	yahoo(settled)
us	07-09	690.69	yahoo(settled)

US 2026-06-28 stays NULL. It is a SUNDAY — the paper book's inception row. No session, no bar, and filling it would invent a price for a closed market. Yahoo returns nothing for that date, which is the correct answer rather than a gap to paper over.

India now has ZERO null benchmark rows; US has one, and it is the Sunday. Consequence worth noting: benchmarkReturnPct baselines off the FIRST non-null bench_nav, which is now 2026-07-06 for both markets — India's benchmark series is exactly aligned with its NAV series, and the US benchmark starts one session after its Sunday inception row.

Open design question, deliberately not changed here. The seven India upstox(yahoo_disagreed) rows carry the AUTHORITATIVE exchange close, but CONFIRMED_BENCHMARK_SOURCES.india admits only upstox+yahoo, so they cannot publish alpha. That rule was written when neither source was trusted over the other; now that Upstox is declared authoritative, withholding alpha because YAHOO was wrong may be stricter than intended. Changing it is an owner decision, not a cleanup.

2026-08-27: **PaperTrader write path audited after the benchmark work. One latent W4 regression closed.**

Checked because confirmBenchmarkSessions was wired into PositionMonitor only, and PaperTrader is the OTHER paper_performance writer. Findings:

- **The two-writer split is sound.** PaperTrader inserts snapshot_type= 'intraday' and ONLY when no row exists for (date, market); PositionMonitor upserts on (date, market) and promotes the row to eod, but only once expectedNewestSession(market) === today. That is why production holds zero surviving intraday rows -- not because the path is dead.
- **LATENT REGRESSION, now fixed.** PaperTrader's insert ladder strips bench_session_date, bench_source and snapshot_type *together* on any undefined-column error. But paper_performance.snapshot_type is NOT NULL DEFAULT 'eod'. So if the missing column was one of the bench ones while snapshot_type existed, the stripped insert would stamp a PREMARKET row as the canonical close -- exactly the W4 defect the one-EOD-writer rule exists to prevent, reintroduced through the fallback. Dormant while all three columns exist. PaperTrader now corrects the label after any legacy-ladder insert.
- **Confirmation pass no longer filters** snapshot_type= 'eod'. A past session's benchmark is settled regardless of how the NAV row was labelled, and the filter would have permanently stranded any row left intraday because PositionMonitor did not run that day.

Not changed: 8 old rows (US 06-28, 07-06/07/09; India 07-06/07/08, 08-17) carry NULL bench_nav. They predate the provenance work and India's policy deliberately requires a stored Yahoo value before claiming upstox+yahoo — filling them from a single source would manufacture provenance rather than recover it.

2026-08-27: **US benchmark — the CONFIRMED rows were the wrong ones.** Same deferred-confirmation fix as India, plus a provenance correction.

Measured against settled VOO closes:

date	stored	source	settled	verdict
08-19	NULL	-	706.91	never resolved
08-20	NULL	-	701.01	never resolved
08-21	703.71	yahoo_quote(provisional)	703.71	exact
08-24	701.83	yahoo_quote(provisional)	701.83	exact
08-25	702.74	yahoo	704.02	0.18% WRONG
08-26	704.20	yahoo_quote(provisional)	704.20	exact

The rows humbly labelled provisional were exact; the rows labelled plain yahoo — which CONFIRMED_BENCHMARK_SOURCES listed as confirmed, authorising an alpha claim — were the inaccurate ones. A Yahoo DAILY BAR read at 16:15 ET is an IN-PROGRESS bar for the session that has just closed: it carries the correct date and an unsettled value, so the exact-session rule cannot catch it. The W5 guard validates the DATE; nothing validated that the value had finished settling.

Two changes. Bare yahoo is removed from CONFIRMED_BENCHMARK_SOURCES.us and replaced by yahoo(settled), which only the confirmation pass writes. confirmBenchmarkSessions is now market-aware and runs for BOTH markets in PositionMonitor: it upgrades provisional rows and FILLS rows that never resolved. A fill reports storedClose: null and deltaPct: null rather than a fabricated 0%, which would read as two sources agreeing when only one ever existed.

Backfilled 08-19..08-26. The two missing sessions now carry alpha of +0.33pp and +1.11pp, previously absent entirely. The US three-session underperformance figure quoted earlier (-1.90pp over 08-21..08-26) is unchanged, because those two endpoints happened to be exact.

All three US guards mutation-verified: treating plain yahoo as settled, fabricating a zero delta for an unresolved row, and letting the US fill rule leak into India each fail a test.

2026-08-27: **India benchmark cross-check was never failing — it was structurally impossible.**

paper_performance recorded yahoo(unconfirmed) for India every session from 2026-08-19. Root cause: Upstox's /v3/historical-candle **never returns the current session**. Verified against every cached payload from 08-19 to 08-27 — the newest bar is always the PREVIOUS trading day. So selectBenchmarkObservation was asked for today's session, correctly found no Upstox bar, and fell back to Yahoo alone. The fetch succeeded every day; it simply never contained the bar being requested. The two upstox rows (08-14, 08-18) exist only because they were backfilled a day later.

Not a token, key-mapping, budget or outage problem — all four were checked and healthy (live API returns HTTP 200; ^NSEI maps correctly; 83 candles cached daily).

Fix is deferred confirmation, the pattern the code comment at the benchmark write already anticipated ("preserve the observed level for the next-day settle pass") but which was never built. confirmIndiaBenchmarkSessions runs in the India PositionMonitor and upgrades earlier provisional rows once Upstox publishes their settled bars. Upstox is authoritative, so the settled exchange close REPLACES the provisional Yahoo value and bench_return_pct/alpha_pct are cleared rather than left stale. Rows already carrying exchange provenance are never revisited.

The disagreement is large and was invisible. Backfilling 08-19..08-27 changed six of seven sessions to upstox(yahoo_disagreed), with gaps up to **0.67%** in a single day. Corrected daily alpha:

date	old bench chg	new bench chg	alpha (pp)
08-25	-0.46%	+0.48%	-0.49
08-26	+0.77%	-0.52%	+0.21
08-27	—	-0.48%	+0.26

India was reported as underperforming on 08-26; against the settled NIFTY close it OUTPERFORMED. Any India alpha figure quoted from a yahoo(unconfirmed) row is unreliable.

An earlier draft of the eligibility rule also required the source string to contain "unconfirmed". Mutation testing showed removing that guard changed nothing (every non-Yahoo source is already excluded), and inspection showed it was wrong: it would have permanently stranded a yahoo_quote(provisional) row. Rule is now "the stored value came from Yahoo", which is the only case where Upstox is a genuine second source.

2026-08-25: **Capital rotation paper execution was enabled and then REVERTED the same hour. Flags are false. Do not re-enable without reading this.**

The enable was argued for on the claim that rotation "has never moved capital". **That claim was false.** Rotation executed two swaps and four sell lots:

date	market	sold	score	bought	score	edge
2026-07-24	us	PLTR	56	CB	74	18
2026-07-27	india	ONGC.NS	53	TCS.NS	100	47

Sell legs realized -0.10% (PLTR) and -2.50 / -4.13 / -3.05% (ONGC.NS x3).

How the claim went wrong: rotation_events.trade_proposal_id and .paper_trade_ids are NULL on every row *including the executed ones*, and that was read as proof of non-execution while status='paper_executed' sat in the same rows. paper_trades.exit_reason='capital_rotation' was never cross-checked. **Never infer "did not happen" from an unpopulated foreign key — confirm against the table that records the effect.**

Do not over-correct either. "All four rotations lost money" is also wrong framing: rotation sells the weakest holding by design, so losing sell legs are expected. Evaluated as swaps, both were fine — CB returned +0.12% over its hold against PLTR's -0.10%, and TCS.NS returned +6.12% against ONGC.NS's ~-3.2%. The July record is not itself a case against rotation.

Nor did rotation cost the PLTR run. PLTR was held 8 calendar days (~6 market days) of a 10-market-day horizon; the unconditional time stop would have closed it around 2026-07-30 anyway. Rotation removed it ~4 sessions early. The mechanism that forfeits moves like that is the time stop — which is what the horizon-extension shadow (jobs 123/124) now measures.

The open question that blocks re-enabling: the paper flags were set false at 2026-08-11 03:33:35 and the registry gates execution "after unsafe early P1 behavior". That behaviour has NOT been identified, and the swap P&L above does not obviously explain it. Establish the cause first.

Current state: `rotation_paper_execute_enabled=false`, `rotation_allow_score_only_paper=false` on both paper rows; both live rows false; `rotation_live_proposals_enabled=false` everywhere. The two `book_type='live'` rows were never modified at any point. `CAPITAL_ROTATION_PAPER_ENABLED` exists in Vercel Production (created ~2026-07-23) with a value that is redacted on pull and therefore unverified — inert while the DB flags are false. Journal: id 344 (the enable, containing the false premise) superseded by the correcting entry.

2026-08-25: **CORRECTION — capital rotation does NOT execute, in either book. Two docs claimed it did.**

Verified against `rotation_config` on <production-project-ref>: all four rows (us/india x paper/live) carry `rotation_shadow_enabled=true`, `rotation_paper_execute_enabled=false`, `rotation_live_proposals_enabled=false`. Across 98 `rotation_events`, both markets, all time, `trade_proposal_id` and `paper_trade_ids` are NULL on every row; every event carries `no_execution: true`. Rotation has never moved capital.

Claimed	Actual
<code>lib/shadows/registry.ts</code> : "Paper execution is enabled; live proposals are disabled"	shadow only; paper execution flag is false
<code>paper-trade/route.ts</code> : "Capital-rotation P1 PAPER execution is live (owner-approved 2026-07-23)"	the call is a no-op behind the false flag

Both corrected in place. This is the same hazard class as the 2026-08-24 protective-stop entry above, inverted: there a doc said a gate was shut while it was open; here two docs said a path executes while it cannot.

Surfaced by tracing why a name that passed the research gate on 30 consecutive sessions never entered the book. Since 2026-08-18 the portfolio constructor has admitted **zero** candidates in either market, and US rotation stopped emitting events on 2026-08-11 — because the `portfolio_constructor / rejected` branch continues past the rotation evaluation, so a candidate denied for lack of room never reaches the mechanism that makes room. India takes the name-cap path, which was already repaired, which is why only US went silent. Full trace: `docs/audits/2026-08-25-rotation-unreachable-trace.md`. **The code fix is proposed, not applied — it awaits owner approval.**

2026-08-24: **Shadow proposals are evidence, never work items.**

`runAutonomousShadow` writes a real `trade_proposals` row per signal so the kernel/sizing decision has somewhere to live. Two defects made those rows indistinguishable from real proposals. Both fixed before the run is ever scheduled (it has never executed: `zero execution_mode='autonomous_shadow'` rows, and `/api/agents/autonomous-shadow/cron` is scheduled nowhere).

1. The insert used `status='pending_review'` and only narrowed to `kernel.shadow_status` ~80 lines later. Every surface that renders an approve control keys off `pending_review`, so that window — permanent if the run threw in between — put a working approve button on synthetic

evidence. Now inserts as `manual_review_required`.

2. Four consumers read `trade_proposals` without filtering `execution_mode`.

The money-path one is agents/trader's 24h "already proposed" dedup set:

unfiltered, shadow rows would enter it and ****starve the real queue of the exact signals that scored best****. Also agents/trader GET (approve queue), markets/smart-money (visible queue), and DashboardShell (desktop notification).

Excluded by `execution_mode`, not by status — a shadow row legitimately carries the same terminal statuses a real proposal does.

`EXCLUDE_SHADOW_FILTER` (`lib/trading/proposal-status.ts`) keeps an explicit

IS NULL arm: the column is nullable (default 'manual') and a bare

<> 'autonomous_shadow' evaluates to NULL — excluding the row — for NULL modes, silently hiding legitimate proposals. Mutation-verified.

Scheduling the cron remains a separate, un-taken step.

2026-08-24: **CORRECTION** — both protective-stop gates are OPEN in production. The "Part E (not yet done)" note below is inverted.

Verified against <production-project-ref> on 2026-08-24:

Gate	This chapter says	Production
<code>strategy_config.protective_orders_enabled</code>	"false-by-default ... STAYS FALSE"	true
<code>PROTECTIVE_PLACEMENT_WORKER_AVAILABLE</code> (<code>lib/protective/coverage.ts:134</code>)	"flip ... to true" as a pending Part E step	already true

`protective_orders` holds **0 rows**, so no broker stop has been placed. But the placement worker is no longer gated shut. This is the inverse of the usual stale-doc hazard: an agent reading this chapter would conclude placement is impossible when it is enabled, and could ship a change on that false assumption.

RESOLVED same day — owner confirmed the activation was unintentional and directed both gates closed.

`strategy_config.protective_orders_enabled` set to false; `PROTECTIVE_PLACEMENT_WORKER_AVAILABLE` set back to false (its own docstring already said "deliberately false" while the value was true). Pre-flip verification showed nothing to orphan: `protective_orders` 0 rows / 0 active, `protective_order_events` 0, `broker_orders` with a `kite_gtt_id` 0. Journalled to `decision_journal` as a risk-reducing `config_change`. Other flags re-verified unchanged and OFF: `live_auto_enabled`, `webull_trade_orders_enabled`, `allocation_enabled`. **Part E is once again closed and reopening it needs explicit owner approval of touch semantics, floor distance, and post-fill policy — not a code edit.**

Also corrected: this chapter and `WORK_LOG.md` both described migration `20260718000000_protective_orders_shadow.sql` as "written as a PROPOSAL and NOT applied to prod". The `protective_orders` table exists, so it HAS been applied. See the reconciliation table at the top of `WORK_LOG.md` for the full set of stale migration blockers cleared on 2026-08-24.

2026-08-20: **US benchmark resolves the just-closed session from a guarded QUOTE**. `bench_nav` was NULL for two consecutive US sessions while NAV itself was correct: at 16:15 ET no vendor has published VOO's settled bar (Yahoo's chart endpoint lacks it, Massive grouped publishes next-day, Massive /range still ends yesterday). The marks path was fixed to use Yahoo QUOTES; the benchmark path was left on the chart endpoint, so alpha stayed uncomputable.

This does not break the "a benchmark observation is a DAILY BAR, not a quote" rule — it respects why that rule exists.

The original defect was an UNSESSION-IDENTIFIED quote stamped with the cron run date. The quote was never the problem; the missing session identity was. There is also a consistency argument: NAV is now marked from the same 16:15 print, so a 16:15 benchmark is LIKE-FOR-LIKE, and pairing a 16:15 NAV against a settled close is the greater inconsistency.

Four guards keep it honest. (1) It fires ONLY on `benchmark_session_mismatch` — a working provider missing one session — never on `benchmark_bars_unavailable`, because with no series at all we cannot tell the data is sane. (2) It fires ONLY when the requested session IS the one that just closed (`expectedNewestSession("us") === expectedSessionDate`), so a quote can never backfill history. (3) A stale quote is refused rather than dated to today. (4) The source reads `yahoo_quote(provisional)` so the settle pass knows to revisit it. A real daily bar always wins; the quote is last resort.

Detector: `tests/benchmark-session-alignment.test.ts` — quote used on a genuine session miss, backfill of an older session refused WITHOUT even calling the quote, stale quote refused, a real bar preferred over the quote, total outage NOT rescued, and a quote-source throw not breaking the path. **Mutation-verified** (dropping the session guard fails). Fixtures use the production shape — a healthy series ending at the previous session — because an empty-series fixture tested a different branch entirely and passed for the wrong reason.

2026-08-20: **Settle pass — US marks finally get a second opinion, one day late.**

The US monitor marks at 16:15 ET from Yahoo, the only vendor carrying the just-closed session at that hour, and Yahoo cannot corroborate Yahoo — so those marks ship uncorroborated. The grouped feed (12,549 tickers, ONE entitled call) publishes the following morning and IS independent. `kairos-settle-check-us (0 13 * * 1-5, 09:00 ET)` compares yesterday's live marks against it.

It writes no money state. No NAV, no `paper_positions.current_price`, no trade. A drift beyond `SETTLE_TOLERANCE_PCT` (0.25%) TAINTS that session's `paper_performance` row with the measured per-symbol drift and net NAV impact, and raises a critical — but nav is left exactly as recorded. The exits for that session already filled at the marked price; restating NAV afterwards would re-decide a closed session, which the frozen-history rule forbids. Labelling it untrustworthy is honest; silently replacing it is not.

Tolerance is looser than the intraday cross-check (0.1%) on purpose. Yahoo's 16:15 print is near-final rather than fully settled. Measured 2026-08-19: exact on NVDA and XAR, 0.07% out on KGC. 0.25% catches a real mismarking — the AV-stale case was 1.00–9.36% — without firing on ordinary settlement revision.

`nothing_to_compare` **neither raises nor resolves.** An unpublished feed is not evidence the marks were right, and "nothing to compare" must never collapse into "everything agreed" — the same failure mode as the W6 liveness checks that could only report green. Marks already labelled `carry_forward` or `entry_cost` are skipped rather than double-reported.

Detector: `tests/settle-check.test.ts` — corroboration, the measured KGC revision tolerated, the AV-stale drift flagged with its NAV impact, stale marks not re-flagged, missing closes counted UNVERIFIABLE not agreed, an empty feed not reading as corroborated, and opposite drifts netting to zero while both stay flagged. Route-shaped assertions pin that it never touches `paper_positions/paper_trades/nav`. **Mutation-verified twice.**

2026-08-20: **Yahoo is now the PRIMARY source for US settled marks — the data was never missing at 16:15 ET, the route was asking the wrong vendor.**

The 2026-08-19 entry below concluded that no vendor publishes the current session's close at 16:15 ET, and proposed moving the US monitor later. **That conclusion was wrong.** Checking the settled 2026-08-19 closes against what each vendor returned during that 20:15 run: Yahoo gave NVDA 217.56 and XAR 284.10 — exact — and KGC 29.92 against a settled 29.90 (0.07%). Alpha Vantage gave 219.74 / 294.87 / 27.12: the PREVIOUS session throughout. Yahoo had it; the chain preferred AV.

Cause. `getSettledDailyQuotes` tried the grouped feed (which publishes NEXT-DAY, so empty at 16:15) and then fell straight to the ordinary chain — Massive snapshot (403), `price_cache` (stale), Alpha Vantage (previous session). Yahoo was relegated to the unresolved tail, which never ran because AV had already answered. Yahoo now sits between grouped and the ordinary chain. It

is keyless and unbudgeted, so preferring it costs no quota, and the schedule is unchanged: exits still run at 16:15, now on correct prices.

Consequence, stated plainly: US marks lose their second opinion. Yahoo cannot corroborate Yahoo — that circularity is exactly what let the ^NSEI provisional value pass as "verified". The cross-check filter now excludes any Yahoo-sourced primary, so US marks are labelled uncorroborated rather than falsely confirmed. Real US corroboration needs the next-day grouped feed via a settle pass; Massive /prev is per-symbol and paced at 12.5s (13 symbols $\approx$ 162s > the 120s route ceiling). NOT built.

Also rejected on evidence: moving the US monitor past 20:00 ET would cross 00:00 UTC, and the route derives today from the UTC date — NAV would be written under the following session.

2026-09-02: **The dispute gate refused a price for seven days because it never checked which SESSION each price came from.**

The 2026-08-19 fix above is sound and stands: a disputed symbol leaves priceMap before the exit loop, so no stop, target or time-stop can fill on it. The defect was in deciding what counts as a dispute. The comparison ran on two prices whose sessions were never checked, so a cross feed one session behind was refused as a vendor disagreement — permanently, because the lag recurred every run.

Production evidence. position-monitor-quote-disputed:india was raised 2026-08-26 and re-reported every run to 2026-09-01: INDUSTOWER.NS (yahoo_india 375 vs upstox 388.8, 3.549%). Yahoo's own closes were **375 on 2026-09-01** and **388.79998779296875 on 2026-08-31** — the "disagreeing vendor price" was the primary's OWN previous session, to four significant figures. The vendors never disagreed about a price; they disagreed about which day it was. Two open positions (INDUSTOWER.NS qty 27, KAMATHOTEL.NS) ran **seven days with no exit evaluation at all.**

Both sides already carried the session and both discarded it. fetchUpstoxCandles parses a date per candle and the cross-check dropped it; the Yahoo quote's retrievedAt is regularMarketTime — the exchange timestamp of the quote, not our fetch time. buildPositionMark's own input type had documented crossPrice as "for the SAME session" since it was written. The contract was stated and never enforced.

Fix (lib/paper/quote-crosscheck.ts). Session gate BEFORE any price comparison, using exchange-local dates (IST/ET, never UTC — an India quote at 01:00 IST is the previous UTC day, which would reintroduce the same off-by-one). Same session → compare exactly as before. Different or unknown session → session_mismatch: **uncorroborated, still priced, exits still evaluated**, reported as a warn naming which side lags. Unknown sessions count as a mismatch rather than being waved through, because failing open would restore the bug for exactly the rows whose provenance cannot be established. The mark builder reads the same verdict, so the mark ledger and the exit gate cannot disagree.

The same alert carried a second symbol with the OPPOSITE cause, and a fix that only aligned sessions would have been wrong to rescue it. KAMATHOTEL.NS (yahoo_india 233.89 vs upstox 225.09) — Upstox matched the 2026-09-01 close of 225.05 and the PRIMARY was the outlier. That is a genuine same-session disagreement and is still refused. Both cases are pinned in tests/quote-crosscheck.test.ts with their real production numbers; removing the session gate fails three of them.

Escalation. Refusing a doubtful price is right for one run and wrong forever — nothing distinguished "refused once" from "refused every run for a week", and the second is worse than either vendor being wrong. A dispute unresolved for DISPUTE_ESCALATION_RUNS (3) raises position-monitor-quote-dispute-persists:<market>, stating how long the position has been unguarded. Days proxy for runs (one run per market per day); a skipped run counts as elapsed, which errs toward escalating sooner.

Contributing factor, deliberately NOT bundled: isYahooQuoteStale accepts an India quote up to **4 days** old as fresh, which makes primary/cross session mismatch likely by construction. Changing it affects every India read path and needs its own decision.

2026-08-19: **CORRECTION — the mark cross-check did not protect exits, and four positions closed on stale prices before it was fixed.**

The entry below claims a disputed quote is "REFUSED for marking AND for stop/target evaluation". That was **false as shipped**. The guard lived only in `buildPositionMark`, which runs AFTER the exit loop; the exit loop reads `priceMap` directly. The 20:15 run corrected the marks and exited anyway.

What it cost. The US chain fell through to Alpha Vantage — which served the PREVIOUS session's close — while Yahoo carried the current one. Six of ten marks were flagged disputed (KGC 9.36%, XAR 3.79%, OXY 2.86%, TSM 2.38%, BAC 1.43%, NVDA 1.00%), and four positions still closed: LULU at ~119.55 (the 2026-08-14 price; the 08-18 close was 119.01) and MSFT at ~487.65 (a carried 08-17 value; 08-18 close 481.63), plus MA and SMCI. Eight lots, now tainted + `excluded_from_learning`; `realized_pnl` left intact to preserve the cash-ledger identity.

Fix. The gate now sits on `priceMap` itself, before any exit is evaluated: a disputed symbol is deleted from the map and reported UNPRICED, so no stop, target or time-stop can fill on it, and a critical names every refused symbol. A price too doubtful to record is too doubtful to sell on. Pinned positionally in `tests/paper-nav-writer-contract.test.ts` — the gate must precede both the exit loop and `buildPositionMark` — and mutation-verified.

Confirmed by the same run (these DID work): the per-position isolation fix and the MSFT residual-lot reconstruction — the run completed done, 14/14 succeeded, `exit_failures=0`, where the previous two runs aborted entirely. India wrote `bench_source=yahoo(unconfirmed)`, exactly the honest label the cross-check contract specifies when only one vendor resolves.

Still unresolved: no vendor publishes the current session's settled close at 16:15 ET. Massive `/prev` returned 2026-08-18 closes when queried at 20:16 UTC on 2026-08-19, and grouped publishes next-day. So the US monitor cannot mark the session it just watched close. US `bench_nav` for 2026-08-19 is NULL for the same reason — correctly refused rather than mislabelled. This needs a run-timing decision (move the US monitor later, or add a next-morning settle pass); it is NOT a code bug and is not built.

2026-08-19: **Position marks are now corroborated by an independent vendor, and a DISPUTED quote fails closed.**

A mark is not a display number — it prices the stop and target checks. Yahoo demonstrably serves PROVISIONAL values it later retracts (the 2026-08-18 ^NSEI case wrote a 0.375% error into `paper_performance`), and the exact-session rule validates the DATE, never the VALUE. So each live mark is compared against a DIFFERENT vendor: US → Yahoo per-symbol (primary chain is Massive-based), India → Upstox (primary is Yahoo). Both keyless and unbudgeted.

Contract. Agreement within `MARK_CROSSCHECK_TOLERANCE_PCT` (0.1% — wider than the index's 5bps because equity feeds differ on adjustment, far below the 0.375% error that motivated this) → `live_quote`, reason names the corroborating vendor. Disagreement → the quote is REFUSED for marking and for stop/target evaluation, the mark falls back to the carried price (or entry cost), and the reason preserves BOTH numbers. No second source → still `live_quote`, but the reason says `uncorroborated` rather than implying it was checked.

Fail-closed is affordable because agreement is the norm. Measured 2026-08-18: Massive `/prev` and Yahoo matched to the cent on every US holding — MSFT 481.63/481.63, NVDA 219.74/219.74, XAR 290.43/290.43.

Detector: `tests/mark-crosscheck.test.ts` — corroborated accepted; disputed refused with the carried mark kept and both values recorded; rounding tolerated; uncorroborated labelled honestly; a cross-source outage cannot demote a good mark; a disputed quote with no carried mark falls to entry cost and never to the disputed price. **Mutation-verified** (removing the guard fails 2).

UNRESOLVED — the grouped-daily fix probably does not fire at 16:15 ET. Measured 2026-08-19 ~00:55 UTC, roughly five hours after the US close: `/v2/aggs/grouped/. . ./2026-08-18` returned **0 rows**, while `/v2/aggs/ticker/{sym}/prev` and Yahoo both already carried the 2026-08-18 close. Grouped publishes next-day. The US PositionMonitor runs fifteen minutes after the close, so `getSettledDailyQuotes` will most likely get nothing from grouped and fall through — meaning US marks stay `carry_forward` and there is nothing to corroborate. `/prev` has the data but is per-symbol and Massive is paced at 12.5s (13 symbols ≈ 162s > the 120s route ceiling). Candidate fixes: move the US monitor later, or add a next-morning settle pass that re-marks the prior session. NOT built — it needs a decision about run timing.

2026-08-19: **India benchmark is now cross-checked against the exchange, because the exact-session rule validates the DATE and never the VALUE.**

The hole W5 could not see. Yahoo's ^NSEI series carries bars whose close is NULL — 2026-01-15, 05-01, 05-28, 06-26, 07-21, 07-22, 07-31, 08-18 in the 1y window — and briefly serves a PROVISIONAL number on those sessions before dropping it. On 2026-08-18 that put **24245.699** into paper_performance when the settled NIFTY 50 close was **24154.9**: 0.375% wrong, written by a code path behaving exactly as designed. W5's exact-session match proved the bar was dated 2026-08-18; nothing proved the number was the close. India had no second source (Massive is US-equities-only), so Yahoo agreeing with itself was the only available "verification".

Method note, recorded because it nearly hid the bug. The India series was first checked by re-fetching from Yahoo and comparing — 24/24 matched, and that proved nothing. Comparing a value against the source that produced it tests self-consistency, not correctness. Only an independent provider could settle it.

Second source: Upstox. A broker API carrying official exchange data, already integrated and unbudgeted. Indices are absent from upstox_instruments (the master is filtered to instrument_type=EQ/segment=NSE_EQ), so fetchUpstoxIndexCandles uses the static key — exact: NSE_INDEX|Nifty 50 resolves, NSE_INDEX|NIFTY 50 returns UDAPI100011.

Contract. Upstox is AUTHORITATIVE; Yahoo is the check. Agreement within BENCHMARK_CROSSCHECK_TOLERANCE_PCT (5bps — an index close is one published number, so a real gap is a fault, not rounding) → source=upstox+yahoo. Disagreement → the exchange value is used and the label says upstox(yahoo_disagreed); the fact is recorded, never hidden. One provider only → upstox(unconfirmed) / yahoo(unconfirmed) — a single-source benchmark beats none, but it must say so. A cross-check outage cannot break the path.

Data corrected: 2026-08-18 India → 24154.9 (source=upstox). The three rows Yahoo could not corroborate (07-21, 07-22, 07-31) are CONFIRMED correct by Upstox to the paisa and are now stamped — they were never wrong, Yahoo simply could not prove them.

Detector: tests/benchmark-session-alignment.test.ts — agreement, the real 2026-08-18 disagreement (exchange value wins, label states it), rounding tolerance, single-provider labelling both ways, both-missing refusal, and a cross-check throw not breaking the path. **Mutation-verified** (neutering it fails 4). One older test asserted source=="yahoo" for India; its premise is now obsolete, so it was updated to pin the invariant it actually existed for — India never spends the US fallback call.

Still one-sided: the US benchmark has Yahoo→Massive, and India now has Upstox+Yahoo. Neither market cross-checks its per-SYMBOL marks; only the benchmark is corroborated.

2026-08-18: **One position's exit failure was blanking the whole book.** The 20:15 US run died on execute_paper_exit denied (MSFT): position_lot_qty_mismatch. closePosition throws on a denial, the throw escaped the per-position loop, and the run aborted BEFORE the mark/NAV block — so no marks, no NAV, and the other 12 US positions never had their stops or targets checked. The 2026-08-14 run died identically on LNC (existing_open_position). Two runs, same shape, two weeks apart.

The RPC denial is CORRECT and stays. MSFT's only lot closed on 2026-08-03 (outcome=win) while its paper_positions row survived, so the parity check refuses to close 0.472499 that no open lot backs. Suppressing that guard would double-count a realized trade. The fix isolates the failure, it does not silence the guard: each position's evaluation is wrapped, a failure is recorded and alerted (position-monitor-exit-failed:<scope>, critical, naming the symbols whose stops went unchecked), and counted as a **failed** unit in the W6 envelope — so the run is error, not healthy, while still completing its other work.

Open data defect, NOT auto-repaired: one orphaned position — MSFT, qty 0.472499, **\$230.41 of phantom value carried in NAV since 2026-08-03.** paper_positions is current-state that "may be removed/closed only through the transactional exit path", so it is surfaced for owner reconciliation rather than deleted. Every other position reconciles exactly (1 of 27 drifts).

Detector: tests/paper-nav-writer-contract.test.ts — the loop body is wrapped; NAV/marks are reached AFTER a failure (positional); failures count as failed; the alert key appears in BOTH the report and resolve paths; the RPC denial still

throws. Mutation-verified twice — and the first version of the alert assertion was decorative (a bare `toContain` passed even with the `reportIssue` key renamed, because the `resolveIssue` occurrence satisfied it), so it was strengthened to count occurrences.

2026-08-18: **W5 US benchmark was never written — the provider ladder's recency guard is too weak for an EXACT session.**

W5 itself works. India recorded `bench_nav 24245.70 / bench_session_date 2026-08-18 / bench_source yahoo` on the 11:15 UTC run. The migration and both write paths are correct. Only US was null, on every row since the migration.

The defect. `fetchUsCandles` accepts the FIRST provider whose newest bar is inside a generic `MAX_BAR_AGE_DAYS = 4` guard. The US `PositionMonitor` runs 16:15 ET — fifteen minutes after the close — and Yahoo has not published the settled VOO daily bar that soon. Its newest bar was 2026-08-14: three days old, therefore "fresh", so the ladder returned it and **never tried Massive, which DID have 2026-08-17 (close 710.27)**. `selectBenchmarkObservation` then correctly refused to store a non-matching session, so the US book recorded no benchmark at all. India was unaffected because its cron runs 1h15m after the NSE close, by which time Yahoo has published. Proof in `av_cache: YAHOO_CANDLES:VOO` at `cache_date 2026-08-17` has `newest_bar 2026-08-14`, while `YAHOO_CANDLES:^NSEI` at the same `cache_date` has 2026-08-17.

A generic age guard cannot answer an exact-session question. "Newest bar is under 4 days old" and "there is a bar for session X" are different predicates, and only the second is what a benchmark observation needs. The fix stays inside `benchmark-observation.ts`: on `benchmark_session_mismatch` — and ONLY that reason — ask the next provider directly. A `benchmarkBars_stale` result means the provider is stranded and is not retried. `fetchUsCandles` is untouched, because its other caller (label maturation) legitimately wants a long series rather than one date.

Rejection reasons stay honest. When neither provider supplies the session, the ORIGINAL rejection is reported — it names the provider the ladder actually chose, not the fallback.

Detector: `tests/benchmark-session-alignment.test.ts` — the fallback resolves the session; no second call is spent when the primary already has it; a stranded provider is NOT retried; both-miss still refuses; India spends no fallback call. **Mutation-verified** (neutering the fallback fails). Dates are RELATIVE, per the note at the top of that file: the stale guard is wall-clock, so pinned literals would drift into the stale branch and start asserting the wrong thing.

2026-08-18: **A market-scoped PositionMonitor run was writing the OTHER market's book.**

Both crons are correctly scoped (`?market=us, ?market=india`), yet the India run at 11:15 UTC wrote 13 US marks and a US `paper_performance` row stamped `snapshot_type='eod'` at **07:15 ET — before the US session opened**. The W4 "ONE canonical EOD writer per market" invariant was broken from a direction W4 did not anticipate: not a second writer racing the same market, but the *other market's schedule* reaching across.

Scoping was defeated inside the route, not at the schedule. Two reads ignored `marketScope`: the `stillOpen` re-read of `paper_positions` (unfiltered select), and `poolByMarket`, built from **every** `paper_portfolio` row. The mark/NAV write loop iterates `poolByMarket`, so it processed both books regardless of scope. Fixed by scoping the re-read and skipping non-scoped markets in the loop before any write.

`snapshot_type` is **no longer an unconditional literal**. Even a legitimate unscoped or manual run must not stamp `eod` on a row built from carry-forward marks hours before the close — the same lie in a different costume. It is now `expectedNewestSession(market) === today ? "eod" : "intraday"`, reusing the post-close predicate added with the grouped-daily work.

Detector: `tests/paper-nav-writer-contract.test.ts` — the guard must sit inside the pool loop *before* the `paper_performance` upsert (positional assertion, not mere presence); the re-read must acquire its market filter before it is awaited; the bare `snapshot_type: "eod"` literal is banned. **Mutation-verified** — removing the loop guard fails the suite. The pre-existing "PositionMonitor is the EOD writer" pin was updated rather than deleted: its intent (PositionMonitor, never PaperTrader, owns the `eod` row) is preserved and now also asserts the post-close condition.

2026-08-18: **The US quote path had no working live source — the Massive key is not entitled to /v2/snapshot.**

What was broken. `fetchMassiveBatchQuotes`, described in code as "the primary batch path", was returning **zero US quotes on every call**: the deployed `MASSIVE_API_KEY` gets 403 `NOT_AUTHORIZED` on every `/v2/snapshot` endpoint and on `/v2/last/trade.if (!res.ok) continue`; swallowed it. The `/markets/etfs` second pass was also dead — that endpoint returns **404** and had never resolved a single ETF. With Massive silent, AV's 25/day budget exhausted since 2026-07-23, and `price_cache` holding only symbols the **research** path scores (never the holdings), the chain collapsed to a stale cached bar — which the pre-2026-08-17 adapter then relabelled fresh.

Cost, measured not estimated. On 2026-08-17 all 13 US holdings were marked at the **2026-08-14** close. Marked position value 7,725.11 vs true 7,667.32 against that session's real closes: NAV overstated **\$57.79**. The reported **+0.239%** was in truth **-0.339%** — **the sign flips, the entire gain was mismarking**. Per-name drift reached **-3.92%** (SMCI), **-3.89%** (INFY), **-3.04%** (MSFT); against a 7% stop that is over half the stop distance, so stop and target evaluation were affected, not merely NAV display.

The fix. The key IS entitled to `/v2/aggs/grouped/locale/us/market/stocks/{date}` — **12,549 tickers, settled OHLCV, one call** — verified against the live API. New `fetchMassiveGroupedDaily + getSettledDailyQuotes` (`lib/data/quotes.ts`); `PositionMonitor`'s US path now uses it. It includes ETFs (XAR, VOO), so the dead `/etfs` pass is deleted rather than repaired, and it carries OHLC so the `dayLow` intraday-stop check survives the move.

Deliberately NOT a blanket swap. Grouped daily is settled/EOD and marked DELAYED. That is correct for a post-close consumer (`PositionMonitor` runs 16:15 ET) and WRONG for intraday callers — `live-portfolio` and `lib/market-data.ts` keep the existing chain. Hence a separate function, not a replacement inside `getBatchQuotes`.

An entitlement failure is now loud. A non-OK snapshot response logs the status and says whether the key is unentitled; a grouped 200-with-zero-rows is reported as "session not published", never as "these symbols have no price".

Detector: `tests/massive-grouped-daily.test.ts` — a 403 must yield no quotes AND log; zero rows must not read as no-price; non-positive closes are refused rather than marking a position at zero; ETFs resolve. `tests/agent-source-pipeline-remediation.test.ts` re-pinned to the settled path.

History annotated, not rewritten. `paper_performance 2026-08-17 US` is `tainted=true` with the full quantified reason and the corrected reading. **nav is left as recorded** — the frozen-history rule in the Scoring Data-Truth Review Protocol forbids re-deciding the past. Earlier US rows (2026-08-10..14) were already tainted by the prior remediation.

Open, unresolved: an unscoped `PositionMonitor` run wrote a US row tagged `snapshot_type='eod'` at 11:15 UTC (07:15 ET, pre-open) on 2026-08-18, which breaks the W4 "one canonical EOD writer per market" invariant from the other market's schedule. Not fixed here.

2026-08-17: **Codex composite/data-truth audit remediation — a stale bar could drive live exit decisions.**

P0-1 (money path). At 16:15 ET Monday 2026-08-17 all 13 US positions were marked, **stop-checked and target-checked** against Friday's close. `priceMap` in `position-monitor` feeds `priceForStopCheck`, the `currentPrice >= priceTarget` partial/target branch, the time-stop fill price AND the W4 mark ledger — one stale number reached every exit decision, not merely NAV display. The monitor's own `!q.stale` guard was correct; the **adapter lied**. Two defects in `lib/data/quotes.ts`: (a) `isStale()` asked "under 4 CALENDAR days old" — Friday's bar is 3.01d on Monday afternoon, so it passed; (b) `parseTickers` stamped `stale:false` unconditionally even when the price fell through to Massive's `prevDay.c`, i.e. last session's close wearing this run's timestamp. Both fixed at the adapter, so every caller inherits the correction.

W9's rule was NOT sufficient and reusing it was wrong. `isFreshSessionDate` delegates to `lastCompletedMarketSession`, which always steps back at least one calendar day and therefore **still** names Friday at 16:15 ET Monday. That leniency is correct for its EOD-cache callers mid-session (today's bar may legitimately not exist yet) and wrong after the close. New `expectedNewestSession(market, now)` in `lib/data/completed-candles.ts` — which already owned the session-close constants — returns TODAY after a trading day's close, else the previous completed session. Callers compare with `>=` so a provisional running-session bar still passes; only the post-close case tightens.

Detector: tests/quote-session-freshness.test.ts pins the exact production case (Friday bar refused Monday 16:15 ET; accepted Sat/Sun when it genuinely IS the last completed session). **Mutation-verified** — restoring lastCompletedMarketSession fails 2 of 4.

P0-2. v_decision_quality recomputed confidence from a hardcoded per-market applicability list and reported India 0.7333 while the scorer had frozen evidence_confidence = 1.0. Both numbers described the same decision, and the paper-fill RPC, /api/kite/order, execute-order and Decision Review all read the view. Migration 20260817200000 makes the **observation the source of truth** (it is the contract the decision was actually made under and cannot drift when a market policy is later edited); the recomputed figure survives as diagnostic-only structural_coverage, and confidence_source records which rule produced each row. Verified behaviour-preserving BEFORE writing: across 4,628 joined rows **0 cross the 0.5 gate in either direction**; 837 change value, none change an outcome. 210 legacy rows with NULL stored confidence fall back to the derived value — without that fallback a usable number becomes NULL → unknown → live BUY fails closed, silently TIGHTENING a gate.

P1-3. The deterministic breakdown veto capped technical score at 20 but the composite renormalised around it, so a strong fundamental could still clear threshold on a confirmed high-volume breakdown. Production carried 7 rows vetoed=true AND entry_eligible=true (CPCAP.NS, SIMO, APP, EXEL, MUTHOOTFIN.NS). **None reached a fill** — unrelated downstream caps caught them, which is luck, not a gate. entryEligible now includes !breakdownVetoed. **New long entries only**; PositionMonitor exits never read the flag, so a breakdown can never block getting OUT.

P1-5 (measurement only). US macro took full 15% weight on 3-of-8 FRED indicators with evidence_confidence=1.0. coverage/coverage_pct/indicators_expected are now persisted so a partial read cannot present as complete. The weight and the MIN_MACRO_INDICATORS=3 floor are **unchanged** — discounting or raising the floor is a live scoring change and needs a frozen shadow counterfactual first.

P1-4 deliberately NOT fixed. Admitting weak technicals (BANKBARODA.NS composite 65 / technical 24) may be real, but the sample is small and hard-coding a technical cutoff now would be tuning on noise. Needs the same date-clustered frozen counterfactual discipline as the exit-policy work.

P2-6. query_learner_config returned strategy_config.score_threshold (52) while the money path uses per-market trading_mandates (60/7%/8%) — the learner could describe and optimise a policy the system does not run. It now returns activeMandates; the legacy field is relabelled legacy_score_threshold_NOT_USED.

2026-08-17: **Price target retargeted — +8% (was +20%).** The +20% swing-mandate target was structurally dead: 0/135 firings, max MFE ever 18.44% (India h10), target above p90 MFE for both markets across 1,885 matured labels. Fix is justified on reachability grounds (a target that never fires is not a target), NOT on return-improvement evidence (ATR counterfactual max t=1.35 — insufficient per predeclared nEffective≥12 floor). New value **8%** sits at approximately p75 MFE for both markets, between average (~6%) and p90 (~12%). Three touch points updated atomically: HORIZON_PRESETS.swing.target_pct (lib/trading-mandate.ts), resolveExecutionRiskReward fallback (lib/trading/trade-plan.ts), and both trading_mandates DB rows + strategy_config display row. W2-full partial exits (restored 2026-08-17) now have a reachable trigger. Evidence frozen read-only in docs/audits/2026-08-17-exit-policy-counterfactual.md.

2026-08-16: **W9 — the remaining price_cache consumers now derive freshness from the BAR'S MARKET DATE.** W1 (lib/data/quote-freshness.ts → assertFreshQuote) closed the fill and live-order boundaries. W9 closes the consumers that read price_cache rows directly and so never went through getQuote at all. One new module, lib/data/price-cache-freshness.ts, holds the rule for a cached BAR and **delegates the verdict to assertFreshQuote** — there is no second rejection taxonomy. A bar is fresh iff its date is at least lastCompletedMarketSession(market); cached_at is never consulted, because a row re-read today is not fresh data.

- lib/portfolio/inputs.ts (**PaperTrader position SIZING**). estimateDailyVolPct read 21 price_cache closes with no coverage or staleness check. For the 101/140 symbols frozen at 2026-07-22 that was not merely stale but **permanently fixed** — the same Jun–Jul dispersion priced into every future trade forever. It now requires coverage (≥15 usable closes) AND freshness, and falls back to DEFAULT_DAILY_VOL **explicitly and observably** (basis: "default" + a reason + the fossil asOf, logged) via the new estimateDailyVolPctDetailed. A zero-dispersion window is also refused — 0 vol reads as infinite position size.

estimateDailyVolPct's bare-number signature is unchanged; paper-trade/route.ts was not touched. The India branch fetches Yahoo live per call and was never affected.

- rescore-check publishes LEARNER FEEDBACK, so a wrong price here corrupts the evaluation layer scoring is calibrated against. It treated the newest cached row as "current" with no as-of check, and when no bar existed at or before the signal date it fell back to the **oldest** row in the window — measuring a window that is not the one being judged. Both now skip with a counted reason; the response carries skipped, stale_symbols, and a degraded flag, and evaluated now means "actually measurable" rather than "had any row at all".
- lib/data/benchmark-series.ts (beta / RS) had no recency check. SPY kept filling so it kept working, but a frozen SPY would have produced a beta that looks measured and is a fossil. It now returns {bars, asOf, stale, reason} via getBenchmarkSeriesStatus; getBenchmarkSeries resolves to [] when stale or under-covered, which downstream already reads as "beta unmeasurable" — so the gate lives at the source and no caller needed changing.
- supabase/functions/_shared/quotes.ts **DELETED**. It carried a divergent, weaker rule: staleness off cached_at (when a row was written) and stale:false for **all** off-hours cache. It had no importer; leaving a second rule available for reuse was the hazard.
- **Display/LLM surfaces labelled:** /api/markets/quote emits asOf/stale/staleNote; deep-dive carries asOf/stale into the LLM bundle with an explicit "do not treat as today's price" instruction; the briefing prompt appends a STALE MARKS warning naming each fossil-marked position, since it reports P&L "now".

Detectors (tests/price-cache-freshness.test.ts, 17 tests): a dense-but-frozen 21-close window must NOT yield a confident vol; a frozen benchmark must report stale and yield []; the deleted Deno file must stay deleted; today's provisional bar must NOT be false-rejected as future-dated. Mutation-verified — neutering the rule fails 6 of them. No migration, no schema change, no production writes.

2026-08-16: **W4/W5 — the NAV invariant now has teeth, and benchmark levels carry their session.** Part of the evaluation-pipeline-integrity remediation; the governing principle is that every fix ships with a check that FAILS when the fix regresses.

W4 — the invariant was a no-op. position-monitor/route.ts computed newNav and invariantExpected from the SAME reduce over the SAME array and compared them: invariantDiff was structurally zero and the violation branch unreachable. It was one of five checks in this incident that could only ever report green. It is replaced by reconcilePersistedNav (lib/paper/marks.ts), which re-reads paper_portfolio.nav, paper_portfolio.cash_balance and paper_performance.nav **out of the database after the write** and compares them against a NAV computed locally from cash plus the mark set, plus the cash-ledger identity and a mark-coverage contract. Both sides are independently sourced, so a dropped write, a rejected column, a partial upsert or a mark missing from NAV now produces a failing check. **A failed reconciliation marks the run error, not done**, and raises a critical paper-nav-reconcile:<market> System Health issue.

W4 — marks now carry provenance. Positions were only re-priced when a fresh quote existed, so NAV silently blended marks of different ages. Every open qty now resolves exactly one mark tagged live_quote / carry_forward / entry_cost with its source and the provider's own observation time; mixed-age NAV raises a paper-nav-stale-marks:<market> warning naming the symbols and the stale share of position value; and every mark is written to the append-only paper_position_marks ledger. **The 2026-08-12 +2.70%/-2.97% NAV round trip stays permanently unattributable** — the ledger prevents a repeat, it cannot recover the past.

W4 — one canonical EOD writer per market. PaperTrader ran a second same-day mark and upsert on the same (date, market) key PositionMonitor writes after the close, so an intraday snapshot could overwrite the EOD row. PaperTrader may now only CREATE today's row when none exists (tagged snapshot_type='intraday'); a 23505 conflict is treated as the EOD writer winning, never as a reason to clobber.

W5 — benchmark session mislabelling. Both writers accepted any positive benchmark *quote*, ignoring stale/source/session, and stamped it with the cron run date: bench_nav 708.42 is VOO's **2026-08-11** close, stored under both 2026-08-12 and 2026-08-13. Benchmark levels now come from session-dated daily bars (lib/paper/benchmark-observation.ts, reusing the existing newestBarIsStale recency guard); the bar's own date and provider are persisted; and a level is written **only** when the

bar's session equals the row's date — a gap is honest, a mislabelled number is not. benchmark-scorecard marks a proven mismatch `source_status='session_mismatch'` (so `loadBenchmarkLevels`, which reads only 'ok', excludes it) and clamps displayed coverage to 100% (US 1M reported 104.5%).

Detectors: `tests/paper-mark-provenance.test.ts` (the reconciliation MUST fail on a disagreeing persisted NAV — the old self-comparison could not), `tests/benchmark-session-alignment.test.ts` (the previous session's close MUST be refused under today's date), `tests/paper-nav-writer-contract.test.ts` (route-shaped: no `invariantExpected`, no second EOD upsert, no benchmark-from-quote, coverage clamped).

Migration `20260816180000_paper_mark_and_benchmark_provenance.sql` **APPLIED 2026-08-17.**
`paper_position_marks` table live; `bench_session_date`, `bench_source`, `snapshot_type` columns on `paper_performance` live. US relative-performance figures now have session provenance.

2026-08-13: **Hybrid protective-stop PLACEMENT WORKER — Parts A–D shipped; Part E (activation) is owner-gated.** The shadow scaffold from 2026-07-18 now has a full broker-placement path, still behind the same two false-by-default gates:

Part A — paper OHLC stop check: `lib/data/quotes.ts` now parses `day.l/day.h` from Massive snapshots into `DeterministicQuote.dayLow/dayHigh`. The position-monitor (`app/api/agents/position-monitor/route.ts`) builds a `dayLowMap` and uses `priceForStopCheck = min(currentPrice, sessionLow)` — a stop touched intraday is real even if price recovers by close. Exit reason is `stop_hit_intraday` (fill at `trailingStop`).

Part B — RH capability file (`lib/protective/robinhood-capabilities.ts`): declares `stop_market` with `timeInForce: ["gtc"]`, `sessions: ["regular"]`, `updateMode: "cancel_replace"`, no lifetime cap. Comments explain: RH GTC stop-market ONLY triggers in the regular session (9:30–16:00 ET); does NOT fire pre-market or after-hours; converts to market order on trigger (no limit floor, but you ARE out).

Part C — broker placement functions: `placeRobinhoodGtcStop()` added to `lib/robinhood-mcp.ts` — same MCP session pattern as `submitRobinhoodOrder`, `timeInForce: gtc`, `type: stop`, returns `{ok, brokerOrderId}` with `needsReconcile` on ambiguous outcomes. `placeKiteStopGtt()` added to `lib/kite.ts` — single-leg GTT (`type: "single"`) SELL CNC LIMIT at `stopPrice` (weaker: can fail to fill on a gap). `lib/protective/kite-placement.ts` is a thin wrapper.

Part D — entry/exit wiring: `lib/protective/placement-worker.ts` is the placement + cancel worker. It reads both gates, derives `stopPrice = entryPrice × (1 – stop_loss_pct%)` from the mandate, evaluates broker eligibility via `evaluateProtection()`, inserts a `protective_orders` row at `status='placing'`, places at broker, moves to `status='active'` (or 'failed'). `execute-order.ts` fires `placeProtectiveStop()` fire-and-forget after every confirmed live BUY ACK. `live-exit-monitor.ts` calls `cancelProtectiveStop()` before submitting a SELL proposal — cancels the resting broker stop to prevent double-sell after the GTC stop independently triggers.

Part E (not yet done): flip `PROTECTIVE_PLACEMENT_WORKER_AVAILABLE = true` in `lib/protective/coverage.ts` and set `protective_orders_enabled = true` in `strategy_config`. Both gates must be opened by owner before any broker stop is placed.

2026-08-13: **Intraday gap protection:** both the position-monitor paper check (Part A above) and the `PositionMonitor` now catch stops touched at any point during the session (via `day.l` from Massive snapshot), not only at the most recent quote. The overnight gap risk for paper trades is narrowed to the gap between last Massive snapshot and session open — true overnight gap is still not covered by paper (and not expected to be; paper has no broker-level orders).

2026-08-07: **Property/Investing isolation is an invariant, stated here so it is testable.** No property table, adapter, scenario, forecast or decision-journal row is read by any securities score, eligibility gate, position sizing rule, order path, exit, promotion gate or broker call. The dependency runs one way only: Property consumes the shared auth, ownership, System Health and source-provenance conventions, and exports nothing into the investing money path.

Property forecasts are **shadow decision support**. They are written state = 'shadow', are never a promise, and are scored only against values observed after the horizon elapsed. The Forecasts workspace **withholds a calibration rate below 10 matured outcomes** per market-and-metric cohort (lib/property/calibration.ts) and shows n regardless — at n=3 an interval-coverage percentage can only read 0, 33, 67 or 100 and would be arithmetic noise presented as evidence.

Property parcel Stage 1 is evidence-only and **collection is disabled** until each county source's machine-use contract is verified. Historical Phoenix deed observations and Austin county appraised/assessed references remain preserved as evidence; the latter are tax references, never public comparable sales or Kairos market-price estimates. The worker exits before credentials, scopes, or downloads, while the UI/API reject new scope activation. Bulk evidence persists no addresses or party names and uses keyed HMAC parcel identities. The UI hard-disables every AVM/market-price/value-range claim. Repeat-sales or hedonic work remains blocked until permitted collection, multiple snapshots, correction handling, temporal validation, market-local sample floors, and measured calibration exist.

Market-local honesty is enforced rather than assumed: the three active sources are US-only, so adapters declare supportsMarket() and the collector records not_applicable for Bengaluru instead of success, 0 rows. **No US value is ever substituted into an India market card**. Private data handling: no plaintext address, mortgage account number, owner name or uploaded document is stored; payloads are encrypted server-side and owner routes never trust a caller-supplied owner_id.

2026-08-01 documentation truth audit: the technical breakdown guard is a hard cap only for an ATR-scaled fall or a 7% high-volume fall. A bottom-quartile weak close is warning-only. This chapter now matches the exact scoring contract in chapter 03 and lib/data/technical.ts.

2026-07-31 scoring input safety: unknown provider taxonomy cannot receive a fabricated P/E benchmark; implausible P/E is omitted; missing availability metadata cannot include a dimension; malformed macro/insider payloads remain unavailable; and ordinary weak closes cannot hard-veto without ATR or volume/move confirmation. Only the Next.js ResearchAgent writes authoritative scores. See features/scoring-data-truth/FEATURE_ARCHITECTURE.md.

Last updated: 2026-07-25 (**ETF allocation cap** — new strategy_config.etf_allocation_cap_pct NUMERIC DEFAULT 30 (migration 20260725130000). Soft guardrail: executeApprovedOrder checks BUY orders for US ETF symbols (isEtfSymbol) and refuses if current ETF portfolio % + this order's notional / NAV would exceed the cap. Fail-open: any DB read error warns and allows through (unlike symbol blocking which fails closed). India market skipped. SELL always allowed. Configurable 0–100 via Settings → Trading → Risk Profile → ETF Allocation Cap.)

Prior: 2026-07-20 (**Guarded kill-switch reset + fail-closed risk reads**. Settings now reads the real market_controls latch and can reset it only through owner-only POST /api/settings/kill-switch: explicit market/book + acknowledgement, originating-book match, current breaker recheck with alert resolution suppressed, durable decision-journal audit, then latch write and matching-alert resolution. Failed config/NAV/history reads block risk increase; the legacy unscoped paper-query retry is removed, so schema/read failure cannot mix US and India risk truth. Trip reason and issue key include paper | live; the conservative per-market latch remains shared across books.)

Last updated: 2026-07-19 (**Webull Trading API transport restored** — lib/brokers/webull-trade/ is now a fully wired, permit-backed broker adapter. Key safety properties: (1) **Nine-gate ladder** (gates.ts) — every Webull order must clear global_trading_enabled, market_control_enabled, circuit_breakers_clear, autonomy_mode_satisfied, single_allowlisted_account (account 605420606 exclusively), orders_feature_flag (webull_trade_orders_enabled=false by default), credential_and_token, risk_checks, quantity_within_mandate; (2) **Permit system** — two permit kinds, preflight (DB flag check only) and order (full 9-gate evaluation); each permit is single-use and expires after 30 s; the transport refuses any request whose path/method is not on the permit's allowlist; (3) **Token preflight** — preflight.ts calls POST /openapi/auth/token/check before gate 7 to resolve live token status (PENDING/NORMAL/INVALID/EXPIRED) and idle-age; assertTokenUsableForOrder() blocks on PENDING (now explicit in types), INVALID, EXPIRED, and idle > 15 days; (4) **Single fetch locus** — liveWebullTransport() is the only function in the codebase that calls fetch() to api.webull.com;

(5) **Signing** — HMAC-SHA1 over path + sorted params + MD5 body hash, `${appSecret}&key`, `x-access-token` header alongside HMAC headers, timestamp freshness enforced; (6) All flags remain false: `webull_trade_orders_enabled=false`, `global_trading_enabled`, and account allowlist require explicit owner activation before any order is possible. Constraint migration 20260718130000 applied: `protective_orders.mode` locked to `'wider_disaster_floor'`, currency NOT NULL, five new DB-enforced invariants on `market/currency/broker-id/floor/order-kind/learning-provenance` — zero rows in table at time of apply, all constraints verified safe.)

Prior: 2026-07-18 (**Hybrid protective-stop SHADOW SCAFFOLD — built UP TO the placement line, no live order ever.** New pure module set under `lib/protective/`: (1) a broker-neutral `BrokerProtectiveCapabilities` matrix (`capabilities.ts`) — protection is capability-driven per order-type/TIF/session/account, never a flat broker boolean; an adapter with no eligible MULTI-DAY order for the exact position is `unprotected-by-broker`, never silently protected; (2) Kite's declared capability (`kite-capabilities.ts`) filled from the PROVEN GTT code — Kite protects via the WEAKER `gtt_limit` (LIMIT child, unfilled-trigger risk surfaced), and its DAY-only regular SL-M is declared and correctly REJECTED as a multi-day floor; (3) a pure disaster-floor calculator (`disaster-floor.ts`) parameterized by (`mode`, `distance`) — Q1 unanswered so mode defaults to `wider_disaster_floor` (outage + catastrophic-loss mitigation, NOT touch-at-analytical-stop) and the distance is a CONFIG input with no hardcoded value; monotonic ratchet so a falling high-water mark can never lower the floor; (4) a pure reconciliation loop (`reconcile.ts`) detecting out-of-band triggers, partial fills, cancels, expiry, broker edits, and corporate-action qty drift — unknown state is always `needs_reconcile`, a trigger without a confirmed fill never closes the book, a gap through a limit child is reported `unprotected` (not filled), expired protection is critical; (5) the state model (`state.ts`) — the `protective_order` record shape + status machine + long-only/cancel-before-replace invariants (total executable SELL never exceeds reconciled held qty; a competing SELL is blocked until cancellation is CONFIRMED). **Exit provenance (Codex's correction):** a disaster-floor fill records `exit_reason = protective_disaster_floor` and `learning_scope = risk_policy_only` — the loss STAYS in P&L, NAV, drawdown, mandate and risk-policy evaluation (real money); ONLY the Learner's signal-weight attribution + genome promotion exclude it, so broker capability can't contaminate weight learning (a generic `excluded_from_learning=true` is insufficient because the evaluation engine also filters that field). **THE MONEY LINE:** `placement-gate.ts` — a false-by-default `strategy_config.protective_orders_enabled` flag gates ALL placement and STAYS FALSE; `planProtectivePlacement()` produces the intended broker action as a plain object and NEVER calls a broker. Deterministic, NO LLM on the money path. US/India never cross (per-market capability scope). Migration 20260718000000 `protective_orders_shadow.sql` written as a PROPOSAL and **NOT applied to prod** (creates `protective_orders` + append-only `protective_order_events`, adds the flag + `learning_scope` columns). 32 acceptance/unit tests in `tests/protective-hybrid-stop.test.ts` (all 14 spec acceptance tests, each falsifiable — mutation-verified: breaking the ratchet fails AT3, breaking the cancel-guard fails AT1). Gated behind owner approval of touch semantics (Q1), floor distance, and post-fill policy before anything goes live. See "Hybrid protective-stop shadow scaffold" below + `features/hybrid-stop/FEATURE_ARCHITECTURE.md`.)

Prior: 2026-07-17 (**Research visibility on the risk surface — a DISPLAY JOIN, coupled to nothing.** `GET /api/portfolio/risk-daily` now attaches a nullable per-holding research block (`score`, `direction`, `scored_at`, `sessions_since`, `days_since`, `state`, `scored_as_holding`) joined from the latest `agent_signals` row per (`symbol`, `market`) — never symbol alone. **Invariant R1: no field of it is read by** `computeHoldingRisk`, `sba-v1`, `constructPortfolio`, **the execution kernel, or any gate** — the risk engine stays research-free BY DESIGN, because a sector is over-cap *because* research liked that sector and letting `analyst_score` also veto the cap double-counts the same signal. **Why it exists: on 2026-07-16 AVGO was 6 days unscored while this panel said "trim", and nothing on screen said so — the AGE is the feature, not the score.** Staleness is measured in market-local **SESSIONS** (reusing `marketSessionsSince`; a Friday score read Monday is 3 days but ONE session — a calendar-day rule would paint the book stale every Monday) and displayed in days. Four non-collapsed states: `fresh` (≤ 2 sessions) · `stale` (warning + day count; annotates the SCORE only, never the action) · `never` (no signal ever — deliberately NOT a link) · `unavailable` (abstained — never rendered as a number). Every annotated row is labelled a screener **candidate** score: `is_holding` is false in **463/463** prod rows, so a `neutral` there does not mean "no exit signal" — the exit question was never asked. Fail-soft: an `agent_signals` error still renders the risk table with an explicit "research unavailable". R1 is pinned behaviorally AND architecturally by `tests/risk-research-annotation.test.ts`, whose coupling detector is itself falsification-tested. Supersedes the unmerged `features/risk-research-integration` (research *ordering* trim absorption — not pursued). No schema change, no migration, no new cron, no LLM. See "Daily Per-Holding Risk Analytics" below + `features/risk-research-visibility/FEATURE_ARCHITECTURE.md`.)

Prior: 2026-07-16 (**Sector-cap breach ALLOCATOR** — `hr-v1` → `hr-v2`. Defect fixed: a sector-cap breach is a property of the SECTOR, so `sectorUtil >= 1` was the identical number for every holding in the sector and `hr-v1` handed EVERY Technology name the identical `trim` (the live AVGO advice: "Trim your position because Technology holdings exceed the 30% sector cap (at 65.6%)") without ever deciding WHICH names absorb the breach or HOW MUCH each gives up — arbitrary and unactionable. New pure module `lib/risk/sector-breach.ts` (`sba-v1`) allocates the breach deterministically by **water-fill**: trim the largest names

in the sector down to a common level L where $\sum \min(w_i, L) = \text{cap}$. Justified over pro-rata (which re-ships the same blanket verdict and leaves the name-cap breach untouched) and over "marginal contribution" (for a sector-weight cap, a name's marginal contribution IS its weight — the same rule with extra abstraction). NAV basis, because live-portfolio-gate enforces the owner's cap as value/NAV — the invested basis would make the advice ~43% wrong. Safety properties: **(1) exits untouched** — the `exit_review` branch is first and unconditional; no allocation, and no absence of one, can delay or suppress a protective-stop/thesis-break exit; **(2) risk-internal** — a function of weights and one owner-set cap, ZERO research/analyst_score coupling; **(3) defaults to honest** — a sector breach with no usable allocation yields `review + missing_inputs: ["sector_breach_allocation"]`, NOT a fallback to the old blanket trim; **(4) sector-unknown degrades honestly** — excluded from every sector total, never bucketed into a synthetic sector, never assumed cap-compliant; **(5) LLM still prose-only** — `parseStrategyNotes (lib/risk/strategy-notes.ts)` can only emit `Map<requestedSymbol, string>`, proven by test. Non-selected names now say hold **with the reason they weren't selected**. Read-only accounts (everything but `<agentic-account-id>`) are labelled advisory-informational. No migration. See "Daily Per-Holding Risk Analytics" below + `features/risk-sector-breach-allocation/FEATURE_ARCHITECTURE.md`.)

2026-07-21 router proof hardening: cohort evaluation is cache-only and cannot lease, call, or enqueue provider work. ResearchAgent copies already-fetched deterministic score inputs into the canonical cache through an internal read-only adapter. Activation now requires separate `safety_pass` and `quality_pass`, a fresh selected proof, and ten distinct validated ResearchAgent `as_of_session` values in a 45-day window for the exact market/policy/code/strategy tuple. Weekend/holiday staged rows do not count. Existing rows default false and cannot authorize cutover. Router remains shadow-only, `router_enabled=false` both markets.

2026-07-31 ADR safety: reviewed ADR identity is explicit, never inferred. SKHY uses the Nasdaq ADS and ADS-basis Yahoo fundamentals; retired/OTC proxies (SKHYV, HXSCL, HXSFC) are rejected by the shared paper/live symbol policy. A thin ADS source becomes unavailable rather than falling through to foreign-underlying per-share data. ADR support adds no live-trading permission and does not bypass broker review or any existing market/account/risk gate.

Prior: 2026-07-16 (**Runtime evidence-degradation guard** — a NEW safety gate on the research entry path, shipped **measure-only**. Problem it solves: scoring renormalizes weights across **available** dimensions, so a dimension dropping out (provider outage OR a routing change) could push a symbol from ineligible→eligible on **missing data rather than new information**. The guard compares each symbol's current availability/quality mask against the last accepted market-local baseline; if a REQUIRED field degrades (fresh→stale beyond ceiling, available→missing, valid→conflict/quarantined) it abstains from any NEW long whose eligibility depends on renormalizing around that degradation. Safety properties, all enforced: **(1) strictly subtractive** — the only transformation is `long → neutral`; short **passes untouched in every mode**, checked before the mode branch AND enforced by a schema CHECK, so no code path can persist a guard event that **created** an entry; **(2) never suppresses an exit** — existing holdings continue through PositionMonitor's normal risk/exit logic, an evidence outage cannot block a stop or mandatory exit; **(3) defaults to abstain** (no baseline + unusable required field ⇒ abstain), and **only clean runs re-baseline**, so a persistent outage never normalizes itself; **(4) fail-safe config** — `EVIDENCE_DEGRADATION_GUARD_MODE` defaults to `measure_only` and an unparseable value ALSO falls back to `measure_only`, so a broken config can neither silently enforce nor silently stop recording; **(5) one aggregated health event per run**, not one alert per symbol. Modes: `off | measure_only | enforce`. **Shipped in** `measure_only` — it records what it **would** abstain and changes no direction. Flipping to `enforce` is an owner decision after observing its logged would-abstain rate. Also: `analyst.consensus` is classified narrative-only (US) / unsupported (India) and excluded from gated intents, so it can never block a cutover. Router itself remains shadow-only, `router_enabled=false` both markets.)

Last updated: 2026-07-15 (LLM-discretion exit hole CLOSED — the last place LLM output could move money. Research direction gate extracted to pure `lib/signal-direction.ts` (unit-tested, `tests/signal-direction.test.ts`): held-position exit ("short") is now DETERMINISTIC — `isHeld && analystScore < mandate threshold` — the LLM's direction field NEVER sets an executable direction (previously an LLM "short" on a held name became an exit signal stored as `deterministic_v1`, and could teach the learner from LLM-created outcomes). SELL capability on holdings preserved per locked rule, now evidence-driven; LLM opinion kept advisory-only in `research_packets.raw_data._original_direction`. LearnerAgent's reassess flag renamed `llm_exit→score_reassess_exit` (score-only trigger); PositionMonitor honors both (legacy drain). Historical contamination verified ZERO (no closed trade ever exited via `llm_exit` or a long→short LLM flip). Entries were already deterministic; paper/live consumers already require `score_source="deterministic_v1"`.)

Prior: 2026-07-15 (Supabase Security Advisor remediation — `20260715120000_security_rls_and_rpc_lockdown.sql`: the public anon API key could read 16 RLS-disabled public tables (incl. `agent_config/learner_config`) and call SECURITY

DEFINER RPCs (kairos_call_agent, activate_evidence_policy, ...) because they carried the default GRANT EXECUTE TO PUBLIC. Fix: RLS deny-all on 15 agent-internal tables + authenticated-read on newsletters (service_role bypasses, so agents/crons unaffected); REVOKE EXECUTE ... FROM PUBLIC on the anon-callable definer RPCs (keeping service_role, and authenticated for the owner's get_daily_ai_count); pinned search_path=public on 15 definer/trigger fns. Verified via get_advisors: 0 ERROR, 0 anon-executable definer functions. Deferred WARNs: 7 always-true policies tighten at multi-tenant, pg_net schema move, Auth leaked-password toggle.)

Prior: 2026-07-11 (Proactive broker-token health check — checkRobinhoodTokenHealth attempts a CAS refresh on the short-lived RH access token from the status route + health-triage cron (6h), reporting broker-token:robinhood only on a genuinely failed/absent refresh; Settings badge shows "Reconnect required" only on a real dead-refresh, else "Connected — valid until <access-token TTL>". See "Proactive broker-token health check" section. Prior: Daily Per-Holding Risk Analytics advisory surface.)

Prior: 2026-07-11 (Codex Phase-B re-review remediation: (Codex#2/#3) the Kite identity gate no longer trusts allowlist *text* alone — verifyKiteTradingIdentity() (lib/kite.ts) now fetches Kite /user/profile and requires the CONNECTED token's user_id to equal strategy_config.active_account_india AND an allowlisted broker_accounts{broker=kite,market=india,role=trading} row. It is enforced at the single placeEquityOrder() choke point, so the canonical/autonomous/exit paths (via kiteAdapter.submitOrder) get the same check as the standalone route — not just the route. Fail-closed: config read err ⇒ 500, unset/absent/view_only ⇒ 403, profile unfetchable / user_id mismatch ⇒ 502/403. (Codex#1) the v2 budget advisory lock DROPS broker from its key (now local_date:market:env) so it matches the market-wide cap it guards — two brokers in one market can no longer take different locks and jointly exceed the cap. (Codex#4) migration 153 rejects non-finite (Infinity/NaN) qty/notional/cap and validates side/env/broker/symbol/order_type enums+identifiers, fail-closed. Migration 153 additive over 152 (never edited).)

Prior: 2026-07-11 (Phase B residuals of 07_08_FULL_APP_REVIEW: A2 the standalone India Kite order route (app/api/kite/order) now enforces a fail-closed identity/allowlist gate — it requires strategy_config.active_account_india to match a broker_accounts{broker=kite,market=india,role=trading} row before reserving budget; unset/absent/view_only ⇒ 403 (no silent fallback), so India live is blocked until an allowlisted Kite trading row is inserted. Canonical-path *unification* still deferred. A4 read-only NAV reconciliation report GET /api/paper/nav-reconcile (owner-gated, zero writes) re-derives nav == cash + $\sum \text{qty} \cdot \text{price}$ per pool. A5 v2 daily-BUY budget window is market-local (America/New_York / Asia/Kolkata), not UTC; advisory lock keyed by local_date:market:broker:env. Dead edge-fn kill-switch copy deleted.)

Prior: 2026-07-11 (Phase A P0 remediation of 07_08_FULL_APP_REVIEW: A1 kill switches take explicit {book, accountId} context — mode no longer inferred from live_auto_enabled; live baseline = account's own snapshot peak not START_NAV; new sellAllowed separates risk-increase from risk-reduction so a trip blocks BUY but not a verified SELL; no_baseline/stale_snapshot fail-close BUY only. A3 durable broker ACK (bounded DB retry → 202 needs_reconcile, never {ok:true}). A4 PositionMonitor NAV write errors now fatal. A5 budget-RPC v1/v2 EXECUTE revoked from public/anon/authenticated.)

Prior: 2026-07-10 (Phase 1 P0: L4 enforcement, conviction normalization, India currency, duplicate SELL, cancel-on-kill BUY-only; Codex P0/P1: breakdown veto, calibration OOS gate, promotion governance.)

Update when any authorization, scoring eligibility, limit, account, order, reconciliation, exit, or kill-switch behavior changes.

2026-08-24 instrument-family safety boundary: taxonomy and metals drivers are measurement-only. The actionable ETF cap and every existing paper/live gate remain unchanged. instrument_family_observations, the owner diagnostics route, and family_uncapped_v1: * shadow rows have no execution consumer. Unknown or leveraged/inverse classification cannot silently enter a special model. Any future family score requires market-local forward evidence, isolated paper, explicit owner promotion, and the same shared money-path controls. The family ledger blocks UPDATE, DELETE and TRUNCATE by grants plus triggers.

2026-07-26 policy-event ledger: US FOMC schedule, official target-range outcomes, and post-event return observations are display/measurement-only. Missing market expectations render unavailable; they never become a neutral or zero surprise. The ledger has no scorer, sizing, paper, live, exit, broker, or India reader. Post-event impacts use only frozen daily-return evidence and are append-only.

Overview

2026-08-06 security boundary: browser-callable

`get_daily_ai_count(p_user_id)` is a SECURITY DEFINER RPC for the owner's daily AI counter. Migration `20260806183039_restrict_daily_ai_count_to_caller.sql` requires `auth.uid() = p_user_id` inside the function, so an authenticated caller cannot query another user's count by substituting a UUID. Service-role access is retained for trusted server work.

Manual and future autonomous orders must pass one shared Execution Gateway. Manual and auto differ only in **who authorizes the proposal**; they do not have separate money-safety implementations.

```
flowchart TD
    INTENT[Trade proposal] --> ACTOR[1. Owner or authorized auto lease]
    ACTOR --> VERSION[2. Deterministic live-approved scoring version]
    VERSION --> ENABLED[3. Global market broker account enabled]
    ENABLED --> KILL[4. Kill switches and critical alerts]
    KILL --> QUALITY[5. Data evidence and mandate]
    QUALITY --> ACCOUNT[6. Fresh target-account NAV positions buying power]
    ACCOUNT --> LIMITS[7. Per-order and portfolio limits]
    LIMITS --> QUOTE[8. Fresh quote spread drift]
    QUOTE --> RESERVE[9. Atomic daily budget and idempotency]
    RESERVE --> PREVIEW[10. Broker preview echo]
    PREVIEW --> SEND[11. Broker submit]
    SEND --> RECON[12. Durable lifecycle sync and protective exit]
```

Unknown, null, stale, malformed, or errored state on a live BUY fails closed. Verified risk-reducing SELL exits remain exempt only from BUY budgets/caps; they still require account, holdings, quote, idempotency, broker, and audit checks.

Completed-session and schedule boundaries (2026-07-31)

Daily ResearchAgent technical evidence is filtered after provider normalization and before scoring, return capture, trade-plan construction, or evidence persistence. A market-local current-date daily candle is admitted only after 16:00 America/New_York (US) or 15:30 Asia/Kolkata (India); malformed/future dates are rejected. Intraday quote consumers are unaffected.

US research, paper-entry, and daily monitor jobs use paired EDT/EST UTC schedules with an exact route-level `local_slot` contract. The seasonal duplicate exits before provider or database work. This prevents the close monitor from shifting to 15:15 ET in winter and prevents entry/research slots from silently moving by an hour.

India headline/event collection is shadow-only. Its intents have no scoring, learner-mutation, paper/live execution, position-monitor, or broker reader; unknown coverage remains unavailable rather than becoming a neutral score.

Gate details

1 — Authorization envelope

Manual: `requireOwner()` + request/CSRF guard + owner approval/Send action.

Future auto L4: all of deployment `AUTONOMOUS_LIVE_ENABLED=true`, `autonomy_level='L4_live_small_auto'`, `live_auto_enabled=true`, unexpired owner lease, authenticated cron worker, and a single-run lease. The gateway (`executeApprovedOrder`) enforces `autonomousWorkerAllowed(autonomy_level)` for non-owner actors — `L3_live_manual` is insufficient and returns 403. The autonomous path cannot use owner-only risk overrides.

Enabling an auto envelope is a human money/config change and is journaled. It is not permission for an LLM to change caps, accounts, strategy lifecycle, or code.

2 — *Signal and scoring eligibility*

Live BUY requires:

- deterministic `score_source`;
- strategy/scoring lifecycle `live_approved` with linked validation evidence;
- market/asset/setup allowed by the mandate;
- unexpired proposal and signal;
- long-only new-position decision;
- no unresolved taint or invalid LLM veto state.

A score threshold or `eligible_for_live_review` flag alone never grants live eligibility.

Breakdown veto (deterministic, runs before momentum math). `scoreTechnical`s (`lib/data/technical`s.ts) evaluates `detectBreakdownVeto` first. A last bar down at least 2.5 ATR, or down at least 7% on at least 1.5x normal volume, caps the technical score at 20 regardless of RSI/EMA. A bottom-quartile close on a down bar is recorded as a warning but does **not** trigger the cap. In plain language: an unusually large, well-confirmed fall can block a misleadingly strong momentum score; a merely weak close supplies context rather than pretending to prove a breakdown. This closes the prior bug where a 12% high-volume reversal scored about 100 because RSI had fallen into the preferred band while price was still above the EMAs. Thresholds remain v0 guardrails needing prospective validation per liquidity bucket.

Promotion governance (`app/api/strategies/versions/route`.ts). A champion can only be promoted with a PASSED validation experiment: the `force_unvalidated` bypass is hard-rejected (400). Demotion of the prior champion is always market-scoped — the former unscoped demote-all fallback is removed and now aborts the promotion on error, so promoting an India challenger never touches the US champion.

Automated validation boundary (migration 170, `activate_strategy_shadow` RPC, `lib/validation/automation`.ts). Automatic challenger validation + shadow routing

(gated per-market by `strategy_validation_automation`, fail-closed) has exactly one automatic lifecycle transition: a PASSED challenger → non-executing shadow_paper.

It **cannot** promote a champion, create a paper fill, move cash, make a broker proposal, or place a live order — the RPC is `service_role-only`, holds a per-market advisory lock, caps at one shadow (`max_active_shadows` 0–1), and refuses any champion/terminal/unvalidated version. Promotion and every execution gate above stay separate and owner-only.

Paper accounting integrity (2026-07-13). Two silent-write bugs are fixed and guarded: (1) `disableTrading` (kill switch) wrote a non-existent `strategy_config.notes` column, so PostgREST rejected the whole update and the switch never actually set `trading_enabled=false` — notes removed, the switch now halts as intended. (2) An earlier `paper_portfolio` update bundled a non-existent `open_positions` column and silently dropped close-proceeds cash credits, understating NAV and tripping a PHANTOM drawdown/kill switch on India (~■197k). The lost cash was reconciled from the trade ledger ($\text{seed} - \Sigma \text{open} - \text{cost} + \Sigma \text{realized}$), and the position-monitor now runs a **ledger reconciliation guard** every cycle: if `cash_balance` drifts from the ledger beyond 0.5% of seed it raises `paper-cash-drift:<market>` (warn) so drift is visible and actionable BEFORE the drawdown breaker acts on corrupted NAV.

Capital rotation is shadow-only (containment restored 2026-08-10). Migration 20260723120000 had enabled paper execution before the TypeScript executor enforced the shadow evaluator's cost, tax, turnover, correlation and economic-edge readiness. It also ignored `rotation_allow_score_only_paper=false`, so four score-only swaps reached paper books. Migration 20260811033335 disables both paper rows. The executor now fails closed on the score-only flag before reading positions or invoking the atomic RPC. Shadow measurement stays enabled; live rotation remains disabled. Re-enabling paper rotation requires a new reviewed change that enforces every P1 readiness blocker in the execution transaction.

New P0 rows now carry a fail-closed P1 readiness contract. Rotation source selection reuses the canonical paper exit-plan projection; post-swap sizing and measured correlation are evaluated over the same market only; persistence counts distinct prior runs; turnover uses filled paper-order notional against same-market NAV; and missing tax lots, economic mapping, configured turnover, correlation, or complete query results remain explicit blockers. These fields are diagnostic only and cannot relax an entry, exit, cash, position, or order gate. Agents -> Rotation exposes the market-local evidence without an enable control.

Per-market pause/kill isolation (migration 171). The pause and kill-switch state was GLOBAL (`strategy_config.app_paused/trading_enabled`), so one market's breaker halted BOTH — India's phantom drawdown even skipped the US research run. Now `market_controls` holds one row per market; the drawdown breaker calls `setMarketPaused(market)`, the kill switch `setMarketTrading(market, false)`, and every gate reads `isPaused(svc, market) / isTradingEnabled(svc, market)` (`lib/market-controls.ts`, fail-closed on read error). A market's trip isolates to that market; the legacy global flags are retained as a **master-kill** that still stops everything. Research is no longer gated by the pause at all (it is measurement — only entry paths pause). Exits keep running during a pause. Owner resumes a single market from the sidebar per-market banner (`/api/settings/pause` with a market).

3 — Trading/broker/account enablement

All global, per-market, broker, and account toggles must be true. Broker resolution fails closed.

US order account is exactly Robinhood agentic account `<agentic-account-id>`. Account `<read-only-account-id>` is read-only for the approved research-holdings use; its NAV/positions cannot size or authorize agentic-account orders. The real implementation currently hardcodes/resolves account IDs; the documentation must not claim otherwise. Credentials/tokens remain encrypted in the vault and never enter code/logs.

4 — Kill switches

`lib/kill-switches.ts` checks per market for daily loss, peak drawdown, and rolling accuracy, disables trading, and creates a critical alert. Submit-time checks must rerun immediately before reserve/send.

One entry per open alpha name (2026-08-06; queue starvation repair 2026-08-10). A fresh score can reassess a held position but cannot create an unvalidated add-to-winner. PaperTrader excludes held alpha names before its per-market top-N new-entry selection and supersedes their entry-ineligible signals; PositionMonitor independently reads the latest deterministic holding signal regardless of that status. The database independently rejects a concurrent bypass. A separately validated and owner-approved add-to-winner policy would need its own risk budget, attribution, and shadow evidence before this rule may change.

Accuracy-gate minimum sample (MIN_ACCURACY_SAMPLE = 20, corrected 2026-07-29). Paper accuracy uses the actual realization timestamp and aggregates every entry lot closed for one symbol at one timestamp into one exit episode. The combined episode return is classified with the currency-neutral breakeven band; breakevens and tainted rows are excluded. The gate trips only after **≥20 directional exit episodes**. Live filled-order pairing is not a kill-switch input: pairing a SELL with the latest BUY is not broker-confirmed lot accounting and is wrong under partial fills. Live daily-loss and drawdown remain immediate from account NAV.

Guarded manual reset (2026-07-20). The per-market strategy toggle is not the breaker latch. Settings reads `market_controls` separately and exposes reset only while its `trading_enabled` is false. Reset names the originating paper | live book, reruns that book's deterministic checks, refuses a mismatched book or active market pause, writes an unresolved audit record before enabling, and resolves it plus only the matching kill alert after the latch write succeeds. It never changes thresholds, accounts, global/security pause, credentials, or autonomy flags. Risk/config reads are strictly market-scoped and fail closed; the old pre-schema unscoped retry is forbidden.

`checkKillSwitches(svc, { market, book, accountId? })` takes an **explicit book/account context** (A1/P0-1). Mode is NO LONGER inferred from `live_auto_enabled` — an L3 manual-live order (`live_auto_enabled=false`) must still measure

real live NAV, so the caller declares the book:

- `book:"paper"`: reads `paper_portfolio / paper_performance` plus realized `paper_trades` exit episodes. A bare-string market arg (`checkKillSwitches(svc, "us")`) is the back-compat paper form.
- `book:"live"`: reads `live_account_snapshots` for the resolved account (`accountId`, else `active_account_{market}`) for daily-loss + drawdown. Accuracy remains analytics-only until a reconciled realized-lot ledger exists.

Live baseline is the account's OWN 90-day snapshot peak, never a static START_NAV — a real \$36 account is not measured against a \$10k paper floor.

Fail-closed for BUY (`result { safe:false, sellAllowed:true }`) on any of: no configured account or no snapshots (`tripped:"no_baseline"`), or newest snapshot older than `KS_LIVE_SNAPSHOT_MAX_AGE_MS` (default 6h, `tripped:"stale_snapshot"`).

Risk-increase vs risk-reduction are separated. The result is `{ safe, sellAllowed, reason?, tripped? }`. A live daily-loss / drawdown trip sets `safe:false` (blocks BUY) but leaves `sellAllowed:true` — a risk-reducing SELL that has passed fresh exact-account held-quantity verification is not blocked. Callers gate as `ksBlocks = side=="sell" ? !sellAllowed : !safe`. A freshness fail-close likewise blocks BUY only. PaperTrader consumes `safe` only for new entries; PositionMonitor's deterministic held-position exits continue independently. `security_locked` still blocks live execution.

Atomic SELL idempotency: `trade_proposals_active_sell_uniq` partial unique index on (`symbol, market`) WHERE `side='sell'` AND `status IN ('pending_review','queued_auto')` enforces at most one active autonomous SELL per position at the DB level. Concurrent exit-monitor runs hitting the same position get a 23505 conflict, not a duplicate SELL.

For L4, any unresolved critical trading/data/reconciliation alert blocks new entries. Cancel-on-kill cancels only resting BUY orders — protective SELL orders are explicitly excluded (canceling an exit increases open exposure). Risk-reducing held-position exits remain allowed where state can be verified.

5 — Data quality and overrides

`data_confidence` uses structural applicable base weights:

`fresh valid applicable base weight / all structurally applicable base weight`

Inapplicable dimensions are omitted from both terms; missing/stale/failed/degraded dimensions stay in the denominator and contribute zero. Post-renormalization `applied_weights` are never the denominator.

Manual owner may use `acceptLowQuality` only with a durable reason written before the order. Auto has no quality or portfolio-risk override. `quality_status=unknown`, missing decision link, or confidence error blocks auto/live BUY.

6 — Fresh account state

NAV, positions, open orders, and buying power must come from the actual target account and meet explicit freshness bounds. There is no `FALLBACK_NAV` for live or auto sizing. If the target account cannot be read, BUY size is zero and SELL authorization fails unless current holdings can be independently verified.

India values remain INR and US values USD. No currency conversion is implicit. If a future cross-currency limit is needed, the FX observation/source/time is explicit and conservative.

7 — Limits and portfolio construction

Current per-order limits live in `strategy_config.max_order_notional_usd / max_order_notional_inr`; daily limits use `max_daily_notional_*` and `max_daily_trades`. Do not claim these are `broker_accounts.notional_cap_usd` unless the schema is actually migrated.

Final BUY size is the minimum of opportunity size, per-order cap, remaining atomic daily budget, buying power, and name/sector/gross/correlation/volatility limits. Quantity rounds down; zero means abstain. SELL that reduces a verified holding is exempt from BUY notional/daily budgets.

8 — Quote and drift

A fresh executable quote is obtained immediately before reservation. Validate positive finite price, retrieval age, spread/liquidity, and drift from proposal/approval. Use a marketable limit collar when the broker schema supports it; never guess tool parameters.

9 — Atomic budget and idempotency

All live submit paths must call the atomic budget-reservation RPC. It counts reserved/submitted/partial/unknown live BUYs and inserts `broker_orders.status='pending_submit'` in the same transaction. Unique active order per proposal is the hard duplicate backstop.

Advisory-lock scope must match the cap's query scope (Codex#1, migration 153). The daily-BUY cap counts/sums `broker_orders` filtered by `market + broker_env='live' + side='buy'` — it is **market-wide**, NOT per-broker. So the serializing advisory lock is keyed `hashtext(local_date:market:env)` and MUST NOT include broker: a broker in the key would let two concurrent live BUYs in one market but different brokers take **different** locks, both read the same pre-order total, and jointly exceed the market cap. Distinct markets still hash distinct and never block each other. (Migration 152 wrongly included broker; 153 narrows it back to the query scope.)

Fail-closed input validation (Codex#4, migration 153). The `SECURITY_DEFINER` RPC rejects malformed service input rather than silently reserving: non-finite numerics (`Infinity/-Infinity/NaN` pass a naive `> 0` check, so `qty`, `estimated_notional`, and `max_daily_notional` are checked explicitly), a non-canonical `p_side/p_broker_env` (e.g. uppercase `'BUY'` would skip the lowercased BUY-cap branch yet still insert a live row), and empty `broker/symbol/order_type` identifiers. A live BUY must additionally carry a positive finite notional.

The current RPC records `approved_by_user=true` for owner and `false` for `autonomous_worker` via `p_execution_actor(v2)`. A read/sum/check in TypeScript is forbidden because concurrent requests can exceed the cap.

10 — Broker preview

Robinhood requires `review_equity_order` before place. Preview must echo account, symbol, side, quantity, and order type. Any missing/mismatch/error blocks submission. Adapter schema is discovered from the live MCP tool list; no LLM constructs parameters.

11 — Submit outcome

- confirmed success with broker ID → submitted;
- clean reject/error → definitive error state;
- timeout/possible success/no broker ID → `unknown_needs_reconcile`, budget remains reserved, retry blocked;
- every transition produces a durable event/audit record.

Email is secondary notification, never the source of truth.

12 — Fill reconciliation and exits

Order sync handles submitted, partial, filled, cancelled, rejected, and unknown states. Partial fill never triggers blind remainder resubmission and available quantity accounts for open SELL orders.

Autonomous BUY is prohibited until a deterministic live protective-exit path exists. Stops/time exits/targets use the same Gateway and verified held quantity. A protection/monitor heartbeat failure disables new autonomous entries. Tax and dividend preferences cannot delay a risk stop.

Autonomy ladder

Use only schema values from migration 124:

Level	Meaning	Live placement
L0_research	research only	none
L1_paper_auto	automated paper	none
L2_shadow	shadow live recommendations	none
L3_live_manual	owner-approved live	owner action required
L4_live_small_auto	future small autonomous envelope	only after architecture phases and explicit enablement
L5_scaled_auto	future scaled envelope	not implemented

Unknown values fail closed. Documentation/UI must not use obsolete names such as `paper_only` or `live_supervised`.

Account allowlist

Account	Market	Role	Allowed use
<agentic-account-id>	US	agentic/trading	only Robinhood account permitted for Kairos orders and order-account sizing
<read-only-account-id>	US	view-only/manual	approved read-only holdings research; never order placement or agentic sizing
configured Kite account	India	trading	official Kite API, INR limits, CNC delivery, separate manual gate today

Every broker/account lookup is scoped by broker, market, role, enabled state, and account ID. No silent default.

Kite verified-identity gate (`verifyKiteTradingIdentity()` in `lib/kite.ts`). The allowlist row is text — it says which account *should* be connected, not which one *is*. So before any Kite submit the gate ALSO fetches Kite `/user/profile` and requires the connected token's `user_id` to equal `strategy_config.active_account_india` (both are the same immutable Zerodha `user_id`, written together by the OAuth callback). It requires the matching `broker_accounts{broker=kite,market=india,role=trading}` allowlist row as well. Fail-closed: config read error $\Rightarrow$ 500; account unset, allowlist row absent, or `view_only` role $\Rightarrow$ 403; profile unfetchable or `user_id` mismatch $\Rightarrow$ 502/403. No silent fallback.

The check is enforced at the single `placeEquityOrder()` **choke point** (Codex#2/#3), which every programmatic Kite submit passes through — the standalone route `app/api/kite/order`, which also runs the check before budget reservation so a mismatch never strands a pending row) AND the canonical/autonomous/exit paths that reach it via `kiteAdapter.submitOrder`. Gating the choke point, not just the route, closes the earlier gap where the canonical path bypassed the route-level block. Because no Kite row exists in `broker_accounts` today, all India live orders currently refuse until the owner inserts an allowlisted Kite trading account whose id matches the connected token — the intended posture until the path is unified onto the canonical `executeApprovedOrder` service.

State tables versus immutable ledgers

Do not describe every financial table as immutable; several require lifecycle updates.

Append-only / no UPDATE or DELETE:

- decision_observations;
- paper_order_events;
- strategy_evaluations;
- evidence_records (subject to its existing immutable design);
- target broker_order_events.

Mutable current-state/audited tables — never hard-delete financial history:

- paper_trades is updated when a trade closes;
- broker_orders is updated as broker lifecycle changes;
- trade_proposals changes approval/execution status;
- paper_positions represents current open state and may be removed/closed only through the transactional exit path.

Every material state transition must have an append-only event/journal record. Cleanup jobs never delete financial/audit history.

Fail behavior matrix

Failure	Manual BUY	Auto BUY	Verified risk-reducing SELL
Auth/actor invalid	block	block	block
Scoring version not live-approved	block	block	allow only if independently triggered by risk exit and holding verified
Quality unknown/low	block unless audited owner override	block, no override	do not block risk exit solely for entry-data quality
NAV/portfolio stale	block or audited manual portfolio override	block, no override	require fresh held quantity; NAV cap exempt
Quote stale/missing	block	block	block
Daily BUY cap full	block	block	exempt
Broker timeout	reconcile/no retry	reconcile/no retry + disable new entries	reconcile/no retry
Exit protection unavailable	owner warned/manual decision	block entry	alert/escalate

Run accounting, not liveness (W6, 2026-08-16)

The defect class. In the 2026-08 evaluation-pipeline incident, five separate checks could only ever report green. Three were monitoring:

Check	Why it could not fail
label-maturation response	returned {"success":true,"matured":0,"skipped":800} for 25 consecutive days
agent_runs.status	recorded done for that same zero-output run
stale-check	asked "did it run", never "did it produce" — and status='error' was written faithfully and read by nothing, so the US PositionMonitor runs that aborted on 2026-08-13 and 2026-08-14 (stops unevaluated for every remaining holding after the failure) counted as healthy

The rejected fix. `assertProductiveRun({attempted, produced})` was proposed and refuted. Zero output is frequently CORRECT — no qualifying signal, no exits needed, no new research. `{"skipped":true,"reason":"weekend + shallow backlog","backlog":0}` is a healthy response. A check that alerts on it trains the operator to ignore the channel.

Layer 1 — within-run accounting (`lib/monitoring/run-accounting.ts`, pure, no schema).

A run reports counts over the units it owned, not a boolean:

```
state    = no_work | no_action | completed | partial | blocked | failed
eligible = succeeded + expected_skip + deferred + unavailable + failed
```

Reason buckets are heterogeneous by design — `expected_skip` (legitimately no work), `deferred` (real work postponed by a budget/cursor), `unavailable` (a needed input was missing), `failed` (tried and errored). Encoding those semantics is exactly why the contract lives in code and not in a mutable config table: a DB row would move the judgement outside code review.

Alerts fire on, and only on:

1. any failed unit (critical) — regardless of how many succeeded alongside it;
2. an impossible reconciliation — the equation not balancing, or a negative count. The job lost track of its own work;
3. `eligible > 0 && succeeded == 0` with no legitimate expected skips, reported **with** the blocker reason so the alert says **why**. A run that can name no blocker says so explicitly.

A mixed run with legitimate `expected_skip` units plus `unavailable/deferred` units is `partial`, not `blocked`: the no-action decisions were completed correctly while the `unavailable` tail still raises a warning. This distinction prevents a full-book PaperTrader run (for example 13 legitimate skips plus 3 stale quotes) from claiming the whole trader was blocked without hiding the actual quote-coverage problem.

`eligible == 0` is `no_work` and is healthy. ****Business metrics** (`trades_filled`, `positions_closed`, `labels_written`) are telemetry and are NEVER health criteria** — a day with zero fills is normal; a day where 12 holdings were eligible and none were evaluated is not.

Layer 2 — cross-run freshness contracts (`lib/monitoring/freshness-contracts.ts`).

Run accounting cannot see a job that reports no_work every day for 25 days: each run is individually healthy and the pipeline is collectively dead. A versioned in-code registry (FRESHNESS_REGISTRY_VERSION, plus a per-contract version so a loosened threshold is a reviewable diff) asserts that each ledger's high-watermark advances within its grace window: price_cache (US, per symbol), observation_labels, decision_observations (US + India).

Per-scope, never aggregate. A table-wide max(date) on price_cache read healthy at Aug 13 while **101 of 140 symbols sat frozen at Jul 22** — one refreshed symbol made the whole table look alive. Contracts with a natural scope are evaluated per scope value against a declared minCoverage. Grace defaults to 96h (weekend + a one-day exchange holiday), matching the off-hours EOD allowance in lib/data/quotes.ts.

Session-aware, and scoped to what is actually refreshed (2026-09-02). Two halves of this contract disagreed with the rest of the system.

The measure. price-cache-us-symbols judged a DAILY bar against a rolling 96-hour calendar window. A calendar grace cannot express "the newest session that should exist": measured Tuesday 12:00Z, the window reaches back to Friday 12:00Z and a Friday bar timestamps at 20:00Z — inside it — so a book stuck on Friday passed. sessionAware: true now evaluates daily contracts against expectedNewestSession, the same rule lib/data/quotes.ts uses to decide stale and the same one the prewarm adopted after the identical bug was found there. Timestamp contracts (label maturation) keep the rolling window, which is the right shape for them. This is STRICTER and will report staleness the lenient window hid — the 85% coverage that breached the contract was measured with the generous rule.

The scope. The contract required freshness for active_us_price_symbols — every US symbol scored in the last 7 days plus every open position — while the research cron prewarmed only THAT RUN'S batch plus the benchmark ETFs. A symbol scored five days ago and absent from today's batch was required fresh and refreshed by nothing. That is not a monitoring nit: it is still scored, still becomes entry-eligible, and is then refused at fill time by the quote gate. Measured 2026-09-01, quote_stale=7 blocked 7 of 10 eligible US candidates; the monitor reported 17/113 scopes past grace. lib/data/prewarm-scope.ts now builds the prewarm list from the SAME definition the contract demands, and PREWARM_RECENT_DECISION_DAYS is pinned to the contract's 7-day window by test, so the two cannot drift apart again.

Widening costs almost nothing: prewarmPriceCache resolves freshness for the whole set in one bulk query and fetches only what is genuinely stale, under the existing deadline. Ordering carries the budget policy — open positions, then today's candidates, then

benchmarks, then the recent-decision tail — so whatever the deadline cuts is the least costly to leave stale. The scope query fails SOFT to the old batch: a wider prewarm is an improvement, never a precondition for the research run.

An **empty read is UNKNOWN and alerts** — "no rows" and "the query is wrong" are indistinguishable, so it is never treated as proof of health. Likewise a job that writes no run-accounting envelope is UNKNOWN, never assumed healthy.

Both layers are read-only over the monitored tables; the only writes are `agent_alerts` rows via `reportIssue/resolveIssue`. No migration, no schema change. Both modules are importable by any agent route; adopting the envelope in a producing route (position-monitor, paper-trade, label-maturation, research/cron) is a separate change.

Launch blockers for L4

- shared execution kernel used by all live paths;
- correct account test (`<agentic-account-id>`) and allowlist verification;
- atomic autonomous budget RPC with true actor audit;
- scoring version lifecycle enforcement;
- fresh agentic-account state with no fallback NAV;
- broker preview echo and idempotency/reconcile path;
- partial-fill/order sync and append-only broker events;
- deterministic live protective SELL path;
- duplicate-cron, timeout, stale-data, DB-failure, and kill-switch chaos tests;
- ■ PA1 shadow evidence — `AutonomousShadow` running, execution kernel in `lib/trading/execution-kernel.ts`
- ■ PA2 Kelly sizing — `computeAutonomousSizing()` in `lib/trading/execution-kernel.ts`; budget dry-run in shadow path; no-fallback NAV enforced (see PA2 section below)
- ■ PA3 broker submit — `lib/trading/autonomous-live.ts`; direct Robinhood REST (`lib/brokers/robinhood/rest-client.ts`) + Kite REST; per-market mode (migration 141); requires `AUTONOMOUS_LIVE_ENABLED=true` in Vercel env

Until all pass, `AUTONOMOUS_LIVE_ENABLED` remains false and L4 is descriptive only.

PA1 shadow path (implemented, deployment flag inactive)

`lib/trading/execution-kernel.ts` → `evaluateAutonomousExecution()` is the single pure gate evaluator shared by the shadow path (PA1) and the future live path (PA2+). It takes a `KernelInput + LiveAutoPolicy` snapshot and returns `KernelResult` — no DB calls, no side effects, deterministic.

Gates evaluated in order:

#	Gate	Fail label
1	AUTONOMOUS_LIVE_ENABLED deployment flag	deployment_flag_inactive
2	live_auto_enabled DB toggle	db_toggle_off
3	Lease not expired	lease_expired
4	Direction = long	non_long_direction
5	Score $\geq$ threshold	score_below_threshold
6	evidence_confidence $\geq$ floor (≥ 0.6)	confidence_below_floor
7	Open positions < cap	max_positions_reached
8	Orders today < cap	max_daily_orders_reached
9	Notional $\leq$ per-order cap (skipped when 0)	per_order_cap_exceeded

`runAutonomousShadow()` in `lib/trading/autonomous-shadow.ts` calls the kernel for each qualifying signal, creates a `trade_proposals` row with `execution_mode='autonomous_shadow'`, and updates status to `queued_auto` (kernel approved) or `manual_review_required` (gate fired).

No broker call, no budget reservation, no order submission in PA1.

PA2 shadow sizing (implemented, deployment flag inactive)

`computeAutonomousSizing()` in `lib/trading/execution-kernel.ts` computes position size for every `queued_auto` proposal. Rules:

Condition	Outcome
live_account_snapshots row missing OR NAV ≤ 0	<code>noSize('no_live_nav')</code> → downgrade to <code>manual_review_required</code>
NAV age > 4h (default)	<code>noSize('stale_nav_Nmin')</code> → downgrade
Quote unavailable or price ≤ 0	<code>noSize('no_current_price')</code> → downgrade
≥ 10 closed paper_trades with P&L	Kelly (half-Kelly from <code>win_rate</code> × <code>payoff_ratio</code> , capped at <code>min(10%, per-order-cap/NAV)</code> , floored at 2%)
< 10 closed trades	Flat <code>position_size_pct</code> from <code>strategy_config</code>
<code>floor(NAV × size_pct / price) < 1</code>	<code>noSize('qty_rounds_to_zero')</code> → downgrade

Per-order cap clamp: `size_pct = min(size_pct, live_auto_max_per_order_usd / NAV)`.

`queued_auto` proposals that survive sizing get `qty`, `estimated_value`, `pct_of_nav`, and `price_at_proposal` populated. Budget dry-run (informational only; not the atomic reservation): reads today's `broker_orders` spend vs `live_auto_daily_cap_usd` and includes the result in `ShadowRunResult.budget_dry_run`. The atomic `reserve_live_order_budget_v2` RPC is NOT called in the shadow path — it is called in the live-submit PA3 path.

PA3 live execution (implemented, requires AUTONOMOUS_LIVE_ENABLED=true)

lib/trading/autonomous-live.ts → runAutonomousLive() is the live execution path.

Triggered by POST /api/agents/autonomous-live/cron at 14:00 UTC weekdays (after research at 13:00 UTC).

Per-market mode (strategy_config, migration 141):

live_auto_mode_[market]	Behavior
off	Cron skips market entirely
manual	TraderAgent creates proposals; owner clicks Approve (existing path)
autonomous	AutonomousLive cron submits live orders

Additional gates (before kernel):

- app_paused=false + security_locked=false + trading_enabled=true
- live_auto_mode_[market]='autonomous' for signal's market
- **Per-market view-only kill switch:** a market is dropped from autonomousMarkets when trading_enabled_[market]=false — the same per-market switch the manual gateways honor also blocks the autonomous path. Flipping a market to view-only stops auto orders for it even if its mode column is still autonomous.
- **Daily-cap fail-closed:** per signal, an effective daily notional ceiling is required. US prefers live_auto_daily_cap_usd (the owner's Live-Auto \$/day guardrail), falling back to max_daily_notional_usd; India uses max_daily_notional_inr (the USD cap is not FX-converted on this path). If the effective ceiling is NULL the signal is blocked (gate_blocked, no_daily_cap_configured) rather than placed uncapped — autonomy must be bounded. The chosen ceiling is passed as p_max_daily_notional to the budget RPC.

Broker execution:

- US: rhPlaceMarketOrder() in lib/brokers/robinhood/rest-client.ts — direct Robinhood REST API using OAuth token from vault (ROBINHOOD_MCP_ACCESS_TOKEN). MCP tools unavailable in serverless.
- India: placeEquityOrder() in lib/kite.ts — existing Kite Connect REST path.

Budget reservation: reserve_live_order_budget_v2 with p_execution_actor='autonomous_worker' → broker_orders.approved_by_user=false.

2026-07-10 audit hardening (Codex full-system audit — lib/trading/autonomous-live.ts):

- The signal query previously selected a nonexistent agent_signals.evidence_confidence column and swallowed the error → the path silently processed **zero** signals every run. Fixed (real confidence column) and query errors are now **fatal** (throw + agent_runs error row).
- **Fresh** checkKillSwitches(svc, { market, book:"live", accountId }) runs per market before evaluation — the real drawdown/daily-loss/accuracy engine against live snapshots, not just cached config booleans. Fail-closed on error; a trip blocks BUY but leaves a verified SELL (sellAllowed).
- **Live market-open guard** (lib/trading/market-calendar.ts): layered — cheap local session/holiday/hours check, then **authoritative Alpha Vantage** MARKET_STATUS for BOTH US and India

(one call, catches **unscheduled** closures, needs no yearly calendar update). Fail-closed on a confirmed CLOSED; when the status source is unreachable, falls back to the session guard + the broker-rejection and quote-freshness backstops. Static US/NSE 2026 holiday lists remain the defense/fallback layer. Autonomous cron split per market (US 15:00 UTC, India 06:00 UTC).

- **Fail-closed gates:** a null lease and a null per-market trading_enabled_* no longer pass; both must be explicitly valid/true.
- **Per-market currency-correct NAV:** US from the Robinhood USD snapshot, India from live Kite INR margins+holdings; a market with no fresh NAV source fails closed (no cross-currency sizing).
- **Daily-cap fail-closed** (live_auto_daily_cap_usd enforced) + **net** per-market open-position count.
- **Idempotent claim:** unique partial index trade_proposals(signal_id, market) WHERE autonomous_live (migration 145) — concurrent/repeated runs can't double-propose+buy the same signal.
- **Money path UNIFIED (R13, 2026-07-10):** both the manual owner gateway (app/api/broker/orders) and the autonomous worker now call one shared server-only service, lib/trading/execute-order.ts::executeApprovedOrder(svc, input, actor). It runs the full invariant set once — autonomy-level, per-market trading flags, fresh checkKillSwitches, G1 decision-quality, account allowlist, fresh-quote notional cap, G3 portfolio limits, price drift, held-SELL, and the atomic reserve_live_order_budget_v2 reservation. The actor envelope distinguishes owner (may supply audited risk overrides) from autonomous_worker (may NOT override any gate; supplies its own live_auto_daily_cap_usd / orders-per-day caps). Autonomy authorization is the upstream deployment flag + DB toggle + lease + kernel + session gates, not an owner click.
- Serverless broker submit RESOLVED (2026-07-10): added a direct-REST Robinhood execution adapter (lib/brokers/adapters/robinhood.ts, registry id robinhood) with submit/status/cancel over REST — works in Vercel serverless, unlike the MCP adapter. Set strategy_config.active_broker_us='robinhood' to route live US orders through it (both the manual gateway and the autonomous worker use it via the shared service). The account is allowlist-validated at the gateway and again in the adapter; the Robinhood live kill switch (robinhood_mcp_enabled) gates both the robinhood and robinhood_mcp ids.
- Remaining before first live dollar: R16 live position-monitor + protective-exit/cancel/reconcile control plane, the J acceptance-test fixtures, shadow soak, and a capped canary. Deployment flag stays false until then.
- Schema reproducibility restored: migrations 143 (live-auto DDL + budget RPC), 144 (RLS), 145.

Outcomes per signal:

- submitted → broker_orders.status=submitted, broker_order_events appended, proposal queued_auto
- needs_reconcile → broker_orders.status=unknown_needs_reconcile, proposal manual_review_required
- broker_error → order not submitted, proposal manual_review_required
- budget_error → RPC threw (cap exceeded), no broker_orders row, proposal manual_review_required
- gate_blocked / sizing_failed → no reservation, no submit

Advisory-only surfaces (read the book, never move money)

Some analytics read the live account book but are structurally severed from the order path. They must never be treated as an order signal, and they never call the Execution Gateway.

Daily Per-Holding Risk Analytics

features/holding-risk-daily — daily /api/agents/holding-risk?market=us|india (pg_cron migration 156, US 21:30 UTC / India 11:00 UTC). Scores **every holding in every live account** — Robinhood Trading

<agentic-account-id>, Robinhood **read-only** <read-only-account-id>, and Kite India — with a deterministic 0–100 risk-control pressure index and a risk posture. Safety properties:

- **Hybrid, deterministic-first.** lib/risk/holding-risk.ts (hr-v3) computes the score **and** the posture (hold / review / trim / exit_review / insufficient_data) with strict precedence: a verified protective-stop or thesis-break → exit_review; **unrealized drawdown ALONE never** triggers exit_review (loss-chasing guard); hard **name**, allocated genuine equity-sector, and correlated-cluster concentration all produce review, never trim, because a global trading reference is not an account-specific sell mandate. Having an order path does not establish that mandate. A future trim posture requires an approved per-account objective/cap and executable share-quantity plan. A correlated cluster produces review until an

allocator identifies which holding and quantity to reduce. A sector breach with ****no usable allocation**** → review; data

confidence < 0.5 → review. An LLM writes **only** the human-readable strategy_note — it ****cannot** change the score, posture, action, or allocation**. lib/risk/strategy-notes.ts makes that provable rather than prompt-dependent: parseStrategyNotes() returns Map<requestedSymbol, string> and drops non-strings and unrequested keys, so model output has no path to any other field. This mirrors the LLM boundary elsewhere: models may explain, never control a numeric limit, a posture, or an order.

- **A sector breach is allocated to names, never applied as a blanket.** A sector-cap breach is a property of the SECTOR — it carries no per-name allocation, so using it directly as a per-name verdict gives every holding in the sector the identical trim with no size (the shipped AVGO defect).

lib/risk/sector-breach.ts (sba-v1, pure, deterministic, ****risk-internal** — no research score, no

LLM) **decides which names absorb the breach and by how much:** water-fill** the sector's largest names down to a common level L where $\sum \min(w_i, L) = \text{cap}$, on the **NAV** basis ($S = c \cdot \text{NAV}$), which

is how lib/risk/live-portfolio-gate.ts enforces the owner's cap on the live money path. Ties need no arbitrary winner (equal weights → equal trims by construction); output order breaks on symbol

ascending. Names below L are **not selected**: they get hold ****plus** the reason they were not

selected — **never a bare verdict (CLAUDE.md "Detail Over Cryptic")**. The allocator is consulted

strictly **BELOW** the exit branch: it can never delay or suppress a risk-driven exit** (the same

invariant lib/evidence/degradation-guard.ts holds for short). Unknown-sector holdings are excluded from every sector total and reported as unknown — never bucketed into a synthetic sector, never assumed cap-compliant. Broad asset-class sleeves (Diversified/International Equity, Fixed Income, Commodities, Digital Assets) are also excluded: they are not equity sectors, so applying the sector cap to IVV/SGOV/GLD/IBIT is a category error. Design + justification:

features/risk-sector-breach-allocation/FEATURE_ARCHITECTURE.md.

- **Wired to NO order path — for ALL accounts.** The strategy_note, posture, and add_capacity flag are advisory. add_capacity means "risk limits have room," **not** a buy signal. Nothing here reaches executeApprovedOrder, the gateway, or a broker. The UI labels the note "advisory" and, for read-only accounts, "advisory only - no order path." Since hr-v3, every account receives concentration review, not trim; allowlisting account <agentic-account-id> for order transport does not create an account-specific objective/cap mandate. The deterministic reason states that no trim is recommended. The readOnlyAccount flag controls advisory transport wording only, not concentration posture. An absent flag defaults to the read-only wording.

- **Legacy snapshots are evidence, not current instructions.** lib/risk/holding-risk-history.ts is the shared read-boundary normalizer for Risk Analytics and newsletters. It preserves the original sourcePosture but renders hr-v1/hr-v2 trim rows as review with the historical reason. The append-only rows are never rewritten, and exit_review is never downgraded.

- **Research is SHOWN next to the verdict, and coupled to nothing** (2026-07-17, features/risk-research-visibility). GET /api/portfolio/risk-daily now attaches a nullable research block per holding — score, direction, scored_at, sessions_since, days_since, state, scored_as_holding — joined from the latest agent_signals row per (symbol, market), never symbol alone (US and India books cannot cross-join). **Invariant R1: this is a DISPLAY JOIN.** No field of it is read by computeHoldingRisk, sba-v1, constructPortfolio, the execution kernel, or any gate; the score/posture/action on the same row were computed research-free and are replayed verbatim from the snapshot. That is deliberate, not an oversight: ***a sector is over-cap *because* research liked that sector, so letting analyst_score also veto the cap double-counts the same signal*** — the risk layer exists to be the one thing not persuaded by the score, and the owner integrates the two views. (This is why the competing features/risk-research-integration proposal — research ***ordering*** which names absorb a trim — was **not pursued**.) tests/risk-research-annotation.test.ts pins R1 both behaviorally (risk bytes identical with the block present vs absent, and across every score/direction/staleness variant) and architecturally (the lib/risk/* modules are checked, comments stripped, for any ***code*** reference to research — the guard is itself falsification-tested, since sector-breach.ts mentions analyst_score in **prose** asserting the invariant and a naive grep would flag it).

- **The AGE is the feature, not the score.** On 2026-07-16 AVGO sat **6 days unscored** while this panel told the owner to trim it, and nothing on screen said so. A 6-day-old 91 and a fresh 91 are not the same claim.

- **Staleness is measured in market-local SESSIONS, displayed in DAYS.** Reuses

marketSessionsSince (lib/trading/paper-exit-policy.ts, built on the market-local holiday calendar in lib/trading/market-calendar.ts) — **not** a calendar-day count, which would paint the whole book stale every Monday (a Friday score read Monday is 3 days but **one** session) and train the owner to ignore the warning. US and India holidays differ and are handled per market.

- **Four states, not collapsed:** fresh ($\leq$ STALE_AFTER_SESSIONS = 2, owner-set) · stale

(warning + "not scored in N days") · never (no signal ever — **not a link**, because a link to nothing is a lie) · unavailable (research abstained on thin evidence — **never** rendered as a number). stale annotates the **score only** — it never dims or alters the risk action or verdict.

- **A score's meaning depends on is_holding.** is_holding is false in **463/463** prod rows: no holding-path score has ever been written, so every annotated row is labelled a screener **candidate** score, not a holding verdict. This matters because the direction gate can only emit short (a deterministic exit) when isHeld is true — a neutral from the screener path does **not** mean "no exit signal"; it means the exit question was never asked.

- **Fail-soft.** If the agent_signals read errors, the risk table **still renders** and the column says "research unavailable" explicitly — never a silent blank, never a fake score. Risk is the product; research is the annotation. A failed read (research: null) is kept distinct from a successful read that found nothing (state: 'never'), so the UI never claims a symbol was never scored when it merely failed to look.

- Every account gets the annotation, including read-only <read-only-account-id> (informational there, as the strategy note already is). Additive only: **no schema change, no migration, no new cron, no provider call, no LLM.** Rows deep-link to /dashboard/research-journal?symbol=&market=.

- **Fails closed.** A missing/stale broker snapshot publishes a failed/insufficient-data run — never yesterday-as-today. Structural-gate failures (missing qty/price/market-value, non-finite inputs, stale quote, non-USD/INR currency) yield insufficient_data with a null score, not a fabricated one.

- **No cross-currency roll-up.** Each run/snapshot carries its own market + currency; USD and INR are never summed. Δ-vs-yesterday only compares runs of the **same** formula_version.

- **Append-only evidence.** holding_risk_runs (lifecycle-guarded: DELETE blocked, identity frozen, status forward-once out of running) + holding_risk_snapshots / account_risk_snapshots (UPDATE+DELETE blocked). Owner-email SELECT RLS; service-role writes; anon REVOKEd. See docs/arch/04-database-schema.md#812-daily-per-holding-risk-analytics-advisory-append-only.

Proactive broker-token health check

checkRobinhoodTokenHealth(svc) in lib/robinhood-mcp.ts is a proactive token-age check. Robinhood access tokens are **short-lived (~days)**, so a naive expiry read would false-alarm on every routine rollover. Instead it mirrors getValidAccessToken: when the access token is past (or within 60s of) expiry **and** a refresh token exists, it **attempts the CAS refresh** (keeping the token warm on the 6h cadence) and reports the critical broker-token:robinhood issue **only** when there is no refresh token or the refresh actually fails — a genuine reconnect-required state, not a short-TTL rollover. On a valid

or successfully-refreshed token it resolves the issue. It runs in two places:

- GET /api/robinhood-mcp/status (Settings → Robinhood card) — so the connection badge cannot lie. A genuinely-dead token renders "● **Reconnect required — token expired**" (red); a healthy token shows "● **Connected — valid until <expiry>**" (green), where <expiry> is the **access-token** TTL (renewed automatically, not a connection deadline). The route returns stale / expires_at / has_refresh.
- POST /api/agents/health-triage (kairos-health-triage, 0 */6 * * *) — runs the check *before* reading agent_alerts, so every 6h it both keeps the token refreshed and surfaces a genuine dead-refresh state even when no order/snapshot path ran to trigger the lazy check in getValidAccessToken.

Downside hedge boundary

The downside hedge is a US paper-book overlay, not a new alpha strategy or live authority.

Ordinary agents block SH, PSQ, DOG, and RWM. Only execute_paper_hedge_fill admits SH/PSQ, after dedicated flags, fresh audited evaluation, state, one-position, cash, and NAV checks. Hedge trades are excluded from learning, cash-funded, and bounded by stop, five-session hold, hysteresis, and cooldown. There is no true short, option, leveraged inverse ETF, India, LLM decision, broker call, or live control.

Why it exists: a dead RH refresh grant makes fetchRobinhoodBrokerAccounts() return an "unknown" account id, which silently drops **all** Robinhood accounts out of holding-risk and freezes live_account_snapshots. The lazy getValidAccessToken reporter only fired when something tried to use the token; this proactive 6h check + honest badge close that gap. The only human-required action (owner reconnects OAuth via the localhost loopback) is triggered solely on a **failed refresh** — the check is advisory + performs the same CAS refresh the order path uses, but reaches no credential-write beyond token renewal and no order path.

Kite GTT and Explicit SELL Safety (2026-07-17)

Kite GTT child orders are LIMIT, including the stop-side child. A trigger is not a guaranteed fill: a gap through the limit can leave protection unfilled. Kairos must report that semantics honestly and may not label the child SL-M.

For the standalone human-confirmed Kite route, an explicit SELL now:

1. reads every recorded non-null GTT id for the India symbol;
2. cancels each at Kite and requires positive confirmation;
3. clears each id in broker_orders;
4. only then submits the explicit SELL.

When a resting GTT exists, the standalone route permits only a full-quantity exit. A partial SELL would cancel protection for the unsold remainder and is

therefore rejected until a reviewed residual-protection workflow exists.

Any read, cancel, or ledger-clear uncertainty fails closed before SELL. If the SELL fails after confirmed cancellation, a critical issue states that the still-held position may now lack broker-side protection. GTT placement/persistence failure after a confirmed BUY is also critical. The broader hybrid protective-stop feature remains a draft and is not enabled by this correction.

Webull Cloud MCP remains query-only and advertises no order scopes or tools. A future Webull Trading API adapter is a separate, signed, sandbox-first money-path feature and is currently unapproved and disabled.

Hybrid protective-stop control plane (2026-07-30)

The earlier shadow scaffold is now reconciled with production schema truth: `protective_orders`, `protective_order_events`, and the false-by-default `protective_orders_enabled` flag are applied. Direct production verification on 2026-07-30 found the flag false and zero protection rows. P1 adds read-only full-quantity coverage evaluation plus an autonomous-live entry interlock. A configuration flag alone cannot permit a BUY: the source-level placement worker is false until a separately reviewed broker-specific lifecycle exists. P1 neither places nor modifies a broker order, and it does not change manual live, paper, or the synthetic live-exit monitor. The older prose below is historical scaffolding; references to the migration as unapplied are superseded.

Coverage requires exact equality between reconstructed held quantity, reconciled held quantity, and broker-protected quantity. Under-protection leaves risk uncovered; over-protection can create an accidental short. Both fail closed. The aggregate coverage verdict must be explicitly true; false/unknown state cannot pass merely because a failing reader returned no per-position findings. P2 must also define an atomic or bounded-compensation workflow for protecting a newly filled BUY; changing the source availability constant is not sufficient.

Built UP TO the placement line and STOPPED. **No live broker order is ever placed by this scaffold — not a \$1 test.** It is a set of pure, deterministic modules under `lib/protective/` plus an UNAPPLIED migration proposal. Nothing here is wired into an order path; the whole thing is inert until the owner approves touch semantics (Q1), the floor distance, and the post-fill protection policy.

Design honesty — protection is MITIGATION, not a guarantee. A stop-market is designed to TRIGGER a market order but neither execution nor price is guaranteed

in a gap/halt/outage. A stop-limit / GTT limit child controls price but MAY NEVER FILL on a gap — the position stays unprotected and that is REPORTED, not called filled. Stops generally cannot fire while the session is closed. The honest label is outage + catastrophic-loss mitigation.

Module	What it is	Money-path posture
capabilities.ts	Broker-neutral BrokerProtectiveCapabilities matrix + evaluateProtection() — capability-driven by order-type/TIF/session/account, NOT a flat broker boolean. GTC ≠ triggers-in-every-session (session eligibility is separate). No eligible multi-day order → unprotected-by-broker, never silent.	pure, no broker calls
kite-capabilities.ts	Kite's declared capability from the PROVEN GTT code: protects via the WEAKER gtt_limit (LIMIT child); the DAY-only regular SL-M is declared and REJECTED as a multi-day floor.	declaration only
disaster-floor.ts	Pure computeDisasterFloor({mode, distance, ...}). Default mode = wider_disaster_floor (Q1 unanswered). Distance is a config input — no hardcoded value. Monotonic ratchet: a falling high-water mark can never lower the floor; a hard max-loss bound can only raise it.	pure, not wired to placement
reconcile.ts	Pure reconcileProtectiveOrder() — detects out-of-band triggers, partial fills, cancels, expiry, broker edits, and corporate-action qty drift. Unknown state → needs_reconcile; trigger-without-fill never closes the book; limit-child gap → unprotected; expiry → critical; residual protection clamped to held qty.	pure, no broker calls
state.ts	The protective_order record shape, status machine (placing→...→needs_reconcile), exit provenance, and long-only / cancel-before-replace invariants.	pure helpers
placement-gate.ts	THE MONEY LINE. False-by-default protective_orders_enabled flag gates ALL placement and stays false. planProtectivePlacement() returns the intended action as a plain object and NEVER calls a broker.	inert — no placement

Exit provenance (Codex's correction). A disaster-floor fill closes the position with exit_reason = protective_disaster_floor and learning_scope = risk_policy_only. The loss is REAL money and STAYS in P&L, NAV, drawdown, mandate evaluation, and risk-policy evaluation. ONLY the Learner's signal-weight attribution and genome promotion exclude it (state.ts includedInSignalWeightLearning() returns false; includedInPnlAndRisk() returns true). A generic excluded_from_learning = true is insufficient because the evaluation engine also filters that field — which would wrongly drop the loss from risk evaluation too. So learning_scope is explicit.

Long-only / cancel-before-replace. Total executable SELL (resting protective SELL + any competing SELL) can never exceed the reconciled held quantity. canSubmitCompetingSell() FAILS CLOSED: a competing SELL is refused while a

resting protective order's cancellation is unconfirmed (else an accidental short), and even after a confirmed cancel a request above held qty is refused. This is the same invariant the shipped standalone Kite route already enforces (cancel-before-sell); a future shared implementation reuses the protocol.

Migration is a PROPOSAL, not applied.

supabase/migrations/20260718000000_protective_orders_shadow.sql creates protective_orders (state table, one active per position+broker via a partial unique index, a protected_qty <= reconciled_held_qty CHECK) and append-only protective_order_events, and adds strategy_config.protective_orders_enabled (default false) plus learning_scope on paper_trades / broker_orders. It is NOT applied to prod — see docs/arch/04-database-schema.md is intentionally not updated until the migration actually runs.

Acceptance tests. tests/protective-hybrid-stop.test.ts implements all 14 spec acceptance tests, each falsifiable (mutation-verified: breaking the ratchet fails AT3; breaking the cancel-guard fails AT1). Prod verified before building: no protective_orders table exists; broker_orders already has kite_gtt_id; no learning_scope column exists; paper_trades exit reasons in use are direction_flip / score_exit (no protective_disaster_floor).

What remains for the owner before anything goes live: approve Q1 (broker-hosted touch execution at all), the disaster-floor distance (recommendation: mandate-ATR with a hard max-loss bound), the minimum ratchet step/cadence, and the build sequence (Kite-first vs Webull-first). Then apply the migration, wire the plan to a reviewed live path, flip protective_orders_enabled to true, and run a single owner-approved small live test per market.

Order-maintenance and ETF-cap correction (2026-07-26): A stale US-order cancel ACK is not a cancellation record. It becomes unknown_needs_reconcile; only a later read-only broker observation may set terminal state. The maintenance cron skips closed US sessions, bounds outbound cancellation and reconciliation to one item each, and the Trading dashboard offers an owner-triggered read-only refresh. A confirmed fill writes filled_qty, avg_fill_price, terminal time, and its proposal result without overwriting requested quantity. ETF allocation now fails closed on unavailable configuration, stale or unpriceable current marks, stale live snapshots, invalid NAV, or missing order price; it uses current marks, not cost basis. India remains outside the US ETF sleeve and SELLS bypass it.

Earnings Event Risk (P0 Shadow)

Earnings proximity and the expiry-bounded ATM straddle proxy are risk annotations, never directional alpha. Exact-expiry selection requires a common call/put strike, valid non-crossed mids, liquidity, bounded spreads, and fresh timestamps. Unknown, conflict, stale, illiquid, or incomplete pagination remains unavailable and cannot be interpreted as no event.

P0 is mechanically behavior-inert: the database permits only shadow rows and requires `behavior_changed=false`; paper/live hooks only log normalized context; the direct live Alpha Vantage blackout remains in force; PositionMonitor and live execution import no earnings-risk decision function. The Risk page exposes warnings and evidence counts without raw broker chains.

The scheduled US holdings monitor is also fail-loud on canonical state reads: errors reading holding-risk runs/snapshots, paper positions, or the PIT earnings calendar mark the agent run error instead of reporting a successful zero-coverage sample. A paper stop at or above spot is stale/malformed and is recorded with no stop-distance comparison; it is never converted with an absolute value into a plausible risk annotation.

Post-Report Daily-Price Repricing Barrier (2026-07-30)

Options risk context remains measure-only, but daily technical direction cannot be trusted immediately after a reported result. When the point-in-time earnings calendar has a recent event that has occurred but the daily candle series does not yet contain its reaction, ResearchAgent writes a **current**, session-validated **neutral** signal. Before-open events may use the completed report-date bar; after-close/unknown events require a later bar. A late actual feed does not reopen stale scoring for a known past event. This is a subtractive data-freshness guard: it blocks new paper/live entries and score/direction exits from that stale daily score, while mechanical stop, target, trailing-stop, and time exits continue. The existing calendar is read only; no provider or options call is added, and no earnings beat/miss is inferred as directional alpha. A calendar read error is an unknown control-plane state and writes a current neutral abstention rather than authorizing stale direction. The latest prior event remains checked until a qualifying post-event bar exists; it does not silently expire after seven calendar days. All earnings collectors share one real ISO-date validator, so impossible provider dates cannot normalize into another session. Normal scoring resumes on the first post-report daily bar.

2026-08-08: **Property address and carrying-cost boundary.** Exact addresses,

loan terms, tax, insurance, and maintenance values remain owner-only encrypted. Owner-provided tax notices and insurance quotes use the same AES-256-GCM vault through `/api/property/owner-evidence`; the UI retains metadata only and does not perform OCR, extraction, automated quote comparison, or automatic cost mutation.

data and never enter an LLM or investing flow. API inputs are size/range bounded. US geocoding fails honestly to `no_match`, `ambiguous`, or `unavailable`; it never invents a parcel. Monthly cost uses the shared

deterministic mortgage engine, while tax/insurance/maintenance stay labelled owner inputs until an official bill or quote is connected. USD and INR remain independent.

Capital Plan boundary (2026-08-08)

Capital Plan is owner-only decision support above the Investing and Property workspaces. It has no broker, lender, payment, transfer, property-purchase, refinance, HELOC-draw, or order route. It reads no paper cash and does not silently read brokerage credentials. Its deterministic mortgage comparison is review-only: it rejects a partial principal payment that would breach the owner-entered reserve plus near-term obligations and never assumes a recast, deduction, or lender permission.

Area snapshots consume already-persisted Property observations rather than calling providers. ZIP/PIN/locality and Bengaluru-local coverage remains `contract_pending`; the system records that absence rather than emitting an ROI or valuation. A future LLM narrator receives only a sealed typed envelope and cannot modify a number, rank, policy, cash transfer, payment, or trade. Capital learning remains shadow-only until timestamped owner outcomes mature.

Time-review exit shadow (2026-09-03)

The unconditional paper time stop remains active. Before that branch runs, `PositionMonitor` now writes an idempotent observation only when an alpha position is at its exact resolved horizon. The versioned `time-review-v1` classifier can qualify only a profitable position with a fresh long score at or above the hold threshold and drawdown no larger than its initial stop distance. It predeclares only +5 and +10 market-session candidates and retains the position's effective mechanical stop. Missing evidence fails closed.

The observer writes only `time_review_exit_observations`; the daily horizon shadow job matures `time_review_exit_outcomes`. Both ledgers are append-only and owner-readable. No position, trade, cash, stop, target, proposal, broker order, score, mandate, or exit reads either ledger. The older daily `horizon_extension_shadow` rows remain frozen historical context and are excluded from readiness.

Source: docs\arch\09-learning-loop.md

Kairos — Learning Loop

2026-09-08: **Score-return IC drift detector shipped (Stage A, detection only).**

Owner asked how to know which shipped feature made the score/return correlation better or worse. `lib/learning/code-version-ic.ts` groups `decision_observations` x `observation_labels` by `code_version` and calls the EXISTING `buildDimensionFindings()` once per group — no second correlation implementation, so it cannot drift from what the Dimension Rank IC panel or the learner report on the same rows. `benchmark_neutral_return` falls back to `fwd_return` per row (production has real `fwd_return`-only rows: US h10 11/2633, India h5 64/934, h10 53/786, h20 91/515 — measured 2026-09-08).

This is DETECTION, not causal attribution — and never claims otherwise.

Every surface (alert text, UI copy) states a dimension's IC "changed AFTER" a `code_version` shipped, never that the version "caused" it: deploys ship weeks apart and the market regime moves between them, so a before/after split on `code_version` is confounded by construction. A real causal claim needs a paired replay (Stage B, not built here).

`lib/learning/ic-regression-alert.ts::detectRegressions` flags the latest `code_version` only when it has ≥ 5 prior `measured_descriptive` versions to compare against AND its mean IC sits more than 2 (warn) / 3 (critical) **historical standard deviations** of those prior versions below their mean — the threshold is derived from each dimension's own observed variance, never a picked absolute IC constant. Fires via the existing `lib/system-health.ts` `reportIssue/reconcileIssues` into `agent_alerts` (no migration needed).

`applyMultipleComparisonsControl` (Benjamini-Hochberg) runs only over cells that already cleared their own n/CI floor; up to ~50 `code_versions` x 6 dimensions is ~300 comparisons, and uncontrolled that would manufacture roughly 15 "significant" cells by chance alone — the same trap that parked "Systematic Pattern Discovery" (WORK_LOG.md 2026-09-05). ~33-37% of `decision_observations` carry no `code_version` (US 2355/6298, India

599/1788, measured 2026-09-08); those rows are bucketed under UNKNOWN_CODE_VERSION and never anchor a regression boundary.

app/api/agents/ic-regression-ledger/route.ts (GET read-only, POST cron/owner-triggered reconcile) and components/dashboard/IcRegressionPanel.tsx (mounted on /dashboard/learning, below Dimension Rank IC) are new. No new table: the ledger computes on read from existing evidence. The POST route was NOT wired into an actual external cron trigger — this repo's dispatch mechanism for dimension-diagnostics's identical contract was not located in vercel.json or .github/workflows/ during this change; see features/score-correlation-drift/FEATURE_ARCHITECTURE.md.

2026-09-01: walkForwardFolds **purged in CALENDAR days while claiming market-horizon purge — labels leaked.** lib/learning/dataset.ts computed purgeCutoffMs = testStart - horizonDays * 86400_000. The horizon is a MARKET-session count (h2/h5/h10/h20/h60/h120), so a nominal 10-day purge spanned only ~6-7 trading sessions and training rows whose label windows still reached into the test window survived it. Every walk-forward result computed with it was optimistically biased.

Now indexed by SESSION, using the observed trading calendar derived from the distinct decision dates in the data — holidays handled without a separate calendar source. Legacy testDays/horizonDays/embargoDays option names are kept as aliases because they always MEANT sessions; only the arithmetic was wrong. Callers updated to the explicit *Sessions names.

The old test could not have caught it: its fixture emitted one row per consecutive CALENDAR day, weekends included, so sessions and calendar days were identical by construction — the fixture encoded the same assumption as the bug. New tests use a weekday-only calendar plus a simulated holiday gap; restoring the calendar-day arithmetic fails 4 of them.

Blast radius: none recorded. validation_experiments and strategy_evaluations are both empty, and the 5 backtest_experiments rows come from lib/edges/oos-runner, which does not call this function. No stored result needs annotating.

2026-08-28/29: **Silent PostgREST truncation swept, and archetype arms now refuse instead of vanishing.** Commits 3aa3753f, ae17fad2.

.limit(n) above PostgREST's server maximum is IGNORED, not an error — the response is capped at 1,000 rows and returns success. Four call sites were actively truncating:

site	rows	actually read
lib/learning/dataset.ts	5,223	1,000 — the learner, oldest fifth of the US cohort
app/api/admin/readiness	5,223	1,000 — gates the autonomous tier
app/api/analytics/atr-exit-evidence	2,976	1,000
app/api/agents/exit-geometry-shadow	2,412	1,000

The dimension loader was the same defect, found first: US h5 had 2,445 rows across 30 dates and read 1,000 across 16, so **every US dimension IC recorded before 3aa3753f used ~40% of the evidence and half the calendar.** India sat under the cap, which is why a cross-market read never exposed it. Dimension plan version v4 → v5; the truncated v4 rows are kept as recorded and superseded.

lib/supabase/paginate.ts (fetchAllRows) is now the single reader: orders by a unique column, treats a short page as the last page, requests one more page when the total is an exact multiple, and refuses past 200 pages rather than return a partial result.

Four latent sites were converted too. `lib/chart-data.ts` is the documented exception — a cache-freshness probe where truncation fails safe by re-fetching.

`lib/learning/dataset.ts` also stopped swallowing read failures into an empty dataset. The old `if (obsErr || !rows?.length) return []` made a broken query indistinguishable from a quiet day.

Archetype arms no longer disappear. `loadRows` inner-joins matured labels, so an arm whose observations have not matured was dropped before grouping and never reached the `eligible-rows` check — in production that erased 6 of 8 US arms and all 3 India arms, reading in the ledger as "never ran". The route now enumerates every arm that has recorded a shadow score and emits an explicit refusal, distinguishing `no_matured_labels` ("Not a negative result") from `no_eligible_rows`. `archetype_ic_runs.cohort` records the population (migration 20260828163000, applied and verified).

Post-fix state, as_of_date 2026-08-29, all rows `cohort = eligible_long`:

market	arms	rows	refusals (no matured labels)	rows with an IC
us	8/8	16	12	2
india	3/3	6	6	0

The only graded arm is `etf_trend`: h10 +0.024 vs champion +0.056, h20 +0.048 vs champion +0.109, on 8-10 sessions against a 20-session floor. `insufficient_evidence`, not a result. **18 of 22 rows are refusals for want of matured labels** — that is the honest state of the learning loop, and it is strictly better than the silence it replaced.

2026-08-28: **Every predictive headline now measures the eligible-long cohort.** `lib/learning/entry-cohort.ts` is the single definition (`entry_eligible = true AND direction = 'long'`), shared by the Alpha Diagnostic Lab's A2, the dimension diagnostics, and the archetype grader so the three cannot drift apart.

The bug: dimension IC (`lib/learning/dimension-diagnostics.ts`) and archetype IC (`app/api/agents/archetype-ic/route.ts`) both computed rank IC over ALL scored observations, including `neutral` and `short` rows the book could never buy. `entryEligible` was loaded but only fed a mean-return summary field, never the IC itself; the archetype select carried no cohort filter at all. Production: 4,075 of 6,592 observations are eligible-long, so roughly 38% of every IC ever reported came from names that were not purchasable.

h10 rank IC	all-scored	eligible-long
us	-0.0101	-0.0768 (21 dates)
india	+0.1046	-0.0083 (17 dates)

Both are `insufficient_evidence` against the 60-date floor. **The India "+0.105 selection edge" cited throughout this chapter was the all-scored number.** Two published diagnoses were retracted for it on the same day (`docs/audits/2026-08-28-sizing-damage-diagnosis.md`, both `CORRECTION` sections).

What changed, all measure-only — no score, weight, threshold, sizing, stop, target, eligibility or trading behaviour was touched:

- Dimension predictive and agent contribution findings compute the headline on the eligible cohort and nest the all-scored figures under `all_scored_context` with an explicit "never cite this" interpretation string. No migration: `finding_type` stays inside its existing `CHECK` constraint.
- Plan version `dimension_diagnostics_p0_v3` → **v4**. The metric changed meaning, so v3 rows are not reinterpreted or compared against v4 ones (frozen-history rule).
- `diagnosticFingerprint` now includes the cohort flag, so an eligibility-rule correction cannot reuse a recorded run.
- The archetype grader filters before grading. An arm with no eligible rows (`etf_trend` scores only ETFs) records `emptyArchetypeResult` — an explicit refusal — rather than vanishing from the ledger and reading as "not run yet". `archetype_ic_runs` has no cohort column, so it stores the eligible-long grade only and says so in `reason`.
- Four mutation-verified detectors: replacing the predicate with `return true` fails all four.

Still unverified: the per-dimension figures that motivated the archetype instrument (us `fundamental` +0.076 `t`=2.40, india `technical` +0.173 `t`=2.51) are all-scored and have NOT been re-derived. The header comment in `lib/learning/archetype-ic.ts` now says so. `lib/edges/*` also computes IC and was not audited here.

2026-08-28: **Alpha Diagnostic Lab P0 shipped** (`features/alpha-diagnostic-lab/`). Read-only funnel diagnosis per market, weekly. A0 data truth gates everything; the strongest verdict is `owner_review` and no money path reads it. Full record incl. the seven defects found by running it: `features/alpha-diagnostic-lab/IMPLEMENTATION_RESULT.md`.

First production run, both markets A0 `pass`, verdict `collect_more` (nothing clears the 60-date review floor):

	us	india
A2 rank IC h10	-0.012 (t -0.22, 17 dates)	+0.105 (t 2.24, 22 dates)
A2 mean quintile spread	-0.006	+0.010
A3 percent profit factor	0.969	1.438
A3 currency profit factor	0.735	0.906
A3 sizing damage	no	YES

SUPERSEDED 2026-08-28 (see the entry at the top of this chapter). The A2 figures in the table above are the ALL-SCORED cohort. On the eligible-long cohort India is -0.0083 and the US -0.0768, so "India picks winners and the sizing destroys them" is not supported — there is no demonstrated selection edge in either entry cohort. The original claim, left here as the record: India picks winners and the sizing destroys them; the US selection does not rank and its quintile spread is negative. The "independent reproduction" of +0.105 against the +0.106 measured on 2026-08-25 was two runs of the same cohort error, not a confirmation.

2026-08-25: **The weighting arms are now graded, and a `fundamental_only` arm was added.**

Seven archetype weight sets record a score for every observation in `shadow_decisions`, and until today nothing evaluated them: the only consumer computed the share of shadow rows that were bullish, and its own comment admits that is not a comparison to the champion. `/api/agents/archetype-ic` (weekly, per market) now grades each `setup_type` by Spearman rank IC against `benchmark_neutral_return`, next to the champion composite measured on the SAME observations - `etf_trend` only ever scores ETFs, so a whole-market champion baseline would compare different universes.

Guards carried over from the dimension-diagnostics work, each mutation-verified: dedupe to one row per (market, symbol, date, `setup_type`) because the research cron writes 2-3x daily; rank IC rather than Pearson; and the overlap-corrected floor `n / horizonDays >= 12` on top of the 20-date floor.

Why the `fundamental_only` arm exists. Measured h10 rank IC, deduped, single stocks:

market	best single dimension	champion composite
us	fundamental +0.076 (t=2.40)	+0.051 (t=0.93)
india	technical +0.173 (t=2.51)	+0.106 (t=2.04)

Both composites rank worse than their own best dimension. `value_inflection` (fundamental 0.45) only half-tests that; the new arm isolates it. It runs in BOTH markets deliberately - India's edge is technical, so the arm is expected to score poorly there, and an arm that only ever runs where it is expected to win proves nothing.

Caller contract, and it is load-bearing: the arm is skipped unless `included.fundamental === true`. `computeWeightedAnalystScore` equal-splits across included dimensions when every included dimension has weight zero, so scoring a `{fundamental: 1.0}` set on a symbol with no fundamental evidence would silently yield an equal-weight technical/sentiment/macro blend - an arm labelled `fundamental_only` containing no fundamental at all.

Timeline. Four of the seven arms only began writing on 2026-08-25 (the multi-expert uniqueness index landed 08-24; before it, only the first archetype in each routing array survived, which is why just `etf_trend` and `quality_momentum` existed). Their h10 labels mature ~2026-09-08; ≥ 20 decision dates lands ~late October. Every read before then returns `insufficient_evidence`, by design.

2026-08-18: **Promotion gate segments IC evidence by PROVIDER REGIME.** Same-day companion to the Yahoo-first candle move below. `edge_ic_history` now contains rows computed on two different data sources, and `app/api/agents/backtest/promote` reads a 1000-day window — so without segmentation the cross-window stability check would compare a Yahoo latest window against a Massive earliest window and report the difference as "stability". `providerRegimeKey()` derives a regime from each row's `provider_report.providerCounts` using the DOMINANT provider (not the exact count map: `{eodhd:20, massive:6}` and `{eodhd:19, massive:7}` are one regime, and keying on counts would over-segment on run jitter and starve the gate). `evaluateGate` then evaluates ONLY the trailing run of windows sharing the latest regime and drops older ones — never blends them.

It fails closed straight after a provider change, with `insufficient_windows_in_provider_regime:n<3:<regime>`. That is the intended answer: after switching candle sources the honest state is "not enough clean evidence yet", not a promotion computed on mixed measurements. A window whose `providerCounts` is missing or empty yields `provider_regime_unknown` and also fails — a window that cannot name the data it was computed on cannot be shown to be like-for-like. Passing no `providerRegimes` preserves the previous behaviour exactly, so existing callers and tests are unaffected.

This route remains DORMANT/measure-only and must not write a policy — the change hardens the gate for whenever promotion is re-enabled; it does not alter live behaviour today. Mutation-verified: neutering the segmentation fails 5 tests. No historical row was rewritten.

2026-08-18: **Edge/IC candles route Yahoo-first for BOTH markets** (`lib/edges/data.ts::resolveCandles`, owner-approved, build-order step 4 of `features/walk-forward-ic-folds/`). Massive is plan-capped at a 2-year lookback, which held US IC history to ~12 usable non-overlapping h20 as-of dates — below the 12/fold floor, so walk-forward IC folds were unbuildable on US. Yahoo serves 5y (~50 dates), keyless and unpaced. It also ends the EODHD exhaustion: Massive per-symbol candles are paced at 12.5s (5/min), so a ~300-symbol EdgeScout run cascaded into EODHD's 20/day free tier (measured 2026-08-17: Massive 12, EODHD 20 = its cap, TwelveData 24, remainder unavailable).

`fetchUsCandles` — **the main research path** — **is untouched**. It has its own local 1y Yahoo fetcher; live `ResearchAgent` scoring inputs did not move. Two files carried a comment claiming otherwise; both were false and are corrected. `resolveCandles` is reached only from `edges/compute.ts` and `edges/ic.ts`, both measure-only, and `research-agent.ts` imports nothing from `lib/edges`.

Discontinuity, stated because a promotion gate reads this table. `edge_ic_history` rows from 2026-08-18 are computed on Yahoo bars where earlier rows used Massive/EODHD/TwelveData. Per-run `providerCounts` records which, so it is attributable rather than silent — but rows either side are NOT like-for-like. `lib/gates/promotion-gate.ts` reads a 1000-day window, so a promotion evaluated across the boundary mixes sources; segment by `providerCounts` before reading an IC change as signal. No historical row was rewritten.

2026-08-01 weight-resolution correction: live `ResearchAgent` scoring reads the market champion, then the static risk-profile baseline. `learning_priors` and `global_signal_weights` are learner configuration/proposal inputs, not hidden live-score fallbacks; challengers remain inert until validation and explicit promotion.

2026-07-28 correction — PROMOTION REMAINS DORMANT, NOT PERMANENTLY CLOSED. The h5 study measured pooled IC sigma ~0.27 and found no useful `mom_12_1` signal at h5 (mean IC 0.0089, `t_HAC` 0.32). It did **not** identify an effective breadth of 17: observed IC variance mixes sampling noise, changing point-in-time membership/coverage, and genuine time variation in factor returns. Inverting it with $1/\sqrt{t(n-3)}$ cannot separate those causes, and an h5 estimate cannot set h20 requirements. The $n=400$

and sector-neutral tests therefore remain legitimate measure-only experiments rather than rejected escape routes. POST /api/agents/backtest/promote still fails closed; no policy can consume these diagnostics.

2026-07-28 US PIT step-4 hardening: us_pit_adv20_top400_v2 ranks membership on a complete 20-session trailing dollar-volume window, shares cached session reads across overlapping as-of dates, excludes partial-window names, and persists one top-400 superset so matched n=200/n=400 tests use the same ranking. Report schema v2 retains successful-date cross-section and complete universe provenance. persist_edge_pit_snapshot () atomically writes exact snapshots to append-only edge_universe_members; it is service-role-only and conflicts fail closed. India PIT membership remains unavailable. These changes improve measure-only evidence and do not enable promotion.

PROMOTION IS DORMANT (2026-07-27). POST /api/agents/backtest/promote fails closed with promotion_evidence_not_oos (503) before any write. Adversarial review found one P0 and three P1 issues that each independently disqualify the current path: promotion is non-atomic (supersede-then-insert can leave a segment with no active policy); the evidence is not out-of-sample (~98.4% window overlap AND a current-liquid universe replayed through past dates — survivorship bias that more weekly runs cannot fix); dsr_z was not a Deflated Sharpe Ratio and is renamed t_margin_vs_trials, with the dsr column now written NULL; and experiment lineage is optional and unbound to the edge/market/horizon/segment it justifies. Re-enable only after features/walk-forward-ic-folds/FEATURE_ARCHITECTURE.md is approved and shipped: frozen experiment lineage → PIT universe/inputs → purged market-session OOS folds → aggregate HAC IC → multiple-testing + cost-adjusted validation → atomic promotion RPC.

Last updated: 2026-09-02 (per-session IC series + t-stat + Learning-page panel; h60/h120 evaluation horizons — see "Evaluation

horizons are decoupled from holding period"; 2026-08-16 label-maturation

coverage + starvation W7/W8 — see

"Label maturation: coverage, budgets and skip accounting"). Prior note

2026-07-27: The deterministic promotion route is implemented

but is governance scaffolding only: production has zero policies, its rolling

IC windows are not OOS, its current-universe history is not PIT, its

trial-adjusted t margin is not DSR, and supersede/insert is not atomic. The

revised features/walk-forward-ic-folds/FEATURE_ARCHITECTURE.md is a blocking

prerequisite before policy promotion or consumption.

Update this file when: the learning flow changes, new guardrails are added to weight mutation, genome parameters change, Phase 1 unlocks, the RAG pipeline changes, or Performance Truth Layer evaluation logic changes.

The big picture

```
flowchart LR
    LEARNER[LearnerAgent\nproposes] --> CHALLENGER[Challenger\na tweaked strategy]
    CHALLENGER --> SHADOW[Shadow test\nscore-only replay, no money]
    CHALLENGER --> VALIDATE[Validation Engine\nreplay on held-out folds]
    VALIDATE -- passes gates --> PROMOTE[Owner-approved activation?]
    PROMOTE -- yes --> CHAMPION[Champion\nthe live strategy]
    PROMOTE -- no --> ARCHIVE[stays a proposal]
    CHAMPION --> RESEARCH[ResearchAgent uses it]
    RESEARCH --> OUTCOMES[new closed trades] --> LEARNER
```

The loop is:

1. ResearchAgent scores stocks using the current **Champion** strategy's weights + genome.

2. PaperTrader acts on high-score signals. PositionMonitor watches and exits positions.
3. Closed trades become training data for **LearnerAgent**.
4. LearnerAgent proposes a **Challenger** strategy with adjusted weights + genome.
5. The **Validation Engine** replays the Challenger on held-out data.
6. If the Challenger passes, it may enter a non-executing shadow automatically. Learning and evidence collection continue without intervention. A separate owner-approved activation is required before it becomes Champion and ResearchAgent uses the new weights on its very next run.

Plan calibration and the LLM boundary (2026-07-30)

Every new ResearchAgent decision stores an indicative stop/objective/horizon in `decision_observations.features.trade_plan`. Nightly label maturation already stores immutable 2/5/10/20/60/120-session forward return, benchmark-neutral return, MFE, and MAE in `observation_labels`.

Evaluation horizons are decoupled from holding period (2026-08-17)

HORIZONS in `app/api/agents/label-maturation/route.ts` is `[2, 5, 10, 20, 60, 120]`. `h60/h120` were added on 2026-08-17. The trading mandate holds 5–15 days; the long horizons are **not** a mandate change and no position is held longer because of them. Every horizon is labelled from the same observation, so the long ones answer a question the $\leq 20d$ labels structurally cannot: **are we exiting too early?**

They were added before they could produce anything, deliberately. Maturation is calendar-bound: `maturityCutoff(60)` is now -85d against an oldest observation of 2026-07-06, so `h60/h120` matched zero rows at merge time and exit on the first empty page. First `h60` label $\approx$ 2026-09-29; roughly 20 independent decision dates by late October. Starting the clock early is the whole point — no engineering compresses forward time.

Setup-expert shadow rows were 2/6 for six weeks (2026-08-25)

From 2026-07-13 to 2026-08-24 only `etf_trend` and `quality_momentum` ever reached `shadow_decisions.value_inflection`, `pre_earnings_proximity_reweight_v1`, both India archetypes, and every `family_uncapped_v1`: * row wrote NOTHING, on any day, in either market.

Cause: migration `163_shadow_decision_idempotency.sql` created

```
create unique index shadow_decisions_observation_policy_uidx
on public.shadow_decisions(observation_id, policy_version_id) nulls not distinct
where observation_id is not null;
```

Every archetype row carries `policy_version_id = null`, so `NULLS NOT DISTINCT` made two experts for ONE observation collide. Postgres rejected the entire batch. Only routes of size one ever landed — ETFs always route to exactly one expert, and US names below the `value_inflection` threshold route to only `quality_momentum`. That is the whole of the surviving data.

It stayed invisible because a PostgREST insert RESOLVES on failure rather than throwing, and the call never read `.error`, so the enclosing try/catch saw nothing.

Fixed by `20260824170000_shadow_decisions_multi_expert_uniqueness.sql`, which re-keys the NULL-policy index to `(observation_id, setup_type)`. Verified on

2026-08-25: all six archetypes plus `family_uncapped_v1:*` now write in both markets. No code change was needed; P0b (India coverage) and P0c (missing archetypes) were both this one index.

Two detectors added, because the outage was silent, not subtle:

1. All three `shadow_decisions` inserts in `research-agent.ts` now read and log `.error`.
2. `/api/admin/shadow-liveness` gained `setup_expert_coverage`. The existing `setup-experts` probe reported "live" every single day of the outage — the table WAS growing, just missing two thirds of its experts. Recency cannot detect a partial outage; the new probe asserts every `market:setup_type` pair the router can emit appeared within 72h.

****Any future uniqueness rule on `shadow_decisions` must treat same-observation/different-setup_type rows as distinct.**** Pinned by `lib/scoring/archetype-coverage.test.ts`.

Consumers widened + an overlap-aware floor (2026-08-24)

`DIAGNOSTIC_HORIZONS` (`lib/learning/dimension-diagnostics.ts`) and `SUPPORTED_HORIZONS` (`lib/learning/plan-calibration.ts`) were both still `[2, 5, 10, 20]`, so the h60/h120 labels would have matured in late September into consumers that never read them. Both now match the labeler.

Widening the diagnostics constant required migration

`20260824220000_diagnostic_horizons_60_120.sql` FIRST: `dimension_diagnostic_runs` constrained `horizon_days` to the short set, and because `runMarket()` rethrows on insert failure, an h60 insert violating that CHECK would have killed the run for every other horizon too. Applied and verified 2026-08-24.

The date floor is now overlap-aware, and that is the load-bearing part.

`MIN_PREDICTIVE_DATES = 20` counts decision dates without regard to horizon, and consecutive forward windows of length `horizonDays` overlap almost entirely:

horizon	20 dates	independent observations
h10	20	2.0
h20	20	1.0
h120	20	0.17

Widening the horizons without this correction would have let the 20-date gate pass at h120 and emit a `measured_descriptive` predictive finding built on roughly ONE non-overlapping window. `MIN_EFFECTIVE_OBSERVATIONS = 12` is applied on top of the date floor via `effectiveObservations(n, horizonDays) = n / horizonDays``; both must clear. A long horizon needs proportionally MORE dates, not the same number. Mutation-verified in `lib/learning/dimension-diagnostics.test.ts`.

Long-horizon labels no longer trust the candle cache.

`price_cache` stores closes adjusted AS OF FETCH TIME, and providers restate adjusted history after a split or distribution. `CandleResolver` short-circuited the provider whenever the cache covered the window, so a covered-but-old slice yielded bars wrong by the split factor — a 2:1 split inside the window reads as a -50% return. Negligible across 10 sessions; months of exposure per label at 60/120. `LONG_HORIZON_REFRESH_DAYS = 60` now forces a provider fetch at the long horizons regardless of coverage. Cost stays bounded by `providerTried` (one fetch per symbol per run), and the fresh bars restate both the in-memory series (`mergeCandles`, later sources win) and `price_cache` itself (`upsert` on `symbol, date`).

****Two long-horizon accuracy caveats remain OPEN and must be stated alongside any h60/h120 finding:****

1. `benchmark_neutral_return` subtracts the raw benchmark return with no beta adjustment. Immaterial over 10 days; over 6 months a high-beta name shows systematic phantom alpha.
2. Survivorship — delisted or acquired names never mature a label and drop out silently, biasing long-horizon results optimistic.

Read long-horizon results by distinct decision DATES, not label count.

Observations scored on one day share that day's market shock and are one draw, not N. At h20 the sample was already only ~10 dates per market when the long horizons were added.

Provider depth is load-bearing. `CandleResolver.providerTried` is keyed `market:symbol`, so each symbol is fetched at most once per run and that single fetch must satisfy the *longest* horizon. India's provider range was widened `3mo → 2y` for exactly this reason: a ~63-bar series cannot cover a 120-session forward window at any decision date, and the fetch is never retried, so short coverage would starve the long horizons silently for the whole run. US needed no change (`lib/data/candles.ts` fetches Yahoo at `range=1y`, ~251 bars). `tests/label-window.test.ts` pins this contract.

`lib/learning/plan-calibration.ts` joins those two truths only when the label horizon exactly matches the original plan horizon. It reports objective reach, stop breach, and realized path statistics. The objective is an exit-policy level, not a predicted terminal price; MAE/MFE cannot establish which level was touched first.

Dimension diagnostics and agent accountability (2026-08-06)

`/api/agents/dimension-diagnostics?market=us|india` runs after label maturation and writes append-only `dimension_diagnostic_runs` and `dimension_diagnostic_findings`. It reads only the existing decision/label ledgers plus signal labels. It reports availability and descriptive session-level rank IC by dimension, and records an agent/version's evidence and decision record. Results below the predeclared 20 qualifying-session floor are `insufficient_evidence`, not a conclusion.

Per-session IC series, t-statistic, and the Learning-page panel (2026-09-02)

`predictiveMetrics` now also emits `sd_session_rank_ic`, `t_stat`, and `session_ic_series` (one `{date, ic, cross_section}` per qualifying session, sorted chronologically). $t_stat = \text{mean} / (\text{sd} / \sqrt{n\text{Effective}})$ — `nEffective`, never the session count**: dividing by `sqrt(sessions)` overstates `|t|` by `sqrt(horizonDays)` (~4.5x at h20) and manufactures significance out of overlap. It is null rather than Infinity when spread is undefined (<2 sessions) or zero.

No plan-version bump: `mean_session_rank_ic` is unchanged, these are additive fields. No migration either — the series lives in the existing `metrics.jsonb`, so it is immutable with its finding, which is what the frozen-history rule wants. The series is attached to the eligible-long headline ONLY; the all-scored context keeps summary stats but gets no line, since it may never be cited as predictive power.

Why a per-session series had to exist before anything could be charted.

mean_session_rank_ic is an EXPANDING-window average over every session to date, so consecutive daily runs share ~93% of their input. Plotting the stored run history draws a cumulative mean converging, not a signal changing. Measured on the real US h5 rows, technical read +0.1394 frozen for nine consecutive days (qualifying_sessions stuck at 6), then oscillated **non-monotonically** (17 → 15 → 17 → 14 → 13) while the truncated loader reshuffled its sample, then settled near -0.13. An expanding window over an append-only label store can only grow; that oscillation is the 1,000-row cap's fingerprint. Drawn as one line it reads "technical decayed through August" — it is 82be100a (cohort) and 3aa3753f (pagination) landing hours apart on 2026-08-28. Usable history is therefore **three run-days**, and the honest chart is the per-session series from the newest run, which already contains every session.

components/dashboard/DimensionICPanel.tsx renders it on /dashboard/learning: horizon selector, a table (mean IC, SD, t, sessions, nEff vs floor, positive-day share, all-scored context, verdict), the per-session chart with a zero reference and an optional rolling mean, and a definitions panel for every column. Session gaps are left open rather than joined — connectNulls={false} — because a connected gap invents an observation on a day the dimension did not qualify. The insufficient_evidence verdict is rendered prominently and is NOT softened by the presence of a chart.

ResearchAgent writes the deployed commit SHA into each new decision observation. Agent contribution findings require that provenance; legacy unversioned rows are explicitly data_degraded and are never silently compared with later releases.

This is **not** LLM punishment/reward or autonomous self-modification. It cannot change a prompt, model, tool access, score, weight, threshold, candidate, strategy, paper/live trade, exit, sizing, or broker action. Multiple agents appearing in one workflow do not receive collaboration credit: the ledger records unattributable_no_paired_shadow until the same market-local opportunity is compared with and without a declared input in a paired non-executing shadow.

Decision Review shows per-symbol paths as illustrative. LearnerAgent receives only same-market aggregate summaries through query_plan_calibration. The LLM may explain the cohort and write a hypothesis, but it has no tool that changes stop, target, or horizon. The existing deterministic MAE/MFE policy remains the only automatic risk-parameter adjustment path and requires at least 60 same-market, exact-horizon eligible-long labels. US and India never share a

cohort.

Benchmark-alpha scorecard boundary (2026-07-13)

/api/agents/benchmark-scorecard writes Phase-1 measurement rows to benchmark_scorecard for paper/live, US/India, and 1W/1M/3M/YTD/1Y horizons.

The math uses common-window rebasing and annualized daily information ratio.

These rows are displayed in Performance Truth and may be read as evidence, but they do **not** change LearnerAgent objectives, promotion gates, posture, sizing, paper fills, or live orders in Phase 1. Phase 2 learner/promotion wiring still requires a separate owner-approved implementation.

The Paper Portfolio chart also supports market-local display comparisons. US offers VOO/QQQ/XLK/XLF; India offers NIFTY 50 plus priceable NIFTY IT, NIFTY Bank, and NIFTY Next 50 ETF proxies. The owner's selection is saved in app_settings, but is deliberately distinct from benchmarks.is_primary: changing what the chart displays cannot change a mandate, learner input, promotion decision, score, fill, size, or order. Secondary daily observations come from existing free providers and are exact-session joined without forward fill.

Champion and Challenger

strategy_versions table (governance)

One row per strategy version. At most one is_champion = true per market at any time.

Field	Meaning
market	us or india — markets evolve independently
is_champion	True for the one active strategy per market
weights_snapshot	5-dim weights that ResearchAgent reads
genome	Trading parameters (see below)
proposed_by	learner or user
backtest_result	Sharpe, Sortino, win_rate, max_dd from Validation Engine
promoted_at	Set only when an owner-approved activation succeeds

The genome

What a Challenger can evolve:

Parameter	Range / options	Notes		
entry_threshold	50–90	analyst_score cutoff to open a position		
exit_stop_pct	3–15%	Stop-loss percentage below entry		

Parameter	Range / options	Notes		
exit_target_pct	10–40%	Price target percentage above entry		
horizon_days	3–30	Time-stop maximum hold days		
position_size_pct	Up to strategy_config.position_size_pct	Can only size DOWN, never above the owner-set cap		
sizing_mode	fixed \	kelly \	confidence_scaled	How position size is calculated
entry.rank_pct_min	0.0–0.95	Default 0.0 = OFF (feature no-op). Minimum within-comparable-group percentile for a NEW long. Hybrid gate: entry requires <code>analyst_score ≥ score_threshold</code> AND <code>rank_pct ≥ rank_pct_min</code> . 0.0 reproduces current selection byte-for-byte. Rank gates cross-group *admission* only; intra-group ordering stays by <code>analyst_score</code> (rank is monotonic in score within a group), so the CLAUDE.md "top-3 by analyst_score win" rule is preserved. Becomes active only via a validated, owner-promoted challenger.		
5 dimension weights	Sum must = 1.0	{fundamental, technical, sentiment, macro, insider}		

Per-market independence

strategy_versions has a market column. LearnerAgent analyzes one market's cohort per run and proposes challengers only for that market. A bad India run cannot shift US scoring weights. India starts on a clone of the US champion as a prior and diverges once it clears the same 10+ closed-trade phase gate.

User visibility

Dashboard → Learning → Active strategy and challengers is the user-facing, market-scoped strategy view. It shows the active champion (or the default baseline when none is promoted), every challenger, its validation verdict and latest experiment metrics, plus prior champions. A challenger with a passed validation shows an owner-only **Request champion activation** control. The server remains the authority and refuses it unless every promotion gate passes; the current OOS

promotion gate is dormant, so the control reports that refusal rather than changing the champion. Learning, validation, and non-executing shadow collection continue automatically. A champion has no fixed expiry: weekly learning and continuous outcome monitoring decide when a new challenger is worth evaluating.

LearnerAgent details

File: app/api/agents/learner/route.ts, app/api/agents/learner-brain/route.ts

Schedule: Fridays 5:00 PM ET

LLM: Claude Opus 4.8 (c laude - opus)

Phase gate

Weight mutation is **blocked entirely** until 10+ closed trades per market exist (Phase 0).

LearnerAgent runs but only writes a "mutation blocked: insufficient trades" note to learning_log. Phase 1 (mutation unlocked) requires the owner to verify the trade set and acknowledge the gate has been cleared.

Run-accounting convention: A LearnerAgent run may reconcile dangling orphan trade rows as maintenance. Its run summary reports that count separately from the market-local total closed-trade corpus (Reconciled ... | Total closed: ...); zero reconciliations never means zero learning data.

OPEN ITEM (2026-07-16): India macro contamination in the historical trade set — NOT yet remediated

Until 2026-07-16, India signals were scored with the **US** macro regime (macro_regime has no market column) and, on 2026-07-13, with a **fortnight-stale** green verdict that was really a failed MacroSentinel run (raw_indicators = []). lib/data/scores.ts now excludes macro for India and age-bounds the row — but **the historical rows already written are still contaminated**.

Measured in prod on 2026-07-16 (entry signal joined to signal_score_history):

India paper_trades	Count
Closed, entered on a macro-contaminated score	4 (2 via stale-green macro_score=100, 2 via US-orange macro_score=60)
Closed, clean (macro already excluded)	3
Open, entered on a macro-contaminated score	13 (3 stale-green, 10 US-orange)
Open, clean	2

Why this is not yet urgent: the mutation gate is 10+ closed trades **per market**; India has 7 closed. The learner is still Phase 0 for India, so no contaminated trade has driven a weight mutation yet. The 13 contaminated **open** positions will close into this set, though — so the

record should be corrected **before** India reaches 10.

Proposal (owner decision — deliberately NOT applied, no rows were mutated): flag rather than delete. The tainted vehicle already exists (`paper_trades.tainted`, `taint_reason`, `excluded_from_learning`; filter in `lib/learning/taint-filter.ts`), so no migration is needed — set `tainted = true`, `taint_reason = 'macro_regime_market_leak'` on the affected rows. `applyLearningTaintFilter` then drops them from learning and from the RAG memory corpus, while `run-evaluation.ts` still **counts** them in P&L (the book really moved — see "Tainted trades" under Performance Truth). Note the flag is coarse: it removes the whole trade from learning, not just its macro dimension. Given India's small *n*, the honest alternative — accept a smaller-but-clean India trade set — is preferable to learning from a US-Fed-scored Indian book.

Tool-use loop (9 tools)

LearnerAgent runs a multi-step tool-calling loop (via `runAgentLoop()`). The 9 tools it has:

1. `get_closed_trades` — recent `paper_trades` with outcomes
2. `get_signal_weights` — current champion weights
3. `get_strategy_versions` — all challengers + their backtest results
4. `get_decision_observations` — scored decisions (including skipped)
5. `query_trade_decisions` — real historical enriched Robinhood trades by regime/action
6. `propose_challenger` — write a new `strategy_versions` row with new weights + genome
7. `run_validation` — trigger Validation Engine on the proposed challenger
8. `get_mentor_insights` — recent coaching notes from MentorAgent
9. `semantic_search_decisions` — pgvector RAG over trade memories (if Jina key present)

Automatic validation & shadow routing (migration 170)

When LearnerAgent creates a challenger it runs `runAutomatedValidation()` (`lib/validation/automation.ts`) **in-process** — this replaced the earlier fire-and-forget localhost request, which was unreliable on cloud cron. The flow, gated by the per-market `strategy_validation_automation` policy (fail-closed: missing row / read error = fully disabled):

1. Policy `enabled=false` → return immediately, no validation (challenger still exists and can be validated manually).
2. Else run the deterministic **Validation Engine** (`validateChallenger`), which writes a `validation_experiments` row (pass or fail) exactly as the on-demand Validate button does.
3. If it **passed** AND `auto_shadow_enabled=true` → call the `activate_strategy_shadow(p_version_id)` RPC, which atomically routes the challenger to `state='shadow_paper'` under a per-market advisory lock and a

max_active_shadows (0–1) capacity cap.

The **only** automatic lifecycle transition is → shadow_paper (non-executing: records what it *would* decide, no fills/cash — see the Shadow section). It can **never** promote a champion, create a paper fill, move cash, make a broker proposal, or place a live order. Champion promotion stays the separate owner-only path and remains fail-closed on a PASSED validation_experiments row.

A Friday cloud recovery cron — kairos-validation-sweep, **21:45 UTC** (POST /api/validation/sweep) — validates up to 5 never-validated challengers per market (state='challenger', validation_experiment_id IS NULL) through the same path, catching challengers created outside LearnerAgent or interrupted before in-process validation completed. Owner controls both switches per market in **Settings → Automatic Strategy Validation** (GET/PATCH /api/settings/validation-automation); disabling preserves every challenger, experiment, and shadow. Feature spec: features/automated-strategy-validation/.

Auto-guard

In addition to the phase gate, LearnerAgent blocks mutation if the last 3 runs have win_rate < 35%. This prevents a losing streak from producing an overfit Challenger.

Per-trade notes

After each trade closes, LearnerAgent writes a 1-sentence outcome summary to learning_log. This is separate from weight mutation — it runs regardless of the phase gate.

Validation Engine

File: lib/validators/backtest.ts, app/api/agents/backtest/route.ts

Deterministic, no LLM. Replays a Challenger vs the current Champion on walk-forward held-out slices of the decision_observations ledger.

Label maturation: coverage, budgets and skip accounting (2026-08-16, W7/W8)

/api/agents/label-maturation returned {success:true, matured:0, skipped:800} for 25 days and every monitoring layer read it as healthy. Labels stop dead at 2026-07-22 in **both** markets, with 1,738 US / 448 India observations old enough to label and unlabelled. Three defects, each individually silent:

1. **Row existence was mistaken for coverage.** usCandles() returned any

non-empty cached slice. `sinceDate` was `decisionDate - 5 days`, so one stale-but-present `price_cache` row satisfied it and the provider fallback below it was **unreachable** — coverage could never self-heal.

2. **The candle memo was range-blind.** It was keyed `market:symbol` but spanned all four horizons, so an h2 request's narrow window was reused for a later h20 request of the same symbol and starved it.

3. **The oldest rows monopolised the budget.** `loadPendingObservations` ordered `ts ASC` and sliced the first 200, so a permanently-failing prefix consumed the entire per-horizon budget every run and newer observations were never reached.

The rule now, in `lib/learning/label-window.ts`:

- `forwardWindow(candles, decisionDate, horizonDays)` is the only coverage test — the entry bar plus `horizonDays` forward sessions, or `null`. Never `candles.length > 0`, never a row count.
- `CandleResolver` re-checks coverage on every request, widens the cache read when a later horizon reaches further back than any previous request, and falls through to the provider **exactly once per symbol per run** when coverage is still short. A per-key promise chain keeps the old memo's "one fetch per symbol" property without its range blindness.
- **Scan budget is decoupled from success budget:** `SCAN_CAP` 2000 observations examined per horizon, `SUCCESS_BUDGET` 200 labels produced. Skipped rows spend the scan budget only. Capacity is split between an oldest-first pass and a deterministic day-rotating cursor (`rotatingOffset`), which visits every page over consecutive days, so the newest observations are always reachable.
- `SkipLedger` reports skips by reason (`no_candles`, `window_incomplete`, `no_entry_price`, `label_unavailable`, `insert_failed`, `exception`) and by symbol, in both the HTTP response and the `agent_runs.result_summary`.

Detector. `zeroOutputWithPending` — `matured === 0 && skipped > 0`, the exact incident signature — writes `agent_runs.status='error'` instead of `'done'`. A fully-drained backlog yields `matured:0, skipped:0` and stays healthy. Regression cover is `tests/label-window.test.ts`, which fails if a stale cache short-circuits the provider, if the memo starves an h20 window after an h2 request, or if a permanently-failing prefix consumes the label budget.

A rejected design, recorded so it is not rebuilt: a `label_attempts` column on `decision_observations`. One observation participates in h2/h5/h10/h20, so a single counter cannot distinguish a transient h20 shortage from a permanent h2

failure and would exclude valid work for the wrong horizon. A horizon-scoped retry ledger keyed (`observation_id`, `horizon_days`) is the fallback, and only if permanent failures survive the coverage fix.

Open: draining the 1,738 US / 448 India backlog and confirming the label high-watermark advances past 2026-07-22 requires production access and has not been done. Until it is, `observation_labels` coverage beyond 2026-07-22 is unproven and the US-vs-India scoring comparison stays in question.

2026-09-15 — capacity, not coverage. Production showed a second, structural backlog: research writes ~160 US and ~50 India decisions a day (one symbol up to three runs a session) against `SUCCESS_BUDGET` 200 labels per horizon per night shared by both markets. US h10 labels stopped at decisions from 2026-08-24 and India h10 was almost entirely unlabelled; recent runs matured 200-782 labels with only `no_candles` skips, so budget exhaustion, not candles, was the cause. Owner-approved fix in `app/api/agents/label-maturation/route.ts`:

- **Label the last observation per market/symbol/session only** (`supersededObservationIds` in `lib/learning/label-window.ts`). Every evaluation already deduplicates to that row, so the earlier runs were budget spent on data nothing reads. The page loader reads same-day siblings directly, because a later run can sit on the next page or past the maturity cutoff.
- **Separate budgets per market.** A cron call without `market` now runs US and India each with the full per-horizon budget instead of sharing one.
- **Run deadline** at 240s of the 300s `maxDuration`; a run that stops with work left says so in `agent_runs.result_summary`.
- **Newest-first instead of oldest-first.** The first production run after the two changes above caught India up (h10 to 2026-08-31) but wrote zero US h5/h10 labels: 1,060 of 1,071 unlabelled US h5 rows and 1,323 of 1,334 h10 rows sat past scan position 2,000 behind an already-labelled July prefix, where neither the oldest-first pass nor that day's rotating cursor reached. The first pass now scans newest-first from the maturity cutoff; the rotating cursor still sweeps older gaps over consecutive days, so a permanently failing old row still cannot block newer work.

Existing labels are not rewritten; earlier same-session duplicates that already have labels keep them. No schedule, schema, score or trading change.

ATR exit-policy evidence (measure-only)

Nightly label maturation also records point-in-time entry ATR, ATR-normalized

MAE/MFE, and three predeclared `close_observed_v1` exit-policy outcomes. The simulation uses completed daily closes and the standard 10 bps haircut, so it does not infer intraday fills from daily OHLC ranges. The authenticated `/api/analytics/atr-exit-evidence` report is strictly market- and horizon-local. Its `reviewable_evidence` status requires at least 60 labels and 12 effective observations, but is not a promotion gate and cannot change PaperTrader, PositionMonitor, live orders, or broker protection. Purged walk-forward and paper shadow approval remain mandatory before any execution-policy proposal.

Eligibility gates (same for US and India)

Gate	Threshold
Sharpe	≥ 0.5
Win rate	$\geq 40\%$

If both gates pass, `eligibility_passed = true` is set on the `experiment_runs` row.

Promotion is blocked (HTTP 412) unless `eligibility_passed = true`. The owner cannot click "Promote" without the Validation Engine having run and passed.

Metrics computed

- Sharpe ratio
- Sortino ratio
- Maximum drawdown
- Win rate
- Expectancy
- Alpha vs benchmark

Walk-forward design

The Validation Engine splits historical data by time, scoring the Challenger only on data it could not have seen when it was proposed. This prevents in-sample overfitting from looking like genuine improvement.

Calibration OOS acceptance gate

`lib/validation/calibration.ts` fits the `pwin_logistic` artifact from walk-forward folds (chronological, purged/embargoed). `acceptCalibrationOOS` computes the Expected Calibration Error (ECE) over the **out-of-sample holdout only** and is **fail-closed**: a model is rejected (`accepted: false`) when the holdout has <30 usable rows, is degenerate (all-win/all-loss or non-finite ECE), or $ECE > 0.1$. `fitAndStoreCalibration` gates the artifact upsert on this verdict — a miscalibrated

model is never written to the live `pwin_logistic` artifact, so the money path never reads a bad `P(win)`. `MIN_OOS_SAMPLES=30` and `MAX_OOS_ECE=0.1` are v0 thresholds needing prospective tuning.

Shadow decisions

A Challenger can be set to "shadow" real runs: it records what it *would have done* on every stock with no fills and no cash. This is a free dress rehearsal — the Challenger accrues a simulated track record before any promotion decision. Off by default. Activated per Challenger in the Strategy Registry.

Instrument-family challengers (2026-08-24)

Family learning is hierarchical, not per ticker: ``market × instrument_family × setup × horizon``. ResearchAgent records one uncapped-v1 shadow comparison for special fund families and stores raw family drivers separately. Diagnostics first collapse repeated symbol/session rows, then collapse substitute vehicles to one `exposure_id × market session`; GLD and IAU therefore contribute one gold sample, not two. US and India never pool evidence.

The initial readiness floor is 60 independent exposure-sessions, at least 30 clean out-of-sample forward observations at a declared horizon, and non-degenerate prediction variance. Passing these counts only permits IC/cost/calibration review; it does not promote a score. Family benchmark alignment, overlapping-horizon uncertainty, net-of-cost results, isolated exploratory paper, and explicit owner promotion remain mandatory. Current production cohorts are below the floor, so the correct state is abstention, not an automatically generated composite.

Feature Registry

LearnerAgent can also propose a **new formula idea** — a new factor to incorporate into scoring. This is written as a human-readable spec with a falsification test. The formula is **never run as arbitrary code** — it is only interpreted through a locked, whitelisted math grammar. AI cannot write executable scoring code directly.

Cross-sectional rank (measure-only → gate, OFF by default)

Files: lib/scoring/rank.ts (computeComparableRank, isRankRejected), app/api/agents/research/cron/route.ts (Pass 2), lib/validation/genome.ts (entry.rank_pct_min). **Spec:** features/cross-sectional-rank/FEATURE_ARCHITECTURE.md.

A deterministic (no-LLM) **second pass** in the research cron, after all symbols score and before PaperTrader. It partitions the eligible pool into comparable groups (market × asset-type × sector), computes each name's within-group empirical percentile (rank_pct), and — **only when the champion genome's** entry.rank_pct_min > 0 — flips rank-losing NEW candidates in agent_signals to status='rank_rejected'. Data-quality gates run before ranking (thin evidence, abstain, evidence-confidence floor, held-position exclusion); groups below their min-sample threshold (US equity 20, India equity 15, ETF 20) fall back to a fixed degraded transform — the "three finalists are not a universe" guard. ETFs are never grouped with single-name equities. Provenance is written to universe_snapshot_scores (rank_quality, comparable_group_key, group_n, rank_eligible).

Resolved (2026-07-11): the CLAUDE.md "top-3 by analyst_score win" rule is preserved — rank gates *cross-group admission* only; rank_pct is monotonic in analyst_score within a group so intra-group ordering is unchanged, and under today's single-group degraded case rank and raw-score selection are identical. Ships OFF (rank_pct_min default 0.0 → byte-stable selection); actionable only through a validated, owner-promoted challenger. Rank is **not** fed into the live P(win)/sizing model — logged as a feature for IC measurement only.

PIT fundamentals ledger (OFF by default)

Files: lib/data/pit-fundamentals.ts (getFundamentalsAsOf, captureFundamentalsFact), capture hook in lib/research-agent.ts. **Table:** fundamental_facts (migration 150). **Spec:** features/pit-fundamentals/FEATURE_ARCHITECTURE.md.

Restatement-safe append-only vintage archive: fundamentals are captured on fetch (fire-and-forget, fail-open, dedup by payload_hash) so "fundamentals as known on date D" is reconstructable and a later restatement can never retroactively change a past as-of read. Not yet wired into live scoring — scoreFundamentals is unchanged and default scores are byte-identical. Purpose: give the Validation Engine and any future walk-forward replay a leak-free fundamentals source.

Historical replay harness (measure-only, OFF)

Files: lib/replay/* (sealed accessor + FutureDataLeakError, packet assembler, gates, reporter). **Tables:** replay_packets, replay_packet_items, replay_eligibility_runs, replay_eligibility_events (migration 149). **Spec:** features/historical-replay-harness/FEATURE_ARCHITECTURE.md.

An offline harness that freezes point-in-time input packets (knowable_at ≤ as_of enforced by a sealed data accessor that throws on any post-cursor datum) and replays the eligibility gates (calibration_oos, thin_evidence, ic, validation, breakdown_veto) to answer "on what date would this strategy first have been eligible?" (first_eligible_asof = MIN(as_of) WHERE passed). Reuses the live gate code (fitCalibration → walkForwardFolds → acceptCalibrationOOS, computeWeightedAnalystScore, isThinEvidence) unchanged so a replay grades on the identical rule that would run live. Runs on in-memory fixtures today; the migration-149 tables persist runs when wired.

Local bulk-evidence worker (measure-only)

Spec: features/local-historical-replay/FEATURE_ARCHITECTURE.md.

Large official datasets remain outside Supabase under the local Kairos evidence store. The worker verifies every manifest/file hash, predeclares the plan in the existing immutable backtest_experiments ledger, runs without provider access, and writes only compact results and fingerprints. Initial scope is an India NSE

price-only OOS IC diagnostic. Backtest reads those results through an owner-only API; the browser cannot start the worker. These records are not promotion, scoring, or trading inputs.

Deterministic portfolio simulation (measure-only, P0)

Files: `lib/simulation/portfolio-simulator.ts` and its fixture tests.

Spec: `features/portfolio-simulation/FEATURE_ARCHITECTURE.md`.

This pure TypeScript accounting engine evaluates predeclared entry/exit events in one native-currency market at a time. It explicitly models cash, costs, whole/fractional-share policy, open-name caps, and deterministic same-session ordering (exits before entries), so released cash can be redeployed exactly once. It does not fetch data, score symbols, select trades, call an LLM, persist rows, or reach a paper/live/broker path. A future sealed replay may use it to compare exit rules whose holding periods differ; no result can change a strategy without the ordinary validation and promotion lifecycle.

Governed external research (disabled P0 contract)

Files: `lib/external-research/contracts.ts` and fixture tests.

Specs: `features/external-research-shadow/FEATURE_ARCHITECTURE.md` and `features/external-research-integrations/FEATURE_ARCHITECTURE.md`.

The initial contract rejects every external artifact until a real source commit, license/SBOM review, synthetic sandbox proof, and explicit release admission exist. It validates a single-market/currency snapshot, exact source/snapshot provenance, bounded payloads, finite numbers, and unsafe object keys. It has no dispatcher, credential, worker, provider, database, score, strategy, paper, live, or broker consumer. This is deliberately a safety foundation, not an enabled Vibe runtime.

Performance Truth Layer

File: `lib/evaluation/run-evaluation.ts`, `/api/agents/evaluation/*`

Panel: `/dashboard/learning`

Mandate-aware, deterministic (no LLM), honesty-first evaluation.

Investment mandates

Named strategy contexts. Default mandates:

- "Swing US 2-20d" (benchmark: VOO)

- "Swing India 2-20d" (benchmark: ^NSEI)

Every agent_signals, paper_trades, and decision_observations row gets a mandate_id stamped at creation time.

Future decision_observations also carry the actual scoring-candle close and a deterministic indicative trade-plan snapshot. This improves point-in-time label truth and lets Research Journal explain approximate entry/risk/target context. Execution does not consume stale absolute research prices: PaperTrader resolves the current bounded policy and anchors prices to the fill, while PositionMonitor continues to own the stored position's exits.

Evaluation metrics

Metric	Notes
Sharpe	Risk-adjusted return
Sortino	Downside deviation only
Max drawdown	Worst peak-to-trough
Win rate	% of trades that closed positive
Expectancy	Average expected profit per trade
Profit factor	Gross profit / gross loss
Alpha	vs benchmark (VOO/^NSEI)
Execution slip	Mean realized vs 0.05% modeled slippage (both are the SAME constant — see below)

Execution cost is an ASSUMPTION, not a measurement (2026-09-02)

spread_applied is a **fraction of price**, and every fill in both markets carries exactly 0.00050 — 205/205 rows, zero variance. The fill path applies a hardcoded 5bps and stores that same constant back, so the "realized" cost is the assumption read back to itself. **Cost cannot be calibrated from our own fills;** doing so would validate the assumption with itself.

Three defects were fixed together:

- **costNet misread the units.** It computed $|\text{spread_applied}| / \text{fill_price} * 100$ as though the column were an absolute price offset, understating cost by a factor equal to the fill price — 0.0009% on a \$57 name, 0.0001% on a \$417 one, against a true 0.05%. `slip()` in the same file already had it right ($\text{fraction} \rightarrow \%$), so the file contradicted itself. Reported cost drag went from ~0.001% to 0.05% per trade.
- **The strategy replay charged nothing.** `compileSpec` left `costPct` unset on entry and exit, so the simulator's multiplier was 1 and every replay result was frictionless — structurally optimistic against the paper book it is meant to be

a counterfactual for. Both sides now pay the same constant.

- **One number, five encodings.** 1.0005 , 0.0005 (x3) and `MODELED_SLIP_PCT = 0.05`

expressed the same quantity in two units across three files. All now derive

from `MODELED_SLIP_FRACTION` in `lib/analytics/performance-metrics.ts`.

What 5bps/side is worth. Conservative for the current universe and size (US mega/large caps, ~\$1k orders on a \$10k NAV sit inside top-of-book; the true cost is about a half-spread, under 1bp on AAPL, with no broker commission). It stops being conservative for thin names, for India, or once NAV grows enough to move the book. `computeFillPrice` has a real bid/ask path, but ask is null in production, so it never engages. Treat cost as a sensitivity band, never a point estimate.

Honesty rules

- **Fewer than 20 trades** → shows `insufficient_sample` instead of a number. No fabricated precision on tiny samples.
- **Tainted trades** (low `data_confidence`) are **counted** here — the book moved, so P&L must not hide them. They are labeled as tainted but included in the totals.
- `health_label` summarizes the overall picture:
 - `insufficient_sample` → not enough data to say anything
 - `negative_or_zero_edge` → strategy has no positive edge
 - `promising_but_unvalidated` → positive metrics but not yet through Validation Engine
 - `validation_required` → ready for formal promotion decision

P1 gate

A weekly Vercel cron counts closed evaluable trades per market. When ≥ 20 accumulate, it fires a System Health info alert: `p1-gate-ready:<market>`. This is the signal to build opportunity-level IC metrics (`decision_observations × observation_labels`).

P0 is book-truth only. The `opp_*` columns in `strategy_evaluations` are null until P1.

RAG trade memory pipeline

```
flowchart LR
    CLOSE[Trade closes\nPositionMonitor] --> INDEX[indexClosedTrade:\nwrite setup as text\nembed with Jina 1024-dim\nstore in trade_memories]
    INDEX --> STORE[(pgvector\ncosine similarity)]
    NEW[New candidate\nResearchAgent] --> RETR[retrieveSimilarTrades:\nembed live setup\nmatch nearest\nrerank top-5 with jina-reranker-v2]
    STORE --> RETR
    RETR --> NOTE[prior similar setups\n3/5 were wins]
    NOTE --> THESIS[injected into thesis prompt\nLLM sees its own track record]
```

Write side — `indexClosedTrade()`

- Triggered on every trade close (`PositionMonitor` + `LearnerAgent` exits)

- Builds a short text: symbol, market, 5 dimension scores, outcome, exit reason, mandate
- Embeds via Jina AI `jina-embeddings-v3` (1024-dim)
- Stores in `trade_memories` (pgvector table in Supabase)
- **Tainted / excluded trades are skipped** — bad-data history cannot poison memory

Read side — `retrieveSimilarTrades()`

- Called by `ResearchAgent` before scoring each candidate
- Fingerprints the live setup as text, embeds with Jina
- Queries pgvector nearest-neighbor (cosine, IVFFlat index) for top-K candidates
- Reranks with Jina `jina-reranker-v2-base-multilingual` to pick genuinely similar past setups
- Returns a one-line summary: `"prior similar setups: 3/5 were wins"`
- Writes a `rag_traces` row for audit

Guardrails

- Ticker filter: a retrieved chunk that doesn't mention the candidate symbol is dropped
- Whole path is **off when JINA_API_KEY is absent** — no key → silent no-op
- Does NOT move money or change weights. Advisory context only.

Signal weights

Current weights (live scoring)

`ResearchAgent` reads the newest promoted `strategy_versions.weights_snapshot` for the symbol's market. If no promoted champion exists, it uses the selected static risk-profile baseline (conservative, balanced, or aggressive), then applies the market mandate's strategy tilt. Missing dimensions are removed and the remaining weights are renormalized. In plain language: only an explicitly promoted market strategy, or the documented profile baseline, can set the live score mix.

`learning_priors` and the global `signal_weights` row are **not** live-scoring fallbacks. `learning_priors` controls which dimensions `LearnerAgent` may study and mutate. `signal_weights` remains a legacy/display row and is used only as a proposal baseline when `LearnerAgent` has no market champion; a proposal still creates an inactive challenger and cannot affect scoring until validation and promotion succeed.

Weight change audit

Legacy prior/global-weight changes are logged to `learning_priors_history` and `signal_weights_history`. The active strategy audit trail is the immutable `strategy_versions` challenger/champion lifecycle. These records are not purged by the cleanup job.

Learner config

`learner_config` table controls per-dimension mutation:

- `learn_from = false` → exclude this dimension from `LearnerAgent` analysis
- `allow_mutation = false` → `LearnerAgent` cannot propose weight changes for this dimension

These are toggled via `/api/agents/learner-controls`.

Promotion gate (deterministic)

The path from "this edge measures well" to "this edge is policy" runs through `lib/gates/promotion-gate.ts` and `POST /api/agents/backtest/promote`. It is the **only** writer to `strategy_policies`, and there is no LLM anywhere on it — the LLM's role ends at proposing a bounded hypothesis into `backtest_experiments`. `strategy_policies.promoted_by` is CHECK-constrained to `'deterministic_gate'`, so that boundary is enforced by the database rather than by convention.

<!-- Superseded 2026-07-27 promotion-gate description retained in git history only.

Inputs

- `edge_ic_history` rows for one `edge_id` + market, scoped to the requested segment (sector → `segment_type='sector'`, regime → `segment_type='regime'`, neither → market-wide rows where `segment_type` is null) and to the horizon band `[horizon_days_min, horizon_days_max]`.
- `backtest_experiments.variants_run` when an `experiment_id` is supplied. Absent one, the gate assumes a single trial — the least punitive assumption, so the other gates have to carry the decision.

The four gates

| Gate | Rule | Why |
|--------------|------------------------------|--|
| Window count | ≥ 3 IC windows | Below this there is no walk-forward to speak of; returns <code>insufficient_windows</code> without evaluating anything else. |
| IC floor | latest window IC ≥ 0.02 | Matches <code>classifyEdgeIC</code> in <code>lib/edges/ic.ts</code> . A decayed edge does not get promoted on its history. |

| Gate | Rule | Why |
|--------------|--|--|
| t-stat | latest window's Newey-West $t \geq 2.0$ | Newey-West because overlapping forward-return windows make naive IID t-stats overstate significance. Latest — not max — see "Which t-stat" below. |
| DSR | $t_{\text{best}} - E[\max t \text{ over } S \text{ trials}] > 0$ | Bailey 2014: $E[\max t] \approx \Phi^{-1}(1 - 1/(2S))$. Testing more variants must not buy significance — this is why <code>variant_budget</code> is locked before the engine runs. |
| IC stability | latest IC > 0 and $\geq 50\%$ of earliest IC | An edge whose estimate halves as data is appended is not stable. This is NOT walk-forward — the windows overlap ~98%, see below. |

Note the interaction worth knowing: `variant_budget` is capped at 20 by the schema, and $E[\max t]$ at 20 trials is ≈ 1.96 — just under the 2.0 t-hurdle. So within the allowed budget range the DSR gate never binds on its own; it tightens the margin rather than rejecting outright. The rejection case is covered by test and only triggers above the schema ceiling.

Outcomes

- **Pass** — supersedes the incumbent active policy for that segment (the partial unique index allows exactly one non-superseded row per segment), then appends a new row. `verdict` is `baseline` when the segment had no incumbent and `variant` when one was superseded. `model_id` is the latest window's `formula_version`, falling back to `run_fingerprint`.
- **Fail** — HTTP 200 with `promoted: false` and machine-readable reason codes. A rejected promotion is a normal outcome, not an error, and nothing is written.
- When `experiment_id` is supplied, the resulting `policy_id` and `completed_at` are written back to `backtest_experiments` to close the lineage loop.

Auth is cron secret **or** an authenticated user; both land on the identical deterministic path, so there is no privileged variant. Tests live in `lib/gates/promotion-gate.test.ts`.

Which t-stat (measured against prod, 2026-07-27)

Three candidate statistics were computed on real `edge_ic_history` rows. Two are unusable and the choice is not cosmetic — it decides every promotion:

| Statistic | Result on prod data | Verdict |
|---|---|--|
| <code>max(t_stat)</code> across windows | <code>dma_trend_slope@20d = 2.83</code> | Rejected — cherry-picks the luckiest of N windows. Same edge's latest window reads 0.55 . This is the exact order-statistic bias the DSR gate exists to remove, so using it made the gate self-defeating. |

| Statistic | Result on prod data | Verdict |
|--------------------------|--|---|
| mean(IC) / SE(IC) pooled | An India edge with 3 windows scored t = 13.77 | Rejected — pooling assumes independent windows. These are ROLLING and overlapping, so the IC series is autocorrelated and the SE collapses. |
| latest window's t_stat | best US = 1.73 , best India = 1.04 | Adopted. Unbiased, no cherry-pick, uses one window of data. Underpowered, which fails closed. |

Correctly pooling overlapping windows (a Newey-West correction **across** windows, not within one) is the real fix and is deferred until there is enough non-overlapping history to justify the machinery.

First real run — zero promotions, and that is the correct answer

Evaluated against every market-wide edge/horizon pair with ≥ 3 windows:

| Market | Pairs evaluated | Pass IC floor | Pass $t \geq 2.0$ | Pass walk-forward | Promotable | Best latest t |
|--------|-----------------|---------------|-------------------|-------------------|------------|---------------|
| US | 33 | 18 | 0 | 15 | 0 | 1.73 |
| India | 24 | 1 | 0 | 3 | 0 | 1.04 |

Nothing promotes in either market. The binding constraint is the t-stat hurdle, and the cause is history length: US IC history starts 2026-07-08, so 6 "windows" sit on ~3 weeks of heavily overlapping data. `strategy_policies` is expected to stay empty until there is materially more non-overlapping history. An empty policy table is the gate working, not the gate broken.

The windows are NOT walk-forward folds

Measured in prod 2026-07-27. Every `edge_ic_history` row is an IC over `history_days = 1000`, and the six US windows span **16 calendar days** end to end — so the first and last windows share **~98.4%** of their data. They are the same backtest re-run weekly with the end date nudged, not out-of-sample folds.

Consequences, and what was done:

- The cross-window check is renamed `ic_stability_pass` (failure code `ic_stability_failed`). It measures whether the IC estimate holds as data is appended — **not** out-of-sample decay. The gate no longer claims a property it does not have.
- `strategy_policies.walk_forward_pass` (the DB column) keeps its name; renaming it needs a migration on an append-only governance table. It stores stability.
- The route now **dedupes by** `window_end`, newest run wins. US edges had 6 rows across only 4 distinct `window_end` values from 6 distinct `run_fingerprints`, with `universe_size` drifting $31 \rightarrow 32 \rightarrow 40$. Undeduped, a same-day re-run reads as fresh evidence and can lift `sample_n` over `MIN_WINDOWS` by itself.

- A span-based guard was considered and **rejected**: with 1000-day windows, genuinely disjoint history needs ~8 years. Waiting does not help either — four more weeks moves overlap from 98.4% to ~95.6%.

What is **not** affected: the Newey-West t -stat **within** a single window (~96 as-of dates). That is real evidence and remains the binding constraint.

The real fix is disjoint folds emitted by the IC engine, proposed in `features/walk-forward-ic-folds/FEATURE_ARCHITECTURE.md` — ****draft, not approved, not implemented****. Its own risk section notes it would likely make promotion **harder** (fewer as-of dates per fold), so the recommendation there is to build it once an edge is close to clearing the hurdle, not before.

-->

Current implementation and audit status

- The route reads one exact evidence horizon. Market-wide evidence is `segment_type='market', segment_value='all'`; sector evidence is explicitly sector-scoped. The production `edge_ic_history` constraint does not permit regime rows, so regime promotion has no evidence and fails closed.
- Duplicate `window_end` rows are collapsed newest-run-wins before evaluation.
- Interactive access is confirmed-owner-only; cron access requires the cron secret. There is no LLM on the path.
- Current gates are at least three distinct rolling windows, latest IC ≥ 0.02 , latest-window Newey-West $t \geq 2.0$, a trial-count-adjusted t margin above zero, and positive/non-halving endpoint IC stability.
- The expected-max- t subtraction is **not** the Bailey/Lopez de Prado Deflated Sharpe Ratio. It omits strategy-return sample length, skewness, and kurtosis. Existing `dsr` names are legacy/misnamed and must be repaired before promotion becomes active.

No edge promotes. Production has zero `strategy_policies` and zero `backtest_experiments`. The best latest US market-wide t -stat is 1.73. India's best latest observation is 1.57 in a Financials sector slice with only two distinct windows, so it fails the evidence-count gate.

Readiness policy correction (2026-09-03). The separate Edge readiness projection previously called windows independent at a flat five-calendar-day gap for every h5/h10/h20 label. That contradicted the fold builder's refusal of `stepSessions < horizonSessions` and counted overlapping forward returns as new evidence. Policy `edge-readiness.v2-horizon-spaced` now requires

$\text{ceil}(\text{horizon_sessions} \times 7 / 5)$ calendar days (7/14/28 respectively). Existing IC rows stay immutable; only the mutable readiness projection is reevaluated, so demotion to collecting is expected and truthful.

Blocking governance defects

The current route is scaffolding, not a production-grade promotion boundary:

1. Rolling windows share about 98.4% of their 1,000-day history.
2. The historical universe is a current-liquid snapshot, not point-in-time.
3. Missing experiment lineage assumes one trial, and experiments are not bound tightly enough to edge/formula/horizon/segment.
4. Supersede then insert is not transactional. The unique index prevents two active policies but cannot prevent zero active policies after insert failure.
5. `strategy_policies.walk_forward_pass` stores stability, and the mutation trigger does not protect every field described as immutable.

No consumer may treat a policy row as approved evidence until the revised `features/walk-forward-ic-folds/FEATURE_ARCHITECTURE.md` is approved and built. That design requires a frozen experiment, PIT universe and inputs, purged market-session OOS folds, aggregate OOS IC evidence, proper multiple-testing control, and one atomic service-role promotion RPC. Waiting for more overlapping weekly windows does not cure these defects.

Time-review outcome maturation (2026-09-03)

The time-review learner is descriptive and measure-only. For each exact-horizon observation it records matched outcomes for the incumbent next-session exit and the predeclared +5/+10-session candidates, including same-window benchmark return, MFE/MAE, retained-stop hits, replacement-candidate availability, and the incremental return versus baseline. Upgrade Path counts only market sessions with an exact review and both outcomes matured; legacy daily evaluations do not advance the gate. Twenty such sessions permits a review, not activation. A sealed market-local portfolio simulation with redeployment, costs, drawdown, turnover, multiple-trial correction, adverse-case review, and owner approval is still required before any time-stop policy change.

Source: docs\larch\KAIRO_S_CONSOLIDATED_ARCHITECTURE.md

Kairos / FinanceOS — Consolidated Architecture

Audience: a new engineer, reviewer, or collaborator who needs to understand the complete application from one document.
Reviewed: 2026-09-21
Status rule: “Active” means the code and production contract are present.
“Paper-only” means it can affect simulated books but never a broker. “Shadow” means evidence collection only. “Blocked” means a named prerequisite is missing.
Historical feature documents are design records; this document describes how the system is connected today.

How a finance professional should read this

Kairos uses familiar investment concepts, but records them more strictly than a typical brokerage screen. A **symbol** is an instrument ticker. A **signal** is a dated research decision. An **entry** is a buy execution. A **position** is open inventory. A **lot** is one accounting slice of an entry or partial exit. A **benchmark** is the market alternative for the same capital. A **shadow** is a research experiment that cannot affect an order. A **session** is a completed US, NSE or UTC crypto period, not merely the time a request ran.

The most important distinction is between a forecast and an allocation. A score of 78 means current evidence ranks a candidate highly; it does not mean a 78% return or an all-in instruction. The portfolio constructor still checks cash, concentration, liquidity, stop distance, correlation, controls and freshness.

1. Product purpose

Kairos is a personal AI-assisted quantitative investing operating system. It is not a chat bot that invents trades and it is not a single-factor stock scanner.

It runs a controlled loop:

```
market/provider evidence
↓
market-local research and deterministic scoring
↓
session-validated signals and decision evidence
↓
paper portfolio construction and simulated fills
↓
position monitoring, exits and reconciliation
↓
closed-trade labels, diagnostics and learning evidence
↓
validated shadows/challengers (never silent promotion)
```

The system supports separate US, India and crypto books. Paper comes before live execution. Any live order must pass broker, data, risk, protection, idempotency and owner-authorisation gates. LLMs can summarize persisted evidence, but they do not have authority to invent a score, target, stop, position size or live order.

2. System boundaries and technology

The web application is Next.js App Router with TypeScript and React. Supabase provides Postgres, Auth, Row Level Security, Vault, Realtime and pg_cron. Server routes use Supabase server/service clients; browser components never receive a service-role key. Owner-only APIs use the confirmed owner gate.

Production schedules run in Supabase pg_cron and call deployed API routes through the Vault-backed `kairos_call_agent` bridge. Local Windows schedules are useful for development but are not the production source of truth.

Main runtime areas:

- `app/dashboard/` and `components/dashboard/`: Investing workspace.
- `app/property/`: owner-recorded property/equity workspace.
- `app/capital-plan/`: capital profile and allocation workspace.
- `lib/data/`: provider adapters, caching and freshness contracts.
- `lib/scoring/`: deterministic equity and crypto scoring.
- `lib/trading/`: exits, ladders, geometry and order policies.
- `lib/brokers/` and `lib/robinhood-mcp.ts`: broker capability and execution adapters.
- `supabase/migrations/`: schema, RPCs, constraints and cron jobs.
- `features/`: feature-level architecture and decision records.
- `docs/arch/`: stable operational architecture chapters.

3. Workspaces and user interface

Investing workspace

`/dashboard` is the owner's operating console. Major routes are:

Route	Purpose
<code>/dashboard</code>	Portfolio summary, NAV/cash, macro banner, health and agent status
<code>/dashboard/research</code>	Symbol scores, dimensions, thesis and dated evidence
<code>/dashboard/research/[symbol]</code>	Price/score history and symbol-level research detail
<code>/dashboard/learning</code>	Champion/challengers, validation and Performance Truth
<code>/dashboard/agents</code>	Agent cards, topology and per-agent diagrams
<code>/dashboard/upgrade-path</code>	Shadow inventory, evidence, blockers and attribution status
<code>/dashboard/markets</code>	Macro, breadth, sectors and benchmark market data

Route	Purpose
/dashboard/smart-money	Options/insider/sentiment evidence and trade queue
/dashboard/india	NSE research, INR paper book and Kite surfaces
/dashboard/live-portfolio	Broker positions, snapshots and live-vs-benchmark charts
/dashboard/journal	Signal → fill → exit → outcome decision journal
/dashboard/mentor	Evidence-based coaching notes
/dashboard/backtest	Historical validation/replay outputs
/dashboard/scanner	Bounded US/India candidate screening
/dashboard/settings	Broker connections, model routing, controls, keys and maintenance

The System Reference surface is owner-only and reads a fixed allowlist. It is not a repository browser. The Agent diagrams are rendered from `public/agent-diagrams/system-map.json`, which is the topology source of truth.

Property and Capital workspaces

Property stores owner-entered properties, financing, carrying costs, evidence imports, market observations and shadow forecasts. It is not an automated AVM. Capital stores allocation profiles, capital decision runs and capital-rotation shadows. Neither workspace can silently place an investing order.

4. Markets, books and instruments

US book

USD paper portfolio, US equity/ETF universe and broker-compatible symbols. The ResearchAgent uses applicable fundamentals, technicals, sentiment, macro and insider evidence. US paper fills support fractional shares. Robinhood is the primary intended execution broker; Webull is a read-only research/broker surface until its trading contract is explicitly enabled.

India book

INR paper portfolio for NSE-compatible equity/ETF symbols. India uses market-local calendar/session handling, local provider evidence and Zerodha/Kite compatibility. India remains whole-share in the paper path unless an instrument contract says otherwise. US macro is never copied into India as if it were local evidence.

Crypto book

Crypto has an isolated USD paper pool. The approved initial paper basket is BTC-USD, ETH-USD and SOL-USD; broker-discovered pairs are recorded separately. Crypto uses public Coinbase/Kraken candles and quotes for research, Coin metadata

where available, and Robinhood MCP only for broker capability/execution truth.

Crypto does not reuse stock P/E, earnings, insider or equity-macro formulas.

Live crypto is disabled. The live path requires account/pair eligibility, fresh executable quotes, preview acceptance, protective-order proof and reconciliation.

5. Data architecture

Every input has a source, observed timestamp, market/session, provenance and quality state. Provider data is cached and quota-accounted. `price_cache` stores OHLCV bars with provider/basis/provenance metadata. Evidence caches prevent the research hot path from repeatedly spending provider quota.

Core data families:

- `agent_signals`: session-validated research decisions and score dimensions.
- `signal_score_history` / decision observations: dated score history and feature snapshots used for divergence and learning diagnostics.
- `paper_portfolio`, `paper_positions`, `paper_trades`: isolated paper books, positions, lots, partial exits and realized outcomes.
- `paper_order_events` / decision journal: immutable event and rationale trail.
- `live_account_snapshots`, `live_performance`, broker order tables: broker truth, never inferred from a paper ledger.
- benchmark observations and scorecards: market-local benchmark curves and expected-session freshness state.
- learning, shadow, validation and attribution tables: append-only evidence for proposed changes.
- System Health tables: failures, warnings, owner actions and auto-expiry state.

Data contracts fail closed. Missing or stale evidence becomes an explicit refusal, not a fabricated zero. Database RPCs serialize cash/position changes and enforce the money-path rules transactionally.

6. Agents and scheduled pipeline

ResearchAgent

Runs market-local research after the relevant session boundary. It gathers a bounded universe, fetches applicable evidence, computes deterministic dimensions, writes signal observations and creates pending signals. It never directly buys.

DeepSeek/alternate model lane

The LLM router can run alternative research prose/model assignments. Model output is tagged and compared for paper outcomes; model prose cannot bypass deterministic score or risk contracts.

PaperTrader

Claims pending signals and performs: market-control check, kill-switch check, session/fresh-price check, score threshold, cash, name/sector limits, duplicate and anti-pyramiding checks, broker-compatible symbol check, and transactional fill. The crypto paper trader is a separate route and RPC mandate branch.

PositionMonitor

Runs after the market's completed bar. It refreshes current marks, high/low touch evidence, highest-price ratchets and exit decisions. A settled bar high may trigger a target even when the close recovers. If stop and target conflict, the policy remains conservative and stop precedence is preserved.

LearnerAgent

Reads mature, non-tainted closed outcomes and produces summaries/challengers. It does not mutate weights simply because a small in-sample result looks good. Promotion requires declared samples, purged/out-of-sample validation, provenance, and owner-approved state transitions.

MacroSentinel and event agents

MacroSentinel creates an advisory market regime from provider-backed indicators. It does not silently halt or throttle trading. Earnings, event maturation, news/sentiment, insider, catalyst and risk-tier agents write evidence or shadows with explicit availability and timestamps.

Supporting agents

Theme/Edge discovery, evidence shadow, evidence cohort, price prewarm, benchmark collectors, holding-risk, broker keepwarm, validation sweep and system-health workers are separate scheduled jobs. Each route has a bounded runtime and reports its own persistence/failure state.

7. Scoring model

The equity composite combines applicable dimensions: technical, fundamental, sentiment, macro and insider. Instrument-aware applicability prevents a stock-only

fundamental formula from masquerading as ETF evidence. A shared weighted scorer validates scores, weights and availability masks; unavailable dimensions do not silently become zero-positive evidence.

Technical inputs include RSI, EMA relationships, trend, volume confirmation and breakdown veto. EMA-200, MACD, relative strength and ADX may be measured shadows until their market-local IC and forward-shadow gates clear. Fundamental inputs are provider-backed and event-aware where possible. Sentiment and insider inputs carry their own coverage flags.

The score threshold creates an eligibility candidate, not a guaranteed return. Dimension rank IC, t-stat and code-version IC are monitored by market, cohort, horizon, label maturity, qualifying sessions and overlap-adjusted effective sample size. Current evidence is insufficient for automatic score-weight promotion.

Crypto score v1 is separate and deterministic: trend/relative strength, structure, volatility/regime and liquidity/execution. Event/news/network features are measure-only until they have timestamped, point-in-time coverage and predictive evidence.

An **IC** is the rank correlation between a score and later return over a declared horizon. A positive IC means higher-ranked names tended to return more in that cohort; it is not a guarantee. The t-stat measures uncertainty. Kairos also tracks qualifying sessions, label maturity and overlap-adjusted effective sample size so overlapping daily observations are not mistaken for independent bets.

8. Portfolio construction and sizing

The constructor applies finite cash, gross exposure, name, sector, correlation, volatility and risk-budget constraints. The live route uses broker equity snapshots and its own protective gates; it must not silently use paper NAV. A cash balance is capacity, not an instruction to buy. Extra cash is deployed only after fresh research, current rank, risk and correlation checks.

The offline sizing replay compares equal planned allocation with equal stop-risk allocation on the same frozen realized opportunities. It models costs, caps, fractional US quantities, whole India quantities, partial exits and sampled drawdown. It cannot claim the globally optimal size or assess missed opportunities.

The historical replay remains evidence-limited: India has 20 distinct symbol roots without canonical price history, and partial-lot/stop provenance still needs reconciliation. The separate top-up experiment is not a production feature.

For a stop-risk diagnostic, the planned amount is:

```
risk fraction = (entry price - stop price) / entry price  
planned notional = NAV × risk budget / risk fraction
```

Cash, name/sector/gross caps, correlation, share increments and transaction costs then bound that amount. Cash is capacity, not a buy signal; unused cash can mean that no candidate passed the current evidence and risk gates.

9. Exit geometry

Stops and targets are recorded with entry provenance, geometry version and observed inputs where available. The ladder supports full target, partial target plus runner protection, trailing protection, structural invalidation and score/direction exit. The no-clock exit policy is intended to let valid winners run, but its historical replay and intrabar semantics remain monitored carefully.

Crypto geometry is native: structural invalidation and ATR/realized-volatility floor, expected costs, liquidity and maximum-loss bound. No unconditional calendar exit is assumed. Paper crypto exits use completed OHLC bars and adverse resolution for ambiguous barriers.

Worked stock trade

For a \$10,000 book, a \$100 entry, \$95 stop and \$115 target, an 8% allocation arm plans \$800 before caps and costs. A 0.5% stop-risk arm sees 5% per-share risk and plans \$1,000 before caps. If the bar high reaches \$115 but closes at \$112, target evidence can trigger the configured partial sale while the close remains the conservative mark. If the same bar's low touches \$95, stop precedence resolves the ambiguity adversely. The outcome is stored at lot level.

10. Benchmarks and charts

US and India benchmark collectors are market-local. Each run records expected session, observed session, provider, benchmark and retry state. A run is done only when every enabled benchmark reaches that market's expected completed session; otherwise it is partial/error. Freshness alerts resolve only after all configured benchmarks advance.

Charts consume canonical persisted observations. They must not mix sessions or use portfolio data from one market with another market's benchmark. Benchmark-relative performance is descriptive until a matched, frozen attribution experiment proves an upgrade-path effect.

Crypto displays BTC buy-and-hold and a frozen equal-weight eligible reference as separate benchmarks; neither is presented as a universal market proxy.

11. Learning, strategies and Upgrade Path

Upgrade Path is an evidence registry, not a list of promises. A path can be:

- **Operational:** proves a collector, guard or persistence contract.
- **Shadow:** records an alternative signal/strategy without changing behavior.
- **Paper cohort:** runs a controlled paper comparison.
- **Matched replay:** compares baseline and variant on the same population/window.

Measured attribution requires common market/window/population, frozen code/config hashes, net cost basis, independent sessions, benchmark, drawdown, turnover, confidence interval and exact variant-minus-baseline arithmetic. Aggregate portfolio P&L cannot be relabelled as proof of one feature. The attribution ledger is empty when no path has a valid producer; that is safer than manufacturing rows.

Capital rotation, external strategies, sector relative strength, catalyst/risk tiers, model comparisons, time/ATR/volatility exit alternatives and leveraged ETF strategies are shadows or proposals until their gates pass. TQQQ/SQQQ/SOXL/SOXS require dedicated volatility/leverage-aware policies and must not inherit ordinary equity targets or stops.

Strategy lifecycle in investment terms

1. Declare universe, signal, holding/exit policy, costs, sizing and benchmark.
2. Run a shadow without changing the incumbent.
3. Wait for labels to mature and evaluate purged/out-of-sample folds.
4. Compare baseline and variant on the same opportunities and sessions.
5. Promote only through an owner-approved state transition.

“Ready” means the evidence contract is complete, not that the strategy wins every week.

12. Broker and live safety

Robinhood MCP is capability-probed for accounts, pairs, quotes, previews, orders, positions and reconciliation. Webull is not assumed to provide crypto execution. Zerodha/Kite is the India execution contract. Broker support is checked for the exact symbol before a live order.

Live order flow must verify: owner/live switch, active account, broker connection, fresh account snapshot, pair eligibility, fresh executable quote, score/geometry, cash/buying power, kill switches, daily loss/drawdown/rate limits, idempotency and protective-order state. A live position without verified protection raises a critical issue and blocks further buys. Viewer accounts can read only the routes explicitly granted to them and cannot mutate owner settings.

13. System Health

Health separates critical failures from informational refusals and owner actions. Examples of critical states are provider quota breaches, stale required market data, failed persistence, broker-token failure, benchmark non-advancement and an unprotected live position. A candidate refused for insufficient history is not by itself a system outage; it is recorded as evidence.

Investigate in this order: expected session → provider/provenance → database row → route response → cron run → UI. A green build or rendered card is not proof that a money-path action succeeded.

14. Production schedules (conceptual)

Schedules are market-local and can have seasonal duplicate invocations that exit before work. The important ordering is:

```
provider/evidence prewarm
→ research and discovery
→ paper-entry attempt
→ post-close price prewarm
→ PositionMonitor
→ label maturation / diagnostics
→ weekly validation and shadow review
```

Crypto runs daily because the market is 24/7, while stock research and paper execution follow US/India sessions. Every job is bounded, idempotent and expected to write an agent_runs heartbeat or an explicit refusal/error.

15. Documentation and change governance

Use this document for the complete connected view. Use:

- docs/arch/ for stable operational chapters;
- features/*/FEATURE_ARCHITECTURE.md for feature contracts and non-goals;
- features/*/IMPLEMENTATION_RESULT.md for shipped deviations;
- PROJECT_DECISIONS.md for owner approvals and reversals;
- WORK_LOG.md for delivery evidence and open work;
- public/agent-diagrams/system-map.json for topology.

Any change to a provider, agent, score dimension, applicability rule, schema/RPC, cron, benchmark, money-path gate, learning/promotion rule or broker contract must update this document and its detailed feature/chapter owner in the same change.

The safest interpretation of any missing evidence is “not proven yet.” Kairos is designed to keep researching and measuring without silently turning a shadow into capital.

16. Finance glossary

Term	Kairos meaning
NAV	Cash plus marked position value for one isolated book
Gross exposure	Total long notional divided by NAV
Stop distance	Entry-to-stop percentage used for risk sizing
Slippage	Difference between expected and simulated/actual fill
IC	Rank correlation between score/rank and forward return
t-stat	Uncertainty statistic for an estimated effect
Purged fold	Validation split with a gap preventing label leakage
Point-in-time	Information actually available at the decision timestamp
Cohort	Explicit population being evaluated
Attribution	Evidence that a named change caused a measured difference
Shadow	Non-authoritative alternative collecting evidence only
Champion	Currently approved strategy configuration
Kill switch	Control blocking new risk while allowing risk reduction

17. Abbreviations

Abbreviation	Expansion	Meaning in Kairos
API	Application Programming Interface	A machine-readable endpoint used by Kairos or a provider
ATR	Average True Range	Volatility measure used in crypto/exit-geometry research
AV	Alpha Vantage	A market-data provider; its quota is centrally enforced
BO	Bombay Stock Exchange	Exchange suffix used by some Indian symbols
DB	Database	Supabase Postgres persistence layer
DDL	Data Definition Language	SQL that creates or changes tables, indexes or functions
DQ	Data Quality	Completeness, freshness and validity checks on evidence
EOD	End of Day	Completed market-session data, not an intraday partial bar
EMA	Exponential Moving Average	Price trend indicator used by technical research
ETF	Exchange-Traded Fund	A listed fund traded like a stock; it has different applicable dimensions
ET	Eastern Time	US market clock used for NYSE/Nasdaq schedules
FDR	False Discovery Rate	Multiple-testing control for research findings

Abbreviation	Expansion	Meaning in Kairos
GDELT	Global Database of Events, Language and Tone	News/event source used in bounded evidence collection
HAC	Heteroskedasticity and Autocorrelation Consistent	Robust uncertainty adjustment for dependent returns
IC	Information Coefficient	Rank correlation between a signal and later return
ID	Identifier	Stable key linking a signal, event, lot or run
INR	Indian Rupee	Currency of the India paper/live book
IST	India Standard Time	India market clock
JSON	JavaScript Object Notation	Structured payload format used in APIs and evidence fields
JWT	JSON Web Token	Auth token used by Supabase sessions
LLM	Large Language Model	Model used for evidence summaries or controlled agent prose
MACD	Moving Average Convergence Divergence	Technical momentum indicator; currently measure-only unless gated
MAE	Maximum Adverse Excursion	Worst excursion against an entry before exit
MCP	Model Context Protocol	Tool bridge used for broker capability and account operations
MFE	Maximum Favorable Excursion	Best excursion in favor of an entry before exit
NAV	Net Asset Value	Cash plus marked value of an isolated portfolio
NSE	National Stock Exchange of India	Primary exchange/calendar context for the India book
NIFTY	NSE Nifty 50	India benchmark index used in market-local comparisons
OCO	One-Cancels-the-Other	Paired protective orders; never assumed available without broker proof
OHLCV	Open, High, Low, Close, Volume	Standard candle/bar fields
OOS	Out of Sample	Data not used to fit or choose a strategy
P&L	Profit and Loss	Realized or unrealized financial result
PIT	Point in Time	Evidence available no later than the decision timestamp
PKCE	Proof Key for Code Exchange	OAuth authorization protection used in broker connection flows
R:R	Risk-to-Reward	Relationship between stop distance and target distance
RLS	Row Level Security	Postgres policies limiting which users can read/write rows
RPC	Remote Procedure Call	Database function invoked atomically by an API route
RSI	Relative Strength Index	Bounded momentum indicator used in technical research
SQL	Structured Query Language	Database query language
T-stat	t-statistic	Estimate divided by its uncertainty; used to qualify evidence
UI	User Interface	Visible dashboard/page layer
USD	United States Dollar	Currency of the US and crypto paper books
UTC	Coordinated Universal Time	Canonical scheduler and crypto-session clock
Vercel	Vercel deployment platform	Hosts the deployed Next.js application
W/L/BE	Win / Loss / Breakeven	Closed-trade outcome categories

Source: features_template\FEATURE_ARCHITECTURE.md

Feature Architecture: <Feature Name>

Status

Architecture status: Draft

Architecture approved: No

Approved scope: None

Approved date: None

Implementation allowed: No

Feature Purpose

Describe what this feature helps the user or system accomplish.

User/System Questions This Feature Answers

- Question 1
- Question 2

Scope

This feature includes:

- Item 1

Non-Goals

This feature does not include:

- Item 1

Current Behavior

Describe existing behavior, if any.

Proposed Behavior

Describe proposed behavior.

User Journey / System Flow

1. Step one
2. Step two
3. Step three

Screen / Page / Module Inventory

List every screen, page, sheet, modal, component, API endpoint, job, or service involved.

UI Architecture

Layout

Components

Sheets / Modals / Drawers

Navigation

Empty State

Loading State

Error State

Success State

System Architecture

Modules

API Contracts

Data Models

Auth / Permissions

Error Handling

Data Architecture

- Required data:
- Optional data:
- Mock vs real vs derived:
- Persistence:
- Validation:

Files Likely To Change

- File/folder 1

Files / Behavior That Must Not Change

- Constraint 1

Acceptance Criteria

- Criterion 1
- Criterion 2

Approval

Architecture approved: No
Approved scope: None
Implementation allowed: No

Source: features\advanced-learning\FEATURE_ARCHITECTURE.md

Advanced Learning — Deflated Sharpe + PBO, Regime-Conditioning, Meta-Labeling

Status: DRAFT (design only, unapproved) . No code, no migration, no deployment.
Last updated: 2026-07-15
Update this file when: the DSR/PBO promotion-gate math, the regime-conditioning weighting method, or the meta-labeling gate contract changes; keep it aligned with docs/arch/09-learning-loop.md (Validation Engine, promotion, Performance Truth, P1 gate), lib/validation/engine.ts, lib/validation/calibration.ts, features/learning-core/FEATURE_ARCHITECTURE.md, and features/decision-review/FEATURE_ARCHITECTURE.md.

1. One-line intent

Upgrade the learning/promotion **brain** with the canonical López de Prado (*Advances in Financial ML*) + academic safeguards we are missing, in priority

order:

1. **Deflated Sharpe Ratio (DSR) + Probability of Backtest Overfitting (PBO)** as a hard, deterministic layer in the **promotion gate** — the #1 overfitting guard.
2. **Regime-conditioning** — a weight *adjustment* conditioned on the current `macro_regime` state, learned per regime cohort with its own sample floor. Evidence-conditioned, **not** a hardcoded bull/bear switch (respects the locked CLAUDE.md pushback rule).
3. **Meta-labeling** — a secondary deterministic model that decides *whether to ACT* on a primary signal (a precision filter), trained on `decision_observations × observation_labels`. Measure-only → shadow → gated.

All three **extend** the existing deterministic Validation Engine / calibration / learner path. None introduces an LLM on the money path, and none creates a parallel truth layer. Performance Truth remains the NAV authority; the LearnerAgent + owner promotion remain the **only** things that change champion weights.

2. Hard boundaries (non-negotiable)

- **Deterministic, no LLM sets any number.** DSR, PBO, regime weight-deltas, and the meta-label model are pure arithmetic / gradient-descent over the ledger — same posture as `lib/validation/engine.ts` (moving-block bootstrap, seeded PRNG) and `lib/validation/calibration.ts` (batch GD logistic, fixed iterations). An LLM may *narrate* the numbers in prose, never produce them.
- **Extends, does not replace.** DSR/PBO are added terms on the existing `validateChallenger` path and are recorded on the existing `validation_experiments / strategy_evaluations` rows. Regime-conditioning is a bounded **delta on top of** the champion genome weights, not a second weight store. Meta-labeling is a *gate flag*, never a score.
- **No parallel truth layer.** Every number is derived from the append-only `decision_observations` (059) × `observation_labels` (060) ledger and the existing `macro_regime` (028) table — the same records LearnerAgent, the Validation Engine, and Performance Truth already read. Performance Truth (`lib/evaluation/run-evaluation.ts`) stays the book-truth / NAV authority; nothing here recomputes NAV.
- **Per-market / per-currency, never mixed.** US cohorts use USD prices + SPY-neutral returns; India cohorts use INR + ^NSEI-neutral. A regime cohort, a DSR trial count, or a meta-label training set is always scoped to a single market. India evolves on its own gate exactly as today (`docs/arch/09-learning-loop.md` "Per-market

independence").

- **Measure-only** → **shadow** → **gated rollout**. Every method ships OFF, computing and recording its verdict without affecting selection/sizing/promotion. Each is activated by a **separate** owner switch, and only after its own sample floor is cleared. The riskiest of the three (meta-labeling gating entries) is the last to ever gate.

- **The champion promotion path stays owner-only and fail-closed**. DSR/PBO can only make promotion **stricter** (add a required PASS), never auto-promote. Meta-labeling can only **withhold** an entry that the deterministic score already permitted — it can never **create** an entry. Regime-conditioning can only nudge a weight within a bounded band; it can never override the entry threshold or the "sum-to-1.0" invariant.

- **Respects the locked "no regime switching" rule**. Regime-conditioning is a smooth, evidence-weighted blend (shrink-to-champion by cohort sample size), not an `if regime == 'red' then bearMode` branch.

3. What already exists (verified against code) — the substrate

Building block	Where	What we reuse
Walk-forward folds (purged + embargoed, anchored)	<code>lib/learning/dataset.ts</code> <code>walkForwardFolds()</code>	The exact leak-free splitter. PBO's combinatorial split reuses the same purge/embargo constants.
Labeled dataset join	<code>lib/learning/dataset.ts</code> <code>loadLabeledDataset()</code>	<code>decision_observations × observation_labels</code> , benchmark-neutral returns at 2/5/10/20d. Meta-label training data + regime cohorting both read this.
Challenger replay objective	<code>lib/validation/engine.ts</code> <code>objectiveTerm()</code> / <code>scoreRow()</code>	Per-row benchmark-neutral log-growth under a weight set. DSR consumes the per-fold return series this already produces.
Bootstrap + seeded PRNG	<code>lib/validation/engine.ts</code> <code>blockBootstrap()</code> , <code>mulberry32()</code>	Determinism pattern; DSR's variance/skew/kurtosis of the return stream and PBO's rank logic reuse the same seed discipline.
Existing promotion gates	<code>lib/validation/engine.ts</code> (<code>p_improvement ≥ 0.80</code> , CI-low, <code>n_effective ≥ 12</code> , <code>≥3/5</code> folds) + <code>activate_strategy_shadow</code> (170) + atomic promotion (161)	DSR/PBO are added as further AND-conditions; the fail-closed PASS row + owner-only promote path are unchanged.
Calibrated P(win) logistic + OOS/ECE fail-closed gate	<code>lib/validation/calibration.ts</code>	Meta-labeling is architecturally a <i>*sibling*</i> of this model: a second logistic, same GD, same walk-forward OOS acceptance discipline (<code>acceptCalibrationOOS</code>), different target (<code>act-vs-skip</code> precision).
Regime state	<code>macro_regime</code> (028): <code>week_of</code> , <code>regime ∈ {green,yellow,orange,red}</code> , <code>danger_score</code> , read via macro-sentinel	Regime label per observation date. Exists but weights are NOT conditioned on it today — this feature is the first consumer for conditioning.
Experiment ledger	<code>validation_experiments</code> (via <code>recordExperiment()</code>), <code>strategy_evaluations</code> (134, append-only)	DSR/PBO verdicts are new columns on these rows. No new provenance store.
EdgeIC statistical toolkit	<code>lib/edges/ic.ts</code> (Spearman, Newey-West SE, t-hurdle)	Reference implementation for the significance discipline; PBO logit and DSR t-stats live beside it, not on the money path.

Building block	Where	What we reuse
Aggregate/per-decision measurement surfaces	features/edge-factor-discovery, features/decision-review	Sequencing peers — DSR/PBO/meta-label verdicts surface through the same Performance Truth / Decision-Review panels, not a new dashboard.

Key implication: we are not inventing infrastructure — DSR/PBO/meta-labeling are

additional deterministic computations over data and splitters that already exist.

The build is a math + gating layer, not a new pipeline.

4. Method 1 — Deflated Sharpe Ratio + PBO in the promotion gate (priority #1)

4.1 Why

The current gate (`validateChallenger`) tests whether a challenger beats the champion on held-out folds with bootstrap significance. It does **not** correct for

selection bias: `LearnerAgent` proposes many challengers over time, and the best of N trials will look good by luck alone. DSR and PBO are the canonical López de Prado corrections for exactly this.

4.2 Deflated Sharpe Ratio (DSR)

DSR adjusts an observed Sharpe for (a) the **number of trials** that were run to find it, (b) the **non-normality** (skew/kurtosis) of the return stream, and (c) the **track length**. It returns a probability that the true Sharpe > 0 given the search.

Deterministic recipe (all inputs already available or trivially derivable):

- 1. Return stream.** For the challenger, take the per-test-row objective series already produced in `validateChallenger` (the challenger arm of `pairedDiffs`, i.e. `objectiveTerm(challengerWeights, row)` across concatenated OOS test rows). This is the leak-free, walk-forward return series — no new replay needed.
- 2. Observed Sharpe** $SR_{\hat{}}$ = mean/std of that series (annualization factor is recorded but DSR works on the raw per-observation SR; we store both).
- 3. Skew γ_3 and kurtosis γ_4** of the same series (deterministic moments).
- 4. Number of trials N** = count of challengers proposed **for this market** in the current learning episode / trial family (see 4.4 — this is the load-bearing input and the biggest measurement decision).
- 5. Expected max Sharpe under the null** SR_0 = the López de Prado closed form for the expected maximum of N independent standard-normal Sharpe estimates (uses the Euler–Mascheroni / Gaussian-quantile approximation — pure arithmetic).
- 6. DSR** = $\Phi\left(\left(SR_{\hat{}} - SR_0 \right) \cdot \sqrt{T-1} / \sqrt{1 - \gamma_3 \cdot SR_{\hat{}} + ((\gamma_4-1)/4) \cdot SR_{\hat{}}^2} \right)$

where T = track length (number of OOS observations), Φ = standard normal CDF.

7. **Gate:** require $DSR \geq DSR_MIN$ (v0 default **0.95**, i.e. 95% confidence the deflated Sharpe is positive) — a **tunable const beside** MAX_OOS_ECE in the calibration module, documented as needing prospective tuning.

4.3 Probability of Backtest Overfitting (PBO)

PBO (Bailey–Borwein–López de Prado, CSCV — Combinatorially Symmetric Cross-Validation) estimates the probability that a configuration selected as best **in-sample** ranks **below median out-of-sample** — the direct probability that our selection is overfit.

Deterministic recipe reusing the existing splitter:

1. Partition the OOS observation matrix into S contiguous, **purged/embargoed** sub-blocks using the **same** purge (horizonDays) and embargo the walk-forward splitter already applies (`lib/learning/dataset.ts`). S even (v0 **$S = 8$** if sample allows, else the largest even S meeting the per-block floor).
2. For each of the $C(S, S/2)$ combinations, designate half the blocks **IS** and half **OOS**.
3. Score the **candidate family** (champion + all challengers in the trial family, §4.4) on IS; pick the IS-best.
4. Record that config's **OOS rank** among the family; compute its logit $\lambda = \ln(\text{rank} / (M+1 - \text{rank}))$.
5. **PBO** = fraction of combinations where the IS-best has $\lambda \leq 0$ (below-median OOS). Require $PBO \leq PBO_MAX$ (v0 default **0.20**).

Because CSCV is combinatorial, cap S so $C(S, S/2)$ stays bounded ($S \leq 10 \rightarrow \leq 252$ combinations) — deterministic and cheap.

4.4 The single load-bearing input: trial count N / the candidate family M

DSR needs N (how many strategies were tried) and PBO needs the **family** of configurations scored together. Both quantify selection bias, so they must count the **same search**. Design decision (v0):

- The **trial family** = all `strategy_versions` rows for this market with `proposed_by='learner'` since the current champion was promoted (i.e. every challenger the learner floated in this episode), **plus** the champion itself.
- N = size of that family. This is directly queryable — no new state — and it is a **conservative** (never-underestimated) count, which is the safe direction for an overfitting guard.
- Recorded on the experiment row (`n_trials`, `trial_family_ids`) so the verdict is

reproducible and auditable.

This is the single riskiest assumption — see §9. If N is undercounted, DSR is
too permissive; the conservative "count every learner challenger this episode"
choice deliberately errs strict.

4.5 Where it plugs in (deterministic, additive)

validateChallenger today returns PASS iff the four existing conditions hold. Add two AND-conditions, evaluated **only when the sample floor is met**:

```
passed = existing_gates_pass
        AND (n_effective >= DSR_PBO_MIN_SAMPLE ? (DSR >= DSR_MIN AND PBO <= PBO_MAX) : true*)
```

* When the sample floor is **not** met, DSR/PBO are ****computed and recorded but not enforced**** (measure-only) — the harness runs from day one, the gate activates only when data matures. The floor mirrors the existing discipline: the engine already refuses <60 observations and `n_effective < 12`; DSR/PBO enforcement adds a higher floor (v0 **n_effective** ≥ 20, matching the Performance Truth 20-trade honesty rule and the P1 gate). Owner flips enforcement per market via a `strategy_validation_automation-style switch (dsr_pbo_enforced)`.

New fields on `validation_experiments` (additive migration, not in this doc): `dsr`, `sr_hat`, `sr0_expected_max`, `n_trials`, `pbo`, `cscv_blocks`, `trial_family_ids`, `dsr_pbo_enforced`, `dsr_pbo_pass`. These **summarize**; they do not replace `passed/p_improvement`.

Boundary: DSR/PBO can only ***block*** a promotion the old gate would have allowed. They never promote, never size, never touch cash. The atomic owner-only promotion RPC (161) still requires a PASS row and an owner click.

5. Method 2 — Regime-conditioning (priority #2)

5.1 Intent & the locked-rule constraint

Learn that (e.g.) ***technical*** weight should be shrunk and ***macro*** weight lifted when `macro_regime='red'`, **without** a brittle bull/bear mode switch. The CLAUDE.md rule is explicit: no explicit regime-detection ***switching***. So regime-conditioning is a **smooth, evidence-weighted, bounded delta** on the champion genome weights — it degrades gracefully to "just use the champion weights" whenever the regime cohort is thin.

5.2 Cohorting

- Stamp each labeled observation with the `macro_regime.regime` in effect on its `ts` (join `decision_observations.ts` → `macro_regime.week_of`, market-local). This is a **read** — `macro_regime` already exists; nothing new is written to the ledger.
- Group the labeled dataset into per-market **regime cohorts** (`{green, yellow, orange, red}`). To avoid 4× sparsity, v0 collapses to two evidence bands — **calm** (`green|yellow`) and **stress** (`orange|red`) — a data-driven coarsening, not a hand-tuned bull/bear branch; the band boundary is a documented const, and the design keeps the full 4-level cohort available for when volume supports it.

5.3 The conditioned weight = champion ⊕ shrunk delta

For each cohort `c` and market, fit the **same calibration logistic** (`lib/validation/calibration.ts` machinery) on the cohort's rows to get the dimension-importance implied by that regime. Convert to a candidate weight vector `w_c`, then **shrink toward the champion** by cohort evidence:

```
w_effective(c) = normalize( (1 - α_c) · w_champion + α_c · w_c )
α_c = min(α_max, n_c / (n_c + K)) # James-Stein-style shrinkage
```

- `n_c` = cohort sample size; `K` = shrinkage constant (v0 **K = 50**); `α_max` caps the maximum deviation from champion (v0 **0.25** — a regime can move a weight by at most a quarter of the way to its cohort-implied value).
- **Sample floor:** when `n_c < REGIME_MIN_SAMPLE` (v0 **30**, matching the calibration `MIN_OOS_SAMPLES`), `α_c = 0` → **exactly the champion weights**. Thin regimes are a no-op, not a guess.
- The sum-to-1.0 genome invariant and the `position_size_pct` cap are re-imposed after the blend; the entry threshold is **never** conditioned (only dimension weights).

This composes with the champion genome as a **pure function** of (champion weights, current regime, cohort evidence). There is no second weight store and no mode flag — the champion remains the single governance object; regime-conditioning is a deterministic transform applied at scoring time, recorded for audit.

5.4 Validation & rollout

- A regime-conditioned challenger is validated by the **same** `validateChallenger` path (including the new DSR/PBO layer) — regime-conditioning is just a different way of producing `weights_snapshot`-equivalent scoring, so it must clear the identical gate before it can ever affect live selection.
- Ships **measure-only**: the conditioned weights are computed and logged to a new `regime_weight_delta`s provenance row (per market × cohort × fit date) but scoring

uses champion weights until the owner enables `regime_conditioning_enabled` per market — and only after the cohort floors are met.

- India and US condition on their **own** `macro_regime` history independently.

Boundary: the delta is bounded (α_{max}), evidence-gated ($\alpha_{\text{c}} \rightarrow 0$ when thin), and re-normalized — it can nudge, never flip. No `if regime` branch selects a strategy; the champion is always the base.

6. Method 3 — Meta-labeling (priority #3)

6.1 Intent

Primary model = the existing deterministic `analyst_score` + entry threshold (decides **side / direction**). Meta-labeling adds a **secondary** deterministic model that decides **whether to act** on a primary BUY — a precision filter that suppresses low-precision entries. It **never** overrides the deterministic score; it only gates **entry eligibility**. (López de Prado, Ch. 3.)

6.2 Model

- **Training data:** `decision_observations` × `observation_labels` where the primary signal was a candidate entry (`entry_eligible = true / score ≥ threshold`). Label = 1 if that entry was a **win** (benchmark-neutral return > 0 at the mandate horizon), 0 otherwise. This is exactly the `loadLabeledDataset` output filtered to would-have-acted rows.
- **Features:** the 5 dimension scores + deterministic context already on the row (availability mask, score-vs-threshold margin, regime band from \$5, `rank_pct` from cross-sectional rank if present). **No look-ahead** — same walk-forward discipline.
- **Fit:** the **same** batch-GD logistic + standardization as `lib/validation/calibration.ts`, producing $P(\text{win} \mid \text{acted})$ — a **precision** estimate, distinct from the sizing $P(\text{win})$ (different training population: acted-only, not all decisions).
- **OOS acceptance:** reuse `acceptCalibrationOOS` verbatim (fail-closed: <30 OOS rows, degenerate class, or $\text{ECE} > 0.1 \rightarrow$ **rejected**, model not stored). Stored (when accepted) as a **new** `model_artifacts.kind='meta_label_logistic'` row — same table, same upsert discipline as `pwin_logistic`.

6.3 The gate (three-stage rollout, the last method to ever act)

1. **Measure-only:** compute P_{meta} for every candidate, record it on the decision

observation / Decision-Review surface, and report **what it would have suppressed** and the counterfactual precision lift. Changes nothing.

2. **Shadow**: a state='shadow_paper' strategy variant applies the meta-gate and records its would-be book vs the champion's — a free dress rehearsal (exactly the existing shadow_paper mechanism, docs/arch/09-learning-loop.md "Shadow decisions"). No fills, no cash.

3. **Gated rollout (owner-approved)**: only after shadow shows a real precision lift over a sufficient sample, and only behind an owner switch (meta_label_gate_enabled per market), the meta-gate may **withhold** an entry the primary model permitted: $\text{act} = \text{primary_entry_eligible} \text{ AND } (\text{P_meta} \geq \text{META_MIN})$. META_MIN is a tunable const with a min-sample floor (v0 ≥ 50 acted observations in the training population before the gate can enforce).

Boundary: meta-labeling is strictly **subtractive** — it can only turn an eligible entry OFF, never turn an ineligible one ON, never change direction, never change size, never write a weight. It is the highest-risk method (it directly withholds trades), so it is sequenced last and gated hardest.

7. C4 sketch

Context

```
flowchart TB
    OWNER[Owner\npromotes / flips switches] --> GATE
    LEARNER[LearnerAgent\nweekly, proposes challengers] --> GATE
    subgraph BRAIN[Advanced Learning brain – deterministic, no LLM]
        GATE[Promotion Gate\n+ DSR + PBO]
        REGIME[Regime-conditioner\nchampion + bounded delta]
        META[Meta-label filter\nact / skip precision]
    end
    end
    LEDGER[(decision_observations\nx observation_labels)] --> BRAIN
    MACRO[(macro_regime)] --> REGIME
    GATE --> EXPR[(validation_experiments\n+ dsr/pbo cols)]
    BRAIN --> PT[Performance Truth\nNAV authority – unchanged]
    GATE -. owner-only promote .-> CHAMPION[strategy_versions\nchampion]
```

Container / component

```
flowchart LR
    DS[lib/learning/dataset.ts\nwalkForwardFolds – REUSED] --> DSR[lib/validation/deflated-sharpe.ts\nDSR + expected-max-Sharpe]
    DS --> PBO[lib/validation/pbo.ts\nnCSCV logit]
    DSR --> ENG[lib/validation/engine.ts\nvalidateChallenger – EXTENDED]
    PBO --> ENG
    CAL[lib/validation/calibration.ts\nGD logistic + acceptCalibration00S – REUSED] --> REG[lib/learning/regime-weights.ts\ncohort fit + shrink]
    CAL --> ML[lib/learning/meta-label.ts\nact/skip precision model]
    MACRO[(macro_regime)] --> REG
    ENG --> EXPR[(validation_experiments)]
    REG --> RWD[(regime_weight_deltas – new, provenance)]
    ML --> MA[(model_artifacts kind=meta_label_logistic)]
    ENG -. records .-> SE[(strategy_evaluations 134 – append-only)]
```

New code (all deterministic, unit-testable pure functions like the existing

walkForwardFolds / blockBootstrap): lib/validation/deflated-sharpe.ts,

lib/validation/pbo.ts, lib/learning/regime-weights.ts, lib/learning/meta-label.ts.

Extended: lib/validation/engine.ts (two added AND-gates, all new fields recorded).

8. Phased rollout (each gate flips separately, measure-only first)

Phase	Ships	Enforcement	Sample floor	Owner switch
A. DSR/PBO harness	Compute DSR, PBO, n_trials on every validateChallenger run; record on experiment rows; surface in Performance Truth	Measure-only	—	none
B. DSR/PBO gate	$DSR \geq 0.95$ AND $PBO \leq 0.20$ become required promotion AND-conditions	Enforced	$n_{\text{effective}} \geq 20$ per market	dsr_pbo_enforced[market]
C. Regime measure-only	Cohort fit + shrunk deltas computed, logged to regime_weight_deltas; scoring still uses champion	Measure-only	—	none
D. Regime conditioning	Conditioned weights used in scoring (still must clear the gate as a challenger)	Enforced	$n_c \geq 30$ per cohort (else $\alpha=0$)	regime_conditioning_enabled[market]
E. Meta-label measure-only	P_meta computed + recorded; counterfactual suppression reported	Measure-only	—	none
F. Meta-label shadow	shadow_paper variant applies gate, records would-be book	Shadow (no cash)	OOS-accepted model	none
G. Meta-label gate	Meta-gate may withhold eligible entries	Enforced	≥ 50 acted obs per market	meta_label_gate_enabled[market]

Phases A/C/E (all measure-only) can be built now; the enforcement phases (B/D/G)

stay dark until the sample floors are cleared — consistent with the existing

Phase-0/Phase-1 discipline and the "when data matures" reality of a fresh book.

9. Single riskiest assumption

Sample size. At this stage the book has very few closed trades (the 10-trade

Phase-0 gate has barely opened, and Performance Truth still shows

insufficient_sample below 20). DSR needs a meaningful track length T, PBO needs

enough purged blocks to form $C(S, S/2)$ splits, regime cohorts fragment the already-

small sample by 2–4×, and meta-labeling needs enough *acted* rows to fit a second

logistic. **Building the harness now is correct and safe (measure-only); activating

any enforcement gate before the per-method sample floors are met would either block

all promotions or fit noise.** The design mitigates this by (a) computing everything

measure-only from day one, (b) per-method floors that mirror the existing 20/30-row

disciplines, (c) shrink-to-champion / no-op fallbacks whenever a cohort is thin, and

(d) conservative trial-count N that errs strict.

The one open decision for the owner (§4.4): *how to count N (the trial family)

for DSR/PBO.* The proposed "every learner-proposed challenger for this market since the current champion was promoted, plus the champion" is conservative and auditable, but it couples the deflation strength to LearnerAgent's proposal cadence — a chattier learner inflates N and makes promotion harder. The alternative (count only *validated* challengers, or a fixed rolling window) is less strict but arguably closer to "trials that could have been selected." This choice sets how aggressively the #1 overfitting guard bites, and should be locked before Phase B enforcement.

10. Acceptance tests (deterministic, no live money)

1. **DSR determinism:** same dataset + same $N \rightarrow$ identical DSR/PBO to full float precision (seeded, like `blockBootstrap`). Snapshot test.
2. **DSR deflation direction:** holding $SR_{\hat{t}}$ fixed, increasing N strictly lowers DSR; increasing negative skew lowers DSR. Property test.
3. **PBO sanity:** a deliberately overfit config (fit-to-noise weights) yields PBO $\rightarrow$ high (> 0.5) on synthetic data; a genuinely predictive config yields PBO low. Fixture test.
4. **Gate is additive, never looser:** any challenger that FAILS the old gate still FAILS with DSR/PBO enabled; DSR/PBO can only flip PASS $\rightarrow$ FAIL. Differential test against `current validateChallenger`.
5. **Measure-only inertness:** with all switches OFF, champion weights, selection, sizing, promotions, and NAV are **byte-identical** to pre-feature (golden-master over a replay window) — mirrors the cross-sectional-rank / PIT-fundamentals "OFF = no-op" proof.
6. **Regime no-op when thin:** cohort with $n_c < 30 \rightarrow \alpha_c = 0 \rightarrow$ conditioned weights exactly equal champion weights (bit-exact).
7. **Regime bounded:** for all cohorts, $\|w_{\text{effective}} - w_{\text{champion}}\| \leq \text{the } \alpha_{\text{max}}$ bound and $\sum w_{\text{effective}} = 1.0$.
8. **Meta-gate subtractive only:** for every observation, $\text{act} \subseteq \text{primary_eligible}$ (the gate never enables a primary-ineligible entry); with the gate OFF, ``act == primary_eligible``.
9. **OOS fail-closed inheritance:** a meta-label model with < 30 OOS rows or $\text{ECE} > 0.1$ is **not** stored to `model_artifacts` (reuse `acceptCalibrationOOS` test vectors).
10. **Per-market isolation:** a synthetic India cohort/trial family never alters any US DSR, PBO, regime delta, or meta-model, and vice-versa.

11. Sequencing vs router + Decision-Review + experiment-lineage

- **Post-router.** This is a *promotion-brain* upgrade; it assumes the scoring/router path and the Validation Engine are stable. It does not touch routing, data provider abstraction, or order execution. Build **after** the router work lands.
- **Complements Decision-Review** (features/decision-review/FEATURE_ARCHITECTURE.md):
Decision-Review is the per-symbol post-mortem UI over the same `decision_observations × observation_labels` ledger. Meta-labeling's measure-only `P_meta` and the DSR/PBO verdicts **surface through** Decision-Review / Performance Truth panels — no new dashboard. Decision-Review answers "was this one decision right?"; Advanced Learning answers "should this *strategy* be promoted, and should we *act* on its signals?" Same evidence tables, different altitude.
- **Complements experiment-lineage:** DSR/PBO records (`n_trials`, `trial_family_ids`, `dataset_hash`) are exactly the lineage/provenance experiment-lineage tracks — this feature *produces* lineage facts (which trials, which family, which verdict) that experiment-lineage *organizes*. Build the DSR/PBO recording columns in a shape experiment-lineage can consume.
- **Ordering within this feature:** #1 DSR/PBO (highest value, smallest surface, pure add to an existing gate) → #2 regime-conditioning (needs cohort volume) → #3 meta-labeling (highest risk, needs the most acted data, gated last). Each measure-only phase is independently shippable and independently reversible.

12. Open items to confirm at approval

1. **Trial-count N definition** (§4.4/§9) — the single biggest decision.
2. DSR_MIN (0.95), PBO_MAX (0.20), S (8), $\alpha_{\max}$ (0.25), K (50), META_MIN, and all sample floors are v0 defaults needing prospective tuning — confirm the starting values.
3. Regime cohorting granularity: 2-band (calm/stress) v0 vs full 4-level — confirm the v0 coarsening.
4. Whether regime-conditioning is expressed as a runtime scoring transform vs a materialized per-regime challenger set (this doc proposes runtime transform + audit row; a materialized variant is possible but adds governance objects).

Source: features\agent-evolution\FEATURE_ARCHITECTURE.md

Feature Architecture: Agent Evolution (source-attributed discovery, expanded genome, US/India parity)

Status

Architecture status: Draft

Architecture approved: No

Approved scope: None

Approved date: None

Implementation allowed: No

Prompted by a second Codex adversarial review pass (2026-07-06), specifically scoped to whether ResearchAgent's parameters, discovery sources, and LearnerAgent's evolution surface are actually appropriate for a world-class long-only swing-trading agent across US and India — as opposed to whether the current code has bugs (that pass is Decision 43, already fixed).

Feature Purpose

Three related gaps, all downstream of the same root cause — the system has no memory of *which discovery source* or *which parameter* a decision came from, so it can't learn which sources/parameters are worth trusting:

- 1. Symbol discovery is unattributed.** `gatherSymbols()` in `lib/research-agent.ts` merges 7 sources (holdings, watchlist, Theme Scout, momentum bucket, value bucket, metals basket, region ETFs, India NIFTY) into one flat list. Once merged, a `decision_observations` row only knows `source: "holding" | "screener"` — Theme Scout picks, momentum-bucket picks, and value-bucket picks all collapse into the same "screener" label. There is no way to ask "did Theme Scout's picks actually make money over the last 90 days?" without hand-joining `watchlist.theme/features.screener.bucket` against `observation_labels` — a query nobody runs today.
- 2. LearnerAgent only evolves 5 numbers.** `update_signal_weight` (Decision

43 hardened it, but its scope is unchanged) can only move `fundamental_weight / technical_weight / sentiment_weight / macro_weight / insider_weight`. Everything else that defines the strategy — score threshold, position sizing curve, stop/target logic, which discovery sources get research budget, India-specific gates — is either a fixed constant in code or a value the user sets by hand in Settings. The "evolution" the product promises is five numbers wide.

3. India is US scoring wearing a currency label. `isIndia(symbol)` swaps Alpha Vantage for Yahoo and skips social/options/insider (honest), but reuses the exact same `score_threshold`, RSI/EMA thresholds (`lib/data/technical.s.ts`), and benchmark logic path as the US. There is no India-specific liquidity filter, no NSE holiday-aware freshness check on candle data, and India's `computeRegimeFeatures` benchmark is `^NSEI` only because someone hardcoded it in — not because it went through the same validation gate a US weight change does.

User/System Questions This Feature Answers

- Which discovery source (Theme Scout, momentum, value, watchlist, held, India NIFTY, metals, region ETF) actually produces profitable signals, and should the system research more/fewer symbols from it?
- If `LearnerAgent` wants to change something other than the 5 score weights (e.g. "raise the score threshold" or "shorten the holding horizon"), is there a governed, validated path to do that, or does it require a manual code change?
- Is India's scoring gated on India-appropriate evidence, or silently reusing US assumptions that were never checked against India data?

Scope

This feature includes:

- A primary `discovery_source` field recorded on every new `decision_observations` row (`'held_position | manual_watchlist | theme_scout | momentum_bucket | value_bucket | india_nifty | india_screen_cache | metals | region_etf | unknown_legacy'`) plus `discovery_sources_all/discovery_metadata` for multi-source provenance. This is additive. It does **not** remove or reinterpret the current `agent_signals.source: "holding" | "screener"` field in v1, because that field is already used by existing UI/reporting paths.

- A read-only **Discovery Source Scorecard** (API + dashboard card) showing, per source: candidates researched, entry-eligible rate, and — once `observation_labels` mature — mean `benchmark_neutral_return`. Advisory only in v1; see Non-Goals.
- An extended **strategy genome schema** (`strategy_versions.genome`, already typed per Phase 3 — see `lib/validation/genome.ts`) covering the fields listed under "Proposed genome fields" below, each with an explicit bounded range and a validation-gate requirement before promotion — no new field becomes live without going through the exact same challenger → validate → promote path `update_signal_weight` already uses.
- An **India evidence contract**: a documented, code-enforced minimum (candle freshness vs NSE trading calendar, minimum liquidity proxy, minimum fundamental-field count) that must hold before a India candidate is eligible to enter `long`, independent of and stricter than the general thin-evidence abstain rule from Decision 43.
- A written decision on whether India gets its own `score_threshold/RSI` bands or continues sharing the US ones, backed by whatever India-labeled ledger data exists at the time (may conclude "not enough India data yet to diverge — keep shared, revisit after N India closed trades").

Non-Goals

This feature does not include:

- **Any automatic reallocation of research budget by source.** The Discovery Source Scorecard is read-only/advisory in v1. Automatically researching more Theme Scout candidates because they scored well recently is a feedback loop that needs its own validation story (a source that looks good on 20 trades can regress) — out of scope until the scorecard has run long enough to have an opinion on its own reliability.
- **Auto-promotion of any genome field.** Every genome-field change flows through the existing challenger/validate/promote lifecycle. Nothing here makes a `strategy_versions` row live without a human promoting it via the Strategy Registry, same as today.
- **Increasing screener candidates above 3/day.** CLAUDE.md locks this at 3; this feature does not touch that number regardless of what the Discovery Source Scorecard shows.
- **Explicit bull/bear regime switching.** CLAUDE.md explicitly wants scoring to adapt via weights, not a hardcoded regime branch. Nothing in the genome expansion below introduces an if/else on regime; regime stays an

observed feature (`features.regime.*`, Phase 3) available to future weight/threshold learning, not a switch.

- **Loosening trade approval.** No genome field, however it evolves, changes `approval_required` gating on `TraderAgent/live` orders. Paper trading and shadow evaluation remain the only channels a genome change can reach before a human promotes it.

- **Building India-specific thresholds speculatively.** This feature defines the `*contract*` (what evidence India needs) and instruments the ledger to measure whether shared vs India-specific thresholds perform differently. It does not invent India-specific RSI/threshold numbers without validated India trade history to back them — CLAUDE.md's "don't run ahead of evidence" principle applies here as much as to weight changes.

Codex Review Amendments Required Before Build

These amendments close ambiguity in the draft architecture and should be treated as part of the target design:

1. **Do not collapse multi-source provenance into one lossy label.** A symbol can be a held position, a manual watchlist item, a Theme Scout candidate, and a screener hit on the same day. Persist one primary `discovery_source` for grouping, but also persist `discovery_sources_all` and `discovery_metadata` so source performance can be audited without losing assisted-source credit.
2. **Keep legacy `agent_signals.source` stable.** The new source fields belong on the decision/research ledger. Do not rename or overload `agent_signals.source` in this PR; existing UI expects the coarse `holding | screener` meaning.
3. **Scorecard must be statistically honest.** It must show sample size, label coverage, benchmark-relative win rate, confidence interval, and insufficient-sample status. A raw average return by source is too easy to overread and will invite bad allocation decisions.
4. **No source-budget feedback loop in v1.** Source performance can be shown to the user, but cannot change `gatherSymbols()` allocation until a separate exploration policy exists with a minimum exploration floor and shadow validation.
5. **Separate provisioned genome fields from live-consumed fields.** `score_threshold` may be added to the genome schema and validation engine in this PR, but `ResearchAgent` must continue using the current approved

runtime threshold path until a later approved promotion/consumption change defines precedence against `strategy_config`.

6. India gates must fail closed for entry eligibility. If the NSE calendar, candle freshness, liquidity proxy, or required fundamentals are unknown, the system may still record an observation, but it must not mark the India candidate entry-eligible.

Current Behavior

- `gatherSymbols()` returns a flat `SymbolEntry[]` with only `isHeld`, `isEtf`, `assetClass`, and an optional `screenerBucket` (momentum/value — added Decision 41, screener-only). Theme Scout candidates arrive via the `watchlist` table read and are indistinguishable from a manually-added watchlist symbol once merged.
- `processSymbol()` sets `source: isHeld ? "holding" : "screener"` on `agent_signals`, and copies the same 2-value source onto `decision_observations` implicitly via the `market/symbol` join — there is no persisted "this candidate came from Theme Scout" flag past the `features.screener` block (which only exists for the momentum/value screener bucket, not Theme Scout/watchlist/held/metals/region-ETF/India).
- `update_signal_weight` (`LearnerAgent`) is the only live weight-mutation tool. `strategy_versions.genome` exists (Phase 3, migration 063) and is validated (`lib/validation/genome.ts`) but nothing currently proposes genome mutations beyond the 5 weights — the genome column is provisioned infrastructure, not yet a used surface.
- India shares `score_threshold`, `stop_loss_pct`, `target_pct` from the same `strategy_config` row as US (no market-scoped `strategy_config` columns exist yet — only `strategy_versions` is market-scoped, per Phase 4). India's NSE holiday gate (Decision 39) only covers 2 fixed-date holidays (Republic Day, Independence Day, Gandhi Jayanti) — floating festivals (Holi, Diwali) are explicitly unmodeled, a known, documented gap.

Proposed Behavior

1. Discovery source attribution

- Extend `SymbolEntry` (`lib/research-agent.ts`) with a required `discoverySource` field, set at the point each source contributes a symbol in `gatherSymbols()`: `held_position`, `manual_watchlist` (manual, non-theme),

theme_scout (watchlist rows with source: 'llm_theme'), momentum_bucket / value_bucket (already tagged via screenerBucket, just renamed into the same enum), metals, region_etf, india_nifty, and india_screen_cache if the broader India scanner cache is later wired into ResearchAgent.

- Extend SymbolEntry with discoverySourcesAll and optional discoveryMetadata. When a symbol appears from multiple sources in one run, keep all sources. Pick the primary source by causal priority:
held_position > manual_watchlist > theme_scout > momentum_bucket / value_bucket > india_screen_cache > india_nifty > metals > region_etf. The scorecard should support both primary-source metrics and assisted-source metrics once enough data exists.
- Persist discovery_source on decision_observations (new column, migration) alongside the existing features. screener_block — additive, does not replace the momentum/value bucket detail already recorded.
- Persist discovery_sources_all and discovery_metadata next to discovery_source. discovery_sources_all is required for new observations and should include the primary source; discovery_metadata may include contributing watchlist row IDs, screener bucket/rank, theme name, or scanner run ID when available.
- New read-only endpoint /api/agents/discovery-scorecard: per discovery_source × market, count of candidates researched (last 90d), entry-eligible rate, and mean benchmark_neutral_return where matured labels exist (reuses lib/learning/dataset.ts's join, grouped by source instead of by fold). Surfaced as a card on the Research Journal's Evolution tab — extends existing UI, no new page.
- The scorecard must report source/market/horizon rows across at least 30d and 90d windows, including nResearched, nLabeled, labelCoveragePct, entryEligibleRate, abstainRate, avgDataAvailability, meanBenchmarkNeutralReturn, medianBenchmarkNeutralReturn, winRateVsBenchmark, bootstrapCiLow, bootstrapCiHigh, and sampleStatus. If a row lacks enough matured labels, show sampleStatus: "insufficient_data" instead of a performance verdict.

2. Genome expansion (validated, shadow-first)

Proposed additional genome fields (each independently bounded, independently gated by the SAME validation path — walk-forward replay via

`lib/validation/engine.ts`'s shared `computeWeightedAnalystScore`, extended per field type below):

Field	Bounds	Validation approach
<code>score_threshold</code>	50–75	Replay: what would entry-eligible rate + benchmark-neutral return have been at this threshold, using labeled history — same walk-forward/bootstrap machinery as weights.
<code>holding_horizon_days</code>	2–20	Already have 2/5/10/20-day labels (<code>observation_labels</code>); replay picks the horizon whose labeled return the strategy is actually evaluated against.
<code>stop_loss_pct</code> / <code>target_pct</code>	bounded %	Already partially dynamic via MAE/MFE percentiles (<code>lib/risk/percentiles.ts</code>); genome field would let a challenger propose a *different percentile* (e.g. p20/p80 instead of p25/p75), validated the same way.
<code>discovery_source_weights</code>	0–1 per source, sum-normalized	Read-only in v1 (see Non-Goals) — recorded in genome as a *proposed* field once the Discovery Source Scorecard has enough history to inform it, not wired to actually change <code>gatherSymbols()</code> 's candidate mix until a future, separately-approved iteration.
<code>sizing_curve_params</code>	half-Kelly cap, floor	Currently hardcoded in <code>lib/risk/sizing.ts</code> defaults; genome field would let a challenger propose different cap/floor, validated against the ledger's realized Kelly-implied returns.

None of these are proposed as an immediate `LearnerAgent` tool. This document recommends building ONE new field (`score_threshold` — cheapest to validate, reuses 100% of existing walk-forward machinery, most directly answerable question: "is 60 the right bar?") end-to-end as a proof of the pattern before expanding to the rest, rather than shipping all five/six new tunable fields in one PR. Each subsequent field follows once `score_threshold` has round-tripped through challenger → validate → promote at least once, on real data.

Important runtime boundary: this PR may provision and validate `genome.entry.score_threshold`, but it must not silently make `ResearchAgent` consume that field. Runtime threshold precedence must be approved separately because today `strategy_config.score_threshold` acts as a user-visible manual control. Until that follow-up is approved, the genome threshold is a shadow candidate parameter only.

3. India evidence contract

Documented (and code-enforced in `lib/data/scores.ts/lib/india-data.ts`) minimum evidence before an India candidate is `entry_eligible`:

- Candle data freshness checked against an NSE trading-calendar-aware cutoff, not a flat day count (extends Decision 39's fixed-holiday gate to also flag stale-but-not-holiday data, e.g. Yahoo feed lag).
- Minimum liquidity proxy (e.g. average daily traded value over N days,

sourced from Yahoo's `averageVolume/price.regularMarketVolume` — needs a concrete threshold, to be set from observed India ledger data once ~60 India observations exist, not guessed today).

- The `hasMinFundamentalFields` gate from Decision 43 already applies to India; this section formalizes it as a **documented** India-specific contract rather than an incidental side effect of a US-shaped check.
- Fail-closed rule: if NSE calendar freshness, latest candle freshness, liquidity proxy, or required fundamentals cannot be determined, the system may still save the observation with an explicit reason code, but it must set `entry_eligible = false` for that India candidate.
- ResearchAgent v1 currently uses the static NIFTY candidate path; the broader India scanner/cache path should not be assumed live for ResearchAgent unless wired explicitly. If that cache is later used as a source, tag it as `india_screen_cache` rather than mixing it into `india_nifty`.
- Explicit open question, not resolved by this document: does India get its own `score_threshold` once enough India-labeled data exists? Recommendation is to instrument (via the Discovery Source Scorecard's `market` dimension, already present) and revisit once India has 60+ matured observations (the same bar Phase 2 validation already uses for US) — not decide now from zero data.

User Journey / System Flow

1. Research cron runs `gatherSymbols()` → every returned `SymbolEntry` now carries a `discoverySource`.
2. `processSymbol()` writes `discovery_source` onto `decision_observations` alongside the existing `score/availability/weighting` fields (Decision 43).
3. Nightly/weekly, the Discovery Source Scorecard endpoint aggregates matured observations by `source × market` — visible on Research Journal.
4. LearnerAgent (future PR, not this one) gains a `propose_genome_field` tool scoped initially to `score_threshold` only, mirroring `update_signal_weight`'s evidence-binding; server recomputes the replay itself, refuses on insufficient data, creates a challenger, never mutates live config directly.
5. Validation Engine's existing walk-forward/bootstrap gate (``lib/validation/engine.ts``) evaluates the challenger; promotion is manual via Strategy Registry, same as weight challengers today.
6. India's evidence contract runs inline in `computeScores/processSymbol` for every India candidate, independent of and prior to the shared

thin-evidence abstain check from Decision 43.

Screen / Page / Module Inventory

- `lib/research-agent.ts` — `SymbolEntry.discoverySource`, `gatherSymbols()`, `processSymbol()` (persist `discovery_source`).
- New migration — `decision_observations.discovery_source` column.
- New app `app/api/agents/discovery-scorecard/route.ts`.
- `app/dashboard/research-journal/page.tsx` — new "Discovery Sources" card on the Evolution tab.
- `lib/validation/genome.ts` — extend bounded-field schema for `score_threshold` (first field only, per Proposed Behavior §2).
- `app/api/agents/learner/route.ts` — future `propose_genome_field` tool (explicitly deferred to a follow-up PR, not built in this pass).
- `lib/india-data.ts`, `lib/data/scores.ts` — India evidence contract checks.
- `lib/learning/dataset.ts` - include source/provenance fields in the labeled observation dataset used by scorecards and later validation.
- Tests for source attribution precedence, legacy unknown-source handling, scorecard insufficient-sample behavior, genome threshold bounds, and India fail-closed gates.

System Architecture

Modules

- Discovery attribution lives entirely in `lib/research-agent.ts` — no new service, extends the existing `SymbolEntry/processSymbol` types.
- Genome validation reuses `lib/validation/engine.ts` and `lib/scoring/weighted-score.ts` (Decision 43) unchanged in mechanism; `objectiveTerm`'s per-row scoring already supports whatever weights a challenger proposes — extending it to also vary `score_threshold` per challenger is a parameter, not a new code path.

API Contracts

- `GET /api/agents/discovery-scorecard?market=us|india&days=90` → `{ sources: [{ source, market, n, entryEligibleRate, meanBenchmarkNeutralReturn | null }] }`. Read-only, no side effects.

Required scorecard response shape:

`GET /api/agents/discovery-scorecard?market=us|india&days=30|90&horizon=2|5|10|20`

returns `{ sources: [{ source, market, horizonDays, windowDays, nResearched, nLabeled, labelCoveragePct, entryEligibleRate, abstainRate, avgDataAvailability, meanBenchmarkNeutralReturn, medianBenchmarkNeutralReturn, winRateVsBenchmark, bootstrapCiLow, bootstrapCiHigh, sampleStatus }] }`.

Read-only, no side effects. `sampleStatus` must distinguish sufficient, insufficient_labels, and legacy_unknown_source.

Data Models

- `decision_observations.discovery_source` text (new, nullable — existing rows have no source recorded, must degrade gracefully, not backfilled).
- `strategy_versions.genome` — already jsonb (migration 063); this feature adds `score_threshold` as a recognized, bounded key, validated by `lib/validation/genome.ts`'s existing bounds-checking pattern.
- `decision_observations.discovery_source` is nullable in DB only for legacy rows; new code paths must write one of the approved source values. Use a check constraint or shared enum list for: `held_position`, `manual_watchlist`, `theme_scout`, `momentum_bucket`, `value_bucket`, `india_nifty`, `india_screen_cache`, `metals`, `region_etf`, `unknown_legacy`.
- `decision_observations.discovery_sources_all` jsonb not null default '[]' records all contributing sources and must include the primary source on new writes.
- `decision_observations.discovery_metadata` jsonb records optional provenance: watchlist row ID, theme, screener bucket/rank, scanner run ID, and stale/gate reason codes.
- Add indexes for scorecard reads: (`market`, `discovery_source`, `ts desc`) and, if query plans require it, a GIN index on `discovery_sources_all`.

Error Handling

- Discovery Source Scorecard degrades to "insufficient data" per source below some minimum n (mirrors Validation Engine's `nObservations < 60` gate) rather than showing a misleading small-sample average.

Data Architecture

- Required data: `decision_observations.discovery_source` (new), `observation_labels` (existing, joined by symbol+market+horizon).
- Optional data: benchmark-neutral return only where labels have matured; scorecard rows show n researched even before any label matures.

- Mock vs real vs derived: fully derived from real ledger rows, no synthetic/seed data.
- Persistence: additive column, fail-soft insert (same pattern as `availability_mask/weighting` in Decision 43 — a missing column must never fail a research run).
- Validation: genome field validation reuses Phase 2's walk-forward + block-bootstrap statistical bar unchanged ($p_improvement \geq 0.80$, CI floor, $n_effective \geq 12$, $\geq 3/5$ folds won).

Files Likely To Change

- `lib/research-agent.ts`
- `supabase/migrations/0NN_decision_observations_discovery_source.sql` (new)
- `app/api/agents/discovery-scorecard/route.ts` (new)
- `app/dashboard/research-journal/page.tsx`
- `lib/validation/genome.ts`
- `lib/india-data.ts`, `lib/data/scores.ts`
- `public/agent-diagrams/system-map.json` — LEARNER/RESEARCH node descriptions need updating once genome fields beyond weights exist (CLAUDE.md's system-map rule).

Files / Behavior That Must Not Change

- `TraderAgent/live` order approval required gating — no genome field reaches live execution without a human approving the order, same as today.
- Screener candidates/day cap (3) — CLAUDE.md-locked, not touched by discovery-source attribution or scorecard.
- No explicit regime branch introduced anywhere in this feature — regime remains an observed feature, not a switch.
- `update_signal_weight`'s existing evidence-binding (Decision 43) — the proposed `propose_genome_field` tool must follow the identical fail-closed-on-fallback-data pattern, not a weaker one.

Acceptance Criteria

- Every new `decision_observations` row has a non-null `discovery_source` matching one of the approved source values, and `discovery_sources_all` includes the primary source. Legacy rows are surfaced as `unknown_legacy`, never silently grouped into a real source.
- Discovery Source Scorecard returns real, non-fabricated aggregates and

clearly marks sources with insufficient sample size rather than showing a misleading number.

- Discovery Source Scorecard exposes label coverage, abstain rate, benchmark-relative win rate, median return, and confidence interval, not only mean return.
- Source scorecard output does not alter research allocation, candidate count, paper trading, or live-trading eligibility in v1.
- `score_threshold` is provisioned in the genome schema and validated by the existing walk-forward engine, but is NOT proposable by `LearnerAgent` until a separately-approved follow-up implements `propose_genome_field` end-to-end (this PR builds the plumbing/contract, not the LLM-facing tool).
- `ResearchAgent` does not consume `genome.entry.score_threshold` until a separate approved runtime-precedence decision says how it interacts with `strategy_config.score_threshold`.
- India evidence contract checks are documented in this file and enforced in code; no India-specific `score_threshold` divergence is introduced without a documented ledger-backed reason.
- India candidates with unknown calendar/freshness/liquidity/fundamental evidence are saved as observations but are not marked entry-eligible.
- Tests cover source attribution precedence, multi-source provenance, scorecard insufficient-sample behavior, legacy rows, genome threshold bounds, and India fail-closed gates.

Approval

Architecture approved: No

Approved scope: None

Implementation allowed: No

Source: features\agent-mind\FEATURE_ARCHITECTURE.md

Feature Architecture: Agent Mind — surfacing what the agents believe, how it evolves, and what macro data means for the book

Status

Architecture status: Implemented (all 3 phases), 2026-07-07

Architecture approved: Yes (user, 2026-07-07)

Approved scope: All three phases

Approved date: 2026-07-07

Implementation allowed: Yes

Implementation notes (2026-07-07)

- Migration 096: `learning_priors_history`, `unique(category, principle)` on `learning_priors`, `macro_interpretations`. Migration 097: daily macro-read crons (us/india). All new tables service-role-only.
- `strategy_versions` already had `parent_version_id` — no schema add needed.
- Phase 1: `/api/agent-mind/priors` (GET/PATCH, owner-only) + Intelligence "Beliefs" tab (view/toggle/add; every change writes history + `decision_journal`).
- Phase 2: `/api/agent-mind/brain` (GET, owner-only) + Intelligence "Brain" tab (champion weights + why, belief drift, learner log, self-invented features, regime posture, track record, evidence-context banner). Pure DB reads.
- Phase 3: `/api/agent-mind/macro-read` (GET owner / POST owner+cron) + Markets "What this means for your book" card. One cheap cached LLM call/day/market.
- No LLM writes any belief; nothing here trades or sizes.

Why this feature exists

Kairos already *forms a view of the market* — it just keeps that view scattered across tables no human reads directly. As the system runs:

- `learning_priors` confidences are meant to drift as predictions resolve (background beliefs like "12-1 momentum works", "widening credit spreads lead equity weakness").
- `strategy_versions` accumulates challenger weight-sets; one per market is the promoted `is_champion` — the app's current working theory of how much each scoring dimension should matter.
- `signal_weights` holds the live per-dimension weights the ResearchAgent actually scores with.
- `learning_log` records the LearnerAgent's weekly hypotheses and outcomes.
- `feature_registry` holds machine-proposed new features and their IC-gated promotion state.
- `macro_regime` / `macro_signals` hold the current macro posture and the raw

economic prints behind it.

- `decision_observations` + `paper_trades` are the ground-truth outcomes that earn or destroy confidence in all of the above.

The user cannot see any of this as a coherent, evolving "mind". This feature surfaces it — read-only, advisory — so the user can (a) see and curate what the agents believe, (b) watch beliefs strengthen/weaken over time and learn the market alongside the system, and (c) understand what a fresh economic print means for the actual book.

This is a VIEWING/curation layer. It never places trades, never changes sizing, and never lets an LLM mutate a belief — belief changes stay owned by the evidence-bound `LearnerAgent` (weekly, statistics-gated). Any LLM used here writes display text only.

Non-Goals

- No new autonomous behavior. Nothing here trades, sizes, or approves.
- No LLM-driven mutation of `learning_priors`, `signal_weights`, or `strategy_versions`. Only the existing `LearnerAgent` (evidence-bound, weekly) changes beliefs. This feature can let the USER manually toggle a prior's enabled flag or edit its text — a human action, logged — but the LLM never does.
- No new market-data spend on the hot path. Macro interpretation (Phase 3) runs only when macro data refreshes (already daily) or on an explicit user click, on a cheap model (DeepSeek/Groq), never per-page-load.
- No change to how confidence is computed. This feature reads the confidence the `LearnerAgent` already maintains; it does not invent a parallel scoring.

Phased scope

The three pieces share data sources and a UI home, but are independently approvable and shippable. Recommended order: Phase 1 (foundation the others read) → Phase 2 (Brain) → Phase 3 (macro interpretation).

Phase 1 — Agent Beliefs panel (surface + curate `learning_priors`)

Purpose

A readable, editable table of every prior the agents reason from, grouped by

category, showing confidence, source, enabled state, and — once history exists — how the confidence has moved.

Data sources (all existing)

- learning_priors (id, category, principle, confidence, source, enabled, notes, created_at).
- Optional new learning_priors_history (see below) to chart confidence drift.

Proposed behavior

- New tab under /dashboard/intelligence (or a card on Settings → Agents): "Agent Beliefs". Groups priors by category (fundamental / technical / macro / insider / general), each row showing the principle text, a confidence bar, source tag, and an enabled toggle.
- Owner-only. Read via a new GET /api/agent-mind/priors (service client, requireOwner). Toggle/edit via PATCH /api/agent-mind/priors (owner-only, logs the change to decision_journal as a human action).
- A small "add prior" form (category, principle, starting confidence, source) so the user can feed a distilled principle from a book/article directly — exactly one testable claim per row, matching the ingestion discipline.
- Sorting by confidence; filter by category; search.

New schema (Phase 1)

- learning_priors_history (prior_id, confidence, changed_at, changed_by ('learner' | 'user'), reason) — append-only, written whenever a prior's confidence or enabled state changes. Lets the UI chart drift and the Brain view (Phase 2) show "strengthening / weakening". The LearnerAgent's existing prior-update path writes a row here; the user-edit path writes one too.
- Migration also de-duplicates and adds a unique(category, principle) constraint to learning_priors (a bug this session left it with every row doubled — already cleaned in data, but the constraint prevents recurrence).

Acceptance criteria

- Every prior visible, grouped, with confidence + source.
- Owner can toggle enabled / edit text / add a prior; each change logged.
- No LLM writes to learning_priors.
- unique(category, principle) prevents duplicate priors.

Phase 2 — Kairos Brain (the unified, evolving belief view)

Purpose

One page that answers "what does the system believe right now, how sure is it, and how has that changed?" — stitched from the separate tables into a single human-readable, evolving narrative. This is the piece that lets the user learn the market alongside the agents.

Data sources (all existing unless noted)

- `learning_priors` + `learning_priors_history` (Phase 1) — background beliefs and their drift.
- `strategy_versions` where `is_champion` per market — the current working theory: the promoted weight-set, its notes (why it was proposed), its validation stats, and its parent lineage (how the champion has changed over generations).
- `signal_weights` — the live per-dimension weights actually scoring today.
- `learning_log` — recent hypotheses the LearnerAgent formed and whether they were confirmed, rejected, or still open.
- `feature_registry` (+ `_history`) — features the system proposed for itself and their IC-gated promotion state (the app inventing new signals).
- `macro_regime` — current regime posture and danger score.
- `decision_observations` + `paper_performance` — the outcome ledger that earns all of the above its confidence, plus realized alpha.

Proposed behavior — sections on /dashboard/intelligence "Brain" tab

1. **Current conviction** — the champion weight-set per market as a readable bar ("fundamental 32%, technical 24%, macro 18%, sentiment 14%, insider 12%") with a one-line "why this is the current theory" from the champion's notes, and how it differs from the previous champion (lineage diff).
2. **Strengthening vs. weakening beliefs** — priors whose confidence rose or fell most over the last N weeks (from `learning_priors_history`), each with the direction and magnitude. This is the "the system is becoming more/less sure of X" view.
3. **Open hypotheses** — what the LearnerAgent is currently testing (from `learning_log`), with status (open / confirmed / rejected) and the evidence count behind each.
4. **Self-invented features** — anything in `feature_registry` the system proposed and where it sits in the IC-gate pipeline (proposed → testing → promoted / killed).
5. **Regime posture** — current `macro_regime` with the danger score and the

threshold adjustment it's currently imposing on new trades.

6. **Track record honesty** — realized alpha, win rate, and the count of resolved predictions, so the confidence is contextualized by how much evidence actually exists (avoid false authority when N is small).

Read path

- GET /api/agent-mind/brain (owner-only, service client) — one aggregated payload assembled server-side from the tables above. No LLM required for the structured view. An OPTIONAL "explain in plain English" button calls a cheap model to narrate the current state (advisory text only, never persisted as a belief).
- Purely read-only. No writes.

New schema (Phase 2)

- None required beyond Phase 1's learning_priors_history. Everything else already exists. (If champion lineage isn't already queryable via a parent_id on strategy_versions, add one — to confirm against live schema first.)

Acceptance criteria

- Single page shows current champion weights + why, belief drift, open hypotheses, self-invented features, regime posture, and track-record context.
- Every number traces to a real table; nothing is LLM-invented.
- Small-N guardrail: confidence is always shown next to the evidence count so a belief with 3 resolved trades never reads as authoritative.

Phase 3 — Macro-to-holdings interpretation (Markets page)

Purpose

Turn the macro data the app already ingests into a plain-language read of what a fresh print means, combined with the other macro signals AND the current book — the missing "so what does this mean for me" layer.

Data sources (all existing)

- macro_signals / macro_regime — the economic prints (GDP, CPI, unemployment, payrolls, retail sales, Fed funds, 2Y/10Y yields) and the regime.
- paper_positions (+ live snapshot) — the current book, its sector/beta mix.
- learning_priors (macro category) — the durable macro principles to reason with (e.g. "rising real yields headwind long-duration growth").

Proposed behavior

- A "What this means for your book" card on the Markets page, generated when macro data refreshes (daily, already scheduled) or on an explicit user click — NEVER on every page load.
- Deterministic inputs assembled server-side (latest prints + regime + book composition + relevant macro priors), handed to a cheap model (DeepSeek/Groq) to produce a short, grounded read: "CPI hotter than expected + real yields rising + you hold 3 long-duration growth names (NVDA, ...) → near-term headwind for that cluster; the macro priors flag this at confidence 0.66." Advisory only.
- Cached per macro-refresh so it costs at most one cheap LLM call/day.

Guardrails

- The model receives DATA, never instructions, and its output is display text only — it cannot change sizing, thresholds, or any belief. It is explicitly told the numbers are ground truth and it may not invent figures.
- Account numbers and any secrets are never in the prompt.
- If the model is unavailable, the card degrades to the raw macro data + regime (which the Markets page already shows) — no hard failure.

New schema (Phase 3)

- Optional `macro_interpretations` (date, market, content, model, created_at) to cache the daily read. Or reuse the existing briefings/newsletter pattern.

Acceptance criteria

- Card appears only after a macro refresh or explicit click; at most one cheap LLM call/day.
- Output ties the print to the regime, the macro priors, and the actual book.
- No LLM output changes any trading behavior; degrades gracefully if the model is down.

Cost summary

- Phase 1 & 2: zero LLM on the structured views (pure DB reads); optional "explain" buttons are on-demand cheap-model calls only.
- Phase 3: at most one cheap-model call per macro refresh (daily), cached.
- No new market-data API spend on any hot path.

Files (indicative, to confirm against live code before building)

- `app/api/agent-mind/priors/route.ts` (new — GET/PATCH, owner-only)
- `app/api/agent-mind/brain/route.ts` (new — GET, owner-only)
- `app/api/agent-mind/macro-read/route.ts` (new — Phase 3, owner-only + cron)
- `components/dashboard/IntelligencePage.tsx` (modify — Beliefs + Brain tabs)
- `components/dashboard/MarketsPage.tsx` (modify — Phase 3 card)
- `supabase/migrations/0NN_agent_mind.sql` (new — `learning_priors_history`, `unique(category,principle)`, optional `macro_interpretations`, optional `strategy_versions.parent_id` if absent)
- `LearnerAgent` prior-update path (modify — also append to `learning_priors_history` when it changes a confidence)

Approval

Architecture approved: No

Approved scope: None

Implementation allowed: No

Source: features\agent-source-pipeline-remediation\FEATURE_ARCHITECTURE.md

Agent Source Pipeline Remediation

Status: APPROVED for implementation by owner on 2026-07-20.

Problem

The US and India pipelines are correctly separated at their principal scoring and paper-accounting boundaries, but an adversarial source audit found active quote coverage, quota, provenance, confidence, and advisory-semantics defects. The most material defect is PositionMonitor's burst of one unpaced Massive request per US holding: an unavailable response silently suppresses every mechanical and conviction exit for that position for the session.

Scope

1. Make paper-position monitoring price-complete per market and visibly report every position that could not be evaluated.

2. Make decision-quality applicability agree with production scoring: India has fundamental, technical, and sentiment dimensions today, not US macro.
3. Scope discovery before external work so one market never spends the other market's provider budget or broker session.
4. Remove Theme Scout from ResearchAgent's wall-clock budget and make capped Alpha Vantage data reserve-only.
5. Record the provider that actually served each evidence item.
6. Stop routine option-chain calls until an approved measure-only experiment has a consumer and prospective acceptance test.
7. Keep Holding Risk's deterministic risk posture intact while adding a separate, read-only alpha context from already-persisted ResearchAgent output.
8. Repair dormant autonomous-live cross-market assumptions while keeping every activation flag unchanged and false.
9. Keep India Learner runs from consuming the US-only macro table and make valid green MacroSentinel runs idempotent.
10. State India scanner coverage from measurable freshness instead of claiming the rotating cache is simultaneously fresh across the full NSE universe.

Non-Goals

- No Router or Vibe cutover.
- No autonomous-live activation and no broker/network order test.
- No score-weight, threshold, mandate, kill-switch, or account change.
- No India macro score. That requires a separately reviewed architecture.
- No conversion of Holding Risk into an execution agent.

Safety Invariants

- No LLM output may set a score, direction, posture, quantity, price, or order.
- A missing quote cannot be treated as a hold; it is an explicit unevaluated state.
- A quote failure for one position cannot suppress exits for positions with valid prices.
- US/USD and India/INR data, outcomes, NAV, mandates, and calibration samples never mix.
- Mechanical exits remain independent of ResearchAgent availability.
- Holding Risk reads persisted alpha evidence only and remains advisory.
- Autonomous-live, Router, Vibe, allocation, and protective-order flags are unchanged.
- Database migrations are forward-only and production definitions are verified after apply.

Acceptance Criteria

- PositionMonitor resolves US prices through one batched request plus existing

cache/reserve fallbacks and reports missing symbols in its run result and Health.

- India quality rows no longer count macro as applicable or missing.
- US research does not call Kite/India screening; India research does not call the US FinancialDatasets screener or US holdings path.
- ResearchAgent no longer invokes Theme Scout inline and no longer fetches Yahoo option chains during routine scoring.
- Evidence records carry actual source and timestamp; unavailable evidence does not claim a successful provider.
- Holding Risk publishes alpha score/direction/age/holding-path provenance as separate context without changing its risk posture or calling ResearchAgent/providers.
- Dormant India autonomous-live uses India pricing, India mandate thresholds, and India-only paper calibration; tests prove cross-market records cannot affect it.
- India Learner reports macro unavailable; a healthy green US macro run is reused.
- Scanner responses expose eligible/fresh counts and do not claim all NSE rows are fresh.
- Focused tests, full Vitest, TypeScript, Next production build, secret scan, migration verification, and read-only production smoke checks pass.

Source: features\agent-transparency-debate\FEATURE_ARCHITECTURE.md

Feature: Agent Transparency + Deep-Dive Debate

Status: DRAFT (Phase 1 in progress) · Owner: Vaibhav · Started 2026-07-04

Motivation

Competitor trading-agents.ai (TradingAgents framework) produces an *argued* per-symbol verdict via an adversarial multi-agent debate. Kairos currently produces only a numeric `analyst_score`. This feature closes that gap while keeping Kairos's edge (loop closure, governance, automation).

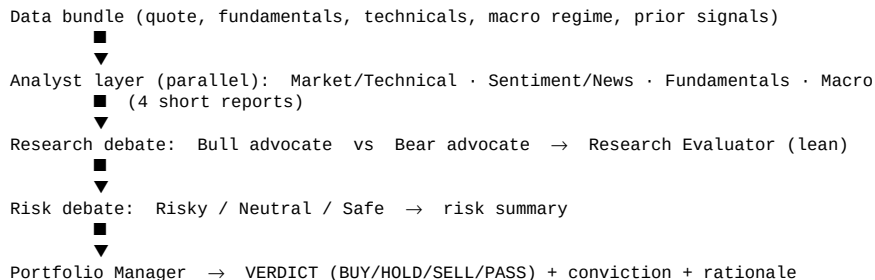
Phases

1. **Deep-Dive Debate engine** + per-symbol panel ← current
2. Agent history/transparency page (#12)

3. Score trajectory + re-scoring (#13)
4. Visual Agents audit + AI-agent vs Flow rename (#14)

Phase 1 — Deep-Dive Debate

Pipeline (on-demand, per symbol)



Model routing (no ANTHROPIC_API_KEY available)

- Analysts + advocates: deepseek-chat (cheap, parallel)
- Research Evaluator + Portfolio Manager: deepseek-reasoner (better reasoning)
- Cost ~\$0.05–0.15 per run. **On-demand only** (button) — never auto/cron.

Governance

- Long-only aware: for a non-held symbol the verdict is BUY or PASS (never SELL).
- Advisory only: verdict may flag high conviction into the pipeline but the deterministic risk gate + approval-required order flow remain authoritative.

Storage — table deep_analyses

```

`id uuid pk · symbol · verdict · conviction int · summary · reports jsonb
(per-agent) · model · tokens_in · tokens_out · cost_usd · created_at`
RLS: service_role all; authenticated read.

```

API

- POST /api/agents/deep-dive { symbol } — runs the pipeline synchronously, stores a deep_analyses row, returns the full result. Auth: user or cron.
- GET /api/agents/deep-dive?symbol=X — latest stored analysis for the symbol.

UI — DeepDivePanel on the symbol page

- "Run Deep Analysis" button.
- Agent pipeline list with status; each agent row expands to its report.
- Final verdict card (verdict, conviction, rationale).
- Shows model + token/cost footer.

Non-goals (Phase 1)

- Streaming progress (synchronous run; loading state in UI).
- Multi-market (US first; India/EU rides with the watchlist workstream).
- Auto-feeding analyst_score (advisory display first; wiring in a later phase).

Source: features\agentic-quant-platform\FEATURE_ARCHITECTURE.md

Feature Architecture: Governed Agentic Quant Platform

Status

Architecture status: Approved

Architecture approved: Yes

Approved scope: Long-only, 2-20 market-day swing-trading research, validation, paper execution, explainability, approval-required Robinhood execution, and a future separately unlocked limited auto-live mode

Approved date: 2026-06-27

Implementation allowed: Yes (phased â€" see Delivery Phases)

Phase 0 complete: 2026-07-01

Phase 1: Not started

Feature Purpose

Kairos will become a governed multi-agent research platform that continuously gathers evidence, develops and validates trading hypotheses, runs realistic paper experiments, explains its reasoning to Vaibhav, and can eventually execute approved trades through the Robinhood agentic account.

The platform is designed to test whether strategies can outperform appropriate benchmarks after costs, taxes, and risk. It does not assume or promise market outperformance. Survival, reproducibility, and auditability take precedence over return.

Approved Product Decisions

- Trading horizon: swing trading, normally 2-20 market days.
- Universe: Robinhood-supported US equities and ETFs that pass dynamic liquidity, price, trading-status, and data-quality filters.
- Direction: long-only in paper and live pathways.
- Data budget: free-first. TradingView is a manual analysis and export tool, not an unattended data API.
- Strategy eligibility: dynamic statistical evidence gate followed by Vaibhav's explicit manual approval.

- Explanation: layered decision journal plus a daily briefing.
- Online research: primary and curated sources establish evidence; unrestricted web and social sources may generate hypotheses but cannot independently justify trades.
- Initial live mode: every live order requires explicit user approval.
- Future live mode: limited auto-live may be enabled only through a separate, strategy-specific, time-bounded manual unlock.
- Tax, dividend, broker, and regulatory knowledge is monitored for change, but behavior-changing rules require governed review.

User/System Questions This Feature Answers

- What does the system currently believe about the market, and what evidence supports that belief?
- What is the agent planning to research, test, paper trade, or propose for live execution?
- Why is a proposed action preferable to waiting or choosing another security?
- Which statements are verified facts, calculations, estimates, interpretations, or unverified hypotheses?
- How did a strategy perform out of sample, after realistic costs, and relative to a benchmark?
- What changed between a champion strategy and its challenger?
- What risks, correlated shocks, dividend events, and tax consequences apply?
- What did the platform learn after a prediction or trade resolved?
- Is the platform healthy, current, and authorized to act?

Scope

This feature includes:

- Point-in-time evidence ingestion and provenance.
- Source-quality hierarchy, conflict detection, quarantine, and abstention.
- Sourced research packets and reproducible hypotheses.
- Deterministic feature computation and a Python validation worker.
- Versioned strategy registry with champion/challenger governance.
- Historical replay, robustness tests, and dynamic eligibility reports.
- Realistic long-only paper execution and portfolio accounting.
- Deterministic portfolio risk and tax-aware decision support.
- Dividend and corporate-action awareness.
- Daily briefing and layered decision journal.
- Approval-required live execution through a single Robinhood gateway.
- A future limited auto-live state with explicit manual unlock.
- Scheduling, monitoring, audit trails, replay, and fail-closed behavior.
- Periodic governed updates to tax, broker, regulatory, and market knowledge.

Non-Goals

This feature does not include:

- Options, crypto, futures, short selling, leverage, or margin borrowing.
- Intraday, high-frequency, or latency-sensitive trading.
- Access to any Robinhood account except agentic account <agentic-account-id>.
- Guaranteed returns or an assertion that Kairos will beat the market.
- Autonomous promotion of a strategy to live trading.
- Silent changes to risk policy, tax rules, execution permissions, or champion strategies.
- LLM-generated authoritative prices, P&L, fills, tax calculations, or risk calculations.
- Treating social posts, unsourced articles, or LLM opinions as verified evidence.
- Tax preparation or personalized legal/tax advice. Broker tax documents and professional advice remain authoritative.
- A big-bang rewrite of the existing Next.js application.

Current Behavior and Known Gaps

The repository already contains research, paper-trade, learner, and trade-proposal routes, a paper-trading migration, and agent dashboards. These are prototypes, not conforming implementations of this architecture.

Phase 0 resolved (2026-07-01):

- ResearchAgent processSymbol now computes all 5 scores deterministically from real fetched data (lib/data/scores.ts). LLM only writes thesis + direction (no scores, no prices).
- PaperTrader uses lib/data/quotes.ts (AV GLOBAL_QUOTE + price_cache). No MCP/LLM for prices.
- Fill price = ask + 0.05% slippage via computeFillPrice() not LLM-estimated.
- Every fill logged to immutable paper_order_events with price source, bid/ask at fill, spread applied.
- LearnerAgent weight mutation blocked by phase gate (requires 10+ closed trades) + auto-guard (3 consecutive bad runs).
- Long-only enforcement: short direction overridden to neutral for non-held positions.
- Paper NAV refresh uses deterministic batch quotes (no MCP cold-start).

Remaining gaps (Phase 1+):

- Evidence store with source hierarchy, quality rules, and append-only evidence records does not exist.
- Corporate actions (dividends, splits, symbol changes) are not tracked.
- Macro data uses single macro_signals row no FRED/ALFRED vintage history.
- Strategy versioning, champion/challenger governance, and eligibility reports do not exist.
- Python Validation Engine does not exist no deterministic backtesting or historical replay.
- Provider adapters are not behind a replaceable contract interface.

Live trade execution remains approval-required. Paper results are not yet statistically valid evidence (no out-of-sample validation or benchmark attribution).

Proposed Behavior

The platform separates probabilistic reasoning from deterministic financial controls.

```

flowchart LR
    DH["Data Hub"] --> RA["Research Agent"]

```

```

RA --> AA["Hypothesis and Analyst Agent"]
AA --> VE["Validation Engine"]
VE -->|pass| SR["Strategy Registry"]
VE -->|fail| RJ["Rejected Experiment Record"]
RJ --> AA
SR --> PE["Paper Execution Engine"]
PE --> LA["Learner Agent"]
LA -->|challenger proposal| AA
SR --> RT["Risk and Tax Engine"]
RT -->|pass| LTG["Live Trade Gateway"]
RT -->|reject| NT["No-Trade Decision Record"]
NT --> LA
LTG --> UA["User Approval"]
UA -->|approve| RH["Robinhood Agentic Account"]
UA -->|reject or expire| NT
RH --> RC["Order and Fill Reconciliation"]
RC -->|failure| SP["Safety Paused"]
DH --> EA["Explainer Agent"]
RA --> EA
AA --> EA
VE --> EA
PE --> EA
RT --> EA
RC --> EA
EA --> DJ["Daily Brief and Decision Journal"]

```

Authority Boundaries

- Data Hub: acquires, validates, timestamps, versions, and stores evidence. It does not reason or trade.
- Research Agent: produces sourced research and labels unverified claims. It cannot create authoritative market facts.
- Hypothesis and Analyst Agent: defines hypotheses and candidate strategy versions. It cannot change a champion or production policy.
- Validation Engine: deterministically computes features, backtests, robustness tests, and eligibility reports.
- Strategy Registry: stores immutable versions and enforces lifecycle transitions.
- Paper Execution Engine: simulates approved paper strategies autonomously with realistic market mechanics.
- Learner Agent: reviews outcomes and proposes challengers. It cannot mutate the running champion.
- Risk and Tax Engine: deterministically calculates sizing, exposure, risk vetoes, dividend implications, and tax flags.
- Live Trade Gateway: the only module authorized to invoke Robinhood order review or submission tools.
- User Approval: Vaibhav is the only authority that may promote a live strategy or approve an initial live order.
- Explainer Agent: summarizes stored evidence, calculations, assumptions, and decisions. It cannot invent missing evidence.

Strategy Lifecycle and Promotion

Allowed states:

1. draft: hypothesis is defined but not validated.
2. testing: immutable version is undergoing historical and robustness tests.
3. rejected: evidence failed; reason and artifacts are retained.
4. paper_candidate: historical evidence passed and paper deployment is allowed.
5. paper_active: frozen version is generating shadow decisions and fills.
6. paper_paused: data, risk, or operational issue prevents valid continuation.
7. eligible: deterministic evidence report passed the dynamic gate.
8. approved_live: Vaibhav promoted the version for approval-required live proposals.
9. live_paused: live use is suspended without deleting history.

10. retired: version can no longer create new decisions.

eligible is not based on a fixed duration or trade count. The Validation Engine must produce a passing evidence report that assesses:

- Positive out-of-sample expectancy after modeled costs.
- Acceptable drawdown, downside risk, and recovery behavior.
- Effective independent sample size and confidence intervals.
- Market-regime coverage and regime-specific failure behavior.
- Benchmark-relative performance and unintended beta/factor exposure.
- Parameter stability and sensitivity.
- Multiple-testing, selection-bias, and overfitting penalties.
- Liquidity, turnover, and capacity.
- Correlation with active strategies.
- Paper-versus-backtest degradation.
- Tail scenarios and stress tests.

Champion/challenger governance:

- Each strategy family has at most one champion for a given universe and horizon.
- LearnerAgent creates a new immutable challenger version.
- Validation Engine determines statistical eligibility.
- Risk Engine can veto deployment but cannot promote it.
- Vaibhav alone promotes an eligible challenger to champion.
- The existing champion remains active until promotion is complete.
- Rejection returns the challenger to paper study or retires it; it does not modify the champion.

Research and Learning Flow

1. Observe a timestamped event or market condition.
2. Produce a sourced research packet with contradictions and missing evidence.
3. Define a falsifiable hypothesis: universe, horizon, entry, exit, expected direction, invalidation, and economic rationale.
4. Define deterministic features computable without an LLM.
5. Freeze a strategy version and dataset manifest.
6. Run point-in-time historical validation.
7. Run transaction-cost, slippage, delay, missing-data, parameter-perturbation, regime, and stress tests.
8. Compare against SPY, an appropriate sector/style benchmark, and a no-trade baseline.
9. Deploy passing versions to paper execution.
10. Evaluate resolved outcomes and create versioned challenger proposals.
11. Run the dynamic evidence gate.
12. Ask Vaibhav to approve or reject any live promotion.

Learning constraints:

- No weight change from one trade or one short observation window.
- No reuse of held-out evidence for tuning.
- One documented experimental change per challenger where practical.
- Failed and rejected experiments are never deleted.
- Safety may pause or retire a strategy automatically; promotion is never automatic.
- Research discoveries may affect future hypotheses but never rewrite historical evidence.
- The system distinguishes learned market evidence from LLM interpretation.

Data Architecture

Evidence Store

Evidence records are append-only and include observed time, effective time, retrieval time, source, source tier, revision, quality state, and payload hash.

Evidence categories:

- Daily OHLCV and corporate-action-adjusted prices.
- Quotes used for paper or live decisions.
- Dividend declaration, ex-dividend, record, and payment dates.
- Splits, mergers, symbol changes, halts, and delistings.
- SEC filings, XBRL facts, and Form 4 transactions.
- Earnings dates, results, estimates, and guidance.
- FRED/ALFRED macro observations with historical vintages.
- Regime inputs such as volatility, breadth, rates, and credit proxies.
- Robinhood account, position, order-review, order, and fill observations.
- External articles and social observations with verification state.

New source data never silently overwrites old evidence. Corrections create new versions.

Knowledge Store

Versioned knowledge records include:

- Market-mechanics rules.
- Approved risk and execution policy.
- Candidate, validated, rejected, decaying, and retired signals.
- Strategy definitions and assumptions.
- Experiment and eligibility reports.
- Regime-specific observations.
- Prediction and trade outcomes.

- Agent lessons tied to evidence.
- User decisions, approvals, rejections, and overrides.
- Tax, dividend, regulatory, and broker rules with effective dates.

A knowledge claim requires citations, confidence classification, effective date, and review state before it can influence a trade.

Immutable Events and Mutable Projections

- Evidence, decisions, strategy versions, experiment results, approval actions, order reviews, orders, fills, and learning events are immutable.
- Current portfolio state, current strategy state, latest quote cache, and dashboard summaries are mutable projections.
- Every projection must be rebuildable from immutable events.
- Corrections are compensating events, not destructive updates.

Source Hierarchy

1. Government, regulator, exchange, issuer, and broker sources.
2. Established structured-data providers.
3. Reputable financial reporting.
4. General web sources.
5. Social media and forums.

Levels 4 and 5 may generate hypotheses. A trade-affecting fact must be corroborated by a higher-tier source. Contradictions are quarantined. Missing or stale required data results in `insufficient_data` and abstention.

Free-First Provider Policy

- Robinhood MCP supplies the executable universe, live account state, and executable-decision quotes where supported.
- SEC EDGAR supplies filings, XBRL data, and insider transactions.
- FRED/ALFRED supplies macro data and historical vintages.
- Issuer investor-relations sources supply dividend and earnings evidence.
- TradingView supports manual analysis, alerts, Pine prototypes, and explicit CSV imports only.
- Historical market data is accessed through a replaceable adapter selected after accuracy, adjustment, coverage, rate-limit, and licensing tests.
- An Alpha Vantage MCP mentioned in external review is not considered available until it is added to `knowledge/CONNECTIONS.md` and its tools pass a contract test.

Validation Engine

The Validation Engine is a deterministic Python worker. Claude or another LLM may propose hypotheses and explain reports but cannot calculate authoritative results.

Responsibilities:

- Build point-in-time dataset manifests.

- Compute deterministic features and labels.
- Run historical replay with purging/embargo where labels overlap.
- Model spread, slippage, latency, dividends, splits, and costs.
- Run robustness and stress tests.
- Calculate performance, risk, uncertainty, capacity, and benchmark metrics.
- Produce signed/versioned experiment and eligibility artifacts.
- Reproduce a run from code version, strategy version, dataset manifest, configuration, and random seed.

Next.js orchestrates user requests and displays results. Supabase persists metadata, immutable events, and artifact references. The Python worker runs behind a job contract so hosting can change without changing strategy definitions.

Paper Execution

Paper and live execution consume the same order-intent, strategy, risk, and sizing contracts. Only the execution adapter differs.

Paper behavior includes:

- Long-only equities and ETFs.
- Market-calendar-aware decisions.
- Bid/ask spread, configurable slippage, delayed fills, rejections, and unfilled orders.
- Cash and settlement constraints matching the intended Robinhood behavior.
- Deterministic daily mark-to-market.
- Dividends, splits, symbol changes, halts, and delistings.
- Strategy-specific 2-20 market-day holding periods.
- Benchmark, after-cost, and after-estimated-tax attribution.
- Append-only order intents, simulated orders, fills, cash movements, corporate actions, and valuation events.

Live Execution and Approval

Allowed execution states:

- disabled
- approval_required
- auto_live_unlocked
- safety_paused

Initial behavior is always `approval_required`. A proposal must display quote, quantity, estimated value, thesis, invalidation, holding period, evidence quality, risk checks, tax flags, portfolio impact, and expiry.

Before submission, the Live Trade Gateway must:

1. Fetch current Robinhood account state and quote.
2. Verify account number `<agentic-account-id>`.
3. Re-run cash, risk, tax, event, liquidity, and freshness checks.

4. Call Robinhood `review_equity_order`.
5. Show any values that changed since proposal creation.
6. Require an unexpired explicit approval.
7. Submit once with an idempotency key.
8. Reconcile order state and fills.

User rejection or expiry creates a no-trade event and does not alter the strategy. A reconciliation failure enters `safety_paused`.

Future `auto_live_unlocked` requires a separate explicit authorization containing strategy version, maximum capital, allowed symbols or universe, risk policy version, start time, expiry time, and revocation state. No agent can create, extend, or broaden this authorization.

Risk Architecture

Hard controls apply regardless of model confidence:

- Volatility-targeted sizing.
- Maximum single-position, sector, factor, and total exposure.
- Daily loss and drawdown circuit breakers.
- Liquidity, spread, and execution-size limits.
- Earnings and corporate-event restrictions.
- Portfolio-correlation and crowding checks.
- LTCM correlated-shock, survival, and liquidity checks.
- Maximum daily order count.
- Stale, missing, or conflicting data veto.
- No averaging down unless explicitly specified and validated.
- No leverage, options, shorting, or non-agentic account access.

Risk failure creates a no-trade event with machine-readable reasons. It cannot be overridden by an LLM.

Dividend and Tax Architecture

Every relevant decision evaluates total return, not dividend yield alone.

Dividend inputs:

- Declaration, ex-dividend, record, and payment dates.
- Amount, frequency, currency, and special-dividend status.
- Expected mechanical ex-date adjustment.
- Payout coverage and sustainability evidence.
- Corporate-action revision history.

Tax-aware outputs:

- Tax-lot cost basis.
- Realized and unrealized gains.
- Short-term versus long-term status.
- Qualified-dividend holding-period estimate.
- Ordinary versus qualified dividend classification.
- Wash-sale exposure across all known relevant lots/accounts where data is available.
- Estimated after-tax expected return.
- Year-to-date realized tax exposure.
- `review_required` when treatment is uncertain.

Tax estimates are advisory. Robinhood tax documents and professional tax guidance are authoritative.

Evolving Rules and Knowledge Governance

Update authority depends on knowledge class:

- Market observations update automatically after data-quality checks.
- Corporate events update automatically after source validation.
- Research findings enter as unverified hypotheses until tested.
- Broker capability changes require contract validation before activation.
- Tax and regulatory changes must come from authoritative sources and create proposed rule versions.
- Risk and execution policy never changes automatically.

Rule-update flow:

1. Detect a source or rule change.
2. Store old and new versions with effective dates.
3. Corroborate when possible.
4. Produce a plain-language difference and impact report.
5. Identify affected strategies, holdings, lots, and pending decisions.
6. Re-run applicable calculations and tests.
7. Require Vaibhav's approval for behavior-changing rules.
8. Activate prospectively without rewriting history.

Ex-dividend data refreshes daily and immediately before any dividend-sensitive proposal or order. Tax sources are checked periodically and when authoritative updates are detected. Uncertainty produces `review_required` and can block tax-sensitive action.

Explainability and User Learning

Daily Briefing

The pre-market brief includes:

- Regime and confidence.
- Portfolio exposure and risk state.
- Earnings, macro, and ex-dividend events.
- Active paper strategies and experiments.
- Candidate opportunities ranked by evidence quality.
- Actions under consideration but not executed.
- Prior predictions versus outcomes.
- New lessons, failed assumptions, and uncertainty.
- Required user decisions.

Material changes can produce an updated brief. Every claim links to evidence or an experiment artifact.

Layered Decision Journal

Each decision exposes:

1. Plain-language summary.
2. Evidence, sources, timestamps, contradictions, and missing data.
3. Signals, calculations, regime, expected return, uncertainty, and benchmark.
4. Strategy version, governance state, risk checks, tax flags, approvals, order review, fills, and resolved outcome.

Every proposal answers why now, why this security, why this size, why act instead of wait, what invalidates the thesis, the strongest opposing case, the correlated-shock risk, and what similar historical experiments showed.

Content labels are explicit: `verified_fact`, `calculated_metric`, `model_estimate`, `agent_interpretation`, `unverified_hypothesis`, and `user_decision`.

User Journey / System Flow

1. Vaibhav opens the daily briefing and sees regime, risks, events, experiments, and pending decisions.
2. ResearchAgent continuously collects sourced observations and proposes hypotheses.
3. Validation Engine tests frozen candidate versions.
4. Passing candidates enter autonomous paper trading.
5. LearnerAgent reviews resolved evidence and proposes challengers.
6. Dynamic evidence gate may mark a challenger eligible.
7. Vaibhav reviews the complete evidence and promotes, rejects, or continues paper study.
8. An approved-live strategy may create a live proposal.
9. Risk and Tax Engine may veto it.
10. Vaibhav reviews the Robinhood order review and approves or rejects.
11. Live Trade Gateway submits and reconciles the approved order.
12. The journal later connects outcomes and lessons to the original decision.

Screen / Page / Module Inventory

Existing screens to evolve

- /dashboard/agents: system health, runs, experiments, champion/challenger status, and safety state.
- /dashboard/trading: paper/live portfolios, proposals, approvals, orders, fills, and kill switch.
- /dashboard/intelligence: sourced research feed and hypothesis inbox.
- /dashboard/markets: eligible universe, evidence quality, regime, events, and signals.
- /dashboard/portfolio: account, tax lots, exposure, dividends, and risk.

New product surfaces

- Daily briefing view.
- Decision journal detail.
- Strategy registry and version comparison.
- Experiment report and eligibility review.
- Data-quality and source-health view.
- Rule-change review.
- Auto-live authorization control, hidden until the future phase is approved.

UI Architecture

Layout

Use the existing dashboard shell and current approved design system. No independent visual redesign is included.

Required Components

- System safety state.
- Evidence-quality badge.
- Fact/estimate/interpretation labels.
- Strategy lifecycle badge.
- Champion/challenger comparison.
- Experiment metric and regime panels.
- Risk and tax check list.
- Source citations and freshness.
- Proposal expiry and changed-value warning.
- Approval/rejection controls.

Empty State

Explain which prerequisite is missing: no source data, no hypothesis, no validated strategy, no paper history, or no proposal. Never display fabricated sample results as real.

Loading State

Show the specific run or data source being awaited and retain the prior timestamped result as stale, not current.

Error State

Show the failed dependency, affected strategies, whether execution is blocked, retry state, and audit identifier.

Success State

Show persisted run/result identifiers and the next lifecycle action. A successful research run is not described as a successful strategy.

System Architecture

Modules

- TypeScript/Next.js orchestration and authenticated API layer.
- Supabase immutable events, evidence metadata, projections, auth, and RLS.
- Python validation worker behind a job contract.
- Replaceable market, filings, macro, issuer, and broker adapters.
- Deterministic paper execution, risk, tax, and live gateway modules.
- LLM research, hypothesis, learner, and explainer modules with structured outputs.
- Scheduler and run monitor suitable for a future Railway worker deployment.

Core Contracts

- EvidenceRecord: source, tier, observed/effective/retrieved times, revision, quality, hash, payload reference.
- ResearchPacket: symbol/universe, claims, citations, contradictions, missing evidence, interpretation, confidence.
- Hypothesis: universe, horizon, direction, entry, exit, invalidation, features, rationale, expected failure modes.
- StrategyVersion: immutable definition, parent, change summary, code/data/config versions, lifecycle state.
- ExperimentRun: strategy version, dataset manifest, costs, metrics, regimes, artifacts, reproducibility metadata.
- EligibilityReport: gate results, uncertainty, failures, risk vetoes, recommendation, generated time.
- OrderIntent: strategy, account, symbol, side, quantity logic, limit logic, expiry, thesis, evidence references.
- RiskDecision: pass/reject, policy version, sizing, exposure, stress results, reasons.
- TaxDecision: lots, holding periods, dividend status, wash-sale flags, after-tax estimate, uncertainty.
- ApprovalRecord: user, exact reviewed payload hash, decision, time, expiry.
- ExecutionEvent: review, submit, broker order, status transition, fill, reconciliation.
- DecisionJournalEntry: linked evidence, calculations, interpretations, governance, and resolved outcome.

Exact database columns and endpoint payload schemas belong in the implementation plan and migrations, derived from these contracts without changing their semantics.

Auth and Permissions

- All user-facing and agent-triggering routes require Supabase authentication.
- Only the superadmin user may approve live strategies, orders, rule changes, or auto-live authorizations.
- Service jobs use a narrowly scoped server credential and cannot call the live gateway unless the authorization contract permits it.
- Robinhood tokens are never persisted in application tables or logs.
- Account <agent-ic-account-id> is enforced at the gateway and verified against the broker response.

Failure Handling

- Missing/stale price: no decision or order.
- Conflicting sources: quarantine and abstain.
- Invalid LLM structured output: discard; never partially persist as evidence.
- Provider outage: pause dependent strategies.
- Missed scheduled run: do not backfill trades using future data.
- Duplicate request: idempotent replay of the prior result.
- Partial fill: reconcile before further dependent orders.
- Account mismatch: immediate safety pause.
- Risk calculation unavailable: no trade.
- Tax uncertainty: warn or block based on policy severity.
- Repeated job failure: bounded retry, dependency pause, and alert.

Scheduling and Operations

- Overnight: data reconciliation and corporate actions.
- Pre-market: evidence refresh, regime calculation, event calendar, and daily brief.
- Intraday: material event and source-health monitoring appropriate for swing trading.
- Post-close: valuation, paper fills, outcome checks, and experiment metrics.
- Weekly: strategy, challenger, decay, and data-quality review.
- Periodic: authoritative tax, regulatory, broker-capability, and source-policy review.

Every run stores inputs, outputs, code/config versions, timing, status, errors, freshness, and retry history.

Testing Architecture

- Unit tests for calculations, lifecycle transitions, sizing, dividends, taxes, and authority boundaries.
- Provider contract tests for freshness, adjustments, rate limits, schema, and error behavior.
- Historical replay tests using point-in-time fixtures.
- Explicit leakage, survivorship, and corporate-action tests.
- Deterministic paper-order and accounting tests.
- Failure injection for stale data, conflict, outage, duplicate order, partial fill, and reconciliation failure.

- Security tests proving only Live Trade Gateway can review or submit Robinhood orders.
- Tests proving initial live orders require an exact, unexpired approval payload.
- Audit reconstruction proving a decision can be reproduced from stored inputs and versions.
- Golden report tests for eligibility and decision-journal outputs.

Delivery Phases

Phase 0: Stabilize Current Agent Loop

- Disable LLM-derived entry and exit prices.
- Disable direct automatic weight mutation.
- Enforce long-only behavior.
- Add a deterministic quote adapter with freshness and provenance.
- Correct paper accounting, valuation, and idempotency.
- Introduce append-only paper events and rebuildable projections.
- Reconcile migration and documentation state.
- Keep all live behavior approval-required.

Phase 1: Data Foundation

- Evidence store, provider adapters, source hierarchy, quality rules, corporate actions, dividends, and macro vintages.

Phase 2: Research Laboratory

- Sourced research packets, deterministic hypotheses/features, Python Validation Engine, experiment artifacts, and Strategy Registry.

Phase 3: Paper Engine

- Realistic shadow execution, benchmarks, attribution, dynamic eligibility, and champion/challenger experiments.

Phase 4: Explainability

- Daily briefing, layered decision journal, rule-change reports, and learning experience.

Phase 5: Approval-Required Live

- Robinhood gateway, exact-payload approval, risk/tax checks, order review, idempotent submission, and reconciliation.

Phase 6: Limited Auto-Live

- Available only under a separately approved architecture amendment after approval-required live execution has been verified.

Files Likely To Change

The implementation plan will narrow each phase. Expected areas include:

- app/api/agents/
- app/api/portfolio/
- app/dashboard/agents/
- app/dashboard/trading/
- app/dashboard/intelligence/
- app/dashboard/markets/
- app/dashboard/portfolio/
- components/dashboard/
- lib/agents/
- lib/data/
- lib/execution/
- lib/risk/
- lib/tax/
- types/index.ts
- supabase/migrations/
- A focused Python validation-worker directory selected in the implementation plan
- knowledge/

Files / Behavior That Must Not Change

- Do not modify AGENTS.md or PRD.md without Architect role and explicit Vaibhav approval.
- Do not access any Robinhood account except <agent-id>.
- Do not enable live execution without the approved gateway and exact approval workflow.
- Do not add options, shorting, leverage, crypto, or intraday scope.
- Do not use an LLM as an authoritative market-data, P&L, fill, tax, or risk source.
- Do not silently replace existing approved visual direction.
- Do not add a data provider or package without the required architecture and user approval.
- Preserve existing useful UI and integrations unless a phase-specific plan explicitly replaces them.

Acceptance Criteria

Architecture-wide

- Every decision is attributable to immutable evidence, deterministic calculations, strategy version, policy versions, and authority actions.
- Every calculation that affects money is reproducible without relying on LLM memory or prose.
- Missing, stale, or contradictory required evidence results in abstention.
- No agent can promote itself, approve its own live strategy, or execute outside the Live Trade Gateway.

Stabilization phase

- No production route asks an LLM for an authoritative security price.
- Paper execution is long-only and accounting identities reconcile from immutable events.
- Paper NAV uses deterministic, timestamped prices and includes modeled execution costs.
- LearnerAgent cannot directly mutate champion weights.
- Existing live proposals remain approval-required.

Research and validation

- A frozen strategy can be replayed from its versioned dataset and configuration.
- Eligibility reports expose every passing and failing gate with uncertainty.
- Failed experiments and rejected challengers remain queryable.
- Benchmarks, costs, regime coverage, and paper/backtest degradation are included.

Live execution

- Only account `<agent:ic-account-id>` can pass gateway validation.
- An order cannot submit without current data, passing risk/tax checks, Robinhood review, and an exact unexpired user approval.
- Duplicate submission does not create a duplicate order.
- Partial fills and broker discrepancies reconcile or trigger `safety_paused`.
- Auto-live remains unavailable until a separately approved later phase.

Explainability

- Daily briefing and decision journal distinguish fact, calculation, estimate, interpretation, hypothesis, and user decision.
- Every displayed claim links to evidence or a stored experiment.
- Every trade proposal shows invalidation, counterargument, correlated-shock risk, tax/dividend flags, and why waiting was rejected.

Implementation Log

Phase 0 — Complete (2026-07-01)

See ARCHITECTURE.md Session 2026-07-01 section for full detail.

All Phase 0 acceptance criteria met:

- All 5 ResearchAgent scores deterministic (no LLM score generation)
- LLM role: thesis + direction only
- PaperTrader uses deterministic quote adapter (`price_cache + AV GLOBAL_QUOTE`)
- Fill price = ask + 0.05% slippage via `computeFillPrice()`
- Immutable `paper_order_events` with UPDATE/DELETE blocked

- LearnerAgent phase gate (≥ 10 closed trades before weight mutation)
- Auto-guard (3 consecutive bad runs blocks mutation)
- Long-only enforcement enforced in code

Phase 0 Addendum — 2026-07-03 Session

Live Portfolio + Trade History Layer added outside original Phase 0 scope (does not block Phase 1 gate).

All-Accounts ResearchAgent Fix

- `lib/research-agent.ts::fetchHoldings()` — uses only the latest snapshot per live account, then unions open paper alpha positions; older snapshots cannot keep closed names in the universe
- Deterministic held-position reassessment is available across live and paper books; holdings bypass discovery caps

Trade Decision Enrichment Pipeline

- Migration 043 applied: `trade_decisions` + `uploaded_trade_files`
- `/api/live-portfolio/enrich` — batch enrichment with AV `TIME_SERIES_DAILY_ADJUSTED`; computes `outcome_score`, `macro_market_regime`, `pattern_tags`
- `/api/live-portfolio/import-csv` — SHA-256 dedup; Robinhood CSV parsing
- `LivePortfolioPage.tsx` — CSV import UI, enrichment trigger, decisions table

LearnerAgent Enhancements

- Tool `query_trade_decisions` added (tool #7) — queries real enriched trades by action/regime/outcome range
- Model upgraded: `deepseek-chat` → `claude-opus-4-8` (89.08% finance accuracy, AIMultiple benchmark)

Phase 1 — Planned [REVIEW PENDING — ChatGPT]

Status: Not started. Gate: Phase 0 complete ■. Can begin.

Key Phase 1 items from original spec + new additions:

1. Evidence store (Evidence categories defined in Data Architecture section above)
2. Corporate actions (dividends, splits, symbol changes) automated tracking
3. Macro data: FRED/ALFRED vintage history (currently: single `macro_signals` row)
4. Provider adapter contracts (replaceable interface in front of AV/FinancialDatasets/Robinhood)
5. **RAG pipeline** [REVIEW PENDING — ChatGPT]
 - Voyage-3.5 embeddings (finance-tuned) for `trade_decisions`
 - Supabase pgvector table: `trade_decision_embeddings(decision_id, embedding vector(1536))`
 - `gte-reranker-modernbert-base` post-retrieval reranker
 - New LearnerAgent tool: `semantic_search_decisions`
 - Migration 044 needed

6. Langfuse observability [REVIEW PENDING — ChatGPT]

- Wrap `lib/llm-router.ts` with Langfuse SDK
- Each agent run = trace; each LLM call = generation span

7. Firecrawl ResearchAgent integration [REVIEW PENDING — ChatGPT]

- Firecrawl MCP active (needs new Claude session)
- Adds `crawl_url` tool to ResearchAgent; max 3 calls/run
- Source quality: Firecrawl output = unverified hypothesis (level 4 in source hierarchy)

8. ResearchAgent model upgrade [REVIEW PENDING — ChatGPT]

- Current: Groq 70B for thesis
- Proposed: Claude Fable 5 (90.34% finance accuracy) — pending cost gate (\$2/day budget)

Approval

Architecture approved: Yes

Approved scope: Governed long-only swing-trading research, paper experimentation, explainability, tax/dividend awareness, and initial approval-required live trading as specified above

Implementation allowed: Yes (phased)

Phase 0 complete: 2026-07-01

Next gate: Vaibhav approves Phase 1 implementation plan (Evidence store, provider adapters, source hierarchy, corporate actions, macro vintages).

Audit-Driven Remediation & Adaptive-Quant Target (2026-07-10)

Consolidates the full-system Codex audit (`CODEX_FULL_SYSTEM_AUDIT_RESULT.md`) and its A–J target spec into one governed roadmap. Owner directive: work every item until done.

Verified current-state truth (spot-checked against live DB + code, not docs)

- **Autonomous-live never places an order.** `lib/trading/autonomous-live.ts` selected `agent_signals.evidence_confidence`, a column that does not exist (real column is `agent_signals.confidence`; `evidence_confidence` lives on `decision_observations`). The query error was swallowed → zero signals every run. DB-confirmed.
- **Autonomous path bypasses the hardened Execution Gateway** — direct broker-client calls, omitting `checkKillSwitches`, account allowlist, G1/G3, quote-drift, preview, held-SELL.
- **India autonomous sizing uses the USD Robinhood NAV account for INR orders** — dimensionally invalid.
- **Migrations 139/140 + `reserve_live_order_budget_v2` are applied in prod but have no files** —

clean DB cannot rebuild. (RPC verified atomic in prod; the gap is reproducibility.)

- **Calibration leakage** — logistic fit on all rows, then "OOS" fold predictions use those same full-data coefficients. Learned P(win) untrustworthy until fold-local.
- **Paper learning loop IS partly real** — owner-promoted champion weights + genome do drive next-run scoring and paper sizing/exits. Live path and feature-registry/edge-lab loops are inert/broken.

AUTONOMOUS_LIVE_ENABLED stays **false** until the money-path unification + acceptance tests pass.

Target architecture (A–J, condensed — the north star)

- **A. Separate prediction / portfolio-construction / execution** into independently testable layers.
- **B. Small mixture of validated setup experts** (quality-momentum, price-trend/RS, PEAD, value inflection, ETF rotation, India quality-momentum) — not one ever-growing composite; not a 6th weight.
- **C. Real learnable genome** (features, weights, routing, entry/exit/sizing/universe/costs) — bounded, hashed, immutably linked to dataset/labels/code/validation. Money ceilings, accounts, kill switches, autonomy mode, secrets, ledger are **not genes**.
- **D. Research tournament**: hypothesis → measure-only → purged walk-forward → shadow → paper A/B → owner live-review → capped canary → scale/retire. Deflated-Sharpe/FDR, lockbox, no "any-horizon-wins".
- **E. Point-in-time, market-specific data plane** (event/available/retrieved times, dated universe + delistings, corporate-action-consistent prices, US+NSE calendars, separate USD/INR NAV).
- **F. Learn from the broad decision population** (every considered candidate incl. rejects), not fills only.
- **G. Performance-truth + attribution** per market (TWR/MWR, alpha/beta, Sharpe/Sortino/Calmar, DD, turnover, net-of-cost, calibration curves).
- **H. Autonomy as one governed control system** — shared execution service, market/account/currency pinned, atomic claim+idempotency, fresh kill-switch, reconciliation, protective exits, short lease, tiny capital, instant disable.
- **I. Honest success metric** — beat an investable benchmark net of costs within DD/tail limits; degrade safely; adapt faster than edge decay; full reproducibility. NOT "beat everyone in every regime".
- **J. 16 acceptance tests** (PIT reproducibility, no future leakage, no US/India cross-contamination, no research→money jump, no duplicate orders, one shared gateway, kill cancels resting orders, RLS matrix, clean-DB replay). Feature is not "done" until these pass.

Remediation checklist (work top-to-bottom; update status inline)

Legend: [] todo · [~] in progress · [x] done

Track 1 — correctness / safety / reproducibility (no new architecture; unambiguous fixes)

- [x] R1 #4 autonomous-live signal query: select real confidence, capture error, FAIL run + alert on error
- [x] R2 #5 add checkKillSwitches(svc, market) to the autonomous path (fresh, per market, fail closed)

- [x] R3 #1 restore migrations 139/140 + reserve_live_order_budget_v2 + broker_order_events DDL from prod → files (143_restore_live_auto_ddl.sql)
- [x] R4 #9 autonomous fail-closed: require valid future lease + per-market trading_enabled_*===true (no null-passes)
- [x] R5 #14 calibration fold-local fitting (fit scaler+logistic inside each train fold; refit-all only for deploy artifact)
- [x] R6 #19 positive-allowlist score_source='deterministic_v1' on paper+trader eligibility (was fail-open negative filter)
- [x] R7 #22 evidence_confidence = weighted structural coverage over the 5 scored dims (not dimension count)
- [x] R8 #33 weighted-score: explicit abstain flag for <2 dims (no meaningless partial number mistaken for a score)
- [~] R9 #16/#31 owner-gate: DONE smart-money, theme-scout GET, mentor evaluate/journal/scores, import-csv. TODO metered public data (options/chain, social/sentiment, charts/*)
- [x] R10 #17/#32 RLS: owner-scoped corporate_actions/evidence_records/experiment_runs/paper_order_events/strategy_versions/trade_decision_embeddings + service-only universe_snapshots* (migration 144)
- [x] R11 #18 vault: PIN stored as salted scrypt hash (lib/vault-pin.ts), legacy-plaintext compat verify (no lockout). Key-value envelope-encryption still TODO (separate, larger).
- [x] R12 #34 redacted literal cron secret from migration 022 text (dead — job unscheduled by 052, CRON_SECRET already rotated). NOTE: git history still contains it (rotation, not rewrite, is the real fix — done).

R13 design note — shared execution service (REVIEW BEFORE CODING)

Problem: the manual gateway (app/api/broker/orders/route.ts) is the hardened path (owner+CSRF, approved/unexpired proposal, dup check, symbol/qty validation, active-broker resolve, autonomy-level + trading + broker enablement, fresh checkKillSwitches, G1 decision quality, account allowlist, fresh-quote + currency notional cap, portfolio gate, held-SELL, drift/preview, atomic reservation). autonomous-live.ts reimplements a WEAKER subset and calls brokers directly. Two implementations = drift + the autonomous path missing controls.

Target: one server-only lib/trading/execute-order.ts::executeApprovedOrder(svc, input, actor):

- actor: { kind: 'owner' | 'autonomous_worker', runId?, csrfOk?, ownerSessionOk? }
- Runs the FULL invariant list once, in order, for both callers. Autonomous may NOT supply any owner-only override (accept-low-quality, force flags); those are rejected when kind !== 'owner'.
- The ONLY broker-write call site in the codebase. Returns a typed result (submitted | rejected | needs_reconcile + broker ref + the durable reservation id).

Safe migration order (each step build+verify before the next):

1. Extract the invariant block from broker/orders/route.ts into executeApprovedOrder with an owner actor — **the manual route delegates to it and behaves identically** (diff = pure move).
2. Verify the manual owner-click path is byte-for-byte equivalent (same gates, same responses).
3. Repoint autonomous-live.ts to call executeApprovedOrder with autonomous_worker — deleting its direct rhPlaceMarketOrder/placeEquityOrder calls and its weaker inline checks.
4. Kite manual route (app/api/kite/order) folded in the same way.

Risk control: step 1–2 must not change manual behavior at all. If any manual gate/response differs, stop. Autonomy stays false throughout. No new capability — only consolidation.

Depends on: R14 (per-market NAV — done), R15 (idempotency — done). Blocks R16 (exit plane).

STATUS: awaiting owner review of this approach before implementation (it modifies the working manual order path).

Track 2 — money-path unification (architecture-gated; build after Track 1)

- [x] R13 #2/#5/#10/#11 shared `lib/trading/execute-order.ts::executeApprovedOrder(svc, input, actor)` — the SINGLE hardened invariant set. Manual gateway delegates (owner actor); autonomous-live delegates (autonomous_worker actor, its own daily caps, no overrides). Autonomy now inherits account allowlist, fresh kill-switch, G1/G3, notional cap, drift, held-SELL, atomic v2 reservation. Serverless caveat RESOLVED: added a direct-REST Robinhood adapter (`lib/brokers/adapters/robinhood.ts, id robinhood`) — submit/status/cancel over REST, works in Vercel serverless. Owner sets `strategy_config.active_broker_us='robinhood'` to route live US orders through it (both manual + autonomous). Flag stays false until R16 + acceptance tests.
- [x] R14 #3/#8 per-market currency-correct NAV (US=RH USD snapshot, India=Kite INR margins+holdings, fail-closed) + per-market NET open-position count (was global filled count)
- [x] R15 #7 atomic signal claim — unique partial index on `trade_proposals(signal_id,market)` WHERE `autonomous_live` (migration 145) + 23505 idempotent skip. (Broker-level client-order-key deferred — RH REST lacks native idempotency.)
- [x] R16 #6 control plane COMPLETE: reconcile (broker-sync polls `getOrder` → fill/partial/status write-back + mismatch alert, uses the robinhood REST adapter) + cancel-on-kill (sync cron cancels resting live orders when a kill switch trips) + LIVE protective exits (`lib/trading/live-exit-monitor.ts` + cron every 30min in-session: reconstructs open live positions from filled `broker_orders`, SELLS via the gateway on stop -8% / target +20% / time 15d; SELLS are held-only, cap-exempt, journaled). Market-HOLIDAY + authoritative AV `MARKET_STATUS` gate in `market-calendar.ts`.
- [x] R17 #12/#28 per-market cron (?market=us 15:00 UTC / ?market=india 06:00 UTC) + `isMarketSessionOpen` session-window guard (no market orders when exchange closed, DST-correct). Full managed-scheduler consolidation still TODO.
- [~] R18 #13 learner baseline now from market champion snapshot + full-vector renormalize to 1.0. TODO #15: transactional per-market promote + `force_unvalidated→paper-only` (strategies/versions route).

Track 3 — statistical + evolution integrity (after Track 2)

- [] R19 #21 label maturation: exchange-calendar sessions + immutable executable decision price (no future candle)
- [] R20 #23/#24 edge lab: dated PIT universe + delistings, predeclared horizon + FDR, Newey-West lag = overlap
- [] R21 #26 rename feature-registry "active"→"research_validated"; wire only via shadow→paper→promote
- [] R22 #27 autonomous sizing: per market/setup/horizon calibrated artifact; unknown edge → size 0
- [] R23 #20 held-exit direction deterministic (LLM advisory-only schema: veto/rationale/risks)
- [] R24 #29/#30 past-trade: RFC-4180 parse, immutable raw rows, transaction taxonomy, position/lot episodes, session/CA-consistent enrichment

Track 4 — target architecture (A–J; phased, data-gated)

- [] R25 skeleton: three-layer contract (alpha model → portfolio constructor → execution policy) interfaces + provenance columns
- [] R26 setup-expert framework (B) with one real expert in shadow
- [] R27 J acceptance-test fixtures; clean-DB replay in CI
- [] R28 performance-truth attribution extension (G)

Docs (`docs/arch/*`, `system-map.json`, `AGENTS.md`, `PRD.md`) reconciled per §7 drift register as each item lands.

Source: features\alpha-diagnostic-lab\FEATURE_ARCHITECTURE.md

Alpha Diagnostic Lab

2026-08-28 v2 amendment: P0 implementation defects found by the independent review are governed by
MEASUREMENT_INTEGRITY_REPAIR_ARCHITECTURE.md. That approved amendment supersedes conflicting cohort, replay, robustness, fingerprint, sample-unit, path-availability, and UI details below. The read-only safety boundary remains unchanged.
Status: APPROVED 2026-08-27 — IMPLEMENTATION IN PROGRESS
Reviewed and corrected before build: four factual claims re-verified against production, one found inverted (target_pct), one dependency found untracked.
See section 2 and the preconditions in section 13.
Architect: Codex / GPT-5, explicitly authorized by Vaibhav on 2026-08-27
Influence: read-only diagnostics only. No score, eligibility, sizing, exit, paper/live portfolio, strategy promotion, proposal, order, or broker path may consume this feature.

1. Decision

Kairos will build one owner-only, market-local Alpha Diagnostic Lab that explains portfolio underperformance as a reproducible funnel:

data truth -> candidate opportunity -> entry selection -> fill conversion -> sizing -> exit path -> cash/redeployment -> costs -> net benchmark-relative return.

The owner approved three defaults:

1. The primary objective is **net excess return against the governed primary benchmark**, subject to a drawdown non-inferiority constraint. Absolute return, win rate, and "win big / lose small" are supporting diagnostics, not the promotion objective.
2. Portfolio counterfactuals redeploy cash released by an exit into the next eligible candidate under deterministic same-session ordering: exits first, then entries ranked by the frozen candidate policy, then lexical symbol/id as the final tie-breaker.
3. Accounting truth includes every real paper trade, including tainted rows.

Learning and policy comparisons use a separate clean cohort that excludes tainted or excluded_from_learning rows. Both cohorts are displayed together; neither may silently substitute for the other.

2. Why this feature exists

The paper portfolios do not consistently beat their governed primary benchmarks, but a single headline alpha number cannot identify the cause. The same visible gap can come from:

- incorrect NAV or benchmark alignment;
- a candidate universe with no benchmark-relative opportunity;
- a useful candidate pool but poor ranking or downstream selection;
- good selected names damaged by size allocation;
- winners surrendered or losers held by the exit path;
- uninvested cash in a rising benchmark;
- costs, stale prices, capacity limits, or failed fills;
- one market or regime being pooled with another; or
- a retrospective result selected from too many trials.

Existing production evidence makes diagnosis urgent but does not license a money-path change.

****Motivating figures re-verified 2026-08-27 after the benchmark-provenance fixes landed the same day.**** Two of the numbers this section originally carried were computed on a contaminated series and are corrected here; the correction does not weaken the case for the Lab, it demonstrates why the Lab must gate every conclusion on A0 data truth.

claim	as drafted	re-verified	note
US 1M excess	~-0.53 pp	-0.67 pp	window boundary; restate with the window pinned
India 1M excess	~-1.76 pp	-0.68 pp	drafted on the Yahoo-contaminated NIFTY series; recomputed on settled Upstox closes
clean profit factor < 1 both markets	asserted	confirmed — US 0.735, India 0.906	currency profit factor, clean cohort
live target vs h10 p75 MFE	"~+20% target sits far beyond"	STALE — live target is 8%	trading_mandates.target_pct = 8 for BOTH markets since 2026-08-03; the code fallback also read 20 until it was corrected 2026-08-27

The target premise is not merely stale, it is **inverted**: against an h10 p75 MFE of ~7.75% (US) and ~8.93% (India), an 8% target is well calibrated, not far beyond reach. A3/A4 must therefore test whether the target is reached and given back, NOT assume it is unreachable. Any residual "+20%" evidence in older

artifacts describes a policy that is no longer live.

Since-inception, on settled benchmark data: India **+0.60 pp** ahead of NIFTY 50, US **-0.74 pp** behind VOO (both from 2026-07-06, 38-39 sessions). India is not uniformly behind, which is itself a reason to diagnose per market rather than pool.

These are hypotheses about where the damage occurs, not permission to tune the policy to this sample.

3. User and product value

User: the single owner/operator of Kairos.

Pain: the app reports whether the portfolio is ahead or behind, but it does not give one reproducible answer to *why*, nor show which proposed repair survives capital constraints, redeployment, costs, and benchmark-relative evaluation.

Expected value: shorten the loop from "portfolio lagged" to a falsifiable, market-local diagnosis; prevent random score/stop/target changes; and focus future build effort on the stage that actually destroys alpha.

4. Scope

In scope

- Paper books only in the first release.
- US/USD and India/INR as fully separate runs.
- Governed primary benchmark only for pass/fail: VOO for US, NIFTY 50 for India.
- Secondary benchmarks as display-only context.
- Persisted production ledgers as the data source; no provider call from a diagnostic run.
- Accounting and clean-learning cohorts.
- Data-truth, alpha-funnel, selection, payoff-path, sizing, portfolio/cash, execution-cost, and robustness tests.
- Immutable experiment identity, dataset/run fingerprints, and trial accounting.
- Owner-triggered runs plus scheduled read-only refreshes.
- A compact Portfolio performance surface with drill-down into the test results.

Out of scope

- Any automatic strategy, weight, threshold, stop, target, horizon, sizing, or universe change.

- Live portfolio evaluation or live-order behavior in P0.
- LLM-authored numeric conclusions or policy parameters.
- Intraday fills inferred from daily candles when event ordering is unknowable.
- Pooling USD and INR, or borrowing evidence from one market to configure another.
- Benchmark shopping: a display comparator cannot become the governed objective.
- New market-data providers or npm packages.
- Automatic promotion. A diagnostic verdict may only recommend continued collection, rejection, or submission for owner review.

5. Existing components to reuse

Need	Existing authority
Portfolio/benchmark scorecards	benchmark_scorecard, lib/analytics/benchmark-alpha.ts
NAV, expectancy, drawdown, costs	paper_performance, paper_trades, lib/analytics/performance-metrics.ts
Candidate decisions and PIT features	decision_observations
Forward, benchmark-neutral, MAE/MFE labels	observation_labels
Dimension and agent diagnostics	dimension_diagnostic_runs/findings, lib/learning/dimension-diagnostics.ts
Exit counterfactual outcomes	observation_labels.atr_exit_outcomes, lib/learning/atr-exit-evidence.ts
Horizon extension evidence	horizon_extension_shadow
Frozen replay boundary	lib/replay/sealed-accessor.ts, lib/replay/cursor.ts
Portfolio accounting simulation	lib/simulation/portfolio-simulator.ts
Immutable experiment lineage	backtest_experiments

The Lab must call or compose these authorities. It must not reimplement a second version of benchmark alignment, label math, technical scoring, or paper accounting.

6. Objective and cohort contracts

6.1 Primary objective

For a fixed market, calendar window, starting capital, and primary benchmark:

$$\text{net_excess_return} = \text{portfolio_total_return_after_costs} - \text{benchmark_total_return}$$

A candidate is directionally better only when paired net excess return is positive against the incumbent on the same sessions. It is not promotion-eligible unless maximum drawdown is non-inferior within a predeclared tolerance and all coverage/robustness gates pass.

Information ratio, daily excess hit rate, turnover, cash utilization, and tail loss are reported. They do not replace the primary paired comparison.

6.2 Accounting cohort

Includes every real closed paper lot and every portfolio/NAV effect, including tainted or excluded rows. It answers: "What happened to the book?"

6.3 Learning cohort

Requires all of:

- `tainted = false;`
- `excluded_from_learning = false;`
- valid market-local fill and exit prices;
- known code/mandate lineage where the test depends on it;
- sufficient benchmark and label provenance for the requested metric.

It answers: "What evidence may be used to compare policies?" Excluded counts and reasons are mandatory output fields.

7. Repeatable diagnostic suite

Every finding carries `market`, `test_id`, `cohort`, `window`, `n_rows`, `n_dates`, `n_symbols`, `coverage`, `metric version`, and a deterministic status of `pass` | `fail` | `insufficient_evidence` | `data_invalid` | `descriptive_only`.

A0 — Data truth and reconciliation

Before any performance conclusion, assert:

- market NAV equals cash plus one mark per open quantity within rounding tolerance;
- fills, realized P&L, lot quantity, and portfolio cash reconcile;
- benchmark close belongs to the same exchange session as the portfolio point;
- portfolio and benchmark returns recompute from their stored levels;
- missing/stale/tainted marks are counted and visible;
- exit rows retain required provenance and no cross-market currency is mixed.

Any failed invariant sets the whole run to `data_invalid`. Downstream values may be shown for debugging but cannot receive a pass/fail interpretation.

A1 — Alpha funnel

Per decision session, count and compare:

`scored` -> `entry_eligible` -> `selected/claimed` -> `filled` -> `closed`.

For each stage, calculate equal-weight benchmark-neutral return at h5/h10/h20,

coverage, and the deterministic reason for attrition. Required comparisons:

- all scored candidates;
- all eligible-long candidates;
- selected candidates;
- actual fills;
- eligible but capacity/cash/name-cap rejected candidates.

This test locates whether alpha is absent at discovery, lost in ranking, or lost between eligibility and execution.

A2 — Entry selection and calibration

Use one observation per (market, symbol, decision_date), earliest eligible row of the day. Report:

- date-clustered rank IC by horizon;
- top-minus-bottom quintile benchmark-neutral spread;
- monotonicity across frozen score bands;
- excess-return hit rate;
- analyst-score calibration and sample per bin;
- results by setup, regime, instrument family, and discovery source only when the cohort floor is met.

Frozen comparisons are incumbent, equal dimension weights, each dimension alone, and leave-one-dimension-out. Each is one declared trial. No ad hoc combination is generated after results are observed.

A3 — Payoff path: "win big, lose small"

For matched closed trades and matured labels, report:

- win rate, average/median winner, average/median loser, payoff ratio;
- percentage-return profit factor (policy quality independent of size);
- currency-P&L profit factor (real capital outcome);
- MFE and MAE distributions;
- $\text{realized_return} / \text{MFE}$ capture ratio where MFE is positive;
- $\text{MFE} - \text{realized_return}$ giveback;
- share of realized losers that were previously positive;
- share of winners/losses contributed by the top/worst five trades;
- results by exit reason and holding period.

Both percentage and currency profit factors are required because a positive

average trade with a negative currency profit factor is direct evidence that sizing damaged the book.

A4 — Exit precedence and counterfactual paths

Replay completed daily bars with the actual entry, stop, target, score-exit, partial-exit, time-stop, and cost conventions. Record when multiple exit conditions were true on the same evaluation session; the incumbent's real precedence decides the reproduced fill and a separate overlap field exposes the classification effect.

Candidate families must be predeclared. MFE/MAE-only rows may measure reachability but cannot infer the order of two barriers touched in the same window. Such rows are ambiguous, never silently assigned a favorable fill.

A5 — Sizing attribution

Replay the same selected trades under:

1. actual historical sizing;
2. equal notional per available slot;
3. capped volatility-normalized sizing;
4. the incumbent size rule with all other policy fields frozen.

Report return and currency P&L by entry-notional quartile, Spearman correlation between notional and later return, concentration, marginal contribution, profit factor, drawdown, and net excess return. The test diagnoses sizing; it does not select a new sizing formula.

A6 — Portfolio construction, cash, and redeployment

Extend the deterministic simulator to produce daily marked NAV, open-position value, cash utilization, benchmark NAV, drawdown, turnover, and rejected-event reasons. Run the same calendar under:

- actual entries/exits/sizes;
- actual selection with equal sizing;
- eligible-candidate equal weight subject to the same name cap;
- actual policy with cash released by exits redeployed into the next eligible candidate;
- an explicit cash sleeve so the cost of abstention is measured rather than assumed.

Ordering is deterministic: exits, then ranked entries, then lexical tie-breaker.

Cash cannot be spent twice and never becomes negative.

A7 — Execution and cost stress

Evaluate the incumbent and every candidate at:

- recorded modeled costs;
- 10, 25, and 50 basis-point round-trip stress;
- next-session-open entry where supported by point-in-time bars;
- stale quote rejection;
- gap-through-stop behavior without assuming a fill better than the observable price convention;
- no same-session redeployment as an adverse operational case.

Report cost drag, turnover, rejected fills, and the fraction of gross alpha that survives after cost.

A8 — Robustness and falsification

- Sealed point-in-time inputs; any future-dated item aborts the run.
- Purged/embargoed walk-forward evaluation for overlapping labels.
- Date-block bootstrap or date-clustered inference; raw row count is never presented as independent sample size.
- Regime holdout and market-local evaluation.
- Label-permutation placebo: a policy must not appear similarly successful on shuffled outcomes.
- Trial-family accounting and a multiple-testing-adjusted review hurdle.
- Re-run identity: identical plan, dataset, and code fingerprints produce the same result document byte-for-byte. This requires a CANONICAL serialization — sorted object keys and fixed-precision numeric formatting — because JSON key order and float repr are otherwise free to vary between runs. Fingerprints are computed over the canonical form, never over `JSON.stringify` default output.

8. Persistence design

Do not create a parallel experiment registry. Extend the existing `backtest_experiments.experiment_type` constraint with `alpha_diagnostic`.

Each diagnostic run inserts its immutable plan row **before** loading the evaluation dataset^{**}. Reuse these columns:

- `market`, `data_cutoff`, `code_version`;
- `trial_family_id`, `trials_considered`, `variant_budget`, `variants`;

- validation_mode, validation_spec, plan_fingerprint;
- universe_fingerprint, dataset_fingerprint, run_fingerprint;
- started_at, completed_at, result_summary.

result_summary uses schemaVersion: 1 and this shape:

```
{
  status,
  objective,
  benchmark,
  accountingCohort,
  learningCohort,
  coverage,
  tests: { A0, A1, A2, A3, A4, A5, A6, A7, A8 },
  diagnosis: [{ stage, contributionPp, confidence, evidence }],
  verdict: "data_invalid" | "collect_more" | "reject_candidate" | "owner_review"
}
```

The migration must preserve existing append-only/write-once guards, service-only grants, and RLS. It must not add a browser write policy. A failed run writes a bounded failure result once; a retry is a new experiment with its own run id.

9. API and UI contracts

API

- GET /api/analytics/alpha-diagnostics?market=us|india
 - owner-only;
 - returns the latest completed run and bounded history;
 - never calls a provider or mutates strategy state.
- POST /api/analytics/alpha-diagnostics?market=us|india
 - owner or cron;
 - inserts the immutable plan, runs A0 first, then remaining tests only if data truth passes;
 - returns run id, coverage, diagnosis, and influence=none.

UI

Add AlphaDiagnosticLab inside the existing Portfolio performance surface below PerformanceTruth; do not add a new main-navigation destination.

The compact view shows, separately for US and India:

- primary benchmark excess over 1W/1M/3M/120-session windows with confidence;
- accounting versus learning-cohort trade counts;
- payoff ratio and both profit factors;
- MFE capture/giveback;
- sizing contribution, cash contribution, exit contribution, and cost drag;
- the first failing funnel stage;

- `insufficient_evidence` rather than a directional verdict when floors fail.

Drill-down shows the nine test cards, their exact sample dimensions, `run/data/code` fingerprints, exclusions, and candidate trial count.

10. Cadence

- **Daily after canonical marks and labels settle:** A0 data truth and compact accounting refresh.
- **Weekly after dimension diagnostics:** A1-A7 descriptive attribution, skipped when the dataset fingerprint is unchanged.
- **Monthly or when a predeclared evidence floor is crossed:** A8 robustness and candidate reviewability.

The scheduler records expected idle windows in shadow-liveness. A missing run is an observability issue; it never triggers a policy fallback or automatic change.

11. Evidence and promotion gates

Descriptive findings may appear with smaller samples, always carrying `n_dates`.

A candidate cannot become `owner_review` unless all hold:

- at least 60 qualifying decision dates and the applicable existing effective-observation floor;
- positive paired net excess return versus incumbent on the same sessions;
- drawdown non-inferiority under the predeclared tolerance;
- no material turnover/cost regression;
- survival under cost stress and the label-permutation placebo;
- multiple-testing adjustment for every configuration tried;
- separate US or India result; no pooled promotion;
- prospective forward-shadow confirmation;
- explicit owner approval through the existing governed promotion path.

The Lab itself cannot promote. Its strongest output is `owner_review`.

12. Failure and edge cases

- Missing benchmark session, stale mark, or NAV mismatch -> entire run `data_invalid`.
- Too few independent dates -> `insufficient_evidence`, never zero or neutral.
- No eligible candidates -> valid funnel result, but no selection conclusion.
- No matched MAE/MFE -> payoff totals remain visible; path metrics unavailable.

- Duplicate symbol/date decisions -> earliest eligible decision only for cross-sectional tests; duplicates remain in the accounting audit.
- Partial exits -> simulate quantity and residual state; do not count residual lots as independent entries.
- Same-day exit and entry -> exit first, cash released once.
- Open positions at cutoff -> mark to the cutoff; never fabricate an exit.
- Candidate requiring unavailable data -> abstain with reason.
- Existing tainted history -> included in accounting, excluded from learning.
- Secondary benchmark selected in UI -> display changes only; diagnostic objective remains the governed primary.

13. Files expected in implementation

Create

- lib/analytics/alpha-diagnostics.ts
- lib/analytics/alpha-diagnostics.test.ts
- lib/analytics/alpha-diagnostic-contract.ts
- app/api/analytics/alpha-diagnostics/route.ts
- components/dashboard/AlphaDiagnosticLab.tsx
- one Supabase migration generated through the approved migration workflow to admit alpha_diagnostic experiment lineage
- features/alpha-diagnostic-lab/IMPLEMENTATION_RESULT.md after verification

Modify

- lib/simulation/portfolio-simulator.ts and tests: daily marks/NAV, benchmark, cash-utilization, drawdown, and run fingerprint output
- **PRECONDITION: this file is currently UNTRACKED in git** (along with features/portfolio-simulation/). An implementation that modifies it is not reproducible from the repository until it is committed. Commit it, or treat A6 as blocked, before build sequence step 2.
- components/dashboard/PerformanceTruth.tsx: mount the compact Lab
- lib/schedule.ts and cron migration only after the owner approves scheduling
- app/api/admin/shadow-liveness/route.ts: expected weekly diagnostic liveness
- docs/arch/04-database-schema.md
- docs/arch/05-crons-and-scheduling.md
- docs/arch/09-learning-loop.md
- public/agent-diagrams/system-map.json only if the scheduled diagnostic is represented as a system flow

No new package is approved or expected.

14. Validation plan

Pure/unit tests

- exact alpha-funnel counts and deterministic attrition reasons;
- percentage versus currency profit-factor divergence;
- MFE capture, giveback, and prior-positive loser calculations;
- duplicated symbol/date deduplication;
- sizing quartile and equal-size counterfactual;
- same-session exit-before-entry redeployment;
- no double-spend, negative cash, or cross-currency simulation;
- cost-stress monotonicity;
- future-dated input throws;
- deterministic fingerprints and byte-identical reruns;
- insufficient/date-clustered status rules.

Integration tests

- owner/cron authorization and browser write denial;
- plan row exists before evaluation reads begin;
- A0 failure prevents interpreted downstream findings;
- result summary is write-once and retry creates a new run;
- US query cannot read India rows and vice versa;
- accounting count includes tainted trades while learning count excludes them;
- no import or call path reaches a provider, scorer, PaperTrader, PositionMonitor, strategy promotion, proposal, order, or broker module.

Production verification

- run one owner-triggered US and one India diagnostic;
- independently recompute one metric from SQL and match the stored result;
- verify service-only grants, RLS, write-once trigger, and migration history;
- load the Portfolio page, switch markets, inspect network/console, and confirm no cross-market result race;
- rerun an identical frozen plan and prove the same run fingerprint and result;
- run the full test suite, production-equivalent typecheck/build, architecture drift checks, migration replay, and secret scan before shipping.

15. Acceptance criteria

1. One screen identifies the measured contribution of data, selection, sizing, exits, cash/redeployment, and costs to market-local benchmark lag.
2. Every conclusion shows independent dates, symbols, exclusions, confidence, and the immutable experiment/run identity.
3. Accounting truth and learning evidence are visibly separate and reconcile to their respective source ledgers.
4. The simulator models calendar capital and redeployment; no per-trade excess comparison is used to rank different holding periods.
5. A target/stop change cannot be recommended solely because it is the best cell in a retrospective grid.
6. US and India cannot share capital, benchmark, thresholds, evidence, or verdict.
7. A diagnostic run has no write path to any money-path table or configuration.
8. The strongest automated verdict is owner_review; activation remains outside this feature.

16. Build sequence after approval

1. Pure diagnostic contracts and A0-A3 fixtures.
2. Extend the portfolio simulator and implement A4-A7.
3. Add A8 robustness, immutable plan/result persistence, and migration tests.
4. Add owner/cron API and production read-only dry run.
5. Add the Portfolio UI and browser verification.
6. Add weekly scheduling only after an unscheduled production run is proven.
7. Complete ship guardrails and implementation record.

Source: features\alpha-diagnostic-lab\IMPLEMENTATION_RESULT.md

Alpha Diagnostic Lab — implementation result

Status: v2.2 measurement repair complete and production-flow verified
Date: 2026-08-28. Influence: none — no money path reads this feature.

Outcome

The independent review's measurement defects are repaired. The Lab now measures the declared eligible-long cohort, preserves the actual opening book and event quantities, uses A2's exact dated statistic in A8, separates initial from current risk geometry, fingerprints dataset content, and renders missing data as missing.

The original P0 production interpretation is **superseded**. In particular, the claim that India selection ranked well and sizing destroyed the edge came from the all-scored cohort, not the cohort that could actually enter.

What changed

area	v2.2 contract
A0	checks duplicate/missing NAV components and benchmark coverage; permits only leading cash-only inception rows
A2	eligible-long filtering before symbol/date dedup; all-scored is context-only; stable pagination prevents the 1,000-row PostgREST cap
A3/A5/A7	distinct entry-date samples; missing outcomes are excluded with explicit coverage, never coerced to zero
A4	matched h10 MFE/MAE; missing excursions are unavailable, not neither_touched
A6	starts from the first untainted EOD mark/performance intersection; seeds persisted mark quantities; preserves partial exits and same-session round-trip causality; rejects unmatched events
A8	exact A2 rows and mean daily Spearman IC; outcomes permuted within date; h10 overlap floor and seeded 2,000-draw placebo
A9	reports initial-stop and current-stop R:R separately; locked-profit trailing stops remain visible
lineage	non-null metric/deploy version plus order-stable content fingerprints over all source ledgers
UI	eligible cohort is the headline; all-scored is labelled context; NULL is —; date units and replay rejection reasons are explicit

No migration and no package were added. No scoring, eligibility, sizing, volatility, stop, target, horizon, exit, paper/live state, proposal, order, or broker behavior changed.

2026-09-18 evidence hardening

A2's selection cohort is now explicitly limited to current-session-validated, deterministic, non-crypto equity signals that remain in the executable signal ledger; advisory/staged/holding-only rows cannot be used to claim entry selection evidence. Its reported t-statistic uses the project-standard horizon-adjusted effective sample size, not the raw count of overlapping forward-return dates. These changes can only make a verdict more conservative.

Production-flow proof

The owner/cron POST route was executed locally against the production ledgers.

It wrote only alpha_diagnostic experiment rows.

market	run id	eligible-long h10 IC	dates	all-scored IC (context)	A6	verdict
US	624b3c85-965d-4e89-8bee-b06729a89160	-0.0768	21	-0.0101	descriptive, 0 rejections, opening delta 0	collect_more
India	d9de3886-b4f8-4ecc-a54c-c9333dc83144	-0.0083	17	+0.1046	descriptive, 0 rejections, opening delta 0	collect_more

A8 is insufficient_evidence in both markets because 17–21 qualifying dates at h10 do not clear the overlap-adjusted 60-date floor. A6 currently estimates the equal-size arm at +0.21 percentage points versus actual in the US and +0.91 points in India over only 7 and 9 mark sessions, respectively. Those values are descriptive and do not authorize a sizing rule.

Two live-flow failures were found and repaired before completion:

1. same-session entry/exit lots were ordered as exits first and rejected; they now carry explicit causal ordering while ordinary exits remain exit-first;
2. .limit(20000) still returned only the server's 1,000-row maximum; the decision ledger now uses stable bounded pagination.

Verification

- Focused adversarial and contract suites passed.
- Full Vitest suite: **2,216 passed, 7 skipped, 0 failed**.
- npx tsc --noEmit: passed.
- npm run build (Next.js production build): passed.
- Production-shaped SQL independently reproduced the cohort reversal and exact opening reconciliations.
- Final live route rows report influence: none, A0 pass, A6 zero rejections, and collect_more for both markets.

Deliberate refusals / remaining evidence gaps

- A1 remains insufficient because the full funnel projection is not persisted.
- A4 refuses barrier ordering when both stop and target were touched and refuses classification when MFE/MAE are unavailable.
- A2/A8 refuse a verdict below the overlap-adjusted date floor.
- The current A6 window is too short for a policy conclusion.
- Historical closed positions cannot recover stop/target geometry that was never

persisted. A9 is therefore an initial-vintage view of retained data.

Architecture:

features/alpha-diagnostic-lab/MEASUREMENT_INTEGRITY_REPAIR_ARCHITECTURE.md.

Source: features\asset-allocation\FEATURE_ARCHITECTURE.md

Feature Architecture — Asset-Class Allocation

Status: Deterministic CORE built + SHIPPED OFF (migration 175). Genome
evolution, the slow rebalancer, sizing wiring, and UI are the validated
follow-up. Owner defaults chosen (below) — change any and say so.
Last updated: 2026-07-13

Built (2026-07-13, ships OFF)

- Migration 175: `strategy\_sleeves(market, sleeve, target\_pct, min\_pct, max\_pct, instruments, enabled) seeded per market + strategy\_config.allocation\_enabled` (default **false** → allocator inert; zero behaviour change today).
- lib/allocation/allocator.ts: deterministic allocate(sleeves, regime) — tilts targets by macro regime WITHIN hard bands, drops disabled sleeves, renormalizes to 100%. computeAllocation(svc, market) returns null unless enabled. NO LLM. Unit-tested (tests/allocator.test.ts).
- **Default decisions used** (owner-tunable in strategy_sleeves):
 - US: equity 70% (0–90), defensive_etf 20% (0–50, SHY/IEF/TLT/GLD), cash 10% (5–100), leveraged **OFF** (0–15).
 - India: equity 70%, defensive_etf 20% (LIQUIDBEES.NS/GOLDBEES.NS), cash 10%.
 - Leveraged sleeve **disabled**; rebalance cadence weekly / 5% deadband (documented; rebalancer not built yet).

Follow-up (NOT built — money path + evolution)

1. Wire the equity-sleeve target into paper-trade sizing (only when enabled).
 2. Slow rebalancer for defensive/cash sleeves (weekly cron, deadband).
 3. Genome allocation block + LearnerAgent proposal + Validation gate.
 4. Allocation UI (current vs target per sleeve).
- Turn it on with allocation_enabled=true only after 1–3 land + are validated.

Original proposal (retained)

Status: **PROPOSAL** — **not built**. Needs owner approval before implementation.

Last updated: 2026-07-13

Problem

Today the book is **long-only single-name equities + a few ETFs** (metals basket, region ETFs) with **cash** as the residual. There is ****no allocation across asset classes**** — no bonds, no defensive/leveraged sleeves, no target weights, no "hold more cash / bonds when the regime turns risk-off." Sizing is per-position (Kelly / position_size_pct); the learner/genome evolves ****scoring weights + entry/exit/horizon/sizing****, NOT the asset mix. So the portfolio cannot shift its ***shape*** to defend or press an edge as markets change — it only picks stocks.

The ask: can the agents figure out a balance across bonds / US ETF / cash / stocks / leveraged ETF for max gain in any market, and keep evolving it?

Design (proposed)

A thin **allocation layer ABOVE the existing stock picker** — it never replaces scoring; it sets how much capital each sleeve gets, and the picker fills the equity sleeve as it does now.

1. Sleeves (per market / currency — never cross-summed)

Define named sleeves with target bands, e.g. (US):

Sleeve	Instruments	Band
equity	scored single names (current pipeline)	0–80%
defensive_etf	bond/treasury ETFs (e.g. SHY/IEF/TLT), gold	0–50%
cash	residual	5–100%
leveraged	leveraged/inverse ETFs — OFF by default, hard-capped, opt-in	0–15%

India mirrors with its own instruments (liquid-bond ETFs, GOLDBEES, cash).

2. Deterministic allocator (no LLM on the money path)

A pure function `allocate(regime, riskProfile, genome) -> targetWeights` maps the existing `macro_regime` signal + risk profile to sleeve targets within the bands. Risk-off → more cash/defensive; risk-on → more equity. Deterministic, auditable, bounded by the bands. (Consistent with the CLAUDE.md rule against fragile explicit bull/bear ***scoring*** switches — this governs ***allocation***, not per-name scoring, and stays inside hard bands.)

3. Learner evolves the mapping (bounded, validated)

Extend the **genome** with an allocation block: the regime→sleeve-target mapping + band edges. The LearnerAgent proposes challenger allocations; they go through the **same champion/challenger + Validation Engine gate** (walk-forward, fail-closed) before a human promotes. So it "figures it out and mutates" — but only within hard bands, only via validated, owner-promoted challengers. Never unbounded, never LLM-driven on the money path.

4. Execution

The trader/paper-trader sizes new entries against the **equity sleeve's** current target, not the whole NAV; defensive/cash sleeves are held as ETF/cash positions rebalanced on a slow cadence (e.g. weekly, with a no-churn deadband) to avoid overtrading. Kill-switch / drawdown / per-market controls all still apply.

Safety

- Leveraged sleeve OFF by default, hard-capped, explicit opt-in.
- All targets clamped to bands in code (not just config).
- Rebalance deadband to prevent churn.
- Paper-first; live only after the same gates as today.
- No cross-currency summing; each market allocates its own pool.

Scope (build order, if approved)

1. Migration: `strategy_sleeves` (targets/bands per market) + genome allocation block.
2. `lib/allocation/allocator.ts` — deterministic `allocate()` + band clamp + tests.
3. Wire equity-sleeve sizing into paper-trade/trader.
4. Slow rebalancer for defensive/cash sleeves (cron, deadband).
5. Genome extension + LearnerAgent proposal + Validation gate + backtest.
6. UI: allocation panel (current vs target per sleeve); docs (arch-08/09 + system-map).

Open questions for the owner

- Which bond/defensive instruments per market? (US: SHY/IEF/TLT/GLD? India: liquid-bond ETF + GOLDBEES?)
- Enable the leveraged sleeve at all, or leave it off?
- Rebalance cadence + deadband tolerance.
- Paper-only first (recommended) before any live allocation.

Source: features\atr-exit-stop\FEATURE_ARCHITECTURE.md

ATR-scaled exit stop — shadow arm

Status: DRAFT — architecture proposal, awaiting owner approval. No code written.
Date: 2026-09-01. Influence if approved: none while shadow . Promotion to live would be a money-path change requiring separate approval.
Evidence: docs/audits/2026-09-01-exit-geometry-diagnosis.md (frozen).

The one hypothesis

Everything below tests exactly one claim, declared before any arm is built:

H1 — Replacing the fixed 7.5% stop with a 2.8x ATR stop reduces premature stop-outs and raises mean benchmark-neutral return, with the time stop and target left unchanged.
--

Declared **directional** (ATR stop $\geq$ fixed stop) and **single**. This is deliberate: the diagnosis came from a 14-arm grid, and the whole risk here is re-testing the grid and calling the winner a finding.

Not being tested, and explicitly out of scope: changing the target, removing or shortening the time stop, changing position sizing, changing eligibility.

Why the stop and not the target

From the frozen diagnosis: the winning arm exits on target 10 times in 900 and times out 706 times — MORE than baseline's 683. Its entire advantage comes from stopping out 131 times instead of 185. Tightening the target ranked LAST in all four cohorts. The target is not the lever.

What the shadow arm does

exit_stop_shadow — measure-only, one arm, no grid.

- For each **entry-eligible long** decision with a matured h10 label and ATR available, evaluate two geometries only: **baseline** (stop 7.5%, target 19.2%) and **candidate** (stop 2.8ATR, target 19.2% — target held identical so the stop is the only difference).
- Record per decision date: resolution (stop / target / timeout / ambiguous), realized benchmark-neutral return under each arm, and the paired difference.

3. Refuse the date when ATR is missing or the resolution is ambiguous under either arm.

Ambiguity is not resolved in the candidate's favour.

4. Write to a new `exit_stop_shadow_runs` table. Nothing reads it but the report.

Reuses `lib/trading/exit-geometry-shadow.ts` (`evaluateGeometry`) rather than adding a second implementation of barrier resolution.

Statistics, declared in advance

- **Paired test.** Both arms evaluate the SAME decisions, so the statistic is the mean paired difference, not two independent means.
- **Date-clustered.** One observation per decision date; symbols within a date are not independent draws.
- **Overlap correction.** $n_{\text{Effective}} = n_{\text{Dates}} / \text{horizonDays}$, the existing `effectiveObservations` contract. At h10, 26 dates is 2.6 effective observations — far below `MIN_EFFECTIVE_OBSERVATIONS = 12`. **The current data cannot clear this floor.** That is the expected initial verdict and it is correct.
- **Multiplicity.** Although only one arm is promoted-eligible, the hypothesis was selected from a 14-arm grid. The family alpha is therefore Sidak-adjusted for $m = 14$:
 $\alpha_{\text{per_test}} = 1 - (1 - 0.05)^{(1/14)} = 0.00366$. A nominal p of 0.04 does not pass.
- **Per market.** US and India are proved separately. India recorded zero target hits and is a different regime; a US result does not transfer.

Promotion gates — all must hold

1. $n_{\text{Effective}} \geq 12$ per market (needs ~120 h10 decision dates; currently 26).
2. Mean paired difference positive with Sidak-adjusted significance at 0.00366.
3. Holds at h5 as well as h10, or the h5 divergence is explained. **Today it does not: the candidate LOSES on US h5 by 0.041pp.** That divergence alone blocks promotion now.
4. Forward shadow period with no peeking at the grid, on decisions made after the arm is declared.
5. Stop-out reduction confirmed as the mechanism — if the gain appears without fewer stop-outs, H1 is wrong even if returns improved.

Failing any gate leaves the arm in shadow. There is no partial promotion.

What would falsify H1

- Stop-out counts converge between arms while returns still differ (mechanism wrong).
- The US h5 loss persists or widens as data matures.
- ATR coverage falls below ~90%, making the candidate a different cohort from baseline.

- Gain concentrates in a single sector — the current US book is heavily gold, and a volatility-scaled stop flatters a trending sector.

Risks worth stating

- **Selection.** H1 was chosen by looking at the grid. The forward shadow is the only real test; the retrospective margin should be treated as an upper bound, not an estimate.
- **A wider stop increases per-trade loss when it does fire.** Mean return can rise while tail loss worsens. The report must carry max adverse excursion and worst-lot loss, not just the mean.
- **ATR is itself estimated** and regime-dependent. A stop that widens in high volatility widens exactly when losses are largest.

What this does NOT propose

- No live stop change. No target change. No time-stop change.
- No grid search. One arm, one hypothesis, one direction.
- No claim that H1 is true. The frozen evidence supports testing it, nothing more.

Sequencing

1. Owner approval of this document.
2. Build the shadow arm + `exit_stop_shadow_runs` migration; verify migration applied.
3. Weekly cron, both markets, measure-only. Expect `insufficient_evidence` for months.
4. Revisit when `nEffective >= 12` — roughly 120 h10 decision dates per market.

At the current rate of ~26 dates per 6 weeks, that is approximately **2027-Q1**. Stating this plainly so the timeline is not a surprise: this is a slow instrument, and building it now is worthwhile only because the data accrues whether or not anyone is watching.

**Source: features\atr-paper-exit-policy\FEATURE
_ARCHITECTURE.md**

ATR Paper Exit Policy

Status: MEASURE-ONLY PHASE APPROVED AND IMPLEMENTED
Date: 2026-07-22

Decision

Do not implement fixed 1.5 ATR partial exits, 2.5 ATR full exits, or a 15-point below entry score hard exit in the current paper loop. Those values are hypotheses, not validated policy parameters, and would create a second exit engine beside the existing transactional PositionMonitor path.

Kairos will first measure ATR-normalized exit outcomes in the existing evidence and validation substrate. A later, explicitly approved version may replace the current risk/reward policy only after it clears market-local, out-of-sample validation and paper shadow evidence.

Why The Handoff Design Cannot Ship As Written

1. `paper_positions`, not `paper_trades`, is the open-state authority. `paper_trades` is an immutable lot/realized-outcome ledger. PositionMonitor already invokes `execute_paper_exit`, which atomically splits partial lots, adjusts the remaining position, credits only the same market pool, and preserves FIFO parity. Storing `partial_exit_qty`, `peak_price`, or trailing state only on `paper_trades` can re-trigger a partial exit on the same open position and break lot/position reconciliation.
2. A 15-point drop from entry is not the current deterministic thesis rule. It would exit an 90-score entry at 74 even while it remains comfortably above the market mandate's validated exit threshold. Current score exits use fresh, same-market evidence and threshold-minus-hysteresis; this avoids treating a still-strong score as a broken thesis.
3. ATR parameters must not be tuned from intuition. Kairos already records MAE, MFE, horizons, and point-in-time observations for the specific purpose of learning a future volatility/pattern-conditioned risk-reward policy. Fixed values would pre-empt that governed surface and contaminate paper outcomes.
4. A once-daily monitor cannot honestly simulate intraday touch-triggered ATR barriers. It may evaluate close-based paper exits only unless an approved, point-in-time intraday fill model is added. It must not call a daily close a fill at a level that may never have been executable.

Existing Policy To Preserve

- PaperTrader binds every fill to an entry-time stop and target through `resolveExecutionRiskReward` and `bindTradePrices`.
- The source is the market-local mandate until at least 60 eligible MAE/MFE

labels support a learned alternative.

- PositionMonitor is the only paper exit executor. It runs time, fresh score/direction, stop, target, and existing one-time half-target logic in a defined order, then calls the service-only `execute_paper_exit` RPC.
- The RPC owns FIFO lot closure, remaining quantity, cash, native currency, and decision journal writes. No new client or route may bypass it.
- US and India stay separate for prices, ATR units, data sources, policy evidence, positions, cash, and evaluation.

Measure-Only Phase (Implemented)

1. Extend immutable `decision_observations / observation_labels`, not the execution tables, with derived ATR-normalized MAE/MFE diagnostics where the entry candle history is adequate. Record null rather than inventing an ATR.
2. Evaluate the versioned `close_observed_v1` family of three stop/partial/target/trail candidates against the same frozen entry opportunities. The simulation observes completed daily closes, matching PositionMonitor's cadence, and applies the existing 10 bps round-trip haircut. It never claims an intraday barrier fill from a daily candle.
3. Expose a read-only, market-and-horizon-local evidence report. A candidate is `insufficient_sample` until it has at least 60 labels and 12 effective horizon-adjusted observations. This status is descriptive only: it cannot promote or activate an exit policy.
4. Write only matured label evidence and health telemetry. No change to `agent_signals`, PaperTrader, PositionMonitor, positions, cash, broker proposals, or live orders.

Implemented contracts

- Entry ATR is read from `immutable decision_observations.features.technical.atr14`. Missing or invalid ATR remains null.
- `observation_labels` owns `entry_atr`, ATR-normalized MAE/MFE, the versioned candidate outcomes, and the policy version. Existing labels are reprocessed only when the version is missing or stale.
- `GET /api/analytics/atr-exit-evidence?market=us|india&horizon=2|5|10|20` returns coverage and candidate aggregates for authenticated owner review.
- No execution module imports the candidate family. The database migration has no trigger or RPC capable of changing positions, cash, signals, or orders.

Still required before paper shadow

After enough prospective labels accrue, add the predeclared purged walk-forward, locked-holdout, drawdown/turnover, and trial-family correction evaluation. A reviewable_evidence aggregate is not approval and cannot skip that gate.

Later Paper Shadow Gate

After a candidate is statistically eligible, run it as a market-local, non-executing shadow beside the incumbent exit policy. It must use the same entry price, quote source, close convention, slippage model, and horizon. It may become a paper policy only after non-inferiority on drawdown and net return, no material turnover regression, reproducible records, and explicit owner approval.

Explicit Non-Goals

- No broker-hosted stops, protective orders, live execution, Webull enablement, router enablement, or change to `protective_orders_enabled`.
- No re-entry exception that bypasses the existing name cap, cooldown, deterministic score, market-control, or kill-switch gates.
- No hardcoded ATR multiplier or score-velocity constant in the money path.
- No `paper_trades` mutation that represents live position state.

Acceptance Criteria For A Future Implementation

- One versioned exit policy owns all partial/full paper exits.
- State lives on `paper_positions` and all transitions use `execute_paper_exit` in a single transaction.
- Every exit is reproducible from versioned evidence and a documented fill convention.
- The policy is separately validated and market-local; it does not borrow US results to configure India or vice versa.
- Existing exits, cash accounting, FIFO parity, long-only constraints, kill switches, and fresh-score protections remain intact.

Evidence Basis

- Kaminski and Lo show that whether a stop-loss adds value depends on the return process and regime rather than a universal parameter. Kairos must therefore validate its own market-local policy out of sample, net of costs.

https://papers.ssrn.com/sol3/papers.cfm?abstract_id=968338

- Hsu, Taylor, and Wang found that strategies selected in sample can fail out of sample even when stop-loss variants are considered. This is why the candidate family, holdout, and trial-history record are mandatory rather than optional.

https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3158101

Source: features\atr-paper-exit-policy\IMPLEMENTATION_RESULT.md

ATR Paper Exit Evidence - Implementation Result

Completed: 2026-07-22
Scope: measure-only; no paper, live, broker, or protective-order activation

Shipped

- Entry-time ATR is read only from frozen `decision_observations.features.technical.atr14`.
- Label maturation writes ATR-normalized MAE/MFE and three deterministic, versioned, close-observed exit-policy outcomes to `observation_labels`.
- Missing historical ATR remains null. Kairos does not reconstruct it from later-adjusted candles.
- The authenticated `/api/analytics/atr-exit-evidence` endpoint reports US or India evidence separately at 2/5/10/20-day horizons.
- Candidate status remains `insufficient_sample` below 60 labels or 12 horizon-adjusted effective observations. No status can activate a policy.
- Label maturation now pages beyond already-completed observations, fixing an older backlog-starvation defect that could strand newer labels.

Safety Proof

- No trading or exit module imports the ATR candidate family.
- The migration adds no trigger, execution RPC, position/cash foreign key, or broker dependency. Production dependency inspection returned zero functions dependent on the new columns.
- `observation_labels` remains RLS-enabled and service-only.

- PaperTrader, PositionMonitor, live exits, broker proposals, and protective orders were not changed.

Verification

- TypeScript: clean.
- Tests: 1,229 passed, 6 skipped; focused ATR/label suite 15 passed.
- Production build: clean on the final implementation tree.
- Supabase migration 20260722110000_atr_exit_evidence_labels.sql: applied to FinanceOS project <production-project-ref>; six columns, three constraints, RLS, and zero execution dependencies verified.
- Production already has 747 decision observations containing frozen ATR. The 361 currently matured labels predate that evidence and are intentionally not backfilled with reconstructed ATR.

Production smoke

- US: 458 labels matured, 54 with valid point-in-time ATR outcomes.
- India: 164 labels matured, 77 with valid point-in-time ATR outcomes.
- Every populated ATR label contains all three close_observed_v1 outcomes; database verification found zero invalid rows.
- Ten- and twenty-day evidence remains sparse or absent until those prospective observations genuinely mature.

Next Gate

Evidence must accrue prospectively. Before any paper shadow proposal, Kairos still requires purged walk-forward evaluation, a locked holdout, drawdown and turnover comparison, trial-family correction, and explicit owner approval.

Source: features\automated-strategy-validation\FEATURE_ARCHITECTURE.md

Feature Architecture: Automated Strategy Validation and Shadow Routing

Status

Architecture status: Approved by owner

Approved date: 2026-07-12

Roles: Codex Architect + Builder

Implementation allowed: Yes

Purpose

Run evidence validation whenever Kairos creates a strategy challenger, without requiring an owner to press a Backtest button. A passing challenger may enter bounded, non-executing shadow evaluation automatically. It must never replace a champion, alter a user mandate, activate paper fills, or enable live trading.

Product Contract

1. LearnerAgent creates an immutable challenger as it does today.
2. If automation is enabled for that market, the same server action runs the deterministic Validation Engine before reporting the learner result.
3. A passing challenger is atomically moved to shadow_paper only when the market has no other automated shadow challenger.
4. ResearchAgent records shadow decisions only; shadow state cannot create orders, paper fills, cash movement, or live proposals.
5. A scheduled sweep retries challengers that have no validation record, so a restart or transient local HTTP failure cannot leave a challenger untested.
6. The owner can disable automation independently for US and India. Disabling stops new automatic validation and shadow activation immediately; it does not delete challengers, experiments, or historical shadow evidence.

Authority

hard safety/risk gates -> user trading mandate -> champion ->
automated validation -> shadow evidence -> owner promotion -> paper/live.

Validation is deterministic and LLM-free. The manual Backtest page remains an exploratory tool and cannot promote a strategy. Passing validation establishes only statistical eligibility for shadow evidence; the existing owner-only, fail-closed champion-promotion RPC remains unchanged.

Data Model

strategy_validation_automation, one row per market:

- market (us | india) primary key;
- enabled: permits automatic deterministic validation;
- auto_shadow_enabled: permits a passing candidate to enter shadow state;
- max_active_shadows: hard bounded at one for the initial release;
- audit timestamps and updated_by.

The schema seeds both markets enabled because this feature is explicitly approved. Missing-table or missing-row behavior is fail-closed: existing manual validation remains available but no automatic state changes occur.

activate_strategy_shadow(version_id) is a service-role-only RPC. It locks the market, confirms an enabled policy and a passed experiment belonging to the challenger, rejects terminal/champion states, enforces the shadow cap, then transitions only that challenger to shadow_paper.

Flows

```
flowchart LR
    L[LearnerAgent] --> C[Immutable challenger]
    C --> V[Deterministic Validation Engine]
    V -->|fail or insufficient evidence| R[Recorded evidence; challenger remains inactive]
    V -->|pass and policy permits| S[Atomic shadow activation]
    S --> D[ResearchAgent shadow decisions]
    D --> O[Owner-only champion promotion]
    O --> P[Existing paper/live gates]
```

The validation sweep considers only challengers with no prior validation experiment. It is bounded per market. A failed experiment is preserved rather than overwritten; a future evidence-refresh design must use an explicit dataset-version rule rather than silently rerunning failed experiments.

TradingView Boundary

TradingView/Pine remains optional, manual corroboration and webhook-based forward-test input. No TradingView UI automation, scraping, or assumed general backtest-results API is part of this architecture. Kairos authority evidence comes from its stored, reproducible US/India data snapshots.

Reversibility

- Set enabled=false to stop automatic validation for one market.
- Set auto_shadow_enabled=false to retain automatic evidence generation but stop automatic shadow routing.
- Existing shadow strategies can be manually paused, rejected, or retired with

the existing registry controls; no history is deleted.

- Removing the scheduled sweep only removes retries. No schema or data rollback is required to return to manual validation.

Files

- supabase/migrations/170_strategy_validation_automation.sql
- lib/validation/automation.ts
- app/api/agents/learner/route.ts
- app/api/validation/sweep/route.ts
- app/api/settings/validation-automation/route.ts
- components/dashboard/ValidationAutomationPanel.tsx
- app/dashboard/settings/page.tsx
- scripts/run-agents.ps1
- scripts/register-tasks.ps1

Non-Goals

- A second historical backtesting engine.
- Automatic champion promotion, paper fills, broker proposals, or live orders.
- TradingView credential/API integration or browser automation.
- Rewriting any existing validation result or shadow decision.

Acceptance Gates

- Per-market disable blocks new automatic validation and shadow activation.
- A new challenger validates synchronously through the server action, with no fire-and-forget local HTTP dependency.
- A passing challenger can create at most one active automated shadow per market.
- Failed/insufficient validation leaves a challenger inactive and is auditable.
- The sweep retries only unvalidated challengers.
- Owner promotion remains mandatory and fail-closed on passed evidence.
- Typecheck, tests, production build, and dependency audit pass.

Source: features\automated-strategy-validation\IMPLEMENTATION_RESULT.md

Automated Strategy Validation Implementation Result

Date: 2026-07-12

Status: Complete

Delivered

- Durable in-process deterministic validation when LearnerAgent creates a challenger. The prior fire-and-forget localhost request was removed.
- Per-market, owner-controlled automation policy for US and India.
- Atomic, service-role-only shadow activation after a passed experiment. The database enforces one active shadow strategy per market.
- Friday cloud retry sweep (`kairos-validation-sweep`, 21:45 UTC) for only challengers that have no validation record.
- Settings controls to disable validation or disable only automatic shadow routing. Disabling preserves all challengers, experiments, and shadows.
- Focused fail-closed policy tests.

Explicit Safety Boundary

This feature cannot promote a champion, create a paper fill, move cash, make a broker proposal, or submit a live order. Existing owner-only champion promotion and all execution gates remain separate.

Production Verification

- Applied and verified `supabase/migrations/170_strategy_validation_automation.sql` against FinanceOS Supabase project `<production-project-ref>`.
- Verified US and India policies seeded enabled with one shadow slot each.
- Verified `activate_strategy_shadow` execute grant is restricted to `service_role` and postgres.
- Called the RPC with a nonexistent strategy and verified the safe `strategy_not_found` response without changing strategy state.
- Verified cloud cron `kairos-validation-sweep` is active at 45 21 * * 5.

Gates

- `npm test`: 313 passed, 6 skipped.

- `npx tsc --noEmit`: passed.
- `npm run build`: passed.
- `npm audit --audit-level=high`: 0 vulnerabilities.

Reversal

Settings -> Automatic Strategy Validation has independent US/India controls:

1. Disable Run validation automatically to return that market to manual validation immediately.
2. Disable Route passing challengers into one shadow-evidence slot to retain automatic evidence while requiring manual shadow routing.

No rollback migration is necessary for normal disablement. Historical evidence is deliberately retained.

Source: features\benchmark-alpha\FEATURE_ARCHITECTURE.md

Feature Architecture - Benchmark Alpha Scorecard

Status: P1 BUILT 2026-07-13; P1E DISPLAY COMPARATORS BUILT 2026-08-24. Phase 1 measurement is implemented; Phase 2 learner/promotion wiring remains unbuilt.
Scope: deterministic measurement first; no order path, no paper fills, no learner mutation in Phase 1.
Build gate: Phase 2 learner/promotion wiring requires a separate owner approval after Phase 1 has accumulated evidence.
Update when built: docs/arch/04-database-schema.md, docs/arch/05-crons-and-scheduling.md, docs/arch/09-learning-loop.md, and public/agent-diagrams/system-map.json.

Purpose

Kairos already stores some benchmark information:

- `paper_performance.bench_nav`, `bench_return_pct`, `alpha_pct` - a single per-market paper benchmark series.
- `live_performance.account_id`, `date`, `equity`, `bench_nav` - a forward-built live account equity curve currently tied to VOO.
- `investment_mandates.benchmark_symbol` and `strategy_evaluations.book_alpha_pct` - mandate/evaluation context.

What is missing is a trustworthy **multi-horizon, per-market, per-book benchmark scorecard** that answers:

- Are paper and live beating the configured benchmark over 1W, 1M, 3M, YTD, and 1Y?
- Is the result statistically usable or just short-window noise?
- Which benchmark is primary for the learning objective, and which are comparison-only?
- Can a benchmark be added without hardcoding VOO/NIFTY everywhere?

This feature belongs inside the existing **Performance Truth Layer**, not beside it. It is a materialized measurement surface and later an input to governed learner/promotion decisions.

Non-Negotiable Design Rules

1. **No cross-currency roll-up.** US USD books and India INR books are never summed, compared, or converted implicitly.
2. **Benchmark currency must match book currency.** US books benchmark against USD-traded benchmarks such as VOO; India books benchmark against INR/index series such as ^NSEI/NIFTY source symbols.
3. **Common-window rebasing only.** Portfolio and benchmark start from the same first date where both have valid levels inside the requested horizon.
4. **No action from noisy windows.** Phase 1 is measurement-only. Phase 2 can only use longer, confidence-qualified windows.
5. **Deterministic only.** No LLM computes, fills, overrides, or gates alpha.
6. **Existing series are source ledgers.** `paper_performance` and `live_performance` remain the source series; `benchmark_scorecard` stores rollups so dashboards/agents read one stable, audited number.

Reviewed Design Issues And Fixes

1. Live aggregation was underspecified

Failure scenario: `live_performance` is keyed only by (`account_id`, `date`) and has VOO in `bench_nav`. A future rollup that sums all rows by date can mix Webull + Robinhood, trading + read-only, or even future India accounts. The resulting "live alpha" would be an accidental account set, not a market book.

Fix: Add explicit live series provenance before live scorecarding:

- Add nullable `market`, `currency`, `broker`, and `book_scope` columns to `live_performance`, backfilled from `live_account_snapshots/broker` registry where possible.
- Phase 1 scorecard uses canonical scopes only:
 - `paper`: one market-level paper pool from `paper_performance`.
 - `live`: one market-level live book per (`market`, `currency`, `book_scope`='all_live_accounts'), summing only accounts whose rows match that market/currency/scope.
- If provenance is missing for any live row in the window, the live scorecard row is `status='insufficient_data'`, not estimated.

2. Info-ratio math was ambiguous

Failure scenario: The proposal said "excess / stdev of daily excess" but did not specify units. If cumulative 1M excess return is divided by daily tracking error without annualization/window scaling, the ratio is not comparable across 1W, 3M, and 1Y.

Fix: Persist both components:

- `excess_return_pct = portfolio_window_return_pct - benchmark_window_return_pct`.

- `tracking_error_daily_pct = sample stdev(daily_portfolio_return_pct - daily_benchmark_return_pct).`
- `info_ratio = mean(daily_excess_return_pct) / stdev(daily_excess_return_pct) * sqrt(252).`
- If tracking error is zero, non-finite, or sample size is too small, `info_ratio=null` and `status='insufficient_data'`.

The dashboard may show cumulative excess for intuitive reading, but learner/promotion logic may only consume the annualized daily information ratio.

3. YTD/1Y edge cases were not guarded

Failure scenario: On January 3, a YTD row might have 1-2 observations and produce a huge positive/negative alpha; on a newly created live account, 1Y might silently use two days and look complete.

Fix: Each row stores `window_start`, `window_end`, `n_observations`, `n_return_days`, and `coverage_pct`. Confidence gates:

Horizon	Minimum return days for display	Minimum for Phase 2 use
1W	4	never used for gating
1M	15	never used for promotion
3M	45	60
YTD	<code>min(20, elapsed trading days - 1)</code>	<code>max(60, 70% of elapsed trading days)</code>
1Y	120	180

Rows below the display floor are stored with `status='insufficient_data'`. Rows below the Phase 2 floor are display-only.

4. Benchmark priceability was too optimistic

Failure scenario: A user adds ^NSEBANK or a sector ETF that the current provider cannot price. If the rollup simply skips it with a log line, the UI and agents can mistake absence for neutrality.

Fix: Add a small benchmark observation ledger and explicit row status:

- `benchmark_price_observations`: one row per (`benchmark_id`, `date`, `component_symbol`) with `close`, `currency`, `provider`, and `source_status`.
- The rollup writes a `benchmark_scorecard` row for every enabled benchmark/horizon/book even when the benchmark is unpriceable, with `status='benchmark_unpriceable'` and `missing_reason`.
- UI shows the failure state; agents treat it as unavailable.

5. Primary benchmark uniqueness must be scoped

Failure scenario: A single partial unique index on (`market`) where `is_primary` works for one benchmark per market, but blend/single representations and future inactive rows can accidentally leave two "primary-like" configs or none.

Fix: Use a normalized `benchmarks` table with stable IDs and an explicit partial unique index:

```
unique (market) where is_primary = true and enabled = true
```

The Settings/API update that flips `primary` must be transactional: clear old `primary` and set new `primary` in one service-role operation. If no enabled `primary` exists for a market, Phase 2 is disabled for that market and Phase 1 rows are comparison-only.

6. Phase 2 could overfit the learner to beta/regime

Failure scenario: A high beta strategy can beat VOO in a risk-on month and get promoted even if it is simply more volatile. Conversely, a defensive strategy can lag in a melt-up and be rejected despite doing its job.

Fix: Phase 2 cannot replace the existing validation gate. It is an additional guard:

- Challenger still must pass walk-forward validation (`validation_experiments.passed=true`) and existing sample/evidence gates.
- Primary-benchmark info ratio is one metric in the Performance Truth row, not the only objective.
- Promotion requires either:
 - positive primary info ratio over the mandate horizon with sufficient sample, or
 - explicit owner override/journaled reason when the mandate intentionally differs from the benchmark.
- Learner objective uses a blended score: validation pass/fold stability first, then primary benchmark IR, then drawdown/cost/slip. No direct mutation from scorecard rows.

7. Posture feedback was too action-like

Failure scenario: A negative 1W alpha row triggers posture tightening, which reduces entries after random weekly noise - the same failure class as the prior phantom drawdown cascade.

Fix: Posture feedback is advisory and slow:

- Only 3M, YTD, or 1Y primary rows may create `trailing-benchmark:*` alerts.
- Requires two consecutive completed rollups below threshold.
- Alert severity is `warn`, not `critical`.
- It never toggles `market_controls`, risk profile, order size, or live-auto settings. It recommends review only.

Data Model - Phase 1

benchmarks

Config rows. One enabled primary per market.

Column	Type	Notes
<code>id</code>	uuid PK	Stable key used by scorecard rows
<code>market</code>	text	<code>us</code> or <code>india</code>
<code>label</code>	text	Human label: VOO, NIFTY 50, QQQ, 60/40
<code>kind</code>	text	<code>single</code> or <code>blend</code>
<code>symbol</code>	text nullable	Single-symbol benchmark display/source symbol
<code>provider_symbol</code>	text nullable	Provider-specific ticker, e.g. VOO, ^NSEI, NIFTY50.NS
<code>currency</code>	text	USD or INR; must match market/book
<code>price_provider</code>	text	<code>massive</code> , <code>yahoo</code> , <code>kite_yahoo</code> , <code>manual_import</code>
<code>is_primary</code>	boolean	One enabled primary per market
<code>enabled</code>	boolean	Disabled rows retained for history
<code>weights</code>	jsonb nullable	Blend components: <code>[{benchmark_id, weight}]</code> ; weights must sum to 1.0

Column	Type	Notes
created_at / updated_at	timestampz	

Seed:

- US: V00, USD, provider massive, primary.
- India: NIFTY 50, INR, provider symbol used by the existing quote path (^NSEI or NIFTY50.NS, whichever the implementation proves priceable), primary.

benchmark_price_observations

Small durable price ledger for benchmark components. This avoids refetching every horizon and makes missing data auditable.

Column	Type	Notes
benchmark_id	uuid FK	
component_symbol	text	
date	date	
close	numeric	
currency	text	
provider	text	
source_status	text	ok, missing, provider_error, unpriceable
error	text nullable	Short provider error; no secrets
PK	(benchmark_id, component_symbol, date)	

benchmark_scorecard

Materialized rollup. This is the always-on number.

Column	Type	Notes
market	text	us or india
currency	text	Must match book currency
book	text	paper or live
book_scope	text	market_paper_pool, all_live_accounts; future account subsets require explicit scopes
benchmark_id	uuid FK	
benchmark_symbol	text	Snapshot display value
is_primary_snapshot	boolean	Snapshot at compute time
horizon	text	1W, 1M, 3M, YTD, 1Y
as_of	date	Rollup date
window_start / window_end	date	Actual common window used
portfolio_return_pct	numeric	Rebased over common window
bench_return_pct	numeric	Rebased over common window
excess_return_pct	numeric	Portfolio minus benchmark
daily_excess_mean_pct	numeric	Mean daily excess return
tracking_error_daily_pct	numeric	Sample stdev of daily excess
info_ratio	numeric	Annualized daily excess / tracking error

Column	Type	Notes
n_observations	int	Common level observations
n_return_days	int	Common daily return pairs
coverage_pct	numeric	Common valid observations / expected trading observations
confidence	text	insufficient, low, medium, high
status	text	ok, insufficient_data, benchmark_unpriceable, book_series_missing, currency_mismatch, stale_series
missing_reason	text nullable	Explain unavailable rows
PK	(market, currency, book, book_scope, benchmark_id, horizon, as_of)	

RLS: owner-read, service-role writes. No public/anon policy. Add indexes on (market, book, book_scope, as_of desc) and (benchmark_id, as_of desc).

Computation Contract

Series loading

Paper:

- Source: paper_performance(date, market, nav).
- Currency: USD for us, INR for india.
- Existing bench_nav/alpha_pct remains for the legacy single-benchmark chart; the scorecard computes its own benchmark series for config-driven rows.

Live:

- Source: live_performance rows with explicit market, currency, and book_scope.
- Aggregation: sum equity by date only within the same (market, currency, book_scope).
- No constant-holdings estimate may feed benchmark_scorecard; estimated live charts stay visually labeled and are not learner inputs.

Benchmark:

- Source: benchmark_price_observations, filled by the rollup from provider adapters.
- For single: one close series.
- For blend: combine component daily returns with fixed weights, rebalanced daily for measurement. Blend weights are normalized only if the stored sum is within a small tolerance; otherwise row status is benchmark_unpriceable.

Windowing

1. Determine target horizon start:

- 1W: as_of minus 7 calendar days.
- 1M: as_of minus 30 calendar days.
- 3M: as_of minus 90 calendar days.

- YTD: January 1 of as_of year.
- 1Y: as_of minus 365 calendar days.
- 2. Filter portfolio and benchmark levels to dates >= target start and <= as_of.
- 3. Inner join by date. Do not forward-fill for scorecard math.
- 4. Use the first joined row as the base for both portfolio and benchmark.
- 5. Compute simple daily returns only for adjacent joined rows separated by exactly one market session. A multi-session gap remains visible in coverage but must not be annualized as one daily return.
- 6. If common rows or return pairs fail the horizon confidence floor, write status='insufficient_data'.

Math

Use fractions internally, persist percentages for UI consistency.

```
portfolio_return_pct = (portfolio_last / portfolio_first - 1) * 100
bench_return_pct     = (bench_last / bench_first - 1) * 100
excess_return_pct    = portfolio_return_pct - bench_return_pct

daily_excess_pct[i]   = portfolio_daily_return_pct[i] - benchmark_daily_return_pct[i]
tracking_error_daily_pct = sample_stdev(daily_excess_pct)
daily_excess_mean_pct = mean(daily_excess_pct)
info_ratio            = daily_excess_mean_pct / tracking_error_daily_pct * sqrt(252)
```

Never divide cumulative excess by daily tracking error.

Surfacing - Phase 1

- Dashboard Alpha Scorecard:
 - Market switch: US/India.
 - Book tabs: paper/live.
 - Rows: horizons.
 - Columns: enabled benchmarks.
 - Cell: cumulative excess return, annualized info ratio, confidence/status badge.
 - Missing/unpriceable rows are visible, not omitted.
- Paper Portfolio comparison chart:
 - Single-select market-local display benchmark (radio/segmented control).
 - US: VOO, QQQ, XLK, XLF.
 - India: NIFTY 50, NIFTY IT (ITBEES), NIFTY Bank (BANKBEES), and NIFTY Next 50 (JUNIORBEES). The latter three are liquid ETF proxies used because their existing free Yahoo daily-bar source was verified priceable; they are labelled as proxies rather than falsely claiming exact US-index equivalence.
 - The owner selection is persisted per market in app_settings and becomes the next chart default across sessions/devices.
 - This is a **display preference only**. It never changes benchmarks.is_primary, mandate benchmark, scoring, learner evidence,

promotion, sizing, paper fills, or orders.

- Portfolio and comparator are joined by exact session date and rebased from their first common observation inside the selected window. No forward fill.

• Agent helper:

- `getAlpha(svc, {market, book, bookScope, horizon, benchmarkId?})`

- Returns only the latest scorecard row with status/confidence.

- Agents must treat non-ok or low-confidence rows as unavailable.

Objective And Feedback - Phase 2

Phase 2 is **not part of the Phase 1 build**. It needs a separate architecture approval after Phase 1 data exists.

Allowed Phase 2 wiring:

1. Add primary benchmark scorecard fields to `strategy_evaluations` snapshots:

- `primary_benchmark_id`

- `primary_benchmark_symbol`

- `primary_horizon`

- `primary_excess_return_pct`

- `primary_info_ratio`

- `primary_alpha_confidence`

2. LearnerAgent may read these fields as evidence but may not mutate weights from them directly.

3. Promotion gate remains: existing walk-forward validation pass first, then benchmark-alpha guard second.

4. Benchmark guard consumes only confidence-qualified 3M, YTD, or 1Y primary rows, never 1W/1M.

5. Alerts are advisory:

- `trailing-benchmark:<market>:<book>` only after two consecutive qualified underperformance rows.

- Warn-only, clears after recovery.

- No automatic pause, kill switch, order-size change, risk profile change, or live-auto change.

Extensibility

Adding a benchmark should be possible by inserting a config row, but only if the provider can price it:

1. Insert benchmarks row as disabled.

2. Run a priceability check for at least 30 recent observations.

3. If priceable and currency matches market, enable it.

4. To change primary, call a service-route transaction that ensures exactly one enabled primary per market.

Blend benchmarks ship after single-symbol benchmarks:

• Component benchmarks must already be enabled and priceable.

• Weights must sum to 1.0.

- Blend currency must match every component and the target market.

Build Order

1. P1A - series/provenance migration

- Add benchmark config/observation/scorecard tables.
- Add live performance provenance columns or an explicit live-account-to-market mapping needed for safe aggregation.

2. P1B - deterministic math library

- Pure functions for common-window rebasing, daily excess, tracking error, info ratio, confidence/status.
- Unit tests for missing dates, zero tracking error, January YTD, short 1W/1M windows, USD/INR mismatch, and live multi-account aggregation.

3. P1C - rollup route + cron

- Owner/cron-gated route.
- Writes every enabled benchmark/book/horizon row, including unavailable statuses.
- No learner/order side effects.

4. P1D - dashboard + helper

- Alpha Scorecard UI.
- getAlpha read helper.
- Docs/system-map updates.

5. P1E - owner-selected display comparators (2026-08-24)

- Seed only verified-priceable, same-currency single-symbol comparators.
- Populate secondary observations through the existing Massive-first US and Yahoo India adapters; the original implementation defined this fetch but failed to call it.
- Persist a separate `portfolio_default_benchmark_<market>` preference; never repurpose the governed `is_primary` flag as a UI preference.
- Expose exact-session daily comparison series through the owner-gated Portfolio API and verify market-switch/request-race behavior in-browser.

6. P2 - gated objective wiring

- Only after P1 evidence and owner approval.
- Add Performance Truth snapshot fields and learner/promotion read-only consumption.
- Add warn-only posture reporter.

Files - Phase 1

- `supabase/migrations/20260713143000_benchmark_alpha_rotation_shadow.sql`
- `lib/analytics/benchmark-alpha.ts`
- `app/api/agents/benchmark-scorecard/route.ts`
- `components/dashboard/AlphaScorecard.tsx`

- tests/benchmark-alpha.test.ts
- lib/analytics/benchmark-display.ts
- app/api/portfolio/performance-series/route.ts
- components/dashboard/BenchmarkPerformanceChart.tsx
- tests/benchmark-display.test.ts
- supabase/migrations/20260824213000_portfolio_benchmark_choices.sql
- docs/arch/04-database-schema.md
- docs/arch/05-crons-and-scheduling.md
- docs/arch/09-learning-loop.md
- public/agent-diagrams/system-map.json

What Not To Build In Phase 1

- Do not change LearnerAgent objectives.
- Do not change promotion gates.
- Do not change risk posture automatically.
- Do not add live-order, paper-fill, sizing, or broker behavior.
- Do not use estimated live backcasts in scorecard rows.
- Do not silently skip unpriceable benchmarks.

Source: features\benchmark-alpha\IMPLEMENTATION_RESULT.md

Benchmark display comparators — implementation result

Shipped: 2026-08-24

Scope: P1E display-only extension to the existing Benchmark Alpha scorecard.

Outcome

- Paper Portfolio exposes one market-local comparison benchmark at a time.
- US: VOO, QQQ, XLK, XLF.
- India: NIFTY 50, NIFTY IT (ITBEES), NIFTY Bank (BANKBEES), NIFTY Next 50 (JUNIORBEES).
- The owner selection persists per market in app_settings and becomes the next display default.

- `benchmarks.is_primary` remains untouched and authoritative for governed analytics. Display selection has no scoring, learning, sizing, fill, or order effect.

Correctness repair

The original scorecard route defined `upsertProviderObservations()` for secondary benchmarks but never invoked it. P1E wires that collector into the daily rollup, using Massive-first/Yahoo-fallback for US and Yahoo daily bars for India. Portfolio and benchmark rows join only on exact dates; missing sessions remain missing rather than being forward-filled.

2026-09-18 freshness hardening

A scorecard now fails closed as `stale_series` when its common portfolio/benchmark return window ends before the session it claims to represent. This prevents a stale comparator from being rendered as OK and silently truncating the portfolio comparison to an old endpoint. It is a display and diagnostic truth repair only; it does not fill missing market data or alter a benchmark.

Data and API

- Migration: `20260824213000_portfolio_benchmark_choices.sql`.
- Preference keys: `portfolio_default_benchmark_us` and `portfolio_default_benchmark_india`.
- Owner-gated API: `GET/PATCH /api/portfolio/performance-series`.

Explicit non-effects

No change to governed primary benchmarks, mandates, ResearchAgent, scoring, LearnerAgent, promotion gates, risk, paper trades, live proposals, or orders.

Source: features\briefing\FEATURE_ARCHITECTURE.md

Feature: Daily Briefing

Status: v1 shipped (2026-07-04). v2 partially shipped; live holding-risk integration shipped 2026-07-21.

v1 (done)

Newsletter-grade email: structured HTML data blocks (accurate, code-built) + short LLM editor's note. Sections: header → editor's note → portfolio health score → market snapshot + regime → agent signals → positions → earnings → learning/phase → footer deep-links. Subject formula with preheader substance. Route: `app/api/briefing/generate/route.ts`.

v2 — REQUIREMENTS (user, 2026-07-04) — NOT yet built

The guiding rule from the user: *****the more details and explanations, the more I like it. Top bullets OK, but explanation and details are MANDATORY.*****

1. **Lighter theme** — not an all-dark background. Mixed/light layout.
2. **Explain every metric** — the Portfolio Health score and each Market Snapshot number must carry a short "what this is + why it matters" line, not just the number. No unexplained figures.
3. **Outlook sections with confidence** — add:
 - **Market outlook** (today + forward) with a stated confidence level.
 - **Positions-I-hold outlook** — per holding, where it's headed and why + confidence.
 - **Future market & position outlook** — near-term expectation + confidence.
4. **Agent activity recap** — what the Research agent and Learner agent actually did (a) on the day of the newsletter and (b) over the past 7 days. Concrete: runs, signals, hypotheses, weight challengers, rescore flags.
5. **Mentor block** — what the Mentor agent has for the user to learn, based on current market + the user's behavior + how they're doing on the learning curve. (Depends on the Mentor AI agent — see features/mentor-agent.)
6. **Structure** — top bullets for scannability, THEN mandatory explanation/detail beneath each. Details are the point, not an afterthought.

Design note for v2: keep the accurate code-built data blocks, but pair each with an LLM-written explanation/outlook (grounded, with confidence), and pull the 7-day agent recap from `agent_runs` + `learner_runs` + rescore flags.

Live Holdings Risk addition (approved and shipped 2026-07-21)

- Each US/India edition reads the latest complete `holding_risk_runs` row per live broker account for that market, then replays its immutable `holding_risk_snapshots` and `account_risk_snapshots` rows.
- The briefing never recomputes risk and never calls a broker or market-data provider for this section. The LLM receives the deterministic postures only as

grounded advisory context and cannot alter them.

- Accounts, markets, and currencies remain separate. Every row must match the selected run's `run_id + market + currency + account_id`; mismatches are discarded instead of cross-summed or relabelled. The `internal` paper-book adapter is excluded from this explicitly live section, and account identifiers embedded in labels are masked to their last four characters.
- The email shows every non-hold posture plus the three highest-risk ordinary holds per account. It states how many lower-priority holds were omitted and links to the complete Risk Analytics page.
- Snapshot date, session age, formula version, confidence, missing inputs, current price, NAV weight, unrealized P&L, score, posture, and deterministic action reason are displayed. Missing values stay unavailable, never zero.
- Every displayed live holding also carries the latest canonical ResearchAgent annotation for the same (`symbol, market`): score, direction, exact researched timestamp, market-session age/freshness, and holding re-score versus candidate score. Only `deterministic_v1 + session_validated=true` rows count; a failed read, abstention, stale result, and never-researched state remain distinct. This is display-only and never changes Holding Risk.
- The email explains that research conviction and portfolio risk answer different questions. Since `hr-v3`, concentration is REVIEW, not TRIM, for every account; a high research score never erases measured exposure, but a global trading reference is never treated as an account-specific sell mandate.
- Failed reads and missing complete runs are visible states. A stale or missing snapshot must never be described as safe/current.
- Legacy `hr-v1/hr-v2` stored trims are immutable evidence, not current advice. The shared history normalizer preserves `sourcePosture=trim` for audit and displays REVIEW in both newsletters and Risk Analytics.

Source: features\broker-compatible-research-universe\FEATURE_ARCHITECTURE.md

Broker-Compatible Broad US Research Universe

Status: **PROPOSED — architecture only**. No schema, ingestion, schedule, score,

eligibility, paper, or live-order behavior changes are authorized by this document.
Last revised: 2026-09-09.
Money-path influence: none . A broker preflight immediately before an order
remains the only broker-tradability authority.

0. Decision summary

Kairos should stop treating its curated candidate queue as if it were a market universe. It should maintain a broad, versioned US **measurement universe** that is independently sourced, point-in-time reproducible, liquid, classified, and separate from the small daily deep-research queue.

It must **not** attempt to make an LLM research every security Robinhood currently offers each day. Robinhood says it offers over 11,200 securities, including stocks, ETFs, CEFs, ADRs and certain OTC equities. That number includes heterogeneous instruments and may change due to corporate actions, halts, account restrictions, or broker policy; it is neither a stable historical universe nor a sector taxonomy.

The proposed result has three deliberately separate populations:

Population	Purpose	Can affect money?
us_instrument_catalog	Current observed exchange/broker instrument identities and metadata	No
us_measurement_universe	Broad, liquid, point-in-time common-stock cohort used for deterministic data and factor/sector measurement	No initially
daily_research_shortlist	Bounded names selected from the measurement universe for expensive news/LLM research and existing scoring	Existing gates only; this architecture adds no authority

This fixes the present evidence problem without implying that all Robinhood instruments are suitable for research or trading.

Measured in production 2026-09-09 (correcting an earlier draft of this section, which claimed "117 canonical sector labels" — that figure was wrong by roughly 4x and pointed at the wrong problem):

Fact	Value
Distinct US symbols ever in decision_observations	167
Distinct US symbols ever in decision_observations (India)	161
Distinct sector labels across those 167 US symbols	30
Distinct sector labels in symbol_profiles overall	37
Observed US symbols with no sector at all	50 of 167 (30%)

The binding constraint is therefore ****classification coverage, not taxonomy fragmentation****. Thirty labels across 167 names is a coarse taxonomy, not a 117-way mess; the actual hole is that nearly a third of the names the system has

made decisions about carry no sector at all. Stage 0 must price that finding first: a 30% null rate may be a Finnhub coverage gap on specific instrument types (ETFs, ADRs) rather than a reason to license a full security master. Do not use this section to justify vendor spend until Stage 0 says which it is.

Feature-lifecycle record

- **Why:** Sector and cross-sectional diagnostics are currently dominated by a small, technology-heavy candidate queue rather than a representative market population.
- **Who:** the Kairos owner.
- **Expected value:** valid sector breadth, auditable research coverage, and a clean comparison between the names the system studies and the names it selects.
- **Shipped at:** not shipped.

1. Verified baseline and constraints

What exists

- `gatherSymbols()` in `lib/research-agent.ts` assembles a bounded daily list from holdings, `research_enabled` watchlist rows, Theme Scout, screeners, and a small relative-strength discovery stream. This is a candidate queue, not a full-market universe.
- `symbol_profiles` caches broad provider profile metadata. The present US source is Finnhub `profile2`, whose `finnhubIndustry` value is useful context but is not a verified canonical GICS classification contract.
- `edge_universe_members` supports immutable point-in-time memberships, but its historical current-liquid source is explicitly survivorship-biased and cannot be silently reused as this feature's historical truth.
- Robinhood REST has an instrument lookup used by the robinhood adapter's `preflightOrder`; that check is account/order-time evidence. It is not proof that a bulk enumeration API is available or licensed.
- **Correction (2026-09-09):** an earlier draft claimed "Robinhood MCP has no instrument-capability tool." That is false. The connected Robinhood MCP exposes `get_equity_tradability`, plus `search`, `run_scan`, `get_scanner_filter_specs` and `get_equity_fundamentals`. What it does **not** expose is a documented ***bulk enumeration*** tool, and every one of those is per-symbol or per-scan and rate-limited. The conclusion is unchanged — no bulk universe from the broker — but the claim must be stated as "no documented bulk-enumeration tool", not as an absence of instrument capability. §9 already requires this be verified from

the actual connected adapter/tool contract rather than assumed.

- `broker_instrument_preflights` is append-only and distinguishes `execution_attempt` from `candidate_probe`; it must continue to keep discovery evidence separate from last-mile enforcement evidence.

External facts to respect

- Robinhood states that it offers over 11,200 securities, but also documents that a security can be untradeable because of a halt, delisting/OTC status, foreign security restriction, or corporate action. [Investments on Robinhood, untradeable securities](#).
- Nasdaq publishes current Nasdaq and other-exchange symbol directories, including security name, exchange, status and test-issue fields, with intraday updates. This is a useful current membership feed—not historical membership. [Nasdaq Symbol Directory definitions](#).
- SEC EDGAR publishes ticker/CIK/exchange associations, but explicitly does not guarantee their accuracy or scope. It is an identity corroborator, not the sole universe or classification source. [SEC EDGAR data access](#).
- NYSE's complete security-master reference products are commercial. We must not claim a free, complete NYSE historical source before licensing and coverage are verified. [NYSE reference data](#).

2. Scope and non-goals

In scope

- A current US instrument catalog sourced from documented exchange/reference feeds.
- A versioned daily measurement-universe snapshot with explicit membership reasons.
- Deterministic, cached profile/price/liquidity collection for its members.
- A normalized classification contract with source, effective timestamp, and confidence/quality state.
- A bounded selection interface that feeds the existing ResearchAgent queue without replacing its entry, risk, or broker gates.
- Owner-visible coverage, source freshness, sector counts, and exclusions in Upgrade Path / Research.

India: deliberately deferred, not forgotten

This document is US-only, and the schema names (`us_instrument_catalog`, `us_measurement_universe`) make that permanent if left unstated. India has the **same** population-bias problem at nearly the same scale — 161 distinct India

symbols in `decision_observations` against 167 US — so the deferral is a sequencing choice, not evidence that India is fine.

India is out of scope for Stage 0 because its sourcing is materially harder: there is no Nasdaq-directory equivalent, NSE/BSE reference data carries its own licensing, and the classification vendor question must be answered separately. Deferring keeps Stage 0 cheap and honest. A later India cohort must be its own architecture with its own sources and its own frozen policy — it must NOT be produced by widening a US-shaped table, and cross-market universe aggregates remain invalid per the Scoring Data-Truth Review Protocol.

Explicit non-goals

- No all-security daily LLM research.
- No claim that every catalogued security is Robinhood-tradable, liquid, or safe.
- No bulk broker probing of 11,200 securities unless the broker documents a read-only bulk capability and its rate/terms have been verified.
- No historical backtest using today's catalog membership.
- No automatic watchlist insertion, paper entry, live entry, score-weight change, broker enablement, or `live_auto_enabled` change.
- No use of the catalog to bypass `symbol_blocklist`, leveraged/inverse ETF policy, account allowlists, or last-mile broker preflight.

3. Population policy

3.1 Catalog: broad observation, not endorsement

`us_instrument_catalog` represents an observed current instrument identity:

- `instrument_id` (internal immutable identity), source IDs, ticker, exchange, name, observed instrument type, active/status, first/last seen timestamps;
- SEC CIK when corroborated, with conflicts recorded rather than overwritten;
- source payload hash, retrieval timestamp, parser version, and source freshness;
- `no_tradable=true` boolean. Broker capability is account- and time-specific.

The catalog accepts common stock, ADR, ETF, CEF, preferred, warrant, unit and OTC as distinct types so the system can explicitly exclude or separately study them.

It is not a trading allowlist.

3.2 Measurement universe: policy-controlled common-stock cohort

Initial measurement is US exchange-listed **ordinary common equity only**. ETFs,

ADRs, CEFs, preferreds, units, warrants, OTC issues, SPAC shells and leveraged or inverse products are excluded from the initial cross-sectional sector cohort and record an exclusion reason. They may receive separate instrument-family studies; they must not dilute common-stock sector statistics.

Each daily member must satisfy a frozen policy version:

1. active primary US exchange listing from a documented source;
2. resolvable identity and a current classification of sufficient quality;
3. at least 60 completed trading sessions of price history;
4. a predeclared liquidity threshold using trailing 20-session median dollar volume;
5. valid close/volume coverage on the prior session;
6. no hard block in `symbol_blocklist` or generic instrument policy.

Do not choose the liquidity threshold now. Stage 0 measures \$2m, \$5m and \$10m 20-day median-dollar-volume arms for size, sector coverage, provider cost and missing bar rate. The eventual threshold, minimum price rule (if any), and maximum cohort size are one frozen policy decision after that evidence. The architecture must not silently select the result with the best realized returns.

Membership is not sector-quota-selected: forcing exactly N names per sector would invent an investable universe that does not exist. The owner-visible report may show undercovered sectors and the shortlist may impose exposure limits later, but the measurement cohort remains a transparent liquidity policy.

3.3 Daily research shortlist: deterministic-wide, AI-narrow

The existing ResearchAgent keeps a bounded deep-research budget. A new deterministic selector may propose a diversified shortlist only from that day's measurement universe, with every inclusion carrying an immutable reason:

- held positions always retain their existing monitoring path;
- current manual watchlist intent remains a separately labeled source;
- candidates are ranked by existing deterministic screens/edge measurements, liquidity, freshness, and an explicit per-sector maximum;
- no score threshold, buy recommendation, or order may be produced merely because a name is in the broad cohort or shortlist.

The selector's **shadow** compares the existing candidate queue with the diversified shortlist on coverage, data quality, score distribution, future labels and benchmark-relative outcomes before the shortlist may replace any current source.

3.4 Current-screen audit and proposed family router

****Verdict:** the current screens are a respectable safety baseline, but are not good enough to be the long-term selection architecture.** They are good at avoiding a silent outage and producing a small list; they are not yet good at choosing the best research allocation across unlike instruments.

Current path	What it does well	Material limitation
Yahoo common-equity momentum/value buckets	Limits to NMS/NYSE, rejects malformed preferred/warrant-style tickers, applies a volume floor, interleaves the two buckets, and has a daily absurd-threshold contract check	Only two static fundamental screens; its "momentum" ranks quarterly revenue growth, not price momentum; volume is shares/day rather than dollar liquidity; it has no spread, sector, correlation, data-quality or forward-outcome test
FinancialDatasets fallback	Keeps a second provider path when Yahoo fails	It has previously become unavailable when credits were exhausted, so it cannot be a silent primary truth source
Relative-strength discovery	Uses completed-session persisted price evidence and limits itself to four candidates	Its present input is a static current-liquid list, explicitly survivorship-biased; it is not a point-in-time broad-universe ranking
Metals / region / ETF paths	Preserve selected holdings and intentionally append known baskets	They are curated static lists, not family-specific screens; ETFs cannot be validly ranked with an operating-company P/E or revenue-growth rule

The Yahoo field-contract cron is valuable and must remain: it detects whether a provider still **applies** a field. It does **not** prove that the screen's economic idea predicts returns, that it works outside technology, or that it selects liquid broker-fillable candidates. "The filter executed" and "the filter has edge" are different claims.

The replacement is a **family router**, not one giant stock screener. It uses the catalog's verified instrument type and classification to choose a predeclared, deterministic candidate screen:

Family	Candidate-screen inputs (initial shadow only)	Explicitly not used
Ordinary common equity	minimum dollar liquidity, price-history/data quality, price relative strength, quality/growth, value, and only point-in-time earnings-revision data once its provenance is available	ETF/fund mechanics or a single universal P/E cutoff
Banks / REITs	same basic liquidity/price requirements plus family-applicable valuation/quality fields	generic operating-company growth screens when the field is economically inapplicable
Broad, sector and thematic ETFs	fund identity, assets/liquidity/spread availability, completed-session trend/relative strength and volatility	company revenue growth, ROE and P/E screens
Metals bullion funds / miners / streamers	instrument-family identity, fund liquidity, completed-session relative strength and volatility; commodity/macro features only after a sourced data contract and shadow evidence exist	treating GLD, GDX and a gold miner as the same issuer type or inventing a commodity signal from LLM text
ADRs, CEFs, preferreds, warrants, OTC, leveraged/inverse products	catalog and exclusion reporting only in the first release	generic automatic candidate selection

Each family screen is a named **candidate source**, never a score adjustment or buy rule. The router records the exact inputs, missing fields, policy version and rank. It feeds a fixed-capacity, correlation-aware shortlist: no family can consume the entire daily research budget; overlapping exposures and duplicate share classes

are deduplicated before the LLM stage.

Before any family screen is used outside shadow, freeze its primary horizon, benchmark, selection cap and material-effect threshold. Evaluate it against (a) the incumbent queue and (b) a transparent liquidity-only control on the same sessions. Report coverage, data failure, turnover, h5/h10/h20 benchmark-relative outcomes, drawdown and concentration. A screen with no adequate mature evidence remains `insufficient_evidence`; it does not earn capital merely by producing attractive names.

4. Data contracts and schema direction

This is a migration plan, not approval to apply one.

4.1 New append-only objects

1. `instrument_catalog_sources` — source/version/license/terms review and latest freshness; one active source policy per purpose, never a hidden fallback.
2. `us_instrument_catalog` — current identity state keyed by stable internal ID.
3. `instrument_catalog_events` — append-only adds, changes, delists, identity and classification conflicts, with raw-hash/source references.
4. `universe_policies` — immutable policy version, inclusion/exclusion predicates, thresholds, source versions, code fingerprint and content hash.
5. `universe_memberships` — immutable (`policy_id`, `as_of_session`, `instrument_id`) membership snapshots plus included and deterministic reason codes. Exclusions are sampled/aggregated in a companion coverage ledger rather than writing millions of rejected rows indefinitely.
6. `universe_collection_runs` — lease/run key, source freshness, input/output counts, errors, budget consumption, catalog and membership fingerprints.
7. `instrument_classifications` — append-only provider observations; normalized sector/industry, taxonomy/version, effective/observed timestamps and quality.
8. `universe_shortlist_assessments` — immutable daily arms, candidate source, selection reason, feature freshness and later label links.

`edge_universe_members` remains owned by Edge discovery. Reusing it would combine two policy lifecycles and make future evidence provenance ambiguous.

4.2 Required invariants

- A membership snapshot cannot be UPDATED, DELETED or TRUNCATED; corrections append a new observation/session with a source-correction link.
- Every computed statistic names `universe_policy_version`, `as_of_session`,

membership fingerprint, data-cutoff timestamp and code version.

- Missing classification is `unclassified`, never guessed from a company name.
- A source outage is `unavailable`, not an empty universe or zero members.
- Delisted and symbol-reused identities never share an internal instrument ID.
- An expired broker preflight cannot be used to advertise a currently tradable instrument, and catalog membership can never satisfy an order preflight.

5. Collection design

Stage 0 — source and capacity proof (read-only)

Before a migration, build only an owner-visible capability report that samples and records:

1. Nasdaq listed and other-listed directory parseability, timestamps, duplicate and test-issue handling;
2. NYSE/NYSE Arca coverage route, licensing and whether a source can legally be stored and used by Kairos;
3. SEC CIK/ticker/exchange corroboration and conflict rate;
4. classification provider coverage, taxonomy consistency and TTL across a stratified sample of common stocks;
5. price/bar and liquidity coverage, provider call cost and practical batch rate;
6. Robinhood REST lookup on a small, rate-limited stratified sample only—reported as corroboration, never assumed to enumerate all account-eligible instruments.

Exit criteria: named source contracts, acceptable licensing, measured missingness, and a budgeted daily cohort size. If any source is not adequate, the program stops at a truthfully incomplete capability report.

Stage 1 — catalog and snapshot (evidence-only)

On each US market session after the reference-data cutoff:

1. fetch and archive raw source manifests/payload hashes;
2. reconcile identities without ticker-only joins;
3. write catalog events and current state;
4. collect deterministic prior-session bars and liquidity in bounded batches;
5. write the frozen membership snapshot and a coverage/failure summary;
6. refresh classifications only when stale, while preserving their source/version;
7. expose freshness and exclusions. No ordinary ResearchAgent change yet.

Collection must be resumable, idempotent, rate-limited, leased, and fail closed for

membership publication: a partial source run is reported as partial and cannot replace a complete snapshot.

Stage 2 — shortlist shadow (still evidence-only)

For each complete measurement snapshot, produce two persisted arms:

- `incumbent_queue`: current ResearchAgent candidate provenance;
- `diversified_measurement_shortlist_v1`: deterministic ranking from the broad measurement membership under fixed per-sector maximum and overall daily cap.

Both arms receive the same score/data pipeline when capacity permits. Report their overlap, sector/size/liquidity coverage, data failures, score distributions, entry eligibility incidence, h5/h10/h20 returns and benchmark-relative results. A name that lacks data stays a named failure; it must not silently drop from one arm.

Stage 2 includes one `family_router_v1` arm per applicable family alongside a liquidity-only control. Do not ship all family screens at once: start with ordinary common equities, then add ETFs and metals only after their data contracts and appropriate controls are verified. This keeps a poor ETF or commodity screen from being hidden inside an apparently successful equity aggregate.

Stage 3 — separately proposed limited integration

Only after Stage 2 has adequate matured labels and no quality regressions may a separate architecture propose changing the existing candidate queue. It must state the exact policy arm, rollout percentage, rollback switch and effects on Research, PaperTrader and live gateways. No live promotion follows automatically.

6. Classification policy

Use a licensed/reference provider chosen in Stage 0 for a normalized taxonomy such as GICS sector plus industry group. Store the raw provider classification alongside the normalized result. Finnhub's coarse industry can remain a display fallback but cannot be treated as the canonical research-universe label without a coverage and mapping audit.

Classification resolution order is deterministic:

1. source-specific stable instrument ID and current official/reference taxonomy;
2. independently corroborated provider mapping;
3. `unclassified` with source failure/conflict reason.

Never infer sector from ticker, LLM text, fund name, or a single peer list. ETFs and

multi-sector conglomerates require separate classification semantics and therefore remain outside the first common-equity cohort.

7. Safety and failure behavior

Condition	Required behavior
Source says no records / parser breaks	Mark run unavailable; retain prior snapshot as historical only; do not publish an empty current one
Identity or ticker conflict	Preserve both claims, mark unresolved, exclude from new measurement membership
Classification missing/stale	unclassified; no sector statistic inclusion
Price/liquidity missing	Exclude with reason; count in coverage report
Broker lookup denied/stale/unsupported	Catalog may remain observed; it cannot enter a broker-compatible shortlist label and cannot advance enforcement evidence
Corporate action/delisting/symbol reuse	Append event and terminate old identity; no ticker-only carry-forward
Provider budget exhausted	Stop new requests, report partial coverage, do not downgrade quality silently
Shortlist output empty	Record an explicit abstention with input/count/failure reasons, never fall back to the curated queue without labeling it

8. Owner-facing surfaces

Upgrade Path is the aggregate program view. It should show:

- policy version, latest complete session, sources and freshness;
- catalog count by instrument type and measurement-member count by sector;
- classification, identity, price and liquidity coverage; exclusions by reason;
- the current broker-probe sample rate and outcome—not a false claim of universal

Robinhood tradability;

- incumbent versus shadow shortlist overlap and matured-outcome evidence;
- clear state: `collecting`, `partial`, `insufficient_evidence`, `ready_for_review`, or `blocked`.

Research should show a symbol's membership source, policy, classification provenance and whether it was selected by the incumbent or shadow shortlist. It must never call the broad catalog a buy list.

9. Acceptance criteria

Architecture / Stage 0

- [] No unsupported claim that Robinhood supplies a bulk, historical or canonical sector universe.
- [] Every source has a documented license/terms, freshness, identity fields, instrument coverage and measured sample failure rate.
- [] Three liquidity arms are measured before a default threshold is selected.

- [] A cost/capacity model proves the deterministic-wide run fits the scheduled provider budget; no LLM budget is used for the full cohort.
- [] Robinhood bulk/candidate-probe limits are verified from the actual connected adapter/tool contract, not assumed from a consumer app screen.

Stage 1

- [] Re-running one session is idempotent; a partial run cannot overwrite a full snapshot.
- [] Every membership statistic is reproducible from a frozen policy, source hash, as-of session and code version.
- [] Sector coverage reports distinguish unclassified, excluded instrument type, and missing provider data.
- [] Mutation tests prove that current catalog membership cannot be reused for a historical snapshot and that a stale broker preflight cannot mark a name tradable.

Stage 2

- [] Both selection arms are persisted before outcomes mature.
- [] The UI shows observations and uncertainty, not a winning-strategy claim.
- [] No score, eligibility, sizing, paper, live order, or broker configuration is changed by this program.
- [] Every family router has a matching liquidity-only control and reports its coverage, failure rate, overlap and matured benchmark-relative outcomes.
- [] The price-relative-strength arm is not promoted while its source universe is the current-liquid, survivorship-biased Edge list.

10. Decisions needed before implementation

1. Approve Stage 0 capability/cost study only, or fund/select a licensed US security-master and classification vendor now.
2. Decide whether the first measurement cohort is common equities only (recommended) or whether ETFs must be a separately governed simultaneous cohort.
3. After Stage 0: freeze the liquidity/minimum-price policy and daily deterministic budget; do not set these from historical returns.
4. Approve any Stage 1 schema migration only after the selected source contracts and data-retention terms are recorded.

11. What this document deliberately rejects

- “Research all 11,200 with AI every day.” This spends the budget on heterogeneous,

often untradeable instruments while making the output less auditable.

- “Use Robinhood category labels.” A broker execution inventory is not a stable security-classification system.
- “Lower the sector sample floor.” That would make a weak cohort look statistically adequate rather than increase the evidence population.
- “Automatically buy all newly discovered names.” Discovery and execution are intentionally different safety states.

Source: features\broker-symbol-tradability\FEATURE_ARCHITECTURE.md

Broker Symbol Tradability Gate — Feature Architecture

Status: Stage 0 approved 2026-09-09; shadow implementation and production schema deployed, enforcement remains gated

Scope: Every live BUY, SELL, and broker-side protective order

Behavior today: Broker-authoritative symbol/side preflights are recorded in shadow mode; they do not yet block or enable orders

Decision

Kairos must never submit a live order merely because a symbol is syntactically valid or belongs to its research universe. Immediately before the irreversible broker call, it must prove that the exact broker, account, environment, instrument, side, quantity, and order type are allowed.

This is a safety gate, not a research filter. Research may study a wider universe. An approved proposal is still not executable until this gate passes.

Contract

Extend BrokerAdapter with a mandatory preflight method:

```
type BrokerInstrumentCapability = {
  broker: string;
  accountId: string;
  env: "paper" | "live";
  market: "us" | "india";
  requestedSymbol: string;
  canonicalSymbol: string;
  instrumentId: string;
  side: "buy" | "sell";
  orderType: "market" | "limit" | "protective_stop" | "protective_target";
  allowed: boolean;
  active: boolean;
  buyAllowed: boolean;
  sellAllowed: boolean;
  closeOnly: boolean;
  fractionalAllowed: boolean;
  lotSize: number;
```

```

tickSize: number | null;
checkedAt: string;
expiresAt: string;
source: string;
reasonCode: string | null;
rawFingerprint: string;
};

```

The check is side-specific. A broker disabling new BUYs must not automatically block a risk-reducing SELL. Conversely, a held symbol is not proof that a SELL is currently accepted. If the broker cannot establish the requested side, the app fails closed and directs the owner to act in the broker app.

Authoritative sources

- Alpaca: GET /v2/assets/{symbol_or_asset_id}; require an active matching asset and tradable=true, plus account/order constraints.
- Zerodha Kite: the daily instruments master establishes the exchange:tradingSymbol, token, lot size, and tick size; a fresh quote key must also exist. The master is refreshed once daily before market open.
- Robinhood: resolve the broker instrument identifier and inspect the instrument/account tradeability response. If the MCP bridge cannot expose this capability, Robinhood live submission remains unsupported through that adapter rather than guessing.
- Future adapters, including Webull, cannot register without implementing this contract and passing the conformance suite.

Placement in the live path

1. Validate proposal and local symbol policy.
2. Select the exact broker and account.
3. Fetch a fresh broker capability (short TTL; no stale fallback).
4. Validate side, quantity/lot, tick, order type, and session restrictions.
5. Persist immutable preflight evidence tied to the proposal/order intent.
6. Reserve the order atomically only when the reservation RPC receives the same preflight fingerprint.
7. Re-check the fingerprint and capability in the adapter immediately before the wire call.
8. Submit once. An ambiguous response is reconciled, never retried blindly.

Both layers are required: the gateway check gives a consistent refusal and audit record; the last-mile adapter check prevents direct routes or future callers from bypassing it.

Paths that must be covered

- lib/trading/execute-order.ts
- every adapter implementing lib/brokers/adapter-types.ts
- app/api/kite/order/route.ts (currently bypasses the generic gateway)
- lib/protective/kite-placement.ts and Kite GTT placement
- autonomous live entry and live-exit monitor paths
- any future ladder leg, stop, target, replacement, or cancel/replace submission

No exception is permitted for an “urgent” automated exit. If validation infrastructure is unavailable, Kairos alerts the owner to exit directly at the broker; it does not submit an unverified order.

Persistence and database gate

Add an append-only `broker_instrument_preflights` table with proposal/order linkage and the fields above. Revoke UPDATE, DELETE, and TRUNCATE from application roles; enforce immutability with triggers and tested grants. The live reservation function must reject missing, expired, mismatched, denied, or already-consumed evidence.

This schema and RPC change requires explicit approval and a checked-in migration before production application.

Tests and rollout

- Adapter conformance tests for unknown, inactive, not-tradable, buy-disabled, sell-disabled, close-only, fractional, wrong lot, wrong tick, delisted, halted, stale, timeout, and symbol-alias cases.
- Mutation test: force each adapter to return `allowed=true`; a denial fixture must fail.
- Static direct-submit inventory test: every broker wire-call site must be registered and guarded.
- Rolled-back database tests for freshness, fingerprint matching, single consumption, RLS, grants, and TRUNCATE denial.
- Shadow for at least 10 market sessions: record would-pass/would-block without changing execution. Investigate every disagreement with actual broker acceptance/rejection.
- Owner review of the shadow report, then a separate approval to enforce on paper, and another explicit approval before live enforcement.

Explicit non-goals

- This does not broaden the research universe.
- It does not enable Alpaca, Robinhood, Kite, Webull, Coin, or live trading.
- It does not infer mutual-fund availability in Zerodha Coin from Kite equity instruments. Coin needs its own instrument/order contract if it is ever automated.
- It does not allow a cached research-provider symbol list to substitute for broker truth.

**Source: features\capital-advisor\FEATURE_ARC
HITECTURE.md**

Capital Advisor

Status

Implemented foundation on 2026-08-08. Capital allocation, securities orders, property purchases, refinancing, HELOC draws, and mortgage prepayments remain unauthorized. See `IMPLEMENTATION_RESULT.md` for the shipped boundary and the explicitly gated future phases.

Decision

Build a new owner-only **Capital Plan** workspace above the existing Investing and Property workspaces. It answers one constrained question:

Given the owner's stated goals, cash, liquidity need, debt terms, and
evidence quality, which use of the next dollar deserves review: keep cash,
reduce debt, add to a governed securities sleeve, or pursue a saved property
thesis?

It is decision support, not a broker, lender, property marketplace, appraisal, or autonomous adviser. It can recommend watch, review, `outside_policy`, or `insufficient_evidence`; it must not issue an imperative buy, sell, borrow, or refinance command.

This is deliberately not a new tab inside Property. The Property workspace is correctly isolated from brokerage accounts and portfolio cash. Capital Plan is the sole cross-workspace reader and consumes read-only, currency-labelled summaries from both sides.

User Jobs

1. Maintain a watchlist of cities, ZIPs, localities, and property theses.
2. Evaluate a rental house, 2-4 unit building, land parcel, or apartment thesis with stated assumptions and downside cases.
3. Compare an optional mortgage principal payment with retaining cash or adding to an investment sleeve on a consistent after-cost, risk, and liquidity basis.
4. Understand why Kairos changed its view and what evidence would change it again.
5. Review historical advice versus subsequent outcomes without allowing a few good calls to create an unsupported allocation rule.

Non-Goals

- No exact ZIP ROI forecast, property AVM, appraisal, listing recommendation, tenant selection, lender approval, or loan offer.
- No scraping Zillow, Redfin listing pages, MLS, housing portals, or Indian listing sites. Source contracts are required even for aggregate feeds.
- No protected-class, demographic, school-rating, crime, or other proxy-based ranking. ZIP/locality is a geographic evidence scope, not a desirability or borrower-risk score.

- No automatic stock sale, broker order, property purchase, mortgage payoff, refinance, HELOC draw, or transfer of capital.
- No cross-currency optimization. US USD, India INR, and each securities market remain independent until an owner explicitly supplies an FX and tax scenario.

Inputs And Privacy

Capital profile

The owner explicitly maintains:

- emergency-reserve months and minimum cash floor;
- investable surplus, recurring monthly surplus, and tax assumptions;
- horizon, liquidity deadline, maximum property concentration, and leverage ceiling;
- property strategy: owner-occupied, long-term rental, 2-4 unit, land, or commercial apartment;
- whether a comparison is an analysis only or a future action the owner might personally execute.

Mortgage and financing profile

For each financing account, store encrypted terms already used by Property:

- principal, rate type, stated rate/index/margin, remaining term, payment, prepayment penalty, recast eligibility, and deductible-interest assumption;
- owner-declared tax treatment and any known lender restriction;
- an optional proposed principal payment and funding source.

The system never infers a tax deduction, lender permission, or credit decision.

Unknown terms produce `insufficient_evidence`.

Cross-workspace read boundary

Capital Plan reads a summarized snapshot, never raw broker credentials, full property address, loan document, or unredacted account number:

Source	Permitted snapshot	Prohibited
Investing	currency, liquid cash, sleeve value, benchmarked outcome range, risk and data-quality state	broker token, order endpoint, individual order payload
Property	geography ID, asset type, underwriting scenario range, financing metrics, evidence freshness	exact address, document payload, lender quote image
Owner profile	declared constraints and assumptions	automatic inference of protected or sensitive characteristics

All snapshots are timestamped and versioned. An expired input cannot become a

current recommendation.

Evidence Model

Area watchlists

`capital_area_watchlists` records an owner-selected city, ZIP, locality, or metro, asset type, intended strategy, budget range, hold horizon, and watch conditions. It is not a universe crawler.

Cadence follows source reality:

- metro evidence can refresh weekly where the approved source publishes weekly;
- ZIP evidence refreshes monthly or at the source's rolling-window cadence;
- locality/PIN evidence is unavailable until a lawful local source is active;
- a data revision creates a new observation, never a silent overwrite.

For example, Redfin says ZIP/neighborhood monthly measures use rolling three-month windows; a weekly ZIP alert therefore has no evidentiary basis. Redfin remains `contract_pending` until reuse terms are recorded.

Underwriting scenarios

`capital_property_theses` and immutable `capital_underwriting_runs` hold only owner inputs and approved market evidence.

Asset type	Required metrics	Must remain separate
Rental home / condo	purchase, rent, vacancy, tax, insurance, maintenance, HOA, management, debt service, sale cost	personal home and commercial property
2-4 unit	unit-level rent and vacancy, debt service, repair reserve	single-family default assumptions
Land	carrying cost, tax, zoning/utilities evidence, liquidity, development assumption	rent, cap rate, and DSCR defaults
Apartment 5+ units	commercial NOI, debt structure, occupancy, cap-exit sensitivity	residential comparable or owner-home model

Outputs include cash flow, cap rate, cash-on-cash range, DSCR, debt paydown, break-even sale price, downside sensitivity, and an IRR range. Every output binds inputs, source IDs, data cutoff, calculation version, and currency.

Securities comparison

The public-equity sleeve contributes only evidence Kairos can support today:

- current liquid cash and policy sleeve state;
- benchmark-relative realized performance and data-quality state;

- a scenario range only when its policy/model has passed its own validation gate.

Kairos must not convert an unvalidated stock score into a precise expected return just to make the property comparison look complete. When no qualified equity-return range exists, Capital Plan shows it as not comparable rather than inventing a winner.

Deterministic Decision Engines

1. Capital safety gate

Before comparing opportunities, reserve:

1. the owner-declared emergency cash floor;
2. known near-term obligations and property repairs;
3. taxes, transaction costs, and required down-payment liquidity;
4. the maximum debt and property-concentration constraints.

If available capital is below this floor, every deploy-capital alternative is `outside_policy`. The tool may still explain the scenario.

2. Mortgage principal payment versus investing

This engine compares **uses of surplus cash**, not total account balances.

For a proposed principal payment, it calculates:

- scheduled interest saved, term reduction, and cash-flow effect separately;
- effective guaranteed return from avoided interest under the owner-declared tax treatment;
- prepayment penalty and the difference between principal reduction and a lender-approved recast;
- liquidity lost and the effect on LTV/CLTV and payment stress;
- downside cases for variable-rate/indexed loans.

For retaining cash or investing, it compares only a qualified scenario range after fees, taxes, liquidity haircut, and drawdown range. A historical market return is not an expected return. The decision summary must show the same horizon for both sides.

Default outcomes:

Condition	Output
Emergency reserve, tax treatment, penalty, or rate terms unknown	<code>insufficient_evidence</code>

Condition	Output
Principal payment violates cash floor or impending obligation	outside_policy
Mortgage saving dominates the qualified alternative across downside cases	review_principal_payment
Qualified alternative dominates but requires an unvalidated forecast	watch, not invest
Ranges overlap materially	indifferent_under_assumptions

No result sells securities, moves money, or tells the owner to skip an employer match, retirement contribution, insurance, or tax obligation. Those are explicitly outside the initial scope.

3. Cross-asset comparison

Every candidate is compared using a declared common frame:

- after-cost nominal and real return range;
- downside loss/range and leverage sensitivity;
- liquidity and time-to-deploy/time-to-exit;
- tax and transaction cost assumptions;
- concentration impact relative to capital policy;
- evidence quality and forecast calibration state.

The primary ranking is **fit to declared constraints**, not highest midpoint ROI. A high leverage property with a wide loss range cannot outrank a liquid investment merely because its base-case IRR is larger.

Agent Structure

```

flowchart LR
    O["Owner inputs and constraints"] --> G["Capital safety gate"]
    I["Investing evidence snapshot"] --> A["Cross-asset allocator"]
    P["Property evidence and underwriting"] --> A
    W["Area watchlists"] --> R["Area observer"]
    R --> P
    G --> A
    A --> L["LLM advisor narrator"]
    A --> J["Capital decision journal"]
    J --> M["Outcome maturation and challenger evaluator"]
    M --> A

```

AreaObserver

Deterministic collector and classifier. It reads only approved, market-local sources and records price/rent/supply/financing context with cadence and freshness. It cannot rank areas or interact with brokerage/lending systems.

UnderwritingEngine

Pure deterministic calculation over owner assumptions and evidence. It creates immutable scenario runs and fails visibly when inputs are missing.

CrossAssetAllocator

Pure deterministic policy engine. It applies cash, leverage, liquidity, concentration, and evidence-quality gates before presenting any comparison. It cannot call a broker, lender, property source, or LLM.

AdvisorNarrator

LLM-only explanation layer. Its tool input is a sealed, typed evidence envelope from the allocator. It may explain why, list assumptions, name missing data, and propose questions for the owner. It cannot alter values, rank candidates, write a recommendation state, or use uncited web text.

CapitalLearner

The learner does not reward or punish an LLM. It measures calibrated outcomes:

- area regime/forecast versus subsequent published market observations;
- underwriting forecast versus owner-entered actual rent, expenses, vacancy, refinance, or sale evidence;
- allocator recommendation versus a declared passive/cash counterfactual.

It may create a new **shadow policy** with a predeclared hypothesis. It may not modify capital policy, a mortgage result, a property thesis, stock score, broker setting, or live allocation. Promotion requires enough independent, matured outcomes, same-market comparison, costs, drawdown, calibration, and an owner approval.

User Experience

Capital Plan overview

Show one current-state panel:

- Capital available after reserve;
- current debt, property, cash, and securities concentration by currency;
- best evidence-backed watch item, if any;
- why no item is actionable when that is the correct answer;
- source freshness and data-quality badges.

Area watch detail

For each saved city/ZIP/locality, show:

- trend and supply/demand charts at the source's true cadence;
- asset-type-specific evidence and missing inputs;
- owner-declared buy/rent/land criteria;
- weekly/monthly change log and alert conditions;
- a cited LLM brief that cannot introduce a new numerical claim.

Mortgage decision detail

Show a side-by-side comparison of pay principal, hold cash, and qualified investment alternatives. Include interest saved, term change, liquidity lost, range comparison, tax/penalty assumptions, and the exact reason for the result. The owner must enter a proposed amount; the app never assumes all extra cash is available.

Decision journal

Every displayed recommendation records policy version, evidence IDs, model version, inputs, output state, explanation hash, owner decision, and later outcome. A later LLM summary cannot rewrite the original rationale.

Data And Compliance Gates

Metro and county scope

Metro-level evidence must not be mistaken for one county. Kairos uses the current OMB/Census metro definitions as a declared coverage registry:

Market	County scope	What can be collected now	What remains gated
Austin-Round Rock-Georgetown	Bastrop, Caldwell, Hays, Travis, Williamson	Whole-metro FHFA/BLS/FRED context; county ACS context after a server Census key	ZIP market feed and any county appraisal/sale feed until that publisher permits automated use
Phoenix-Mesa-Chandler	Maricopa, Pinal	Whole-metro FHFA/BLS/FRED context; county ACS context after a server Census key	Maricopa recorded-sale feed and any Pinal equivalent until each source contract is separately approved
Bengaluru	city-level only	RBI/NHB city series after direct release-contract validation	locality/PIN/parcel/sale collection and every automated guidance-value lookup

Every county row carries its county FIPS and ACS release vintage. ACS is annual context (income, median value, gross rent, and rental vacancy), not current sale-price evidence, a desirability score, or an allocation input. A local source may not be activated for the whole metro merely because one county has a public download page.

- Aggregate US source use requires a documented contract. Redfin publishes broad market metrics across metros and ZIPs, but its releases can be revised;

it remains inactive until Kairos records permitted reuse and revision policy.

- Census ACS may provide slow-moving geographic context, not a near-term price signal; current API access requires a key.
- FHFA, FRED, and BLS remain market context only according to their current data contracts.
- No protected-class or proxy feature may enter a geography score. The Fair Housing Act prohibits discrimination in housing and housing-related lending.
- This is a private, single-owner decision-support system. Before serving other users, accepting compensation, or executing personalized securities advice, obtain legal/compliance review. Automated personalized investment programs can be subject to investment-adviser requirements.

Delivery Sequence

P0: Architecture and policy only

Create this document, a decision record, and a reusable evidence-envelope contract. No new UI, LLM, source, or money-path behavior.

P1: Capital profile and manual scenarios

Add encrypted owner inputs, deterministic mortgage principal comparison, and manual property underwriting. Results are explanation-free until the deterministic outputs are verified.

P2: Area watchlists and source-backed monitoring

Add city/ZIP/locality watchlists, true-cadence collection, source health, and immutable snapshots. Start with Austin/Phoenix metro data; do not activate ZIP or Bengaluru sources without a contract.

P3: Read-only cross-asset comparison

Add the cross-workspace snapshot boundary, safety gate, and constraint-fit comparison. Exclude unvalidated stock expected-return claims.

P4: LLM narrative

Add the sealed AdvisorNarrator after P1-P3 are verified. It explains only deterministic facts and is logged in the decision journal.

P5: Shadow learning

Mature outcomes, compare predeclared challenger policies with the same-market

baseline, and expose calibration. No automatic promotion or capital movement.

Acceptance Criteria

- An owner can compare a stated principal payment against cash/investment only after all required terms and reserve constraints are explicit.
- All calculations are deterministic, reproducible, versioned, and currency isolated.
- Every LLM statement maps to cited evidence IDs; it cannot change a numerical output or execute an action.
- ZIP/locality monitoring respects each source's cadence and availability.
- A property or securities forecast with inadequate evidence cannot win a comparison by displaying a fabricated ROI.
- No Capital Plan path invokes a broker, lender, trade route, property purchase flow, or personal-document provider.
- Challenger policies remain shadow-only until matured outcome and owner approval gates pass.

References

- Redfin market-data methodology and publication cadence:
<https://www.redfin.com/news/data-center/methodology/>
- Census API available data and geography coverage:
https://www.census.gov/data/developers/guidance/api-user-guide.Available_Data.html
- HUD Fair Housing rights and obligations:
<https://www.hud.gov/stat/fheo/rights-obligations>
- SEC Investor Bulletin on robo-advisers:
<https://www.investor.gov/introduction-investing/general-resources/news-alerts/alerts-bulletins/investor-bulletins-45>

Source: features\capital-advisor\IMPLEMENTATION_RESULT.md

Capital Advisor Implementation Result

Implemented: 2026-08-08

Shipped

- New owner-only /capital-plan workspace, reachable from both Investing and Property through the three-way workspace switch.
- AES-256-GCM encrypted Capital Profile, immutable profile snapshots, area watches, and immutable decision-run records.
- Deterministic partial-mortgage-prepayment comparison. It separates gross scheduled-interest savings, an owner-declared tax assumption, prepayment penalty, term reduction, and the hard cash-floor check. It never assumes a lender recast and never initiates a payment.
- Read-only property-versus-market scenario comparison. Both return ranges are owner-entered and clearly remain assumptions; cash floor, liquidity, and concentration prevent a higher midpoint from becoming a false recommendation.
- Weekly database-only area-watch snapshots after Property collection. These read existing observations only, use no new provider calls, and record metro_only or contract_pending rather than producing a ZIP/PIN/locality forecast without an approved source contract.
- Explicit metro-county coverage: Austin-Round Rock-Georgetown includes Bastrop, Caldwell, Hays, Travis, and Williamson; Phoenix-Mesa-Chandler includes Maricopa and Pinal. A server-key-gated ACS collector is ready to record annual county income, rent, value, and vacancy context for every declared county. It is not active without CENSUS_API_KEY and source approval, and it never represents annual ACS data as sale-price evidence.
- A sealed future narrator envelope in lib/capital-advisor/narrator.ts. It contains no address, lender identifier, account number, execution instruction, or authority to alter the deterministic decision state.

Deliberately Not Activated

- No live broker, bank, lender, or payment integration.
- No automatic property search, scraping, AVM, exact ZIP/PIN ROI claim, or Bengaluru-local conclusion. Current market evidence is metro-level only where the Property source contract is active.
- No LLM narrative call, allocation mutation, or auto-learning policy. The sealed envelope exists so a future narrator cannot receive raw private data or write back to a calculation. Outcome learning remains blocked until real, timestamped owner outcomes exist.
- No cross-currency ranking. A Capital Profile is explicitly USD or INR and a comparison uses one chosen currency only.

Verification

- `npx tsc --noEmit`
- `npx vitest run tests/capital-advisor.test.ts` (4 tests)

Required Before Expanding

1. Apply and verify `20260808223000_capital_advisor_foundation.sql` in the production Supabase project.
2. Obtain an approved aggregate source contract before enabling ZIP/PIN/locality collection or alerts.
3. Define matured-outcome semantics and a minimum sample before a shadow policy can be evaluated. A narrator must remain explanation-only.
4. Configure a Census API key and explicitly activate `census-ac`s; configure a HUD token separately for the existing annual FMR reference. Confirm machine-use terms separately for every county's parcel or recorded-sale feed.

Source: features\capital-rotation\FEATURE_ARCHITECTURE.md

Feature Architecture - Capital Rotation

Status: P0 SHADOW BUILT 2026-07-13 . Paper/live execution remains disabled and unbuilt.
Scope: deterministic opportunity-cost reallocation for PAPER first, LIVE approval proposals later.
Last updated: 2026-07-13
Update when built: docs/arch/03-agents.md, docs/arch/04-database-schema.md, docs/arch/08-risk-and-safety.md, docs/arch/09-learning-loop.md, public/agent-diagrams/system-map.json.

One-line decision

Build capital rotation as a **deterministic evaluator inside the existing entry flows**, not as a new autonomous reallocation agent. It may only propose or execute a rotation after the candidate has passed every normal entry gate except cash. PositionMonitor still owns exits. Live rotation is approval-required and two-leg reconciled.

Problem

When a market-local book is fully invested and a materially better candidate appears, the current paper/live flows skip the buy with `insufficient_cash`. Existing exits are absolute risk or thesis exits: score-below-exit, stop, target, time-stop, and partial-profit. The missing behavior is opportunity-cost reallocation: sell the weakest still-held, still-valid position only when the new candidate is

materially better after cost, tax, turnover, and safety gates.

Non-negotiable rules

1. **No LLM on the money path.** LLM output may explain a signal upstream, but rotation selection, thresholds, tax/cost math, and execution decisions are deterministic.
2. **Per-market and per-currency only.** US/USD and India/INR books are separate. No cross-market cash, NAV, alpha, turnover, or holdings aggregation.
3. **Existing gates remain authoritative.** Rotation cannot bypass owner auth, app pause, per-market pause, kill switch, drawdown breaker, market controls, account allowlists, live autonomy level, approval-required live mode, name/sector/gross caps, quality gates, correlation gates, volatility gates, re-entry cooldown, or position-count caps.
4. **PositionMonitor exits win.** If a holding would exit today for stop, target, time-stop, score-below-exit, direction flip, thesis break, or partial-profit workflow, rotation must not treat that holding as a discretionary funding source.
5. **Paper execution must be atomic.** A paper rotation is one transaction-like sell-and-buy operation. The system must not sell W and then leave the book idle because the buy failed.
6. **Live execution is two-leg and reconciled.** The live path creates an approval-required rotation proposal. The sell leg must confirm before the buy leg is allowed. Partial fills or unknown status stop the workflow in `needs_reconcile`; no automatic buy follows.
7. **Default off.** Shadow measurement is first. Paper auto-rotation and live proposals have separate flags and are both false by default.

Prioritized design issues found and required fixes

P0 - Sell-then-buy can create unintended de-risking

Failure scenario: PaperTrader sells the weakest holding, then the candidate buy fails because price moved, cap checks changed, the fill RPC rejects, or the daily notional cap is exhausted. The portfolio is now unintentionally in cash even though the rotation did not complete.

Fix: P1 paper execution must be a single atomic operation, preferably a Supabase RPC that validates the same snapshot, inserts both trade legs, updates/deletes the sold position, inserts the new position, adjusts `paper_portfolio.cash`, writes `rotation_events`, and aborts all writes on any failed invariant. If the existing paper fill RPC cannot support this, add a dedicated `execute_paper_rotation` RPC before enabling paper auto-rotation.

P0 - Live rotation cannot be a one-click opaque sell-and-buy

Failure scenario: The app submits a live sell, assumes proceeds are available, then submits a buy while the sell is partially filled, rejected, delayed, or in unknown status. This can overspend, bypass budget truth, or buy before the account is reconciled.

Fix: Live rotation emits a `trade_proposals` rotation record with explicit sell and buy legs, `approval_required`, market/account/currency, idempotency key, and full reason JSON. Approval submits only the sell leg through the canonical execution gateway. The buy leg is eligible only after broker-confirmed sell fill and refreshed account/budget data. Any partial fill, unknown state, broker error, or stale account snapshot moves the proposal to `needs_reconcile` and blocks the buy.

P0 - Rotation can fight PositionMonitor

Failure scenario: A holding is near a stop or has a score-below-exit signal today, but rotation sells it as "weakest" and records the action as an opportunity-cost rotation. That hides the true exit reason and corrupts learning labels.

Fix: Rotation source eligibility must run after PositionMonitor predicates or reuse a shared deterministic evaluateExitEligibility helper. Holdings with any current exit, partial-profit, near-stop, near-target, stale quote, stale score, or pending order state are excluded. If a holding has an exit reason, PositionMonitor owns the exit and the learning label.

P0 - Entry gates must be replayed after the hypothetical swap

Failure scenario: Candidate C passed standalone checks before the sell, but after replacing W with C the book breaches sector exposure, symbol/name cap, gross exposure, correlation, volatility, max positions, or daily notional limits.

Fix: The evaluator must apply a hypothetical portfolio state: remove W, add C at proposed size, then run the same portfolio-construction gates used by paper/live entry. Rotation is allowed only if the post-rotation book passes every gate.

P1 - Score-only edge is too noisy for auto-rotation

Failure scenario: A one-day score fluctuation makes C beat W by the margin, causing a buy-then-rotate-out whipsaw. This is the same risk class as a phantom weekly drawdown cascade, but now it sells holdings.

Fix: Paper auto-rotation requires persistence: C must beat W by the configured margin in at least two consecutive eligible research runs, or the latest run plus a prior run inside a configured freshness window. Shadow mode may log single-run opportunities, but auto paper cannot act on them. Live proposals require persistence and confidence-qualified benchmark-alpha data when the alpha term is enabled.

P1 - Cost/tax hurdle is underspecified

Failure scenario: edge_value is not mapped to dollars/rupees, so a taxable gain or spread/slippage cost can be smaller than a score gap but larger than the actual expected economic benefit.

Fix: Define expected value deterministically:

```
expected_edge_value =  
    target_notional *  
    clamp(expected_excess_return_C - expected_excess_return_W, -cap, cap) *  
    confidence_discount
```

Costs subtract estimated sell spread, buy spread, commissions/fees if any, market-impact/slippage buffer, and tax drag. For paper, tax drag is modeled and logged. For live, tax drag must use lot-level cost basis when available. If cost basis or holding period is unavailable and tax_sensitivity is medium/high, fail closed.

P1 - Turnover budget needs a ledger definition

Failure scenario: The system treats only the sell notional as turnover in one place and sell+buy in another, allowing more churn than the mandate intended.

Fix: Define rotation turnover as sell_notional + buy_notional, measured against same-market book NAV for the current calendar month. Budget consumption is checked before execution and logged to rotation_events. If investment_mandates.turnover_budget_monthly is null, rotation budget is zero unless an explicit rotation override exists.

P1 - Benchmark-alpha dependency can silently degrade

Failure scenario: Benchmark-alpha is unavailable, so rotation falls back to score-only without changing risk level. Live proposals then optimize on a weaker edge measure than expected.

Fix: P0/P1 shadow may write alpha_status = unavailable and use score-only as measurement. Paper auto-rotation may run score-only only if rotation_allow_score_only_paper = true and the edge margin is widened. Live rotation proposals

cannot use score-only fallback; they require confidence-qualified primary-benchmark alpha or remain blocked.

P1 - Correlation and duplicate exposure need post-swap evaluation

Failure scenario: W is sold and C is bought, but C is highly correlated with another retained holding, creating self-competition and no real portfolio improvement.

Fix: Correlation checks must compare C against the whole post-swap portfolio, not only W. Reject same issuer, duplicate ETF exposure, blocked symbols, leveraged/inverse products, and high-correlation substitutes unless an explicit allowlist exists.

P2 - Audit trail needs lifecycle and idempotency

Failure scenario: A rotation decision is logged after execution with a partial reason. A retry or cron duplicate creates duplicate sell attempts or impossible-to-reconstruct learning labels.

Fix: rotation_events is append-only and lifecycle-based: evaluated, rejected, planned, paper_executed, live_sell_submitted, live_sell_confirmed, live_buy_submitted, completed, aborted, needs_reconcile. Every row includes idempotency key, market, currency, book/account, candidate, source holding, scores, alpha inputs/status, cost/tax model, turnover budget, gate outcomes, snapshot timestamps, and actor.

Target architecture

Components

CapitalRotationEvaluator lives in lib/trading/capital-rotation.ts and is pure/deterministic. It returns a plan, not side effects.

```
type RotationDecision =
  | { action: 'reject'; reason: string; audit: RotationAudit }
  | { action: 'shadow'; plan: RotationPlan; audit: RotationAudit }
  | { action: 'paper_execute'; plan: RotationPlan; audit: RotationAudit }
  | { action: 'live_propose'; plan: RotationPlan; audit: RotationAudit };
```

Call sites:

- app/api/agents/paper-trade/route.ts: after C has passed every normal entry gate except cash, call evaluator at the insufficient_cash branch.
- app/api/agents/trader/route.ts: proposal generation only. It may produce a rotation proposal, never submit an order directly.
- PositionMonitor is not a call site for discretionary rotation. It owns exits and may expose reusable exit predicates.

Evaluator inputs

- Market-local candidate signal C: symbol, market, currency, score, direction, confidence, horizon, expected alpha if available, quote, target notional.
- Market-local open positions with fresh quote, fresh score, stop/target state, holding age, lot/cost basis if available, pending order state.
- Same-market book state: cash, NAV, exposure by name/sector, position count, monthly turnover used, daily notional used.
- Mandate/config: turnover_budget_monthly, min_holding_days, max_holding_days, tax_sensitivity, max_position_pct, score threshold, exit threshold, rotation flags.
- Safety state: app pause, market controls, kill switch, drawdown breaker, live autonomy mode, account allowlist, approval-required state.

- Benchmark-alpha state: primary benchmark excess return, confidence, status, window metadata.

Source holding eligibility

A holding W is sellable only if all are true:

- Same market and currency as candidate C.
- Still held, no pending order, no stale quote/score, no account/book mismatch.
- Not below exit threshold, not direction-flipped, not stop/target/time-stop/partial-profit eligible today.
- Not near stop or near target inside configured buffers.
- Holding age is at least `min_holding_days`.
- Selling W would not violate tax/cost guardrails.
- Selling W plus buying C improves the post-swap portfolio after all gates replay.

Decision rule

Rotation is allowed only if all are true:

1. C is eligible and blocked only by cash.
2. W is the weakest sellable holding by fresh deterministic score, with alpha tie-break when available.
3. $\text{score}(C) - \text{score}(W) \geq \text{rotation_margin}$.
4. Edge persists across the configured run count/window.
5. Expected edge value after confidence discount is positive after all costs and tax drag.
6. Monthly turnover budget remains available after `sell_notional + buy_notional`.
7. Post-swap book passes every entry and portfolio gate.
8. C is not a duplicate/correlated substitute for retained holdings.
9. Per-run and per-day rotation caps are not exhausted.
10. The feature flag for the current phase/action is enabled.

Data model

rotation_events

Append-only audit ledger. No UPDATE or DELETE except service migration repair.

Required columns:

- `id`, `created_at`, `owner_user_id`
- `market`, `currency`, `book_type` (paper or live), `account_id` nullable for paper
- `idempotency_key` unique per candidate/run/book
- `status`
- `candidate_symbol`, `source_symbol`
- `candidate_signal_id`, `source_position_id`

- candidate_score, source_score, score_edge
- benchmark_id, alpha_status, candidate_alpha, source_alpha, alpha_edge
- sell_notional, buy_notional, turnover_consumed
- cost_model_json, tax_model_json, gate_results_json, audit_json
- trade_proposal_id nullable, paper_trade_ids nullable

RLS: owner read only; service role writes. Public, anon, and authenticated direct writes revoked.

Config

Prefer a small rotation_config table keyed by owner/market/book type, or explicit strategy_config columns if the codebase already centralizes strategy flags there. Required flags:

- rotation_shadow_enabled default true for measurement only once built.
- rotation_paper_execute_enabled default false.
- rotation_live_proposals_enabled default false.
- rotation_allow_score_only_paper default false.
- rotation_margin_score, rotation_persistence_runs, rotation_cooldown_days, max_rotations_per_run, max_rotations_per_day.

Implementation must not overload global trading_enabled; rotation has its own flags in addition to all existing money-path flags.

Phasing

P0 - Shadow-only measurement

Built 2026-07-13. The evaluator and append-only audit rows are implemented. On insufficient_cash, PaperTrader records whether a rotation would have been eligible and why. No sell, no buy, no proposal. This validates frequency, churn pressure, score persistence, priceability, and missing tax/lot data.

P0 readiness completion (2026-07-22): New shadow rows reuse the canonical paper exit-plan projection so a holding already due for a time, score, stop, or target exit cannot be mislabeled as a rotation source. They also record distinct-run persistence, current-month paper turnover, mandate budget/tax sensitivity, the existing 5 bps-per-leg paper slippage floor, post-swap constructor replay, and measured candidate-to-remaining-book correlation from frozen symbol_daily_returns. Missing or truncated evidence fails closed and is recorded in p1_blockers; no value is guessed as zero. Agents -> Rotation exposes these blockers per market. This is measurement and visibility only: it does not grant P1 permission.

P1 - Paper auto-rotation, still default off

Owner approval, 2026-09-18: build the missing evidence and accounting

contracts for this phase. The execution flag remains default-off and may be enabled per market only after the new common-window, net-of-cost gate reports a passing result. This approval does not authorize live rotation.

Implemented P1 evidence contract (2026-09-18): every cash-constrained candidate now records (a) a 20% owner-approved monthly gross turnover ceiling, (b) a traceable single paper cost-basis lot rather than a merged/add-to-position lot, (c) a matched candidate-versus-holding forward-return score-edge estimate collapsed to non-overlapping sessions with a positive lower confidence bound, (d) post-swap portfolio-constructor admissibility, and (e) complete candidate correlation coverage. A failed or missing contract blocks execution before the book or atomic RPC is touched. `ready_for_review` is evidence readiness only; it is not a forecast of portfolio or benchmark outperformance.

After P0 evidence is reviewed, enable market-by-market paper execution only. Requires atomic paper RPC, persistence, post-swap gate replay, turnover budget, cost/tax model, and complete audit rows.

Current enforcement (2026-07-22): P1 is not approved. A production audit found the US paper flag enabled while several required gates above were still absent. It was reset to false, and migration 20260722185000 adds a database constraint that prevents either market from enabling paper rotation. The old money-moving RPC is replaced by a claim-verifying stub that always returns `p1_guardrails_incomplete` and performs no ledger mutation. P0 shadow measurement is the only reachable phase until a later migration removes that constraint and introduces a fully reviewed P1 transaction.

Containment correction (2026-08-10): Migration 20260723120000 later removed that containment and enabled both paper rows, but the caller only rechecked score spread, persistence, cooldown and count caps. It did not enforce the P1 readiness result and did not read `rotation_allow_score_only_paper`. Four paper rotations executed; their small realized cohort was negative in both markets. Migration 20260811033335 disables paper execution again, and the caller now refuses score-only execution when the default-false flag is not explicitly enabled. P0 measurement remains active. This is containment, not a claim that the four-trade result proves rotation lacks edge.

P2 - Live approval proposals

After P1 shows low churn and net-positive behavior, add live proposal generation. Owner approval is required. Sell and buy legs are reconciled separately through the canonical gateway. No autonomous live rotation in this phase.

P3 - Optional owner-approved automation

Out of scope for this architecture. Would need a separate architecture review, broker canary evidence, live reconciliation evidence, and explicit owner approval.

What is sound in the original design

- The core product idea is useful: fully invested books need an opportunity-cost path, otherwise the system holds mediocre but not-yet-exit positions while better candidates are skipped.
- Keeping rotation deterministic and per-market is correct.
- Using mandate fields such as turnover budget, min holding days, tax sensitivity, and max position percent is the right fit.
- Putting live behind approval-required proposals is correct.
- Using benchmark-alpha as an edge tie-break is appropriate once benchmark-alpha has confidence-qualified data.

Recommended build order

1. [Done] Add P0 shadow evaluator and `rotation_events` audit table.
2. [Done] Reuse the canonical paper exit-plan projection so rotation cannot relabel true exits.
3. [Done for P0 measurement] Replay the post-swap paper constructor and measured candidate correlation; live remains out of scope.
4. [Done for P0 measurement] Add persistence, turnover, friction, tax-evidence, and truncation tests.
5. [Blocked] Validate a score-to-forward-return mapping, configure an owner-approved turnover budget, and collect independent-session evidence before designing a new atomic P1 RPC.
6. [Done] Add owner-visible market-local rotation readiness reporting.
7. Add live proposal generation only after P1 paper evidence is reviewed.

Single riskiest assumption to validate first

The riskiest assumption is that the current deterministic score plus benchmark-alpha has enough signal stability to justify selling a still-valid holding. Validate this in P0 shadow mode before allowing any paper or live rotation.

Explicit non-goals

- No autonomous live sell-to-buy workflow.
- No rotation across US and India.
- No score-only live fallback.
- No new standalone reallocation agent that initiates orders independently.
- No LLM override or LLM-generated tax/cost/edge math.
- No implementation code until this architecture is approved.

Source: features\catalyst-scout\FEATURE_ARCHITECTURE.md

CatalystScout — Feature Architecture

Status: Stage 1 LIVE (measure-only shadow) . No gate, no sizing, no order-path change.
Author: Claude (Sonnet 5), 2026-09-18. Approved by Vaibhav 2026-09-18 (item #1 of the
achaljhawar/1rok gap-analysis plan — see PROJECT_DECISIONS.md Decision 79's fast-follow list).
Sibling document: features/risk-tier-gate/FEATURE_ARCHITECTURE.md (same session, same discipline).

1. The gap this closes

Kairos's earnings handling has always been **defensive only**: the $\pm 2/5$ day earnings blackout in `app/api/agents/paper-trade/route.ts` exists to avoid trading INTO an earnings print, and analyst target upside is explicitly `observational_only` (0 points) in `scoreFundamentals`. Nothing in this codebase scores catalyst **upside** — an asymmetric near-term setup where the evidence already points one direction before the print. achaljhawar/1rok's Catalyst agent's prompt discloses a clean, adaptable scoring shape for exactly this (base 50, components earnings-setup/expectation-asymmetry/non-earnings-catalyst/insider-signal/event-risk-penalty) — their agent is LLM-judged; this document adapts the disclosed **shape**, not their implementation, into a deterministic Kairos component.

2. Inputs — all already in scope, zero new fetches

Every input `computeCatalystScout` (`lib/scoring/catalyst-scout.ts`) needs is a variable already computed by the time `research-agent.ts` reaches the `decision_observations` insert:

Input	Existing source	Was it already fetched?
<code>daysToEarnings</code>	<code>earnings-calendar</code> batch read	Yes — same value the earnings blackout uses
<code>analystConsensusScore</code>	<code>lib/data/analyst.ts</code> <code>scoreAnalyst()</code> (Finnhub)	Yes — already fetched, currently only LOGGED (<code>decision_observations.features.analyst</code>), never scored, per that call site's own comment: "captured as LOGGED EVIDENCE... NOT fed into the live weighted score yet"
<code>insiderScore / insiderAvailable</code>	<code>scores.insider_score</code> , <code>included.insider</code>	Yes — the live insider dimension
<code>capitalFlow5d</code>	<code>webullExtended.capitalFlow.largNet5d</code>	Yes — already fetched Webull smart-money flow, already written to <code>decision_observations.features.capital_flow_5d</code>
<code>breakdownVetoed</code>	<code>scores.evidence.technical.breakdown_veto.vetoed</code>	Yes — the existing crash/meme-reversal veto

No new provider, no LLM call, no schema dependency beyond the existing `decision_observations.features.jsonb` column.

3. The five components (adapted from 1rok's disclosed ranges)

Component	Range	Logic
Earnings setup	-20 to +25	Only scored when earnings are within 10 days; direction/magnitude from analyst-consensus skew. Known-but-distant earnings contribute exactly 0 (available evidence, no near-term setup) — distinct from unknown (null, excluded)
Expectation asymmetry	-15 to +20	Analyst-consensus skew from neutral (50), independent of earnings timing — a standing asymmetry read, not earnings-gated
Non-earnings catalyst	-10 to +15	Sign of Webull 5-day capital-flow net (this pass: sign only, not magnitude-scaled — see §5)
Insider signal	-10 to +10	insider_score skew from neutral, gated on the SAME insiderAvailable flag the live composite uses (never reads the default-fill 50 as if it were evidence)
Event risk penalty	-25 to 0, always computed	Earnings within 5 days: -15. Active breakdown veto: -10. Both together caps at -25 and forces classification = "high_risk_binary_event" regardless of the setup direction

`totalScore = clamp(50 + sum(available components) + eventRiskPenalty, 0, 100).`

4. Same stated asymmetry as risk-tier

The four evidence-gated components contribute zero, not a neutral default, when unavailable — coverage (0-4) travels with every row for the same reason risk-tier's does. The event-risk penalty is different in kind: it is a rule (not an evidence read) that defaults to 0 when its trigger conditions aren't met, which is a defensible "no known reason to flag extra risk," not a fabricated claim — so it is not counted in coverage and always contributes.

5. What was deliberately not built (yet)

- **Corporate-actions non-earnings catalysts** (M&A, spinoffs, buybacks — the `corporate_actions` table already exists per `system-map.json`) are NOT wired into `nonEarningsCatalyst` this pass; only Webull capital-flow sign is. A real event-type read is a natural fast-follow.
- **Capital-flow magnitude scaling** — this pass uses sign only (+8/-6 fixed), not a magnitude-scaled contribution. Needs a defensible normalization (vs. float, vs. ADV) before scaling by size.
- **Binary vs. non-binary event classification** (1rok's own distinction: FDA/M&A rulings vs. product launches/investor days) is not modeled — Kairos has no data source for non-earnings binary events yet.
- Any consumption of `catalyst_shadow` by scoring, sizing, or entry gating — a separate, evidence-gated promotion decision, same as risk-tier-gate.

6. Acceptance criteria

1. Zero change to `analyst_score`, direction, sizing, entry/exit gates, or any order path.
2. No new provider call, no new fetch — every input already existed in this function's scope.

3. Fail-soft: wrapped in its own try/catch; a thrown error never blocks the research decision it's attached to.
4. coverage always present so a low or high totalScore can be read against how much real evidence it rests on.

Source: features\correlation-aware-construction \FEATURE_ARCHITECTURE.md

Correlation-Aware Portfolio Construction — FEATURE ARCHITECTURE

Status: Reviewed / measurement prerequisite required / P0-P1 activation not approved. Design only.
Last updated: 2026-07-16.
Per-market applicability: both — correlation gating/sizing is strictly per-market (US/USD and India/INR pools are separate; never cross-summed). Cross-market correlation is display-only awareness, never a gate .
Sequencing: folds in after the in-flight Codex paper-autonomy fixes (they own paper - trade/position-monitor right now) and alongside the holdings-priority research fix.
Update this file when: the risk inputs, estimator, selection rule, or the gating/display boundary change.

0. 2026-07-16 adversarial correction (authoritative over the earlier draft)

The earlier draft overstates what is reusable from Holding Risk. Do **not** wire P0/P1 as originally written:

- Holding Risk computes pairwise correlations in memory only among **already-held** names. It persists per-name cluster summaries, not the pairwise matrix needed to estimate a new candidate's marginal risk.
- A new candidate is not part of that Holding Risk run, so there is no measured candidate-to-book correlation to consume at entry.
- The persisted per-holding beta is currently derived from sector beta in the account enrichment path; it is not a uniformly measured symbol-to-benchmark beta contract. India can replace the account-level aggregate beta, but that does not make every persisted holding beta measured.
- PaperTrader already estimates candidate daily volatility, but the India implementation fetches candles synchronously on the entry path and the US path depends on whatever history exists in price_cache. This does not satisfy the proposed no-provider-call, point-in-time evidence contract.
- P1 intentionally reorders already-eligible candidates. Therefore “no previously rejected name becomes eligible” can apply only to **safety-gate rejection**, not selection rank; otherwise P1's stated purpose is impossible.

Required prerequisite before activation

1. During ResearchAgent's existing candle fetch, append a compact, immutable per-symbol return observation with market, symbol, as_of, available_at, provider/source, return dates, daily vol, benchmark beta when measurable, overlap count, and input fingerprint. Do not make another provider call for this record.

2. Build candidate-to-held pair estimates from observations where `available_at <= decision_at`, with at least 60 shared sessions, deterministic shrinkage, and sector-proxy fallback explicitly labeled `low_confidence`.
3. Run the measured penalty in **shadow only** and persist baseline size/rank, shadow size/rank, evidence IDs, fallback reason, and every eligibility flip.
4. Activate P0 only after a frozen-cohort test proves no order is larger and no baseline safety rejection becomes eligible. Activate P1 separately after shadow evidence shows the reordering improves diversification without weakening score, mandate, liquidity, capacity, or execution gates.

Until those prerequisites exist, the production constructor remains on its current conservative volatility/sector-proxy behavior. Faking candidate correlations from held-name cluster averages is prohibited.

Implementation notes — §0 item 1 SHIPPED (2026-07-16), items 2-4 NOT started

Item 1 (the per-symbol return-observation contract) is built. **It activates nothing**. Items 2, 3 and 4 remain open; the constructor is untouched and still runs on `beta: null, dailyVol: null` + the sector-proxy constants.

Piece	Where
Observation builder + capture (pure, deterministic, no LLM)	<code>lib/data/return-observations.ts</code>
Shared run-level benchmark series	<code>lib/data/benchmark-series.ts</code>
Capture hook (concurrent, fail-soft, joined before return)	<code>lib/research-agent.ts</code> → <code>processSymbol</code> , immediately after the existing <code>candle Promise.all</code>
Tables (append-only, owner-read RLS)	<code>symbol_return_observations</code> summaries + <code>symbol_daily_returns</code> frozen per-session series
Coverage report (owner-gated)	GET <code>/api/analytics/return-observation-coverage?minShared=60</code>
Unit tests	<code>lib/data/return-observations.test.ts</code>

Hook point. The capture piggybacks on the candles `processSymbol` already fetches for scoring — it makes **no provider call of its own**. The candle `*source*` (previously discarded by `.then(r => r.candles)`) is now carried alongside the bars so the observation records its provenance. The benchmark series was **hoisted, not added**: `getRegimeFeatures` previously fetched `SPY/^\NSEI` inline; both it and the capture now share one promise-deduped 30-min cache, so the run still makes exactly one benchmark load per market. US benchmark closes come from `price_cache` (a DB read, not a provider call).

Added per-run cost. Provider calls: **+0**. The same already-fetched candles produce one summary row plus idempotent per-session rows. Unchanged sessions deduplicate by `symbol/market/date/source/basis/input` fingerprint; provider revisions append instead of rewriting history.

PIT enforcement is doubled up. The builder drops any bar dated after `available_at` before computing anything (symbol `*and*` benchmark series), and the table carries check `(as_of <= (available_at at time zone 'utc')::date)` so a lookahead row is not storable even if the builder were bypassed.

Beta honesty. `benchmark_beta` is written **only** when genuinely measured vs the market's own benchmark (`us→SPY, india→^\NSEI`) over ≥ 60 shared sessions with non-zero benchmark variance. Otherwise it is `null` and `beta_unmeasurable_reason` records why. A DB check constraint makes `beta` and the reason mutually exclusive, so **a sector proxy cannot be written into the beta column** — the failure mode §0 called out.

Per-session series (Option A). `symbol_daily_returns` stores each frozen simple return with both session dates, source, `available_at`, explicit `adjusted_close/raw_close` basis, closes, and an immutable input fingerprint. Item 2 can align exact shared sessions point-in-time without refetching candles. Revisions remain separately auditable; an estimator must select the latest row available at its decision cutoff and must not compare incompatible price bases silently.

1. Verified current state (not assumed — read from code)

The machinery is ~80% built and **fed fake inputs**:

Piece	Reality
Sector cap (max_positions_per_sector, default 3, market-local)	Enforced ■
Portfolio Constructor (lib/portfolio/constructor.ts) — name/sector/gross/vol/correlation rules at entry	Runs , and is correctly subtractive ("never increases a size, never force-sells the book") ■
beta, dailyVol inputs	paper-trade passes beta: null, dailyVol: null → constructor falls back to a flat 0.02 (2%/day) vol for every name ■
Correlation used in the vol budget + Rule-5 haircut	Proxied from sector : corr = sameSector ? CORR_SAME_SECTOR : CORR_CROSS_SECTOR — two constants, not measured co-movement ■
max_avg_pairwise_corr (0.7)	Explicitly "informational bound; not solved for directly" ■
Real measured beta + aligned-return correlation	Computed daily by holding-risk (lib/risk/holding-risk.ts, lib/risk/correlation.ts → computeCorrelationClusters) — but powers the Risk Analytics display only , never the entry decision ■

Consequence: two instruments in different nominal "sectors" that move identically (e.g. a semi ETF and a semi name; two USD-revenue India IT names) pass the entry gate as if diversifying. The knobs exist; the numbers exist; **the wiring between them does not**.

2. Why this matters more than raising the position cap

The research budget already caps position count (a US run scores ~30 of ~102; 72 deferred). Once holdings are re-scored daily, 10/market ≈ 20 holdings ≈ most of a run's capacity, leaving ~10 discovery slots. **15-20/market would starve discovery or holdings-freshness**.

So the lever is **quality of the 10, not quantity**: 10 genuinely uncorrelated names beat 15 correlated ones. **Do this before revisiting the cap**.

3. Non-negotiable boundaries

- **Deterministic** — no LLM anywhere in estimation, selection, or sizing.
- **Subtractive only** — preserve the constructor's invariant: rules may only **shrink** a size or reject a new entry. Never increase a size, **never force-sell an existing position** because a correlation estimate moved.
- **Per-market gating only**. Correlation/vol constraints are computed and applied **within** a market's pool. Cross-market correlation is **informational** and must never size, gate, or reject.
- **Point-in-time**. Correlation/beta/vol must be computed from returns **available at the decision timestamp**. No lookahead; a decision cannot use a correlation estimated with later data.
- **Fail-safe, not fail-open**. Missing/insufficient risk inputs must degrade to the **current conservative behavior** (sector proxy), never to "no constraint".
- **No new provider calls on the entry path**. Reuse the existing daily holding-risk computation + price_cache; entry must not burst providers.

4. Statistical rigor (why the naive version is wrong)

At 10-24 names with limited history, **sample correlation matrices are notoriously unstable**. Naive implementation will reject good trades on noise.

1. **Shrinkage** (Ledoit-Wolf style) toward a structured target (sector-average or identity) — never raw sample correlation.

2. **Minimum overlapping history** (~60 sessions) per pair; below it, **fall back to the sector proxy** and label the estimate `low_confidence`.

3. **Penalty, not hard gate**. Correlation reduces a candidate's effective attractiveness/size; it does **not** hard-reject at $\text{corr} > 0.7$. A hard threshold on a noisy estimate is a coin-flip rejector.

4. **Beta $\neq$ correlation — keep both, they answer different questions:**

- **Beta (to market benchmark)** → *how much market exposure does the book carry?* (10 names all at $\beta 1.5$ = a $1.5\times$ levered market bet, even if uncorrelated to each other.)

- **Pairwise correlation / clusters** → *how redundant are these names with each other?*

Both are needed; conflating them hides one of the two risks.

5. **Estimate provenance**: every risk input stored with `as_of`, `source`, and `confidence` so a decision can be audited and a low-confidence input can be treated conservatively.

5. Phases

P0 — Wire the real inputs (highest value, cheapest)

Feed measured beta / dailyVol / pairwise correlation — **from the existing daily holding-risk computation** — into the constructor at entry, replacing `beta: null`, `dailyVol: null` and the sector-proxy constants. Apply shrinkage + min-history fallback to the sector proxy. **Turns existing dead knobs live with no new data source and no new provider calls.**

P1 — Correlation-penalized selection

Rank candidates by **score per unit of *marginal* risk added to the current book**, not raw score. Greedy: for each candidate, compute its marginal contribution to book variance given current holdings; penalize accordingly.

This is the owner's stated intent: *a 70-score name uncorrelated to the book should be able to beat an 85-score name that is a clone of what we already own* — and should be able to displace it as a second/third choice.

Deterministic + auditable: persist the penalty and the marginal-risk term in the decision rationale so "why did it pick #4 over #2" is answerable.

P2 — "These move together" view (owner-facing)

Surface correlation **clusters** (reuse `computeCorrelationClusters`) on the Risk/Portfolio surface: which stocks/ETFs actually co-move, with coverage + confidence labels. Display-only.

P3 — Cross-market awareness panel (display-only, never gates)

Show US↔India co-movement at the **owner/total** level — the blind spot per-market views structurally cannot see (e.g. India IT is largely USD/US-client revenue → correlates with US tech; a US tech drawdown can hit **both** pools at once).

Hard requirement: NSE closes before NYSE opens — **naive same-day cross-market correlation is an artifact**. Must use **lag-aligned** returns (India day T vs US day T-1, or a documented overlapping-window convention) and treat **INR/USD as its own factor**. Label explicitly: *informational; not used for sizing or eligibility.*

6. Acceptance / rollback

- P0 must be a **no-op-or-safer** change: with real inputs the constructor may only shrink or reject relative to today's behavior on the same cohort; prove on a frozen cohort that no position gets **larger** and no previously-rejected name becomes eligible.
- Missing inputs ⇒ identical behavior to today (sector proxy).
- Rollback = a config flag reverting to the sector-proxy constants; no schema loss.

- Tests: shrinkage estimator determinism; min-history fallback; PIT correctness (no lookahead); subtractive invariant (never upsizes / never force-sells); per-market isolation (US inputs never enter India's book).

7. Open decisions (owner / Codex)

1. **Estimator:** Ledoit-Wolf shrinkage vs simpler constant-correlation shrinkage? (Recommend constant-correlation target first — fewer moving parts at $N \approx 10-24$.)
2. **Penalty strength:** how hard should correlation discount score in P1? Must be a **config value**, not hardcoded, and ideally learnable later — but fixed until the learner has a validated edge.
3. **Beta budget:** should the book carry an explicit **portfolio-beta cap** (distinct from vol/correlation), so 10 uncorrelated-but-all-high-beta names still get constrained? (Recommend yes, config, default off until measured.)
4. **Cross-market lag convention:** India T vs US T-1, or overlapping-window? Must be fixed + documented before P3 renders a number.
5. **Do NOT raise the position cap until P0+P1 ship** — confirm this sequencing.

Source: features\cross-sectional-rank\FEATURE_ARCHITECTURE.md

Cross-Sectional Ranking — Feature Architecture

STATUS: IMPLEMENTED (P0–P2), OFF by default (entry.rank_pct_min = 0.0). Migration 151 applied 2026-07-11.

Last updated: 2026-07-11

Author: Claude (P0–P2 shipped: lib/scoring/rank.ts, cron Pass 2, genome param, migration 151. P3–P4 validation-replay/learner-proposal still pending.)

Open flag RESOLVED (2026-07-11) — "top-3 by analyst_score win" (§9 borderline item): the rank gate governs **cross-group admission only**; final ordering of admitted candidates stays by `analyst_score` (`PaperTrader.order("analyst_score", desc)` unchanged). Within a comparable group `rank_pct` is monotonic in `analyst_score`, so intra-group top ordering is byte-identical, and under today's single-/small-group degraded case (the common case) rank selection and raw-score selection are identical. The divergence only appears once the universe is large enough for multiple valid sector groups — precisely when cross-sectional rank has value. This is behavior-neutral today because the feature ships OFF (`rank_pct_min` default 0.0) and can only activate via a validated, owner-promoted challenger. No conflict with the locked CLAUDE.md decision.

Scope: US equities, US ETFs, India NSE equities; long-only new positions; 2–20 trading-day swing horizon.

Parent doctrine: features/scoring-methodology/FEATURE_ARCHITECTURE.md §5 ("Universe and comparable groups") and §3 ("comparable-universe rank + uncertainty"). This document is the concrete, buildable slice of that named P1 gap.

1. Problem statement (the named P1 gap)

Today a candidate becomes a Long entry **only** through an **absolute** per-stock gate:

- lib/research-agent.ts (processSymbol, ~line 1180):

```
signalDirection = analystScore >= (scoreThreshold ?? 60) ? "long" : "neutral";
```

scoreThreshold comes from the champion genome (genome.entry.score_threshold, bounded [50,75]) → strategy_config.score_threshold → min_analyst_score → 60.

- app/api/agents/paper-trade/route.ts (~line 150) re-applies the **same** absolute floor:

```
.eq("status","pending").eq("direction","long")
.gte("analyst_score", scoreThreshold)
.order("analyst_score", { ascending: false }).limit(10)
```

An absolute 0–100 composite is a **calibration-fragile** entry rule. analyst_score is a renormalized weighted sum (lib/scoring/weighted-score.ts), not a probability, so a fixed cutoff of 60 silently means "trade more" on strong tapes and "trade nothing" on weak tapes even when the *relative* best names are unchanged. It cannot answer the question a swing book actually needs: **"Is this the strongest candidate available in its comparable group today?"**

What already exists (do NOT rebuild)

The **measure-only** half of cross-sectional rank is already shipped:

- **Migration 137** (supabase/migrations/137_universe_snapshots.sql) created universe_snapshots and universe_snapshot_scores(universe_snapshot_id, symbol, analyst_score, rank_pct, decision_observation_id).

- **Migration 136** added decision_observations.universe_snapshot_id.

- The research cron (app/api/agents/research/cron/route.ts, ~lines 152–215) already:

1. inserts one universe_snapshots row per run (source: "mixed"),
2. after all symbols score, computes rank_pct = (# symbols scoring below s) / (n-1) over the whole run, and
3. writes universe_snapshot_scores.

- processSymbol accepts universeSnapshotId and stamps it on decision_observations.

Gaps this feature closes:

1. The existing rank_pct is **never consumed** by any selection path — it is logged and forgotten.
2. It is computed over a **mixed universe** (US equities + ETFs + metals + region ETFs + India names all pooled), violating scoring-methodology §5 ("never rank ETFs against companies", per-market/sector comparable groups, minimum sample sizes). It also ranks **all** scored symbols including thin-evidence / vetoed ones, violating "rank composites **after** data-quality gates".
3. Rank is computed **after** the per-symbol entry decision is already written, so it structurally cannot gate entry today.

2. Design goals & non-negotiables

Goal: make cross-sectional rank an **actionable, correct** input to the new-entry decision — layered on top of (not instead of) a conservative absolute floor — while leaving sizing, execution, and the SELL-for-holdings path untouched.

Respect these locked decisions (from CLAUDE.md and scoring-methodology §2):

- Rank is a **scoring refinement**, not a regime switch. **No explicit bull/bear mode** is introduced (§3).
- **Dual-bucket momentum/value** screener is unchanged; rank sits downstream of scoring, not inside discovery (§3).
- **≤ 3 screener candidates/day**; rank can only *reduce* the actionable set, never expand it (§4).
- **Long-only for new positions**; the held-position exit branch (isExitSignal) is not touched (§4).
- An LLM never generates rank, score, direction, or eligibility (scoring-methodology §2).
- Comparable-universe transforms use a **recorded** reference universe for the same market/date; **the three daily finalists are not a valid reference universe** (scoring-methodology §5 / §2).

3. Where rank must live in the pipeline

Rank is intrinsically a **whole-universe** operation: a symbol's percentile is undefined until every peer in its comparable group has scored. The current pipeline decides direction/entry_eligible **per symbol, streaming**, before peers finish. Therefore rank cannot be a drop-in inside processSymbol's existing gate.

Chosen shape: a deterministic second pass in the cron, after scoring, before PaperTrader.

```
Pass 1 (unchanged): processSymbol × N → writes analyst_score, per-dim scores,
                    decision_observations, agent_signals(direction, status='pending')
                    Data-quality state already computed here:
                    thinEvidence, includedDims, evidence_confidence,
                    breakdown veto (baked into technical_score), abstain
```

```
Pass 2 (NEW, deterministic, no LLM): rankAndGate(runId, universeSnapshotId)
  a. Build the ELIGIBLE ranking pool = scored symbols that passed data-quality gates
  b. Partition pool into comparable groups (market × asset-type × sector/bucket)
  c. Compute rank_pct within each group (empirical percentile)
  d. Persist rank_pct + rank_quality to universe_snapshot_scores (replaces the
     current naive mixed-pool computation)
  e. Apply the rank+floor entry policy → update agent_signals.direction/status
     for NEW candidates only (holdings & exits untouched)
```

```
PaperTrader (minimally changed): consumes the gated signals as today
```

Why a second pass and not gate inside processSymbol:

- **decision_observations** is **append-only** (mutation-blocking trigger, per scoring-methodology §9) — we must not rewrite the PIT observation. Rank is logged as an *additional* universe_snapshot_scores row keyed to the observation, exactly what migration 137 was built for.
- **agent_signals** is mutable (PaperTrader already flips status there) — it is the correct surface to flip a rank-rejected candidate from direction='long' → neutral/status='rank_rejected'.

Two placement options for Pass 2, both acceptable; recommend **(A)** for auditability:

- **(A) Cron reconciliation (recommended)**. Extend the existing block in research/cron/route.ts (~line 193) that already computes rank_pct. It runs once, sees the whole universe, writes correct grouped ranks, then applies the gate to agent_signals before it chains PaperTrader (~line 249). Fully deterministic, easy to unit-test, one write site.
- **(B) PaperTrader join**. Leave agent_signals alone; have PaperTrader join universe_snapshot_scores and apply the rank gate at selection. Less invasive to research, but scatters the entry policy across two agents and re-derives the pool at fill time. Rejected as primary.

CLAUDE.md coupling note: `decision_observations.universe_snapshot_id` and the whole `universe_snapshot_scores` table must be confirmed present in the target DB before Pass 2 ships (they are migrations 136/137). Per the global "verify migration before merging" rule, a `list_migrations / information_schema` check is required at build time; the new columns in §7 add migration ~145.

4. Comparable groups & the eligible ranking pool

4.1 Data-quality gates run *BEFORE* ranking (the "rank composites after gates" requirement)

A symbol enters its comparable group's ranking pool **only if** all of the following hold (all already computed in Pass 1):

Gate	Source	Rejected symbols
Not thin evidence	<code>isThinEvidence(includedDims)</code> (<code>weighted-score.ts</code>) — ≥ 2 usable dims	excluded from pool, <code>direction=neutral</code> (already)
Not abstained	<code>signalDirection !== "neutral"</code> from Pass 1's mechanical gate	excluded
Breakdown veto not fired	already folded into <code>technical_score</code> cap 20 (<code>detectBreakdownVeto</code>)	stays in pool but scores low naturally
Evidence confidence floor	<code>decision_observations.evidence_confidence</code> $\geq$ <code>RANK_MIN_CONF</code> (default 0.60, mirrors scoring-methodology §7)	excluded from pool
Is a new candidate	<code>isHeld === false</code>	holdings never enter the new-entry rank pool

Excluded symbols still get a `universe_snapshot_scores` row with `rank_pct = null`, `rank_quality = 'excluded_<reason>` for auditability, but are **not** rank-eligible for entry. This is the operational meaning of "rank composites after data-quality gates".

4.2 Comparable groups (scoring-methodology §5, made concrete)

Partition the eligible pool by:

- **US equity / ADR:** `market='us' × sector` (`decision_observations.features.fundamental.sector`) when the sector has $\geq$ **RANK_MIN_GROUP_EQUITY (20)** eligible names that day; else fall back to `market='us' × asset-type=equity` (all US equities pooled).
- **US ETF:** `market='us' × ETF category` (broad / sector / thematic / leveraged-excluded). **Never** ranked against single companies. Given daily ETF counts are tiny (metals basket + region ETFs, typically < 5), ETFs almost always fall to the degraded path (§4.3).
- **India equity:** `market='india' × NSE sector` when $\geq$ **RANK_MIN_GROUP_INDIA (15)**; else `market='india' × large/mid liquidity bucket`.

Groups are keyed off fields already present in `decision_observations.features` and entry metadata — no new fetch. Sector for US comes from AV OVERVIEW. Sector (already stored in fundamental evidence); India sector from the screen cache / Yahoo mapper.

4.3 Small-group degraded path (the "three finalists are not a universe" guard)

If a comparable group has fewer than its minimum-sample threshold **eligible** names:

- Do **NOT** invent a percentile from 3–5 names.
- Mark `rank_quality = 'degraded'` and fall back to a **pre-registered fixed transform**: `rank_pct := clamp01((analyst_score - RANK_FLOOR_LO) / (RANK_FLOOR_HI - RANK_FLOOR_LO))` with fixed constants (e.g. 45 →

0, 80 → 1). This makes the rank gate collapse to a slightly-softened absolute gate exactly when there is no valid cross-section — the honest degenerate behavior.

- A degraded rank may drive **paper** selection but is flagged so the learner and any future live path can treat it as lower-quality evidence.

Because the realistic daily universe today is small (~6 screener + watchlist + holdings, one market per cron), **most runs will start on the degraded path**. That is acceptable and honest: the value of true cross-sectional rank grows as the PIT universe widens (scoring-methodology §5/§12). The architecture is correct now and improves automatically as universe breadth increases.

5. How rank composes with the genome threshold (the entry policy)

Hybrid gate = absolute floor AND relative rank. Both must pass for a NEW long entry:

```
entry_long ■ (analyst_score >= score_threshold)      # existing genome floor, unchanged
             AND (rank_pct >= rank_pct_min)           # NEW cross-sectional gate
             AND passed §4.1 data-quality gates
             AND not thin evidence
```

Rationale for **AND**, not replace:

- **Floor alone** (today) trades the weak-tape problem described in §1.
- **Rank alone** would trade "the best of a bad universe" on a day when *every** candidate is weak — a real failure mode the absolute floor exists to prevent. Keeping the floor means: on a day where the top-ranked name still can't clear an absolute conviction bar, we trade nothing. This is the correct humility prior (doctrine §3).
- **AND** = "trade only names that are both objectively decent (floor) **and** the relative best available (rank)." This is what a disciplined swing book does and it is the tightest, safest composition.

Genome extension

Add one evolvable parameter to the genome (strategy_versions.genome, documented in docs/arch/09-learning-loop.md):

Parameter	Range	Default	Notes
entry.rank_pct_min	0.0 – 1.0	0.0 (feature OFF)	Minimum within-group percentile for a NEW long. 0.0 = today's behavior exactly (rank never rejects). Champion promotion bounds it, e.g. [0.5, 0.9].

Backward-compatibility is exact: a champion with no rank_pct_min (or 0.0) reproduces current selection byte-for-byte. This makes the whole feature a **no-op until a challenger carrying rank_pct_min > 0 is validated and promoted** — satisfying "no scoring change affects live until owner promotion" (scoring-methodology §15) and the CLAUDE.md pushback mandate.

Top-3/day preservation

Rank is a **filter that only removes candidates**. It never lifts the screener's ≤3/day target, PaperTrader's limit, position caps, or cash constraints. When rank_pct_min > 0, the actionable set is a **subset** of today's absolute-floor survivors, so daily churn can only **decrease**. No conflict with the top-3 rule.

Ordering note (CLAUDE.md "top 3 by analyst_score win"): within a single comparable group, rank_pct is a monotonic transform of analyst_score — so the **ordering** of the top-3 within a group is identical to ordering by raw score. Rank changes **which** names clear the bar (relative-to-peers), not the intra-group ranking of those that do. PaperTrader's .order("analyst_score", desc) can stay. No locked-decision conflict.

6. Rank vs the calibrated P(win) model — feature, not gate (yet)

`lib/validation/calibration.ts` fits a logistic P(win) over the 5 dimension scores (DIMS) and gates the `pwin_logistic` artifact on walk-forward ECE. Two candidate roles for rank:

- **As a GATE (selection):** covered in §5 — this is the actionable use now, and it is deterministic and testable.
- **As a FEATURE (calibration/sizing):** `cross_sectional_rank_pct` is a genuinely different axis from raw dimension scores (it encodes *relative* strength, which the absolute dims cannot). It is a strong candidate to become a 6th feature in the P(win) logistic.

Recommendation: log rank as a feature now; do NOT feed it into the live P(win) model until it earns an IC track record. This mirrors exactly how analyst consensus and `days_to_earnings` are already handled in `processSymbol` (logged into `decision_observations.features`, graded by the learner, promoted through the IC-gated Feature Registry — never wired straight into live sizing). Concretely:

- Pass 2 writes `features.cross_sectional.rank_pct`, `rank_quality`, `comparable_group_key`, `group_n` into the observation's companion evidence (via `universe_snapshot_scores`, since `decision_observations` is append-only and already written in Pass 1).
- The learner's IC tooling (`lib/edges/ic.ts`) can then measure Spearman IC of `rank_pct` vs forward returns per market/setup/horizon.
- Only after rank clears the same OOF/IC bar the calibration §10 rules demand does adding it to DIMS (a `deterministic_v2` concern) get proposed. **Not in this feature's scope.**

This keeps the money path (sizing off calibrated P(win)) unchanged while rank proves itself.

7. Persistence & migrations (additive only)

Reuse migration 137 tables; add columns, do not create a parallel truth store (scoring-methodology §15).

Migration ~145 (verify next number at build time — repo is at 144):

```
-- universe_snapshot_scores: richer rank provenance (all nullable, additive)
alter table public.universe_snapshot_scores
  add column if not exists rank_quality      text,          -- 'ok' | 'degraded' | 'excluded_thin' | 'excluded_conf' | 'excluded_held'
  add column if not exists comparable_group_key text,        -- e.g. 'us:equity:technology'
  add column if not exists group_n          int,            -- eligible names in the group that day
  add column if not exists rank_eligible     boolean;        -- passed §4.1 gates → counted in the group

-- agent_signals: record why a candidate was rank-rejected (audit; PaperTrader already
-- reads status). No new features blob — canonical PIT stays in decision_observations.
alter table public.agent_signals
  add column if not exists rank_pct          numeric,
  add column if not exists rank_rejected     boolean default false;
```

- `decision_observations` gets **no** new columns — `universe_snapshot_id` (136) already links it; rank lives in the companion `universe_snapshot_scores` row (keyed by `decision_observation_id`). This respects the append-only invariant.
- New status value: `agent_signals.status = 'rank_rejected'` for candidates that cleared the floor but failed the rank gate (distinct from `expired/neutral` so the Research Journal can explain "scored well but wasn't the best available today").
- Range check `rank_pct ∈ [0,1]` already exists on `universe_snapshot_scores`; mirror as `NOT VALID` on `agent_signals` then validate.

8. Learning-loop / validation evaluation of a rank challenger (no regime logic)

A challenger that sets `rank_pct_min > 0` must be replayable by the Validation Engine on held-out folds **without** any regime branch.

- **Reference universe reconstruction:** the validation replay already reads `decision_observations` per PIT day. To evaluate the rank gate, it joins `universe_snapshot_scores` (or recomputes grouped `rank_pct` from the day's eligible observations using the same §4 grouping function — the function must be **shared** between live Pass 2 and validation, exactly as `computeWeightedAnalystScore` is shared between `research-agent.ts` and the engine). This guarantees the challenger is graded on the identical rank rule that would run live.
- **Walk-forward integrity:** rank is computed **within each PIT day** from data available that day — it introduces **no** look-ahead (a day's percentile only uses that day's cross-section). Purge/embargo rules are unchanged.
- **Metrics:** top-K precision / top-minus-median spread (scoring-methodology §10) are the natural fit — a rank gate should **improve** `precision@K` if it has edge. Champion-vs-challenger is paired on the same opportunity set.
- **Eligibility gates unchanged:** Sharpe ≥ 0.5 , win rate $\geq 40\%$ (docs/arch/09). No new gate needed; rank is just another genome dimension the engine already knows how to replay.
- **No regime detection:** the challenger contains a single scalar `rank_pct_min`. Its adaptivity is emergent (on a weak day fewer names clear the paired floor+rank bar), exactly the "scoring naturally adapts" principle the CLAUDE.md pushback mandate protects. There is no bull/bear state, no regime table read for the gate.

9. Explicit non-conflict checklist (CLAUDE.md locked decisions)

Locked decision	How this design respects it
No explicit bull/bear regime switching	Rank is a within-day percentile scalar; no regime state, no mode switch. §2, §8.
Dual-bucket momentum/value screener	Untouched — rank is downstream of scoring, screener discovery unchanged.
≤ 3 screener candidates/day	Rank only removes candidates from the actionable set; churn can only fall. §5.
Top-3 by <code>analyst_score</code> win	Within a comparable group, <code>rank_pct</code> is monotonic in <code>analyst_score</code> ; intra-group top ordering is identical. §5.
Long-only for NEW positions	Gate applies only to <code>isHeld === false</code> candidates; the <code>isExitSignal SELL</code> path is not read or modified. §4.1, §5.
SELL capability for existing holdings	Holdings never enter the rank pool; their exit signals bypass the gate entirely. §4.1.
No agent complexity before the weekly learner has run	Ships OFF (<code>rank_pct_min</code> default 0.0); becomes active only via a validated, owner-promoted challenger. §5.
LLM never generates score/direction/eligibility	Pass 2 is fully deterministic, no LLM call. §3.

Only borderline item: "top 3 by `analyst_score` win" **could** be read as "selection must be by raw absolute score." This design keeps raw-score ordering **within** a comparable group (monotonic) and only changes the **cross-group admission** rule. If the owner intends the phrase to mean "never let a lower-raw-score name beat a higher one anywhere," note that with a **single** comparable group (the common degraded/small-universe case today) the two are identical — so the divergence only appears once the universe is large enough to have multiple valid sector groups, which is precisely when cross-sectional rank has value. Flag for explicit owner sign-off.

10. Phased build plan & effort

Phase	Deliverable	Gate to advance	Effort
P0 — measure-only correctness	Replace the cron's naive mixed-pool rank_pct (research/cron ~L193) with the §4 grouped, gated computation + migration ~145 columns. Rank still not consumed for entry. Shared computeComparableRank() in lib/scoring/.	Unit tests: grouping, small-group degraded path, exclusion reasons, monotonicity within group. Migration verified applied.	~1.5 days
P1 — genome plumbing (still off)	Add entry.rank_pct_min to genome type + DEFAULT_GENOME (0.0), bounds at promotion, docs/arch/09 update. No behavior change.	Byte-stable selection with default genome (regression test).	~0.5 day
P2 — Pass 2 entry gate + shadow	rankAndGate() in cron after scoring: flip agent_signals for rank-rejected NEW candidates; write rank_rejected/status='rank_rejected'; Research Journal reason. Runs only when champion rank_pct_min > 0. Add shadow scoring so a shadow challenger's rank decisions are recorded (mirrors existing shadow_decisions).	Acceptance tests §11; shadow evidence accrues; no live/paper change while champion default.	~2 days
P3 — validation replay	Shared rank function wired into Validation Engine; challenger with rank_pct_min replayable on held-out folds with top-K precision metric.	Paired champion/challenger replay reproduces live rank decisions; walk-forward no-leakage test.	~2 days
P4 — learner proposes rank challengers	Learner's propose_challenger may set rank_pct_min; IC of rank_pct feature reported. Owner promotes if validated.	Same Sharpe≥0.5 / win-rate≥40% gates; owner promotion.	~1 day
(later) P5 — rank as P(win) feature	Out of scope here. Only after rank clears the IC/OOF bar (§6).	scoring-methodology §10 calibration gates.	—

Total to actionable-but-off (P0–P2): ~4 days. To validated & promotable (P0–P4): ~7 days.

11. Acceptance tests

- Same universe snapshot + same genome → byte-stable rank_pct, rank_quality, and entry decisions.
- ETFs never share a comparable group with single-name equities; an ETF-only group with < min sample is degraded, never a fabricated percentile.
- A comparable group with < min eligible names uses the pre-registered fixed transform and is flagged degraded — the "three finalists are not a universe" test.
- Thin-evidence, abstained, low-confidence, and held symbols are **excluded** from the ranking pool (row written with rank_eligible=false, rank_pct=null).
- With rank_pct_min = 0.0, selection is **identical** to pre-feature behavior (regression).
- With rank_pct_min > 0, the actionable set is always a **subset** of the absolute-floor survivors (never larger).
- Held-position SELL/exit signals are unaffected by any rank_pct_min value.
- Within one comparable group, ordering by rank_pct equals ordering by analyst_score.
- Validation replay of a rank_pct_min challenger reproduces the live Pass-2 decisions on the same PIT day (shared function).
- No regime table is read anywhere in the rank path.

12. Risks & open questions

1. **Small daily universe (biggest risk).** Realistic per-market daily eligible counts (~6 screener + watchlist + a few holdings) are **below** the min-sample thresholds, so most runs use the degraded fixed-transform path — meaning true cross-sectional rank rarely engages until the PIT universe widens. *Mitigation:* the degraded path is honest (collapses to a softened absolute gate); the feature's payoff scales with universe breadth work already on the scoring-methodology roadmap. **Open question:** should we widen the daily screener pull (currently RESEARCH_SCREENERS_MAX=6) specifically to feed ranking? That trades against the ≤ 3 -candidate/overtrading rule — the pull can widen for *ranking universe* while the *actionable* cap stays ≤ 3 . Needs owner decision.
2. **Sector attribution quality.** Grouping by OVERVIEW.Sector depends on that field being populated; missing sector → falls to the market×asset-type group. Acceptable but coarsens groups. India sector coverage is thinner still.
3. **Two-pass latency.** Pass 2 runs inside the 150s cron maxDuration after scoring. It is a pure in-memory computation over already-fetched results + a couple of writes — negligible, but must not re-fetch anything.
4. **Interaction with RESEARCH_CANDIDATE_CAP.** gatherSymbols already caps the candidate map at 10 before scoring; ranking operates on whatever scored, so the cap upstream still governs how many names *reach* the pool. Confirm this ordering is intended (rank ranks the post-cap survivors).
5. **analyst_score monotonicity assumption.** The "ordering preserved within group" claim holds because rank_pct is the empirical percentile of analyst_score within the group. If a future deterministic_v2 ranks on a rankScore distinct from analyst_score, revisit the PaperTrader ordering note (§5).
6. **Degraded-rank live eligibility.** Should a degraded rank ever be allowed to authorize an autonomous-live order? Recommend **no** — degraded rank is paper/shadow-only evidence; live auto keeps requiring evidence_confidence and live_approved regardless. Confirm.

13. What this does NOT do

- Does **not** add or read any market-regime / bull-bear detection (CLAUDE.md locked).
- Does **not** change the dual-bucket screener, its criteria, or its candidate count.
- Does **not** raise the ≤ 3 actionable candidates/day cap or any position/cash cap — it can only tighten.
- Does **not** remove or alter SELL-for-holdings; the exit path is untouched.
- Does **not** feed rank into the live P(win)/sizing model (logged as a feature only, IC-gated — §6).
- Does **not** let an LLM produce rank, score, direction, or eligibility.
- Does **not** rewrite append-only decision_observations rows (rank lives in universe_snapshot_scores).
- Does **not** change any live behavior on ship — default rank_pct_min = 0.0 reproduces today exactly until a validated challenger is owner-promoted.
- Does **not** relabel the scorer deterministic_v2 (scoring-methodology §1); this is an additive gate on deterministic_v1.

```
# Source: features\crypto-native-platform\FEATURE_ARCHITECTURE.md
# Crypto-Native Research, Execution, and Learning
```

```
> Status: **Approved for Stages A-D by Vaibhav, 2026-09-16.** This does not
> authorize a live crypto order. Stage E remains preview/tap-approve only and
> Stage F requires a separate owner decision after its gates pass.
>
> Owner intent: Kairos should research, paper-trade, evaluate, and eventually
> execute crypto with the same rigor as equities – while treating crypto as a
> different market, not as a stock with larger stop percentages.
```

```
## 1. Decision
```

```
**Why:** The current Stage 3 crypto book collects useful paper evidence, but it
```

uses three selected assets, daily bars, a stock-shaped score, fixed 15%/25% levels, and no benchmark or dedicated learner. That is an evidence collector, not a credible crypto execution system.

****Who:**** The Kairos owner, initially using the existing isolated crypto paper pool and later an explicitly enabled Robinhood Crypto account.

****ROI:**** A falsifiable, execution-aware crypto sleeve can prove whether there is a net-of-cost edge before capital is at risk. The moat is disciplined refusal and auditability, not an LLM inventing coin theses.

****Non-goal:**** "Trade every coin", always be invested, or enable autonomous live crypto because a paper signal looks attractive. No live stage is released merely because this design is implemented.

2. Pressure-tested operating facts

- Robinhood supports crypto market, limit, stop, and stop-limit orders, but Kairos must capability-probe the account and pair before treating any order type as available. Robinhood says a sell stop converts to a market order and may execute with up to a 5% buffer; that is not a guaranteed exit price.
- Crypto trades 24/7 and Robinhood notes scheduled maintenance exceptions. A weekday stock cron and a daily close price cannot claim continuous protection.
- Displayed mid price is not executable. Robinhood distinguishes bid/ask from mid and warns that spread varies materially by coin and liquidity.
- Limit/stop-limit orders can fail to fill. Therefore a target and a stop are *policies plus reconciled order state*, not two numbers recorded on a position.

Sources: [Robinhood crypto order types](https://robinhood.com/us/en/support/articles/order-types/), [Robinhood buying and selling crypto](https://robinhood.com/us/en/support/articles/360001298246/), [Robinhood crypto routing](https://robinhood.com/us/en/support/articles/360022216832/).

3. Architecture decisions

3.1 Separate crypto research lane and candidate universe

The active universe is ****not**** every listed token and is never a static hardcoded list. A daily `CryptoUniverseCollector` reads the broker's tradeable-pair inventory and writes a point-in-time snapshot with account eligibility, jurisdiction availability, quote availability, observed spread, and 30/90-day history coverage. It admits a pair to research only when all are true:

1. broker says the pair is tradable for the connected account;
2. at least 90 completed UTC daily bars and the required intraday history exist;
3. executable quote is current and spread is below a configured, evidence-backed ceiling; and
4. the pair is neither halted, maintenance-blocked, nor on a manual deny list.

The collector cannot buy, score-promote, or mutate a watchlist. It only creates the candidate population and records why a pair was refused.

3.2 A crypto-native deterministic score

`CryptoScore v1` is deterministic and versioned. It has no stock P/E, earnings, analyst-target, insider, or US-equity-macro substitute. Inputs are:

Dimension	Purpose	Initial role
Trend / relative strength	Trend across 1d, 4h, and 1h bars; rank only against eligible crypto pairs	candidate selection
Volatility / regime	ATR, realized volatility, gap/jump frequency, BTC market regime	sizing and stop geometry
Liquidity / execution	bid-ask spread, quote age, traded-volume proxy, recent fill/slippage history	hard eligibility veto
Market structure	distance from moving averages, breakout/retest, drawdown state, volume confirmation	entry/exit evidence
Event / sentiment	provider-backed crypto news only, timestamped and availability-tracked	measure-only until predictive IC clears a gate
Network data	only if a licensed provider supplies point-in-time, symbol-consistent data	measure-only; no fabricated "fundamentals"

No LLM can emit a score, tradeable flag, stop, target, or regime. An LLM may summarize already-persisted deterministic evidence for the UI, clearly labelled as commentary.

3.3 Dedicated crypto genome, not the equity genome

Create `crypto\_strategy\_versions`, `crypto\_strategy\_evaluations`, and `crypto\_shadow\_configs` rather than widening equity genome tables. A crypto genome may vary only bounded, interpretable parameters:

- timeframe family (daily/4h/1h), trend lookbacks, and breakout confirmation;
- ATR / realized-volatility stop multiple and invalidation rule;
- target policy (fixed reward multiple versus deterministic trailing policy);
- maximum spread, maximum quote age, minimum history, and concentration cap;
- risk-per-trade and total crypto-sleeve risk.

It may ****not**** mutate into leverage, martingale/recovery sizing, averaging down, unbounded turnover, or a new order type. Candidate/champion/shadow transitions reuse the existing one-champion state machine but have their own evidence tables and crypto-only purged walk-forward evaluation.

3.4 Deterministic entries, stops, targets, and exits

At entry, `CryptoExitGeometry` receives a frozen executable quote, current volatility regime, entry setup invalidation level, and expected costs. It emits:

- `stop`: the more conservative of structural invalidation and an ATR-volatility floor, but never wider than the configured maximum loss;
- `target`: a deterministic reward/risk or trailing-policy level *only if* expected net reward after worst-case spread/slippage exceeds the required threshold;
- `size`: risk budget divided by stop distance plus expected execution cost, bounded by a crypto sleeve cap, coin cap, and cash availability;
- `refusal`: if any input is stale, spread is excessive, stop distance is too wide/narrow, or net expected reward fails after costs.

There is no unconditional calendar exit. A position can exit only because the protective stop is triggered, target/trail policy fires, the deterministic thesis is invalidated, or a safety kill switch closes/requires owner intervention.

Every exit decision stores its geometry inputs and version, so actual net return can be compared against expected return, expected loss, quote spread, and fill slippage.

3.5 Paper execution before live execution

Paper must model bid/ask, fees, partial fill, rejection, cancellation, and quote age. It cannot keep filling at candle close. `CryptoPaperExecutionAdapter` will consume the same quote contract as future live execution, record a fill model version, and create a simulated order lifecycle.

Paper targets/stops are evaluated from intraday high/low only after that bar is complete; ambiguous bars resolve adversely. This preserves the Stage 3 safety rule while permitting a higher-frequency shadow.

3.6 Live execution safety kernel

Live code is built behind `CRYPTO\_LIVE\_ENABLED=false` and a separate owner-only settings switch. It starts at preview-only, then owner tap-approve, then may become autonomous only after evidence gates. It must require all of:

1. broker onboarding, account eligibility, pair availability, crypto buying power, and a fresh quote verified immediately before preview and submit;
2. deterministic `CryptoExitGeometry` pass and a stored idempotency key;
3. a successful broker preview whose expected price/cost is within the policy tolerance; and
4. a fresh global and crypto-specific kill switch, risk budget, max exposure, daily loss, order-rate, and duplicate-order checks.

Because broker-native OCO/brackets are not assumed available, **a live entry is rejected until the execution supervisor proves both sides of protection are in force or explicitly records why it cannot establish them.** The supervisor reconciles broker orders/positions every minute, detects partial fills and rejections, cancels stale sibling orders, and raises a critical alert when a live position lacks a verified protective state. It never silently replaces a missing broker stop with an application-only promise.

3.7 Benchmarks and Crypto Markets workspace

Add `/dashboard/crypto` rather than hiding crypto in a stock page. It shows:

- universe funnel: broker-listed → eligible → researched → scored → paper/live;
- each score dimension, source timestamp, quote/spread, and deterministic reason for buy, refusal, hold, or exit;
- separate paper/live NAV, cash, positions, realized/unrealized P&L and every order lifecycle event;
- benchmark comparison against BTC buy-and-hold plus a frozen, equal-weight eligible-universe reference. Neither benchmark is presented as a universal crypto market proxy; both have point-in-time constituents and session labels;
- strategy table: version, paper/live state, weeks observed, trades, net return, max drawdown, win rate, profit factor, IC and uncertainty. No "winner" badge before a predeclared minimum sample and out-of-sample gate.

The existing `/dashboard/trading` Crypto Watch remains a compact summary linked to this workspace.

4. Delivery stages and gates

Stage	Deliverable	Cannot do	Promotion gate
A	Broker inventory + quote capability probe, data contracts, workspace shell	score, trade, write a live order	verified pair/onboarding
B	Crypto score v1, candidate funnel, feature ledger, IC pipeline	paper entry	completed labels and source-quality coverage
C	Crypto genome + execution/exit-geometry shadows	execute paper/live	purged OOS evidence and net-cost replay
D	Intraday paper lifecycle, benchmarks, strategy comparison	live order	minimum paper fills/exits, reconciliation and adverse-fill test
E	Preview-only then owner tap-approve live adapter	autonomous live order	real broker preview/reconciliation drills and explicit owner
F	Autonomous live, if ever	change genome without gate	sustained shadow/paper/live readiness, independent review, owner switch

5. Acceptance criteria

1. Every tradable candidate was broker-eligible at the recorded time; every excluded candidate has a recorded reason.
2. A crypto score is reproducible from immutable input timestamps and version.
3. No current-day/incomplete candle, stale quote, mid-price-only quote, or missing spread can enter paper or live geometry.
4. A mutated rule that bypasses liquidity, max-risk, stale-quote, or broker eligibility must fail a test.
5. Reported performance is net of modeled/actual fees, spread, and slippage; paper and live are never co-mingled.
6. A live position without confirmed protective execution state creates a critical alert and blocks further entries.
7. Live crypto remains disabled until Stage E's explicit owner action. Stage F is a new decision, not a default.

6. Explicit deferrals

Paper recovery implementation note – 2026-09-21

The existing Stage 3 paper basket remains BTC-USD/ETH-USD/SOL-USD; this repair does not expand executable names or enable live trading. Public research always prioritizes these three before bounded broker-discovered names, normalizes USD symbols, and labels missing broker members non-tradeable. Native pipeline identity uses `score\_source`; constrained discovery `source` is `screener`. The previously committed daily 00:45 UTC paper schedule is now applied. Non-cron entry invocation requires the owner, not merely any signed-in viewer.

The fill RPC now has a crypto-only mandate branch (score 60, at most three names, native claimed signal and reviewed basket required). Equity mandate lookup is unchanged. The guarded migration refuses unexpected RPC source drift and was first executed inside a rolled-back transaction. RPC mandate version is integer 1; the descriptive policy name stays in the snapshot, not in the integer argument.

- no leverage, perpetuals, options, staking yield, lending, DeFi, transfers, or custody/wallet functionality;
- no on-chain metric until it is licensed, point-in-time, and coverage-audited;
- no claim that a high paper win rate or a single strategy is a proven edge;
- no copying equity benchmark, target, stop, or genome parameters into crypto.

Source: features\data-availability-layer\FEATURE_ARCHITECTURE.md
Feature Architecture – Data Availability Layer

> 2026-07-31 correction: the India GDELT sentiment assumptions in this historical design did not survive production evidence (0/310 usable obs)

> Status: **PROPOSAL** – draft, Codex-reviewed, not built. Needs owner sign-off. NOTHING here is wired into the running app yet – the shipped

> Scope: guarantee no scoring dimension / agent flow is ever starved of input data on a max-symbol day, using only free + already-keyed sources

> Last updated: 2026-07-13

> Update when built: `docs/arch/02-tech-stack.md`, `docs/arch/03-agents.md`, `docs/arch/05-crons-and-scheduling.md`, `docs/arch/09-learning-1

One-line decision

Put a **Data Availability Layer** in front of ResearchAgent: a **paced**, cache-first refresh cron fills Supabase evidence caches ahead of scoring; ResearchAgent scores from cache snapshots and only makes bounded, last-resort live calls. Resilience for thin dimensions (fundamentals) comes from **cache TTL + stale-serve + pacing**, NOT from fallback breadth – because on the free tier the fallbacks mostly do not exist. (Design from the Codex review, corrected to only live-validated sources.)

Why not "just add more fallbacks inside processSymbol()"

- Vercel serverless: no shared in-memory rate state across invocations, and a cron can't spin-wait 17 min for a 5/min provider. Synchronous per-symbol fallback chains blow both the cron `maxDuration` and the provider limits.
- On a max day (measured: **US 42 symbols, India 13**), synchronous scoring would need ~84 Massive calls (5/min) and 84 Finnhub calls (60/min) in one burst.

LIVE-VALIDATED provider truth (tested 2026-07-13 – the correction to Codex)

Every entry below was hit with the real key this session. **A provider may only enter a chain after it is live-validated; "documented as free" is not enough** (FMP, FinancialDatasets, TwelveData, EODHD all **looked** usable and were not).

Dimension	Market	Provider	Result	In chain?
Fundamental	US	Finnhub `/stock/metric+/profile2`	works (PE, margin, ROE, EPS, rev-growth, sector)	primary
Fundamental	US	Yahoo `quoteSummary` (crumb)	endpoint returns data (profitMargins, returnOnEquity, revenueGrowth, forward PE, sec	
Fundamental	US	SEC EDGAR `xbrl/companyfacts`	endpoint returns data (Revenues, NetIncomeLoss, EPS, StockholdersEquity). Free/no-key;	
Fundamental	US	FMP `/stable/ratios-ttm`	premium-gated on free	no
Fundamental	US	FinancialDatasets snapshot	\$0 credit balance	no
Fundamental	US	TwelveData `/statistics`	premium-only (403)	no
Fundamental	US	EODHD `/fundamentals`	free = EOD prices only	no
Fundamental	US	Massive `/financials/v1/ratios`	plan NOT_AUTHORIZED	no

```
| Fundamental | India | Yahoo `quoteSummary` (crumb) | ■ endpoint returns data for `.NS` (RELIANCE.NS: margin 0.076, ROE 0.091, rev-growth 0.
| Fundamental | India | Finnhub (NSE) | ■ "no access to this resource" | no |
| Fundamental | India | TwelveData / EODHD (NSE) | ■ premium / symbol-not-found | no |
| Sentiment | US+India | GDELT `tonechart` | ■ works for any company name, free, no daily cap | **backbone** |
| Sentiment | US | StockTwits | ■■ unreliable (frequent 403) | opportunistic first |
| Sentiment | US | AV `NEWS_SENTIMENT` | ■ but 25/day HARD cap | reserve only |
| Insider | US | Massive `/filings/vx/form-4` | ■ works (P/S transaction detail) | **primary** |
| Insider | US | SEC EDGAR Form 4 | ■ free/official (was returning unavailable in practice – re-verify) | secondary |
| Insider | India | NSE corporate PIT disclosures | ■■ exists in `lib/nse-data.ts` – UNVALIDATED for scoring | validate before claiming |
| Technical | US | Massive aggregates | ■ works | primary |
| Technical | US | TwelveData `/time_series` | ■■ on free tier (8/min, 800/day) – validate | secondary |
| Technical | US | AV daily | ■ capped | reserve |
| Technical | India | Yahoo chart | ■ works (`lib/india-data.ts`) | primary |
| Technical | India | Upstox candles | ■■ `UPSTOX_ACCESS_TOKEN` present – validate | secondary |
```

Honest verdicts (corrected after Codex review 2026-07-13)
 "Endpoint validated" (the URL returns the fields with our key/handshake) is NOT the same as "integrated + reliable." The three US sources are **not interchangeable** – they carry different metric bases and must be used in DISTINCT roles, not treated as equivalent fallbacks:

- **SEC EDGAR** = primary reported/fundamentals** (TTM margin, ROE, revenue growth from official XBRL). Per-field coverage varies: sampled AAPL/MSFT/NVDA/JPM covered; TSM is IFRS (different taxonomy), BRK.B lacked the standard diluted-EPS tag. So coverage is measured **per field**, not "universal third provider."
- **Finnhub** = primary profile + computed-ratio fields** (sector, TTM P/E, EPS).
- **Yahoo** = opportunistic forward-looking fields** (forwardPE) + the **SOLE** India source. Unofficial – pace + fail-soft + monitor; never a guaranteed backbone.
- **AV OVERVIEW** = capped emergency reserve.**
- **US fundamentals**: genuine multi-source, but role-split, not "3 equal fallbacks."**
- **India fundamentals**: coverage exists (Yahoo) but resilience does NOT – single unofficial source.** Keep the explicit single-source risk; on failure → `provider\_unpriceable` + renormalize (like ETFs), never faked neutral 50.
- ETFs legitimately have no fundamentals/insider – "not applicable," never starvation.
- **Reality gap**:** these are PROPOSAL-only for US. Current shipped US chain is still Finnhub → FMP → AV (`lib/data/fundamentals.ts`); SEC + US-Yahoo are NOT wired yet, so they do not protect the running app until built.

Corrected provider chain matrix (endpoint-validated; role-split, not equal fallbacks)

```
| Dimension | Market | Ordered chain |
|---|---|---|
| Fundamental | US | **reported** SEC EDGAR (TTM) + **profile/ratios** Finnhub + **forward** Yahoo (opportunistic) → AV `OVERVIEW` (reserve)
| Fundamental | India | Yahoo quoteSummary (crumb, monitored single-source) → stale-cache-serve → else `provider_unpriceable` |
| Technical | US | Massive → TwelveData `/time_series` *(if validated)* → AV daily (reserve) |
| Technical | India | Yahoo chart → Upstox *(if validated)* |
| Sentiment | US | StockTwits (opportunistic) → GDELT tonechart → AV NEWS (reserve) |
| Sentiment | India | GDELT tonechart → NSE announcements *(if validated)* |
| Insider | US | Massive Form 4 → SEC EDGAR → AV (reserve) |
| Insider | India | NSE PIT *(if validated)* → else not-enough-data |
```

(Twelve Data `/time\_series`, Upstox, NSE PIT, Yahoo-crumb each carry a **validation gate** – a live probe that must pass before the source is enabled in config. This gate is the durable fix for the "looked wired, wasn't" failure mode.)

Metric equivalence + provenance (the hard part – Codex correction)

Different providers report the SAME dimension on DIFFERENT bases. Swapping a provider must never silently change what a score MEANS. So every fundamental field is stored with provenance, and derived metrics use consistent periods:

- **Per-field provenance**:** store `metric\_basis` (e.g. `ttm` | `forward` | `annual` | `quarterly`), `period\_end`, `source`, `taxonomy` (`us-gaap` | `ifrs`), and `fetched\_at` for EACH field – not one label for the whole overview.
- **SEC period selection** (must not take "latest" blindly):** companyfacts mixes annual, quarterly, and YTD observations. The adapter must:
 - TTM margin from **matching-period** TTM revenue and TTM net income (sum of 4 consecutive quarters, same units), not latest annual ÷ latest quarterly.
 - ROE on **average equity** ((begin+end)/2), not the latest ending equity.
 - Revenue growth from **equivalent periods** (TTM vs prior TTM, or FYvsFY).
 - Handle **US-GAAP** and **IFRS** tags (TSM sampled as IFRS); fall through when the standardized tag is absent (BRK.B lacked the standard diluted-EPS tag).
 - Keep **TTM P/E** (SEC EPS + price) distinct from Yahoo/Finnhub **forward** P/E – never blend the two into one field.
- **Per-field coverage, not per-provider**:** SEC covers major facts for most large caps but NOT all fields for all issuers, so availability is tracked at the field level. A field with no consistent-period value is `genuine\_no\_data`, and the fundamental score renormalizes over the fields it DOES have.
- The scorer already renormalizes over available dims; extend that to **available fields within** the fundamental dimension so partial SEC coverage still scores.

Architecture

1. Cache-first evidence store

Per (symbol, market, dimension): `evidence\_cache(symbol, market, dimension, payload, source\_used, fetched\_at, ttl\_days, stale\_ok)`. ResearchAgent reads this, never the

provider directly.

2. Per-dimension freshness TTL (the real load-killer)
On a 42-symbol day, most symbols already have fresh evidence, so few live calls fire.
- Fundamentals: **14 days** (financials change quarterly).
- Insider: 1-3 days.
- Sentiment: 1 day.
- Technicals: current trading day / latest EOD.

3. Priority classes (whom to freshen first, not "trim to 3")
1. Held live/paper positions – always freshen.
2. Open candidates likely to cross threshold – second.
3. Watchlist – third.
4. Broad screener names – last; may score on stale fundamentals.
Score ALL applicable symbols; only *freshen* expensive dims for priority 1-2 daily.

4. Supabase-backed pacing (not in-memory – serverless-safe)
- `provider\_limits(provider, min\_interval\_ms, daily\_budget, burst, enabled)`
- `provider\_call\_ledger(provider, started\_at, cache\_key, status, http\_status, throttle\_reason)`
- `provider\_refresh\_jobs(provider, symbol, dimension, market, priority, status, next\_attempt\_at)`
- `providerCachedFetch()`: fresh-cache-hit → return; else acquire a **Supabase lease via RPC/advisory lock** enforcing `last\_started\_at + min\_interval\_ms <= now()`; no slot → return stale cache + enqueue a refresh job. **Never spin-wait in Vercel.**
- Min gaps: Massive 12.5s, GDELT 5.5s, TwelveData 7.8s, AV 15s (+ hard daily reserve), Finnhub 1.2s, SEC 250-500ms, Upstox/Yahoo/NSE 1-2s cache-first.

5. Refresh cron separate from research cron – BOUNDED + resumable
A prewarm/refresh cron drains `provider\_refresh\_jobs` at each provider's safe pace BEFORE the research cron runs. Research scores from warm cache; live calls become bounded exceptions. **A single Vercel function cannot run ~17 min** (Hobby caps ~60s; even Pro maxDuration is bounded) – so the prewarm CANNOT be one long drain. Instead: each invocation processes only as many jobs as fit in a hard wall-clock budget (e.g. <45s), leases per-provider slots, marks jobs `next\_attempt\_at`, and **returns** – leaving the rest queued. The job is scheduled to fire repeatedly (pg_cron every few minutes) so the queue drains across MANY short invocations, never one long one. State lives in Supabase (`provider\_refresh\_jobs` + `provider\_call\_ledger`), so progress survives across invocations. Cold-start of a full 42-symbol universe simply takes several prewarm ticks, not one function call.

6. Starvation invariant + telemetry (first-class)
Invariant: *No applicable scoring dimension may be unavailable solely because a provider budget/rate/burst limit was exhausted. It may be unavailable only because every configured source returned genuine no-data, the symbol is structurally inapplicable, or no validated free source exists for that market/dimension.*
- Per symbol/dimension record: `applicable, available, source\_used, source\_chain\_attempted, unavailable\_reason`.
- Reasons: `not\_applicable | genuine\_no\_data | provider\_rate\_limited | provider\_daily\_budget | provider\_auth\_missing | provider\_unpriceable | stale\_cache\_used | chain\_exhausted`.
- System Health (extends the existing low-confidence-research alert):
`data-availability:<market>:<dim>` – warn < 85% availability, critical < 70%, detail names the throttled/exhausted provider.

7. Evidence provenance labels
Fix stale "Alpha Vantage" labels in the journal to show the real `source\_used` (Finnhub / Yahoo / GDELT / Massive / SEC).

Capacity proof (max day: US 42, India 13) – after TTL + prewarm
With 14-day fundamental TTL and priority-class freshening, a steady-state day freshens only priority-1-2 fundamentals (~5-10), all sentiment (1-day TTL) via GDELT, insider (1-3d) via Massive. Worst-case COLD start (empty cache) is absorbed by the prewarm cron over its window, never the scoring cron:
- AV: target 0, ≤10 reserve; **never 42** → under 25/day. ■
- Finnhub: ≤ (priority fundamentals × 2) paced at 1.2s → seconds. ■
- Massive: prewarmed at 12.5s gaps → ~17 min cold, off the scoring path. ■
- GDELT: ≤ (US+India sentiment) at 5.5s → prewarm window. ■
Any chain that still can't cover 42 (e.g. India fundamentals) is reported as `provider\_unpriceable`, not silently starved.

Build order – SHIPPED status (2026-07-13)
1. ■ **DONE** – US sentiment → StockTwits → GDELT tonechart → AV reserve (commit 504cf44).
2. ■ **DONE (adapted)** – Supabase-backed pacing: `provider\_pacing` + `try\_acquire\_provider\_slot` RPC (migration 176) in `providerCachedFetch()`, for the HARD-limited providers (Massive/GDELT). Serve-stale on no-slot. (Full `provider\_limits`/ledger/refresh-jobs deferred with the prewarm cron.)
3. ■ **PARTIAL** – source chains are code-ordered (Finnhub→Yahoo→SEC→FMP→AV; Massive→EDGAR→AV; StockTwits→GDELT→AV). Not yet a config registry with per-source validation gates.
4. ■ **DONE** – `data-availability:<market>:<dim>` telemetry (warn<85%/crit<70% among applicable).
5. ■ **DONE (core)** – bounded+resumable prewarm cron (`/api/agents/prewarm`, `prewarmSymbol`, pg_cron `kairos-prewarm-us/-india`) warns `av\_cache` (the evidence cache) ahead of scoring, paced so it never bursts. ■ Explicit priority classes (holdings-first) + a dedicated `provider\_refresh\_jobs` queue not built – the bounded tick + 14d-TTL cache-hit skip approximate it.
6. ■ **NOT BUILT** – evidence provenance labels in the journal (still may show generic sources). Also deferred: SEC EDGAR fundamentals (spot-check failed – needs the `frames` API).
7. ■ **DONE (fundamentals adapters)** – SEC companyfacts adapter (`lib/data/sec-fundamentals.ts`, consistent latest-annual basis, avg-equity ROE, FY-vs-FY growth, US-GAAP+IFRS, symbol→CIK map,

per-field coverage) + US Yahoo `quoteSummary` (reuses `fetchIndiaOverview`, crumb cache + fail-soft) wired into the US chain. ■ Still-unvalidated ■■ sources (Upstox, TwelveData `time\_series`, NSE PIT/announcements) remain behind a future live probe. Insider adapter (Massive Form 4) shipped earlier (commit 3d742c4).

Note: "endpoint returns data" ≠ "correct metric" – a live spot-check of the SEC-derived margin/ROE against ≥5 names is still worth doing before fully trusting the SEC tier.

Riskiest assumption

Fundamentals redundancy now rests on two **unofficial** sources (Yahoo quoteSummary, and Finnhub's free tier) plus one official (SEC EDGAR, US-only). The residual risks:

1. **Yahoo is unofficial** – the crumb/cookie flow or field shapes can change without notice; it is the SOLE India fundamentals source. Mitigation: cache aggressively (14-day TTL), fail soft to `provider\_unpriceable`, and health-alert if Yahoo's India availability drops. A second India source (e.g. Screener.in/Tickertape scrape, or NSE fundamentals) remains an open follow-up – nice-to-have, not blocking.
2. **EDGAR needs a maintained symbol→CIK map + a price** to derive P/E; margin/ROE/growth come straight from XBRL. Low risk (official, stable) but more mapping code.

The earlier "no free India fundamentals source" blocker is RESOLVED (Yahoo `.NS` live-validated), so this is no longer a launch blocker – just a monitoring concern.

Non-negotiables

- Free cloud only – never a paid tier/proxy/VPS. Every source free + keyed in vault/Vercel.
- Deterministic – no LLM on the scoring/fallback path.
- Forward-only – no backfill of historical signals.
- Per-market / per-currency isolation; US and India chains independent.
- A source enters a chain ONLY after a live validation probe passes.

Source: features\data-provider-abstraction\FEATURE_ARCHITECTURE.md
Data Provider Abstraction + Per-Dimension Routing – FEATURE_ARCHITECTURE (DRAFT)

Status: **DRAFT** – awaiting approval. No code written yet.

Date: 2026-07-07

Trigger: `av\_budget` hit 183 vs the 25/day free cap → ~70% of symbols get no real technical/fundamental data every run (observed as technical=50 / fundamental=55 across GLD/IBIT/XAR). Root cause: every heavy per-symbol dimension is forced through Alpha Vantage's 25/day free tier.

All provider capabilities below were verified live (2026) against official provider docs, not assumed. Sources in MARKET_DATA_API_INVENTORY.md + this session's research.

1. Problem (verified)

Per US research symbol, current code spends ~4 AV calls, all through `avCachedFetch` which increments the single shared `av\_budget` counter:

```
| Dimension | AV call | File |
|---|---|---|
| Technicals (candles) | `TIME_SERIES_DAILY_ADJUSTED` | `lib/research-agent.ts:608` |
| Fundamentals | `OVERVIEW` | `lib/research-agent.ts:622` |
| Insider | `INSIDER_TRANSACTIONS` | `lib/research-agent.ts:49` |
| News sentiment | `NEWS_SENTIMENT` | `lib/social-sentiment.ts:125` |
```

Plus two confirmed waste bugs:

- **FD eats AV budget** – `app/api/agents/research/scan/route.ts:80` routes FinancialDatasets `FD\_SNAPSHOT` through `avCachedFetch`, incrementing `av\_budget`. FD calls burn AV's 25 slots.
- **Scanner double-fetches** – `research/scan/route.ts:53,64,65` calls AV `RSI`, `GLOBAL\_QUOTE`, `EMA50` per symbol, redundant with research's own candle fetch that already computes RSI/EMA locally.

24 symbols × ~4 calls = ~96 AV calls per run; two runs/day = ~190. Cap is 25.

2. Design – provider abstraction

Replace the single AV-specific `avCachedFetch` with a generic cached fetcher keyed by provider, each with its own daily budget counter. NO trading-execution code changes.

2a. `lib/data/providers.ts` (new)

```
type ProviderId = "alpha_vantage" | "massive" | "finnhub" | "fmp"
```

```
| "alpaca" | "upstox" | "marketaux" | "gdelt" | "yahoo"
```

```
| "sec_edgar" | "fred" | "financialdatasets";
```

```

interface ProviderSpec {
id: ProviderId;

dailyBudget: number | null; // null = no daily cap (rate-limited only)

budgetKey: string; // av_budget-style counter row key

keyEnv?: string; // env var holding the key (absent = keyless)

expiresAt?: string | null; // ISO date the credential expires; null = never.

// For JWT tokens (Upstox), auto-derived by decoding
// the token's exp claim rather than hardcoding.
}

### Credential expiry tracking (user-requested)

Most keys never expire (Finnhub/FMP/FRED/Massive = indefinite until revoked).
Token-based providers do: Upstox (1yr JWT), Kite (daily), Robinhood (OAuth
refresh). The registry carries an `expiresAt` per provider:

- For **JWT tokens** (Upstox), `expiresAt` is derived by base64-decoding the
  token's `exp` claim at read time – no manual date entry, stays correct if the
  token is rotated. (Verified: current Upstox token exp = 2027-07-08.)
- For **static keys**, `expiresAt` = null` ("No expiry").
- A daily `provider-credential-check` reporter (reuses the existing
  `lib/system-health.ts` issue funnel) raises a WARN at **30 days** before
  expiry and CRITICAL at **7 days / expired**, auto-resolving when the
  credential is refreshed. Same pattern already used for `broker-token:kite`.

### 2b. `providerCachedFetch(provider, cacheKey, url, opts)` (generalizes avCachedFetch)

Same reserve-before-spend + day-cache + stale-fallback logic already in
`av-cache.ts`, but:
- increments a **per-provider** counter (`provider_budget` table, keyed by
  `(provider, date)`), so FD no longer touches AV's budget;
- the 7-day stale-cache cap already added this session carries over;
- `avCachedFetch` becomes a thin wrapper: `providerCachedFetch("alpha_vantage", ...)`

### 2c. Migration `NNN_provider_budget.sql`

- New `provider_budget(provider text, cache_date date, calls int, primary key(provider,cache_date))`.
- New `provider_budget_increment(p_provider text, p_date date)` RPC (mirrors
  `av_budget_increment`).
- Keep `av_budget` for backward-compat during rollout; migrate reads after.
- `av_cache` table reused as-is (already provider-agnostic by cache_key).

---

## 3. Per-dimension routing table (the core decision)

Primary → fallback chain per dimension. Fallback only fires when primary is
unavailable/over-budget. Availability-mask semantics (this session's fixes)
still apply: a dimension with no live data is EXCLUDED from weighting, never
scored as fake-neutral.

### US

| Dimension | Primary (new) | Fallback | Was | Provider notes (verified) | |
|---|---|---|---|---|---|
| Candles/technicals | **Massive** (`/v2/aggs`) | Alpaca → AV | AV | Massive=Polygon rebrand, key already configured, no daily cap |
| Quotes | **Massive** (already) | Alpaca IEX → AV | Massive/AV | unchanged primary |
| Fundamentals | **FMP free** (250/day) | FD credits → AV | AV | AV `OVERVIEW` | FMP 150+ endpoints, statements/ratios free |
| Insider | **SEC EDGAR** (free, unlimited) | AV | AV | EDGAR route already exists (`app/api/markets/edgar-insiders`) |
| News sentiment | **Finnhub free** (60/min) | Marketaux → GDELT | AV | AV `NEWS_SENTIMENT` | per-ticker scores, free |
| Analyst ratings/revisions | **Finnhub free** | FMP | (none today) | NEW dimension – recommendation trends + price targets free |
| Earnings calendar | **Finnhub free** | AV | AV | free |
| Macro/regime | **FRED** (key added) | AV | AV | official, keyed, drafted next |
| Options (advisory) | Yahoo | – | Yahoo | unchanged |

### India

| Dimension | Primary (new) | Fallback | Was |
| ---|---|---|---|
| Candles/technicals | **Upstox** (analytics token, free, hist to 2000) | Yahoo | Yahoo |
| Quotes | **Upstox** LTP | Yahoo | Yahoo |
| Fundamentals | **Upstox** (ISIN-keyed statements/ratios) | Yahoo quoteSummary | Yahoo |
| Corporate actions | **Upstox** (splits/div/bonus) | NSE/Yahoo | (weak today) |
| Insider/PIT | NSE `corporates-pit` | – | NSE (unchanged) |
| Execution | Kite (unchanged) | – | Kite |

```

Upstox verified: Analytics token = 1-yr, read-only GET, no static IP for data, covers all four India data needs.

4. Keys required (user creates; Claude wires)

Provider	Key/env	Free?	Needed for
Massive	`MASSIVE_API_KEY`	(have)	yes US candles/fundamentals/news
Finnhub	`FINNHUB_API_KEY`	yes	US news/analyst/earnings
FMP	`FMP_API_KEY`	yes (250/day)	US fundamentals
Upstox	`UPSTOX_ACCESS_TOKEN`	yes (1-yr)	India everything
FRED	`FRED_API_KEY`	(added)	yes US macro
Alpaca	`ALPACA_KEY_ID`/`ALPACA_SECRET`	yes	US fallback (optional)
Marketaux	`MARKETAUX_API_KEY`	yes	news fallback (optional)

GDELT + SEC EDGAR = keyless.

Minimum to end the AV-limit problem: ****Massive (have) + Finnhub + FMP + Upstox****. Alpaca/Marketaux are fallback-only, deferrable.

5. Rollout (safety-first, no trading-behavior change)

1. Provider abstraction + `provider\_budget` migration (no routing change yet).
2. Split FD off AV budget (`fdCachedFetch`) – immediate AV relief, zero new keys.
3. Kill scanner's redundant AV RSI/EMA/QUOTE calls.
4. Route US candles → Massive (key already present).
5. Wire Finnhub/FMP as keys land; each dimension behind a per-provider adapter with the existing availability-mask fallback (missing → excluded, not faked).
6. Route India → Upstox as token lands.
7. FRED macro adapter (drafted, enabled last).
8. Settings → Data Providers ****capacity dashboard**** (user-requested – "always know where the ceiling is"). Per provider, per market:
 - ****Limit****: daily cap (FMP 250) or per-minute rate (Massive 5/min, Upstox 500/min) or "no cap".
 - ****7-day rolling average**** real calls/day, from `provider\_budget` history (the counter logs one row per provider per day, so the average is a plain query). Also today's live count.
 - ****Headroom / projected ceiling****: `limit – avg`, and for capped providers a "days/x of growth until you hit the cap" figure (e.g. "FMP: 60/250 avg – room for ~4x your current universe").
 - ****Cache hit rate**** (real calls ÷ requested) so you see caching working.
 - ****Credential expiry**** date + days-remaining countdown (green >30d/no-expiry, amber <30d, red <7d/expired), driven by registry `expiresAt`.
 - ****Bottleneck flag****: highlights whichever provider is closest to its ceiling so the single limiting dimension is always visible at a glance.

5b. Redundancy – every dimension has ≥2 sources (never starve)

Each dimension routes primary → fallback(s). Fallbacks fire automatically when the primary is over-budget/rate-limited/erroring. Critically, EVERY dimension already has a ****keyless or already-owned**** fallback, so agents never starve even with zero extra signups:

All keys below are ADDED + live-tested unless marked. Fallback chains (primary → ... → keyless) mean no dimension can fully starve.

Dimension	Primary	Fallback 1	Fallback 2	Fallback 3 (keyless)
US candles/EOD	Massive	EODHD	Twelve Data	Alpha Vantage (have)
US fundamentals	FMP	EODHD	Massive financials	Alpha Vantage
US corporate actions	EODHD	AV	–	–
US news	Finnhub	Marketaux	–	GDELT (keyless)
US insider	SEC EDGAR	(keyless)	Alpha Vantage	–
US analyst	Finnhub	FMP	–	–
India candles/quotes	Upstox	–	–	Yahoo (keyless)
India fundamentals	Upstox	–	–	Yahoo (keyless)
India corp actions	Upstox	–	–	NSE/Yahoo (keyless)
Macro	FRED	Alpha Vantage	–	–

* = optional, not added. GDELT/Yahoo/SEC-EDGAR keyless → every row survives a provider outage.

LIVE-TEST CORRECTION (2026-07-08)

The prior research claim that EODHD/Twelve Data free tiers cover India NSE is ****FALSE – verified by hitting the APIs****:

- EODHD free `RELIANCE.NSE` → "Ticker Not Found" (NSE = paid tier).
- Twelve Data free `RELIANCE/NSE` → HTTP 404 "available starting with the Grow plan" (NSE = paid).

So ****EODHD + Twelve Data add US redundancy only**** on their free tiers. India

redundancy stays **Upstox (primary, has fundamentals+corp-actions) → Yahoo (keyless)** – which is fine; Upstox is free and complete. Adding paid India coverage (EODHD ~\$60/mo or Twelve Data Grow) is deferred until/unless the free Upstox→Yahoo chain proves insufficient. US now has 4-deep candle redundancy and gains clean corporate-actions data (EODHD) that was previously weak.

Capacity at expanded 20 US + 20 India fresh/day (+ holdings ≈ 40 US / 30 India)

Provider	Limit	Load at 40 US/30 India	Verdict
Finnhub	60/min, no daily cap	~160 calls (~3 min)	fine
FMP	250/day ~80/day (40×2)	fine – still 3× headroom	
Massive	5/min, no daily cap	~40 candle calls	fine w/ throttle
Upstox	500/min, no daily cap	30 India	fine

20/20 expansion is comfortably within free tiers. FMP (the only daily cap) only becomes the bottleneck past ~100 fresh US symbols/day – at which point Twelve Data or Massive-financials absorb the overflow, or FMP \$22/mo.

Each step is independently shippable and reversible. Steps 2–4 need **no new keys** and alone likely get US under the 25/day AV cap.

6. Out of scope (explicitly not this feature)

- TradingView integration (no data API – signal-webhook feature, separate track).
- Any paid tier (EODHD/Twelve Data/Massive Starter) – only if free proves short.
- Order-execution / broker changes – untouched.
- Sector-median P/E (needs a sector-median data source – separate).

7. Open questions for approval

1. Approve the routing table as-is, or adjust any primary/fallback?
2. Build all 8 rollout steps, or start with the no-new-key relief (steps 1–4) and pause for keys before 5–7?
3. Data Providers panel (step 8) now, or defer as its own UI task?

Source: features\data-source-policy\FEATURE_ARCHITECTURE.md
Data Source Policy + Canonical Evidence Router – Feature Architecture

> 2026-07-31 current-state correction: India per-symbol GDELT sentiment is retired from active scoring after zero usable production coverage.

> Status: **DRAFT FOR CLAUDE + OWNER REVIEW. DESIGN ONLY.**
> Date: 2026-07-13
> Owner request: stop rewriting agent code and quota logic whenever a provider or MCP tool changes. Give the owner simple market/evidence controls while Kairos > automatically handles provider selection, freshness, pacing, fallback, and audit.
> Implementation gate: no code, migration, provider activation, scoring change, or > deployment until this architecture is reviewed and Vaibhav says `Proceed`/`Code it`.

1. Decision

Build one deterministic **Canonical Evidence Router** between agents and external data providers. Agents request an evidence **intent**, never a provider or tool:

```
await evidenceRouter.resolve({
```

```
  market: "us",
```

```
  symbol: "AAPL",
```

```
  intent: "fundamentals.reported",
```

```
  asOf: decisionTs,
```

```
  runId,
```

```
});
```

Settings exposes `Auto`, `Prefer <provider>`, `<Provider> only`, and `Off` per market + evidence intent. `Auto` is the default and recommended mode. The router owns the provider chain, durable cache, contract validation, quota/pacing, stale policy, source health, and provenance.

Raw MCP tool names and provider URLs are never user-configurable. Adding or changing a provider requires one allowlisted adapter, not edits across agents.

2. Why this is needed now

Kairos currently has several overlapping abstractions:

- ``lib/data/provider-interface.ts`` exposes provider-first methods, but only AV/FMP are substantially implemented and its result types do not carry field-level basis, period, currency, or provenance.
- ``lib/data/provider-fetch.ts`` centralizes HTTP caching/budgets but cannot carry MCP calls, treats ``dailyBudget=null`` as both rate-only and unknown, and does not express capability/market/entitlement.
- ``lib/data/fundamentals.ts`` hardcodes a provider chain and maps everything into an Alpha Vantage-shaped string object, losing per-field source and metric basis.
- ``lib/data/webull-data.ts`` separately opens MCP sessions and uses process-local caching, bypassing the durable provider budget/pacing/cache path.
- Settings → Data is observational only. It cannot safely alter routing.
- ``lib/data/evidence.ts`` has a closed source-name union that omits current sources, while ``computeScores()`` writes inaccurate generic source labels.

This feature consolidates those paths. It does not add a second parallel data system and does not replace the immutable evidence ledger.

Relationship to older feature documents

After owner approval, this document supersedes the provider-routing, cache-control, quota-model, and Settings-control sections of:

- ``features/data-provider-abstraction/FEATURE_ARCHITECTURE.md``
- ``features/data-availability-layer/FEATURE_ARCHITECTURE.md``

Those documents remain historical evidence of why the current adapters/pacing were built. Their already-shipped behavior remains the legacy fallback until this router passes rollout gates. Any provider capability claim that conflicts with this document must be re-probed rather than copied forward.

3. Confirmed defects to repair in the implementation

The following were verified against the live read-only Webull MCP contract on 2026-07-13. They are acceptance-test requirements, not optional cleanup.

P0 – Webull calls silently fail

``get_analyst_rating``, ``get_analyst_target_price``, ``get_stock_forecast_eps``, and ``get_financial_indicators`` require:

```
{ "symbol": "AAPL", "category": "US_STOCK" }
```

The current adapter sends only ``{ symbol }``. Because it is fail-soft, all calls return ``null`` without making the failure visible to the research run.

****Fix:**** the typed Webull adapter owns constant ``category: "US_STOCK"``; callers cannot omit or override it.

P0 – Webull payload parsing is wrong

- Forecast EPS is an array of fiscal periods with fields including ``est``, ``actual``, and ``reported``; the current parser unwraps the first object and does not recognize ``est``.
- Financial indicators are ``{ currency, values: { metric: [{ fiscal_year, fiscal_period, value }] } }``; the current numeric flattener treats arrays as scalar values and returns no usable data.
- Analyst ratings use ``strong_buy``, ``buy``, ``hold``, ``under_perform``, ``sell``, and ``number``. The current parser ignores ``under_perform``, biasing consensus upward.

****Fix:**** use explicit schema validators and period selection. Do not use generic first-record unwrapping for financial data.

P1 – Cache is not serverless-durable

The Webull six-hour ``Map`` exists only inside one Vercel process. Cold starts and parallel instances repeat calls, while a negative result can remain hidden in a warm instance.

****Fix:**** all provider responses use Supabase-backed ``evidence_cache_v2``. A small process cache may remain only as a non-authoritative micro-cache.

P1 – Provenance is inaccurate

``computeScores()`` records fundamentals/ohlcw evidence as Alpha Vantage even when Finnhub, Yahoo, Massive, Upstox, or another provider served it.

****Fix:**** the resolver returns canonical field-level provenance; scoring writes the actual evidence references and policy version.

P1 – Unknown quota is displayed as unlimited

Settings currently renders ``dailyBudget=null`` as ``no cap (rate-limited)``. For Yahoo/Webull and other providers, no published daily limit may mean ****unknown****,

not unlimited.

****Fix:**** model daily-limit knowledge, rate-limit knowledge, and subscription entitlement as separate fields. UI must display `Unknown` and use conservative pacing whenever a limit is unknown.

P2 – Diagnostic surface is broader than necessary

`POST /api/broker-mcp/[broker]/tools` exists only so a cron helper can enumerate tools. Tool discovery is now complete and the route exposes provider descriptions to anyone holding `CRON\_SECRET`.

****Fix:**** after contract fixtures are captured, remove the POST alias. Keep an owner-only GET diagnostics route only if Settings needs it; otherwise remove the route. It must never call tools or return credential/token data.

4. Scope

In scope

- US and India evidence routing.
- Fundamentals, analyst consensus, daily bars, quotes, sentiment, insider, earnings/calendar, corporate actions, and macro evidence intents.
- Code-owned provider capabilities and typed adapters.
- Versioned owner policy with `Auto`/preferred/only/off modes.
- Durable cache, pacing, refresh queue, call ledger, health, and diagnostics.
- Settings product controls and provider capacity visibility.
- Compatibility adapter for existing AV-shaped scoring inputs during migration.
- Webull research adapter fixes and shadow validation.

Explicitly out of scope

- Any broker/order routing or order scopes.
- Enabling Webull orders or changing its `orderCapable=false` gate.
- New signal weights or making Webull analyst consensus a scored dimension.
- Letting an LLM select providers, quotas, fallbacks, or data values.
- Arbitrary provider URLs/tool names entered through Settings.
- Python sidecar/GitHub Actions worker in this phase.
- Paid provider upgrades.

5. Non-negotiable invariants

1. Provider routing is deterministic; no LLM participates.
2. US and India resolve independently. No cross-market or cross-currency merge.
3. Every scored field records provider, basis, period, currency, observation time, retrieval time, and quality state.
4. `unknown` quota never means unlimited.
5. A missing provider/config/cache read fails unavailable, never fabricated neutral.
6. Structurally inapplicable evidence remains distinct from unavailable evidence.
7. A provider switch must not silently change metric meaning (for example forward P/E cannot replace TTM P/E under the same canonical field).
8. Policy changes apply only to future research runs. Each run freezes one active policy version per market.
9. A provider outage cannot touch cash, positions, proposals, or broker orders.
10. Database configuration can select only providers/capabilities compiled into the allowlisted code registry.
11. Raw provider prose is never inserted directly into an LLM prompt. Only schema-validated, bounded canonical fields may be rendered.
12. Existing scoring availability-mask + renormalization semantics remain intact.
13. A policy cannot become production-active if its only material effect is removing scored evidence in a way that newly creates entry eligibility. `Only`/`Off` for score-affecting intents remain diagnostic/shadow unless separately approved as a scoring-policy change.
14. Router price intents are research/chart evidence only. Execution sizing, fills, stops, broker reconciliation, and order previews retain their existing verified money-path quote sources until separately architected.

6. Product behavior

Default owner experience

Settings → Data gains two views:

1. ****Routing Policy**** – the primary view.
2. ****Provider Health**** – capacity, credentials, freshness, errors, and diagnostics.

Routing Policy is a market-tabbed matrix:

Evidence	Mode	Preferred	Effective chain	Freshness	Status
--- ---	---	---	---	---	---
Fundamentals	Auto	Recommended	Finnhub → Webull → Yahoo → AV	14d	Healthy
Analyst consensus	Auto	Webull	Webull → Finnhub	3d	Healthy
Daily prices	Auto	Massive	Massive → EODHD → AV	EOD	Healthy
Sentiment	Auto	GDELT	StockTwits → GDELT → AV reserve	1d	Degraded

The provider dropdown lists only validated providers supporting that market and

intent. A provider must be `production\_eligible` for score-affecting intents; contract validity alone is insufficient. Unhealthy, disconnected, unentitled, or contract-invalid providers are disabled with a short reason.

Modes

- `Auto` – use the code-owned recommended chain, health, fresh cache, and capacity.
- `Prefer` – place the selected provider first, then use the safe Auto fallbacks.
- `Only` – use only the selected provider/cache; unavailable means unavailable. This is an advanced diagnostic option and displays a starvation warning. For a scored intent it is shadow-only.
- `Off` – intent is intentionally unavailable. For scored intents this is shadow-only because removing a dimension can change renormalized scores.

Owner should not manage quotas routinely

Provider limits and pacing defaults live in the code registry and are updated once per provider adapter. Settings shows them but does not require repeated adjustment. Advanced controls permit conservative overrides only:

- lower daily cap;
- larger minimum interval;
- larger reserved-call buffer;
- disable provider;
- shorten stale allowance.

The UI cannot raise a limit above the code-known ceiling unless the provider's entitlement/tier is also explicitly changed and contract-probed.

Change and rollback UX

- Save creates an immutable policy version; it does not mutate the active pointer.
- A confirmation summarizes changed rows and says “Applies next research run.”
- Activate atomically updates the active-policy pointer for one market.
- `Restore Auto` creates/activates a new all-Auto version.
- Recent versions show who changed them, when, and a compact diff.
- Disabling the router feature flag immediately returns that market to the legacy chain without deleting policies or cache.

7. C4 system context

C4Context

title System Context - Kairos Data Source Policy

Person(owner, "Vaibhav", "Chooses evidence-source policy")

System(kairos, "Kairos FinanceOS", "Research, scoring, paper and live workflows")

System_Ext(usProviders, "US Data Providers", "Webull, Finnhub, SEC, Massive, Yahoo, AV")

System_Ext(indiaProviders, "India Data Providers", "Kite, Upstox, Yahoo, NSE, GDELT")

SystemDb_Ext(supabase, "Supabase", "Policy, cache, queue, health, evidence ledger")

Rel(owner, kairos, "Reviews health and activates policy", "HTTPS")

Rel(kairos, usProviders, "Fetches allowlisted US evidence", "HTTPS/MCP")

Rel(kairos, indiaProviders, "Fetches allowlisted India evidence", "HTTPS")

Rel(kairos, supabase, "Freezes policy and persists evidence state", "PostgREST/RPC")

8. C4 containers

C4Container

title Container Diagram - Evidence Routing

Person(owner, "Vaibhav", "Owner")

System_Ext(providers, "External Providers", "Structured evidence APIs and MCP")

System_Boundary(kairos, "Kairos") {

Container(settings, "Settings Data UI", "Next.js Client", "Policy and health controls")


```

Container(api, "Policy and Diagnostics API", "Next.js Route Handlers", "Owner-gated reads and version activation")
Container(research, "Research and Prewarm", "Next.js Agents", "Requests canonical evidence intents")
Container(router, "Canonical Evidence Router", "TypeScript", "Deterministic cache, policy and fallback resolution")
Container(adapters, "Provider Adapters", "TypeScript", "Allowlisted typed provider contracts")
ContainerDb(db, "Evidence Control Store", "Supabase Postgres", "Versions, cache, leases, queue, health and ledgers")
}

```

```

Rel(owner, settings, "Selects Auto/preferred/off", "Browser")
Rel(settings, api, "Reads health and proposes versions", "JSON/HTTPS")
Rel(api, db, "Creates and activates policy versions", "PostgREST/RPC")
Rel(research, router, "Resolves evidence intent", "TypeScript")
Rel(router, db, "Reads frozen policy/cache and records attempts", "PostgREST/RPC")
Rel(router, adapters, "Calls validated capability", "Typed interface")
Rel(adapters, providers, "Fetches data", "HTTPS/MCP")
Rel(adapters, db, "Stores canonical cache result", "PostgREST")

```

9. Router components and dynamic flow

C4Component

title Component Diagram - Canonical Evidence Router

```

Container(research, "Research Agent", "Next.js", "Scoring workflow")
ContainerDb(db, "Supabase", "Postgres", "Durable control state")
Container_Ext(provider, "External Provider", "HTTP/MCP", "Evidence source")

Container_Boundary(router, "Evidence Router") {
  Component(intentCatalog, "Intent Catalog", "TypeScript", "Canonical schemas and applicability")
  Component(policyResolver, "Policy Resolver", "TypeScript", "Loads frozen version and provider order")
  Component(cacheResolver, "Cache Resolver", "TypeScript", "Fresh/stale provider snapshots")
  Component(capacityGate, "Capacity Gate", "Supabase RPC", "Atomic budget and pacing lease")
  Component(adapterRegistry, "Adapter Registry", "TypeScript", "Allowlisted provider capabilities")
  Component(validator, "Schema Validator", "TypeScript", "Canonicalizes and rejects bad payloads")
  Component(auditWriter, "Audit Writer", "TypeScript", "Provenance, attempts and health")
}

Rel(research, intentCatalog, "Requests intent")
Rel(intentCatalog, policyResolver, "Supplies capability requirements")
Rel(policyResolver, cacheResolver, "Checks ordered candidates")
Rel(cacheResolver, capacityGate, "Requests live-call lease on cache miss")
Rel(capacityGate, adapterRegistry, "Permits bounded call")
Rel(adapterRegistry, provider, "Calls allowlisted operation", "HTTPS/MCP")
Rel(adapterRegistry, validator, "Returns provider payload")

```

Rel(validator, auditWriter, "Records accepted/rejected evidence")

Rel(auditWriter, db, "Persists state")

Rel(cacheResolver, db, "Reads cache")

Rel(policyResolver, db, "Reads policy version")

Resolution sequence

1. Validate market/symbol/intent and determine structural applicability.
2. Load the policy version frozen at research-run start.
3. Build provider candidates from the code registry and policy mode.
4. Remove providers that are disabled, contract-invalid, unauthorized for the market/intent, circuit-open, or known to lack entitlement.
5. Read fresh canonical cache in provider order.
6. If no fresh hit, atomically reserve budget and pacing for at most two synchronous provider attempts within the caller's wall-clock budget.
7. Validate provider payload and canonical metric semantics.
8. Persist canonical cache + append-only evidence/attempt records before returning evidence that can influence scoring.
9. If live fetch cannot proceed, return policy-allowed stale evidence and enqueue a refresh job. Otherwise return typed unavailable status.
10. Never spin-wait for a provider slot inside Vercel.

10. Canonical intent catalog

Initial intent IDs are code constants, not arbitrary DB strings:

Intent	Markets	Canonical result	Default fresh TTL	Stale ceiling
`price.quote`	US/India	price, currency, market time	2 min/session	1 day, display only
`price.daily_bars`	US/India	adjusted OHLCV series	current EOD	3 trading days
`fundamentals.reported`	US/India	margin, ROE, EPS, revenue growth	14 days	45 days
`fundamentals.valuation`	US/India	TTM/forward P/E with basis	3 days	14 days
`analyst.consensus`	US	initially rating counts, target range, forecast EPS	3 days	14 days
`sentiment.news`	US/India	aggregate tone + sample count	1 day	3 days
`insider.transactions`	US/India	if supported normalized filings	1 day	7 days
`events.earnings`	US/India	event time and estimates	1 day	no stale past event
`events.corporate_actions`	US/India	dividend/split/action	1 day	no stale past ex-date
`macro.regime_inputs`	market-scoped	point-in-time macro values	provider cadence	one cadence

TTL values are policy defaults, not proof that the underlying observation is current. `observedAt`/`periodEnd` remain explicit.

11. Canonical contracts

type Market = "us" | "india";

type EvidenceIntent =

| "price.quote"

| "price.daily_bars"

| "fundamentals.reported"

| "fundamentals.valuation"

| "analyst.consensus"

| "sentiment.news"

| "insider.transactions"

| "events.earnings"

| "events.corporate_actions"

| "macro.regime_inputs";

type EvidenceQuality =

| "fresh"

| "stale"

| "partial"

```

| "conflict"
| "quarantined"
| "unavailable"
| "not_applicable";

interface FieldProvenance {
    providerId: ProviderId;
    providerField: string;
    basis: "ttm" | "forward" | "annual" | "quarterly" | "spot" | "eod";
    periodEnd?: string;
    observedAt?: string;
    retrievedAt: string;
    currency?: "USD" | "INR";
    unit: "fraction" | "currency" | "per_share" | "count" | "ratio" | "text";
}

interface CanonicalField<T> {
    value: T;
    provenance: FieldProvenance;
}

interface EvidenceEnvelope<T> {
    schemaVersion: "evidence-v1";
    market: Market;
    symbol?: string;
    intent: EvidenceIntent;
    quality: EvidenceQuality;
    payload: T | null;
    provenance: FieldProvenance[];
    providersAttempted: ProviderId[];
    policyVersionId: string;
    cacheState: "fresh" | "stale" | "miss";
    unavailableReason?: UnavailableReason;
    resolvedAt: string;
}

`UnavailableReason` is an enum including `auth_missing`, `entitlement_missing`,
`contract_invalid`, `rate_limited`, `daily_budget`, `timeout`, `provider_error`,
`schema_invalid`, `genuine_no_data`, `chain_exhausted`, `disabled_by_policy`, and
`not_applicable`.

## 12. Provider adapter contract

interface ProviderAdapter<I extends EvidenceIntent> {
    readonly providerId: ProviderId;

```

```

readonly intent: I;

readonly contractVersion: string;

fetch(request: ProviderRequest<I>, ctx: ProviderCallContext): Promise<ProviderResult<I>>;

validate(raw: unknown): ProviderResult<I>;

toCanonical(result: ProviderResult<I>): CanonicalPayloadByIntent[I];
}

```

Provider specs are code-owned:

```

interface ProviderSpec {

id: ProviderId;

label: string;

transport: "http" | "mcp";

markets: readonly Market[];

capabilities: readonly EvidenceIntent[];

dailyLimitState: "known" | "none" | "unknown";

dailyLimit?: number;

rateLimitState: "known" | "none" | "unknown";

rateLimitCalls?: number;

rateLimitWindowSeconds?: number;

minIntervalMs: number;

reserveCalls: number;

entitlementRequired: boolean;

credentialRef?: string;

trustTier: 1 | 2 | 3 | 4 | 5;

official: boolean;

}

```

DB overrides can only make these limits more conservative unless a code change updates the provider entitlement/ceiling.

13. Webull adapter specification

Allowed research tools

The research adapter allowlist contains only:

```

- `get_analyst_rating`
- `get_analyst_target_price`
- `get_stock_forecast_eps`
- `get_financial_indicators`
- later, after fixtures: `get_income_statement`, `get_balance_sheet`,
  `get_cash_flow`, `get_company_profile`, `get_financial_alert`,
  `get_stock_industry_comparison`

```

No tool name is read from DB or request input. No order tool is imported.

Analyst normalization

```

- Send `{ symbol, category: "US_STOCK" }`.
- Validate symbol/category echoed by Webull.
- Normalize string numeric fields with finite/range checks.
- Consensus weights: `strong_buy=100`, `buy=80`, `hold=50`,
  `under_perform=20`, `sell=0`.
- Compare the sum of rating buckets with `number`; if mismatch exceeds one, mark
  partial/conflict and preserve raw counts without producing a normalized score.
- Target price preserves mean/median/high/low/currency/effective date.
- Forecast EPS selects the nearest future `reported=false` fiscal period. It never

```

```

treats historical `actual` EPS as forecast EPS.

### Fundamentals normalization

- Request explicit `type` (`ANNUAL` initially) and bounded `count` (2-5).
- Parse each metric array by fiscal year/period; never flatten arrays generically.
- `net_margin` and `roe` are stored as fractions, not percentages.
- Preserve provider currency and period.
- Do not infer TTM from annual data. Add quarterly/TTM derivation only under a
  separate tested adapter version.
- Map only explicitly validated fields. Unknown numeric leaves remain diagnostics,
  not canonical scoring inputs.

### Caching and failure behavior

- Analyst TTL 3 days; financials TTL 14 days.
- Negative cache only deterministic `genuine_no_data` for 6 hours.
- Auth, entitlement, timeout, schema, and provider errors are not long negative
  cached; they update health/circuit state and may use stale canonical cache.
- One MCP session may be reused, but calls are sequential until Webull documents or
  tests prove concurrent calls on one session are safe.
- Token and provider error redaction remains mandatory.

### Scoring posture

Webull analyst evidence remains narrative/measurement-only. It cannot change the
deterministic analyst score until it has sufficient point-in-time outcomes and an
approved scoring architecture. Webull reported fundamentals may become a fallback
only after shadow comparison proves field semantics against existing sources.

## 14. Database design

Before implementation, list live migrations/tables/functions. Migration 176's
`provider_pacing`/`try_acquire_provider_slot` exists in production but is not
tracked in this repository; the implementation must first add an idempotent repair
migration matching the live schema. Never blindly recreate or rename it.

### `evidence_policy_versions` – immutable

| Column | Type | Rule |
|---|---|---|
| `id` | uuid PK | generated |
| `market` | text | checked `us`/`india` |
| `version` | int | unique per market |
| `router_enabled` | bool | false during rollout |
| `created_by` | uuid | owner |
| `created_at` | timestampz | audit |
| `change_note` | text | bounded owner note |

Rows never change after insertion. UPDATE/DELETE is trigger-blocked.

### `active_evidence_policy` – mutable pointer

| Column | Type | Rule |
|---|---|---|
| `market` | text PK | `us`/`india` |
| `policy_version_id` | uuid FK | same-market immutable version |
| `activated_by` | uuid | owner |
| `activated_at` | timestampz | audit |

The activation RPC updates this pointer under a per-market advisory lock. A trigger
or RPC validation prevents a US pointer from referencing an India policy. Policy
history remains immutable and rollback means pointing to a prior version or creating
a new all-Auto version. Activation also validates that the target version has exactly
one valid rule for every required intent before changing the pointer.

### `evidence_policy_rules` – immutable children

| Column | Type | Rule |
|---|---|---|
| `policy_version_id` | uuid FK | cascade prohibited |
| `intent` | text | code-validated enum |
| `mode` | text | `auto`/`prefer`/`only`/`off` |
| `preferred_provider` | text nullable | required for prefer/only |
| `max_age_seconds` | int | conservative bounded range |
| `stale_max_seconds` | int | >= max age, bounded |
| `max_sync_attempts` | int | 0-2 |
| `advanced_config` | jsonb | future compatible, size bounded |

Primary key `(policy_version_id, intent)`.

### `provider_runtime_config` – current operational controls

| Column | Type | Rule |
|---|---|---|

```

```
| `provider_id` | text PK | must exist in code registry at runtime |
| `enabled` | bool | false blocks live calls, cache may remain visible |
| `daily_limit_state` | text | `known`/`none`/`unknown` |
| `daily_limit_override` | int nullable | cannot exceed code ceiling |
| `rate_limit_state` | text | `known`/`none`/`unknown` |
| `rate_limit_calls_override` | int nullable | cannot exceed code ceiling |
| `rate_window_seconds_override` | int nullable | cannot shrink code window |
| `min_interval_ms_override` | int nullable | cannot be lower than code floor |
| `reserve_calls_override` | int nullable | >= code minimum |
```

Owner-read; writes only through owner-gated API using the service role after code validation. No direct authenticated writes.

`provider\_capability\_status` – provider/intent maturity

```
| Column | Type | Rule |
|---|---|---|
| `provider_id`/`market`/`intent` | text | composite PK |
| `contract_state` | text | `unverified`/`valid`/`invalid` |
| `maturity_state` | text | `discovered`/`contract_valid`/`shadow_validated`/`production_eligible` |
| `entitlement_state` | text | `unknown`/`active`/`inactive` |
| `contract_version` | text | must match code adapter |
| `last_probe_at` | timestamptz | operational evidence |
| `last_shadow_evaluation_id` | uuid nullable | proof link |
```

Maturity is capability-specific: Webull analyst can be eligible while Webull fundamentals remains shadow. The owner API may disable a capability, but promotion to `production\_eligible` requires a code-known adapter version plus a passing shadow evaluation. A DB row can never create a capability absent from the code registry.

`evidence\_policy\_evaluations` – append-only shadow proof

Stores candidate policy version, baseline production version, market, symbol/run sample, coverage delta, schema failures, provider disagreement, score delta, entry-eligibility flips, call usage, and pass/fail reasons. Production activation of a score-affecting policy requires a passing evaluation and explicit owner action.

`evidence\_cache\_v2` – mutable current cache

```
| Column | Type | Rule |
|---|---|---|
| `market`/`symbol`/`intent`/`provider_id` | text | composite identity |
| `request_fingerprint` | text | contract + normalized request hash |
| `schema_version` | text | canonical schema |
| `payload` | jsonb | canonical only, size bounded |
| `provenance` | jsonb | canonical field/source sidecar; array only |
| `quality_state` | text | fresh/partial/conflict/quarantined |
| `observed_at`/`period_end` | timestamptz/date | semantic time |
| `fetched_at`/`expires_at`/`stale_until` | timestamptz | cache policy |
| `currency`/`basis` | text | checked canonical values |
| `payload_hash` | text | integrity/dedup |
```

Primary key `(market, symbol, intent, provider\_id, request\_fingerprint)`. Market-wide intent uses a reserved symbol such as `\_\_MARKET\_\_`, never null ambiguity.

`provider\_call\_ledger` – append-only operational audit

Records provider, intent, market, symbol hash/plain symbol, run ID, policy version, cache outcome, lease outcome, started/completed timestamps, latency, HTTP/MCP status, normalized error code, response bytes, and contract version. It stores no tokens, URLs containing keys, headers, or raw error bodies.

`provider\_refresh\_jobs` – durable queue

One active job per `(market,symbol,intent,provider\_id,request\_fingerprint)`. Claiming uses a service-role-only `claim\_provider\_refresh\_jobs()` RPC with `FOR UPDATE SKIP LOCKED`, lease expiry, bounded batch size, attempt count, and `next\_attempt\_at`. Jobs dead-letter after bounded retries and raise System Health issues.

Existing tables retained

- `evidence\_records` remains the immutable decision evidence ledger.
- `provider\_budget` may remain as the daily rollup but becomes derived from or atomically updated alongside `provider\_call\_ledger`.
- `av\_cache` remains during compatibility rollout; it is not the router cache.
- `provider\_pacing` remains the atomic cross-process lease table.

RLS and grants

- Enable RLS on every new table.
- Owner-email SELECT policy for policy/health/cache summaries where UI needs it.
- No `anon` access.
- No direct `authenticated` INSERT/UPDATE/DELETE.
- Service role performs agent writes.
- SECURITY DEFINER RPCs set `search\_path`, validate all enums/ranges, revoke PUBLIC,

```

    and grant only `service_role`.
- Append-only triggers block UPDATE/DELETE on policy versions, rules, evaluations,
  and call ledger;
  only the active-policy pointer is mutable through its service-role RPC.

## 15. Policy APIs

### `GET /api/settings/data-routing?market=us`

Owner-only. Returns active version, rules, resolved effective chains, provider
availability, latest unactivated version, and change history. It never returns
credential values.

### `POST /api/settings/data-routing/versions`

Owner-only. Validates the complete matrix server-side and creates a new immutable
version without activating it. A service-role RPC allocates the next market version
under an advisory lock and inserts the header + complete rule set in one transaction.
It rejects unsupported market/intent/provider combinations.

### `POST /api/settings/data-routing/activate`

Owner-only. Calls atomic activation RPC. Activation never starts a research run and
never calls a provider. It affects only future runs. For a score-affecting change,
the RPC requires a passing `evidence_policy_evaluations` row against the currently
active baseline and explicit owner confirmation. Non-scored display/narrative policy
may activate after contract validation without the score-delta gate.

### `POST /api/settings/data-routing/restore-auto`

Owner-only. Creates and activates a new default version; does not mutate history.

### `GET /api/settings/data-providers`

Upgrade the existing endpoint to report daily-limit state, rate-limit state, known
limits, entitlement, contract status, cache hit rate, last success, consecutive
failures, circuit state, and source freshness. Remove the claim that every null
daily budget is uncapped.

## 16. Capacity, pacing, and health

### Atomic call admission

One RPC combines daily-budget reservation and minimum-interval lease. Counting a
call before external spend is conservative. The ledger distinguishes `reserved`,
`started`, and `completed` so unused reservations can be measured but never reused
in a race-prone way.

### Circuit breaker

Per provider + intent:

- open after 3 consecutive contract/auth/schema failures;
- transient timeout/5xx opens only after a higher threshold;
- cooldown is bounded and visible;
- one half-open probe is leased atomically;
- genuine no-data for one symbol does not degrade global provider health.

### Health states

`healthy | degraded | exhausted | disconnected | unentitled | contract_invalid |
unknown`. Health affects Auto routing but never alters a frozen canonical payload.

### Telemetry

- cache hit/stale/live/miss rates;
- success and schema-rejection rates by provider/intent;
- p50/p95 latency;
- fresh coverage by market/intent;
- calls and reserved headroom;
- fallback frequency and provider disagreement;
- refresh queue depth/oldest age/dead letters.

Existing `data-availability:<market>:<dimension>` alerts remain. Add provider-specific
issues only when actionable; avoid one issue per symbol.

## 17. Fundamental merging and conflict rules

Fundamentals are not a first-provider-wins blob. Canonical fields resolve separately:

- provider must support the required basis;
- freshest valid field in policy order wins;
- no blending of annual/quarterly/TTM/forward values;
- source-specific fields may coexist in one envelope with field provenance;
- material same-basis disagreement marks `conflict` and excludes that field until

```

- a trusted-tier rule resolves it or the conflict clears;
- a partial envelope is available only if it satisfies the intent's minimum field contract; otherwise it is unavailable and scoring renormalizes.

Compatibility phase maps canonical fields into the legacy AV-shaped object only at the scorer boundary. The mapping includes a provenance sidecar and is deleted after the scorer consumes canonical fields directly.

18. Security and abuse analysis

- ****SSRF/tool injection:**** provider URLs and MCP tool names exist only in code.
- ****Prompt injection:**** discard unstructured provider prose; render bounded canonical values through templates. Filing/news text remains in its separate sanitized path.
- ****Prototype/property abuse:**** validators create new plain canonical objects and use own-property checks; no arbitrary dotted-path traversal over provider objects.
- ****Credential leakage:**** service role and OAuth tokens remain server-side; logs store normalized error codes, never raw authorization/URL/header/error payloads.
- ****Payload bombs:**** response byte limit, JSON depth/array count bounds, timeout, and per-tool result schemas.
- ****Cross-market contamination:**** cache and policy keys include market; adapters check provider market/category and canonical currency.
- ****Configuration escalation:**** Settings cannot add order capability, tool names, scopes, URLs, or raise unknown quotas to unlimited.
- ****Money-path isolation:**** router modules are imported by research/prewarm/read APIs, not broker adapters or `execute-order.ts`.

19. Rollout and reversibility

Phase 0 – schema reality and fixtures

1. Verify live tables/functions/migrations, especially out-of-band migration 176.
2. Capture sanitized contract fixtures for the Webull read tools and representative US symbols: AAPL, MSFT, JPM, BRK.B, one ETF, one no-data symbol.
3. Add tests before changing the adapter.

Phase 1 – repair Webull without scoring impact

1. Add required `US\_STOCK` category and explicit parsers.
2. Persist canonical Webull analyst/fundamental cache.
3. Keep analyst narrative/measurement-only.
4. Keep Webull fundamentals shadow-only; compare with Finnhub/Yahoo for at least five trading days and report coverage/disagreement.
5. Remove or narrow the diagnostic tools route.

Phase 2 – canonical core behind flags

1. Add canonical contracts, intent catalog, adapter registry, durable cache, call ledger, pacing repair migration, and refresh queue.
2. Seed one all-Auto policy per market with `router\_enabled=false`.
3. Dual-resolve legacy and canonical routes; canonical output is logged only.

Phase 3 – Settings policy control

1. Add owner APIs and Routing Policy/Provider Health UI.
2. Activate policy versioning while router remains shadow-only.
3. Browser-test desktop/mobile, disabled options, version diff, restore Auto, and unknown-quota labels.

Phase 4 – controlled research cutover

1. Enable router for paper research in US only.
2. Compare availability, canonical scores, provider calls, and decision deltas.
3. Enable India independently after its own evidence.
4. No live-autonomy change; signals continue through all existing gates.

Phase 5 – cleanup

1. Move remaining direct provider calls behind adapters.
2. Replace inaccurate evidence source labels.
3. Retire provider-first abstraction and `av.cache` only after no callers remain.
4. Update architecture/database/cron docs and system map.

Disable/rollback

Set the per-market router feature flag false. The legacy chain resumes next run; policy versions, ledgers, and cache remain for audit. No migration rollback or data deletion is required.

20. Required tests

Unit/contract

- Every adapter fixture validates and canonicalizes expected fields.
- Webull missing category fails test; adapter always supplies `US\_STOCK`.
- EPS chooses nearest future unreported `est` row.

- Analyst denominator includes `under\_perform`; count mismatch becomes conflict.
- Financial period arrays preserve basis/period/currency and fraction scale.
- Unknown quota is not treated/displayed as unlimited.
- Unsupported provider/intent/market combinations are rejected.
- Forward and TTM fields never substitute for one another.
- Raw untrusted keys/prose cannot enter canonical objects/prompts.

Router

- Frozen policy is stable when owner activates a new version mid-run.
- Auto/prefer/only/off produce deterministic candidate orders.
- Only/off on scored intents cannot production-activate without separate approved scoring-policy treatment.
- Fresh cache avoids provider calls.
- No lease returns stale/enqueues; never spin-waits.
- Max two synchronous attempts and hard wall-clock bound.
- Market/currency keys cannot cross-hit cache.
- Contract-invalid provider is skipped in Auto.
- Provider-only mode returns unavailable rather than hidden fallback.
- Failure to write trade-affecting evidence blocks that evidence from scoring.

Database/security

- One active-policy pointer per market under concurrent activation.
- Score-affecting activation fails without a passing evaluation against the current baseline, and a stale evaluation cannot authorize a newer version.
- Append-only tables reject update/delete.
- Queue claims cannot double-lease.
- anon/authenticated writes are denied; owner reads work.
- SECURITY DEFINER functions have fixed search path and service-role-only execute.
- No secrets/raw token-bearing errors are stored.

Integration/product

- Research run stamps policy version and exact field provenance.
- Shadow evaluation reports entry-eligibility flips, not only average score delta.
- Availability mask and weight renormalization match legacy semantics.
- Webull outage does not change India and cannot reach any order path.
- Settings dropdown contains only validated market-capable providers.
- `Restore Auto` creates a new auditable version.
- Browser verifies responsive table/control layout and no text overlap.

Release gates

- `npx tsc --noEmit`
- full unit/integration test suite
- `npm run build`
- dependency/security audit
- schema verification against live Supabase after migration
- read-only production contract probes
- Playwright Settings checks at desktop and mobile widths
- explicit proof: Webull `orderCapable=false`, no order scope, no order calls

21. Product/trader acceptance criteria

The feature is successful when:

1. Vaibhav can leave every row on Auto and never rebalance quotas manually.
2. Switching a provider needs one policy activation, not agent code edits.
3. Adding a tool/provider requires one adapter + registry entry + fixtures.
4. Every research decision explains which sources and periods supplied its fields.
5. Provider degradation is visible before it silently starves a scoring dimension.
6. Provider changes do not produce phantom score jumps from unit/basis drift.
7. US and India can be enabled, disabled, and rolled back independently.
8. No data-source control can place or authorize a trade.

22. Recommended initial defaults

Keep these as reviewed defaults, not permanent truths:

- ****US reported fundamentals:**** Auto; Finnhub first while Webull shadows. Promote Webull fallback only after comparison passes. SEC remains excluded until its period/concept derivation is corrected.
- ****US analyst consensus:**** Webull primary, narrative/measurement-only.
- ****US bars/quotes:**** existing Massive-first chain.
- ****India fundamentals:**** Yahoo monitored single-source with durable stale cache; do not pretend resilience is solved.
- ****India bars/quotes:**** existing Upstox/Kite/Yahoo roles, based on entitlement.
- ****Sentiment/insider/macro:**** preserve current chains during router shadow phase.
- ****All policy modes:**** Auto.
- ****Router flags:**** off until dual-run evidence passes separately per market.

23. Riskiest assumptions to validate first

1. Webull financial ratios remain available under the owner's authorized capability

- and subscription during unattended cron runs, not just interactive probes.
- 2. Provider values that share a label also share the intended period/basis/unit.
- 3. The legacy scoring path can be compatibility-mapped without changing historical score semantics before the canonical scorer migration.

The first implementation proof should therefore be a five-day, no-score-impact Webull contract/coverage comparison with durable fixtures and provenance.

24. Review questions for Claude

1. Is immutable per-market policy versioning sufficient for reproducible research, or should each run copy the complete rule JSON into its run ledger as well?
2. Should provider call admission extend the existing `provider\_pacing` RPC or use a new combined budget+pacing RPC after live schema verification?
3. Are the canonical intent boundaries narrow enough to prevent metric-basis drift?
4. Is two synchronous provider attempts the correct Vercel bound, with remaining work delegated to the refresh queue?
5. Should Webull fundamentals remain shadow for five trading days or require a larger symbol/sector sample before becoming a fallback?
6. Does any existing agent still consume direct provider payloads in a way this migration order misses?

```
# Source: features\decision-review\FEATURE_ARCHITECTURE.md
# Decision Review / Counterfactual – Feature Architecture

> Status: **IMPLEMENTED (Phase A – measure-only, read-only)**. No migration, no writes.
> Last updated: 2026-07-15
>
> **Build note (2026-07-15):** Shipped as a read-only tab in Research Journal.
> Files: `app/api/decision-review/route.ts` (owner-gated SELECT-only join of
> `decision_observations` × `observation_labels` × `v_decision_quality`) and
> `components/dashboard/DecisionReviewPanel.tsx` (4th tab:
> `Daily Funnel | Evolution | Score Tracker | Decision Review`,
> `/dashboard/research-journal?tab=review`).
>
> **DB-reality deviations from the draft below:**
> - **No `v_decision_review` view created** – used the API-only deterministic join
>   ($9 open decision 3, "API-only path avoids a migration entirely"). Read-only,
>   no migration, honors the hard "no migration" boundary.
> - **Horizons surfaced are data-driven, not fixed at 2/5/20.** As of build, only
>   `horizon_days` 2 and 5 have matured in `observation_labels`; 10/20 back-fill as
>   they mature. The API returns the canonical horizons that actually have labels;
>   the "1d" question ($4) is moot – 1d is not in the pipeline and none was added.
> - **20-floor gate is live** (`MATURED_FLOOR = 20`): a cohort with fewer matured
>   outcomes at a horizon renders "N/20 matured – insufficient sample, not yet
>   actionable"; at/above the floor it shows hit-rate + avg fwd + avg alpha.
> - **Per-symbol attribution is decorative**, always tagged "n=1 · ILLUSTRATIVE"
>   (locked owner decision). No LLM prose pass was wired (kept fully deterministic).
> Update this file when: the per-symbol decision-review data contract, the
> attribution method, or the UI surface changes; keep it aligned with
> `docs/arch/09-learning-loop.md` (Performance Truth / P1 gate) and
> `features/edge-factor-discovery/FEATURE_ARCHITECTURE.md` (aggregate EdgeIC).
```

1. One-line intent

For a single symbol we scored on date **D**, show: **what we scored** (analyst_score + the 5 dimension scores), **how the stock actually moved** over the next **1d / 5d / 20d** (market-local adjusted close), **the gap** (score-implied direction vs realized), **which dimension most plausibly pushed the score wrong**, **how confident** that read is, and a **plain-English** "what we'd learn."

This is the missing **per-symbol post-mortem**. We already measure **aggregate** predictive power (EdgeIC on the broad universe; `v\_decision\_quality` per-decision data quality). Decision Review is the per-decision, per-symbol surfacing of the **already-existing** `decision\_observations` × `observation\_labels` ledger. It is the concrete UI for the "opportunity-level IC" work that `docs/arch/09-learning-loop.md` and the P1 gate already name as the next measurement layer.

It aligns with item **#4 "Counterfactual trade journal"** on the native roadmap in `features/external-research-integrations/DEEP\_CANDIDATE\_CAPABILITY\_AUDIT.md` ("Advisory history only; never changes ledgers, positions, prices, or trades").

2. Hard boundaries (non-negotiable)

- **Read-only / advisory / measurement-only.** Decision Review reads existing records and renders them. It **never** writes to `agent\_signals`, `paper\_trades`, `learning\_priors`, `strategy\_versions`, weights, genome, or any order path.
- **Not a parallel truth layer.** It does **not** create a new provenance/outcome store. It reads the append-only `decision\_observations` ledger (059) and the

matured `observation\_labels` (060) – the *same* records the LearnerAgent and Performance Truth Layer read. If a number is shown here, it came from those tables (or a deterministic view over them), never from a fresh recomputation that could diverge from the ledger.

- **LearnerAgent remains the only thing that changes weights**, on its own weekly evidence, behind the Phase-0 (10+ closed trades) gate. Decision Review can *inform* the owner and *feed the same evidence tables* the LearnerAgent already reads, but it proposes nothing and mutates nothing.
- **No LLM sets any number.** Every score, return, gap, and attribution value is deterministic SQL/arithmetic over the ledger. An LLM may only render the already-computed numbers into the one-line "what we'd learn" *prose* (optional, and clearly labeled as narration of fixed numbers – same posture as existing thesis text).
- **Market-local, never mixed.** US decisions are compared to USD prices and SPY; India decisions to INR prices and ^NSEI. A view/API filtered by `market` never joins a USD entry price to an INR forward path. Currency is carried on every row (`decision\_observations.currency`).

3. Data sources – everything needed already exists

3.1 `decision\_observations` (migration 059) – the point-in-time score

Append-only (a mutation trigger `dobs\_block\_mutation` blocks UPDATE/DELETE). One row per candidate scored by ResearchAgent (filled OR rejected). Columns this feature reads:

Column	Use in Decision Review
id, ts, market, symbol	Row identity, market scope, score date **D**
analyst_score	The headline score under review
fundamental_score, technical_score, sentiment_score, macro_score, insider_score	Per-dimension scores (the attribution inputs)
direction	long / short / hold as scored (defines the "was it right?" sign)
weights_used (jsonb)	Dimension weights actually applied → dimension contribution = weight × (score-50)
availability_mask (jsonb)	Which dimensions were real vs unavailable – an unavailable dim cannot be "wrong"
features (jsonb)	Raw sub-features (already parsed by `v_decision_quality`)
price_at_decision, currency	Entry basis (native currency) – the anchor for forward returns
entry_eligible, action, score_threshold	Whether we would have acted; held vs skipped context
signal_id	Link to `agent_signals` when a signal was written
mandate_id (migration 135)	Mandate/benchmark context (Swing US 2-20d → V00/SPY; Swing India 2-20d → ^NSEI)

3.2 `observation\_labels` (migration 060) – the matured forward outcome

One row per `(observation\_id, horizon\_days)`, written **only after maturity** by the `label-maturation` cron (`app/api/agents/label-maturation/route.ts`) so features can never see the future. Horizons already computed: **2, 5, 10, 20 trading days**.

Column	Use
horizon_days	2 / 5 / 10 / 20 – maps to the "next 1d/5d/20d" UI (see §4 note on the 1d question)
fwd_return	(exit-entry)/entry – 10bps` cost haircut – the actual move
benchmark_return	Same-window SPY (us) / ^NSEI (india) return
benchmark_neutral_return	fwd_return – benchmark_return – alpha vs beta separation
max_adverse_excursion / max_favorable_excursion	Path risk inside the window (was the thesis ever right/wrong intraperiod)
entry_price, exit_price	Displayed anchors
matured_at	Freshness / "not matured yet" state

3.3 `v\_decision\_quality` (migration 122) – per-decision dimension bookkeeping

Already computes, per observation: `applicable\_dims`, `real\_dims`, `missing\_dims`, `degraded\_dims`, `decisive\_dim` (highest-weight real dimension), `data\_confidence`, `confidence\_band` (high/med/low). Decision Review **reuses** this for the confidence column and to gate attribution to *real, non-degraded* dimensions only.

3.4 `edge\_ic\_history` / EdgeIC (migrations 132; `lib/edges/ic.ts`) – the aggregate we EXTEND

EdgeIC measures cross-sectional rank-IC of *edges* on the *broad candle universe*. Decision Review is the **complementary** view: not "does this factor rank winners across the universe" but "for this one decision on this one symbol, how did our composite score line up with what happened". Decision Review **references** EdgeIC as context ("technical-momentum edge IC at 20d = 0.04, t=2.1") but does **not** recompute or duplicate it, and never writes to `edge\_ic\_history`.

3.5 Forward-price source (for spot-checking / rendering only)

The forward returns themselves come from `observation\_labels` (already matured). If the UI wants to *render a small price sparkline* from D to D+20, it reads cached candles the same way maturation does – `price\_cache` then `fetchUSCandles` fallback (`lib/data/candles.ts`) for US, `fetchIndiaCandles` (`lib/india-data.ts`) for India. The **numbers of record** are always the label row, not a fresh fetch, to prevent drift from the ledger.

3.6 New table/column decision – **prefer none; one optional derived view**

- **No new base/truth table.** The ledger already holds score + outcome.
- **No new provenance layer.** `decision\_observations` is the provenance.
- **One optional read-only VIEW** `v\_decision\_review` (deterministic join of 3.1 × 3.2

× 3.3, per market, per horizon) MAY be added to keep the query in one audited place. A view is not a truth layer – it stores nothing. ****Open question in §9**** is whether even this is worth it vs. an API-only join.

- The "opportunity-level IC" columns the roadmap already reserved (``strategy_evaluations.opp_*``, null until P1 per ``docs/arch/09-learning-loop.md``) are the correct home for any ****aggregate**** roll-up this feature surfaces – extend those, do not invent siblings.

4. Point-in-time forward returns (no look-ahead, market-local)

The maturation cron already implements this correctly; Decision Review ****consumes**** it and must not re-derive it differently.

1. ****Entry anchor**** = ``decision_observations.price_at_decision`` (the close/quote used at scoring time), else the first candle on/after date ****D****. Native currency.
2. ****Window**** = the first ``H`` candles ****strictly after**** the entry date (``c.date > entryDate``) – so D itself is never counted, and a horizon is only labeled once ``H`` trading days have actually elapsed (maturity). This is what prevents look-ahead: an un-elapsed horizon has ****no**** label row and renders as "maturing".
3. ****Return**** = ``(exit - entry)/entry - 0.001`` (10 bps round-trip cost haircut, ``LABEL_COST_HAIRCUT``).
4. ****Benchmark**** = SPY for us, ``^NSEI`` for india, ****same window****, same rule → ``benchmark_return``; ``benchmark_neutral_return`` = ``fwd - benchmark`` isolates alpha.
5. ****Market isolation**** = every query filters ``market`` and reads ``currency``; a US label is never compared to an INR benchmark and vice-versa. Mandate benchmark (``V00/^NSEI``) comes via ``mandate_id``.

****The "1d" question.**** Labels exist at 2/5/10/20. The task asks for 1d/5d/20d. Two honest options (see §9): (a) surface ****2d/5d/20d**** and label the near bucket ``-1-2d`` (zero new pipeline work, no new provider calls); or (b) add ``horizon_days = 1`` to the ``HORIZONS`` array in the maturation cron (one-line change, still measure-only, back-fills naturally). Recommendation: ****a)**** for the first cut – 1d is the noisiest, least decision-relevant horizon for a 2-20d swing mandate, and adding it risks implying precision we don't have.

5. The per-symbol view (what the screen shows)

For a chosen ``(symbol, market)`` and a chosen decision date ****D**** (or the latest matured decision), one ****Decision Review card**** per horizon, plus a symbol-level history table.

5.1 Header – the decision under review

- Symbol, market badge, date ****D****, ``analyst_score``, ``direction``, whether it was ``entry_eligible`` / acted / skipped, ``confidence_band`` (from ``v_decision_quality``), mandate + benchmark.

5.2 Score vs actual move (per horizon 2/5/20)

- ****Scored**** ``analyst_score`` and score-implied direction.
- ****Actual**** ``fwd_return``, ``benchmark_neutral_return``, MAE/MFE (path).
- ****Gap**** a deterministic, sign-aware gap metric:
 - Directional hit/miss: did ``sign(direction)`` match ``sign(benchmark_neutral_return)``?
 - Magnitude gap: score mapped to an ``ordinal`` expectation bucket (high score ⇒ expect top-tercile forward return) vs realized tercile – ****ordinal, not a fabricated regression**** (we deliberately avoid implying the score is a calibrated return forecast; it isn't).

5.3 Per-dimension attribution – "which dimension was most wrong"

Deterministic, and ****honest about its limits****:

- Only dimensions that were ****real and non-degraded**** (``availability_mask`` + ``v_decision_quality.degraded_dims``) are eligible – an unavailable dimension cannot be blamed.
- ****Contribution**** of dimension ``d`` to the decision = ``weights_used[d] × (score_d - 50)`` (how far, and in which direction, ``d`` pushed the composite off neutral).
- ****"Most wrong"***** = the eligible dimension whose contribution most ****opposed**** the realized ``benchmark_neutral_return`` sign, weighted by ``|contribution|``. Example: score was pushed long mostly by ``sentiment`` (+high contribution), stock fell vs benchmark ⇒ sentiment is flagged as the most-wrong dimension for this decision.
- ****"Most right"***** symmetric, for balance.
- Displayed as a small horizontal contribution bar, colored by whether each dim helped or hurt ****this realized path**** – explicitly labeled "for this one outcome," never as a causal weight-change recommendation.

5.4 Confidence

- ``confidence_band`` from ``v_decision_quality`` (data completeness), ****plus**** a sample/maturity caveat: a single symbol-decision is ``n=1``. Per-symbol attribution is always shown with an explicit ``n=1`` anecdote – see aggregate below" marker.

5.5 Plain-English "what we'd learn"

- A templated, deterministic sentence assembled from the computed fields, e.g.:
****"On 2026-06-30 we scored NVDA 78 (long), driven mainly by technical (+12) and sentiment (+9). Over 20d it returned -4.1% vs SPY (-5.3% alpha). Sentiment most opposed the outcome. This is one decision (n=1); the sentiment dimension's 20d IC**

```

    across all decisions is +0.01 (t=0.7, not significant) – no weight change is
    warranted on this evidence."*
- Optional LLM pass only *rephrases* this fixed string; it introduces no new numbers.

### 5.6 Symbol history + aggregate roll-up (the honest layer)
Because n=1 per decision is meaningless, the symbol page's default emphasis is the
**aggregate over that symbol's decisions** (and, more importantly, over the
**regime/dimension cohort**): hit-rate, mean `benchmark_neutral_return`, and a
per-dimension "how often did this dim's push agree with the outcome" – only shown once
the cohort clears a sample floor (reuse the Performance Truth **20-trade / P1** honesty
rule; below floor → `insufficient_sample`, no fabricated precision).

```

```
---
```

```
## 6. C4 sketch
```

```
### 6.1 Context
```

flowchart TB

```

Owner([Owner / operator]) -->|reads, never edits| DR[Decision Review\ntab in Research Journal]
DR -->|read-only SQL| LEDGER[(decision_observations 059\nobservation_labels 060\nv_decision_quality 122)]
MAT[label-maturation cron\nnighly, existing] -->|writes matured labels| LEDGER
RA[ResearchAgent\nexisting] -->|append-only score rows| LEDGER
DR -.reads for context only.-> EIC[(edge_ic_history 132)]
DR -.n=1 caveat points owner to.-> PT[Performance Truth / P1\nstrategy_evaluations.opp_*]
DR -. never writes .-x LEARN[LearnerAgent / weights / orders]

```

```
### 6.2 Container
```

flowchart LR

```

subgraph UI[Next.js dashboard]
    TAB[/dashboard/research-journal?tab=review\nDecisionReviewPanel.tsx/]
end

subgraph API[Next.js route handlers - read only]
    EP[GET /api/decision-review\n?market&symbol&date&horizon]
end

subgraph DB[Supabase Postgres]
    V[(v_decision_review\noptional deterministic view)]
    O[(decision_observations)]
    L[(observation_labels)]
    Q[(v_decision_quality)]
end

TAB --> EP --> V
V --> O & L & Q
EP -. optional sparkline .-> PC[(price_cache / providers)]

```

```

Everything in the API/DB path is `SELECT`-only. No route in this feature issues
`INSERT`/`UPDATE`/`UPSERT` to any table.

```

```
---
```

```
## 7. Where it lives in the UI
```

```

A **fourth tab in the existing Research Journal** page
(`app/dashboard/research-journal/page.tsx`), sibling to **Score Tracker**:
`Daily Funnel | Evolution | Score Tracker | Decision Review`

```

- Reached via ``/dashboard/research-journal?tab=review`` (mirrors the existing ``?tab=scores`` redirect pattern; a ``/dashboard/decision-review`` route may redirect here for discoverability).
- **Market-scoped** via the existing market toggle/param (``?market=us|india``); the panel never renders mixed-market rows.
- Deep-link from Score Tracker: "review this decision" on any historical score point opens the same symbol/date in Decision Review. Deep-link from the Research Journal Daily Funnel row and from ``/dashboard/symbol/[symbol]``.
- Reuses existing dashboard chrome (``PageHeader``, theme tokens ``T``, tab button styling)
 - no new layout system, per Drift Prevention.

8. Phased rollout (measure-only first)

Phase A – Surface (measure-only, no new pipeline).

Read ``decision_observations × observation_labels × v_decision_quality``; render the per-horizon card (2/5/20), score-vs-actual, and the symbol history table. Attribution shown but heavily n=1-caveated. **No migration** beyond the optional ``v_decision_review`` view. Ships behind the same measure-only posture as EdgeIC.

Phase B – Attribution + aggregate honesty.

Add the deterministic per-dimension contribution/most-wrong logic and the cohort/regime aggregate roll-up gated by the 20-trade/P1 sample floor. Wire the "what we'd learn" template. Still writes nothing.

Phase C – Optional 1d horizon + Performance Truth link.

If owner wants true 1d, add ``horizon_days = 1`` to the maturation ``HORIZONS`` (one-line, measure-only). Surface the aggregate per-dimension agreement into the reserved ``strategy_evaluations.opp_*`` columns **as read-through display**, feeding the **same** evidence the `LearnerAgent` already consumes – never a new recommendation channel.

Never a phase: auto-adjusting weights from Decision Review. That stays the `LearnerAgent`'s weekly, gated job.

9. Acceptance tests

1. **Read-only proof:** grep the feature's API + panel for any write verb (``insert|update|upsert|delete|.rpc(`)` → zero hits against ledger/weight/order tables).
2. **No look-ahead:** a decision whose newest horizon hasn't elapsed shows "maturing" and **no** fabricated return; forcing a render never fetches post-cursor candles for the number of record (it reads only the existing label row).
3. **Market isolation:** a US symbol page never displays an INR price or an `^NSEI` benchmark; India never shows SPY. Currency badge matches ``decision_observations.currency``.
4. **Ledger fidelity:** every number on the card equals the corresponding ``decision_observations`` / ``observation_labels`` field byte-for-byte (no recompute drift).
5. **Attribution eligibility:** a dimension with ``availability_mask[d] = false`` or in ``degraded_dims`` is never flagged "most wrong."
6. **Sample honesty:** with a below-floor cohort, the aggregate panel shows ``insufficient_sample``, not a number (mirrors Performance Truth's 20-trade rule).
7. **Determinism:** same inputs → same card on repeat load; the optional LLM prose pass, if disabled, leaves every number and the templated sentence intact.
8. **Boundary:** disabling/deleting this feature changes no ledger row and no weight – it is pure projection.

10. Riskiest assumption (call it out loud)

Forward-return attribution ≠ causation, and per-symbol samples are tiny (n=1).

Flagging a dimension as "most wrong" for a **single** realized path is an anecdote: the stock's move is dominated by market/sector beta and idiosyncratic noise, not by which sub-score we over-weighted. If the UI presents per-symbol attribution as actionable, the owner (or a future `LearnerAgent` reading it) could chase noise – exactly the overfitting the Phase-0 "10+ closed trades" rule and the EdgeIC t-stat hurdles exist to prevent.

Mitigation (locked into the design): per-symbol/per-decision attribution is always labeled n=1 and is decorative; the **decision-relevant** number is the **aggregate/regime/dimension cohort** roll-up, shown only above the Performance-Truth sample floor, with the aggregate EdgeIC t-stat displayed alongside so a non-significant dimension is visibly non-significant. Decision Review **informs**; it never proposes, and weight change stays the `LearnerAgent`'s gated, aggregate-evidence job.

11. Open decisions for owner / Claude

1. **(Biggest)** Attribution methodology & how loudly to show per-symbol "most wrong": is the deterministic `weight × (score-50)` vs realized-sign contribution rule the right definition, and should per-symbol attribution be **shown** at all in Phase A or deferred until the aggregate cohort layer (Phase B) exists to anchor it? (This is the single point where the feature could either become genuinely useful or quietly encourage overfitting.)

2. Add `horizon\_days = 1` now (true 1d) or surface 2/5/20 and label the near bucket "-1-2d"? (Recommendation: defer 1d.)
3. `v\_decision\_review` deterministic view vs API-only join? (Both are read-only; the view centralizes the contract, the API-only path avoids a migration entirely.)

Source: features\dimension-diagnostics\FEATURE_ARCHITECTURE.md
Dimension Diagnostics Architecture

> Status: approved design, not implemented.

>

> Date: 2026-08-06

>

> Decision: Kairos will measure why a scored decision did or did not work, create > bounded hypotheses for repairs, and validate them without automatically changing > research scores, paper trading, live trading, exits, sizing, thresholds, or a > champion strategy.

> Accountability addition (2026-08-06): this P0 program also records an > **agent contribution ledger**. It measures evidence quality and outcome > attribution for an agent/version; it does not reward or punish an LLM, mutate > its prompt/model/tools, or infer that several agents caused a result merely > because they appeared in the same workflow.

1. Problem and decision

Kairos already stores the exact evidence, availability mask, applied weights, score, action, and later 2/5/10/20-session outcomes for a research decision. That answers what happened to a decision. It does not yet provide one governed, market-local process that distinguishes:

1. **Evidence degradation:** a dimension was missing, stale, delayed, or had an invalid source contract.
2. **Predictive decay:** an available dimension no longer ranked later outcomes usefully for its declared market, instrument cohort, and horizon.
3. **Regime dependence:** a dimension works only in a predeclared, observable market regime.
4. **Portfolio/execution effects:** a promising research decision was not entered, was position-limited, or realized differently because of fills, stops, targets, cash, taxes, or correlations.

The wrong response is to let an LLM, a short run of trades, or an in-sample chart rewrite a live weight. It would confuse the four causes above and repeatedly fit noise. The right response is a **diagnostic-to-challenger lifecycle** using the existing decision, feature, validation, shadow, and strategy-version truth layers.

This feature is intentionally not a claim that Kairos can guarantee benchmark outperformance. It makes changes falsifiable and reversible instead of silently optimizing after the fact.

1.1 Agent accountability, not agent punishment

Agents are software roles, not economic actors. A reward/punishment loop that updates their behavior after a trade would optimize a sparse, confounded P&L sample and invite persuasive explanations, selective abstention, or other reward-hacking behavior. Instead, P0 records an immutable scorecard for each agent/version:

- decision coverage and label maturity;
- evidence availability, freshness and point-in-time integrity;
- market-local, benchmark-neutral decision outcomes at each horizon;
- the decision funnel (`scored` -> `eligible` -> `admitted` -> `filled` -> `exited`), with portfolio/execution exclusions shown separately;
- explicit `insufficient\_evidence` rather than a positive or negative verdict.

No scorecard changes an agent's model, prompt, tool access, schedule, score weight, decision authority or trading permission. A future collaboration claim requires an existing paired or randomized shadow where the same opportunity was evaluated with and without the specific input. Until then, `collaboration` is reported as **unattributable**, not rewarded.

2. Non-negotiable boundaries

- US and India diagnostics never share observations, labels, trial counts, benchmarks, cohorts, currency, confidence, candidates, or promotion evidence.
- The canonical facts remain `decision\_observations` and `observation\_labels`. `feature\_registry`, `edge\_signals`, `strategy\_versions`, validation experiments, and `shadow\_decisions` remain their existing sources of truth. This feature does not create a third provenance or promotion system.
- A diagnostic run may read completed, point-in-time-safe evidence only. It never refetches current provider snapshots to explain a historical decision.
- A missing, stale, malformed, or inapplicable input produces an explicit finding; it cannot be counted as a neutral score or evidence that a factor failed.
- The LLM may summarize a deterministic finding in plain language. It cannot set

- a formula, parameter, threshold, cohort, winner, or activation state, and it never runs on the scoring, order, exit, or broker path.
- No diagnostic result may write `agent\_signals`, `paper\_positions`, paper cash, live proposals/orders, exits, risk limits, `trading\_mandates`, score thresholds, or champion weights.
- A candidate can become a non-executing, market-local shadow only through the existing validation/strategy lifecycle. Promotion remains owner-gated and is currently dormant under the existing OOS promotion guards.

3. Existing truth layers and data contract

flowchart LR

D["Immutable decision_observations"] --> L["Matured observation_labels"]

D --> Q["Availability and provenance audit"]

L --> Q

Q --> R["Dimension diagnostic findings"]

R --> H["Bounded repair hypothesis"]

H --> V["Existing sealed replay and validation"]

V --> S["Existing non-executing shadow"]

S --> P["Existing owner-gated promotion"]

P --> C["Market-local champion"]

3.1 Reused primary records

Existing truth	Required use here
--- ---	
`decision_observations`	Exact as-of score, raw feature snapshot, availability mask, applied weights, decision/action, source and code/strategy
`observation_labels`	Matured, horizon-specific forward, benchmark-neutral, MAE/MFE and cost-adjusted labels. No immature label is eligible
`fundamental_facts`, `edge_signals`, `edge_signal_inputs`	Provider/freshness/PIT diagnosis for a particular fact or feature version.
`strategy_versions`, validation experiments, `shadow_decisions`	The only challenger, evaluation, shadow, and promotion lifecycle.
Paper/live trade and order ledgers	Separate execution attribution only. They cannot be mixed into a factor IC calculation.

All joins bind `market`, decision timestamp, horizon, formula/feature version and strategy version. Labels are joined by decision identity, never merely symbol.

3.2 Additive diagnostic records

The implementation may add only these append-only, owner-readable records after a schema/RLS review:

Table	One row per	Purpose
--- ---		
`dimension_diagnostic_runs`	market, declared analysis plan, as-of cutoff, horizon, code hash	Immutable run identity, input counts, excluded
`dimension_diagnostic_findings`	run, subject type/key, cohort, finding type	Measured dimension availability, freshness, PIT integrity, a
`dimension_repair_candidates`	finding-backed, structured candidate	A bounded reweight, availability-policy repair, feature retirement, c

`subject\_type` is `dimension`, `agent`, or `collaboration`. Only `dimension` and `agent` have P0 metrics. A P0 `collaboration` finding is always `unattributable`; it exists to make the missing paired counterfactual explicit.

These are diagnostic index rows, not alternate evidence stores. They reference source record IDs and immutable fingerprints; they do not copy or recompute a second set of historical features or labels. Use service-only writes, owner-read RLS, append-only triggers, market-scoped unique keys, and no browser write grant.

4. Diagnostic taxonomy

4.1 Evidence availability and freshness

For each market, dimension, instrument kind, provider policy and session cohort, measure the rate of `available`, `unavailable`, `inapplicable`, stale, malformed, and abstained decisions. Inspect source/fact timestamps and known-at boundaries.

This finding can justify a data-contract repair, cache/pacing change, or a more honest abstention. It cannot support a directional weight increase or decrease.

4.2 Predictive behavior

Only mature, availability-valid observations are evaluated by:

- rank IC and uncertainty at its declared forward horizon;
- top-bucket versus comparable median benchmark-neutral return;
- hit rate, tail loss/MAE and turnover-sensitive cost result;
- stability across independent session blocks and predeclared regimes.

The baseline score and every standalone dimension are evaluated separately. Correlation is descriptive until it has passed the sealed validation pathway. No finding says a dimension is "wrong" merely because a few held trades lost.

4.3 Regime and instrument applicability

Regimes must be predeclared from observed, market-local state available at the decision timestamp. A market-wide US regime never labels India evidence. Company fundamentals do not diagnose ETFs, leveraged ETFs, REITs, banks, or ADRs unless their feature-pack contract explicitly permits it.

Regime segmentation consumes trial budget. It cannot be used after looking at the outcome to find the one period where a factor happened to work.

4.4 Portfolio and execution attribution

Research merit, entry admission, and realized trade performance are reported as separate stages:

scored -> eligible -> admitted -> filled -> held -> exited -> realized

This prevents a factor from being blamed for a cash cap, a correlation veto, an earnings barrier, a stop, a partial exit, a stale order, or an unavailable broker. Conversely, a good research score does not excuse a harmful execution policy.

4.5 Agent contribution attribution

P0 reports an agent's own decision and evidence record, not a causal reward. A ResearchAgent score can be measured against mature benchmark-neutral outcomes; a data collector can be measured on valid/fresh/PIT-safe evidence; and an execution role can be measured on filled/held/exited bookkeeping. These metrics are never combined into one cross-role reward number.

The collaboration of ResearchAgent, an LLM explanation, a data adapter and PaperTrader is not identifiable from one realized trade. P0 therefore records `unattributable\_no\_paired\_shadow` until an existing non-executing shadow compares the same decision with and without a declared input. Any later marginal-credit method must be proposed separately, use market-local paired decisions, and stay outside the money path.

5. Statistical and governance controls

5.1 Minimum evidence

Each run declares floors before observing results: distinct market sessions, symbols per session, mature labels, independent time blocks, and cohort coverage. The floor is horizon-specific. Four US h20 labels or thirteen India h20 labels are displayed as `insufficient\_evidence`, never interpreted as a repair signal.

Do not infer effective breadth from IC dispersion. Report observed counts and uncertainty directly. A ticker-level diagnostic is attribution only, never a parameter-learning sample.

5.2 Trial-family control

Every formula, weight change, missing-data policy, regime segment, instrument pack, horizon, and combined strategy counts as a declared trial. The trial plan is frozen before outcome access, includes retired/failed candidates, and uses the existing walk-forward and multiple-testing requirements. There is no hidden grid search and no "try one more combination" after a weak result.

5.3 Candidate types

Candidates are structured, finite, and versioned:

1. **Data-contract repair:** correct a demonstrable availability, units, freshness, or PIT defect. It requires regression tests and reclassification of affected history; it is not an alpha candidate.
2. **Availability-policy candidate:** change how a valid but missing dimension causes abstention or renormalization. Requires paired replay because it can alter eligibility.
3. **Feature candidate:** add, retire, or replace one declared feature family under `feature\_registry` and the Feature-Pack Validation architecture.
4. **Strategy candidate:** a bounded market-local weights/genome version in the existing `strategy\_versions` lifecycle.

An LLM may only select from predeclared candidate templates and explain why a deterministic finding merits review. Server-side validation recreates every number; free-text cannot become a candidate specification.

6. Lifecycle and phasing

P0: measurement and visibility only

1. Define typed analysis plans and the diagnostic taxonomy.

2. Build a scheduled, idempotent market-local diagnostic from the existing decision/label ledger.
3. Write append-only run/finding rows with source fingerprints and explicit insufficient-data states.
4. Write a per-agent/version contribution finding alongside the dimension findings, plus an explicit unattributable collaboration finding.
5. Display status in Dashboard Learning/Upgrade Path and Strategy Portfolio Lab: evidence collected, exclusions, confidence, finding class, candidate status, and next gate. The global market switch selects the market; no cross-market aggregate exists.

P0 makes no candidate, provider call, score, trade, or strategy change.

P1: bounded candidate creation

A passing P0 finding can create a structured, reviewable candidate only if it has a frozen trial-family slot and complete PIT provenance. Data-contract repairs follow their normal bug-fix path. Feature and strategy candidates remain measure-only until sealed evaluation.

P2: sealed historical replay

Use the existing historical replay and walk-forward architecture. Evaluation is paired against the current market-local champion on the same opportunities and records costs, turnover, coverage, abstentions and benchmark-relative results. An unavailable historical dataset or weak PIT coverage is a refusal, not a fallback to current data.

P3: forward non-executing shadow

Only a sealed-replay survivor may create an existing `shadow\_decisions` strategy shadow. It reads the same decision snapshots and records counterfactual outputs; it creates no fills, cash movement, exits, proposals, or orders.

P4: promotion readiness

The existing owner-gated promotion path may consider a candidate only after its predeclared historical, forward-shadow, cost, safety, and benchmark-relative criteria pass. The system can recommend "ready for owner review" but never promotes itself. Current promotion safeguards remain authoritative and dormant until their documented PIT/OOS blockers are resolved.

7. UI and audit behavior

The user-facing view must explain the result without promising a prediction:

- ****Dimension health:**** availability/freshness/PIT status separate from measured predictive behavior.
- ****Evidence:**** distinct sessions, symbols, mature labels, exclusions, horizon, cutoff, source/strategy/formula versions, and trial-family count.
- ****Finding:**** `healthy`, `data\_degraded`, `predictive\_uncertain`, `predictive\_weakened`, `regime\_limited`, `execution\_confounded`, or `insufficient\_evidence`.
- ****Candidate trail:**** exact rule, reason, parent finding, replay/shadow status, rejection reason and immutable links to existing records.
- ****Per decision:**** the existing Research Journal/Decision Review keeps showing what was used, missing, observed-only, and the later realized path. It must not imply a single stock proved or disproved a dimension.

The UI reads persisted records only. Opening it cannot trigger a provider query, LLM call, replay, or candidate evaluation.

8. Acceptance criteria

- Every diagnostic row is market-local and refuses a mixed-market/currency input.
- Each result binds to exact decision IDs, mature label IDs, feature/formula and strategy versions, code hash, cutoff, trial plan and source fingerprints.
- Missing/stale/PIT-invalid evidence is reported as a data-quality result and is excluded from predictive calculations according to the declared plan.
- Factor diagnostics cannot read trade outcomes as substitutes for research labels; execution attribution cannot change a predictive result.
- Fewer than the declared sample floor yields `insufficient\_evidence` with no candidate auto-created.
- A diagnostic run, LLM summary, candidate row, UI request, cron failure, or malformed input cannot change score, direction, eligibility, thresholds, sizing, positions, cash, exits, proposals, broker calls, or champion status.
- Tests prove append-only behavior, idempotency, owner/RLS isolation, market isolation, PIT cutoff enforcement, trial-family accounting, no-lookahead, and no money-path imports/readers.
- Any strategy candidate reuses existing validation, shadow, and promotion rows; no parallel champion, provenance, or evaluation truth is introduced.
- Agent contribution results never trigger a model/prompt/tool/configuration mutation, task disablement, privilege change, score change or broker action.
- A collaboration result is `unattributable` unless it references paired, market-local shadow decisions with and without the declared input.

9. Build order and dependencies

1. Reconcile field-level provenance in existing decision/label records and add read-only diagnostic plan types/tests.
2. Add the P0 schema, RLS, append-only triggers and deterministic runner.
3. Surface P0 results in the existing Learning and Upgrade Path views.
4. Establish enough mature market-local labels before interpreting results.
5. Integrate candidate creation with the existing feature/strategy lifecycle.
6. Use the already-approved sealed replay and forward-shadow gates before any owner review; do not implement automatic activation.

Dependencies: `features/feature-pack-validation/FEATURE\_ARCHITECTURE.md`,
`features/technical-factor-calibration/FEATURE\_ARCHITECTURE.md`,
`features/walk-forward-ic-folds/FEATURE\_ARCHITECTURE.md`,
`features/historical-replay-harness/FEATURE\_ARCHITECTURE.md`, and
`docs/arch/09-learning-loop.md` remain authoritative for their respective subsystems. When P0 ships, update architecture chapters 04, 05/operations as applicable, and 09 in the same implementation change.

10. Explicit non-goals

- Autonomous self-reweighting, self-promotion, or self-retirement of a live scoring dimension.
- Per-symbol or per-sector parameter fitting from a small number of outcomes.
- LLM-selected indicators, strategy combinations, or trades.
- New provider polling, historical backfill, intraday alpha, or broker behavior.
- Replacing the current Feature-Pack Validation, replay, strategy version, or promotion architecture.

Source: features\dimension-diagnostics\IMPLEMENTATION_RESULT.md
Dimension Diagnostics P0 Implementation Result

> Shipped: 2026-08-06

>

> Scope: P0 measurement and visibility only. P1 candidate creation, P2 sealed replay, P3 forward shadow, and P4 promotion remain blocked by their separate evidence and governance gates.

Delivered

- `dimension\_diagnostic\_runs` and `dimension\_diagnostic\_findings`: append-only, RLS-enabled, service-only diagnostic index tables.
- `/api/agents/dimension-diagnostics?market=us|india`: owner-readable and cron-callable deterministic runner/read model.
- Weekday post-label pg_crons: `kairos-dimension-diagnostics-us` at 23:20 UTC and `kairos-dimension-diagnostics-india` at 23:25 UTC.
- Market-local availability and descriptive session-level rank-IC findings for every existing score dimension and horizon (2/5/10/20 sessions).
- Agent contribution records plus an explicit `unattributable\_no\_paired\_shadow` collaboration result.
- Upgrade Path registration and Learning page summary. Both surfaces are read-only and make no provider or LLM call.

Hard boundaries verified in implementation

- Reads only `decision\_observations`, `observation\_labels`, and signal labels.
- Does not import ResearchAgent scoring, PaperTrader, PositionMonitor, strategy mutation, proposal, broker, or provider modules.
- Results below 20 qualifying daily cross-sections of at least five names are marked `insufficient\_evidence`; no repair candidate is created.
- US and India are separate route invocations, separate rows, separate cron jobs, and separate UI queries.
- Collaboration is deliberately not assigned credit from shared workflow output. It remains unattributable unless a future paired market-local shadow exists.
- The initial `dimension\_diagnostics\_p0\_v1` and `\_v2` rows remain immutable. The active `dimension\_diagnostics\_p0\_v3` plan records agent contribution by agent label plus the decision's code version. ResearchAgent now writes the deployed Vercel/GitHub commit SHA on new observations. Historic observations without that field remain explicitly `data\_degraded`, rather than being silently blended with later deployments.

Verification

- Focused Vitest: dimension diagnostics and shadow-registry contracts.
- TypeScript check.
- Production migration: RLS enabled with no browser policy/grant, append-only triggers present, and both pg_cron jobs active.

Not shipped

No score/weight/threshold change, agent reward/punishment, agent configuration mutation, candidate creation, replay, shadow strategy, promotion, paper/live trade, exit, sizing, provider call, LLM call, or broker action.

Source: features\discovery-risk-and-instrument-policy\FEATURE_ARCHITECTURE.md
Discovery, Instrument Policy, Sizing, and Exit Control

****Status:**** Partially implemented. The auditability work below is now deployed as research-only provenance; every behavior-changing phase remains separately gated.

****Decision:**** Kairos must first prove why it did or did not trade a candidate, widen deterministic discovery, and distinguish instruments by risk contract. It must not respond to missed winners by concentrating 50% of a portfolio, enabling Webull order types, or allowing an LLM to alter a stop, target, or size.

Implementation Status (Verified 2026-08-01)

The safe foundations in this document are already present in production and must be reused rather than rebuilt:

- `universe\_snapshots` and `universe\_snapshot\_scores` preserve the per-run candidate population and rank provenance.
- `discovery\_snapshot\_members` records each symbol admitted to a market-scoped research run, including its source, held/ETF status, asset class, and screener bucket. It is append-only, service-only, and read by no scoring or money path.
- `decision\_observations.discovery\_source` records the research source, and `pipeline\_stage\_events` records portfolio-constructor, risk-plan, and execution outcomes. The Research Journal consumes this lineage as the current Miss Review surface.
- `instrument\_registry` is deployed as L0 observational classification with `new\_entry\_allowed=false`; execution code does not read it.
- Capped half-Kelly code and the out-of-sample calibration gate are present, but no accepted `pwin\_logistic` model is stored for either market. Flat mandate sizing remains authoritative.
- ATR partial/target/trailing variants are collecting measurement evidence only; PositionMonitor still owns the incumbent deterministic paper exit policy.

Therefore, the unbuilt work starts at ****better discovery inputs and expanded classification metadata****, while sizing activation, intraday exits, leveraged-paper entries, and live protective orders remain deliberately blocked by evidence or a separate owner approval.

****2026-08-01 implementation:**** the US relative-strength candidate bucket now reuses fresh, completed-session `edge\_signals` already collected by EdgeScout. Its existing bounded US post-close run rotates through the current liquid universe rather than only the watchlist. It admits at most four non-ETF symbols with positive six-month relative-strength and 52-week-high proximity z-scores. It makes no provider call, changes no score or threshold, and is recorded as `edge\_relative\_strength` with its exact input provenance. India remains unchanged because an equivalent candidate contract has not been verified.

****2026-08-01 Journal refinement:**** the Miss Review surface exposes the persisted admission metrics (and only those metrics) for an EdgeScout-admitted symbol. It must label them as research-admission provenance, never score contribution, prediction, or trade rationale. This adds no provider call and has no reader on any money path.

****Earnings/revision admission remains blocked:**** the current free earnings capture correctly preserves first-observed actuals and pre-announcement consensus vintages, but does not carry a provider-confirmed common EPS basis. Observed pairs can therefore be incomparable. Do not rank or admit candidates from `actual / consensus` until the capture contract records comparable basis, currency, fiscal period, and availability for both values, then passes a separate point-in-time review. The existing calendar and repricing safety readers remain unchanged.

1. Problem Statement

Recent US production evidence shows that the system did research MSFT and MU but did not create paper trades. The current research backlog is small, while PaperTrader has frequently reduced new orders below its 0.5% minimum because the book is already near the 80% gross-exposure cap. Theme Scout is a six-name/week, headline-to-LLM attention feed; the US screener is a six-name quality/value screen. Neither is a systematic earnings/revision/relative-strength discovery engine.

The product needs four separate upgrades:

1. Explain every missed or rejected candidate end to end.
2. Build deterministic, point-in-time discovery buckets without treating headlines or broker tools as alpha.
3. Classify each instrument and apply an appropriate risk/monitoring contract.
4. Improve sizing and exits only when market-local, out-of-sample evidence allows it.

2. Non-Negotiable Invariants

- US/USD and India/INR pools, evidence, policies, outcomes, and limits never mix.
- No LLM decides eligibility, direction, quantity, stop, target, partial exit, or broker action. LLMs may explain completed deterministic decisions and propose hypotheses for review.

- Existing pause, market-control, kill-switch, drawdown, freshness, name, sector, gross, volatility, correlation, cooldown, and approval gates remain authoritative.
- A new component may only observe, reject, or shrink until it has a separately approved promotion record. It may not make a candidate more eligible by default.
- Paper and live orders continue through the current atomic execution paths. No new ledger, cash, or position truth is created.
- Stops are risk controls, not guaranteed prices. A broker-resident stop is a second line of protection, not proof against a gap or a halt.

3. Architecture

flowchart LR

A["PIT discovery snapshots"] --> B["Candidate funnel ledger"]

B --> C["Deterministic research score"]

C --> D["Portfolio constructor and entry gates"]

D --> E["Paper or live execution gateway"]

E --> F["Position / protective monitoring"]

F --> G["Immutable outcomes and calibration"]

G -. "only after OOS gates" .-> D

H["LLM explanation / retrospective"] -. "never money path" .-> B

I["Robinhood + Webull router shadows"] -. "proven provenance only" .-> C

4. Phase 0: Candidate-to-Outcome Truth Ledger

Goal

For every candidate, answer: **was it discoverable, queued, researched, eligible, blocked, filled, exited, and how did it perform afterward?**

Build

- Reuse, rather than duplicate, existing truths: ``discovery_snapshot_members`` is the append-only admission record; ``decision_observations`` is score truth; ``universe_snapshot_scores`` is rank truth; and ``pipeline_stage_events`` is portfolio/execution truth.
- Bind a review by market, symbol, source snapshot, signal/observation id, mandate/policy version, and rejection code. Add fields only to the existing ledger that owns that stage; never create a parallel score, position, or P&L truth.
- Add a market-scoped Miss Review UI: filter symbol/date/source and show the exact first terminal reason. It must show ``unknown``, not invent a reason for legacy rows.
- Add a weekly report for top ex-post movers from a frozen point-in-time eligible universe. This is diagnostic coverage, not a new trading signal.

Acceptance

- New MSFT/MU-style reviews identify an exact admission source and terminal stage, or explicitly report missing legacy lineage.
- The report can distinguish ``never discovered`` from ``qualified but gross-cap denied``.
- It produces no provider calls and has no scoring, sizing, or order consumer.

5. Phase 1: Deterministic Discovery Buckets

Theme Scout remains an attention source, capped at its current small size. It is not a primary alpha engine. Add three independently budgeted US buckets, then design India equivalents only where point-in-time data coverage exists:

Bucket	Candidate contract	Initial status
---	---	---
Earnings / revisions	Known earnings result, surprise/guidance/revision evidence and <code>`available_at`</code>	Shadow / candidate only
Relative strength / breakout	Liquid point-in-time universe, completed-session price/volume, sector and benchmark-relative ranks	Shadow
Quality acceleration	Existing fundamentals screen, but batched/cache-backed and ranked by acceleration rather than only value	Candidate

The first row shipped is the **US candidate-only** relative-strength half. It uses the current EdgeScout universe and is explicitly not an alpha claim or a point-in-time historical backtest universe. The canonical score and every portfolio gate remain the sole determinant of a trade.

Rules:

- Each bucket produces a bounded daily snapshot (for example, 10 names), expires in three market sessions, and deduplicates into the existing research queue.
- Discovery is separate from selection. A name entering a bucket does not receive a score boost, paper fill, allocation, or live eligibility.
- Rank only within a homogeneous market/instrument universe. ETFs never rank against companies; no US/India cross-ranking.
- All price/fundamental observations must carry source, ``as_of``, ``available_at``, and corporate-action conventions. Unknown provenance means unavailable, not neutral.

- Start with coverage and forward-return reporting. Any score/sizing consumer needs a later point-in-time walk-forward and multiple-testing review.

Broker data policy

- Robinhood and Webull fields enter only through the Evidence Router cache, provider ledger, schema validation, and shadow comparison. No per-symbol MCP fan-out in the ResearchAgent deadline.
- Prioritize broker ****earnings calendar/results, financial statements, analyst revision/consensus, quotes, and tradability**** as read-only evidence contracts.
- Robinhood ``review_equity_order`` remains a fail-closed execution safety check.
- ``get_equity_technical_indicators``, Level 2, scanner popularity, options flow, and broker watchlists are not score inputs until they demonstrate incremental, point-in-time value. Level 2 belongs to order-quality measurement, not daily alpha.

6. Phase 2: Instrument Registry and Monitoring Contracts

Create a server-owned, versioned ``tradable_instruments`` policy registry. It replaces static ticker inference as the authority for new special instrument classes.

Required fields: market, currency, instrument kind, sector/industry, exposure tags, leverage class, underlying, sleeve, liquidity policy, event calendar contract, research cadence, monitoring cadence, paper/live eligibility, effective dates, source, and reviewer/version.

Class	Current truth	Phase policy
Ordinary equity / ADR	Generic daily model	Keep current model; ADR listing/priceability is a separate validation gate
REIT	Only broad ETF/context; no FFO/AFFO model	Classify first; add FFO/AFFO, occupancy, debt maturity, and rate-event evidence before
AI / semis / nuclear-power	Partial static coverage	Theme Scout may notice them
Crypto ETF	Proxy ETFs only; no direct crypto trading	Isolated exposure tag; underlying tracking and crypto-event context, never a general
Leveraged / inverse ETF	Generic flow blocks them	Default deny; only a separate US paper sleeve may later allow explicit long 2x/3x index

The registry is initially read-only classification. It may deny an unsafe instrument but cannot authorize a new one until the relevant sleeve architecture is approved.

7. Phase 3: Sizing and Capital Allocation

Immediate correction

Do not raise the per-name cap above 12% and do not allow 50-60% concentration. A raw analyst score is not a probability of winning, and a stop cannot cap gap loss.

Keep the current 80% gross cap until a separate allocation decision is approved. The new ledger must show whether rejected entries were correct risk denials or an avoidable cash-reserve policy. The owner-facing setting must call the remaining balance either ``strategic_cash_reserve`` or ``deployable_cash``; it must never imply idle buying power when gross exposure prevents entries.

Later calibrated sizing

- Reuse ``pwin_logistic`` and ``lib/risk/sizing.ts``; do not introduce another Kelly implementation.
- Enable only when a market-local model has at least 60 mature observations, a walk-forward holdout with at least 30 usable rows, non-degenerate outcomes, and out-of-sample calibration error at or below the existing threshold.
- Use half-Kelly only, with a 2% floor and 10% cap, then pass it through the existing constructor. Every portfolio constraint can only shrink or reject it.
- Log predicted probability, payoff estimate, proposed size, final size, and every constructor reduction. Roll back to flat mandate size on any gate failure.

No score threshold, risk profile, name cap, or cash reserve changes occur in this phase without a separate owner decision backed by the miss ledger.

8. Phase 4: Exit and Profit-Taking Evidence

Incumbent policy

Current paper exits are deterministic: entry-time stop/target, daily trailing stop using the position's own initial stop distance, fresh-score safeguards, time exits, and one partial profit at target where quantity permits. This remains the baseline.

Evidence path

- Continue the existing ATR exit-policy measurement family. It is not executable.
- Require market-local prospective labels, purged walk-forward comparison, locked holdout, turnover/cost sensitivity, drawdown non-inferiority, and correction for the complete trial family before any paper shadow.
- A selected candidate runs beside the incumbent in paper shadow using the same close-based fill convention. Only an explicit approval can change PositionMonitor.
- Stops and targets are immutable on entry. The system cannot widen a stop, average down, or convert a loss into a new higher-risk thesis.

9. Leveraged ETF and Intraday Protection

This phase inherits and supersedes no safety rule in ``features/leveraged-etf-and-intraday-execution/FEATURE_ARCHITECTURE.md``.

1. ****L0:**** registry/allowlist and observability only; all leveraged and inverse products remain blocked from generic flows.

2. ****L1:**** isolated US measure-only candidates; capture quote age, spread, underlying tracking, volatility, event risk, and would-enter/would-exit records.
3. ****L2:**** one paper-only long 2x/3x broad index/sector position; 3% name, 5% total sleeve, no single-stock or crypto leveraged funds, no India sleeve.
4. ****L3:**** 15-minute paper would-exit comparison, then risk-reducing paper exits only after quote freshness, duplicate-exit, and false-trigger tests pass.
5. ****L4:**** distinct live review. A broker-resident GTC disaster floor plus broker reconciliation is mandatory. App polling is secondary protection only.

Email/push alerts may notify the owner, but never constitute an execution signal. A verified in-session quote/broker state may create a risk-reducing SELL only in an approved live phase; it can never create a BUY outside the approved entry session.

10. Delivery Order and Estimates

Order	Deliverable	Estimate	Money-path effect
0	Candidate funnel ledger + Miss Review	3-5 engineering days	None
1	US discovery snapshots and coverage report	5-8 days	Shadow only
2	Router broker evidence contracts / parity	4-7 days after Router gates	Shadow only
3	Instrument registry and class visibility	3-5 days	Default-deny only
4	Capital-reserve decision surface and sizing audit	2-4 days	No automatic limit change
5	ATR exit walk-forward / paper shadow	Evidence-dependent; weeks of labels	No execution initially
6	Leveraged sleeve L0-L3	2-4 weeks after phases 0-5	Paper-only, separately approved
7	Broker protective orders / live intraday exits	Separate live approval	Live, fail-closed

The estimates are implementation effort, not evidence time. Validation windows cannot be compressed by coding faster.

11. Explicitly Deferred / Rejected

- No 50-60% single-name allocation.
- No LLM-selected or LLM-sized trades.
- No options-flow, social popularity, Level 2, or Fibonacci weight in the money path.
- No direct crypto trading, inverse ETF trading, single-stock leveraged ETFs, margin, shorting, extended-hours automation, or automatic broker routing.
- No Webull trading activation. Its richer order types are execution features, not evidence of selection alpha.

12. Required Documentation Updates Per Build

Every approved implementation updates the relevant sections of `docs/arch/03-agents.md`, `docs/arch/04-database-schema.md`, `docs/arch/05-crons-and-scheduling.md`, and `docs/arch/08-risk-and-safety.md`, plus the matching agent Mermaid diagram and `WORK\_LOG.md`. The System Reference must link this document as the governing plan.

Source: features\downside-hedging\FEATURE_ARCHITECTURE.md
Downside Hedging - Feature Architecture

> Status: APPROVED FOR BUILD (2026-07-15). US PAPER ONLY. SHIPS OFF.

Purpose

Add bounded downside protection without true short selling, margin, options, or leveraged products. Kairos may buy an explicitly allowlisted unleveraged inverse ETF as a small portfolio hedge after deterministic macro and market confirmation. The hedge is portfolio insurance, not an alpha candidate.

Locked Decisions

- US paper book only. No India proxy is invented.
- Initial instruments: `SH` and `PSQ`, both -1x daily-reset inverse ETFs.
- Every 2x/3x, volatility, commodity-inverse, and single-name inverse product remains blocked.
- Normal research, PaperTrader, Trader, rotation, and live execution continue to reject every leveraged/inverse ETF. Only the dedicated paper-hedge RPC has a narrow `SH`/`PSQ` exception.
- No LLM may arm, select, size, enter, exit, or mutate the hedge.
- Entry is cash-funded. Capital Rotation cannot sell an alpha holding to fund it.
- Default target is 5% NAV; hard ceiling is 8%; one hedge position maximum.
- Maximum holding period is five market days. Normal stop protection still works.
- Performance is judged on whole-book drawdown, volatility, turnover, and benchmark-relative return after hedge drag, never hedge P&L alone.

Deterministic State Machine

`off -> armed -> active -> exit\_pending -> cooldown -> off`

Entry requires both:

1. Macro confirmation: latest US `macro\_regime.danger\_score` >= 60.
2. Market confirmation: SPY is below its 50-session SMA and either 20-session return <= -4% or 20-session drawdown <= -6%.

The condition must persist for two completed evaluations. `PSQ` is selected only when QQQ is at least three percentage points weaker than SPY over the same 20-session window; otherwise `SH` is used.

Exit requires danger <= 45, SPY above SMA50, and non-negative five-session return, persisting for two evaluations. The five-market-day time stop and 5% price stop can exit sooner. Missing/stale data never enters and never fabricates an exit.

Data Model

- `downside\_hedge\_config`: US-only controls, seeded disabled.
- `downside\_hedge\_state`: mutable current state and confirmation streaks.
- `downside\_hedge\_events`: append-only evaluation/lifecycle ledger.
- `paper\_positions.position\_role` and `paper\_trades.position\_role`: `alpha` or `hedge`; hedge trades are always `excluded\_from\_learning=true`.

Owner-authenticated users may read configuration/state/events through owner RLS. Only service-role routes/RPCs write. Anonymous access is revoked.

Execution

`POST /api/agents/downside-hedge` runs after the US close. It loads macro state and point-in-time daily bars, evaluates the pure state machine, and appends an event.

- `enabled=false`: no evaluation or provider call.
- `enabled=true`, `paper\_execute\_enabled=false`: shadow measurement only.
- both true: an `enter` decision creates a non-alpha control signal and calls `execute\_paper\_hedge\_fill`, which rechecks configuration, state, allowlist, one-position limit, cash, and NAV cap under DB row locks before delegating to the existing atomic paper-fill function.
- an `exit` decision marks only the hedge position `hedge\_exit`; PositionMonitor closes it through the existing paper accounting path.

PositionMonitor never applies ordinary analyst-score/direction exits to a hedge. It retains price stop and time stop behavior. After a hedge disappears, the next controller run reconciles `exit\_pending`/`active` to cooldown before re-entry.

Product Surface

Settings -> Agents contains a Downside Hedge panel showing state, latest reason, target/cap, instruments, and separate Shadow and Paper Execution toggles. Paper execution cannot be enabled unless shadow is enabled. There is no live toggle.

Acceptance Criteria

1. Default/live DB state is OFF for both flags.
2. Generic research, paper, trader, rotation, and live gateway still block `SH`, `PSQ`, and all leveraged/inverse products.
3. LLM text cannot affect any evaluator result.
4. One failed/missing/stale input cannot create an entry.
5. RPC rejects non-US, non-allowlisted, over-cap, non-cash-funded, duplicate, or improperly sourced fills.
6. Hedge trades cannot enter learner/validation datasets.
7. US and India books are never summed or mutated together.
8. Typecheck, full Vitest, production build, live migration/RLS/OFF-state checks, and Vercel deployment pass without placing a live order.

Source: features\duration-signal-measurement\FEATURE_ARCHITECTURE.md
Duration Signal – Measurement Only

****Status:**** DEFERRED by owner decision, 2026-08-20. Not approved, not implemented.

****Revive when:**** the US equity book clears its own evidence floor (`n` >= 60` closed trades with h10 labels AND `nEffective` = n/10 >= 12` per r

****Date:**** 2026-08-18

****Scope:**** measure whether a duration (bond) signal has predictive power. No sizing, no orders, no allocation.

Why this exists

Asked: does Kairos recommend buying bonds when bonds rally, and plan entries/exits?
Answer, verified against production: ****no****.

- `TLT, IEF, SHY, HYG, LQD, BND, AGG, GOVT` – ****never scored, never traded****. Zero rows in `decision\_observations` and `paper\_trades`.
- Only `SGOV` (56 obs) and `GLD` (53 obs) exist from that family. SGOV is an ultra-short T-bill fund – a cash equivalent, not a duration position.
- `strategy\_sleeves` carries a `defensive\_etf` sleeve (SHY/IEF/TLT/GLD, 20% target, `enabled=true`), but `computeAllocation` returns null unless `strategy\_config.allocation\_enabled=true`, which is ****false****. Even when enabled, `paper-trade/route.ts` reads only the ****equity**** sleeve, and only to TIGHTEN the equity cap. Nothing buys the defensive sleeve.
- `TLT/IEF/HYG` in `price-cache-fill`'s REGIME list is tracking only; it feeds no buy decision. MacroSentinel's regime score feeds the macro DIMENSION of an equity score; nothing maps it to duration.

Why NOT to route bond ETFs through the existing scorer

This is the central design decision. Scoring `TLT` with today's pipeline would produce a number that looks like evidence and is not.

1. ****A fabricated fundamental.**** `scoreFundamentals` returns a flat ****55**** for any ETF ("no company fundamentals; neutral 55 baseline") and `fundamentalDataAvailable: isEtf || ...` marks it ****available****. So a constant enters at 30% base weight and is counted as real evidence. `v\_decision\_quality` meanwhile lists ETF applicable dims as `[technical, sentiment, macro]` – excluding fundamental. Two sources disagree about the same decision, which is exactly the defect fixed in `20260817200000\_decision\_quality\_single\_source`.
2. ****Equity technicals do not describe duration.**** RSI/EMA/ADX on TLT measure price momentum of a rates instrument; the actual drivers are level, slope, real rates, and policy expectations.
3. ****Macro is a 15% side input**** when for duration it is the whole thesis.
4. ****Sentiment is StockTwits chatter**** on a bond fund – thin to meaningless.
5. ****The exit mandate does not fit.**** A bond ETF would inherit the equity swing mandate: 8% target, 7% stop, 10-day horizon. TLT rarely moves 8% in ten sessions, so the target would be structurally unreachable – the exact defect documented in `docs/audits/2026-08-17-exit-policy-counterfactual.md`, rebuilt knowingly.

****Therefore: a separate, isolated measurement lane. Not a new symbol class in the equity screener.****

Proposed architecture

Isolation boundary (the load-bearing rule)

Modelled on the Property/Investing invariant in `docs/arch/08`: the dependency runs ONE WAY. Duration observations are written and read only by this lane. ****No** securities score, eligibility gate, sizing rule, order path, exit, promotion gate, learner, or broker call reads them. ****Enforced by a coupling test in the style of `tests/risk-research-annotation.test.ts` (which pins invariant R1 and is itself falsification-tested), not by convention.**

Data – zero new providers, zero new keys, zero cost

```
| Input | Source | Already integrated? |
|---|---|---|
| 2y / 10y yields (`DGS2`, `DGS10`) | FRED | yes – `FRED_SERIES` |
| Fed funds, target range | FRED | yes |
| CPI (real-rate proxy) | FRED | yes |
| TLT / IEF / SHY daily bars | Yahoo chart | yes – keyless, unbudgeted, unpaced |
```

FRED and Yahoo both carry `dailyBudget: null`. This adds no load to the AV (25/day) or EODHD (20/day) tiers.

Features (deterministic, no LLM)

Level (`DGS10`), slope (`DGS10 – DGS2`), 20-day change in each, policy gap (`DGS2 – FEDFUNDS`), and a real-rate proxy (`DGS10 – trailing CPI YoY`). Each carries its own FRED observation date – a monthly series is stamped with the period it belongs to, never the run date. Missing input $\Rightarrow$ the feature is ****unavailable****, never a neutral placeholder. That rule is the whole point of the lane.

Storage

New table `duration\_observations` (append-only), plus forward labels at h5/h10/h20/h60 reusing the existing label contract (`LABEL\_COST\_HAIRCUT`, forward-window coverage from `lib/learning/label-window.ts`). A separate table – not a `market` tag on `decision\_observations` – so no existing consumer can pick these rows up by accident.

Predeclared evidence floor – set BEFORE any data exists

Copying the discipline already enforced in `aggregateAtrExitEvidence`:

- date-clustered IC (one decision date = one draw; same-day observations share a shock)
- `n >= 60` AND `nEffective = n / horizonDays >= 12`
- below that the lane reports `insufficient\_sample` and ****may not be cited as evidence****

Kill condition – also predeclared

If, after the floor is met, date-clustered IC is indistinguishable from zero at $|t| < 2$, the lane is ****deleted, not tuned****. Written down now so it cannot be relaxed once the data is in.

Operator view

****Steelman.**** Duration is a genuinely different return driver, the FRED inputs are already paid for, and measurement-only is cheap. If a real signal exists, a future defensive sleeve would have evidence behind it instead of a hunch.

****Counter, and my recommendation.**** The equity book has ~16 clean US sessions, no demonstrated edge, and both markets sit far below the `nEffective >= 12` floor for the strategy that already exists. Retail duration timing is also a crowded,

well-studied space where the honest prior is "hard". The real cost here is not compute – it is attention: a second asset class multiplies the measurement burden before the first one is measurable.

****I would defer this**** until the equity book clears its own evidence floor. If it is built anyway, it must stay a measurement lane with the isolation test and the kill condition, so it cannot quietly become a sizing input.

Explicitly NOT in scope

No sizing. No orders. No allocation change. No `strategy\_sleeves` activation.
No `allocation\_enabled` flip. No change to the equity screener, mandate, or exits.
Nothing in this lane may be read by the money path.

Open questions for the owner

1. Build now, or defer until the equity book clears its floor? (I recommend defer.)
2. US only, or India too? India has no FRED equivalent; the India branch would need a different rate source and is not costed here.
3. Should the ETF fundamental-55 inconsistency be fixed separately? It affects the ETFs already being scored today (SGOV, GLD, XAR, VOO), independent of this lane.

Source: features\earnings-aware-risk\FEATURE_ARCHITECTURE.md
Feature Architecture: Earnings-Aware Risk

Status

Architecture status: Approved P0 - implemented 2026-07-29
Architecture approved: Yes
Approved scope: Measure-only US paper/live annotations, India proximity,
display-only holding warnings, normalized append-only evidence
Implementation allowed: P0 only

No behavioral policy is approved. `block` and `size\_down` remain unavailable.

Decision

Use earnings proximity and an options-derived move proxy as ****risk context****, not as a sixth alpha score.

Build the first release as measure-only:

- US paper: annotate every otherwise-eligible entry.
- US live: annotate, while preserving the existing Alpha Vantage earnings blackout exactly as-is.
- India: annotate earnings proximity when available; options risk is `unavailable`.
- Existing positions: surface a warning only. PositionMonitor and live exits do not change.

No block, size reduction, stop widening, score change, promotion input, or automated exit is allowed until the shadow record passes the acceptance gates in this document and the owner separately approves the behavior.

Why This Belongs In Risk, Not Alpha

Short-dated option prices around a known earnings event contain information about the market price of uncertainty. They do not provide a reliable sign for the stock's next move. Skew and flow may contain directional information in some settings, but retail-visible volume cannot reliably identify opening versus closing trades, buyer versus seller initiation, or multi-leg hedges.

Kairos may therefore use:

- earnings timing as event-risk metadata;
- a same-expiry ATM straddle as an expiry-bounded move proxy;
- the ratio of that proxy to the planned stop distance as context.

Kairos must not use:

- put/call ratio, "unusual calls", or skew as automatic long conviction;
- the move proxy as a directional forecast;
- options fields in `analyst\_score` or the LearnerAgent objective.

This feature does not need directional IC validation because it makes no directional alpha claim. It ****does**** need operational and calibration validation: coverage, quote quality, date accuracy, implied-versus-realized move calibration, and counterfactual effects on entries and outcomes.

Verified Current State

Area	Current behavior

```

| US PaperTrader | No earnings gate or annotation at the fill choke point |
| US TraderAgent | Already blocks proposals from 2 calendar days after through 5 calendar days before an Alpha Vantage earnings date |
| Existing live gate | Fetches Alpha Vantage directly, fails open on timeout/unavailable data, and is not shared with PaperTrader |
| `lib/data/earnings.ts` | Finnhub US and Yahoo India proximity helper; returns days only and loses source/as-of/confidence metadata |
| `lib/data/earnings-pit.ts` | Captures point-in-time earnings vintages |
| `lib/options-signal.ts` | On-demand Yahoo nearest-expiry chain; not on a trading path |
| Current `ivPercentile` | Not a 52-week IV percentile. It locates ATM IV inside the current chain's strike-IV range |
| Current unusual-flow labels | Comments and summary overstate contracts as "smart money" evidence |
| Paper audit | `pipeline_stage_events` records stage decisions |
| Live audit | `trade_proposals.risk_check_reasons` records proposal risk checks |

```

The new feature must not accidentally replace or weaken the existing live blackout. A unified policy can replace it only after shadow coverage parity and an explicit owner-approved cutover.

Verified Broker Data Surfaces

The connected broker surfaces were re-probed on 2026-07-29. Availability is useful, but tool presence alone is not evidence that a field is correctly timestamped, complete, or stable enough for a money-path decision.

```

| Source | Earnings | Options | Verified state | Intended role |
|---|---|---|---|---|
| Finnhub | Yes | No | Existing US `fetchDaysToEarnings()` source | Current US calendar source and independent fallback |
| Alpha Vantage | Yes | No | Existing live TraderAgent blackout source | Preserve until unified resolver proves parity |
| Webull MCP | Yes | No | Live `tools/list`: 71 tools. `get_stock_earnings_calendar` returned AAPL dates, actual/estimated EPS, and actual/estimated IV |
| Robinhood MCP | Yes | Yes | Live `tools/list`: 52 tools, including `get_earnings_calendar`, `get_earnings_results`, `get_option_chains`, `get_unusual_flow` |
| Yahoo | Yes for India; US fallback possible | Yes | Existing India calendar helper and current on-demand US options implementation | India calendar

```

Webull's published Cloud MCP surface does **not** expose an equity-options chain, despite exposing stock quotes and many fundamentals tools. Do not infer options support from Webull's brokerage product or separate trading documentation.

Robinhood is the strongest candidate for the US move proxy because the connected MCP advertises contract discovery plus current quotes. Before selection as a provider, an owner-run allowlisted probe must verify:

- exact expiry and strike fields;
- same-contract bid, ask, sizes, timestamp, and open interest;
- whether quotes are real-time, delayed, indicative, or entitlement-dependent;
- response pagination and maximum contracts per request;
- behavior outside market hours and for illiquid names;
- rate limits and token/session failure behavior.

Broker OAuth calls do not consume Finnhub, Alpha Vantage, or Yahoo quotas, but they are not quota-free by assumption. They require independent pacing, provider-call accounting, caching, and health monitoring. A disconnected broker must become `unavailable`; it must never silently change entry behavior.

Source resolution

Earnings dates should be prewarmed and resolved from the existing point-in-time calendar cache rather than fan-out to three providers synchronously at every entry. Finnhub, Webull, and Robinhood observations retain source and as-of metadata. A disagreement beyond one market session is `conflict`, not a majority vote.

For US option snapshots:

1. Robinhood is the preferred candidate only after its payload probe passes.
2. Yahoo remains a shadow-only fallback with an explicit unofficial-source tag.
3. Webull returns `unsupported`, not `unavailable`, for options.
4. No broker order or option-order tool is used. Read and execution allowlists remain separate.

Risk Contract

Earnings event

```

type EarningsEventRisk = {
  market: "us" | "india";
  symbol: string;
  reportDate: string | null;
  reportSession: "bmo" | "amc" | "during_session" | "unknown";
  sessionsUntilReport: number | null;
  source: string | null;

```

```

observedAt: string;

confidence: "confirmed" | "estimated" | "unknown";

status: "available" | "unknown" | "conflict";

};

```

Calendar sessions, not UTC duration or raw calendar-day subtraction, determine proximity. If sources disagree beyond one market session, status is `conflict`; the system must not silently choose the more convenient date.

Options move proxy

```

type EarningsMoveProxy = {

market: "us";

symbol: string;

observedAt: string;

quoteAsOf: string | null;

reportDate: string;

reportSession: EarningsEventRisk["reportSession"];

expiry: string;

spot: number;

strike: number;

callBid: number;

callAsk: number;

putBid: number;

putAsk: number;

callMid: number;

putMid: number;

moveProxyPct: number;

stopDistancePct: number | null;

stopToMoveRatio: number | null;

quality: "usable" | "wide_spread" | "stale" | "illiquid" | "unavailable";

reason: string;

};

```

The calculation is:

$\text{moveProxyPct} = (\text{ATM call mid} + \text{ATM put mid}) / \text{contemporaneous spot}$

It is labelled ****expiry-bounded ATM straddle move proxy****, not "earnings implied move". The premium contains event variance, non-event variance through expiry, volatility risk premium, rates/dividends, and bid/ask effects.

Deterministic Chain Rules

1. Select the earliest standard expiry that is after the earnings event and leaves at least one tradable post-event session. Account for BMO versus AMC; unknown timing makes the proxy lower-confidence.
2. Fetch that exact expiry. The current Yahoo call only reads `options[0]` and is insufficient.
3. Select the nearest strike present in both calls and puts.
4. Use mids only when both bid and ask are finite, non-negative, and ask is not below bid. Last trade is never a substitute.
5. Require a configurable maximum spread-to-mid ratio on each leg, nonzero open interest or quoted size, a fresh quote timestamp, and a contemporaneous spot.
6. Reject crossed, stale, zero-premium, missing-leg, or implausible results.

7. Cache the raw normalized snapshot by `(symbol, expiry, observed market session)`. Never overwrite a snapshot used by a decision.
8. Every provider is fail-soft and source-labelled. Robinhood requires an allowlisted payload probe; Yahoo is unofficial and remains a shadow fallback. A capability probe and provider-health metric are required before shadow runs.

The first build must rename or remove the current `ivPercentile` label and remove "smart money" claims from unusual-flow summaries. Neither field is part of this feature's decision contract.

Policy

```
type EarningsRiskPolicy = {
```

```
version: number;
```

```
mode: "shadow" | "block" | "size_down";
```

```
market: "us" | "india";
```

```
proximitySessions: number;
```

```
maxLegSpreadToMid: number;
```

```
maxQuoteAgeSeconds: number;
```

```
minOpenInterest: number;
```

```
sizeMultiplier: number;
```

```
};
```

Initial production configuration is permanently pinned to `shadow` until an owner-approved migration/config change activates another mode.

`earningsRiskVerdict()` is pure and deterministic. It may only preserve or reduce entry eligibility/size. It can never:

- increase a score, size, price target, or holding horizon;
- widen or remove a stop;
- suppress a sell or any PositionMonitor/live-exit action;
- convert unknown data into "no earnings";
- borrow US options evidence for India.

In shadow mode, unavailable data records `unknown` and does not block. In a future active mode, unavailable behavior must be decided explicitly; it cannot inherit a generic fail-open default.

Money-Path Integration

Paper

Run the pure verdict after the fill-bound stop/target plan exists and before the atomic `execute\_paper\_fill` call. In P0, only log the counterfactual verdict to `pipeline\_stage\_events`; do not alter the RPC inputs or claim lifecycle.

Capital rotation must use the same annotation contract. A rotation cannot bypass measurement merely because it enters through the slot/cash replacement path.

Live

Run the shared annotation before proposal insertion and persist it inside `trade\_proposals.risk\_check\_reasons`.

The existing Alpha Vantage `-2/+5` blackout remains authoritative during P0. The new resolver must not catch an error and skip that existing gate. Replacement requires:

1. coverage parity over predeclared observations;
2. no regression in known-date blocking;
3. an owner-approved policy version;
4. one atomic cutover that removes the direct Alpha Vantage path.

Autonomous live execution must re-check the persisted policy version and verdict freshness before submitting an approved proposal, just as it re-checks other latched money-path controls.

Existing positions

P0 may show:

- earnings date/session;

- sessions remaining;
- usable move proxy;
- stop-to-move ratio;
- data-quality state.

It must not cause PositionMonitor or the live exit monitor to trim, close, move a stop, or alter a target. Gaps can jump resting or synthetic stops, so ``stopToMoveRatio < 1`` is a warning, not proof that the stop is wrong.

Persistence And Provenance

Do not create a parallel evidence truth layer.

- Source payload/fingerprint references use existing ``evidence_records``.
- Paper decision context uses ``pipeline_stage_events.detail``.
- Live decision context uses ``trade_proposals.risk_check_reasons``.
- If cross-flow analytics cannot be made reliable over those ledgers, add one append-only ``earnings_risk_observations`` table only after schema approval. It must reference the existing signal, paper event/proposal, evidence record, policy version, and source snapshot rather than duplicating source truth.

No raw option chain is exposed to the browser. Owner-facing APIs return only the normalized fields above. RLS is owner-read; writes are service-only and append-only.

Shadow Metrics And Acceptance Gates

Predeclare the shadow window before collecting outcomes. Minimum gates:

1. At least 60 otherwise-eligible US entry decisions and at least 20 distinct earnings events.
2. Earnings-date coverage at least 95% for symbols where the legacy live gate found a date.
3. Date disagreement, reschedule, and unknown rates reported separately.
4. Usable option snapshot coverage reported by liquidity tier; no silent exclusion of thin names.
5. Compare proxy with absolute close-to-close and close-to-next-open event moves.
6. Report empirical exceedance rates for ``0.5x``, ``1.0x``, and ``1.5x`` proxy bands.
7. Counterfactual report: entries blocked/sized down, later P&L, MAE/MFE, stop-outs, and sample sizes. No claim based only on average P&L.
8. No paper/live eligibility or size change while mode is ``shadow``.

These observations test calibration and product usefulness, not directional IC. A future block/size-down proposal must specify its expected trade-count cost and must default to no change when evidence is inconclusive.

Build Order

1. Correct misleading existing option labels and add golden parser tests.
2. Add bounded read-only Robinhood earnings/options contract probes and persist only schema fingerprints plus normalized test results.
3. Add Webull earnings into the source-aware, session-aware earnings-event resolver; retain Finnhub/Alpha Vantage observations through cutover.
4. Add exact-expiry Robinhood fetch, Yahoo fallback, and pure quote-quality/move-proxy calculation.
5. Add pure ``earningsRiskVerdict()`` with ``shadow`` as the only enabled mode.
6. Wire paper, rotation, and live annotation without changing behavior.
7. Add owner-facing Backtest/Risk visibility and provider-health coverage.
8. Accumulate the predeclared shadow record.
9. Review evidence and separately decide whether to keep annotation-only, activate bounded size-down, or reject the behavioral feature.

Tests Required Before P0 Ships

- BMO, AMC, unknown timing, weekend, holiday, and rescheduled event cases.
- Exact expiry selection and same-strike call/put pairing.
- Missing leg, zero bid, crossed quote, stale quote, wide spread, and no-OI cases.
- US/India isolation.
- Paper fill and capital-rotation paths both produce annotation.
- Existing live blackout still blocks every case it blocked before.
- Annotation failure cannot alter eligibility, sizing, or exits.
- Idempotent retries do not create conflicting observations.
- Browser payload excludes raw chain data.

Sources

- Cboe explains that short-dated options around earnings primarily indicate the magnitude, not direction, of the expected reaction and compares straddle pricing with historical moves:
<https://www.cboe.com/insights/posts/what-options-data-may-indicate-about-mag-7-earnings>
- Chung and Louis document that earnings-event option returns and implied versus realized volatility vary with pre-event conditions, which is why calibration cannot be assumed:
https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2886040
- SEC material notes that disseminated option quotes can be stale during market

data stress, supporting explicit freshness and quote-quality gates:
<https://www.sec.gov/rules-regulations/2001/09/firm-quote-trade-through-disclosure-rules-options>

Owner Decision

Recommended approval, when requested:

- P0 measure-only implementation;
- US paper + US live annotation;
- India proximity annotation only;
- existing live blackout preserved;
- existing positions display-only;
- no scoring, sizing, entry, stop, target, or exit behavior change.

Do not approve `block` or `size\_down` yet.

P0 Implementation Result

Implemented in `lib/risk/earnings-risk.ts` and recorded in
`IMPLEMENTATION\_RESULT.md`. Production is pinned to policy version 1
`mode='shadow'`; the database rejects any other mode and rejects
`behavior\_changed=true`.

Holdings coverage extension (2026-07-30)

The original P0 hooks observed an options move proxy only after an entry reached PaperTrader or TraderAgent. A bounded weekday US holdings monitor now reads open paper positions plus the latest complete per-account live-risk snapshots and records `decision\_kind='holding'` observations. Event discovery merges the persisted PIT cache with one bounded Robinhood calendar read, so a cache miss does not hide a known holding's earnings date. Per-symbol earnings/options calls are requested only when that event is inside the holding horizon. This extends risk visibility to names already owned without turning options into an all-watchlist provider fan-out or a directional score. India remains proximity-only and has no scheduled options leg.

Shared earnings-date parser (2026-08-02) – open item CLOSED

The follow-up recorded in `IMPLEMENTATION\_RESULT.md` \$"Priority 3 completed – provider parsers audited" ("Normalizing validation into a single shared date parser is the tidier fix and is left as a follow-up, since it touches all four collectors") is now done.

`normalizeRealIsoDate` in `lib/date-only.ts` is the one parser. Every collector routes through it:

```
| Collector | Before | Now |
|---|---|---|
| `robinhoodEarningsObservation` | validated (own inline regex) | `normalizeRealIsoDate` |
| `earnings_calendar` cache | **unvalidated** – `String(data.report_date)` | `normalizeRealIsoDate` |
| Webull `parseEarningsCalendar` | **unvalidated** – `String(row.expected_publish_date)` | `normalizeRealIsoDate` |
| Finnhub / Yahoo-India base (`fetchEarningsDateObservation`) | validated | `normalizeRealIsoDate` |
| `fetchIndiaEarningsDate` | **unvalidated** – `new Date(Number(raw) * 1000)`, no range check | `normalizeRealIsoDate` (epoch-aware) |
```

The parser accepts `YYYY-MM-DD` (with or without a time suffix) and a provider epoch in seconds or milliseconds. It returns `null` for a wrong format, an empty string, `null`/`undefined`, a non-string non-number, an out-of-range or negative epoch, and for calendar days that do not exist (`2026-02-30`) which JavaScript's `Date` would otherwise roll forward into a different, valid-looking event day. A bare 8-digit `20260731` stays a rejection rather than being read as an epoch.

Rejection is fail-closed for every risk-increasing decision: the collector emits no observation, so the symbol resolves to `status: "unknown"` (or `"conflict"` when observations disagree) with `reportDate: null` and `sessionsUntilReport: null`. It never coerces to a nearby date and never defaults to "no earnings soon". `tradingSessionsBetween` keeps its load-bearing ordering – validation before the `from == to` shortcut, returning `number | null` – as defence-in-depth behind the parser, pinned by `tests/earnings-risk.test.ts`. Coverage lives in `tests/date-only.test.ts` and `tests/webull-research-data.test.ts`.

Policy version 1, `mode='shadow'`, is unchanged. This is a refactor plus a correctness fix: no score, weight, threshold, direction, eligibility, sizing, entry, exit, promotion or broker behaviour changed.

Source: features\earnings-aware-risk\IMPLEMENTATION_RESULT.md
Earnings-Aware Risk P0 Implementation Result

Date: 2026-07-29
Status: Complete and deployed to production schema
Policy: `v1`, `shadow` only

Shipped

- Source-aware US earnings resolution across the existing PIT calendar,

- Finnhub, Webull, and Robinhood; India uses its market-local calendar/Yahoo source. A disagreement beyond one market session is `conflict`.
- Robinhood read-only contract allowlist and owner/cron schema diagnostics. Order, review, cancel, account, and position tools are structurally excluded.
- Exact post-event expiry, bounded option-instrument pagination, nearest common call/put strike, timestamp/liquidity/spread validation, and an expiry-bounded ATM straddle move proxy. Yahoo is a labelled fail-soft shadow fallback.
- Paper entry and capital-rotation annotations before execution. The annotation cannot alter the atomic fill or rotation arguments.
- Live proposal annotation while preserving the existing Alpha Vantage `-2/+5` blackout exactly. No live execution code reads the annotation.
- Owner-facing Portfolio Risk panel with current-holding event warnings and acceptance-window counts. No raw option chain reaches the browser.
- Append-only `earnings\_risk\_observations` with owner-read RLS, service-only insert, immutable rows, retry idempotency, and database checks that pin `policy\_mode='shadow'` and `behavior\_changed=false`.
- Post-DDL advisor findings were resolved: the owner policy caches `auth.jwt()` through a scalar subquery and `proposal\_id` has a covering partial index.

Live Proof

The bounded AAPL probe on 2026-07-29 resolved the same 2026-07-30 AMC date from the cache, Finnhub, Webull, and verified Robinhood calendar. It selected the 2026-07-31 expiry and the true \$340 common strike, then normalized timestamped Robinhood bid/ask, sizes, and open interest. The quote exceeded the 15-minute freshness ceiling and was correctly classified `stale`; no counterfactual action was treated as usable. Retrying the probe left one ledger row.

Behavior Boundary

This release does not change scores, eligibility, sizing, stops, targets, holding horizons, proposals, fills, rotations, or exits. Future activation requires 60 otherwise-eligible US entry decisions, 20 distinct events, the remaining calibration gates in `FEATURE\_ARCHITECTURE.md`, and a separate owner approval.

Post-implementation review fix – 2026-07-29

Independent adversarial review of `65c1c5be`. All twelve stated review priorities were checked; production DB invariants were verified against the live database rather than the migration files.

Finding 1 (MED, fixed) – `tradingSessionsBetween` fabricated session counts

Proven by execution against the shipped code:

Input	Returned	Should be
--- --- ---	---	---
`to = "not-a-date"   **282**`	unknown	unknown
`to = "2030-01-01" (beyond the 400-iteration guard)   **282**`	unknown	unknown
`to = ""   **280**`	unknown	unknown

The loop exited on its guard without signalling that the target was never reached, returning whatever it had counted. Two reachable consequences:

1. `app/api/portfolio/earnings-risk/route.ts` computes `sessionsUntilReport` from a `\*\*persisted\*\*` `report\_date`, so a malformed or far-future stored date rendered a fabricated "trading sessions until report" on the owner's Risk page.
2. The `conflict` check compared two such values. `Math.abs(null)` is `0`, and two unparseable dates each scored 282 – so they compared as `\*\*agreeing\*\*`, producing `status: "available"` from data whose distance could not be computed. That directly contradicts this feature's own rule that unknown or conflicting data must never become a confident date.

****Fix.**** `tradingSessionsBetween` now returns `number | null`, rejects non-ISO and impossible dates (`2026-02-31`) up front, and returns `null` when the target is not reached inside the guard. The conflict check treats `null` as a conflict rather than agreement. Both call sites already typed `sessionsUntilReport` as `number | null`, so the unknown propagates without a signature change at the boundary. Five regression assertions added.

No behavioural change to trading: the value was already shadow-only.

Verified clean

- ****Shadow isolation (paper):**** `earningsRisk` appears at seven sites – one declaration, one assignment, two log fields, two record calls, one null guard. Never in arithmetic or a branch condition. The rotation observation sits inside the pre-existing `atNameCap || cashShort` branch and is `try`-wrapped.
- ****Legacy live blackout:**** AV fetch, CSV parse and `diffDays >= -2 && diffDays <= 5` unchanged. `skipped.push` and `continue` sit ****after**** the annotation's `try/catch`, so a throwing annotation cannot weaken the block.

- ****False ATM under incomplete pagination:**** guarded by ``hasMore && maxStrike < spot -> return null``.
- ****Read allowlist:**** ``callRobinhoodResearchReadTool`` re-checks the name at runtime and returns ``tool_not_allowlisted``, so a cast cannot bypass it.
- ****Owner API:**** ``requireOwner()`` gates on ``OWNER_EMAIL`` ****and**** ``email_confirmed_at``; another authenticated user receives 403.
- ****No raw payload to the browser:**** the portfolio route selects an explicit column list containing no raw/chain/payload column.
- ****Production DB (queried live, not inferred from migrations):****
 - ``policy_mode CHECK (= 'shadow')``, ``behavior_changed CHECK (= false)``, RLS enabled, owner policy uses ``(select auth.jwt())``, ``authenticated` = SELECT only, `service_role` = SELECT + INSERT only (UPDATE and DELETE both return `42501`)`, ``anon`` no grants, plus a ``BEFORE DELETE OR UPDATE`` immutability trigger and 19 domain CHECKS.

Open, not fixed

- ****Quota/latency (LOW, not a defect):**** the annotation is awaited inline before sizing on every paper entry. The Robinhood calendar is cached 30 minutes, but ``get_option_chains``, ``get_option_instruments`` (up to 5 sequential pages) and ``get_option_quotes`` are not. Fail-soft, so it cannot break a fill, but it adds real latency and broker quota per entry.

Priority 3 completed – provider parsers audited

Date-format validation is applied ****inconsistently**** across the four earnings observation parsers:

```
| Parser | Validates the date? |
|---|---|
| `robinhoodEarningsObservation` | **Yes** – `if (!/^d{4}-d{2}-d{2}$/.test(date)) return null` |
| `earnings_calendar` cache | **No** – `String(data.report_date)` accepted as-is |
| Webull | **No** – `String(row.nextDate).slice(0, 10)` accepted as-is |
| Finnhub / Yahoo base | Relies on the shared helper, not re-checked here |
```

Not currently exploitable, because the Finding 1 fix closes it at the gate: ``tradingSessionsBetween`` validates ****before**** the ``from == to`` shortcut, so a single malformed observation compared against itself returns ``null``, the conflict check treats ``null`` as a conflict, and ``resolveEarningsEventRisk`` returns ``status: "conflict"`` with ``reportDate: null``. A bad date from any parser now fails closed rather than propagating.

That is defence-in-depth, not a substitute for validating at the parser. The per-parser gap is recorded here and pinned by a regression test (``treates a self-comparison of a malformed date as unknown, not zero``) so the protection cannot be removed silently. Normalizing validation into a single shared date parser is the tidier fix and is left as a follow-up, since it touches all four collectors.

****FOLLOW-UP DONE (2026-08-02).**** The table above is history, not current state. All collectors now normalize through the single parser ``normalizeRealIsoDate`` in ``lib/date-only.ts``, including ``fetchIndiaEarningsDate``, which was a fifth unvalidated entry point this audit missed (it converted a Yahoo epoch with no range check). See ``FEATURE_ARCHITECTURE.md`` §"Shared earnings-date parser (2026-08-02) – open item CLOSED". The ``tradingSessionsBetween`` ordering above is unchanged and remains the defence-in-depth layer.

Numeric coercion elsewhere is sound: quote legs coerce with ``Number(...)`` and ``calculateMoveProxy`` rejects non-finite, crossed, or stale inputs rather than computing through them.

Source: features\earnings-expectations-peer-delta\FEATURE_ARCHITECTURE.md
Earnings Expectations & Peer Delta – Feature Architecture

```
> Status: **DRAFT – architecture only. Awaiting Vaibhav's approval before any Builder work.**
> Author: Claude (Sonnet 5), Architect role. 2026-09-03.
> Scope: design only. No migration, provider activation, scoring change, candidate emission,
> shadow-decision write, or order-path change is authorized by this document.
> Update this file when: source contracts, PIT capture rules, peer methodology, η definition,
> combination-testing methodology, or Upgrade Paths integration change.
> Companion: `docs/audits/2026-09-03-earnings-expectations-peer-delta-codex-brief.md` (adversarial
> review brief – every load-bearing claim below is re-derivable from that brief's SQL and sources).
```

0. Verdict up front

****RESHAPE**, then GO for a narrow Stage 0/1, with two hard gates before anything wider.**

The core idea – that forward-looking, peer-relative estimate revisions carry information a current-fundamentals/price-momentum scorer cannot see – is real and has primary-source support (Hameed, Morck, Shen & Yeung 2010, §5). But the prompt's scope, taken as written, is too large for what the data and the provider budget can support today. Three things must shrink or defer, and one operational risk (Alpha Vantage's shared call budget) must be resolved before Stage 1 commits to a

symbol count:

1. ****No management-guidance extraction in Stage 1.**** It is the single most complex, LLM-adjacent, look-ahead-prone part of the prompt, and nothing downstream needs it to get a first η measurement. Own-company and peer estimate REVISIONS (not guidance) are sufficient to test the core hypothesis. Guidance extraction becomes a ****Stage 2+**** decision, gated on Stage 1 clearing its own bar.
2. ****No new lifecycle taxonomy.**** The prompt proposes ``unavailable → collecting → measurable → shadowing → eligible → rejected/retired``. Kairos already has one shared ``ShadowLifecycle`` enum (``lib/shadows/registry.ts:1-9``) used by all 21 registered Upgrade Path programs: ``collecting | ready_for_review | blocked | armed | paper_active | idle | off | not_applicable``. This feature reuses that enum at the top level and puts the requested finer granularity in a nested ``stage`` field – exactly the pattern ``technical-calibration`` already uses (``edge_readiness_status.stage`` nested inside a ``collecting`/`ready_for_review`` lifecycle). Two lifecycle vocabularies on one Upgrade Paths page is the redundant-system the brief itself warns against.
3. ****India is fully deferred****, not merely de-prioritized. No India source contract has been verified (\$4.5); this document proposes no India work beyond a not-yet-run capability probe.
4. ****Alpha Vantage's call budget is shared and its true production ceiling is unverified.**** Production consumed 35-71 AV calls/day over the last 10 days (measured, \$4.4) while the code's own default budget constant (``lib/data/provider-fetch.ts:29``) is 25/day – meaning either a paid tier is active that isn't visible in this session's ``.env.local``, or the gate is already being exceeded. ****This is the single largest feasibility risk in the whole feature and is not resolved here.**** Stage 0 must confirm the actual contracted plan before Stage 1 commits to any symbol count.

With those four changes, Stage 0 and Stage 1 are buildable now, cheaply, and Stage 1's evidence (\$9.2) will tell us honestly whether Stage 2 (peer baskets) is worth building at all.

1. Verified current-state audit

Every number below was re-derived against the production database (`<production-project-ref>`) or read from the live source file on 2026-09-03. See the companion Codex brief \$2 for the exact SQL.

1.1 What exists today

```
| Claim in the originating prompt | Verified | Detail |
|---|---|---|
| `lib/data/earnings-pit.ts` captures upcoming US consensus vintages + first-observed actuals | **TRUE** | 303 lines. Append-only; never overw
| Capture covers only a small default US watchlist | **TRUE** | Called from the existing earnings-refresh cadence on whatever symbol set that
| Production: 56 consensus snapshots, 19 symbol/report-date events, 9 symbols, 0 basis-comparable actual-vs-consensus events | **TRUE, exact m
| `decision_observations`: 5,911 US rows, 4,125 with analyst-recommendation evidence | **TRUE, exact match** | `features->'analyst'` is preser
| Analyst `*targets*` in only 157 US decision observations | **TRUE, exact match, wrong field path in the prompt** | The prompt implied a top-le
| `scoreAnalyst()` is measure-only, not in the live composite | **TRUE, and the variable-naming trap that would mislead a future auditor is re
| Live fundamental scorer uses P/E (sector-relative), margin, ROE, EPS, YoY revenue growth; no guidance/revision dimension | **TRUE** | `lib/
| Earnings dates affect risk/repricing but not scoring inputs | **TRUE, three distinct mechanisms, correctly separated already** | (a) `lib/r
| `symbol_profiles.peers`: 97/187 US profiles, 534 relationships, 0/51 India | **TRUE, exact match** | `sum(array_length(peers,1))` over US =
| `/api/peer-moves` is display-only, same-session peer price moves, never scored | **TRUE** | Module-level comment states it explicitly; veri
| `guidance_cut`/`guidance_raised` exist in the vocabulary, zero production events | **TRUE** | `lib/events/vocabulary.ts:73-86`. Not verific
| The relationship graph failed its coverage study and must not be revived | **TRUE, and the failure mode is instructive** | `features/relatio
```

1.2 New findings this review made (not in the originating prompt)

1. ****Finnhub's own documentation states its earnings-calendar EPS/revenue are non-GAAP**** – verified verbatim from ``https://finnhub.io/docs/api/earnings-calendar``: ****"EPS and Revenue in this endpoint are non-GAAP, which means they are adjusted to exclude some one-time or unusual items."**** This is unconditional, not row-specific, so it ****can safely resolve the prospective basis contract**** (\$4.2) – but only prospectively; the 56 existing rows with ``eps_basis = null`` are not retroactively reinterpreted (frozen-history rule).
2. ****Finnhub's free earnings calendar caps historical depth at one month****: ****"Free Tier: 1 month of historical earnings and new updates."**** This explains why 51 days of production history (2026-07-08 to date) yields only 9 symbols and 19 events – it is not an early-feature artifact, it is a hard provider ceiling. ``earnings-pit.ts`` cannot backfill a longer PIT history from Finnhub at any price on the free tier; only forward accumulation from first capture is possible, which the module's own comment already states.
3. ****Finnhub's `/stock/eps-estimate` endpoint is `Premium: Premium Access Required`****, and even its documented free-tier-adjacent shape is thinner than Alpha Vantage's free endpoint – no revision history, no 7/30/60/90-day deltas, just current avg/high/low/analyst-count. This settles decision \$12.2 below.
4. ****Alpha Vantage's `EARNINGS\_ESTIMATES` is live and free with Kairos's own configured key**** – verified with two real, read-only GET calls against ``ALPHA_VANTAGE_API_KEY`` (IBM and EXEL, a mid-cap Kairos holding, not just a mega-cap). Both returned real data, not a premium-lock message. Fields: ``eps_estimate_average/high/low``, ``eps_estimate_analyst_count``, ``eps_estimate_average_{7,30,60,90}_days_ago``, ``eps_estimate_revision_{up,down}_trailing_{7,30}_days``, and the revenue equivalents – for both ``fiscal year`` and ``fiscal quarter`` horizons in one call. This is the multi-horizon, revision-history, analyst-count source the prompt hypothesized, confirmed real.
5. ****Alpha Vantage's overall daily call budget is shared and its true ceiling is unverified.**** ``lib/data/provider-fetch.ts:29`` defaults ``AV_DAILY_BUDGET`` to 25/day; the checked-in ``.env.local`` does not override it; production consumed 35, 71, 60, 36, 8, 45, 67, 56, 67, 47 calls across the last 10 days (``provider_budget`` table) – already over the local default on 7 of 10 days. Either Vercel's production env sets a higher budget than this session can see, or the gate is already being routinely exceeded. ****This must be confirmed before any symbol count is committed.****
6. ****Finnhub `/stock/peers` supports a `grouping` parameter**** (``sector | industry | subIndustry``, default ``subIndustry``) that ``lib/data/symbol-profile.ts`` does not currently pass – it is already

using the narrowest, most defensible grouping by default, which is a stronger starting point for peer candidates than the architecture originally assumed.

7. `signal_score_history` has no `analyst_target` or `analyst_upside_pct` column, though `research-agent.ts:2213-2214` inserts both on every research run (`error: scoreHistErr` is captured but the surrounding code path was not traced far enough to confirm whether the whole insert fails or only those keys are silently rejected by PostgREST). **This is a pre-existing defect, out of scope for this architecture, and is flagged to Codex in the companion brief** rather than fixed here – fixing it would be implementation, and this document is architecture only.

2. Problem and user value

Kairos's live composite scorer sees: current fundamentals (P/E, margin, ROE, EPS, revenue growth – all trailing/TTM), current technicals, current sentiment, current macro regime, current insider activity. It has **no forward-looking axis** – nothing that answers "is the market's opinion of this company's *future* improving or deteriorating, and how does that compare to its peers?"

That is a genuinely different information set from momentum (which is backward-looking price) and from current fundamentals (which are backward-looking accounting). Hameed, Morck, Shen & Yeung (2010, NBER w15833) found that stocks with thin analyst coverage co-move with the price moves of heavily-covered peers in the same industry, asymmetrically – well-covered names lead, thinly-covered names follow, not the reverse. That is direct empirical support for exactly the peer-relative, asymmetric-influence design this feature proposes (§7).

User value, concretely: a bounded, auditable signal – "this company's forward estimates are improving *faster than its peers*" – tested honestly against the existing eligible-long cohort, on the same walk-forward/purged/cost-adjusted discipline every other dimension in this codebase is held to, with no shortcut to "it correlates with returns" as a stopping point (acceptance criterion #19).

3. Explicit non-goals

This feature does **not**, at any stage without a separate governed decision:

- Change scoring, eligibility, sizing, stops, targets, exits, or orders.
- Claim or model supplier/customer/partner economic relationships (that is the killed relationship graph's domain; see §1.1's last row – this feature works with **comparable-company peers**, a different and already-provider-covered data type, never economic linkage).
- Extract management guidance from any source in Stage 0 or Stage 1.
- Activate any India data path.
- Compare a symbol against the whole market as its "peer" population.
- Treat `symbol_profiles.peers` as a verified, versioned peer basket – it may only seed candidates.
- Let an LLM generate combinations and pick the backtest winner (§8).
- Invent a second lifecycle taxonomy for Upgrade Paths.

4. Source-capability findings

4.1 Alpha Vantage `EARNINGS_ESTIMATES` – verified live, recommended primary source

GET https://www.alphavantage.co/query?function=EARNINGS_ESTIMATES&symbol={SYMBOL}&apikey={KEY}

Confirmed with the project's real key against `IBM` and `EXEL` (2026-09-03, two read-only GET requests, no state changed). Response: one `estimates[]` array per symbol, mixing `"fiscal year"` and `"fiscal quarter"` horizon rows, each carrying:

- `eps_estimate_average/high/low`, `eps_estimate_analyst_count`
- `eps_estimate_average_{7,30,60,90}_days_ago` – a **built-in revision trail**, no need to reconstruct 7/30/60-day deltas from our own snapshot cadence
- `eps_estimate_revision_up_trailing_{7,30}_days` / `..._down_trailing_{7,30}_days` – **revision breadth**, i.e. contributor-level information without needing contributor-level data
- `revenue_estimate_average/high/low`, `revenue_estimate_analyst_count`

No documented `entitlement` gate or "premium" label on this endpoint (unlike `TIME_SERIES_INTRADAY`, which the same doc page marks `Trending Premium`). No basis field – Alpha Vantage does not state GAAP vs. non-GAAP for this endpoint anywhere found in this pass; **treat as unknown basis** until a further probe resolves it, same discipline as Finnhub before its docs were checked.

Does not cover: Q+2 explicitly as a distinct labeled row (only "fiscal quarter", which appears to be the *next* quarter – needs confirmation with a second real quarter's data before Stage 1 assumes a Q+2 row exists), and does not return a `snapshot_at`/`available_at` – Kairos must stamp its own capture timestamp as the point-in-time marker, exactly as `earnings-pit.ts` already does for Finnhub consensus.

4.2 Finnhub earnings calendar – confirmed non-GAAP, already wired for actuals

Verified verbatim from Finnhub's own documentation page: **"EPS and Revenue in this endpoint are non-GAAP, which means they are adjusted to exclude some one-time or unusual items ... Estimates are sourced from both sell-side and buy-side analysts."** Also confirmed: **"Free Tier: 1 month of historical earnings and new updates"** – a hard depth ceiling, not a coverage gap to fix.

Decision (see §12.3): this resolves the prospective basis contract. **New Finnhub-sourced captures**

from the date this is approved forward may set `eps\_basis = 'non\_gaap'` deterministically, since the statement is unconditional across the endpoint, not row-specific. ****The existing 56 rows with `eps\_basis = null` are NOT retroactively reinterpreted**** – they were captured under real uncertainty and stay that way; only new rows benefit. This is a plan-version-style change: a new capture-policy version starts, the old one's rows keep their original (unknown) basis forever.

4.3 Finnhub `/stock/eps-estimate` – confirmed premium, ruled out

Explicitly labeled `Premium: Premium Access Required` on Finnhub's own page. Even ignoring the paywall, its documented shape (`epsAvg/High/Low`, `numberAnalysts`, `period`, no revision fields) is thinner than Alpha Vantage's free endpoint. ****Not recommended as a source at any tier.****

4.4 Alpha Vantage call budget – UNRESOLVED, the top operational risk

`lib/data/provider-fetch.ts:29` sets the code-level default at 25 calls/day. Production's own `provider\_budget` table shows ****35, 71, 60, 36, 8, 45, 67, 56, 67, 47**** calls/day over the ten days ending 2026-09-03 – exceeding the code default on 7 of 10 days. This means one of:

(a) Vercel's production environment has `AV\_DAILY\_BUDGET` set to a value this session's `.env.local` doesn't reflect (a paid AV tier), or
(b) the budget gate (`providerCachedFetch`) is soft/advisory and is already being routinely exceeded against a real 25/day ceiling, which would mean AV calls are already at risk of being rate-limited or rejected by the provider.

****This document does not resolve which.**** It is Decision §12.7's dependency: no Stage 1 symbol count should be committed until the actual contracted Alpha Vantage plan is confirmed (ask Vaibhav, or check Vercel's env directly with appropriate access). A weekly-cadence capture across even 60 symbols is 60 calls the first week and thereafter proportional to how often estimates actually move – cheap against a paid multi-hundred/day plan, impossible against a real 25/day free ceiling shared with every existing AV consumer (fundamentals scoring, sector P/E lookups, symbol profiles' India overview reuse).

4.5 India – no verified source contract, correctly out of scope

No India-specific probe was run in this pass (explicitly deferred, matching the prompt's own instruction not to assume US/India parity). `symbol\_profiles.peers` is `[]` for all 51 India profiles by design (no free peers source found previously). Alpha Vantage's international coverage for NSE/BSE tickers, if any, is unverified – India symbols in this codebase use a Yahoo-style `.NS`/`.BO` suffix convention that does not obviously match Alpha Vantage's symbol format. ****A separate Stage 0b India probe is required before any India activation; this document proposes none.****

4.6 Management guidance sources – surveyed, not selected for Stage 0/1

- ****SEC EDGAR full-text search + 8-K EX-99.1 press releases****: has real point-in-time timestamps (`acceptanceDateTime`, confirmed reliable by the relationship-graph P0 probe: "Works. ... intraday-precise"), free, and is the only source with genuine PIT guarantees. But guidance figures inside a press release are unstructured prose – extraction needs either a deterministic regex/table parser (fragile across issuers) or an LLM (needs the full §10 boundary). ****Not built in Stage 0/1.****
- ****Investor presentations / earnings call transcripts****: even less structured, no reliable free source with PIT timestamps at Kairos's current scale. Ruled out for now.
- ****Recommendation for whenever guidance extraction is revisited****: start with a ****deterministic-first parser**** against the small set of issuers that publish a structured guidance table in their EX-99.1 (a real, testable subset – not assumed universal), and reserve the LLM path for the residual only after the deterministic parser's own coverage is measured, per the user's own instruction to evaluate deterministic-first before defaulting to an LLM.

5. Pressure test – steelman then attack

****Steelman.**** The value case is not "another momentum proxy." Estimate revisions are opinion-updates by third parties who often have private-ish information advantages (buy-side/sell-side channel checks), and the Hameed et al. finding gives a real, citable mechanism (information diffusion lag across thinly- vs. widely-covered peers) rather than a data-mined correlation. The existing scorer structurally cannot see this axis – it has no forward-looking dimension at all.

****Attack, and how the design answers each:****

- ****Does this duplicate price momentum?**** Partially, unavoidably – estimate revisions correlate with recent price action because analysts react to news too. The architecture requires a `technicalCorrelation` diagnostic (§9, mirroring `measureSectorSignal`'s own honest-prior check against `technical`) reported alongside every η measurement, exactly as `sector-regime.ts` already does for its own candidate dimension – "A high correlation here means this is technical in slow motion, which is a reason to stop rather than a detail" (verbatim precedent from the existing code).
- ****Is coverage sufficient to avoid selection bias?**** At 157/5,911 (2.7%) for the ****existing**** AV target field, and unverified for the new `EARNINGS\_ESTIMATES` endpoint until Stage 0 measures it across Kairos's real universe – this is an open question Stage 0 exists to answer, not assumed away. If coverage skews toward mega-caps with heavy analyst followings, the shadow's cohort will silently exclude exactly the thinly-covered names the Hameed et al. mechanism is about – the architecture requires reporting coverage ****by market-cap tier**** (already available via `symbol\_profiles.market\_cap\_tier`) so this bias is visible, not assumed absent.
- ****Is comparing to "all companies" invalid?**** Yes, unconditionally – no peer aggregate in this design may include the whole market; see §7.
- ****Are peer relationships symmetric?**** No – Hameed et al. is direct evidence they are not; see §7.2.
- ****Does this duplicate the killed relationship graph?**** No, if the boundary in §3 holds: peers here are Finnhub sub-industry comparables (a provider-classified taxonomy, already free and covering

97/187 US names), not disclosed economic relationships (which failed on structural SEC disclosure sparsity, a different failure mode entirely). The discipline that must be inherited, though: the graph's P0 probe measured real coverage on the real universe *before* building anything – this architecture's Stage 0 does the same for both the estimates source and the peer-basket source.

- *Does guidance extraction need an LLM?* Possibly, eventually, but not for Stage 0/1's core hypothesis test – deferred per §3.
- *Cost/entitlement/freshness feasible on free-cloud-only?* Estimates: yes, confirmed live and free (§4.1), pending the budget question in §4.4. Guidance: deferred, so moot for now.
- *Should India be deferred?* Yes, unconditionally, per §4.5 and the prompt's own instruction.

6. Required conceptual separation

Implemented as ****distinct, separately-typed evidence records****, never collapsed into one score:

```
| # | Concept | Table (see §11) | Never conflated with |
|---|---|---|---|
| 1 | Analyst consensus | `earnings_expectation_snapshots` (kind=`consensus`) | Management guidance |
| 2 | Management guidance | `management_guidance_observations` | Analyst consensus |
| 3 | Realized results | existing `earnings_calendar.eps_actual_first` (reused, not duplicated) | Consensus, guidance |
| 4 | Market-implied expectation | **out of scope** – no options-derived expectation is proposed in this document; noted as a possible future |
| 5 | Peer expectations | `earnings_expectation_snapshots` rows for peer symbols, aggregated via  $\eta$  (§7) | Own-company expectations |
| 6 | Price reaction | `reaction_residual` derived measurement (§7.3), computed against `price_cache`/benchmark series already in use elsewhere |
| 7 | Economic relationships | **explicitly not modeled** – the killed relationship-graph's domain | Peer comparables (§3, §7) |
```

7. Peer methodology and η

7.1 Candidate sourcing, not accepted membership

`symbol\_profiles.peers` (Finnhub `/stock/peers`, default `grouping=subIndustry`, already free and already fetched for 97/187 US names) ****seeds**** a candidate list. It is never read directly as the accepted peer basket by any measurement code – acceptance requires a versioned, reviewable row in `peer\_group\_versions`/`peer\_group\_members` (§11.3) with an `effective\_from` timestamp, so that acceptance criterion #8 ("peer membership used at time T is the version available at T") is mechanically enforceable, not a convention.

Basket size: ****5–15 companies****, matching the user's own bound. If Finnhub's candidate list returns fewer than 5 after ETF/instrument-family filtering (see acceptance criterion #11), the symbol is `peer\_basket\_unavailable`, not padded with weaker matches.

7.2 Asymmetric weighting – leader/follower, not a flat average

Hameed et al. (2010) found the co-movement is one-directional: thinly-covered names follow heavily-covered ones, not the reverse. This architecture's peer weight is therefore a function of ****relative analyst coverage**** (`eps\_estimate\_analyst\_count`, already returned by the Alpha Vantage endpoint – no new data source needed) between the subject symbol and each peer, not a flat 1/N. A peer with materially more analyst coverage than the subject gets a higher weight in the subject's η ; a peer with materially less gets a lower one. The exact weighting function (e.g. rank-based vs. count-ratio) is a Stage 2 design decision, not committed here – Stage 2 must show the weighting choice does not itself manufacture the effect (a leave-one-peer-influence check, per the required robustness list).

****Report order**** (which peer reports first) and ****historically measured lag**** are listed in the prompt as candidate weighting inputs; both require a longer observation history than Stage 1's data will have. Deferred to a Stage 2+ refinement once base η has enough history to measure lag honestly.

7.3 η and companion measurements

$\eta(\text{symbol}, \text{horizon}) = \text{standardized_own_estimate_revision}(\text{symbol}, \text{horizon})$

– $\text{weighted_median_peer_revision}(\text{peer_basket}(\text{symbol}), \text{horizon})$

- ****Weighted median****, not mean, per the user's own instruction – robust to one peer's outlier revision dominating a small (5–15 member) basket.
- **`standardized\_own\_estimate\_revision`**: the `eps\_estimate\_average` change over the relevant window (7/30/60/90-day trail, all provided directly by Alpha Vantage – §4.1), standardized within the symbol's own history so the units are comparable across names with very different EPS scales.
- ****A regression-residual construction is explicitly deferred****, not chosen initially – the bounded predeclared-family discipline (§8) argues for starting with the simplest defensible construction and only adding a second construction as its own separately-tested hypothesis, never as a same-family variant selected post-hoc.

Companion measurements, exactly as specified in the originating prompt, computed only where the underlying inputs exist (§4 – guidance-dependent ones are inert until Stage 2+ guidance capture exists, but their formulas are frozen here so they are not redesigned ad hoc later):

- $\text{guidance_gap} = \text{guidance_midpoint} / \text{pre_release_consensus} - 1$ – ****inert in Stage 1**** (no guidance capture yet); formula frozen for when Stage 2+ guidance lands.
- $\text{peer_revision_diffusion}$ = weighted fraction of valid peers whose own revision is positive – computable in Stage 1 from estimates alone.
- $\text{reaction_residual} = \text{symbol_post_event_return} - \text{matched_peer_basket_return}$ – computable once a basket exists; reuses existing benchmark-series/price-cache infrastructure (`getBenchmarkSeries`,

```

    already used elsewhere in `research-agent.ts`), not a new price-fetch path.
- `peer_lead_signal` – requires the lag-measurement work in §7.2; Stage 2, not Stage 1.

### 7.4 What a peer criterion explicitly is NOT

A peer is a Finnhub sub-industry comparable with an owner-reviewed, versioned, effective-dated acceptance record. It is never: a disclosed supplier/customer, a "similar-sounding" LLM-generated guess, an unreviewed raw Finnhub list read live at scoring time, or a basket that changes composition without a new version row.

---

## 8. Combination discovery – bounded, predeclared

No LLM may generate candidate interactions or select a winning backtest. The predeclared family for Stage 3 is drawn only from the user's own listed hypotheses, and only the subset whose inputs exist after Stage 1 (own/peer estimate revisions) – guidance-dependent hypotheses (`guidance_gap × price above EMA20`, `guidance_cut × breakdown veto`, etc.) are frozen as formulas, not scheduled, until Stage 2+ guidance capture exists:

Testable from Stage 1 data alone:
- upward  $\eta$  × positive technical trend
- current-quarter beat × falling Q+1/Q+2 expectations
- current-quarter miss × raised future guidance *(needs guidance – frozen)*
- inexpensive peer-relative valuation × improving forward revisions
- positive expectations × acceptable volatility
- negative expectation shock × existing exit/horizon-extension conditions

That is five testable interactions initially – well inside "small predeclared family." Every one of the mandatory methodology items the user listed (frozen formula/version identifiers, PIT construction, market/horizon-local folds, purging/embargo per horizon, baseline-vs-baseline-plus-feature on the identical eligible cohort, multiple-testing/FDR control, incremental IC, after-cost turnover, sector/size/vintage/regime robustness, peer-membership sensitivity, leave-one-symbol and leave-one-sector influence, a prospective forward shadow, no automatic promotion) is required and none of it is invented fresh – it reuses infrastructure that already exists and was verified live in this session's earlier work:

- Purged folds + horizon-matched step spacing: `lib/edges/folds.ts` (already enforces `stepSessions >= horizonSessions` and refuses the overlap that `edge-readiness.ts`'s old `INDEPENDENT_WINDOW_DAYS = 5` bug allowed – this feature's independence rule must use the SAME horizon-aware spacing Codex's 2026-09-03 fix now applies, not a third bespoke rule).
- Rank IC / Spearman with the existing floors: `MIN_PREDICTIVE_DATES = 20`, `MIN_EFFECTIVE_OBSERVATIONS = 12`, `MIN_CROSS_SECTION = 5` (used across the codebase's other dimension diagnostics – reused verbatim, not reinvented).
- Šidák/multiple-testing correction and Deflated Sharpe / PBO machinery already used by the Alpha Diagnostic Lab and the OOS orchestrator (`lib/edges/oos-experiment.ts`'s `multipleTesting.method: "trial_adjusted_t_margin"`).
- Cost: `MODELED_SLIP_FRACTION` (`lib/analytics/performance-metrics.ts`, fixed 2026-09-02) for after-cost turnover, so this feature's cost accounting matches the paper book and the replay exactly rather than inventing a fourth cost constant.

---

## 9. Upgrade Paths contract

One new `ShadowProgramDefinition` entry, id `earnings-expectations-peer-delta`, reusing the existing `ShadowLifecycle` enum at the top level (§0.2). Reports separately for US and India (India will read `not_applicable` until §4.5's probe exists and is approved).

{
id: "earnings-expectations-peer-delta",
name: "Earnings expectations & peer delta",
category: "Scoring",
markets: ["us", "india"], // india reports not_applicable until its own Stage 0 probe
...
}

Nested `stage` detail (matching the `edge_readiness_status` precedent, not a new top-level enum):
- `unavailable | collecting | measurable | shadowing | eligible_for_review | rejected | retired`

Reported fields, per market:
- collection coverage (symbols with a captured snapshot / total in-universe), freshness, latest successful run
- independent earnings dates covered, basis-comparable event count (mirrors the existing `/api/calendar/earnings/coverage` report's own methodology – reused, not reinvented)
- Q+1 / Q+2 / FY+1 horizon coverage (FY+1 confirmed available from AV; Q+2's presence unconfirmed)

```

- per §4.1, must be measured before claimed)
- peer-basket coverage (% of in-universe symbols with an accepted ≥5-member basket)
- h1/h5/h20 incremental rank IC with an HAC t-stat/uncertainty interval (reusing `neweyWestLag` from `lib/edges/folds.ts`)
- benchmark- and peer-relative expectancy, average winner, average loser, payoff ratio, profit factor – **not win percentage as the headline**, per the explicit instruction; hit rate is a secondary field only
- turnover and modeled cost (from `MODELED\_SLIP\_FRACTION`), max drawdown
- sector/size/regime stability, contribution above current dimensions (technical-correlation diagnostic from §5, reported prominently, not buried)
- weeks of prospective shadow stability, blockers, next evidence needed

Refreshes from the producer's **latest completed market-session evidence** on the existing one-minute page-refresh loop already used by every other Upgrade Path program (`UpgradePathPage.tsx`) – no new refresh cadence, and a cron invocation is never itself presented as proof the producer generated fresh evidence (matching the exact defect class Codex's 2026-09-03 review found and fixed in the Router readiness check).

10. LLM boundary

Not exercised in Stage 0 or Stage 1 – no LLM call is proposed until guidance extraction (Stage 2+) is separately approved. When it is:

- LLM output is a **candidate observation only**, never a direct write to a scored/decision table.
- Must quote/anchor the exact source passage and document (SEC filing accession number + exact paragraph), matching the mutation/replay requirement.
- Deterministic validation (metric/period/units/range/timestamp) runs on every candidate before acceptance; ambiguous values enter a review/quarantine state, never a silent default.
- Extraction confidence is recorded but is never trading confidence, and never overrides a deterministically-computed score/direction/size/stop/target/exit/order.
- No missing value becomes a neutral zero – matches every other dimension's `available: false` convention already used throughout `lib/data/scores.ts`, `lib/data/analyst.ts`, etc.
- Prompt, model, schema version, and extraction version are recorded per observation.
- A deterministic-first parser is evaluated against the structured subset of issuers before any LLM path is built, per §4.6.

11. Data model

Reuses `lib/edges/\*` (folds, PIT universe, OOS runner), the Canonical Evidence Router's provenance conventions, and the existing `earnings\_calendar` table for realized actuals – **no duplicate provenance, outcome, experiment, or promotion system is created.**

11.1 `earnings\_expectation\_snapshots` (new – replaces the ad-hoc consensus-only `earnings\_consensus\_snapshots` for THIS feature's purposes; the existing table is left untouched and still serves `earnings-pit.ts`'s PEAD-precursor use case unchanged)

```
create table public.earnings_expectation_snapshots (
  id uuid primary key default gen_random_uuid(),
  symbol text not null,
  market text not null default 'us' check (market in ('us','india')),
  kind text not null check (kind in ('consensus')), -- extend only by owner review
  metric text not null check (metric in ('eps','revenue')),
  fiscal_period text not null, -- e.g. '2026Q4', '2026FY'
  horizon_label text not null check (horizon_label in ('fiscal_quarter','fiscal_year')),
  value_avg numeric,
  value_high numeric,
  value_low numeric,
  analyst_count integer,
  revision_up_7d integer,
  revision_down_7d integer,
  revision_up_30d integer,
  revision_down_30d integer,
```

```

value_7d_ago numeric,
value_30d_ago numeric,
value_60d_ago numeric,
value_90d_ago numeric,
basis text, -- 'unknown' until a source contract proves it; AV: unknown per $4.1
currency text not null default 'USD',
source text not null, -- 'alpha_vantage'
source_call_id text, -- correlates to provider_call_ledger, for replay
captured_at timestampz not null default now(), -- our own PIT stamp; AV returns none
unique (symbol, market, metric, fiscal_period, horizon_label, source, captured_at)
);

Idempotency key: `(symbol, market, metric, fiscal_period, horizon_label, source, captured_at)` –
append-only, one row per capture event, never updated. A later capture with an unchanged value is
still a new row (unlike `earnings-pit.ts`'s dedupe-on-unchanged behavior) because the revision-count
fields (`revision_up_7d` etc.) are themselves informative even when the average estimate hasn't moved
– dropping "unchanged" snapshots here would lose that signal.

### 11.2 `management_guidance_observations` (schema frozen now, producer not built until Stage 2+)

create table public.management_guidance_observations (
id uuid primary key default gen_random_uuid(),
symbol text not null,
market text not null default 'us' check (market in ('us','india')),
metric text not null check (metric in ('eps','revenue')),
fiscal_period text not null,
guidance_low numeric,
guidance_high numeric,
guidance_midpoint numeric generated always as (
case when guidance_low is not null and guidance_high is not null
then (guidance_low + guidance_high) / 2 end
) stored,
is_withdrawal boolean not null default false, -- distinct from a numeric cut — acceptance #3
basis text,
currency text not null default 'USD',
source_document_id text not null, -- SEC accession number or equivalent
source_quote text not null, -- the exact anchored passage, mandatory for LLM-derived rows
extraction_method text not null check (extraction_method in ('deterministic','llm')),
extraction_version text,
llm_model text,
llm_prompt_hash text,
occurred_at timestampz not null, -- when made public
available_at timestampz not null, -- when Kairos could first know it
review_state text not null default 'pending' check (review_state in ('pending','accepted','quarantined')),

```



```

unique (symbol, market, metric, fiscal_period, source_document_id)
);

A withdrawal is `is_withdrawal = true` with `guidance_low`/`guidance_high` both null – never encoded
as a numeric cut (acceptance criterion #3).

### 11.3 `peer_group_versions` / `peer_group_members`

create table public.peer_group_versions (
id uuid primary key default gen_random_uuid(),
subject_symbol text not null,
market text not null default 'us' check (market in ('us','india')),
effective_from timestampz not null,
effective_to timestampz, -- null = current
candidate_source text not null, -- 'finnhub_peers_subindustry'
review_state text not null default 'pending' check (review_state in ('pending','accepted','rejected')),
reviewed_by text,
reviewed_at timestampz,
unique (subject_symbol, market, effective_from)
);

create table public.peer_group_members (
version_id uuid not null references public.peer_group_versions(id),
peer_symbol text not null,
weight numeric, -- null until §7.2's weighting function is approved
primary key (version_id, peer_symbol)
);

Enforces acceptance criteria #8 (membership at T = version available at T) and #9 (a peer reporting
after the target cannot leak into the target's earlier decision – enforced by every consumer
filtering `peer_group_versions.effective_from <= decision_ts` and requiring each peer's OWN
`earnings_expectation_snapshots.captured_at <= decision_ts`, not merely the basket's effective date).

### 11.4 Derived feature table

Reuses `edge_signals` (already the home for cross-sectional candidate dimensions like
`rel_strength_6m`, `high_52w_proximity` – see `lib/research/relative-strength-discovery.ts`) rather
than a new table. New `edge_id` values: `earnings_eta`, `peer_revision_diffusion`,
`reaction_residual` (Stage 1+); `guidance_gap`, `peer_lead_signal` (Stage 2+, inert until guidance
exists).

### 11.5 Shadow-run summaries

Reuses the existing `edge_ic_history` / `edge_readiness_status` shape (§9) rather than a parallel
experiment/outcome system – Stage 3's shadow runs write to the same tables `technical-calibration`
already writes to, tagged by `edge_id`.

### 11.6 RLS posture

Every new table: RLS on, `deny anon`, `authenticated` may `SELECT` only, all writes via
`service_role` – the exact pattern `earnings_consensus_snapshots`'s own migration already
establishes (`supabase/migrations/20260715210100_earnings_consensus_snapshots_vintages.sql:29-35`),
reused verbatim.

---

## 12. Decisions

### 12.1 GO / RESHAPE / NO-GO

**RESHAPE.** See §0.

### 12.2 Best first estimate source

```

****Alpha Vantage `EARNINGS\_ESTIMATES`**, confirmed live and free with the project's existing key, multi-horizon, with built-in revision history – richer than Finnhub's paywalled thinner alternative (\$4.1-4.3). This is not "given existing credentials and quota" hand-waved – it is two live, verified, read-only calls against the actual production key.**

12.3 Can Finnhub's non-GAAP basis safely unblock prospective comparisons?

****Yes, prospectively only.**** Verified verbatim, unconditional across the endpoint (\$4.2). New captures from approval-date forward may set `eps\_basis = 'non\_gAAP'`. The 56 existing `null`-basis rows are never rewritten.

12.4 Smallest defensible management-guidance source for Stage 0

****None – Stage 0/1 build no guidance capture at all**** (\$0, \$4.6). If forced to name the smallest defensible source for a future stage: SEC EDGAR 8-K EX-99.1 press releases via full-text search, deterministic-parser-first.

12.5 What qualifies a company as a peer

Finnhub `/stock/peers?grouping=subIndustry` candidate, owner-reviewed and accepted into a versioned, effective-dated `peer\_group\_versions` row. 5-15 members. Never `symbol\_profiles.peers` read live. Never an economic-relationship claim.

12.6 η construction

****Weighted median**** initially (\$7.3), per the user's own instruction. Regression-residual explicitly deferred to a separately-tested Stage 2+ hypothesis, not a same-family variant.

12.7 Minimum coverage to build Stage 1

****Unresolved pending \$4.4.**** Proposed floor once the AV budget question is answered: ≥ 60 US symbols with both FY+1 estimate coverage and an accepted ≥ 5 -member peer basket, reaching the existing `MIN\_CROSS\_SECTION = 5` floor on a real trading day. This number is a placeholder pending the budget answer, not a commitment.

12.8 Evidence required before Stage 3 reports a number

Reuses the existing dimension-diagnostics floors verbatim (\$8): `MIN\_PREDICTIVE\_DATES = 20`, `MIN\_EFFECTIVE\_OBSERVATIONS = 12`, `MIN\_CROSS\_SECTION = 5`, horizon-aware independent-window spacing (matching Codex's 2026-09-03 fix, not the flat 5-day rule that fix replaced).

12.9 Exact promotion gates

Reuses `edge\_readiness\_status`'s two-tier historical/validation gate structure verbatim – no new promotion-gate design. A validated feature may request shadow review; it cannot self-promote past that (matches `technical-calibration`'s own `maximumInfluence`).

12.10 What stays permanently display-only

Peer-move-style "what's happening" surfaces (raw revision news, a guidance calendar if ever built) – never a trade signal without clearing the full Stage 3/4 gate, matching `/api/peer-moves`'s own existing, unambiguous precedent.

12.11 Feasible India work now

None. A Stage 0b India capability probe (Alpha Vantage NSE/BSE symbol format, alternative India estimate sources) is the only India work this document proposes, and it is not scheduled – it requires a separate approval.

12.12 Does any part revive the relationship graph?

No, if the boundary in \$3/\$6/\$7.4 is enforced. The risk is real enough to name explicitly: if a future stage ever tries to reweight peers by "how much revenue symbol A gets from symbol B," that IS the killed graph, under a new name, and must be refused.

12.13 What's missing from the originating prompt

The Alpha Vantage shared call-budget risk (\$4.4) – the prompt asked about "provider cost, entitlement, call volume, freshness, and free-cloud feasibility" in the abstract, but the concrete production number (35-71 calls/day already in flight against a 25/day code default) was not knowable without querying `provider\_budget`, and it is the single largest feasibility gate on Stage 1's scope. Also missing: what happens to this feature's evidence if the champion scorer's weights change mid-collection (the existing dimension-diagnostics machinery handles this via `analysis\_plan\_version`; this feature should adopt the same versioning discipline, not a bespoke one – noted here so a Builder doesn't have to rediscover it).

13. Phased implementation (unchanged from the originating prompt's shape, reflecting \$0's reshape)

Stage 0 – source-contract probes

- Confirm the real Alpha Vantage plan/quota (\$4.4) – ****blocks everything else****.
- Measure `EARNINGS\_ESTIMATES` coverage across Kairos's real US universe (not just IBM/EXEL), broken down by market-cap tier (\$5).
- Confirm whether a `fiscal quarter` row beyond the immediate next quarter (i.e. a true Q+2) exists

in a real multi-quarter response.

- No India probe scheduled (§4.5, §12.11) unless separately approved.
- Stop cheaply if coverage or basis integrity fails – same discipline as the relationship-graph P0.

Stage 1 – immutable expectation capture (US only)

- Capture `earnings\_expectation\_snapshots` on the existing earnings-refresh cadence, symbol count set by Stage 0's budget answer.
- No scores, candidate admission, or trades. No guidance capture.
- Reuse Canonical Evidence Router provenance conventions; RLS per §11.6.

Stage 2 – audited peer baskets and η (US only)

- `peer\_group\_versions`/`peer\_group\_members`, seeded from Finnhub, owner-reviewed acceptance.
- Compute η , `peer\_revision\_diffusion`, `reaction\_residual` into `edge\_signals`.
- No economic-relationship claims (§3, §12.12). No India.

Stage 3 – feature and interaction shadows

- Register the five Stage-1-computable predeclared interactions (§8).
- Reuse `lib/edges/folds.ts`, the existing IC/FDR/cost machinery (§8).
- Surface through the existing `ShadowLifecycle`-based Upgrade Paths entry (§9), nested `stage` field.

Stage 4 – promotion proposal only

- A passing shadow creates a reviewable challenger proposal via the existing promotion-proposal pattern. Cannot alter Champion weights, eligibility, sizing, stops, targets, exits, or broker actions. Owner approval mandatory.

Guidance capture (management_guidance_observations, guidance-dependent interactions, LLM boundary exercised) is a separately-approved Stage 2.5+/5+ decision, not scheduled by this document.

14. Acceptance criteria

All 20 from the originating prompt are load-bearing here; mapped to the concrete mechanism that enforces each:

1. Beat + future-guidance-cut stay separate facts – `earnings\_calendar.eps\_actual\_first` (realized) and `management\_guidance\_observations` (guidance) are different tables, never merged.
2. Guidance midpoint is a generated column from low/high (§11.2) – low/high are never dropped.
3. Withdrawal is `is\_withdrawal = true` with null low/high, never a numeric cut (§11.2).
4. A post-announcement estimate cannot masquerade as pre-announcement consensus – enforced by `available\_at`/`captured\_at` comparison, the same `isStrictlyPreAnnouncementVintage` discipline `earnings-pit.ts` already implements, reused not reinvented.
5. GAAP actual vs. adjusted estimate – `basis` column on both sides; comparison code must require matching, non-null basis (mirrors `/api/calendar/earnings/coverage`'s own `hasComparableConsensus` check verbatim).
6. Q+2 cannot join to a Q+1 actual – enforced by `fiscal\_period` + `horizon\_label` exact match in every join.
7. A later restatement cannot overwrite first-observed – append-only tables, no UPDATE except the existing `restated\_eps` pattern on `earnings\_calendar`, unchanged.
8. Peer membership at T = version at T – `peer\_group\_versions.effective\_from/effective\_to` filter, mandatory in every consumer (§11.3).
9. A peer reporting after the target cannot leak – dual timestamp check, target's `decision\_ts` against both the peer's own snapshot `captured\_at` and the peer basket's `effective\_from` (§11.3).
10. US/India never pool – every table's `market` column is part of every aggregation `group by`, and India stays `not\_applicable` throughout Stage 0-4 of this document (§4.5).
11. ETFs excluded – reuses `classifyInstrument`/`KNOWN\_US ETFS` (already used by the relationship-graph P0 probe and `lib/asset-classification.ts`), not a new classifier.
12. No single symbol/sector silently dominates – required leave-one-symbol/leave-one-sector influence checks (§8), reported alongside every IC number.
13. LLM extraction mutation cannot alter the money path – §10, no LLM output writes to a scored table directly; Stage 0/1 don't exercise the LLM path at all.
14. Empty/insufficient cohorts report unavailable, not zero/neutral – matches every dimension's existing `available: false` convention (`scoreAnalyst`, `scoreFundamentals`, etc.).
15. Upgrade Paths distinguishes scheduled/succeeded/fresh/measurable/eligible – nested `stage` field (§9), reusing `edge\_readiness\_status`'s own stage vocabulary rather than inventing a new one.
16. Replacing eligible cohort with all-scored causes a detector to fail – mutation test required in Stage 3's test suite, same discipline as this session's cost-fix mutation checks.
17. Relaxing as-of timestamp or basis predicate causes a detector to fail – mutation test required.
18. A shadow reproduces from immutable evidence IDs and formula versions – `source\_call\_id`, `extraction\_version`, `analysis\_plan\_version` (adopted from dimension-diagnostics, §12.13).
19. Shows the feature adds value over the existing score, not mere return correlation – baseline-vs-baseline-plus-feature comparison on the identical eligible cohort is mandatory (§8), not a standalone IC number.
20. No implementation phase activates automatically – every stage transition in §13 requires the explicit approval gate in `AGENTS.md` ("Approved", "Proceed", "Code it", etc.).

15. Rollout / rollback

- Stage 0/1: no rollback needed – pure capture, no reads by any other system.
- Stage 2: peer baskets are versioned and additive; disabling the feature means the Upgrade Paths entry stops advancing its `stage`, nothing else changes.
- Stage 3/4: shadow-only by construction; rollback is "stop calling the shadow route," identical to every other measure-only shadow in this codebase.

- No stage in this document writes to any table another system reads, so there is no rollback scenario that requires a migration reversal.

16. Costs and operational risks

1. ****Alpha Vantage shared budget – unresolved, top risk.**** \$4.4, \$12.7, \$12.13.
2. ****Finnhub free-tier one-month depth**** caps how much PIT history this feature (for actuals; it already reuses `earnings\_calendar`) can ever have without a paid tier – a ceiling, not a bug.
3. ****Coverage/selection bias by market-cap tier**** – must be measured and reported, not assumed away (\$5).
4. ****`signal\_score\_history` schema drift**** (\$1.2 item 7) – a pre-existing, unrelated defect this review surfaced; flagged to Codex, not fixed here.
5. ****Peer-basket review burden**** – an owner-reviewed acceptance step (\$7.1, \$11.3) does not scale to thousands of symbols; acceptable at Kairos's current ~187-symbol US universe, would need a different design at 10x that size.

17. Unresolved decisions requiring Vaibhav's approval

1. Confirm the real Alpha Vantage plan/quota (\$4.4) before any Stage 1 symbol count is set.
2. Approve or reject this document's Stage 0/1 scope (\$13) as the first Builder task.
3. Confirm the peer weighting function for \$7.2 once Stage 2 has enough data to choose between candidates (not a Stage 0/1 blocker, flagged for later).
4. Decide whether Stage 2.5+/5+ guidance extraction is ever built, and if so, approve its own architecture separately – this document deliberately does not schedule it.
5. Decide whether a Stage 0b India probe is worth running at all before India work of any kind.

18. Files a Builder would create or modify (Stage 0/1 only – nothing beyond is authorized by this document)

****Create:****

- `lib/data/earnings-expectations.ts` – `EARNINGS\_ESTIMATES` fetch + capture, mirroring `earnings-pit.ts`'s structure (fail-soft per symbol, append-only writes).
- `supabase/migrations/<date>\_earnings\_expectation\_snapshots.sql` – \$11.1's table, RLS per \$11.6.
- `app/api/calendar/earnings/expectations/coverage/route.ts` – coverage report, mirroring `/api/calendar/earnings/coverage/route.ts`'s exact structure and caveats style.
- `tests/earnings-expectations.test.ts` – capture logic, idempotency key, PIT-vintage rules.

****Do not touch:****

- `lib/data/earnings-pit.ts`, `earnings\_consensus\_snapshots` – left exactly as-is; this feature does not replace or migrate that table.
- Any file under `lib/data/scores.ts`, `lib/research-agent.ts`'s scoring/weighting logic, any eligibility/sizing/exit/order path – Stage 0/1 write no scored dimension.
- `symbol\_profiles.peers` fetch logic – unchanged; Stage 2, not Stage 0/1, is when peer versioning begins, and even then the existing fetch is a candidate SOURCE, not something to rewrite.
- `lib/shadows/registry.ts`'s `ShadowLifecycle` type – reused, never redefined.

Source: features\earnings-repricing-barrier\FEATURE_ARCHITECTURE.md
Earnings Repricing Barrier

Status: Approved corrective safety change, implemented 2026-07-30.

Problem

Daily technical inputs can remain pinned to the pre-earnings close after an AMC or BMO release. On 2026-07-30, MSFT was a held position and received a deterministic `short` from a July 29 close despite a July 29 after-close result. That is a stale-data decision, not an informed reaction.

Rule

When the existing PIT earnings calendar has a recent event that has already occurred, a research score cannot direct an entry or a score/direction exit until the daily candle series contains the event reaction. A before-open report may use the completed report-date bar; an after-close or unknown-session report requires a later bar. A known past event activates the barrier even when the actual-result feed is late.

- New candidate: direction becomes `neutral` and is ineligible for PaperTrader. The signal remains session-validated so consumers see this current abstention rather than falling back to an older pre-event direction.
- Held position: direction becomes `neutral`, preventing stale score/direction exits. Price stops, targets, trailing stops, and time exits are unchanged.
- The calendar lookup is read-only and provider-free. It never infers that an earnings beat means buy, and it never uses options as directional alpha.
- Calendar read failure is an unknown control-plane state and therefore writes a current neutral abstention. It cannot authorize a stale score. The lookup has no seven-calendar-day expiry: the most recent prior event remains checked until a qualifying post-event bar exists.

- Every collector uses the same real `YYYY-MM-DD` validator. Impossible values such as `2026-02-31` are unavailable, never normalized into another day.

Scope

No migration, new provider, options call, score-weight change, manual-live change, or paper/live order is added. The existing next research session after a complete post-event daily bar resumes normal deterministic scoring.

Source: features\edge-calibration-readiness\FEATURE_ARCHITECTURE.md
Edge Calibration Readiness Monitor

Status: Approved
Owner: Kairos
Decision date: 2026-07-21
Scope: Deterministic, measure-only US and India evidence governance

1. Problem

Weekly EdgeIC runs collect immutable factor evidence, but Kairos does not currently answer three operational questions reliably:

1. How many independent weekly windows have accumulated for each factor, market, and horizon?
2. Has collection stalled or degraded?
3. When is the evidence sufficient to build the next validation phase or request a shadow review?

Relying on a human to remember to check in six to eight weeks is fragile. Treating a single `shadow\_eligible` IC row as readiness is worse: it ignores stability, point-in-time quality, costs, turnover, and multiple testing.

2. Decision

Build a deterministic readiness monitor over the existing append-only `edge\_ic\_history` ledger. It publishes progress to the Edge dashboard and emits a one-time informational notice when a factor crosses a governance milestone.

It has no LLM and no authority over scoring, signals, mandates, positions, cash, proposals, brokers, or orders.

3. Milestones

3.1 `collecting`

Fewer than six independent market-wide windows exist. Windows use the latest snapshot for each `window\_end`. Policy `edge-readiness.v2-horizon-spaced` requires at least $\lceil \text{horizon_sessions} \times 7 / 5 \rceil$ calendar days between selected windows (7/14/28 days at h5/h10/h20), so manual reruns, provider corrections, and overlapping forward-return periods do not inflate progress. The earlier flat five-day rule is retained only as historical policy context; projections may demote when reevaluated under v2, while immutable IC history remains untouched.

3.2 `needs\_stability`

Six windows exist, but the bounded historical diagnostic does not satisfy every Kairos policy gate:

- minimum observation count of 72 in every selected window;
- positive IC in at least five of six windows;
- median IC at least 0.02; and
- median Newey-West t-statistic at least 1.5.

These thresholds are conservative Kairos governance policy, not a claim of universal statistical truth. They may be versioned later; old results remain immutable.

3.3 `ready\_for\_validation\_build`

The historical stability gates pass, but fewer than four independent rows carry the future evidence quality `pit\_walk\_forward\_cost\_adjusted\_fdr` with non-null turnover and net-of-fee IC. This milestone means only: build or run the point-in-time, walk-forward, cost-adjusted, multiple-testing-controlled validation layer.

3.4 `ready\_for\_shadow\_review`

At least four independent validation windows exist and all of these gates pass:

- evidence quality is exactly `pit\_walk\_forward\_cost\_adjusted\_fdr`;
- net-of-fee IC is positive in at least three of four windows;
- median net-of-fee IC is at least 0.01; and
- turnover is finite and non-negative in every validation window.

This remains a review milestone. It never changes an Edge lifecycle state and never

grants production, paper, or live trading permission.

3.5 FDR Contract Before Any Validation Row Can Qualify

``pit_walk_forward_cost_adjusted_fdr`` is not a label a route may set by convention. Before the first row claims that evidence quality, the validation design must specify and persist: the complete predeclared trial family (factor formula/version, market, horizon, segment, and parameter variants), the family size, raw p-value, and the Benjamini-Hochberg procedure at ``q = 0.05``. The correction is applied independently per market and validation family; US and India are never pooled. Missing/non-finite p-values, an unknown family size, or a changed family definition fail closed and keep the row below the validation milestone.

The current EdgeIC implementation does **not** yet supply this contract or immutable candle vintages. Consequently no current row may claim ``pit_walk_forward_cost_adjusted_fdr``, and the monitor may report only collection or historical-stability progress until a separate approved validation-layer build.

4. Data Model

``edge_readiness_status`` is a latest-state projection keyed by ``(edge_id, market, horizon)``:

- stage and policy version;
- observed/required windows and positive-window count;
- median IC/t-stat, minimum observations, latest window/session;
- validation-window counts and median net-of-fee IC;
- deterministic gate details;
- separate first-notified timestamps for each readiness milestone; and
- evaluation timestamp.

The immutable source remains ``edge_ic_history``. The projection may be updated because it is not an evidence ledger. RLS is enabled and access is service-role only, matching ``edge_market_status``.

5. Scheduling And Alerts

A provider-free daily cron runs after the Monday EdgeIC time slot. It:

1. reads only market/all rows;
2. collapses same-window provider revisions to the latest snapshot;
3. evaluates every factor/market/horizon independently;
4. upserts the latest projection;
5. emits at most one batched informational notice per market/milestone/run for newly qualifying factor-horizons;
6. emits a warning when a market has no successful IC snapshot for more than ten days; and
7. resolves the stall warning after recovery.

Daily evaluation is deliberately cheap and idempotent: it reads only Supabase rows and makes no market-data request. This lets a missing weekly IC run become visible on day 11 rather than waiting for another weekly monitor invocation.

One-time readiness notices use separate milestone timestamps in the projection. Dismissing a notice does not cause it to reopen every week, and a regression from a later milestone cannot re-notify an earlier one. A later policy/formula version gets a new identity and can notify independently.

6. Dashboard

The Edge page shows:

- market and horizon;
- stage;
- weekly-window progress;
- positive-window count;
- median IC and minimum sample;
- validation-window progress; and
- the next required action.

No visible status may use the phrase “trade ready.” Empty or failed reads render as unavailable, never as zero evidence or ready.

7. Safety And Failure Modes

- US and India are never combined.
- Sector rows are diagnostic and never satisfy market readiness.
- Same-window reruns and horizon-overlapping windows cannot increase the independent-window count.
- Legacy/unverified rows may count only as retrospective diagnostics, never as validation windows.
- Null/NaN metrics fail their gate.
- A route or DB failure reports a monitor warning but cannot affect EdgeIC itself.
- Readiness alerts are informational; stalled collection is warning severity.
- No provider calls are made by the monitor.

8. Acceptance Criteria

- Pure evaluator tests cover deduplication, weekly spacing, thin samples, unstable signs, historical readiness, validation readiness, market isolation, and nulls.
- Route source contains no score, signal, mandate, position, cash, proposal, broker, or order write.
- One-time notification state survives alert dismissal.
- Dashboard exposes truthful progress and the next gate.
- Cron, migration, RLS, TypeScript, tests, production build, and live smoke pass.

Source: features\edge-factor-discovery\FEATURE_ARCHITECTURE.md
Edge/Factor Discovery + Signal Validation + Regime Filter

****Status: P0/P1 MEASURE-ONLY BUILT. P2+ NOT APPROVED OR WIRED.****

Author: Claude (Kairos). Date: 2026-07-08. Reviewed/updated by ChatGPT (Codex), 2026-07-08 – source-backed methodology review through 2026. Intended reviewers: other LLMs / quant-literate readers. This doc is written to be self-contained enough to critique without the codebase in hand.

0. TL;DR

Kairos today generates trade ideas by having an LLM read ~15 news headlines and invent "themes," then scores candidates on 5 soft dimensions and paper-fills the top ones. That makes the ****LLM the alpha source**** – a black-box, news-chasing, overfit-prone origin that the systematic-trading literature explicitly warns against.

This proposal moves Kairos to the ****standard institutional shape****:

1. ****Discovery = a catalog of explicit, academically-grounded EDGES**** (factor, technical, calendar, structural, information), each a deterministic formula – not an LLM guess.
2. ****Every signal must earn its way in**** via an ****Information Coefficient (IC) stability gate**** (rolling IC, IR, t-stat, decay) before it is allowed to size normal paper capital or become live-eligible. Tiny, capped exploratory paper may be allowed for candidate edges so the system can learn without pretending the edge is proven.
3. ****A ranked Regime Filter**** starts as a continuous size scaler, scored on protection / cost / stability / ****cross-market reach****, replacing today's implicit, score-only regime handling. A hard on/off switch is a later, evidence-gated owner decision.
4. ****The LLM is demoted**** from alpha source to ****research synthesizer****: hypothesis generation from literature, explanation, code/report drafting, orchestration. It never originates the number that sizes a position.
5. ****Validation stays fees-first + walk-forward OOS + Monte Carlo**** (extends the existing Validation Engine), so nothing trades live without surviving that.

The existing evolution machinery (genome, champion/challenger, shadow decisions, learner, calibration, Build 4a slip) is ****reused, not replaced**** – this proposal gives it a better ***input*** (validated factor signals) and a better ***gate*** (IC + regime), rather than rebuilding the loop.

0A. ChatGPT reviewer correction – methodology update

The direction is correct, but the first draft over-restricts exploration and overstates regime gating. The safer target architecture, based on current factor-research practice through 2026, is:

1. ****ThemeScout remains attention discovery only.**** It may add candidates to the watchlist, but it never creates tradeable alpha or trade permission.
2. ****EdgeScout / FactorScout becomes the deterministic alpha factory.**** It computes formula-based edge values and validates them statistically.
3. ****Candidate edges may enter shadow/exploratory paper with tiny caps before full IC proof.**** Otherwise the system cannot explore. Normal paper sizing and any live eligibility still require validation.
4. ****Regime starts as a continuous size scaler, not a hard on/off switch.**** A hard block can be added later only after measured evidence and owner approval.
5. ****Point-in-time data becomes a first-class gate.**** No factor result is trusted unless every input has **`source`, `as\_of\_date`, `available\_at`**, revision policy, and corporate-action adjustment policy.
6. ****Long-only validation is required.**** Kairos does not run long/short books; IC is useful, but the decisive test is whether the top-ranked long-only bucket beats cash/benchmark after realistic costs.
7. ****Newly discovered factors face stricter multiple-testing gates than academically-prior factors.**** A t-stat around 2 can be enough for known priors to enter shadow/paper; newly discovered/data-mined edges need a higher hurdle such as t-stat > 3 or FDR/q-value control before active/live eligibility.

1. Motivation – what the literature says vs. what we do

Sources the owner supplied (SetupAlpha Medium series: "Scientific Workflow for

```
| Principle | Kairos today | Gap |
|---|---|---|
| Ideas come from **academic factors / documented edges** (SSRN, arXiv), not vibes | LLM invents "themes" from 15 headlines | No factor basis
| **Test the signal first** – rolling IC/IR stability, t-stat, judged at the *median* parameter (not the best) | Whole *strategies* are walk-
| **Regime filter** as explicit scaler first, ranked on protection/cost/stability/**reach** (works SPY+QQQ+India proxy, not one chart) | Impl
| **Fees-first, worst-case** friction | Build 4a slip tracking + notional caps | Partial – decent |
| **Stack multiple edge categories** | Single LLM-score edge | One edge, not stacked |
| **Understand WHY (causal); never trade a black-box signal** | LLM *is* the signal (black box) | Inverts the principle |
| **Judge by out-of-sample, not in-sample; expect overfit if you test many signals** | Shadow decisions + validation exist | IC/deflated-Sharpe
```

— — —

****Goals****

- **Non-goals****

- — —

[illegible]

■ factor · technical · calendar · structural · information ■

[illegible]

▼

[REDACTED]

[REDACTED]

▼

[illegible]

■ rolling IC, IR, t-stat, half-life/decay, turnover, net-of-■

▼

■ ENSEMBLE / COMPOSITE SCORE (IC-weighted blend of active ■

■ gated by

■ REGIME FILTER (ranked, per market) → advisory/size scale ■

existing pipeline: research signals $\rightarrow$ paper-trade (freshness/claim) $\rightarrow$

LLM (DeepSeek/Claude via agent config) sits BESIDE this: proposes new edges

— — —

4. The Edge Library

- ****Factor**** (cross-sectional, academically documented): 12-1 momentum, value (E/P, FCF yield, B/P), quality (ROIC, gross-profitability, accruals), low-volatility, size, short-term reversal, earnings-revision/PEAD, investment/asset-growth.
- ****Technical / trend****: distance to SMA/EMA/KAMA/WMA, breakout (n-day high), RSI, MACD state, ATR-normalized momentum.
- ****Calendar / time****: turn-of-month, day-of-week, holiday drift, seasonality, FOMC drift – used as **conditioners**, not standalone.
- ****Structural****: index-inclusion drift, ETF-flow pressure, sector rotation, post-earnings-announcement-drift window.
- ****Information**** (later phase; needs alt-data): insider-cluster buys, 13F flow, news-sentiment surprise vs. estimate. (This is where the **current** LLM/news work is repurposed – as an **information edge input**, not the whole engine.)

Reuse: the ****Feature Registry**** (migration ~065-era, whitelisted-grammar-only) already exists for exactly this "safe formula" purpose – extend it, don't reinvent.

```
## 5. Signal computation + store
```

```
`edge_signals(symbol, date, edge_id, raw_value, z_value, universe_id)`.
```

Cross-sectional standardization + winsorization is the standard pre-IC step.
Neutralization (sector/beta) optional per edge (flag in metadata).

— — —

6. The IC/IR validation gate (the heart of it)

```
- **IC_t = Spearman rank-corr( z_value_t , forward_return_{t→t+h} )** per period.  
(Rank IC is robust to outliers vs. Pearson.)
```

- **Mean IC**, **IC volatility**, **IR** = $\text{mean(IC)}/\text{std(IC)}$ (Grinold's fundamental law: $\text{IR} \approx \text{IC} \cdot \sqrt{\text{breadth}}$).
- **t-stat** of IC series vs. 0; require significance.
- **Decay / half-life**: IC by horizon (1,5,10,20d) – is the edge fresh or stale?
- **Turnover** implied by the signal → fee drag (fees-first: net-of-cost IC).
- **Stability**: rolling IC must not flip sign / collapse (judge the *plateau*, not the peak – the article's core point about not cherry-picking the best parameter).

Promotion state machine (per edge, per market):

candidate → measure_only → shadow → exploratory_paper → active_paper → live_eligible → live_approved

- candidate → measure_only: deterministic formula compiles, inputs exist, and the economic rationale is explicit.
- measure_only → shadow: point-in-time input availability is proven and the signal has enough historical observations to compute rank-IC.
- shadow → exploratory_paper: small positive gross IC, positive direction in multiple windows, and no major data-quality defects. Allocation is tiny and capped; this is exploration, not proof.
- exploratory_paper → active_paper: mean rank-IC $\geq \tau_{ic}$ (e.g. 0.02–0.03 for daily equity factors), $\text{IR} \geq \tau_{ir}$, positive **net-of-fee** IC, stability over K windows, turnover/liquidity acceptable, and correlation to existing active edges < 0.8 .
- active_paper → live_eligible: WFO/OOS + Monte Carlo pass, benchmark-relative long-only top-bucket alpha positive after costs, and decay monitor clean.
- live_eligible → live_approved: owner approval only.

Statistical hurdle:

- Academically-prioried/known factors may enter shadow or exploratory paper with t-stat ≈ 2 when other gates pass.
- Newly discovered/data-mined factors need a stricter multiple-testing hurdle before active/live eligibility: t-stat > 3 or explicit FDR/q-value control.
- Overlapping forward returns require autocorrelation-aware standard errors (e.g. Newey-West) rather than naïve IID t-stats.

active_paper → benched when rolling IC/alpha decays below a floor for M windows. benched → retired after prolonged failure. All transitions are logged and owner-visible.

This gate is what stops "test 100 signals, one looks amazing by luck" (the overfitting failure mode). Complement with **deflated Sharpe / multiple-testing correction** (López de Prado) on the number of edges trialed.

7. Ensemble → existing genome

active edges are blended into a **composite cross-sectional score** per symbol, weighted by each edge's recent IC/IR (shrunk toward equal-weight to avoid over-fitting the weights – this is where the **existing Learner + genome weights** plug in: the genome already carries per-dimension weights and the learner already mutates them evidence-bound). The composite replaces today's "5 soft LLM dimensions" as the score that drives entry threshold + Kelly sizing (which already read the champion genome – see genome-live.ts).

Net effect: minimal change downstream. `research_agent` emits the composite score into `agent_signals`; everything after (paper-trade freshness/claim, monitor, learner, validation) is untouched.

8. Regime filter (ranked, portable, scaler first)

A dedicated module, evaluated per market, that outputs `{state: risk_on | risk_off | neutral, scale ∈ [0,1]}`. In the first implementation this is **advisory/measure-only**, then a **continuous size scaler**. It must not become a hard new-entry block until the scaler has measured protection/cost evidence and the owner explicitly approves the harder behavior. Exits always remain allowed.

Candidate filters (implement several, rank them, pick per the article's rubric): SMA(100–300) above/below, EMA, KAMA (adaptive – caught COVID in the study), WMA, Hull, TEMA, n-day-high freshness, higher-lows, trend-quality (R^2 of price vs. time), price-within-x%-of-high. **Rank each on:**

1. **Deal** – MAR / return-vs-drawdown improvement over buy-and-hold.
2. **Crash coverage** – % of each bear (dot-com, 2008, COVID, 2022) avoided.
3. **Cost** – annual return given up (the insurance premium).
4. **Reach** – must work on SPY **and** QQQ **and** (crypto/India proxy), scored at the **median** parameter across its whole range, not the best.

Chosen filter (+ runner-ups as an ensemble vote) becomes the regime **scaler**. Stored in `regime_filters` (definition + rolling scorecard) and `regime_state` (current per market). This augments the implicit score-only regime and the `macro_regime` table's advisory-only role (macro stays as a *slow* overlay).

Push-back note (CLAUDE.md): the repo's locked decision says *no explicit*

bull/bear switching* – "scoring should adapt." This proposal **re-opens that decision deliberately**, because the regime-filter literature is strong and the current implicit approach has no measured protection/cost. Flag for owner: this is a conscious contradiction of an existing locked rule and needs explicit sign-off. The approved first cut honors the old rule: keep regime as measure/advisory first, then a *sizing scaler* only. Never hard on/off without a new owner approval.

9. Money-safety invariants (unchanged, must hold)

- Additive migrations only; verify applied before schema-coupled code ships.
- No autonomous live trading; owner click + `requireOwner` on every live order.
- No LLM places/cancels live orders, mutates money limits, mutates the active live strategy, or approves its own promotion. Here the LLM is even *further* from execution (it only proposes catalog entries as `candidate`).
- Append-only ledgers (paper_trades, paper_order_events) – inserts only.
- US(USD)/India(INR) money limits stay currency-separated.
- Long-only new positions; SELL only if held.
- Nothing goes `active`/live without passing IC gate **and** the existing fail-closed Validation Engine (WFO OOS + Monte Carlo) **and** owner promotion.

10. Data model (new, additive)

- `edge\_catalog(edge\_id, name, category, formula\_spec, inputs, rationale, expected\_sign, horizon\_days, data\_source, references, status, created\_at)`
- `edge\_signals(symbol, date, edge\_id, market, raw\_value, z\_value, universe\_id)` (partitioned/indexed by date+market; this is the big table)
- `edge\_universe\_members(universe\_id, market, symbol, as\_of\_date, source, included\_reason, created\_at)` – records exactly which symbols were eligible for a universe/date so IC results are auditable and survivorship bias is visible.
- `edge\_signal\_inputs(edge\_signal\_id, input\_name, source, as\_of\_date, available\_at, revised\_at, adjustment\_policy, raw\_ref)` – mandatory audit trail for point-in-time validation. No edge may be promoted if its inputs cannot prove when the app could have known the value.
- `edge\_ic\_history(edge\_id, market, window\_end, ic, ic\_ir, t\_stat, horizon, net\_of\_fee\_ic, turnover, status\_after)`
- `regime\_filters(filter\_id, definition, params, scorecard jsonb, active bool)`
- `regime\_state(market, date, state, scale, filter\_id, detail)`

All read-heavy analytics; no changes to money tables.

11. Integration points (existing files, indicative)

- `lib/research-agent.ts` – replace 5-dim soft score with the composite edge score; keep LLM call only for the written thesis/explanation. (Path verified: it is `lib/research-agent.ts`, not `lib/agents/research-agent.ts`.)
- Feature Registry / whitelisted grammar – `lib/validation/feature-compiler.ts` (verified) hosts the safe-formula grammar; extend it for edge formulas.
- `lib/validation/engine.ts` (Validation Engine, verified) + `app/api/agents/backtest/\*` – add IC gate + deflated-Sharpe; extend WFO to the edge composite.
- `lib/validation/genome-live.ts` + Learner – consume IC-weighted edge blend as the weights it tunes (evidence-bound mutation already exists).
- `app/api/agents/paper-trade/route.ts` – unchanged (freshness/claim/sizing all reused); it just fills better-founded signals, now also gated by `regime\_state`.
- Macro Sentinel (`macro\_regime`) – demoted to slow overlay beneath the fast regime filter.
- Dashboard – new read-only panels: edge catalog + IC scorecards, regime state.

12. Rollout phases (each independently shippable + measurable)

****Current reality (2026-07-18):**** P0 and P1 exist, but the first broad run correctly collapsed the initial watchlist-only IC results to approximately zero. No edge is shadow-eligible in production. Migration
`20260718140000\_edge\_evidence\_collection\_hardening` restarts bounded prospective US/India collection, stores sample/provenance metadata, and moves lifecycle state from the misleading global `edge\_catalog.status` to `edge\_market\_status`. Historical IC remains labeled `retrospective\_current\_universe`; it cannot authorize P2.

- ****P0 – Edge library + signal store (measure-only).**** Implement 6-8 price/volume technical edges only, compute daily, store `edge\_signals`, and record the exact `edge\_universe\_members` snapshot. No effect on trading.
- ****P1 – IC gate + dashboard.**** Compute rolling IC/IR/t-stat; classify edges; surface scorecards. Still measure-only. This alone tells us whether ANY current idea has real edge.
- ****P2 – Composite score in SHADOW.**** Blend active edges; run alongside the live LLM/current score via the ****existing shadow_decisions**** machinery (no fills). Compare current score vs. factor score vs. blended score on live,

- contemporaneous opportunities.
- ****P3 – Exploratory paper.**** Candidate/known-prior edges that clear data-quality and early IC checks may receive tiny capped paper allocation. This is explicit exploration, not proof, and is excluded from live eligibility until later gates pass.
- ****P4 – Regime filter (measure + rank).**** Implement filters, score them on history, and run as an advisory/continuous size scaler first. Hard blocking is a later owner decision after evidence.
- ****P5 – Active paper.**** Validated composite score + regime scaler drive the PAPER book. LLM is demoted to explanation. Watch calibration + Performance-Truth for several weeks.
- ****P6 – Live (owner-gated, unchanged safety).**** Only after WFO OOS + Monte Carlo + long-only benchmark-relative alpha + owner promotion, and only for edges that stayed `active\_paper` through the soak.

Each phase reuses existing infra (shadow, validation, calibration, genome), so risk is incremental and reversible.

P1 evidence-hardening gate before P2

P2 stays blocked until all of the following are true per market:

- prospective daily captures have real `observed\_at` timestamps and immutable input fingerprints; legacy synthetic next-session timestamps are labeled `legacy\_unverified`;
- `n\_obs`, universe size, sampled dates, provider coverage, and evidence quality are persisted and visible;
- lifecycle status is market-scoped; US and India cannot overwrite each other;
- at least 60 prospective trading sessions exist (120 preferred for 20-day horizons), and historical results are not represented as point-in-time proof;
- turnover and cost-adjusted long-only bucket alpha are populated before any paper-capital phase;
- Fibonacci, if added, is measure-only and must beat non-Fibonacci placebo levels.

13. Risks / failure modes to critique

- ****Small-sample IC noise**:** with a tiny paper history, IC estimates are unstable. Mitigation: compute IC on a broad historical universe (not just our filled trades) via the price-history backfill; treat live paper as confirmation only.
- ****Multiple-testing / overfitting the catalog**:** trialing many edges inflates false positives. Mitigation: deflated Sharpe, hold-out, cap the number of simultaneously-active edges, prefer academically-prior edges.
- ****Regime filter = one more overfit dial**:** the article shows band/adaptive filters wobble. Mitigation: judge at median parameter + require cross-market reach; prefer the simplest filter that clears the bar.
- ****Data coverage for India**:** factor inputs on NSE via Yahoo/Kite are thinner – IC gate may bench most edges there initially (acceptable; fail-closed).
- ****Turnover/fees eat factor edges**:** enforce net-of-fee IC from the start (fees-first).
- ****Contradicts the locked "no explicit regime" decision**** – see §8; needs owner sign-off.

14. Open questions for reviewers (LLMs / quants)

1. Is rank-IC + IR + t-stat + deflated-Sharpe a sufficient signal gate, or should we add combinatorial purged cross-validation (CPCV) from day one?
2. IC thresholds: what τ_{ic} / τ_{ir} are defensible for daily US equity factors on a ~1-3k name universe? (We proposed $IC \geq 0.02-0.03$, $IR \geq 0.3$ – too lax/strict?)
3. Regime as ****hard on/off**** vs ****continuous sizing scaler**** – which is more robust given it will also gate India with thin data?
4. Ensemble weighting: IC-weighted vs. equal-weight vs. risk-parity across edges – which best resists weight overfitting at our scale?
5. Where exactly should the LLM stay useful – only literature→hypothesis and explanation, or also as a **conditioner** (e.g. news-surprise as one information edge feeding the IC gate like any other)?
6. Minimum history before an edge may go `active` (calendar time vs. number of independent IC observations)?
7. Anything structurally missing vs. how real agentic-quant platforms (e.g. QuantConnect + factor libraries, WorldQuant-style alpha pools) are built?

15. References

- Grinold & Kahn, **Active Portfolio Management** – IC/IR, fundamental law of active management ($IR \approx IC \cdot \sqrt{\text{breadth}}$).
- López de Prado, **Advances in Financial Machine Learning** – deflated Sharpe, purged/combinatorial CV, backtest overfitting.
- Asness / AQR – factor premia, implementation shortfall, overfitting warnings.
- Owner-supplied: SetupAlpha "Scientific Workflow 2026", "60+ Market Edges 2026", "20 Trend-Based Regime Filters" (Medium); Investopedia "Blend Technical and Fundamental Analysis".

- Existing Kairos infra this builds on: Feature Registry (whitelisted grammar), Validation Engine (WFO, fail-closed), shadow_decisions, strategy genome + Learner (evidence-bound), calibration, Build 4a slip tracking.
- Harvey, Liu & Zhu, "...and the Cross-Section of Expected Returns" – new/data-mined factors need higher multiple-testing hurdles; t-stat > 3 is a useful rule of thumb for newly discovered factors.
- Bailey & López de Prado, "The Deflated Sharpe Ratio" – selection bias and multiple testing inflate backtest Sharpe; use DSR/PSR-style correction.
- Benhenda, "Look-Ahead-Bench" (2026) – point-in-time LLM finance workflows must explicitly test for look-ahead bias and decay across temporally distinct market regimes.
- Azevedo, Hoegner & Velikov, "The Expected Returns on Machine-Learning Strategies" – ML/anomaly strategies can work after costs, but only with careful treatment of transaction costs, post-publication decay, and stale historical data.
- Baldi-Lanfranchi, "Transaction-cost-aware Factors" (2024) – factor construction should optimize the trade-off between exposure and rebalancing costs; especially relevant for high-turnover edges like momentum.
- Research Affiliates / Robeco / Man Group factor implementation notes – practical factor investing must account for turnover, liquidity, crowding, risk management, and investability, not just gross historical returns.

****No implementation until owner approves + reviewers vet. Theme Scout has been reverted to a plain news-driven scout in the interim (it is not the alpha source).****

Reviewer changelog (ChatGPT, 2026-07-08)

- Updated author line to record ChatGPT/Codex methodology review.
- Corrected the TL;DR: IC validation is required for normal sizing/live eligibility, but tiny capped exploratory paper is allowed so the system can learn.
- Corrected the regime design: start with continuous size scaling; hard on/off is deferred until evidence and owner approval.
- Added Section 0A with the intended separation: ThemeScout = attention discovery; EdgeScout/FactorScout = deterministic alpha factory.
- Replaced the simple `candidate → active` promotion with a full lifecycle: `candidate → measure\_only → shadow → exploratory\_paper → active\_paper → live\_eligible → live\_approved`.
- Added stricter statistical gates for newly discovered/data-mined edges: t-stat > 3 or FDR/q-value control before active/live eligibility.
- Added autocorrelation-aware standard error requirement for overlapping forward returns.
- Added mandatory point-in-time input audit table `edge\_signal\_inputs`.
- Updated rollout to include exploratory paper and active paper as separate phases.
- Added recent/current references through 2026 covering look-ahead bias, transaction-cost-aware factors, ML strategy implementability, and multiple testing.

Source: features\event-aware-fundamentals-adr\FEATURE_ARCHITECTURE.md
Event-Aware Fundamentals and Exchange-Listed ADRs

Status: Approved for implementation by owner on 2026-07-31

Decision

Kairos will reuse company-reported facts across agents through one event-aware freshness policy and will treat reviewed US exchange-listed ADRs as a distinct instrument kind. ADRs may be researched and paper-traded in the US/USD book. Live compatibility uses every existing pause, kill-switch, account, portfolio, broker-review, and order-acknowledgement gate; this feature does not enable live trading or assert that a broker supports a symbol.

Why

ResearchAgent and ThemeScout currently share same-day provider cache rows, but the active Finnhub metric/profile path normally polls again the next UTC day. Reported margins and revenue do not require daily polling outside an event window, while valuation and earnings dates do. Treating the whole provider response as one freshness class either wastes calls or serves stale valuation.

ADRs also cannot be treated as ordinary domestic issuers. Foreign private issuers generally do not provide Section 16 Form 4 evidence, and some providers resolve an ADR symbol to the foreign underlying. A live 2026-07-31 probe for SKHY returned Korean-underlying EPS from Finnhub but USD ADS-basis fields from Yahoo. Mixing those units can corrupt valuation and comparability.

Verified SK hynix Identity

- Current US instrument: `SKHY`, Nasdaq Global Select Market.
- Temporary when-issued symbol: `SKHYV`, retired after 2026-07-10.

- Each SKHY ADS represents one-tenth of a KRX `000660` common share.
- Legacy `HXSC` is a restricted/limited OTC GDR and is not a substitute.
- Sources: Nasdaq Trader Alert 2026-37, SEC Form F-1/424(b)(4), Citi DR record.

Only `SKHY` is added to the reviewed ADR registry. OTC/foreign-exchange symbols remain excluded from automatic discovery and live execution.

Freshness Contract

One batch lookup of `earnings\_calendar` annotates symbols before provider work:

```
| State | Reported-fact freshness |
|---|---|
| Report date within the prior 3 days or next 14 days | 1 day |
| No near event, but a calendar record exists | 7 days |
| Earnings date unknown | 7 days, fail conservatively |
```

Issuer profile/classification uses 30 days. Price bars remain completed-session daily data. Earnings dates remain daily inside the existing calendar path. Because the legacy FintHub metric response mixes reported and valuation fields, its maximum freshness is seven days rather than Router's future 14-day reported contract. The Router may later split `fundamentals.reported` (14 days) from `fundamentals.valuation` (3 days) after parity approval.

An event-window refresh does not change scores by itself. It only makes the next normal scoring pass fetch current facts. Provider failures continue to return bounded cached data or unavailable; they never fabricate a fresh observation.

Agent Boundaries

- ResearchAgent is the only authoritative scorer and receives the annotated TTL.
- ThemeScout validates ticker existence with a quote/instrument lookup, not a complete fundamental fetch. Its candidates are still scored later by ResearchAgent.
- Prewarm uses the same annotated symbol entries and cache rules.
- Router shadow collection remains separate and cannot affect scores or orders.
- Learner records `asset\_class='adr'`, preventing ADR evidence from being silently pooled as a domestic issuer when segment diagnostics become available.

ADR Scoring Contract

- Market/currency: US / USD, never cross-summed with the India/INR book.
- Technical, sentiment, US macro, options, and analyst applicability match a US-listed equity when the source supports the symbol.
- Fundamental source order: ADS-aware Yahoo first, then only compatible fallbacks.
- Insider/Form 4: structurally not applicable and excluded from weight renormalization, not assigned a neutral score.
- New ADRs require a reviewed registry entry or a future verified instrument classifier. Symbol suffix guessing is forbidden.

Trading Contract

- PaperTrader accepts `adr` as a US asset class and uses USD/fractional paper sizing.
- Live fund-concentration classification treats `adr` as an equity, not an ETF.
- The Robinhood MCP `review\_equity\_order` remains mandatory before placement.
- A broker rejection or unsupported symbol fails closed. No fallback to an OTC symbol, foreign listing, or alternate broker occurs automatically.
- OTC instruments require a separate design: whole-share sizing, limit-day-only orders, liquidity/spread gates, market-tier and disclosure checks, and explicit broker support. They are out of scope here.

Acceptance Criteria

1. `SKHY` is classified as `adr`, enters US research from the watchlist, and persists `asset\_class='adr'`.
2. SKHY fundamentals prefer Yahoo ADS-basis values; FintHub's Korean-underlying profile cannot win the ADR chain.
3. ADR insider evidence is inapplicable and consumes no insider provider call.
4. An eligible long SKHY signal can enter the paper selection/fill path without a special-case rejection.
5. Live classification recognizes ADR as non-fund equity but every existing live and broker review gate remains in force.
6. ThemeScout ticker validation consumes no full-fundamental call.
7. Near-earnings symbols use one-day reported freshness; ordinary symbols use seven days; profiles use thirty days.
8. Tests, architecture chapters, system map, and research diagram agree.

Reversal

Remove SKHY from the reviewed registry and restore one-day freshness constants. No trade, historical signal, or immutable evidence row is rewritten.

Source: features\event-ledger\FEATURE_ARCHITECTURE.md
Event Ledger – Feature Architecture

Status: ****Step 1 approved and shipped 2026-08-05. Steps 2-5 remain proposals.****
Author: Claude · 2026-08-05
Related: `features/theme-tracking/`, `features/walk-forward-ic-folds/`,
`features/edge-factor-discovery/`, `features/india-scorer-discrimination/R3\_DIMENSION\_FEASIBILITY.md`.

1. What this is for

Record dated, typed market events and their observed forward paths, so that recurring event patterns become a ****counted base rate**** instead of a remembered story.

The motivating example is the "TACO" pattern – an aggressive policy announcement causes a selloff, and the actor has historically reversed within weeks, so the reaction is fadeable. Whether or not that specific pattern survives contact with data, the shape is the right one: identifiable trigger → measurable reaction → conditional forward distribution.

****This proposal builds only the ledger.**** It does not build a strategy, a signal, or a scoring input.

2. Why a ledger before anything else

Three constraints, all measured in this repo rather than assumed.

****The sample is tiny and will stay tiny.**** The motivating pattern has perhaps 5-15 instances. At roughly one event per month, ****30 instances is ~2.5 years****. No amount of engineering shortens that; only recording earlier does.

****A causal story makes a small sample feel decisive.**** "We know he backs out" is exactly the kind of narrative that survives scrutiny it has not earned. This system already measured ****IC ≈ 0 across ~1,700 decision observations**** in `DIAGNOSIS.md` §8 and `R2\_TECHNICAL\_SATURATION\_COUNTERFACTUAL.md` – far more data than an event pattern will ever have, and it could not separate any variant from zero. An $n=10$ pattern with a compelling explanation is the canonical overfit.

****The false-discovery control does not exist yet.**** `walk-forward-ic-folds` Open Decision #3 (false-discovery procedure) and #4 (cost-adjusted portfolio test) are both unresolved and both gate steps 8-9 there. Any mechanism that lets an agent mint and name its own patterns multiplies the trial count – which is precisely the condition under which that missing control matters most. A ledger is safe to build now because it makes no claim; a discovery loop is not.

****There is also no event data at all today.**** No structured, dated, typed policy or macro event exists anywhere in the schema. GDELT supplies news volume; `lib/allocation/international-policy.ts` is portfolio allocation policy, not political events. Nothing can be measured against nothing.

3. Scope

****In scope:**** an append-only ledger of typed events, their market reaction, and their matured forward path; plus a read-only base-rate report.

****Out of scope, explicitly:****

- Any scoring dimension. An event is market-wide or sector-wide, so its per-date cross-sectional σ within the affected set is zero – the same defect that disqualified NSE FII/DII in `R3\_DIMENSION\_FEASIBILITY.md` and that already afflicts US `macro\_score` (σ 2.26) and `insider\_score` (94.1% at one value). It would shift the composite level, not improve ordering, and with ****11.6% of US rows within ±2 points of the gate**** (`DIAGNOSIS.md` §14e) any level shift is a threshold change.
- Any entry, exit, sizing, or broker behaviour.
- Automatic strategy generation. `feature\_registry` already exists for agent-proposed formulas and its lifecycle (`proposed` → `quarantined` → `measure\_only` → `retired`) deliberately has ****no path to a scoring state****. That boundary is not relaxed here.

4. Design

4.1 `market\_events` – append-only

One row per event occurrence:

- `event\_type` – from a ****controlled vocabulary****, not free text
(`policy\_tariff\_announced`, `policy\_tariff\_reversed`, `fed\_rate\_decision`, `sanction\_announced`, ...)
- `occurred\_at` – the timestamp the event became public, which is the only

```

timestamp that can support a point-in-time claim
- `market` - `us` | `india` | `global`
- `direction` - `escalation` | `de_escalation` | `neutral`
- `magnitude` - nullable, type-specific and documented per type
- `source_url`, `source_name`, `observed_at` - provenance
- `notes` - free text, never parsed

**The controlled vocabulary is the load-bearing choice**, and it is the same
lesson `theme-tracking` produced three days earlier: free-text LLM-minted names
drifted into 42 strings over 13 runs, 32 appearing exactly once, with
"Cybersecurity" arriving as six variants. An event type minted per occurrence
cannot be counted, and counting is the entire purpose.

### 4.2 `market_event_outcomes` - matured forward paths

Per (event, horizon): benchmark return, benchmark-neutral return, max adverse and
favourable excursion. Mirrors `observation_labels`, which already does this for
decisions, so the maturation job and the statistics are the same shape rather
than a second implementation.

### 4.3 Ingest - manual first, deliberately

Events are entered by the owner, or by a narrow adapter against a source with a
**stable timestamped machine-readable contract**. No LLM classifies an event into
a type.

Manual entry sounds weak and is correct here: at ~1 event/month the ingest volume
is trivial, and a hand-entered event with a real `occurred_at` and a cited source
is worth more than an automated one whose timestamp is the moment a news
aggregator noticed it. Look-ahead enters through sloppy timestamps, and this is
the one field the whole feature rests on.

### 4.4 Read-only base-rate report

Per `event_type` × horizon: n, mean and median forward return, hit rate,
dispersion, **and n displayed next to every figure**. Below a declared minimum
(suggest 20) the report states that the sample cannot support an estimate rather
than printing one.

---

## 5. What would make this actionable, and when

Not before all of:

1. ≥20 instances of a single `event_type` with matured outcomes;
2. a stated forward horizon and objective fixed **before** the estimate is read;
3. a false-discovery correction across every event type tested - i.e.
   Open Decision #3 resolved;
4. a cost- and slippage-adjusted result, since fading a reaction trades into
   the widest spreads of the move.

Until then the honest use is a **displayed observation with its instance count**,
informing a human decision. That is genuinely useful at n=10, where a systematic
rule is not.

---

## 6. Risks

**R1 - narrative overrides the count.** The most likely failure is acting at n=8
because the story is good. Mitigation: n printed beside every number; the report
refuses to estimate below the floor.

**R2 - non-stationarity.** A single-actor behavioural pattern works until it does
not, and fading a selloff is short volatility: the failure is large, sudden, and
not helped by a stop. The ledger cannot fix this; any future strategy proposal
must price it explicitly.

**R3 - look-ahead through timestamps.** If `occurred_at` drifts toward "when we
recorded it", every backward measurement is contaminated. Mitigation: separate
`occurred_at` from `observed_at` and treat any row where they are implausibly
close as suspect.

**R4 - event-type proliferation.** Each new type is another trial. Mitigation:
the vocabulary is owner-reviewed, and the count of tested types is reported with
the results.

---

## 7. Sequencing

1. `market_events` + `market_event_outcomes` + controlled vocabulary. No UI, no
   signal. Append-only, RLS, `service_role` INSERT-only.
2. Maturation job reusing the `observation_labels` pattern.

```


3. Read-only base-rate report with n-floors.
4. Markets-page display, once a type reaches the floor.
5. Any strategy proposal – separate, and gated on \$5 in full.

Step 1 is small, additive, and reversible.

8. Open questions for the owner

1. ****Approve step 1?****
2. ****Which event types to seed?**** Recommend starting with one – the motivating policy-reversal pattern – rather than a broad taxonomy. Each extra type is an extra trial against an unresolved false-discovery control.
3. ****Backfill?**** Historical instances could be entered manually with cited sources. Worth doing ***only*** if ``occurred_at`` can be sourced accurately; reconstructing dates from memory would produce exactly the look-ahead in R3. Note the precedent: the ``theme_observations`` backfill three days ago keyed on ``created_at`` and produced misattributed rows, because the underlying table had never recorded run identity. A backfill is only as good as the timestamp it can cite.

9. Step 1 – shipped 2026-08-05

``lib/events/vocabulary.ts``, ``market_events``, ``market_event_outcomes``, and an owner-gated ``GET`/`POST`/api/events``. Migration applied and verified.

****Vocabulary seeded with the trade-policy family only**** – ``policy_tariff_announced`` and ``policy_tariff_reversed`` – per §8 Q2. The paired announce/reverse types make the ***interval between them*** measurable, which is the quantity the pattern actually claims. Every additional type is another trial against an unresolved false-discovery correction, so growth is an owner-reviewed edit rather than a convenience.

****Guards verified against the live database, not assumed.**** Two deliberate violations were attempted and both were rejected, leaving the table empty:

```
| attempted | result |
|---|---|
| `occurred_at` two days in the future | rejected by `market_events_occurred_before_observed` |
| `market = 'mars'` | rejected by the market CHECK |
```

Grants confirmed: ``market_events`` holds ``INSERT, REFERENCES, SELECT, TRIGGER`` – no ``UPDATE, DELETE`` or ``TRUNCATE``. ``market_event_outcomes`` additionally keeps ``UPDATE`` so a horizon can be re-matured if a price source is corrected, but never ``DELETE`` or ``TRUNCATE``. Append-only is a grant property, not a writer convention.

****Ingest is manual and owner-gated****, with ``source_url`` and ``source_name`` mandatory. An event with no citation cannot be re-verified, and an unverifiable ``occurred_at`` is precisely the look-ahead in R3. The route also fails closed on an unrecognised ``event_type`` and returns the known list rather than recording the string – the theme ledger failure mode, avoided by construction.

``GET`` returns ledger contents with per-type counts and an explicit note that ****no base rate is computed there****. A rate requires matured outcomes and the declared minimum n from §4.4; nothing in step 1 produces one.

Backfill – done 2026-08-05 (§8 Q3 answered: yes, with cited sources)

``scripts/seed-market-events.ts`` – ****19 rows**, every one carrying a source URL**, routed through the same validators as ``POST`/api/events`` rather than raw SQL, so the vocabulary and timestamp guards apply to seeded rows too. Idempotent via the ``(event_type, market, occurred_at)`` ``UNIQUE`` constraint; a second run inserted 0.

```
| market | announced | reversed |
|---|---|---|
| us | 7 | 7 |
| india | 3 | 2 |
```

****Timestamp precision is recorded, not assumed.**** Two rows are cited to the hour (Apr 2 2025 Rose Garden, after the US close; Apr 9 2025 Truth Social, ~13:00 ET intraday). The rest are date-only and are stamped ****23:59:00Z** of the event date** – deliberately, because stamping 00:00Z would place the event up to a day ***before*** it became public and a forward return measured from there would quietly include pre-announcement drift. That is R3 exactly. End-of-day UTC is the conservative direction: after the US cash close, before the next India open. The precision marker is written into ``notes`` so a reader can see which is which.

****No `global` rows.**** An event that moved both books is two rows, one per market, each measurable against its own benchmark. A ``global`` row would invite a pooled US+India statistic, and those must never cross-sum.

Every count here is ****below the §4.4 floor of 20****, which is the expected and

correct outcome – the base-rate report must refuse to estimate on it.

10 – Steps 2 and 3, shipped 2026-08-05

`lib/events/outcomes.ts` (pure), `POST /api/agents/event-maturation`
(`kairos-event-maturation`, weekdays 16:10 UTC, jobid 116), and owner-gated
read-only `GET /api/events/base-rate`.

The anti-look-ahead rule is the feature

A measurement starts at **the first session whose CLOSE falls after
`occurred\_at`**, using a per-market close hour (`us` 20:00Z, `india` 10:00Z).
Anchoring on the close rather than the date is the whole point: an announcement
made at 13:20 ET is tradable at that day's US close, one made after the bell is
not, and starting from a pre-announcement close folds the market's reaction INTO
the "forward" return and makes the pattern look prescient.

Verified against the live ledger, not just fixtures:

```
| event | market | entry |  
|---|---|---|  
| 2025-04-09 **17:20Z** (intraday) | us | **2025-04-09** – same session, 13:20 ET is before the close |  
| 2025-04-09 **17:20Z** (same instant) | india | **2025-04-11** – after the IST close, and 04-10 was an NSE holiday |  
| 2025-02-01 23:59Z (Saturday) | us | 2025-02-03 (Monday) |
```

An unelapsed horizon returns **null**, never 0. A zero would pull every mean
toward zero while inflating n – the same class as the count-not-span bug this
repo already shipped once in sector returns.

Excursions (MAE/MFE) are measured from `start + 1`, not from the entry bar: we
enter at the entry bar's close, so that bar's intraday range is already past, and
for an intraday-cited event part of it happened **before** the announcement.
This was a real defect in the first implementation, caught by its own test.

Two artefacts found by running it on real data

benchmark_neutral_return is identically 0 for every market-wide event,
because the subject IS the benchmark. The report initially preferred the neutral
leg, which would have printed every tariff base rate as exactly zero – reading
as a finding and being an artefact. `cohortValue()` now selects raw return for
market-wide cohorts and benchmark-neutral for idiosyncratic ones. An
idiosyncratic outcome without an aligned benchmark is excluded rather than
falling back to raw return; the base-rate response exposes that exclusion count
so an incomplete benchmark series cannot silently shrink or contaminate a cohort.

price_cache cannot mature this ledger: it reaches back only to 2025-07-22
for SPY, while the earliest recorded event is 2025-02-01. Maturation uses the
repo's existing keyless Yahoo chart source with `adjusted: true` – deliberately
not the live scoring path's default, because a return study must not read a
split or distribution as alpha.

The report refuses

Below `MIN_INSTANCES = 20` the summary carries **nulls**, not numbers with a
caveat beside them – a caveat is something a reader skips; an absent number is
not. `n` is returned either way, and the count of event types tested is returned
with the results per R4.

Live state 2026-08-05: **57 outcomes, 12 cohorts, max n = 7. Nothing estimated.**
Market is part of the grouping KEY, not a filter applied afterwards, so a pooled
US+India row cannot be produced by accident.

Vocabulary extended – guidance, by owner review

`guidance_cut` / `guidance_raised`, chosen over further macro types **because it
is idiosyncratic**. A market-wide event hits every name in the affected set
together, so its per-date cross-sectional variance is zero by construction: it
can shift a composite's level but never improve its ordering. That is the
measured defect that disqualified NSE FII/DII and that already afflicts US
`macro_score` and `insider_score`. Per-company events are the only ones with real
dispersion, so they are the only ones worth minting.

`EventTypeDefinition.idiosyncratic` drives `requiresSymbol()`: an idiosyncratic
event with no subject has nothing to compute a return ON, so it could never
mature and would sit in the ledger permanently deflating n. `POST /api/events`
rejects it at the door.

Guidance rows still need a source. The type exists; populating it is the next
problem, and it is harder than tariffs – per-company, higher frequency, and the
`occurred_at` must be the release or call, not the period covered.

Not yet built (steps 4-5)

Markets-page display (gated on a type reaching the floor) and any strategy
proposal (gated on \$5 in full, including the unresolved false-discovery control).

Source: features\exit-geometry-provenance\FEATURE_ARCHITECTURE.md
Exit-geometry provenance and shadow-baseline repair

> Status: ****APPROVED** – paper-only evidence repair. ****** Owner approval: 2026-09-12.
> Money-path influence: ****none**. ****** This work records and measures the incumbent;
> it does not alter a stop, target, sizing rule, paper exit, live order, or broker setting.
> Implementation result: 2026-09-12 – complete locally; 2,664 tests pass and
> TypeScript is clean. The isolated Next build was stopped after an extended
> silent OneDrive/Webpack compile, so no build pass is claimed.

Why

Kairos resolves entry geometry from either the market mandate or the market-local, entry-candidate MAE/MFE ledger. The chosen values are bound to a fill, but the existing journal record omits the percentile values, horizon, mandate version and calculation time needed to reproduce that decision. Separately, the general exit-geometry shadow still uses a stale hard-coded 7.5%/19.2% baseline even though execution uses a different market-local plan. A shadow cannot judge a policy it did not actually measure.

Scope

1. Extend the in-memory execution-risk result with an immutable provenance object: mandate inputs/version, resolved horizon, observed-at time, sample count, and, for a ledger result, the actual MAE/MFE percentile values and percentile parameters.
2. Write that object into append-only `pipeline\_stage\_events.risk\_plan` detail for paper and autonomous-live proposal creation. It is evidence only; existing tables and executions remain authoritative.
3. Make the read-only geometry-shadow endpoint load each market's current mandate rather than embed 7.5%/19.2%. It must stamp the baseline source and explicitly report that it measures a mandate baseline, not a reconstruction of historical per-entry plans.
4. Correct stale comments and tests that still claim 7.5%/19.2% is live.

Deliberate limits

- Do not turn an ATR multiplier, structural low, fundamental score, volume profile, or R multiple into a stop/target rule.
- Do not use a current percentile calculation to rewrite history or claim it was the plan used by an older trade.
- Do not persist a new table or migration: `pipeline\_stage\_events` is already the append-only decision journal and the change is additive JSON.
- Do not make the general shadow a promotion gate. It remains read-only and must continue to refuse ambiguous barrier ordering.

Acceptance criteria

- [x] A newly created risk-plan event contains a versioned provenance snapshot; malformed/missing learned data is visibly represented as a mandate fallback.
- [x] Paper and autonomous-live proposal paths use the same provenance constructor.
- [x] Geometry-shadow results contain no hard-coded 7.5%/19.2% incumbent and state the exact baseline used for each market.
- [x] Tests prove no input can silently turn invalid learned values into a ledger plan, and prove a supplied market mandate changes the shadow baseline.
- [x] Typecheck and relevant tests pass. No production migration or trade occurs.

Source: features\exogenous-risk-evidence\FEATURE_ARCHITECTURE.md
Exogenous Risk Evidence Layer

> Status: ****P0 FOUNDATION BUILT; LEGACY US INTEGRITY + EXPLAINABILITY REMEDIATED; P1-P3 INGESTION NOT YET ENABLED.****
> Date: 2026-08-24
> Owner approval: observation-first build approved 2026-08-01. No scoring, paper,
> live, exit, sizing, or broker activation is approved.

1. Decision

Build one canonical, point-in-time evidence layer for facts that affect markets but do not belong to a company's fundamental, technical, insider, or ordinary news-sentiment score.

The layer has three deliberately separate classes:

1. ****Domestic regimes**** – US macro/Fed and India macro/RBI.
2. ****Global spillovers**** – Fed impact on India, global oil, USD/INR, and global risk conditions.
3. ****Discrete shocks**** – tariffs, sanctions, supply disruptions, and conflicts.

The first releases are record-only and shadow-only. They cannot alter a score, direction, position size, paper trade, live trade, stop, target, or broker order. Admission to a money path requires separately measured, point-in-time,

market-and-sector-specific incremental value.

****P0 implementation (2026-08-01):**** migration `20260801150000\_exogenous\_risk\_p0` adds the two append-only, owner-read/service-write-only ledgers described below and removes the obsolete India Macro Read cron, which had already refused its market before an LLM call or write. No ingestion adapter is enabled until an official source exposes a stable, timestamped machine-readable contract. Empty ledgers are intentional and are not treated as neutral macro evidence.

2. Why This Is Needed

Kairos currently has real but fragmented coverage:

Input	Current state	Problem
US macro / Fed	US-only FRED MacroSentinel; FOMC ledger is record-only	Correct for the US, but cannot be reused as an India domestic regime
India FII/DII	NSE factual line in the India thesis	Useful daily flow context, but not a regime and not scoreable alone.
Oil, gold, crypto products	Some ETFs/watchlist symbols and Markets display tiles	A visible instrument is not a validated factor or a symbol
Tariff and conflict news	May appear in per-symbol sentiment/LLM prose	No authoritative event identity, effective time, exposure mapping.
India macro	Explicitly unavailable in scoring	This is safer than borrowing the US regime, but leaves a genuine coverage gap.

The bad alternative is a universal "risk score" that mixes stale CPI, a same-day oil move, an LLM's interpretation of a conflict, and US Fed policy, then applies it to every US and Indian symbol. That would be non-reproducible and would make the earlier India-US macro leakage possible again.

3. Invariants

- **No cross-market substitution.**** India never reads a US domestic regime; the US never reads an India domestic regime. Global observations are explicitly tagged `scope=global\_spillover` and have separate US/India transmission rules.
- **Point in time.**** Every observation records `observed\_period`, `published\_at`, `available\_at`, source URL, source revision/vintage when available, and raw-payload fingerprint. A later revision never rewrites a value knowable at an earlier decision time.
- **No fabricated neutral.**** Missing, stale, partial, or contradictory evidence is `unavailable`, not a score of 50.
- **Deterministic ingestion and math.**** No LLM classifies an event, chooses a sector, computes a regime, or changes an exposure. LLMs may only summarize already persisted evidence for display.
- **Off the money path until proven.**** Research, paper, live, exits, sizing, allocation, and brokers do not import this layer in P0-P3.
- **Append-only evidence.**** Corrections append a new observation/supersession; they do not rewrite historical facts or impact calculations.
- **Market-local accounting.**** US results remain USD/SPY-or-V00-relative; India results remain INR/NIFTY-relative. They are never cross-summed.
- **Structural confidence denominator.**** For a fixed applicable indicator set, missing/degraded inputs never disappear through post-fetch renormalization. The numerator contains only valid evidence; the denominator remains the full applicable structural weight, and evidence coverage is displayed separately.

4. Source Policy

4.1 Approved primary sources

Evidence	US	India	Cadence	Initial role
Domestic policy rate / decision	Existing FRED target range + FOMC ledger	RBI MPC decision and RBI DBIE policy-rate series	Event / daily	Domestic regime evidence
Inflation / activity	Existing FRED series	MoSPI CPI and IIP releases	Monthly	Domestic regime evidence
Rates / financial conditions	Existing Treasury/FRED series	RBI DBIE 10Y G-Sec, policy corridor, liquidity, INR/USD	Daily/monthly	Domestic regime evidence
Portfolio flows	Existing US risk indicators only	NSE FII/DII; SEBI/NSDL reference fallback	Daily	India flow overlay
Oil	EIA Brent/WTI	PPAC Indian Basket, with EIA Brent as global comparator	Daily/weekly	Global spillover
Official trade actions	USTR / Federal Register	Indian Ministry of Commerce / DGFT notices when a machine-readable, timestamped source is		

The primary India authority is RBI DBIE for monetary/financial series and MoSPI for CPI/IIP. NSE FII/DII is not promoted into a domestic regime by itself.

4.2 Explicitly deferred or prohibited sources

- Do not scrape CME FedWatch, news pages, social posts, or an RBI page to invent a market-implied policy expectation. A future expectation adapter needs a lawful, timestamped, pre-decision source.
- Do not use Yahoo, TradingView, a crypto exchange, or a chart tile as the authoritative macro source. They may be display/fallback sources only after a separate source-policy approval.
- Do not use a general LLM, GDELT tone, or arbitrary headlines to produce a tariff/war severity score.
- Gold:silver ratio, crypto breadth, and generic geopolitical sentiment are not P0 inputs. They are hypothesis candidates only after the core evidence layer works.

5. Canonical Data Contracts

The implementation must extend the existing evidence/provenance conventions, not create a disconnected second audit system. It should reference existing

`evidence\_records`, `provider\_call\_ledger`, `evidence\_cache\_v2`, frozen
`symbol\_daily\_returns`, and the US `policy\_rate\_events` ledger by stable IDs or
fingerprints.

5.1 `exogenous\_observations`

Append-only raw facts, one record per source observation/vintage:

```
type ExogenousObservation = {  
  id: string;  
  market: "us" | "india" | "global";  
  scope: "domestic" | "global_spillover";  
  seriesKey: string; // e.g. india.cpi_combined_yoy, global.brent_usd  
  value: number | null;  
  unit: string;  
  observedPeriod: string;  
  publishedAt: string;  
  availableAt: string;  
  source: string;  
  sourceUrl: string;  
  sourceRevision: string | null;  
  payloadFingerprint: string;  
  quality: "fresh" | "stale" | "partial" | "unavailable";  
};
```

No data consumer receives a naked number without its availability and as-of
metadata. Source cache keys must include the source, series, market, period, and
revision identity.

5.2 `market\_regime\_runs`

Immutable output of a versioned deterministic regime function. It must have a
`market` column from day one. This is the eventual replacement for reading the
unmarketed legacy US `macro\_regime` table; it does **not** modify that table in
the first phase.

```
type MarketRegimeRun = {  
  id: string;  
  market: "us" | "india";  
  domesticState: "supportive" | "neutral" | "adverse" | "unavailable";  
  globalSpilloverState: "supportive" | "neutral" | "adverse" | "unavailable";  
  formulaVersion: string;  
  inputFingerprint: string;  
  computedAt: string;  
  eligibleInputCount: number;  
};
```

Domestic and global states are separate fields. They must never be added into a
single value without a validated formula version.

5.3 `exogenous\_events` and `event\_impact\_observations`

An event carries an authoritative source, effective time, affected jurisdictions,
and a bounded taxonomy (`policy\_rate`, `tariff`, `sanction`, `supply\_disruption`).
It does not store a model-generated severity or inferred symbol list. A separate,
versioned exposure map may connect a **verified** event class to sectors or

instruments. Impact rows calculate 1/5/20 session raw and same-market benchmark excess returns only from frozen return evidence.

Existing FOMC tables remain the authoritative US policy implementation. A later adapter links them to this common contract rather than copying or rewriting them.

6. Transmission Maps

There is no valid all-symbol response to oil, Fed, tariffs, or war. Every future mapping must be versioned, deterministic, and conservative:

```
| Driver | US examples | India examples | Rule before admission |
|---|---|---|---|
| Fed / US yields | long-duration technology, banks, REITs | FPI-sensitive financials, INR-sensitive importers/exporters | Measure separately
| Oil rise | energy producers/refiners may benefit; airlines may suffer | upstream producers may benefit; airlines, chemicals, OMCs may suffer
| INR depreciation | US domestic impact usually none | exporters and import-dependent firms differ | Use RBI exchange evidence and a reviewed
| Tariff | importer/exporter and supply-chain effects differ | India exporter/importer effects differ | Only an explicit, reviewed exposure l
| Crypto move | crypto ETFs, miners, exchanges | no general India equity effect assumed | Apply only to explicitly classified crypto exposures
| Gold:silver ratio | no default equity implication | no default India equity implication | Research-only until an IC study clears. |
```

7. Phased Build Order

P0 – Truth and observability (2-3 engineering days)

1. Publish a source capability matrix: authority, cadence, expected delay, allowed use, fallback, staleness limit, and API quota/cost.
2. Add System Health checks for source freshness and observation coverage, not one alert per missing symbol.
3. Audit and relabel existing Markets tiles as `display-only` where appropriate.
4. Remove the now-harmless `macro-read-india` no-op cron after verifying no consumer depends on it.

Acceptance: the UI and health surface distinguish `not built`, `unavailable`, `state`, and `display-only`; no money-path import changes.

P1 – India domestic macro shadow (5-7 engineering days)

1. Implement server-only official adapters for RBI DBIE and MoSPI. Start with repo rate, 10Y G-Sec, INR/USD, CPI Combined YoY, and IIP growth.
2. Persist release vintages in `exogenous\_observations`; capture source release time rather than using fetch time as a fake release time.
3. Add daily frozen NSE FII/DII snapshots with source date, bounded stale policy, and no accidental reuse of the `gdelt` provider identity.
4. Compute an India domestic regime in `market\_regime\_runs`, shadow-only.
5. Compare regime outputs to NIFTY/sector outcomes by released-data time, with no score or trade effect.

Acceptance: an India run can be reproduced using only observations available at that run's `computedAt`; a failed source yields `unavailable`, not calm; the US path cannot query India records and vice versa.

P2 – Global spillover evidence for both markets (4-6 engineering days)

1. Link the existing FOMC outcome ledger to the evidence contract; do not duplicate its event or impact records.
2. Add Fed-to-India as a separate global-spillover observation, never an India domestic-policy observation.
3. Add Brent/WTI from EIA for the US and the PPAC Indian Basket for India.
4. Add a versioned, read-only oil/FX/FPI exposure hypothesis map that defaults to no mapping for unknown symbols.
5. Record per-sector, per-market impact observations from frozen returns.

Acceptance: a Fed shock can be shown as global context for India while the India domestic regime remains unchanged; all displayed effects state their source and as-of time.

P3 – Discrete-event ledger (5-8 engineering days)

1. Implement an event ledger for a small authoritative taxonomy only.
2. Ingest USTR/Federal Register trade actions and a separately validated Indian official trade-notice source. Start with policy/tariff events, not wars.
3. Permit an owner-reviewed event record for conflicts/supply disruption when an official source and effective timestamp exist. The application must not auto-infer an event from news sentiment.
4. Record impacts and candidate sector hypotheses; display them as observation, not trade recommendations.

Acceptance: every event has a source URL and effective/known-at time; a missing exposure link cannot suppress or promote a symbol.

P4 – Hypothesis evaluation (minimum 8-12 weeks of new evidence; no fixed calendar promise)

1. Use the existing point-in-time replay, Edge IC, FDR, and walk-forward infrastructure to test each `one` hypothesis separately.

2. Require adequate event count, market/sector sample floor, post-cost result, stability across windows, and no worse drawdown/turnover than the baseline.
3. Promote at most one validated, narrow use at a time: initially a subtractive `long -> neutral` eligibility guard in paper shadow. Never promote an event feature that directly creates a buy, exits a holding, or widens risk limits.
4. Require an owner sign-off and a reversible feature flag before paper action; live remains disabled pending a separate approval.

The key constraint is data time, not coding time. CPI/IIP and RBI decisions are monthly/event-driven, so reliable validation cannot be compressed into a week.

8. Explicit Deferrals

- ****Gold:silver ratio:**** add only as a shadow Edge-lab factor after P2. It has no default portfolio action because its equity relationship is regime dependent.
- ****Crypto macro factor:**** add only for instruments classified as crypto exposure; a broad US/India equity penalty is unsupported.
- ****War score:**** do not build one. Record verified events and their observed impacts; the product must not pretend it can quantify an unfolding conflict.
- ****Policy expectations:**** retain `unavailable` until a lawful pre-decision feed is selected. Official outcomes cannot reconstruct prior market expectations.
- ****Live usage:**** completely out of scope.

9. Safety, Security, and Rollback

- All ingestion routes are server/cron-only and owner-triggerable; browser code reads a redacted, owner-authenticated projection only.
- New evidence and impact tables use RLS deny-by-default, no anon grants, service-role-only writers, append-only triggers, indexes on `(market, series\_key, available\_at)` and event/impact keys.
- Raw payloads are bounded, schema-validated, and fingerprinted; no cookies, headers, tokens, or arbitrary fetched HTML enter the database or UI.
- Every phase has a feature flag whose disabled state leaves today's US macro, India scoring, paper, live, and exit behavior byte-for-byte unchanged.
- Rollback disables readers/jobs; it never deletes evidence or rewrites past signals/trades.

10. Validation Plan

1. Adapter fixtures: normal release, revision, late release, malformed payload, source throttle, stale fallback, duplicate fingerprint, and unavailable.
2. Boundary tests: US consumers cannot access India domestic observations; India cannot access US domestic observations; global observations cannot be mislabeled as domestic.
3. PIT replay tests: a value published after `as\_of` is rejected even when its observation period is earlier.
4. Falsification tests: unknown symbol exposure, absent benchmark, missing expected policy rate, incomplete return horizon, and missing source must all leave scores/directions/orders unchanged.
5. Production proof before any promotion: write fresh source rows, then verify their as-of metadata, regime run fingerprints, market isolation, and a UI projection with no provider URL/token leakage.

11. Decision Record

- ****Why:**** The user needs auditable macro and event context across both books without mixing USD/INR, importing unreliable news judgement, or spending research-provider quota.
- ****Who:**** Vaibhav operating separate US and India paper/live portfolios.
- ****ROI:**** Better diagnosis and future evidence-backed risk gating; no promised alpha until the layer passes the existing validation process.
- ****Build decision:**** ****GO** for P0-P3 record/shadow architecture; DEFER all score, paper, live, exit, gold/silver, crypto, and war-score admission.

12. References

- RBI DBIE: <https://dbieold.rbi.org.in/DBIE/>
- MoSPI release calendar: https://mospi.gov.in/sites/default/files/Advance_Release_Calendar.pdf
- SEBI FPI source index: <https://www.sebi.gov.in/curation/fpi.html>
- RBI MPC schedule: <https://www.rbi.org.in/scripts/PublicationsView.aspx?id=23139>
- EIA oil data: https://www.eia.gov/dnav/pet/pet_pri_spt_s1_w.htm
- PPAC Indian Basket: <https://ppac.gov.in/index.php/prices/international-prices-of-crude-oil>
- USTR Section 301 source: <https://ustr.gov/issue-areas/enforcement/section-301-investigations>

13. Implementation checkpoint – 2026-08-24

The August remediation reuses the existing ledgers rather than creating a second macro or tariff subsystem:

- ``lib/data/macro-regime-integrity.ts`` is now the single US legacy integrity contract used by MacroSentinel, the stock scorer, the allocator, and the Agent Mind macro narrative. It recognizes the eight declared dimensions and their fixed weights, rejects unknown/duplicate/forged dimensions, requires at least 6/8 dimensions and 75% structural-weight coverage, and uses the fixed structural denominator (48 maximum danger points). A failed provider cannot

- become a confident regime by renormalizing over the inputs that happened to return.
- The Markets page shows every available US dimension's description, raw value, reliability weight, exact danger-point contribution, coverage, as-of week, and the formula. This is the legacy `100 - danger\_score` overlay currently used by the US scorer; it is not described as symbol-specific tariff exposure.
- The existing `market\_events` ledger is displayed for both US and India with source links, public-known timestamps, market tags, and an explicit stale-ledger warning. It remains owner-reviewed and measurement-only. The official Federal Register API is free and suitable for a future candidate inbox, but a keyword hit must not automatically choose announcement vs reversal, exposure, or a trade effect.
- India continues to state `unavailable` for macro rather than borrowing the US FRED verdict or treating FII/DII flow as a complete regime. P1 remains the official RBI/MoSPI point-in-time collector and shadow regime described above.
- No item in this checkpoint changes live/paper eligibility, sizing, exits, or broker behavior beyond rejecting an under-covered legacy US macro verdict.

Operational fixes shipped with this checkpoint: the label-maturation freshness monitor now reads the true newest watermark rather than an arbitrary PostgREST page; the US price-cache contract monitors active decision/open-position symbols instead of retired historical cache members; missing active symbols remain in the coverage denominator; legitimate all-skip paper runs are healthy `no\_action` while deferred/unavailable/failed work remains blocked; monitor query/update failures are reported instead of silently swallowed.

Independent review of `57c63cf3` – 2026-08-01

Adversarial pass over the commit and its production effects.

Finding 1 (HIGH, fixed) – append-only was breakable by TRUNCATE

`20260801150000` protected both tables with a **`BEFORE ROW UPDATE OR DELETE`** trigger. A row-level trigger **`**does not fire on TRUNCATE**`**, and `service\_role` retained the TRUNCATE grant. Confirmed against production by decoding `pg\_trigger.tgtype`:

exogenous_observations_no_mutation -> BEFORE ROW DELETE UPDATE (no TRUNCATE)

market_regime_runs_no_mutation -> BEFORE ROW DELETE UPDATE (no TRUNCATE)

Failure scenario: any code path or operator holding the service key could `TRUNCATE public.exogenous\_observations`, erasing the entire point-in-time evidence ledger, while the table comment, the migration, the feature doc and the Upgrade Path all still asserted append-only. No error, no trigger, no trace.

`**Fix**` – `20260801160000\_exogenous\_risk\_truncate\_guard.sql`, applied and verified: statement-level `BEFORE TRUNCATE` triggers on both tables, and `UPDATE/DELETE/TRUNCATE` revoked from `service\_role`. Grants and triggers are now two independent barriers rather than one, matching `earnings\_risk\_observations`.

No issue found

- **`**Migration idempotency**`** `create table if not exists`, `create index if not exists`, `create or replace function`, `drop ... if exists` before every trigger and policy. The `cron.unschedule` runs in a `DO` block that swallows its own error, so re-running is safe.
- **`**RLS and grants**`** RLS enabled on both. Owner policy is `(select auth.jwt() -> 'email') = '<owner-email>'`. `authenticated` = `SELECT only`; `anon` has **`**no grant at all**`** and is refused with `42501`. `20260801150500` correctly repairs the first migration's over-revoke – RLS filters rows but does not grant table privileges, so the owner-read policy would otherwise have been inert.
- **`**Trigger function safety**`** `exogenous\_risk\_append\_only()` is SECURITY **`**INVOKER**`** (not DEFINER) with `search\_path` pinned, and EXECUTE is revoked from public/anon/authenticated (`proacl` = postgres, service_role only).
- **`**Constraint correctness**`** `check ((market = 'global') = (scope = 'global\_spillover'))` is a biconditional, so it enforces **`**both**`** directions in one clause: a global row must be `global\_spillover`, and a `global\_spillover` row cannot be domestic. `available\_at` >= `published\_at` blocks inverted provenance. Fingerprints are `^[a-f0-9]{64}\$`; `source\_url` must be `https://`.
- **`**India cron absent in production**`** `cron.job` matching `%macro-read%` returns exactly one row – `kairos-macro-read-us`. `kairos-macro-read-india` is gone.
- **`**Money path**`** No reference to either table outside `lib/shadows/`. No scorer, signal, PaperTrader, PositionMonitor, TraderAgent, sizing, Router or broker consumer.
- **`**Upgrade Path honesty**`** `lifecycle` resolves only to `idle` or `collecting` – it cannot render "active". `cronJobs: []`, `callAccounting: "not\_applicable"`, and `currentInfluence` states no reader exists. Global rows are fetched alongside the market but the surfaced text states they are context only and never cross-summed with a market book; the series set is keyed `market:series\_key`, so US/USD and India/INR cannot merge.

Verification limit worth recording

UPDATE and DELETE were probed through PostgREST against a predicate matching **zero rows**, which returns 204 without the trigger ever firing. That is inconclusive as a runtime proof. The append-only guarantee is evidenced instead by trigger presence and type in `pg_trigger` plus the revoked grants – not by an executed mutation. Exercising it properly would require inserting a throwaway row into an immutable ledger, which is not worth the permanent artifact.

Source: features\external-research-integrations\FEATURE_ARCHITECTURE.md
Governed External Research Integrations

> Status: **REVISED DRAFT FOR CLAUDE + OWNER REVIEW. DESIGN ONLY.**
> Date: 2026-07-13
> Owner decision requested: approve an extensible but zero-trust way to add
> selected GitHub projects, libraries, and skills to Kairos without granting
> them broker, production database, or money-path authority.
> Build gate: no implementation, dependency installation, migration, provider
> activation, credential issue, deployment, or order-path change until this
> document is reviewed and Vaibhav says `Proceed`/`Code it``.

Document Authority

This file is the **only implementation authority** for this feature. Before a build starts, its approved version must contain the precise data contract, migration plan, rollout and acceptance criteria. The catalog, Vibe deep-dive, and deep candidate audit in this directory are evidence-only intake records: they explain why a capability was accepted, deferred, or rejected, but never override this architecture or authorize implementation.

The first native capability remains blocked until Evidence Router shadow parity and the entry-eligibility-flip guard are complete. It must extend existing Kairos records, including `experiment_runs` and immutable `evidence_records`; it must not introduce another provenance truth layer.

1. Decision

Kairos remains the **system of record**, the **Performance Truth Layer**, and the sole **money path**. External repositories and skills are never installed into the Next.js application as trusted agents. They are admitted one at a time as version-pinned, capability-scoped **Research Integrations**.

This document does **not** approve a Quant Research Worker now. The immediate work is the Canonical Evidence Router Phase 2 foundation, followed by narrowly scoped in-repository indicator hygiene where it has proven value. The worker framework is reserved for the first genuinely untrusted or LLM-driven external integration, such as a Vibe-derived advisory workflow.

When that future worker is justified, it receives immutable router evidence envelopes plus a Kairos universe snapshot; it may calculate approved research artifacts and returns schema-validated results through a narrow Kairos gateway. It cannot read broker credentials, submit/preview/cancel an order, change strategy configuration, mutate ledger rows, or call production Supabase with a service-role key.

An Integration Registry controls whether a capability is disabled, in contract test, research-only, shadow-only, or retired. There is **no execution-capable state** for a third-party integration.

This gives Kairos a durable answer to both questions:

1. New useful repositories can be added later without rewriting agents.
2. A repository update, compromise, deletion, license change, or provider outage cannot silently affect a real account or an existing decision.

2. Why This Is Better Than "Just Use Skills"

Skills and open-source projects are useful accelerators, but they are not a durable trading product boundary. They can change dependencies, prompts, network behavior, data sources, licenses, or security posture without notice. Kairos has requirements that a generic skill does not own:

- US/USD and India/INR pools are never cross-summed.
- Paper and live books are distinct, with live approval explicit by default.
- A score, evidence snapshot, candidate strategy, fill, and outcome are independently auditable.
- No LLM is permitted on a deterministic decision, sizing, or execution gate.
- Provider quotes used for execution remain separate from research evidence.

The right use of a skill is therefore: **generate or test a hypothesis inside a constrained research workflow**, not decide or execute a trade.

3. Goals And Non-Goals

Goals

1. Add a new library, skill, or permitted fork through a repeatable review and release process.
2. Make useful capabilities portable: indicator calculation, hypothesis generation, adversarial research, backtesting, document extraction, and validation reports.
3. Make every output reproducible from its pinned integration version, input snapshot, policy version, and strategy version.
4. Reuse and extend the existing `decision\_observations`, `strategy\_versions`, `experiment\_runs`, immutable `evidence\_records`, validation, Performance Truth, benchmark-alpha, and evidence-ledger architecture rather than create a parallel learning/trading system.
5. Make disable and rollback immediate and harmless to existing Kairos flows.

Non-goals

- No external repository is a broker, order router, portfolio source of truth, or direct Supabase client.
- No arbitrary GitHub URL, package name, container image, prompt, provider URL, MCP tool, or command may be entered from Settings.
- No self-updating integrations, floating `latest` tags, or remote code fetch at runtime.
- No third-party code is copied before a license and provenance review.
- No model training, autonomous live trading, or new data-provider entitlement is enabled by this feature.
- No worker, job-token issuer, worker table, or Settings marketplace is built merely to calculate indicators Kairos already obtains or can compute as pure, tested TypeScript.
- No OpenBB code/service is used in the initial plan. Its current AGPLv3 license requires a separate legal/product decision.

4. Architecture Principles And Invariants

1. ****Kairos owns orders.**** Only Kairos-owned, reviewed broker adapters can reach a broker. The existing gate sequence is repeated immediately before submit.
2. ****Read-only by construction.**** Worker input is a snapshot; its write path is limited to an append-only integration run/artifact ledger.
3. ****No credentials cross the boundary.**** A worker never receives broker tokens, OAuth cookies, `CRON\_SECRET`, Supabase service-role credentials, user browser cookies, or raw vault material.
4. ****No network by default.**** A research worker uses supplied snapshots. Any approved outbound provider access is an explicit, domain-allowlisted adapter owned by the Canonical Evidence Router, never a repository-defined URL.
5. ****Determinism on the money path.**** LLM and skill output is advisory text or a structured proposal. It cannot set scores, direction, sizing, a gate result, or an execution action.
6. ****Schema before prose.**** All cross-boundary input and output is validated against a versioned JSON schema. Free-form repository/provider text is never interpolated directly into an LLM prompt or a trading decision.
7. ****Market isolation.**** Every snapshot, run, artifact, experiment, and result has exactly one `market` and native `currency`. No aggregate result can mix `us/USD` and `india/INR`.
8. ****Point-in-time fidelity.**** Inputs carry `as\_of`, `retrieved\_at`, source provenance, price adjustment basis, and policy version. The worker cannot query a newer fact when replaying a historical date.
9. ****Failures are abstentions.**** Timeout, invalid output, quota exhaustion, missing price, unexpected network attempt, or policy mismatch returns an unavailable result. It never manufactures neutral evidence or a pass.
10. ****Versioned and reversible.**** Every run records its exact build digest and configuration digest. Disabling an integration stops new runs without changing existing scores, experiments, paper positions, or orders.

5. C4 Context

C4Context

title System Context - Governed External Research Integrations

Person(owner, "Vaibhav", "Reviews integrations, research results, and live proposals")

System(kairos, "Kairos", "System of record, evidence, validation, paper/live governance")

System_Ext(github, "Approved upstream projects", "Libraries, skills, and source repositories")

System_Ext(data, "Approved market data providers", "Reach Kairos only through typed evidence adapters")

System_Ext(brokers, "Broker accounts", "Reach Kairos-owned broker adapters only")

Rel(owner, kairos, "Reviews, enables, disables, and promotes", "HTTPS")

Rel(github, kairos, "Provides reviewed source/release metadata only", "Release review")

Rel(kairos, data, "Retrieves typed research evidence", "HTTPS via allowlisted adapters")

Rel(kairos, brokers, "Submits owner-approved orders", "Kairos-owned broker adapter")

Upstream projects do not connect to brokers, accounts, or the primary Kairos database. A GitHub project is source material, not a peer system.

6. C4 Container Boundary

C4Container

title Container Diagram - Kairos Research Integration Boundary

Person(owner, "Vaibhav", "Owner")

System_Boundary(kairos, "Kairos") {

Container(web, "Kairos web/API", "Next.js 15", "Settings, research, validation, paper/live approvals")

ContainerDb(db, "Kairos database", "Supabase Postgres", "Evidence, decisions, ledgers, policies, artifacts")

Container(router, "Canonical Evidence Router", "Kairos TypeScript", "Typed provider routing, cache, pacing, provenance")

Container(workerApi, "Future Research Worker Gateway", "Kairos API", "Issues scoped jobs and validates results")

Container(worker, "Future isolated compute container", "Pinned container in GitHub Actions", "No secrets or network; calculates approved research artifacts")

Container(exec, "Kairos Execution Gateway", "Kairos TypeScript", "Final owner/gate/broker checks; never callable by worker")

}

System_Ext(data, "Market-data providers", "Typed evidence providers")

System_Ext(broker, "Broker APIs/MCP", "Real account access")

Rel(owner, web, "Controls policy and reviews artifacts", "HTTPS")

Rel(web, db, "Reads/writes governed records", "Supabase")

Rel(web, router, "Requests evidence intents", "In-process")

Rel(router, data, "Fetches allowlisted typed evidence", "HTTPS")

Rel(web, workerApi, "Creates research job", "Internal API")

Rel(workerApi, db, "Reads frozen snapshots; appends artifacts", "Restricted DB role")

Rel(workerApi, worker, "Future: dispatches immutable job", "GitHub Actions workflow")

Rel(worker, workerApi, "Future: writes result file for trusted wrapper", "No network from compute container")

Rel(web, exec, "Sends owner-approved proposal only", "In-process")

Rel(exec, broker, "Submits after final checks", "Kairos-owned adapter")

The future compute container has no relationship to the broker, credentials vault, browser, data providers, or callback endpoint. A trusted Actions workflow wrapper, outside the `--network none` compute container, validates the output and posts it to Kairos using a short-lived callback proof. The wrapper is therefore a reviewed Kairos component, not third-party code. The absence of a worker-to-broker or worker-to-provider path is an intentional security control, not a convention.

7. Capability Classes And Authority Matrix

Class	Examples	Can read	Can write	Never allowed
`indicator`	TA-Lib, internally reimplemented formula	Frozen OHLCV snapshot	Derived indicator artifact	Network, LLM prompt, score/or
`backtest`	Kairos replay engine, approved quant library	Frozen decision/evidence/price snapshots	Experiment result artifact	Live/cu
`research_advisory`	Vibe-derived research workflow	Bounded facts, approved documents, prior artifacts	Hypothesis or critique artifact	
`document_extract`	Parser/OCR library	User-approved document copy	Structured extraction artifact	External upload, unrestricted files
`data_adapter`	Kairos-owned Yahoo/EDGAR/Webull adapter	Intent and provider config	Evidence cache/ledger only	Arbitrary URLs or raw f

There is deliberately no `execution`, `portfolio\_write`, `config\_write`, `database\_admin`, or `broker\_read` third-party capability class.

State machine

proposed -> license_review -> contract_test -> research_only -> shadow_only

-> retired

- `proposed`: metadata exists, no code/image installed or run.
- `license\_review`: source and license are reviewed; still cannot run.
- `contract\_test`: pin runs only on synthetic fixtures with network disabled.
- `research\_only`: may create visible, non-actionable artifacts from snapshots.
- `shadow\_only`: its structured hypothesis may be evaluated by Kairos's existing deterministic shadow/validation flow, still with no fills or proposals.
- `retired`: no new jobs; historic results remain immutable and visible.

No transition from any integration state can enable paper fills, live proposals, or live orders. Those are controlled only by existing Kairos strategy and broker governance states.

8. Integration Registry

The registry is code-owned plus database-configured. Database rows select only compiled, reviewed integration identifiers; they cannot name a URL, command, Docker image, npm package, Python package, or MCP tool.

Proposed logical records

Record	Purpose	Required fields
integration_catalog	Immutable identity of a reviewed integration	id, slug, capability_class, source_repo, license_spdx, owner
integration_releases	One reviewed build/version	integration_id, source_commit, release_tag, artifact_digest, dependency_lock
integration_policies	Per-market enablement and resource limits	release_id, market, state, max_jobs_day, timeout_seconds, capacity
integration_runs	Append-only job header	run_id, release_id, policy_version, market, currency, snapshot_hash, status, strategy
integration_artifacts	Append-only validated outputs	run_id, artifact_type, schema_version, payload_hash, storage_ref, quality
integration_security_events	Blocked/failed security events	run_id, event_type, severity, redacted_context, created_at
upstream_watch_events	Detected upstream changes	integration_id, observed_ref, change_kind, review_status, observed_at

These are design names, not approved migrations. All financial and research history remains append-only. RLS must be owner-only for UI reads; the worker uses a job-specific backend credential that can create only its own run/artifact rows through narrowly scoped RPCs.

Required release manifest

Every release must contain a checked-in manifest such as:

```
type IntegrationManifest = {
  slug: "ta_lib_indicators" | "vibe_research_adapter";
  capability: "indicator" | "research_advisory" | "backtest";
  sourceRepo: string;
  sourceCommit: string; // full immutable SHA, never a branch
  licenseSpdx: string;
  artifactDigest: string; // image/package digest
  dependencyLockDigest: string;
  inputSchemaVersion: string;
  outputSchemaVersion: string;
  networkMode: "none";
  permittedMarkets: Array<"us" | "india">;
  permittedArtifacts: string[];
};
```

The manifest is reviewed code. Settings can enable only a manifest that has an approved release record.

9. Data Contracts

9.1 Frozen worker input (future only)

The Canonical Evidence Router is the only evidence/provenance contract. A future gateway does not re-create provider, freshness, quality, policy, or field provenance types. It freezes the router's existing `EvidenceEnvelope` values and adds only the market-local universe and permitted existing Kairos references. This prevents a parallel truth layer.

```
type ResearchSnapshot = {
  snapshotId: string;
  snapshotHash: string;
  market: "us" | "india";
  currency: "USD" | "INR";
  asOf: string;
  createdAt: string;
  evidencePolicyVersionId: string;
  strategyVersionId?: string;
  mandateId?: string;
  universe: Array<{ symbol: string; assetClass: string }>;
  evidence: Array<EvidenceEnvelope<unknown>>;
  decisionObservations?: Array<{ observationId: string; decisionAt: string; features: unknown; outcome?: unknown }>;
};
```

Rules:

- The snapshot includes one market and one currency only.
- `asOf` is a hard ceiling. Historical replay uses only evidence known at or before that instant.
- `EvidenceEnvelope` retains the Canonical Evidence Router's `intent`, quality, cache state, provider attempts, policy version, unavailable reason, and field-level provenance. The integration layer adds none of these concepts.
- The worker sees normalized router facts, not provider tokens, URLs, cookies, raw credential errors, browser sessions, or user account data.
- Live position quantities/account balances are absent unless a future, separate architecture explicitly permits a redacted risk-analysis snapshot. That future capability still may not execute.

9.2 Worker result envelope

```
type WorkerResult = {
  runId: string;
  snapshotHash: string;
  integrationSlug: string;
  sourceCommit: string;
  artifactDigest: string;
  resultKind: "indicator_set" | "backtest" | "hypothesis" | "critique" | "document_extract";
  status: "completed" | "abstained" | "invalid" | "failed";
  quality: {
    warnings: string[];
    inputCoveragePct: number;
    reproducible: boolean;
  };
  payload: unknown;
```

producedAt: string;

};

The gateway verifies `runId`, snapshot hash, manifest identity, schema, bounded payload size, numeric finiteness, market/currency match, and allowed artifact type. Any failure becomes `invalid`, emits a security/health event, and stores no candidate strategy, score, or trade action.

9.3 Artifact-specific restrictions

- An `indicator\_set` contains derived values and formula/version metadata, never a buy/sell recommendation.
- A `backtest` includes assumptions, universe, date bounds, transaction costs, price basis, benchmark, and point-in-time coverage. It cannot mark itself eligible or promoted.
- A `hypothesis` contains a human-readable proposal plus a structured, whitelisted feature/gate specification. It cannot contain executable code.
- A `critique` may identify evidence gaps or contradictory assumptions, but it cannot revise a score, ledger, or configuration.

10. Controlled Flows

A. Research advisory flow (Vibe-style integration)

1. Owner enables a pinned `research\_advisory` release for one market in `research\_only` state.
2. Kairos gathers already validated evidence through the Canonical Evidence Router and freezes a bounded snapshot.
3. Gateway sends the snapshot and an allowed task template to the worker.
4. Worker returns a hypothesis/critique artifact with citations to snapshot IDs, not external free-form sources.
5. Kairos renders it as **External Research - Advisory** with evidence coverage, version, and warnings.
6. A human may convert a suitable hypothesis into an existing Kairos feature registry proposal. The whitelisted feature grammar and Validation Engine remain the only path toward shadow evaluation.

The artifact cannot change `agent\_signals`, `strategy\_versions`, a paper order, or a broker proposal by itself.

B. Indicator/backtest flow (TA-Lib-style integration)

1. Research or an owner-triggered experiment requests an approved indicator or backtest capability.
2. Gateway creates a market-local point-in-time snapshot, including adjusted price basis and price/evidence provenance.
3. Worker computes output with no network.
4. Gateway validates and persists an artifact.
5. The existing deterministic validator, not the worker, performs walk-forward folds, purging/embargo, cost treatment, benchmark calculation, and promotion gate checks.
6. A passing result may create a `validation\_experiments` record and at most a `shadow\_paper` candidate under existing owner/automation controls.

C. Current money path remains unchanged

sequenceDiagram

participant W as External Worker

participant K as Kairos Research/Validation

participant P as PaperTrader

participant O as Owner

participant E as Kairos Execution Gateway

participant B as Broker

W-->>K: Advisory artifact or derived metrics only

K-->>K: Validate deterministically and optionally shadow

K-->>P: Existing approved champion signal only

P-->>K: Existing paper ledger events

K-->>O: Existing live proposal, if independently eligible

O->>E: Explicit approval

E->>E: Recheck all money-path gates

E->>B: Submit through Kairos-owned adapter

There is intentionally no arrow from `W` to `P`, `E`, or `B`.

11. Security Threat Model And Controls

Threat	Failure scenario	Required control
Supply-chain compromise	A dependency or upstream release becomes malicious	Pin full commit + artifact digest; lock dependencies; SBOM; v
Prompt injection	A news article, README, or provider string asks a worker/LLM to change behavior	Treat external text as data; schema/bou
Credential theft	Third-party code reads env/vault/browser token	Dedicated worker runtime has no secrets, host mounts, shell access, or p
Broker bypass	Worker calls a broker/API/MCP tool directly	Network deny by default; no broker DNS/egress; no credentials; execution gatew
Database corruption	Worker obtains broad database access	Job-scoped signed token plus append-only RPCs; no direct database connection; M
Data exfiltration	Worker sends account/history data to an arbitrary host	No egress by default; only sanitized snapshot; outbound block
Replay leakage	Backtest reads data that was unavailable at decision time	Snapshot `asOf`, evidence observation/retrieval times, sealed m
Resource exhaustion	Bad library hangs, forks, or allocates memory	Per-run CPU/memory/time/file-size limits; concurrency cap; cancel wat
Dependency drift	A version update changes numerical results silently	Golden fixtures, deterministic result hashes/tolerances, release co
License contamination	Code is copied from incompatible source	License review before source import; source/commit attribution; permitted

Security controls that are mandatory before any worker exists

1. Separate runtime identity from the Next.js/Vercel runtime.
2. No Docker socket, host filesystem mount, SSH agent, Windows user profile, or `.env\*` file in the worker environment.
3. Read-only filesystem except an ephemeral working directory.
4. Egress disabled. The first release must require **zero network**.
5. A short-lived, single-use job token bound to `runId`, artifact digest, snapshot hash, expiry, and permitted result schema.
6. Gateway idempotency: a job can append its result once; retries cannot duplicate strategy candidates or artifacts.
7. Redacted logs: no raw snapshot payloads, access tokens, provider errors, account numbers, or LLM prompts in central logs.
8. A global integration kill switch plus per-integration/per-market disable.
9. Worker output parser rejects prototype-pollution keys, unexpected nested data, executable code, URLs outside cited snapshot references, and non-finite values.

12. Upstream Review And Update Policy

An upstream update is an input to a review queue, never a deployment event.

Intake checklist for any new repository

1. Establish the desired capability and confirm Kairos does not already own it.
2. Record repository owner, canonical URL, full commit, release/tag, license, language/runtime, maintenance signal, dependency graph, and market relevance.
3. Check license compatibility before copying code or creating a fork.
4. Review source for secrets, shell execution, dynamic code loading, outbound network behavior, telemetry, subprocesses, filesystem access, and credential assumptions.
5. Define the minimum snapshot and artifact schema before installation.
6. Run only synthetic fixtures with network disabled.
7. Add golden tests, failure tests, resource-limit tests, and a kill-switch test.
8. Approve research-only state manually. No installation can skip these steps.

Update lifecycle

1. A scheduled **metadata-only** watcher records a new upstream release/commit.
2. A reviewer compares license, manifest, dependency lock, SBOM, and relevant source diff against the pinned release.
3. A candidate image is built in an isolated CI environment and scanned.
4. It runs fixture and regression suites against the same snapshots as the old release. Numerical output differences require an explanation.
5. The candidate may enter `contract\_test`; it cannot replace the active pin.
6. An owner promotes the release by changing the active registry pointer.
7. The old release remains available for rollback until the configured retention period has elapsed.

If an upstream repository is deleted or abandoned, active pinned artifacts remain reproducible. For a capability worth keeping, Kairos may maintain a private or internal mirror/fork **only after its license permits that**. Kairos must never depend on cloning GitHub at request time.

13. Candidate Mapping From Reviewed Projects

Candidate	Valuable capability	Initial treatment	Explicit boundary
TA-Lib	Standard technical indicators	Deferred candidate only if pure TypeScript/provider parity is insufficient	No worker or native de
Vibe-Trading	Research-team workflows, hypothesis/critique artifacts, backtest evidence patterns	Later `research_advisory` adapter after	
ML4T	Walk-forward/backtest methodology and test corpus ideas	Reference only	Reimplement only narrow, reviewed methods in Kairos tests
Qlib	Experiment/data/feature lifecycle concepts	Reference only until a later model-research phase	No framework import into money path
OpenBB	Provider capability/adaptor architecture	Architecture reference only	No source/runtime use under this plan because current rep

| FinRL, Qbot, strategy collections | Hypothesis discovery | Reference only | No imported strategy may bypass existing validation/promotion gate
| QuantDinger, AI Berkshire | Explainability/research-journal ideas | Product/reference only | No security/execution code reuse without separation

Every candidate must pass the intake checklist independently. A positive review of one project does not create trust in a future version, fork, package, or skill.

14. Product Surface

Add one Settings area, **Research Integrations**, only after the backend safety model exists. It is operational, not a marketplace.

Registry view

For each approved integration show:

- Capability and plain-language purpose.
- Market scope and current state.
- Pinned source commit/release, artifact digest, and license.
- Last contract test, last run, error rate, timeout rate, and output coverage.
- Inputs it can receive and artifacts it can produce.
- A clear statement: ``Cannot access accounts or submit orders``.
- Enable/disable control requiring owner confirmation; disable is immediate.

Artifact view

Research/Journals can show an externally produced artifact only with:

- ``Advisory`` or ``Derived Indicator`` label.
- Snapshot as-of time, market/currency, policy and strategy versions.
- Integration version and data coverage warning.
- Evidence links to Kairos snapshots.
- A visible path to inspect the deterministic validation result, if one exists.

No UI wording implies an artifact is a trade command, prediction guarantee, or live-account recommendation.

15. Observability And Health

Track per integration, release, market, and artifact type:

- jobs requested/started/completed/abstained/invalid/failed;
- queue age, runtime, CPU/memory limit hits, and timeout rate;
- snapshot coverage and stale/unavailable evidence ratio;
- schema validation failures and blocked egress attempts;
- result reproducibility/hash mismatch rate;
- artifact-to-hypothesis, hypothesis-to-validation, and validation-to-shadow conversion rates;
- zero financial metric attribution: integrations do not get credit for returns until an independent Kairos strategy lifecycle has evidence.

Health degradation disables new jobs at the policy layer. It does not delete artifacts or alter an existing champion/paper/live position.

16. Phased Build Order

Prerequisite - Canonical Evidence Router Phase 2 foundation

This is the next implementation work, not the integration framework. It creates the one provenance/capacity substrate that future integrations must consume:

- canonical contracts and intent catalog;
- immutable policy versions and active market pointers;
- durable evidence cache, provider-call ledger, refresh queue, and pacing repair;
- typed adapter registry and independently testable adapters; and
- all-Auto policy seeds with ``router_enabled=false`` and dual-resolution logging.

Its disjoint foundation work may be parallelized in isolated worktrees: policy migration, cache/ledger/queue migration, canonical contracts, capacity RPC repair, and one adapter per worker. The router stays shadow-only until its own Phase 2 evidence proves score/availability/entry-eligibility parity. Do **not** build its research cutover in parallel with the foundation.

Exit gate: canonical and legacy outputs are available for comparison; no provider choice can affect production scoring, paper fills, or live proposals.

Phase 1 - In-repository technical indicator hygiene

Do not create a worker just to run TA-Lib. First inventory the 6-10 technical formulas Kairos actually uses. For each, choose either the current validated provider value or a small pure TypeScript implementation using Kairos price snapshots. Add formula/version metadata and golden fixtures against known values.

- No new external runtime or native dependency by default.
- No scoring-weight, entry, paper-fill, or execution change in this phase.
- Do not duplicate an Alpha Vantage/other router value without a documented

semantic reason (period, smoothing, adjustment basis, or availability).

****Exit gate:**** exact formula semantics, price basis, warm-up behavior, missing-data behavior, and fixture tolerances are documented and tests are green.

Phase 2 - Deferred isolated compute framework

Build this only after a genuine untrusted/LLM-driven integration has a narrowly defined value that cannot be met in-repo. The initial cloud-only target is a GitHub Actions job using a pinned Ubuntu runner and a pinned compute image.

- The workflow wrapper receives a single-use dispatch/callback proof; the `docker run --network none` compute container receives neither that proof nor any other secret.
- The compute container writes a bounded result file; the reviewed wrapper validates it and performs the authenticated callback to Kairos.
- Pin every workflow action by full commit SHA, restrict `GITHUB\_TOKEN` permissions, do not expose Actions secrets to the compute container, and do not upload market snapshots as public artifacts.
- GitHub Actions is quota-limited for private repositories. Dispatch must stop at a conservative included-minute/storage threshold and paid overage must remain disabled. It is free only while within the account's included allowance.
- A future persistent/free-cloud fallback, if truly needed, requires a separate deployment, network, and secrets architecture review. Do not substitute a local Windows service.

****Exit gate:**** synthetic jobs prove the container has no egress/secrets, the wrapper is the only callback identity, results are reproducible, and disabled state cannot dispatch work.

Phase 3 - External advisory research adapter (Vibe candidate)

- Complete source-license-security review of the exact Vibe release first.
- Add a narrowly defined advisory task template and structured hypothesis/critique output schema, using only frozen `EvidenceEnvelope`-derived snapshots.
- Permit no external tool/network use from the compute container.
- Require a human conversion into the existing feature registry proposal flow.

****Exit gate:**** prompt-injection corpus passes; artifacts cannot alter a score, strategy, paper position, proposal, or execution path; owner finds UI useful.

Phase 4 - Optional capability expansion

Potential additions are evaluated individually: document extraction, additional indicator libraries, Qlib-inspired experiment tooling, or a permitted local fork. Each repeats the same intake and rollout process. There is no blanket approval for a GitHub topic, organization, or skill collection.

17. Rollback And Incident Response

Normal disable

Owner disables an integration globally or per market. Gateway rejects queued/new jobs immediately. In-flight jobs are cancelled or their results rejected. Existing artifacts remain visible as historical records, clearly marked disabled.

Security incident

1. Disable the affected release globally.
2. Revoke its job-token issuer/worker identity.
3. Block the image/artifact digest from future dispatch.
4. Preserve redacted run/security evidence for review.
5. Confirm no broker, vault, or direct database path existed; inspect gateway audit events for rejected attempts.
6. Roll back to a previously approved release only after its digest and tests are reverified.

No incident response action mutates historical financial ledgers. Any questionable artifact is marked `security\_quarantined` and excluded from future research views and validation inputs.

18. Acceptance Criteria

The future isolated-compute implementation is acceptable only when all are proven:

1. A disabled integration cannot start a job.
2. A worker has no broker/vault/Supabase-service-role/browser credentials.
3. DNS/egress to broker domains, arbitrary hosts, and provider domains is blocked.
4. A worker cannot reach any order, proposal, settings, or direct DB endpoint.
5. A job token cannot be reused, altered for another snapshot, or used after expiry.
6. Invalid, oversized, non-finite, unexpected-schema, or wrong-market output is rejected and produces no candidate strategy or score mutation.
7. A US job cannot contain or create India/INR data, and vice versa.
8. Re-running the same pinned release against the same snapshot produces the same result within defined numerical tolerance and records both provenance.

9. A worker timeout/kill/egress attempt fails closed and leaves existing Kairos research, paper, live proposals, and positions unchanged.
10. Upstream update detection cannot update an active release automatically.
11. The UI accurately states state, pin, inputs, outputs, health, and lack of account/execution access.
12. Existing benchmark-alpha, capital-rotation, learner, PaperTrader, PositionMonitor, and broker tests remain unchanged/green with every integration disabled.

19. Open Decisions For Claude And Owner

1. **Future worker hosting:** GitHub Actions is the proposed cloud-only initial runtime, but private repositories have included-minute/storage quotas rather than an unconditional free tier. Recommendation: use it only with a strict no-paid-coverage dispatch guard; do not run a local Windows service and do not run native worker compute inside Vercel.
2. **Artifact storage:** Supabase JSONB versus object storage with hashed references. Recommendation: small structured results in Postgres; large reports/arrays in private storage with immutable hash references.
3. **License policy:** which licenses are approved for dependency, permitted fork, reference-only, and prohibited categories. Recommendation: explicitly prohibit copyleft runtime dependencies until reviewed; record SPDX per release.
4. **Vibe source boundary:** whether to reimplement a minimal adapter from public behavior, use a permitted pinned fork, or keep it as a manual/offline research tool. Recommendation: prove utility manually first, then choose the smallest permitted integration.
5. **Data Policy dependency:** should any provider-dependent integration wait for the Canonical Evidence Router? Recommendation: yes. Its snapshot must embed the router's `EvidenceEnvelope`, not a second provenance type. In-repo formula fixture work can proceed without provider access but cannot create a new competing evidence path.
6. **Review authority:** who may move `contract\_test` to `research\_only`. Recommendation: owner approval plus a completed security/code review; no automation can promote a release.

20. What This Document Explicitly Does Not Approve

- Installing a GitHub project directly into the Next.js app.
- Executing a repository's install script on the developer workstation.
- Giving Vibe, TA-Lib, Qlib, OpenBB, or any future integration a live account, broker, MCP, API-vault, service-role, or shell credential.
- Replacing Kairos's deterministic scoring, validation, benchmark, paper ledger, capital rotation, or execution gateway.
- Enabling Webull orders, autonomous live trading, or a new provider from this architecture.
- Treating a repository's README, stars, claimed backtest, or LLM output as evidence of trading edge.

21. Recommended Owner Decision

Approve the authority boundary in principle, but do **not** build a worker now. Build the Canonical Evidence Router Phase 2 foundation behind flags and shadow comparison first. Then do the small in-repository technical-indicator hygiene work only where it has a demonstrated gap. Vibe and every other untrusted repository remain deferred until there is a concrete need for the isolated GitHub Actions compute pattern and its boundary has been independently reviewed.

Source: features\external-research-shadow\FEATURE_ARCHITECTURE.md
External Research Shadow Runtime

> Status: **APPROVED FOR P0/P1 ISOLATED FOUNDATION ONLY (2026-08-10).**
> Last reviewed: 2026-07-15 by Codex.
> Authority: subordinate to `features/external-research-integrations/FEATURE\_ARCHITECTURE.md`.
> Scope: optional, record-only research compute. No money-path authority.

1. Decision

Do not run a "self-sourcing" external repository as the first phase. Self-sourcing requires outbound network and usually credentials, which contradicts the required `docker run --network none` compute boundary. Independent data can be valuable, but acquisition must occur in a reviewed Kairos-owned adapter before untrusted compute.

The only approved conceptual runtime is:

1. trusted Kairos acquisition freezes a minimal public-market-data snapshot;
2. untrusted, pinned compute receives that snapshot with no network or secrets;
3. a trusted wrapper validates and ingests a bounded artifact; and
4. Kairos evaluates it as research evidence only.

No external repository directly becomes a Kairos agent, candidate-discovery source, Challenger, scorer, provider adapter, or broker dependency. A useful result may motivate a separately specified, clean-room, Kairos-owned deterministic feature.

2. Preconditions

The production third-party runtime must not be built until all are true:

- Router cutover and eligibility guard are complete for the target market;
- experiment lineage can reference Router policy/evidence fingerprints without a third provenance system;
- an exact repository commit, license, transitive dependency lock, and capability have passed source/security/legal review;
- the same hypothesis cannot be tested more cheaply and safely in native Kairos;
- an owner-approved resource budget, kill switch, retention policy, and threat model exist; and
- a synthetic fixture run passes before any real snapshot is exported.

P0/P1 may build only Kairos-owned contracts, synthetic fixtures, source-release metadata, validation, and an owner-disabled registry. It may not clone, execute, or package an upstream repository; export provider data; issue worker credentials; dispatch GitHub Actions; or accept a third-party artifact. This makes the feature optional and later than native experiment lineage.

3. Trust Boundaries

Trusted control plane

A Kairos-owned workflow repository and reviewed wrapper may:

- construct a job manifest from an approved integration registry entry;
- download a prebuilt, digest-pinned compute image or build from a reviewed lock;
- place the immutable input snapshot in a bounded read-only mount;
- launch ``docker run --network none`` with hard resource restrictions;
- validate/scan the output after the container exits; and
- upload only the accepted artifact and trusted wrapper logs.

Untrusted compute plane

The repository code receives:

- no GitHub token, OIDC token, environment secret, provider key, broker token, Supabase credential, browser cookie, portfolio/account data, or callback proof;
- no outbound or inbound network;
- no host Docker socket, privileged mode, device access, or writable host path;
- a read-only input mount and one size-bounded output directory;
- a read-only root filesystem where compatible, non-root UID, dropped capabilities, PID/memory/CPU/time limits, and a process/file-size limit; and
- one approved command with no user-supplied shell fragment.

The repository cannot upload artifacts itself. The trusted wrapper regains control after exit, rejects unsafe files/links/archives, validates JSON, and uploads only the normalized result.

GitHub-hosted runner ephemerality is defense in depth, not the sandbox. Job-level Actions ``container:`` is not treated as proof of network isolation; the reviewed wrapper must execute the explicit no-network container and test egress denial.

4. GitHub Actions Security Baseline

- workflow exists only in a Kairos-owned private control repository;
- no ``pull_request``, ``pull_request_target``, issue-comment, or fork-controlled trigger;
- manual/scheduled dispatch selects only code-known integration IDs;
- top-level ``permissions: { contents: read }`` or stricter; no write permission;
- no cloud OIDC, deployment environments, production secrets, or persistent runner;
- every third-party Action is pinned to a full commit SHA and reviewed;
- no untrusted cache restore, reusable workflow from a floating ref, or mutable tag;
- concurrency is one, timeout and daily job caps are enforced before dispatch;
- no paid overage; quota exhaustion is an abstention;
- source/image/dependency digests, action SHAs, runner image, and wrapper version are written to the run record; and
- hostile-output fixtures test ANSI/log injection, symlinks, path traversal, zip bombs, huge JSON, NaN/Infinity, deep nesting, HTML/Markdown/script, and bad market.

5. Immutable Snapshot Contract

Do not export mutable ``evidence_cache_v2`` rows directly. Create an immutable research snapshot manifest that references existing evidence and lineage records:

```
type ExternalResearchSnapshot = {
```

```
  snapshotId: string;
```

```
  snapshotHash: string;
```

```
  schemaVersion: string;
```

```
  market: "us" | "india";
```

```
  currency: "USD" | "INR";
```

```

asOf: string;
policyVersionId: string;
experimentRunId: string;
universeVersion: string;
symbols: string[];
evidenceRefs: Array<{
evidenceRecordId: string;
envelopeHash: string;
intent: string;
availableAt: string;
}>;
files: Array<{ path: string; sha256: string; bytes: number }>;
};

The materialized files contain only the minimum normalized public-market facts
needed by the capability. Exclude owner identity, watchlist rationale, account,
holdings, orders, fills, cash, broker, credentials, internal prompts, and raw
provider responses. A historical snapshot enforces `availableAt <= asOf`.

Provider acquisition remains in Router-owned adapters with normal quota, license,
and terms controls. An external project's data loader, URL, key, browser automation,
or MCP tool is never admitted into the untrusted container.

```

6. Output Contract

```

type ExternalResearchArtifact = {
schemaVersion: string;
runId: string;
snapshotHash: string;
integrationId: string;
sourceCommit: string;
imageDigest: string;
market: "us" | "india";
asOf: string;
status: "completed" | "abstained";
artifactKind: "hypothesis" | "critique" | "numeric_signal" | "backtest_report";
coverage: { eligibleN: number; resolvedN: number };
payload: unknown;
};

```

The trusted gateway checks exact IDs/hashes, market/currency, schema/version, bounded strings/arrays/numbers, finite numeric values, symbol allowlist, payload size, and allowed artifact kind. Rationale is stored/rendered as escaped plain text; URLs, executable code, raw HTML/Markdown, files, and tool instructions are rejected. Invalid output records a run failure but does not persist a partial advisory.

7. Records Without A Parallel Truth Layer

Extend the approved experiment-lineage model with:

- `external\_research\_runs`: experiment ID, integration release, snapshot fingerprint, wrapper/action/image digests, resource use, status, and failure code;
- `external\_research\_artifacts`: run ID, kind, schema, payload hash, storage reference, validation status, and retention class; and

- security events containing normalized codes only.

These records reference existing `experiment\_runs`, Router policy/evidence records, and labels. They do not copy source provenance into a third ledger. Do not create an `external\_advice` table that pretends a repo generated a Kairos decision, and do not manufacture `decision\_observations` to improve sample size.

RLS is owner-read; only a service-role RPC owned by the trusted gateway may append a result for the exact issued run. Tables are append-only and reject update/delete.

8. Evaluation

External output is compared only on a predeclared common universe, as-of time, horizon, benchmark, and cost model. Numeric signals must be alignable to existing observations without selecting only names the repo liked. Report missingness and abstentions.

Every repository, commit, prompt/configuration, signal definition, regime slice, and horizon is part of the trial family. Use walk-forward/holdout evaluation and DSR/PBO governance. "Who beats us where" is not a product claim unless confidence, coverage, multiple testing, and incremental value over the champion are shown.

An external artifact can remain advisory indefinitely. Promotion means writing and validating a native Kairos feature under a new architecture; it never means wiring the upstream runtime into candidate discovery or trading.

9. Licensing And Supply Chain

Runtime execution does not make license obligations disappear. Before admission:

- verify the exact commit's license and every bundled/transitive dependency;
- define whether the use is copy, modification, distribution, or network service;
- obtain explicit legal/product approval for copyleft, source-available, unclear, missing, or changed licenses;
- generate/store SBOM and vulnerability/malware scan results;
- mirror source/artifacts only when the license permits it; and
- never auto-sync upstream. A new commit is a new release and repeats review/tests.

Clean-room reimplementations applies to ideas later rebuilt in Kairos. It is not a substitute for complying with the license of code actually executed.

10. Phases

1. ****P0 synthetic harness:**** Kairos-owned toy program, synthetic data, egress-denial and hostile-output tests. No third-party repo and no production snapshot.
2. ****P1 one reviewed capability:**** one permissively licensed, exact-pinned repo on a minimal immutable snapshot, record-only, owner-triggered, no LLM required.
3. ****P2 repeated shadow:**** scheduled bounded runs after retention/cost review; common-universe evaluation, still no influence.
4. ****P3 native hypothesis:**** if evidence warrants, design a separate clean-room Kairos feature and send it through normal measure/shadow/validation governance.

Vibe is not automatically P1. Its breadth and agent/tool surface increase review cost; a minimal adapter must be justified and source-audited first.

11. Disable, Incident Response, Acceptance

A code-owned global kill switch and per-integration policy stop dispatch immediately. Revoke wrapper credentials, disable workflow, quarantine unvalidated artifacts, and retain immutable audit metadata. Existing Kairos research/scoring/trading continues unchanged because there is no runtime dependency.

Acceptance requires a proven egress-denial test, zero secrets exposed to compute, hostile artifact rejection, reproducibility from hashes, per-market isolation, quota stop, append-only/RLS tests, and explicit proof that no import/call path reaches signals, strategy activation, broker adapters, or order execution.

12. Owner Decisions Before Build

1. Whether this optional framework is worth building after native lineage.
2. The first narrow capability, exact commit, and license class.
3. Maximum daily Actions minutes/storage and retention.
4. Whether private GitHub Actions is acceptable as the initial control plane after the sandbox threat model is independently reviewed.

Source: features\external-skill-observation\FEATURE_ARCHITECTURE.md
External Skill Observation Adapter

> Status: ****DRAFT - NOT APPROVED FOR IMPLEMENTATION****
> Scope: one optional, record-only experiment; no trade influence

Decision

Do not wire Claude interactive skills, their prompts, or their broker/data tools into

ResearchAgent. `nse-trading-toolkit` is a prompt workflow with optional Groww MCP and Yahoo/yfinance dependencies, not a deterministic library with a stable output contract. `garchmethod` may be useful later for a volatility hypothesis, but it is not evidence that its code or recommendations should enter the money path.

The smallest credible experiment is a Kairos-owned deterministic GARCH(1,1) ****observation**** calculated from Kairos-frozen daily returns, written beside existing research evidence for a single market. It is not sourced from the installed skill and does not consume a provider key beyond the candle data Kairos already owns.

Boundaries

- Input: one market-local, immutable daily-return snapshot; no holdings, account, broker, token, prompt, or external MCP access.
- Compute: a pinned, reviewed implementation in an isolated offline job, or clean-room native deterministic code after a separate license/security review.
- Output: `garch\_vol\_forecast`, model/version/hash, return-window end date, convergence status, and unavailable reason. Finite values only.
- Consumer: evidence ledger/read-only dashboard only. It cannot modify score, rank, sizing, stop, target, eligibility, PaperTrader, PositionMonitor, proposals, or orders.

Why This Is The Right First Step

It tests a measurable claim: whether a market-local volatility forecast improves forecast calibration or risk diagnostics over realized-volatility baselines. It avoids the unsafe alternative of treating a natural-language trading framework as an agent input, and it preserves the option to reject the idea with no production impact.

Evaluation And Promotion

Compare the observation with predeclared realized-volatility baselines using frozen walk-forward windows, separately by market, with sample floors and costs where a future sizing simulation needs them. Register every model/order/window variant in the trial family. A later sizing change requires its own approved architecture and must apply in PaperTrader's sizing construction before the execution gateway, never inside the gateway.

Explicit Non-Goals

- No automated invocation of Claude skills, Python scripts, Groww/TradingView MCP, yfinance, or upstream repo data loaders.
- No API-key sharing, network-enabled GitHub compute, external code execution, or provider-quota bypass.
- No direct VCP/CANSLIM/sector-rotation feature added to `agent\_signals`.
- No use for US and India together; each model, evidence set, and conclusion is local.

Prerequisites And Acceptance

Start only after Router cutover/lineage prerequisites and sufficient labeled outcomes exist for the target market. The build must prove no call path from the observation to trading, validate deterministic replay from a snapshot hash, report unavailable rather than fabricate a forecast, and retain the full trial lineage. Any external repository execution remains governed by `features/external-research-shadow/FEATURE\_ARCHITECTURE.md`.

Source: features\external-strategy-discovery\FEATURE_ARCHITECTURE.md
External strategy discovery

> Status: ****REVISION 2 – DRAFT, not implementation-ready. No code written.****
> Date: 2026-09-01. Influence if approved: ****none.**** Every stage is measure-only;
> paper activation and live activation are separate, later owner decisions.
>
> Independent review ruled ****(c): a corrected Stage 0****, and found several claims
> in revision 1 wrong. All corrections below were verified against the code and
> production before being accepted. Review brief:
> `docs/audits/2026-09-01-external-strategy-discovery-codex-brief.md`.
>
> Supersedes `features/strategy-library-shadow/FEATURE\_ARCHITECTURE.md` (2026-09-01),
> which proposed the same idea without the legal boundary, without the
> combination design, and while assuming a foundation that does not fully exist.

Purpose

Turn a public catalogue of published trading strategies into a controlled research funnel that answers a portfolio question, not a return question:

> ****What role does this strategy play, and does adding it improve the portfolio
> we already have?****

Explicitly NOT "crawl 200 strategies and activate the winners."

1. Legal and ethical boundary – binding

The catalogue's [`robots.txt`](https://www.quantifiedstrategies.com/robots.txt) permits crawling, but its [copyright policy](https://www.quantifiedstrategies.com/copyright/) **prohibits** copying or reproducing content¹, while allowing fair-use reference with attribution.

Therefore, as a hard constraint on any implementation:

- **Store only public metadata**: title, URL, category, publication/update date.
- **Store only independently written rule specifications**, in our own words, with source attribution. A rule is an idea; the article is their property.
- **Never store** article text, charts, images, paid rules, or their code.
- **Exclude** `/shop`, `/member`, `/amember`, paid downloads and gated content.
- **Rate-limit politely**, and prefer requesting written permission before automating hundreds of article fetches.
- Firecrawl (or any crawler) is a **local** research-ingestion tool, never a production dependency. Firecrawl was unavailable in both the Claude and Codex sessions of 2026-09-01 (`CONNECT_TIMEOUT`); the pages are static HTML so a plain fetch suffices.

Catalogue size measured 2026-09-01: **408** unique internal links, **~356** candidate content pages, **70+** categories.

2. What the foundation ACTUALLY is (verified, not assumed)

An earlier draft of this proposal described these as an existing foundation. They were checked against production on 2026-09-01:

component	state	evidence
--- --- ---		
<code>lib/simulation/portfolio-simulator.ts</code>	exists , but produces NO NAV path	returns <code>endingCash</code> , <code>positions</code> , <code>fills</code> , <code>rejections</code> , <code>lib/validation/</code> (8 modules) real , but answers a different question validates alternative SCORING WEIGHTS against decision observations
<code>strategy_templates</code> / <code>strategy_sleeves</code>	real 7 rows each	
<code>backtest_experiments</code>	real and exercised	18 rows: 13 alpha diagnostic, 3 OOS IC, 2 historical replay. The immutable experiment ledger
<code>strategy_evaluations</code>	0 rows — but this is NOT the relevant gap	written by <code>lib/evaluation/run-evaluation.ts:121</code> ; it is the mandated
<code>validation_experiments</code>	0 rows — THIS is the proof gap	written by <code>lib/validation/engine.ts:140</code> , the actual validation-engine
<code>strategy_template_shadow_configs</code>	not deployed <code>to_regclass</code> NULL; migration untracked	
<code>features/strategy-portfolio-lab</code> , <code>features/portfolio-simulation</code>	untracked drafts not in git	

CORRECTION accepted. Revision 1 claimed the empty `strategy_evaluations` proved the validation pipeline had never run. Wrong — that table belongs to a different subsystem. Producing a row in it would not prove external-strategy validation. The relevant empty ledger is `validation_experiments`.

Also corrected: revision 1 cited the portfolio simulator as able to produce the promised metrics. It cannot. Alpha Lab A6 gets its NAV path from its own `runPortfolioCalendar` in `lib/analytics/alpha-diagnostics-portfolio.ts`, not from the simulator. Any replay needs that marking layer built explicitly.

Consequence for sequencing. The problem is not merely that the pipeline is unproven — it is that the cited components **answer different questions** and are not connected into an external-strategy evaluation path. The validation engine grades scoring weights, not price rules. The simulator produces fills, not a NAV path. Neither gap is closed by "exercising" them.

Still: do not build a second backtester. Build the **missing seam** — a rule compiler, a deterministic NAV/benchmark marker, and a sealed result written to the experiment ledger that already works.

The shadow migration must NOT be applied unchanged. Its fingerprint covers only market, kind and template IDs, omitting operator, weights, rule version and trial family, so two different combinations can collide or be silently rewritten. It also has no immutable-config trigger and no append-only lifecycle ledger.

3. The funnel

public catalogue (metadata only)

- > reproducibility filter
- > frozen deterministic rule specification
- > point-in-time replay
- > cost / stress / walk-forward OOS gates
- > at most 3 forward shadows per market
- > owner-reviewed paper sleeve

-> tightly capped live trial (separate approval)

The app may recommend `continue`, `retire`, `prefer A`, `prefer A+B`, `pause`, or `submit for owner review`. **It may never activate a strategy on its own.**

4. Reproducibility filter – what is excluded and why

Excluded by construction, before any statistics:

- **Intraday and exact-close systems.** This book is EOD with a 2-20 market-day swing mandate. A close-entry-on-the-same-close rule is also a look-ahead: the catalogue's own Turnaround Tuesday article acknowledges this. Where a rule needs closing information to act, replay must use **next-open** execution.
- **Futures, forex, crypto, CFDs, options, leveraged products.** Out of mandate.
- **Short-selling and market-neutral pairs.** The mandate is long-only for new positions.
- **Paid or incompletely specified rules.** We would be testing a guess.
- **Rules needing data we do not have** point-in-time: sentiment history, market breadth, index-constituent history.
- **India translations of US effects** without an independently stated India hypothesis. A US seasonal effect is not evidence about NSE.

Also classified separately, not excluded:

- **Index-level calendar rules** (Turnaround Tuesday, Turn of the Month, Santa Claus Rally) produce the SAME signal for every symbol on a date. They carry **zero cross-sectional rank information** – the identical defect measured in `macro_score` on 2026-09-01, where the value was constant across 49 of 49 US dates. They must be evaluated as **market-timing / exposure overlays**, never mixed into a selection composite.

5. First trial family – ROLE SLOTS FIRST, names after a data audit

CORRECTED. Revision 1 named six strategies. Review found the family poorly balanced: roughly **five of six are index / exposure / rotation rules**, with only NR7 potentially supplying instrument selection. It also barely touches the known "win big, lose small" exit gap – the one place this book has measured evidence of a problem (73.7% of exits are the clock; `partial_target` returns a mean +20.20% on 5 lots).

Freeze ROLE SLOTS before naming anything:

```
| slot | count | purpose |
|---|---|---|
| entry rule | 2 | instrument selection |
| exit / holding rule | 2 | winner capture, loss limitation |
| exposure overlay | 1 | market timing, evaluated separately from selection |
| defensive allocator | 1 | drawdown reduction |
```

Names are chosen **after** the data audit below, not before.

Data sufficiency must be proven first

Review measured the cached history: SPY / QQQ / IWM / GLD hold only ~280 sessions, so after a 200-day warm-up **SPY MA yields roughly 80 decision sessions**. TLT and IEF hold ~35 sessions – unusable.

A replay that silently runs on 35 sessions produces a number, and that number is worthless. **Stage: acquire and seal sufficient licensed OHLCV and corporate-action history before any strategy is named.**

Six, not two hundred, remains right: selecting the best historical curve from hundreds is severe selection bias. See [Deflated Sharpe Ratio](https://doi.org/10.2139/ssrn.2460551) and [Harvey, Liu & Zhu](https://www.nber.org/papers/w20592).

6. What every replay must measure

Performance: net benchmark excess return; CAGR and total return; Sharpe, Sortino, max drawdown; profit factor and average win / average loss; MFE/MAE and winner-capture ratio; turnover, exposure, capital utilisation; worst month and tail loss.

Integrity – the part that decides whether any of the above means anything:

- point-in-time universe and adjusted prices (`lib/edges/pit-universe.ts` exists and fails closed; use it rather than the survivorship-biased curated list)
- signal `known_at` versus executable session
- **next-bar fills** wherever closing information is required
- spread, slippage, commissions
- delisting and corporate-action handling

- walk-forward OOS folds with purge and embargo
- ****parameter perturbation instead of best-parameter selection****
- trial-family count and multiple-testing adjustment
- comparison against buy-and-hold AND against the current Kairos champion

****Portfolio value:**** correlation with existing strategies; incremental benchmark alpha; whether it improves drawdown or merely duplicates existing exposure; whether earlier exits genuinely redeploy capital better.

7. Combination discovery

Extends the predeclared-combination design already sketched in
`features/strategy-portfolio-lab`. ****Not an unconstrained optimiser.****

For strategies A and B, simulate four portfolios:

****CORRECTED.**** Revision 1 proposed `A`, `B`, `A+B`, `champion+A+B` and omitted the two most important marginal comparisons. The required set is:

champion <- baseline

champion + A <- marginal value of A

champion + B <- marginal value of B

champion + A + B <- the joint decision

Standalone `A`, `B` and `A+B` remain useful diagnostics but cannot answer
"does adding this help the book we already hold".

****Shapley caveat.**** Exact attribution requires resimulating every necessary subset. Path dependence does not invalidate Shapley, but capital reuse and fill ordering make the characteristic function path-dependent, so ****event ordering and simultaneous-entry allocation must be frozen**** before any subset is run.

Three permitted forms

1. ****Parallel sleeves**** – independent trading, separate risk budgets. For strategies differing in return, holding period or regime.
2. ****Confirmation**** – enter only when both agree. Reduces false positives, but also reduces trade count and delays entry.
3. ****Regime routing**** – A under a predeclared regime, B under another. Permitted ****only**** when the routing rule is frozen before replay. The app may not search retrospectively for the regime split that flatters the curve.

No optimised free weights initially. Equal-risk, or explicitly declared fixed weights.

Complementarity report

Return correlation over the full period ****and specifically on losing days and high-volatility days**** – low average correlation is insufficient, because correlations rise under stress. Plus: signal overlap and duplicate-position rate; common-loss frequency; horizon and capital-use overlap; incremental excess return, Sharpe and Sortino; change in max drawdown and expected shortfall; change in payoff ratio and profit factor; turnover and cost increase; share of combination profit from its largest few trades; regime-by-regime contribution; and whether one strategy merely dilutes a stronger one.

Exact attribution

With combinations capped at three strategies, every subset is at most ****eight simulations****, so marginal contribution is deterministic:

portfolio with strategy - portfolio without strategy = marginal contribution

Shapley-style attribution for return, drawdown reduction and cost is therefore exact, not estimated. This surfaces cases such as: A produces the return while B cuts drawdown; B looks profitable only because A frees capital; A and B are the same exposure renamed; C helps in calm markets and causes nearly every joint tail loss.

`lib/simulation/portfolio-simulator.ts` is the engine for these subsets.

Hypothesis-guided proposal, not brute force

Combinations are proposed from ****portfolio coverage gaps****, not from historical return ranking: mean reversion + trend; short horizon + long horizon; equity + defensive rotation; high-win-rate/small-win + low-win-rate/large-winner; return sleeve + drawdown hedge; US strategy + separately governed India strategy with no cross-currency pooling.

An acceptable proposal states an economic reason first – "RSI mean reversion

struggles in persistent declines; the 200-day trend rule avoids much of that regime; test them regime-routed" – and freezes the pair and operator before looking at results.

Overfitting controls

Six strategies give 15 pairs, 20 triples and several operators; testing all and picking the best nearly guarantees a lucky winner. See

[Harvey & Liu, "Evaluating Trading Strategies"](https://people.duke.edu/~charvey/Research/Published_Papers/P116_Evaluating_trading_strategies) and [Bailey, Borwein & López de Prado](https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2739335).

- at most **one** active FORWARD-SHADOW combination per market – this must not prevent multiple immutable OFFLINE experiments, which are cheap and sealed
- at most **three** constituents
- combination, operator and weights **declared before replay**
- **every** attempted combination increments the trial-family count
- failed combinations stay on the record
- no tuning against the sealed OOS period
- forward shadow required after replay
- new combinations proposed from coverage gaps, never from return ranking

8. Per-instrument customisation

The owner's requirement – "some strategies work for some stocks and ETFs and not others" – is correct as intuition and is the most dangerous item here if implemented naively. An independent test per (strategy, instrument) over ~113 US symbols is a 20,000-arm search on per-symbol samples mostly under 40 observations.

CORRECTED – revision 1 was wrong as an implementation proposal. Hierarchical partial pooling is directionally right **eventually**, but it cannot be built on the taxonomy as it stands. Start with **predeclared class-level estimates**; add empirical-Bayes per-symbol deviations only after the classes and sample floors exist.

Required treatment:

1. Estimate one **global** effect per strategy.
2. Estimate per-instrument deviations and **shrink toward the global estimate** in proportion to each instrument's sample size (James-Stein / empirical Bayes).
3. Prefer **instrument-CLASS** grouping – but **the existing taxonomy is not adequate for this**. `lib/scoring/instrument-taxonomy.ts:36` identifies ETFs, metals, banks, REITs and broad operating companies; it has **no liquidity bucket**, no realised-volatility bucket, and no meaningful sector for most operating companies. Those buckets must be built and predeclared first.
4. Report the shrunken estimate as the headline; raw per-symbol only alongside its sample size and marked unstable.

8b. Per-symbol strategy SWITCHING over time – test the premise first

The owner's follow-up: "each symbol can move around from one strategy to another... symbols behave differently and follow a strategy certain times. Can we monitor that and switch strategy to trade based on this?"

The intuition is real

A name genuinely can trend for months, then chop for months. Momentum and mean-reversion regimes are a documented feature of equity returns, and a symbol's character does change with its business, its volatility and its ownership.

Why this is nonetheless the most dangerous request in this document

1. **It is the largest search space here by far.** Section 8's per-symbol slice is already ~20,000 arms. Adding "and it changes over time" multiplies that by the number of candidate switch points. Every additional degree of freedom makes a flattering history easier to find and a real effect harder to distinguish.
2. **Retrospective switch detection is trivially easy and almost always noise.** Given any price series and any two strategies, one can always find the split dates that make the combination look excellent. That procedure has no out-of-sample content.
3. **It brushes a locked project decision.** `CLAUDE.md` pushes back on explicit "bull/bear mode" switching as "fragile and adds moving parts". Review's refinement, accepted: that prohibition is **relevant but not dispositive** – per-symbol state adaptation is materially different from one global market regime switch. It earns a measure-only premise test; it does not earn a switcher now.

The premise test – CORRECTED

Revision 1 proposed a rank correlation of strategy affinity between adjacent periods. Review found that **wrong** as a primary test and currently

unanswerable**, and the author agrees on both counts:

- Rank persistence is at best a *diagnostic*. It does not test whether switching **makes money after costs**, which is the only question that matters.
- The data cannot support it. 26 distinct eligible-long h10 dates per market is ~**2.6 effective observations** under the system's own overlap heuristic.

Primary test instead – nested walk-forward policy:

1. In a training window, estimate the preferred strategy per symbol.
2. **Freeze the assignment.**
3. Trade it in the next window, out of sample.
4. Compare net-of-cost results against three honest baselines: the **champion**, the **best static strategy** for that symbol, and an **equal-weight ensemble**.

Transition persistence and top-two retention become secondary diagnostics, not the verdict.

Purge and embargo, specified: purge every observation whose realized outcome interval crosses the train/test boundary; embargo by the longest permitted holding/outcome horizon – currently **20 market sessions**.

```
> Do NOT reuse `walkForwardFolds`. lib/learning/dataset.ts:107` documents  
> itself as market-horizon purged but computes `purgeCutoffMs = testStart -  
> horizonDays * DAY` with `DAY = 86400_000` (:120`) – calendar milliseconds.  
> A nominal "10-day" purge is therefore about 6-7 trading sessions, so labels  
> leak across the boundary. This is a live defect in existing learning code, not  
> just a constraint on this feature, and is flagged separately for repair.
```

Timing: this test is **not answerable today**. It needs its own accrual period. Proposing a test that cannot report is its own failure mode, so it is scheduled last and gated on independent history existing.

If it ever does get built

- Switch triggers frozen before replay, expressed as observable state, never as dates.
- A minimum dwell time per assignment, so the system cannot thrash.
- Switching counted in the trial family: **k** strategies × **r** regimes is **k×r** arms, not **k**.
- Compared against the honest baselines – best single static strategy for that symbol, and the champion – not against the worst constituent.
- Transaction costs of switching charged in full; a switcher that changes stance often can lose to a static rule purely on turnover.

Recommendation: run the persistence test in stage 3, decide from its result, and build nothing before it reports.

9. Promotion policy

A candidate advances only when **all** hold:

1. survives frozen historical replay after realistic costs
2. persists across periods and reasonable parameter perturbations
3. adds portfolio-level value rather than duplicating the champion
4. passes the multiple-testing-adjusted gate at the **full** trial count
5. forward-shadow fills match modelled execution
6. produces enough independent forward observations
7. **owner explicitly approves paper activation**
8. a later, separately approved live trial starts under a small risk cap

A strategy designated as a **hedge** may carry modest or negative standalone return, but must reduce downside enough to justify its carrying cost. It may not be quietly graded against easier return-strategy criteria.

10. Blocking gaps found by review (C7)

Each must be closed or explicitly accepted before the stage that depends on it:

- **PIT universe excludes ETFs and refuses India.** **lib/edges/pit-universe.ts:259`.** Every fixed-ETF strategy therefore needs a **separate point-in-time instrument/data contract**.
- **price_cache` has no immutable provider/basis/version provenance**, so adjusted history can be restated after splits or dividends and a "sealed" replay silently changes. Sealed replay needs versioned price provenance.
- **No NAV path** from the simulator (section 2). Sharpe, Sortino, drawdown, benchmark alpha and stress-day correlation need a deterministic marking layer.
- **Same-session fill ordering is lexical**, which privileges one strategy when capital is scarce. Parallel sleeves need **reserved budgets or pro-rata arbitration**, frozen before any subset runs.
- **Trial-family counting can reset.** Verified in production:

```
`trial_family_id` = 'local-nse-technical-v1' has 2 experiments each recording
`trials_considered` = 1, and **13 of 18 rows carry no `trial_family_id` at
all**. An immutable, atomically-incremented trial ledger is required before any
multiple-testing claim means anything.
- **Retirement is mutable state** in the proposed shadow table. It needs an
  **append-only event history** with reason, evidence snapshot and actor.
- **Source adaptations are new specifications.** "Turnaround Tuesday, adapted to
  next-open" is a tested variant and **must increment the trial count**.
- **India stays blocked** until its PIT membership, corporate-action and
  benchmark contracts are independently complete.
- **The first family must test exits**, not only entries and exposure.
```

11. Sequencing – Stage 0R (review ruling: option (c))

1. **Stage 0R – reconcile the foundation.** Track the approved documents, map each ledger to its real purpose, and **revise, not deploy**, the shadow migration.
2. **Immutable trial-family ledger.** Atomically count every rule, parameter variant, adaptation, combination and rerun, across sessions.
3. **Prove the orchestration seam.** Compile ONE frozen rule into events, run the existing simulator plus a new deterministic NAV/benchmark marker, and persist a sealed result in `backtest\_experiments`. **Include a negative-control strategy** – if a deliberately worthless rule scores well, the seam is wrong.
4. **Prove data sufficiency.** Acquire and seal enough licensed OHLCV and corporate-action history **before** naming candidates.
5. **Metadata-only catalogue**, preferably after written crawling permission.
6. **Freeze the role-balanced family**, including exit/holding strategies.
7. **Historical replay + the full champion-relative combination matrix.**
8. **One forward-shadow combination per market.**
9. **Revisit switching** only after enough independent history exists.

Explicitly NOT to be built yet: the per-symbol switcher, the per-symbol empirical-Bayes model, any arbitrary combination optimiser, a production crawler, or the current shadow migration.

Approval sought for **steps 1-3 only**.

11b. Stage 0R steps 1-3: what shipped, and what the seam found

Completed 2026-09-01. Every defect below was invisible to unit tests and was found only by running on 1,280 real VOO bars – which is the entire argument for step 3 existing before any strategy is judged.

#	defect	fix
1	exits carried no `quantity`; `portfolio-simulator.ts:136` rejects those as `invalid_exit`. **1 fill, 96 rejections** compiler tracks	
2	`seamVerdict` returned `pass: true` on that run – it inspected only the controls, never the rejection rate fails above a 10% rejection	
3	`cashAllocation` pinned to INITIAL cash while real cash drifted; at 100% allocation one losing round trip made every later entry unaffor	
4	`alwaysInControl` used `positionSizePct: 0.1` regardless of universe size, so on one symbol it ran at **8.7% utilisation** and showed -	

Defect 4 was surfaced by the gate added for defect 2 – the tightened verdict immediately failed a run the lenient one had passed.

Final verified run: `pass: true`, **zero rejections** across all three specs (97/97 and 233/233 fills). Controls behave: never-trades returns exactly 0.00% with 0.00% drawdown; always-in reaches 91% utilisation and lands -18.39pp against the index it holds, which is the drag of exiting every 10 sessions and re-entering next open.

The rule itself: 200-session VOO trend returns **46.73% against buy-and-hold's +87.26%** over ~5 years – it underperforms by 40pp while cutting drawdown from 25.50% to 13.27%. Believable for a trend filter in a rising market. It is a full-sample replay with no costs, no walk-forward and no multiple-testing adjustment, so under the promotion policy it cannot advance past `replay only`.

Sealing is REFUSED, and that is the schema working

`backtest\_experiments` already enforces a full manifest for `historical\_replay` (`backtest\_experiments\_historical\_replay\_manifest\_required`), including **`validation\_mode` = 'purged_temporal_oos'**. This seam runs a full-sample replay with no OOS split, so writing that value would misstate how the number was produced. `POST` therefore returns `409 sealing\_requires\_purged\_oos` rather than forcing a full-sample result into a slot reserved for purged ones.

Two things this exposed for later:

- The contract also requires `edge\_id`, `formula\_version`, `universe\_policy\_version`, and a `validation\_spec` at schemaVersion `kairos.historical-replay.v1`. Sealing is a real piece of work, not a field map.
- `backtest\_experiments.trials\_considered` means *variants in this experiment* and is capped at `variant\_budget` (max 20). That is **not** the same quantity as the trial family's running total in `trial\_family\_ledger`, which is the multiple-testing denominator and grows without bound. The two must not be conflated when sealing is implemented.

The trial ledger earned its place immediately

Changing `alwaysInControl` from `positionSizePct 0.1` to `1 / universe.length` produced a NEW fingerprint and registered as **trial 4**, alongside the original at trial 3. A fix to a control is itself a new specification. Without the ledger that change would have been invisible and the denominator would have stayed at 3.

12. Honest assessment

Recommendation: go, at Stage 0R. The concept is worth pursuing; the architecture was not implementation-ready, and revision 1 got four things wrong that mattered:

1. Pointed Stage 0 at `strategy\_evaluations`, which belongs to a different subsystem – the real gap is `validation\_experiments`.
2. Treated the portfolio simulator as able to produce Sharpe/drawdown/alpha. It returns fills and realized P&L; there is no NAV path.
3. Proposed four combination portfolios that omitted `champion+A` and `champion+B`, the two most informative marginals.
4. Proposed a per-symbol shrinkage model on a taxonomy that has no liquidity, volatility or sector buckets to shrink toward.

A fifth correction is a live defect in existing code, not this feature: `walkForwardFolds` purges in calendar milliseconds while documenting itself as market-horizon purged, so a "10-day" purge is 6-7 sessions and labels leak.

Three cautions stand:

- **The foundation is not just unproven, it is unconnected.** The validation engine grades scoring weights; nothing in the repo can currently replay a price rule end to end.
- **Data may be the binding constraint before statistics are.** ~80 usable SPY decision sessions after warm-up, ~35 for TLT/IEF.
- **The risk is a good-looking result, not a bad strategy.** Five claims from this book were retracted in the past week, every one from reading a number drawn from the wrong population or an unrun path. A negative control in step 3 is cheap insurance against the seam itself being wrong.

13. Follow-up: Strategy Evidence Scorecard (separate architecture)

Review also proposed a **Strategy Evidence Scorecard** – a third tab on the Strategies page (`Fit Scores · Algo Library · Live Evidence`) showing, per strategy and per combination: benchmark-relative alpha, **payoff ratio**, profit factor, winning weeks (not streaks), evidence count and stage (replay / shadow / paper / live), and a governed status from `insufficient\_evidence | collecting | promising | working | degrading | failed | retired`.

Deliberately **not** a win-rate leaderboard, which would reward exactly the pathology this book already shows: many small wins and one large loss.

`StrategyGovernancePanel.tsx:33` already surfaces signal count, win rate, average return, Sharpe, drawdown and alpha, and `lib/analytics/performance-metrics.ts:105` already computes expectancy and profit factor. Missing: strategy-level weekly return history, payoff ratio per run, benchmark-relative weekly consistency, evidence-stage separation, governed status, and combination rows.

Status thresholds are financially load-bearing and **must be frozen** in architecture before any strategy is graded against them. This needs its own approved document; it is not covered by this one.

Source: features\factor-quantile-diagnostics\FEATURE_ARCHITECTURE.md
Factor quantile + stability diagnostics (alphalens method port)

> Status: **DRAFT** – awaiting approval. No code written.
> Created: 2026-09-02
> Scope: `lib/learning/dimension-diagnostics.ts` and the Learning-page panel.
> Measure-only – nothing here can reach a score, size, entry, exit or order.
> Source of method: [alphalens-reloaded](https://github.com/stefan-jansen/alphalens-reloaded)
> (Apache-2.0, 642★). Method is ported; no dependency is added.

1. Why, in one line

Rank IC answers **"does the ordering correlate with returns?"**. It cannot answer **"is the ordering monotonic, and is the top-minus-bottom spread worth trading?"** – and those are the questions that decide whether a dimension is real.

2. What we already have vs what alphalens computes

alphalens function	Kairos today	Gap	
--- --- ---			
`factor_information_coefficient`	per-session Spearman rank IC	-	
`mean_information_coefficient`	`mean_session_rank_ic`	-	
IC by date (series)	`session_ic_series` (2026-09-02)	-	

```
| mean_return_by_quantile | nothing | monotonicity unknown |
| compute_mean_returns_spread | nothing | no tradeable magnitude, no SE |
| factor_rank_autocorrelation | nothing | stability unknown |
| quantile_turnover | nothing | low priority – we do not trade a factor portfolio |
```

3. The three measures to add

3.1 `mean_return_by_quantile`

Per session, bucket the cross-section into quintiles by the dimension's score and take each bucket's mean benchmark-neutral forward return. Average per bucket across sessions.

Why it matters here. A small positive IC is consistent with two very different worlds: a clean monotonic gradient ($Q1 < Q2 < Q3 < Q4 < Q5$), or a flat middle with one extreme tail dragging the correlation. The first is tradeable; the second is an artifact. Sentiment's `+0.0886` and technical's `-0.1325` are currently indistinguishable between those cases.

3.2 `compute_mean_returns_spread` (top minus bottom, with standard error)

Q5 – Q1 per session, then mean and standard error across sessions.

Why it matters here. This is the only number in the whole diagnostic that is denominated in `returns` rather than correlation units – the answer to "how much is this dimension worth". Its standard error also yields significance directly, using the same `nEffective` overlap correction already applied to IC, without inventing a second statistical convention.

3.3 `factor_rank_autocorrelation`

Correlation between a dimension's cross-sectional ranks on consecutive sessions.

Why it matters here, specifically. It would have diagnosed `macro` on sight: a market-wide scalar has an undefined/degenerate rank autocorrelation because it has no cross-sectional variance at all – the same fact that makes its IC `0.0000` by construction. More usefully it separates two failure modes we cannot currently tell apart for `technical`: a stable signal that is genuinely inverted (autocorr high, IC negative – actionable, flip or drop it) versus a signal whose ranking is noise from day to day (autocorr near zero – a data problem, not an alpha problem).

4. Constraints this must respect

1. **Cohort.** Headline on eligible-long (`lib/learning/entry-cohort.ts`), all-scored as labelled context. Same rule as every other predictive finding.
2. **Floors.** `MIN_PREDICTIVE_DATES` 20 and `MIN_EFFECTIVE_OBSERVATIONS` 12 still gate any verdict. Quintiles need their own floor: a session contributes only if every bucket has a minimum membership (proposed: 3), because a quintile built from one name is not a quintile. Sessions below it are excluded and counted, never silently folded in.
3. **No migration.** These live in the existing `metrics` jsonb beside `session_ic_series`, immutable with their finding.
4. **No plan-version bump.** `mean_session_rank_ic` is unchanged; the fields are additive. Same reasoning as the 2026-09-02 series addition – and the same check: the existing numbers must come back byte-identical.

5. The caveat that must ship WITH the numbers

Quintiles on the eligible-long cohort are computed on a distribution **truncated** at the entry threshold – everything below `analyst_score >= 60` is already gone. So "Q1" there is not the factor's true bottom quintile; it is the bottom of the survivors. The top-minus-bottom spread on that cohort therefore understates a factor's full range and is **not** comparable to a published alphas tear sheet.

Both cohorts get computed for exactly this reason, and the panel must label which is which – the same collider problem measured on 2026-09-02, where component ICs roughly halved between the eligible and all-scored cohorts.

6. Deliberately NOT in scope

- `quantile_turnover`, `factor_weights`, `factor_returns`, `factor_alpha_beta`, `factor_cumulative_positions`: these describe a portfolio built FROM a factor. Kairos does not trade single-dimension portfolios; the composite plus the entry gate decides. Building them would measure a strategy that does not exist.
- Any alphas dependency. It is Python; the money path is TypeScript on serverless. The method ports; the library does not.
- Any change to weights, thresholds or eligibility. Measure-only.

7. Expected outcome, stated in advance

Most dimensions will read `insufficient_evidence` – 29 qualifying sessions at h5 is `nEffective` 5.8 against a floor of 12, and adding measures does not add evidence. Predeclaring that matters: the value here is that a **future** verdict is better-founded, not that today's numbers change. If a monotonic quantile gradient appears alongside a sub-floor IC, that is still not permission to act.

8. Size

One module change (`dimension-diagnostics.ts`), one panel section, tests. No migration, no new dependency, no new route, no schedule change.

```
# Source: features\feature-pack-validation\FEATURE_ARCHITECTURE.md
# Feature-Pack Validation Architecture

> Status: P0 built; P1 technical measurement already collecting; feature-registry
> lifecycle hardened to measure-only. P2-P5 remain design-gated.
>
> Date: 2026-08-02
> Owner decision required before P1 or any feature can influence a decision.

> P0 result (2026-08-02): `lib/feature-packs/catalog.ts` is the typed
> applicability/status read model used by Research Journal and Strategy Library.
> It has no data fetcher and no score, signal, paper, live, exit, sizing, broker
> or feature-registry writer. Strategy templates are explicitly reference/manual
> tools until a later bounded shadow compiler accepts their supported rules.
>
> Governance correction (2026-08-02): the legacy registry had an automatic
> `quarantined -> active` transition after a small IC screen. `active` was only
> an observational logger in current code, but its name contradicted the
> architecture and invited misuse. The lifecycle is now
> `proposed -> quarantined -> measure_only -> retired`; legacy rows migrate to
> `measure_only`. This changes no scoring or trade behavior.
>
> Money-path regression corrected (2026-08-10): commit `82c932a3` collected the
> planned fields but also added them directly to `deterministic_v1`. EMA-200,
> MACD, ADX, relative strength, FCF yield, debt/equity, gross margin, PEG and
> 52-week proximity are again evidence/measurement only. Regression tests assert
> that populating these fields cannot change the canonical score.
```

1. Decision

Kairos will evolve from one generic five-dimension score into a small, governed library of **market-local, instrument-appropriate feature packs**. A feature may move from `catalogued` to `measure\_only`, `shadow`, `paper\_challenger`, and only then to an owner-promoted champion. No popular indicator becomes an active score input merely because a provider exposes it or a historical chart looks good.

This is not per-ticker parameter fitting. Evidence is evaluated at the smallest predeclared cohort with enough independent observations, for example US liquid semiconductors, India large-cap financials, or US broad ETFs. A single stock is used for attribution and review, never as sufficient evidence to learn its own MACD period, stop, or fundamental formula.

2. Why

The user needs a system that can answer all of these honestly:

1. Which facts and indicators did this decision use?
2. Which ones were unavailable, stale, or structurally inapplicable?
3. Which feature families work for this market, instrument type, and horizon after realistic costs?
4. When a candidate feature loses, did it fail statistically, economically, or simply lack enough evidence?

The app already has useful pieces, but they are not yet one governed program:

```
| Existing component | Truth today | Role in this design |
|---|---|---|
| `scoreFundamentals` / `scoreTechnical` | Current deterministic v1 score | Baseline champion, unchanged until a later promotion |
| `fundamental_facts` | Append-only captured fundamental vintages | Fundamental provenance source; not a full historical backfill |
| `edge_signals` / `edge_signal_inputs` | Immutable scalar factor observations | Canonical measure-only feature evidence |
| Technical Factor Calibration | Measures v1 technical composite, relative strength, MACD/ATR and signed ADX | Initial technical trial family |
| Historical replay / walk-forward work | Offline, sealed, point-in-time validation substrate | Required validation path, not a live predictor |
| `strategy_versions`, `shadow_decisions` | Existing lifecycle and shadow comparison substrate | Only promotion/shadow lifecycle; do not create
```

3. Non-negotiable boundaries

- US and India data, currencies, labels, benchmarks, feature values, trial counts, scorecards, and promotions never mix.
- An ETF, leveraged ETF, operating company, ADR, bank, and REIT do not inherit a company-fundamental pack by default.
- Every feature has a deterministic formula, version, units, applicable instrument kinds, source/provenance rule, freshness rule, missing-data rule, and corporate-action policy.
- Feature families are de-correlated by design. RSI, MACD, EMA state, ADX and Bollinger are not five independent votes merely because they all describe the price path.
- The LLM may propose a hypothesis and explain results. It cannot select a winner, change a threshold, choose a per-symbol parameter, or write a

tradeable feature value.

- A failed fetch, unverified timestamp, stale fundamental, thin cohort, or unclassified instrument produces `unavailable` / `inapplicable` / `abstain`. It never silently becomes a neutral or favorable feature.
- This program cannot bypass the existing app pause, market controls, drawdown breaker, entry quality gates, paper/live approval requirements, or broker controls. Until a version is explicitly promoted, it has no money-path reader.

4. The feature packs

Feature **families** are the unit of selection. A pack may contain one predeclared representative from a family, not an indicator zoo.

4.1 Shared price and liquidity pack

Applicable to listed equities, ADRs and eligible ETFs when completed daily candles are available.

Family	v1/current fact	Candidate additions	Rule
Trend/momentum	RSI(14), EMA20/50 state, 20d trend	60d relative strength; EMA200 state	Test one state/return representation at a time
Volatility/risk	ATR14, latest return/range, breakdown veto	realized/downside volatility, gap frequency	Risk/veto context is not autom
Participation	volume vs 20d average	dollar-volume and spread proxy	Current-bar volume never appears in its own baseline
Trend-strength	none in v1	signed ADX14	P0 technical challenger
Momentum confirmation	none in v1	MACD histogram / ATR	P0 technical challenger

Fibonacci, Elliott Wave, arcs/fans, and discretionary chart patterns are not candidate score inputs. They may be visual/context aids, but they have no deterministic and reproducible feature contract in this program.

4.2 Company quality, value, and growth pack

Applicable only to operating companies and reviewed ADRs with a compatible reported-fundamental basis. All reported values must use the exact `fundamental\_facts` vintage known at the decision timestamp.

Family	Current v1 input	Candidate addition	Initial classification
Relative valuation	sector-relative P/E	P/B; FCF yield; PEG only with dated consensus	measure-only
Profitability	profit margin; ROE; EPS sign	gross profitability / assets; margin trend	measure-only
Balance sheet	none	debt/equity and liquidity/leverage context	measure-only
Growth	quarterly revenue growth YoY	earnings growth and revenue/margin acceleration	measure-only
Earnings event	calendar/repricing safety barrier	actual-vs-consensus surprise, revisions, post-event age	measure-only after PIT source

Analyst target price remains display-only. It is not an alpha feature and cannot substitute for dated analyst estimate history.

4.3 Instrument-specific policies

Instrument class	Eligible packs	Explicit exclusions / special requirement
US / India operating company	shared price/liquidity + company pack + local benchmark/sector context	Company values must be market-local
Reviewed US ADR	shared price/liquidity + company pack on the ADS-compatible source	No US Form 4 assumption; foreign-underlying substitution
Broad or sector ETF	shared price/liquidity + ETF category/benchmark context	Company P/E, margin, ROE, EPS, debt/equity and insider packs
Leveraged ETF	shared price/liquidity with explicit underlying-trend, realized-volatility, gap and holding-duration fields	No company pack
Bank / financial	price/liquidity initially	A bank-specific pack needs capital, asset-quality and net-interest-margin data; do not misuse
REIT	price/liquidity initially	A REIT pack needs FFO/AFFO, occupancy, leverage and rate sensitivity; do not treat EPS/P/E as sufficient

When classification is missing or too coarse, select the conservative generic price/liquidity shadow pack only. It cannot authorize a new specialized score.

5. Canonical data contract

This feature must extend existing truth layers, never create a parallel feature database.

flowchart LR

A["Provider / local evidence"] --> B["PIT facts and completed candles"]

B --> C["Versioned deterministic feature definitions"]

C --> D["Edge evidence ledger"]

C --> E["Exact decision snapshot"]

D --> F["Sealed historical replay + walk-forward evaluation"]

F --> G["Challenger lifecycle"]

G --> H["Owner-gated paper champion"]

H --> I["Existing paper/live safety gates"]

5.1 Reused stores

- ``fundamental_facts`` is the only reported-fundamental vintage ledger. A feature snapshot retains the fact id, report period, filing/known-at date, restatement sequence and provider.
- Completed OHLCV is the only source for price features. Formula inputs retain candle date, adjustment basis, provider, input fingerprint and as-of cursor.
- ``edge_signals`` stores one scalar value per feature/version/symbol/date/market; ``edge_signal_inputs`` records the input fingerprint and provenance.
- ``decision_observations.features`` is the exact feature/policy snapshot that produced a research decision. ``research_packets.raw_data._scores.evidence`` is the detailed UI/audit payload. Neither is overwritten.
- ``strategy_versions``, validation experiments and ``shadow_decisions`` remain the only lifecycle/evaluation records for a challenger. Do not create a second "feature winner" or activation table.

5.2 Feature definition requirements

The existing ``feature_registry`` is extended only after a schema audit. Each versioned definition must declare:

```
type FeatureDefinition = {  
id: string; // immutable, includes formula version  
family: "trend" | "volatility" | "participation" | "quality" | "value" | "growth" | "event";  
markets: Array<"us" | "india">;  
instrumentKinds: string[];  
horizonDays: number[];  
formula: string; // whitelisted deterministic grammar, not code from an LLM  
inputs: string[];  
sourcePolicy: string;  
freshnessPolicy: string;  
knownAtPolicy: string;  
adjustmentPolicy: string;  
missingPolicy: "unavailable" | "inapplicable" | "abstain";  
trialFamilyId: string;  
status: "catalogued" | "measure_only" | "shadow" | "retired";  
};
```

Changing formula, period, normalization, universe, missing-data treatment, source policy, or applicable instrument set creates a new id and increments the trial family. Existing evidence is immutable.

6. Evidence method

6.1 Unit of learning

The primary unit is:

market x instrument pack x comparable cohort x feature version x horizon

Examples: ``US x operating_company x liquid semiconductors x macd_atr_12_26_9 x 10 sessions``, or ``India x operating_company x NIFTY IT x relative_strength_60d x 5 sessions``.

Individual tickers receive an attribution view, not bespoke tuning. A ticker may be marked ``insufficient_symbol_history``; that is an honest state, not an invitation to optimize around a handful of its outcomes.

6.2 Predeclared trial families

Before the first run, the system freezes the candidate list, parameter choices, market, packs, horizons, cohort definitions and evaluation plan. It counts all members, including losers and retired candidates, in the multiple-testing correction. The initial family is intentionally small:

1. Current technical composite v1;
2. 60-day benchmark-relative strength;

3. MACD(12,26,9) histogram / ATR14;
4. signed ADX14;
5. one PIT quality/value composite after fundamental source qualification.

No grid search across RSI/EMA/MACD parameters, sector-specific tuning, or post-hoc "best combination" is allowed in P0-P2.

6.3 Required validation

All claims are market-local and use the sealed replay/historical intake path.

- Purged, embargoed, non-overlapping walk-forward folds; all transforms fit on training folds only.
- Rank IC and Newey-West uncertainty, plus long-only top-bucket hit rate and top-minus-median spread.
- Net-of-spread, slippage, fees and applicable tax assumptions; turnover is a first-class failure condition.
- Stability by independent time block, comparable cohort and benign/adverse market regime. A good aggregate that fails every stressed period is not ready.
- Paired champion-versus-challenger results on the same opportunities.
- Multiple-testing adjustment across the frozen trial family. Any existing approximate DSR implementation must be repaired before it becomes a promotion argument.

Historical replay establishes eligibility only. A feature then collects forward shadow outcomes before an owner may promote it to a paper champion. Live remains separately gated by the existing validation, risk, broker and approval controls.

7. Research and UI behavior

Every Research Journal record must identify:

- active pack and scoring version;
- facts actually used, their source, `known\_at`, freshness and contribution;
- applicable but missing/degraded facts and the exact abstain/renormalization treatment;
- observed-only facts, clearly labeled as not used;
- active feature-family overlap warnings, so multiple price transforms do not read as independent confirmation.

Score Tracker continues to compare immutable research points. The Upgrade Path gets one row per feature trial family and market: collection coverage, completed folds, shadow outcomes, provider calls, status, rejection reason, next gate and an honest ETA. The header market switch is authoritative; India never shows a US-only conclusion as an India result.

8. Build order

P0 - catalog and audit, no new feature influence

1. Reconcile `feature\_registry`, Edge registry, `fundamental\_facts`, instrument classification and existing documentation into one typed catalog.
2. Mark every current Strategy Library condition as `supported`, `shadow\_only`, `manual\_only`, or `unsupported`; reject unsupported conditions from automated strategy shadows rather than approximating them.
3. Add data-contract tests: units, dates, source, freshness, corporate-action basis, market/instrument applicability and missing-data behavior.
4. Surface active/observed/unavailable facts in Research Journal and Score Tracker using stored evidence only. No additional provider calls from the UI.

P1 - existing technical candidates

1. Finish the approved Technical Factor Calibration trial family unchanged.
2. Use the existing Edge ledger to collect v1, relative strength, MACD/ATR and signed ADX on completed candles with no extra provider request per feature.
3. Run US and India separately; publish only measure-only results.

P2 - reported fundamentals qualification

1. Qualify US and India reported facts source-by-source, including a filing or conservative known-at date, units, restatements, sector mapping and event freshness.
2. Extend the existing PIT ledger and snapshot references; do not backfill current provider snapshots as if historically known.
3. Start only four candidates: gross profitability, debt/equity context, FCF yield, and revenue/margin acceleration. P/B, PEG, surprise and revisions wait for a provider with their required dated inputs.

P3 - sealed replay and challenger creation

1. Run predeclared pack/candidate trials through the existing historical replay and walk-forward architecture.
2. Compile only passed candidates into a market-local `strategy\_versions` challenger with source/trial lineage.
3. Compare it in non-executing shadow decisions against the current champion.

P4 - paper promotion, separately approved

An owner can promote one validated, market-local challenger to `paper\_active` only after historical gates and a sufficient forward shadow sample pass. It replaces no live version automatically. PaperTrader then still applies all existing admission, position, risk, cash and exit gates.

P5 - specialist packs, separately proposed

Bank, REIT and leveraged-ETF packs require their own data qualification and risk-control review. They are not extensions of the generic equity pack.

9. Acceptance criteria

- The same immutable input snapshot and definition version produces the same feature value and reason codes.
- No feature family produces a provider call per UI view or per duplicate indicator in a research run.
- A company feature cannot apply to an ETF; US macro cannot count as India evidence; a missing instrument type cannot inherit a specialist pack.
- Every historical fundamental read enforces `known\_at <= as\_of`; a violation throws and invalidates the replay.
- New feature rows cannot change `analyst\_score`, direction, eligibility, position sizing, paper positions, live proposals, exits or broker orders in P0-P3.
- Trial count includes failed/retired variants and rejects arbitrary new combinations after evidence collection begins.
- UI shows missing and observed-only facts distinctly from scored facts.
- A failed candidate remains visible with formula/version, sample limits and rejection reason; it cannot be silently removed from the trial record.
- Tests prove US/India isolation, instrument applicability, feature-version immutability, no look-ahead, fold-only fitting and no money-path reader.

10. Out of scope

- LLM-selected scores, rules, or parameter tuning.
- A universal per-stock model or a claim that the app can learn a stock's behavior from a few trades.
- Adding every named technical indicator.
- Intraday alpha, broker-side stops, TradingView scraping, or options direction scoring.
- Automatic paper or live promotion.

11. Authority and related documents

This is the governing integration design for feature selection. It does not replace the implementation-level specifications below:

- `features/technical-factor-calibration/FEATURE\_ARCHITECTURE.md` owns the initial MACD/ADX measurement family.
- `features/pit-fundamentals/FEATURE\_ARCHITECTURE.md` owns reported-fact vintage/known-at semantics.
- `features/historical-replay-harness/FEATURE\_ARCHITECTURE.md` and `features/walk-forward-ic-folds/FEATURE\_ARCHITECTURE.md` own replay and OOS validity.
- `features/strategy-portfolio-lab/FEATURE\_ARCHITECTURE.md` owns the bounded template-shadow UI/lifecycle.
- `docs/arch/03-agents.md`, `04-database-schema.md` and `09-learning-loop.md` remain the running-system source of truth and are updated only when a phase is actually implemented and verified.

Source: features\feature-pack-validation\IMPLEMENTATION_RESULT.md
Feature-Pack Validation: P0 Result

> Shipped: 2026-08-02
> Commit: see the matching `feat: add feature pack catalog` history entry.

Delivered

- Added `lib/feature-packs/catalog.ts`, a typed read model of current feature lifecycle and instrument applicability. It records active v1 facts, measure-only candidates, observed-only facts and unsupported indicators.
- Research Journal now labels the record's instrument family and separates current v1 facts, measured-only candidates and inapplicable facts. It reads stored decision evidence only and makes no provider call.
- Strategy Library now says plainly that templates are reference/manual tools. Each card distinguishes manual Scanner conditions, shadow-only conditions and Scanner-unsupported conditions. Its buttons are labeled manual scan/backtest.
- Added regression tests for ETF fundamental exclusion, leveraged-ETF classification precedence, and explicit template-condition support states.

Not delivered intentionally

- No MACD, ADX, fundamental candidate, strategy template, score, threshold,

paper trade, live proposal, position, exit, sizing, broker or Router behavior changed.

- No database migration or `feature\_registry` lifecycle mutation was made.
- P1 technical measurement, P2 PIT fundamental qualification, P3 replay/shadow, P4 paper champion, and P5 specialist packs remain governed by the architecture and their existing feature specifications.

Follow-up governance hardening (2026-08-02)

- Replaced the legacy automatic `active` status with `measure\_only`. The database migration renames legacy rows and the weekly evaluator can no longer create a score-eligible lifecycle state.
- Research evidence now writes `measured\_feature\_values`, an explicit observation-only payload. No current reader uses it for score, eligibility, sizing, paper/live proposals, exits, or broker execution.
- The existing EdgeScout/EdgeIC technical trial family remains the P1 source of truth. P2 needs source-qualified market-local reported facts, P3 needs the sealed walk-forward gate, P4 remains disabled, and P5 needs specialist data contracts; none are represented as completed functionality.
- Dashboard -> Upgrade Path now registers all of these gates separately for the selected US or India market: technical calibration, PIT fundamental qualification, specialist packs, and challenger validation. The page reports actual ledger coverage where one exists and an explicit idle state where no evidence collection has been approved.

Verification

- `npx vitest run tests/feature-pack-catalog.test.ts tests/research-journal-controls.test.ts tests/scoring-evidence-contract.test.ts`
- `npx tsc --noEmit`
- `npm run build`

Independent review correction – 2026-08-02

Adversarial review of `17876da4..a51ae122`. Findings and the fixes applied.

****HIGH – the Research funnel reported live inputs as inapplicable.****
`instrumentFamily()` in `lib/feature-packs/catalog.ts` matched only the `InstrumentKind` vocabulary (`us\_equity`, `india\_equity`, ...). Journal rows supply `identity.asset\_type` from `JournalAssetType` (`company`, `india\_company`), and `identity.instrument\_kind` is `null` for any decision written before instrument capture existed. Both inputs therefore resolved to `unknown`, and `featureAuditForInstrument()` returned every `active\_v1` feature as `\*inapplicable\*`. The funnel then told the owner that RSI(14), EMA20/50, 20-day trend, volume, ATR, sector-relative P/E, profit margin, ROE, EPS sign and revenue growth did not apply to a decision those inputs had in fact scored.

Production scope at the time of review, `decision\_observations` over 30 days: US 1,744 rows with 726 carrying an instrument kind, India 410 with 197. About 58% of recent decisions rendered the false panel.

Fixed by teaching `instrumentFamily()` the journal vocabulary, and by making `featureAuditForInstrument()` return empty lists for an `unknown` family – an unclassified instrument is unknown, not proven incompatible. The funnel now states plainly that the decision stored no classification and points at the quant audit instead.

****MEDIUM – three fundamental conditions were labelled shadow-gated.****
`FUNDAMENTAL\_CONDITION\_STATE` marked `fcf\_yield\_min`, `debt\_equity\_max` and `gross\_margin\_min` as `measure\_only`, which the Strategy Library rendered as "Shadow evidence required before use". All three ship inside `AlgoStrategy.scan\_filters` and are sent to the Financial Datasets screener by `app/api/agents/research/scan/route.ts`, so the manual Scanner does evaluate them. Corrected to `manual\_only`. `macd\_cross\_up` remains shadow-gated and `price\_above\_ma200` / `volume\_surge` remain unsupported – the scan route reads only `rsi\_min`, `rsi\_max` and `price\_above\_ma50` from `conditions.technical`.

****LOW – a provider error was recorded as a narrative timeout.****
The 10s advisory-thesis race in `lib/research-agent.ts` returned the same `narrative-timeout` fallback for a rejected `callLLM` as for an elapsed timer, so an outage read as latency in the LLM ledger. The catch path now labels `narrative-error` with `durationMs: 0`.

****LOW – stale vault paths in user-visible strings.**** `ScannerPage`, `PortfolioRiskPage` and the `execute-order` no-keys error still said "Admin → Vault". Updated to "Settings → System → API Vault".

Verified clean

- ****Money path.**** `trader`, `kite/order`, `lib/kite.ts` and `execute-order.ts` changes are wording only; every gate predicate is byte-identical. Broker routing still resolves through `active\_broker\_us` / `active\_broker\_india`.
- ****Removing the legacy global broker picker changes no configuration.****
`PATCH /api/settings/risk-profile` applies `broker` only under

- ``if (broker !== undefined)``, so omitting it leaves the stored value untouched.
- **Settings labels match reality.** ``lib/trading-mandate.ts`` never reads ``strategy_config``, so the claim that entry threshold, stop, target and horizon are market-local is accurate. Stop/target/threshold state still round-trips into ``strategy_config`` on save, but nothing on the money path reads those columns.
- **Deep links.** ``?tab=profile|preferences|access`` alias to ``account`` and ``?tab=agents`` to ``trading``; ``/dashboard/admin`` still redirects to ``?tab=admin``. The four MCP OAuth callbacks still emit ``?tab=agents`` and land correctly through the alias.
- **Admin surface.** Removing the duplicate sidebar item removed no access: the ``admin`` tab was already in the tab list, and ``/api/admin`` gates on ``role in (admin, superadmin)`` server-side.
- **Feature-registry governance.** Production ``feature_registry_status_check`` is ``('proposed','quarantined','measure_only','retired')``; ``active`` is gone and no code writes it. ``nextFeatureRegistryStatus()`` has no edge that reaches a score-eligible state. Nothing reads ``measured_feature_values``.
- **Direction stays deterministic** under the thesis timeout. ``resolveSignalDirection()`` uses ``llmDirection`` only to compose a note; an empty narrative cannot change eligibility. The losing race branch carries a ``.catch``, so no unhandled rejection and no leaked timer.
- **Screener aliases cover the real filters.** ``pe_ratio`` and ``profit_margin`` are the only legacy field names present in ``scan_filters``, and both are in ``LEGACY_FIELD_ALIASES``. Symbol extraction still requires a non-empty ``ticker`/`symbol``, so no arbitrary symbol can enter research.
- **RSI alias.** ``computeTechnicals()`` emits ``rsi14``; ``ScoreTrackerPanel`` read ``evidence.rsi``, which never existed. The ``rsi14 ?? rsi`` fix is correct.

Not verifiable locally

The Financial Datasets ``POST /financials/search/screener`` request and response shape could not be exercised: the account balance is \$0.00. The ``search_results`` key and the ``{filters, limit}`` body are taken from the provider's published contract, not from an observed response.

Gates

```
`npx tsc --noEmit` clean · `npx vitest run` 1506 passed / 7 skipped ·
`npm run build` clean · `git diff --check` clean.
```

Source: features\forecast-calibration\FEATURE_ARCHITECTURE.md
Forecast Calibration and Earnings-Event Assurance

Status: Approved by owner direction and implemented in phases on 2026-07-30.

Problem

Kairos already stores each deterministic score, the indicative stop/objective plan, and immutable 2/5/10/20-session realized paths. Those records are not joined into an explicit plan-calibration view or exposed as a bounded LearnerAgent tool. Separately, earnings/options risk is measured only after an entry reaches PaperTrader or TraderAgent, so an existing holding can lack a fresh pre-event volatility observation.

The MSFT July 2026 incident exposed a related freshness defect: a pre-event daily bar was allowed to produce a post-event direction.

Semantic Boundary

The indicative ``profit_objective`` is an exit-policy level, **not** a predicted terminal price. Retrospective language must therefore report:

- decision/reference price;
- stop floor and profit objective;
- planned horizon;
- realized horizon close;
- maximum favorable and adverse excursion;
- whether the path reached the objective or breached the stop; and
- the fraction of the objective reached.

It must not call the objective a forecast mean or describe a one-stock miss as evidence that a strategy parameter is wrong.

Architecture

flowchart LR

D["Immutable decision_observations
score + trade plan"] --> C["Deterministic plan calibration"]

L["Immutable observation_labels
return + MFE + MAE"] --> C

C --> U["Decision Review
per-symbol illustrative path"]

C --> A["Market/horizon cohorts
N and readiness"]

A --> LA["LearnerAgent read-only tool
hypothesis prose only"]

A --> RR["Existing MAE/MFE policy
N >= 60, deterministic"]

E["PIT earnings calendar"] --> G["Post-event repricing barrier"]

G --> S["Current neutral signal
until reaction bar exists"]

H["US paper/live holdings"] --> O["Bounded earnings/options shadow"]

E --> O

O --> R["Existing append-only
earnings_risk_observations"]

Deterministic Calibration

The reusable calculator consumes one original trade plan and one matured label. It refuses malformed, non-candidate, stale, or currency-incoherent plans.

For the label whose horizon exactly matches `horizon\_sessions`:

- `objective\_reached` = MFE >= target_pct`;
- `stop\_breached` = MAE <= -stop_loss_pct`;
- `objective\_reach\_ratio` = MFE / target_pct`;
- terminal return, benchmark-neutral return, MFE, and MAE remain distinct.

MAE and MFE alone do not reveal event order. If both levels were touched, the calculator reports both and never claims which exit would have happened first.

Per-symbol output is always illustrative. Cohorts become reviewable at 20 matured paths, while risk-parameter admission keeps the existing stricter floor of 60 same-market, same-horizon eligible-long labels.

LearnerAgent Boundary

The LLM may:

- read deterministic aggregate calibration;
- explain likely failure modes;
- write a hypothesis for future validation; and
- point to insufficient evidence.

The LLM may not:

- calculate labels or overwrite realized outcomes;
- change stop, target, horizon, score, or weights from one event;
- treat personal trade history as market-wide evidence; or
- bypass the existing challenger/promotion gates.

The existing deterministic MAE/MFE policy remains the only automatic stop/objective adjustment path and requires at least 60 matching labels.

Options Monitoring

Options remain US earnings-risk evidence, never a sixth directional score. Cboe's own explanation is explicit that earnings options primarily estimate move magnitude rather than direction.

The scheduled shadow monitors only:

- open US paper positions; and
- symbols in the latest complete US live holding-risk snapshots.

It deduplicates symbols, caps work per run, and discovers events from the persisted cache plus one bounded Robinhood calendar call. That batch fallback closes a cold-cache gap without creating N per-symbol requests. It requests an option chain only when the event falls inside that position's horizon. Otherwise-eligible entries continue to be measured at the existing PaperTrader/TraderAgent choke points. Rejected/watchlist symbols do not consume option calls merely to inflate a sample.

Every observation is append-only, idempotent per market session, and `behavior\_changed=false`. It cannot score, size, enter, exit, or mutate a position.

Earnings Repricing Rules

- Before-market-open result: the report-date daily bar can contain the reaction.
- After-close result: a daily bar strictly after the report date is required.

- Unknown session: use the conservative after-close rule.
- A known event that is already in the past activates the barrier even when the actual-result feed is late.
- A current neutral signal is written so consumers cannot fall back to an older directional signal.
- Mechanical stop, target, trailing-stop, and time exits remain independent.

Acceptance Criteria

1. MSFT-style AMC input plus only the report-date bar produces `neutral`.
2. BMO input plus a completed report-date bar releases the barrier.
3. A late actual feed cannot reopen stale scoring after a known past event.
4. Decision Review shows plan versus realized path without calling the objective a terminal-price prediction.
5. LearnerAgent receives only aggregate deterministic calibration and cannot mutate risk parameters from it.
6. The holding monitor writes only shadow observations and never calls options for India or for an event outside the position horizon.
7. No US/India data is aggregated together.

Sources

- Cboe, "What Options Data May Indicate About Mag 7 Earnings":
<https://www.cboe.com/insights/posts/what-options-data-may-indicate-about-mag-7-earnings/>
- Rompolis, "Pricing event risk: evidence from concave implied volatility curves," Review of Finance (2025):
<https://doi.org/10.1093/rof/rfaf016>
- Novy-Marx, "Backtesting Strategies Based on Multiple Signals," NBER Working Paper 21329:
<https://doi.org/10.3386/w21329>

Source: features\health-control-and-paper-pnl\FEATURE_ARCHITECTURE.md
Health Control And Paper P&L
Status: APPROVED AND IN BUILD (owner request, 2026-07-20)

Problem

1. A kill-switch trip latches `market\_controls.trading\_enabled=false`, but the Settings market toggle only changes `strategy\_config`. The user therefore cannot inspect or safely reset the actual latch from the UI.
2. `paper\_trades.realized\_pnl` is correctly null while a trade is open, but the Trades tables render only that terminal value. The current mark-to-market P&L already available on `paper\_positions` is not shown beside the open lot.
3. Live holdings already show current P&L. Live orders and research signals do not share one canonical, cross-broker execution ledger and must not be presented as if uploaded history were broker-confirmed live trades.

Design

Guarded reset

- Add an owner-only Settings API that returns each market latch and reset reason.
- A reset names the market and the book that tripped (`paper` or `live`).
- The server reruns `checkKillSwitches` for that exact book before enabling the market latch. An unsafe, stale, or missing live baseline refuses the reset.
- A successful reset clears the matching health issue and writes an immutable `decision\_journal` entry. It never changes thresholds, global pause, security lock, broker credentials, account selection, or live-autonomy flags.
- Future trip reasons and issue keys include the book. The market latch remains intentionally conservative and shared: a trip in either book blocks new exposure for that market until its originating book passes a guarded reset.

Current paper P&L

- Keep `Realized P&L` terminal and immutable.
- Add `Current P&L` beside it for open BUY lots only.
- Compute deterministically from persisted values already fetched:
 $(\text{paper_positions.current_price} - \text{paper_trades.fill_price}) * \text{paper_trades.qty}$
- Join strictly by normalized market + symbol. Multiple open lots use the same current mark but retain their own fill price and remaining lot quantity.
- Missing/non-finite marks, unmatched lots, SELL rows, and closed trades render unavailable; they never substitute fill price or zero.
- Expose mark timestamp in the cell tooltip. No provider call is added.

Live parity boundary

- Live Portfolio remains the source for broker holdings and unrealized P&L.
- Research signals remain market-level evidence, not account-level executions.
- A future live Trades tab may read only reconciled `broker\_orders`, scoped to market + active account + live environment. Kite/Webull/Robinhood parity must be proven before calling it unified. Uploaded CSV decisions remain analysis, not the live order ledger.

Acceptance criteria

- Settings shows the real per-market latch and its reason.
- Reset cannot succeed unless the originating book currently passes all brakes.
- Successful reset is audited and resolves only matching kill-switch alerts.
- Open paper trade rows show current amount and percent at the persisted mark; closed rows show only realized P&L.
- US and India are never joined or summed; currency follows the selected market.
- No external API call, LLM call, broker call, order, threshold, or autonomy flag is introduced by the P&L display.

Source: features\historical-evidence-intake\FEATURE_ARCHITECTURE.md
Governed Historical Evidence Intake

****Status:**** APPROVED for implementation by owner direction on 2026-07-29
****Scope:**** offline acquisition, validation, normalization, and historical replay only
****Money-path status:**** unreachable; no scorer, trader, monitor, cron, or broker reads this store

1. Decision

Kairos will maintain a local, immutable historical-evidence store for deterministic backtests. It will acquire official bulk data where available, pin any community artifact to an exact commit, validate and hash every file, quarantine failures, and feed normalized records only to the existing sealed replay layer.

This is not a universal market-data lake. The useful first scope is:

1. India daily equity prices and point-in-time traded universe from NSE bhavcopy.
2. India historical corporate actions from NSE, required before adjusted-return use.
3. US as-filed fundamentals from SEC quarterly Financial Statement Data Sets.
4. US macro vintages from FRED/ALFRED for as-known regime replay.
5. A diagnostic-only S&P 500 historical-membership fixture for cross-checks.

No free GitHub repository found in the intake review replaces a survivor-bias-free US security master plus adjusted prices and delisting returns. Kairos must continue using its existing Massive point-in-time membership/price entitlement for recent US experiments or purchase licensed data for older promotion-grade US tests.

2. Why Local, Not Supabase

The raw SEC archive is gigabytes and India daily files are hundreds of megabytes. Uploading those rows to Supabase would waste the free database/storage budget and make local reproducibility depend on network access.

Raw and normalized files live under:

%KAIROS_EVIDENCE_DIR%

raw/<source>/<version>/...

normalized/<dataset>/<version>/...

manifests/<dataset-id>.json

quarantine/<dataset-id>/...

catalog.json

Default: %USERPROFILE%\kairos\evidence`. The store is outside the repository and outside OneDrive. Only small schemas, acquisition code, tests, and example manifests are committed.

Supabase receives only the existing immutable experiment identity, dataset fingerprint, result summary, and (when explicitly materialized) sealed replay packet metadata. Raw bulk rows never enter a browser or Vercel runtime.

The executable boundary is defined in

features/local-historical-replay/FEATURE_ARCHITECTURE.md: a local, network-free worker verifies these manifests, predeclares one immutable experiment, computes the diagnostic, and uploads only compact fingerprints/results. The application does not read this directory and the raw store is not uploaded anywhere.

3. Source Decisions

Source	Capability	Authority	Intake decision	Evidence class
SEC Financial Statement Data Sets	quarterly ZIPs of primary statements, as filed, 2009+	SEC official	adopt; prefer over bulk	company
SEC companyfacts.zip	all XBRL company facts, nightly	SEC official	catalog only; do not download by default (1.29 GB compressed and r	
NSE daily bhavcopy	all daily CM rows, OHLC/volume/trade fields	NSE official	adopt	promotion-capable only with actions and coverage
NSE corporate-actions archive	splits, bonuses, dividends and effective dates	NSE official	adopt; adjusted returns refuse without it	
FRED/ALFRED	observation vintages and real-time periods	Federal Reserve Bank of St. Louis	adopt exact series used by MacroSentinel	
chartiny/nse-*	normalized mirrors of NSE reports	community, MIT repo	fallback/probe only; pin commit and cross-check official sample	
jugaad-data	NSE/RBI downloader and caching patterns	community	reference implementation only; do not add Python runtime dependency	
hanshof/sp500_constituents	daily historical S&P membership since 1996	community, MIT	adopt as a small diagnostic fixture, never as	


```
| `FinanceDatabase` | current global symbol/classification catalog | community, MIT | optional display/entity-resolution input only | context
| TradingView CSV | manually loaded chart bars/indicators | user-licensed manual export | accepted only for named-symbol diagnostics; never p
| FNSPID and similar news corpora | historical prices/news | research/community corpus | defer pending license, source, timestamp, and leakage
```

Repository code is never dynamically executed. Data files are downloaded over HTTPS; community files are pinned to immutable commit URLs. GitHub outages cannot remove already-hashed local copies.

4. Manifest Contract

Each acquisition writes a manifest before normalization:

```
interface EvidenceDatasetManifestV1 {

schemaVersion: "kairos.evidence-dataset.v1";

datasetId: string;

sourceId: string;

sourceAuthority: "official" | "community" | "manual";

evidenceClass: "promotion_candidate" | "diagnostic" | "reference";

market: "us" | "india" | "global";

dataKinds: Array<"ohlcv" | "universe" | "fundamental" | "macro" | "corporate_action">;

sourceVersion: string;

retrievedAt: string;

files: Array<{

relativePath: string;

sourceUrl: string;

sha256: string;

bytes: number;

mediaType: string;

}>;

coverage: { start: string; end: string; expectedFiles?: number; receivedFiles: number };

normalization: {

schemaVersion: string;

codeGitSha: string;

codeSha256: string;

status: "pending" | "valid" | "quarantined";

};

limitations: string[];

datasetFingerprint: string;

}
```

The dataset fingerprint is SHA-256 over canonical manifest fields and ordered file hashes. A changed upstream file creates a new dataset version; it never mutates an existing valid version.

5. Canonical Records

Daily bar

```
`market, exchange, symbol, session_date, open, high, low, close, volume,
turnover, currency, price_basis, source_file_sha256`
```

Raw exchange prices remain raw. Adjusted prices are a derived dataset with a separate fingerprint and the exact corporate-action manifest in its lineage.

Fundamental fact

```
`cik, accession, symbol_at_filing, form, filed_at, accepted_at, period_start,
period_end, fiscal_year, fiscal_period, taxonomy, tag, unit, value, statement,
source_file_sha256`
```

Reads use `accepted\_at <= as\_of`; if the compact SEC data exposes only filing date, daily replay treats it as available after that session and records the coarser clock. Amendments are additional accessions, not in-place replacements.

Macro observation

```
`series_id, observation_date, realtime_start, realtime_end, value, unit,
frequency, source_file_sha256`
```

An as-of read selects the row whose real-time interval contains the decision date. Latest revised values must never be stamped onto historical decisions.

Corporate action

```
`market, symbol, action_type, ex_date, record_date, ratio_or_amount, currency,
announced_at, source_file_sha256`
```

Unsupported or ambiguous actions quarantine the affected symbol/date range. The normalizer never guesses a split ratio from a price jump.

6. Validation and Quarantine

All gates fail closed:

1. HTTPS and allowlisted host.
2. Successful status and non-empty body.
3. SHA-256 and byte count recorded.
4. Expected archive members and headers present.
5. Dates parse and remain inside requested coverage.
6. OHLC invariants: positive prices, `high >= max(open,close)`,
`low <= min(open,close)`, non-negative volume.
7. One canonical row per source key; conflicting duplicates quarantine.
8. Market/currency contract: US/USD, India/INR.
9. Session coverage report distinguishes exchange holidays from missing files.
10. Corporate-action completeness is required for adjusted-return datasets.
11. Symbol mappings are time-varying; current ticker text is not a permanent ID.
12. No normalized record may have `knowable\_at > replay\_as\_of`.

Quarantined files remain preserved with reasons and hashes but cannot be resolved by the replay accessor.

7. Existing Architecture Integration

This extends, rather than duplicates, current truth layers:

- `lib/replay/packet-assembler.ts` remains the sole raw-to-frozen boundary.
- `lib/replay/sealed-accessor.ts` remains the network-free read boundary.
- `edge\_universe\_members` remains the persisted PIT universe table.
- `fundamental\_facts` remains the live forward-capture ledger.
- `backtest\_experiments` remains the immutable experiment identity.

The local resolver emits the existing `RawRecord` contract, extended additively with first-class `macro` and `corporate\_action` item types. It does not add a second scoring API or a second experiment ledger.

Default resolver order for offline replay:

1. exact immutable local dataset named in the experiment manifest;
2. existing persisted PIT snapshot with matching policy/fingerprint;
3. refuse.

There is no live-provider fallback inside a bound replay.

8. Safety Boundary

- No environment switch can make this store affect live scoring.
- No API route exposes raw files.
- No cron downloads or normalizes data in P0.
- No LLM parses, validates, transforms, or selects financial rows.
- Community and manual sources are diagnostic by default.
- A promotion experiment must name the exact dataset fingerprint before the run.
- India and US datasets, currencies, universes, actions, and results remain separate.

9. Build Order

1. Manifest, hashing, allowlist, atomic download, quarantine, and catalog CLI.
2. FRED/ALFRED exact-vintage acquisition for the existing MacroSentinel series.
3. SEC quarterly ZIP acquisition and schema validation; selective normalization for the experiment universe rather than expanding every fact permanently.

4. NSE official bhavcopy acquisition and normalization.
5. NSE corporate-action acquisition and adjusted-return derivation.
6. Local replay resolver to `RawRecord`, plus sealed-accessor tests.
7. Diagnostic community fixtures pinned to commits.
8. Coverage report; only then run new India and macro PIT diagnostics.

10. Acceptance Criteria

- Re-running an acquisition with unchanged bytes is idempotent.
- Changed bytes at the same URL produce a new version or quarantine; never overwrite.
- Interrupted downloads leave no valid manifest.
- Path traversal from ZIP member names is rejected.
- Invalid rows cannot appear in normalized output.
- An as-of macro/fundamental read cannot see later revisions/amendments.
- India adjusted returns refuse when corporate actions are missing.
- A replay bound to a dataset works with network disabled.
- No import changes any current score, signal, paper position, broker proposal, or order.
- Catalog output names coverage, source authority, evidence class, limitations, and exact fingerprints without exposing secrets.

11. Residual Limitations

1. Free US history older than the existing Massive entitlement remains unsuitable for broad promotion-grade, survivor-safe equity tests.
2. SEC statements solve fundamentals, not prices, corporate actions, or delisting returns.
3. NSE bhavcopy gives a point-in-time traded set, not necessarily every temporarily suspended but still listed security.
4. Community historical-index files are useful cross-checks, not authoritative truth.
5. Historical news/sentiment remains deferred because publication-time corrections, corpus selection, redistribution rights, and prompt-injection surfaces are harder than the incremental value currently justifies.

12. Implementation Record

Implemented on 2026-07-29. The exact current manifests, fingerprints, coverage, row counts, and residual exclusions are recorded in `ACQUISITION\_RECORD.md`. Only entries marked `currentNormalizer: true` by the catalog are candidates for new experiment binding.

Source: features\historical-replay-harness\FEATURE_ARCHITECTURE.md
Historical Replay Harness – Frozen Point-in-Time Eligibility

> ****STATUS: SUPERSEDED IN PART.**** The sealed primitives were implemented. The
> parallel storage proposal below is historical context and is not authoritative
> for new bulk-data runs.

> New bulk-data experiments follow
> `features/local-historical-replay/FEATURE\_ARCHITECTURE.md` and extend the existing
> immutable `backtest\_experiments` ledger.

<details>
<summary>Historical implementation notes (source-preserved)</summary>

Last updated: 2026-07-29

</details>

Owner: Vaibhav
Requirement: `CLAUDE\_CODE\_POST\_UPGRADE\_FIX\_PROMPT.md` – Required test ****15****.

1. The requirement (verbatim)

> ****Test 15:**** "MU/INTC/SNDK/GME-style frozen historical packets run without future
> data and report whether/when each model becomes eligible – no handpicked future-return
> dates."

And its sibling, test 14 (the guard this harness must not violate):

> ****Test 14:**** "point-in-time label/universe fixtures cannot access future records."

Read together: build a replay that, for a handful of named symbols, freezes the world as it was on a sequence of past as-of dates, runs the real scoring → calibration → eligibility pipeline against each frozen slice, and reports ****the first date each model would have cleared its eligibility gate**** – chosen by the pipeline, never cherry-picked by a human who already knows the outcome.

The adversarial framing from the prompt applies: "Treat real money as adversarial." The whole point of this harness is to make it ****impossible**** to accidentally answer "would this model have caught the MU/INTC run-up?" with a number that secretly peeked at the run-up.

The harness does not invent an eligibility concept. It replays the gates that already exist:

"Model becomes eligible" in test-15 language **the earliest as-of date at which one of these gates flips from fail to pass, using only data dated $\leq$ that as-of date.** That is the single number the harness reports per (symbol-cohort, model, setup, horizon).

— — —

The PIT machinery is largely built; this harness assembles frozen inputs *for* it.

- ```
The leak surfaces this harness must close
```

- — —

Three components, each a thin layer over existing code.

[illegible]

■ per (symbol, as-of date): ■

- [illegible]

- [illegible]



The cursor steps forward one trading day at a time across the run window. At each step it re-seals to the new `asOf` and re-runs the gates. Walk-forward calibration inside a step still uses `walkForwardFolds()` with its existing purge/embargo – so there are **two** layers of time-safety: the outer sealed cursor (no packet item after asOf enters the step at all) and the inner purge/embargo (no train label window bleeds into a test fold).

### C. Eligibility reporter – the honest output

For each (cohort-symbol or cohort, model, setup, horizon), the reporter walks the per-date verdicts in ascending date order and records the **first** date the gate passes:

symbol model setup horizon first\_eligible\_asof gate\_margin n\_oos ece

MU quality\_catalyst\_moment breakout 10d 2023-01-19 +0.031 41 0.074

MU turnaround\_inflection inflection 20d 2022-11-08 +0.012 36 0.088

INTC turnaround\_inflection inflection 20d never (window ends 2024-06-30) — —

SNDK (delisted 2016; excluded — see §8 open question)

GME fast\_breakdown\_defense crowding 5d 2021-01-27 (VETO fired, not entry)

- `first\_eligible\_asof` is **selected** by the gate, not by a human. There is no "pick the date MU doubled" input anywhere in the pipeline.
- Forward return is reported **only** as a consequence column, computed strictly after `first\_eligible\_asof`, and is never fed back into selection, sizing, or gate thresholds within the run. It answers "and then what happened?" – it does not choose the date.
- "never eligible in window" is a first-class, honest result. A model that never clears its gate on MU across the whole replay window reports `never`, not a fudged partial pass.

---

## 5. Proposed storage schema (describe only – DO NOT create)

Four additive tables. Names/shapes are a proposal; the migration is out of scope for this draft and must not be applied without approval (and, per the global schema rule, verified against the target DB before any dependent code ships).

- **replay\_packets** – one row per (symbol, as-of date). Columns (proposed): `id`, `cohort` (text, e.g. `semis\_memory\_2022`), `symbol`, `market`, `as\_of` (date), `manifest\_hash` (sha256 over the frozen item set), `publication\_lag\_assumptions` (jsonb), `created\_at`. Immutable after write.
- **replay\_packet\_items** – the frozen inputs. `id`, `packet\_id` (fk), `item\_type` ('ohlcv', 'fundamental', 'news', 'universe'), `knowable\_at` (timestampz – the date this record was public), `source`, `source\_tier`, `payload` (jsonb), `payload\_hash`. The `knowable\_at ≤ packet.as\_of` invariant is what the sealed accessor enforces at read time and what a DB check/backfill test asserts at write time.
- **replay\_eligibility\_runs** – one row per replay execution. `id`, `cohort`, `model\_kind`, `setup`, `horizon\_days`, `window\_start`, `window\_end`, `packet\_manifest\_hash` (proves which frozen inputs were used), `code\_git\_sha`, `created\_at`.
- **replay\_eligibility\_events** – per (run, symbol, as-of) gate verdict. `run\_id`, `symbol`, `as\_of`, `gate` ('calibration\_oos', 'validation', 'ic', 'thin\_evidence', 'breakdown\_veto'), `passed` (bool), `margin` (numeric), `n\_oos`, `ece`, `detail` (jsonb). The reporter's "first\_eligible\_asof" is a `MIN(as\_of) WHERE passed` query over this table.

Reuse existing tables where possible: labels/returns can still live in `observation\_labels`, and the accessor can read `evidence\_records` filtered by `effective\_at ≤ as\_of` for any inputs already captured there, rather than duplicating them into packets.

---

## 6. How it plugs into the existing walk-forward code

- The harness **imports** `walkForwardFolds` and `loadLabeledDataset` from `lib/learning/dataset.ts` unchanged – but feeds them observations assembled from sealed packets rather than the live `decision\_observations` join.
- It **imports** `fitCoefficients` / `predictPWin` / `acceptCalibrationOOS` from `lib/validation/calibration.ts` unchanged. The eligibility verdict per as-of date is literally `acceptCalibrationOOS(...).accepted` computed on the packet-sealed folds.
- It **imports** `computeWeightedAnalystScore` from `lib/scoring/weighted-score.ts` so the replay's analyst\_score is identical to production's.
- The only **new** code is: (a) the packet assembler, (b) `SealedDataAccessor` + `FutureDataLeakError`, (c) the reporter, (d) the cohort fixtures. The gates themselves are reused verbatim – this is what makes the report trustworthy (it tests the real gates, not a reimplementation, satisfying the fix prompt's "call the actual services, not duplicate pure logic" rule).

---

## 7. Phased build plan & effort

| Phase | Deliverable | Effort (ideal-dev-days) |

```

|---|---|---|
| **P0 – sealed accessor + leak test** | `SealedDataAccessor`, `FutureDataLeakError`, and a unit test proving a deliberately future-stamped i
| **P1 – packet assembler (prices + news)** | Offline builder for OHLCV and news packets from historical provider endpoints; manifest hashing
| **P2 – fundamentals as-of** | Filing-date-based fundamental freezing incl. the publication-lag assumption + manifest recording. The hard, h
| **P3 – replay cursor + reporter** | Step-forward cursor wiring the sealed accessor into the reused gates; `replay_eligibility_events` writes
| **P4 – cohort fixtures + test 15** | MU/INTC/GME (and SNDK caveat) fixtures; the actual `tests/replay-eligibility.test.ts` asserting each m
| **P5 – schema migration (gated)** | The four tables, additive, applied only after owner approval + target-DB verification per the global sch
| | **Total** | **~8 dev-days** |

```

P0 is independently valuable and low-risk – it can land and satisfy the test-14 guard before the heavier packet work. Recommend building P0→P4 as offline/measure-only (no money path touched, consistent with the fix prompt's "shadow/measure-only first" doctrine); P5 last.

---

## ## 8. Risks / open questions

1. **Fundamental publication dates.** The whole honesty of the fundamentals freeze rests on knowing the *filing/acceptance* date, not the fiscal period. If the chosen provider only exposes fiscal period, the publication-lag heuristic is an assumption that must be visible in every report. Open question: does FinancialDatasets `get_filings`/`get_income_statement`` reliably return an accepted/field date for the 2020-2023 window for these symbols? Needs a spike before P2.
2. **SNDK (SanDisk) was delisted (acquired by Western Digital, 2016).** Historical data for a delisted ticker is spotty and the current providers/universe don't carry it. Options: (a) drop SNDK and document why, (b) source a one-off frozen CSV for it, (c) substitute a still-listed memory/NAND name (WDC/MU) and note the substitution. **Owner decision needed.** The requirement says "SNDK-*style*", so a documented substitution is likely acceptable.
3. **GME is an exit/veto case, not an entry case.** The honest expected result for GME is that the fast-breakdown/crowding defense (fix prompt Phase 2 §4) fires a **veto**, not that an entry model becomes "eligible." The reporter must represent veto-fired distinctly from entry-eligible so GME isn't scored as a missed long.
4. **Small-n calibration on a 4-symbol cohort.** `acceptCalibration00S`` needs ≥30 00S rows. A single symbol across a replay window won't reach that from its own observations. The cohort/universe must be broad enough at each as-of date to produce a real holdout – i.e. the replay universe is the *point-in-time liquid set*, and the named symbols are the ones we *report* on, not the entire training set. This must be designed in from P3.
5. **Corporate actions (splits/dividends) inside the window.** Frozen prices must be split-adjusted *as known at each as-of date*, which is subtle (a split that happens after asOf must not be retro-applied to pre-asOf candles). Needs explicit handling in P1.
6. **Cost/rate limits of historical backfill.** Assembling multi-year daily packets for a cohort burns provider budget (`provider-fetch.ts`` daily caps). Assembly should be a throttled, resumable one-time job, not a cron.

---

## ## 9. What this harness does NOT do

- **It is not a backtester and does not simulate P&L, fills, slippage, or position sizing.** It answers one question: *when would each model have become eligible?* Realized-return backtesting is the Validation Engine's job (`lib/validators/backtest.ts``); this harness only reports the forward return *after* the eligibility date as a read-only consequence.
- **It does not touch the money path.** No orders, no `strategy_config``, no live/paper execution, no autonomous flags. Consistent with the fix prompt: measure/shadow only.
- **It does not promote or demote any model.** Eligibility here is a historical diagnostic, not a live promotion signal. Promotion stays governed by the existing Validation Engine + owner review (`docs/arch/09-learning-loop.md``).
- **It does not fix survivorship bias in the live universe** (`lib/edges/universe.ts``). It works around it *for the replay* by carrying per-packet universe membership, but building a true versioned PIT index membership (fix prompt Phase 2 §2) is a separate effort.
- **It does not hand-pick outcome dates** – by construction. The absence of any "future return date" input is the feature, and P0's leak test is what proves it.
- **It does not re-fetch on replay.** Once packets are frozen, a run is a pure function of frozen inputs + gate code, reproducible via the manifest hash.

---

## ## 10. Docs to update when this ships (per CLAUDE.md)

- `docs/arch/09-learning-loop.md`` – new "Historical Replay Harness" section under Validation.
- `docs/arch/04-database-schema.md`` – the four `replay_*` tables (only after migration applied and verified).
- `public/agent-diagrams/system-map.json`` – only if the harness becomes a scheduled/agent flow (currently proposed as an offline tool, so likely no system-map change).
- This file: flip STATUS to approved and record the approval date.

---

# Source: features\holding-risk-daily\FEATURE\_ARCHITECTURE.md  
# Feature: Daily Per-Holding Risk Analytics

**Status:** SHIPPED – current formula `hr-v3``

**Last updated:** 2026-07-21

**Owner:** Vaibhav

**Update this file when:** the per-holding risk score formula, the strategy-decision

rules, the daily cron cadence, or the snapshot schema changes.

```
> **Sector-cap breach allocation (`hr-v1` → `hr-v2`, 2026-07-16):** the posture rule
> below ("hard concentration/cluster breach → `trim`") was wrong for the SECTOR cap.
> A sector breach is a property of the sector, not of any name in it, so it gave
> every holding in the sector the identical `trim` with no size. The per-name
> allocator – which names absorb the breach, how much each gives up, and why a name
> was NOT selected – is specified in
> **features/risk-sector-breach-allocation/FEATURE_ARCHITECTURE.md**, which is the
> authority for that rule, its NAV denominator, and the read-only advisory labelling.
> This file remains the authority for the score, the cron, the snapshot schema, and
> the LLM prose boundary. No migration; `formula_version` carries the change.
```

```
> **Account-intent correction (`hr-v2` → `hr-v3`, 2026-07-21):** a generic Kairos
> trading cap is not an approved sell mandate for a long-term/read-only account.
> Concentration breaches on every live account now produce `review`, never `trim`.
> Broad asset-class sleeves (`Diversified Equity`, `International Equity`, Fixed
> Income, Commodities, Digital Assets) are not equity sectors and are excluded
> from `max_sector_exposure_pct`. Correlation remains measured but produces
> `review` until a deterministic allocator identifies the holding and quantity.
> Having an order path is not proof of an account-specific concentration mandate.
> A future `trim` requires both that mandate and a deterministic executable share-
> quantity plan. Protective-stop/thesis `exit_review` precedence is unchanged. No
> schema or order-path change.
> Stored `hr-v1`/`hr-v2` rows remain immutable audit evidence, but current Risk
> Analytics and newsletter read surfaces expose their original posture separately
> and render legacy `trim` as `review` with an explicit historical-reason label.
```

## Intent (owner, verbatim)

```
> **Canonical account labels (2026-07-21):** `broker + account_id` remains the
> stable identity and join key. The latest verified `live_account_snapshots.nickname`
> is presentation metadata only. Live Portfolio and Risk Analytics render
> `<nickname> . <broker> ....<last4>`; if the nickname is unavailable they render
> `<broker> ....<last4>`. The read API enriches historical runs at response time,
> so immutable risk evidence is never rewritten when an account is renamed. Full
> account IDs remain authenticated API request keys but are never rendered as
> visible account labels; raw snapshot rows are not returned. Nickname lookup
> failure degrades to the masked fallback without blocking risk results.
```

```
> Risk analytics on left panel – add live OTHER accounts, each holding's risk
> analytics, score, and details of risk and what should be our strategy for each
> one of them, updated everyday.
```

## Decisions (approved 2026-07-11)

- \*\*Strategy source: Hybrid.\*\* A deterministic engine computes the per-holding risk score (0-100) AND the risk posture (`hold` | `review` | `trim` | `exit\_review`). A risk-only engine must **not** recommend adding capital: `add` requires a separate current, actionable, deterministic alpha signal plus the existing portfolio/execution gates. The risk page may display `add\_capacity=true` (risk capacity exists), but that is not an add recommendation. An LLM writes only the human-readable explanation around that fixed decision – it cannot change the score or the action. Complies with PROJECT\_DECISIONS ("deterministic services calculate risk; LLMs may propose/explain") and the safety rule ("LLMs may not control money limits, accounts, promotion, order submission").
- \*\*Scope: all live accounts.\*\* Every Robinhood account (Trading `<agentic-account-id>`, read-only `<read-only-account-id>`, any others) + Kite India. The strategy line is **advisory** for all accounts and wired to NOTHING – it never reaches the order path. Only `<agentic-account-id>` can ever act on a call, and only via the existing owner-click Execution Gateway.

## Non-goals / guardrails

- No auto-execution. Strategy text is display-only.
- New positions long-only; `exit`/`trim` calls are advisory, not orders.
- Additive migration only; daily snapshots are an append-only history table.
- No cross-currency roll-up. USD and INR accounts are scored and trended independently; the feature never adds USD market value to INR market value.
- A score is a **risk-control pressure index**, not a probability of loss and not a return forecast. UI copy must not present it as predictive confidence.
- Missing/stale inputs produce `insufficient\_data` or a lower `data\_confidence`; the engine must not replace unavailable beta, correlation, sector, quote, or cost-basis evidence with a confident neutral score.
- Unrealized loss alone never creates an `exit\_review`. Selling solely because a holding is down is a loss-chasing rule. Exit review requires a deterministic protective-stop/thesis-break/hard-risk-breach reason with its evidence recorded.
- Read-only accounts stay read-only. Without an approved account-specific objective/cap mandate, concentration produces `review`, not a sell instruction. A verified protective-stop/thesis break may still produce advisory `exit\_review`.

## Data model (additive)

`holding\_risk\_runs` (append-only, one row per market/account computation):



- `id` uuid, `run\_key` text unique, `market`, `currency`, `broker`, `account\_id`, `account\_label`, `status` (`running|complete|failed|partial`)
- `source\_captured\_at`, `started\_at`, `completed\_at`, `formula\_version`, `input\_hash`, `data\_confidence`, `missing\_inputs` text[], `error`
- `run\_key` is deterministic for `market` × `account` × local trading date × `formula\_version` × `input\_hash`; concurrent/retried crons claim it through a unique insert. Do not use check-then-insert idempotency.

`holding\_risk\_snapshots` (append-only, one row per run × holding):

- `id`, `run\_id` uuid FK holding\_risk\_runs(id), `captured\_on` date, `market`, `currency`, `broker`, `source`, `source\_captured\_at`, `account\_id`, `account\_label`
- `symbol`, `sector`, `qty`, `current\_price`, `average\_cost`, `market\_value`, `weight\_pct`, `beta`, `realized\_vol\_pct`, `unrealized\_pnl\_pct`
- `holding\_risk\_score` int, `risk\_label`, `risk\_drivers` jsonb (deterministic detail)
- `risk\_posture` text (`hold|review|trim|exit\_review`), `action\_reason` text (deterministic), `add\_capacity` boolean (never an order signal)
- `data\_confidence` numeric, `missing\_inputs` text[], `formula\_version` text
- `strategy\_note` text (LLM prose, nullable – populated best-effort, never blocks)
- `created\_at`

`account\_risk\_snapshots` (new append-only table, one row per `holding\_risk\_runs.id`): the existing in-code `RiskMetrics` roll-up persisted so the page can show Δ-vs-prior-comparable-run and a trend. This table does **not** exist today; “existing” refers only to the TypeScript `RiskMetrics` contract. Store `currency`, `source\_captured\_at`, `formula\_version`, `data\_confidence`, and `missing\_inputs` here too.

Schema rules:

- Unique `(run\_id, symbol)` for holding rows and unique `run\_id` for account rows. Tables have INSERT-only service-role policies plus UPDATE/DELETE-blocking triggers. Reruns insert a new run only when inputs/formula changed; they never upsert or rewrite a prior snapshot.
- Owner-authenticated SELECT only; no anon access. Account identifiers and positions are sensitive.
- Numeric checks: finite, `market\_value` ≥ 0, `0` ≤ `weight\_pct` ≤ 1, `0` ≤ `score` ≤ 100, `0` ≤ `data\_confidence` ≤ 1.
- No cascade that can delete snapshot history. A failed run remains as evidence.

## Compute path

1. Add a pure, versioned module `lib/risk/holding-risk.ts` with
 

```
computeHoldingRisk(holding, ctx) → { score, label, drivers[], riskPosture, actionReason, addCapacity, dataConfidence, missingInputs }
```

 Pure function; `ctx` carries account/portfolio totals, owner-approved risk limits, sector exposure, correlation evidence, beta/volatility, quote freshness, and data availability. Do not enlarge `lib/portfolio-risk.ts` further; reuse its types/calculations where sound, but keep daily snapshot logic isolated and testable.
  - V1 score is formula-versioned and based on **limit utilization**, not uncalibrated “TBD” weights:
    - name concentration: 30 points, scaled against `max\_name\_exposure\_pct`;
    - sector concentration contribution: 20 points, scaled against `max\_sector\_exposure\_pct`;
    - volatility/beta contribution: 15 points, scaled against the portfolio volatility/beta budget, only when real price evidence is available;
    - correlated-cluster contribution: 15 points, from computed aligned-return correlations, never the small static `KNOWN\_CORR` map alone;
    - drawdown/stop-distance pressure: 10 points; drawdown alone cannot trigger exit;
    - liquidity/event/idiosyncratic flags: 10 points when supported by fresh data.
 Each component is clamped 0-its cap and recorded in `risk\_drivers`.
  - If required structural fields (qty, price, market value, account total, currency) are missing/non-finite, return `insufficient\_data` with no numeric score. Optional missing dimensions reduce `data\_confidence` and are excluded from the numerator without renormalizing the score to appear fully confident.
  - Posture precedence: verified protective-stop/thesis-break → `exit\_review`; direct-name, allocated real-sector, and correlated-cluster concentration produce `review` for every account because global references are not account-specific sell mandates; incomplete or conflicting evidence → `review`; otherwise `hold`. `add\_capacity` means only that owner-approved risk limits have room.
2. New cron `POST /api/agents/holding-risk` (cron-secret gated):
  - accept only `market=us|india`; use verified market calendars and the market-local completed session date;
  - acquire the append-only run claim before external/LLM work;
  - fetch all live accounts (reuse `fetchRobinhoodBrokerAccounts` / Kite), but persist only broker-verified account identities. Today `fetchKiteBrokerAccount()` reports placeholder `accountId="kite\_india"`; it must instead return the verified Kite `user\_id` that matches `active\_account\_india`, or the run fails closed for that account;
  - **Robinhood source (shipped 4f0bec7, 2026-07-12):** `fetchRobinhoodBrokerAccounts` no longer uses the classic REST client (`api.robinhood.com` 401s `rejected client id` for the MCP-scoped vault token – it never returned

```

holdings). It now calls `captureAllRobinhoodAccounts()` in
`lib/robinhood-mcp.ts`: one agentic-MCP session, `get_accounts` enumerates
all 6 RH accounts, per-account `get_equity_positions` + `get_portfolio`, and
a batched `get_equity_quotes` over deduped held symbols to price holdings.
`agentic_allowed=false` gates order placement only, not reads, so all 6
accounts feed risk while order placement stays restricted to ``.
The same capture backs `refreshViaMcp()` (all 6 accounts upserted into
`live_account_snapshots`, not only the active one);
- capture one coherent, bounded input snapshot per account. Never combine a
Robinhood account timestamp with another account's holdings or a later quote;
record source timestamps and reject/mark stale inputs;
- compute per-holding + per-account risk deterministically,
- LLM pass (hybrid): given the fixed score+action, write `strategy_note` per
holding. The prompt contains only structured facts; output is length-bounded,
schema-validated, and cannot introduce new numbers/actions. Batch notes to
control cost. Best-effort; empty note on LLM failure never blocks the row.
- insert snapshots and mark the run complete in one database transaction/RPC.
Never `upsert` append-only snapshot rows. Partial DB failure marks the run
failed/partial and does not publish it as latest.
3. Keep `GET /api/portfolio/holdings` unchanged for live view. Add owner-gated
`GET /api/portfolio/risk-daily?market=us|india&accountId=...` that returns the
latest complete run plus the previous complete run with the same
account/market/currency/formula version. Do not compare across a formula change.

```

#### ## UI (PortfolioRiskPage.tsx)

- Holdings table gains columns: **Risk** (score chip, colored) and **Action** (hold/review/trim/exit-review pill). `insufficient_data` is shown explicitly, not as risk 0.
- Row expander shows `risk_drivers` bullets + `strategy_note` prose +  $\Delta$  vs yesterday.
- Header cadence flips `as-needed`  $\rightarrow$  `daily` with "Last computed: <date>" from the snapshot, the source-data as-of time, formula version, and confidence. The live Refresh remains separate and must not silently overwrite the daily historical run.
- Account tabs never aggregate currencies. Read-only accounts show "advisory only"; the agentic trading account still offers no order button from this feature.
- Account tabs use the canonical presentation label above. Never use a transport-adapter fallback label or expose a full broker account number in UI copy.

#### ## Cron

`holding-risk` – daily, per market (`?market=us`, `?market=india`), after the respective exchange close **and** after the authoritative account snapshot refresh. If the refresh is absent/stale, publish a failed/insufficient-data run, not yesterday's risk as today's. The route is owner-or-valid-cron-secret gated, concurrent runs are serialized by the DB run claim, and provider/LLM failures are recorded per account. Exact times and dependency ordering go in `docs/arch/05-crons-and-scheduling.md` on ship.

#### ## Failure and safety behavior

- Account enumeration/config/profile error: fail that account closed; do not substitute another account or the combined portfolio.
- Stale quote/account snapshot: no actionable posture; `insufficient_data`.
- Missing sector/beta/correlation: lower confidence and list the missing inputs; never silently use "Other", beta 1.0, or zero correlation as verified fact.
- LLM unavailable/invalid: persist deterministic row with null note.
- Snapshot insert/transaction failure: run is failed and is not returned as latest.
- Duplicate cron/retry: unique run claim returns the existing run; no duplicate holding rows and no mutation.
- Formula change: new `formula_version`; trends/deltas are not compared across versions without an explicit compatibility transform.
- Corporate actions/symbol changes: preserve broker symbol and normalized symbol, with the raw broker payload hash/provenance; never infer P&L across a split from unadjusted values.

#### ## Required verification before ship

- Pure formula tests at every threshold and with NaN/Infinity/null inputs.
- US and India fixtures proving no currency/account cross-contamination.
- Duplicate/concurrent cron test and transaction rollback test.
- Stale/missing broker snapshot and provider failure tests.
- Kite token user-id mismatch test; unknown Robinhood account remains advisory.
- Append-only trigger tests (UPDATE/DELETE rejected) and owner-only RLS tests.
- LLM output cannot alter score, posture, drivers, symbols, or numbers.
- Clean migration replay, `tsc --noEmit`, full unit suite, build, and authenticated Risk page smoke test.

#### ## Cross-doc updates required on ship (same commit)

- `docs/arch/05-crons-and-scheduling.md` – new daily cron.
- `docs/arch/08-risk-and-safety.md` – advisory-only strategy line, read-only scope.
- `docs/arch/04-database-schema.md` – two new tables.
- `public/agent-diagrams/system-map.json` – new HoldingRisk node + history entry.

- Migration verified applied to prod DB before the reading code ships.

## ## Reviewer changelog (ChatGPT)

- Corrected the append-only/upsert contradiction by adding run-level idempotency, insert-only snapshots, immutable triggers, and a publish-only-complete rule.
- Split risk capacity from alpha: removed risk-only 'add' recommendations and prohibited loss-only exit calls.
- Defined V1 score semantics/components so the implementation is not left to arbitrary thresholds; added confidence/abstention for missing data.
- Added market/currency/account/source timestamps and prohibited USD/INR or cross-account aggregation.
- Corrected the Kite reuse assumption: current broker code uses the placeholder 'kite\_india'; daily risk must use the broker-verified Kite 'user\_id'.
- Added coherent source snapshots, stale-data behavior, transactional publication, concurrent cron handling, formula versioning, security/RLS, failure modes, and acceptance tests.

---

# Source: features\hybrid-stop\FEATURE\_ARCHITECTURE.md  
# Hybrid Protective Stops

- > Status: P1 implemented: read-only coverage and autonomous-entry interlock.
- > Broker placement/cancel/replace remains deliberately unimplemented and disabled.
- > Scope: US and India live positions. Paper remains unchanged unless separately approved.

## ## Current status and phasing (2026-07-30)

The production schema was checked directly on 2026-07-30. 'protective\_orders', 'protective\_order\_events', and 'strategy\_config.protective\_orders\_enabled' exist; the flag is false and there are zero protective-order rows. Migration presence is not treated as deployment proof.

P1 is active but cannot send broker traffic:

- 'lib/protective/coverage.ts' accepts protection only when exactly one matching broker order is active, unexpired, has exactly one broker identifier, and has protected and reconciled quantities that exactly match the reconstructed holding. Partial, oversized, duplicate, expired, or unreconciled records are unprotected; an oversized SELL floor could create a short after a trigger. The aggregate coverage result must also be explicitly true: unknown/inconsistent aggregate state with no per-position detail still blocks entry.
- 'runAutonomousLive()' blocks new autonomous live BUYS before provider work or a proposal write unless the feature flag, a future broker-specific placement and reconciliation worker, and full coverage for all managed live positions pass.
- The worker availability is a source-level false constant. A Settings, database, environment, or LLM change cannot turn it on.

The existing-position interlock is necessary but not sufficient for P2. A future BUY workflow must prove how the newly filled quantity becomes broker-protected without an unbounded gap; merely changing the source constant to true is not an approved placement implementation.

P1 does not place, amend, cancel, or query a broker order. It does not alter PaperTrader, PositionMonitor, manual live orders, scoring, learning, or the existing synthetic live-exit monitor. Broker-hosted protection remains a separate P2 activation and canary decision.

## ## Shadow scaffold status (2026-07-18)

The owner approved BUILDING the shadow scaffold – everything UP TO the placement line, and it STOPS there. \*\*No live broker order is placed. Not a \$1 test. Ever.\*\* The spec below remains authoritative for the eventual live design; this section records what shipped as inert scaffolding.

\*\*Built (all shadow / no placement), under 'lib/protective/':\*\*

- 'capabilities.ts' – the broker-neutral 'BrokerProtectiveCapabilities' matrix + 'evaluateProtection()' (capability-driven; no flat broker boolean; no eligible multi-day order → 'unprotected-by-broker'; GTC ≠ every-session triggering).
- 'kite-capabilities.ts' – Kite's capability declared from the PROVEN GTT code ('gtt\_limit' weaker protection; DAY-only SL-M rejected as a multi-day floor).
- 'disaster-floor.ts' – pure 'computeDisasterFloor({mode, distance, ...})'. Default 'mode = wider\_disaster\_floor' (Q1 unanswered); distance is a config input, no hardcoded value; monotonic ratchet (a falling high-water mark can't lower it).
- 'reconcile.ts' – pure 'reconcileProtectiveOrder()' (out-of-band triggers, partial fills, cancels, expiry, broker edits, corp-action drift; unknown → 'needs\_reconcile'; trigger-without-fill never closes the book).
- 'state.ts' – the 'protective\_order' record shape + status machine + exit provenance ('protective\_disaster\_floor' / 'learning\_scope = risk\_policy\_only') + long-only / cancel-before-replace invariants.
- 'placement-gate.ts' – \*\*THE MONEY LINE.\*\* False-by-default 'protective\_orders\_enabled' flag gates ALL placement and stays false; 'planProtectivePlacement()' never calls a broker.

```

Schema (applied, still inert):
`supabase/migrations/20260718000000_protective_orders_shadow.sql`, hardened by
`20260718130000_harden_protective_orders_invariants.sql`. The schema is only a
ledger/control plane until a separately approved P2 broker worker exists.

Tests: `tests/protective-hybrid-stop.test.ts` – all 14 spec acceptance tests,
each falsifiable (mutation-verified).

Still gated behind owner approval before anything goes live: Q1 (touch
semantics), floor distance, ratchet step/cadence, and the Kite-first vs
Webull-first build sequence – see "Open Owner Decisions" below.

Decision

Kairos currently owns close-based stop, target, trailing, thesis, and time exits. A
broker-hosted order changes an analytical close-based rule into an intraday-touch
execution rule. That is a strategy change, not merely outage protection.

The recommended design is therefore a wider disaster floor, not a duplicate
of the analytical stop:

- Kairos remains authoritative for analytical exits and the trailing ratchet.
- The broker holds a wider, static protective floor for app or scheduler outages.
- Broker fills are reconciled back into the Performance Truth Layer.
- A broker floor may only move upward and may never increase risk.

Do not activate placement until the owner explicitly approves touch semantics,
floor distance, post-fill policy, and the relevant broker-specific P2 canary.

Verified Broker Capabilities

Broker	Verified capability	Constraint	Design consequence
Kite	GTT `single`/`two-leg` trigger with child orders represented as `LIMIT`	A trigger creates a limit order; a gap can leave it unfilled	
Kite regular order	`MARKET`, `LIMIT`, `SL`, `SL-M`; DAY/IOC/TTL validity	A regular protective order is not a multi-day substitute for GTT	
Robinhood agent API	Existing captured tool schema is insufficient proof for a new stop integration	Public support pages do not define Kite	
Webull	Trading API documents stop orders; Cloud MCP is query-only	Trading API is a different signed integration. Trailing-stop orders are not supported	

Broker-Neutral Capability Matrix

Protection is driven by a broker-neutral state machine that reads declared
capabilities per adapter – never by per-broker `if` branches in the protocol.
The credential store (`api_key_vault`) is generic; broker behavior is conditional,
so each adapter declares only combinations it has proved.

BrokerProtectiveCapabilities {
 scope: {
 market: Market
 instrumentTypes: InstrumentType[]
 sides: ("sell_long")[]
 accountModes: AccountMode[]
 }
 orders: Partial<Record<"stop_market" | "stop_limit" | "gtt_limit", {
 timeInForce: ("day" | "gtc" | "broker_managed")[]
 sessions: ("regular" | "extended" | "overnight")[]
 updateMode: "modify_in_place" | "cancel_replace" | "none"
 maxLifetimeDays: number | null
 oco: boolean
 }>>
}

- A flat broker-level boolean is forbidden. Support varies by order type,
instrument, session, TIF, account entitlement, and environment. For example,
Webull documents a GTC stop-market in `CORE`, but that does not prove the same
stop works in extended or overnight sessions.
- GTC means the order persists across trading days; it does not mean the
order can trigger in every session. Session eligibility is separate.
- An adapter with no eligible multi-day order for the exact position is declared

```

- \*\*synthetic-only\*\*** – the position is flagged unprotected-by-broker, never silently treated as protected.
- Preference order: ``stop-market`` where verified (designed to trigger a market order, without a guaranteed execution or fill price) → `GTT/`stop-limit`` as **\*\*weaker\*\*** protection with unfilled-trigger risk surfaced in the UI.
- When a broker adds a verified capability, its adapter updates the matrix after a fresh schema capture; the shared protocol is unchanged.

## ## Protection Is Mitigation, Not a Guarantee

State this honestly in code comments and the UI – do not let it read as "loss protection":

- A **\*\*stop-market\*\*** is designed to trigger a market order, but neither execution nor price is guaranteed during gaps, halts, venue/broker outages, or rejected orders. A floor at \$90 can fill at \$75 or remain unresolved.
- A **\*\*stop-limit / GTT limit child\*\*** controls price but may **\*\*never fill\*\*** on a gap → the position stays unprotected, which must be reported, not called filled.
- Stops generally **\*\*cannot execute while the session is closed\*\***. A resting floor does not cover the gap between close and the next open.

The honest label is **\*\*outage + catastrophic-loss mitigation\*\***, not guaranteed loss protection.

## ## Required State Model

One authoritative ``protective_order`` record per live position and broker:

- ``position_id`, `broker_account_id`, `market`, `symbol``
- ``broker_order_id`` or Kite ``trigger_id``
- ``mode = disaster_floor``
- analytical stop, broker floor, trigger and limit prices
- status: ``placing`, `active`, `triggered`, `filled`, `canceling`, `canceled`, `failed`, `needs_reconcile``
- broker version/update timestamp, expiry, last reconciliation timestamp
- immutable reason and correlation/idempotency id

The record references the existing position and execution ledger. It must not create a parallel position, cash, or P&L truth layer.

**\*\*Exit provenance.\*\*** A disaster-floor fill closes the position with ``exit_reason = protective_disaster_floor``, DISTINCT from every strategy exit (stop, target, trailing, thesis, time). The Learner excludes it from **\*\*signal-weight attribution and genome promotion\*\***, while the Performance Truth Layer, NAV, realized P&L, drawdown, mandate evaluation, and risk-policy evaluation MUST include the loss. A generic ``excluded_from_learning = true`` is insufficient because the existing evaluation engine also filters that field. Use explicit attribution such as ``learning_scope = risk_policy_only`` (exact schema decided during implementation). Broker execution quality is evaluated separately from the strategy signal that originally opened the position.

## ## Execution Protocol

### ### Entry

1. Pass all existing money-path gates and place the live BUY.
2. Reconcile the BUY fill and held quantity.
3. Compute a deterministic disaster floor from the approved mandate.
4. Reserve one protective-order intent using a stable idempotency key.
5. Place the broker floor and persist its broker id and expiry.
6. If placement, persistence, or immediate read-after-write reconciliation fails, create a critical health issue, mark the position unprotected, and latch a **\*\*broker-account + market entry circuit breaker\*\***. Existing exits and risk-reducing protection writes remain enabled. Clear the latch only after a later reconciliation proves the correct active protection and quantity.

### ### Ratchet

- Only update when the approved floor has risen by the minimum step.
- Kite uses modify-in-place for the existing GTT where possible.
- Any cancel/replace broker must confirm cancellation before replacement.
- Never lower a floor and never leave two sell orders capable of exceeding the reconciled holding.

### ### Explicit Exit

1. Read and lock the current protective-order state.
2. Cancel the resting protection and confirm cancellation.
3. Re-read reconciled held quantity.
4. Submit a SELL no larger than that quantity.
5. If cancellation cannot be confirmed, fail closed and alert. Do not submit a competing SELL that could create an accidental short.

The current standalone Kite path rejects a partial explicit SELL when a resting GTT exists because it cannot atomically rebuild protection for the residual.

The standalone Kite route now follows this cancel-before-sell rule; a future

shared implementation must reuse the protocol rather than add a second path.

### ### Reconciliation

Every live snapshot must detect out-of-band triggers, partial fills, cancellations, expiry, and broker-side edits. Reconciliation closes or adjusts the Kairos position using actual fills and appends execution evidence. Unknown state is `needs\_reconcile`, never assumed active or canceled.

Corporate actions and broker expiry are first-class reconciliation inputs. A split, symbol change, delisting, stale instrument id, or approaching GTC/GTT expiry invalidates the assumed price/quantity and requires a fresh broker snapshot before replacement. Renewal uses the same cancel/replace quantity invariant and may not create overlapping sell capacity.

### ## Safety Invariants

- No LLM computes, places, modifies, or cancels protection.
- US and India accounts, cash, orders, and quantities never cross.
- Existing pause, market controls, kill switch, drawdown breaker, approval mode, notional limits, and account allowlist all remain upstream of **new entries**.
- Entry halts never block explicit exits, reconciliation, placement of missing protection on an already-held position, or an upward risk-reducing ratchet. They block lowering/removing protection except inside the locked explicit-exit or verified replacement protocol.
- Total executable SELL quantity can never exceed reconciled held quantity.
- A trigger without a confirmed fill does not close the book.
- A gap through a Kite limit child is reported as unprotected, not called filled.
- Expired protection is a critical health state.
- Paper and live outcomes are not mixed across different exit semantics.

### ## Acceptance Tests

1. A cancel failure prevents the explicit SELL.
2. A replace cannot create two executable protective orders.
3. A falling high-water mark cannot lower the floor.
4. A Kite trigger with no fill remains open/unprotected and raises an issue.
5. An out-of-band fill reconciles with actual quantity and price.
6. Partial fills leave the correct residual protection.
7. Expiry and broker-side cancellation are detected.
8. Every pause/kill/account/approval gate blocks new entries; an entry halt cannot block exits or risk-reducing protection repair on an existing position.
9. US and India fixtures cannot read or mutate the other market.
10. Paper results remain on their existing close-based semantics.
11. A disaster-floor fill records `exit\_reason = protective\_disaster\_floor`, is excluded from signal-weight attribution, and remains included in P&L, drawdown, mandate, and protection-policy evaluation.
12. An adapter with no eligible multi-day protective order yields an unprotected-by-broker position, never a silently-protected one.
13. Capability fixtures reject unsupported order-type/TIF/session/account combinations; `GTC` alone never implies extended-hours triggering.
14. Split, expiry, partial-fill, and broker-edit fixtures cannot leave aggregate executable SELL quantity above the reconciled holding.

### ## Open Owner Decisions

1. Approve broker-hosted touch execution at all?
2. Disaster-floor distance: fixed percentage beyond analytical stop, volatility multiple, or mandate-specific value? Recommendation: mandate-specific ATR rule with a hard maximum loss bound.
3. Minimum ratchet step and update cadence.
4. Whether paper receives a separately versioned touch-fill simulation before live.
5. **Build sequence** – genuine tension, owner call:
  - **Kite-first** (this spec's build order): lifts already-written, proven GTT code into the shared path and delivers India protection without waiting for a new broker entitlement.
  - **Webull-first** (Codex's recommendation): build the signed `webull\_trade` adapter as a reference against a richer capability set + a sandbox, but delays any floor until Webull entitlement and lifecycle proof are complete.The tradeoff is ship-protection-soonest vs build-the-abstraction-right-first. My lean: adopt the capability-flag interface up front (from Codex) but implement **Kite first** against it, since its code is proven and it delivers protection before the Webull entitlement/sandbox path is even confirmed.

### ## Build Order After Approval

1. Capture current broker schemas and write failure-injection contract tests.
2. Add protective-order state and reconciliation without placing orders.
3. Shadow the computed floor and broker capability status.
4. Implement Kite sandbox/manual path first.
5. Implement a verified Robinhood path only after live schema capture.
6. Run a single owner-approved small live test per market.
7. Keep Webull in its separate feature and disabled.

### ## Primary Sources

```

- Kite GTT lifecycle and LIMIT child orders: https://kite.trade/docs/connect/v3/gtt/
- Webull stock order types, TIF, sessions, and examples: https://developer.webull.com/apis/docs/trade-api/stock/
- Robinhood's public agent tool inventory (schema still requires live capture): https://robinhood.com/us/en/support/articles/trading-with-you

Source: features\hybrid-stop\IMPLEMENTATION_RESULT.md
Hybrid Protective Stops: P1 Implementation Result

Date: 2026-07-30
Status: Implemented and intentionally non-executable at a broker.

What P1 Adds

- A deterministic, account-local `position_id` for managed live positions.
- Read-only full-coverage evaluation against the existing `protective_orders` ledger. A row counts only when it is the sole exact match, active, unexpired, broker-identified, and covers the full reconstructed quantity.
- A fail-closed autonomous-entry interlock. New autonomous live BUYS require:
 1. `protective_orders_enabled=true`;
 2. a future broker-specific placement/reconciliation worker; and
 3. full coverage for every current managed live position.

What P1 Does Not Do

- It does not call, place, alter, cancel, or read back a broker order.
- It does not change paper trading, manual live trading, the existing synthetic live-exit monitor, scoring, learning, or allocation.
- It does not activate Webull, Robinhood, Kite GTT, or any feature flag.

The placement-worker constant is false in source code. Therefore a database or Settings change cannot accidentally create autonomous positions without a broker-resident protection implementation.

P2 Prerequisites

P2 is a separate approved broker-by-broker change. It requires an account and instrument-specific capability proof, sandbox/manual canary, idempotent place/read-after-write/reconcile lifecycle, cancel-before-competing-sell protocol, and a failure-injection test suite. Robinhood has no verified stop-order adapter; Kite GTT has limit-child gap risk; Webull remains disabled pending its own sandbox.

Source: features\india-benchmark-fallback\FEATURE_ARCHITECTURE.md
India secondary-benchmark fallback

> Status: **DRAFT – scope only, awaiting owner approval. No code written.**
> Date: 2026-09-01. Influence if approved: **none on the money path.** Writes only
> `benchmark_price_observations`, which feeds the comparison chart and the
> scorecard. No scorer, sizing, exit, order or broker path reads it.

The problem, stated precisely

India secondary benchmarks have exactly one price source: Yahoo. Measured 2026-09-01, all three ETFs are missing the **2026-08-31** session while the India primary `^NSEI` has it:

benchmark	provider	missing sessions
NIFTY 50 (`^NSEI`)	yahoo	0
BANKBEES	yahoo	**2026-08-31**
ITBEES	yahoo	**2026-08-31**
JUNIORBEES	yahoo	**2026-08-31**

The book has a NAV for 2026-08-31, so the comparison chart drops that session from every India ETF comparison.

Why the US fix does NOT cover this

`pickFreshestProvider` (shipped 2026-09-01, `lib/data/benchmark-ingest.ts`) chooses **one provider's whole series** by which reaches furthest forward. That solves the US failure, which was *tail staleness* – XLK stuck eight days behind.

India's failure is a different shape: an **interior hole**. Yahoo is the freshest source (it has 09-01), so freshest-wins keeps Yahoo and the 08-31 gap survives. Adding a second India provider under the current mechanism would change nothing.

This needs a per-date merge, not a per-provider choice. Stating that plainly because it is the whole reason this is a separate piece of work rather than a config line.

What already exists (no new integration required)

- `fetchUpstoxCandles(symbol, days)` – `lib/data/upstox.ts:78`, working today.

```

- All three ETFs resolve in the cached instrument master:  
``BANKBEES -> NSE_EQ|INF204KB15I9`, `ITBEES -> NSE_EQ|INF204KB15V2`,  
`JUNIORBEES -> NSE_EQ|INF732E01045` (verified in `upstox_instruments`,  
2,714 rows, refreshed 2026-08-28).`
- Upstox already backs the India **primary**: ``paper_performance.bench_source``  
shows ``upstox+yahoo`` (32 rows) and ``upstox(yahoo_disagreed)`` (7 rows).
- The cross-check and disagreement vocabulary already exist for the primary.

So this is wiring an adapter that is already in production, to symbols it can already price.

## Two constraints that shape the design

**\*\*1. Upstox never returns the current session.\*\*** ``v3/historical-candle`` lags by one session by design – this is why ``confirmBenchmarkSessions`` and deferred confirmation exist for the primary. Upstox is therefore a **settled-bar backfill source**, not a freshness source. That is exactly the right shape for filling yesterday's hole, and exactly the wrong shape for a freshest-wins contest.

**\*\*2. The Upstox token is environment-based and intermittent.\*\***  
``lib/data/upstox.ts:20`` reads ``process.env.UPSTOX_ACCESS_TOKEN``; there is no vault entry. Production evidence of intermittency: 2 of 42 India primary rows fell back to ``yahoo(unconfirmed)`` (2026-08-28 and 2026-09-01) when Upstox was unavailable. The design must treat Upstox as **sometimes absent**, never as a guaranteed second opinion.

## Proposed design

Extend secondary-benchmark ingestion for India only:

1. Fetch Yahoo as today (primary source, keeps the current session).
2. Fetch Upstox for the same symbol.
3. **\*\*Merge per date.\*\*** Yahoo wins any date it has. Upstox fills dates Yahoo is missing. Each row keeps its own ``provider``, which the table already stores per row, so a merged series is fully attributable.
4. Where BOTH have a date, compare. If they disagree beyond a tolerance, keep Yahoo's value and mark the row ``source_status`` accordingly – reusing the existing disagreement vocabulary rather than inventing a second one. Do not silently prefer either.
5. If Upstox is unavailable (no token, throttled, empty), behave exactly as today and let the existing ``benchmark-stale`` alert speak.

The merge helper is pure and belongs beside ``pickFreshestProvider`` in ``lib/data/benchmark-ingest.ts``, unit-tested with the real 08-31 shape.

## What this deliberately does NOT do

- No change to the India **primary** benchmark path. It already has Upstox, a cross-check and deferred confirmation; touching it risks a money-path-adjacent series for no stated benefit.
- No US change. Freshest-wins is correct there and is now verified.
- No new provider, no new dependency, no new table, no migration.
- No attempt to make Upstox a freshness source. It structurally cannot be.

## Open questions for you

1. **\*\*Is it worth it?\*** These are **display comparators**. The cost of the gap is one missing session on a chart line, now visibly flagged rather than silent. The honest case for doing it is completeness of the comparison series; the honest case against is that nothing downstream consumes it and the same effort could go to a money-path item.
2. **\*\*Disagreement tolerance.\*\*** The primary uses an explicit percentage. Reuse the same number, or set one per ETF? I would reuse it and revisit only if it fires.
3. **\*\*Token durability.\*\*** Upstox tokens are short-lived. If refresh is manual, this fallback degrades to "works when someone refreshed the token", which is worth knowing before building rather than discovering later.

## Estimate

Small: one merge helper plus tests, one branch in ``upsertProviderObservations``, no schema change. The risk is not size – it is that a merged multi-provider series is harder to reason about than a single-source one, and the thing it fixes is currently one missing point on a comparison chart.

**\*\*My recommendation: defer.\*\*** The gap is now visible rather than silent, which was the actual defect. I would rather spend the next unit of work on something the book reads.

---

# Source: `features\india-markets-parity\FEATURE_ARCHITECTURE.md`  
# India Markets Verification And Hardening

> Status: **\*\*REVIEWED DESIGN DRAFT. NOT APPROVED FOR IMPLEMENTATION.\*\***  
> Last reviewed: 2026-07-15 by Codex.



> Scope: India Markets page correctness and resilience; no scoring or trading.

### ## 1. Correct Current-State Assessment

India Markets is implemented, but not yet production-proven:

- `MarketsPage` displays NIFTY 50, SENSEX, BANK NIFTY, India VIX, and ten sector-index tiles.
- `IndiaBreadth` fetches only the first ten fixed NIFTY-50 constituents. It is a sample, not NIFTY-50 market breadth.
- The client component imports `lib/india-data.ts` and calls Yahoo directly. Browser-side Yahoo access cannot rely on the server-only `User-Agent` header or Next.js revalidation semantics and bypasses Router pacing, durable caching, provenance, and health telemetry.
- `IndiaGapNote` conflates a transient India-source failure with a structural absence of an India equivalent.
- The page heading remains US-specific when India is selected.

The feature is therefore a verification and hardening project, not a greenfield parity build. Sector and breadth UI must not be built twice.

### ## 2. Invariants

1. The browser never calls Yahoo, NSE, RBI, Kite, Upstox, or a provider directly.
2. US/USD and India/INR snapshots, caches, labels, and health are isolated.
3. Missing or partial data is displayed as unavailable/partial, never zero or a fabricated neutral market state.
4. Markets remains advisory. No tile changes scores, sizing, orders, or exits.
5. An unofficial source is capability-probed, cached, paced, and fail-soft; it is never described as guaranteed, unlimited, or an authoritative NSE feed.
6. Structural product gaps and transient operational failures use different UI states and System Health codes.

### ## 3. Target Contract

Create one owner/cron-gated server boundary that returns a frozen India market snapshot. It must consume Canonical Evidence Router contracts or a temporary compatibility adapter that is explicitly retired at Router cutover.

```
type IndiaMarketsSnapshot = {
 market: "india";
 currency: "INR";
 asOf: string;
 fetchedAt: string;
 status: "complete" | "partial" | "unavailable";
 policyVersionId: string;
 indices: Array<{
 symbol: string;
 label: string;
 price: number;
 changePct: number;
 observedAt: string;
 source: string;
 quality: "fresh" | "stale";
 }>;
 sectors: Array<{
 symbol: string;
 label: string;
 price: number;
 changePct: number;
```

```

observedAt: string;

source: string;

quality: "fresh" | "stale";

}>;

breadth: null | {

 universeId: "nifty_50";

 universeAsOf: string;

 eligibleN: number;

 resolvedN: number;

 advanced: number;

 declined: number;

 unchanged: number;

 unavailable: number;

 coveragePct: number;

 quality: "complete" | "partial";

};

unavailable: Array<{ component: string; reasonCode: string }>;

};

```

The response contains no raw provider payload, token-bearing error, URL, cookie, or arbitrary prose. Numeric fields must be finite and currency/basis validated.

#### ## 4. Data Acquisition

- Use server-side, code-allowlisted adapters only.
- Prefer a fresh durable cache; serve bounded stale data with an explicit timestamp when allowed; enqueue bounded refresh work rather than burst on page load.
- Fetch index/sector symbols with bounded concurrency and atomic provider pacing.
- Use Kite/Upstox only when the owner's entitlement and contract allow it. Provider selection is policy-controlled, not hardcoded in the component.
- Yahoo is an unofficial fallback. Contract-test symbol semantics and adjustment basis; treat CORS, throttling, schema drift, and no-data as expected failures.
- Never scrape arbitrary NSE/RBI pages from a client or silently change source.

#### ## 5. Breadth Correctness

Choose one honest product:

1. **\*\*Full NIFTY-50 breadth:\*\*** use a versioned, current constituent snapshot; resolve all eligible names from a common observation window; show breadth only at a pre-approved coverage floor and report `resolvedN/eligibleN`.
2. **\*\*Ten-name sample:\*\*** retain current behavior but rename it "10-name NIFTY sample," disclose coverage, and never use the word breadth or compare it with full US breadth.

Recommended target is full NIFTY-50 breadth built asynchronously from cache, not 50 provider calls during page render. Constituents need effective dates so historical replay does not use today's index membership. Advance/decline compares the current session's valid reference close on a consistent adjustment basis; unchanged and unavailable names remain in the denominator report.

#### ## 6. Product States

- `loading`: bounded skeleton while the server request is active.
- `complete`: data plus `asof`/source-quality summary.
- `partial`: render available rows and explicit coverage; no healthy-looking total.
- `temporarily\_unavailable`: India capability exists but failed/staled out.
- `not\_supported`: no approved India equivalent exists.

The India page heading and explanatory copy name India instruments. Leveraged bull/bear pairs remain `not\_supported`; they must not be synthesized. TradingView embeds are optional third-party display surfaces, not a parity requirement or evidence.

#### ## 7. Macro And Regime Boundary

Do not build an “India recession sentinel” in this feature. RBI policy rate, CPI, WPI, IIP, yields, INR, and FII/DII flows differ in release cadence, revision, economic meaning, and source authority. FII/DII flow is not a macro regime proxy.

A future India macro architecture must define official series, publication and revision vintages, release calendar, staleness, deterministic regime math, and validation. Until then, show a structural gap. LLM prose cannot create regime math.

## ## 8. Verification Plan

1. Add server-adapter fixture tests for success, partial payload, throttling, schema drift, stale fallback, bad currency, and non-finite values.
2. Verify no provider hostname appears in the client bundle/network log.
3. Browser-test the actual context switch, persistence, refresh, failure, and stale states; `?market=india` is not the context contract unless separately designed.
4. Verify India requests never read/write US cache keys and vice versa.
5. Verify the page always exits loading after timeout/error.
6. Verify full breadth coverage arithmetic and constituent-version identity, or verify the ten-name product is consistently labeled as a sample.
7. Confirm desktop/mobile layout and market-specific heading with Playwright.

## ## 9. Build Order

1. Live-test the current market switch and capture current browser/network failures.
2. Add the server-side snapshot contract and cache compatibility layer.
3. Remove direct client imports/calls to `lib/india-data.ts`.
4. Correct headings, transient-vs-structural states, and timestamps.
5. Decide full breadth versus explicitly labeled sample; implement only one.
6. Add System Health aggregation by component/provider, not one alert per symbol.
7. Revisit optional India-only displays after source contracts are proven.

## ## 10. Rollback And Acceptance

Disable the India snapshot endpoint/flag and render an honest temporary-unavailable state; US remains untouched. Acceptance requires zero direct browser provider calls, reproducible market-local cache keys, bounded loading, explicit partial coverage, and no scoring/order imports. No Supabase migration or provider activation is authorized by this document until the design is owner-approved.

## ## 11. Implementation Notes (2026-07-15)

Built to this design. Files/routes:

- `lib/india-markets/adapter.ts` – server-only, allowlisted Yahoo chart adapter. Pure validate core `toAdapterQuote(symbol, FetchOutcome, now)` handles success / no\_data / throttled / http\_error / network\_error / bad\_currency (currency must be INR) / non\_finite. `fetchQuotes` runs bounded concurrency + pacing. Never called authoritative.
- `lib/india-markets/constituents.ts` – versioned NIFTY-50 snapshot (`universeId:"nifty\_50"`, `universeAsOf:"2026-07-01"`, `BREADTH\_COVERAGE\_FLOOR=0.8`).
- `lib/india-markets/snapshot.ts` – types + pure `buildSnapshot` / `computeBreadth`. Status: `unavailable` (0 index rows) / `complete` (all indices+sectors + breadth complete) / `partial` (anything else). Breadth counts only resolved names; unresolved stay in the denominator (`resolvedN/eligibleN`, `coveragePct`).
- `app/api/markets/india/route.ts` – the ONLY India market-data surface. `GET` = cache-only and never contacts Yahoo from a page request. `POST` = owner/cron-gated FULL fill incl. NIFTY-50 breadth through one deduplicated, paced stream; persistence failure returns an error instead of false success. System Health aggregates ONE issue by overall status (`india-markets-degraded`), not per-symbol.

Breadth choice: \*\*Option 1 (full NIFTY-50 breadth)\*\*, built asynchronously by the scheduled `POST` fill (not during page render), with a versioned constituent snapshot and 80% coverage floor. The GET path carries the last full breadth block forward and shows `temporarily\_unavailable` until the first fill runs.

Cache isolation: new India-only table `india\_market\_snapshot` (migration `20260715160000`), fully separate from US `price\_cache`. India data is never written to `price\_cache` and vice-versa – no cross-read is possible.

Cron: `20260715160500\_india\_market\_snapshot\_cron.sql` plus audit correction `20260715220000...` – post-close full fills at 10:15 / 10:35 UTC (15:30 IST close, no DST) via `kairos\_call\_agent` (injects `x-cron-secret`). Both migrations applied to the target DB and verified.

Client (`components/dashboard/MarketsPage.tsx`): removed all direct `lib/india-data` Yahoo calls; India path fetches `/api/markets/india` with a 15s client abort so loading always exits. Product states: loading skeleton, complete/partial (available rows + explicit coverage), `TempUnavailableNote` (transient), `NotSupportedNote` (structural – leveraged pairs / macro sentinel / TradingView, never synthesized). Market-aware heading names India instruments. Verified: no provider hostname in the client static bundle; the markets page chunk references `api/markets/india`. Gates: `tsc`, `vitest` (423 pass), `next build` all green.

---

# Source: features\instrument-aware-scoring\FEATURE\_ARCHITECTURE.md

## # Instrument-Aware Research and Scoring

```
Status: Approved; P0/P1 measurement implemented; 2026-09-15 gold evidence repair + oil exposure pack (measure-only, $11)
Owner: Vaibhav
Architect/Builder: Codex / GPT-5
Date: 2026-08-24
```

### ## 1. Problem

Kairos recognizes some funds and metals, but the actionable v1 scorer still renormalizes one market-level five-dimension policy and caps every ETF-like score at 65. This makes structurally different exposures look interchangeable: gold bullion, silver bullion and a gold-miner equity fund can all become an eligible 65. India symbols are also classified as India equities before fund subtype is known.

The system must model an economic exposure before choosing a tradable vehicle. It must not create a bespoke model per ticker: that would overfit. The learning unit is 'market × instrument\_family × setup × horizon' and promotion requires independent forward evidence.

### ## 2. Non-negotiable invariants

- Existing v1 score, paper selection and every live-money path remain unchanged during measurement. New fields are 'measure\_only' and cannot authorize trades.
- New positions remain long-only. Holdings exits are never suppressed.
- LLMs may explain evidence or propose a challenger; they cannot mutate policies, limits, scores, code or orders.
- US and India never share promotion evidence.
- Missing, stale, disputed or unclassified evidence is unavailable, never neutral.
- Multiple runs for one symbol/session count once in evaluation.
- Near-substitute vehicles (GLD/IAU) represent one exposure, not two independent alpha ideas or samples.
- Append-only ledgers are never updated or deleted.

### ## 3. Canonical taxonomy

The initial families are 'operating\_company', 'adr', 'bank', 'reit', 'broad\_equity\_etf', 'sector\_etf', 'thematic\_etf', 'fixed\_income\_etf', 'gold\_bullion\_fund', 'silver\_bullion\_fund', 'gold\_miners\_fund', 'metal\_producer\_equity', 'royalty\_streaming\_equity', 'india\_etf', 'leveraged\_or\_inverse\_etf', and 'unknown'.

Initial exposure identities include 'gold\_spot', 'silver\_spot', 'gold\_miners', 'us\_broad\_equity', 'us\_sector:', 'us\_rates:', and 'india\_index:'. Classification is deterministic, version-stamped and persisted with the immutable decision. Unknown classification cannot silently inherit a special model.

GDX/GDXJ are miner-company funds, not bullion. KGC/NEM/AEM/GOLD are producers. FNV/WPM/RGLD are royalty/streaming companies. GLD/IAU are substitute gold vehicles. India fund symbols such as GOLDBEES.NS and LIQUIDBEES.NS are funds, not operating companies.

### ## 4. Evidence packs

Shared completed-session features (trend, participation and volatility) remain available where applicable. The first measure-only family pack records:

- gold: real-yield change, broad-dollar change, gold return and technical state;
- silver: gold inputs plus silver return and silver-minus-gold relative return;
- miners: gold return, miner return, miner-minus-gold return and technical state;
- funds: benchmark/exposure identity and later qualified spread, premium/discount, tracking difference, expense and concentration evidence;
- producer/streamer equities: company evidence remains applicable; commodity sensitivity and qualified operational evidence are additive challengers.

FRED series 'DFI110' and 'DTWEXBGS' are official/free. Price-derived values use the settled 'price\_cache' contract. Every field carries source, as-of date, freshness and status. No family composite becomes actionable in this phase.

### ## 5. Discovery and selection

Discovery produces exposure candidates. Vehicle selection happens later:

1. detect 'gold\_spot' opportunity;
2. evaluate the gold evidence pack;
3. if a validated policy is eligible, select one permitted vehicle (for example GLD or IAU) using liquidity, spread, tracking and account support;
4. apply portfolio exposure, correlation and money gates.

The existing always-on metal basket stays for evidence continuity, but duplicate vehicles cannot count as independent opportunities or promotion samples.

### ## 6. Evaluation and promotion

Diagnostics use the last frozen observation per symbol and market session. They

report independent sessions, usable 5/10/20-session labels, score variance, cap saturation and excess returns versus the family benchmark. Overlapping horizons require Newey-West or block-bootstrap uncertainty. A zero-variance/cap-saturated score has no valid IC.

Promotion is separate per market/family/setup. Minimum starting gate: 60 independent sessions, at least 30 clean out-of-sample observations, non-degenerate predictions, net-of-cost positive evidence, acceptable calibration and explicit owner approval. These are floors, not proof of profitability.

#### ## 7. Rollout

1. **\*\*P0 – taxonomy and diagnostics:\*\*** persist versioned family/exposure context; report current cap saturation and label coverage. No score change.
2. **\*\*P1 – feature collection:\*\*** collect family evidence on every qualifying decision. No score change.
3. **\*\*P2 – challenger evaluation:\*\*** build family-local shadow policies and correct benchmarks. No paper/live change.
4. **\*\*P3 – isolated exploratory paper:\*\*** one exposure-level candidate and dedicated risk sleeve after P2 passes.
5. **\*\*P4 – active paper:\*\*** owner promotion; v1 remains rollback champion.
6. **\*\*P5 – live eligibility:\*\*** requires separate canary acceptance and owner click.

#### ## 8. Acceptance criteria

- GLD/IAU resolve to `gold\_bullion\_fund` + `gold\_spot`; SLV to silver; GDX to `gold\_miners\_fund`; Indian known ETFs never resolve to `operating\_company`.
- Decision observations expose taxonomy version, family, exposure and measure-only evidence with timestamps/status.
- Diagnostics deduplicate same-session runs and explicitly reject IC when score variance or clean labels are insufficient.
- No new table, route or field is read by paper/live execution.
- Golden taxonomy/evidence tests, full Vitest, production build, migration proof, RLS/security advisors and end-to-end production read verification pass.

#### ## 9. Explicitly deferred

Copper, uranium, agriculture, futures/options, leveraged/inverse funds, intraday models, proprietary DXY, and autonomous promotion are separate designs. Oil moved out of this list on 2026-09-15 as a measure-only exposure pack only (\$11); oil scoring, eligibility and trading remain deferred. No performance claim is made until forward evidence exists.

#### ## 10. Implemented baseline (2026-08-24)

P0/P1 measurement is shipped: deterministic taxonomy, immutable family evidence, an owner-only bounded diagnostics route, uncapped family shadow decisions, and Research Journal disclosure. Migrations `20260824183930` and `20260824185010` and hardening migration `20260824190302` are applied and verified in the FinanceOS project. The second migration backfills only deterministic taxonomy; it deliberately does not reconstruct historical feature values.

The initial production audit confirms why promotion is blocked. After collapsing same-symbol session duplicates, gold bullion has 58 symbol-sessions but only 37 independent exposure-sessions; gold-miner and silver funds each have 23. Cap-at-65 rates are about 47% for bullion and 61% for miner/silver funds. India has no qualifying historical curated-fund cohort in this ledger yet. These are diagnostic facts, not evidence of edge; every family remains below the 60-exposure-session floor and the new features remain non-actionable.

Delivered behavior and verified deviations are recorded in `features/instrument-aware-scoring/IMPLEMENTATION\_RESULT.md`.

#### ## 11. Gold evidence repair and oil exposure pack (2026-09-15)

Owner-approved ("approved, do 1-3") after an audit prompted by oil and gold moves. Everything here is measure-only: no score, eligibility, sizing, paper, live, exit or broker path reads it.

##### ### 11.1 Gold evidence audit – what was actually wrong

Production evidence (FinanceOS project, 2026-09-15):

- **\*\*Not a defect:\*\*** 100 of 168 `gold\_bullion\_fund` rows have no features. All of them are `taxonomy-backfill.v1` rows before 2026-08-25, which \$10 already says do not reconstruct history. Every live row since 2026-08-25 carries features.
- **\*\*Defect 1, mislabeled price window:\*\*** `returnPct` returned first-to-last over every cached row the shared query returned (~100 rows per symbol, back to 2026-04-21), not 20 bars. As of 2026-09-11 GLD recorded -7.17% when the real 20-bar return was -0.05%; SLV -15.14% vs -0.07%; GDX +4.42% vs +10.00%. Fixed to exactly the last 20 settled bars; `INSTRUMENT\_FEATURE\_VERSION` bumped to `instrument-family-features.v2`. v1 rows are immutable and must be evaluated separately from v2.
- **\*\*Defect 2, frozen FRED inputs:\*\*** `av\_cache` served DFII10 observation

2026-08-31 for every cache day 2026-09-02..09-11 and DTWEXBGS 2026-09-04 for 09-12..09-15, while FRED itself had 2026-09-11 for both. Production FRED calls failed and `providerCachedFetch` carry-forward rewrote the old payload into each new day's slot, so the 7-day fallback bound never expired. The shared-cache root cause is tracked as a separate task because it affects every today-only provider. In scope here: per-series staleness (DFII10 7d, DTWEXBGS 10d because it is released weekly, crude 10d) and a System Health `warn` (`family-evidence-fred-stale:<series>`) instead of a silent stale label.

Correction to an earlier chat claim: metals do not fail to trade because of `scoreMode=measure\_only` – no paper path reads score mode. They can enter at the mandate threshold (52), but `ETF\_SCORE\_CAP=65` keeps them below equity candidates that score 66+.

### ### 11.2 Oil exposure pack

`loadOilExposureEvidence` (same module, same day-cached inputs) attaches `decision\_observations.features.oil\_exposure\_evidence` for curated symbols only:

| Market | Class                   | Symbols                       |
|--------|-------------------------|-------------------------------|
| US     | crude_oil_fund          | USO, BNO                      |
| US     | energy_sector_fund      | XLE, XOP                      |
| US     | upstream_producer       | OXY, EOG, COP, DVN, FANG, APA |
| US     | integrated_major        | XOM, CVX                      |
| US     | refiner                 | MPC, PSX, VLO                 |
| US     | oilfield_services       | SLB, HAL, BKR                 |
| India  | upstream_producer       | ONGC.NS, OIL.NS               |
| India  | downstream_marketer     | BPCL.NS, IOC.NS, HINDPETRO.NS |
| India  | integrated_conglomerate | RELIANCE.NS                   |

Features: FRED `DCOILWTIC0` and `DCOILBRENTU` 20-observation % change, Brent 5-observation % change, and settled USO 5/20-bar returns (FRED spot crude lagged 6 days on 2026-09-15, so USO is the same-week proxy). Unknown symbols get nothing – no sector-name heuristic. No expected sign is stored (see §11.3). Deviation from `features/exogenous-risk-evidence` P2: FRED republishes EIA spot prices under the existing key, so no new provider; the PPAC Indian Basket stays deferred. The Research Journal funnel shows the pack as an "Oil exposure · measurement only" panel next to the family-evidence panel (display only).

### ### 11.3 Frozen counterfactual – does the oil-sign hypothesis hold?

Read-only, no historical decision rewritten. Last observation per symbol-session, joined to `observation\_labels.benchmark\_neutral\_return`. Crude = USO settled close: trailing 10 bars strictly before the decision date; forward window approximated as `horizon\_days × 1.4` calendar days. Evidence is 2026-07-07..2026-09-02.

| Group                       | h  | n (sessions) | corr(crude fwd, excess)                  | corr(crude trailing 10, excess) | excess when crude up / down |
|-----------------------------|----|--------------|------------------------------------------|---------------------------------|-----------------------------|
| india_omc (BPCL, IOC)       | 5  | 33 (19)      | +0.11   -0.14   -0.06% (29) / -0.42% (4) |                                 |                             |
| india_omc                   | 10 | 33 (19)      | +0.54   -0.39   -0.44% (29) / +2.38% (4) |                                 |                             |
| india_upstream (ONGC)       | 10 | 16 (16)      | +0.86   -0.89   -4.51% (12) / +0.66% (4) |                                 |                             |
| india_integrated (RELIANCE) | 10 | 7 (7)        | -0.06   -0.18   -0.45% (4) / -0.20% (3)  |                                 |                             |
| us_upstream                 | 10 | 19 (19)      | +0.94   -0.67   -0.50% (17) / +8.64% (2) |                                 |                             |
| us_integrated               | 10 | 20 (20)      | +0.90   -0.58   -0.26% (18) / +7.02% (2) |                                 |                             |
| us_refiner                  | 5  | 8 (6)        | +0.83   -0.51   +5.82% (3) / +6.74% (5)  |                                 |                             |

Findings, stated at the strength the data allows:

1. US energy and India upstream move with crude over the same window, as expected.
2. \*\*The "OMCs are hurt when crude rises" sign is not supported.\*\* BPCL/IOC excess returns co-moved weakly positively with crude. The earlier chat warning was textbook reasoning, not evidence, and is withdrawn as a rule candidate. BPCL's paper losses came from 11 re-entries in three weeks closed by time stops, not from benchmark underperformance (BPCL 5-day excess was +0.37%, 15/22 positive).
3. Every group shows negative correlation between the trailing crude run-up and the next window's excess return: entries made after crude rallies lagged. This is one regime (crude +34% in July then -14% in late July), overlapping windows, 2-3 symbols per group, and a US-listed crude proxy on Indian dates. It is a hypothesis for the new pack to test forward, not a rule.

Gate before any oil feature becomes actionable: the \$6 floors per market and exposure class (60 independent sessions, 30 clean out-of-sample observations), measured on v2 evidence with Newey-West or block-bootstrap uncertainty, plus owner approval.

---

# Source: features\instrument-aware-scoring\IMPLEMENTATION\_RESULT.md  
# Instrument-Aware Scoring – Implementation Result

\*\*Implemented by:\*\* Codex / GPT-5  
\*\*Date:\*\* 2026-08-24  
\*\*Commit:\*\* `b2c226d1` plus documentation/ledger hardening follow-up  
\*\*State:\*\* P0/P1 live in measurement; P2-P5 evidence-gated

## Shipped

- `instrument-taxonomy.v1` deterministically separates operating companies, ADRs, banks, REITs, broad/sector/thematic/fixed-income funds, bullion funds, miner funds, metal producers, royalty/streaming companies, known India ETFs, leveraged/inverse funds, and unknowns.
- ResearchAgent stores family, exposure, benchmark, taxonomy version and score mode inside the immutable decision snapshot.
- Metal-family measurements record dated FRED real-yield/dollar inputs and settled GLD/SLV/GDX return relationships. They contribute zero score points.
- `instrument\_family\_observations` is an append-only measurement ledger. RLS is enabled; browser writes are denied; UPDATE, DELETE and TRUNCATE are blocked by grants/triggers while service-role INSERT remains available.
- Curated historical metal/fund rows were taxonomy-backfilled without inventing unavailable feature values.
- The owner-only diagnostics route collapses repeated symbol-session rows and substitute vehicles to independent exposure-sessions before readiness counts.
- The Research Journal shows the observed family/exposure and timestamped family evidence, explicitly separated from the actionable v1 score.
- An uncapped v1 family shadow row records whether the universal ETF cap destroys rank information. It cannot create a fill or modify policy.

#### ## Verified production baseline

The backfilled US cohort has 37 independent gold-bullion exposure-sessions, 23 gold-miner-fund sessions and 23 silver-fund sessions. Cap-at-65 saturation is approximately 47%, 61% and 61% respectively. India has no qualifying historical curated ETF cohort in the new ledger. Every family therefore remains below the 60 independent-exposure-session floor.

#### ## Approved deviations and deferrals

- No family composite was invented before feature-level forward evidence exists.
- No v1 score, threshold, paper selection, sizing, exit or live-money behavior changed.
- Exposure-level vehicle selection, family-local walk-forward evaluation, exploratory paper sleeves and promotion remain P2-P5. They cannot be declared complete merely because their code could be written; the forward observations and owner decision are part of the acceptance contract.

#### ## Verification

- Focused tests: 14 passed.
- Full suite: 2,044 passed; 7 skipped.
- Next.js production build: passed.
- Supabase migrations `20260824183930`, `20260824185010`, and `20260824190302`: applied and directly verified.
- Production Vercel deployment: Ready.

---

# Source: features\international-equity-allocation\FEATURE\_ARCHITECTURE.md  
# International Equity Exposure Architecture

```
> Status: **P0-P2A SHIPPED (read-only). P2 comparative study and P3-P4 remain unapproved.**
> Date: 2026-07-27
> P0-P2A shipped: current US paper-book visibility plus an append-only VXUS
> policy, issuer-mandate snapshot, and weekly operational-shadow assessment. It
> has no target, provider runtime, allocation, paper-trading, or live-trading
> effect. Comparative P2 and P3-P4 require separate approval.
```

#### ## P0 Implementation (2026-07-27)

- `lib/allocation/international-exposure.ts` calculates a current US paper-book view from existing persisted paper marks only. It uses a deliberately narrow, static country-ETF map; other ETFs remain unavailable rather than inferred.
- `components/dashboard/InternationalExposurePanel.tsx` renders the read-only Portfolio panel with a disabled/no-action state, recognized-country exposure, unclassified-ETF disclosure, and cost-mark disclosure.
- The panel is hidden on the India view. It cannot read India holdings or mix USD and INR. No position, policy, instrument registry, provider ledger, or money-path writer was added.

#### ## P1 Implementation (2026-07-27)

- Migration `20260727100000\_international\_allocation\_p1.sql` added service-only `international\_allocation\_policies`, append-only `fund\_exposure\_snapshots`, and append-only `international\_allocation\_assessments`. Browser roles have no table grants and cannot execute the refresh RPC.
- The sole policy is `us\_non\_us\_broad\_core\_v1`: `market=us`, `core=VXUS`, `construction=broad\_core`, `status=observe`, and `\*\*null target/band\*\*`. The observed `instrument\_registry` row remains `new\_entry\_allowed=false`.
- The first issuer snapshot preserves Vanguard's broad developed/emerging ex-US mandate and source URL. It is deliberately `partial`: it does not claim a point-in-time country-weight breakdown.
- The owner/cron-gated assessment route can append a current US paper-book observation only. It calls no provider and cannot generate a candidate,

alter allocation, or place an order. P2A weekly scheduling is now active; comparative P2 remains unapproved.

## ## P2A Implementation (2026-07-27)

- `kairos-international-allocation-shadow` runs once weekly at Monday 03:30 UTC through the existing protected Vercel transport.
- Each P2A row records a no-action proposal, explicitly null costs/tax drag, source coverage state, snapshot/position fingerprints, and one shadow week. A partial index-mandate snapshot or unset target can only suppress action.
- This is an operational evidence track for the approved VXUS policy. It does **not** compare no-international, VXUS, and VEA+VWO strategies yet: the latter two require separately approved policies and point-in-time source snapshots. Country satellites remain out of scope.

## ## 1. Decision

Kairos should eventually support **non-US equity exposure inside the US/USD book** through reviewed, US-listed international ETFs. It must not create an `international` market, a third cash pool, or an automatic country-ETF trading strategy.

This is strategic geographic diversification, not a short-horizon alpha claim. The first product is a read-only allocation decision surface that answers:

1. How much of the US equity sleeve is already exposed outside the US?
2. Is the portfolio materially concentrated in US geography, sectors, or common global revenue sources?
3. Does the owner's approved target call for a broad non-US core, and is the actual exposure outside its allowed band?
4. Is a proposed fund a broad core, a complementary developed/emerging sleeve, or a concentrated country satellite that duplicates existing exposure?

The current allocation core remains off. This proposal does not change `allocation\_enabled`, score a symbol, add watchlist candidates, consume a new provider quota, place a paper trade, or place a broker order.

## ## 2. Why This Is the Right Boundary

A US broker account can obtain international equity exposure through US-listed funds without operating a foreign-market execution system. A broad vehicle such as [VXUS](https://investor.vanguard.com/investment-products/etfs/profile/vxus) tracks a developed-and-emerging-markets index excluding the United States. That is materially different from a single-country vehicle such as [INDA](https://www.ishares.com/us/products/239659/INDA), which the issuer describes as a targeted India exposure and currently has a large financials weight. Broad geographic diversification can reduce home-country concentration, but it is not a promise of higher return or a signal to tactically rotate every week. [Vanguard's research](https://corporate.vanguard.com/content/corporatesite/us/en/corp/articles/making-case-international-equity-allocation) also frames the benefit as diversification and volatility reduction, not a market-timing rule.

The existing India pipeline is a separate INR book. It has its own market hours, research, paper pool, risk controls, and benchmark. A US-listed India ETF is a USD instrument with India geographic exposure; it belongs in the US book if ever approved. It must not be combined with, rebalance against, or treated as a substitute for India-pipeline holdings.

## ## 3. Non-Negotiable Invariants

1. **Two market pools only.** A US-listed international ETF has `market=us`, `currency=USD`; the underlying geography is metadata. India remains `market=india`, `currency=INR`. No US/India NAV, P&L, cash, cap, performance, or exposure percentage is cross-summed.
2. **Strategic sleeve, not a generic candidate.** Broad and country ETFs do not enter the ordinary 2-20 day ResearchAgent queue or compete with single-name scores. They use a slow allocation/rebalance process only after approval.
3. **One core construction at a time.** The system may hold either one broad ex-US core, or a deliberate developed-plus-emerging split. It cannot call overlapping cores diversification.
4. **Country funds are satellites.** A single-country ETF never satisfies the international-core target. Its geographic, sector, issuer, and existing-book overlap must be shown before it can be proposed.
5. **No US-macro proxy.** Existing US MacroSentinel cannot decide whether an international ETF is attractive. Future domestic/global evidence is read-only until it demonstrates incremental, point-in-time value by fund archetype.
6. **No LLM decision authority.** An LLM may explain persisted allocation facts; it cannot choose a fund, set a target, determine overlap, or trigger an order.
7. **Default deny.** An unregistered ETF, stale fund facts, unavailable geographic look-through, or ambiguous classification produces `unavailable` and no proposal, never a neutral or buy recommendation.
8. **Existing brakes remain authoritative.** Any future paper/live action still passes app pause, market controls, kill switch, drawdown breaker, market



session, account scope, mandate, caps, idempotency, and live approval gates.

## ## 4. Exposure Model

### ### 4.1 Portfolio hierarchy

#### US/USD book

- > equity sleeve
- > domestic US equity exposure
- > international-core sleeve (optional, slow allocation)
- > broad ex-US OR developed + emerging split
- > country satellite sleeve (optional, smaller and independently capped)

#### India/INR book

- > unchanged India equity/defensive/cash sleeves

`international\_core` and `country\_satellite` are US-book sleeve labels, not markets. They extend the existing `strategy\_sleeves` concept only after an approved migration and must retain its per-market key.

### ### 4.2 Permitted construction patterns

| Pattern                                             | Intended use                                                                     | Initial status                               |
|-----------------------------------------------------|----------------------------------------------------------------------------------|----------------------------------------------|
| One broad ex-US core, e.g. VXUS                     | Default diversified international exposure across developed and emerging markets | Candidate for P0 measure                     |
| Developed ex-US + emerging split, e.g. VEA + VWO    | Owner deliberately wants a controlled developed/emerging mix                     | Candidate for P0 measure                     |
| Single-country, e.g. INDA                           | Explicit small satellite hypothesis or owner-directed concentration              | Observe only; not a core and not an ordinary |
| Multiple India country ETFs, e.g. INDA + EPI + INDY | Apparent variety with heavy geographic overlap                                   | Prohibited without look-through proof        |

The product must not preselect tickers as an investment recommendation. The issuer's current fund facts, expense ratio, liquidity, holdings, index method, and broker fractional eligibility are data to verify at proposal time, not permanent constants in source code.

### ### 4.3 Target and band semantics

The owner selects a target as a percentage of the **US equity sleeve**, not of total personal net worth, because Kairos does not have a complete, reconciled view of every external asset and liability. It shows both denominators clearly:

- `US book NAV`: useful for execution capacity only.
- `US equity sleeve`: the only denominator for this allocation policy.
- `Known Kairos portfolio`: informational only; never represented as total wealth.

No default percentage is set in code. The initial UI asks for a target and rebalancing band only after P0 data has exposed the current geographic concentration. A target of zero remains the shipped default.

Bands create **slow** actions:

- below lower band: a future proposal may buy the selected core;
- inside band: hold, regardless of recent relative performance;
- above upper band: a future proposal may stop new core buys or propose a slow rebalance, subject to taxes, costs, and approval;
- unavailable/stale input: hold and surface the gap.

No band breach is an immediate sell signal and no international allocation rebalance can override a protective position exit.

## ## 5. What Determines Where Exposure Belongs

### ### 5.1 Core decision matrix

| Input                              | What it answers                                                                               | Allowed initial effect                                |
|------------------------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------|
| Current fund look-through          | Country, region, sector, and issuer overlap across held ETFs                                  | Display and duplicate-exposure warning                |
| Current US-book holdings           | Concentration in US/foreign revenue, sectors, and names when reliable tags exist              | Display only; unknown exposures                       |
| Owner target and band              | Desired strategic non-US share of US equity sleeve                                            | Display a deterministic below/inside/above-band state |
| Fund eligibility                   | Registration, exchange, currency, instrument archetype, liquidity, fee, and stale-fact checks | Refuse proposal when inco                             |
| Tax and account constraints        | Whether a sale/rebalance could be uneconomic                                                  | Refuse or require explicit owner approval             |
| Domestic/global exogenous evidence | Global risk context and country-specific stress                                               | Context only in P0-P2; never a timing score           |

### ### 5.2 Geographic look-through rules

1. Store fund composition snapshots with `as\_of`, source URL, retrieval time, holdings/fund-facts fingerprint, and coverage percent. Never silently use today's holdings for a historical allocation decision.
2. Report `known`, `unknown`, and `overlap` portions separately. Do not

- renormalize known holdings to 100% when coverage is incomplete.
- 3. Use country/region weights first. Sector/issuer overlap is supplementary and must be constrained by source coverage.
- 4. For a country satellite, show overlap with the India/USD ETF sleeve and any directly held Indian ADR/US-listed exposure. It does not read India/INR position data into US sizing.
- 5. A fund-of-funds, derivative, or opaque product needs explicit classification; it is not inferred from its ticker or marketing name.

## ## 6. Data and Architecture Fit

### ### 6.1 Reuse, do not create a parallel truth layer

The build must extend these existing contracts:

| Existing component                                        | Required use                                                                                             |
|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Instrument Registry                                       | Reviewed archetype, market, currency, execution eligibility, and default-deny behavior                   |
| Strategy Sleeves                                          | Eventual US-book target/band configuration; no global singleton setting                                  |
| Existing live/paper position and NAV projections          | Current US-book exposure inputs, preserving account scope and currency                                   |
| Evidence Records, Provider Call Ledger, Evidence Cache v2 | Provenance for all externally sourced fund and risk facts                                                |
| Symbol Daily Returns                                      | Later frozen, same-market measurement only; no current-price backfill for historical claims              |
| Features/Exogenous-Risk-Evidence                          | Future domestic/global context source; remains observation/shadow-only until independently admitted      |
| Existing allocation core                                  | Eventual deterministic band calculation, still off until its current sizing/rebalancer work is completed |

New records, if P1 is approved, should be a versioned `fund_exposure_snapshot` and an immutable `allocation_assessment`, each linked to the above evidence fingerprints. They must not replicate prices, positions, evidence cache, or broker account data.

### ### 6.2 Required instrument archetypes

| Archetype                      | Market/currency | Research path                 | Allocation role                              |
|--------------------------------|-----------------|-------------------------------|----------------------------------------------|
| International Equity Core      | US/USD          | Excluded from generic scoring | Optional strategic core                      |
| International Equity Developed | US/USD          | Excluded from generic scoring | Optional complementary split only            |
| International Equity Emerging  | US/USD          | Excluded from generic scoring | Optional complementary split only            |
| Country Equity Satellite       | US/USD          | Excluded from generic scoring | Optional capped satellite, never core        |
| India Equity                   | India/INR       | Existing India path           | Unchanged; not a substitute or counterweight |

## ## 7. Phased Build and Timing

Calendar estimates are engineering ranges, not evidence promises. A phase does not advance just because its dates have elapsed.

### ### P0 - Inventory and read-only decision surface (2-3 engineering days)

1. Add an owner-reviewed starter policy catalog with archetypes and no enabled instruments. Do not seed an allocation target.
2. Inventory existing US-book ETF/ADR exposure and static overlap warnings.
3. Build a Portfolio > Allocation read-only section: US book separately, selected construction status, known/unknown geographic coverage, duplicate warnings, target-band state, data freshness, and 'no action' state.
4. Prove that no item enters ResearchAgent, ThemeScout, PaperTrader, PositionMonitor, capital rotation, or live trader.

Acceptance: with no policy selected the surface states that international allocation is disabled; it neither fetches a new market-data feed nor changes the research backlog or provider-call ledger.

### ### P1 - Authoritative look-through and policy records (3-5 engineering days)

1. Add source-backed fund exposure snapshots and the append-only allocation assessment record.
2. Enforce one core construction and prohibit undocumented overlap.
3. Add fund eligibility/staleness/cost/liquidity checks and account/tax warning states.
4. Apply RLS and service-only writer protections; verify the target production schema and client denial before claiming completion.

Acceptance: every displayed country allocation has an as-of source snapshot, coverage percentage, and immutable evidence fingerprint. A stale or partial snapshot yields 'unavailable' and no proposed action.

### ### P2 - Paper allocation shadow (minimum 8-12 weeks of observations)

1. Recalculate target-band decisions on a slow cadence (weekly at most) without creating paper positions or consuming broker/provider execution quota.
2. Record proposed action, suppressed action, costs, coverage, and exact input fingerprints.
3. Compare three mutually exclusive policy families: no non-US sleeve, broad core, and developed/emerging split. Country satellites are evaluated as a separate hypothesis, not merged into the core result.
4. Measure turnover, costs, dividend/tax assumptions, drawdown, concentration, and exposure stability. No retrospective current-holdings reconstruction.

Acceptance: zero cross-currency records, zero generic ResearchAgent entries, and reproducible weekly proposals. Operational evidence is healthy for at least 8-12 weeks; long-horizon performance claims remain unproven.

### P2B - Historical static-allocation diagnostic (implemented, not a promotion gate)

1. Backfill only `V00` and `VXUS` adjusted-close history through two bounded, cache-only Yahoo chart requests. Massive's production entitlement returned only about two years of archive data, below the required history floor. This does not add either symbol to Markets tiles, research, scoring, or execution.
2. Compare one predeclared US/USD policy pair only: `100% V00` against `80% V00 / 20% VXUS`, monthly rebalanced at the matched-session close with a recorded 5 bps one-way cost assumption. The 20% is a diagnostic constant, not an owner target or recommendation.
3. Require at least three years of matched sessions. Until then, persist and display `insufficient\_history`; never fill missing dates from current prices or a provider call in the replay request.
4. Persist the exact cache/configuration fingerprint and result in a service-only append-only replay ledger. Show return, volatility, drawdown, information ratio, cost drag, and three contiguous reporting windows, but never designate a winner or alter policy state.

Acceptance: the replay makes zero provider calls, produces no paper/live order, does not mutate a target/band/status, and clearly says it is not a reconstruction of the Kairos paper/live book. It is useful historical context only; PIT fund composition, distributions, taxes, and a separately predeclared owner target are still required before a performance-oriented promotion.

### P3 - Paper execution, only after P2 and allocation-core completion

1. Finish the already-deferred allocation sizing/rebalancer work with a US-only international sleeve and a band/deadband.
2. Require a separate paper-only feature approval and a tax/cost-safe proposal ledger. Rotation cannot use an international sleeve to bypass its own gates.
3. Restrict actions to the selected core construction and owner cap. Country satellites remain disabled unless their separate study passes.

Acceptance: an action is serialized per US pool, idempotent, gate-complete, and cannot force-close a single-name position or override a stop/target.

### P4 - Live consideration (no fixed date)

Requires all of: explicit owner approval, paper evidence, longer-horizon historical/PIT evaluation, broker and fractional-share eligibility, tax review, and a separate live-order architecture review. It must start in manual approval mode. No inference from the presence of a Webull or Robinhood connection is permission to enable it.

## ## 8. Validation Standard

This architecture deliberately does **not** promise that international exposure will outperform US equities. The validation question is whether a chosen policy delivers the stated diversification objective after realistic implementation costs, without creating unacceptable concentration or churn.

Before a performance-oriented promotion, require:

1. Point-in-time fund composition and return inputs, including delistings, distributions, fees, rebalances, and corporate actions as applicable.
2. Walk-forward comparisons against the policy's declared baseline, with no target tuning on the test window.
3. A long enough span to include distinct currency/rate/regime periods. An 8-12 week shadow proves operations, not strategic allocation quality.
4. Costs, spreads, taxes, and account constraints applied before any claimed benefit.
5. Market-local reporting: US allocation results in USD and against its selected US-book benchmark; India results, if separately designed later, in INR and against NIFTY. Never create an aggregate alpha number.

Until then, global risk evidence and relative strength can explain context only; they cannot time entry or exit from the international sleeve.

## ## 9. UI Contract

The eventual Portfolio > Allocation section is a dense operational surface, not a recommendation feed. It shows:

- US-book current/target/band values and the explicit denominator;
- selected construction (`disabled`, `broad core`, or `developed + emerging`);
- country/region weights with known/unknown coverage;
- detected duplicate exposure and country-satellite caps;
- source as-of time, quality, and missing-data reason;
- current policy result: `disabled`, `hold`, `below band`, `above band`, or `unavailable`;
- a full audit trail of inputs and hypothetical actions.

It must visibly distinguish an educational allocation observation from an approved paper proposal and from a broker order. No buy/sell button appears in P0-P2.

### ## 10. Explicit Non-Goals

- No direct foreign exchange or foreign-broker trading.
- No third market or pseudo-currency pool.
- No use of India paper/live cash to fund a US-listed India ETF.
- No automatic country rotation, momentum chasing, or weekly macro timing.
- No addition of VXUS, VEA, VWO, INDA, EPI, INDY, or similar funds to the generic stock watchlist merely because they are available.
- No use of an LLM, TradingView, social/news sentiment, or a chart pattern to choose geographic allocation.
- No live allocation change, provider quota expansion, or API key requirement in P0-P2.

### ## 11. Owner Decisions Needed Before P1

1. Is the first objective broad diversification only, or is a tactical country satellite allowed at all? Recommendation: broad diversification only.
2. What is the desired non-US share of the US **equity sleeve** and allowed deadband? Recommendation: choose after P0 displays actual look-through; default remains zero until explicitly set.
3. Is the permitted construction one broad core, or a developed/emerging split? Recommendation: one broad core initially because it has the least overlap and operating complexity.
4. Are any US-book accounts tax-sensitive enough that a rebalance must always be manual? Recommendation: yes by default until tax-lot and account-policy facts are fully modeled.
5. Should any country satellite be evaluated later? Recommendation: defer it until the broad-core policy and look-through data are operating correctly.

### ## 12. Recommended Next Step

Approve **P0 only**. It is the smallest useful answer to “when, how, and where”: show existing exposure, select no more than one construction, make overlap visible, and keep all trading paths unchanged. P1-P4 remain gated by P0 evidence and separate approvals.

---

# Source: features\known-anomalies\FEATURE\_ARCHITECTURE.md  
# Known-Anomaly Research Backlog

> Status: **REVIEWED DESIGN DRAFT. NOT APPROVED FOR IMPLEMENTATION.**  
> Last reviewed: 2026-07-15 by Codex.  
> Scope: measure-only anomaly research. No score, rank, sizing, order, or exit effect.  
> Market applicability: US feasibility first. India is unavailable until an  
> independently validated point-in-time event and expectation contract exists.

#### ## 1. Decision

Do not begin by implementing PEAD as a scored feature. Begin with a bounded US data-feasibility study. Kairos currently stores upcoming earnings estimates, but the refresh path writes `eps_actual = null`; Webull supplies a current consensus snapshot, not a durable pre-announcement estimate history. The app therefore does not yet possess the point-in-time actual/expectation pair required for PEAD or the vintages required for revision momentum.

This backlog is subordinate to `features/advanced-learning` and the existing Edge/Validation governance. Each distinct signal definition is a separate trial and counts toward multiple-testing controls. A combined signal is another trial, not a free improvement.

#### ## 2. Non-Negotiable Invariants

1. No LLM output changes a deterministic score, eligibility result, size, order, stop, target, or exit.
2. Every feature is point-in-time: `available_at <= decision_at`. Event date alone is insufficient.
3. US and India are evaluated separately. Currency and return pools never mix.
4. A negative observation is measurement only. It cannot enter downside hedging, suppress candidates, or create a short without a separately approved feature.
5. Promotion defaults to no change and uses the existing Challenger, validation, lineage, and Performance Truth layers.
6. Data comes through the Canonical Evidence Router and existing evidence provenance. Do not create an anomaly-specific provider or truth layer.
7. The existing `post_earnings_drift` archetype is not evidence that PEAD exists: it currently reacts to pre-earnings proximity and reweights existing dimensions. A true PEAD edge must use a different versioned identifier until that naming collision is resolved.

#### ## 3. P0: US PEAD Data Feasibility

### Required event record

For each issuer/reporting period, the immutable event snapshot must contain:

- canonical symbol and issuer identity;
- fiscal period and period end;
- announcement timestamp and session ('before\_open', 'during\_session', 'after\_close', or 'unknown');
- first-reported actual EPS, basis, currency, source, and 'available\_at';
- last valid consensus EPS strictly before the announcement, snapshot timestamp, contributing analyst count, basis, currency, and source;
- later correction/restatement links without overwriting the original value;
- split/corporate-action adjustment metadata; and
- Router policy version and evidence-record references.

Unknown announcement session, incompatible EPS bases, a post-release estimate, zero/near-zero scaling denominator, or missing provenance causes abstention.

### Candidate surprise definitions

Pre-register and evaluate separately:

1. ``pead_analyst_price_scaled_v1 = (actual_eps - consensus_eps) / pre_event_price``.
2. ``pead_analyst_abs_scaled_v1 = (actual_eps - consensus_eps) / max(abs(consensus_eps), epsilon)``, with a documented epsilon and winsorization.
3. A time-series SUE only after a restatement-safe historical EPS series exists; scale the unexpected change by its own historical variability.

Do **not** divide surprise by analyst dispersion. Dispersion measures disagreement, may be absent or near zero, and can create unbounded values. It may be tested as a separate conditioning variable once point-in-time analyst-level coverage exists.

### Feasibility output

P0 produces a coverage report, not 'edge\_signals':

- eligible events / all events by year, sector, market-cap bucket, and session;
- consensus age and analyst-count distributions;
- basis/currency conflicts and correction rates;
- source outages and survivorship/delisting coverage; and
- estimated provider call/storage cost.

Proceed only if the owner approves explicit coverage floors after seeing this report. Do not label PEAD "zero data risk" or "data already available."

## 4. P1: Measure-Only PEAD Edge

After P0 approval:

1. Freeze the definition, universe, horizon set, cost model, and winsorization.
2. Assign returns from the first tradable session after the verified publication time. An after-close result cannot use that same close.
3. Evaluate fixed forward horizons aligned with Kairos styles, with overlapping-return robust errors, sector/year breakdowns, event clustering, liquidity, delistings, and realistic spread/slippage.
4. Evaluate long-only behavior. Negative-surprise names are an excluded/observed cohort, not automatic shorts.
5. Write measure-only edge observations linked to existing decision observations and evidence snapshots. Never manufacture a decision observation solely because an external event exists.
6. Count every definition, horizon, filter, and combination in the declared trial family used by DSR/PBO and promotion review.

Only an approved, out-of-sample Challenger may later consume the edge.

## 5. Deferred Candidates

### Analyst-revision momentum

Blocked until Kairos stores immutable estimate vintages. A current Webull target or forecast cannot reconstruct revisions. A valid contract needs estimate timestamp, fiscal period, basis, analyst count, revision breadth, stable analyst universe, and strict pre-decision availability. Evaluate EPS and target revisions separately.

### Filing-language change (Lazy Prices)

US-only feasibility initially. Use point-in-time EDGAR filings, issuer/form/period identity, acceptance timestamp, amendment links, section-aware extraction, and a versioned tokenizer/diff algorithm. Never compare a filing with a later amendment that was not yet known. Plain deterministic similarity is the first candidate; embeddings or LLM interpretation require a separate untrusted-compute review.

### PEAD.txt-style textual surprise

This is not generic earnings-call tone. The cited research constructs a numerical text-based earnings-surprise measure. Reproducing it requires a separately reviewed document corpus, labels, training/evaluation protocol, and point-in-time controls.

It is not an incremental prose feature for P1.

Short interest, 13F flows, options skew, and seasonality remain intake ideas only.

#### ## 6. Validation And Promotion Gates

- common, point-in-time universe and benchmark;
- market-local net returns after costs;
- walk-forward splits with an untouched final holdout;
- sample floors per year/regime/sector, not only aggregate 'N';
- Newey-West or event-cluster-aware uncertainty where observations overlap;
- declared trial-family count and advanced-learning DSR/PBO controls;
- incremental value over the current champion, not standalone significance;
- stable effect direction and no dependency on one provider/year/sector; and
- owner-approved Challenger promotion. No automatic promotion.

#### ## 7. Build Order

1. Resolve the 'post\_earnings\_drift' naming/semantic collision.
2. Specify and capability-probe a US point-in-time earnings contract.
3. Run P0 coverage only; retain no scoring effect.
4. If approved, add immutable event snapshots through Router provenance.
5. Build one pre-registered PEAD definition and evaluation.
6. Add other definitions one at a time as counted trials.
7. Consider revision momentum, filing changes, and India only after their own evidence contracts pass feasibility.

#### ## 8. Acceptance And Rollback

P0 passes when every reported numerator/denominator is reproducible from immutable source references and no post-event fact enters a pre-event snapshot. P1 passes only when reruns from the same snapshot produce byte-equivalent feature values and the edge remains measure-only. Disable by stopping new edge computation; immutable history remains. No positions, signals, policies, or orders require rollback.

#### ## 8b. Implementation Note – Data Capture Enabler (2026-07-15)

Build-order steps 1-2 (plus the vintage table and a feasibility read) are implemented as **data capture only**. No scored feature, no 'edge\_signals', no scoring/sizing/order/exit effect exists. Deterministic; no LLM.

- **Naming collision (step 1) resolved.** 'lib/scoring/archetypes.ts': the archetype formerly 'id: "post\_earnings\_drift" / label "Post-Earnings Drift" is renamed to 'id: "pre\_earnings\_proximity\_reweight\_v1" / "Pre-Earnings Proximity Reweight". Behavior (weights, 'daysToEarnings' <= 10' routing, shadow role) is unchanged – only the identifier/label/comment. The 'pead\_' namespace is now reserved for a future true surprise-based edge. Historical 'shadow\_decisions.setup\_type = 'post\_earnings\_drift'' rows remain queryable under the old string; the rename starts a new series going forward.
- **First-observed provider actuals (step 2).** Migration '20260715210000\_earnings\_pit\_actual\_capture\_columns.sql' adds 'eps\_actual\_first', 'revenue\_actual\_first', 'actual\_available\_at', 'announcement\_session', 'eps\_basis', 'actual\_currency', 'actual\_source', 'restated\_eps', 'restated\_available\_at', 'restated\_source', 'market' to 'earnings\_calendar'. 'lib/data/earnings-pit.ts' fills 'eps\_actual\_first' **once** and never overwrites it; migration '20260715220000\_...' enforces that immutability in Postgres. A later differing provider print is logged to 'restated\_eps'. This is the first value Kairos observed, not proof that a sparse polling cadence captured the issuer's literal first print. Source: Finnhub earnings calendar (free, no daily cap, already wired); its 'hour' field maps to the announcement session.
- **Consensus vintages (step 3).** Append-only table 'earnings\_consensus\_snapshots' (migration '20260715210100\_...') accumulates the last-valid pre-announcement consensus. A new vintage is appended only when the consensus changes since the last snapshot; a database trigger blocks UPDATE/DELETE. The conservative timestamp rule rejects consensus captured on or after the US report date, preventing delayed actual publication from creating look-ahead. 'analyst\_count' is null on the free Finnhub calendar (surfaced in the coverage report). Finnhub also does not identify GAAP versus adjusted EPS, so provider identity is never treated as accounting basis; these observations remain measurement-only and ineligible until another source proves basis comparability.
- **Feasibility read (§3).** 'GET /api/calendar/earnings/coverage', owner-gated ('requireOwner'), read-only. Reports eligible events (both a first-reported actual AND a pre-announcement consensus vintage with explicitly matching basis/currency) by year and session, plus analyst-count coverage, corrections, and basis conflicts. It is a coverage report, not 'edge\_signals'.
- **Cadence.** 'kairos-earnings-pit-capture' invokes 'POST /api/calendar/earnings/refresh' daily at 02:10 UTC; the capture runs after the cache bust, fail-soft. RLS is on for both tables (authenticated-read, service-role-write), verified via 'information\_schema' + 'pg\_policies'.

#### ## 9. Primary Research References

- Bernard and Thomas, post-earnings-announcement drift:  
<https://www.jstor.org/stable/2491062>

- Philadelphia Fed, PEAD.txt project and revisions:  
<https://www.philadelphiafed.org/the-economy/banking-and-financial-markets/pead-txt-post-earnings-announcement-drift-using-text>
- Cohen, Malloy, and Nguyen, Lazy Prices:  
<https://onlinelibrary.wiley.com/doi/abs/10.1111/jofi.12885>

## ## 10. Owner Decisions Before Build

1. Approve P0 as US coverage measurement only.
2. Approve one analyst-surprise definition after the coverage report; do not approve a combined definition in advance.
3. Set coverage and sample floors from observed feasibility, not guesses.
4. Keep India blocked until a separate point-in-time data proof exists.

---

# Source: features\learning-core\FEATURE\_ARCHITECTURE.md  
# Learning Core Rebuild – Feature Architecture

**\*\*Status (2026-07-06): FULL ROADMAP BUILT.\*\*** Phase 1 (decision ledger), all P0 improvements (transactional fill RPC, cron-gap detector, vites  
**\*\*Origin:\*\*** Codex agent-architecture review (`CODEX\_AGENT\_REVIEW\_RESULT.md`). Verdict: Kairos is a governed adaptive *\*scoring\** system, not yet

**\*\*Goal:\*\*** Turn the learning plane into a statistically trustworthy, self-evolving engine – the "world-class evolving quant agent" bar – without

**\*\*Design constraints (keep):\*\*** deterministic scoring (no LLM numbers); LLM = hypothesis/feature proposal + narrative only; per-market isolation

---

## ## The three phases

### ### Phase 1 – Scientific ground truth (KEYSTONE)

Give the learner a valid observation→outcome dataset. Nothing above this is trustworthy until it exists.

- **\*\*`decision\_observations`\*\*** (new, immutable, append-only): one row per SCORED candidate – **\*\*including rejected ones\*\*** (not just filled long)
  - `id, ts, market, symbol`
  - `code\_version` (git sha), `strategy\_version\_id`, `weights\_snapshot\_hash`, `data\_availability\_mask` (which of the 5 dims had real data)
  - raw + transformed features (the dimension sub-inputs, not just the 5 scores): stored as `features jsonb`
  - `analyst\_score`, per-dim scores, `entry\_eligible` (passed threshold?), `action` (bought/skipped + reason), `fill\_price/qty` if filled, `ex`
  - Written by ResearchAgent for EVERY candidate it scores (filled or not).
- **\*\*`observation\_labels`\*\*** (new): forward outcomes computed AFTER horizon maturity, never before:
  - `observation\_id, horizon (2d|5d|10d|20d), fwd\_return, benchmark\_neutral\_return, sector\_neutral\_return, max\_adverse\_excursion, max\_favorab`
  - Uses corporate-action-adjusted prices; realistic spread/slippage/fees. A nightly "label maturation" job fills these as each horizon comes
  - Realized policy P&L (`paper\_trades`) is kept SEPARATE as an execution-policy label, not the alpha label.
- **\*\*Walk-forward dataset builder\*\***: purged + embargoed folds around validation boundaries so overlapping swing returns aren't leaked across tr
- **\*\*Quarantine 10yr personal trades\*\*** from alpha: they feed the Mentor/behavioral model only (provenance + policy-era tags); may generate hyp

Migrations: `059\_decision\_observations.sql`, `060\_observation\_labels.sql`.

Jobs: `POST /api/agents/label-maturation` (nightly cron, per market). ResearchAgent writes observations inline.

### ### Phase 2 – Validation Engine (make evidence mandatory)

- **\*\*Deterministic validation service\*\*** (`lib/validation/`): fit regularized multivariate models (ridge/elastic-net for excess return, logisti
- **\*\*Champion-vs-challenger on paired opportunity sets\*\*** (same symbols/periods): paired return differences, posterior P(improvement), Sharpe/S
- **\*\*Promotion state machine\*\*** (server-enforced, fail-closed): `draft → offline\_testing → shadow\_paper → eligible → approved`. `promote\_cham
- **\*\*Promotion objective = risk-adjusted return\*\*** (net expected log-growth / Sharpe under fixed exposure), NOT win-rate – so R:R and expected
- **\*\*Calibrated P(win) + sizing/exit module\*\***: fit a calibrated probability + a separate, independently-validated sizing/exit policy against th
- LearnerAgent's `update\_signal\_weight` no longer picks the number – it proposes a hypothesis; the Validation Engine fits + gates.

### ### Phase 3 – Controlled evolution (genuinely evolve)

- **\*\*Typed strategy genome\*\*** (extend `strategy\_versions`): feature defs/transforms, missing-data policy, entry/rank threshold, universe/liquid
- **\*\*Feature registry + discovery\*\***: LLM proposes a machine-readable feature spec (rationale, formula, inputs, lag, expected sign, horizon, fa
- **\*\*Shadow A/B + bounded exploration\*\***: fan every daily observation to champion + a small set of challengers (record hypothetical decisions/f
- **\*\*Regularized regime conditioning\*\***: point-in-time regime features (trend/vol/breadth/rates/dispersion); mixture-of-experts / partial-poolin
- **\*\*Predeclared promotion objective\*\*** (e.g. benchmark-neutral expected log-growth under fixed exposure) + non-inferiority gates; versioned, re

---

## ## Cross-cutting workstream – Risk-Reward Optimization (first-class goal)

**\*\*Problem today:\*\*** sizing is a flat `position\_size\_pct` (10%) and R:R is a fixed profile ratio (~2.85:1) – identical on a barely-qualifying an

This spans all three phases:

- **\*\*Phase 1 (ground truth):\*\*** `observation\_labels` capture **\*\*MAE/MFE\*\*** (max adverse/favorable excursion) per candidate per horizon – the raw m
- **\*\*Phase 2 (learn + validate the R:R module):\*\***
  - Fit a **\*\*calibrated P(win) / expected-payoff model\*\*** (logistic for sign, quantile/regression for magnitude) – NOT the raw analyst\_score, wh
  - Learn **\*\*sizing + exit as a SEPARATE validated module\*\*** (review #4) against MAE/MFE + return labels: how big, where the stop, where the tar
  - **\*\*Promotion objective = risk-adjusted return\*\*** (net expected log-growth / Sharpe under fixed exposure limits), NOT win-rate. So the engine
- **\*\*Phase 3 (make it evolvable + conditional – part of the genome):\*\***
  - **\*\*Conviction-scaled sizing\*\*** – fractional-Kelly or volatility-target sizing from calibrated P(win) × payoff, capped by exposure/concentrat
  - **\*\*Dynamic R:R\*\*** – ATR/volatility- and pattern-conditioned stops & targets (e.g. stop = k·ATR, target sized to the pattern's historical MFE
  - **\*\*Regime-conditioned posture\*\*** – the regime router (P3) adjusts sizing aggression + R:R by regime (e.g. tighter/smaller in high-vol revers
  - All bounded by the risk layer + human live-capital gate; sizing/exit changes go through the same walk-forward validation + promotion gate

**\*\*Guardrails:\*\*** fractional-Kelly (≤ ½-Kelly) not full; hard per-name and per-sector exposure caps stay human-set (moral-hazard rule); sizing m

## ## Phase 1 detail (the piece up for approval now)

**\*\*Why first:\*\*** every other phase optimizes against a label. If the label/dataset is biased (symbol-join, filled-longs-only, policy-P&L, leakage).

**\*\*Scope of the build:\*\***

1. Migration `059\_decision\_observations` + `060\_observation\_labels` (+ indexes; RLS off like sibling agent tables). Applied manually (MCP den).
2. ResearchAgent (`lib/research-agent.ts`): after scoring each candidate, write a `decision\_observations` row (features + availability mask + ...).
3. Label maturation job `app/api/agents/label-maturation/route.ts` + cron: nightly, per market; for observations whose horizons have matured, ...
4. Walk-forward dataset builder `lib/learning/dataset.ts`: assemble purged/embargoed folds from observations+labels (read-only; consumed by PL ...)
5. Learner: repoint `query\_signals\_with\_outcomes`/`query\_score\_correlation` to read the matured label dataset (horizon-aligned, all candidates ...)
6. Docs + system-map (new LEDGER node) + PROJECT-DECISIONS (Decision 33).

**\*\*Not in Phase 1:\*\*** the statistical fitting, promotion gate, genome, shadow – those are Phase 2/3.

**\*\*Risk/cost:\*\*** additive, resilient; no change to live behavior until observations accrue. Storage grows (~N candidates/day/market × features)

---

**## Phase 1 – RESOLVED decisions (refinement, 2026-07-06)**

1. **\*\*Feature granularity = full raw features, for free.\*\*** `computeScores` already returns an `evidence` object per dimension holding the raw ...
2. **\*\*Observation trigger = every symbol that reaches `computeScores`\*\*** (holdings + screener candidates + NIFTY candidates), whether it fills ...
3. **\*\*Horizons = 2 / 5 / 10 / 20 trading days. Active markets only\*\*** (write observations only for markets currently in `market\_focus`) – match ...
4. **\*\*Label sources / benchmarks:\*\*** US fwd prices from price\_cache/AV/Massive, benchmark SPY. India from Yahoo `NS` candles, benchmark `^NSEI ...
5. **\*\*No backfill.\*\*** Point-in-time features can't be honestly reconstructed for past signals, so the ledger starts fresh from deploy. `signal\_ ...
6. **\*\*Phase 1 improves the DATA, not yet the METHOD.\*\*** Once matured labels exist, the learner's correlation tools read the horizon-aligned, al ...
7. **\*\*Retention:\*\*** prune `decision\_observations` + `observation\_labels` older than 18 months (a later cron); bounded storage.

**\*\*Volume estimate:\*\*** ~10-20 scored candidates/market/day × 2 markets ≈ 30-40 rows/day + 4 label rows each → well under 1M rows/year. Trivial

**\*\*Deliverables locked for Phase 1:\*\*** migrations 059/060 (manual apply), ResearchAgent observation write, `label-maturation` cron (+ register- ...)

---

**## Build 1 – Genome as LIVE control (2026-07-08, user-approved)**

**\*\*Gap found:\*\*** Phase 3 shipped a typed, hard-bounded `StrategyGenome` + feature registry (`lib/validation/genome.ts`, migrations 063/064) that is written and hashed onto every signal for provenance – but **\*\*no live code read it\*\***. The decision path took `score\_threshold` from `strategy\_config` and derived exit/sizing from hardcoded percentiles (`0.25/0.75`, horizon `10`, cap = `position\_size\_pct`). So a challenger that the Validation Engine proved better by tightening a stop or raising the threshold got promoted, yet **\*\*traded identically to the champion\*\*** – the genome was a manifest, not a control. Build 1 closes that: the promoted champion's genome now governs live behavior.

**\*\*Wired (additive, no new migration – genome column already live):\*\***

- `lib/validation/genome-live.ts` (new): `loadChampionGenome(supabase, market)` – reads the promoted champion `strategy\_versions.genome`, hydrates onto `DEFAULT\_GENOME`, re-checks bounds, and returns `{genome, source, hash}`.
- Best-effort: missing champion / null genome / out-of-bounds → `DEFAULT\_GENOME` (`source: "default"`), never throws.
- `lib/research-agent.ts`: entry threshold reads `champion.genome.entry.score\_threshold` (genome added to the champion select), falling back to `strategy\_config`.
- `app/api/agents/paper-trade/route.ts`: `getGlobalMaeMfePercentiles` now takes the genome's `horizon\_days` + `exit.stop\_mae\_pctile/target\_mfe\_pctile`; Kelly sizing takes `sizing.cap\_pct/floor\_pct` and `sizing.mode` selects half-Kelly vs flat. Genome `source`/`hash` stamped into the portfolio-constructor stage log.

**\*\*Money-safety invariants (locked):\*\***

- **\*\*Genome cap clamped to the owner's `strategy\_config.position\_size\_pct`\*\*** – `min(genome.cap\_pct, position\_size\_pct)`. The learning loop can size DOWN but NEVER raise per-position exposure above the owner-set limit. Money limits stay human. Portfolio Constructor name/gross/vol caps still run after.
- **\*\*Behavior-preserving by construction.\*\*** `DEFAULT\_GENOME` = `{score\_threshold 60, stop\_mae\_pctile 25, target\_mfe\_pctile 75, horizon 10, cap 10, floor 2}`, the exact values the live path used before. A market with no genome-bearing champion is byte-for-byte unchanged. New behavior triggers only after a genome-bearing challenger passes the fail-closed validation gate AND is owner-promoted – the most-gated path in the app. No autonomy change; owner click still required for live.

**\*\*Deliberately OUT of Build 1 (deferred, not dropped):\*\*** `universe.\*` (screener routing), `features.included` + `regime.router` (need feature-registry activation – Build 5-adjacent). Shadow-version decisions still score off the champion threshold, not each shadow version's own genome (separate enhancement).

---

# Source: features\learning-integrity\FEATURE\_ARCHITECTURE.md  
# Learning Integrity - Taint Detection, Exclusion & Learner Recovery

**\*\*Status:\*\*** DRAFT - awaiting approval. No implementation code until approved.

**\*\*Author:\*\*** Claude (Opus 4.8), reviewed/updated by ChatGPT, 2026-07-08.

**\*\*Decision inputs (user, 2026-07-08):\*\***

- Primary mechanism: **\*\*tag + exclude\*\*** (non-destructive; deletion optional).
- Auto-taint threshold: **\*\*moderate\*\*** (`data\_confidence < 0.5`) - conditional on a proven-correct, deterministic confidence calculation.
- Scope now: **\*\*architecture draft only\*\***.



---

## ## 1. Problem

The research scorer can emit a high-conviction signal from partial data - for example, GLD/SGOV/IBIT on 2026-07-07 scored high while the logged evidence showed missing fundamentals, zero technical candles, and unknown macro. The weighted score then renormalized onto the dimensions that happened to be present.

Those decisions can flow into paper/live books and, after outcomes mature, into the LearnerAgent and Validation Engine. Bad data can therefore become bad trades, bad labels, and bad strategy mutations.

We need:

1. A deterministic way to detect low-evidence decisions from already-logged facts.
2. A non-destructive way to stop them polluting learning and validation datasets.
3. A recovery path if a bad learner mutation still lands.
4. A reusable, safe teardown path to replace ad-hoc manual SQL cleanup.

## ## 2. Non-goals

- No re-fetching or re-scoring historical decisions.
- No live trading-path dependency on this layer.
- No auto-delete by default; delete/reset remains manual, backed up, and dry-run-gated.
- No implementation code until Vaibhav approves this architecture.

---

## ## 3. Existing primitives to reuse

```
Need	Existing table/column
Per-decision evidence	`decision_observations`: append-only; `availability_mask` jsonb, `features` jsonb (`weighting.{base_weights,applied}`)
Forward labels	`observation_labels`: `observation_id`, horizon returns, MAE/MFE
Versioned strategies / rollback	`strategy_versions`: `weights_snapshot`, `genome`, `is_champion`, `parent_version_id`, `promoted_at`, `ret`
Prior history	`learning_priors_history`: `prior_id`, `confidence`, `enabled`, `reason`, `changed_by`, `changed_at`
Shadow / challenger eval	`shadow_decisions`: `observation_id`, `policy_version_id`, `would_enter`, `score`
Paper trade to decision link	`paper_trades.signal_id` -> `decision_observations.signal_id`
Live order to decision link	`broker_orders.proposal_id` -> `trade_proposals.id` -> `trade_proposals.signal_id` -> `decision_observations.signal_id`
```

### ### Current schema constraints that matter

- `decision\_observations.id` is `bigserial`.
- `decision\_observations.signal\_id` is `uuid`.
- `paper\_trades.signal\_id` is `uuid`.
- `broker\_orders` currently has no `signal\_id`; live quality must join through `proposal\_id`.
- `trade\_proposals.signal\_id` is currently declared as `bigint` in migration 037. It must be migrated to `uuid` before live-order taint joins.
- `decision\_observations` is append-only by trigger; do not add mutable taint columns there.
- `observation\_labels` and `shadow\_decisions` reference `decision\_observations(id)` with `on delete cascade`; the reset/teardown feature must cascade.
- Any code that reads new quality/tag columns must ship only after the additive migration is applied. No schema-coupled runtime fallback should be needed.

Implication: rollback is a strategy-version pointer operation, not data surgery. The taint layer should be a thin additive layer over mutable trade/order projections plus a deterministic view over the immutable observation ledger.

---

## ## 4. Design

### ### 4.1 `v\_decision\_quality` - deterministic quality view

Create a read-only SQL view `v\_decision\_quality` over `decision\_observations`. It must not mutate the append-only ledger. It covers both historical and future rows.

Concept:

applicable\_dims = structural scoring dims derived from the logged row only

applicable\_weight = sum(base\_weights[d]) for d in applicable\_dims

real\_dims = dims in applicable\_dims where availability\_mask[d]=true and dim is not degraded

available\_weight = sum(base\_weights[d]) for d in real\_dims

data\_confidence = available\_weight / applicable\_weight

Use `features.weighting.base\_weights` for `base\_weights`. Do **not** use `weights\_used` or `features.weighting.applied\_weights` as the denominator; those are post-renormalization weights and can make a one-dimension decision look fully covered.

The view must not call or guess the TypeScript `applicableDimensions()` function. It has to derive structural applicability from values already logged in

```

`decision_observations`:

- `market = 'india': applicable scoring dims are `fundamental`, `technical`, `macro`.
- `features.fundamental.is_etf = true`: applicable scoring dims are `technical`, `sentiment`, `macro`.
- US non-ETF: applicable scoring dims are `fundamental`, `technical`, `sentiment`, `macro`, `insider`.
- US ADR insider exclusion is not reliably reconstructable from current rows unless a structured flag is logged. Until then, if `availability_mask.insider` is not set, assume exclusion.
- If `applicable_weight <= 0`, malformed base weights, or a missing base-weight key would affect the denominator, emit `data_confidence=NULL`.

The view emits:

- `observation_id`
- `signal_id`
- `market`
- `symbol`
- `applicable_dims text[]`
- `real_dims text[]`
- `missing_dims text[]`
- `degraded_dims text[]`
- `decisive_dim text`
- `data_confidence numeric`
- `confidence_band text` (`high >= 0.75`, `med >= 0.5 and < 0.75`, `low < 0.5`)
- `quality_status text` (`ok` or `unknown`)
- `taint_reason text`

4.2 Degraded-dimension rules

A dimension is degraded when it is technically present but not real evidence.

Historical fallback rules:

- `fundamental`: evidence note contains "No fundamental data available" or provider/rate-limit text. ETF "no company fundamentals" is structured.
- `macro`: `features.macro.regime = 'unknown'`.
- `technical`: `features.technical.dataPoints < 15`.
- `sentiment`: no provider-backed basis, e.g. bullish/bearish both absent/zero and no AV/news sentiment.
- `insider`: unavailable/too few transactions text or `availability_mask.insider=false`.

String matching is a historical bridge only. For future rows, ResearchAgent should log
a structured quality object:

{
 "quality": {
 "fundamental": { "state": "ok|missing|degraded|inapplicable", "reason": "..."},
 "technical": { "state": "ok|missing|degraded|inapplicable", "reason": "..."},
 "sentiment": { "state": "ok|missing|degraded|inapplicable", "reason": "..."},
 "macro": { "state": "ok|missing|degraded|inapplicable", "reason": "..."},
 "insider": { "state": "ok|missing|degraded|inapplicable", "reason": "..."}
 }
}

`v_decision_quality` should prefer `features.quality` and fall back to legacy string
heuristics only for older rows.

4.3 Correctness requirements

These are acceptance criteria:

1. **Pure / no I/O.** The quality calculation reads only database columns. No network, LLM, live prices, or provider calls.
2. **Reads logged weights.** It uses `features.weighting.base_weights` and the scorer's own logged evidence. It does not reconstruct live state.
3. **Two-stage fail behavior.**
 - During measure-only rollout, malformed/missing `weighting`, `availability_mask`, or structural inputs produce `data_confidence = NULL`.
 - After golden tests and historical flag-rate checks pass, learner and validation datasets must require `quality_status='ok'` and `excluded=false`.
4. **Trading path stays independent.** Order submission and portfolio code must not depend on this layer to place orders. This layer can tag, but cannot block.
5. **Flag != exclude.** Computed confidence and mutable exclusion are distinct. A bad flag is reversible by owner override.
6. **Golden tests + monitor.** Unit/golden tests assert worked examples against real or fixture rows. A daily System Health reporter emits tags.

Moderate auto-flagging (`data_confidence < 0.5`) turns on only after all of the above are green. Until then, ship measure-only.

4.4 Worked checks

These cases must pass as golden tests:

- Bad GLD/SGOV/IBIT-style ETF row: applicable `{technical, sentiment, macro}`; technical missing, macro degraded, only sentiment real -> confidence 0.5.
- Legit ETF with real technicals, real macro, real sentiment -> confidence near 1.0 -> clean.
- Full-coverage US equity with all applicable dims real -> confidence 1.0 -> clean.
- India equity with real fundamentals, real technicals, real macro -> confidence 1.0 -> clean.

```

```

- Malformed historical row with no `features.weighting.base_weights` -> `quality_status='unknown'`, `data_confidence=NULL`, not auto-tainted

5. Trade/order tagging

Add nullable columns, additive and safe:

- `paper_trades`: `data_confidence` numeric, `quality_status` text, `tainted` boolean default false, `taint_reason` text, `excluded_from_learning` boolean
- `broker_orders`: same five columns
- `observation_labels` or the ledger-quality join path: must expose equivalent quality/exclusion fields to validation and feature-learning code

Population:

- **At creation - paper:** PaperTrader joins `v_decision_quality` by `paper_trades.signal_id` -> `decision_observations.signal_id` and writes the row
- **At creation - live:** Execution Gateway joins by `broker_orders.proposal_id` -> `trade_proposals.id` -> `trade_proposals.signal_id` -> `decision_observations.signal_id`
- **Backfill:** One-time job tags existing `paper_trades` and `broker_orders`.
- **Override:** Owner can flip `excluded_from_learning` manually; every override writes `decision_journal` or an admin audit row.

Taint rule after enablement:

if quality_status='ok' and data_confidence < 0.5:
 tainted = true
 excluded_from_learning = true
 taint_reason = 'low data_confidence: <degraded/missing dims>'
else if quality_status='unknown' or data_confidence is null:
 measure-only: leave untainted
 learning-enabled: exclude from learner/validation until reviewed

6. Learner and validation filtering

The primary pollution fix is not deletion. It is dataset filtering.

Closed-trade learner queries must include:

coalesce(excluded_from_learning,false) = false
and coalesce(tainted,false) = false
and data_confidence is not null
and coalesce(quality_status,'unknown') = 'ok'

Apply the equivalent filter to every learning input path:

- `app/api/agents/learner/route.ts` closed `paper_trades` queries.
- `computeScoreCorrelation()` fallback over `paper_trades`.
- `lib/learning/dataset.ts::loadLabeledDataset()` / validation engine reads from `decision_observations x observation_labels`.
- `app/api/validation/feature-check/route.ts` feature IC evaluation.
- `shadow_decisions` promotion metrics.

For ledger-based paths, do one of:

1. Add `data_confidence`, `quality_status`, and `excluded_from_learning` to `observation_labels` at label creation time; or
2. Join `v_decision_quality` by `observation_labels.observation_id`.

Do not depend only on `paper_trades` tags. The validation engine and feature discovery
learn from the observation ledger, not only filled paper trades.

7. Learner mutation recovery

Reuse `strategy_versions` and `learning_priors_history`.

- Every weight mutation should already create a new `strategy_versions` challenger row with `parent_version_id` and `weights_snapshot`.
- Recovery is: retire the bad version, re-promote the parent for the same market, and log the reason.
- `revert_learner(market, to_version?)`: owner-gated admin op. `to_version` defaults to current champion's parent.
- Prior restoration should use `learning_priors_history` only when the mutation actually changed priors. If prior changes are unrelated user changes, ignore.

Mutation guardrails:

- Reject a candidate trained on any tainted/excluded trade or unknown-quality ledger row after enablement.
- Reject a candidate if validation used any row with `quality_status <> 'ok'` or `data_confidence is null`.
- Clamp per-run weight deltas server-side.

```

- Shadow quarantine: a new candidate must accumulate `K` shadow outcomes before promotion to champion / production sizing.

---

## ## 8. Reusable safe teardown

Owner-gated admin route. Dry-run is the default.

Verbs:

- `exclude\_tainted?market=X`: flags only, no deletion.
- `reset\_paper?market=X&dryRun=true`: returns counts and exact operations.
- `reset\_paper?market=X&confirm=true`: creates backup under `backups/`, then transactionally resets mutable paper state.

`reset\_paper` may touch only mutable projections for the chosen market:

- `paper\_portfolio`
- `paper\_positions`
- `paper\_trades`
- pending paper-only proposals/orders, if explicitly scoped

It must not delete or mutate:

- `decision\_observations`
- `observation\_labels`
- `shadow\_decisions`
- `paper\_order\_events`
- `learning\_log`
- `strategy\_versions`
- `evidence\_records`

The reset route must use service-role only, owner auth, dry-run counts, backup-first, and explicit market scoping.

---

## ## 9. What each part protects against

```
Failure	Guard
Bad data -> high score	`v_decision_quality.data_confidence` measures evidence coverage
Tainted trade -> learner	`excluded_from_learning` + quality filters
Tainted observation -> validation/feature learning	`v_decision_quality` join or label-level quality columns
Confidence calc bug -> silent mass exclusion	measure-only rollout, unknown state, golden tests, monitor, owner override
Bad mutation lands	champion-parent revert + shadow quarantine
Manual cleanup breaks app	dry-run + backup + append-only-respecting reset
```

---

## ## 10. Rollout

1. **Phase 1 - Measurement:** `v\_decision\_quality` view, structured future `features.quality` logging, golden tests, monitor. No auto-taint yet.
2. **Phase 1B - Pollution-proofing:** trade/order taint columns, observation-label or ledger-quality join, backfill, learner/validation/feature learning.
3. **Phase 2 - Mutation safety:** `revert\_learner()`, mutation guardrails, shadow quarantine.
4. **Phase 3 - Ops:** admin exclude/reset route with dry-run and backup.

---

## ## 11. Open questions

- `K` for shadow quarantine. Default proposal: 10 closed shadow outcomes per market, but validate against observed signal frequency.
- Exact `features.quality` shape for future rows.
- Historical degraded-dim heuristics for old rows.
- `trade\_proposals.signal\_id` type migration from `bigint` to `uuid`.
- Moderate threshold 0.5: validate flag rate across full history before enabling. It should catch the true bad set, not a broad percentage of rows.

---

## ## 12. Diagram / system-map impact

The learner <- trades <- decision\_observations flow gains a quality gate. On build, update `public/agent-diagrams/system-map.json` and the affected learner/paper-trader diagrams, then append a history entry per project convention.

## ## Reviewer changelog (ChatGPT)

- Section 3: Updated author line to "reviewed/updated by ChatGPT, 2026-07-08" as requested.
- Section 3: Added an explicit migration-order rule: code that reads new taint/quality columns must ship only after its additive migration is complete.
- Section 4.1: Tightened the `data\_confidence` denominator rule to fail `unknown` when structural/base weights are malformed instead of falling through.
- Section 5: Added the requirement that validation/feature-learning paths get equivalent quality/exclusion fields through `observation\_labels`.
- Sections reviewed with no additional change: Sections 1, 2, 7, 8, 9, 10, 11, and 12 were consistent with the known schema/safety constraints.

---

# Source: features\leveraged-etf-and-intraday-execution\FEATURE\_ARCHITECTURE.md

## # Leveraged ETF Sleeve and Intraday Execution Architecture

**\*\*Status:\*\*** APPROVED 2026-09-13 – L0-L1 implementation only; no paper or live execution

**\*\*Date:\*\*** 2026-07-26

**\*\*Scope:\*\*** US paper book first. India, live trading, inverse ETFs, options, and extended-hours trading are explicitly out of scope.

### ## 1. Decision

Kairos should not enable leveraged ETFs through the generic ETF path. A long leveraged ETF is a distinct instrument class with a daily-reset return objective, path-dependent multi-session results, and materially higher gap and execution risk. The correct first product is an isolated, paper-only US sleeve that is disabled by default.

The proposed initial envelope is:

```
Limit	Initial policy
One leveraged ETF	3% of US paper NAV
Entire leveraged ETF sleeve	5% of US paper NAV
Number of leveraged ETF positions	1
Market	US / USD only
Direction	Long leveraged only
Default state	Disabled
```

The 3% and 5% limits are ceilings, not target allocations. A 5% sleeve can lose its entire paid value, or about 5% of the US portfolio at the entry-time NAV, if its products go to zero; a 3x product also creates roughly 15% initial index-equivalent exposure before path effects. Stops do not eliminate gap risk. These are risk-envelope defaults, not evidence-based alpha parameters, and must not be raised automatically.

The broader goal is to borrow institutional \*discipline\*, not to imitate Renaissance or Medallion. Kairos does not have their proprietary data, dense diversified signal library, low-latency execution, capacity research, or decades of independent outcomes. The relevant, achievable standard is reproducible point-in-time research, realistic costs, strict market-local risk limits, out-of-sample promotion, and operational monitoring. No return target or claim of Medallion-like capability is valid.

### ### Quant capability target

```
Capability	Kairos direction	What not to pretend
Data provenance	Freeze evidence, policy, code, and quote fingerprints per decision	That a retail/free-data feed is an institutional cons
Research validation	Point-in-time replay, walk-forward, purging, costs, and regime slices	That a backtest proves a live edge
Portfolio construction	Market-local constraints, concentration/correlation controls, and explicit sleeves	That a small number of paper m
Execution	Fresh quote, spread, drift, idempotency, reconciliation, and broker protections	That cron polling can compete with a market m
Learning	Separate populations and promotion defaults-to-no-change	That an LLM or a short sample can discover stable alpha
```

These are the useful quantitative similarities to pursue. Additional indicators, more agents, or frequent retraining are not substitutes for these controls and are not part of this feature.

### ## 2. Existing State and Gaps

#### ### Existing controls that must remain in force

- `lib/trading/symbol-policy.ts` blocks leveraged and inverse ETFs from generic research, paper, and live paths.
- `lib/asset-classification.ts` identifies a static set of known US ETFs, including several leveraged products. This is insufficient as an authority for a new money path because it does not model leverage factor, direction, issuer, or status.
- The generic US ETF allocation cap in `lib/trading/execute-order.ts` is a global ETF cap. It is not a leveraged-sleeve cap and cannot authorize a leveraged trade.
- Paper entry attempts currently occur at two deterministic in-session windows: US at 15:15 UTC and 19:15 UTC, and India at 09:40 IST and 13:15 IST. The US wakes are 11:15/15:15 EDT and 10:15/14:15 EST, respectively. They are not random or continuously polling entries, but static UTC cron cannot preserve an exact ET minute across daylight saving.
- The live auto worker and live exit monitor exist behind false-by-default deployment and database gates. They must remain disabled for this feature's paper phases.

#### ### Gaps that block an enablement

1. Static ticker lists cannot safely classify all future leveraged, inverse, single-stock, or renamed funds.
2. The current technical score is a general equity score. It does not measure daily reset/path risk, underlying trend persistence, intraday liquidity, or execution spread for a leveraged product.
3. Paper PositionMonitor is not a reliable intraday protective system. A periodic quote check cannot protect an outage or overnight gap.
4. Existing paper results are not an isolated leveraged-ETF experiment and cannot justify sizing or live activation.

### ## 3. Instrument Classification and Allowlist

Create a server-owned `tradable\_instruments` policy record rather than inferring from

a ticker. The initial record fields are:

```
type InstrumentPolicy = {
symbol: string;
market: "us" | "india";
currency: "USD" | "INR";
assetType: "equity" | "etf";
leverageClass: "none" | "long_2x" | "long_3x" | "inverse" | "leveraged_inverse" | "unknown";
underlyingSymbol: string | null;
sleeve: "core" | "leveraged_us" | "blocked";
enabledForPaper: boolean;
enabledForLive: boolean;
effectiveFrom: string;
effectiveTo: string | null;
source: string;
reviewedAt: string;
};
```

Rules:

- Only explicit `long\_2x` or `long\_3x`, `sleeve='leveraged\_us'`, and `enabledForPaper=true` may reach the isolated paper candidate flow.
- `inverse`, `leveraged\_inverse`, `unknown`, single-stock leveraged products, crypto leveraged products, and every unlisted ticker remain blocked. The existing paper-only hedge exception remains separate and may not use this sleeve.
- The first allowlist must contain only broad, liquid, long index/sector products. It must not include single-stock or crypto leveraged ETFs. Each addition is an owner-reviewed policy change with an audit record, not a watchlist side effect.
- A policy lookup/read error is a money-path denial. It may not fall back to the old static ticker set.

#### ## 4. Leveraged-ETF Decision Model

No LLM may decide eligibility, instrument class, size, stop, target, or exit. LLM output can explain a completed deterministic decision only.

The leveraged model is a second deterministic gate after the normal new-long gates; it can only reject or reduce an otherwise valid candidate.

#### ### Required measurements

| Measurement                                  | Purpose                                                              | Initial use                                                     |
|----------------------------------------------|----------------------------------------------------------------------|-----------------------------------------------------------------|
| Underlying trend and persistence             | Daily-reset leverage benefits more from persistent than choppy moves | Require a pre-declared, paper-val.                              |
| Realized volatility and ATR-normalized range | Detect path-risk / volatility drag conditions                        | Reject or reduce exposure above a market-local                  |
| Gap and overnight risk                       | Daily stops do not control discontinuous loss                        | Require a maximum expected loss and separate overnight policy   |
| ETF quote age, bid/ask spread, dollar volume | Avoid paying large implementation shortfall                          | Fail closed on stale/unavailable quotes or exco                 |
| Underlying/ETF tracking sanity               | Catch stale, dislocated, halted, or bad price data                   | Reject on abnormal divergence, stale underlying, or a           |
| Portfolio beta/correlation                   | Prevent duplicate exposure (for example, QQQ plus TQQQ)              | Enforce against the US book using the existing corre            |
| Event risk                                   | Avoid automatic entry immediately before known binary events         | Start with an exclusion policy for underlying index/sector even |

MACD, RSI, EMA, ATR, volume confirmation, and relative strength may be evaluated as features in the isolated experiment. They are not sufficient by themselves, and none should receive a permanent weight without point-in-time, costed, market-local evidence. Fibonacci levels, Elliott wave, and similar discretionary chart patterns are not part of this money path because they add degrees of freedom without a validated decision contract.

#### ### Sizing and exits

```
leveraged_notional = min(
normal_new_long_notional,
3% of current US paper NAV,
remaining 5% US leveraged sleeve capacity,
```

liquidity/spread-constrained notional,

volatility-scaled risk budget,

all existing name/sector/gross/correlation/cash constraints

)

- Recalculate current NAV and all marks in USD at the same decision timestamp.
- Never use cost basis, stale cache values, or cash alone for sleeve capacity.
- A generic 7% stop / 20% target must not be copied blindly. The experiment records candidate volatility-normalized exits, but the active paper policy starts with a conservative deterministic stop/time-risk plan defined before entry and immutable on the trade record.
- No capital rotation into or out of this sleeve in the first phase. It would mix the new experiment with the established alpha book and amplify churn.
- The LearnerAgent cannot mutate the core champion or learn core signal weights from these outcomes. Leveraged outcomes have their own evaluation population.

## ## 5. Execution Sessions

Regular-session hours are 09:30-16:00 ET for US equities and 09:15-15:30 IST for NSE equities. Kairos must remain regular-session-only. Extended-hours orders, random polling, and "trade whenever a score changes" are out of scope.

The opening and closing periods have higher uncertainty, volatility, and often worse spreads. A study of intraday trading activity finds U-shaped volume and price variability, and liquidity research also documents wider spreads at the edges of the day. Therefore, the product uses fixed decision windows, not a claim that any single minute is universally optimal.

| Flow | US equities / ordinary ETFs | India equities | Leveraged-US ETF sleeve |  
|---|---|---|---|  
| Research | Existing morning and afternoon cycles | Existing morning and midday cycles | Reuse only a fresh same-session deterministic snapshot  
| New entry | Keep the existing two in-session wakes; normalize any future exact local-time policy in the route, not only in UTC cron | Keep a  
| Routine exit review | Existing PositionMonitor policy | Existing PositionMonitor policy | Same end-of-session plan plus a separately qualified  
| Emergency exit | Not a reason to add random entry polling | Not a reason to add random entry polling | Risk-reducing only: immediately action

The exact 11:00 ET leveraged entry time is a conservative starting experiment, not a universal alpha claim. It must be compared with the existing fixed windows using timestamped, costed paper evidence before it becomes a permanent policy. A cron may wake the endpoint more broadly, but 'America/New\_York' time-window code must be the single execution authority; DST can never silently move a policy window.

### ### Intraday exits: correct order of implementation

1. **\*\*Do not add an intraday exit cron using stale/free daily prices.\*\*** It could sell on a bad mark and create a false sense of protection.
2. Build a quote-quality contract: executable quote timestamp, bid, ask, spread, source health, halt/session status, and broker-held quantity.
3. In paper, record every would-exit at a fixed 15-minute cadence, but do not alter the established book until mark quality and false-trigger statistics are known.
4. For any future live entry, a broker-resident disaster floor/protective order plus reconciliation is required. The app monitor is a second line, not the primary protection against an outage or gap.
5. A risk-reducing verified SELL may run outside entry windows during the regular session. It remains subject to held-quantity/reconciliation guards, never a BUY budget, and never opens a short position.

## ## 6. Evidence and Promotion

The sleeve has a separate immutable experiment lineage. Each decision stores the instrument-policy version, underlying and ETF quotes, features, rule version, entry window, model outcome, transaction-cost assumptions, and market-local session date.

Required evidence before any expansion:

- point-in-time replay with correct fund availability and no survivorship additions;
- walk-forward evaluation and purged/time-separated validation;
- costs and conservative bid/ask/slippage sensitivity, not close-to-close returns;
- distinct regime slices (trending, high-volatility, mean-reverting) and a minimum number of independent setups, not daily observations counted as independent;
- comparison with the same underlying unlevered ETF and a cash/no-trade baseline;
- maximum drawdown, gap, false-exit, quote-quality, and reconciliation reporting;
- an owner-approved paper evidence threshold. Failure means no change, not a lower threshold.

No result from this experiment may promote a core strategy version, relax a generic ETF cap, expand the live auto lease, or change India logic.

## ## 7. Phased Delivery

### ### L0 - policy and observability

- Add the classified instrument registry, owner-visible sleeve status, and a strict

- default-deny rule.
- Add a pure, tested market-local execution-window policy. It changes no existing schedule or order behavior in L0; later phases use it to enforce an approved ET window irrespective of UTC/DST.
- Keep all leveraged instruments blocked from generic research and execution.
- Add no cron, no provider, no live broker call, and no scoring influence.

#### ### L1 - measure-only research

- Produce deterministic candidate feature records only for explicitly allowlisted instruments, paced separately from the core research quota.
- Capture quote quality, regime, normalized volatility, and would-enter/would-reject decisions. No paper fills.

#### ### L2 - isolated paper sleeve

- Enable one US paper-only position under the 3%/5% envelope.
- Use the dedicated 11:00 ET entry window, immutable entry plan, and separate outcomes/analytics.
- Enable 15-minute \*would-exit\* observations only after quote contract tests pass.

#### ### L3 - paper risk monitor

- If L2 shows reliable marks, activate deterministic risk-reducing paper exits at the approved cadence. Continue end-of-session monitoring as the fallback.
- Demonstrate no duplicate exits, no stale-mark exits, and correct market-local session behavior in failure injection.

#### ### L4 - live design review only

- This is not authorization to trade live. It requires broker-native protective support, reconciliation, a live-specific allowlist, live account mandate, small owner-approved lease, and a separately approved architecture.

### ## 8. Acceptance Criteria

- An unclassified, inverse, leveraged-inverse, single-stock, crypto, India, or expired allowlist instrument is denied before any score, claim, fill, or order.
- The sleeve cannot exceed 3% for one name or 5% in aggregate using current US NAV.
- India INR holdings and capacity never participate in US sleeve calculations.
- A missing policy, quote, mark, NAV, spread, or correlation read rejects a new leveraged entry; it never falls back to the generic ETF path.
- Leveraged results are excluded from core LearnerAgent promotion/weight mutation.
- A repeated worker, stale quote, quote divergence, halt, partial exit, broker uncertainty, or kill switch cannot create a duplicate order or oversell.
- New entries occur only in their approved regular-session window. Verified risk-reducing exits may occur during regular session outside that window.
- No production flag, mandate, account, generic ETF cap, or live order capability changes in L0-L3.

### ## 9. Sources

- FINRA, [The Lowdown on Leveraged and Inverse Exchange-Traded Products](https://www.finra.org/investors/insights/lowdown-leveraged-and-inverse)
- FINRA, [Non-Traditional ETFs FAQ](https://www.finra.org/rules-guidance/key-topics/etf/non-traditional-etf-faq): suitability must consider volatility
- Wei (1992), [Intraday Variations in Trading Activity, Price Variability, and the Bid-Ask Spread](https://doi.org/10.1111/j.1475-6803.1992.tb00001.x)
- NYSE, [Trading Information](https://www.nyse.com/trade/trading-information?os=io\_): US core session hours.
- NSE, [Market Timings and Holidays](https://www.nseindia.com/resources/exchange-communication-holidays): NSE regular equity session hours.

### ## 10. Explicit Non-Goals

- No claim of Renaissance/Medallion equivalence.
- No HFT, market making, order-book prediction, extended-hours trading, options, inverse ETF trading, or cross-market/currency netting.
- No new LLM authority, external GitHub skill runtime, or provider-quota increase.
- No live leveraged trade, even manually, under this architecture without the L4 approval and a distinct live safety review.

### ## 11. Implementation record – 2026-09-13

L0-L1 is implemented locally in commit pending verification: the only accepted measurement symbols are long 3x `TQQQ` (underlying `QQQ`) and `SOXL` (`SOXX`). `SQQQ` and `SOXS` remain inverse-blocked. The collector records a single 11:00-11:14 ET observation with quote/underlying freshness, spread and raw volatility/trend/liquidity fields, but its only possible decision is `observe\_only`. It cannot create a score, candidate, paper fill, broker call, or live order.

The schema is deliberately unapplied while the FinanceOS production migration lineage is unreconciled. There is also no scheduled collector yet: the current quote contract has no verified executable bid/ask provider. A daily job must not be enabled until it can supply those inputs rather than substituting a daily OHLC cache. L2-L4 remain unimplemented and unapproved.

---

# Source: features\live-auto-trading\FEATURE\_ARCHITECTURE.md



```

Live Auto Trading — Feature Architecture

Last updated: 2026-07-10
Status: IMPLEMENTING PA3 — per-market off/manual/autonomous mode (approved by owner)
Authors: Claude Sonnet 4.6; security and execution architecture reviewed by ChatGPT/Codex, 2026-07-10
Scope: Per-market autonomous mode (US via Robinhood direct REST; India via Kite REST). Both markets support off/manual/autonomous independent.

1. Safety correction and goal

The previous draft was not safe to implement. It used the wrong Robinhood account number (`605420606`), bypassed the hardened Execution Gateways.

Authorized order account: `<agentid>` only. Read-only account `<read-only-account-id>` may be used only for the explicitly approved.

The goal remains valid: after Kairos proves a scoring version in shadow/paper evidence, Vaibhav may enable an autonomous **risk envelope**. In

2. Required architecture

There must be one shared execution kernel for manual and autonomous orders.

Research/Scoring → eligible signal → proposal
↓
authorization context
manual_owner | autonomous_worker
↓
executeApprovedProposal()
(same account/risk/quote/budget/order gates)
↓
broker adapter
↓
broker_order + append-only events
↓
sync/reconcile partial fill/fill/cancel/reject
↓
live position + protective exit

Shared execution kernel

Extract the money-moving logic in `app/api/broker/orders/route.ts` into a server-only function, for example:

executeApprovedProposal({
 proposalId,
 env: "live",
 actor: { kind: "owner", userId } | { kind: "autonomous_worker", runId },
})

The owner route keeps `requireOwner()` and calls this function. A new cron/worker route uses `verifyCronSecret()`, obtains a single-run lease,
The kernel owns all gates: broker/account resolution, autonomy policy, kill switches, signal quality, scoring-version eligibility, fresh account.

3. Authorization and activation

Autonomy uses independent keys that all must pass:

1. deployment flag `AUTONOMOUS_LIVE_ENABLED=true` (false/absent by default);
2. `strategy_config.autonomy_level = 'L4_live_small_auto'`;
3. `live_auto_enabled = true`;
4. unexpired `live_auto_enabled_until` lease;

```

5. global and US trading switches on;
6. Robinhood MCP enabled and token healthy;
7. active account resolves from the allowlist to ``<agentic-account-id>`` with role ``trading``;
8. active scoring strategy is ``live_approved`` with linked validation evidence;
9. no unresolved critical trading/data/reconciliation alert.

Unknown or missing values fail closed. Do not replace the deployment kill constant with only a DB boolean. The environment flag is a release

Settings changes require owner authorization, CSRF/request guards, a recent session or vault-PIN re-auth, typed ``ENABLE AUTO``, and an append-

LLMs and agents cannot change any activation key, cap, account, strategy lifecycle, or lease.

---

## 4. Database changes (additive)

Use the next real migration number. Apply and verify migrations before code.

### ``strategy_config``

`live_auto_enabled` boolean not null default false

`live_auto_enabled_until` timestampz

`live_auto_policy_version` integer not null default 1

`live_auto_daily_cap_usd` numeric

`live_auto_max_per_order_usd` numeric

`live_auto_min_evidence_confidence` numeric

`live_auto_max_open_positions` integer

`live_auto_max_orders_per_day` integer

Add non-negative/range checks. Defaults must be conservative and never larger than the existing ``max_order_notional_usd``, ``max_daily_notional`

### ``trade_proposals``

Add ``execution_mode` (`'manual'`|`'auto'`)`, ``policy_snapshot` jsonb`, ``auto_run_id` uuid`, and ``auto_decided_at``. Existing proposal statuses should

Add only schema-compatible status values after inspecting the current CHECK constraint. Suggested proposal states: ``pending_review``, ``approved`

### Budget reservation and actor audit

The current ``reserve_live_order_budget`` serializes per market/day correctly, but hardcodes ``approved_by_user=true``. Do not use it unchanged f

Create a new versioned RPC (for example ``reserve_live_order_budget_v2``) rather than accidentally creating an overloaded function with ambiguous

- be ``SECURITY DEFINER`` only if necessary;
- set a fixed safe ``search_path``;
- validate every enum/value and reject null/invalid notional for live BUY;
- accept ``execution_actor`` and persist ``approved_by_user = (actor='owner')``;
- use a market-local-day advisory transaction lock;
- count reserved/submitted/partial/unknown BUY notional so timeouts cannot reopen budget;
- exempt held-position SELL exits from BUY budget;
- insert the ``pending_submit`` broker row in the same transaction;
- be revoked from ``PUBLIC``, ``anon``, and ``authenticated``, granted only to ``service_role``;
- preserve the unique active-order-per-proposal backstop.

RLS remains enabled on exposed tables; service-role-only tables have explicit revoked grants. Run Supabase advisors and verify RPC permissions

### Append-only execution events

Add ``broker_order_events`` as an append-only lifecycle ledger if it does not already exist. ``broker_orders`` remains mutable current state. Each

---

## 5. Autonomous proposal eligibility

A proposal may be queued for auto execution only when all are true:

- market is US;
- account target is exactly ``<agentic-account-id>`` after allowlist resolution;
- side is BUY for a new long or SELL for a verified held position;
- signal source is deterministic and scoring strategy lifecycle is ``live_approved``;
- signal/quote/proposal are within configured freshness windows;
- evidence confidence meets the auto threshold with no override;
- no LLM veto, taint, unknown quality, or unresolved provider substitution;
- mandate permits the asset/setup;
- no earnings/ex-dividend/corporate-action blackout required by the approved setup policy;
- a protective exit can be established and monitored;
- proposal has a deterministic idempotency key and policy snapshot.

`analyst\_score >= 80` alone is not eligibility. A 0-100 heuristic is not a calibrated probability. Prefer validated `expected\_return\_bps`, and

The autonomous path has **\*\*no\*\*** `acceptLowQuality` or `acceptPortfolioRisk` override. Those remain owner-only manual actions with audited reasons.

---

## ## 6. Pre-submit gate order

Every gate is rerun immediately before broker submission:

1. actor authorization and single-worker lease;
2. policy lease/version and current strategy lifecycle;
3. trading flags, broker token, and account allowlist;
4. kill switches and critical-alert check;
5. proposal expiry, source/version, direction, quality, and mandate;
6. fresh agentic-account portfolio/positions; no fallback NAV;
7. valid integer quantity and held quantity for SELL;
8. minimum of per-order/global/auto caps in USD;
9. portfolio concentration/correlation/volatility limits;
10. fresh executable quote, max age, spread, and price-drift collar;
11. atomic daily budget/idempotency reservation;
12. `review\_equity\_order` with symbol/side/qty/account echo verification;
13. broker submit;
14. durable result/event write before returning.

Any read error, null, stale state, malformed preview, or failed audit on a money path fails closed. Do not use `FALLBACK\_NAV`. Do not downgrade.

For swing entries, prefer a marketable limit with a price collar if the MCP tool schema supports it. If only market orders are supported, consider

---

## ## 7. Exit protection is a launch blocker

Autonomous BUY cannot launch before an operational live exit path exists.

Minimum:

- current agentic holdings synced frequently during market hours;
- protective stop policy recorded at entry;
- deterministic held-position SELL through the same Execution Gateway;
- SELL quantity capped to verified available quantity after open orders/partial fills;
- stop/time/target/kill-switch exits prioritized above new entries;
- protective SELL is never blocked by BUY notional/daily budget;
- if broker-native stop/bracket support exists, preview and place it deterministically;
- if no broker-native protection exists, the runtime must have heartbeat/staleness alerts and the L4 allocation must remain tiny.

If protective exit cannot be installed or the monitor is stale, reject the BUY and disable/expire L4. Tax or dividend preferences may influence

---

## ## 8. Order lifecycle and reconciliation

External exactly-once execution cannot be assumed. Use durable at-most-once intent plus reconciliation:

- reserve before submit;
- send a client idempotency key if supported;
- ambiguous timeout/no broker ID → `unknown\_needs\_reconcile`, keep budget reserved, block symbol/proposal retry;
- clean rejection → `error/rejected`, release only according to explicit RPC policy;
- partial fill → persist filled and remaining quantities; never blindly resubmit remainder;
- order-sync worker queries broker order state during market hours and resolves submitted/partial/unknown states;
- reconciliation compares broker orders, fills, positions, proposals, and local ledger;
- any mismatch opens a critical alert and blocks new autonomous entries while allowing verified risk-reducing exits.

Kill-switch behavior: stop new orders and attempt to cancel resting entry BUY orders if an authenticated, tested cancel tool exists. Never cancel

Email is notification only. Durable DB alerts/events are the source of truth; email failure must not erase the order result.

---

## ## 9. Position sizing

Sizing is deterministic and bounded:

raw opportunity size = portfolio constructor(calibrated edge, uncertainty, volatility, correlation)

final notional = min(raw size,

global per-order cap,

auto per-order cap,

remaining daily auto budget,

mandate/name/sector/gross limits,

## available buying power)

If calibrated edge or NAV is unknown, autonomous BUY size is zero. Paper and manual pathways may show a proposal, but auto must abstain. Integ  
The LLM cannot size. A “great opportunity” can increase size only through the validated numeric opportunity model and only up to every hard c

---

### ## 10. Rollout

#### ### PA0 – architecture/schema/UI only

- additive migrations, RLS/grants/checks;
- Settings card and audited time-bounded enablement;
- deployment flag remains false;
- no autonomous order route.

#### ### PA1 – shadow autonomous decisions

- extract and test shared execution kernel;
- auto worker evaluates proposals and records would-submit/would-block reasons;
- no broker submit;
- compare shadow decisions to manual outcomes for at least the approved evidence window.

#### ### PA2 – reconciliation and live exits

- broker-order event ledger and sync worker;
- partial-fill/unknown reconciliation;
- deterministic live held-position exits/protective policy;
- chaos tests for timeouts, duplicate cron, stale account, quote failure, and DB failure.

#### ### PA3 – tiny L4 live

- owner promotes a scoring version to `live\_approved`;
- explicit deployment flag + 24-hour DB lease;
- one order at a time, tiny caps, liquid allowlist, no unresolved critical alerts;
- watched first executions and rollback drill.

#### ### PA4 – evidence-based expansion

Increase limits only by owner action after net live results, reconciliation accuracy, drawdown, and operational SLOs pass. No automatic cap ex

India auto trading is a separate architecture. It must use Kite’s official HTTP API through the same execution kernel, INR-only limits, CNC pr

---

### ## 11. Acceptance tests

- Any account other than `<agentic-account-id>` is rejected before reserve/preview/submit.
- Auto caller cannot call the broker adapter directly; static test verifies only shared kernel imports it.
- Deployment flag false, expired lease, invalid autonomy level, or DB read error blocks auto.
- Two concurrent runs for one proposal yield one reservation and at most one submit intent.
- Concurrent different proposals cannot exceed daily count/notional.
- Auto broker row records `approved\_by\_user=false` and autonomous run ID.
- Unknown outcome remains budget-reserved and cannot be retried.
- Preview mismatch on account/symbol/side/qty blocks submit.
- Missing/stale NAV or holdings blocks BUY and SELL authorization as appropriate; no fallback NAV.
- Low/unknown quality and portfolio-risk overrides are impossible in auto.
- Scoring version not `live\_approved` cannot auto trade regardless of score.
- Partial fill cannot cause oversell or automatic remainder resubmit.
- Protective exit failure blocks new entry.
- Kill switch blocks entries, preserves/permits risk-reducing exits, and handles resting BUY cancellation safely.
- Secrets/tokens never appear in logs, events, emails, or raw payloads.

---

### ## 13. PA3 Implementation Details (2026-07-10)

#### ### Per-market mode

New columns on `strategy\_config` (migration 141):

- `live\_auto\_mode\_us` TEXT DEFAULT `manual` CHECK IN (`off`,`manual`,`autonomous`)
- `live\_auto\_mode\_india` TEXT DEFAULT `manual` CHECK IN (`off`,`manual`,`autonomous`)

Three modes per market:

- `off` – autonomous-live cron skips this market entirely
- `manual` – TraderAgent creates proposals; owner clicks Approve (existing flow, unchanged)
- `autonomous` – execution kernel + live broker REST submit from cron, no per-order click

#### ### New cron: autonomous-live (14:00 UTC, weekdays)

Runs AFTER research (13:00 UTC / 9 AM ET) so same-day signals exist.

Route: `POST /api/agents/autonomous-live/cron` – timing-safe CRON\_SECRET auth.

Shadow cron at 07:30 UTC unchanged.

### US execution path (no MCP in serverless)

Direct Robinhood REST via `lib/brokers/robinhood/rest-client.ts`:  
- Token from vault key `ROBINHOOD\_MCP\_ACCESS\_TOKEN`  
- `GET https://api.robinhood.com/instruments/?symbol=X` → instrument URL  
- `POST https://api.robinhood.com/orders/` – market, gfd, account=<agentic-account-id>  
- `submitRobinhoodOrder()` (MCP) is NOT called – requires live MCP session context unavailable in serverless

### India execution path

Calls `placeEquityOrder()` from `lib/kite.ts` directly – same REST path as the owner-click flow.  
Note: requires fresh daily Kite token; if stale, order = unknown\_needs\_reconcile.

### Budget reservation

Uses `reserve\_live\_order\_budget\_v2` with `p\_execution\_actor='autonomous\_worker'` which sets `approved\_by\_user=false`.

### Missing migration 139 column fix

`trade\_proposals.market` was not added by migration 139 but IS written by autonomous-shadow.ts – shadow cron inserts fail. Migration 141 adds

---

## 12. What Claude must not do

- Do not implement the old direct `submitRobinhoodOrder()` branch.
- Do not use `605420606`; it is wrong.
- Do not remove the deployment kill flag.
- Do not build PA3 before PA1/PA2 evidence and Vaibhav approval.
- Do not use market-data/portfolio fallbacks on autonomous money paths.
- Do not duplicate the Gateway's checks in TraderAgent; extract and reuse one kernel.
- Do not call a same-app HTTP route internally for execution.
- Do not enable autonomous BUY without live exit and reconciliation.
- Do not mutate caps, lifecycle, account, or code through any LLM/tool loop.

---

# Source: features\live-exit-ladder-parity\FEATURE\_ARCHITECTURE.md  
# Live Exit Ladder Parity – Architecture Proposal

> Status: \*\*Approved and implemented; production state/shadow tables verified\*\*  
> (2026-09-12). Both monitors now call the same ladder core. The 2026-09-12  
> correction makes the stop use the prior persisted high-water mark before the  
> current observation can ratchet the trail; `live\_auto\_enabled` remains off.  
> Owner-directed 2026-09-09, sequenced ahead of Guardian Stage 1 and any live  
> enablement: "before any live enablement, build live partial-exit/ladder  
> parity and test it against the paper behavior."  
> Money path: YES. This changes how live positions are exited.

---

## 1. The gap, precisely

| behavior                        | paper (`position-monitor`)                    | live (`live-exit-monitor`) |
|---------------------------------|-----------------------------------------------|----------------------------|
| stop exit                       | yes, trailing                                 | yes, **fixed**             |
| target exit                     | **partial** (sell half, keep runner)          | **full close**             |
| runner protection after partial | stop raised to `max(entry, trailing)`         | n/a                        |
| trailing stop                   | `max(prev stop, highest × anchorPct)`         | none – stop never moves    |
| trail distance                  | position's own `initial_stop_loss / avg_cost` | n/a                        |
| highest-price tracking          | persisted `highest_price`                     | none                       |
| time stop                       | no – Decision 74                              | no – Decision 74           |

So a live winner that reaches target is fully closed; the paper equivalent banks half and lets the rest run behind a breakeven-or-better stop. A live winner that runs never tightens its stop at all – it keeps its original stop from entry indefinitely.

## 2. The real obstacle: live positions have no persistent state

This is why the gap exists, and it is the whole design problem.

`paper\_positions` is a mutable row per position carrying `highest\_price`, `stop\_loss` (moved as it trails), `initial\_stop\_loss` (immutable anchor), and `price\_target`. `position-monitor` reads it, computes, and writes the new trailing stop back each run.

Live has no such row. `reconstructAccountLivePositions` (`lib/trading/live-position-ledger.ts`) rebuilds positions \*\*statelessly\*\* from `broker\_orders` fills every run – FIFO lot consumption, average entry derived from buy fills, policy read from each lot's `policy\_snapshot`. Nothing about the position's \*history\* since entry survives between runs.

Two consequences that make this more than "add a partial-sell branch":

1. \*\*A partial-target rule would re-fire every run.\*\* With no memory that half

was already sold, the monitor would sell half of the remainder every 15 minutes while price stayed above target – bleeding the position out in fractions. The FIFO reconstruction does see the SELL fill and reduces qty, but nothing marks \*why\* it was sold, so the rule can't distinguish "already took profit here" from "fresh position at target."

2. **\*\*Trailing requires a persisted high-water mark.\*\*** Recomputing the highest price seen since entry from bars each run is possible but is a different (and more expensive) mechanism than paper's, and would diverge the moment bar coverage has a gap.

### ## 3. Design

#### ### 3.1 Reuse the paper ladder math verbatim – do not reimplement

``paperPartialTargetQuantity`` and ``paperRunnerStopPrice`` (``lib/trading/paper-quantity.ts``) are already pure, already tested, already market-aware (India whole-share floor vs US fractional scale). Live calls the same two functions. **\*\*Any divergence in ladder arithmetic between paper and live would be a bug by construction if these are shared – which is exactly the property parity requires.\*\*** No second implementation.

#### ### 3.2 Add the missing state, minimally

A new table ``live_position_state``, one row per (account, symbol), holding only what reconstruction cannot derive:

```
create table live_position_state (
 account_id text not null,
 symbol text not null,
 market text not null check (market in ('us', 'india')),
 state_mode text not null default 'legacy'
 check (state_mode in ('legacy', 'shadow', 'executable')),
 highest_price numeric, -- trailing anchor since entry
 trailing_stop numeric, -- last computed stop; never lowered
 partial_taken_at timestampz, -- non-null = broker-confirmed partial target fill
 partial_qty numeric, -- what was sold at partial, for audit
 opened_at timestampz not null, -- guards stale rows across re-entries
 direction_flip_armed_session timestampz, -- first fresh held-short score session
 updated_at timestampz not null default now(),
 primary key (account_id, symbol)
);
```

``opened_at`` is the guard against the re-entry trap: if the reconstructed position's ``firstBuyAt`` is newer than the stored ``opened_at``, the row is stale (position was fully closed and re-entered) and must be reset, not inherited – otherwise a new position would start life believing profit was already taken.

#### ### 3.3 Exit precedence – identical to paper, in the same order

1. **\*\*Stop\*\*** (trailing, never lowered) → full close.
2. **\*\*Target\*\*** and no ``partial_taken_at`` → submit ``paperPartialTargetQuantity``. Only a broker-confirmed non-zero fill may set ``partial_taken_at`` and move the runner stop to ``paperRunnerStopPrice(entry, trailingStop)``.
3. **\*\*Target\*\*** and ``partial_taken_at`` already set → no action (the runner is managed by the trailing stop from here, same as paper).
4. **\*\*Fresh score below entry threshold\*\*** → full close immediately; a confirmed two-session held-short direction flip → full close. Both share paper's freshness and debounce policy.

Trailing stop is recomputed every run as ``max(stored trailing_stop, highest_price × anchorPct)`` where ``anchorPct`` derives from the lot's own policy stop distance – matching paper's ``initial_stop_loss / avg_cost`` logic rather than a hardcoded percentage.

#### ### 3.4 Live-specific hazard paper does not have: partial fills

Paper exits are synthetic and always fill completely. A live partial-target

SELL can fill partially or not at all. The stamp must therefore be written **\*\*from the observed fill\*\***, not at submission – otherwise a rejected partial marks the position as "profit taken" and permanently disables its target branch. Proposal: stamp ``partial_taken_at`/`partial_qty`` only after the SELL appears with non-zero ``filled_qty`` as ``filled`/`partially_filled`` in ``broker_orders``, reconciled on the following run. Requested quantity is never substituted for an unconfirmed partial fill.

#### ## 4. Testing parity – the actual acceptance bar

The owner's framing was "test it against the paper behavior." Concretely:

A shared fixture of price paths (runner, stop-out, gap-through-stop, target-then-reverse, target-then-continue) is driven through **\*\*both\*\*** engines' decision functions, asserting identical exit sequences – same trigger, same quantity, same resulting stop. Any divergence is either a bug or a deliberately documented live-only difference (\$3.4 is the only expected one).

This requires extracting paper's decision logic into a pure function the test can call without a DB, mirroring how ``manual-fill-detection.ts`` was extracted from its route. That extraction is part of this work, not a prerequisite.

#### ## 5. Explicitly NOT in this proposal

- **\*\*No change to entry, sizing, or the stop distance itself\*\*** – only what happens to an already-open live position.
- **\*\*No change to ``live_auto_enabled`` or any autonomy gate.** This makes the live exit engine match paper; it does not enable anything.
- **\*\*No pyramiding / scaling in.\*\*** Paper's anti-pyramid rules are untouched.
- **\*\*No Guardian Stage 1 interaction.\*\*** Suggested stops for manual fills remain detection-and-alert only; this proposal governs Kairos-originated live positions, which have a ``policy_snapshot`` lineage a manual fill does not.
- **\*\*India live\*\*** is in scope for the shared math (the helpers are already market-aware) but India live orders remain manual-confirm, so no behavior reaches an order path there.

#### ## 6. Open questions

1. **\*\*Shadow state isolation (resolved):\*\*** shadow rows use ``state_mode=shadow``; executable runs read only ``state_mode=executable``, so a hypothetical trail or partial cannot affect an order after enablement.
2. **\*\*Reconciliation cadence for \$3.4.\*\*** The next run is 15 minutes later; whether that is tight enough for a partially-filled protective sell needs a decision.
3. **\*\*Backfill for existing live positions.\*\*** Any position already open when this ships has no ``live_position_state`` row. Proposal: initialize ``highest_price`` to the current price and leave ``partial_taken_at`` null – conservative, since it means an already-past-target position could take a partial it "should" have taken earlier, rather than assuming history it cannot know.

---

# Source: features\live-trading-hardening\FEATURE\_ARCHITECTURE.md

# Feature Architecture: Live-Trading Hardening (proposal consolidation, order-status sync, snapshot parse, pre-send confirmation, broker-side

#### ## Status

Architecture status: P1 + P2 Implemented; P3-P5 Draft (unapproved)

Architecture approved: P1 + P2 ("build", 2026-07-07)

Approved scope: P1 (proposal consolidation) + P2 (Robinhood order-status sync)

Approved date: 2026-07-07

Implementation allowed: P1 + P2 done; P3-P5 still gated

#### ### Implementation notes (2026-07-07)

- **\*\*P1\*\*** – ``trade_queue`` retired as a live path. ``/api/agents/trade``, ``/trade/approve``, ``/trade/reject`` now return 410. All generation → ``/api/agents/trader`` (canonical ``trade_proposals``). Repointed: AgentsPage "Run TraderAgent" button, the dead ``/api/markets/smart-money`` API (now reads ``trade_proposals`` aliased), and TradingPage + PortfolioPage approve/reject (→ ``/api/agents/trader``). ``smart-money/page.tsx`` already read ``trade_proposals`` (the real-money hazard was already patched). Follow-up flagged: TradingPage/PortfolioPage "queue" tabs still render raw ``agent_signals``, so their Approve needs a proposal id – product-UX decision (spawned task).
- **\*\*P2\*\*** – ``robinhood_mcp`` adapter ``getOrder`` implemented via a deterministic ``get_equity_orders`` callTool (``mcpToolJson``-parsed, escaped-safe) → state map. The ``kairos-broker-sync`` loop now transitions Robinhood orders submitted→filled, writes ``filled_qty`/`avg_fill_price`` to ``broker_orders`` and ``fill_price`/`fill_qty`/`filled_at`` (status=executed) back to the ``trade_proposal``, resolves the ``order-needs-reconcile`` System Health alert, and journals the fill.

#### ### Still Draft / not built

- **\*\*P3\*\*** live-account snapshot parse fix, **\*\*P4\*\*** pre-send confirmation screen, **\*\*P5\*\*** broker-side protective stops (the real live-risk fix). Unapproved.

#### ## Why this feature exists

The in-app Robinhood MCP live-order path now works end-to-end (first real order placed 2026-07-07). But the surrounding machinery has correctness gaps and a real live-risk hole that surfaced during that first order. This feature makes live trading **correct**, trackable, safe to operate by hand, and – critically – **protected by broker-side stops** so a live position isn't left unguarded when the app/cron isn't running. It does NOT add autonomous execution (separate, later, deliberately gated) and does NOT add limit-order entries (low value for this app's daily-cadence style).

Sequenced in phases; each is independently approvable and shippable. Recommended order: P1 → P2 → P3 (small) → P4 → P5.

#### ## Non-Goals

- **No autonomous live execution.** Every live order still requires a human approve + send. `trading_mode='auto'` stays unimplemented; its architecture is a separate doc drafted only after a paper track record justifies it.
- **No limit-order entries.** Daily-cadence trades on liquid names are fine with market orders; limit-order management (unfilled orders, cancel/replace, GTC tracking) is complexity this app's edge doesn't need. Revisit only if a real need appears (e.g. routinely trading illiquid names).
- **No standalone live-orders history page** in this feature – `broker_orders` plus Robinhood's own app already hold the record. A minimal orders list may ride along with P4, but a polished history/analytics page is out of scope.
- No change to the paper-trading loop's simulation math.

---

#### ## Phase 1 – Consolidate the proposal system (`trade_queue` → `trade_proposals``)

##### ### Purpose / current bug

Two tables model "a proposed trade":

- `trade_queue`` – written by the legacy generator `app/api/agents/trade`` and READ by the Smart Money Trade Queue UI (`/api/markets/smart-money``).
- `trade_proposals`` – written by the main `app/api/agents/trader`` and the ONLY table the Execution Gateway (`app/api/broker/orders``) reads.

So the UI shows `trade_queue`` rows, but the Gateway acts on `trade_proposals``. The paper/live "send" buttons pass the displayed row's id as a `proposal_id`` to the Gateway – which looks it up in `trade_proposals``. If the ids don't align (they don't in general), the Gateway either 404s or, worse, **acts on a different proposal with the same id**. This is a real-money correctness hazard – it's why the Send-LIVE button was NOT shipped.

##### ### Proposed behavior

- `trade_proposals`` becomes the single source of truth for proposals.
- `/api/markets/smart-money`` (and any UI reading `trade_queue``) reads `trade_proposals`` instead; the Trade Queue renders `trade_proposals`` rows, so the id sent to the Gateway is guaranteed correct.
- The legacy `/api/agents/trade`` generator (writes `trade_queue``) is retired or repointed to write `trade_proposals``. Its approve/reject routes (`/api/agents/trade/approve|reject``) are consolidated onto the `trade_proposals`` approve/reject the trader route already uses.
- `trade_queue`` is left in place (not dropped) but no longer written/read; a follow-up migration can archive it once verified unused.

##### ### Files (verify against live code before building)

- `app/api/markets/smart-money/route.ts`` (read `trade_proposals``)
- `components/dashboard/SmartMoneyPage.tsx`` (field-name mapping to `trade_proposals`` columns; the queued Send-LIVE button becomes safe here)
- `app/api/agents/trade/route.ts`` + `trade/approve|reject`` (retire/repoint)
- Any other reader of `trade_queue`` (grep before changing).

##### ### Acceptance criteria

- The Trade Queue UI renders `trade_proposals``; a row's id equals the `proposal_id`` the Gateway resolves – verified by sending a paper order and confirming the Gateway operates on the same row shown.
- No code path sends a `trade_queue`` id to the Gateway.
- No behavior change to how proposals are generated/approved beyond the table.

---

#### ## Phase 2 – Robinhood order-status sync (`unconfirmed` → filled``)

##### ### Purpose / current gap

After a live Robinhood order is placed it sits at Robinhood state `unconfirmed``, then fills. The app never learns: `lib/brokers/adapters/robinhood-mcp.ts``'s `getOrder()` is a stub, and the sync loop (`app/api/broker/orders/sync``) has no Robinhood branch. So `broker_orders.status`` stays `submitted`` and the fill price/qty/position never update in-app.

##### ### Proposed behavior

- Implement the Robinhood adapter's `getOrder(brokerOrderId)`` via a deterministic MCP `get_equity_orders`` (or the single-order variant if the server exposes one) `callTool``, parsed with `mcpToolJson`` (the escaped-content-safe helper added



```

when fixing the first order). Map Robinhood order `state`
(unconfirmed/confirmed/partially_filled/filled/cancelled/rejected) → the
adapter's `BrokerOrderState.status` union.
- The existing sync cron (`kairos-broker-sync`, every 30 min market hours)
already iterates DISTINCT brokers in open orders – it will pick up
`robinhood_mcp` orders once `getOrder` works. Update `broker_orders`
(status, filled_qty, avg_fill_price, raw_last_state redacted) + on fill write
back `trade_proposals.fill_price/fill_qty/filled_at` and a `decision_journal`
fill entry.

Files
- `lib/brokers/adapters/robinhood-mcp.ts` (`getOrder`, maybe `cancelOrder`)
- `lib/robinhood-mcp.ts` (a `queryRobinhoodOrder(id)` helper if useful)
- `app/api/broker/orders/sync/route.ts` (confirm Robinhood orders flow through)

Acceptance criteria
- A placed Robinhood order transitions in-app `submitted` → `filled` after the
sync runs, with the real fill price/qty recorded.
- `mcpToolJson` parsing handles Robinhood's escaped text content (no false
"unparseable" like the first order hit).
- Tokens/account numbers never persisted in `raw_last_state`.

Phase 3 – Live-account snapshot parse fix (small)

Purpose / current bug
`refreshViaMcp` (in `app/api/live-account/refresh-snapshot`) connected and ran
`get_accounts`/`get_equity_positions`, but its regex parse of the escaped MCP
text content left `equity`/`buying_power`/`portfolio_value` NULL and
`positions_json` empty. So the live holdings view shows nothing.

Proposed behavior
- Reparse using `mcpToolJson`: extract account equity/buying_power/portfolio_value
and the positions array from the parsed object shape (to confirm against a real
`get_accounts`/`get_equity_positions` response), not a flat regex.
- Same fix applies to `robinhoodHeldQty` (the sell-if-held gate) – parse the
positions via `mcpToolJson` so live SELLS validate correctly.

Acceptance criteria
- After a cloud MCP snapshot refresh, `live_account_snapshots` for the trading
account shows non-null equity + a populated positions array.
- `robinhoodHeldQty` returns a correct held quantity for a known holding.

Phase 4 – Pre-send confirmation screen (real-money "don't fat-finger")

Purpose
Today a live order is sent with only a JS `confirm()` (or a raw console call).
Before committing real money the user should see Robinhood's OWN reviewed
numbers (from `review_equity_order`) – symbol, side, qty, order type, estimated
price/cost, account – and explicitly confirm THAT.

Proposed behavior
- A new owner-only endpoint `POST /api/broker/orders/review` runs the Gateway's
pre-flight (all gates) + the broker's `review_equity_order` and returns the
reviewed order preview WITHOUT placing anything.
- The Send-LIVE flow becomes: click → modal shows the reviewed preview (price,
qty, est cost, account, cap headroom, any gate warnings) → user confirms →
Gateway places (reusing the already-fetched review where possible).
- Applies to all brokers (Alpaca/Kite show their equivalent preview).

Files
- `app/api/broker/orders/review/route.ts` (new, owner-gated)
- `lib/robinhood-mcp.ts` (a `reviewRobinhoodOrder()` that returns the parsed
preview; today review happens inside `submitRobinhoodOrder` – split it out)
- `components/dashboard/SmartMoneyPage.tsx` (confirmation modal) – depends on P1
so the button acts on the right table.

Acceptance criteria
- No live order is placed without the user seeing the broker-reviewed numbers
and confirming them.
- The review endpoint never places an order (pure preview) – verified.

Phase 5 – Broker-side protective stops (the real live-risk fix)

Purpose / current gap (most important item)
Stops/targets are managed APP-SIDE: `PositionMonitor` polls prices and sends a
market exit when a stop/target is hit. Fine for paper (simulated). For LIVE
money it's dangerous: if the app or its cron isn't running (deploy down, cron
missed, overnight gap), a live position has **NO broker-side stop** – it can gap
against you with nothing protecting it. Real capital needs a resting stop at the

```

broker.

```
Proposed behavior (to be refined against Robinhood's MCP order schema)
- When a LIVE entry fills, place a corresponding **broker-side protective stop**
 (a resting stop-loss order at the MAE-derived stop price the app already
 computes), if Robinhood's MCP supports stop orders (confirm via `tools/list` /
 the `place_equity_order` schema: `type: stop` / `stop_price`, or a bracket/OCO
 facility). Prefer a native bracket/OCO if available so the stop auto-cancels on
 target fill.
- If Robinhood does NOT support a resting stop via MCP: surface this clearly and
 keep app-side monitoring as the only stop, with an explicit UI warning that
 live positions are only protected while the app is running (honest about the
 limitation rather than pretending there's a safety net).
- `PositionMonitor` becomes broker-env aware: for a LIVE position with a
 broker-side stop already resting, it should NOT also fire its own market exit
 (avoid double-exit / racing the broker stop). It reconciles against the
 broker's open orders first.
- All stop placements go through the same Gateway gates + are deterministic (no
 LLM), same as entries.
```

```
Open questions to resolve before building (verify, don't guess)
- Does Robinhood's agentic MCP `place_equity_order` accept `type: "stop"` /
 `stop_price`, and/or a bracket/OCO? (Inspect the live tool schema.)
- Partial-fill handling: stop qty must track the actual filled qty, not the
 ordered qty.
- Cancel/replace semantics for adjusting a trailing stop broker-side vs
 app-side.
```

```
Acceptance criteria
- A LIVE entry fill results in a resting broker-side stop at the intended price
 (if supported), verified on the account.
- PositionMonitor does not double-exit a live position that has a broker stop.
- If broker stops are unsupported, the UI states plainly that live stops are
 app-side only (running-app dependent).
```

---

## Cross-cutting guardrails (all phases)

```
- Every live order/stop/cancel goes through the Execution Gateway's full gate
 set (owner, CSRF, kill switches, `robinhood_mcp_enabled`, per-market
 trading_enabled, notional cap, fresh-quote/drift, allowlist), deterministic,
 no LLM in the write path.
- `needs_reconcile` semantics preserved; no auto-retry of any write.
- No secrets/account numbers in persisted `raw_last_state`.
- `robinhood_mcp_enabled` remains the fast kill switch; `max_order_notional`
 stays a hard cap.
```

## Deferred to their own specs (explicitly not this feature)

```
- **Autonomous live mode** (`trading_mode='auto': auto-approve + auto-send +
 enforced limits + master autonomous kill-switch) – separate doc, gated on a
 paper track record.
- **Limit-order entries** – only if a concrete need appears.
- **Standalone live-orders history/analytics page.**
```

## Approval

Architecture approved: No  
Approved scope: None  
Implementation allowed: No

---

# Source: features\llm-council-shadow\FEATURE\_ARCHITECTURE.md  
# LLM Council – Shadow Annotation

```
Status: DEFERRED by owner decision, 2026-08-20. Not approved, not implemented.
Revive when: the US equity book clears its own evidence floor (`n >= 60` closed trades with h10 labels AND `nEffective = n/10 >= 12` per m
Date: 2026-08-20
Scope: measure whether a multi-model review would have improved decisions. Writes no direction, no size, no order.
```

## The question this answers

Not "should an LLM help decide" – that is unanswerable today. The answerable  
question is:

> \*\*When the council disagrees with the deterministic scorer, who turns out to be right?\*\*

That has a real answer, from data we already collect, in about three months. Until  
it is answered, wiring model opinion into the money path is a guess wearing a  
number.

## Why this shape and not the obvious one

The obvious design – several models score a stock, take the median, gate on it –

does not remove non-determinism. It **moves** it. The median of three stochastic estimates is a stochastic estimate with a deterministic wrapper. `PROJECT_DECISIONS`` already closed this door on 2026-07-15: an LLM `direction`` on a held name became an executable exit and taught the learner from LLM-created outcomes. The fix made direction deterministic (`lib/signal-direction.ts``) and demoted the model's opinion to `research_packets.raw_data._original_direction`` – advisory, stored, read by nothing.

**This proposal is that same pattern, applied deliberately and measured.**

**Where an LLM genuinely adds something**

Worth being precise, because "LLMs are smart" is not an argument.

| Dimension            | Current input                                 | Can a model add?                                                        |
|----------------------|-----------------------------------------------|-------------------------------------------------------------------------|
| Fundamental          | P/E, FCF yield, insider buying – real numbers | <b>No.</b> Arithmetic on fetched fields. A model would only paraphrase. |
| Technical            | RSI/EMA/ADX/ATR – deterministic               | <b>No.</b>                                                              |
| Macro                | FRED regime, 3/8 indicators   Marginal.       |                                                                         |
| <b>Sentiment</b>     | StockTwits bull/bear %, GDELT tone            | <b>Yes.</b> These are thin proxies for "what is actually being said".   |
| <b>News / events</b> | not scored at all                             | <b>Yes.</b> M&A, guidance, litigation, supply shocks live in prose.     |

So the council's job is narrow: **read the unstructured evidence the numeric pipeline cannot, and say whether it contradicts the score.** Not "rate this stock".

**Architecture**

**The isolation boundary (load-bearing)**

Modelled on the Property invariant and the risk-research display join (invariant R1). **No council output is read by any scorer, eligibility gate, sizing rule, order path, exit, promotion gate, or learner.** Enforced by a coupling test in the style of `tests/risk-research-annotation.test.ts`` – which pins R1 and is itself falsification-tested – not by convention.

**Flow**

- Runs AFTER** the deterministic score exists, on entry-eligible decisions only. It never sees a symbol the scorer has not already judged.
- Grounding contract.** The prompt carries ONLY data already fetched this run: the frozen `features`` blob, the score breakdown, and the fetched news/sentiment payloads. Each fact carries its own as-of timestamp. Recalled facts are forbidden – the same \$1 rule already in the research prompt ("you do not know prices, P&L, RSI values; every number must trace to a tool call made in THIS run").
- N models, independently.** 3 by default from the 7 already wired (`lib/llm-keys.ts``: anthropic, deepseek, groq, gemini, grok, openai, glm). Independent calls, no shared context – otherwise they are one opinion.
- Structured verdict only:** `{ agrees: bool, concern: enum, confidence: 0-1, citation: string }`. Free prose is stored but never parsed into a field – the `parseStrategyNotes`` precedent, where model output can only ever land in one string column and that is provable rather than prompt-dependent.
- A referee model** sees the deterministic score, the fetched evidence, and the N verdicts, and emits one `council_verdict``. It is stored. **It decides nothing.**

**Storage**

New table `council_annotations``, append-only, keyed to `observation_id``. A separate table – not a column on `decision_observations`` – so no existing consumer can pick it up by accident, and so it can be dropped wholesale if the answer is "no".

**Cost – measured, not guessed**

Current volume: **145 observations today, 208 distinct symbols over 7 days, 786 entry-eligible decisions in 7 days** ≈ 112/day eligible.

At 3 models + 1 referee = 4 calls per eligible decision ≈ **450 calls/day**. That is the honest number and it is not small. Mitigations, in order:

- Score **only entry-eligible** decisions (112/day, not 145 × all).
- DeepSeek is already the default everywhere and is cheap; `llm_call_log`` shows only 3 distinct models used in 30 days, so provider spread is a new cost.
- Cap per run and sample if the cap binds – a sampled measurement still answers the IC question, it just answers it slower.

**This is the main argument against building it now.** It is worth stating plainly rather than burying.

**Predeclared success criterion – set BEFORE any data**

Copying the discipline in `aggregateAtrExitEvidence``:

- The unit of analysis is a **disagreement**: council says `agrees=false`` on an entry-eligible decision.
- Outcome = the existing forward label at h10 (`observation_labels``), so no new

```

labelling machinery.
- **Date-clustered** – one decision date is one draw; same-day decisions share a
 market shock.
- Floor: `n` >= 60` disagreements AND `nEffective` =  $n / 10 \geq 12$ ` before the
 comparison may be cited at all.
- Success = disagreements underperform agreements by a date-clustered `t` >= 2`.

Kill condition, also predeclared

If the floor is met and `|t| < 2`, the lane is **deleted, not tuned**. Written now
so it cannot be relaxed once the data is in.

Explicitly NOT in scope

No direction. No sizing. No orders. No exits. No promotion input. No change to
`analyst_score`, the mandate, or any gate. Nothing here may be read by the money
path. Cross-model *agreement* is not a green light and is not surfaced as one.

Operator view

For: news and sentiment are genuinely under-served, the models are already
wired, and shadow measurement risks nothing. If disagreements predict
underperformance, that is a real edge and you would have earned it honestly.

Against, and my recommendation: the equity book has ~16 clean US sessions, no
demonstrated edge, and sits below the `nEffective` >= 12` floor for the strategy
that already exists. Four separate defects shipped into this system on 2026-08-20
alone. The binding constraint is not ideas – it is verification capacity. ~450
LLM calls/day is a real recurring cost for a question that cannot be answered for
three months.

I would defer until the equity book clears its own evidence floor. If built
anyway, it must stay a shadow lane with the coupling test and the kill condition,
so it cannot quietly become a sizing input the way the 2026-07-15 exit hole did.

Open questions for the owner

1. Build now, or defer until the core is measurable? (I recommend defer.)
2. If now: 3 models, or 2 plus a referee to halve the cost?
3. US only, or both markets? India's sentiment coverage is already 0% (GDELT
 sparse), so India is where a model would add most – and where grounding data
 is thinnest, which cuts both ways.

Source: features\local-historical-replay\FEATURE_ARCHITECTURE.md
Local Historical Replay Worker

Status: APPROVED and implemented by owner direction on 2026-07-29
Scope: deterministic offline research, immutable experiment lineage, read-only app visibility
Money-path status: unreachable; diagnostic results cannot change scoring, promotion, positions, cash, orders, or broker state

1. Decision

Kairos will run large historical experiments on the operator's machine, where the
verified evidence store already lives. Vercel and the browser never read the raw
files. Supabase stores only the predeclared experiment, dataset/universe/run
fingerprints, compact result summary, and timestamps.

The worker extends `backtest_experiments`, the existing immutable experiment
lineage. It does not create another result ledger and does not populate the older
`replay_eligibility_` tables.

Initial executable scope is an India, price-only, cross-sectional OOS diagnostic
of `kairos_technical_score_v1` using official NSE bhavcopy. US bulk SEC facts and
FRED vintages remain available to later sealed feature studies, but Kairos does
not claim a US price backtest until a survivor-safe adjusted-price source is bound.

2. Runtime Boundary

```

flowchart LR

```
CLI["Local operator CLI"] --> VERIFY["Verify manifest and every file hash"]
```

```
VERIFY --> PREDECLARE["Insert immutable backtest_experiments plan"]
```

```
PREDECLARE --> REPLAY["Two-pass network-free OOS replay"]
```

```
REPLAY --> COMPLETE["Write compact result and fingerprints once"]
```

```
COMPLETE --> API["Owner-only read API"]
```

```
API --> UI["Backtest historical runs"]
```

## VERIFY -. "raw files stay local" .-> STORE["User profile evidence store"]

The CLI may contact Supabase only to register and complete lineage. After predeclaration, the replay calculation has no provider client and makes no market data API call. It reads the manifest-bound local files only.

### ## 3. Immutable Plan

An experiment is inserted before normalized market rows are read. Its plan fixes:

- dataset IDs and expected SHA-256 fingerprints;
- market, data cutoff, date window, horizon, cadence, and fold layout;
- exact edge/formula version and git commit;
- point-in-time universe rule and liquidity lookback;
- minimum cross-section and action-exclusion policy;
- one predeclared variant and diagnostic evidence class.

`plan\_fingerprint` is unique. A completed plan is never overwritten. Re-running the same plan returns the existing result; changing any input creates a new plan.

Post-run defects do not rewrite that result. An append-only `backtest\_experiment\_quality\_reviews` row marks the artifact `accepted\_diagnostic` or `invalidated`, records the reason, and may point to its replacement. Backtest must show this review state prominently.

### ## 4. India Replay Method

1. Verify the NSE daily-bar and corporate-action manifests byte-for-byte.
2. First pass builds a trailing 20-session turnover rank from date-ordered bars.
3. On each predeclared non-overlapping as-of date, select the top 200 eligible equities using only information available on or before that date.
4. Reject symbol/date observations with a split, bonus, rights issue, demerger, or ambiguous price-affecting action in the feature/label window. Raw dividends are not converted into total returns, so the run is explicitly diagnostic.
5. Second pass loads only the union of selected symbols plus `NIFTYBEES`.
6. Compute the exact production technical composite from candles through `as\_of`. Compute the forward return only after `as\_of`; immature labels are dropped.
7. Calculate raw-rank Spearman IC per date. No winsorization precedes ranking. Aggregate mean IC, sample sigma, IC IR, and Newey-West t-stat.
8. Persist the bounded per-date audit series plus dataset, universe, and run fingerprints.

The result is research evidence, not a simulated portfolio claim. Costs, capacity, tax, dividends, and execution are not represented and the UI must say so.

### ## 5. Database Contract

`backtest\_experiments.experiment\_type = 'historical\_replay'` uses the existing service-role-only table and mutation guard. Required plan fields include formula, horizon, validation mode, trial family, universe policy, data cutoff, exact code commit, structured validation spec, unique plan fingerprint, and predeclared variant/count. Completion requires all three SHA-256 fingerprints and a structured result. No anonymous or authenticated table grant is added.

### ## 6. App Surface

`GET /api/agents/backtest/historical`:

- calls `requireOwner()` before using the service client;
- returns only bounded historical-replay rows and compact summaries;
- never returns raw evidence paths, credentials, or source files;
- has no POST method and cannot start a process on the user's desktop.

Backtest shows market, status, edge, date range, evaluated/skipped dates, cross-section, mean IC, sigma, HAC t-stat, evidence class, dataset fingerprint, and limitations. It visually separates these runs from legacy signal replay.

### ## 7. Safety and Non-Goals

- No LLM in acquisition, features, labels, statistics, or persistence.
- No raw upload to Supabase, Vercel, Git, or the browser.
- No network fallback when a local file is missing or invalid.
- No use by ResearchAgent, PaperTrader, TraderAgent, PositionMonitor, LearnerAgent, promotion RPCs, strategy policies, or broker adapters.
- No automatic schedule. The operator runs the CLI for an approved fixed plan.
- No inference that one diagnostic run validates a strategy. Promotion remains separately fail-closed under existing PIT/walk-forward governance.

### ## 8. Acceptance Gates

1. A changed byte or manifest fingerprint aborts before result computation.
2. The experiment plan exists before the first normalized row is read.
3. No candle after `as\_of` enters feature computation.
4. No label is shorter than the declared session horizon.
5. Exact-plan reruns are idempotent; completion is write-once.

6. Anonymous/authenticated clients still cannot access the table.
7. The app API rejects non-owner callers.
8. Focused tests, TypeScript, production build, and one real local run pass.

---

# Source: features\macro-dimension-role\FEATURE\_ARCHITECTURE.md  
# Macro dimension – role correction

> Status: **\*\*Stage 1 COMPLETE (measure-only, shipped). Stages 2–3 not approved.\*\***  
 > Created: 2026-09-02 · Stage 1 results added 2026-09-02  
 > Scope: how `macro\_score` enters the entry decision. Money path.  
 > Owner has confirmed "trade less in a dangerous regime" IS an intended policy  
 > (\$7), which changes the Stage 2 recommendation – see §9.

## 1. What macro is today

`fetchMacroScore` (`lib/data/scores.ts:403`) returns a **\*\*single market-wide scalar\*\*** derived from the weekly `macro\_regime` row (`week\_of` is a Monday; `kairos-macro-sentinel` runs Mondays 12:30 UTC). It is US-only by construction – all 8 indicators are US FRED series – and India correctly returns `available: false`, so India's weights renormalize onto the other four dimensions.

It carries **\*\*weight 0.15\*\*** in `computeWeightedAnalystScore`  
 (`fundamental 0.30 / technical 0.25 / sentiment 0.20 / macro 0.15 / insider 0.10`).

Production, 2026-08-01 onward:

| market | macro available  | distinct macro values        | avg analyst score |
|--------|------------------|------------------------------|-------------------|
| us     | 100% (3,970 obs) | 6 (one per week)             | 62.99             |
| india  | 0% (1,144 obs)   | 1 (placeholder 50, excluded) | 74.81             |

## 2. The defect, stated precisely

**\*\*Macro is not broken as information. It is misplaced as a ranking input.\*\***

On any given day macro is a constant `c` across every symbol. The composite is

$\text{analyst} = 0.30f + 0.25t + 0.20s + 0.15c + 0.10i$

Since `0.15c` is identical for every candidate, it **\*\*cannot change the ordering of candidates\*\***. Removing macro and renormalizing gives  
 `(0.30f + 0.25t + 0.20s + 0.10i) / 0.85` – a positive scalar multiple of the same quantity, so the ordering is **\*\*identical\*\***. This is algebra, not a measurement, and it needs no evidence floor to assert.

Two consequences follow:

1. Macro's 0.15 weight buys **\*\*exactly zero\*\*** information about **\*\*which\*\*** stocks are selected.
2. Its entire effect lands on the **\*\*level\*\*** of every score at once, and therefore on **\*\*how many\*\*** candidates clear the absolute entry threshold of 60.

So macro is, de facto, a **\*\*market-timing gate implemented as a weight\*\*** – which makes its timing behaviour implicit, uncalibrated, and invisible. Nobody declared "at danger\_score X, buy less." It emerges from the product of a weight and a threshold that were each chosen for unrelated reasons.

> This is the correct reading of the `macro` row in the Dimension Rank IC panel:  
 > `mean IC 0.0000, SD 0.0000, t -, positive days 0%`. That is **\*\*not\*\*** a  
 > measurement that macro has no predictive value. Its cross-sectional IC is zero  
 > **\*\*by construction\*\***, because a constant has no cross-sectional variance. The  
 > panel must not be used to argue macro should be dropped as information.

## 3. Measured impact

Paired symbols across the 2026-08-24 macro update (73 symbols present both days):

| dimension          | mean Δ 08-21 → 08-24 | contribution to composite |
|--------------------|----------------------|---------------------------|
| fundamental        | 0.00                 | 0.00                      |
| technical          | +7.73                | +1.93                     |
| sentiment          | +0.62                | +0.12                     |
| <b>**macro**</b>   | <b>**+28.00**</b>    | <b>**+4.20**</b>          |
| insider            | -0.55                | -0.06                     |
| <b>**analyst**</b> | <b>**+5.85**</b>     | –                         |

Macro supplied roughly **\*\*72% of the entire book-wide re-rating\*\***. Eligibility share moved **\*\*52.6% → 76.0%\*\***; **\*\*16 of 73 symbols became newly eligible and 0 lost eligibility\*\***, with no company-specific evidence changing.

Weekly macro values since 2026-08-01: `57, 67, 65, 47, 75, 69`. The 47→75 range is 28 points, i.e. a **\*\*±4.2 point\*\*** swing on every US score from a signal that

cannot rank anything.

## 4. Why this matters more than it looks: the rank gate is off

`lib/scoring/rank.ts` implements the cross-sectional half of a **hybrid entry gate** (absolute threshold AND within-group percentile). It is **inactive**: `rank_pct_min` defaults to 0 and is only enabled from the champion genome (`app/api/agents/research/cron/route.ts:557-570`). Neither champion has a genome at all, so `rank_pct_min` is null.

Verified: `rank_pct` is null and `rank_rejected` is 0 on **every** US signal for the 14 sessions from 2026-08-18 to 2026-09-02.

This is **off by design, not broken** – the code comment says default 0 means "gate inactive  $\Rightarrow$  we do NOT touch agent\_signals at all". But the consequence is that **the absolute threshold is currently the only active entry gate**, which is precisely what makes macro's level shift fully load-bearing. A rank-based gate is immune to a common level shift by construction.

## 5. Options

**A.** Remove macro from the composite; express regime as a declared policy. Ordering is provably unchanged (§2). The regime signal becomes an explicit gate – a sizing throttle or an entry pause at a predeclared danger level. **Pro:** the timing behaviour becomes visible, calibratable and testable. **Con:** choosing the policy threshold needs its own evidence, which does not exist yet.

**B.** Make macro cross-sectional. Give each symbol a regime sensitivity (sector beta), so macro can actually rank. **Con:** requires a sensitivity model that does not exist – `lib/scoring/instrument-taxonomy.ts` has no volatility or liquidity buckets. Large speculative build.

**C.** Activate the rank gate. Set `rank_pct_min` on the champion genome so selection is cross-sectional. A common level shift then cannot change who is selected, which neutralises the macro churn without touching the macro dimension at all. **Already built and tested; needs a genome, not new code.**

**D.** Do nothing to live behaviour; shadow first. Record counterfactual eligibility with macro excluded, for N sessions, and decide from that.

## 6. Recommendation

**D then A, with C evaluated in parallel.**

Stage 1 (measure-only): shadow the macro-excluded composite, recording per session how many candidates change eligibility. No live change. This is the only part with no evidence risk.

Stage 2 (needs Stage 1): move macro out of the composite and into a declared regime policy, with a predeclared threshold and its own falsification test.

Stage 3 (separate decision): whether to activate the rank gate. This is the highest-leverage available change and requires no new code, but it alters live selection and therefore needs its own approval and its own shadow.

**Explicitly NOT proposed:** tuning or zeroing the macro weight from the Dimension Rank IC table. That IC is zero by construction and is not evidence about macro's value.

## 7. Open question for the owner

Does the intended behaviour "trade less in a dangerous regime" exist as a deliberate policy? If yes, it should be stated as one and Stage 2 implements it. If it was never intended and the level effect is incidental, Stage 2 is simply a removal. The answer changes what Stage 2 builds.

---

## 8. Stage 1 results (measure-only, 2026-09-02)

Implemented as `lib/learning/macro-counterfactual.ts` + `GET /api/agents/macro-shadow?market=us|india`. It re-scores every recorded decision with macro excluded, using the `PRODUCTION` scorer (`computeWeightedAnalystScore` plus `capEtfLikeScore`) so the shadow cannot drift from live scoring. Measure-only: it writes nothing and nothing reads it.

**This turned out to be a retrospective replay, not a forward shadow.** The ledger already stores `weights_used` (effective, post-renormalization), `availability_mask`, all five dimension scores, the threshold and `entry_eligible` – 100% populated across all 5,672 US rows. Waiting N sessions to collect what was already recorded would have delayed the answer for nothing.

### Replay fidelity had to be fixed twice before any number was publishable

| stage | match rate | cause |

```

|---|---|---|
| first run | 0.8646 | ETF cap not applied |
| after `capEtflLikeScore` | 0.9721 | legacy pre-cap cohort remains |
| analysed set | 1.0000 | unreplayable rows excluded, not guessed |

1. **ETF cap omitted.** 768 rows failed because production caps ETF-like scores
 at 65 after weighting (`research-agent.ts:1714`). This is a KNOWN trap – the
 Evidence Router hit it identically ("recorded=65, replayed=<uncapped>", 45
 failures across 8 symbols), which is why `capEtflLikeScore` is exported. The
 first version of this module repeated the documented mistake.
2. **Legacy cohort.** 158 rows still fail, max gap 35 points, and only 13 are
 within ±1 – so NOT rounding. 149 of 158 fall in 2026-07-06..07-22, ETFs
 recorded at `analyst_score = 100` where the capped replay says 65: scored
 before the cap existed. A row whose PRESENT cannot be reproduced cannot have
 its counterfactual trusted, so those rows are excluded from the flip counts
 and from the denominator, never counted as "unchanged".

> A mismatch RATE alone cannot distinguish "3% off by one point" (rounding,
> ignorable) from "3% off by 35" (a missing rule). Reporting only the rate nearly
> ended this investigation at the wrong conclusion; `replayMaxAbsMismatch` and
> `mismatchesByDate` exist because of that.

```

### The finding

Analysed 5,514 of 5,672 US decisions (158 excluded above).

```

metric	value
certain eligibility losses without macro	**160 (2.9%)**
possible gains (UPPER BOUND)	53
unchanged	5,207
mean score delta	-0.68

```

**\*\*Macro is already doing what the owner wants, and the sign flips with the regime – which is the point:\*\***

```

week	macro	mean delta without macro	certain losses	possible gains
08-18 ... 08-21	**47** (risk-off)	**+2.2 to +2.6**	0-3	3-20
08-24 ... 08-28	**75** (calm)	**−1.3 to −2.0**	6-13	0
08-31 ... 09-02	69	−0.9 to −2.1	3-11	0

```

At macro 47 removing macro RAISES scores – the low regime reading was suppressing the book. At macro 75 removing it LOWERS them – the calm reading was expanding the book. That is exactly "trade less in a dangerous regime", already happening.

## 9. Revised recommendation

\$6 proposed moving macro out of the composite. **\*\*Stage 1 substantially weakens that\*\***, and the owner has since confirmed the timing behaviour is intended.

The mechanism is crude – a level shift against an absolute threshold, whose magnitude is a by-product of a weight and a cutoff each chosen for other reasons – but its DIRECTION is correct and its size is modest (2.9% of decisions, ~1-3 score points). Ripping it out would remove a working behaviour to fix an aesthetic complaint.

So Stage 2 is no longer "remove macro". It is: **\*\*keep the behaviour, make it explicit and calibratable.\*\*** Express the regime response as a declared policy with a named threshold and its own falsification test, so the amount of suppression is a choice rather than an artefact. Whether that policy should throttle SIZING rather than eligibility is the open design question – sizing degrades smoothly, whereas an eligibility cliff moves a slab of the book at once.

Still **\*\*not\*\*** proposed: changing macro's weight on the basis of the Dimension Rank IC table. That IC is zero by construction.

Stage 3 (activate the cross-sectional rank gate) is unchanged and still the highest-leverage available change: it needs no new code, only a champion genome with `rank\_pct\_min`, and it makes selection immune to common level shifts by construction. It alters live selection, so it needs its own approval and shadow.

---

```

Source: features\manual-trade-guardian\FEATURE_ARCHITECTURE.md
Manual Trade Guardian – Architecture Proposal

```

```

> Status: **Stage 0 approved 2026-09-09 and IMPLEMENTED.** Migration applied
> and verified (`agentic_position_ledger`, `learner_runs.live_trades_closed`/
> `live_win_rate`). Cron `kairos-manual-fill-detect` registered (`*/15 13-21
> * * 1-5`). Route: `app/api/agents/manual-fill-detect/cron/route.ts`. Pure
> core: `lib/trading/manual-fill-detection.ts`, tested
> (`tests/manual-fill-detection.test.ts`, 11 cases). Stage 1 (the
> Stage 1 is implemented 2026-09-09: an owner-armed software-stop plan,
> immutable plan-event ledger, owner panel on Live Portfolio, and a gated

```



```
> monitor. It is deliberately **not** a `trade_proposals` approval card: that
> existing control executes a SELL immediately, which is unsafe for arming.
> Owner ask (2026-09-08): "if I buy manually in the agentic account, add the
> right stop loss ASAP; I'll sell manually myself; track manual vs app-placed."
> This document is the architecture gate required before any code is written.
```

```
> **2026-09-09 remediation:** the initial implementation did not satisfy its
> own position-ledger contract. The first snapshot had no baseline state,
> partial sells were skipped, full exits were hard-coded manual, missing cost
> basis became current price, and client roles retained TRUNCATE. The approved
> repair adds lease-backed bootstrap state, records every quantity transition,
> supports aggregate order attribution plus explicit unknown, preserves null
> cost basis, adds the market-calendar gate, and reconciles the schema/grants
> through `20260909153000_guardian_and_broker_preflight_safety.sql`.
```

---

## 1. What's true today (verified against code + production, not assumed)

- **\*\*`PositionMonitor` never watches live Robinhood positions.\*\*** It reads only `paper\_positions` (`app/api/agents/position-monitor/route.ts`). A live position – manual or agentic – gets no trailing stop, no exit logic, nothing.
- **\*\*`strategy\_config.protective\_orders\_enabled = false`\*\*** in production. Even where protective-order placement exists (Kite side), it's off by default.
- **\*\*`broker\_orders` has no source column\*\***, and it only ever gets a row when *\*Kairos\** proposes an order. `broker/orders/sync` reconciles Alpaca/Kite fills back onto rows that already exist – it never discovers a fill Kairos didn't create. A manual Robinhood-app buy creates **\*\*no row anywhere\*\***.
- **\*\*Robinhood order placement in this codebase supports `market` and `limit` only\*\*** (`RobinhoodOrderInput.type`, `lib/robinhood-mcp.ts:585`). There is no broker-side stop order wired for Robinhood. Kite's "stop" path (`lib/kite.ts:223`) is actually a plain LIMIT order placed at the stop price – also not a broker-native stop.
- **\*\*Consequence: "attach a stop loss" cannot mean a broker-resident stop order today.\*\*** It has to mean the same thing it means for paper positions – a software watch that submits a market/limit SELL when the price breaches the level, running on Kairos's own cron. This must be stated to the owner plainly in the UI, not implied to be a real broker stop.
- **\*\*No cron runs more often than daily.\*\*** `vercel.json`'s finest schedule is `autonomous-live` at once/day per market. "ASAP" is therefore bounded by whatever polling cadence this feature adds – realistically 5-15 minutes on Vercel's free-tier cron granularity, not real-time. Stating this honestly matters more than the word "ASAP" – a false real-time claim is the kind of cryptic status this project's own conventions forbid.
- **\*\*`trade\_proposals` already has exactly the shape an approval card needs:** `symbol, side, qty, order\_type, thesis, key\_risks, status, approval\_expires\_at, robinhood\_order\_id, account\_number, execution\_mode`. This is the existing owner-approval mechanism (Settings → whatever surfaces pending proposals today) and should be reused, not duplicated.
- **\*\*`lib/research-agent.ts` already computes `stop\_loss\_pct`\*\*** from `tradingMandate` for every paper entry (`lib/research-agent.ts:1786`). The same function is the correct source for a manual fill's stop – not a new formula.
- Account allowlist is unchanged by this feature: `` is the only order-permitted account; everything else stays read-only.

## 2. Design

### 2.1 Detection (new, tight-cadence cron)

New route `POST /api/agents/manual-fill-detect/cron`, cron-secret gated, scoped to `` only.

Each run:

1. Fetch live positions for `` via `fetchRobinhoodBrokerAccounts` (same call `holding-risk` already makes – reuse, don't re-implement).
2. Compare against a new **\*\*position-ledger snapshot\*\*** (see schema below) – the last known qty per symbol this feature itself recorded.
3. A qty increase with **\*\*no matching open/filled `broker\_orders` row\*\*** for that symbol/qty/window = a manual fill. A qty increase that DOES match an already-known `broker\_orders` fill is Kairos's own trade – not manual, ignore.
4. Every fill (manual or matched) gets written to the position ledger with `source: 'manual' | 'agentic'`. This is the "track what was done manually vs by the app" ledger the owner asked for, independent of whether a stop ever gets attached.

Cadence: as tight as the free-tier cron budget allows (proposal: every 15 minutes during US market hours only, to keep the polling budget small and honest about latency). Exact number is an implementation decision, not an architecture one – flagged as open below.

### 2.2 Stop computation (reuse, not reinvent)

On detecting a manual fill: call the same stop-loss function

`research-agent.ts` uses for paper entries, with the mandate's  
`stop\_loss\_pct` and the fill's entry price. Output: one suggested stop price,  
nothing else. No new stop-sizing model.

### 2.3 Approval – arm a plan, never submit on approval

**\*\*This is the load-bearing decision in this document.\*\*** The first draft said a  
`trade\_proposals` SELL could be used for this approval. That was wrong: its  
existing approval handler executes the SELL at once. Guardian instead creates  
a dedicated `guardian\_protection\_plans` record in `pending\_approval`:

account\_id: <agentic-account-id>

symbol, qty, entry\_price, stop\_price

status: pending\_approval → armed

The owner sees it in the dedicated Live Portfolio **\*\*Manual Trade Guardian\*\***  
panel and taps **\*\*Arm stop\*\***. This records owner attribution plus an immutable  
`armed` event. It submits no broker order. The UI states plainly that this is  
a Kairos software stop, not a broker-resident stop.

**\*\*Why not auto-place:\*\*** the project's own push-back mandate flags "Running  
real TraderAgent orders without approval\_required mode." An unattended stop  
placed the instant a fill is detected is the same class of autonomy jump that  
rule exists to catch, even though the intent here is protective. If the owner  
later wants it fully automatic, that is a separate, explicit decision –  
Section 4.

### 2.4 Once a stop is armed – who watches it?

Because Robinhood has no broker-native stop, a sibling monitor reads an armed  
plan only after **\*\*both\*\*** `AUTONOMOUS\_LIVE\_ENABLED` and  
`strategy\_config.live\_auto\_enabled` are on, the app is not paused/locked, the  
active US account is exactly `<agentic-account-id>`, and the US market is open. It then  
requires a fresh quote at or below the committed stop, atomically claims the  
plan, creates one `autonomous\_live` market SELL proposal, and calls the same  
hardened execution gateway used by the live exit monitor. The gateway still  
re-verifies held quantity, broker allowlist, autonomy level, market controls,  
and kill switches. Any failed/ambiguous submission becomes  
`trigger\_needs\_reconcile`; it never silently retries another SELL.

### 2.5 Manual sell – explicitly out of scope for automation

The owner said they'll sell manually themselves. Nothing here watches for or  
reacts to a manual sell beyond recording it in the position ledger  
(`source: manual`) when qty drops with no matching Kairos sell order.

## 3. Schema (applied and verified 2026-09-09)

create table agentic\_position\_ledger (

id bigint generated always as identity primary key,

account\_id text not null, -- '<agentic-account-id>' only, checked

symbol text not null,

qty numeric not null,

avg\_cost numeric,

source text not null check (source in ('manual','agentic')),

detected\_at timestamptz not null default now(),

matched\_broker\_order\_id bigint references broker\_orders(id),

suggested\_stop\_price numeric,

stop\_proposal\_id bigint references trade\_proposals(id),

created\_at timestamptz not null default now()

);

-- append-only by convention (matches broker\_orders/observation\_labels idiom);

-- RLS: service-role write, owner read, same posture as broker\_orders.

Stage 1 adds `guardian\_protection\_plans` (the current state, one per detected  
manual-buy ledger entry) and append-only `guardian\_protection\_events`. The

database function `create\_guardian\_protection\_plan` inserts both atomically:  
there can be no armed/visible plan without its `created` evidence event.

#### ## 4. Explicitly NOT in this proposal

- **\*\*Does not touch `strategy\_config.live\_auto\_enabled` or `autonomy\_level`.**  
That is a separate decision the owner has not yet made (raised, not answered, this session).
- **\*\*Does not place any order automatically.** Detection and stop-sizing are automatic; placement is always owner-approved.
- **\*\*Does not add a broker-native stop order type.** None exists for Robinhood in this codebase; building one is a distinct, larger scope (would need Robinhood MCP support that hasn't been confirmed to exist) and is not assumed here.
- **\*\*Does not watch Kite/India manual fills.** The owner's ask was specifically the agentic (Robinhood, ``lib/protective/`); extending detection there is a future, separate ask.

#### ## 5. Decisions implemented

1. **\*\*Poll cadence:\*\*** existing 15-minute US-market-hours Guardian cron. This is monitoring latency, not a real-time or broker-native guarantee.
2. **\*\*Sell order on breach:\*\*** market. A stop-loss's safety objective is to reduce exposure; a limit could remain unfilled during a fast fall.
3. **\*\*No minimum notional floor:\*\*** every detected manual buy with a real cost basis receives a suggestion. Noise is cheaper than leaving a small test position falsely unprotected.

#### ## 6. Sequencing

Detection + ledger → pending Guardian plan → owner arms the plan → later verified breach → gateway-gated market SELL. The user has not enabled the existing live switch, so the deployed monitor presently refuses before quote or broker work. No external order was sent while implementing this feature.

---

# Source: features\market-local-risk-configuration\FEATURE\_ARCHITECTURE.md  
# Market-Local Risk Configuration

> Status: **\*\*DRAFT - NOT APPROVED FOR IMPLEMENTATION\*\***  
> Trigger: 2026-07-26 configuration audit

#### ## Decision Needed

US and India should have independently configurable execution risk limits. Their genomes already evolve independently, but the owner sizing ceiling is still read from the global `strategy\_config.position\_size\_pct`. That is an unsafe hybrid: a change intended for one currency/book silently changes the other market's maximum new-paper position size.

Do not choose different numeric values speculatively. The correct immediate change is ownership and provenance, not an India-specific risk opinion without sufficient market-local outcomes.

#### ## Current State

- `trading\_mandates` is per-market and currently controls score threshold, default stop loss, target, holding horizon, open-name cap, and signal freshness.
- Production US and India mandate rows currently both use `60 / 7% / 20% / 10 names`. Equal values are valid defaults; they are not evidence that the settings should be permanently shared.
- PaperTrader reads `strategy\_config.position\_size\_pct` as the flat-size fallback and as the cap on either market's champion genome. This is global.
- Existing positions persist their entry stop/target and mandate snapshot. Changing a mandate is gate-only for new entries unless its explicit existing-positions policy is approved and executed.
- The US ETF allocation cap is a US instrument-class policy. It must not be copied to India; a future India fund/ETF sleeve needs a separately classified INR policy.

#### ## Scope

1. Add `max\_position\_pct` to `trading\_mandates`, constrained to 1-30 and seeded from the owner-approved current global ceiling for each market.
2. Make PaperTrader load this strict market-local value before sizing. It becomes the flat fallback and the upper cap for that market's genome/Kelly result.
3. Stamp the resolved cap and mandate version into each paper fill's existing mandate snapshot/decision journal record.
4. Keep `strategy\_config.position\_size\_pct` as a deprecated UI-preset/template only until settings migration is complete; it must not be an execution fallback once market mandates are required.
5. Expose US and India controls separately in Settings with an explicit statement that edits affect future entries only.

#### ## Non-Goals

- No retroactive resize, forced exit, stop rewrite, or cross-market normalization.
- No automatic genome parameter divergence or learner-authorized money-limit change.
- No India ETF policy inferred from US ETF tickers.
- No live-order behavior change; this is first a paper-path configuration correction.

## ## Safety Requirements

- Missing/malformed market mandate or max-position value fails closed for new paper entries; it never falls back to the other market or global value.
- US and India pools, currencies, position counts, NAV, and caps remain isolated.
- A champion genome may size down but never above that market's owner cap.
- Settings writes are owner-authenticated, versioned, audited, and do not alter existing position snapshots.
- Database migration includes a CHECK constraint, RLS verification, and a default-free backfill that explicitly seeds both current market rows.

## ## Acceptance Criteria

- A US cap edit cannot change an India simulated order, and vice versa.
- A missing India cap blocks only India new entries with an auditable reason.
- Existing positions preserve their stored entry plan after a settings edit.
- Tests cover flat and Kelly sizing, both markets, missing configuration, snapshot provenance, and no cross-currency aggregation.
- `docs/arch/03-agents.md`, `04-database-schema.md`, `08-risk-and-safety.md`, the System Map, migration tracker, typecheck, tests, build, and production schema/RLS verification are updated before ship.

## ## Build Order

1. Approve initial US and India max-position values.
2. Add migration and strict mandate type/loader support.
3. Change PaperTrader sizing and immutable fill provenance.
4. Add separate Settings controls and tests.
5. Verify production schema, then observe paper entries before considering live reuse.

---

# Source: features\markets-data\FEATURE\_ARCHITECTURE.md  
# Markets Data – Price-Cache Fill Architecture

> Status: **Implemented** (2026-07-15)  
> Scope: the Markets page display tiles only. **Never** on the money / scoring / sizing / order path.

## ## Problem

The Markets page (`components/dashboard/MarketsPage.tsx`) renders ~30 ETF tiles:

- **Regime proxies** – SPY, QQQ, IWM, TLT, IEF, HYG, UUP, GLD, VIXY, DIA
- **11 sector XLS** – XLK, XLF, XLE, XLV, XLI, XLY, XLC, XLP, XLU, XLRE, XLB
- **Leveraged sentiment pairs** – TQQQ/SQQQ, SOXL/SOXS, SPXL/SPXS, FAS/FAZ, UGL/GLL

Each tile group had its own API route that fetched its symbols with a **concurrent burst** of per-symbol Massive `/prev` calls on page load:

- `app/api/markets/synthesis/route.ts` – `Promise.allSettled(REGIME\_TICKERS.map(fetchQuote))` (8-9 concurrent)
- `app/api/markets/overview/route.ts` – 15 symbols in parallel
- `app/api/markets/quotes/route.ts` – up to 20 symbols in parallel (the leveraged pairs)
- `app/api/charts/sector-returns/route.ts` – 11 sectors, staggered but still bursty

Massive's **free tier rate-limits** at ~5 requests/minute (confirmed: `HTTP 429 – "You've exceeded the maximum requests per minute"`). So most of each burst 429'd and `Promise.allSettled` silently dropped the rejected quotes → IWM/TLT/HYG/UUP/GLD and the leveraged pairs rendered `–`. Worse, `markets/synthesis` then **cached the degraded read** (briefings table, `session='synthesis'`) for the whole day, so a single unlucky page load froze a mostly-empty synthesis until someone manually force-refreshed. `charts/sector-returns` reads a `price\_cache` table that nothing proactively filled, so Sector Performance / Breadth showed "Price cache empty".

**Free-cloud-only constraint:** upgrading Massive's tier is off the table. The fix has to live entirely within ~5 req/min.

**Solution:** one paced daily fill, then read the cache

### 1. Daily fill job – `app/api/agents/price-cache-fill/route.ts`

Runs pre-market on weekdays (pg\_cron, see below). Cron-gated (`verifyCronSecret`) for the scheduled run, owner-gated for manual runs (`requireOwner`). `maxDuration = 60`.

**Pacing** – the key move. Massive is Polygon-compatible, so it exposes the **grouped-daily** aggregates endpoint:

GET <https://api.massive.com/v2/aggs/grouped/locale/us/market/stocks/{date}?adjusted=true>

This returns **every US ticker's** prior-session OHLC in **one call** (~12k rows). The fill job makes that single request, filters the result to our 31-symbol universe, and upserts into

`price\_cache`. So the entire daily fill costs **one Massive request** – it respects the ~5/min limit trivially, with no burst and no long pacing loop. The most-recent completed session is resolved by walking back from yesterday over weekdays (a market holiday just yields an empty grouped result and we step back another day); at most a handful of paced attempts.

**Fallback** (only if the grouped endpoint is ever unavailable / entitlement-denied): sequential per-symbol `/prev`, gated by the shared serverless-safe `try\_acquire\_provider\_slot` lease (12.5 s = 5/min), **bounded** to a 45 s wall-clock budget and **resumable** – it only fetches symbols missing the expected session, so the 13:45 retry tick (and subsequent days) drain any remainder. This mirrors the `prewarm` / `evidence-shadow` bounded-resumable pattern.

**Idempotency**. Before doing anything (unless `?force=1`), it checks that **every** universe symbol has the most-recent session (`shouldSkipFill`) and skips only then. It used to probe SPY alone as a marker; one off-schedule run advanced SPY and every later tick skipped, freezing XLK/QQQ/DIA a session behind (prod 2026-07-17). Upserts are keyed by the `price\_cache` primary key `(symbol, date)`.

**Health**. Emits a System Health `warn` (`reportIssue`, key `price-cache-fill-degraded`, auto-expires at UTC midnight) **only** on a large shortfall (<60 % of the universe cached, or the fallback made zero progress while symbols are still missing). A clean fill calls `resolveIssue`. A couple of illiquid names missing from one snapshot is normal and does **not** alert.

Coverage is read back from `price\_cache` (`fillCoverage`), **not** counted from what the tick fetched. The per-symbol fallback deliberately skips symbols an earlier tick already filled, so fetch-counting understated coverage by exactly the amount already fixed – prod 2026-07-17 raised **incomplete (3/31)** listing 28 symbols as Missing and warning their tiles would show "-" while the cache held every one. Freshness is judged against the session actually served (`chosenDate`), so a holiday walk-back does not report the whole universe missing for a session that never existed. `filled` (this tick) and `cached` (the system) are both returned, never conflated.

### ### 2. Tiles read the warm cache

- **markets/synthesis** now reads the latest `price\_cache` bar per regime ticker in **one batch query** (`readCachedQuotes`), computing `changePct = (close - open) / open` from the cached daily bar (identical to what `/prev` returned). Only symbols still missing from the cache are backfilled, and that backfill is **sequential + lease-gated** (`fetchQuotePaced`) so it can never burst. Normal page loads resolve entirely from cache → **zero Massive calls on load**.
- **markets/quotes** already had a `price\_cache` fallback (it computes change from the last two closes); once the cache is filled the leveraged pairs resolve from it.
- **charts/sector-returns** reads `price\_cache` and is kept warm by the daily fill. Its own lazy backfill remains as a cold-start safety net. **Superseded assumption (corrected 2026-07-17)**: this section previously claimed "the cache accumulates history over successive fills", implying the 1W/1M/3M/6M/1Y windows would fill themselves in. They do not on any useful horizon – the daily fill adds ONE session per weekday, so a 1Y window needs a year of ticks. See "Sector period windows" below for the honesty guard and the explicit backfill that replace it.
- **markets/overview** keeps its 5-minute in-memory + Next `revalidate` cache; it benefits from the warm provider state but was not the frozen-for-a-day offender.

### ### 3. Never cache a degraded synthesis

`markets/synthesis` now only writes its daily briefings cache when a **strong majority** of the 7 regime scoring signals resolved (`scored >= 5`). A sparse read (cache not filled yet, or a transient Massive failure) is returned to the caller but **left uncached**, so the next fill / regeneration completes it instead of freezing it for the day.

### ### 4. Macro "what this means for your book" stays fresh

`app/api/agent-mind/macro-read` already regenerates daily via `kairos-macro-read-us/india` crons and returns a `stale` flag when its cached date ≠ today. The `MacroReadCard` on the Markets page now **self-heals**: on load, if the read is stale it auto-triggers one regeneration (guarded by a `useRef` one-shot, reset when the market flips) so the card is never frozen on a previous day's text. Advisory-only, cached per day → at most one cheap LLM call per viewer per day.

### ## Schedule

pg\_cron (migration `20260715140000\_price\_cache\_fill\_cron.sql`), verified registered in prod  
cron.job` (jobid 71/72, active):

```
Job	Schedule (UTC)	Calls
`kairos-price-cache-fill`	Weekdays 13:25	`POST /api/agents/price-cache-fill`
`kairos-price-cache-fill-retry`	Weekdays 13:45	`POST /api/agents/price-cache-fill`
```

Both fire before the 14:00 UTC morning briefing. 13:25 UTC ≈ 9:25 AM ET pre-market. EDT/EST caveat: set for EDT (summer); shift +1h at the November changeover, consistent with the other `kairos-\*` jobs.

### ## Invariants preserved

- **Free-cloud-only** – one Massive request/day; no paid tier, no VPS.
- **Deterministic** – prices and the regime scoring math are pure data; the LLM is used **only** for the advisory synthesis / thesis / macro-read prose, never for prices or the regime call.
- **Per-market never mixed** – this fill is US ETFs only. India tiles keep their own Yahoo/GDELT sources (`IndiaSectors`, `IndiaBreadth`, `IndiaGapNote`); US-only tiles stay US-only.
- **Off the money path** – `price\_cache` is display data. Nothing here feeds scoring, sizing, or orders.

- **\*\*Mobile-first\*\*** – no UI layout changes; the existing responsive tiles just get real data.

## Sector period windows – honesty guard + backfill (2026-07-17)

### The defect

Markets → Sector Performance switching 1W/1M/3M/6M/1Y did not change the data. `price\_cache` held exactly **\*\*two\*\*** sessions per sector ETF (2026-07-14, 2026-07-15), because the daily fill had only been scheduled two days earlier and adds one session per tick. Every cutoff (`now` – 7/30/90/180/365) therefore preceded both bars, every window selected the SAME two bars, and every period returned the SAME one-day move.

**\*\*This was an honesty bug, not just missing data.\*\*** The guard was `if (candles.length < 2) return null` – a COUNT check, not a SPAN check. Two bars pass it and produce a number the UI renders as "1Y XLK return +2%" when it is a one-day move. A display asserting a period the data cannot support is a false return.

### The contract – `lib/markets/sector-returns.ts` (pure, unit-tested)

A window may only report a return when the cached bars **\*\*actually span it\*\***. Otherwise `returnPct: null` plus a machine-readable `reason` the UI renders as what/why/next. Three independent checks, each covering a different failure mode – all are load-bearing:

```
Check	Constant	Catches
Start edge – oldest bar within N days of the cutoff	`START_GRACE_DAYS = 4`	Missing EARLY history on long windows (6 months of data must
Span coverage – `span >= days × floor`	`MIN_SPAN_COVERAGE = 0.4`	DEGENERATE spans on short windows. The absolute grace cannot protect a
End edge – newest bar within N days of today	`STALE_GRACE_DAYS = 7`	A window ending at a stale point – a "1Y return" ending three weeks
```

Reason codes: `no\_data`, `single\_bar`, `insufficient\_history`, `stale\_cache`. `summariseCoverage()` folds the per-sector rows into one note; the UI renders it verbatim (never a bare "-").

### PostgreSQL 1000-row cap (found by the backfill)

`sector-returns` batch-read all 11 ETFs unpaginated. PostgreSQL caps a response at **\*\*1000 rows\*\***; a full 1Y window is  $\sim 273 \times 11 \approx 3000$ , so it silently returned only the four alphabetically-first sectors (XLB, XLC, XLE, XLF) complete and the other seven with ZERO rows. Latent while the cache held 22 rows; it bit the instant real history landed. The read is now paginated. The honesty guard degraded correctly here – it reported `no\_data` rather than inventing a number.

### Backfill

See `docs/arch/05-crons-and-scheduling.md`.  $\sim 400$  days of daily bars for the 11 sector XLs, **\*\*one provider call per symbol\*\*** (`range/1/day` returns the full series in one response), lease-paced at 5/min, wall-clock bounded, resumable across ticks, riding the existing `kairos-price-cache-fill` schedule. Total cost: 11 requests. Note the fix order – the honesty guard is correct with **\*\*zero\*\*** backfill and ships independently of it.

### India parity

India has **\*\*no equivalent defect\*\***. `MarketsPage` gates the whole US analytics block behind `{!isIndia && ...}`; India renders its own `SectorHeatmap` from NSE sector indices, which is a **\*\*single-session\*\*** snapshot with **\*\*no period selector\*\*** – so it cannot make a period claim its data cannot support. Structural gaps already carry an explicit `NotSupportedNote`. Per-market and per-currency; nothing is cross-summed. No change was needed and none was made.

## Files

- `app/api/agents/price-cache-fill/route.ts` – new fill job; + sector history backfill (2026-07-17)
- `lib/markets/sector-returns.ts` – span-aware honesty layer (pure; `tests/sector-returns-span.test.ts`)
- `app/api/charts/sector-returns/route.ts` – honesty guard + paginated read (2026-07-17)
- `components/charts/SectorPerformanceChart.tsx` – insufficient-history / partial-coverage states
- `components/dashboard/SectorBreadth.tsx` – per-clause scope labelling + reason instead of "-"
- `app/api/markets/synthesis/route.ts` – cache-first read + strong-majority cache gate
- `components/dashboard/MarketsPage.tsx` – `MacroReadCard` self-heal on stale
- `lib/schedule.ts` – display metadata entry
- `supabase/migrations/20260715140000\_price\_cache\_fill\_cron.sql` – pg\_cron schedule
- `docs/arch/05-crons-and-scheduling.md` – cron chapter entry

---

# Source: features\mentor-agent\FEATURE\_ARCHITECTURE.md  
# Feature: Mentor AI Agent

Status: DRAFT → building v1 (2026-07-04).

## What's out there (and the gap)

- **\*\*Trade-journal analytics\*\*** (Edgewonk, TraderVue, Chartlog): compute stats on your behavior – win rate by setup, time-of-day, position-sizing errors, revenge trading. Descriptive, not coaching. You read charts and self-diagnose.
- **\*\*Trading-psychology coaches\*\***: human, expensive, not data-grounded in YOUR fills.
- **\*\*Robo-advisors / SA "health scores"\*\*\***: rate your portfolio, not your \*behavior\*.

**\*\*Nobody combines\*\***: your actual decision data + live market regime + a personalized AI coach that reasons about \*you\* + a learning curriculum with milestones. That's

the gap Kairos fills.

```
What we should have – MentorAgent (true AI tool-use agent)
Not a scorer (the old /api/mentor/evaluate just graded a single thesis). A real
agent that, on demand (and feeding the briefing), pulls the user's behavior +
learning progress + market context + trading principles, reasons over them, and
returns personalized coaching.

Pipeline
runAgentLoop on `deepseek-reasoner` (tool-calling works; no ANTHROPIC_API_KEY).
Tools:
- `query_behavior` – trade_decisions outcome_score patterns (what the user does
 well/badly across regimes), buy vs sell accuracy, recurring pattern_tags;
 closed paper_trades outcomes.
- `query_learning_progress` – closed-trade count, Phase 0/1 status, recent
 LearnerAgent hypotheses, rescore flags.
- `query_market_context` – current macro regime + index posture.
- `read_principles` – human-written learning_priors (Bayesian market principles).
- `finish` – structured coaching (below).

Output (stored in `mentor_insights`)
- `grade` (0-100) + `confidence` (0-1) on the user's current trading discipline/progress.
- `strengths` [] – what they're doing right (grounded in their data).
- `focus_areas` [] – the 1-3 things to work on, each with why.
- `lesson` – ONE lesson tailored to the current market regime AND their gaps.
- `market_note` – how today's regime should shape their behavior.
- `next_milestone` – the concrete next step (e.g. "close 4 more trades to unlock
 weight-tuning; journal a thesis for each").

Governance / honesty
- Advisory/educational only – never places trades, never changes strategy.
- Grounded: reasons only over the user's real data + principles; if data is thin
 (Phase 0, few trades), it says so and coaches on process, not outcomes.

Surfaces
- `/dashboard/mentor` – a "Coach" panel: run button + latest insight card.
- Briefing v2 – a Mentor block (see features/briefing).
- Visual Agents – Mentor tagged ■ AI Agent (done Phase 4).

API
- `POST /api/agents/mentor-coach` – runs the loop, stores an insight, returns it.
- `GET /api/agents/mentor-coach` – latest stored insight.

Storage – `mentor_insights`
`id` · grade int · confidence numeric · strengths jsonb · focus_areas jsonb ·
 lesson text · market_note text · next_milestone text · model · tokens_in/out ·
 created_at`. RLS: service all; auth read.

Source: features\momentum-factors\FEATURE_ARCHITECTURE.md
Momentum / Growth Factors – Feature Architecture

> **STATUS: DRAFT – NOT APPROVED.** Proposal for review. No code until explicit sign-off.
> Owner decision pending. Created 2026-07-10.

1. Problem

The live `deterministic_v1` scorer is tilted value + mean-reversion and structurally
favours the exact stocks that become multibaggers (Micron, SanDisk, NVDA, Intel-type
cycle-turn / re-rating moves):

- RSI curve peaks at 60-72 and penalizes RSI > 75 – hype movers run overbought for weeks.
- P/E dimension marks high-multiple growth names "expensive" – the re-rating engine (earnings
 acceleration) isn't measured, only the P/E level.
- No relative strength vs market/sector (the top empirical momentum factor).
- No earnings-estimate revision momentum (the re-rating driver).
- No revenue/earnings acceleration (2nd derivative – the cycle-turn detector).
- No 52-week-high proximity / breakout / volume accumulation.

We want to catch these earlier – WITHOUT curve-fitting to a handful of remembered winners
(survivorship) and without turning a value engine into a muddle that does neither job well.

2. Non-goals / guardrails (push-back mandate)

- Not a hardcoded "hype detector." Every new factor must clear the existing IC gate
 (EdgeScout/EdgeIC, `lib/edges/*`) on the broad universe before it can influence live scoring –
 the same gate that already correctly rejected the naive price/volume edges as noise.
- Not a replacement for the value/quality dimensions. Momentum is ADDED; the champion
 weights (per market) decide when it dominates – that's what the learning loop is for.
- No look-ahead. All factors computed point-in-time from data available at decision time.
- No money-path change. Scoring only; sizing/gates/broker untouched.

3. Proposed factors (all deterministic, 0-100 or z-scored)

| Factor | Definition | Data source |
```

```

|---|---|---|
| Relative strength | stock total return – benchmark return over 1/3/6mo, percentile-ranked | daily series (have) + SPY/^NSEI |
| Estimate revision | Δ consensus EPS/target over 30/90d (up = bullish) | FinancialDatasets / AV estimates |
| Revenue acceleration | sign+magnitude of QoQ growth 2nd derivative | FinancialDatasets income statements |
| Earnings acceleration | same on EPS | FinancialDatasets |
| 52w-high proximity | `price / 52w_high` (breakout tell) | daily series (have) |
| Volume breakout | volume z-score on up-days vs base | candles (have) |

```

## 4. Two architecture options (decide at review)

**\*\*Option A – new 6th dimension `momentum\_score`.\*\***

Aggregate the factors into one 0-100 dimension; extend the weight vector to 6, extend champion `weights\_snapshot`, validation replay, and genome. Cleanest conceptually; touches the weight schema + validation contract (bigger blast radius).

**\*\*Option B – factors as IC-gated \*features\* feeding existing dimensions (RECOMMENDED).\*\***

Each factor enters `feature\_registry` (migration 064), is IC-validated via the edge lab, and on promotion contributes to the **\*\*technical\*\*** (RS, 52wH, volume-breakout) or **\*\*fundamental\*\*** (estimate revision, acceleration) dimension via the whitelisted compiler. No new weight slot; the governed discovery path we already built does the work. Slower to "turn on," but validation-first and reversible.

Recommendation: **\*\*B\*\*** – it's the path the learning-core was built for and avoids overfitting.

## 5. Archetype interaction

`lib/scoring/archetypes.ts` already exists. A "momentum/breakout" archetype would stop hard-fading high RSI **\*when\*** RS + acceleration confirm – regime/archetype-routed, not global. In scope as a follow-on once  $\geq 1$  momentum factor is IC-promoted.

## 6. Validation gate (hard requirement)

No factor influences a live score until: IC  $|p| \geq$  threshold with Newey-West t-stat, out-of-sample across walk-forward folds, on the **\*\*broad\*\*** universe (not the 4 winners). Fail-closed. This is the antidote to survivorship.

## 7. Phasing

- **\*\*P0\*\*** – compute + log the 6 factors as shadow evidence on every candidate (no score effect).
- **\*\*P1\*\*** – IC-measure each in the edge lab; promote only those that clear.
- **\*\*P2\*\*** – wire promoted factors into their dimension (Option B) or the 6th dimension (Option A).
- **\*\*P3\*\*** – momentum archetype (conditional RSI-fade removal); let per-market weights adapt.

## 8. Open questions for review

1. Option A (new dimension) vs B (IC-gated features)?
2. Benchmark for RS: SPY/^NSEI only, or sector ETF too?
3. Estimate-revision data: FinancialDatasets sufficient, or add a second source?
4. Acceptable AV/FD budget increase for the new fetches (day-cache + budget guard applies)?

---

# Source: features\multi-tenant\FEATURE\_ARCHITECTURE.md  
# Feature Architecture: Multi-Tenant (Friends & Family)

## Status

Architecture status: Draft – DEFERRED (2026-09-14)

Architecture approved: No

Approved scope: None

Approved date: None

Implementation allowed: No

> **\*\*DEFERRED, not cancelled (2026-09-14).\*\*** The owner chose a much smaller  
> nearer-term path: read-only viewer access, designed in  
> `features/shared-viewer-access/FEATURE\_ARCHITECTURE.md`. That document is the  
> only one proposed for implementation. This one remains the design of record for  
> full per-user tenancy (own book, own broker, own genome, per-user learning) if  
> and when that is wanted; the viewer feature does not preclude it.

>  
> Two premises here were re-verified against production on 2026-09-14 and are  
> **\*\*stale in detail\*\***, though the framing holds. Correct current figures: 213  
> public tables all have RLS enabled but only 21 carry any owner column; 72  
> tables are pinned to the owner's email, ~10 are `auth.uid()`-scoped, and **\*\*34\*\***  
> are reachable by any authenticated user – 7 of those writable. Also note that  
> dashboard pages read through `createServiceClient()` (266 files), which  
> bypasses RLS entirely, so RLS is defence-in-depth here and never the delivery  
> path. Re-derive these numbers before reviving this design.

> DESIGN ONLY. No code, no migration file, no migration applied, no deployment.  
> This document is a proposal awaiting the owner's explicit approval gate  
> (CLAUDE.md "Architecture-First Mode"). Nothing here ships until the owner says  
> "Approved / Proceed / Implement this".

---



## ## Feature Purpose

Let the owner invite a small, vetted circle (friends & family) to use Kairos for their **own** internal paper/research use. Each invited user gets an isolated account: their own onboarding, their own market-data + LLM + broker keys, their own paper book, and – the interesting part – a **fresh champion genome and learning loop** that adapts on **their** trades, never on anyone else's. The system explores whether **aggregate, privacy-preserving** meta-insights can be shared across users without ever sharing an individual's weights or trades.

This is the transition from a **single-owner personal tool** (one hardcoded email, global-singleton tables scoped by RLS to that email) to a **narrow multi-tenant tool** (N users, every per-user row carries ``user_id``, RLS scoped to ``auth.uid()``), while keeping the free-cloud-only, deterministic-money-path, and per-market/currency-isolation invariants intact.

## ## User/System Questions This Feature Answers

- How does the owner let a specific friend in – and **only** that friend – without opening public registration?
- When a new user logs in for the first time, what must they provide before any agent runs on their behalf, and how do we stop agents running until they do?
- How is user A's genome/learning kept provably separate from user B's?
- Whose API keys and LLM spend pay for a given user's runs? (Answer: always their own – the owner never pays a guest's usage.)
- What, if anything, is safe to learn **across** users, and how do we share it without leaking any individual's trades or weights?
- How do we migrate the current single-owner data so the owner becomes "user 1" with zero loss?

## ## Scope

This feature includes:

- A tenancy model: ``user_id`` on every **per-user** table + Supabase RLS keyed on ``auth.uid()`` (replacing the owner-email RLS), with an explicit shared-vs-per-user table classification and a single-user→multi-user backfill.
- Owner-approved signup: an invite/allowlist gate. New emails cannot self-activate.
- A guided onboarding flow that blocks all agent activity until complete.
- Per-user genome + per-user learner (the locked 10-closed-trade Phase-0 rule, applied per user per market).
- A cross-user learning design that shares **only** opt-in, aggregate, k-anonymized meta-insights – never individual weights or trades.
- Security/isolation: RLS proof obligations, per-user vault isolation, owner-admin boundary, per-user cost isolation.
- C4 context + container diagrams, a phased rollout, and acceptance tests.

## ## Non-Goals (explicitly OUT of scope)

- **Billing / metering / subscriptions.** Friends & family = free. Lago stays a ``later_product_idea`` (per ``features/external-research-integrations/REPOSITORY_CAPABILITY_CATALOG.md``).
- No Stripe wiring beyond the dormant legacy columns already on ``profiles``.
- **Public / open self-service signup.** Invite-only, owner-gated, forever (for this feature).
- **Any shared money path.** No pooled capital, no shared book, no cross-user order routing. Each user's book is theirs alone.
- **Live trading for invited users, initially.** Paper-first. Guests are hard-capped to paper + research. Live broker order placement stays owner-only until a separate, later, explicitly-approved phase.
- **Hand-tuning of weights by users.** Users influence their own genome only indirectly (risk profile, mandate, and their realized outcomes). No user – not even the owner-for-a-guest – edits a genome's raw weights by hand.
- Changing the deterministic no-LLM-on-money-path rule, per-market/per-currency isolation, or the free-cloud-only constraint.

---

## ## Current Behavior (verified against code, 2026-07-15)

Kairos is **single-user, owner-gated** to one email. Concretely:

```
Layer	Today	File
Owner identity	`OWNER_EMAIL` = "<owner-email>" (hardcoded constant)	`lib/auth/owner.ts`
Page gate	`middleware.ts` signs out any session whose `email !== OWNER_EMAIL`; gates `/dashboard`, `/admin`	`middleware.ts`
API gate	`requireOwner()` returns 403 unless `user.email === OWNER_EMAIL && email_confirmed_at`	`lib/auth/require-owner.ts`
RLS	Owner-email predicate: `(auth.jwt() -> 'email') = '<owner-email>'` on the sensitive tables; `service_role` bypasses RLS for all	
Profiles	`profiles` PK = `auth.users.id`; `handle_new_user()` trigger auto-creates a row on signup and stamps `role='superadmin'` for the	
```

**Which tables already carry ``user_id``:** only the **legacy consumer-app** tables from the original FinanceOS product – ``holdings``, ``predictions``, ``watchlist`` (``UNIQUE(user_id, symbol)``), plus ``profiles`` (PK = ``auth.users.id``). These predate the agent system and are effectively dormant for the agent flows.

**Which tables are global singletons with NO ``user_id``** (owner-scoped only by the

email-RLS + the fact that one person uses the app) – the agent system's entire data plane:

- **Strategy/config:** ``strategy_config`` (a `*single row*`), ``strategy_versions`` (per-market champion/challenger), ``agent_config``, ``learner_config``, ``learning_priors``, ``learning_priors_history``, ``learning_log``, ``signal_weights_history``, ``investment_mandates``, ``market_controls``, ``strategy_validation_automation``, ``strategy_sleeves``.
- **Research/signals:** ``agent_signals``, ``signal_score_history``, ``decision_observations``, ``research_packets``, ``research_queue``, ``edge_signals``, ``edge_ic_history``, ``universe_snapshot``.
- **Paper book:** ``paper_portfolio``, ``paper_positions``, ``paper_trades``, ``paper_order_events``, ``paper_performance``.
- **Live book:** ``broker_accounts``, ``live_account_snapshots``, ``live_performance``, ``broker_orders``, ``broker_order_events``, ``trade_proposals``, ``decision_journal``.
- **Learning/RAG:** ``trade_memories`` (pgvector), ``experiment_runs``, ``strategy_evaluations``, ``benchmark_scorecard``, ``rotation_events``.
- **Secrets:** ``api_key_vault`` – a `*single global vault*` keyed by ``key_name`/`provider`` UNIQUE; resolution is vault-first / env-fallback, service-role-locked (``lib/llm-keys.ts``, ``lib/vault-pin.ts``). LLM keys and broker OAuth tokens all share this one store.

**Genuinely-shared market facts (NOT user data – must stay global):** ``macro_regime``, ``macro_signals``, ``india_screen_cache``, ``evidence_records``, ``fundamental_facts``, ``corporate_actions``, ``benchmarks``, ``benchmark_price_observations``, ``provider_pacing``, ``provider_call_ledger``, ``evidence_cache_v2``, the evidence-policy tables, and the market-knowledge principle base (``knowledge/``). These describe *the market*, identically for everyone, and are expensive to recompute per user.

**Scheduling today** is global: Vercel crons, Windows Task Scheduler (``scripts/run-agents.ps1``), and Supabase ``pg_cron`` each fire once and operate on the single owner's book, parameterized only by `?market=us|india``. There is **no per-user fan-out** anywhere.

**No onboarding, invite, or allowlist flow exists** for users today – this is greenfield.

---

## Proposed Behavior

### 1. Tenancy model

The core move: `**add `user_id` uuid NOT NULL REFERENCES auth.users(id)`` to every per-user table, and switch RLS from the owner-email predicate to ``user_id = auth.uid()``. **Shared market-fact tables keep no ``user_id`` and stay readable by all authenticated users.**

#### 1a. Table classification (the load-bearing decision)

| Class               | Rule                                                                    | Tables (representative)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Per-user            | <code>(add `user_id`, RLS `= auth.uid()`)</code>                        | Anything that is one user's config, book, signals, learning, or secrets   <code>`strategy_config`</code> , <code>`strategy_versions`</code> , <code>`agent_config`</code> , <code>`learner_config`</code> , <code>`learning_priors`</code> , <code>`learning_priors_history`</code> , <code>`learning_log`</code> , <code>`signal_weights_history`</code> , <code>`investment_mandates`</code> , <code>`market_controls`</code> , <code>`strategy_validation_automation`</code> , <code>`strategy_sleeves`</code> .                                                 |
| Shared market facts | <code>(NO `user_id`, RLS: read-all-authenticated, service-write)</code> | Identical for everyone; describes the market, not a per-user book   <code>`macro_regime`</code> , <code>`macro_signals`</code> , <code>`india_screen_cache`</code> , <code>`evidence_records`</code> , <code>`fundamental_facts`</code> , <code>`corporate_actions`</code> , <code>`benchmarks`</code> , <code>`benchmark_price_observations`</code> , <code>`provider_pacing`</code> , <code>`provider_call_ledger`</code> , <code>`evidence_cache_v2`</code> , the evidence-policy tables, and the market-knowledge principle base ( <code>`knowledge/`</code> ). |
| Secrets             | <code>(per-user, but never client-readable)</code>                      | Per-user vault rows; service-role only, never returned raw   <code>`api_key_vault`</code> → <code>**add `key_name`/`provider` UNIQUE; resolution is vault-first / env-fallback, service-role-locked`</code>                                                                                                                                                                                                                                                                                                                                                         |

> \\* **strategy\_config** is the sharpest change. It is a `*single-row*` table today  
> (``SELECT ... LIMIT 1`` everywhere). Multi-tenant requires it to become  
> **one row per user** (drop the singleton assumption; key by ``user_id``). Every  
> call site that does "read the config row" must become "read *this user's* config  
> row". This is the highest-touch refactor and the biggest source of latent  
> single-tenant assumptions. Same pattern for ``paper_portfolio`` (already per-market;  
> becomes per-user-per-market) and ``market_controls``.

#### 1b. Composite scoping keys

Kairos already isolates by **market** (``us`` | ``india``) and **currency**. Multi-tenant adds **user** as the outermost scope. The canonical grain of a book row becomes `*(user_id, market)*`; of a champion, `*(user_id, market, is_champion)*`. No row is ever summed across ``user_id``, exactly as no row is summed across currency today. All existing per-market uniqueness constraints gain ``user_id`` as the leading column (e.g. ``strategy_versions`` champion-uniqueness → `UNIQUE(user_id, market) WHERE is_champion``).

#### 1c. RLS pattern (replaces owner-email)

For every per-user table:

-- read/write only your own rows

CREATE POLICY user\_rw ON <table>

FOR ALL TO authenticated

USING (user\_id = (SELECT auth.uid()))

WITH CHECK (user\_id = (SELECT auth.uid()));

`service\_role` continues to bypass RLS – but see §6: server/agent code must now **explicitly stamp and filter `user\_id`** on every service-role query, because the DB will no longer protect it (the email predicate is gone). This is the single biggest correctness risk and gets an acceptance test.

Owner/admin visibility (support, debugging) is **not** a blanket RLS bypass. It is a separate, audited, read-only admin surface (§6) that uses `service\_role` with an explicit `user\_id` filter and writes an access-log row – never an "owner sees all via RLS" shortcut.

#### 1d. Migration from single-user (backfill owner as user 1)

1. Resolve the owner's `auth.users.id` (call it `OWNER\_UID`).
2. For each per-user table: `ALTER TABLE ... ADD COLUMN user\_id uuid;` then `UPDATE ... SET user\_id = OWNER\_UID WHERE user\_id IS NULL;` then `ALTER COLUMN user\_id SET NOT NULL` + add FK + index `(user\_id, ...)`.
3. Replace owner-email RLS policies with the `auth.uid()` policies above.
4. `strategy\_config` / `paper\_portfolio` / `market\_controls`: the existing rows are stamped `OWNER\_UID`; the singleton reads become per-user reads.
5. Shared-fact tables: unchanged except tightening RLS to "read-all-authenticated, service-write" (they were owner-email or service-only before).
6. Backfill is **idempotent and reversible** (guard on `WHERE user\_id IS NULL`), applied per CLAUDE.md's schema-migration rule (verify applied to target DB via `information\_schema` before any dependent code ships).

Because every existing row is stamped `OWNER\_UID`, the owner's book, genome, learning history, and vault survive the migration **byte-for-byte** – the owner simply becomes user 1.

### 2. Owner-approved signup (invite / allowlist)

No open registration. A new `user\_invitations` table is the gate:

| Column       | Type             | Notes                                                                 |
|--------------|------------------|-----------------------------------------------------------------------|
| email        | text PK (citext) | Invited address, normalized                                           |
| invited_by   | uuid             | Owner's uid                                                           |
| status       | text             | invited \ activated \ revoked                                         |
| role         | text             | guest (default) \ owner                                               |
| capabilities | jsonb            | e.g. { live_trading: false, markets: ["us"] } – paper-only by default |
| invited_at   | timestampz       |                                                                       |
| activated_at | timestampz       |                                                                       |
| revoked_at   | timestampz       |                                                                       |

Flow:

1. Owner adds an email in an **Admin → Invites** panel (owner-only, guarded by a new `requireRole('owner')`).
2. The email is added to `user\_invitations` (status='invited'). Supabase Auth signups are configured **invite-gated**: a `handle\_new\_user()` update rejects (or immediately deactivates) any signup whose email is not `status='invited'`. This replaces the "reject everyone but OWNER\_EMAIL" logic in `middleware.ts`.
3. The invited person signs in (magic-link / OAuth). `handle\_new\_user()` flips the invite to `activated`, creates their `profiles` row with `role='guest'`, and marks onboarding incomplete.
4. `middleware.ts` and `requireOwner` → `requireUser` change: instead of comparing to a hardcoded email, they check `profiles.role`/invite status. `OWNER\_EMAIL` stays only as the bootstrap seed for user 1 and the owner-admin check.
5. Revocation: owner sets `status='revoked'`; middleware signs the session out and blocks re-entry. (Data is retained, not deleted – deletion is a separate, explicit, out-of-scope action.)

### 3. Onboarding (agents gated until complete)

A new `onboarding\_state` per user (columns on `profiles` or a small `user\_onboarding` table) with a hard gate: **no agent, cron, or research run executes for a user until `onboarding\_complete = true`**. The gate is enforced in the agent entry points (the per-user run resolver, §7) – a user who hasn't finished onboarding is simply skipped by the schedulers.

Guided steps (a wizard; each writes to the user's own rows):

1. **What Kairos does** – explainer + the hard rules (paper-first, deterministic money path, you bring your own keys, your data is isolated).
2. **Market-data keys** – the user enters **their own** provider keys (Alpha Vantage, FinancialDatasets, etc.) into **their** vault rows. Nothing runs on the owner's keys. Missing required keys → that capability stays dark.
3. **LLM model + key** – the user picks a per-flow model and supplies the matching provider key (reuses the existing Settings → AI Models UX and `lib/llm-keys.ts`, now `user\_id`-scoped). No key → that flow is disabled for them (never silently billed to the owner).
4. **Broker connect (paper-first)** – optional; read-only holdings import only. Live order placement is **not** offered to guests in this phase.

5. **\*\*Risk profile + mandate\*\*** – `Conservative | Balanced | Aggressive` and a default `investment\_mandate` (horizon/benchmark). These seed the user's `strategy\_config` and their fresh genome (\$4).
6. **\*\*Confirm\*\*** – sets `onboarding\_complete = true`, which arms their schedulers.

#### ### 4. Per-user genome + learning loop

Each user gets a **\*\*fresh champion genome per market\*\***, seeded at onboarding from a neutral default prior (the same academic prior the single-owner app seeds today – 5 equal-ish dimension weights + default genome ranges), **\*\*not\*\*** from any other user's genome. From there:

- The user's `strategy\_versions` rows (`user\_id`-scoped) hold **\*their\*** champion + challengers. `UNIQUE(user\_id, market) WHERE is\_champion`.
- Their LearnerAgent run reads **\*\*only their\*\*** closed `paper\_trades` / `decision\_observations` and proposes challengers for **\*\*their\*\*** genome.
- The locked **\*\*Phase-0 gate applies per user\*\***: weight mutation is blocked until that user has 10+ closed trades in that market. A user with 3 trades stays on the seeded champion; the owner with 200 trades is on their evolved champion. The gate, the auto-guard (last-3-runs win-rate < 35%), and the "size down only, never above the user-set cap" genome rule are unchanged – just per-user.
- Their Validation Engine replays walk-forward on **\*\*their\*\*** `decision\_observations` only. Their promotion decisions touch only their champion.
- **\*\*Full isolation invariant\*\***: \*user A's outcomes never touch user B's weights.\* There is no code path, RLS policy, or query that reads one user's trades to mutate another's genome. This is the divergence acceptance test (\$Acceptance): two users fed different trade sets must produce measurably different champions.

Users influence their genome **\*\*only\*\*** through risk profile, mandate, and realized outcomes – never by editing raw weights (the CLAUDE.md push-back mandate is preserved and now applies per user).

#### ### 5. Cross-user learning (the interesting part)

The design question the owner cares about most. Two hard boundaries first, then the narrow safe channel.

**\*\*Never shared (hard NO)\*\*:**

- Individual weight genomes / `weights\_snapshot` / champion parameters.
- Any per-user trade, signal, position, P&L, or `decision\_observation`.
- Anything from which a single user's book or identity could be reconstructed.

**\*\*Already shared, safely (existing priors)\*\*:** the **\*\*academic principle base\*\*** (`knowledge/`) and the deterministic scoring grammar. These are literature-derived priors, identical for everyone, and carry no user data. They stay shared – this is not new leakage.

**\*\*The proposed narrow channel – opt-in, aggregate-only meta-insights\*\*:**

A separate, deterministic **\*\*MetaLearner\*\*** (offline, owner-run, no money path) computes **\*cohort-level\*** observations across users who have **\*\*opted in\*\***, subject to strict privacy guards, and emits them only as **\*\*priors nudges bounded in magnitude\*\***, never as another user's weights:

- Example output: **\*\*"across opted-in users, the momentum-dimension weight tended to rise in high-volatility regimes" – a \*\*direction\*\*, not a value, not attributable to anyone.**
- **\*\*k-anonymity / min-cohort\*\***: an aggregate is emitted only if it is computed over **\*\*≥ K opted-in users\*\*** (proposed K ≥ 5) **\*and\* ≥ N trades**, so no cohort can be narrowed to one person. Below the floor → no insight emitted.
- **\*\*Aggregation before storage\*\***: only means/medians/rank-correlations over the cohort are ever persisted; raw per-user inputs are read under service-role, reduced in-memory, and discarded. No per-user row is copied into the meta store.
- **\*\*Overfitting guards\*\***: meta-insights are advisory **\*priors\***, bounded to a small max nudge (e.g. ±X% of a dimension weight), pass the same walk-forward validation before they can influence anyone, and are rate-limited (at most one meta-update per cadence). They can **\*\*never\*\*** bypass a user's own Phase-0 gate or promote a user's champion.
- **\*\*Opt-in + revocable\*\***: default **\*\*OFF\*\***. A user opts in explicitly during onboarding or Settings; opting out removes them from all future cohorts.
- **\*\*No money path\*\***: the MetaLearner writes to a `meta\_insights` (aggregate) table only; it never places an order, moves cash, or auto-mutates a champion.

This gives "collective intelligence" its upside (regime-conditioned prior nudges) while keeping every individual's weights and trades private and every user's book adapting primarily to their own outcomes.

#### ### 6. Security / isolation

- **\*\*RLS proof obligation\*\***: Every per-user table has an `auth.uid()` policy; a test suite (\$Acceptance) asserts, per table, that user B receives **\*\*zero\*\*** rows of user A via the anon/authenticated client. The dangerous change is that removing the owner-email predicate means **\*\*server code is now the last line of defense\*\*** on service-role queries – so every service-role read/write must carry an explicit

```

`.eq('user_id', uid)`. A lint/code-review checklist + an integration test that
runs each agent as user B and asserts it never reads user A's rows.
- **Per-user vault isolation.** `api_key_vault` gains `user_id`;
 `UNIQUE(user_id, key_name)`. `getProviderKey()` / `setProviderKey()` /
 broker-token resolution all take a `user_id`. A user can never read or use
 another user's key; keys remain write-only from the UI and are never returned
 raw. Env-fallback becomes **owner-only** (guests must bring their own keys — see
 cost isolation).
- **Owner-admin boundary.** The owner is a superadmin for *support* (read-only,
 audited), not a data-plane superuser. Admin reads go through a dedicated
 service-role surface that filters by the target `user_id` and writes an
 `admin_access_log` row. No "owner sees everything" RLS bypass.
- **Cost isolation.** Each user's LLM and market-data calls resolve **their** vault
 keys only; env-fallback is disabled for guests. A guest with no key → the flow is
 disabled, not silently charged to the owner. `llm_call_log` is `user_id`-scoped so
 spend is attributable per user. This directly satisfies the free-cloud-only /
 "users bring their own keys" memory constraint.
- **Live-trading lockout.** Guests' `capabilities.live_trading = false` is enforced
 at the order gate (in addition to the existing 9 safety gates + autonomy ladder).
 A guest cannot reach a live broker order path regardless of settings.

```

### ### 7. Scheduling (per-user fan-out)

Today's crons operate on one book. Multi-tenant needs the schedulers to \*\*fan out over active, onboarded users\*\*:

- Each global cron endpoint (`/api/agents/research/cron`, `paper-trade`, `position-monitor`, `learner`, `performance`, briefings, ...) changes from "run once" to "for each user where `onboarding\_complete` AND NOT paused", run scoped to that `user\_id`. Shared market-fact refreshes (macro, screen cache, evidence) run \*\*once, globally\*\*, not per user — so N users don't multiply the market-data spend.
- Free-cloud-only constraint: Vercel cron count/time budgets are fixed, so fan-out is a \*\*sequential loop inside one cron invocation\*\* (bounded, small N), not N separate cron entries. Per-user provider pacing (`provider\_pacing`, migration 176) already exists and extends naturally.
- Guests are paper-only, so the live-order crons stay owner-scoped in this phase.

---

## ## System Architecture

### ### C4 — Level 1: System Context

#### C4Context

title Kairos Multi-Tenant — System Context

Person(owner, "Owner", "Superadmin. Invites users, uses Kairos, runs admin/support")

Person(guest, "Invited User (F&F)", "Paper/research only. Brings own keys")

System(kairos, "Kairos", "Multi-tenant paper-trading + research OS")

System\_Ext(supabase, "Supabase", "Postgres + Auth + RLS + pg\_cron + pgvector")

System\_Ext(llm, "LLM providers", "Anthropic / DeepSeek / Groq / (per-user keys)")

System\_Ext(mkt, "Market-data providers", "Alpha Vantage / FinancialDatasets / NSE (per-user keys)")

System\_Ext(brokers, "Brokers", "Robinhood / Kite (owner live; guests read-only paper)")

Rel(owner, kairos, "Invites, uses, supports")

Rel(guest, kairos, "Onboards, uses paper+research")

Rel(kairos, supabase, "Per-user RLS data plane, cron fan-out")

Rel(kairos, llm, "Scoped to each user's vault key")

Rel(kairos, mkt, "Shared facts once; user calls on user keys")

Rel(kairos, brokers, "Owner live; guest read-only")

### ### C4 — Level 2: Container

#### C4Container

title Kairos Multi-Tenant — Containers

```

Person(owner, "Owner")

Person(guest, "Invited User")

Container_Boundary(app, "Next.js app (Vercel)") {
 Container(web, "Web UI", "Next.js/React", "Dashboard, onboarding wizard, Admin-Invites")
 Container(mw, "Auth middleware", "Edge", "Invite/role gate (replaces owner-email gate)")
 Container(api, "API routes", "Route handlers", "requireUser() + user_id-scoped")
 Container(agents, "Agent runtime", "Server", "Research/Trader/Learner/Monitor per-user run resolver")
 Container(sched, "Schedulers", "Vercel cron / pg_cron / Task Scheduler", "Fan out over onboarded users; shared facts once")
 Container(meta, "MetaLearner (offline)", "Server, owner-run", "Aggregate, k-anon, opt-in only. No money path")
}

ContainerDb(db, "Supabase Postgres", "Per-user tables (user_id + RLS auth.uid()); shared-fact tables; per-user vault;
meta_insights (aggregate)")

System_Ext(providers, "LLM + market-data + brokers")

Rel(owner, web, "Invite, use, admin")

Rel(guest, web, "Onboard, use")

Rel(web, mw, "Session")

Rel(mw, api, "role/invite-checked")

Rel(api, db, "user_id-scoped (RLS + explicit filter)")

Rel(agents, db, "Reads/writes this user's rows")

Rel(sched, agents, "Per onboarded user")

Rel(agents, providers, "User's own keys")

Rel(meta, db, "Reads opted-in cohort; writes aggregate only")

Screen / Page / Module Inventory (new or changed)

- **New:** Admin → Invites panel; Onboarding wizard (6 steps); per-user Settings
 (keys/LLM/risk/mandate now `user_id`-scoped); opt-in-to-meta toggle.
- **Changed:** `middleware.ts` (invite/role gate), `lib/auth/require-owner.ts` →
 add `requireUser()` / `requireRole()`, `lib/llm-keys.ts` + vault helpers (take
 `user_id`), every agent entry point (per-user run resolver + onboarding gate),
 every cron endpoint (fan-out), all `strategy_config`/`paper_portfolio` single-row
 reads.
- **New tables:** `user_invitations`, `user_onboarding` (or columns on `profiles`),
 `meta_insights` (aggregate-only), `admin_access_log`.
- **Altered tables:** `user_id` added to every per-user table (§1a) + RLS swap.

Phased Rollout

Phase	Goal	Exit criteria
P0 – Owner-only (backfill)	Add `user_id` everywhere, backfill owner as user 1, swap RLS to `auth.uid()`, refactor singleton reads. **	
P1 – 1 invited user	Invite flow + onboarding gate + per-user vault + per-user genome seeding + cron fan-out (N=2).	A second user on
P2 – N users	Harden fan-out under cloud budgets; per-user pacing; admin support surface + access log.	N (small) users run within Ver
P3 – Cross-user meta (opt-in)	MetaLearner: aggregate, k-anon (K≥5), bounded prior nudges, validated, opt-in.	Meta-insights emit only
***(Later, separate approval)**	Guest live trading, billing/metering.	Out of scope for this feature.

Acceptance Tests

1. **Tenant isolation (data plane).** As authenticated user B, every per-user table
 returns 0 rows belonging to user A (direct PostgREST + through each API route).
2. **RLS policy coverage.** Automated check: every table classified per-user has an
 `auth.uid()` policy and no lingering owner-email or `USING(true)` policy.
3. **Service-role filter guard.** Run each agent as user B; assert (via query log /

```

- integration harness) it never reads or writes a row with `user\_id = A`.
4. **\*\*Onboarding gate.\*\*** A user with `onboarding\_complete = false` is skipped by all schedulers; no `agent\_signals`/`paper\_` rows are ever created for them.
  5. **\*\*Invite gate.\*\*** A non-invited email cannot activate; a revoked user is signed out and blocked; only `status='invited'` can complete signup.
  6. **\*\*Per-user genome divergence.\*\*** Seed user A and user B with different closed-trade sets past the Phase-0 gate; assert their champion `weights\_snapshot`/`genome` differ and that neither's LearnerAgent ever read the other's trades.
  7. **\*\*Vault isolation.\*\*** User B cannot read/use user A's provider or broker key; `getProviderKey(uid=B)` never resolves A's vault row; env-fallback is owner-only.
  8. **\*\*Cost isolation.\*\*** A guest with no LLM key gets the flow disabled (not run on the owner's key); `llm\_call\_log` attributes every call to the correct `user\_id`.
  9. **\*\*Owner backfill fidelity.\*\*** Post-migration, the owner's champion, trade ledger, and vault are byte-identical to pre-migration (hash compare).
  10. **\*\*Meta privacy.\*\*** MetaLearner emits nothing below K users / N trades; the `meta\_insights` store contains no per-user trade, weight, or identity; opting out removes a user from all subsequent cohorts.
  11. **\*\*Live lockout.\*\*** A guest cannot reach any live broker order path irrespective of settings (`capabilities.live\_trading=false` enforced at the gate).

---

## ## Open Decisions (for the owner)

**\*\*Biggest open decision – how much cross-user aggregate learning to allow, vs full isolation.\*\*** The spectrum:

- **\*\* (A) Full isolation.** No cross-user learning at all. Simplest, zero leakage risk, most defensible privacy story. Each user is an island; the only shared prior is the existing academic `knowledge` base. Cost: no "collective intelligence" upside.
- **\*\* (B) Opt-in aggregate nudges (this doc's §5 proposal).** k-anonymized ( $K \geq 5$ ), bounded, validated, revocable prior nudges – direction-only, never values, never attributable. Upside: regime-conditioned collective priors. Cost: real engineering + a standing privacy/overfitting surface to maintain, and a small but non-zero inference-risk to reason about continuously.

Recommendation: **\*\*ship P0-P2 with (A) full isolation\*\***, and treat (B) as a distinct, separately-approved P3 only after there are enough opted-in users for  $K \geq 5$  to even be meaningful. Everything else in this doc (tenancy, invites, onboarding, per-user genome, security) is required regardless of which way (A)/(B) goes.

Secondary open decisions:

- Keep `strategy\_config` as one-row-per-user, or split the truly-global knobs (autonomy master-kill) from per-user knobs into two tables? (Proposal: per-user row; owner keeps a separate global master-kill.)
- Cron fan-out as a sequential loop vs. a durable per-user job queue if N grows (proposal: sequential loop now; revisit at N beyond a handful).

---

## ## Update triggers (per CLAUDE.md)

If/when approved and implemented, the SAME change must update:  
 `docs/arch/04-database-schema.md` (new `user\_id` columns + RLS), `08-risk-and-safety.md` (guest live-lockout + per-user gates), `09-learning-loop.md` (per-user genome + MetaLearner), `05-crons-and-scheduling.md` (per-user fan-out),  
 `06-env-variables.md` (owner-only env-fallback), `07-coding-conventions.md` (service-role `user\_id` filter rule), and `public/agent-diagrams/system-map.json`.

---

# Source: features\nav-restructure\FEATURE\_ARCHITECTURE.md  
 # Feature: Left-nav restructure (funnel order + surface LLM config)

<details>  
 <summary>Historical implementation notes (source-preserved)</summary>

Last updated: 2026-07-12

</details>

Update this file when: NAV\_SECTIONS changes, a route is added/removed/renamed, or the LLM-config entry point moves.

## ## Problem

- ~20 sidebar items; grouping ("Daily / Portfolio / Agents & Research / Discovery / Learn / Settings") doesn't mirror the user's actual funnel.
- "Morning Briefing" is the home page but reads like a feature, not the app home.
- The per-flow LLM model picker is buried at Agents → "■ LLM Config" tab (default tab is Paper), so users look in Settings and don't find it.
- Three portfolio surfaces (Paper / Live-US / India) read as duplicates.

## ## Decision (this pass – LOW RISK, no routes removed)

Reorder + relabel `NAV\_SECTIONS` in `components/dashboard/DashboardShell.tsx` to follow the

```

funnel **Overview → Markets → Signals → Portfolio → Research → Discovery → Learn → Settings**,
and:
1. Rename the home item "Morning Briefing" → ***Home*** (route `/dashboard` unchanged).
2. Split the old "Daily" into **Overview** (Home, Markets) + **Signals** (Intelligence,
 Smart Money) – pure regrouping, same routes.
3. Rename "Agents & Research" → ***Research***; keep Agents/History/Research Journal/Scores.
4. Add a ***LLM & Models*** item under Settings that deep-links to
 `/dashboard/agents?tab=llm-config`. `AgentsPage` now reads `?tab=` to open that tab
 directly. This is the fix for "I can't find where to change the model."

```

No page component is merged in this pass. All existing routes keep working; only the sidebar order/labels and one new deep-link change.

```

Done (2026-07-12, second pass – shipped behind redirects, no deep-link 404s)
- **Live Portfolio** = US + India: LivePortfolioSwitch renders US (LivePortfolioPage) or
 India (IndiaLivePanel) by the global market switch. /dashboard/india redirects.
- **Signals** = Intelligence + Smart Money: Smart Money is a tab on the Intelligence page
 (data via owner-gated /api/markets/smart-money). /dashboard/smart-money redirects.
- **Research** = Research Journal + Score Tracker: ScoreTrackerPanel is a tab on Research
 Journal. /dashboard/scores redirects.
- **Agents** absorbs Agent History: AgentHistoryPanel is a "History" tab; AgentsPage reads
 ?tab=. /dashboard/agents/history redirects.
- **Settings** absorbs Automation + Admin + LLM Config: AutomationPanel + AdminPanel +
 LLMConfigPanel are tabs. /dashboard/admin and /dashboard/settings/automation redirect.

```

Deliberately NOT merged: Paper Portfolio stays separate from Live Portfolio (sim vs real money – kept visibly distinct for safety).

```

Known cosmetic follow-up
Merged panels (Smart Money, Score Tracker, Admin, Automation, India Live) still render
their own PageHeader inside the container that also has one → a double header on those
tabs. Functional; trim by adding an optional `embedded` prop that suppresses the inner
PageHeader when rendered as a tab.

```

```

Files
- `components/dashboard/DashboardShell.tsx` – `NAV_SECTIONS` array.
- `components/dashboard/AgentsPage.tsx` – initial `tab` from `useSearchParams().get("tab")`.

```

```

```

```

Source: features\new-symbol-and-ipo-discovery\FEATURE_ARCHITECTURE.md
New Symbol, Listing, and Pre-IP0 Discovery

```

```

> Status: PARTIALLY IMPLEMENTED – P0 plus the filing-discovery slice of P1 shipped 2026-09-09. Exchange-listing events, identity reconciliation
> Last revised: 2026-09-09 by Codex after code-path and source review.
> Money-path influence: none until a later, separately approved promotion.

```

```

0. Decision summary

```

Kairos should discover and study newly listed instruments, but it must not use the owner watchlist as a machine discovery database or infer that discovery means a symbol is safe to score or buy.

The recommended first release is an evidence-only candidate registry:

1. ingest authoritative issuer filings and listing events;
2. resolve issuer, listing, symbol, exchange, and instrument identity;
3. verify the exact target broker/account can buy the instrument;
4. collect point-in-time scores, prices, liquidity, and forward outcomes;
5. compare predeclared 20/40/60-session admission policies in shadow;
6. show the evidence in Research and Upgrade Path.

It must not add candidates to `watchlist`, make them entry-eligible, create paper positions, or place live orders during this release.

```

Feature-lifecycle record

```

- \*\*Why:\*\* Kairos can otherwise miss new companies and ETFs, while manually adding every listing is incomplete and does not produce evidence about when a new listing becomes suitable for the existing strategy.
- \*\*Who:\*\* the single Kairos owner.
- \*\*Expected value:\*\* broader, broker-relevant research coverage and a measurable answer to whether newly listed instruments add benchmark-relative return.
- \*\*Shipped at:\*\* 2026-09-09 (evidence-only SEC daily-index filing discovery, candidate/event/filing registry, Research and Upgrade Path read)

```

1. Verified baseline

```

```

1.1 What exists

```

- `watchlist.research\_enabled` controls whether a watchlist row enters the ordinary ResearchAgent batch. It is an owner research preference, not an instrument-trust state.
- `gatherSymbols()` reads only research-enabled watchlist rows. Its watchlist partition currently reduces every non-manual watchlist source to the generic `watchlist` discovery source. Adding `source='new\_listing\_scan'` to a watchlist row



- alone would therefore lose the provenance needed by LearnerAgent.
- A technical score becomes usable at 15 completed candles. EMA50 requires 50 bars; EMA200 is measure-only and requires 200. The providers commonly return about 251 bars, but 251 is not an entry requirement.
- New long entries already require at least two usable score dimensions, the normal score threshold, no earnings-repricing barrier, and no breakdown veto.
- `broker\_instrument\_preflights` and each execution adapter can record exact broker/account/symbol/side capability. The current Stage-0 program is measure-only and normally runs immediately before an attempted live order.
- `evidence\_records`, `doc\_chunks`, `decision\_observations`, return labels, and Upgrade Path already provide most of the evidence and outcome infrastructure.
- The currently observed Robinhood MCP tool catalog has no tool for requesting or managing a primary-market IPO allocation. Robinhood documents IPO Access as a conditional-offer workflow with eligibility checks and potentially partial or zero allocation. This means Kairos cannot automate IPO Access through the current MCP surface. It does not justify a permanent or industry-wide claim.

### ### 1.2 What does not exist

- No authoritative, persistent new-listing event feed.
- No stable issuer-to-listing identity spanning pre-ticker CIK, later ticker, exchange, symbol changes, postponements, withdrawals, or delistings.
- No candidate lifecycle distinct from the owner watchlist.
- No IPO/new-listing cohort in discovery provenance or LearnerAgent.
- No measured answer for whether 20, 40, 60, or another number of sessions is the right admission delay.

## ## 2. Scope and safety boundaries

### ### In scope

- US issuer filing discovery: S-1, S-1/A, F-1, F-1/A, 424B4 and related status changes, keyed by CIK and accession number.
- US exchange-listing event discovery for ordinary equities, ADRs, direct listings, ETFs, and SPAC-related listings, with explicit instrument-type cohorts.
- Exact broker/account buy-capability corroboration.
- Point-in-time research and outcome capture with no trading influence.
- A Research-page candidate panel and an Upgrade Path shadow program.
- A source-adapter contract that can support India later without changing the state machine.

### ### Out of scope

- Requesting primary IPO allocations.
- A day-one or "hot IPO" entry path.
- New IPO-specific score weights, targets, stops, sizing, or exits.
- Silent insertion into the owner watchlist.
- Automatic paper or live eligibility.
- Changing `live\_auto\_enabled`, account allowlists, autonomy level, or broker enforcement mode.
- India ingestion until an NSE/BSE source passes the same provenance, licensing, freshness, and coverage checks.

An owner may still read research or manually follow a candidate. An owner-directed manual trade remains governed by the existing confirmation, risk, and broker preflight controls; this feature does not create a hidden manual-trading prohibition.

## ## 3. Source policy

### ### 3.1 Filings

Use SEC EDGAR directly as the primary US filing source. `data.sec.gov` is a regulator source, requires no API key for public data, and publishes submissions in near real time. Requests must use the SEC-required identifying User-Agent, bounded concurrency, backoff, and immutable accession-number provenance.

Robinhood `get\_sec\_filing\*` tools may be tested as a corroborating adapter only after their live schemas and content coverage are captured. They are not currently in `ROBINHOOD\_RESEARCH\_READ\_TOOLS` and must not be assumed available to a new route merely because a historical tool inventory named them.

### ### 3.2 Listing events

`search`, `create\_scan`, and `run\_scan` are not accepted as an authoritative new-listing feed. Search requires a candidate query, and a scanner is not proven to emit every new equity and ETF or preserve listing dates.

Before choosing a provider, Stage 0 must compare candidate sources against these requirements:

- stable source event ID;
- exchange and first-trade/effective date;
- symbol, name, instrument type, and status;
- amendments, postponements, withdrawals, symbol changes, and delistings;
- daily publication cadence and documented correction behavior;

- licensing that permits Kairos storage and use;
- measured coverage against a second independent source.

Official exchange listing notices or directories are preferred. Nasdaq's Daily List is an example of the required event shape, but it is a data product and must not be selected until access, licensing, cost, and NYSE/other-exchange coverage are verified.

### ### 3.3 Broker truth

An exchange listing is not proof that the selected broker/account supports it. Discovery adapters propose candidates; broker adapters corroborate exact capability.

For each listed candidate, run a read-only one-unit BUY preflight against the target broker/account and persist the result to `broker\_instrument\_preflights` with `proposal\_id=NULL`. Add a required `purpose` discriminator:

- `execution\_attempt` – the existing last-mile order-path observation;
- `candidate\_probe` – a discovery/admission observation that cannot imply an order.

Backfill existing rows as `execution\_attempt`. Upgrade Path's broker-enforcement gate must count only `execution\_attempt`; this program counts only `candidate\_probe`. Without that separation, automated candidate probes would falsely advance the existing ten-session broker-enforcement gate. The candidate stores only a pointer to the latest applicable preflight; the append-only preflight ledger remains the evidence source.

A denied, unsupported, stale, account-mismatched, or symbol-mismatched result can never be treated as allowed. A broker result may expire and must be refreshed before any future admission decision.

### ## 4. Domain model

Do not add `first\_seen\_at` to `watchlist`. Create a dedicated machine-candidate model.

#### ### 4.1 `listing\_candidates` – current state

```
create table public.listing_candidates (
id bigint generated always as identity primary key,
market text not null check (market in ('us','india')),
issuer_key text not null,
listing_key text not null,
symbol text,
company_name text not null,
exchange text,
instrument_type text not null check (instrument_type in
('operating_company','adr','direct_listing','etf','closed_end_fund',
'spac','unit','warrant','preferred','unknown')),
state text not null check (state in
('pre_listing','announced','listed_observing','paper_admitted',
'withdrawn','postponed','rejected','delisted')),
announced_at timestampz,
first_trade_date date,
first_seen_at timestampz not null,
last_seen_at timestampz not null,
source text not null,
source_event_id text not null,
source_url text not null,
source_payload_hash text not null,
```

```

latest_preflight_id bigint references public.broker_instrument_preflights(id),
admission_policy_id bigint,
admitted_at timestamptz,
created_at timestamptz not null default now(),
updated_at timestamptz not null default now(),
unique (market, listing_key)
);

```

`issuer\_key` is a US CIK when known and otherwise a source-scoped identifier.  
`listing\_key` is the stable source/exchange listing identity. `first\_trade\_date` comes from an authoritative listing event. `first\_seen\_at` records Kairos discovery latency and must never substitute for listing age.

#### ### 4.2 `listing\_candidate\_events` – append-only lineage

Every discovery, correction, state transition, symbol mapping, filing link, broker probe, rejection, and admission decision appends an event with:

- `candidate\_id`, `event\_type`, `event\_at`, `effective\_at`;
- prior and next state where applicable;
- source, source event ID/URL, payload hash;
- deterministic reason code and calculations;
- code version and policy version.

UPDATE, DELETE, and TRUNCATE are forbidden to authenticated and service roles by grants and triggers. Corrections append a superseding event; they do not rewrite history.

#### ### 4.3 `issuer\_filings` – append-only filing identity

Store one row per accession/document version:

- CIK, accession number, form, filed/accepted/effective timestamps;
- primary document URL, issuer name, candidate ID when resolved;
- raw document hash, retrieval timestamp, parser version and quality state;
- structured extracted facts and risks with section/page anchors;
- amendment/supersedes accession link.

The unique identity is accession number plus primary-document hash. A new S-1/A is a new row. `decision\_journal` receives a concise owner-visible summary that links to these immutable filing rows; it is not the source of truth.

Full filing text may be chunked after issuer identity is resolved. Do not put a fake ticker into `doc\_chunks.symbol` for a pre-ticker company. Either add a separately approved issuer-key retrieval contract or defer chunk ingestion until a real symbol mapping exists.

#### ### 4.4 `listing\_admission\_policies` and assessments

Store each frozen policy arm with its session floor, evidence requirements, broker requirements, liquidity rule, primary outcome, inference plan, code version and content hash. Once referenced by an assessment, it is immutable.

`listing\_admission\_assessments` is append-only and records candidate, policy, as-of session, `would\_admit`, every reason code, evidence IDs, calculations and code version. A unique key on `(candidate\_id, policy\_id, as\_of\_session)` makes retries idempotent. `listing\_candidates.admission\_policy\_id` references this policy table only after an owner-approved promotion.

#### ### 4.5 Access

- Service role: insert/update current candidate state through a constrained RPC; append events and filings.
- Authenticated owner: read only.
- Anon: no access.
- State transitions use compare-and-swap and append their event atomically.

#### ## 5. State machine

```

pre_listing ■■■listing announced■■■> announced
announced ■■■first trade verified■■■> listed_observing
announced ■■■delay■■■> postponed
pre_listing/announced ■■■cancelled■■■> withdrawn

```

listed\_observing ■■■approved policy passes■■■> paper\_admitted

listed\_observing ■■■invalid/unsupported■■■> rejected

listed\_observing/paper\_admitted ■■■delisted■■■> delisted

Only `paper\_admitted` can enter the ordinary new-buy ResearchAgent candidate pool in a future promoted stage. Pre-listing and observing candidates are researched by the dedicated shadow collector and never set `decision\_observations.entry\_eligible=true`.

No state in this table grants live eligibility. Live execution still requires the normal signal/proposal/autonomy chain and a fresh last-mile broker preflight.

## ## 6. Collection and APIs

### ### 6.1 Scheduled collectors

Run on US business days, not weekly:

1. `listing-filing-discovery` – incrementally reads new/changed SEC submissions.
2. `listing-event-discovery` – reads the approved listing-event adapter.
3. `listing-identity-reconcile` – resolves CIK/listing/symbol without guessing.
4. `listing-broker-capability` – refreshes exact account BUY capability after the listing becomes active and when prior evidence expires.
5. `new-listing-shadow` – captures a bounded rotation of point-in-time features, scores, liquidity and prices; it creates no executable signal.
6. Existing label maturation computes forward returns from immutable observations.

Each collector uses a lease/run key, pagination cursor, bounded wall-clock budget, idempotent source IDs, explicit error state, and an `agent\_runs` or equivalent run record. Empty provider responses are unavailable, not “zero new listings.”

### ### 6.2 Owner APIs

- `GET /api/listing-candidates` – filters by state, type, broker capability and age.
- `GET /api/listing-candidates/[id]` – source lineage, filings, evidence and shadow outcomes.
- `POST /api/listing-candidates/[id]/follow` – creates or updates a manual owner watchlist relationship without changing candidate trust or admission state.
- `POST /api/listing-candidates/[id]/research` – refreshes filing analysis only; never changes eligibility.

Use candidate ID or CIK, not a free-form “ticker-or-CIK” parameter. A company may not have a ticker yet, and ticker reuse/symbol changes make free-form identity unsafe.

## ## 7. Provenance in ResearchAgent

Add explicit discovery sources only when the shadow collector is wired:

- `new\_listing\_observation` – measure-only dedicated collection;
- `new\_listing\_admitted` – future paper-admitted candidate.

Update together:

- the `DiscoverySource` TypeScript union;
- `discovery\_snapshot\_members` CHECK constraint;
- discovery-ledger serialization;
- Research Journal filters and labels;
- carry-forward provenance;
- tests proving the source survives candidate → queue → research → observation.

Do not route a candidate through the generic watchlist partition, which would relabel it `watchlist` and make cohort learning impossible.

## ## 8. Admission-policy shadow

### ### 8.1 Mainline and challengers

- **\*\*Mainline:\*\*** newly listed candidates remain in `listed\_observing`; no automatic paper admission.
- **\*\*Challenger A:\*\*** admission assessment at 20 completed exchange sessions.
- **\*\*Challenger B:\*\*** assessment at 40 completed exchange sessions.
- **\*\*Challenger C:\*\*** assessment at 60 completed exchange sessions.

These are predeclared measurement arms, not three promises to trade. Session age is computed from `first\_trade\_date` using the market calendar. Every arm also requires:

- a fresh exact broker/account BUY capability result;
- at least two genuinely usable applicable scoring dimensions;
- completed-candle provenance with no stale or current-intraday bar;
- valid quote and existing liquidity/risk checks;
- no earnings-repricing or breakdown veto;
- the ordinary score threshold.

An arm records `would\_admit=false` plus reason codes when any requirement fails.

### ### 8.2 Cohorts

Never pool these without reporting them separately:

- operating-company IPO;
- ADR/foreign issuer;
- direct listing;
- ETF launch;
- SPAC IPO or de-SPAC/symbol conversion.

Each candidate is compared with point-in-time matched established instruments from the same market, sector/instrument family, size and liquidity bucket, and calendar window. The benchmark and comparison policy are frozen before labels mature.

### ### 8.3 Outcomes

At h5, h10 and h20, report:

- benchmark-neutral return and rank IC;
- maximum favorable/adverse excursion using executable-bar assumptions;
- close-to-next-open gap and realized volatility;
- spread/liquidity availability and broker denial rate;
- score coverage, dimension availability and breakdown-veto incidence;
- entry-threshold crossing rate, win rate, profit factor and drawdown;
- matched-control delta with confidence intervals.

Do not select a policy from win percentage or raw P&L alone. Listing clusters share market conditions, so the inference unit includes distinct listings and independent listing-week clusters. Results remain `insufficient\_evidence` until both (a) a predeclared lower bound of 30 distinct listings and 12 independent listing-week clusters and (b) the sample required by a frozen power/MDE calculation have matured for the relevant cohort and primary horizon. Thirty is an analysis floor, not proof of edge. If the power requirement takes a long time, the correct result is to keep abstaining.

Any later promotion requires:

1. no unresolved source or broker disagreements;
2. adequate candidate and matched-control coverage;
3. a positive primary-horizon matched-control estimate whose confidence interval clears the frozen material-underperformance tolerance, plus no contradictory sign across the secondary horizons;
4. no materially worse drawdown or liquidity failure than controls;
5. a frozen policy version and code fingerprint;
6. explicit owner approval.

Promotion first permits **\*\*paper admission only\*\***. Live influence requires a separate architecture and approval after forward paper evidence.

### ## 9. UI

Add a "New Listings" section to the existing Research surface, not a new top-level page and not the watchlist table.

For each candidate show:

- company, symbol when assigned, exchange and instrument type;
- lifecycle state and authoritative first-trade date;
- latest filing/amendment and source link;
- broker/account capability with checked/expiry timestamps;
- completed sessions and score/data coverage;
- 20/40/60 challenger assessments and explicit refusal reasons;
- benchmark-relative shadow outcomes once matured.

Upgrade Path owns the aggregate program view: freshness, candidates by state, coverage, unresolved identity/source/preflight conflicts, matured sample counts and promotion readiness. "Working" must never be shown merely because prices rose.

### ## 10. Failure behavior

| Failure                                    | Required behavior                                          |
|--------------------------------------------|------------------------------------------------------------|
| ---                                        | ---                                                        |
| Listing source unavailable or stale        | Record unavailable; do not infer no listings               |
| Filing amendment cannot be fetched         | Keep prior version, record gap; do not call it current     |
| CIK-to-symbol mapping ambiguous            | Keep unresolved; never guess                               |
| Listing postponed/withdrawn                | Append transition; stop eligibility clock                  |
| First-trade date missing                   | Remain `announced`; no session-age calculation             |
| Broker capability unsupported/denied/stale | `would_admit=false`; never coerce to allowed               |
| Fewer than two usable dimensions           | Abstain and record missing dimensions                      |
| Shadow collector exceeds budget            | Persist cursor and resume; do not starve ordinary research |
| Candidate already manually watched         | Preserve owner settings; link identities only              |
| Symbol changes or is reused                | New listing identity/event; never rewrite old observations |
| Schedule misses a run                      | Surface stale status in System Health and Upgrade Path     |

### ## 11. Build sequence

### ### P0 – source and schema proof

- Verify live Robinhood capability schemas without calling an order tool.
- Compare at least two listing sources for coverage, corrections, licensing and cost.
- Build rolled-back schema/RLS/immutability tests.
- No scheduled writes, scoring or trading.

**\*\*Implemented 2026-09-09:\*\*** append-only candidate/event/filing tables with RLS and immutable evidence triggers; a separate `candidate\_probe` preflight purpose that cannot advance the existing execution-enforcement gate; migration and active cron verification. The source comparison and Robinhood-schema corroboration remain open.

### ### P1 – evidence-only US collectors

- Filing, listing, identity and broker-capability collection.
- Candidate detail and Research UI.
- Mainline remains abstain.

**\*\*Implemented slice:\*\*** the weekday `kairos-listing-discovery-us` job reads bounded SEC daily master indexes for S-1/S-1-A/F-1/F-1-A/424B4 metadata and persists issuer filing lineage. The Research Journal's New Listings tab and Upgrade Path show that evidence and the unresolved state. This is explicitly *\*not\** an exchange-listing event collector: SEC filing metadata does not set `first\_trade\_date`, ticker, exchange, or broker capability. Those remaining P1 collectors must not be claimed as shipped until an approved authoritative source passes the source contract.

### ### P2 – shadow admission study

- Bounded point-in-time feature capture.
- 20/40/60 arms, matched controls and label maturation.
- Upgrade Path reporting.

### ### P3 – paper admission, only after evidence and approval

- Promote one frozen admission policy.
- Add `new\_listing\_admitted` to the ordinary research pool without using watchlist.
- Enforce the new-listing gate independently in the paper-entry consumer.
- Mutation-test that changing the gate to `return true` creates test failures.

### ### P4 – possible live consideration

Deferred. Requires forward paper evidence, last-mile broker-preflight enforcement, and a separate explicit owner decision.

India follows P0-P2 only after an NSE/BSE listing source passes the same source contract. The schema and policy engine remain market-neutral; no `.NS` guessing is allowed as listing identity.

### ## 12. Acceptance criteria for the first approved release

1. No discovered candidate is inserted into `watchlist` automatically.
2. No candidate affects `entry\_eligible`, paper fills, live proposals or orders.
3. Every candidate has an immutable source event, payload hash and retrieval time.
4. Filing amendments remain separate accession-version evidence.
5. Listing age uses authoritative `first\_trade\_date`, never discovery time.
6. Exact broker/account/symbol/BUY capability is recorded without submitting an order.
7. Source, identity, broker and data-quality disagreements fail closed and appear in System Health/Upgrade Path.
8. `new\_listing\_observation` provenance survives every persistence boundary.
9. The 20/40/60 policies run on identical point-in-time data and cannot influence execution.
10. Empty, stale and unsupported states are distinguishable in API and UI.
11. Unit, integration, mutation, full-suite, typecheck and isolated production build pass.
12. Any migration is verified in production with rolled-back mutation/immutability checks before a route reads it.

### ## 13. Source references

- SEC EDGAR APIs: <https://www.sec.gov/search-filings/edgar-application-programming-interfaces>
- SEC IPO risk bulletin: <https://www.sec.gov/investor/alerts/ipo-investorbulletin.pdf>
- Robinhood IPO Access: <https://robinhood.com/us/en/support/articles/about-ipo-access/>
- Robinhood IPO request process: <https://robinhood.com/us/en/support/articles/how-to-request-ipo-shares/>
- Nasdaq Daily List product description: <https://www.nasdaqtrader.com/trader.aspx?id=DailyListPD>

---

# Source: features\paper-exit-plan-display\FEATURE\_ARCHITECTURE.md  
# Paper Portfolio Exit Plan Display

Status: approved for implementation (owner requested, 2026-07-21)

### ## Decision

Show one compact `Exit plan` section for every open paper position. Do not show a single "agent exit price": Kairos can exit through several

## ## Why

The owner needs to understand why an open paper position is still held and which condition would close it. The existing position card shows P

## ## Scope

- Project a read-only exit plan from the open position, per-market trading mandate, effective champion/user horizon, and latest same-market v
- Show the persisted trailing/protective stop. This is the current executable paper stop and may ratchet upward; it never moves down.
- Show the persisted profit target when present. After a partial target exit the database clears the target, so the UI must say no remaining t
- Show current score versus the per-market exit threshold and whether the score is fresh enough to act.
- Show current position age versus the effective time horizon, including whether the position is grandfathered onto its entry horizon.
- Show the first currently met condition using PositionMonitor's precedence: time, score, stop, target.
- Reuse the same projection in the Paper Portfolio Trades table, immediately after Score. Only an open BUY trade in the same market and symbol

## ## Boundaries

- Display-only. It cannot write signals, positions, trades, mandates, cash, controls, broker orders, or provider state.
- No market-data/provider calls. It reuses persisted position prices and signals.
- US and India are keyed and resolved separately; currencies and signals never cross.
- Missing/stale research means mechanical exits only. It must never be converted to score zero.
- The manual Close Position command remains separate and unchanged.
- This does not add or change an exit rule, threshold, target, stop, or schedule.

## ## Data Contract

`PaperExitPlan` contains:

- market and position identity
- current persisted price, stop, and optional target
- latest deterministic score, threshold, timestamp, market-session age, and freshness
- open-position weekday age, effective horizon, and horizon source
- current projected state: `hold`, `time\_exit\_due`, `score\_exit\_due`, `stop\_exit\_due`, or `target\_exit\_due`

The projection is assembled on the server. The client receives only display data and performs no decision calculation.

## ## Edge Cases

- No target: display `No remaining target`; do not invent one.
- No score: display `Awaiting held-name research · mechanical exits active`.
- Stale score: display its age and `mechanical exits only`.
- Partial target: the remaining lot has no target and a breakeven-or-higher persisted stop.
- Hedge positions: omit score exits; retain their stop and effective short horizon.
- Invalid numeric data: display unavailable, never coerce to zero.
- A due state is a projection at the persisted price/time. PositionMonitor remains the sole autonomous execution authority.

## ## Acceptance Criteria

1. Every open paper position renders an Exit plan section without changing layout width on mobile.
2. Stop, target, score/freshness, and time horizon match the inputs PositionMonitor would read.
3. US positions cannot receive India scores or mandates, and vice versa.
4. Missing/stale scores cannot produce `score\_exit\_due`.
5. Null targets remain null.
6. The page performs no external provider request and no write.
7. Pure projection tests cover precedence, stale/missing score, null target, grandfathered horizon, and market identity.
8. The Trades table places Exit plan between Score and Realized P&L, and closed trades never receive a current plan.

---

# Source: features\paper-portfolio-benchmarks\FEATURE\_ARCHITECTURE.md  
# Paper Portfolio – market-aware benchmarks + page cleanup

\*\*Status: SHIPPED. 2026-07-08.\*\*

- migration 130 (applied): `paper\_performance.bench\_nav`/`bench\_return\_pct`.
- paper-trade route computes V00 (US) / NIFTY `^NSEI` (India) into bench\_\* + alpha for BOTH markets (US also mirrors into legacy spy\_\* for back-compat).
- metrics route surfaces bench\_return\_pct (+ benchmarkLabel) so India gets a benchmark.
- PortfolioPage gauge is market-aware ("Alpha vs V00"/"Alpha vs NIFTY") and reads the stored bench; BenchmarkChart plots the per-market bench line from bench\_return\_pct (dropped the hardcoded US V00/QQQ fetch – fixed the US-index-on-India leak).
- LiveHoldingsTab is market-aware: US → Robinhood (\$), India → Zerodha Kite (■) via new owner-gated `/api/kite/holdings` (read-only). Fixes the Robinhood-on-India leak.

<details>

<summary>Historical implementation notes (source-preserved)</summary>

Last updated: 2026-07-08

</details>

## ## Why

The Paper Portfolio page has three real defects and one missing feature, all surfaced from the live dashboard:

1. \*\*Benchmark is inconsistent (US) and absent (India).\*\*

- US: the paper-trade route stores alpha vs **SPY** (``paper_performance.alpha_pct``, [app/api/agents/paper-trade/route.ts:639]), but the UI gauge independently fetches **V00** and computes a *second, different* alpha labelled "Alpha vs V00" ([components/dashboard/PortfolioPage.tsx:199,225,736]). Two benchmarks for one number.
  - India: the route explicitly skips benchmark computation (``if (market==="us")``), so India stores no benchmark at all – yet the UI still renders the hardcoded "Alpha vs V00" needle on the India view (reads a meaningless  $\sim +0.0\%$ ).
  - The Risk page footer already promises "US vs SPY, India vs NIFTY" – India has no NIFTY series wired, so the promise is unmet.
2. **US-broker text leaks onto the India view (bug).** ``LiveHoldingsTab`` renders "Robinhood Live Positions" + "Robinhood Trading account \*\*\*\*8641" unconditionally ([PortfolioPage.tsx:535,609]) – including when ``activeMarket === "india"``, whose broker is Kite, not Robinhood. The trade-queue is already market-guarded at :733; LiveHoldings is not.
  3. **No benchmark line on the return chart.** The "Portfolio vs Benchmark (% return)" chart currently plots only the portfolio return line; the benchmark it names is not drawn. User wants the benchmark drawn alongside portfolio gain/loss, for BOTH markets.
  4. **General cleanup** of the page (redundant/again-fetched benchmark, dead labels).

Non-goal: this does not touch the fill engine, sizing, money limits, or live-order paths. Read/display layer + one additive benchmark-series write for India.

## Proposed design

- ### A. One benchmark per market, computed server-side (single source of truth)
- **US benchmark = SPY** (already computed and stored in ``paper_performance``). Retire the client-side V00 fetch + client-side alpha. The gauge and the chart both read the stored ``spy_return_pct`` / ``alpha_pct``. Label becomes **"Alpha vs SPY"**. (Decision point for you: keep SPY, or switch the stored benchmark to V00? SPY is already wired end-to-end; recommendation: keep SPY, drop V00.)
  - **India benchmark = NIFTY 50** (``^NSEI`` via the existing India quote path – same Yahoo ``.NS``/index source India NAV already uses). Add its close + return to the India ``paper_performance`` rows, mirroring the US SPY fields. This is the only data-layer change: the route's per-market perf block gains an India branch that fills ``bench_nav``/``bench_return_pct``/``alpha_pct`` from NIFTY instead of leaving them null.

**Schema:** ``paper_performance`` currently has US-specific ``spy_nav``/``spy_return_pct``. Two options:

- (A1, recommended) rename-free, additive: add generic ``bench_nav`` numeric, ``bench_return_pct`` numeric columns (nullable). US writes SPY into them (and keeps `spy_*` for backward-compat), India writes NIFTY. UI reads the generic columns. One additive migration, verified applied before UI reads them.
- (A2) reuse ``spy_*` columns for both markets (semantically wrong name for India). Rejected – misleading column name.

- ### B. Market-aware UI (no cross-market leakage)
- Gauge label + benchmark name derived from ``activeMarket``: **"Alpha vs SPY"** (US) / **"Alpha vs NIFTY"** (India). No hardcoded "V00".
  - ``LiveHoldingsTab`` (Robinhood) renders **only when** ``activeMarket === "us"``. On India, either hide it or show a Kite-scoped equivalent (out of scope here – hide for now, matching the existing trade-queue guard pattern).

- ### C. Benchmark overlay on the return chart (both markets)
- "Portfolio vs Benchmark (% return)" plots two lines: portfolio cumulative %-return (existing) + benchmark cumulative %-return (**SPY** US / **NIFTY** India), both rebased to 0% at the window start, from ``paper_performance`` series. Legend + the headline number becomes portfolio – benchmark = alpha for that window.

- ### D. Cleanup
- Remove the client V00 fetch/compute path once the gauge/chart read server values.
  - Delete the now-dead "Alpha vs V00" hardcoded label.
  - Keep the empty-state copy (US "No open positions...") – correct as-is; it will populate once the paper-fill fix (migrations 126-128) opens US positions.

- ## Data contract / touch list (for approval)
- **Migration (additive):** ``paper_performance.bench_nav``, ``bench_return_pct`` (nullable). Verify applied via `information_schema` before UI reads them.
  - `app/api/agents/paper-trade/route.ts` – per-market perf block: US fills `bench_*` from SPY (already fetched); add India branch filling `bench_*` from NIFTY (``^NSEI``).
  - `components/dashboard/PortfolioPage.tsx` – market-aware label; read stored `bench_*`; drop client V00 fetch; guard ``LiveHoldingsTab`` to US; add benchmark line to the chart.
  - **Docs:** `SYSTEM_OVERVIEW.md` (benchmark section) + `system-map.json` only if an agent-to-agent flow changes (it does not – display layer + one perf field).

## Locked decisions (2026-07-08)

1. **US benchmark = V00.** Move the server-side benchmark computation from SPY to **V00** in the paper-trade perf block; store into the new generic ``bench_*` columns. Historical ``spy_*` rows are left as-is (V00=SPY, both S&P 500); the new series is V00 forward. UI label = "Alpha vs V00". Retire the client-side V00 fetch – the value now comes from the server like India's.



2. **\*\*India benchmark = NIFTY 50 (^NSEI)\*\*** via the existing India index source. Written into `bench\_\*` on India `paper\_performance` rows. UI label = "Alpha vs NIFTY".

3. **\*\*India live tab = stub a Kite holdings tab.\*\*** Instead of hiding LiveHoldings on India, render a market-aware live-holdings tab: US → Robinhood block (existing); India → a **\*\*Kite holdings\*\*** block reading the India account's positions **\*\*read-only\*\*** (reuse the existing India live/holdings source the India Live page already uses – no new order path, no new broker-write surface). Guard: Robinhood block renders only for US, Kite block only for India – no cross-market leak either way.

### Revised touch list

- **\*\*Migration (additive):\*\*** `paper\_performance.bench\_nav`, `bench\_return\_pct` (nullable). Verify applied via information\_schema before UI reads.
- **\*\*paper-trade/route.ts:\*\*** US perf block computes V00 (was SPY) into bench\_\*; new India branch computes NIFTY (^NSEI) into bench\_\*.
- **\*\*PortfolioPage.tsx:\*\*** market-aware label + benchmark line on the chart; drop client V00 fetch; `LiveHoldingsTab` becomes market-aware (Robinhood=US, Kite=India read-only).
- **\*\*Kite holdings read:\*\*** reuse the existing India live-holdings data source (same one the India Live + Signals page uses); read-only, no order/mutation path.
- **\*\*Docs:\*\*** SYSTEM\_OVERVIEW.md benchmark section + "Last updated" bump. system-map.json only if a flow changes (it does not).

**\*\*Awaiting explicit approval to implement\*\*** (say "approved / implement / code it"). One caveat to confirm: switching US to V00 means the stored benchmark series changes provider mid-history (SPY rows before, V00 after) – acceptable since both track the S&P 500. Flag if you'd rather backfill V00 across history instead.

---

# Source: features\per-user-broker-risk\FEATURE\_ARCHITECTURE.md  
# Feature Architecture: Per-User Broker Connections & Private Risk Analytics

## Status

Architecture status: Approved  
Architecture approved: Yes (owner, 2026-09-14)  
Approved scope: Phases 0-3  
Approved date: 2026-09-14  
Implementation allowed: Yes, phase by phase

**\*\*Open Decision 1** (the daily Kite login) is RESOLVED: option (a). **\*\*The owner accepts that each India guest must log in to Zerodha every trading day, on the condition that staleness is shown **\*\*loudly\*\*** – on the page and in the email – rather than a stale number being presented as current. That obligation is binding on Phases 2 and 3: a risk figure computed from a token that expired is never rendered as if it were today's.**

Relationship to the other two documents:

| Document                                                                                                                                               | Status |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| --- ---                                                                                                                                                |        |
| `features/shared-viewer-access/`   <b>**Built.**</b> Phases 0-2 shipped 2026-09-14. Read-only viewers, no data of their own.                           |        |
| <b>**This document**</b>   Adds a <b>*private*</b> data plane on top: a guest connects their own broker and sees risk analytics on their own holdings. |        |
| `features/multi-tenant/`   <b>Deferred.</b> Full tenancy (own book, own genome, per-user learning). This feature is a strict subset of it.             |        |

This is the first feature that gives a non-owner **\*\*private rows of their own\*\***. That is a category change from viewer access, and most of the design below exists to contain it.

---

## Feature Purpose

Let an invited guest connect their own Robinhood (US) and/or Zerodha Kite (India) account, read-only, and see risk analytics computed on **\*\*their\*\*** holdings – never the owner's, and never pooled. Optionally, receive a daily risk email if they opt in.

The owner's book, research, learning and paper portfolio are unaffected. Nothing about this feature lets a guest trade, and nothing lets the owner see a guest's holdings.

## Scope

- Per-user, encrypted, **\*\*read-only\*\*** broker credentials for Robinhood and Kite.
- A per-user live holdings snapshot and a per-user risk analytics run.
- Private risk pages showing only the signed-in user's own accounts.
- Opt-in daily risk email, per user, with unsubscribe.
- Tenant isolation (`user\_id` + RLS + service-role query guards) on the private data plane only.

## Non-Goals (explicitly OUT of scope)

- **\*\*Any order path for a guest.\*\*** No place, cancel, exit, stop, or Guardian action. Read-only broker scopes only, enforced structurally (below), not by convention.

```

- **Any pooling.** A guest's holdings never enter the owner's research, scoring,
 learning, paper book, or shadow evidence – and never another guest's.
- **Owner visibility into guest holdings.** The owner administers *access*, not
 positions. The owner is not an exception to isolation here.
- Per-user paper books, genomes, or learning loops (that is `multi-tenant/`).
- Guest access to the owner's live portfolio, which stays owner-only.

Current Behavior (verified against code + production, 2026-09-14)

Everything broker-related is single-account by construction.

Concern	Today	Evidence
Broker credentials	One global row per fixed `key_name` in `api_key_vault`	`lib/brokers/mcp-driver.ts` `vaultSet`, `lib/brokers/mcp-regis
`api_key_vault` shape	`key_name, key_value, provider, ...` – **no user column**	production `information_schema`
Live holdings	`live_account_snapshots`, `live_performance` – **no user column**	production
Risk output	`holding_risk_runs`, `holding_risk_snapshots` – **no user column**, owner-email-pinned RLS	production
Risk inputs	`fetchRobinhoodBrokerAccounts` / `fetchKiteBrokerAccount` + `fetchUsCandles` / `fetchYahooCandles`	`app/api/agents/holding-r
Scheduling	`kairos-holding-risk-{us,india}`, `kairos-live-snapshot`, `kairos-broker-keepwarm` – one account each	`cron.job`
Email	Resend (or SMTP) via `lib/providers/email`	`lib/providers/email/index.ts`

The one piece of good news, and it is load-bearing

Market data does **not** scale per user. `lib/data/candles.ts` fetches through
`providerCachedFetch`, day-cached per symbol under a per-provider budget. Two
users holding AAPL cost one AAPL candle fetch, not two. The same is true of
`price_cache`.

So the per-user cost is **broker API calls only** – which are per-account by
nature and not something caching can fix, but are also not metered against the
market-data budgets that are already strained (Alpha Vantage over its default,
Massive at ~5 req/min).

The three hard problems

1. Credentials. `api_key_vault` is a global key/value store addressed by fixed
names. It cannot hold two users' Robinhood tokens. It also mixes *app* provider
keys (Massive, Alpha Vantage) with *broker* credentials; overloading it further
would put guest secrets in the same table the owner's admin vault UI edits.

2. Kite tokens expire daily. Zerodha's access token is good for one trading
day; the work log already records "a stale Kite daily token just skips the day".
Today that is one daily login for the owner. With N guests it is **N daily
logins**, each of which only that guest can perform. This is a Zerodha platform
constraint, not something the design can engineer away, and it is the single
biggest UX risk in this feature.

3. Cron fan-out returns. The viewer feature's cost guarantee was "no per-user
crons". This feature breaks that by necessity – but only for two jobs, and the
order-path jobs must be excluded outright:

Job	Fans out?	Why
`kairos-live-snapshot`	yes	per-account holdings refresh
`kairos-holding-risk-{us,india}`	yes	per-account risk
`kairos-broker-keepwarm`	yes	per-account OAuth refresh
`kairos-manual-fill-detect`	**no – never**	order-path; guests are read-only
`kairos-live-exit-monitor`	**no – never**	places protective exits

Proposed Behavior

1. Credentials: a new table, not the vault

`user_broker_credentials`, separate from `api_key_vault`:

- Keyed `(user_id, broker)`; `broker` in `('robinhood','kite')`.
- Ciphertext only. Encrypted with a server-held key (env), never with anything a
 client can reach. The plaintext token is never returned by any API, never
 logged, never rendered – the UI shows connection *state*, not secrets.
- `scope` column, constrained to `read_only` at the check-constraint level. There
 is no value that means "can trade", so a future bug cannot widen it silently.
- RLS: a user reads only their own row; **no read policy for the owner**. The
 owner sees that a connection exists via a separate non-secret status view, not
 the credential row.
- Revoking viewer access (Phase 1 of the viewer feature) must also disable the
 credential – one revocation, both effects.

The existing global `api_key_vault` keeps the owner's own broker tokens unchanged.
Do not migrate the owner into the new table in the same change; that is a separate,
later cleanup with its own rollback.

2. Read-only enforcement, structurally

```

The Robinhood MCP surface exposes ``place_equity_order``, ``cancel_equity_order`` and friends. "Guests are read-only" must not rest on nobody calling them:

- Guest credentials are loaded through a **separate client factory** that binds only the read tools (``get_equity_positions``, quotes, balances). The order tools are not reachable from that object at all.
- Every order-capable route keeps ``requireOwner()``. Being a connected guest grants nothing on those routes.
- The ``<agentic-account-id>``-only order-placement rule in ``CLAUDE.md`` is unchanged and unaffected; guest accounts are never candidates for it.

### ### 3. Isolation of the private data plane

``user_id`` (not null, FK ``auth.users``) on: ``user_broker_credentials``, ``user_account_snapshots``, ``user_holding_risk_runs``, ``user_holding_risk_snapshots``, ``user_risk_email_prefs``.

**New tables, not new columns on the owner's tables.** Adding ``user_id`` to ``live_account_snapshots`` / ``holding_risk_snapshots`` would put guest rows in the tables the owner's live pages, kill-switch baseline and Guardian read – one missed filter and a guest's position silently enters an owner money-path calculation. Separate tables make that class of bug impossible rather than merely tested-for.

RLS: ``user_id = auth.uid()`` for reads; service-role for writes. Per ``docs/arch/07-coding-conventions.md``, every service-role read of these tables carries an explicit ``user_id`` filter – RLS is the backstop, the filter is the mechanism.

### ### 4. Per-user risk pipeline

One job per market iterates **connected, non-revoked** users, and for each:

1. Fetch holdings with that user's read-only credential.
2. Compute risk with the existing ``computeRiskMetrics`` / ``computeCorrelationClusters`` – unchanged code, different input.
3. Write ``user_holding_risk_`` rows scoped to that user.

Candles come from the shared cache, so a symbol a guest holds that the owner's universe **ALREADY** covers is free. **Correction to an earlier claim in this document** that market data does not scale per user at all: a guest symbol **OUTSIDE** that universe has no cached day-row, so the first risk run for it costs a real provider fetch, and the day-cache only makes it free from the second reader onward. The honest statement is that cost scales with the number of **DISTINCT** new symbols guests introduce, not with the number of guests – which is much flatter than per-user, but not zero. Phase 2 must measure that set before fanning out. A user whose credential is missing, expired or revoked is **skipped** with a recorded reason, never silently, and never falls back to the owner's data.

### ### 5. Opt-in daily risk email

> **STATUS 2026-09-16** – the opt-in UI is **WITHDRAWN**; the backend is intact. The toggle was removed from ``/dashboard/user-settings``, so no guest can currently turn this on. Everything below still exists and still runs: the table, the hourly ``kairos-user-risk-email`` cron, the ``send_hour_utc`` honouring, the one-send-per-day index and the unsubscribe route. What does not work is **DELIVERY** – the send goes through the shared ``<verified-email-sender>`` sender, which Resend permits only to the Resend account owner, so a guest who enabled it received nothing and had no way to tell why. Offering a switch that silently does nothing is worse than offering none. Restore the UI once either a verified domain or the SMTP transport (``EMAIL_PROVIDER=smtp``, implemented 2026-09-16) is configured; ``/api/user-prefs`` and the preference rows were deliberately left untouched so that is a one-file change.

- Off by default. ``user_risk_email_prefs`` holds the opt-in, the send hour, and an unsubscribe token.
- Content is built **only** from that user's own ``user_holding_risk_`` rows. The owner's book, research, and signals are not included – that would turn a risk report into a recommendation, which is the thing this feature deliberately is not.
- One send per opted-in user per day, with a per-send audit row and a hard cap so a scheduling bug cannot mail in a loop.
- Every email carries a working unsubscribe that flips the opt-in without a login.

### ### 6. Cost model

| Cost                          | Scales with                         | Mitigation                                                                                              |
|-------------------------------|-------------------------------------|---------------------------------------------------------------------------------------------------------|
| Broker API calls              | users × jobs/day                    | Only 3 jobs fan out; snapshot cadence for guests can be lower than the owner's 2-hourly                 |
| Market data (candles, quotes) | <b>distinct symbols</b> , not users | Already day-cached per symbol; overlap is free                                                          |
| LLM                           | should be <b>zero</b> for guests    | Guest risk is the deterministic scorer only. No LLM prose note. Stated as a rule, enforced by the guest |
| Email                         | users × 1/day                       | Trivial at this scale                                                                                   |

The rule that keeps this true: **a guest-triggered path may call a broker, and may read cache, but may never call an LLM or a metered market-data provider outside the shared cache.**

---

## ## Phased Rollout

| Phase | Content                                                                                          | Gate to next                                                                   |
|-------|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| 0     | Schema + RLS + encryption, no UI, no job. Tables exist and are empty.                            | Isolation matrix passes with two seeded test users                             |
| 1     | Connect/disconnect flow for the signed-in user; read-only client factory; connection status only | **DONE 2026-09-14.** A guest can connect                                       |
| 2     | Per-user snapshot + risk job; private risk page                                                  | **DONE 2026-09-14.** A guest sees only their own; owner's pages byte-identical |
| 3     | Opt-in daily email + unsubscribe                                                                 | **DONE 2026-09-14.** Send audit shows exactly one mail per opted-in user       |

Phases 2 and 3 each add scheduled work; neither may ship before the cost rule in §6 is verified against real usage – including the corrected form of it: the count of distinct symbols guests hold that are NOT already in the owner's cached universe.

**Phase 2 as shipped**, and how the §6 cost gate was actually cleared.  
The gate said Phase 2 must measure the set of guest symbols outside the cached universe before fanning out. Tracing the candle path made a stronger answer available than a measurement: `fetchYahooCandles` is Yahoo's **keyless** chart endpoint – no API key, no quota, no budget line – while `fetchUsCandles` starts there and then falls back through Massive → EODHD → TwelveData → Alpha Vantage, several already over budget. So the guest pipeline (`lib/risk/guest-risk.ts`) imports the unmetered fetcher and nothing else, and a held symbol Yahoo cannot serve is recorded UNCOVERED with its correlation reported **unknown** rather than escalated to a paid provider or silently treated as zero correlation. Guest metered-provider spend is therefore **structurally zero** rather than measured and hoped about, which satisfies §6's own rule ("may never call an LLM or a metered market-data provider outside the shared cache") more strongly than the measurement would have. Enforced at the imports by `tests/guest-risk-budget.test.ts`. What genuinely scales per guest is one broker call per user per market per day, against that user's own Zerodha quota, not ours.

The job is `POST /api/agents/user-holding-risk?market=us|india`, cron-gated, scheduled 15 minutes after the owner's own job for the same market (`'20260914190000_user_holding_risk_cron.sql'`, applied and verified). It writes only `user_*` tables, records every uncomputable user as `skipped` with a reason, and runs no LLM. The page is `/dashboard/my-risk` – deliberately **not** `/dashboard/risk`, which is the owner's Daily Per-Holding Risk dashboard over the owner's live account book. (The first draft of this phase did write to that path, which would have replaced the owner's dashboard **and**, since the path was simultaneously added to `VIEWER_PAGES`, aimed the owner's book at every guest. It was caught before commit and is now asserted against.)

**Phase 1 as shipped**. `/dashboard/connections` (viewer-reachable) plus `/api/broker-connections` (GET state, POST disconnect) and `/api/broker-connections/kite/login`. Zerodha registers one redirect URL per app, so guests reuse `/api/kite/callback`; which account a token belongs to is decided from the HMAC-signed state cookie (`guest:<userId>`), never from the session, because a session can be absent on a redirect and falling back to the owner path would let a guest's token overwrite the owner's vault entry. The guest branch never touches `api_key_vault`, `storeAccessToken`, `broker_accounts` or `strategy_config`. Staleness is rendered as a full-width amber banner reading "Your figures are out of date." with a Reconnect action – the owner's stated condition for accepting the daily login. Robinhood is declared unsupported and shown as "Coming soon" rather than offering a button that fails. No job runs and no risk figure is computed yet; connecting today stores a credential and nothing else consumes it.

## ## Acceptance Tests

As with the viewer feature, negative tests are issued **directly** against PostgREST with each user's JWT, not through the UI.

- Guest A cannot read guest B's credential, snapshot, risk run, or email prefs.
- The **owner** cannot read any guest's credential row or holdings. (Deliberate: the owner administers access, not positions.)
- A guest's session cannot reach any order-capable route, and the guest client factory exposes no order tool.
- A revoked viewer grant disables the broker connection and stops the per-user jobs on the next run – not merely the navigation.
- A guest's holdings never appear in the owner's `live_account_snapshots`, `live_performance`, kill-switch NAV baseline, paper book, research inputs, learner evidence, or any shadow ledger. Asserted by row counts before/after a guest run.
- An expired Kite token produces a recorded skip with a reason, never a fallback to the owner's account and never a partial risk number presented as complete.
- No guest-triggered path calls an LLM – asserted against `llm_call_log` across a full guest cycle.
- Unsubscribe works without a login and stops the next send.
- Plaintext credentials appear in no response body, no log line, and no error message.
- Full suite, typecheck, production build, migration applied and rollback-verified.

## ## Open Decisions (for the owner)

- The daily Kite login.** Each India guest must log in to Zerodha every trading day or their risk report silently goes stale. Options: (a) accept it and make

- staleness loudly visible; (b) India guests get US-only analytics; (c) defer India entirely to a later phase. **\*\*Recommendation:** (a), with the staleness shown on the page and in the email. **\*\*** This should be decided before Phase 0 – it changes what gets built.
- \*\*Who pays for broker-driven load\*\*, and what per-guest cap applies. This is lighter than the market-data question (broker calls are not metered against the strained provider budgets) but should still have a stated ceiling.**
  - \*\*Snapshot cadence for guests\*\*** – the owner's is 2-hourly; daily is likely enough for risk analytics and costs a fraction.
  - \*\*Does the owner get \*any\* visibility into guest holdings?\*** This design says no. If the answer is yes for support reasons, it must be an explicit, audited, guest-consented path – not an implicit owner exemption.

### On the regulatory question

Worth stating plainly because it differs from the viewer feature: computing risk analytics on a person's **\*\*own\*\*** holdings, shown only to them, is materially less exposed than sharing the owner's research and signals with people who may trade on them. It is analysis of their data, for them, with no recommendation. That is not legal advice and the gate on the viewer feature's research page still stands – but this feature does not widen it, and the daily email must stay descriptive (exposure, concentration, correlation) rather than prescriptive (buy/sell/hold) to keep it that way.

## Update triggers (per CLAUDE.md)

If approved and implemented, the SAME change must update:

- `docs/arch/02-tech-stack.md` (per-user broker credential store),
- `docs/arch/04-database-schema.md` (new tables + RLS),
- `docs/arch/05-crons-and-scheduling.md` (**\*\*the per-user fan-out jobs – this feature does add crons, unlike shared-viewer-access\*\***),
- `docs/arch/06-env-variables.md` (credential encryption key),
- `docs/arch/08-risk-and-safety.md` (guest read-only boundary, order-path exclusion),
- `PROJECT\_DECISIONS.md`, and `public/agent-diagrams/system-map.json` (a new per-user data plane is a flow change).

## Addendum 2026-09-14 – Phase 3, the dial, and what a digest may contain

Owner asked for Phase 3 plus three things from Seeking Alpha's Pre-market Portfolio Digest: the gauge dial, per-stock score detail, and score history.

**\*\*Built.\*\*** The dial (`lib/risk/risk-gauge.ts`, one source of bands and needle position for both surfaces), a "why it reads this way" breakdown derived from the same stored figures the page already shows, portfolio score history, and per-holding detail with weight history. Phase 3 itself: opt-in daily email, hourly cron honouring each user's `send\_hour\_utc`, a database-enforced one-send-per-user-per-day cap, and a no-login unsubscribe.

**\*\*The dial is deliberately not SA's dial.\*\*** Theirs is a RATING – Strong Sell to Strong Buy, green means "buy this". Ours measures how much risk a portfolio carries – green means "less concentrated, less leveraged to the market". Same shape, opposite kind of claim. The labels stay Low/Moderate/Elevated/High so the difference is legible without reading the docs.

**\*\*Two SA sections were NOT built, and need an owner decision before they are.\*\***

- \*Ratings breakdown ("7 Strong Buy, 4 Buy, 13 Hold").\*** For a guest this could only come from the owner's `agent\_signals`. §5 of this document rules that out: "the owner's book, research and signals are not included – that would turn a risk report into a recommendation, which is the thing this feature deliberately is not." The owner's own `dashboard/risk` does show `agent\_signals` (DISPLAY ONLY), but the owner reading their own research is a different act from mailing per-stock buy/sell ratings to friends and family. This is the regulatory line already flagged as a gate to clear outside the repo, not a UI decision.
- \*Breaking news per holding, and yesterday's gainers/losers.\*** News needs a metered provider call per guest symbol, and quotes for symbols outside the owner's cached universe are not in `price\_cache`. Both break §6's rule that a guest path may never call a metered provider outside the shared cache. A cache-only gainers/losers restricted to already-cached symbols is feasible and cheap; a full one is not. Not built pending a decision on which.

**\*\*Why the explanation is templated, not written by an LLM.\*\*** §6 forbids a guest-triggered LLM call, but the stronger reason is that an explanation which can drift from the number it explains is worse than none. Every line is derived from the stored figures.

**\*\*One defect worth recording.\*\*** The email trimmed drivers to the top four by weight, and the "no usable price history" caveat carries the lowest weight by design so it never leads – so the trim dropped exactly the line that stops the email overclaiming. Drivers now carry `kind: "driver" | "caveat"` and callers trim drivers only.

---

```

Source: features\performance-truth\FEATURE_ARCHITECTURE.md
Performance Truth Layer – Feature Architecture

<details>
<summary>Historical implementation notes (source-preserved)</summary>

Last updated: 2026-07-09 (v2 – post Codex BLOCKER/HIGH review)

</details>

Status: **P0 – awaiting approval before any implementation**

Goal

Build a deterministic, mandate-aware, cost-adjusted evaluation layer that
answers one question:

> "Did this strategy create repeatable, benchmark-relative edge in the mandate
> it claims to serve?"

Two evaluation targets (both required):
1. **Book/trade metrics** – what actually happened to the paper book (closed trades + NAV)
2. **Opportunity-level metrics** – what would have happened across the full scored opportunity set (decision_observations × observation_labels)

No live trading impact. No LLM weight optimization. Append-only. Deterministic.

Reuse inventory – what already exists

Analytics math (`lib/analytics/performance-metrics.ts`)
All math already built. Full reuse, zero changes:
- `navToReturns()`, `sharpe()`, `sortino()`, `maxDrawdown()`
- `expectancy(trades[])` – expects rows with `{ pnl_pct, outcome }` directly (NOT `{ returnPct }`)
- `costNet(trades[])`, `slip(trades[])`
- `calibration(rows[])` – expects `{ predicted: number (0-1), win: boolean }` (NOT `{ score, won }`)
- Returns `Metric { value, n, insufficient }` – abstains correctly when n < min

API (`app/api/agents/performance/metrics/route.ts`)
Reads `paper_performance + paper_trades`, returns full metric set per market.
Additive change: accept `?mandateId=` for trade-level filtering. NAV metrics
remain whole-book (paper_performance has no mandate column); label them clearly
as book-level, not mandate-specific. Mandate-specific Sharpe comes from
opportunity-level evaluation in `strategy_evaluations`.

UI (`components/dashboard/PerformanceTruth.tsx`)
Full metric tiles already render. **Two additive changes only:**
(1) mandate selector dropdown, (2) evaluation history table below existing tiles.
Existing tiles untouched.

Tables (confirmed columns from migrations audit)
- `paper_trades` – closed via `closed_at IS NOT NULL` (no `status` column)
 - Execution quality: `expected_price`, `realized_slip_pct`, `fill_status`, `spread_applied`, `data_confidence`, `tainted`, `taint_reason`,
- `paper_performance` – canonical NAV plus seed-based total/daily P&L, cumulative
 outcome counts/win rate, `alpha_pct`, and `bench_return_pct` per `(date, market)`;
 NOT per mandate. Both runtime writers use the same derivation helper so a later
 mark-to-market cannot leave stale/default analytics behind.
- `decision_observations` – immutable append-only; full feature scores, weights_used, signal_id
- `observation_labels` – matured forward returns per observation (used by learning dataset)
- `lib/learning/dataset.ts` – `loadLabeledDataset()` already joins decision_observations × observation_labels
- `edge_universe_members` – **NOT point-in-time**; current-liquid/survivorship-biased.
 Do NOT use for promotion-grade PIT universe evaluation. Flag all evaluations that
 use this as `universe_pit_safe: false` in strategy_evaluations.

Validation engine (`lib/validation/engine.ts`)
Walk-forward replay, block bootstrap, p-value. Already persists to
`validation_experiments` table and updates `strategy_versions.validation_experiment_id`.
This is the canonical challenger evidence ledger. The new `strategy_evaluations`
table is a summary layer that REFERENCES `validation_experiments` via FK – it
does NOT replace it.

Auth helper
Use `requireOwner()` from `lib/auth/require-owner.ts` – not `requireOwnerSession()`.

Net-new work

Migration A – `investment_mandates`

create table public.investment_mandates (
id uuid primary key default gen_random_uuid(),

```

```

name text not null,
market text not null check (market in ('us', 'india')),
horizon text not null check (horizon in (
'swing_2_20d', 'position_1_6m',
'long_term_1y_plus', 'income_dividend'
)),
-- 'intraday' excluded: no intraday data/execution model exists
benchmark_symbol text not null,
min_holding_days int,
max_holding_days int,
evaluation_horizon_days int[] not null default array[5, 10, 20],
max_position_pct numeric not null default 10, -- advisory/evaluation only; NOT wired to broker gateway
turnover_budget_monthly numeric,
allowed_asset_types text[] not null default array['equity', 'etf'],
allowed_signal_families text[] not null default array['momentum', 'quality', 'technical', 'sentiment', 'macro'],
tax_sensitivity text not null default 'medium'
check (tax_sensitivity in ('low', 'medium', 'high')),
income_preference text not null default 'none'
check (income_preference in ('none', 'dividend', 'growth')),
execution_model text not null default 'conservative_close'
check (execution_model in ('conservative_close', 'optimistic_close')),
-- NOT a live-trading authorization flag. Advisory/review eligibility only.
-- Broker gateway NEVER reads this. Live orders always require owner-click gate.
eligible_for_live_review boolean not null default false,
mandate_version int not null default 1,
active boolean not null default true,
archived_at timestamptz,
created_at timestamptz not null default now()
);

alter table public.investment_mandates enable row level security;

-- Deny by default; service-role routes bypass RLS. Add owner policies only when direct client reads needed.

-- Seed default mandates (maps to existing swing behavior — all existing signals default to these)
insert into public.investment_mandates
(name, market, horizon, benchmark_symbol, min_holding_days, max_holding_days)
values
('Swing US 2-20d', 'us', 'swing_2_20d', 'VOO', 2, 20),
('Swing India 2-20d', 'india', 'swing_2_20d', 'NIFTY50.NS', 2, 20);

```

```

Migration B — `strategy_evaluations` (append-only, trigger-guarded)
create table public.strategy_evaluations (
id uuid primary key default gen_random_uuid(),
mandate_id uuid not null references public.investment_mandates(id),
-- Snapshot of mandate AT EVALUATION TIME. Mandate rows are mutable;
-- evaluations must be reproducible even if mandate config later changes.
mandate_snapshot jsonb not null,
market text not null,
evaluated_at timestamptz not null default now(),
evaluator_version text not null, -- VERCEL_GIT_COMMIT_SHA or 'local'

-- Input dataset bounds (for idempotency detection)
dataset_hash text, -- sha256 of sorted trade IDs
window_start date,
window_end date,

-- Trade counts (always filled)
n_trades_total int not null default 0,
n_trades_evaluable int not null default 0, -- excludes tainted + excluded_from_learning
tainted_count int not null default 0,
excluded_count int not null default 0,
n_observations int, -- decision_observations count in window

-- Book metrics (from closed paper trades + paper NAV; whole-book Sharpe)
-- Null when n_trades_evaluable < 20
book_sharpe numeric,
book_sortino numeric,
book_max_drawdown numeric,
book_win_rate numeric,
book_expectancy_pct numeric,
book_alpha_pct numeric,
book_benchmark_symbol text,
book_cost_adjusted_return_pct numeric,
book_slip_vs_modeled_bps numeric,

-- Opportunity-level metrics (from decision_observations × observation_labels)
-- Null until observation_labels for this window have matured
opp_n_labeled int,
opp_hit_rate numeric, -- fraction of scored signals where actual beat benchmark
opp_benchmark_neutral_expectancy numeric, -- avg(label - benchmark_label) across observations

```



```

opp_ic numeric, -- information coefficient (score vs label rank)
opp_t_stat numeric,
universe_pit_safe boolean not null default false, -- true only when PIT universe membership exists

-- Walk-forward validation reference
validation_experiment_id bigint references validation_experiments(id),
walk_forward_folds jsonb, -- [{fold, start_date, end_date, n_effective, p_improvement, passed}]

-- Display-only health flag (NOT a live-trading promotion gate)
-- 'insufficient_sample' | 'negative_or_zero_edge' | 'promising_but_unvalidated' | 'validation_required'
health_label text not null default 'insufficient_sample',
health_reason text,

-- FULL promotion gate (separate from health_label):
-- Requires validation_experiments.passed=true, p_improvement, fold wins, drawdown cap,
-- benchmark-neutral expectancy, cost-adjustment, taint ratio check.
-- NOT implemented in P0 display layer. Wired at Learner promotion gate in P1.
promotion_eligible boolean not null default false,

created_at timestamptz not null default now()
-- APPEND-ONLY: trigger below blocks updates/deletes
);

alter table public.strategy_evaluations enable row level security;

create index se_mandate_market_idx on public.strategy_evaluations(mandate_id, market, evaluated_at desc);

-- Append-only enforcement (same pattern as decision_observations)
create or replace function se_no_update() returns trigger language plpgsql as $$
begin raise exception 'strategy_evaluations is append-only'; end; $$;
create trigger se_no_update_trigger
before update or delete on public.strategy_evaluations
for each row execute function se_no_update();
Migration C - `mandate_id` FK on existing tables
-- Nullable first — backfill after seed inserts before making NOT NULL
alter table public.agent_signals add column mandate_id uuid references public.investment_mandates(id);
alter table public.paper_trades add column mandate_id uuid references public.investment_mandates(id);
alter table public.decision_observations add column mandate_id uuid references public.investment_mandates(id);
Backfill SQL (run after seed, before any NOT NULL constraint):
-- Assign all existing rows to the matching default mandate by market
update public.agent_signals set mandate_id = (
select id from public.investment_mandates

```

```

where name = case when market = 'india' then 'Swing India 2-20d' else 'Swing US 2-20d' end
and active = true limit 1
) where mandate_id is null;

```

-- Same for paper\_trades

```

update public.paper_trades set mandate_id = (
select id from public.investment_mandates
where name = case when market = 'india' then 'Swing India 2-20d' else 'Swing US 2-20d' end
and active = true limit 1
) where mandate_id is null;

```

-- Same for decision\_observations (market column exists since mig 059)

```

update public.decision_observations set mandate_id = (
select id from public.investment_mandates
where name = case when market = 'india' then 'Swing India 2-20d' else 'Swing US 2-20d' end
and active = true limit 1
) where mandate_id is null;

```

---

### File wiring – `mandate\_id` propagation

Every boundary that creates a signal, observation, paper fill, or validation run must read the default mandate and attach `mandate\_id`. Files to update:

```

File	What to change
`lib/research-agent.ts`	When inserting `agent_signals`, resolve default mandate from `investment_mandates` by market; pass `mandate_id`
`lib/deepseek-agent.ts`	Same – if it also inserts agent_signals or decision_observations
`app/api/agents/paper-trade/route.ts`	When calling paper-fill: pass `mandate_id` from linked agent_signal
`execute_paper_fill` RPC (migration)	Accept `mandate_id` param; write to paper_trades
`lib/learning/dataset.ts` → `loadLabeledDataset()`	Add optional `mandate_id` filter to JOIN
`lib/validation/engine.ts`	Accept optional `mandate_id`; filters observations to mandate before walk-forward

```

---

### New file: `lib/evaluation/run-evaluation.ts`

Reuses all math from `lib/analytics/performance-metrics.ts`. No LLM. No weight mutation.

```

import {
 navToReturns, sharpe, sortino, maxDrawdown,
 expectancy, costNet, slip, calibration
} from "@lib/analytics/performance-metrics";
import type { SupabaseClient } from "@supabase/supabase-js";
import { createHash } from "crypto";

export async function runEvaluation(mandateId: string, market: string, supabase: SupabaseClient) {
 // 1. Fetch mandate (fail closed on error)
 const { data: mandate, error: mandateErr } = await supabase
 .from("investment_mandates").select("*").eq("id", mandateId).single();
 if (mandateErr || !mandate) return { ok: false, error: mandateErr?.message ?? "mandate not found" };

 // 2. Closed, non-tainted, evaluable trades for this mandate

```

```

// paper_trades closes via closed_at (no status column)
const { data: allTrades, error: tradesErr } = await supabase
 .from("paper_trades")
 .select("pnl_pct, spread_applied, realized_slip_pct, expected_price, fill_price, analyst_score, closed_at, tainted,
 excluded_from_learning")
 .eq("mandate_id", mandateId)
 .eq("market", market)
 .not("closed_at", "is", null);
if (tradesErr) return { ok: false, error: tradesErr.message };

const trades = allTrades ?? [];
const taintedCount = trades.filter(t => t.tainted).length;
const excludedCount = trades.filter(t => t.excluded_from_learning && !t.tainted).length;
const evaluable = trades.filter(t => !t.tainted && !t.excluded_from_learning);

// Dataset hash for idempotency detection (sort by closed_at so order doesn't matter)
const tradeIds = [...trades].sort((a, b) => a.closed_at < b.closed_at ? -1 : 1).map(t => t.closed_at);
const datasetHash = createHash("sha256").update(JSON.stringify(tradeIds)).digest("hex").slice(0, 16);
const windowStart = trades.length ? trades.reduce((a, b) => a.closed_at < b.closed_at ? a : b).closed_at?.slice(0, 10) : null;
const windowEnd = trades.length ? trades.reduce((a, b) => a.closed_at > b.closed_at ? a : b).closed_at?.slice(0, 10) : null;

// 3. NAV series for whole-book metrics (paper_performance has no mandate column)
const { data: navRows, error: navErr } = await supabase
 .from("paper_performance")
 .select("date, nav, alpha_pct, bench_return_pct")
 .eq("market", market).order("date");
if (navErr) return { ok: false, error: navErr.message };

// 4. Compute book metrics (reuse existing math — do NOT reimplement)
const returns = navToReturns((navRows ?? []).map(r => r.nav));
// expectancy() expects rows with pnl_pct field directly
const expM = expectancy(evaluable);
const sharpeM = sharpe(returns);
const sortM = sortino(returns);
const maxDDM = maxDrawdown(returns);
const costM = costNet(evaluable);
const slipM = slip(evaluable);
// calibration() expects { predicted: 0..1, win: boolean }
const calibM = calibration(
 evaluable.map(t => ({ predicted: Number(t.analyst_score ?? 50) / 100, win: (t.pnl_pct ?? 0) > 0 })),
);

```

```

const alphaPct = (navRows ?? []).length
? (navRows![navRows!.length - 1].alpha_pct ?? null)
: null;

// 5. P0 display health label (NOT a promotion gate)
const n = evaluable.length;
const MIN_EVAL = 20;
let healthLabel = "insufficient_sample";
let healthReason = Need ${MIN_EVAL} evaluable trades, have ${n};
if (n >= MIN_EVAL) {
 if (sharpeM.insufficient || sharpeM.value <= 0) {
 healthLabel = "negative_or_zero_edge";
 healthReason = Sharpe ${sharpeM.insufficient ? "N/A" : sharpeM.value.toFixed(2)} ≤ 0;
 } else if (sharpeM.value < 0.5) {
 healthLabel = "promising_but_unvalidated";
 healthReason = Sharpe ${sharpeM.value.toFixed(2)} < 0.5; run walk-forward validation;
 } else {
 healthLabel = "validation_required";
 healthReason = Sharpe ${sharpeM.value.toFixed(2)} ≥ 0.5; confirm with walk-forward;
 }
}

// promotion_eligible requires validation_experiments.passed=true — NOT set here in P0

// 6. Persist (append-only — never upsert)
const { error: insertErr } = await supabase.from("strategy_evaluations").insert({
 mandate_id: mandateId,
 mandate_snapshot: mandate, // snapshot mandate at evaluation time
 market,
 evaluator_version: process.env.VERCEL_GIT_COMMIT_SHA ?? "local",
 dataset_hash: datasetHash,
 window_start: windowStart,
 window_end: windowEnd,
 n_trades_total: trades.length,
 n_trades_evaluable: n,
 tainted_count: taintedCount,
 excluded_count: excludedCount,
 // Book metrics (whole-book NAV — not mandate-scoped Sharpe, document this)
 book_sharpe: sharpeM.insufficient ? null : sharpeM.value,
 book_sortino: sortM.insufficient ? null : sortM.value,

```

```

book_max_drawdown: maxDDM.insufficient ? null : maxDDM.value,
book_win_rate: expM.insufficient ? null : expM.winRate, // use winRate, not value
book_expectancy_pct: expM.insufficient ? null : expM.value,
book_alpha_pct: alphaPct,
book_benchmark_symbol: mandate.benchmark_symbol,
book_cost_adjusted_return_pct: costM.insufficient ? null : costM.value,
book_slip_vs_modeled_bps: slipM.insufficient ? null : slipM.value * 10000,
// Opportunity metrics: null until observation_labels mature (P1)
opp_n_labeled: null,
opp_hit_rate: null,
opp_benchmark_neutral_expectancy: null,
opp_ic: null,
opp_t_stat: null,
universe_pit_safe: false, // edge_universe_members is NOT PIT-safe
// Walk-forward: null until validation_experiments run referencing this mandate
validation_experiment_id: null,
walk_forward_folds: null,
health_label: healthLabel,
health_reason: healthReason,
promotion_eligible: false, // P0: never set to true here
});
if (insertErr) return { ok: false, error: insertErr.message };

return {
 ok: true,
 n: trades.length,
 n_evaluable: n,
 health_label: healthLabel,
 health_reason: healthReason,
 sharpe: sharpeM,
};
}

New routes

`POST /api/agents/evaluation/run` (owner-gated, no cron access)
// Body: { mandateId: string, market: 'us' | 'india' }
// Auth: requireOwner() — no cron secret accepted
// Calls: runEvaluation() → returns { ok, health_label, n, n_evaluable, sharpe }
// No live trading data exposed or mutated

```

```

`GET /api/agents/evaluation/results` (owner-gated)

// Query: ?mandateId=&market=&limit=20

// Returns: strategy_evaluations rows, latest first

// Does NOT expose: mandate_snapshot (strip to keep response lean)

UI additions to `PerformanceTruth.tsx` (additive only)

1. **Mandate selector** (top of card)
 - Dropdown from `investment_mandates` where `market = currentMarket AND active = true`
 - Default: `Swing US 2-20d` / `Swing India 2-20d`
 - Drives `?mandateId=` on existing `/api/agents/performance/metrics` call for trade metrics
 - Note label: "NAV/Sharpe is whole-book – trade metrics below are mandate-filtered"

2. **Evaluation history table** (below existing tiles)
 - Reads `GET /api/agents/evaluation/results?mandateId=&market=`
 - Columns: Date | Trades(eval/total) | Sharpe | MaxDD | Alpha | Health | WF Pass
 - "Run Evaluation" button → POST → spinner → refresh
 - Table is read-only. No edit/delete.
 - Show `universe_pit_safe` as a warning chip when false.

Pass/fail gate – P0 vs promotion

P0 (this feature): health_label display only. Four states:
- `insufficient_sample` – n_evaluable < 20
- `negative_or_zero_edge` – Sharpe ≤ 0
- `promising_but_unvalidated` – Sharpe 0-0.5
- `validation_required` – Sharpe ≥ 0.5 (requires walk-forward to become promotion-eligible)

Promotion gate (P1, wired at LearnerAgent): all of:
- `validation_experiments.passed = true`
- `p_improvement` threshold met
- ≥ 3/5 folds won
- `n_effective` ≥ min by horizon
- Benchmark-neutral expectancy > 0
- Max drawdown within mandate limit
- Tainted/evaluable ratio below threshold
- No unresolved data-quality warnings

`promotion_eligible` in strategy_evaluations is always `false` in P0. Only the
LearnerAgent promotion gate (after walk-forward) sets it via a new row insert.

Opportunity-level evaluation (current design gap – add to P1 scope)

P0 evaluates only closed paper trades (book truth). This is insufficient for
signal quality proof because trades are policy-selected.

The repo already has the infrastructure:
- `decision_observations` – all scored candidates (not just traded ones)
- `observation_labels` – matured forward returns
- `lib/learning/dataset.ts` – `loadLabeledDataset()` joins these
- `lib/validation/engine.ts` – walk-forward replay

P1 addition: populate `opp_` columns in strategy_evaluations:
- Filter `decision_observations` by `mandate_id` and horizon window
- Join to `observation_labels` (matured only)
- Compute `opp_ic`, `opp_t_stat`, `opp_hit_rate`, `opp_benchmark_neutral_expectancy`
- Mark `universe_pit_safe: true` only when a proper PIT universe snapshot exists

What NOT to do (hard constraints)

- No LLM in evaluation path – deterministic only
- Do NOT mutate `strategy_config`, money limits, or strategy weights from evaluation
- Do NOT make `mandate_id` NOT NULL until backfill verified in production
- Do NOT add `intraday` mandate – no intraday data or execution model exists
- Do NOT use `eligible_for_live_review` as a broker gateway signal – advisory label only
- Do NOT replace `validation_experiments` with `strategy_evaluations` – they serve different roles
- Do NOT call `expectancy()` with `{ returnPct }` – it expects `{ pnl_pct }`
- Do NOT call `calibration()` with `{ score, won }` – it expects `{ predicted (0-1), win }`
- Do NOT query `paper_trades` with `.eq("status", "closed")` – use `.not("closed_at", "is", null)`
- Do NOT call `requireOwnerSession()` – use `requireOwner()` from `lib/auth/require-owner.ts`
- Do NOT set `promotion_eligible = true` in P0 – reserved for P1 LearnerAgent gate

```

## P1 follow-ups (separate feature docs)

1. **Opportunity-level evaluation** – populate `opp\_\*` columns using `decision\_observations × observation\_labels` + `lib/validation/engine.ts`
2. **Data quality ledger** – `data\_quality\_log` table per (symbol, date, dimension) with source/freshness/confidence; feed into `data\_confidence`
3. **Conservative paper execution model** – stale quote guard, partial fills, "would\_not\_fill" outcomes, market-hours check per market
4. **ResearchDecision output shape** – extend agent\_signals to include `expected\_return\_bps`, `evidence\_quality`, `missing\_evidence[]`, `position`

---

## Acceptance criteria

- [ ] `investment\_mandates` seeded with Swing US + Swing India defaults
- [ ] All new `paper\_trades` inserts get `mandate\_id` from linked `agent\_signals`
- [ ] All new `agent\_signals` get `mandate\_id` set in `lib/research-agent.ts`
- [ ] All new `decision\_observations` get `mandate\_id` before insert
- [ ] `runEvaluation()` uses `.not("closed\_at", "is", null)` – not `status`
- [ ] `expectancy()` called with `pnl\_pct` field; `book\_win\_rate` stored from `expM.winRate`
- [ ] `calibration()` called with `{ predicted: score/100, win: pnl\_pct > 0 }`
- [ ] All Supabase errors return `{ ok: false, error: message }` – no silent failures
- [ ] `strategy\_evaluations` rows never updated/deleted (trigger enforced)
- [ ] `promotion\_eligible` is always `false` in P0
- [ ] `mandate\_snapshot` populated with mandate row JSON at evaluation time
- [ ] `dataset\_hash` populated so UI can detect duplicate/same-dataset reruns
- [ ] RLS enabled on `investment\_mandates` and `strategy\_evaluations` (deny by default)
- [ ] `eligible\_for\_live\_review` NOT read by any broker gateway or order placement code
- [ ] `/api/agents/evaluation/run` uses `requireOwner()` and returns 401 without session
- [ ] PerformanceTruth.tsx shows "NAV/Sharpe is whole-book" label + mandate selector + history table
- [ ] Existing metric tiles (Sharpe/Sortino/MaxDD/Calibration) continue working unchanged
- [ ] `lib/validation/engine.ts` – NOT changed in P0; `validation\_experiments` remains canonical

---

## Migration summary

| # | Table                 | Change                          | Risk                                     |
|---|-----------------------|---------------------------------|------------------------------------------|
| 1 | investment_mandates   | NEW + RLS                       | Low – no FK deps initially               |
| 2 | strategy_evaluations  | NEW + RLS + append-only trigger | Low – no existing code reads it          |
| 3 | agent_signals         | ADD mandate_id (nullable)       | Low – nullable, backfill after seed      |
| 4 | paper_trades          | ADD mandate_id (nullable)       | Low – nullable                           |
| 5 | decision_observations | ADD mandate_id (nullable)       | Low – trigger already blocks non-inserts |

All migrations additive. No existing columns modified. No existing behavior changed.

---

# Source: features\pipeline-data-and-timing\FEATURE\_ARCHITECTURE.md  
# Pipeline Data and Timing Remediation

Status: SHIPPED

Owner: ResearchAgent and scheduling control plane

Decision: PROJECT\_DECISIONS.md Decision 61

Markets: US and India, always isolated

## Implementation verification

- Completed-session filtering is applied after provider normalization and before scoring, return capture, evidence persistence, or trade-plan
- US paired seasonal crons are active in production and every command carries the expected market-local slot. Duplicate seasonal invocations are rejected
- India GDELT sentiment is unsupported by the active scorer. The replacement India news/event collector writes shadow-only intents that no decision engine reads
- Production migration `20260731210000\_market\_local\_crons\_and\_india\_news\_shadow.sql` is applied.
- Verification passed on 2026-07-31: 168 test files passed (1 skipped), 1,466 tests passed (7 skipped), TypeScript passed, and the Next.js prod build passed
- The first production India shadow canary processed 12 symbols, found relevant RSS headlines for 9, wrote 13 append-only provider-call rows and 13 shadow rows

## Problem

Three production facts require corrective work:

1. Intraday research can receive a provider's still-forming daily candle. That can make a daily technical score depend on an incomplete session
2. Fixed UTC US crons preserve UTC, not New York wall-clock time. After a daylight-saving transition, the close monitor can run before the US market
3. India news sentiment is nominally applicable but produced 0 available observations in the latest 310 production decisions. GDELT rejects the first 100

Historical replay remains diagnostic-only. Its accepted India run (111 dates, mean IC 0.0125, HAC t 1.32, unstable fold signs) does not authorize new decisions

## Decisions

### 1. Daily scoring uses completed sessions only

ResearchAgent filters normalized daily candles immediately before any score, return capture, trade-plan, or evidence calculation. A candle data row is not used until the session is complete

This boundary belongs after provider normalization, not inside a provider adapter, because quote/chart consumers may legitimately need an intraday candle

### 2. US schedules are market-local contracts

Each intended US local slot is scheduled at both possible UTC hours. The route admits only the invocation whose New York local `HH:mm` equals the slot

Only cron requests carrying `local\_slot` are checked. Owner/manual and closed-day catch-up invocations without the parameter preserve their execution

```
Flow	Local contract	EDT UTC	EST UTC
US research AM	09:00 ET	13:00	14:00
US paper entry AM	11:15 ET	15:15	16:15
US research PM	14:00 ET	18:00	19:00
US paper entry PM	15:15 ET	19:15	20:15
US position monitor	16:15 ET	20:15	21:15
```

### 3. India sentiment is removed from active applicability

Until a replacement is validated, India news sentiment is structurally unavailable rather than repeatedly requested and silently renormalized

### 4. Replacement evidence is shadow-only

A daily post-close India shadow collects official NSE corporate announcements when its cookie-gated public endpoint is reachable and bounded

It stores sanitized, source-labelled headline/event evidence in the existing `evidence\_cache\_v2` and provider-call lineage in `provider\_call`

The shadow prioritizes open paper holdings, then enabled watchlist symbols, and rotates the remaining bounded workload by least-recently-observed

The program can be promoted only through a separate decision after enough distinct market sessions demonstrate symbol relevance, freshness, and

## Data Contracts

Shadow evidence uses these canonical intents:

- `event.corporate\_announcement\_shadow`, provider `nse\_corporate\_announcements`
- `sentiment.news\_headlines\_shadow`, provider `google\_news\_rss`

Each `evidence\_cache\_v2` row binds market, symbol, provider, request fingerprint, payload hash, observed/as-of timestamps, expiry, quality score

No scorer, ResearchAgent, PaperTrader, PositionMonitor, TraderAgent, LearnerAgent, or broker route may read either shadow intent.

## Flow

flowchart LR

Cron["Market-local cron slot"] --> Guard["DST/local-slot guard"]

Guard -->|admitted| Research["ResearchAgent"]

Providers["Daily candle providers"] --> Complete["Completed-session filter"]

Complete --> Research

Research --> Signals["Market-local signals"]

Signals --> Paper["PaperTrader"]

Signals --> Monitor["PositionMonitor"]

IndiaCron["India post-close shadow"] --> NSE["NSE announcements"]

IndiaCron --> RSS["Google News RSS"]

NSE --> Cache["evidence\_cache\_v2"]

RSS --> Cache

Cache -. "no scoring reader" .-> Review["Upgrade Path review"]

## Safety Invariants

- US and India data are never cross-summed or cross-used.
- Partial current-session bars cannot reach daily scoring.
- A duplicate seasonal UTC invocation performs no provider, scoring, position, or order work.
- India shadow evidence cannot affect a score, threshold, signal, trade plan, position, cash balance, proposal, or broker order.
- Unknown news coverage remains unavailable; it is never converted to neutral sentiment.
- Provider text is untrusted, bounded, and non-executable.
- Historical evidence stays offline/diagnostic until a separately approved promotion gate passes.

## Acceptance Gates

- Unit tests cover US summer/winter and India local dates around regular close.
- Unit tests prove current-session candles are stripped before close and retained after close.
- Route tests prove the wrong seasonal UTC invocation exits before its worker runs.
- India `applicableDimensions` excludes sentiment and no active India research path calls GDELT.
- Shadow parser/relevance tests reject malformed, irrelevant, or oversized feed items.
- Upgrade Path shows India-only schedule, coverage, calls, blockers, and next action.
- Repository-wide tests, TypeScript, production build, migration verification, and production cron inspection pass.
- Architecture chapters, schedule reference, agent diagrams, system map, and work log are updated in the same change.



```

Non-Goals

- No signal weight or threshold change.
- No automatic historical-data ingestion into scoring.
- No LLM sentiment classifier.
- No options, broker, paper/live order, or position-management change.
- No claim that headline volume predicts direction.

Source: features\pit-fundamentals\FEATURE_ARCHITECTURE.md
Point-in-Time (PIT) Fundamentals – Feature Architecture

STATUS: DRAFT – awaiting owner approval
Owner: –
Author: architecture proposal (deep-audit P1 remediation)
<details>
<summary>Historical implementation notes (source-preserved)</summary>

Last updated: 2026-07-10

</details>

1. Problem statement

The research/scoring pipeline must use financial data as it was known on the decision date. Today it does not. A revenue/EPS figure that is later restated can retroactively change what a past decision "should" have seen, which corrupts both backtests and the learning loop. This is a named P1 gap in the deep-audit remediation.

Two distinct leaks exist, and the fix must address both:

- Leak A – vintage leak on any re-fetch/backfill. All fundamentals today are fetched as current TTM snapshots with no report date or filing date attached. If we ever re-score a past date, run a fundamentals-based backtest over history, or backfill, we pull today's (possibly restated) numbers and stamp them onto a past decision. There is no stored vintage to prevent this.
- Leak B – unversioned evidence behind a frozen score. The live score is frozen at decision time (see §3), but the fundamental inputs that produced it are stored as a loose JSON blob with no report-date / filing-date / vintage key. We cannot prove a stored score used as-known data, cannot detect when a later restatement changed the underlying TTM window, and cannot reconstruct the exact inputs for audit.

2. How fundamentals are fetched today (grounded in code)

2.1 Fetch path

- lib/research-agent.ts (processSymbol, ~L933-948) fetches all data in parallel. US fundamentals go through fetchUsOverview(symbol, avFallback) in lib/data/fundamentals.ts.
- fetchUsOverview calls fetchFmpOverview → FMP ratios-ttm + key-metrics-ttm (day-cached, own 250/day budget), falling back to Alpha Vantage OVERVIEW (fetchAVOverview, lib/research-agent.ts ~L734).
- India uses fetchIndiaOverview (Yahoo quoteSummary), see lib/india-data.ts.
- All three map into the same AV-OVERVIEW-shaped Record<string,string>:
 PERatio, ProfitMargin, ReturnOnEquityTTM, EPS, QuarterlyRevenueGrowthYOY, AnalystTargetPrice, Sector, etc.

Key fact: none of these fields carry a report period or filing date. ratios-ttm / key-metrics-ttm / AV OVERVIEW are all trailing-twelve-month, as-of-now rollups. AV OVERVIEW exposes only LatestQuarter (a single date on the whole object, not per-field). So the current fetch layer is structurally incapable of PIT – the date dimension is simply absent.

2.2 Scoring path

- lib/data/scores.ts → scoreFundamentals(overview, isEtf, currentPrice) (~L73-158) reads the OVERVIEW fields and produces a deterministic fundamental_score + an evidence object (pe_ratio, profit_margin, roe, eps, revenue_growth_yoy, analyst_upside_pct, sector, ...).
- computeScores (~L296) assembles the 5 sub-scores and fire-and-forget writes evidence_records rows via writeBatchEvidence (fundamental payload, source alpha-vantage, quality_state). No date-of-record on the fundamental payload.

2.3 Persistence path (where evidence lands)

- agent_signals.signal_breakdown (jsonb) – per-dimension evidence for "today".
- decision_observations.features (jsonb) – the learning fuel. Written at lib/research-agent.ts ~L1370-1427. Already stamps schemaVersion: "v1" and decisionTs (ISO timestamp), and spreads scores.evidence (incl. the fundamental evidence). Table is append-only,

```

```

immutable** (trigger blocks UPDATE/DELETE – see `docs/arch/04-database-schema.md`).
- **`evidence_records`** – immutable, `payload_hash` dedup.
- **`signal_score_history`** – append-only per-symbol scores.

```

### ### 2.4 Learning / backtest consumers

```

- `lib/learning/dataset.ts` → `loadLabeledDataset` joins
 `decision_observations` × `observation_labels` and returns the **stored** scores
 + `availability_mask` + forward-return labels. `walkForwardFolds` does
 purge/embargo splitting. It **replays stored scores**, it does not re-fetch
 fundamentals – so today's learner is accidentally PIT-safe *for the score
 number*, but blind to whether that number's inputs were later restated (Leak B).
- `lib/validators/backtest.ts` (Validation Engine) similarly replays stored
 observations. Any *new* fundamentals-based factor or a re-score/backfill that
 re-hydrates fundamentals from a live provider would introduce Leak A.

```

**Net finding:** the frozen-score design already prevents the crudest leak for the \*existing\* 5-dim replay. The exposure is (1) any future backfill / re-score / fundamentals-driven backtest, and (2) the absence of a vintage key that lets us audit or detect restatements. Both require capturing report-date + filing-date and versioning restatements – the subject of this proposal.

---

## ## 3. Design

### ### 3.1 (a) Capture fundamentals with report-date AND as-of/filing-date

Every fundamental fact gets **three** dates:

```

- `report_period` (a.k.a. fiscal period end / report date) – *what period the
 number describes* (e.g. `2026-03-31`, Q1 FY26).
- `filing_date` / `accepted_date` – *when that number became public* (the SEC
 acceptance timestamp, or provider's `acceptedDate`/`filingDate`).
- `captured_at` – *when Kairos first observed this value* (our own vintage clock).

```

The PIT rule for reads: a fact is **known on date D** iff  
 `filing\_date` ≤ D (fall back to `captured\_at` ≤ D when the provider gives no  
 filing date). Never `report\_period` ≤ D alone – a period can end weeks before it  
 is filed, and using period-end would leak the not-yet-published number.

### ### 3.2 (b) Storage model that versions restatements (describe only – no DDL here)

Proposed new tables (append-only, following the existing ledger convention):

**fundamental\_facts** – one immutable row per (symbol, metric window, vintage).

| Column            | Type       | Notes                                                                                 |
|-------------------|------------|---------------------------------------------------------------------------------------|
| `id`              | uuid PK    |                                                                                       |
| `symbol`          | text       |                                                                                       |
| `market`          | text       | `us` \ `india`                                                                        |
| `metric_set`      | text       | `ttm_overview` (matches today's OVERVIEW rollup) \ `quarterly` \ `annual`             |
| `report_period`   | date       | Fiscal period end the values describe                                                 |
| `fiscal_period`   | text       | `Q1`/`Q2`/`Q3`/`Q4`/`FY` (nullable)                                                   |
| `filing_date`     | date       | When it became public (nullable if provider omits)                                    |
| `values`          | jsonb      | The OVERVIEW-shaped field map (`PERatio`, `ProfitMargin`, ...)                        |
| `source`          | text       | `fmp` \ `alpha_vantage` \ `yahoo` \ `financialdatasets`                               |
| `restatement_seq` | int        | 0 = as-first-observed; 1,2... = later restatements of the SAME `report_period`        |
| `is_latest`       | bool       | Convenience flag: newest vintage for (symbol, report_period)                          |
| `payload_hash`    | text       | UNIQUE per (symbol, report_period, source, values-hash) – dedups identical re-fetches |
| `captured_at`     | timestampz | Our vintage clock                                                                     |

Restatement handling: a new fetch for an existing `(symbol, report\_period)`  
 whose `values` differ from the newest stored vintage inserts a **new row** with  
 `restatement\_seq` = prev+1 and flips the old row's `is\_latest=false`. Nothing is  
 ever mutated in place – the original as-first-reported row survives forever. This  
 is what makes "a later-restated revenue must not retroactively change a past  
 score" enforceable.

**fundamental\_fetch\_log** (optional, ops) – every provider hit with  
 throttle/shape outcome, so a restatement can be traced to a specific fetch.

Also add, on `decision\_observations.features` (no migration – it's jsonb):

```

- `fundamentals_vintage`: { fact_id, report_period, filing_date, restatement_seq, captured_at }
 so each stored decision points at the exact immutable vintage it consumed.

```

### ### 3.3 (c) Read API – "fundamentals as known on date D"

New module `lib/data/fundamentals-pit.ts`:

```
getFundamentalsAsOf(symbol, market, asOf: Date, opts?): Promise<{
```

```
overview: Record<string,string>; // same shape scoreFundamentals reads today
```

```
vintage: { fact_id, report_period, filing_date, restatement_seq, captured_at } | null;
```

source: string;

>

Query: newest `fundamental\_facts` row for `symbol` where  
`COALESCE(filing\_date, captured\_at) <= asOf`, ordered by that date desc then  
`restatement\_seq` \*\*asc-at-or-before-asOf\*\* (i.e. pick the vintage that was the  
latest \*known\* one on `asOf`, not a future restatement). Returns the OVERVIEW map  
untouched, so \*\*`scoreFundamentals` needs zero changes\*\* – it keeps reading a  
`Record<string,string>`.

- \*\*Live path\*\* (`asOf = now`): `getFundamentalsAsOf(symbol, market, new Date())`  
simply returns the newest known vintage – behaviourally identical to today, but  
now it also writes/reads the vintage record.
- The existing `fetchUsOverview` becomes the \*\*capture writer\*\*: on each live  
fetch it upserts into `fundamental\_facts` (append vintage on change), then  
hands the same overview to the caller. This is the single ingestion point.

### 3.4 (d) ResearchAgent + learning dataset consume it consistently

- \*\*ResearchAgent (live)\*\*: replace the raw `fetchUsOverview(...).overview` at  
`lib/research-agent.ts` ~L948 with `getFundamentalsAsOf(symbol, market, now)`,  
and record `features.fundamentals\_vintage`. Capture-on-fetch means live scoring  
builds the PIT archive as a side effect.
- \*\*Learning dataset\*\* (`lib/learning/dataset.ts`): \*\*unchanged for the \*existing\*  
5-dim replay (it already reads frozen scores – keep it that way). It gains one  
capability: when a future factor or the Validation Engine needs to \*recompute\* a  
fundamental at a historical decision, it MUST call  
`getFundamentalsAsOf(symbol, market, observation.ts)` – never a live fetch.  
Add a lint/guard note in `lib/validators/backtest.ts` that fundamentals in  
replay come only through the PIT read API. This is what makes backtest and live  
scoring agree: both resolve fundamentals through one date-parameterised function.
- \*\*Restatement-drift detection (Leak B)\*\*: a periodic job can compare  
`decision\_observations.features.fundamentals\_vintage.fact\_id` against the  
current `is\_latest` vintage for that `report\_period`; a mismatch flags the  
observation as \*restated-after-decision\* (surfaced to the learner as a taint  
signal, not a mutation). The stored score is never changed.

### 3.5 (e) Backfill strategy for existing history

We \*\*cannot\*\* reconstruct as-first-reported values for the past – no current  
provider serves original-vs-restated history (see §4). Honest plan:

1. \*\*Forward-only PIT from go-live.\*\* Start capturing vintages now; the archive  
becomes correct from day one and compounds. Mark all pre-go-live decisions  
`pit\_status = "pre\_archive"` in features (they keep their frozen scores).
2. \*\*Best-effort seed\*\* for actively-traded symbols: one snapshot per current  
`report\_period` using FMP `income-statement` / `financial-reports-dates`  
which \*do\* expose `date` / `fillingDate` / `acceptedDate` – stamped  
`restatement\_seq = 0`, `source\_note = "seed\_latest\_restated"` so it is never  
mistaken for a true first-report vintage.
3. \*\*Never rewrite\*\* existing `decision\_observations` (append-only trigger forbids  
it anyway). Backfill only populates the new `fundamental\_facts` table.

---

## 4. Data-source capability (does the provider expose filing dates / restated?)

| Provider (role today)                                                                                                   | Report date                        | Filing date     | Original-vs-restated    |
|-------------------------------------------------------------------------------------------------------------------------|------------------------------------|-----------------|-------------------------|
| --- --- --- ---                                                                                                         |                                    |                 |                         |
| FMP `ratios-ttm` / `key-metrics-ttm` (**current US fundamentals**, `lib/data/fundamentals.ts`)                          | ■ (TTM rollup)                     | ■               | ■                       |
| FMP `income-statement` / `financial-reports-dates` (not used yet)                                                       | ■ `date` / `calendarYear`+`period` | ■ `fillingDate` | ■ `acceptedDate`        |
| Alpha Vantage `OVERVIEW` (fallback)   ~ `LatestQuarter` only                                                            | ■                                  | ■               |                         |
| FinancialDatasets `get_financial_metrics` / `get_income_statement` (MCP, referenced in a doctrine prompt but not wired) | ■                                  |                 | ■ `report_period`       |
| Yahoo `quoteSummary` (India)   partial                                                                                  | ■                                  | ■               |                         |
| Specialized PIT vendors (Compustat Point-in-Time, S&P)                                                                  | ■                                  | ■               | ■ (out of budget/scope) |

**Conclusion:** no in-budget provider gives true as-first-reported history. The  
only way Kairos gets real PIT is to **become its own point-in-time archive** by  
snapshotting filing-date-stamped fundamentals on every fetch going forward, and  
appending a new vintage row whenever values for a `report\_period` change. Filing  
dates ARE obtainable (FMP `income-statement.acceptedDate`, FinancialDatasets  
`report\_period`), so the capture side is feasible without a new paid vendor.

---

## 5. Phased build plan + effort

| Phase                 | Scope                                                                                                                      | Effort |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------|--------|
| --- --- ---           |                                                                                                                            |        |
| **P0 – Schema**       | `fundamental_facts` (+ optional `fundamental_fetch_log`) migration; verify applied via `list_migrations` before code ship  |        |
| **P1 – Capture**      | Make `fetchUsOverview` / `fetchIndiaOverview` upsert vintages (append-on-change, filing-date from FMP `income-statement`)  |        |
| **P2 – Read API**     | `lib/data/fundamentals-pit.ts` `getFundamentalsAsOf`; unit tests against a hand-built multi-vintage fixture (mirrors `w`)  |        |
| **P3 – Wire live**    | ResearchAgent consumes `getFundamentalsAsOf(now)`; stamp `features.fundamentals_vintage`. No change to `scoreFundamentals` |        |
| **P4 – Replay guard** | Route all replay/backtest fundamentals through the PIT API; add restatement-drift detector + a `pit_restated_after_`       |        |

| **\*\*P5 – Backfill\*\*** | Forward-only default + best-effort seed for the active universe; `pre\_archive` tagging. | ~1 day |

Total ≈ **\*\*6 engineer-days\*\***, shippable phase-by-phase behind the existing append-only-ledger conventions. Docs to touch on ship:  
`docs/arch/04-database-schema.md` (P0), `docs/arch/02-tech-stack.md` (new FMP endpoint / capture note), `docs/arch/09-learning-loop.md` (P4), and  
`public/agent-diagrams/system-map.json` if the ResearchAgent→learner data flow node descriptions change.

---

#### ## 6. Effect on existing scores

- **\*\*No existing score changes.\*\*** `decision\_observations` is append-only; frozen scores stay frozen. Live scoring output is numerically identical on go-live (same provider, same fields) – we only **\*add\*** a vintage record alongside.
- The learner gains a new **\*observed\*** taint feature (restated-after-decision); per CLAUDE.md's pushback mandate it is **\*\*logged evidence, not a new weighted dimension\*\*** until it earns an IC track record.

---

#### ## 7. Risks / open questions

1. **\*\*Filing date availability & cost.\*\*** FMP `income-statement` gives `acceptedDate`, but it is a separate call – does the 250/day FMP budget absorb one more day-cached call per symbol? (Likely yes given day-cache; verify.)
2. **\*\*India filing dates.\*\*** Yahoo `quoteSummary` does not expose filing dates cleanly; India may have to fall back to `captured\_at` as the as-of clock (still correct forward, just coarser). Acceptable?
3. **\*\*TTM windows don't have a single filing date.\*\*** A TTM rollup blends 4 quarters; the honest `filing\_date` for a TTM `metric\_set` is the filing date of its **\*most recent\*** constituent quarter. Confirm that convention.
4. **\*\*Restatement detection sensitivity.\*\*** Provider re-computations / rounding can flip a value trivially. Need a tolerance threshold before appending a new `restatement\_seq` vintage to avoid vintage churn.
5. **\*\*Should the seed backfill exist at all?\*** It stamps latest-restated values as `restatement\_seq=0`, which is **\*not\*** truly first-reported – risk of it being mistaken for real PIT. Option: forward-only, skip seed entirely.
6. **\*\*`get\_financial\_metrics` (FinancialDatasets MCP) wiring.\*\*** It exposes `report\_period` cleanly and could become the canonical PIT fundamentals source instead of FMP TTM. Bigger change; out of this proposal's default scope but noted as the strongest long-term option.

---

#### ## 8. What this does NOT do

- Does **\*\*not\*\*** reconstruct as-first-reported fundamentals for history predating go-live – no in-budget provider offers that.
- Does **\*\*not\*\*** change `scoreFundamentals` math, the 5-dim weights, or any score value already stored.
- Does **\*\*not\*\*** add a new weighted scoring dimension (restatement taint is logged evidence only, per pushback mandate).
- Does **\*\*not\*\*** re-fetch or mutate `decision\_observations` / `evidence\_records` (append-only ledgers stay immutable).
- Does **\*\*not\*\*** purchase a specialized PIT data vendor (Compustat/S&P) – explicitly out of scope; flagged as the only path to true pre-history original-vs-restated.
- Does **\*\*not\*\*** cover point-in-time **\*prices\*** or **\*index membership\*** survivorship – separate audit items.

---

# Source: features\policy-event-ledger\FEATURE\_ARCHITECTURE.md  
# US Policy Event Ledger

#### ## Decision

Build a US-only, record-only FOMC ledger so Kairos can distinguish a scheduled Fed decision, the pre-decision market expectation, the realized target range, and later observed symbol/benchmark returns. The existing MacroSentinel remains a slow US macro regime. This feature is not a score, trade, sizing, or exit input.

#### ## Sources And Boundaries

- **\*\*Official outcome:\*\*** FRED `DFEDTARL` / `DFEDTARU`, whose source is the Federal Reserve and whose observations are the effective target range. A scheduled event is marked decided only after both ranges are observed after its date.
- **\*\*Expectation:\*\*** immutable snapshots are supported, but no source is enabled. CME's supported FedWatch API is paid; do not scrape the public webpage or fabricate probabilities. A future licensed adapter writes only pre-decision snapshots with source URL, source name, and capture time.
- **\*\*Impact:\*\*** compound only rows already present in `symbol\_daily\_returns`. No event route fetches quotes or candles, so it cannot consume research API quota or cause a late provider revision to masquerade as a decision-time fact.

- **\*\*Scope:\*\*** US FOMC only. India receives no Fed score, event, or inferred RBI substitute. A separate RBI design is required later.

#### ## Data Model

1. `policy\_rate\_events`: one mutable schedule/outcome row per FOMC date.
2. `policy\_rate\_expectation\_snapshots`: append-only pre-event expected ranges.
3. `policy\_event\_impacts`: append-only 1D/5D raw and SPY-relative returns.

An impact row includes an input fingerprint and its available time. It can be recomputed when more frozen daily-return data exists, but it never overwrites a previous calculation. Raw symbol returns may be recorded without a benchmark; SPY-relative excess stays null until identical frozen benchmark sessions exist.

#### ## Operation

`POST /api/agents/policy-events` is a daily server-only job. It upserts the official calendar, reads FRED target ranges, resolves past scheduled outcomes, and derives impacts from existing US daily-return evidence. It does not invoke ResearchAgent or an LLM. The read API powers the Markets card.

#### ## Acceptance Criteria

- A public/anonymous client cannot read or write ledger tables.
- An expectation after the decision is rejected; a missing expectation renders as unavailable, never zero or "hold expected."
- FRED failures preserve existing records and produce no invented decision.
- Event impacts require complete 1D/5D sessions and calculate excess only when both symbol and SPY returns are comparably available.
- No scorer, trader, PaperTrader, PositionMonitor, or broker adapter imports the ledger.

---

# Source: features\portfolio-simulation\FEATURE\_ARCHITECTURE.md  
# Deterministic Portfolio Simulation

Status: Approved by owner for isolated implementation on 2026-08-10

#### ## Decision

Kairos will add a deterministic, market-local portfolio simulator for evaluating research and exit hypotheses. It is an analytical engine only: it cannot read or write live/paper portfolio state, scores, strategies, signals, proposals, orders, or broker records.

#### ## Why this exists

The existing historical replay and exit-path shadows correctly measure individual decision paths, but they cannot rank rules with different holding periods. A rule that exits earlier frees cash sooner; a per-trade comparison cannot observe the subsequent redeployment of that cash. The simulator closes that measurement gap.

#### ## Invariants

1. One run has exactly one market and native currency: `us/USD` or `india/INR`.
2. Every input event carries an execution session and a known-at timestamp. The caller is responsible for supplying only point-in-time-safe inputs.
3. Cash, fills, costs, position caps, fractional-share policy, and benchmark are explicit inputs. No hidden global configuration is read.
4. A rejected, unavailable, malformed, stale, or out-of-order event is an abstention recorded in the result, never a fabricated fill.
5. Equal timestamp events are ordered deterministically: exits, then entries, then lexical symbol/order key. This permits same-session redeployment without double-spending cash.
6. The simulator applies only deterministic input instructions. It does not calculate a score, choose an instrument, invoke an LLM, or generate an order.
7. Results are immutable experiment artifacts when persistence is later added; the initial core is pure TypeScript with fixture tests.

#### ## Scope

P0 builds a pure simulation core and fixtures. It supports long-only entry and exit events, entry costs, exit costs, explicit quantity or cash allocation, market-local cash, whole/fractional share rules, an open-name cap, and daily NAV. It returns every accepted/rejected event, cash history, holdings, realized P&L, and a run fingerprint supplied by the caller.

P0 intentionally excludes live data acquisition, intraday assumptions, tax-lot selection, borrowing, options, leverage, automatic strategy selection, and database persistence. Those require separate data/execution architectures.

#### ## Integration boundary

The first consumer will be a future owner-triggered historical experiment using

sealed input packets. Exit shadows, challenger validation, and benchmark analysis may consume results only as diagnostics. A separate approved evaluation must still decide whether a rule is useful; a simulation result cannot change any policy.

### ## Acceptance criteria

- US and India runs cannot be combined or cross-funded.
- An exit can release capital for a later same-session entry exactly once.
- Cash never becomes negative; every rejection has a reason.
- Fractional quantity is rejected when the market policy disallows it.
- Costs and realized P&L reconcile from immutable simulated fills.
- Re-running identical inputs produces byte-identical results.
- No import path from simulator reaches a provider, Supabase, scorer, agent, PaperTrader, live execution gateway, or broker adapter.

---

# Source: features\portfolio-sizing-replay\FEATURE\_ARCHITECTURE.md  
# Historical sizing comparison

Status: owner-approved diagnostic comparison, 2026-09-20.

Why: separate position sizing from selection and partial-exit effects.  
Who: owner evaluating US and India portfolios independently.  
Expected benefit: auditable capital-allocation evidence; no promised return uplift.  
Implementation verified locally: 2026-09-21 (17 focused tests; TypeScript clean).  
Historical portfolio comparison: not yet available; see DATA\_READINESS.md.

### ## First comparison

Implement a pure, network-free replay with one chronological evidence tape shared by two arms: equal allocation per new entry versus equal planned stop risk. Initial diagnostic parameters are explicit inputs, not trading defaults. Do not optimize them on the outcomes. Replay realized entry opportunities and freeze their exit timestamps, prices and fractions of the original lot. This conditions on what the app actually bought; it cannot assess missed opportunities or claim a globally optimal portfolio. Group partial exits under the original entry ID.

Both arms start with identical cash, no positions and the same market/currency. NAV includes cash and contemporaneously available marks. Explicit mark events must precede sizing; stale/missing marks invalidate the paired run. Entries have fixed name, sector and gross exposure caps. Cash may never become negative. US quantities use six decimals, India quantities use whole shares. Exit fractions apply to the original simulated quantity, with the final exit clearing rounding residue. Do not fabricate a liquidation for positions still open at the cutoff.

The plan explicitly declares execution-cost basis points and maximum mark age. Historical prices that already include spread must not have that spread charged again. Validate cost provenance before interpreting net returns. NAV drawdown is measured only at supplied observations and must be labeled sampled drawdown.

### ## Required evidence

- Complete, untainted chronological tape, immutable event IDs and ordering.
- Original stop with evidence timestamp no later than entry; a later trailing stop or a closed lot's captured exit stop is not an entry stop.
- Positive finite fill prices, original quantities and sector-at-entry.
- Marks available at each decision and at the common end timestamp.
- Explicit initial cash, costs and common cutoff; deposits must be represented in a future cash-flow-aware extension, not mistaken for performance.

Missing or invalid evidence returns `invalid` with no arm returns. Never silently drop bad episodes or backfill missing stops from current positions. Historical snapshots and production ledgers remain untouched. A local CLI consumes an operator-reviewed JSON fixture; it does not read credentials or call a broker.

### ## Reporting boundary

Report final NAV, return, cash, gross traded notional, sampled maximum drawdown, and every accepted/skipped entry. Hash the whole tape and plan. Differences are conditional diagnostic comparisons, not causal proof, significance, or Upgrade Path promotion evidence. There is no database writer or trading consumer.

### ## Separate top-up experiment (pending)

Do not add top-ups to the first comparison. It requires fresh historical research for held and unheld alternatives, an independently frozen ranking rule, portfolio constraints and an explicit residual exit policy for newly added lots. Cash is capacity, not an entry signal. Today's known winners must never choose yesterday's top-ups. Production anti-pyramiding remains unchanged.

### ## Acceptance

Tests cover finite cash, concurrent positions, proportional partial exits,

whole/fractional rounding, loss amplification, future-dated stops, stale marks, invalid exits and identical deterministic runs. Real-data readiness is reported separately from synthetic test success. No weights, broker caps, or live switches are changed.

---

# Source: features\property-address-and-carrying-costs\FEATURE\_ARCHITECTURE.md  
# Property Address And Carrying Costs

Status (2026-08-08): **\*\*Implemented.\*\*** Address and current carrying-cost fields use the existing encrypted `property\_assets.encrypted\_payload`.

Source capability decision (2026-08-08): see  
[`SOURCE\_CAPABILITY\_AUDIT.md`](SOURCE\_CAPABILITY\_AUDIT.md) before adding any parcel, tax, insurance, or property-value automation. In particular, the Phoenix sales-feed licence remains unverified and no search page, payment page, or private portal may be automated.

## Decision

Property uses one workspace-wide market context: `US - Austin`, `US - Phoenix`, or `India - Bengaluru`. Every market-aware page inherits it.

There are three deliberately separate location contracts:

1. **\*\*Explore a market:\*\*** ZIP for the US, PIN/locality for Bengaluru. No county or parcel identifier is required from the owner.
2. **\*\*Link an owned property:\*\*** optional exact address, encrypted at rest and never exposed to an LLM. The owner-gated server may resolve non-  
USPS parcels.
3. **\*\*Link parcel evidence:\*\*** background-only and source-specific. A parcel identifier is never an owner-facing navigation concept and no parcel  
evidence is required.

## Address Flow

flowchart LR

O["Owner enters address"] --> A["Owner-gated assets API"]

A --> G["Official geography resolver"]

G --> E["Resolution state and geography"]

A --> V["AES-256-GCM property vault"]

E --> V

V --> U["Owner-only property record"]

For US addresses the resolver is the Census Geocoding Services API. It can return standardized address, ZIP, county name, and county GEOID. In

Bengaluru stores owner-entered address/locality/PIN encrypted but has no automated parcel resolver in this phase.

## Address Verification – Is This A Real Address? (2026-09-08)

Geography resolution and address *\*existence\** are two different questions and are stored as two different fields. Census answers "which county". USPS answers "is this deliverable". They can disagree; both are kept.

`lib/property/address-verify.ts` returns exactly one of five states, persisted with the property as `details.addressVerification` inside the same AES-256-GCM encrypted payload (no new column, no migration) together with `checkedAt` so a stale check is visible:

| State          | Meaning                                                               | Owner-facing consequence                                       |
|----------------|-----------------------------------------------------------------------|----------------------------------------------------------------|
| verified       | USPS returned a standardized deliverable address                      | Only state shown in accent colour; standardized form displayed |
| not_found      | USPS was reached and found no such address                            | Flagged amber; the property still saves                        |
| not_configured | No `USPS_CONSUMER_KEY`/`USPS_CONSUMER_SECRET`                         | "Address verification is not configured" + exact enable steps  |
| no_validator   | India – outside USPS coverage, nothing free wired                     | "Unverified – no free validator for this market"               |
| unavailable    | USPS configured but erroring, timing out, or rejecting the credential | Flagged amber; opens System Health `property-address`          |

**\*\*Fail closed.\*\*** A USPS `200` that carries no standardized street *\*and\** ZIP is `not\_found`, not a pass – USPS echoes input on partial matches and treating that as verified would defeat the feature. Nothing but a real USPS match ever renders as verified.

**\*\*Flag, do not forbid.\*\*** An unverified address never blocks a save. The owner may know a property the postal database does not; the record simply carries an honest unverified flag.

Two entry points: `POST /api/property/address-verify` is an optional owner-gated pre-save preview that writes nothing, and `/api/property/assets` re-verifies on every save so the persisted flag can never drift from the persisted address. The address itself is never logged, echoed in a URL path, or sent to an LLM; only the verification state travels.

Non-goals: no Indian address validator (deferred until a free, licence-clean one exists), no parcel identity, no address autocomplete, and no bulk re-verification

sweep of existing records — a record is re-checked when it is next saved.

## ## Carrying-Cost Contract

The monthly total is deterministic:

``principal and interest + annual tax / 12 + annual insurance / 12 + annual maintenance / 12 + HOA + other monthly costs``

Mortgage principal and interest reuses ``calculateMortgage()`` from the shared property scenario engine. Tax, insurance, and maintenance are owned by the owner.

## ## Charts

Historical market charts use: ``1M`, `6M`, `YTD`, `1Y`, `5Y`, `10Y`, `20Y`, `All``. A range with no official observations renders an honest empty chart.

## ## Safety And Acceptance

- Market selection persists locally and is shared by Markets, My Properties, Opportunities, Financing, Forecasts, and Valuation Evidence.
- Switching markets never converts or cross-sums currencies.
- API payload size, text length, and numeric ranges are bounded before encryption.
- Exact address and costs remain inside the existing owner-only encrypted vault.
- A geocoder outage does not prevent saving the private record and does not fabricate resolution.
- Austin no longer asks the owner for a TCAD property ID; Phoenix bulk collection remains ZIP-scoped.
- Property workflows remain isolated from investing scores, agents, orders, and strategy promotion.

## ## Editable Records And Owner History (2026-08-08)

The current ``property_assets`` row is editable owner state. Every create or save also writes one encrypted, append-only ``property_asset_history`` snapshot through an atomic service-role RPC. The history contains the valuation, loan, and carrying-cost inputs needed to reproduce the chart point, but deliberately does not duplicate the exact address.

An owner can choose the snapshot's effective date, including a prior date, but cannot select a future date. The chart renders the recorded value, derived equity, and derived monthly carrying cost in the asset's own currency. It never cross-sums USD and INR.

Records are archived rather than deleted. The history foreign key uses ``ON DELETE RESTRICT``, history mutations and truncation are rejected by database triggers, and table/function access is service-role-only. Existing records begin their history at their next save because their current encrypted payload cannot be truthfully backfilled by SQL.

## ## Official References

- US Census Geocoding Services API: [https://geocoding.geo.census.gov/geocoder/Geocoding\\_Services\\_API.html](https://geocoding.geo.census.gov/geocoder/Geocoding_Services_API.html)
  - Maricopa Assessor address and parcel search: <https://www.mcassessor.maricopa.gov/page/home/help/searching/>
  - Travis Central Appraisal District property search: <https://traviscad.org/propertysearch/>
- # HUD rental-reference adapter (2026-08-08)

## ## Decision

Kairos may collect the official HUD Fair Market Rent (FMR) response for the **Austin** and **Phoenix** HUD metro areas only, after an operator supplies a free HUD User API token as the server-only ``HUD_FMR_API_TOKEN`` environment variable and explicitly changes the ``hud-fmr`` source record to ``active``.

The collector records one annual bedroom-specific ``rent_reference_*`` observation for each HUD published bedroom category (studio through four-bedroom), source key ``hud-fmr``, FMR-year source version, and the FMR-year effective date. The browser never receives the token. If the source remains pending or the token is absent, the workspace states that the source is unavailable; it does not render a zero, reuse an old value as current, or fabricate a rent.

## ## Boundaries

- This is an **area-level affordability reference**, not a property rent estimate, comparable, appraisal, valuation input, forecast, underwriting input, portfolio recommendation, or rental-price target.
- The adapter accepts no address, ZIP, parcel, owner record, or bedroom input. Even if HUD returns a Small Area FMR array, it deliberately chooses only the documented ``MSA level`` row. This prevents an official affordability table from becoming a false claim about a particular property.
- Bengaluru is structurally ``not_applicable``; no US rent reference is copied into the India workspace.
- A missing token is a typed ``hud_fmr_token_unconfigured`` unavailable state, recorded in the append-only source-run ledger as ``partial``, not a collection failure and not a silent no-data state.

## ## Official verification

HUD's FMR API documentation specifies the ``Authorization: Bearer`` token flow, the ``/fmr/data/{entityid}`` endpoint, FMR-year response, and bedroom fields. HUD requires an account-created access token, so the adapter is server-only and starts ``contract_pending``; there is no private browser key. Source:



<https://www.huduser.gov/portal/dataset/fmr-api.html>

HUD's API terms permit retrieval, display, and analysis of its public data and require the displayed HUD User attribution. The Property Market Explorer renders that exact attribution beside the source references. The terms also set a 60-queries-per-minute maximum; annual, two-metro collection is materially below that limit. Source: <https://www.huduser.gov/portal/dataset/api-terms-of-service.html>

## ## Implementation result

Implemented on 2026-08-08 in `lib/property/sources.ts` and the bounded Property collection route. `tests/property-ingestion.test.ts` covers annual normalization, the MSA-only SAFMR-array rule, token absence, and Bengaluru exclusion. TypeScript and focused tests pass. The adapter has no schema migration because `property\_market\_observations.metric\_key` is a governed text contract; the new metric is documented in this feature record.

---

# Source: features\property-address-and-carrying-costs\IMPLEMENTATION\_RESULT.md  
# Implementation Result

Implemented on 2026-08-08:

- Workspace-wide Austin, Phoenix, and Bengaluru market context.
- Historical chart windows from 1M through All.
- Optional encrypted exact-address fields in My Properties.
- Official US Census geography resolution with explicit failure states.
- Deterministic monthly carrying-cost breakdown using the shared mortgage engine.
- Owner-entered property-tax, insurance, maintenance, HOA, and other-cost inputs.
- Removal of owner-facing Austin parcel-ID collection.
- Unit coverage for cost arithmetic, chart cutoffs, and Census response parsing.
- Editable records with an effective-date, encrypted value/cost snapshot on every save.
- Archive-only removal and an owner-recorded value, equity, and carrying-cost chart per property.
- Encrypted owner tax-notice and insurance-policy/quote evidence at `/property/imports`; metadata is listable but document text is never returned.
- Explicit `Incomplete` monthly cost when no cost inputs were recorded; missing data is never represented as zero.
- Owner-only overview truth surface: active private-record count and non-sensitive market/type status are fetched live without selecting encryption.

Not claimed: parcel resolution, AVM, property-specific insurance quote, automatic tax bill lookup, or Bengaluru parcel data. The source capabilities are limited.

---

# Source: features\property-decision-workspace\FEATURE\_ARCHITECTURE.md  
# Kairos Property Decision Workspace

Status (2026-08-07): \*\*P0-P4 and valuation-evidence Stage 1 schema applied to production and verified.\*\*  
Collection is live for the US markets (FHFA HPI, FRED MORTGAGE30US, BLS LAUS) on a weekly cron; Opportunities, Financing and Forecasts persist through owner-gated encrypted APIs. The encryption master is configured consistently in Vercel and GitHub Actions. Still NOT done: authenticated visual verification, production route smoke tests, and every India source. Valuation Stage 1 is evidence-only; no AVM, lender integration, or automated transaction is authorized or enabled. See `IMPLEMENTATION\_RESULT.md` for the verified findings list and the honest deferral list.

Update (2026-08-08): the Property shell now has one persistent market context ('US - Austin', 'US - Phoenix', 'India - Bengaluru') inherited by market-aware pages. My Properties accepts an optional exact address inside the existing encrypted vault, resolves US geography through Census without claiming a parcel match, and calculates source-labelled monthly carrying costs. Historical market charts use 1M/6M/YTD/1Y/5Y/10Y/20Y/All. See `features/property-address-and-carrying-costs/FEATURE\_ARCHITECTURE.md`.

Initial market packs: Austin Central Texas, Phoenix Metro Arizona, and Bengaluru Karnataka. They are independent markets, never a blended property universe.

Update (2026-09-08): Amortization & Refi (Stage 1) adds a pure-math page that rebuilds a loan's full amortization schedule (`lib/property/amortization.ts`, wrapping the existing `buildAmortizationSchedule`/`evaluateRefinance` in `lib/property/scenarios.ts` rather than a second implementation) from owner-entered original principal, rate, term, and start date, splits it at today into paid-to-date and remaining, and compares the current rate against today's FRED 30-yr average for a refinance breakeven. "Today's rate" reads the `mortgage\_rate` observation the existing `GET /api/property/overview` already returns – no second FRED client, no new API route, and no `property\_scenarios` row is written; everything computes client-side and nothing is persisted. Closing costs are a required, editable input defaulted to 2% of the current balance, never a hidden constant. US-only, matching `FredMortgageAdapter` coverage: Bengaluru gets the amortization split but not the refinance comparison, with an explicit reason shown rather than a blank section.

## ## Product Boundary

Kairos Property is a top-level workspace for personal property, market, and financing decision support. It shares Kairos ownership, authentication, source

provenance, system health, and forecast-calibration discipline. It has separate tables, APIs, schedules, privacy controls, and decision logic from Investing.

It answers four auditable questions:

1. Is a market improving, cooling, or uncertain relative to its own history?
2. Does buying, selling, holding, renting, refinancing, or using a HELOC make sense under stated assumptions and downside cases?
3. Which user-selected locations meet a declared screening rule?
4. How accurate have earlier Kairos forecasts been by market and horizon?

It never claims to know the exact right time, an exact property price, or lender approval.

## Workspace UX

The persistent application header contains:

`Kairos | Investing | Property`

The selected workspace changes the entire left navigation. Investing retains its US/India market selector. Property has a distinct location context such as `United States / Austin Metro` or `India / Bengaluru`.

Property navigation:

```
Area	Purpose
Overview	Market pulse, source freshness, alerts, tracked decisions.
Market Explorer	Country, state, metro, ZIP, locality, and type trends.
My Properties	Owner-entered homes, land, and rentals.
Opportunities	Buy, sell, rent, and land scenarios.
Financing	Mortgage, refinance, HELOC, and loan-against-property scenarios.
Amortization & Refi	Full amortization schedule split into paid-to-date and remaining; refinance breakeven vs. today's FRED 30-yr rate.
Forecasts and Learning	Forecasts, actuals, calibration, and confidence.
Data Sources	Source definitions, freshness, coverage, licenses, and health.
```

`Opportunities` is a private decision notebook, never a broker-connected listing or purchase workflow. The Property workspace must not import trading agents, paper positions, broker clients, portfolio cash, strategy champions, or stock scores.

## System Context

flowchart LR

U["Owner"] --> W["Kairos Property Workspace"]

W --> V["Private Property Vault"]

W --> M["Market Evidence Service"]

W --> C["Deterministic Scenario Engine"]

W --> F["Forecast And Calibration Ledger"]

M --> O["Approved Data Sources"]

C --> V

F --> M

F --> C

LLMs may summarize completed, cited scenario outputs. They may not see full addresses or loan documents, calculate financial outputs, select a data source, make an unlogged forecast, or trigger a transaction.

## Market Packs

### Austin Central Texas

Scope: Austin Metro, Travis County, and an owner-managed ZIP watchlist.

- FHFA HPI: repeat-sales price trend at available metro/county/ZIP levels.
- Redfin Data Center: inventory, new/pending listings, days on market, price cuts, cancellation/delisting, and sale-to-list trends.
- FRED/Freddie Mac: national mortgage-rate context.
- Census ACS, BLS, HUD FMR: income, employment, and rental reference ranges.
- Travis County/TCAD official parcel fields where a field/license audit permits.
- User-provided MLS exports, lender quotes, tax notices, insurance quotes, rent rolls, and comparable-sales evidence for property-level decisions.

Texas assessment is always labelled `assessment`, not market value or sold price. It cannot substitute for closed-sale comparables.

### ### Phoenix Metro Arizona

Scope: Phoenix Metro and an owner-managed ZIP watchlist. Use FHFA, Redfin, FRED, Census, BLS, and HUD source families. Maricopa County official parcel/tax data is a future adapter only after an explicit field, license, and rate-limit audit. No national series may masquerade as an individual-property valuation.

### ### Bengaluru Karnataka

Scope: Bengaluru plus an owner-managed locality registry. The canonical local unit is locality, with optional PIN/ward/taluk; a US ZIP model is not reused.

- NHB RESIDEX housing, land, and rental indices when its published coverage and segment identify an applicable Bengaluru geography.
- RBI quarterly Bengaluru HPI as an independent city-level reference.
- RBI macro/policy rate context for financing conditions.
- Bhoomi, Kaveri, and BBMP only for owner-directed title, land-record, guidance-value, or tax-document review.
- Owner-imported broker comps, loan offers, leases, and registration evidence.

Guidance value is statutory/reference context, never a promised sale value. Automated scraping of Indian property portals is forbidden.

## ## Evidence Collection Policy

### ### Source priority

1. Official API/download with a documented metric definition.
2. Licensed data expressly permitting intended use.
3. Owner upload with source, date, geography, and document type.
4. Bounded official/public-policy page capture with URL and content hash.
5. Narrative news as explanation-only context.

Firecrawl/browser automation may only fetch a permitted official/public-policy page with no structured endpoint. It is not a listing crawler, comparable-sales source, rent feed, or property-search engine. Zillow, Realtor, Redfin listing pages, MLS portals, Magicbricks, Housing, and similar sites remain excluded unless a future written license explicitly permits the exact use.

### ### Cadence and bounds

| Source class                   | Austin                       | Phoenix                      | Bengaluru               | Retention                  |
|--------------------------------|------------------------------|------------------------------|-------------------------|----------------------------|
| Mortgage and macro             | weekly                       | weekly                       | monthly or policy event | normalized observations    |
| Listing market conditions      | weekly metro and monthly ZIP | weekly metro and monthly ZIP | unavailable by default  | source hash per release    |
| Official price rent land index | monthly or quarterly         | monthly or quarterly         | quarterly               | all released values        |
| Parcel/tax context             | monthly/on request           | monthly/on request           | owner request only      | selected fields only       |
| Owner comps and quotes         | on import                    | on import                    | on import               | encrypted per owner policy |

There is no daily nationwide ZIP crawl. `property\_geography\_registry` contains the initial metros and owner-selected ZIPs/localities. A monthly coverage job can recommend a new geography only when adequate published data exists and a declared owner rule requests it.

Every metric records source, source version/hash, geography, property segment, native unit, `as\_of`, `published\_at`, `collected\_at`, revision state, and availability. Missing data is `unavailable`, never zero or neutral.

## ## Data Model And Privacy

All tables use a `property\_` prefix and owner-only RLS. Service-role writes are limited to approved collection workers.

| Record                         | Purpose                                                            |
|--------------------------------|--------------------------------------------------------------------|
| `property_geographies`         | country hierarchy, ZIP/PIN/locality, geometry reference, currency. |
| `property_market_observations` | immutable normalized market time series.                           |
| `property_source_runs`         | source health, calls, costs, rows, and errors.                     |
| `property_assets`              | encrypted address/documents and owner asset metadata.              |
| `property_financing_accounts`  | encrypted loan terms, balances, rates, and payments.               |
| `property_scenarios`           | immutable versioned buy/sell/rent/finance calculations.            |
| `property_forecasts`           | declared forecast, horizon, cutoff, range, and model version.      |
| `property_forecast_outcomes`   | matured actuals and calibration metrics.                           |
| `property_decision_journal`    | recommendation state, evidence references, and owner action.       |

Full addresses, loan balances, credit data, documents, account numbers, and lender quotes are encrypted before persistence. Logs use property ID and coarse geography only. Full-address enrichment needs explicit per-request owner consent and an allowlisted provider; default calls use a derived ZIP/locality/CBSA.

## ## Decision Engines

### ### Market regime

The descriptive regime uses only market-local data: price momentum/deceleration, inventory and listing-pending balance, days on market, sale-to-list, price-cut/cancellation/delisting pressure, affordability shock, employment/income, and rent-to-price data only when both measures share compatible geography/period.

Allowed output: `tight`, `balanced`, `buyer\_favoring`, `cooling`, or `data\_insufficient`, always with visible drivers and missing inputs. It must not call cooling a recession or invent a severe-drop probability before a validated forecast program exists.

#### ### Financing

US supports purchase mortgage, refinance, and HELOC. Bengaluru supports home-loan and loan-against-property scenarios; it must not display HELOC as an Indian product.

Required outputs: amortization, interest, payment stress, LTV/CLTV when an owner valuation exists, refinance NPV/break-even/fee sensitivity, HELOC index-margin rate shocks and liquidity impact, and lender-quote comparison. Benchmark rates are context only, not offers. The engine returns `actionable`, `watch`, `not\_economic\_under\_assumptions`, or `insufficient\_inputs`.

#### ### Buy sell rent and land

Every scenario exposes its inputs: price, down payment, financing, tax, insurance, HOA, maintenance, vacancy, management, rent, renovation, closing costs, holding period, selling cost, and exit-price range. Outputs are cash flow, cap rate, cash-on-cash, DSCR, equity, debt paydown, IRR range, sensitivity, and break-even sale price. Land is a distinct type with no rental-yield default.

#### ## Forecasting And Learning

Forecasting is a market-local shadow. Initial targets are index change, rent index change, and mortgage-rate range, never individual parcel/home value.

Each forecast records input fingerprints, data cutoff, geography, property type, unit, horizon, model version, baseline/bear/base/bull range, confidence, and known coverage gaps. Outcome maturity attaches actual values and error metrics.

The initial baseline is latest level/trend plus a transparent uncertainty range. A complex model may enter an advisory state only after it beats that baseline in sealed time-ordered replay and forward shadow for the exact market and horizon. Learning can lower confidence or retire a model; it cannot automatically make a recommendation more aggressive.

#### ## Chart Specification

1. Indexed price trend: country/state/metro/ZIP/locality, latest observation date and lag shown per series.
2. Supply demand: active/new/pending listings, months supply, days on market.
3. Price behavior: sale-to-list, price cuts, cancellations, delistings/relistings.
4. Affordability: mortgage rate, representative payment, income and rent burden.
5. Rental economics: rent trend, rent-to-price, yield, and input confidence.
6. Market pressure: visible drivers and data coverage, not a decorative score.
7. Forecast fan chart: bear/base/bull range, actuals, and calibration history.
8. Property detail: amortization, refinance/HELOC break-even, cash-flow waterfall, and downside sensitivity.

Every chart presents source, definition, geography, unit, as-of date, revision, and unavailable reason. It must not place incomparable assessments, listing prices, transaction indices, and median prices on one unlabeled axis.

#### ## Source Manifest

```
Source	Permitted initial use	Explicit limitation
FHFA HPI	US repeat-sales price-trend context by published geography	Not a parcel valuation or live comparable feed.
Redfin Data Center	Published market-condition downloads and methodology-defined aggregates	No listing-page crawling or assumed resale r
Freddie Mac via FRED	Weekly national mortgage-rate context	Not an individual lender offer or HELOC quote.
Census ACS, BLS, HUD FMR	Area income, employment, and rent-reference context	FMR/ACS are not current property rent comps.
Travis/Maricopa official records	Geography, parcel, assessment, and tax context after adapter audit	Assessment is not sale price; field
NHB RESIDEX and RBI HPI	Bengaluru/India published price, land, rent, and macro trend context	Quarterly/city-level observations are not
Karnataka government records	Owner-directed land/title/guidance/tax reference	No automated title conclusion or market-value assertion.
Owner uploads	MLS comps, loan offers, rent rolls, tax/insurance documents	Provenance and encryption are mandatory.
```

The source implementation record must preserve the official URL, terms/permission review date, metric dictionary, schedule, expected geography/segment coverage, rate limit, and fallback behavior. A source may not become active solely because it is easy to crawl.

#### ## Security And Safety

- No Property route/table/worker imports broker, paper-trade, portfolio-cash, score, execution, or Investing agent modules.
- Property data never appears in LLM prompts, generic analytics, or logs.

- No scraping behind logins, CAPTCHAs, robots exclusions, terms, or MLS controls.
- No lending, appraisal, title, tax, legal, or real-estate-agent claim.
- Every recommendation links to a versioned scenario and evidence set.
- Owner export/delete removes Property data without touching Investing evidence.

## ## Delivery Plan

| Phase | Build                                                                    | Influence              |
|-------|--------------------------------------------------------------------------|------------------------|
| P0    | workspace shell, geography registry, source catalogue, source-run health | display only           |
| P1    | Austin, Phoenix, Bengaluru market packs and comparable charts            | display/alerts only    |
| P2    | private vault and deterministic scenario/finance calculators             | owner decision support |
| P3    | comp/quote imports and decision journal                                  | owner decision support |
| P4    | forecast ledger, replay, and forward calibration                         | shadow only            |
| P5    | evidence-qualified market/financing recommendation states                | advisory only          |

## ## Acceptance Criteria

1. Workspace switch makes Property and Investing unmistakable.
2. Each metric exposes source, geography, as-of date, unit, freshness, and state.
3. Austin, Phoenix, and Bengaluru never cross-sum or share unqualified metrics.
4. Collection is bounded, idempotent, quota-accounted, and never crawls forbidden listing sites.
5. Full address and finance data never reach LLMs, analytics, or logs.
6. Forecasts cannot become recommendations before exact-market calibration.
7. Property cannot affect securities research, trading, brokers, or execution.

---

# Source: features\property-decision-workspace\IMPLEMENTATION\_RESULT.md  
# Kairos Property – implementation result

Date: 2026-08-07 · Production project `<production-project-ref>`  
Status: \*\*P2-P4 and valuation-evidence Stage 1 shipped. Schema, secrets, production deployment, parser self-checks, and the no-scope download guard are verified. Authenticated visual verification remains pending.\*\*

---

## ## 1. Findings, by severity, and how each was fixed

Every one of these was found by reviewing the work rather than by a test failing, and every fix below was verified against the live database.

### CRITICAL – append-only was breakable by TRUNCATE  
Five evidence tables were defended by \*\*BEFORE ROW\*\* triggers only. A row trigger \*\*does not fire on TRUNCATE\*\*, and `service\_role` still held the grant, so every "append-only" property ledger could be erased with every stated guarantee apparently intact. This is the identical hole closed on 2026-08-01 for the exogenous-risk tables (`20260801160000`), reintroduced.

\*Fix:\* statement-level `BEFORE TRUNCATE` triggers on `property\_scenarios`, `property\_forecasts`, `property\_forecast\_outcomes`, `property\_market\_observations`, `property\_decision\_journal`, plus `REVOKE UPDATE, DELETE, TRUNCATE ... FROM service\_role` so grants and triggers are two independent barriers. \*\*Verified live:\*\* TRUNCATE is rejected with `property\_market\_observations` is append-only and cannot be truncated, and `service\_role` holds neither TRUNCATE nor DELETE on any of the five.

### HIGH – the migration could only ever be applied once  
`CREATE TRIGGER` has no `IF NOT EXISTS` in any Postgres version, so a re-run would abort. A migration that cannot be re-applied is not idempotent and will break any replay or environment rebuild.

\*Fix:\* drop-then-create inside a `DO` loop. \*\*Verified\*\* by re-running the whole trigger/grant block: succeeds, five TRUNCATE triggers still present.

### HIGH – silent logical duplication in observations  
`property\_market\_observations` uniqueness key includes the \*\*nullable\*\* `source\_version`. Postgres treats NULLs as distinct, so two rows with identical source, geography, metric, `as\_of` and `revision\_state` and a NULL version would \*\*both\*\* insert.

\*Fix:\* replaced with a unique index using \*\*NULLS NOT DISTINCT\*\*. Safe to do without a backfill because the table held zero rows at the time.

### HIGH – revisions were silently suppressed  
Every adapter filtered `as\_of <= since`. A source republishing a corrected value for an existing period could therefore never be ingested, and the `revision\_state` column and the revision-aware uniqueness key could never fire. The schema modelled revisions; the collector made them unreachable.

\*Fix:\* `keepObservation()` always admits a `revised` observation and applies the `since` lower bound only to `initial` ones. The unique index remains the thing that prevents a true duplicate. Unit-tested in `tests/property-ingestion.test.ts`.

```

MEDIUM — national files re-downloaded once per market
FHFA's multi-megabyte master JSON and FRED's full history were fetched **per
market** with `cache: "no-store"` explicitly defeating Next's caching — nine
upstream calls for a three-market run.

Fix: a per-invocation cache in `lib/property/sources.ts`, cleared at the start
of every run. Deliberately **not** TTL-based: a TTL would serve a stale national
file into a later scheduled run. **Measured 9 → 4**, and 4 is correct because BLS
needs one request per metro.

MEDIUM — dishonest market coverage reporting
Bengaluru recorded `success, 0 rows` for US-only sources, making a structural
coverage gap indistinguishable from a quiet collection day.

Fix: adapters declare `supportsMarket()`; the collector records
`not_applicable` with a reason and `request_count = 0`; the `outcome` CHECK was
extended to allow it. The Markets page states the gap in words instead of showing
three empty charts.

MEDIUM — provider-call accounting overstated usage
`property_source_runs.request_count` recorded `1` per adapter call even when the
run cache served the payload with no network request — inflating the very ledger
used to justify the collection cadence.

Fix: records the actual number of upstream fetches attributable to that run.

MEDIUM — duplicated scenario math in the UI
`OpportunitiesWorkspace` carried its own `monthlyPayment()` and `metrics()`
alongside the server's `evaluateRentalEconomics()`. Two implementations of one
formula drift, and then the displayed number and the stored number disagree with
no way to tell which is right.

Fix: the component renders exactly what the engine computed and stored. The one
remaining local calculation (deriving annual debt service from the typed rate and
down payment) imports the same `calculateMortgage()` the server uses.

MEDIUM — a 5xx rendered as "no data"
`PropertyMarketData` never checked `response.ok`, so an outage and an empty
market looked identical.

Fix: explicit load-error state that says it is a load failure. Same guard added
to the Opportunities, Financing and Forecasts workspaces.

LOW — 2,889 mortgage points plotted raw
Fix: 1Y/5Y/Max window plus downsampling to 220 points, always retaining the
true latest observation so the "as of" line cannot disagree with the chart.

```

---

## ## 2. What was implemented

- **Schema**: six private/evidence tables, RLS on, browser grants revoked, encrypted payload columns, append-only enforced by trigger **and** grant.
- **Ingestion**: FHFA HPI, FRED `MORTGAGE30US`, BLS LAUS as small native adapters; per-invocation cache; revision-aware admission; honest per-market outcomes.
- **Crons**: `kairos-property-collect` (Sun 10:00 UTC) and `kairos-property-forecast` (Sun 10:30 UTC). Weekly because these sources publish monthly/quarterly.
- **Encryption**: AES-256-GCM, versioned, fail-closed (`lib/property/crypto.ts`).
- **Opportunities**: scenarios persist through the server engine; DSCR and decision state surfaced; USD/INR never combined.
- **Financing**: accounts persist; refinance and rate-shock use the shared engine; cumulative-benefit chart with a breakeven marker plus a stacked principal/interest amortization curve.
- **Markets**: windowed, downsampled charts; observed vs shadow-forecast drawn as distinct series with a band; honest empty, error and not-applicable states; source attribution rendered.
- **Forecasts & learning**: new `GET /api/property/forecasts` returning forecasts joined with matured outcomes; forecast-range-versus-actual chart; calibration table that **withholds** any rate below 10 matured outcomes while always showing `n` (`lib/property/calibration.ts`, 8 unit tests).

## ## 3. Gates

```

`npx tsc --noEmit` clean. `npx vitest run` **1704 passed / 7 skipped**.
`npm run build` clean.

```

Two tests failed on two separate runs and passed on a clean re-run both times. The flaky pair has not been identified. It is recorded here rather than ignored.

## ## 4. Production verification

```

check	result

```

```
11 property tables present	yes
RLS enabled on all	yes
`service_role` TRUNCATE/DELETE on the 5 evidence tables	**none**
TRUNCATE actually blocked	yes, by trigger
`NULLS NOT DISTINCT` on the observation identity index	yes
Both crons registered	yes, Sundays 10:00 / 10:30 UTC
Supabase security advisors, Property findings	none above INFO; the INFO `rls_enabled_no_policy` entries are deliberate (server-only tables)
```

**\*\*Ingested:\*\*** Austin and Phoenix each hold FHFA HPI (196 quarters from 1977), FRED `MORTGAGE30US` (2,889 weeks from 1971) and BLS LAUS (65 months).  
**\*\*Bengaluru holds zero\*\*** rather than borrowing US values.

#### ## 5. Source coverage, honestly

```
market	price index	mortgage rate	unemployment	note
Austin	FHFA HPI ■	FRED ■	BLS LAUS ■	full US pack
Phoenix	FHFA HPI ■	FRED ■	BLS LAUS ■	full US pack
Bengaluru	■	■	■	all three active sources are US-only; recorded `not_applicable`, never substituted
```

Bengaluru requires RBI/NHB bounded official export or owner-directed encrypted import. Both remain `contract\_pending`; neither is built.  
 Census ACS and HUD FMR are `contract\_pending` pending a key/token.

#### ## 6. Genuinely deferred – not complete

- \*\*No Playwright or visual verification.\*\*** The pages are auth-gated and no desktop/mobile screenshot pass was run. Responsive classes are reused from the existing shell but are unverified visually.
- \*\*Bengaluru data\*\*** – no India source is implemented.
- \*\*`rent\_index`\*\*** is forecast-capable in code but no adapter produces it, so it will never populate.
- \*\*Open design question, undecided:\*\*** `MORTGAGE30US` is a **\*\*national\*\*** series stored per market, so 2,889 identical rows exist for Austin and again for Phoenix. It works, but it duplicates and misrepresents a national series as market-local. A cleaner model is one `national` geography that markets reference. Left as an owner decision, not silently changed.

#### ## 7. Completion audit and valuation Stage 1 (2026-08-07)

The oldest-first observation-limit defect was corrected before the first forecast cron: market overview now bounds each metric independently from newest rows, and forecasts select the newest coherent source series, collapse revisions deterministically, project the declared horizon in observed periods, bind source provenance, and freeze the exact matured observation. Production contains zero legacy forecasts, so no stale forecast required supersession.

Property valuation remains evidence-only. Production now has owner-selected scope configuration plus append-only bulk snapshot, write-event, parcel, and deed-linked transfer ledgers. Phoenix accepts ZIP scopes; Austin accepts TCAD parcel IDs that are HMACed before persistence. No address, party name, or raw parcel ID is stored. The monthly GitHub Action exits before download without a scope, pins action SHAs, uploads no raw artifacts, and runs no model. The UI says `NO AVM`, `NO MARKET-PRICE ESTIMATE`, and `NO PARCEL VALUE RANGE`.

The identical Property encryption master is configured in local development, Vercel production/preview/development, and the FinanceOS GitHub Action. GitHub also has the FinanceOS Supabase URL and service-role key; no value was logged.

Production deployment `dpl\_9QQQZaxASYtQMkSMGrf3BaZ1f9H6` is live at <https://financeos-phl.vercel.app>. GitHub Actions run `` completed both matrix jobs: each parser self-check passed and each source returned `NO\_SCOPE` before downloading. This is the intended state until the owner adds a Phoenix ZIP or Austin parcel through the authenticated UI.

#### ## 8. Mobile workspace parity (2026-08-08)

The Property shell now uses a compact mobile header with an explicit navigation menu instead of rendering the full brand, workspace switcher, and eight-link horizontal strip above every page. Shared page grids collapse without horizontal overflow; actions become full-width; stat rows reduce to two and then one column; charts use a stable phone height; and data rows retain their field names when desktop headers disappear. Overview, Market Explorer, and Data Sources now have the same responsive contract as the shared workspace pages. The stale Data Sources statement that no collectors exist was also removed.

Verified on production deployment `dpl\_B3zY6VdvZ6uPjh2ZmqYGwGyWJzUf`: all eight Property destinations had viewport width equal to document scroll width at 390px; the collapsed menu exposed eight links; the 1440px layout retained its 236px sidebar; and the browser console had no warnings or errors.

The follow-up device-safe-area audit found one gap the browser-width test could not simulate: unlike Investing, Property did not clear iOS's translucent status bar even though the root viewport uses `viewport-fit: cover`. The mobile brand header and navigation now include the top/left/right safe-area insets, the main

content includes the bottom home-indicator inset, and the shell uses `100dvh`.

The first mobile-navigation revision also hid the workspace switch and placed Property's hamburger on the opposite side from Investing. Property now follows the Investing interaction contract: a sticky safe-area top bar with the menu button at far left, a slide-in left drawer, backdrop dismissal, and an explicit Investing link in both the top bar and drawer workspace switch.

## ## 9. Page guidance parity (2026-08-08)

Every Property destination now carries the same collapsible, page-specific help contract as Investing: **What's here** and **What to look for**. Guidance is attached to the shared `PropertyPageFrame`, so Properties, Markets, Valuation, Imports, Opportunities, Financing, Forecasts, and Sources cannot drift into different help implementations. The Overview uses that same frame. Guidance states evidence and estimate limitations explicitly; it never represents a county assessment, manual scenario, or shadow forecast as a market-value claim.

---

# Source: features\property-owner-evidence\FEATURE\_ARCHITECTURE.md  
# Property Owner Evidence Intake

Status: Implemented 2026-08-08.

### ## Purpose

An owner can preserve the text of a property-tax notice or an insurance quote/policy inside the Property workspace, without turning an upload

### ## Boundary

- New route/page only: `/property/imports` and `/api/property/owner-evidence`.
- Does not modify `MyPropertiesWorkspace.tsx`, `/api/property/assets`, the shared shell, market data collection, valuation, forecasts, scores, or lender-quote.
- Existing generic `/api/property/imports` is deliberately unchanged. This new surface cannot break future comps, rent-roll, or lender-quote.
- Existing `property\_imports` storage is reused. No migration is needed.
- Records are append-only by product policy. This phase has no delete, edit, file upload, OCR, extraction, or LLM processing.

### ## Contract

Only `tax\_notice` and `insurance\_quote` are accepted. The server verifies the owner, validates a bounded label/date/market/text payload, comp

The user-facing form states that a document is evidence only. Tax and insurance values remain owner-entered in the private property record un

### ## Acceptance

- Unauthenticated or non-owner callers cannot read or write evidence.
- A missing encryption key rejects writes with no plaintext fallback.
- The UI supports all shared property markets but never cross-sums or converts money.
- Content is capped at 1 MB and exact source text does not enter logs, URLs, or browser lists.
- Tests reject unsupported types, invalid dates, and unknown markets.

---

# Source: features\property-valuation\FEATURE\_ARCHITECTURE.md  
# Property Valuation Evidence

Status (2026-08-08): **Stage 1 schema is shipped, but collection is disabled.**  
The previous pre-download `NO\_SCOPE` guard remains valid but is no longer enough: both county collectors are stopped before credentials, scopes, or downloads until their individual machine-use contracts are verified.

Source-audit correction (2026-08-08): the claimed public/free Phoenix R102 sales source is now **licence-unverified**. Maricopa's official data-sales catalogue lists Sales Affidavits as a paid product. No new feed use, expansion, or free-availability claim is authorized until the cited ArcGIS attachment's publisher, licence, and machine-use permission are recorded. See `features/property-address-and-carrying-costs/SOURCE_CAPABILITY_AUDIT.md``.

Parent: `features/property-decision-workspace/FEATURE\_ARCHITECTURE.md`

There is no AVM, repeat-sales index, hedonic model, comparable-sale engine, or parcel value claim in Stage 1. The word `valuation` names the user problem; the current product surface is an evidence ledger.

### ## 1. Product Boundary

Property evidence remains inside the Property workspace. It cannot affect a security score, research eligibility, sizing, order, exit, strategy promotion, or broker call. It never initiates a property or financing transaction.

The owner approved two different evidence tiers because the underlying law and data differ:

| Market | Stage 1 capability | Explicitly unavailable |  
|---|---|---|



| Phoenix | Recorded deed/transfer observations for owner-selected ZIPs | Active AVM or parcel value range |  
| Austin | FHFA metro trend plus TCAD county appraised/assessed reference for owner-selected parcels | Public comparable sales or market-price  
| Bengaluru | No parcel source in this feature | Any substituted US evidence |

## ## 2. Why Phoenix And Austin Differ

### ### Phoenix

This design previously treated a cited ArcGIS ZIP as a public R102 Sales Affidavits source. That assertion is not yet source-verified: Maricopa's official data-sales catalogue lists Sales Affidavits as a paid product. No production collection may rely on that attachment until its legal and technical status is resolved. If approved, its columns would include parcel number, month-level sale date, reported price, deed number/date/status/type, property class, situs fields, parties, finance fields, assessor quality codes, and personal-property flags.

Stage 1 reads only the fields needed for identity, quality filtering, ZIP scope, and the recorded transfer. It never reads or stores grantor/grantee names, mailing addresses, or owner fields. Situs addresses are not persisted or hashed.

The file is a latest-transfer snapshot, not transaction history. A changed row can be a correction rather than a new transaction. Therefore:

- event identity is HMAC(parcel) + deed number + deed date;
- each source release remains a separate immutable observation;
- corrections are preserved rather than overwritten;
- later repeat-sale work must collapse reviewed deed identities, not diff rows and assume every change is a sale.

Official references:

- <https://www.arcgis.com/sharing/rest/content/items/f3484c72a938497286adc4e5de7e9963?f=pjson>
- <https://www.arcgis.com/sharing/rest/content/items/f3484c72a938497286adc4e5de7e9963/data>
- [https://www.mcassessor.maricopa.gov/file/data\\_sales/R102\\_Sales%20Affidavit\\_Layout\\_ST42025.pdf](https://www.mcassessor.maricopa.gov/file/data_sales/R102_Sales%20Affidavit_Layout_ST42025.pdf)

### ### Austin

Texas does not provide a public individual-sale-price feed suitable for comparable-sales modeling. Texas Tax Code 22.27 protects voluntarily disclosed sale-price information held under confidentiality, and Government Code 552.149 protects related private-entity data held by appraisal authorities.

TCAD does publish certified appraisal exports. They contain distinct `market`, `appraised`, and `assessed` fields. Kairos stores the exact county terminology; it does not collapse those fields into a sale price.

Austin can display:

1. FHFA Austin metro HPI trend already collected by Property Markets.
2. TCAD county appraised and assessed references for a selected parcel.
3. The tax year and source release.

The UI must always state: **\*\*County appraisal reference, not a market price.\*\***  
No Texas AVM may be trained on county assessments, because that would fit one model to another model and falsely present the result as market evidence.

Official references:

- <https://traviscad.org/publicinformation/>
- current certified appraisal export linked by that page;
- official export layout ZIP linked by that page.

## ## 3. Source And Licence Boundary

Both sources are approved only for the current owner's private, non-commercial decision support. No open-data or redistribution licence is asserted.

Maricopa has separate commercial-purpose request rules and fees. TCAD's public download page does not grant a commercial redistribution licence. Any future commercialisation, redistribution, or multi-user product must stop these feeds until a new declared-use, fee, permission, and legal review is completed.

County data carries no accuracy guarantee. Recorded transfers are evidence, not automatically arm's-length comparable sales. County appraisals are tax-system model outputs, not independent appraisals or expected sale prices.

## ## 4. Privacy Contract

The bulk worker receives the same 32-byte Property encryption master through a GitHub Actions secret. It derives domain-separated HMAC-SHA256 lookup keys:

HMAC(master, "property:parcel:v1\0" + normalizedParcelId)

## HMAC(master, "property:sale-event:v1l0" + parcelKey + deedIdentity)

Plain SHA-256 is forbidden because parcel IDs and addresses are enumerable. Raw parcel/account IDs are accepted only by an owner-gated server route and are converted before persistence. Parcel HMACs are not returned to the browser.

The bulk worker does not persist or log:

- plaintext parcel/account identifiers;
- owner, grantor, or grantee names;
- mailing or situs addresses;
- raw source archives;
- full input rows.

### ## 5. Bounded Collection

Large county archives never enter a Vercel request. The monthly ``.github/workflows/property-evidence.yml`` worker runs on an ephemeral GitHub Actions runner.

Safety properties:

1. It queries active scopes before any download. No scope means no provider call.
2. Phoenix persists only selected five-digit ZIPs.
3. Austin persists only selected HMAC parcel IDs.
4. Raw archives live only in a temporary directory and are never artifacts or caches.
5. GitHub actions are pinned to commit SHAs with ``contents: read`` only.
6. Maricopa requires the exact audited 44-column R102 header and fails closed on schema drift.
7. TCAD uses the smaller certified fixed-width export, not the multi-gigabyte special JSON export, and follows the official layout positions.
8. Every run records source hash, release ID, scope fingerprint, rows seen, selected rows, actual inserted rows, and rejection counts.
9. No row-level source content is written to logs.

The service-role secret is still broad. It is accepted only because FinanceOS already uses it in a private repository action and all target tables are server-only. A future multi-user product must replace it with a narrowly scoped ingestion service or database role before activation.

### ## 6. Data Model

| Table                                        | Mutability            | Purpose                                                           |
|----------------------------------------------|-----------------------|-------------------------------------------------------------------|
| <code>`property_valuation_scopes`</code>     | Mutable configuration | Phoenix ZIP or Austin HMAC parcel scopes                          |
| <code>`property_bulk_snapshots`</code>       | Append-only           | Source release, file/scope fingerprints, normalization counts     |
| <code>`property_bulk_snapshot_events`</code> | Append-only           | Write started/completed/failed ledger with actual rows            |
| <code>`property_parcel_snapshots`</code>     | Append-only           | TCAD county references and selected parcel attributes per release |
| <code>`property_sales`</code>                | Append-only           | Maricopa deed-linked transfer observations per release            |

Append-only means UPDATE, DELETE, and TRUNCATE are blocked by grants and triggers. RLS is enabled and browser grants are revoked. Owner APIs use the server-side service client only after the existing owner gate succeeds.

The sale uniqueness key includes source, deed-based event HMAC, and source snapshot. This deliberately retains a correction in a later release. Current state is derived by a view/query with documented precedence; evidence is never rewritten.

TCAD releases must preserve tax year and release class. Supplemental rows do not overwrite certified or preliminary rows. Display precedence is latest valid supplement, then certified, then preliminary, while every observation remains.

### ## 7. UI Contract

``/property/valuation`` shows evidence state and owner scope controls.

Phoenix:

- selected ZIP scopes;
- source status, last snapshot, actual row count, and errors;
- explicit ``NO AVM``, ``NO MARKET-PRICE ESTIMATE``, and ``NO PARCEL VALUE RANGE`` labels.

Austin:

- selected private parcel scope;
- latest county appraised/assessed values with tax year;
- FHFA metro trend chart;
- persistent statement that county values are tax references, not sale prices.

Zero rows, no configured scope, provider failure, and unavailable capability are four different states and must never share copy.

## ## 8. Learning And Later Stages

Stage 1 makes no prediction. It only accumulates auditable source snapshots.

### ### Phoenix Stage 2: repeat-sale ZIP index

Blocked until at least two dated source snapshots exist and reviewed deed identity creates genuine same-parcel repeat transactions. Before activation it requires:

- correction/retraction handling;
- arm's-length quality policy using deed status and assessor codes;
- minimum event and ZIP sample floors;
- temporal validation with no future release leakage;
- a declared benchmark and error metric.

### ### Phoenix Stage 3: hedonic interval

Blocked until Stage 2 is calibrated. It may use permitted parcel attributes and must output an interval, not a point. It requires temporal holdout testing, out-of-sample error by ZIP and property type, and at least ten matured outcomes before any calibration rate is shown.

### ### Phoenix Stage 4: owner parcel interpolation

Blocked until Stage 3 passes. It inherits the ZIP model's measured error and must be labelled as interpolation, not an independent appraisal.

### ### Austin

There is no later AVM stage under the current source contract. New public, licensed arm's-length sales evidence would require a new architecture decision.

## ## 9. Go/No-Go Gates

Stage 1 may run only when:

- both database migrations are applied and verified;
- Vercel and GitHub use the identical Property encryption master;
- the owner has configured a bounded ZIP or parcel scope;
- the source page, schema/layout, licence posture, and file size still match the audited contract;
- parser self-check, TypeScript, tests, and production build pass;
- production RLS, grants, append-only triggers, and advisor results are clean.

No later stage is enabled merely because its code could be written. Evidence, sample size, and calibration gates are product behavior, not implementation delay.

---

# Source: features\property-valuation\IMPLEMENTATION\_RESULT.md  
# Implementation Result

Updated on 2026-08-08:

- Stage 1 tables and immutable historical evidence remain in place.
- `maricopa-sales` and `tcad-appraisal` are database-enforced as `contract\_pending`; existing scopes were deactivated by `20260808194000\_defer\_unverified\_property\_valuation\_sources.sql`.
- The scope API now rejects activation, and the bulk worker returns before credentials, scopes, or downloads for either source.
- The GitHub workflow retains only parser and disabled-source checks. It has no schedule or Supabase/encryption secrets.
- The valuation UI explains the source-contract block instead of offering a collection control.

Not implemented: new county collection, sale comparables, AVM, repeat-sales index, or hedonic model. These require documented and permitted source contracts first.

---

# Source: features\property-value-intelligence\FEATURE\_ARCHITECTURE.md  
# Property Value Intelligence

### ## Status

Implemented through value evidence, index references, and index scenarios on 2026-08-08. This design replaces the ambiguous single owner-entered `value` field with source-labelled property value evidence and an explainable market-index reference. It does **not** authorize a Zillow/Redfin-style AVM until market-specific data and validation gates are satisfied.

### ## Problem

`My Properties` currently stores one owner-entered value as of a chosen date. That value can be a purchase price, an appraisal, a tax assessment, or an opinion, but the system cannot tell which. A historical chart of that mixed field is not a defensible estimate, cannot explain a change, and cannot be calibrated against an eventual sale.

The owner needs three distinct answers:

1. What is known about this property from explicit evidence?
2. What does that evidence imply when rebased by an auditable local market index?
3. What are the 1-, 3-, and 5-year bear/base/bull scenarios, why do they differ, and how accurate has the system historically been?

## ## Product Decision

Build a **Property Value Intelligence** surface with three strictly separated layers:

| Layer             | Output                                                               | May claim                                           | Must not claim                 |
|-------------------|----------------------------------------------------------------------|-----------------------------------------------------|--------------------------------|
| Value evidence    | Purchase, appraisal, sale, owner estimate, tax reference, owner comp | What an identified source says at a date            | Current market                 |
| Indexed reference | Rebased range from one admissible base value plus one market index   | "Market-index-adjusted reference" and exact formula |                                |
| Forecast          | 1/3/5-year market-index bear/base/bull ranges                        | Scenario range with assumptions and confidence      | Exact future property price or |

The interface must never collapse these outputs into one number. No LLM may produce a value, alter a range, select evidence, or write an observation. An LLM can only summarize an already-computed result with its evidence IDs.

## ## Current Capability By Market

| Market    | Evidence now                                         | Indexed reference                                                         | AVM / exact estimate                            |
|-----------|------------------------------------------------------|---------------------------------------------------------------------------|-------------------------------------------------|
| Austin    | Owner purchase/appraisal/estimate and FHFA metro HPI | Yes, if the owner chooses a dated base value                              | No. Texas has no permitted, n                   |
| Phoenix   | Owner evidence and FHFA metro HPI                    | Yes, if the owner chooses a dated base value                              | No until a permitted parcel/sales source is act |
| Bengaluru | Owner purchase/appraisal/estimate                    | Not until an RBI/NHB source has a documented reusable export/API contract | No. No approved                                 |

No US series is used for Bengaluru. USD and INR are never summed or compared as one book.

## ## Data Model

### ### `property\_value\_observations`

Add an owner-only, append-only encrypted ledger. Public metadata is deliberately minimal: `id`, `asset\_id`, `owner\_id`, `market`, `currency`, `observed\_on`, `kind`, `provenance`, `created\_at`, and `supersedes\_id`. Amount, source URL, document reference, comp notes, appraisal company, and rationale stay inside a versioned AES-256-GCM payload.

Allowed `kind`:

- `purchase\_price`
- `owner\_estimate`
- `documented\_appraisal`
- `observed\_sale`
- `county\_appraised\_reference`
- `county\_assessed\_reference`
- `owner\_comparable`

Allowed `provenance`:

- `owner\_entered`
- `owner\_document`
- `official\_reference`

Market-index results and scenarios are derived records in `property\_value\_references`; they are never written back into the evidence ledger as if they were an observation.

Triggers and grants must block update, delete, and truncate. A correction writes a new observation with `supersedes\_id`; it does not rewrite history.

### ### `property\_value\_references`

Append-only deterministic output ledger for indexed references and forecast scenarios. It binds every result to:

- asset, market, currency, and result kind;
- selected evidence observation and its amount/date;
- source observation IDs and index values at the base and cutoff dates;
- formula/model version and input fingerprint;
- cutoff, horizon, lower/base/upper values, and coverage state.

This table is not a cache and cannot be edited in place. A newly computed result

supersedes the old result while preserving prior reasoning.

## Deterministic Calculations

### Market-index-adjusted reference

Only available when the selected base evidence and a compatible local price index are both present:

$\text{indexed\_reference} = \text{base\_amount} * (\text{index\_at\_cutoff} / \text{index\_at\_base})$

The UI shows base amount/date/type, source, both index values, formula, index publication date, and an uncertainty band. The band starts wider than a point forecast and is based on observed historical forecast error only once a market has ten matured outcomes; before then it displays 'unvalidated' rather than inventing calibration.

The reference is unavailable, not zero, if no compatible index exists, the base predates the series, currency differs, or the index is stale.

### 1/3/5-year scenario range

The forecast targets the \*\*market index\*\*, then translates the range to the chosen base evidence only as a labelled scenario. It does not forecast an exact parcel price.

- 1 year: persistent-trend baseline plus historical volatility band.
- 3 and 5 years: same baseline compounded only within an explicit cap and wider uncertainty; no apparent precision from decimal projections.
- Current numeric driver: local price-index momentum and its observed volatility. Mortgage-rate and local-unemployment series are collected as labelled context, not model features, until a predeclared point-in-time shadow proves a market-local improvement against this baseline. Missing inputs are named.
- No national mortgage or labour measure may become a local property fact.
- The owner may run a separate manual assumption scenario, but it is never written to the forecast-evaluation ledger as a Kairos forecast.

Every forecast records a cutoff and later compares itself only with a subsequent published index observation. Historical replay must use only information available by the cutoff date.

## UI

`My Properties` gains a \*\*Value evidence\*\* panel:

- Add purchase price/date separately from a current owner estimate.
- Add an appraisal, sale, county reference, or owner comparable with a source label and optional encrypted evidence link.
- Show a timeline with separate visual series; a county reference never joins a sale-price line.
- Replace the ambiguous `Your value` label with a clear evidence-type selector.

Each property also gains a \*\*Value intelligence\*\* card:

- Latest evidence, indexed reference, and forecast state appear as separate rows.
- "Why this changed" lists changed source observations, selected base evidence, index movement, model version, and missing inputs.
- 1Y, 3Y, and 5Y tabs show bear/base/bull paths, input date, freshness, formula, and calibration status.
- `unavailable`, `insufficient\_evidence`, `unvalidated`, and `not\_applicable` are first-class states.

No value card is shown for a property until the owner chooses an evidence base. No recommendation to buy, sell, refinance, or borrow follows from a range.

## AVM Gate

A per-property "Kairos Estimate" is prohibited unless all are true for the exact market/property class:

1. A documented, permitted source for recent arm's-length sales and relevant property attributes is active.
2. The data has point-in-time release/version semantics and recorded revision handling.
3. The model is trained and tested with temporal, geography-aware splits; no random split or post-sale feature leakage.
4. It beats an index-rebased baseline out of sample with published median and tail error by market/property class.
5. At least ten matured forward outputs establish range calibration, and an ongoing error monitor can retire the model.
6. The result is labelled estimate/range, never appraisal or lender-ready value.

Austin and Bengaluru fail gate 1 today. Phoenix fails gate 1 while Maricopa collection remains `contract\_pending`.

## ## Delivery Sequence

1. Migration and encrypted append-only evidence ledger; migrate the current generic owner value as `owner\_estimate` without overwriting history.
2. Evidence-type UI, source labels, immutable timeline, and import linking.
3. Deterministic index-rebased reference for Austin/Phoenix only, with a provenance-rich API and unavailable states.
4. Persisted 1/3/5-year shadow scenario ledger and forecast-vs-index maturity job; wire the existing Forecasts & Learning calibration surface.
5. Owner-comparable import workflow, still no automatic listing/MLS scraping.
6. Market-specific AVM only after the hard gate above. This is deliberately not scheduled by calendar.

## ### Data-use rule

Free data already available to Austin/Phoenix is not automatically a better forecast feature. FHFA price history is the current numeric baseline. FRED mortgage rates and BLS local unemployment are retained as separate, market-local context. The running BLS collector retains its tested recent window; a longer historical intake needs its own reliable provider contract and release-vintage check before it can support an accuracy claim. A candidate may replace the baseline only after a predeclared rolling-origin, no-look-ahead comparison improves both error and interval calibration on the same market.

## ## Acceptance Criteria

- A purchase price is never displayed as a current estimate without its type and date.
- Every derived reference is reproducible from stored evidence/index IDs and a versioned formula.
- A missing/stale index produces unavailable, never a fabricated value.
- Austin, Phoenix, and Bengaluru remain isolated by market and currency.
- No LLM, securities route, broker, score, agent, or money path reads any new table.
- Append-only/RLS/grant/trigger tests, dated-formula tests, no-look-ahead forecast tests, owner-route tests, and mobile/desktop visual tests pass.

## ## References

- Zillow describes its value estimate as a model using public records, MLS, home characteristics, prior sales and trends, and explicitly says it is not an appraisal: <https://www.zillow.com/zestimate/>
- Redfin describes MLS access, nearby recent sales, home/neighbourhood inputs, market coverage thresholds, and that its estimate is not an appraisal: <https://www.redfin.com/redfin-estimate>
- Current Kairos source boundary:  
`features/property-address-and-carrying-costs/SOURCE\_CAPABILITY\_AUDIT.md`

---

# Source: features\property-value-intelligence\IMPLEMENTATION\_RESULT.md  
# Implementation Result

Implemented on 2026-08-08:

- `property\_value\_observations`: encrypted, owner-only, append-only dated value evidence for purchase price, owner estimate, appraisal, sale, county reference, and owner comparable records.
- `property\_value\_references`: encrypted, owner-only, append-only provenance ledger for market-index-adjusted references and 1/3/5-year index scenarios.
- My Properties now includes Value intelligence, keeping source evidence, derived reference, and shadow scenarios visibly distinct.
- Austin and Phoenix derive only from FHFA price-index history when the owner records compatible dated evidence. Bengaluru returns not-applicable until an approved local price-index source exists.
- Every calculation records its formula, index dates and values, source key, model version, and unvalidated calibration state. A user explicitly selects which dated evidence point may become the reference base; recording an appraisal, estimate, or note never changes a derived reference by itself.
- Local BLS unemployment and FRED mortgage rates are optional context. An upstream outage preserves existing observation history and records a typed unavailable result; it never changes a price scenario or makes FHFA price collection appear failed.

Not implemented: a Zillow/Redfin-style AVM, property-level market-price claim, listing scrape, MLS integration, automated comparable selection, or any lending recommendation. Those remain blocked by the architecture's source and temporal validation gates.

---

# Source: features\property-zip-area-context\FEATURE\_ARCHITECTURE.md  
# Property Stage 3 – ZIP-Level Area Context

## ## Status

Implemented 2026-09-08. Owner-approved 2026-09-08.

### ## Problem

Property Stage 1/2 give metro- and county-grain market context (FHFA, FRED, BLS, HUD, Census ACS) and owner-entered per-property value evidence ('property-value-intelligence'). Neither answers "what is the ZIP this property actually sits in doing?" – the grain a home owner actually thinks in.

### ## Product decision

Add one more free, keyless, official source – **Zillow Research ZHVI**, ZIP grain – as a strictly separate **area context** series next to each tracked property's own recorded value. It must never be blended in.

### ### Honesty constraint (hard requirement)

Per Zillow's own [ZHVI User Guide](https://www.zillow.com/research/zhvi-user-guide/), ZHVI is a smoothed, seasonally-adjusted weighted average of the **middle third** of homes in a region. It is:

- NOT this property's value
- NOT an AVM or Zestimate substitute
- NOT ever written into, or used to derive, the owner's recorded `value`

The UI (`components/property/ZipAreaTrend.tsx`) renders it as a clearly labelled, visually separate block: level, 3M/12M change, as-of date, source name/link, and an explicit "why this is separate" line. No LLM path reads or summarizes this table.

### ## Data flow

Zillow Research ZHVI (ZIP CSV, ~120MB, no key)

-> ZillowZhviZipAdapter (lib/property/sources.ts)

filters to Metro ~ Austin-Round Rock / Phoenix-Mesa, last 13 monthly cols

-> POST /api/property/collect (existing weekly kairos-property-collect cron, no new schedule — ZHVI's monthly release is polled weekly; idempotent)

-> property\_zip\_observations (new table, append-only, migration)

20260908120000\_property\_zip\_zhvi\_observations.sql — applied + verified)

-> GET /api/property/zip-trend?market=&zip= (new, owner-gated)

-> components/property/ZipAreaTrend.tsx, rendered per property row in

MyPropertiesWorkspace.tsx keyed off the property's own postal code

### ## Why reuse the existing adapter/cron pattern

`lib/property/sources.ts` already runs four keyless US adapters (FHFA/FRED/BLS/HUD) on one weekly collector with a per-invocation fetch cache and a `property\_source\_runs` ledger. `ZillowZhviZipAdapter` follows the same `supportsMarket` / `fetch` shape and the same `PropertySourceUnavailableError` contract, so a Zillow outage or format change reports as a typed `unavailable`/`failed` run row – never a silent empty dataset. It rides the same cron rather than getting a new one because the system-map convention is to extend an existing scheduled-flow node when a source joins the flow it already belongs to, not fork a near-duplicate.

### ## Deliberately left out

- **No new cron.** Weekly polling of a monthly-updated source is the same pattern already used for BLS (monthly) and HUD (annual) on this cron.
- **No full ZHVI history.** Only the trailing 13 monthly columns are kept per run (enough for a 3M/12M trend); a longer chart is a future ask, not this one.
- **No streaming parser.** The adapter downloads the full ~120MB CSV via the existing `fetchText` cache and filters in memory. Ceiling: if this times out or exceeds function memory in production, upgrade to a streaming line-filter (flagged with a `ponytail:` comment at the call site) – not built preemptively.
- **No India/Bengaluru coverage.** Zillow does not publish ZHVI outside the US; `supportsMarket` returns false for `bengaluru`, matching every other US-only adapter's `not\_applicable` (not empty-success) reporting.
- **No AVM, no Zestimate, no blended "estimated value."** Explicitly out of scope by the honesty constraint above.

```

Source: features\python-runtime\FEATURE_ARCHITECTURE.md
Python Runtime — Feature Architecture

Status: **Implemented (measure-only). Not wired into any decision path.**
Shipped 2026-08-03. **Lane A retired from Vercel 2026-09-21** — see the note below; only Lane B (GitHub Actions) and the local `scripts/python`

1. Why

Kairos is TypeScript, but a large share of the statistics it needs already exists,
tested and peer-reviewed, in free Python libraries. Newey-West HAC standard
errors, Spearman correlation, the deflated Sharpe ratio and purged-fold machinery
are one `statsmodels` import away; in TypeScript they are hand-rolled.

`lib/edges/ic.ts` is the concrete case. It implements Spearman plus a
Bartlett-kernel Newey-West standard error by hand, and until now was only ever
checked against itself. A reference implementation is the difference between
"our IC t-stat is 2.1" and "our IC t-stat is 2.1 and we can prove the estimator
is right."

Standing constraint: **$0 cloud.** Both lanes below are free tiers already in use.

2. Two lanes, split by what each can survive

	Lane A	Lane B
Where	Vercel Python Function	GitHub Actions
Trigger	HTTP POST from TypeScript	Weekly cron / manual dispatch
Time budget	30s (`maxDuration`)	30 min (`timeout-minutes`)
Dependency budget	500MB bundle, cold-start sensitive	effectively unbounded
Use for	An answer needed *now*, modest deps	Backtests, ML, long rotations

Choosing between them is a latency question, not a taste question. If the caller
is waiting, Lane A. If nothing is waiting, Lane B — it has no timeout pressure
and no bundle limit.

> **RETIRED 2026-09-21.** The Vercel Python Function bundled statsmodels+scipy+numpy+pandas at 243.83 MB per deployment (measured in the build)

Lane A — `api/py/ic.py` (historical; now `scripts/python/ic.py`)

`POST /api/py/ic`. Spearman rank IC per cross-section, then a Newey-West HAC
standard error over the IC series.

- **Auth:** `x-cron-secret` compared against `CRON_SECRET` with
 `hmac.compare_digest`, **failing closed when `CRON_SECRET` is unset — the
 same contract as `verifyCronSecret` in `lib/auth/cron.ts`. Never anonymous.
- **Pure:** JSON in, numbers out. No database, no market data, no LLM.
- **Bounded:** `MAX_PERIODS` 2000, `MAX_OBS_PER_PERIOD` 5000, `MAX_BODY_BYTES`
 4MB, so one request cannot pin a serverless CPU.
- `newey_west_lag()` mirrors `neweyWestLag` in `lib/edges/evidence.ts` so the two
 runtimes pick the same lag from the same horizon/step.

Vercel endpoint convention: the class must be named `handler` and subclass
`BaseHTTPRequestHandler`. Each `.py` under `api/` becomes its own function.

Lane B — `scripts/python/lane_b_analysis.py` + `.github/workflows/lane-b-python.yml`

Weekly (`30 4 * * 0`) plus `workflow_dispatch`. Template for long-running work.

- `permissions: contents: read` — no write scope.
- `concurrency` group prevents overlapping runs.
- Self-check runs **before** the analysis; a broken build fails loudly first.
- **Dry-run by default.** With `SUPABASE_URL` / `SUPABASE_SERVICE_ROLE_KEY`
 absent it prints its payload and exits 0, so a missing secret cannot produce a
 half-write.

Secrets to add under *Settings → Secrets and variables → Actions* before it
writes anything: `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY` (secrets), and
`LANE_B_TABLE` (a repository *variable*, not a secret).

3. Verification

Both self-checks run locally and pass.

Lane A cross-checks the TypeScript implementation, which is the whole point:

python nw_se = 0.015654709709910813

```



ts\_reference\_nw\_se = 0.015654709709910807

Agreement to ~1e-17 over 60 periods at lag 4. `lib/edges/ic.ts`'s hand-rolled Bartlett-kernel Newey-West is correct.

Lane B self-check: OLS over 500 synthetic points, slope 0.0198, t 63.0, R<sup>2</sup> 0.889.

`npx tsc --noEmit` clean and `npm run build` clean \*\*with Python present in the tree\*\* – the primary integration risk (Python breaking the Next.js build) is cleared.

**\*\*Not verified:\*\*** actual Vercel deployment of the Python function. Cold start, real bundle size, and the deployed auth path are unmeasured. The first deploy is the test.

---

#### ## 4. Bundle and cold start

Python Vercel Functions include **\*\*all files reachable at build time\*\*** – there is no tree-shaking. `vercel.json` therefore scopes `api/py/\*.py` with `excludeFiles` covering `.next`, `node\_modules`, `app`, `components`, `lib`, `tests`, `docs`, `features`, `knowledge`, `public`, `supabase`, `scripts`, `types` and `backups`. Without that the Next.js tree ships inside the Python bundle against a 500MB limit.

`requirements.txt` is deliberately minimal – `statsmodels` transitively pulls numpy, scipy and pandas, which is already most of the budget. Every addition costs cold-start latency. Keep it short.

Python cold starts are slower than Node. Acceptable for crons and analysis; **\*\*not\*\*** acceptable on a page render.

---

#### ## 5. Boundaries

- **\*\*Measure-only.\*\*** Nothing here is wired into ResearchAgent, PaperTrader, PositionMonitor, the learner, the Router, or any broker path. No existing cron calls Lane A.
- No score, eligibility, sizing, entry, exit, promotion or broker behaviour is affected.
- Wiring any Python output into a money decision is a **\*\*separate proposal requiring owner approval\*\***. Cross-checking an estimator is not the same as letting it decide anything.
- Two dependency systems now coexist (`package.json` and `requirements.txt`). That is the ongoing cost of this approach and is accepted deliberately.

---

#### ## 6. Out of scope

Not built, and not to be added without a reason: a shared Python client library, a generic RPC layer, per-request package installs, ML model serving, or any Python on a user-facing render path.

---

# Source: features\quote-dispute-session-alignment\FEATURE\_ARCHITECTURE.md  
# Quote cross-check – session alignment and dispute escalation

> Status: **\*\*DRAFT – awaiting approval.\*\*** No code written.  
> Created: 2026-09-02  
> Scope: `app/api/agents/position-monitor/route.ts` mark cross-check. Money path  
> (a disputed symbol gets no stop, target or time-stop evaluation).

#### ## 1. The live symptom

`position-monitor-quote-disputed:india`, CRITICAL, open since 2026-08-26 and re-reported every run through 2026-09-01. Two open India positions – `INDUSTOWER.NS` (qty 27) and `KAMATHOTEL.NS` – have been reported `unpriced` on every PositionMonitor run for seven days.

Unpriced is not cosmetic. The route deletes a disputed symbol from `priceMap` **\*before\*** the exit loop, deliberately:

> "A price too doubtful to record is far too doubtful to sell on, so a disputed  
> symbol is removed from priceMap and reported UNPRICED – no stop, no target,  
> no time-stop fill."

That per-run judgement is correct. Its multi-run consequence is not: **\*\*two positions have been running with no exit evaluation at all for a week.\*\***

#### ## 2. Root cause: the comparison has no session alignment

The cross-check compares two numbers without requiring that they describe the same market session.

```
const bars = await fetchUpstoxCandles(sym, 5);

const newest = bars.length ? bars[bars.length - 1] : null;

if (newest && ...) { crossPrice[sym] = newest.close; } // date DISCARDED

...

const deltaPct = (Math.abs(live - cross) / cross) * 100;

if (deltaPct > MARK_DISPUTE_REFUSE_PCT) { /* refuse */ }

`fetchUpstoxCandles` parses `date` for every candle (`lib/data/upstox.ts:95`)
and the cross-check throws it away. The primary side has the same information
and also discards it: `fetchYahooQuote` sets
`retrievedAt = new Date(m.regularMarketTime * 1000)` – Yahoo's **exchange
timestamp for the quote**, not our fetch time (`lib/india-data.ts:62`).

Both sides already carry the session. Neither is checked.

Proof from production

`INDUSTOWER.NS`, alert text: `yahoo_india 375 vs upstox 388.8, 3.549%`.

session	Yahoo close (price_cache)
2026-09-01	**375**
2026-08-31	**388.79998779296875**
```

The "disagreeing vendor price" **is** Yahoo's own previous session, matching to four significant figures. The two vendors do not disagree about a price. They disagree about which day it is, and a one-session lag is being reported as a 3.5% vendor dispute – permanently, because the lag recurs every run.

This also violates the module's own stated contract: **"no cross price simply means the mark is recorded as uncorroborated, never as disputed."** A date-mismatched bar is precisely a cross price that cannot be used, so it should produce `uncorroborated`, not `disputed`.

### The second symbol is NOT the same bug

`KAMATHOTEL.NS``: `yahoo\_india 233.89 vs upstox 225.09, 3.910%`. Upstox's 225.09 matches the 2026-09-01 close of 225.05; Yahoo's 233.89 matches no recent close. Here the **primary** is the outlier and the cross is right – a genuine data problem, where refusing the price is the correct outcome.

So the two symbols in one alert have opposite causes. A fix that only aligns sessions would clear `INDUSTOWER` correctly and must still refuse `KAMATHOTEL`.

> Contributing factor, NOT bundled into this change: `isYahooQuoteStale` allows > an India quote to be up to **4 days** old and still count as fresh > (`lib/india-data.ts:47`). That tolerance makes a primary/cross session > mismatch likely by construction. Worth revisiting for exit evaluation > specifically, but it is a separate decision with its own blast radius.

## 3. Proposed change

1. **Carry the session on both sides.** Derive the primary's session date from `regularMarketTime` in exchange-local time (IST for India, ET for US), and keep the Upstox candle's own `date`.
2. **Compare only same-session prices.** Equal session dates → compare as today.
3. **Different sessions** → `uncorroborated`, not `disputed`. The mark is recorded, the position stays priced, and exits ARE evaluated. Emit a `warn` naming the lag and which side is behind – never a `critical`, and never a silent pass.
4. **Same session and delta** > `MARK_DISPUTE_REFUSE_PCT` → unchanged. Genuine disputes keep failing closed exactly as now. `KAMATHOTEL.NS` stays refused.
5. **Escalate persistence.** A dispute unresolved across N consecutive runs (proposed N = 3) raises a distinct alert stating that a position has been unguarded for N runs. An indefinitely unguarded position is a worse outcome than either vendor being wrong, and today nothing distinguishes "refused once" from "refused for a week".

Point 5 matters even if the session fix is perfect, because a genuine dispute can also persist – as `KAMATHOTEL.NS` may.

### Explicitly NOT proposed

- Loosening `MARK_DISPUTE_REFUSE_PCT`. The threshold is not the problem; comparing different days is.
- Auto-selecting a "winner" vendor. Choosing which feed to trust on a genuine

disagreement is a policy decision, not a bug fix.  
- Changing the 4-day India quote-staleness tolerance (see §2 note).

#### ## 4. Falsification

The fix is wrong if, after it ships, `INDUSTOWER.NS`-shaped cases (cross price equal to the primary's \*previous\* close) still raise `disputed`, or if a genuine same-session disagreement stops being refused. Both are directly testable against the recorded alert values.

#### ## 5. Blast radius

Only the mark cross-check. No change to scoring, sizing, entry, order routing, or the exit RULES themselves – the change is about which prices are allowed to reach the existing exit evaluation. Affects US and India, since the cross-check is shared.

#### ## 6. Same-session coverage follow-up (2026-09-03)

The session gate fixed the safety defect but exposed a coverage defect: the monitor still asked Upstox's historical daily-candle endpoint for the second opinion. That settled series can legitimately remain on the prior session after the close. The monitor now batches the same symbols through Upstox's exchange market-quote endpoint, using `last\_price` and `last\_trade\_time`; an unknown timestamp remains explicitly uncorroborated. This is the existing second provider on the existing read-only analytics token, not a new vendor or money-path authority.

---

# Source: features\relationship-graph\FEATURE\_ARCHITECTURE.md  
# Relationship Graph and Cross-Company Propagation - Feature Architecture

> Status: \*\*P0 feasibility study RUN AND FAILED. Recommendation: do not build. Design retained for the record.\*\*  
> Last updated: 2026-07-16 – P0 produced a provisional KILL; enhanced reproducibility rerun is required before final sign-off. See [P0\_COVERAGE\_STUDY].  
> Previous revision: 2026-07-15 by Codex after primary-source and Kairos architecture review.  
> Scope: design only. No migration, provider activation, scoring change, candidate insertion, or order path is authorized.  
> Update this file when: source contracts, identity resolution, relationship schema, EdgeScout integration, validation gates, or money-path become relevant.

#### ## 0. P0 outcome (2026-07-16) – measured, not estimated

The P0 study §11 authorized was run against live SEC EDGAR over Kairos's real US universe (92 US non-ETF symbols → \*\*79 domestic 10-K filers\*\*). Full report and method: [P0\_COVERAGE\_STUDY.md](./P0\_COVERAGE\_STUDY.md). Probe: `scripts/sec-customer-coverage-probe.mjs`.

```
P0 exit question (§11)	Measured result
Named-customer coverage **with a revenue %**	**5 / 79 = 6.3%** (CHD, SWK, AMT, CELH, CRWV – 10 links total)
Named, but no usable exposure %	2 / 79 (QCOM ≥10% floor only; BX $-denominated, non-traded funds)
Identity resolution on those 10 strings	**60% resolved**, 20% ambiguous, 20% unresolved; a silent false positive (BCRED→BX) appeared immediately
Point-in-time `available_at`	**Works.** `acceptanceDateTime` on 79/79, intraday-precise; amendments carry their own timestamps, so as-of
Freshness	exposure weight is **~6.5 months stale at median**, up to 381 days; refreshes annually
Cost	**$0**, 167 requests, ~3 min, ~169 MB/year raw
```

\*\*Answer to §16 open decision 1 ("Is free disclosed-customer coverage high enough to justify P1?"): No.\*\*

The single riskiest assumption named in §17 – whether Kairos can obtain enough point-in-time, correctly resolved, economically weighted public relationships for its actual US universe – \*\*failed on measurement\*\*. US GAAP requires the \*amount\* of a ≥10% customer, not the \*name\*, and issuers overwhelmingly take the anonymity option on exactly the highest-exposure links (NVDA 22%, ENPH 39%, AMAT 19%, MU 17%, INTC 43%; FFIV/FSLR/RGTI literally letter or number their customers). §5.3's caution was correct and is now evidenced.

\*\*Recommendation: do not build P1-P5.\*\* n=5 cannot support the §9.1 cross-sectional experiment or the §9.2 evaluation battery; it cannot even reach EdgeIC's `nObs >= 12` preliminary diagnostic. Per §17, retain the peer-move display and stop. Revisit only if the covered universe reaches ~500+ US issuers (the falsifiable precondition derived in the study); re-running the probe then costs three minutes.

#### ## 1. Review verdict

The product idea is valid: economically linked firms can transmit information slowly, and customer returns have historically predicted supplier returns. The previous V3 document was not build-ready, however. It made five unsafe leaps:

1. It stored a bullish/bearish `direction` on a relationship. A supplier/customer/competitor relationship is not intrinsically bullish or bearish; an event has a sign and a relation determines how that event may transmit.
2. It proposed a second provenance and learning system instead of reusing Kairos's Canonical Evidence Router, `evidence\_records`, `edge\_\*` lab, research queue, and decision ledger.
3. It treated Finnhub peers as economic links. Similar-company peers are display context, not verified customer, supplier, or competitor relationships.
4. It overstated free data coverage. Public filings do not guarantee that every major customer is named, and FinancialDatasets segmented financials describe product/business segments, not a complete customer-supplier graph.
5. It elevated two very recent arXiv preprints into a production recommendation. They are useful hypotheses, not sufficient evidence to ship embedding propagation into Kairos.

\*\*Correct product decision:\*\* do not build a general LLM relationship graph first. Start with a

narrow, falsifiable, US-only, measure-only **\*\*disclosed customer momentum\*\*** experiment inside the existing Edge/Factor lab. Add graph extraction, event propagation, candidate discovery, or model embeddings only after each earlier layer proves coverage and out-of-sample incremental value.

## ## 2. Product purpose

The feature should eventually answer four distinct questions without presenting speculation as fact:

1. **\*\*Relationship:\*\*** What verified economic relationship exists between two issuers?
2. **\*\*Exposure:\*\*** How economically important is that relationship, and to which party?
3. **\*\*Event:\*\*** What point-in-time event or return shock occurred at the leading issuer?
4. **\*\*Measured implication:\*\*** Has Kairos observed that this relation/event type predicts the linked issuer in the target market and horizon after costs?

The user-facing output must keep those layers separate. A relationship card may say:

```
> Supplier A disclosed Customer B as 18% of revenue in its 10-K filed on date D.
> Customer B rose 7% over the last 20 sessions. Kairos is measuring whether similar links
> predict Supplier A; this is not a recommendation.
```

It must not say "Supplier A will rise" merely because an LLM extracted a link.

## ## 3. Non-negotiable boundaries

- **\*\*No LLM on the money path.\*\*** An LLM may propose an entity/link assertion from bounded, sanitized filing text. It cannot set a score, direction, threshold, position size, order, long suppression, or exit.
- **\*\*No relationship output affects trading in P0-P3.\*\*** It is display-only or measure-only.
- **\*\*No direct hedge integration.\*\*** The governed downside hedge is a portfolio-level US market overlay. A bearish company relationship must never activate or size `SH`/`PSQ`.
- **\*\*No automatic shorting.\*\*** Negative observations remain research evidence. Kairos remains long-only for new single-name positions.
- **\*\*No competitor or peer inference from a provider list.\*\*** `symbol\_profiles.peers` and the shipped peer-move strip remain context-only and outside this graph.
- **\*\*No parallel provenance store.\*\*** Provider acquisition uses the Canonical Evidence Router; decision evidence links to existing evidence and provider ledgers.
- **\*\*No auto-promotion.\*\*** Passing an EdgeIC measurement does not alter the Champion, scorer, P(win), sizing, candidate priority, or live eligibility. Any use requires a separately approved, validated Challenger or feature-registry change.
- **\*\*Per-target-market activation.\*\*** US and India measurements, validation, configuration, and promotion remain independent. Currency amounts are never cross-summed.
- **\*\*Fail closed on ambiguity.\*\*** Unresolved issuer identity, missing source availability time, stale relationships, missing exposure basis, or conflicting assertions produce no signal.

## ## 4. Existing Kairos systems to reuse

This feature extends existing architecture; it must not replace it.

```
Concern	Canonical Kairos owner	Required integration
Provider choice, pacing, caching, source provenance	`lib/evidence/*`, `EvidenceEnvelope`, evidence policy/cache/call ledger	Add relationship evidence
Decision evidence	`evidence_records`, append-only `decision_observations`	Store IDs/ hashes that identify the exact inputs known at decision
Factor measurement	`edge_catalog`, `edge_signals`, `edge_signal_inputs`, `edge_ic_history`	Register graph-derived measurements as versioned
Candidate backlog	`research_queue` and ResearchAgent discovery attribution	Only a later approved phase may enqueue a linked issuer; no pre-emption
Outcome labels	`decision_observations` x `observation_labels`	Use for candidate-source evaluation only after candidate discovery exists
Strategy governance	Feature Registry, Validation Engine, Shadow, Champion/Challenger	Required before any measured edge can influence a score
Display-only peers	`symbol_profiles.peers`, peer-move MVP	Remains separate and cannot seed verified graph edges
```

The current `DiscoverySource` union does not include `relationship\_graph`. That is intentional. It is added only when P3 candidate-shadow architecture is separately approved.

## ## 5. What the research supports - and does not support

### ### 5.1 Peer-reviewed prior: customer momentum

Cohen and Frazzini (2008), *"Economic Links and Predictable Returns"*, supports a narrow claim: publicly disclosed customer returns predicted subsequent supplier returns in their historical sample. Their customer return was sales-weighted, their portfolios were formed monthly, and the reported drift persisted beyond the initial customer shock.

This supports testing a **\*\*sales-weighted customer-return factor\*\***. It does not prove that:

- every partnership, competitor, or co-mention edge predicts returns;
- an arbitrary 8-K has a mechanically knowable sign for every neighbor;
- a 1-day or 5-day Kairos implementation retains the historical edge after costs;
- the effect works unchanged in today's large-cap universe or in India; or
- an LLM-extracted graph is superior to disclosed customer links.

Primary source: [Cohen and Frazzini, Journal of Finance (2008)](<https://pages.stern.nyu.edu/~afrazzin/pdf/Economic%20Links%20and%20Predictable%20Returns.pdf>)

### ### 5.2 Recent preprints: hypothesis grade only

- [Supply Chain Propagation of Textual Signals (arXiv:2606.29290)](<https://arxiv.org/abs/2606.29290>) reports promising results for FinBERT/graph-propagated 10-K features, but it is a June 2026 preprint.

single-author preprint using 255 S&P 500 firms over 2011-2025. It is not an independently replicated production standard.

- [Cross-Stock Predictability via LLM-Augmented Semantic Networks (arXiv:2604.19476)](<https://arxiv.org/abs/2604.19476>) reports that LLM edge filtering improved a historical long-short result on S&P 500 constituents from 2011-2019. The abstract does not establish point-in-time constituent handling, implementation costs, or suitability for Kairos's long-only books.

Therefore embedding propagation is an **offline research candidate**, not the P1 implementation. It must prove incremental out-of-sample value over the simple disclosed-customer baseline after costs and multiple-testing correction.

### ### 5.3 Disclosure reality

SEC EDGAR is free and official, and current filing metadata/XBRL APIs require no API key. SEC fair-access guidance currently caps automated access at 10 requests/second and requires a declared user agent. That is a ceiling, not a service-level guarantee or an unlimited quota.

Major-customer disclosures are incomplete for graph construction:

- US GAAP can require disclosure that a customer contributes 10% or more of revenue and the amount/segment, but customer identity is not always required.
- Regulation S-K was modernized in 2020 into a more principles-based business-description rule. Kairos must not assume every 10% customer is named.
- FinancialDatasets `segmented\_financials` exposes product/business segment revenue. It is not proof of customer identity or customer-level revenue exposure.

Source contracts are therefore capability-probed and coverage-measured before architecture approval. Missing identity/exposure is `unavailable`, never guessed.

Primary sources: [SEC EDGAR APIs](<https://www.sec.gov/search-filings/edgar-application-programming-interfaces>), [SEC developer fair-access guidance](<https://www.sec.gov/about/developer-resources>), and [SEC 2020 Regulation S-K modernization](<https://www.sec.gov/rules-regulations/2020/08/modernization-regulation-s-k-items-101-103-105>).

## ## 6. Correct conceptual model

### ### 6.1 Issuers are not ticker symbols

Graph nodes represent legal issuers, not ticker text. A separate point-in-time mapping connects an issuer to one or more tradable instruments. This prevents symbol changes, share classes, ADRs, cross-listings, delistings, and duplicate economic exposure from corrupting the graph.

Minimum identity keys:

- US issuer: CIK plus normalized legal name; LEI when available.
- India issuer: reviewed exchange/company identifier from an approved NSE/BSE/SEBI source.
- Tradable instrument: `(market, exchange, symbol, currency, valid\_from, valid\_to, is\_primary)`.

An LLM may return a name mention. A deterministic resolver must map it to exactly one issuer and instrument. Zero or multiple matches are quarantined.

### ### 6.2 Relationships are neutral assertions

A durable assertion describes an economic fact:

- supplier supplies customer;
- customer buys from supplier;
- parties have a material partnership/JV; or
- an issuer disclosed material dependence on another issuer.

The assertion stores no bullish/bearish direction. It separately records:

- relation type and orientation;
- exposure percentage and whose revenue/cost basis it describes;
- extraction quality and verification state;
- economic-validity interval (`valid\_from`, `valid\_to`); and
- knowledge-time fields (`source\_published\_at`, `available\_at`, `retrieved\_at`).

### ### 6.3 Events and propagation observations are separate

An event or leading-company return shock is a time-stamped observation. A propagation measurement joins an event/return to relationships that were known and economically valid **before** the target decision timestamp.

Conceptually:

leading input (customer return or validated event)

x economic exposure

x relationship verification quality

x age/validity rule

= measure-only propagation raw value

This is not an analyst score. Cross-sectional normalization and forward-return evaluation remain in the Edge/Factor lab.

## ## 7. Proposed data architecture

These tables are proposals only. A future implementation requires an approved migration plan and live-schema verification.

### ### 7.1 `issuer\_entities`

Stable legal entities.

| Column                           | Purpose                              |
|----------------------------------|--------------------------------------|
| `issuer_id`                      | UUID primary key                     |
| `legal_name`, `country`          | Reviewed identity                    |
| `cik`, `lei`, `india_company_id` | Nullable unique external identifiers |
| `identity_status`                | `verified`, `ambiguous`, `retired`   |
| `created_at`                     | System time                          |

### ### 7.2 `issuer\_instruments`

Bitemporal issuer-to-security map.

| Column                                     | Purpose                                                |
|--------------------------------------------|--------------------------------------------------------|
| `issuer_id`                                | FK to issuer                                           |
| `market`, `exchange`, `symbol`, `currency` | Tradable identity; market and currency always explicit |
| `valid_from`, `valid_to`                   | When the mapping was economically true                 |
| `available_at`, `retrieved_at`             | When Kairos could know it                              |
| `is_primary`                               | Prevent duplicate ADR/share-class candidate emission   |

### ### 7.3 `relationship\_assertions`

Append-only facts, not mutable current-state rows.

| Column                                                           | Purpose                                                                                              |
|------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| `assertion_id`                                                   | UUID primary key                                                                                     |
| `subject_issuer_id`, `object_issuer_id`                          | Oriented issuer pair                                                                                 |
| `relation_type`                                                  | Narrow grammatical enum; start with `supplier_to_customer` (subject is supplier, object is customer) |
| `exposure_pct`, `exposure_basis`, `exposure_period_end`          | Nullable; never mix revenue and cost exposure                                                        |
| `valid_from`, `valid_to`                                         | Economic validity interval                                                                           |
| `source_published_at`, `available_at`, `retrieved_at`            | Point-in-time availability                                                                           |
| `evidence_record_id`, `router_policy_version_id`, `payload_hash` | Links canonical Kairos provenance; no ad hoc `evidence_ref` blob                                     |
| `source_locator`, `source_span_hash`                             | Accession/document/section and hash of the cited bounded span                                        |
| `extractor_version`, `schema_version`                            | Reproducibility                                                                                      |
| `assertion_action`                                               | `assert` or `retract`; an accepted retraction ends a prior assertion without mutating it             |
| `verification_status`                                            | `proposed`, `accepted`, `rejected`                                                                   |
| `supersedes_assertion_id`                                        | Corrections add rows; UPDATE/DELETE remains blocked                                                  |
| `created_at`                                                     | System time                                                                                          |

A deterministic current/as-of view resolves the accepted assert/retract supersession chain. An old row remains historically accepted; the later accepted row determines current state without rewriting history. Raw LLM confidence is not stored as economic truth. Verification confidence is derived from objective checks such as identity uniqueness, source class, citation-span match, and numeric exposure extraction.

### ### 7.4 Reuse `edge\_\*` tables for measurements

Do not add a `propagation\_signals` truth layer. Register, for example:

edge\_id: customer\_momentum\_sales\_weighted\_v1

input: trailing customer return known at t + accepted customer relationship known at t

raw\_value: sum(exposure\_weight\_j \* customer\_return\_j)

expected\_sign: +1

default lookback: 20 trading sessions

evaluation horizons: 5, 10, 20 trading sessions

Write values to `edge\_signals`; write relationship assertion IDs, price evidence references, availability timestamps, and adjustment policy to `edge\_signal\_inputs`; write evaluation to `edge\_ic\_history`. A missing exposure produces unavailable for the sales-weighted edge. An unweighted variant, if tested, is a separate edge ID and therefore a separate statistical trial.

## ## 8. Acquisition and extraction pipeline

### ### 8.1 Router-owned acquisition

Before build, define and review narrow Router intents such as:

- `filings.document`
- `relationships.disclosed`

The Router owns provider policy, cache, pacing, freshness, payload hashes, capability status, and call ledger. SEC should be the initial authoritative US source. FinancialDatasets may be a policy option only after a live capability/entitlement probe proves the required customer-level contract.

### ### 8.2 Untrusted-document isolation

Filings and news are untrusted text even when hosted by an official source.

- Strip scripts, active content, hidden markup, and irrelevant exhibits.
- Bound bytes, tokens, nesting, entities, and output tuples.
- The extraction model gets no tools, network, shell, credentials, memory writes, or order APIs.
- Treat instructions inside documents as data, never system instructions.
- Require strict schema validation, an exact source locator, and a supporting span hash.
- Reject unknown relation enums, self-links, unresolved entities, future timestamps, invalid percentages, and output not grounded in the supplied span.
- Store model/prompt/schema versions for reproducibility.

### ### 8.3 Verification state machine

raw document

-> proposed assertion (untrusted extraction)

-> deterministic identity/source/numeric checks

-> accepted OR rejected

-> later filing may append a superseding assertion

Only accepted assertions enter display or EdgeScout. P0 should include owner review because the covered universe is small and the cost of a false link is high. Automation can be reconsidered after precision is measured against a labeled fixture set.

## ## 9. Statistical and trading correctness

### ### 9.1 Pre-register the first experiment

The first trial is only `customer\_momentum\_sales\_weighted\_v1`:

- US primary common equities only;
- accepted disclosed customer relationships only;
- relationship and instrument mapping available before the as-of timestamp;
- 20-session customer return, evaluated at 5/10/20-session supplier horizons;
- benchmark-neutral and raw target returns both retained;
- split/dividend-adjusted prices with explicit availability and adjustment policy;
- no parameter search selecting the best lookback after seeing results.

Other relation types, unweighted links, event semantics, embeddings, neutralization schemes, and horizons are separate registered trials. They increase the trial-family count used by DSR/PBO/FDR governance; they are not silent variants of one factor.

### ### 9.2 Long-only decision metric

Cohen-Frazzini and both recent preprints emphasize long-short portfolios. Kairos is long-only for new names, so a long-short Sharpe cannot authorize adoption. Required evaluation includes:

- top-bucket long-only benchmark-relative return after modeled costs;
- hit rate, drawdown, turnover, capacity, and sector concentration;
- rank IC/ICIR with overlapping-horizon correction;
- stability across time, sectors, issuer size, and relationship age;
- comparison with ordinary momentum to prove incremental value; and
- explicit unavailable/coverage rates so sparse disclosure is not mistaken for selectivity.

The existing EdgeIC `nObs >= 12` classifier is a preliminary measure-only diagnostic, not a production promotion gate for this new sparse/data-mined family. Promotion must use the canonical Edge/Factor plus Advanced Learning governance and cannot be weaker than its multiple-testing, walk-forward, sample-floor, cost, and owner-approval rules.

### ### 9.3 No immediate candidate or score effect

Even a promising measurement remains in `edge\_\*`. To affect ResearchAgent later, a separate design must choose exactly one governed route:

1. **Discovery route:** enqueue a linked issuer for normal deterministic scoring, with no score bonus; or
2. **Feature route:** log a normalized relationship feature in the point-in-time decision snapshot, then promote it only through Feature Registry + Challenger validation.

It must not do both in one experiment, because selection and score effects would become inseparable. Long-candidate suppression is also a money-path change and requires the same gates.

## ## 10. Market and session isolation

### ### Initial scope

- P0-P2 are US-only. India is `not\_applicable`, not silently empty.
- India requires a separately validated official filing/entity source and its own coverage study.
- Yahoo/Finnhub peer lists cannot substitute for Indian economic relationships.

### ### Cross-market relationships

The graph may eventually represent a real US-customer/India-supplier relationship, but it does not cross-sum books or currencies. The propagation observation belongs to the **target instrument's** market, currency, mandate, benchmark, calendar, and owner controls.

Cross-market evaluation must prove `available\_at <= target\_decision\_ts` using both exchange calendars and time zones. An event released after the target market closed cannot be treated as known for that session. ADR and local shares are one issuer with multiple instruments; candidate deduplication selects the configured primary instrument per market.

## ## 11. Phased build order and rollback

### ### P0 - source and identity feasibility, display fixtures only

1. Live-probe SEC and any optional provider contract on 25-50 representative US issuers.
2. Measure named-customer coverage, exposure coverage, entity-match precision, and conflicting disclosure rate.
3. Build a reviewed fixture set with positive, absent, ambiguous, superseded, and prompt-injection cases.
4. Exit criterion: documented coverage plus high-precision entity/assertion verification. If the free data is too sparse, stop. Sparse is an acceptable result.

No graph table, candidate, score, or LLM cron is required merely to run the feasibility probe.

### ### P1 - accepted relationship ledger and read-only display

Add issuer/instrument/assertion storage behind `relationship\_graph\_enabled=false`. Display only accepted disclosed customer links with source, exposure basis, age, and "not a recommendation". No competitor seeding and no propagation.

### ### P2 - customer momentum in the existing Edge lab

Compute `customer\_momentum\_sales\_weighted\_v1` into `edge\_\*`, measure-only. The Edge dashboard shows coverage and caveats. It cannot write `agent\_signals`, `research\_queue`, `decision\_observations`, paper positions, proposals, or orders.

### ### P3 - candidate shadow, only after P2 evidence

If P2 passes its pre-registered tests, design a separate candidate-discovery shadow. It records which target names would have been added, but does not add them to ResearchAgent. Compare their normal deterministic outcomes with the existing universe.

### ### P4 - separately approved paper integration

Only after P3 prospective evidence and owner approval may one route from Section 9.3 become a validated Challenger. Start US paper-only and OFF. Live remains a later, separate decision.

### ### P5 - optional embedding/LLM network experiment

Run only as an isolated offline experiment after the simple baseline exists. It must beat the baseline out of sample after costs and trial correction. Model artifacts are pinned and hashed; provider/model updates create a new trial, never silently replace the old one.

### ### Disable/rollback

Each phase has an independent per-market flag. Disabling stops new acquisition, extraction, measurement, or candidate shadow while preserving immutable evidence and results. A rollback never deletes or rewrites history.

## ## 12. Security, operational, and cost controls

- RLS on every new public table; no `anon` access; owner read only where UI needs it; service-role writes; fixed `search\_path` and service-role-only grants for any SECURITY DEFINER RPC.
- Append-only assertion and evaluation ledgers reject UPDATE/DELETE. Corrections append a superseding row.
- Bounded extraction batch, wall-clock limit, per-document size cap, retry cap, dead-letter state, and health alert aggregation.
- SEC requests use a declared contact user agent, shared pacing, conditional caching, and the Router ledger. Stay materially below the SEC's maximum access rate.
- No source is described as unlimited. Unknown quota/entitlement remains unknown.
- No raw filing body, model prompt, provider error, URL query credential, token, or service key is written to logs.
- Data retention distinguishes small immutable source-span hashes from large cached filing bodies; large bodies receive an explicit TTL and storage budget.
- Provider, parser, prompt, model, schema, and policy versions are pinned in every extraction run.

## ## 13. Product UX requirements

The eventual UI is evidence-first and novice-readable:



- **Verified relationship:** issuer names, relation orientation, exposure and basis if known.
- **Source and age:** filing type, accession/link, filed date, relationship validity.
- **What happened:** leading return/event and the exact as-of time.
- **System posture:** `display only`, `measuring`, `shadow candidate`, or later governed state.
- **Confidence honesty:** use `verified`, `ambiguous`, `stale`, `conflicting`, or `unavailable`; never render an LLM's self-assigned confidence percentage.
- **No recommendation language** before an approved integration phase.

The UI must not collapse "peer", "supplier", "customer", "partner", and "competitor" into one generic related-stock list.

#### ## 14. Required tests for any future build

##### ### Identity and temporal correctness

- Symbol rename/share-class/ADR mappings resolve to the correct issuer as of date.
- Ambiguous company names and private/unlisted entities quarantine rather than guess.
- Assertions unavailable at the target decision time cannot enter a signal.
- Superseded relationships remain historically queryable but disappear from later as-of views.
- Cross-market session/time-zone tests prevent look-ahead.

##### ### Extraction and security

- Prompt instructions embedded in filing text cannot alter schema or invoke tools.
- Output without an exact source span, valid locator, and unique entity match is rejected.
- Percentages above 100, mixed exposure bases, self-links, unknown enums, and future dates reject.
- Parser/model/version changes create distinct artifacts and do not rewrite assertions.

##### ### Statistical honesty

- Sales-weighted formula matches a fixed fixture exactly.
- Missing exposure yields unavailable rather than zero/equal-weight substitution.
- Each alternate parameter/model is counted as a distinct trial.
- Point-in-time universe, price adjustment, costs, and benchmark are reproducible.
- Long-only results are reported alongside IC/long-short diagnostics.
- Ordinary momentum comparison identifies whether the graph adds incremental information.

##### ### Boundary tests

- P0-P2 imports cannot reach `agent\_signals`, `research\_queue`, PaperTrader, PositionMonitor, TraderAgent, broker adapters, hedge controller, or execution gateway.
- Relationship flags default OFF independently by market and phase.
- A disabled or unavailable provider produces no fabricated relationship or signal.
- No graph value changes scoring, entry eligibility, P(win), sizing, exit, or orders without a separately approved and validated strategy version.

#### ## 15. Acceptance criteria for architecture approval

Before implementation is approved, the owner/reviewer should require:

1. P0 coverage evidence from live free sources, not provider marketing or memory.
2. A canonical Router intent/adaptor contract and point-in-time provenance mapping.
3. A reviewed issuer/entity-resolution contract.
4. A fixed initial edge specification and trial-family registration.
5. Explicit proof that the build extends `edge\_\*` and existing evidence ledgers.
6. Separate US and India applicability; India may remain unsupported.
7. A no-money-path import test and flags OFF by default.
8. A cost/storage/cadence estimate that fits current free-cloud budgets.

#### ## 16. Open decisions

1. --Is free disclosed-customer coverage high enough to justify P1?-- **ANSWERED 2026-07-16:** No – 5/79 filers (6.3%) yield a named customer with a revenue share. See §0 and `P0\_COVERAGE\_STUDY.md`. Decisions 2-6 below are now moot unless the revisit precondition (~500+ covered US issuers) is met.
2. Should accepted assertions require owner review indefinitely, or only until fixture precision clears a pre-registered threshold?
3. Should the first later integration be candidate discovery or a logged feature? Do not combine both.
4. What minimum prospective calendar span and independent cross-sections are required beyond the current preliminary EdgeIC classifier before P3?
5. Which official India filing/entity source can provide a lawful, stable, point-in-time contract?
6. Where should bounded extraction run? Prefer the existing trusted research cron for reviewed filing extraction; use an isolated worker only if untrusted code/models are introduced.

#### ## 17. Recommended decision now

Approve only **P0 feasibility research**, not implementation of the graph. Do not prioritize this ahead of the Canonical Evidence Router cutover and first trustworthy learning-loop run. If P0 later shows sparse named-customer coverage, retain the peer-move display and stop; Kairos should not build a large graph whose apparent precision comes from guessed relationships.

The single riskiest assumption is not whether customer momentum once existed. It is whether Kairos can obtain enough **point-in-time, correctly resolved, economically weighted public relationships** for its actual US universe to measure that anomaly honestly and cheaply today.

---

```

Source: features\research-journal-controls\FEATURE_ARCHITECTURE.md
Feature: Research Journal Controls and Market-Local Clocks

Status: SHIPPED
Owner: Vaibhav
Date: 2026-07-21

Intent

- Score Tracker must support clearing all selected symbols.
- Its existing point filters must also identify matching symbols across the bounded candidate universe, with explicit commands to select or add those matches to the chart.
- Daily Funnel must support one-date and all-date views plus a symbol filter.
- The persistent US and India status pills must show each market's current local time in addition to the existing session state and open/close schedule.

Contracts

Score Tracker

- Candidate symbols remain the current market's watchlist/holdings/custom set.
- Candidate discovery and persisted chart selections are market-scoped. The US live-portfolio endpoint is never used to populate India's picker.
- Filter matching queries `signal_score_history` through the existing owner-only endpoint, in chunks of at most 50 symbols. It does not call a provider.
- A symbol matches when at least one stored score point satisfies the current period, score-band, direction, source, and date filters.
- Active filters narrow the displayed symbol chips. They never silently change chart selection. `Select filtered` replaces selection, `Add filtered` unions matches into selection, and `Clear selection` removes every chart line.
- The global market switcher remains the sole US/India authority.

Daily Funnel

- Default remains one market-local date and the latest decision per symbol for that date.
- `All dates` returns at most the 250 newest immutable observations for the selected market, or for one validated symbol when filtered. The response marks truncation honestly; it never claims unbounded completeness.
- Symbol filtering is server-side and exact after trim/uppercase normalization.
- Each all-date card is keyed by observation ID and shows its market-local date, so repeated observations for one symbol remain distinct and expandable.
- Existing stage, paper/live position, outcome, evidence, and terminal-state joins remain display-only.

Market Header

- US displays current `ET`; India displays current `IST`.
- Existing status label and schedule detail remain visible.
- Time updates every 30 seconds and uses IANA time zones; no fixed UTC offsets.

Non-Goals

- No score recalculation, signal selection, provider call, order, portfolio, or risk behavior change.
- No cross-market chart or journal result.
- No unbounded historical read or infinite-scroll build in this phase.
- No new database table, migration, package, or scheduled job.

Acceptance

- Clear selection produces an empty chart state.
- Filter matching is computed over candidates that were not previously selected.
- Select/add filtered affects only the current market.
- All-date Daily Funnel can show multiple observations for one symbol and states the 250-row bound.
- Desktop and 390px mobile layouts contain all controls without overlap.

Source: features\research-journal\FEATURE_ARCHITECTURE.md
Feature: Research Journal (daily funnel report + learning evolution)

Status: BUILT (v1 shipped 2026-07-06; v2 novice-first upgrade approved 2026-07-12) · Owner: Vaibhav · Started 2026-07-06

Motivation

Today, "why did the agent do/not do X" requires manually querying 4+ tables (`decision_observations`, `trade_proposals`, `paper_trades`, `learner_runs`) and cross-referencing by hand – no single place shows the full chain for a symbol on a given day, and no single place shows whether the learning loop is actually improving over weeks/months. `app/dashboard/agents/history` exists but never reads `decision_observations` – it's unrelated to this.

The ask: a daily report showing, per symbol scored that day – which

```

screeener bucket/criteria flagged it, its full score breakdown, why it passed or failed at each pipeline stage (research threshold → portfolio constructor → trade execution), and a separate longer-horizon view of how the screener, LearnerAgent, and feature registry are evolving.

## What already exists (verified against live schema, 2026-07-06)

`decision\_observations` (Phase 1 ledger) already logs, per symbol per research run: `analyst\_score`, `score\_threshold`, `entry\_eligible`, sub-scores (fundamental/technical/sentiment/macro/insider), and a `features` jsonb blob that already contains honest per-dimension notes (e.g. `"no macro data"`, `"ETF — no company fundamentals"`). This is real, live data — verified via a sample query (IAU/GDX/SLV, 2026-07-06: score 52 < threshold 60 → correctly rejected, with the exact reason visible in `features`).

## Gaps (why this can't be built as a pure read-only aggregation today)

1. **\*\*No screener-stage tag.\*\*** `decision\_observations.features` has no `bucket` (momentum/value) or `criteria\_matched` field. ResearchAgent's dual-bucket scoring runs today but doesn't record which bucket/rule flagged a candidate.
2. **\*\*No per-symbol downstream trail.\*\*** Once a candidate clears the research threshold, Portfolio Constructor's shrink/reject decision and the trade-proposal/fill outcome exist in separate tables with no shared key back to the originating `decision\_observations` row beyond `signal\_id` (which exists on `decision\_observations` but isn't written-to / consistently joined by the downstream stages today).
3. **\*\*No feature-registry status-history log.\*\*** `feature\_registry` has a current `status` (proposed/quarantined/active/retired) but no history of *when* it changed — the Evolution tab needs a timeline, not a snapshot.

## Proposed architecture

### Principle: instrument at the source, don't reconstruct after the fact

Reconstructing "why" from final-state tables is lossy (a proposal that got rejected by Portfolio Constructor before ever becoming a `trade\_proposals` row leaves no trace today). Instead, write one small journal row at each stage transition, all keyed by `signal\_id`, then join for display.

### Schema changes

**\*\*A. `decision\_observations.features` — add two fields at write time\*\***  
(`lib/research-agent.ts`, no migration needed — same jsonb column):

```
features.screener = { bucket: "momentum" | "value", criteria_matched: string[] }
```

Populated from the existing dual-bucket scoring logic in `lib/research-agent.ts` (the bucket assignment already happens in code — this just records which one won and why, instead of discarding it).

**\*\*B. New table `pipeline\_stage\_events`\*\*** (append-only, mirrors `decision\_journal`'s generic shape but scoped to this funnel so queries stay cheap and don't compete with the general journal):

```
create table pipeline_stage_events (
```

```
id bigserial primary key,
```

```
signal_id uuid not null,
```

```
symbol text not null,
```

```
market text not null default 'us',
```

```
stage text not null, -- 'research' | 'portfolio_constructor' | 'proposal' | 'execution'
```

```
outcome text not null, -- 'passed' | 'rejected' | 'shrunk' | 'filled' | 'expired'
```

```
reason text, -- human-readable, e.g. "sector cap: 3/3 Tech positions already held"
```

```
detail jsonb, -- stage-specific numbers (e.g. shrink %, correlation haircut applied)
```

```
created_at timestamptz not null default now()
```

```
);
```

Written by: `lib/research-agent.ts` (stage='research'), the Portfolio Constructor call inside `app/api/agents/paper-trade/route.ts` (stage='portfolio\_constructor'), the trade-proposal/approval flow (stage='proposal'), and `execute\_paper\_fill`/broker-order paths (stage='execution'). Each write is one extra `insert` at a point where the decision is already being made — no new decision logic, just recording the

existing one.

**\*\*C. `feature\_registry\_history`\*\*** (small, append-only): `id, feature\_id, from\_status, to\_status, reason, created\_at` – written wherever `feature\_registry.status` is currently updated (feature-check route, learner's `propose\_feature` tool).

### API

- `GET /api/agents/research-journal?date=YYYY-MM-DD&market=us` – for each symbol with a `decision\_observations` row that day, join `pipeline\_stage\_events` (by `signal\_id`) into an ordered stage list, plus the terminal state (rejected-at-research / rejected-at-constructor / proposed-pending / filled / expired).
- `GET /api/agents/research-journal/evolution?market=us&days=90` – time series: LearnerAgent weight deltas (`learner\_runs`), feature-registry status transitions (`feature\_registry\_history`), calibration drift (`model\_artifacts`), shadow-decision agreement rate (`shadow\_decisions` vs realized champion decisions).

### UI – new page `/dashboard/research-journal`

Two tabs, matching the existing dark-theme card/pill conventions:

- **\*\*Daily Funnel\*\*** – date picker (default today), per-symbol expandable rows: score breakdown → screener bucket/criteria → stage-by-stage pass/reject with reason → terminal badge. Empty state: "No candidates scored yet – research runs at 9 AM ET" (mirrors existing empty-state language elsewhere in the app).
- **\*\*Evolution\*\*** – line/step charts: weight deltas over time per dimension, feature-registry promotion/retirement timeline, calibration Brier-score trend, shadow-decision agreement rate. Each metric card states plainly when there isn't enough history yet (no false precision on thin data – matches the Goal tracker's feasibility-sentence convention).

### Docs required in the same change (per CLAUDE.md)

- `public/agent-diagrams/system-map.json` – this doesn't add a new agent-to-agent data flow, but it DOES add new observability into existing flows (research → constructor → proposal → execution). Add a note to each touched node's description (RESEARCH, TRADER/PROMOTE area) rather than a new node, plus a `history` entry.
- `PROJECT\_DECISIONS.md` – one entry once approved and built.
- `WORK\_LOG.md` – session entry.

### Non-goals (v1)

- Real-time streaming of the funnel as it happens (this is a post-hoc daily report, not a live pipeline visualizer).
- Auto-alerting on funnel anomalies (e.g. "0 signals passed today") – that's already partially covered by the existing stale-run / 0-signal alert in `research/cron`; this feature is for understanding *why*, not re-alerting.
- Multi-market side-by-side view in v1 – one market at a time. India is now supported through the same market-scoped picker and local-calendar rules.

## Acceptance

- `pipeline\_stage\_events` inserts are additive and fail-soft (a logging failure must never block or alter the actual trading decision it's describing – same convention as `decision\_journal` writes elsewhere).
- Daily Funnel renders correctly with zero events (first day after ship).
- Evolution tab is honest about thin history ("not enough learner runs yet to show a trend") rather than drawing a misleading chart from 1-2 points.
- tsc + build pass; migrations handed over as clickable links + full paths.

## V2 – novice-first evidence and context (approved 2026-07-12)

### Product question

The journal must answer, in order: **\*\*what is this security; why did it appear; what changed; what does Kairos currently think; how reliable is that view; what could invalidate it; and what happens next?\*** Exact scores and raw indicators remain available, but are the audit layer rather than the headline.

### Three-layer information architecture

1. **\*\*Decision summary (default/collapsed):\*\*** asset identity/type, discovery state (`new`, `recurring`, `holding`, `re-entry` when recorded), action (`research`, `watch`, `wait`, `paper candidate`, `hold`, `exit review`, `avoid`), score, evidence coverage, decision confidence, why-now, strongest positive, largest risk, and next step.
2. **\*\*Evidence and context (expanded):\*\*** grounded thesis, technical translation, material catalysts/news context when stored, theme/peer context when supported, counter-evidence, invalidation conditions, history/score change, and source-safe links for independent verification.

3. **Quant audit (expanded):** dimension values, structural/applied weights, point contributions, availability/freshness, raw evidence, and pipeline events.

#### ### Truth and safety invariants

- Keep **available-evidence score**, **structural evidence coverage**, and **decision confidence** separate. A 98 computed from two dimensions is not displayed as five-dimension/high-confidence evidence.
- Missing/inapplicable dimensions display `Not used`, never a neutral-looking numeric score. They contribute zero points and lower coverage only when the dimension is structurally applicable.
- The LLM may summarize already-recorded evidence. It cannot invent prices, financials, news, peers, confidence, action, targets, or invalidation levels. Future observations may show the deterministic mandate-based indicative plan defined in `features/research-trade-plan/FEATURE_ARCHITECTURE.md`; it is explicitly not LLM-authored and is re-priced from the actual fill downstream.
- News/theme/social context does not authorize a trade. It may explain, confirm, reduce confidence, or veto through a separately validated deterministic rule.
- No per-card provider fan-out on initial page load. The immutable stored decision renders first. Optional current enrichment is on-demand, cached, bounded, and explicitly newer than the historical decision.
- Point-in-time history is never rewritten. V2-derived presentation fields are computed at read time or written additively for future observations.

#### ### Asset-aware context

- **Company:** business/sector, growth, profitability, cash flow, leverage, valuation, estimates/earnings when available.
- **ETF/fund:** exposure, holdings/concentration, liquidity, expense/flows, geography/currency hedge and structural risks. Company fundamentals and insider activity are inapplicable.
- **India company:** NSE symbol normalization and NSE/BSE/company-announcement links; India-specific provider freshness.
- Asset type comes from recorded `agent_signals.asset_class` plus the canonical symbol classifier. Never infer ETF/company status from missing fundamentals.

#### ### History and novelty

For all symbols in the selected daily run, one bounded history query computes first-seen date, observation count, previous score/direction and change. Existing holdings are always labeled `holding`; otherwise first observation is `new` and later observations are `recurring`. `re-entry` requires a recorded re-entry event and is never guessed from elapsed time.

#### ### Technical translation

Translate deterministic evidence into novice language: trend, momentum, moving-average posture, volume confirmation and extension/breakdown risk. RSI/MACD are diagnostics, not independent votes; correlated indicators must not be double-counted. Never call RSI near 50 "strong" or low volume "confirmation."

#### ### External validation links

Build links deterministically from validated symbols: TradingView, Yahoo Finance, SEC/company research for US, NSE/BSE for India, and issuer pages only when a verified issuer URL exists. Links are labeled external and are never scraped as application data or treated as scoring evidence.

#### ### V2 phased delivery

- **V2.0 (this build):** decision/coverage/confidence separation, asset type, novelty/history, novice action/technical translation, missing-dimension honesty, score change, and deterministic TradingView/Yahoo/SEC/NSE/BSE links. Uses only stored data; no scoring or order-path change.
- **V2.1 (built 2026-07-12):** on-demand cached provider-backed news panel with source, publication/fetch timestamps, title deduplication, safe HTTPS links and an explicit warning that current context post-dates the historical decision. India fails honestly until a validated free NSE-news adapter is available.
- **V2.2:** point-in-time peer/theme comparison after comparable-group and provider coverage validation. Measure-only until incremental value is proven.

#### ### V2 acceptance

- A novice can identify the security state, current action, confidence, strongest evidence, largest gap and next step without expanding raw score construction.
- ETF examples (including EUAD) never display company-fundamental/insider evidence as applicable; India links and symbols resolve correctly.
- A high score with 2/5 applicable dimensions visibly reports limited coverage and cannot be labeled high confidence.
- Historical and on-demand current context are visually/time separated.
- API reads are bounded; partial context failure does not hide the stored decision.
- Current-news enrichment is owner-gated, opt-in per symbol, cache/budget guarded, deduplicated and never changes the recorded score/action.
- TypeScript, unit tests, production build, and US/India browser checks pass.

#### ## V2.3 – symbol decision and execution timeline (built 2026-08-24)

### ### Correctness defect

`paper\_trades` is a lot ledger, not an event ledger. A normal long entry keeps `order\_side='buy'` after it is sold; the exit action is stored on that same row as `exit\_at`, `exit\_price`, `realized\_pnl\_pct`, and `exit\_reason`. The Fundamentals page previously rendered `order\_side` literally. In production this presented 46 closed US lots and 91 closed India lots as BUYS and displayed no SELLS.

### ### Presentation contract

- Expand each untainted paper BUY lot into a BUY fill at `executed\_at` and, when closed, a SELL fill at `exit\_at`/`closed\_at`. An explicit SELL ledger row is one SELL event and is never inverted into a fictitious BUY.
- Show live `trade\_proposals` as proposals with their status. A proposal is an app decision, not an execution and must never be labelled as a trade/fill.
- Show `broker\_orders` separately as order lifecycle events; only a positive `filled\_qty`, `filled`, or `partially\_filled` state is an execution.
- The Fundamentals table's Trade column is the latest actual execution across paper and live venues. The symbol detail timeline may also show non-executed proposals/orders, with venue, stage, status, timestamp, side, quantity, price, reason, and P&L when the ledger contains it.
- All reads are owner-gated, bounded, symbol-and-market scoped, and exclude only rows explicitly tainted. Missing prices remain `—`; they are never fabricated.
- BUY/SELL chart markers use normalized paper actions and are market scoped. Recommendation direction remains separate in score history.

### ### Non-goals

This is a read-model correction only. It does not change research direction, eligibility, sizing, paper fills, proposals, broker submission, exits, or any append-only ledger.

---

# Source: features\research-trade-plan\FEATURE\_ARCHITECTURE.md  
# Feature: Research-Time Indicative Trade Plan

Status: APPROVED / BUILDING (2026-07-20) · Owner: Vaibhav · Builder: Codex

### ## Problem

Research Journal explains why a symbol passed or failed, but does not show the native-currency price used by research or translate the active mandate into an approximate risk/return plan. Downstream execution already creates absolute stops and targets at fill time, so adding independent LLM-generated prices would create two conflicting truths on the money path.

### ## Decision

Record a deterministic, point-in-time **indicative plan** from the candle already used by ResearchAgent. Show it in Research Journal as:

- research reference price and market date/source;
- for an eligible new long: approximate initial risk floor, profit objective, percentages, and target horizon;
- for a rejected/neutral symbol: reference price plus `No entry plan`;
- for a held symbol: reference price plus `PositionMonitor owns the executable exit plan` so a newly computed scenario cannot overwrite the position's fill-bound stop/target.

This is not a predicted fair value, limit order, broker stop, guaranteed exit, or new score dimension. FINRA warns that a stop price is not a guaranteed execution price and can execute materially differently in volatile markets; the product must not label an indicative level as an execution promise.

### ## Contracts

#### ### Shared deterministic module

`lib/trading/trade-plan.ts` owns two pure contracts:

1. `buildIndicativeTradePlan`: binds the latest validated research candle to the current per-market mandate. It performs no fetch and no LLM call.
2. `resolveExecutionRiskReward`: selects fill-time stop/target percentages. A valid ledger MAE/MFE distribution with at least 60 eligible-long outcomes for the same market/horizon uses the approved clamps (maximum 10% MAE stop and 40% MFE target); malformed/thin learned data falls back to the current mandate.

Both contracts reject non-finite/non-positive prices and preserve native market currency (`USD` for US, `INR` for India).

#### ### Persistence

No migration is required. Use the existing immutable ledger:

- `decision\_observations.price\_at\_decision`: latest candle close used by scoring;

- `decision\_observations.currency`: existing native-currency field;
- `decision\_observations.features.trade\_plan`: versioned indicative plan;
- `agent\_signals.stop\_loss\_pct` / `take\_profit\_pct`: research-time mandate snapshot for downstream audit/comparison, never an executable absolute price.

Historical null rows remain unchanged. The Journal must report them as unavailable rather than backfilling with current prices.

## ## Agent Ownership

- **ResearchAgent**: records the point-in-time reference and indicative plan.
- **PaperTrader**: fetches a fresh market-local quote, re-resolves current mandate/validated MAE-MFE policy, anchors absolute levels to the actual fill, and logs planned-versus-bound values. Research prices never authorize a fill.
- **PositionMonitor**: remains the sole owner of paper stop, target, trailing, score-exit, and time-exit decisions from `paper\_positions`. Research cannot loosen or replace those levels.
- **Live Trader/broker adapters**: unchanged. Existing quote freshness, approval, pause, kill-switch, account, and order gates remain authoritative.
- **LearnerAgent**: may evaluate eventual MAE/MFE outcomes through the existing observation ledger. It cannot invent or directly mutate plan levels.

## ## Safety Invariants

1. No LLM-generated number enters the plan or money path.
2. US and India plans never share prices, currencies, mandates, or learned samples.
3. A missing/stale/non-positive research price produces `unavailable`, never a fabricated level.
4. Research levels cannot bypass signal/session freshness, portfolio, cash, pause, kill-switch, approval, broker, or execution-price gates.
5. Fill-time levels are anchored to the actual fill, not the research close.
6. Held-position exits always use the stored position plan; the Journal labels research levels as context only.
7. Stop/target prices are estimates, not guaranteed executions.

## ## UI

Add an unframed `Indicative plan` row in each Research Journal symbol detail:

- `Reference`, `Initial risk floor`, `Profit objective`, `Horizon`;
- market-native currency formatting;
- source/as-of metadata;
- concise disclosure: `Repriced at fill; not an order or guaranteed execution.`

Do not show red/green prices for rejected/neutral symbols as though a trade is planned. Do not add controls that place orders from the Journal.

## ## Acceptance Criteria

- A new US and India observation stores the actual latest scoring-candle close, native currency, source/date, and a versioned plan without extra provider calls.
- Eligible new longs show approximate entry/risk/target; rejected and held names show truthful non-entry/position-owned states.
- PaperTrader uses valid ledger percentiles when available, applies documented bounds, falls back on malformed/thin data, and anchors levels to fill price.
- Journal API/UI render old null-price rows honestly.
- Focused unit tests cover invalid values, both currencies, held/rejected states, learned-policy bounds/fallback, and fill anchoring.
- TypeScript, full unit suite, production build, and diff checks pass.

## ## Reversal

Remove the Journal rendering and stop writing `features.trade\_plan`. Existing JSONB observations remain harmless audit history. The fill-time resolver can be returned to mandate-only behavior independently; no position or order migration is required.

## ## V2: Position And Outcome Truth (Approved 2026-07-20)

The Journal must not make the research-time scenario look like the currently managed trade. It adds three independent, explicitly labelled overlays:

1. **Current paper position**: read from `paper\_positions` for the same market and symbol. Show actual average cost, current price, stored initial/current stop, target, quantity, and update time. These stored fields remain owned by PaperTrader and PositionMonitor.
2. **Current live holding**: read only from the latest `live\_account\_snapshots` row for the configured active account. Never make a broker request from the Journal and never merge accounts or markets. A live holding is `managed` only when a filled Kairos BUY proposal for that exact active account carries a valid deterministic execution policy; otherwise it is truthfully labelled `unmanaged\_by\_kairos` and no stop/target is invented.
3. **Realized paper outcome**: join `paper\_trades` by the exact `signal\_id` and show fill, exit, P&L, close time, and exit reason. Symbol-only matching is forbidden because it can attach a later trade to an older decision.

The API also reports exit-policy learning readiness as eligible same-market, same-horizon long labels `n / 60`. Thin data continues to use the mandate. This is observability only and cannot lower the 60-sample gate.

### ### Dormant Live-Exit Contract

Before live autonomy can ever be enabled, every autonomous BUY proposal records the deterministic fill-time percentage policy, horizon, mandate version, and policy source in `trade\_proposals.policy\_snapshot`. Absolute levels are rebound to the broker's actual average fill.

The dormant live exit monitor must:

- include only filled orders whose `proposal\_id` resolves to the configured active account; missing proposal/account lineage fails closed;
- reconstruct remaining lots FIFO so partial sells remove quantity and cost basis together;
- use the remaining lots' recorded policies, with the current per-market mandate only as a clearly marked fallback for legacy Kairos lots;
- use the fill-recorded resolved horizon and count market sessions exactly as Paper PositionMonitor does; legacy Kairos lots fall back to the mandate;
- never auto-manage broker holdings that predate Kairos order lineage;
- surface database read failures rather than silently treating them as no positions.

These changes do not activate `AUTONOMOUS\_LIVE\_ENABLED`, `live\_auto\_enabled`, broker-hosted protection, Webull trading, or any order canary. Owner approval, account capability proof, and a small manually approved canary remain separate deployment gates.

---

# Source: features\research-universe-fix\FEATURE\_ARCHITECTURE.md  
# Research Universe Fix – Feature Architecture

\*\*Status:\*\* APPROVED → IMPLEMENTING  
\*\*Date:\*\* 2026-07-09  
\*\*Author:\*\* Claude / Opus 4.8

---

### ## Problem

US paper fills = 0 individual stocks. Root cause confirmed (see `P0\_BUILD\_RESULT.md` in edge-factor-discovery):

`gatherSymbols()` in `lib/research-agent.ts` inserts all symbols into a single `Map`, with holdings first. `slice(0, RESEARCH\_MAX\_SYMBOLS=10)`

Evidence: 4 days of `agent\_signals` decompose exactly as 10 holdings + 3 metals + 3 region ETFs = 16. Zero individual stocks ever enter the re

---

### ## Root Cause (single line)

```
const nonMetals = Array.from(result.values()).slice(0, cap); // cap=10 = all holdings
```

Holdings fill the entire cap before watchlist or screener stocks even enter the slice.

---

### ## Fix

**\*\*Separate the holdings-monitor budget from the new-buy candidate budget.\*\***

Holdings are already owned. They need to be scored always (SELL signals, exit timing, position monitor). They should NEVER compete with or co

New-buy candidates (watchlist stocks from Theme Scout + manual adds + screener stocks) need their own budget cap.

### ### Changed invariants

| Before                                                                                                                                      | After |
|---------------------------------------------------------------------------------------------------------------------------------------------|-------|
| --- ---                                                                                                                                     |       |
| One shared cap (`RESEARCH_MAX_SYMBOLS=10`) applies to holdings+watchlist+screener together   Holdings uncapped (always all). Candidates use |       |
| Watchlist/screener stocks evicted when holdings fill cap   Watchlist stocks first, then screener stocks, up to `RESEARCH_CANDIDATE_CAP`     |       |
| Screener max 3 stocks   Screener max `RESEARCH_SCREENERS_MAX` (default 6)                                                                   |       |

### ### Preserved invariants (no changes)

- `isHeld: true` on holdings → LLM prompt includes `heldNote` → SELL signals possible
- `isHeld: false` on new candidates → long-only new-position enforcement in PaperTrader
- Metals basket appended after cap (unchanged)
- Region ETFs appended after cap (unchanged)
- India holdings/candidates handled independently (unchanged)
- No schema changes, no migrations

---



## Code Changes

**\*\*One file: `lib/research-agent.ts`\*\***

Replace the `result = new Map()` block (lines ~419-440) with two separate structures:

holdingEntries[] — uncapped, always all holdings

candidateMap — watchlist first, then screener, capped at RESEARCH\_CANDIDATE\_CAP

nonMetals — [...holdingEntries, ...candidateMap.slice(0, candidateCap)]

### Environment variables

| Var                    | Old | New default                                               | Notes                                              |
|------------------------|-----|-----------------------------------------------------------|----------------------------------------------------|
| RESEARCH_MAX_SYMBOLS   | 10  | deprecated                                                | No longer read; remove from Vercel env to clean up |
| RESEARCH_CANDIDATE_CAP | 10  | Max new-buy candidates (watchlist+screener) per run       |                                                    |
| RESEARCH_SCREENERS_MAX | 6   | Max screener-sourced stocks in candidates (raised from 3) |                                                    |

### Expected batch after fix (US)

- Holdings: 10 ETFs (always)
- Candidates: up to 10 (watchlist stocks first, then screener fill up to 6 of the 10 slots)
- Metals: 4 (unchanged)
- India region ETFs: 3 (unchanged)
- **\*\*Total: ~27 max\*\*** (vs. 16 previously, which were 100% ETFs)

---

## Safety Analysis

- Holdings still scored → SELL signals preserved
- `isHeld` flag correct per entry → PaperTrader long-only gate unaffected
- No change to scoring logic, sizing, order path, live trading
- No DB schema change
- No migration
- Additive: batch gets larger (more API calls), but within existing rate limits and budgets

---

## Second Potential Blocker (to verify post-deploy)

`runScreener()` calls FinancialDatasets `screen\_stocks`. FD key confirmed in vault. But screener output needs to be non-empty for any candidate

---

## Diagram update

System Map and SYSTEM\_OVERVIEW do not need updating for this fix – the agent-to-agent flow is unchanged; only the internal symbol prioritization

---

# Source: features\risk-research-visibility\FEATURE\_ARCHITECTURE.md  
# Risk Analytics – Research Visibility

```
> Status: **APPROVED – built.** Owner approved 2026-07-17 with $10 decided.
> Author: Claude / Opus 4.8. Date: 2026-07-17.
> Supersedes the approach in `features/risk-research-integration/FEATURE_ARCHITECTURE.md`
> (branch `worktree-agent-abaf0b16ef6af4175`, unmerged) – see $9.
>
> Implementation: `lib/research/risk-annotation.ts` (pure), the `research` block on
> `GET /api/portfolio/risk-daily`, `PortfolioRiskPage.tsx`, and the
> `?symbol=&market=` deep link on the research journal. Tests: `tests/risk-research-annotation.test.ts`.
```

## 1. What this is

Show, per holding on Risk Analytics: **\*\*the research score, its direction, and how old it is\*\*** – with a click through to that symbol's research

## 2. What this is NOT – the load-bearing constraint

**\*\*Research does NOT enter the risk computation.\*\*** Not the score, not the posture, not the action, not the trim allocation. This is a **\*\*display**

Why, restated so a future reader does not "improve" it:

- `lib/risk/holding-risk.ts` has zero research coupling **\*\*by design\*\***. `sba-v1` (the water-fill sector-breach allocator, shipped 2026-07-17)
- A sector is over-cap **\*\*because\*\*** research liked that sector. Letting `analyst\_score` also veto the cap **\*\*double-counts the same signal\*\***. The
- The owner integrates the two views. That is where judgment belongs.

**\*\*Invariant R1:\*\*** no field introduced here may be read by `computeHoldingRisk`, `sba-v1`, `constructPortfolio`, the execution kernel, or any g

## 3. Why the age matters more than the score

This is the actual feature. On 2026-07-16, AVGO was **\*\*6 days unscored\*\*** while Risk Analytics told the owner to trim it – and nothing on screen

Per CLAUDE.md "Detail Over Cryptic": the cell must say what/why/next, never a bare number.

```

4. Staleness – measured in SESSIONS, displayed in DAYS

Do not measure staleness in calendar days. Research runs per market on weekdays. A Friday score read on Monday is 3 calendar days old but
- **Measure**: market-local *sessions* elapsed since `scored_at` (US and India have different calendars/holidays – `market_controls` / the ex.
- **Display**: "N days ago" (owner's call – plain and readable) **plus** the session-based state.

States:

State	Rule	Render
`fresh`	≤ `STALE_AFTER_SESSIONS`	score + direction, normal
`stale`	> `STALE_AFTER_SESSIONS`	**warning** styling + "not scored in N days"
`never`	no signal for (symbol, market)	"never scored" – a distinct state, NOT stale
`unavailable`	research ran but abstained (thin evidence)	"abstained – thin evidence", NOT a score

`never` ≠ `stale` ≠ `unavailable`. Collapsing them is the exact bug class this batch removed.

5. Data

Source: latest `agent_signals` per `(symbol, market)` – `analyst_score`, `direction`, `conviction`, `created_at` (→ `scored_at`), `is_ho
Join key: `(symbol, market)`. Never symbol alone – `market` is authoritative; per-market/per-currency never cross-summed.

Delivery: extend `GET /api/portfolio/risk-daily` (already returns per-holding rows and is owner-gated) with a nullable `research` block per
research: {
score: number | null,
direction: 'long' | 'neutral' | 'short' | null,
scored_at: string | null, // ISO
sessions_since: number | null, // market-local
days_since: number | null, // display
state: 'fresh' | 'stale' | 'never' | 'unavailable',
scored_as_holding: boolean | null, // see §6
} | null

Two known traps:
1. **`insider_score` = 50` is a default-fill, not evidence.** The honest record is `decision_observations.availability_mask`. Do not surface a
2. **`is_holding` matters.** Before the starvation fix, every AVGO row was `is_holding: false` – scored as a *screener candidate*, not as a h

6. Click-through

Target: `/dashboard/research-journal?symbol=<SYM>&market=<us|india>` → auto-expand that symbol's entry.

Gap: `app/dashboard/research-journal/page.tsx` currently expands via local `toggle(symbol)` state and reads **no** `searchParams`. Deep-L
Rules: `never`-state symbols are not links (a link to nothing is a lie). The link must carry `market` – `.NS` symbols are unambiguous but

7. Acceptance tests – each must be able to FAIL

#	Test
T1	**R1**: `computeHoldingRisk` / `sba-v1` output is byte-identical with the `research` block present vs absent. (Falsifiable: wire the s
T2	A score older than `STALE_AFTER_SESSIONS` renders `stale` + day count; one inside renders `fresh`.
T3	**Friday score, read Monday, threshold = 1 session → `fresh`.** (Falsifiable: a calendar-day implementation fails this.)
T4	No signal → `never`, not `stale`, not score 0, and **not a link**.
T5	An abstained (thin-evidence) signal → `unavailable`, never rendered as a number.
T6	US and India books never cross-join: an India `.NS` symbol resolves only against `market='india'` rows.
T7	A `is_holding:false` row is labelled as a candidate score, not a holding verdict.
T8	The risk API still returns holdings when `agent_signals` is empty/errors – research absence must never blank the risk table (fail-soft)

8. Failure modes

Mode	Behavior
`agent_signals` read fails	Risk table still renders; research column shows an explicit "research unavailable" – **never** silently blank,
Research hasn't run today	Correct and expected → `stale` with the day count. This is the feature working.
Symbol held in 2 accounts	One research row per (symbol, market); it is a property of the symbol, not the account. Same annotation both pl
India research abstains (sentiment 0% available)	`unavailable`, not a score.

9. Relationship to the earlier proposal

`features/risk-research-integration/FEATURE_ARCHITECTURE.md` proposed research **ordering** which names absorb a sector trim. **Not pursued.**

10. Owner decisions – DECIDED 2026-07-17

```

All four are settled. This section is a record, not an open question.

```
#	Decision	**DECIDED**	Rationale as approved
Q1	`STALE_AFTER_SESSIONS`	**2**	The merged holdings rotation bounds worst-case staleness at ~2 runs, so `1` would flag ordinary rotation.
Q2	Does `stale` also mark the row's `risk action` as lower-confidence?	**NO**	`stale` annotates the SCORE only. It must NOT dim or a
Q3	Show `conviction` alongside `analyst_score`?	**OMIT**	Identical to `analyst_score` in every prod row inspected. Revisit only if the
Q4	Mobile (375px) columns	`score + age badge only`	Direction and `scored_as_holding` go behind the existing row expander. Mobile-fi

Verified against prod after the decision (2026-07-17, project `<production-project-ref>`):

- **Q3 is right, and for a better reason than stated.** `analyst_score = conviction` in
 458 of 463 rows – not all of them. The 5 exceptions (AAPL 58/62, NVDA 55/48,
 TSLA 36/58, MSFT 53/52, META 69/68) are all from **2026-06-28** and all carry
 `score_source = null` – the pre-`deterministic_v1` LLM-scored era. **Since
 `deterministic_v1` shipped, the two have never diverged.** So `conviction` is
 redundant *by construction* under current scoring, not merely *empirically*.
 Revisit if a future scorer decouples them.
- **Q1's threshold does what it was chosen to do.** AVGO's latest row (scored
 2026-07-13, read 2026-07-17) is **4 sessions** old → `stale`. The India book
 (scored 2026-07-17 04:00 UTC) is `fresh`. The threshold separates the two.

10a. What prod actually looks like (measured, not assumed – 2026-07-17)

Three findings that shape the implementation:

1. **`is_holding` is `false` in 463 of 463 rows – 100%.** Not an AVGO quirk: **no
 holding-path score has ever been written.** Every score on the Risk page is a
 screener-candidate score. $5 trap 2 is therefore the *common* case, and
 `scored_as_holding` is labelled on every row rather than as an edge case.
2. **There is no abstain representation in `agent_signals`.** `analyst_score` is
 non-null in all 463 rows; no column encodes "ran but abstained". The
 `unavailable` state is therefore **defensive, and currently unreachable in
 prod** – it fires only if research ever writes a null/NaN score, so that an
 abstain degrades honestly instead of rendering as `0`. T5 pins it against a
 fixture. **This is the one state with no production witness.**
3. **`never` currently has no production witness either.** Every holding in every
 account's latest run resolves to a signal. The state still exists and is
 tested – a book with a new position would hit it immediately – but it is not
 observable today.

10b. Spec corrections found while building

- **$6's premise was wrong.** `app/dashboard/research-journal/page.tsx` *did* read
 `searchParams` (for `tab`), and the `toggle(symbol)` expander described there
 lived in a local `FunnelTab` component that **was never rendered** – dead the day
 it was written. The page has always delegated to `components/dashboard/ResearchFunnel.tsx`.
 Deep-link support therefore went into `ResearchFunnel.tsx`; the dead `FunnelTab`
 was deleted rather than modified, since it was the source of the misreading.
- **The session helper already existed.** `marketSessionsSince(createdAt, now, market)`
 in `lib/trading/paper-exit-policy.ts` (built on `isMarketHoliday` in
 `lib/trading/market-calendar.ts`) is the market-local session authority. Reused,
 not reimplemented, per $4.

11. Out of scope
Live-trading accounts the app cannot trade (e.g. `read-only-account-id`) get the same annotation – it is informational there, as the strategy

Source: features\risk-sector-breach-allocation\FEATURE_ARCHITECTURE.md
Feature: Sector-Cap Breach Allocation (risk-internal, deterministic)

Status: SHIPPED - allocator `sba-v1`, posture consumer `hr-v3`
Last updated: 2026-07-21
Owner: Vaibhav
Update this file when: the allocation rule, its denominator basis, the breach
arithmetic, the roles (`absorb` / `not_selected` / `no_breach` / `sector_unknown`),
or the read-only advisory labelling changes.

Parent feature: `features/holding-risk-daily/FEATURE_ARCHITECTURE.md` – this file
is the authority for the allocator; that file remains the authority for the score,
the cron, the snapshot schema, and the LLM prose boundary.

> **Current posture boundary (`hr-v3`, 2026-07-21):** this allocator remains the
> correct simulation for a genuine equity-sector breach, but its `absorb` output
> is internal diagnostic evidence only. Every live account displays measured
> concentration under `review`, with no trim recommendation, because the current
> limits are global references rather than account-specific mandates. An order-
> permitted account is not an exception. Asset-class buckets
> such as Diversified Equity, Fixed Income, Commodities, and Digital Assets never
> enter this sector allocator.

1. The defect this exists to fix
```

Risk Analytics shipped this for AVGO, a live holding in the read-only Robinhood account `<read-only-account-id>`:

```
> "Strategy note (advisory): Trim your position because Technology holdings exceed
> the 30% sector cap (at 65.6%), with a risk score of 63..."
```

**\*\*A sector-cap breach is a property of the SECTOR, not of any one name.\*\*** In `lib/risk/holding-risk.ts` (`hr-v1`) the posture branch read:

```
const sectorBreach = sectorUtil !== null && sectorUtil >= 1;
```

```
if (nameBreach || sectorBreach || clusterBreach) { riskPosture = "trim"; ... }
```

`sectorUtil` is the same number for every holding in the sector. So **\*\*every\*\*** Technology name received the identical `trim` verdict, with identical prose, and the engine never decided **\*\*which\*\*** names should absorb the breach or **\*\*how much\*\*** each gives up. The advice is arbitrary (no name is distinguished) and unactionable (no quantity). Trimming all six Tech names by an unstated amount is not a plan.

Note the same run also emitted `trim` for AVGO while ResearchAgent had not scored AVGO since 07-13 (see WORK\_LOG, holdings-starvation entry). That is a separate bug; it is **\*\*not\*\*** evidence that risk should read research scores. See §7.

## 2. What this builds

`lib/risk/sector-breach.ts` – a pure, versioned (`sba-v1`) allocator:

```
> Given a sector that is over its cap and the reduction required to reach the cap,
> decide **which** holdings absorb it and **how much** each gives up.
```

Deterministic. No LLM. No research score. No I/O, no clock, no randomness – same inputs always yield the same allocation. It is called by the holding-risk cron before `computeHoldingRisk`, and its per-name result is threaded into `HoldingRiskContext.sectorBreachAllocation`.

## 3. Denominator: NAV, and why it decides the arithmetic

The breach amount is **\*\*not\*\*** basis-independent. Let `S` = sector market value, `D` = the denominator the cap is measured against, `c` = cap fraction:

```
Denominator	Does a sale change `D`?	Value that must be sold
NAV (cash-inclusive)	No – proceeds become cash, still inside `D`	`X = S - c·D`
Invested (cash-excluded)	Yes – proceeds leave `D`	`X = (S - c·D) / (1 - c)`
```

At Tech 65.6% / cap 30%: NAV basis → sell **\*\*35.6pp\*\***. Invested basis → sell **\*\*50.9pp\*\***. Picking the wrong basis makes the advice wrong by ~43%.

**\*\*Kairos' sector cap is NAV-relative.\*\*** `lib/risk/live-portfolio-gate.ts` – the code that actually enforces `max\_sector\_exposure\_pct` on the live money path – builds the book as `valuePct = position value / NAV × 100`. That is the authoritative reading of the owner's limit. `lib/risk/holding-risk.ts` already scores **\*\*name\*\*** concentration against `accountTotalValue` (NAV) for the same reason.

So the allocator takes `navValue` (not a generic "denominator"), and the required reduction is the simple `X = S - c·D`. The field is named `navValue` precisely so a future caller cannot silently feed an invested-total and get 43%-wrong advice.

### 3.1 Known inconsistency this fixes (and one it does not)

`hr-v1` fed `sectorWeightPct` from `computeRiskMetrics().sectorBreakdown`, whose `weightPct` is **\*\*invested-relative\*\*** (`lib/portfolio-risk.ts` L307-318: `totalValue = holdings.reduce(...marketValue)`). So `hr-v1` compared name concentration against NAV and sector concentration against invested – two different denominators against two caps the owner set on one basis. The cron now derives sector weights on the NAV basis from the allocator, so the sector driver and the allocation agree, and both agree with the name driver.

**\*\*Not fixed here (deliberate):\*\*** `computeRiskMetrics().sectorBreakdown` remains invested-relative for the account roll-up display. Changing it touches the paper path, `constructPortfolio` inputs, and the Risk page's sector bars – out of scope for a risk-verdict fix. The two numbers can differ for a cash-heavy account. Recorded as a follow-up.

## 4. The allocation rule – WATER-FILL (level-down, largest-first)

**\*\*Rule.\*\*** Trim the largest positions in the breached sector down to a single common level `L`, chosen so the sector lands exactly on the cap:

```
> find `L` such that $\sum \min(w_i, L) = c$ over the sector's names (weights as
> fractions of NAV), then $\text{trim}_i = \max(0, w_i - L)$.
```

Names with  $w_i \leq L$  are **\*\*not touched\*\***. Names with  $w_i > L$  absorb the breach,

largest first, and all end at the same weight `L`.

Solved in closed form: sort weights ascending, and for `k = 0..n-1` test the candidate `L = (c - prefix[k]) / (n - k)`, accepting the first `k` where `L ≥ w[k-1]` and `L ≤ w[k]`. `Σ min(w, L)` is continuous and non-decreasing in `L`, from `0` to the sector weight, so exactly one `L` exists whenever `0 < c < sectorWeight`.

#### ### 4.1 Justification

**\*\*Why not pro-rata?\*** Scale every name in the sector by `c / sectorWeight`.  
- **\*For:\*** expresses no view on which name is worse – appropriate, since a risk engine with no conviction input has no such view.  
- **\*Against, decisive:\*** **\*\*every\*\*** name is trimmed, so every name still gets the same blanket "trim" verdict – the exact defect in §1, re-shipped with numbers attached. And it preserves the sector's concentration profile: after a pro-rata cut the book is at the cap but just as lopsided, so the **\*name\*** cap – a second limit the owner set – is left as breached as it was. Rejected.

**\*\*Why not "highest marginal contribution to the breach" first?\*** For a **\*sector weight\*** cap, a name's marginal contribution to the sector weight **\*\*is its weight\*\*** –  $\partial(\Sigma w) / \partial w = 1$ . So this rule is identical to largest-first with an extra abstraction that invites smuggling correlation or beta into the ordering. Those are separate risk components with their own caps and their own missing-data paths; folding them into the sector allocation would make one number's absence silently reshuffle who sells. Rejected as a distinct rule.

**\*\*Why water-fill wins.\*\*\*** Among all allocations that reach the cap **\*\*by reducing only\*\*** ( $0 \leq t \leq w$ ), the water-fill is the one that minimises the largest residual name weight **`max w`** – and, more generally, minimises  $\Sigma (w)^2$  (the sector's Herfindahl). Sketch: any feasible allocation with two names at  $a > b \geq L$  where the water-fill would have levelled both must have **`max w`**  $\geq a > L$ ; levelling weakly reduces both the max and the sum of squares while holding the total fixed. So the water-fill discharges the sector breach **\*\*and\*\*** does the most it can for the name-concentration cap at the same time – the two limits the owner actually set point the same way. It also yields real "hold" verdicts for small names, which is what makes the output actionable rather than a blanket.

**\*\*Honest cost.\*\*\*** Water-fill sells your biggest name first, which is often your best name. The risk engine cannot know that – it has no conviction input **\*\*by design\*\*** (§7). Absent a view, a same-sector shock hits every name in the sector and expected loss scales with weight, so reducing the largest exposures is the risk-correct action. If the owner wants conviction to reorder who sells, that is a Research↔Risk coupling decision, not a change to this allocator.

#### ### 4.2 Determinism and tiebreaks

- **\*\*Ties in weight need no arbitrary winner.\*\*\*** Water-fill is a continuous function of the weight vector: two names at identical weight receive identical trims by construction. There is no "first one wins" step that a tie could turn arbitrary. **\*(Test: `equal weights receive equal trims`.)\***
- **\*\*Comparison tolerance.\*\*\*** The fill runs in weight-fraction space (values  $-0-1$ ), not currency, so a fixed **`EPS = 1e-12`** is meaningful at every account size.
- **\*\*Output ordering\*\*** (the only place a tiebreak is needed): absorbers are ordered by **`trimPct`** descending, then **`symbol`** ascending (lexicographic). **`rank`** follows that order. Sector summaries are ordered by **`sectorWeightPct`** descending, then **`sector`** ascending.
- **\*\*No materiality floor.\*\*\*** A name is **`absorb`** iff its trim is  $> 0$ . A floor would have to redistribute the dropped remainder, which reintroduces an arbitrary choice for a cosmetic gain.

#### ### 4.3 Degenerate inputs

```
Input	Behavior
`navValue` ≤ 0 / non-finite	Empty result; every symbol `sector_unknown`-style honest reason is **not** used – the caller's structural gate
`maxSectorExposurePct` ≤ 0 or ≥ 100	Cap unusable → sector reported `no_breach` with the cap echoed; never a fabricated breach.
Sector weight ≤ cap	`no_breach` for every name in it; `requiredReductionPct` = 0.
Non-finite / negative `marketValue`	That position is excluded from its sector's total and reported as `sector_unknown` (unvalued), never a
```

#### ## 5. Per-name output (Detail Over Cryptic)

**`SectorBreachAllocation`** per symbol, with a **`role`**:

```
Role	When	`action_reason` says
`absorb`	breached sector, `w > L`	*what*: trim N pp of NAV, to X% (= value in account currency). *why*: sector is P% vs the C% cap → P
`not_selected`	breached sector, `w ≤ L`	*what*: hold. *why*: ***Technology is over its 30% cap (65.6% of NAV) and 35.6pp must come out
`no_breach`	sector within cap	sector weight vs cap, no reduction required.
`sector_unknown`	sector null/empty/"Other", or unvalued	"sector unknown – excluded from sector-cap allocation: neither counted toward a
```

#### ## 6. Posture wiring (`lib/risk/holding-risk.ts`, current `hr-v3`)

Two new **\*\*optional\*\*** **`HoldingRiskContext`** fields:  
**`sectorBreachAllocation?: SectorBreachAllocation`** | null, **`readOnlyAccount?: boolean`**.

Everything else – the score, the six components, the caps, the confidence weights, `add\_capacity` – is unchanged. Scores do not move except through the NAV-basis denominator correction in §3.1.

Precedence (the exit branch is untouched and stays first):

1. verified protective-stop / thesis-break → `exit\_review` – **unconditionally**, regardless of any allocation. Drawdown alone still never triggers it.
2. `nameBreach` || `sectorSelected` || `clusterBreach` → `review`, where `sectorSelected` = `sectorBreach` && `allocation.role` == "absorb". The allocator remains internal evidence; its simulated reduction is not exposed as advice. No concentration condition creates a sell instruction.
3. `sectorBreach` **and** no usable allocation → `review` + `missing\_inputs: ["sector\_breach\_allocation"]`. Reason: the sector is over cap but the engine cannot say whether **this** name should absorb it.
4. `dataConfidence` < 0.5 → `review` (unchanged).
5. otherwise → `hold`; when `role` == "not\_selected" the hold carries the allocator's why-not sentence instead of "within owner-approved risk limits" (which would be a lie while the sector is over cap).

**Default-to-honest, never default-to-confident.** Case 3 is the `degradation-guard` discipline: a legacy caller that supplies no allocation gets `review` – the **old** blanket-trim behavior is not preserved as a fallback, because a fallback to the bug is the bug.

`formula\_version` is now `hr-v3`: the posture semantics changed again, and `risk-daily` must not diff a v3 posture against a prior formula. No migration – `formula\_version` is `text`, `risk\_posture` keeps its existing CHECK values, and the allocation surfaces in the existing `action\_reason` (text) and the sector driver's `detail` (jsonb).

#### ## 7. Explicitly out of scope

- **No `analyst\_score` / `agent\_signals` / conviction coupling.** The entire point is that this needs **zero** Risk↔Research coupling: the allocator is a function of weights and one owner-set cap. The competing proposal (`features/risk-research-integration/FEATURE_ARCHITECTURE.md`, branch `worktree-agent-abaf0b16ef6af4175`, unmerged) is untouched.
- **No order path.** The route places, previews, and cancels nothing. Only `<agentic-account-id>` may ever place an order, and only via the owner-click Execution Gateway, which this feature does not call.
- **No LLM on any number.** The LLM still writes `strategy\_note` prose only, and is now handed the allocation as read-only context it must explain, never alter. `lib/risk/strategy-notes.ts` extracts the parse step so this is provable: `parseStrategyNotes()` returns `Map<symbol, string>` and **cannot** express a score, a posture, a trim, or a symbol that was not asked for.
- No migration. No schema change.

#### ## 8. Advisory labelling

`readOnlyAccount` is set by the cron: `false` only for `<agentic-account-id>` (the sole order-permitted account per CLAUDE.md), `true` for every other account including `<read-only-account-id>` – where AVGO sits. `readOnlyAccount` changes advisory transport wording only; concentration is `review` for both account classes. `exit_review` reasons carry:

- read-only → "Advisory only – this account is read-only in Kairos; the app cannot trade it."
- `<agentic-account-id>` → "Advisory only – this feature places no order; any action requires owner approval in the Execution Gateway."

Both are advisory. The distinction tells the owner whether an order path exists at all, rather than implying one does.

#### ## 9. Acceptance tests (falsifiable – each states the implementation that fails it)

`tests/sector-breach.test.ts`, `tests/holding-risk.test.ts`, `tests/strategy-notes.test.ts`.

```
#	Test	Fails when
1	Tech 65.6% of NAV vs 30% cap → `requiredReductionPct` ≈ 35.6`, and `Σ trimPct` ≈ 35.6`	the invested-basis formula (would give 50.9), or
2	Post-trim sector weight equals the cap exactly; every `targetWeightPct = min(w, L)`	the fill level is mis-solved
3	Reproducible: two runs over the same input deep-equal; input array shuffled → identical `bySymbol`	any iteration-order or `Map`-order
4	Two names at identical weight get identical `trimPct`; ordering ties break `symbol` ascending	any "first/last one wins" branch
5	A small name below `L` gets `role: "not_selected"`, `trimPct` == 0, and a reason naming its sector, the cap, the breach size, and why	
6	Every name in a breached sector gets its own `trimPct`, and not all are equal	blanket-trim (`hr-v1`) – all names identical
7	`protectiveStopHit` + `role: "not_selected"` → still `exit_review`	the allocator being consulted before the exit branch
8	`thesisBreak` + `role: "not_selected"` → still `exit_review`	same
9	Sector breached, no allocation supplied → `review`, `missing_inputs` contains `sector_breach_allocation`, posture is not `trim`	
10	Sector breached, `role: "absorb"` → `review`, no simulated sell quantity in the action reason	diagnostic allocation leaking as advice
11	Sector breached, `role: "not_selected"`, nothing else breached → `hold` and reason is not "within owner-approved risk limits"	th
12	Name breach + `role: "not_selected"` → `review`, with no trim recommendation	global reference leaking as a sell mandate
13	Cluster breach + `role: "not_selected"` → `review`, with no sell quantity	unallocated overlap leaking as advice
14	India: INR account, `NS` symbols, India sector labels → identical allocation shape; `no_breach` when within cap	any US-GICS or USD
15	US and India inputs with equal weights produce equal `trimPct` and never a summed/cross-market total	cross-market contamination
16	`sector: null` / `Other` / `""` → `role: "sector_unknown"`, excluded from every sector total, reason says "sector unknown"; the othe	
17	A sector that is entirely unknown-sector names produces no `no_breach` claim for them	silent cap-compliance
18	`parseStrategyNotes` given { "AVGO": { "trim_pct": 99, "risk_posture": "hold", "ZZZZ": "x" } } returns no AVGO note, no `risk_posture` key,	
```

```

| 19 | `parseStrategyNotes` over unparseable/absent LLM text → empty map, never throws | prose failure blocking a deterministic row |
| 20 | Both read-only and order-permitted accounts keep concentration at `review`; transport wording remains accurate | order permission being
| 21 | `navValue` unchanged, cap 0/100/NaN → `no_breach`, never a fabricated breach | an unguarded cap divide |

Plus the parent feature's existing `hr` suite (structural gate, component caps,
loss-only-never-exit, purity) must stay green.

10. Cross-doc updates shipped with this

- `docs/arch/08-risk-and-safety.md` — "Daily Per-Holding Risk Analytics" section.
- `features/holding-risk-daily/FEATURE_ARCHITECTURE.md` — pointer to this file.
- `WORK_LOG.md` — entry.
- `public/agent-diagrams/system-map.json` — the diagram is unchanged: no
 agent-to-agent flow, handoff, table dependency, schedule, or learning-loop edge
 changed; this is internal to the HoldingRisk node's own compute. But the
 `HOLDINGRISK` node's own `description` asserted `hr-v1` and the old
 "hard concentration/cluster breach -> trim" rule, which is now false — so the
 node description is corrected and a `history` entry appended.
- No migration.

Source: features\risk-tier-gate\FEATURE_ARCHITECTURE.md
Risk-Tier Gate — Feature Architecture

> Status: Stage 1 LIVE (measure-only shadow). No gate, no sizing, no order-path change.
> Author: Claude (Sonnet 5), 2026-09-18. Approved by Vaibhav 2026-09-18 (item #4 of the
> `achaljhawar/1rok` gap-analysis plan — see PROJECT_DECISIONS.md).
> Source of the idea: `github.com/achaljhawar/1rok`'s Risk Agent, whose prompt discloses an
> exact deterministic-looking scoring shape (base 30, component maxes, tier/investability
> thresholds) even though their implementation is LLM-judged. This document adapts that
> disclosed shape into a 100% deterministic Kairos component — no LLM, no new provider.

1. What this is, and isn't

Is: a 5-component additive risk composite (`lib/risk/risk-tier.ts`, `computeRiskTier`) computed
from data ResearchAgent already fetches every run — AV `COMPANY_OVERVIEW` (beta, debt/equity, profit
margin, gross margin), `lib/data/technical.ts` (ATR-multiple of the last bar's move, the existing
breakdown veto), `macro_regime.danger_score` (US-only, same availability gate as `macro_score`), and
days-to-next-earnings (already computed for the earnings blackout). Written to
`decision_observations.features.risk_tier_shadow` on every scored candidate.

Isn't: a gate. It reads nothing that isn't already fetched, writes nothing that anything else
reads, and blocks no signal, no sizing, no order. `analyst_score`, direction, sizing, and every
existing gate are byte-for-byte unchanged.

2. Why measure-only first

Same discipline as every other new signal in this codebase (instrument-family evidence, oil
exposure, the technical calibration shadow): a new composite must accumulate real evidence and prove
its predictive value before it is trusted to block or shrink a real position. This one has an
additional reason to wait: an honest asymmetry in how it's built (§4) that a gate cannot tolerate but
a shadow can.

3. The five components

Component	Max	Inputs	Availability
Volatility	30	`beta` (AV overview), `atrMultipleMove` (last bar's move in ATR units)	Excluded (null) when both missing
Fragility	30	`debt_to_equity`, `profit_margin`, `gross_margin`	Excluded when all three missing
Concentration	20	none — no segment/customer/geography data source exists in this codebase	A diversified fund (`broad_equity_etf`/`secu
Macro	15	`macro_regime.danger_score`, US-only	Same gate as `macro_score`: excluded for India, stale rows, or an `unknown` verdict
Special	20	`daysToEarnings` ≤ 5, active technical `breakdown_veto`, short-interest squeeze setup (`short_percent_float` > 20% + `short	
`totalRisk = clamp(30 + sum(available components), 0, 100)`. Tiers: LOW ≤30, MODERATE ≤50, HIGH ≤70,
VERY_HIGH >70 (1rok's own disclosed thresholds). Investability label (not yet consumed anywhere):
`full` <50, `medium` 50-69, `small` 70-85, `not_investable` >85.

4. The asymmetry, stated not hidden

This is an additive penalty system, not a renormalized weighted average like `analyst_score`. A
missing component contributes zero risk points — not a neutral default. That means a symbol with
thin data reads lower risk than one with full data showing the identical real risk. This is the
opposite bias from `analyst_score`'s "abstain when evidence is thin" convention, and it is the
reason this ships as a shadow: `coverage` (0-5, how many components were available) travels with every
row specifically so a future consumer — or a human reviewing the Research Journal — can tell "low
risk" from "low evidence" apart. A future promotion to a real gate must either fix this asymmetry
(e.g., require a coverage floor, or make missing-evidence itself count as risk) or explicitly accept
it with eyes open — this document does not resolve that, it flags it.

5. What was deliberately not built (yet)

- Short interest — added 2026-09-18. Yahoo's `defaultKeyStatistics` module (`shortPercentOfFloat`,
`shortRatio`) was already being fetched for every US symbol (`fetchIndiaOverview` in
`lib/india-data.ts`, called by `fetchUsOverview`'s Finnhub-gap-fill pass) but never parsed. Added a

```

```

`BONUS_COPY_ONLY` list in `lib/data/fundamentals.ts` so these two fields ride along only when
Yahoo is already being called for some other gap – deliberately NOT added to the `FILLABLE`
gap-trigger list, because Finnhub can never supply them, which would have forced an extra Yahoo call
on every single US symbol (including ones with full Finnhub coverage) as a silent side effect. Real
coverage is therefore partial, not universal, by design – same honest-unavailable convention as
everything else here.
- Options-derived risk (IV percentile, put/call ratio) – a real input Irok's risk agent uses that
Kairos has no free data source wired for today. Still a fast-follow, not in this pass.
- Institutional ownership trend – a different signal from the existing Form 4 insider score;
separate future item.
- Any consumption of `investability` or `tier` by sizing, entry, or the order path – that is the
eventual point of building this, but it is a separate, evidence-gated promotion decision, exactly
like every other measure-only signal here (instrument-family scoreMode, oil exposure, technical
calibration).

6. Acceptance criteria

1. Zero change to `analyst_score`, direction, sizing, entry/exit gates, or any order path.
2. No new provider call – every input is already fetched by the existing ResearchAgent run.
3. Fail-soft: a thrown error in `computeRiskTier` or its inputs must never block the decision it's
attached to (wrapped in its own try/catch, same as `instrumentFamilyEvidence`/`oilExposureEvidence`).
4. `coverage` is always present alongside `totalRisk` so a low score can never be read as "verified
low risk" without checking how much evidence it rests on.

Source: features\robinhood-agentic-evidence\FEATURE_ARCHITECTURE.md
Robinhood Agentic Evidence – Feature Architecture

> Status: P0 approved and implemented. Later phases require a separate owner-approved design and evidence gate.
> Scope: US only. Robinhood's MCP capability surface is observed, not trusted as a data provider or trading policy.

P0: Capability Snapshot

The weekly route opens an authenticated MCP session and invokes only `initialize`,
`notifications/initialized`, and `tools/list`. It stores the sorted tool names,
tool count, and a SHA-256 fingerprint of the name plus input-schema contract in
the append-only `broker_mcp_capability_snapshots` ledger. Raw descriptions and
schemas are intentionally not persisted because they are untrusted remote input.

It does not invoke `tools/call`; therefore it cannot consume data-provider quota,
read accounts or positions, create research evidence, change scores, change
eligibility, create paper orders, or create live orders. An unavailable connection
is an observation, not a System Health incident or a trading gate.

Deferred Phases

1. Owner-run, allowlisted contract probes for `get_financials`, earnings, and
technical tools. Each probe is bounded and records no raw payload in an LLM
context.
2. A `robinhood` adapter inside the existing Canonical Evidence Router, using
its cache, pacing, provider-call ledger, provenance, health, and schema
validation. No parallel evidence truth layer.
3. Shadow-only field-level comparison and point-in-time walk-forward validation.
A broker field cannot alter a score merely because it is available.
4. Separately, retain `review_equity_order` as a fail-closed pre-trade safety
gate. Capture its contract only after an approved execution-specific design;
it is not part of research evidence.

No scanner, Level 2, tax lots, options, or Robinhood data call is enabled by P0.

Source: features\robinhood-crypto\FEATURE_ARCHITECTURE.md
Robinhood Crypto – Feature Architecture

> Status: Stage 3 LIVE (paper trading) – legacy_v1 scoring + own $10k paper pool + dedicated
> entry/exit routes + Trading UI watch section + per-coin symbol-detail page.
> Author: Claude (Sonnet 5), 2026-09-16. Approved by Vaibhav 2026-09-16.
> Evidence-gate override (owner-directed): Stage 3 was gated on MIN_PREDICTIVE_DATES (20)
> evidence sessions per this doc's original text below. At approval time (2026-09-16) the
> `crypto_basket` had run ~12 days since 2026-09-04 – below the floor. The owner explicitly
> directed the override rather than waiting; scoring quality for BUY/SELL calls at this sample
> size is unproven. Recorded, not silently applied – see PROJECT_DECISIONS.md.
> Scope: PAPER trading only. No live order authorized – Stage 4 remains a separate future decision.
>
> Implementation log:
> - Stage 0 (db57bfd1): RH capability probe – onboarded, 58 pairs, BTC/ETH/SOL selected
> - Stage 1 (120935f4): `InstrumentFamily="crypto"`, `CRYPTO_SESSION_CUTOFF_UTC`, session guards, 14 tests
> - Stage 2 (be9b1b9b): `scoreMode="measure_only"`, IFE composite (technical+macro+sentiment), AV DIGITAL_CURRENCY_DAILY
> - Stage 2b (f29e0ee6): `crypto_basket` always-feeds BTC/ETH/SOL into research runs (parallel to metals_basket)
> - Stage 2c: Trading page "Crypto Watch" section – evidence progress, coin scores (US-only)
> - Stage 3 (2026-09-16): `scoreMode="legacy_v1"`; own paper pool via a THIRD `market='crypto'`
> value scoped to the 4 paper-ledger tables only (`paper_portfolio/positions/trades/performance`
> – none carry a CHECK constraint, verified) – NOT `instrument_family`, and NOT threaded through

```



```
> `app/api/agents/paper-trade` or `position-monitor` (those are equity/India-calibrated: portfolio
> constructor, correlation shadow, capital rotation, Kelly/genome sizing, all keyed to
> `market:"us"|"india"` through library functions backed by market-CHECK-constrained tables –
> forking that router was a materially larger, riskier change than Stage 3 needs). Two small
> dedicated routes instead: `app/api/agents/crypto-paper-trade` (flat-sizing entries, one position
> per coin, price = latest completed AV DIGITAL_CURRENCY_DAILY close – same source the signal was
> scored against) and `app/api/agents/crypto-position-monitor` (mechanical OHLC-aware stop/target
> exits + daily NAV snapshot; no time-stop). Both reuse the existing atomic `execute_paper_fill` /
> `execute_paper_exit` RPCs (fully generic on `market`, verified by reading their SQL bodies) and
> the existing `checkKillSwitches`/`isPaused`/`isTradingEnabled` machinery (`lib/market-controls.ts`
> widened from `us`/`india` to include `crypto` – its `norm()` helper silently coerced any
> other value to `us`, which would have let a crypto kill-switch trip disable US equity trading).
> `decision_journal.market`'s CHECK (migration 084) also had to widen: `execute_paper_exit`
> unconditionally inserts a `decision_journal` row using the position's own market, and the
> original `('us','india')` CHECK would have rolled back every crypto exit – found by reading the
> RPC body before shipping, not in production. Migration:
> `supabase/migrations/20260916020000_crypto_paper_pool.sql`.
> UI: Trading page's Crypto Watch section now shows the pool NAV, open positions, and recent
> closed trades (linking to `/dashboard/symbol/<coin>`); the symbol-detail page hides the Options
> tab and swaps `SymbolFundamentals` for a technical+macro composite view when the symbol is
> crypto (`app/api/charts/crypto-overview`).
> No genome/portfolio-constructor parity attempted ($2.4/$4 – future decision once real paper
> history exists). No benchmark comparison for the crypto book (no crypto analog to V00/ANSEI).
```

## 0. What this reopens, and what it doesn't

Decision 2 (2026-06-27) excluded crypto from the original long-only equity/ETF scope. This document proposes amending that exclusion for **paper trading only** – the same starting point every other asset class in this codebase got (US equities started paper, India started paper, ETFs started paper). Live crypto trading is explicitly **not** proposed here; it would need its own future decision after paper history exists, exactly like equities' own L3/L4 `lib/autonomy.ts` ladder already requires for everything else.

## 1. Verified current state

- Zero cryptocurrency-trading code exists anywhere in `lib/` or `app/` – confirmed by grep for every real crypto-trading reference (`place\_crypto\_order`, `BTC-USD`, `cryptocurrency`, etc.). Every other "crypto" hit in the codebase is Node's `crypto` module (hashing/encryption) or unrelated (`lib/property/crypto.ts`).
- Robinhood's MCP surface has real write capability: `place\_crypto\_order`, `preview\_crypto\_order`, `cancel\_crypto\_order`, plus reads (`get\_crypto\_positions`, `get\_crypto\_orders`, `get\_crypto\_quotes`, `get\_currency\_pairs`, `get\_crypto\_account\_onboarding\_info`). None are allowed or called by Kairos's `lib/robinhood-mcp.ts` today (`ROBINHOOD\_RESEARCH\_READ\_TOOLS` covers only earnings + options data).
- **Unverified**: whether the agentic account is actually crypto-onboarded at Robinhood (`get\_crypto\_session\_onboarding\_info` was not called this session – no live MCP session available). This is Stage 0, not assumed either way.
- The scoring/session schema keeps its `market` in `('us','india')` contract. Crypto does **not** become a third scoring market: it remains `market='us'` plus `InstrumentFamily='crypto'`. The isolated paper ledger is the narrow exception: its existing free-text `market` discriminator uses `crypto`, and the two generic infrastructure constraints needed by its paper exit (`market\_controls`, `decision\_journal`) admit that value. This avoids rewriting the 34+ scoring market constraints or routing crypto through US/India session, purge, and embargo primitives.
- The right existing mechanism already exists and is already live: `lib/scoring/instrument-taxonomy.ts`'s `InstrumentFamily` union (`operating\_company`, `adr`, `bank`, `reit`, five ETF families, three metals-adjacent equity families, `india\_etf`, `leveraged\_or\_inverse\_etf`, `unknown`) plus `InstrumentPolicy.scoreMode: "legacy\_v1" | "measure\_only" | "blocked"`. This registry was **already** built for staged rollout of exactly this kind of non-standard instrument – gold/silver bullion funds, miners, royalty streamers all got their own family with their own benchmark/exposure/score-mode before being trusted. `decision\_observations.features.instrument\_family\_evidence` already has 119 measure-only rows from this system running live. Crypto reuses this, not a new mechanism.

## 2. Design

### 2.1 Market stays `us`, family becomes `crypto`

Robinhood crypto settles in USD through the same agentic account already used for US equities. Add `crypto` to `InstrumentFamily`, `market: "us"`, `scoreMode: "blocked"` initially (matching the registry's own existing precedent – a family starts `blocked`, earns `measure\_only`, then `legacy\_v1` only after evidence). No `market` CHECK constraint anywhere needs to change.

### 2.2 The 24/7 problem – the one genuinely new piece of infrastructure

Crypto trades every hour of every day; every purge/embargo/session-fold/repricing-barrier primitive in this codebase assumes a closed market with a definable "next session." Two options, and this document picks the second:

- **(a) Bend crypto into the existing session model** – pick an arbitrary daily UTC cutoff and call it "the crypto session close." Rejected: this manufactures a fake session boundary for an asset that has none, and every purge/embargo calculation (`walkForwardFolds`, the h2/h5/h10/h20 forward-return labels) would be silently measuring something different from what it measures for equities – a stealth basis mismatch of the exact shape this codebase's own dimension-diagnostics work has repeatedly found and fixed (edge-readiness's flat-vs-horizon-aware spacing bug, the quote cross-check's session mismatch).

- **\*(b) A declared crypto session convention, explicit and separate.** A fixed daily UTC cutoff (e.g. 00:00 UTC) IS used, but it is documented and coded as `CRYPTO\_SESSION\_CUTOFF\_UTC`, a **convention for aggregation, not a claim about market structure** – the way `earnings-pit.ts`'s own comments distinguish a real fact from an operational convenience. `walkForwardFolds`, `computeSpearmanIC`'s date-clustering, and any purge/embargo logic that touches crypto rows must read this constant rather than the market-timezone functions used for `us`/`india`. This is a genuinely new piece of shared code (`lib/data/crypto-session.ts`), not a reuse of the existing `America/New\_York`/`Asia/Kolkata` functions with a new timezone bolted on – a fake "always open" timezone would break every place that assumes a session has a close.

### 2.3 Scoring composite – reuses the ETF precedent, doesn't invent a sixth dimension

Crypto has no P/E, no earnings, no analyst coverage, no insider transactions. Four of the five existing scoring dimensions (`fundamental`, `insider`, and effectively `sentiment`'s current news-driven sourcing) are structurally inapplicable or thin. `scoreFundamentals` already has a neutral-baseline branch for ETFs (`isEtf` parameter) – crypto gets the same treatment, not a new dimension:

- `fundamental`: neutral baseline (matches ETF precedent) – `dq.fundamentalDataAvailable = false`, excluded from the weighted composite via the existing `included`/renormalization mechanism (`computeWeightedAnalystScore`), not a fabricated neutral-50.
- `technical`: fully applicable – RSI/EMA/momentum compute the same way off crypto candles.
- `sentiment`: applicable if a crypto-relevant news/social source exists (needs a Stage 0 provider check – Alpha Vantage's news-sentiment endpoint covers some crypto tickers; unverified for Kairos's specific candidate list).
- `macro`: the existing US macro regime applies as-is; crypto is a US-account instrument.
- `insider`: not applicable (no insiders) – excluded, same mechanism as ETFs.

Net: a 2-3 dimension composite (technical, sentiment if available, macro), renormalized through the exact mechanism that already handles a dimension being unavailable – no new scoring function.

### 2.4 Genome – reuses existing bounds, does not fork per-family yet

Per the paired research plan (`docs/audits/2026-09-04-broker-capability-expansion-plan.md` §4), genome stays shared with equities for now. The one parameter worth flagging for Vaibhav's attention: `horizon\_days ∈ {2,5,10,20}` and `sizing.cap\_pct [5,15]` were bounded for equity volatility; crypto's realized volatility is materially higher, so the \*existing\* bounds may already be conservative enough to be safe (a wide stop/target band absorbs more volatility) – this needs measurement once paper data exists, not a guess now. No genome change proposed in this document.

### 2.5 Paper pool – new, not shared with the US equity pool

Reusing the existing \$10k US equity paper pool would let a crypto flash-crash swing the account NAV a paper-equity strategy is being judged against, contaminating the exact comparison the Alpha Diagnostic Lab and champion/challenger validation depend on. Precedent: India got its own pool for exactly this kind of contamination reason (Decision "Cross-market Trading Mandates"). Proposed: a third pool, `paper\_portfolio` row keyed `(market='us', instrument\_family='crypto')` or an equivalent scoping column – exact schema left to the migration design, not committed here.

## 3. Phased implementation

### Stage 0 – capability + coverage probe

- Verify agentic-account crypto onboarding (`get\_crypto\_account\_onboarding\_info`) – live call, read-only, no order.
- Verify RH crypto quote/candle data coverage for a real candidate universe (BTC, ETH, and whichever others RH lists) sufficient for technical scoring.
- Verify a sentiment source exists for crypto (Alpha Vantage news-sentiment coverage check).
- Stop cheaply if onboarding or data coverage fails.

### Stage 1 – instrument family + session convention (no trading)

- Add `crypto` to `InstrumentFamily`, `scoreMode: "blocked"`.
- Build `lib/data/crypto-session.ts` and wire it into any purge/embargo/fold code that will ever see a crypto row – additive, existing `us`/`india` paths unchanged.
- No scores, no candidates, no trades.

### Stage 2 – technical+sentiment+macro composite, measure-only

- `scoreMode: "measure\_only"`, matching the existing family-evidence pipeline (`instrument\_family\_evidence`) – nothing trades yet, but the composite's own predictive power gets measured honestly before it's trusted.

### Stage 3 – paper trading

- `scoreMode: "legacy\_v1"` for crypto only after Stage 2's evidence clears the same floors every other dimension in this codebase is held to (`MIN\_PREDICTIVE\_DATES`, etc.).
- New paper pool (§2.5). PaperTrader/PositionMonitor gain a crypto-aware path – reusing the existing per-market pattern, scoped by instrument family instead of market.
- Genome stays shared with equities (§2.4).

### Stage 4 – live (separate future decision)

- Not proposed here. Would need its own decision record and its own pass through the L3/L4 autonomy ladder exactly like equities did, after real paper history exists.

## 4. Acceptance criteria

1. No scoring/session market CHECK constraint changes – crypto research is `market='us'` plus a

```

new `InstrumentFamily`. Only the isolated paper-ledger/control/exit path may use
`market='crypto'`, and it must remain excluded from US/India aggregates by construction.
2. Crypto rows never enter a US-equity purge/embargo/fold calculation through the equity session
functions – mutation test: routing a crypto row through `America/New_York`-based session logic
must fail a detector.
3. Crypto's paper NAV never sums into the US equity paper pool's NAV.
4. `scoreMode` never jumps a stage – `blocked` → `measure_only` → `legacy_v1` in order, each gated on
the evidence the corresponding stage specifies.
5. No crypto order of any kind (paper or live) exists in Stage 0-2 code.
6. Live crypto trading requires its own separately-approved decision – this document authorizes
paper only.

5. Unresolved decisions requiring Vaibhav's approval

1. Approve Stage 0-3 (paper only) – or a narrower subset.
2. Which coins/candidates (RH's crypto list is not large; needs Stage 0's own inventory).
3. New paper pool starting balance and currency (USD, but how much – separate from the $10k US
equity pool per $2.5).
4. Whether the `CRYPTO_SESSION_CUTOFF_UTC` convention ($2.2) is the right call, or whether a
different aggregation convention is preferred.

Source: features\robinhood-mcp-integration\FEATURE_ARCHITECTURE.md
Feature Architecture: In-App Robinhood MCP Client – read-only snapshot + human-gated live order execution + allowlist-backed account select

Status

Architecture status: Implemented (OAuth + deterministic MCP path), 2026-07-07
Architecture approved: Yes (user, 2026-07-07)
Approved scope: OAuth 2.1 flow + deterministic MCP client + Gateway hardening + account allowlist. Live order execution stays behind robinhood
Approved date: 2026-07-07
Implementation allowed: Yes

Verified OAuth metadata (Robinhood live well-known, 2026-07-07)

Confirmed by fetching the endpoints directly – NOT guessed:
- authorization_endpoint = https://robinhood.com/oauth
- token_endpoint = https://api.robinhood.com/oauth2/token/
- registration_endpoint = https://agent.robinhood.com/oauth/trading/register
- scope = internal · PKCE S256 · token_endpoint_auth_method = none (public client)
- resource = https://agent.robinhood.com/mcp/trading
- No revocation_endpoint is published → Robinhood's own Agentic Trading
 dashboard is the authoritative remote revoke; in-app Disconnect wipes local
 tokens.

Remaining runtime unknown (fail-closed, not guessed)

The exact `place_equity_order` / `review_equity_order` argument *schema* over
MCP is discovered at runtime via `tools/list`; the adapter maps canonical
order fields onto the returned inputSchema and ABORTS if a required field
can't be confidently filled – it never sends a guessed real-money payload.
There is no Robinhood sandbox, so the FIRST live order should be a tiny qty,
watched, to confirm the field mapping before trusting it.

Revision history

- 2026-07-06 – original draft: read-only snapshot via DeepSeek+MCP, no order execution.
- 2026-07-07 (a) – scope expanded at user request: live order execution via the existing broker-adapter/Gateway pattern; `live_account_source`
- 2026-07-07 (b) – **this rewrite**: full security review (18 findings) folded in as binding rules. Biggest changes: **no LLM anywhere in the

Feature Purpose

Today, refreshing the live Robinhood account snapshot (equity, buying power,
positions for the read-only Trading account `<read-only-account-id>`) depends on
`execClaude` (`lib/claude-exec.ts`) – Windows-desktop-only, fails on Vercel.
Robinhood order execution (account `<agentic-account-id>`) is fully manual: approving a
proposal generates a natural-language command the user pastes into their own
Claude Code session with Robinhood MCP access.

This feature adds an in-app path for both concerns, using Robinhood's real
remote MCP server (`https://agent.robinhood.com/mcp/trading`, confirmed via
the user's `claude.json`; it exposes `get_accounts`, `get_equity_positions`,
`place_equity_order`, `review_equity_order`, `cancel_equity_order`,
`get_equity_orders`, among others):

1. **Read path** – a deterministic MCP client fetches the account snapshot
 server-side (Vercel or local), replacing/coexisting with `execClaude`,
 selected by a new `strategy_config.live_account_source` switch
 (`claude_exec` | `robinhood_mcp`, default `claude_exec`).
2. **Write path** – Robinhood becomes a third broker adapter in
 `lib/brokers/registry.ts` (alongside Alpaca and Kite), flowing through the
 Execution Gateway with the same two-human-click gate (Approve →
 Send to broker). The only change vs today is transport: a typed MCP call
 instead of a manual paste. **Who decides never changes.**

```

3. **\*\*Account selector\*\*** – a ``broker_accounts`` allowlist table + ``active_account_us`/`active_account_india`` so future additional accounts are user-selectable but never implicitly tradable.

## BINDING IMPLEMENTATION RULES (non-negotiable – deviation = reject the PR)

These rules exist because a security review found the original draft ambiguous enough to permit dangerous implementations. Each rule is stated so compliance is mechanically checkable.

### R1. No LLM in the write path – ever.

``submitOrder()`` MUST be a direct, deterministic ``client.callTool({ name: "place_equity_order", arguments: {...} })`` using `@modelcontextprotocol/sdk`'s `Client`` over ``StreamableHTTPClientTransport``. Arguments are built exclusively from the validated ``trade_proposals`` row and ``broker_accounts`` – never from, through, or "confirmed by" any LLM. No LLM output is ever parsed to derive order parameters, order status, or ``ok:true``. (Rationale: ``trader/route.ts`` already documents the hallucinated-fill failure mode this prevents.)

### R2. The read path is ALSO deterministic by default.

The snapshot fetch calls ``get_accounts`` + ``get_equity_positions`` directly via ``callTool`` and parses the structured results with a strict zod schema: numbers must be finite, ``symbol`` must match ``^[A-Z.-]{1,10}$``, arrays bounded (`≤500` positions). No LLM is required to call two read tools. An LLM summarizer MAY be layered on top for display text only, subject to R3.

### R3. LLM data-hygiene rules (applies to any optional summarizer).

- Vault tokens (``ROBINHOOD_MCP_``), ``CRON_SECRET``, and raw account numbers NEVER appear in any LLM prompt or LLM-visible tool argument. Account numbers are replaced with opaque labels ("Trading account") before prompting.
- MCP tool results are data, never instructions: the system prompt states this, and no numeric/actionable value from LLM output is written to any table. Loop iterations hard-capped.

### R4. OAuth 2.1 – do NOT copy the Kite route-pair internals.

- Kite's callback runs unauthenticated with no ``state`` (fine for Kite's checksum-signed flow, unsafe for OAuth). The Robinhood flow MUST have:
- ``state``: `≥128-bit` random, stored in an `HttpOnly, Secure, SameSite=Lax`, short-TTL (10 min) signed cookie; verified and single-use in the callback.
  - PKCE S256; the verifier stored in the same signed cookie (serverless-safe – in-memory storage does not survive between ``/login`` and ``/callback`` invocations on Vercel).
  - The callback REQUIRES an authenticated owner session (``createClient().auth.getUser()`` + the middleware owner gate) before exchanging the code – prevents login-CSRF/token-injection.
  - Exact registered ``redirect_uri``, strict string compare. Post-auth redirect is HARDCODED to ``/dashboard/settings?rhmcpc=connected`` – no ``next`` param.
  - Request the MINIMUM scope set Robinhood offers. If read-only scopes exist, the snapshot connection uses only those; if only a bundled trading scope exists, that fact is surfaced in the Settings UI text.
  - Related fix shipped alongside: ``app/auth/callback/route.ts`` must validate its ``next`` param (single leading ``/``, no ``//``, ``\\``, or ``@``) – current code allows ``?next=@evil.com`` open redirect.

### R5. Token handling.

- Tokens live only in ``api_key_vault`` (service-role-only as of migration 089) and are sent only in the ``Authorization`` header – never in URLs, never logged, never included in ``broker_orders.raw_last_state`` (redact before persisting any ``raw`` payload; also redact account numbers).
- Refresh is single-writer: compare-and-swap on the vault row (``UPDATE ... WHERE updated_at = <value read>``), losers re-read instead of overwriting – prevents the rotating-refresh-token race between concurrent cron + user invocations from bricking the connection.

### R6. Gateway modifications (the Gateway is NOT "unchanged").

``app/api/broker/orders/route.ts`` today lacks several gates the original draft wrongly claimed it had. This feature MUST add, in order, before ``submitOrder``:

1. ``guardOrderRequest(req)`` (Host/Origin check) – extended to read the allowed host from an ``APP_BASE_URL`` env var so it works on Vercel (current guard allowlists localhost only; do NOT delete the guard to "fix" Vercel).
2. ``env`` MUST be explicitly present in the request body; reject if absent. No silent ``?? "paper"`` default on an order-placing route.
3. ``broker.envs.includes(orderEnv)`` checked in BOTH directions (a live-only broker rejects paper; a paper-only broker rejects live). The adapter additionally self-rejects wrong env (defense in depth).
4. ``checkKillSwitches(supabase)`` – kill switches currently run only at proposal build/approve, not at submission. They MUST also run here.
5. Existing checks retained: proposal ``status='approved'``, ``approval_expires_at``, global ``trading_enabled`` + per-market ``trading_enabled_us`/`trading_enabled_india`` (live env).
6. Submit-time re-validation (live env): ``qty`` positive integer `≤` hard cap;

fresh quote fetched and ``qty * quote ≤ max_order_notional`` (new ``strategy_config`` column, default `≤15%` of latest ``live_account_snapshots.equity``); price drift vs ``proposal.price_at_proposal`` `≤` the same threshold ``trader/route.ts`` uses at approval; ``symbol`` matches `^[A-Z\.-]{1,10}$`; ``side='sell'`` allowed only if the symbol exists in the current holdings snapshot (long-only for new positions per CLAUDE.md).

7. Duplicate-submit hardening: partial unique index ``broker_orders(proposal_id) WHERE status IN ('pending_submit', 'submitted', 'partially_filled')`` – the existing check-then-insert race allows double-submit on concurrent clicks.

### R7. Ambiguous-failure semantics (the case that duplicates real orders). If ``place_equity_order`` times out or errors AFTER possibly succeeding, the adapter MUST NOT resubmit. It marks the order row ``status='unknown_needs_reconcile'`` and the sync loop (or a manual button) calls ``get_equity_orders`` to reconcile before any human is allowed to retry. Where Robinhood's MCP supports it, use ``review_equity_order`` → place sequence and/or a client-order-id for idempotency. No automatic retry of any write call, ever.

### R8. Account allowlist – fail closed, hardcode stays.

- Validation lives in BOTH the Gateway (before ``submitOrder``) and inside ``submitRobinhoodOrder()`` (last code before the wire): resolved account must exist in ``broker_accounts`` with ``role='trading'`` and matching market.
- ANY error reading ``broker_accounts`/`active_account_us`` ABORTS the order. The silent fallback-to-default pattern used by ``getActiveBroker()`` for broker selection is explicitly FORBIDDEN for account selection.
- The existing hardcode (``AGENTIC_ACCOUNT = "<agentid-account-id>"`` in ``trader/route.ts``) is NOT removed in this feature. Removal is a separate, later change gated on the allowlist being verified live. Until then both checks run.
- New accounts default ``role='view_only'``. Becoming a trading target requires BOTH explicitly setting ``role='trading'`` AND selecting it as ``active_account_us`/`active_account_india``. No auto-discovery from broker APIs marks anything tradable.
- ``broker_accounts`` ships service-role-only (no anon/authenticated grants, no permissive policy) – a client-side writer to this table would defeat the entire allowlist. Migration must be verified applied to the live DB before the validation code merges (global schema rule).

### R9. Kill switches and flags (contradiction in prior draft resolved).

- ``robinhood_mcp_enabled`` (new, `**default FALSE**` – a live-order integration's kill switch must not ship pre-armed ON; user flips it on in Settings after connecting): when false, BOTH the MCP snapshot fetch and the MCP order path are blocked. The last cached snapshot in ``live_account_snapshots`` remains viewable – "blocked" means no new MCP calls, not hidden data.
- ``trading_enabled_us`` off: blocks live orders (all US brokers), never blocks any read.
- In-app Disconnect: deletes vault tokens even if Robinhood's revocation endpoint is unreachable (same contract as ``disconnectKite()``), and the Settings UI states that Robinhood's own Agentic Trading dashboard is the authoritative, app-independent kill switch.

### R10. Cron/auth surface fixes shipped with this feature.

- New shared ``verifyCronSecret(req)`` helper using ``crypto.timingSafeEqual`` and rejecting when ``CRON_SECRET`` is unset/empty; used by the new routes (and the snapshot routes it touches). Repo-wide migration of the other `~30`===`` comparisons can follow separately.
- ``GET /api/live-account/snapshot`` currently returns live equity/positions with NO auth – it MUST require an authenticated owner session before this feature writes fresher data into it.
- The ``robinhood_mcp`` snapshot branch writes to ``live_account_snapshots`` directly via the service client – NOT via the existing loopback HTTP POST that ships ``CRON_SECRET`` to ``NEXT_PUBLIC_APP_URL``.

## Scope

- OAuth 2.1 client per R4: ``/api/robinhood-mcp/login`` + ``/api/robinhood-mcp/callback``.
- Token storage per R5 in ``api_key_vault``.
- ``lib/robinhood-mcp.ts``: OAuth helpers, vault-backed token storage/refresh (CAS), ``queryRobinhoodAccount()`` (read, R2), ``submitRobinhoodOrder()`` (write, R1/R7/R8 – called only by the broker adapter).
- ``lib/brokers/robinhood-mcp.ts``: adapter implementing the standard interface (``id: 'robinhood_mcp'``, ``envs: ['live']``, ``isConfigured()``, ``submitOrder()``), registered as a second US-market option.
- Gateway modifications per R6.
- ``strategy_config.live_account_source`` + Settings selector ("Local – Windows Scheduler + Claude Code" vs "Cloud – Robinhood MCP").
- ``broker_accounts`` table + ``active_account_us`/`active_account_india`` + Settings UI per R8 (manual add form: user types the account number and picks the role explicitly; no bulk import).
- Settings card "Robinhood MCP (Live Account + Orders)": connection status (``connected: boolean`` + ``updated_at`` only – never token material),

Connect/Re-authorize, Disconnect, `robinhood\_mcp\_enabled` toggle,  
`live\_account\_source` selector, authoritative-kill-switch note.

## ## Non-Goals

- \*\*Any autonomous order placement.\*\* Every order still requires (1) human Approve on the proposal, then (2) human "Send to broker" click. This feature changes transport for step 2 only.
- Removing the manual Claude-Code-paste flow (stays available: simply don't select `robinhood\_mcp` as `active\_broker\_us`).
- Removing the `` hardcoded (separate later change, R8).
- Options trading: `place\_option\_order` is never called; the adapter supports equities only.
- Auto-discovery of accounts from broker APIs.
- Building the OAuth routes before the user has confirmed Robinhood's actual endpoints/scopes against the Agentic Trading dashboard.

## ## Data Models

- `api\_key\_vault`: `ROBINHOOD\_MCP\_ACCESS\_TOKEN`, `ROBINHOOD\_MCP\_REFRESH\_TOKEN` (service-role-only per migration 089 – already applied live).
- `strategy\_config.live\_account\_source` – text, `claude\_exec` | `robinhood\_mcp`, default `claude\_exec`.
- `strategy\_config.robinhood\_mcp\_enabled` – boolean, \*\*default false\*\* (R9).
- `strategy\_config.max\_order\_notional` – numeric, default null → computed as 15% of latest live equity (R6.6).
- `strategy\_config.active\_account\_us` / `active\_account\_india` – text, default today's hardcoded values.
- `broker\_accounts` – `id, broker, market, account\_number, label, role` ('trading' | 'view\_only'), created\_at; service-role-only; seeded `agent-ic`.
- `broker\_orders` – new partial unique index per R6.7; new status value `unknown\_needs\_reconcile` (R7). RLS enabled + client grants revoked (R7).

## ## Error Handling

- Read-path MCP/OAuth failure → snapshot stays stale, `{ok:false, error}` logged (Decision 45 contract). Never throws away the cached snapshot.
- Write-path failure before the wire → order row `status='error'` with the real message. Possible-success ambiguity → R7 reconcile flow. No silent retry anywhere.
- Account/allowlist resolution failure → abort (fail closed, R8).

## ## Files / Behavior That Must Not Change

- Phase 0: proposals always `pending\_approval`; auto-approve never permitted.
- Long-only enforcement for new positions; SELL only on held positions – now also enforced at Gateway submit time (R6.6).
- The existing `execClaude`-based mentor/portfolio/chart-data features.
- `trader/route.ts`'s manual-paste flow remains available and unchanged except for adding allowlist validation alongside (not replacing) the account hardcoded.

## ## Acceptance Criteria (each is a hard review gate)

1. No code path derives order parameters or order status from LLM output (R1). Grep-level check: `place\_equity\_order` appears only inside `submitRobinhoodOrder()`, and that function contains no LLM call.
2. Snapshot fetch works with zero LLM calls (R2); any summarizer failure cannot block or alter stored numbers.
3. OAuth callback rejects: missing/mismatched `state`, replayed `state`, absent owner session (R4).
4. Tokens never appear in URLs, logs, LLM prompts, or persisted `raw` payloads (R5) – verified by grep + a redaction unit test.
5. Gateway rejects: absent `env`, env not in `broker.envs`, kill-switch tripped, notional over cap, price drift over threshold, sell of unheld symbol, account not in allowlist (R6, R8) – each with a unit test.
6. Ambiguous submit timeout produces `unknown\_needs\_reconcile`, never a resubmit (R7) – unit test with a mocked timeout.
7. `robinhood\_mcp\_enabled=false` blocks new MCP calls (read+write) while cached snapshot stays viewable; default is false (R9).
8. Disconnect wipes local tokens even when Robinhood is unreachable; Settings names Robinhood's own dashboard as the authoritative kill switch (R9).
9. `GET /api/live-account/snapshot` requires owner auth (R10).
10. Adding a `broker\_accounts` row never makes it tradable without explicit `role='trading'` + active-account selection (R8).
11. All new tables/columns verified applied to the live DB before dependent code merges (global schema rule).

## ## Approval

Architecture approved: No  
Approved scope: None  
Implementation allowed: No

---

# Source: features/router-cutover\FEATURE\_ARCHITECTURE.md  
# Canonical Evidence Router Controlled Cutover

> Status: \*\*REVIEWED DESIGN DRAFT. NOT APPROVED FOR IMPLEMENTATION OR CUTOVER.\*\*

> Last reviewed: 2026-07-15 by Codex.  
> Companion authority: `features/data-source-policy/FEATURE\_ARCHITECTURE.md`.  
> Scope: score-affecting Router activation and rollback, independently per market.

## ## 1. Current State And Correction

The Router is shadow-only. `resolveEvidence` rejects ordinary callers while the active immutable policy has `router\_enabled=false`; only an explicit shadow caller may bypass that gate. No scorer currently consumes `EvidenceEnvelope`.

Cutover is not “flip a boolean.” Policy versions are immutable. Enabling or disabling the Router means creating and atomically activating a new complete market-local policy version. The policy ID is frozen at run start.

The earlier proposed runtime check (“score with and without the changed dimension”) is insufficient. It cannot reliably distinguish a genuine new fact from provider availability, and one-symbol recomputation misses cohort-rank changes. Eligibility safety therefore has two separate controls: a same-input policy activation gate and a runtime evidence-degradation gate.

## ## 2. Invariants

1. US and India policies, universes, caches, evaluations, signals, and rollback are independent. Activate one market and one score-affecting intent family at a time.
2. A run freezes policy version, strategy/genome, universe, as-of, and evidence snapshot. Mid-run policy activation affects only the next run.
3. Source changes must preserve field semantics: period, basis, unit, currency, adjustment, observation time, and knowledge time. “Same or better” coverage is not semantic parity.
4. No missing, stale, conflicting, quarantined, or schema-invalid evidence is converted to zero/neutral.
5. Provider/routing change alone cannot create a newly eligible long.
6. Existing trading, market-control, kill-switch, drawdown, mandate, rank, approval, and execution gates remain unchanged and are rechecked at their existing sites.
7. Cutover changes research evidence acquisition only; it grants no live autonomy.

## ## 3. Intent Classification

Maintain a code-owned classification:

- `score\_affecting`: canonical fields consumed by deterministic dimensions;
- `eligibility\_affecting`: prices/events used by filters, freshness, ranks, or gates;
- `narrative\_only`: display/context that cannot change score or eligibility; and
- `unsupported`: no valid market capability.

Do not require `analyst.consensus` parity for India or make US narrative analyst data a scoring cutover blocker. Each scorer field declares its required intent, minimum field contract, acceptable quality/freshness, and legacy mapping.

## ## 4. Frozen Dual-Run Evaluation

For each candidate policy version, build one immutable evaluation cohort per market:

1. freeze universe, as-of, active baseline policy, candidate policy, strategy/genome, threshold, ranks, and price basis;
2. resolve/cache raw provider observations once where contracts permit;
3. construct legacy and candidate canonical inputs from the same knowledge cutoff;
4. run the full deterministic cohort scorer and all pre-trade eligibility/rank gates;
5. persist field, quality, availability, score, rank, direction, and eligibility deltas with evidence fingerprints; and
6. classify every divergence with a bounded code and reviewer note.

The comparison must include symbols where either path abstains or fails. It cannot compare only jointly successful rows. A reverse-shadow run after cutover reuses the same frozen observations/cache; it must not double provider bursts.

### ### Semantic parity

Each canonical field has an explicit comparator:

- exact match for enums, dates, currency, basis, period, and nullability;
- documented numeric tolerance only after unit/period normalization;
- no tolerance across TTM/quarterly/annual/forward or adjusted/unadjusted bases;
- field-level provenance and conflict state retained; and
- unexplained disagreement fails the candidate, even if aggregate score is close.

## ## 5. Activation-Time Eligibility-Flip Gate

Before a score/eligibility-affecting policy may activate, its immutable evaluation must prove on the full cohort:

- zero unexplained `ineligible/abstain -> eligible` transitions;
- zero newly eligible transitions caused only by source availability, stale fallback, field omission, conflict resolution, basis mapping, or weight renormalization;
- no material adverse rank displacement attributable only to missingness;

- no lower required-field coverage versus baseline beyond a pre-approved non-inferiority margin; and
- no schema, currency, market, provenance, or secret-redaction failures.

A genuine evidence-value change may create eligibility only when field semantics are equivalent, the observation is valid at the shared as-of, and the divergence is explicitly classified and owner-approved. Added provider coverage is first measured; it is not silently treated as approval to trade more names.

The activation API/RPC must bind approval to the exact candidate version, baseline version, evaluation ID, evaluation code version, strategy version, market, and expiry. A stale evaluation cannot authorize a new policy or changed scorer.

#### ## 6. Runtime Evidence-Degradation Guard

Activation parity cannot prevent later outages. At each research run:

1. evaluate each symbol against the code-owned minimum field/quality contract;
2. compare the current availability/quality mask with the last accepted market-local baseline and record transition reasons;
3. if a required field degrades ('fresh' -> 'stale beyond ceiling', 'available' -> 'missing, valid' -> 'conflict/quarantined'), abstain from any new long whose eligibility depends on renormalizing around that degradation;
4. apply the existing thin-evidence floor, but do not treat two dimensions as a universal assurance when a required gate field is absent;
5. existing holdings continue through PositionMonitor's normal risk/exit logic; evidence outage cannot suppress stops or mandatory exits; and
6. emit an aggregated health event, not one alert per symbol.

This guard is deterministic and records baseline/current masks, score/rank counterfactuals, reason codes, and policy/evidence IDs. It defaults to abstain for new entries, never to a more permissive score.

#### ## 7. Release Evidence Floors

Calendar days alone are insufficient. For each market and intent family require:

- at least ten distinct trading sessions within the selected session's prior 45 calendar days and a representative liquid universe;
- symbol-intent sample counts and sector/industry coverage declared in the report;
- coverage non-inferiority with uncertainty, not only a point estimate;
- organic cache/live/fallback/no-data cases plus forced timeout, quota exhaustion, schema drift, provider disagreement, and complete-primary-provider outage drills;
- zero unexplained entry flips and zero market/currency/provenance failures; and
- owner review of every material semantic divergence.

Exact numeric sample floors and tolerances are approved from observed traffic before release and then versioned. They cannot be weakened in Settings. India must satisfy its own floors; US results provide no evidence for India.

#### ## 8. Cutover Sequence

1. Complete score-field ownership and legacy-to-canonical compatibility mapping.
2. Add frozen dual-run evaluation and activation binding.
3. Add runtime degradation guard and cohort-rank tests.
4. Keep legacy acquisition warm; run and document a rollback drill.
5. Pilot a narrative-only intent if useful, then one score-affecting intent family in one market behind a code-owned compatibility adapter.
6. Create and activate a new 'router\_enabled=true' policy only after all gates pass.
7. Run candidate primary plus legacy reverse-shadow from shared frozen inputs.
8. Expand intent families, then the second market, through separate evaluations.
9. Remove legacy code only after a separately approved observation period and a release proving no rollback dependency remains.

Do not flip all intents or both markets together.

#### ## 9. Rollback And Circuit Breakers

Rollback creates/activates a complete disabled/legacy policy version; it does not mutate history. Before cutover, prove that legacy credentials, pacing, cache, schemas, and code still work under realistic load. Define rollback SLO and owner/automatic triggers for:

- unexplained new eligibility or rank displacement;
- required-field coverage below the approved floor;
- cross-market/currency/basis/provenance failure;
- sustained schema/contract errors, circuit open, or quota exhaustion; and
- evidence-record persistence failure.

Automatic rollback may switch acquisition policy but cannot alter signals/orders already created. New-entry research pauses during ambiguous partial rollback; the normal position/execution safety system continues for holdings and exits.

#### ## 10. Data And Audit Changes Required Before Build

Extend 'evidence\_policy\_evaluations' or a linked append-only detail table to retain:



- frozen cohort/snapshot and strategy/genome fingerprints;
- candidate/baseline policy and evaluation-code versions;
- per-field semantic/quality/availability deltas;
- score, rank, direction, and eligibility before/after;
- classified flip cause and reviewer disposition;
- coverage denominators, outage-test results, call usage, and expiry; and
- activation/rollback proof.

Do not store only aggregate `eligibility\_flips`. RLS is owner-read, writes are service-role-only, RPCs use fixed `search\_path`, grants exclude anon/authenticated, and evaluation rows are append-only.

#### ## 11. Required Tests

- policy and strategy freeze across mid-run activation;
- full-cohort ranking under one added/removed/degraded dimension;
- false-to-true eligibility caused by renormalization is blocked;
- genuine same-contract value change is classified, not confused with availability;
- added coverage cannot bypass activation review;
- stale/conflict/quarantined/missing and zero are distinct;
- required-field gate overrides the generic two-dimension floor;
- reverse shadow makes no duplicate live provider burst;
- unsupported India intents abstain without reducing US coverage;
- policy activation rejects stale/wrong-market/wrong-baseline evaluation;
- rollback under load restores the proven legacy path within the SLO;
- position exits and all execution brakes remain active during Router outage; and
- no Router/provider configuration can call an order adapter.

Release still requires typecheck, full tests, production build, schema/RLS/grant verification, read-only production probes, and per-market shadow/rollback reports.

#### ## 12. Acceptance

Cutover is acceptable only when a future run can reproduce exactly which policy, facts, availability transitions, score, rank, and gates produced each decision; a provider outage cannot create a new long; both markets can roll back independently; and the owner sees an explicit evaluation rather than a generic "parity passed."

#### ## 13. Owner Decisions Before Build

1. Approve the intent/field ownership matrix.
2. Approve observed sample floors, tolerances, evaluation expiry, and rollback SLO.
3. Choose the first market and first score-affecting intent family after shadow data is sufficient.
4. Approve the exact legacy-retirement observation period separately from cutover.

---

#### ## 14. Implementation Notes – Prerequisites Built (2026-07-16)

```
> **NOT A CUTOVER.** `router_enabled` remains `false` for BOTH markets; no enabled
> policy version exists; no scorer consumes `EvidenceEnvelope`. Verified in
> production after this change: both markets' active policy version is v1,
> `router_enabled=false`, and zero evaluations exist. This section records the
> machinery that must exist *before* a future owner-approved cutover.
```

##### ### §3 – Intent classification (`lib/evidence/intent-classification.ts`)

Code-owned. `classifyIntent(intent, market)` returns  
`score\_affecting | eligibility\_affecting | narrative\_only | unsupported`. Per §3:  
`analyst.consensus` is \*\*narrative\_only in US, unsupported in India\*\*, and  
`gatedIntents()` excludes it in both – it can never block a scoring cutover, and  
India parity for it is not required. `SCORER\_FIELD\_CONTRACTS` declares, per field:  
required intent, minimum field contract (`minFields`/`minCount`), acceptable  
quality + freshness ceiling, allowed bases, structural applicability, and the  
legacy module/symbol it maps to today. `contractClassViolations()` is asserted by  
tests, so a narrative intent cannot quietly acquire a scoring dimension.

\*\*Required (gating) fields:\*\* `technical.daily\_bars` (all shapes) and  
`fundamental.reported\_core` (equity/ADR only). Everything else is optional and  
renormalizes exactly as today. `sentiment`/`insider` are deliberately NOT required  
– they are genuinely sparse (india:sentiment, us:insider), and requiring them  
would abstain on healthy runs.

##### ### §6 – Runtime degradation guard – \*\*SHIPPED IN MEASURE-ONLY MODE\*\*

- Pure core: `lib/evidence/degradation-guard.ts`. Impure shell:  
`lib/evidence/degradation-runtime.ts`. Wired into `lib/research-agent.ts`  
immediately after `resolveSignalDirection`.
- \*\*Mode:\*\* `EVIDENCE\_DEGRADATION\_GUARD\_MODE` ∈ `off | measure\_only | enforce`,  
\*\*defaulting to `measure\_only`\*\* – it records what it \*would\* abstain and  
changes no direction. An unparseable value falls back to `measure\_only`: a  
broken config can never silently enforce, nor silently stop recording.
- \*\*Strictly subtractive.\*\* `applyDegradationGuard` has exactly one  
transformation: `long → neutral`. `short` (deterministic exit on a holding) and

- `neutral` return untouched in *every* mode – checked before the mode branch, so no configuration can make the guard interfere with a sell. This is also enforced in the schema (`degradation_guard_is_subtractive` CHECK), so no future code path can persist a guard event that created or upsized an entry.
- **Trigger:** *a required, applicable* field is unusable **and** the scorer renormalized around it (§6.3 – “eligibility depends on renormalizing around that degradation”). A required field unusable but *not* renormalized around did not shape the decision, so it does not abstain. §6.4 holds: the required-field gate fires even when  $\geq 2$  other dimensions are present.
- **Defaults to abstain:** *no baseline + unusable required field* → `no_baseline_required_field` → abstain. A persistent outage keeps abstaining; it never normalizes, because only a clean run is promoted to baseline.
- **One current-condition health event per market** (`evidence-degradation:<market>`), never one per symbol or historical run; the detail names the current run and a clean run resolves the condition. Run-level history belongs in the append-only degradation-event ledger, not System Health.
- **Known limitation (honest):** the legacy path supplies availability and contract satisfaction but *not* observation age, so `ageSeconds` is reported as `null` rather than an invented zero. In practice the legacy path therefore exercises `available→missing` and `contract_broken`, not `fresh→stale`. The core handles age/quality transitions and they are unit-tested; real ages arrive with the Router once an intent family is cut over.

### ### §4/§5 – Frozen dual-run evaluation + activation binding

- `lib/evidence/evaluation/parity.ts` – semantic comparator. Semantic axes (nullability → provenance → basis → period → currency → unit → adjustment → conflict) are all checked **before** any value comparison, so a numeric tolerance is unreachable while the two values describe different facts. Cross-family bases (TTM/quarterly/annual/forward) and adjusted/unadjusted are hard mismatches even when the numbers are identical. Unknown fields default to exact-match.
- `lib/evidence/evaluation/cohort.ts` – freezes universe/as-of/baseline/candidate/strategy/threshold/ranks/price basis; scores **both** paths with the production scorer and production direction gate (never a re-implementation); ranks cohort-wide; classifies every flip with a bounded cause. Rows where either path abstains or fails are first-class. Artifact causes (availability, stale fallback, field omission, conflict resolution, basis mapping, renormalization) can never create eligibility; a `genuine_value_change` is classified as such but cannot self-approve. Becoming *more* conservative never blocks.
- `lib/evidence/evaluation/persist.ts` + `activate_evidence_policy_bound()` – the RPC binds approval to candidate + baseline + evaluation ID + evaluation code version + strategy version + market + expiry, and additionally requires the evaluation's baseline to still be the *active* policy and every flagged divergence to carry an approving review row. Production-probed: unknown/stale evaluation and invalid market are both refused.

### ### §8 – Massive candle adapter split (Codex pre-cutover finding)

`price.daily_bars` was one *massive* adapter wrapping `fetchUsCandles()`, which silently fell back Massive→EODHD→TwelveData while reporting `providerId: "massive"`. Three harms: provenance named a nominal source; `mode: "only"` could not actually pin a provider; three providers' pacing/budget accounted as one. Now `lib/evidence/adapters/bars.ts` builds one adapter per source (`massive-bars-v1`, `eodhd-bars-v1`, `twelvedata-bars-v1`), each hitting exactly one provider and refusing a payload that claims another source. The **router** owns the fallback via the registry chain. Contract version bumped from `us-bars-v2` so multi-source cache rows are not read back under the new single-source contract. Note: the resolver's `MAX_SYNC_ATTEMPTS=2` means the third source is reached via the refresh queue, not synchronously – a deliberate Vercel wall-clock bound.

### ### Migration

`supabase/migrations/20260716210000_router_cutover_prerequisites.sql` → `evidence_field_baselines` (mutable – it is a moving reference point, not evidence), `evidence_degradation_events`, `evidence_evaluation_details`, `evidence_evaluation_reviews` (all append-only via `no_mutate`), plus frozen-cohort and binding columns on `evidence_policy_evaluations`. RLS on + owner-read policy on all four; anon has no grants; writes are service-role only; the RPC is *security definer* with `search_path=public` and executable by `service_role` only.

### ### Deferred after the 2026-07-21 hardening build

- Rollback SLO and any future weakening of approved floors remain owner decisions. Section 16 locks the current session, coverage, score, rank, and TTL values.
- The outage drills (§7) – forced timeout, quota exhaustion, schema drift, provider disagreement, and complete-primary-provider outage remain unbuilt.
- §9 rollback drill + circuit-breaker triggers.
- The owner-only activation API exists, but no general evaluation-management UI is built and both active policies remain disabled.

---

### ## 15. Implementation Notes – Cohort Builder Built (2026-07-18)

> Historical v1 record. Section 16 supersedes its provider-burst allowance,

```
> availability-only scoring, placeholder thresholds, and manual-only schedule.

> **STILL NOT A CUTOVER.** Verified in production immediately before and after this
> change: both markets' active policy is v1 with `router_enabled=false`, and no
> scorer consumes `EvidenceEnvelope`. This step only makes the FIRST parity
> evaluations exist. It never activates anything.
```

### What was built

```
`lib/evidence/evaluation/cohort-builder.ts` – the piece §14 deliberately deferred
because "a builder that fetches is the step that risks provider bursts."
Exposed as `GET/POST /api/agents/evidence-cohort?market=us|india&limit=N[&dryRun=1]`
(owner- or cron-gated, mirroring `evidence-shadow`).
```

### §4.2 – ONE frozen observation set, and why the reuse is structural

The governing constraint is that a dual-run must not double provider calls. The build resolves **one** market-local frozen observation set via the EXISTING `resolveEvidence` (`allowDisabledPolicy: true`), which shares `evidence_cache_v2` with the `evidence-shadow` harness – so a warm cache short-circuits at zero cost.`

`assembleCohort` is pure and takes the snapshot as an argument`. It has no resolver and no network. The reverse-shadow leg therefore cannot re-fetch: its only path to evidence is the frozen `Map`. The reuse is a structural property, not a caching optimisation that a later refactor could quietly undo.`

An earlier draft resolved twice (primary + reverse) and was **caught** by its own ledger proof – the reverse pass made 12 real bursts. That is precisely the defect the constraint exists to prevent, and it is why `verifyReverseShadowReuse` now confirms every `(symbol,intent)` is served from the frozen set rather than re-resolved.`

### The two legs – and what v1 honestly compares

- **legacy** = a real recorded production decision. `agent_signals` joined to its `research_packets.raw_data` supplies the exact persisted availability mask (`data_quality`) and the weights that actually drove the score (`profile_weights`). The mask is rebuilt with research-agent's own rule, so the leg reproduces the recorded analyst_score` exactly` – asserted per symbol and reported as `legacyReproduction` (27/27 matched across the first two runs). A reconstruction that did not reproduce production would be a guess, not a baseline.`
- **candidate** = the SAME frozen dimension scores, with availability decided by the router snapshot.

**v1 scope is AVAILABILITY + eligibility-flip parity**, and says so. Legacy never persisted raw field values or per-field provenance, so a value/basis/period comparison against it would be fabricated. Both legs therefore carry the field's canonical contract semantics, `value`/`periodEnd` are left null (not compared), and the real serving provider is recorded for audit. Note the consequence, stated plainly: because both values are null, `compareField` short-circuits at the nullability axis, so basis/period/currency/unit are not exercised on the production path in v1 – they are exercised in tests, and become live when the legacy path is itself routed. This mirrors the honesty already shipped in the degradation guard (ageSeconds: null`).`

Dimension scores are **not re-derived** from router evidence – no scorer consumes `EvidenceEnvelope` yet, and inventing one here would mean the evaluation stopped predicting the thing it claims to predict. The production scorer and production direction gate are reused unchanged, via `evaluateCohort`.`

### First evaluations (real, persisted)

| Market | Symbols | Passed     | Flips                              | Finding                                                                                         |
|--------|---------|------------|------------------------------------|-------------------------------------------------------------------------------------------------|
| US     | 15      | <b>yes</b> | 0 new-eligible / 0 new-ineligible  | Required fields at parity: <code>technical.daily_bars` and `fundamental.reported_core` b</code> |
| India  | 12      | <b>no</b>  | 0 new-eligible / 10 new-ineligible | <code>coverage_below_non_inferiority_margin` on `technical.daily_bars`: candidate 0.09</code>   |

The India failure is the gate working, not a defect: US results provide no evidence for India (§7), and the two were evaluated independently. Scores moved materially in the US cohort (deltas -28 to +5) without a single threshold crossing – verified per symbol rather than inferred from the zero-flip count.

### Ledger proof (§4.2), from `provider_call_ledger``

| Run                  | Ledger rows | Fresh cache | Real bursts |
|----------------------|-------------|-------------|-------------|
| US primary           | 50          | 40          | 10          |
| US reverse-shadow    | <b>0</b>    | 0           | <b>0</b>    |
| India primary        | 12          | 6           | 6           |
| India reverse-shadow | <b>0</b>    | 0           | <b>0</b>    |

The reverse leg wrote **zero ledger rows** while serving all 75 (US) and 60 (India) `(symbol,intent)` pairs from the frozen set. Dual-run provider cost therefore equals the single primary pass – the primary's bursts are the explicit cache misses the constraint allows. A denied` lease is deliberately not counted as a burst: it is the pacing system refusing a call, i.e. the protection working.`

```

Deliberately NOT done

- No migration. Every column `persistEvaluation` writes already existed.
- No activation, and no change to `router_enabled` (still false, both markets).
- $7 numeric floors, tolerances, and evaluation TTL remain owner decisions ($13.2).
 The builder's `coverageNonInferiorityMargin` (0.05) and `maxAdverseRankDisplacement`
 (5) are **placeholders pending that approval**, not approved thresholds.
- ADR shaping: `shapeOf` maps every non-ETF/non-metal symbol to `equity`, so a US ADR
 over-cludes the insider field. That is conservative (a coverage miss, never a
 fabricated eligibility) but it is an approximation, not a modelled distinction.

16. Approved Evidence-Hardening Build (2026-07-21)

Decision record

- **Why:** the daily Router shadow is fresh, but the cutover cohort ran only once,
 could spend provider quota on cache misses, and its single `passed` bit did not
 distinguish entry safety from loss of useful score evidence.
- **Who:** the single Kairos owner operating separate US/USD and India/INR books.
- **ROI:** make cutover evidence repeatable and quota-neutral while preventing a
 technically conservative but materially blinder candidate from being described
 as full parity.
- **Shipped at:** `866548b3` + `cac4cffa`; production verified 2026-07-21.

Scope and invariants

1. **Cohorts are cache-only.** A cohort may read fresh or policy-allowed stale
 `evidence_cache_v2` rows. It may not acquire a provider lease, enqueue a refresh,
 or call an adapter. A cold field remains unavailable and the evaluation records
 the miss. `primary_provider_calls` must equal zero or the evaluation fails.
2. **Research-output compatibility bridge.** During the existing ResearchAgent
 fetch, Kairos writes canonical cache rows from the data already in memory:
 fundamentals, bars, sentiment, macro, and insider. This is one bounded database
 upsert, not another provider request. Provenance names the compatibility bridge
 and retains the actual serving source in the canonical payload. It is a
 transition adapter, not a new source; future provider-native policies must be
 evaluated separately before removing legacy acquisition.
3. **Market-wide evidence is market-wide.** `macro.regime_inputs` uses the reserved
 `__MARKET__` cache key and is resolved once per market/run. It is never fanned
 out into one provider/cache operation per symbol. India macro remains
 unavailable: the US MacroSentinel cannot be relabelled as India evidence.
4. **Exact score-input replay.** Bridge payloads carry the deterministic dimension
 score and the score input/evidence that produced it. The candidate leg uses that
 score rather than reusing the legacy score by assumption. Provider-native
 canonical payloads must derive the dimension score through the same production
 scoring functions before they can satisfy this gate.
5. **Two machine verdicts.** `safety_pass` covers required-field floors, semantic
 failures, and artifact-created new eligibility. `quality_pass` covers all
 score-affecting coverage loss, material total-score drift, and adverse rank
 displacement. `passed = safety_pass AND quality_pass`; activation requires all
 three fields true.
6. **Rolling proof.** An enabled policy cannot activate from one snapshot. The
 bound activation RPC requires ten distinct market-session evaluations for the
 same market/candidate/baseline/evaluation-code/strategy tuple, all safety- and
 quality-passing within the selected session's prior 45 calendar days, with
 the selected evaluation still unexpired. The session date comes from
 executable ResearchAgent rows (`session_validated=true`, `as_of_session`), so
 weekend/holiday staged rows cannot inflate the count. US and India
 histories never satisfy one another.
7. **Schedule only after hardening.** Run one cohort after each market's final daily
 shadow tick. The route remains shadow-only and both active policy versions keep
 `router_enabled=false`.
8. **Evidence binds the real candidate.** Seed one immutable, inactive
 `router_enabled=true` candidate per market by copying that market's active rules.
 Cohorts resolve cache order under that exact candidate ID. The active pointer is
 not changed, so production remains on the disabled baseline until the bound
 activation RPC eventually clears every gate.

Approved numeric floors

- Rolling sessions: **10** distinct market-local sessions.
- Required-field coverage non-inferiority margin: **5 percentage points**.
- Any loss of an optional score-affecting dimension is a quality failure when the
 cohort loss exceeds **5 percentage points**; structurally inapplicable fields are
 excluded from the denominator.
- Material aggregate score drift: absolute candidate-vs-legacy difference greater
 than **2 points** for any symbol is a quality failure until owner-reviewed under a
 future provider-change policy.
- Missingness-caused adverse rank displacement: **3 or more places** is a quality
 failure.
- Evaluation validity remains **72 hours**; this is freshness for the selected
 evaluation, not a substitute for the ten-session history.

Data changes

```

Add immutable evaluation columns `safety\_pass boolean not null`, `quality\_pass boolean not null`, and nullable `market\_session\_date date` sourced from validated ResearchAgent sessions (old rows have no valid session proof). Existing evaluations default both verdicts false, so an old availability-only run can never authorize cutover. Harden `activate\_evidence\_policy\_bound()` to require the selected row and the rolling ten-session history. No new provenance table is introduced: bridge payloads live in the existing Router cache and immutable decision detail remains in `evidence\_policy\_evaluations` / `evidence\_evaluation\_details`.

#### ### Acceptance

- A cohort test fails if its resolver attempts a provider on a cache miss.
- A warm compatibility row reproduces its recorded dimension and aggregate score.
- Losing macro or sentiment can remain entry-safe but cannot be `quality\_pass=true`.
- Macro produces one market-key lookup regardless of cohort size.
- India bars become observable from the Upstox/Yahoo candles already fetched by ResearchAgent; no extra Upstox/Yahoo request is made.
- The recurring jobs persist evaluations but cannot activate a policy.
- Every evaluation binds the inactive enabled candidate and the currently active disabled baseline; a baseline/candidate change starts a new ten-session series.
- The activation RPC refuses old evaluations, fewer than ten passing sessions, wrong-market history, or any safety/quality failure.
- Both production policies remain `router\_enabled=false` after deployment.

#### ### Production verification (2026-07-21)

- Applied migrations `router\_evidence\_hardening` and `router\_cohort\_session\_source` in the FinanceOS Supabase project.
- Vercel production deployments for both implementation commits reached Ready.
- US evaluation `a2226bb7-e114-464d-96b8-01ff3684a85a` bound candidate v2 to baseline v1 for session 2026-07-20. India evaluation `78770c01-4583-49ac-b605-2e82f1730458` did the same for session 2026-07-21.
- Both evaluations recorded zero primary and reverse provider calls with ledger proof holding. Both failed safety and quality on current cache coverage, which is the required fail-closed startup behavior rather than permission to cut over.
- Active US and India policies remain v1 with `router\_enabled=false`. The bound activation RPC is executable by `service\_role` only.
- Gates: TypeScript clean; 1,162 tests passed / 6 skipped; production build clean.

---

#### ## 16. Parity Evidence State And The ETF Reproduction Bug (2026-08-03)

##### ### Where the gates actually stand

Daily dual-run evaluations have been running since 2026-07-20. Measured from `evidence\_policy\_evaluations`:

```
	India	US
sessions evaluated	10	10
sessions passing all machine gates	**3**	**1**
`safety_pass`	8/13 runs	2/14 runs
`quality_pass`	3/13 runs	3/14 runs
eligibility flips ($7 floor: zero)	**34**	3
schema failures	0	0
outage drills recorded ($7 requires them)	**0**	**0**
owner reviews of divergences	**0**	**0**
```

**Nothing is approvable.** \$7 requires ten *passing* sessions, zero unexplained entry flips, the full outage-drill set, and owner review of every material divergence. Three of those four are unmet in both markets, and the drill set has never been started.

Router state is correct and unchanged: `router\_enabled = true` exists on v2 for both markets, but `active\_evidence\_policy` points at v1 (`router\_enabled=false`) for both. The candidates are inert, exactly as \$14 intended.

##### ### The dominant US blocker was our bug, not the router's

`legacy\_reproduction\_failed` accounted for **45** of ~90 US failures, on eight symbols — DBA, DXJ, EUAD, FEZ, IBIT, IVV, SCHD, VTV — every one an ETF.

Production caps ETF-like scores at `ETF\_SCORE\_CAP = 65` in `lib/research-agent.ts` after the weighted score. The evaluation called `computeWeightedAnalystScore` directly and skipped it, in **both** legs:

- `lib/evidence/evaluation/cohort.ts` — comment claimed "the EXACT production scorer" while omitting the cap;
- `lib/evidence/evaluation/cohort-builder.ts` `scorePathForReport` — the legacy-reproduction replay.

Production evidence, 2026-08-03, verbatim:

VTV recorded=65 replayed=76 EUAD recorded=65 replayed=82

IVV recorded=65 replayed=72 FEZ recorded=65 replayed=75

Every `recorded` is exactly the cap; every `replayed` is the uncapped weighted score. The legacy-reproduction check exists to prove the harness froze the right mask and weights – instead it was reporting its own omission as evidence that the frozen cohort was wrong.

The cap shipped ~2026-07-22 (US ETF scores stop exceeding 65 from 07-23 in `decision\_observations`); the evaluator was never updated to match.

**\*\*Fix.\*\*** `capEtfLikeScore(score, isEtfLike)` exported from `lib/scoring/archetypes.ts` and applied at both call sites, keyed on `shape === "etf" || shape === "metal"`. The metal case matters: `symbolShapeOf` checks `isMetal` first, so a metal fund never reports `etf` even though production sets `isEtf: true` on the metals basket – keying on `etf` alone would leave GLD/SLV uncapped and reopen the same failure.

Applying it to the candidate leg as well is not optional. Fixing only the replay would have made `score\_delta` compare a capped legacy against an uncapped candidate for the same symbols, converting a reproduction failure into a silently wrong drift measurement.

### What this does and does not change

It does **not** move any market closer to activation on its own – it makes the US measurement mean what it claims. Expect the next US evaluations to drop the 45-occurrence failure class and expose whatever remains underneath. The other blockers are untouched and still real:

- `score\_drift\_exceeds\_quality\_limit` – 29 US, 29 India occurrences. The US cases pair with `availability\_gain` on TSM, MELI, NVDA, INTC, KR, W: the candidate finds *more* data than legacy. That is not automatically good – `artifact\_created\_eligibility:TSM` shows one case where the router made a name eligible that legacy did not, which \$5 treats as a flip to explain, not a win.
- `unexplained\_flip:SBIN.NS` – India, \$7 zero-tolerance.
- India's 34 eligibility flips across 10 sessions.
- Zero outage drills.

No tolerance was widened and no gate was relaxed to accommodate any of this.

---

# Source: features\score-correlation-drift\FEATURE\_ARCHITECTURE.md  
# Score-Return Correlation Drift Detector (Stage A)

> Owner-approved 2026-09-08. **\*\*DETECTION ONLY, not causal attribution.\*\***  
> Stage B (paired-replay causal attribution) is explicitly out of scope and not  
> built here.

## Problem

"We keep adding features – how do we know which feature made the correlation between price and scores better or worse, so the app can tell if a feature breaks it?"

## What this answers, and what it deliberately does not

This flags "a dimension's rank IC changed AFTER code\_version X shipped". It never claims "X caused it". Deploys ship weeks apart and the market regime moves between them, so a before/after split across code\_version is confounded by construction – every surface (alert text, UI copy, this doc) must say "changed after", never "caused by". A real causal claim needs a paired replay (same market-local opportunities scored with and without the change) – that is Stage B and is not built here; see `docs/arch/09-learning-loop.md`'s existing "collaboration" finding type for why an unpaired comparison is already refused elsewhere in this codebase (`unattributable\_no\_paired\_shadow`).

## Reused, not reimplemented

`lib/learning/code-version-ic.ts` groups `decision\_observations` (joined to `observation\_labels`) by `code\_version` and calls the EXISTING `buildDimensionFindings()` (`lib/learning/dimension-diagnostics.ts`) once per group. That function already does the one thing that makes a rank-IC number trustworthy: per-session Spearman IC (via `computeSpearmanIC`, `lib/validation/feature-check.ts`), averaged, with the overlap-aware `nEffective = sessions / horizonDays` floor and the eligible-long cohort restriction (`lib/learning/entry-cohort.ts`). No second correlation implementation was written – this file only groups rows and reads back the numbers `buildDimensionFindings` already computes, so it cannot drift from what the Dimension Rank IC panel or the learner report for the same rows.

`benchmark\_neutral\_return` falls back to `fwd\_return` per row (both columns on

`observation\_labels`, migration `060\_observation\_labels.sql`) – production has real fwd\_return-only rows (US h10: 11/2633; India h5: 64/934, h10: 53/786, h20: 91/515 – measured 2026-09-08) and dropping them would understate n for exactly the small-sample cells this feature exists to police.

## Cohorts that never blend

- **Market.** US and India are always separate cells; never aggregated (per CLAUDE.md's Scoring Data-Truth Review Protocol, point 7).
- **code\_version.** ~33-37% of `decision\_observations` carry `code\_version IS NULL` (US 2355/6298, India 599/1788 – measured 2026-09-08). Those rows are bucketed under `UNKNOWN\_CODE\_VERSION` and are NEVER used to anchor a regression boundary – an unknown version could span several real deploys, so treating it as one boundary would manufacture a false one. The UI never charts it either.

## Multiple-comparisons control

Up to ~50 code\_versions x 6 dimensions x up to 6 horizons is a large comparison surface. `applyMultipleComparisonsControl` (Benjamini-Hochberg) runs only over cells that already cleared their own n/CI floor (`measured\_descriptive`); `insufficient\_evidence` cells are excluded from the correction entirely – they never get a p-value or a BH verdict, because there is nothing to correct. This is the exact trap `WORK\_LOG.md` (2026-09-05) parked "Systematic Pattern Discovery" over: an uncontrolled ~300-comparison table would manufacture roughly 15 "significant" cells by chance alone at  $\alpha=0.05$ .

Every cell without at least `MIN\_PREDICTIVE\_DATES` qualifying sessions AND `MIN\_EFFECTIVE\_OBSERVATIONS` effective (overlap-corrected) observations – both from `dimension-diagnostics.ts`, unchanged here – renders `insufficient\_evidence` with no IC, no CI, and no p-value. Never a number with no support behind it.

## Rolling regression alert

`lib/learning/ic-regression-alert.ts::detectRegressions` walks each `market x horizon x dimension` code\_version series (chronological by first-seen date, `UNKNOWN\_CODE\_VERSION` excluded) and flags the LATEST version only if:

1. At least `MIN\_PRIOR\_VERSIONS\_FOR\_BASELINE` (5) prior `measured\_descriptive` versions exist – with fewer, "historical variance" is a guess, not an estimate, and guessing a threshold is exactly what this exists to avoid.
2. The latest version's mean IC sits more than `WARN\_SD\_MULTIPLE` (2, `warn`) or `CRITICAL\_SD\_MULTIPLE` (3, `critical`) **historical standard deviations** of the prior versions below their mean. The threshold is derived from each dimension's own observed variance, not a picked absolute IC constant – a naturally noisy dimension needs a bigger absolute move to trigger than a stable one.

A firing alert calls `lib/system-health.ts::reportIssue` (existing `agent\_alerts` table, no migration) with `issueKey = ic-regression:<market>:<dimension>:<horizon>:<code\_version>`, so a recurring condition refreshes one open row instead of duplicating, and `reconcileIssues` auto-resolves an alert whose condition is no longer active. Detail text always states IC before/after, n before/after, the code\_version boundary, and the "changed after, not caused by" framing.

## Surfaces

- `app/api/agents/ic-regression-ledger/route.ts` – `GET` (owner-gated, read-only, computes the ledger + a pure non-writing regression pass) and `POST` (cron-secret or owner; recomputes and reconciles `agent\_alerts` via `reconcileRegressionAlerts`). Same pattern as `app/api/agents/dimension-diagnostics/route.ts`.
- `components/dashboard/IcRegressionPanel.tsx` – mounted on `/dashboard/learning` (`LearningPage.tsx`, below the existing Dimension Rank IC panel). Per-dimension IC-over-code\_version chart with 95% CI band, regression alert banners, and a table (n, mean IC, CI, nEff, BH significance, verdict). No statistic renders without its n.

## Explicitly not built (Stage B)

- Paired-replay causal attribution (same opportunities scored with/without a declared code change) – the only way to say a version CAUSED a change rather than preceded it.
- A persisted ledger table. The ledger is computed on read from `decision\_observations` + `observation\_labels` (both already indexed for this access pattern by the existing dimension-diagnostics query) – no migration was needed for Stage A, and adding persistence before the detection logic is validated would be premature.
- Wiring `POST /api/agents/ic-regression-ledger` into an actual cron schedule/vercel.json/GitHub Actions entry. The endpoint is written to the same cron-secret contract as `dimension-diagnostics`, but this repo's actual

```

external trigger for that endpoint was not located in `vercel.json` or
`.github/workflows/` during this change (see
`docs/arch/05-crons-and-scheduling.md`), so wiring a new schedule without
confirming the existing mechanism would guess at infrastructure this change
did not verify. The route works correctly when called manually or by the
owner's existing external scheduler; owner action needed to add the actual
trigger.

Source: features\score-price-divergence\FEATURE_ARCHITECTURE.md
Score / Price Divergence – Feature Architecture

Status: **Implemented** (owner-approved 2026-09-08; production evidence collection active)

Problem

Kairos records score history and the price used at each research decision, but the
owner cannot see them on one chart. The legacy rescore job also judges only the
latest signal per symbol, resets its clock whenever a new signal arrives, assumes
the US calendar, and writes unstructured global prose into `learning_log`. That
cannot reliably answer whether conviction is falling while price keeps rising.

Scope

1. Upgrade the existing Research Journal **Score Tracker**. With one selected
symbol it plots the recorded decision price on a right axis and composite,
fundamental, technical, sentiment, macro, and insider scores on a 0-100 left
axis. Multiple selections retain the existing composite comparison.
2. Join price and scores through immutable `decision_observations`, scoped by
market and canonical research session. Never join by symbol alone.
3. Replace the legacy latest-signal heuristic with a market-local, append-only
measurement shadow. It identifies opposing five-session moves:
score down at least 5 points while price rises at least 2%, or the inverse.
Three-session events are diagnostic context only.
4. Mature h5/h10/h20 benchmark-neutral outcomes from existing
`observation_labels`. Expose evidence to LearnerAgent and Upgrade Path, but
grant neither automatic score nor money-path authority.

Integrity rules

- US and India are always separate.
- Use `features.technical.as_of` as the research session when present; otherwise
the observation timestamp is explicitly labelled as a fallback, not silently
presented as an exchange close.
- Within a symbol/session, use the latest observation deterministically.
- Compare only observations with the same `score_source`, `scoring_version`,
availability mask, and applied weights. A methodology change is not alpha.
- Require a positive finite `price_at_decision` and deterministic score source.
- The evidence ledger is append-only. Historical score rows are not rewritten.
- Readiness requires at least 30 primary events across 20 distinct end sessions,
with both h5 and h10 outcomes. Readiness means review, never activation.

Data model

- `score_price_divergence_runs`: daily market-local heartbeat and counts.
- `score_price_divergence_events`: immutable window endpoints, deltas, dimension
attribution, provenance fingerprints, and primary/diagnostic classification.
- `score_price_divergence_outcomes`: immutable horizon outcomes copied from the
canonical observation-label ledger.

All tables use owner-read RLS, service-role insert, and update/delete/truncate
guards. The event natural key makes scheduled retries idempotent.

Runtime and UI

- `POST /api/agents/score-price-divergence?market=us|india` runs after label
maturation each weekday and appends a heartbeat even when it finds no event.
- The old `/api/agents/rescore-check` route remains as a compatibility alias but
now requires explicit market scope and returns structured evidence.
- Upgrade Path shows freshness, primary-event count, matured evidence and gates.
- LearnerAgent may summarize the ledger. It cannot mutate a weight from this
evidence alone.

Explicit non-goals

No change to scoring formulas, eligibility, sizing, target/stop/ladder geometry,
time stops, paper/live execution, or broker behavior. No causal claim is made
from a divergence: rising price can make valuation scores fall mechanically.

Acceptance criteria

1. A single-symbol Score Tracker visibly overlays actual recorded price and all
selectable score dimensions; multi-symbol comparison still works.
2. Same ticker in the other market cannot enter either chart or evidence.

```



3. Daily rescoring does not reset the measurement window.
4. Duplicate observations in one session collapse deterministically.
5. Version/mask/weight changes break the comparison window.
6. Empty and stale runs are visible, never reported as clean evidence.
7. Full tests, typecheck, production build, and a real owner-authenticated UI/API exercise pass before completion is claimed.

---

# Source: features\scoring-data-truth\FEATURE\_ARCHITECTURE.md  
# Scoring Data-Truth Audit

\*\*Status:\*\* Corrective fixes implemented; semantic redesigns remain proposed  
\*\*Date:\*\* 2026-07-31  
\*\*Markets:\*\* US and India, always evaluated separately  
\*\*Money path:\*\* Yes. Research scores can authorize paper and future live entries.

### ## 1. Problem

The scorer's formulas were individually plausible, but provider payloads did not always satisfy the formula's assumed contract. Static review and synthetic tests therefore passed while production data produced systematically distorted scores. The critical example was treating Finnhub `finnhubIndustry` as a GICS sector and silently applying a P/E norm of 20 whenever its value did not match a GICS key.

Finnhub documents Profile 2 as returning its own `finnhubIndustry` classification; the free response does not provide the paid profile's GICS `gsector` field:  
<https://www.finnhub.io/docs/api/crypto-profile>

### ## 2. Production Findings Reproduced

Snapshot queried on 2026-07-31.

```
Finding	Production result
Finnhub fundamental rows, last 7d	425
Rows silently using unmatched/default P/E 20	291 (68%)
Finnhub P/E median / p90 / max	30.56 / 153.84 / 8,827.05
Finnhub P/E > 200	31 / 425
US technical score exactly 20, last 45d	320 / 1,635 (19.6%)
India technical score exactly 20	55 / 405 (13.6%)
US technical score exactly 100	315 / 1,635 (19.3%)
India technical score exactly 100	163 / 405 (40.2%)
Usable dimensions, last 7d US	technical 687; sentiment 664; macro 687; insider 288
Usable dimensions, last 7d India	fundamental 200; technical 200; sentiment/macro/insider 0
```

Availability is read from `decision\_observations.features.weighting.included\_dims`, not from non-null score columns. Score columns retain neutral placeholders even when a dimension is excluded.

India sentiment is wired, but GDELT supplied zero usable observations in the measured window. The defect is availability, not missing source code. Core learner, validation, champion, and performance records are market-scoped; the unresolved issue is score comparability, not direct US/India pooling on the money path.

### ## 3. Corrective Build

#### ### 3.1 Fundamental taxonomy and P/E

- Provider mappers stamp `SectorTaxonomy`.
- Known Finnhub industries use an explicit, reviewable industry-to-sector crosswalk.
- Ambiguous or unknown industries do not receive a fabricated sector norm.
- P/E values outside `(0, 200]` are unpriceable for this component. They are omitted, not winsorized and not assigned a maximum bearish penalty.
- Other valid fundamental fields continue scoring when P/E is omitted.
- Evidence records expose mapping and omission status.

`MAX\_SCORABLE\_PE=200` is a data-sanity bound, not an alpha threshold. The valuation ladder already saturates long before 200, so larger values add no ranking information and are commonly caused by near-zero trailing earnings.

#### ### 3.2 Breakdown veto

Only either of these can hard-veto:

1. A down move of at least 2.5 ATR.
2. A decline of at least 7% on at least 1.5x average volume.

A bottom-quartile close on an ordinary down day remains evidence as a warning but no longer independently forces technical score 20. Matured labels did not support that condition as a hard classifier: US h2 was positive on average with a 61.8% win rate, and India h5 was positive with an 80% win rate in the available small cohort.

#### ### 3.3 Availability and one scorer

- Fundamental, sentiment, macro, and insider inclusion require explicit

```
`dataQuality.*Available === true`; missing fields fail closed.
- A macro row with missing/out-of-range `danger_score` is unavailable.
- `available=true` insider evidence without a finite 0..100 score is unavailable.
- The Supabase `research-agent` Edge Function is retired with HTTP 410.
- `/api/agents/research/cron` plus `lib/research-agent.ts` is the sole scorer.
- The US scanner fallback is market-scoped. Deep Dive reads same-market signals and holdings and never attaches the US macro regime to India.
```

#### ## 4. Counterfactual Impact

A read-only SQL reconstruction over 21 days applied the new P/E mapping, P/E omission, and technical-veto rule to frozen observations and their recorded effective weights.

|       | Market | n   | Old pass | Estimated new pass | Up flips | Down flips | Old avg | Est. new avg |
|-------|--------|-----|----------|--------------------|----------|------------|---------|--------------|
| US    | 1,461  | 640 | 741      | 106                | 5        | 56.97      | 58.49   |              |
| India | 368    | 274 | 284      | 17                 | 7        | 73.62      | 75.49   |              |

This is diagnostic only. It infers historical provider taxonomy where old evidence did not stamp it and does not replace a point-in-time replay. Historical signals, positions, and trades are never rewritten. New behavior begins with future research.

#### ## 5. Still Proposed, Not Implemented

These change strategy semantics and require owner approval plus shadow evidence:

1. **\*\*Insider redesign.\*\*** Shadow `current` versus `exclude-insider` and a purchase-only event feature. Do not invent a new baseline from the same small sample.
2. **\*\*Per-market threshold calibration.\*\*** The mandate already stores separate thresholds. Calibrate precision, return, pass rate, and turnover per market; do not force numeric score equivalence across different available dimensions.
3. **\*\*True PEAD.\*\*** Add point-in-time actual-versus-consensus earnings surprise and drift as a shadow archetype. Options-implied move remains risk, not surprise direction.
4. **\*\*India exit policy.\*\*** Diagnose time-stop concentration against MFE/MAE and fresh score paths before changing hold periods or exit thresholds.
5. **\*\*Technical ceiling.\*\*** Scores at 100 are saturated (US 19.3%, India 40.2%). Evaluate a versioned shadow formula; do not tune the live formula from this audit.

#### ## 6. Acceptance Invariants

- No unknown provider taxonomy silently receives a valuation benchmark.
- No invalid P/E can become confident bearish evidence.
- A weak close alone cannot hard-veto.
- Missing availability metadata excludes a dimension.
- Exactly one production component can write authoritative ResearchAgent scores.
- No corrective deployment writes an order or mutates historical trades.

#### ## 7. Shared composite input contract (2026-09-18)

Research and offline validation share `computeWeightedAnalystScore`. The Astra audit corrective pass now rejects nonfinite/negative weights, weights whose total differs from one by more than 0.000001, non-boolean availability, and included scores outside finite 0..100. Invalid inputs throw an explicit error rather than producing a directional score or silently rescaling a broken configuration.

Excluded scores never participate in arithmetic, including in the thin-evidence fallback. A single available dimension keeps its original base weight and returns `abstain=true`; absent placeholders no longer contribute to that diagnostic number. Valid multi-dimension calculations retain their existing formula and rounding. The existing zero-included-weight equal-split policy is unchanged.

This closes input validation and unavailable-value contamination only. Availability mask calibration, OOF expected returns, and dimension challengers remain separate pending work; this patch supplies no evidence of increased investment returns.

---

# Source: features\scoring-methodology\FEATURE\_ARCHITECTURE.md  
# Scoring Methodology Upgrade – Feature Architecture

**\*\*Last updated:\*\*** 2026-07-10  
**\*\*Status:\*\*** REVIEWED / IMPLEMENTATION-READY, but every rollout phase still requires Vaibhav approval  
**\*\*Authors:\*\*** Claude Sonnet 4.6; reviewed and materially corrected by ChatGPT/Codex, 2026-07-10  
**\*\*Scope:\*\*** US equities, US ETFs, and India NSE equities; long-only new positions; 2-20 trading-day swing horizon.

---

#### ## 1. Objective and honest benchmark

Kairos must rank opportunities reproducibly, abstain when evidence is weak, learn only from point-in-time outcomes, and promote a scoring vers

1. point-in-time universe and data;
2. compact, economically motivated features;
3. cross-sectional expected-return ranking with uncertainty;
4. independent portfolio/risk construction;

5. purged walk-forward validation, costs, and multiple-testing controls;
6. shadow → paper → small-live promotion with rollback.

Research supports starting with momentum/trend, liquidity, volatility, industry-relative strength, and carefully lagged fundamentals. Nonlinear

### Current assessment

| Layer               | Current state                           | Target                                                                    |
|---------------------|-----------------------------------------|---------------------------------------------------------------------------|
| Candidate discovery | Mixed static/watchlist/screener sources | PIT universe snapshot + discovery-source attribution                      |
| Features            | Five coarse 0-100 dimensions            | Compact raw features with source/as-of/quality metadata                   |
| Ranking             | Renormalized linear heuristic           | Setup-specific alpha score + comparable-universe percentile               |
| Direction           | LLM can return long/neutral/short       | Deterministic entry/exit decision; LLM explanation only                   |
| Confidence          | Count/availability semantics            | Structural applicable-weight coverage + model uncertainty                 |
| Learning            | Five-weight challenger                  | Versioned feature/model/setup genome; statistics proposes, human promotes |
| Validation          | Existing walk-forward and IC pieces     | Purged nested walk-forward, costs, trial accounting, OOF calibration      |

The existing five-dimension scorer remains a transparent baseline (`deterministic\_v1`). It must not be relabeled `deterministic\_v2`.

---

## 2. Non-negotiable invariants

- An LLM may not generate `analyst\_score`, `rank\_score`, expected return, probability, direction, weights, size, or eligibility.
- DeepSeek comparison signals are `llm\_advisory`, never `pending`, and never consumable by PaperTrader or TraderAgent.
- New positions are long-only. A held-position exit is `exit\_candidate`/SELL, never a synthetic "short" forecast.
- Missing, stale, failed, degraded, and structurally inapplicable data are distinct.
- The denominator of evidence confidence is \*\*structurally applicable base weight\*\*, never post-renormalization `applied\_weights`. The numerator is evidence.
- No model is trained or evaluated on information unavailable at `decision\_ts`.
- Cross-sectional transformations use a recorded reference universe for the same market/date. The three daily candidates are not a valid reference.
- A scoring version cannot become paper-active or live-eligible merely by changing a string or Settings value.
- LLM explanations can veto/reduce a candidate only through a bounded, evidence-citing schema; they can never rescue a failed deterministic gate.
- Scoring chooses opportunities. Portfolio construction and Execution Gateway independently decide whether and how much may be traded.

---

## 3. Target pipeline

PIT universe snapshot

- provider observations (value, source, observed\_at, available\_at, retrieved\_at)
- feature snapshot + per-feature quality
- deterministic eligibility/liquidity/event gates
- asset/setup router
- setup raw-alpha models
- comparable-universe rank + uncertainty/evidence adjustment
- deterministic long\_candidate | watch | abstain | exit\_candidate
- LLM explanation / bounded veto (optional; never rescue)
- agent\_signals summary + append-only decision\_observations canonical snapshot
- matured labels at 2/5/10/20 trading days
- IC, precision@K, calibration, net return, turnover, drawdown
- purged walk-forward challenger comparison
- shadow → paper → live-review lifecycle

### Separation of concerns

- \*\*Universe layer:\*\* defines what could have been selected at the time.
- \*\*Alpha layer:\*\* estimates relative expected return and uncertainty.
- \*\*Decision layer:\*\* applies evidence, event, contradiction, and mandate gates.
- \*\*Portfolio layer:\*\* sizes from calibrated edge, volatility, correlation, current book, and caps.
- \*\*Execution layer:\*\* validates account, quote, limits, budget, idempotency, and broker state.

No layer may silently absorb another layer's responsibility.

---

## 4. Canonical contracts

type Market = "us" | "india";

```

type AssetType = "equity" | "etf" | "adr";

type DataState =
| "ok"
| "inapplicable"
| "missing"
| "stale"
| "provider_failed"
| "degraded";

interface FeatureValue<T = number> {
value: T | null;
state: DataState;
source: string;
observedAt: string | null; // market timestamp of the observation
availableAt: string | null; // first timestamp Kairos could legally know it
retrievedAt: string;
staleAfter: string | null;
}

type SetupType =
| "quality_momentum"
| "value_inflection"
| "post_earnings_drift"
| "etf_trend"
| "india_quality_momentum"
| "india_sector_rotation";

interface ScoringFeatureSnapshot {
schemaVersion: "1";
scoringVersion: string;
symbol: string;
market: Market;
assetType: AssetType;
decisionTs: string;
marketSessionDate: string;
universeSnapshotId: number;
benchmarkSymbol: string;
sectorId: string | null;
features: Record<string, FeatureValue<number | boolean | string>>;
sourcePayloadHashes: string[];

```

```
}
```

```
interface SetupScore {
 setupType: SetupType;
 rawAlphaScore: number; // stable deterministic composite; not a probability
 rankScore: number; // 0..100 percentile within recorded comparable universe
 evidenceConfidence: number; // 0..1 structural coverage, §7
 contradictionPenalty: number; // 0..1 direct penalty, §8
 regimeScaler: number; // bounded [0.75,1.00] in initial release
 finalScore: number; // rankScore × confidence × (1-penalty) × scaler
 pWin: number | null; // calibrated OOF model only
 expectedReturnBps: number | null; // calibrated OOF model only, net estimate separate
 predictionIntervalBps: [number, number] | null;
 action: "long_candidate" | "watch" | "abstain" | "exit_candidate";
 reasonCodes: string[];
 scoreSource: "deterministic_v1" | "deterministic_v2" | "llm_advisory";
 scoringVersion: string;
}
```

`analyst\_score` remains a compatibility/display field during migration. For v2 it equals rounded `finalScore`; consumers must use `score\_source`.

---

#### ## 5. Universe and comparable groups

Every run persists the eligible universe before scoring, including inclusion reason, market, asset type, sector, liquidity fields, and effects.

Comparable groups:

- US equity: market/date/sector where sample  $\geq 20$ ; otherwise market/date/asset type.
- US ETF: market/date/ETF category (broad, sector, thematic, fixed income where supported); never rank ETFs against companies.
- India equity: market/date/NSE sector where sample  $\geq 15$ ; otherwise market/date/large/mid-cap liquidity bucket.

Winsorization and z-scores are fitted within the **training/reference** group only. If the group is too small, use pre-registered fixed transforms.

Universe membership must record survivorship limitations. The current `edge\_universe\_members` list is useful for measure-only work but is not for trading.

---

#### ## 6. Feature set and setup experts

##### ### Shared price/liquidity features

- 5/20/60/120 trading-day total return (120d may be absent in early phases)
- 12-1 momentum when enough history exists
- benchmark- and sector-relative 20/60d momentum
- realized volatility and downside volatility
- distance to 52-week high
- volume surprise using prior 20 sessions (exclude current bar from denominator)
- average traded value and, where available, spread/price-impact proxy
- EMA20/50/200 state; EMA is a state feature, not four duplicate indicators

Corporate actions must be adjusted consistently. A split/dividend discontinuity cannot become momentum or a fake drawdown.

##### ### Fundamental/event features

- sector-relative value and quality composites using only fields with PIT `available\_at`
- earnings surprise/revision and post-event age
- next earnings/ex-dividend dates as event/risk metadata
- stale filing age and provider quality

If PIT availability cannot be proven, the feature is excluded from training and trading, not backfilled with today's value.

##### ### India-specific context

- NIFTY 50 and sector-relative strength
- India VIX state
- INR/USD and crude-oil change as bounded context features
- FII/DII flow only after a reliable timestamped source exists

US `macro\_regime` must not be reused as India macro evidence. India context begins as a small scaler/feature and earns influence through valid

### Setup experts

The six existing archetypes remain, but formulas are **priors to validate**, not truths:

```
Setup	Asset	Load-bearing evidence	Initial role
`quality_momentum`	US equity/ADR	positive 20/60d trend, liquidity, quality if PIT	primary
`value_inflection`	US equity/ADR	PIT relative value/quality + technical stabilization	shadow until PIT fundamentals proven
`post_earnings_drift`	US equity/ADR	timestamped event + surprise/revision + volume/trend	shadow until event feed proven
`etf_trend`	US ETF	category-relative trend, liquidity, vol	primary
`india_quality_momentum`	India equity	NIFTY/sector-relative trend + liquidity + PIT quality	primary price-only first
`india_sector_rotation`	India equity	stable timestamped sector-index data	shadow until source reliability proven
```

Router rules are deterministic and pre-registered. An event-qualified symbol can be evaluated by the event expert; otherwise the asset-specific

Initial raw-alpha models are monotonic, regularized linear composites over transformed features. Do not hardcode arbitrary `z(...)` formulas

---

## 7. Evidence confidence — exact math

For setup `s`, let `w\_f` be the pre-registered structural weight of feature group `f`. Let `A\_s` contain only groups structurally applicable

denominator =  $\sum w_f$  for  $f \in A_s$

numerator =  $\sum w_f$  for  $f \in A_s$  where state(f) = ok and freshness passes

data\_confidence = numerator / denominator

- `inapplicable` is omitted from both numerator and denominator.
- `degraded`, `missing`, `stale`, and `provider\_failed` remain in the denominator and contribute zero to the numerator.
- Never use post-renormalization `applied\_weights` in either term.
- A missing hard-required feature causes `abstain` regardless of aggregate confidence.
- A malformed/zero denominator produces `quality\_status=unknown` and abstain for actionable paths.

This must align with `v\_decision\_quality` and `features.weighting.base\_weights`; the v2 writer must preserve the structural weights, included

Initial policy:

- ` $<0.60$ `: abstain;
- ` $0.60-0.74$ `: shadow/watch only;
- ` $\geq 0.75$ `: eligible for the remaining gates.

These are policy defaults, not learned alpha thresholds. Auto-live may require a stricter threshold and never accepts a quality override.

---

## 8. Contradictions and direction

Contradiction penalties are direct additive values, capped at 1.0. Do not multiply a penalty by a second copy of its own weight.

let penalty = 0;

if (bullishEvent && relMomentum20d < -0.03) penalty += 0.30;

if (valueRank > 70 && momentum60d < -0.10) penalty += 0.25;

if (highAttention && volumeSurge < 0.80) penalty += 0.20;

if (hostileRegime && fragileLiquidity) penalty += 0.25;

return Math.min(1, penalty);

- ` $\leq 0.20$ `: clean;
- ` $0.20-0.40$ `: watch/shadow or deterministic size haircut in the portfolio layer;
- ` $> 0.40$ `: abstain.

Machine-readable hard vetoes—illiquidity, upcoming earnings policy, missing required evidence, stale quote, corporate-action anomaly—run before

Direction/action order:

1. Determine whether the symbol is held in the relevant paper/live book.
2. Evaluate held-position exit rules independently; output `exit\_candidate` only for held quantity.
3. For new positions, apply deterministic setup/evidence/contradiction/event gates.
4. Output `long\_candidate`, `watch`, or `abstain`.
5. Ask the LLM for explanation and optional bounded veto.
6. A valid veto can only downgrade `long\_candidate` → `watch`; it cannot upgrade any result.

The LLM schema may output summary, risks, catalysts, and `{vetoed, category, citedEvidenceIds}`. It may not output any numeric trading field.

---

## ## 9. Persistence and migrations

All migrations are additive and must be applied before schema-coupled code ships. Use the next real migration number discovered at build time.

### ### `agent\_signals` summary columns

Add nullable columns with constraints: `score\_source`, `scoring\_version`, `setup\_type`, `rank\_score`, `final\_score`, `evidence\_confidence`, ``.

Do **not** add a second full `features` blob to `agent\_signals`; `decision\_observations.features` is the canonical PIT snapshot. Duplication v

Add numeric range checks as `NOT VALID`, backfill/inspect legacy rows, then validate in a later step. Add a DB constraint so new `status='pending`

### ### `decision\_observations`

This table already has `features jsonb`, `availability\_mask`, and UUID `signal\_id`, and is append-only. Add only missing summary columns: `score`

Never update or delete historical observations to “upgrade” them. New logic writes new rows/versioned snapshots.

### ### Version lifecycle

Extend the existing `strategy\_versions` lifecycle rather than creating an unrelated activation switch. Each scoring challenger records `scoring`

draft → measure\_only → shadow\_paper → paper\_active → live\_review\_eligible → live\_approved → retired

Only owner promotion changes lifecycle. Learner/LLM may propose a challenger; it cannot activate one. PaperTrader consumes `paper\_active`; Tra

---

## ## 10. Validation, calibration, and learning

### ### Labels

- Forward returns at 2/5/10/20 **trading-day** horizons.
- Benchmark-neutral and sector-neutral returns.
- Corporate-action-adjusted prices.
- Labels mature only after the horizon; late/provider-revised labels are versioned, never overwritten silently.

### ### Required evaluation

- Spearman rank IC and Newey-West uncertainty by market/setup/horizon.
- Top-K precision/hit rate and top-minus-median spread for this long-only system.
- Net-of-spread/slippage/fees/tax-assumption expectancy and turnover.
- Brier score, log loss, and expected calibration error for `pwin`.
- Sharpe/Sortino, max drawdown, benchmark alpha, concentration, and regime/year stability.
- Champion-vs-challenger paired results on the same opportunity set.
- Trial count/effective number of variants, Deflated Sharpe or a documented multiple-testing correction.

### ### Walk-forward rules

- Purge samples whose forward-label windows overlap the test window; apply an embargo.
- Fit transforms, imputers, coefficients, and calibrators on each training fold only.
- Produce out-of-fold predictions for every validation metric.
- The current `lib/validation/calibration.ts` fits coefficients on the full dataset and then scores fold test rows with those same coefficients.
- Do not use `60 observations` as a universal green light. Require enough effective, non-overlapping observations for model complexity and bot

### ### Learning boundary

The LearnerAgent may:

- discover hypotheses/features;
- request deterministic experiments;
- propose bounded challenger configs;
- summarize failures.

It may not directly change active weights, thresholds, setup routing, code, money limits, or lifecycle state. Numeric optimizers/regularized t

---

## ## 11. Data-source policy

Provider tiers and quotas change; `docs/DATA\_PROVIDER\_MATRIX.md` (or the existing canonical provider review) must record current plan, quota,

- Massive/Polygon, Alpha Vantage, FMP, Finnhub, Upstox, Robinhood, and Kite are official/API sources where configured.
- Yahoo endpoints are an unofficial, no-SLA fallback and cannot be the sole live-auto dependency.
- TradingView paid UI access does not grant an API license; CSV/manual validation is allowed, scraping/auth automation is not an architectural
- Fallbacks may supply research/shadow data only when source equivalence is validated. Provider substitution is recorded in the snapshot and o
- API exhaustion results in cached-with-freshness, alternate approved provider, or abstain. Never fabricate or ask an LLM for market data.

---

## ## 12. Build phases and exact gates

### P0 – provenance safety (no formula change)

1. Migration: source/version/summary columns and safe constraints.
2. Tag current ResearchAgent rows `deterministic\_v1`, version `v1.0`; DeepSeek rows `llm\_advisory`.
3. PaperTrader explicitly requires the current `paper\_active` deterministic version.
4. TraderAgent requires `live\_approved` scoring lifecycle for live BUY proposals.
5. Replace LLM direction for new entries with a deterministic v1 threshold/evidence gate; held exits remain separate.

### P1 – measure-only feature/universe snapshots

1. Add PIT universe snapshot and feature computation.
2. Persist canonical snapshots in `decision\_observations.features`.
3. Verify corporate-action adjustment, source timestamps, reference-universe ranks, and US/India market calendars.
4. No change to paper/live selection.

### P2 – shadow setup experts

1. Implement pure scorers and deterministic router.
2. Persist all expert scores as shadow evidence; only v1 remains actionable.
3. Run IC/stability/cost analysis over broad PIT opportunities.

### P3 – OOF calibration and expected return

1. Correct the calibration leakage.
2. Fit regularized baseline models per setup/market only when sample gates pass.
3. Store OOF predictions, model artifact/hash, transformation parameters, and trial-family count.

### P4 – paper champion/challenger

1. Owner promotes v2 to `paper\_active` after validation.
2. Shadow v1 and v2 on the same opportunity set.
3. UI explains features, data quality, comparable group, abstentions, and outcome history.

### P5 – live review

Require statistically and economically positive OOF/paper evidence, stable performance across time slices, acceptable drawdown/turnover, no un

---

## 13. Acceptance tests

- Same snapshot + same version produces byte-stable score/reason codes.
- DeepSeek/LLM signals cannot be claimed by PaperTrader or TraderAgent even if status is accidentally set pending.
- ETF cannot enter equity setup or fundamental denominator.
- India cannot consume US macro as valid India evidence.
- Degraded/stale/failed required data remains in confidence denominator and contributes zero numerator.
- `applied\_weights` are never used for confidence math.
- Contradiction penalty can reach abstain threshold; tests cover each combination.
- New unheld short/SELL is impossible; held exit quantity cannot exceed verified holdings.
- Rank computation fails to watch/abstain when the comparable universe is absent or too small.
- Current-bar volume is excluded from its own baseline; all N-day features have no off-by-one/look-ahead.
- Split/dividend-adjusted return tests for US and India.
- Walk-forward preprocessing and calibration fit only on fold train data.
- Pending actionable signal requires deterministic source and active version.
- Auto/live eligibility requires `live\_approved`; score threshold alone is insufficient.
- Missing provider data produces abstain or approved fallback with provenance, never neutral 50 as real evidence.

---

## 14. Evidence basis

- [Gu, Kelly & Xiu, \*Empirical Asset Pricing via Machine Learning\* (RFS 2020)](<https://academic.oup.com/rfs/article/33/5/2223/5758276>): nonlin
- [Harvey, Liu & Zhu, \*...and the Cross-Section of Expected Returns\*]([https://www.nber.org/system/files/working\\_papers/w20592/w20592.pdf](https://www.nber.org/system/files/working_papers/w20592/w20592.pdf)): facto
- [Bailey & López de Prado, \*The Deflated Sharpe Ratio\*](<https://www.davidhbailey.com/dhbpapers/deflated-sharpe.pdf>): correct selection bias,
- [Bailey et al., \*The Probability of Backtest Overfitting\*]([https://papers.ssrn.com/sol3/papers.cfm?abstract\\_id=2326253](https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2326253)): ordinary holdouts a
- [Jegadeesh/Titman momentum evidence](<https://www.nber.org/papers/w7159>): momentum is a reasonable prior, not permission to skip contemporary

## 15. What Claude must not improvise

- Do not implement all phases in one PR.
- Do not invent migration numbers; inspect the repo first.
- Do not create a second feature truth store.
- Do not hardcode thresholds as "validated"; mark priors and lifecycle state.
- Do not make Yahoo/TradingView scraping a live dependency.
- Do not train a complex model on the current handful of paper trades.
- Do not make any scoring change affect live trading until its migration, tests, shadow evidence, and owner promotion exist.

## 16. Provider Data-Truth Amendment (2026-07-31)

Formula review is not sufficient acceptance for a scoring feature. Every dimension must publish provider taxonomy/units, availability semantics, silent-default hit rate, production distribution including floor/ceiling mass, and a market-local frozen counterfactual. The corrective baseline and remaining semantic proposals are recorded in `features/scoring-data-truth/FEATURE\_ARCHITECTURE.md`.

---



```
Source: features\sector-regime-dimension\FEATURE_ARCHITECTURE.md
Sector-by-regime dimension

> Status: **DRAFT – awaiting approval. BLOCKED on a data prerequisite ($3).**
> Created: 2026-09-02
> Scope: a new cross-sectional scoring input. Money path if promoted; measure-only first.
```

## 1. Why this is the only macro idea worth building

`macro` is one market-wide scalar per day. Its rank IC is exactly `0.0000` \*\*by construction\*\* – a constant has no cross-sectional variance, so it cannot rank one candidate above another. Everything EPB Research and Bravos Research publish, and everything Smart X Terminal charts, is \*also\* a market-wide series: adding the Cyclical Economy index, HY spreads or an oil model makes that scalar better informed and it will still rank nothing. Those belong in the Stage-2 regime policy (`features/macro-dimension-role`), not in the composite.

Sector is different. \*\*Sector varies per symbol.\*\* A dimension of the form "how does this symbol's sector behave in the current regime" is the first macro input that could actually order a cross-section – which is the only way macro earns a place in the weighted score rather than acting as a hidden market timer.

## 2. The measure

For each symbol on each session:

sector\_regime\_score(symbol, date)

= f( relative strength of symbol's sector vs the market,

conditioned on the macro regime phase on that date )

Sector relative strength comes from the sector ETFs already prewarmed daily by `price-cache-fill` (XLK, XLF, XLE, XLV, XLI, XLY, XLC, XLP, XLU, XLRE, XLB). Regime phase comes from the existing `macro\_regime` row. Neither needs a new provider.

## 3. BLOCKER – the symbol→sector mapping is not fit for this

Measured 2026-09-02, and this is why the feature cannot start today:

```
fact	value
US symbols with a decision observation	**162**
of those, symbols carrying a sector in `symbol_profiles`	**68 (42%)**
distinct sector labels across 82 profiled symbols	**23**
median symbols per label	**~2**
`sector_breadth_history` rows / sectors / days	**14 / 2 / 11**
```

Two independent defects:

**Coverage.** 58% of the US cross-section has no sector at all. Under the availability-mask contract such a symbol is excluded, so the dimension would score a minority of the book and the score would renormalize around its absence for everyone else – the exact shape that makes a dimension look available when it is not.

**Taxonomy.** The 23 labels sit at mixed levels of granularity: `Semiconductors` (15) and `Technology` (12) coexist, so an identical chip maker can land in either; `Banking` (2) and `Financial Services` (7) split one GICS sector; `Communications` (2), `Telecommunication` (3) and `Media` (2) are three names for one. `normalizeSector` in `lib/scoring/rank.ts` only lowercases and trims – it performs no mapping, so these remain distinct groups.

`sector\_breadth\_history` exists but holds 14 rows across 2 sectors and 11 days. It is not a usable input.

### 3.1 This already degrades a feature we shipped

`rank.ts` partitions the cross-section into `market × asset-type × sector` groups and requires `RANK\_MIN\_GROUP\_EQUITY\_US = 20` members, falling back to a market-wide `all` group when a sector group is thinner. With a median of ~2 symbols per label, \*\*essentially every sector group falls below 20 and collapses into the fallback.\*\* So the "cross-sectional rank" gate – the Stage-3 change recommended as the highest-leverage available – would not actually be ranking within sectors today. It would be ranking against the whole market while reporting a sector-partitioned design.

That is worth verifying independently before Stage 3 is enabled, and it is a second reason to fix the taxonomy first.

### 3.2 RE-MEASURED 2026-09-09 – the taxonomy fix shipped; the blocker moved

The taxonomy defect described above **is fixed.** `normalizeSector` in `lib/scoring/rank.ts` now delegates to `canonicalSectorKey`

(`lib/scoring/sector-taxonomy.ts`), which maps provider labels to canonical GICS sectors and returns `null` for unmapped labels so they take the honest fallback rather than a wrong peer group. The §3 text above describing `normalizeSector` as "lowercase and trim only" is historical – kept for the record, no longer true.

Coverage also improved materially:

```
fact	2026-09-02	2026-09-09
US symbols with a decision observation	162	167
of those, carrying a sector in `symbol_profiles`	68 (42%)	**117 (70%)**
distinct RAW sector labels	23	30
```

**\*\*But the blocker is still real, and its cause has moved from code to universe breadth.\*\*** Applying the now-live crosswalk to current production labels, all-time distinct US symbols per canonical sector:

```
canonical sector	members	vs. `RANK_MIN_GROUP_EQUITY_US = 20`
Technology (17 `Technology` + 15 `Semiconductors`)	**32**	clears
Financials (12 `Financial Services` + 2 `Banking`)	14	below
Consumer Discretionary (9 `Retail` + 4 `Hotels, Restaurants & Leisure`)	13	below
Energy	11	below
Communication Services (`Media`)	6	below
Health Care (3 `Biotechnology` + 2 `Life Sciences Tools & Services`)	5	below
Materials (`Metals & Mining`)	5	below
Real Estate	3	below
Utilities	3	below
Industrials (`Electrical Equipment`)	3	below
Consumer Staples (`Beverages`)	2	below
```

**\*\*One sector of eleven clears the floor.\*\*** The other ten still collapse into `market:assetType:all`, exactly as §3.1 describes – but now because the universe is too small and too tech-concentrated, not because the mapping is broken.

This count is also the OPTIMISTIC case: it is all-time distinct symbols with any observation. `rank.ts` partitions on **\*\*per-session eligible\*\*** counts, which are strictly smaller, so on a given session even Technology may not clear.

Consequence for this feature: sector-relative ranking cannot be meaningfully evaluated today for ten of eleven sectors, regardless of the taxonomy fix. The remaining options are a materially wider universe, a lower floor for thin sectors (which trades away the statistical protection the floor exists to provide, and would need its own justification), or accepting that the dimension only ever applies to Technology. None of these is a code defect to fix.

### 3.3 The shadow has never run and cannot be surfaced today

Verified 2026-09-09:

- **\*\*No schedule.\*\*** No `cron.job` entry, nothing in `vercel.json`, nothing in `lib/schedule.ts`.
- **\*\*No persistence.\*\*** `app/api/agents/sector-regime-shadow/route.ts` only reads (`observation\_labels`, `symbol\_profiles`, `price\_cache`). It writes nothing; its IC report is returned in the HTTP response and discarded.
- **\*\*No results table\*\*** exists for it.

So "run the shadow and surface it on Upgrade Path" requires adding persistence and a schedule first – it is not merely a matter of invoking the route. Given §3.2, doing so before the breadth problem is resolved would persist an IC computed over one usable sector.

## 4. Staged plan

**\*\*Stage 0 – mapping (prerequisite, no scoring change).\*\*** One canonical sector per symbol at a single level of granularity (GICS sector, 11 buckets), covering the traded universe, with the source and as-of date recorded. `symbol\_profiles` already has `sector` and `industry` columns to hold it. `normalizeSector` gains a real mapping so `Semiconductors` → `Technology`, `Banking` → `Financial Services`, `Telecommunication/Media` → `Communication Services`. Success is measured, not assumed: coverage of decision symbols and count of distinct labels, both reported.

**\*\*Stage 1 – measure only.\*\*** Compute the sector-regime score for history and run it through the diagnostics shipped today (rank IC, quantile gradient, spread with `Effective`, rank autocorrelation). No weight, no eligibility effect.

**\*\*Stage 2 – decide.\*\*** Only with evidence clearing the pre-declared floors, and only as a governed promotion.

## 5. The statistical trap this must not fall into

A sector signal assigns the **same value to every symbol in a sector**. With 11 GICS sectors, a 78-name cross-section carries at most **11 distinct values**, not 78. Its effective breadth is the number of sectors, not the number of names.

Rank IC computed over 78 names would therefore overstate the independent information: the 78 observations are ~11 clusters, and the correction is the same in spirit as the 'nEffective' overlap correction already applied across time. **Breadth must be corrected across the cross-section as well as across sessions**, or a sector bet will look far more significant than it is – the identical error to the naive t of 3.09 versus the corrected 1.38 measured on technical Q1 today.

This is the single easiest way for this feature to produce a false positive, and it must be built in from the first measurement rather than discovered later.

#### ## 6. Honest prior

This may simply reproduce what 'technical' already captures. Sector relative strength IS a momentum measure, and technical's rank IC is currently negative at every horizon with its bottom quintile the best performer. A sector-momentum dimension could be a slower-moving copy of a signal that is currently ranking backwards. Stage 1 must therefore report the **correlation between the sector-regime score and the existing technical score**, and a high correlation is a reason to stop, not a detail.

#### ## 7. Not in scope

Adding EPB/Bravos market-wide series to the composite (they cannot rank – see §1); any change to weights, thresholds or eligibility; any new data provider. Sector ETFs and 'macro\_regime' already exist.

---

# Source: features\sector-relative-strength\FEATURE\_ARCHITECTURE.md  
# Sector Relative Strength – SUPERSEDED

> Status: **WITHDRAWN 2026-09-09**. Do not implement this document.  
> Superseded by 'features/sector-regime-dimension/FEATURE\_ARCHITECTURE.md',  
> which already covers this idea with a more rigorous evidence path.

---

#### ## Why this document was withdrawn

It proposed building sector relative strength into 'EdgeScout' as a 4th admission z-score. That was wrong on three counts, all found in review:

1. **It claimed sector-ETF relative strength did not exist.** It does: 'lib/learning/sector-regime.ts' ('sectorRelativeStrength') computes each sector ETF's trailing return minus SPY, strictly point-in-time, and 'app/api/agents/sector-regime-shadow/route.ts' maps all 11 GICS sectors to their SPDRs and joins them to eligible long decisions. Building a second implementation inside EdgeScout would have been duplicate machinery that could silently drift from the first.
2. **It claimed per-symbol sector classification was ready.** It is not. See 'sector-regime-dimension' §3 – the blocker is coverage and taxonomy, not the strength calculation. The withdrawn document generalized from a 5-symbol convenience sample.
3. **It treated an EdgeScout admission change as low-risk.** Admission is not measure-only: changing it changes which symbols 'ResearchAgent' ever sees, which changes what the learner subsequently learns from. That belongs behind an admission shadow, not a "small additive term."

#### ## Measured update to the blocker (2026-09-09)

'sector-regime-dimension' §3 measured the blocker on 2026-09-02. Re-measured today against production:

```
fact	2026-09-02	2026-09-09
US symbols with a decision observation	162	167
of those, carrying a sector in 'symbol_profiles'	68 (42%)	117 (70%)**
distinct sector labels	23	30**
```

Coverage improved substantially. **Taxonomy fragmentation got worse** – 30 labels now describe what should be 11 sectors. The §3 blocker stands, and its taxonomy half has regressed since it was written.

Note that 'lib/scoring/sector-taxonomy.ts' already contains a working crosswalk that normalizes provider strings into 11 canonical sectors – it is already used by 'resolveSectorPeBenchmark' for P/E benchmarking. The blocker is that 'normalizeSector' in 'lib/scoring/rank.ts' does not use it; it only lowercases and trims. The mapping largely exists; it is not wired into the

```

path that needs it.

Additional finding – the shadow has never run

Beyond the review's corrections, verified 2026-09-09:

- **No schedule.** No `cron.job` entry, nothing in `vercel.json`, nothing in
 `lib/schedule.ts`.
- **No persistence.** `sector-regime-shadow/route.ts` only reads
 (`observation_labels`, `symbol_profiles`, `price_cache`). It writes
 nothing. Its IC report is returned in the HTTP response and discarded.
- **Therefore it cannot be surfaced on Upgrade Path today** – there is no
 standing evidence to surface. "Run and surface the existing shadow"
 requires adding persistence and a schedule first.

Agreed sequencing (owner-directed, 2026-09-09)

1. Fix the symbol→sector taxonomy blocker – wire `rank.ts`'s
 `normalizeSector` to the existing `sector-taxonomy.ts` crosswalk so 30
 labels collapse to 11 canonical sectors. This also un-breaks the
 cross-sectional rank gate described in §3.1, which currently falls back
 to market-wide ranking while reporting a sector-partitioned design.
2. Add persistence + a schedule to the sector-regime shadow so it produces
 standing evidence.
3. Surface it on Upgrade Path.
4. Require the **sector-level** IC – not the inflated name-level IC – to be
 positive, robust, and meaningfully distinct from the existing technical
 score's momentum.
5. Only if that clears: an admission shadow comparing EdgeScout's top-four
 with and without sector strength, because admission is not measure-only.
6. Only after that: a small bounded admission boost with a rollback switch.

Nothing in steps 1-5 touches the money path.

Source: features\sector-scoring-integrity\FEATURE_ARCHITECTURE.md
Sector scoring integrity – cyclical, macro weighting, and dead scoring code

> Status: **REVISION 3 – DRAFT. F1 and F2 NOT approved and NOT enabled.
> F3 IMPLEMENTED AND SHIPPED (commit `07d142ab`, on `origin/main`).**
> Date: 2026-08-29. Influence if approved: money path (`analyst_score` → direction gate → paper buys).
>
> Review record: `docs/audits/2026-08-29-sector-scoring-codex-brief.md` (request) and the
> independent verdict of 2026-08-29, which found two of revision 1's stated mechanisms did
> not match the live implementation. Both are corrected below and the corrections change
> the recommendations, not just the wording.

The live scoring path (verified)

research-agent.ts:1711 computeScores() <- deterministic, sole producer

|- lib/data/scores.ts:589 scoreFundamentals()

| |- :103 resolveSectorPeBenchmark() -> :70 SECTOR_PE_NORM

|- fetchMacroScore(supabase, market, now) <- no symbol parameter

|- scoreTechnicals / scoreSentiment / normalizeInsiderScore

-> research-agent.ts:1861 weightOf = applyStrategyTilt({fw,tw,sw,mw,iw}, strategy_preference)

-> research-agent.ts:1869 computeWeightedAnalystScore(scoreOf, included, weightOf)

-> analyst_score -> entry_eligible -> proposals

Corrected in revision 2: live base weights come from the risk-profile / champion
mandate (`fw/tw/sw/mw/iw`) passed through `applyStrategyTilt`, **not** from
`lib/scoring/archetypes.ts`. Revision 1 quoted archetype weights as the live baseline and
was wrong. Production records show a normal US macro weight of **15%**, not the 10% quoted
previously. `archetypes.ts` weights belong to the shadow arms, not the live scorer.

Confirmed: there is exactly one producer of `decision_observations`; 6,494 rows carry
`deterministic_v1/v1.0`. The 210 older untagged rows stay historical and are not
reinterpreted (frozen-history rule).

F1 – Semiconductors fold into `technology`, and P/E is an ADDITIVE adjustment

`lib/data/scores.ts:82` maps `semiconductors -> technology`; `SECTOR_PE_NORM.technology = 30`.

Corrected mechanism. Inside `scoreFundamentals`, P/E is not a normalised component. It
is an additive adjustment to a running score, keyed on `ratio = pe / norm`:

```

```
ratio	adjustment
< 0.7	**18**
< 1.0	+8
< 1.4	-3
< 2.0	-12
>= 2.0	**22**
```

Renormalisation happens only between the five **top-level dimensions** in `weighted-score.ts`. Nothing renormalises inside the fundamental dimension.

**This invalidates revision 1's F1-b.** That option claimed removing the P/E term would let "the availability mask renormalise the remaining fundamental inputs". No such mechanism exists. Removing the term simply deletes its adjustment – which for an expensive name (ratio >= 2.0) **raises** the fundamental score by 22 points. Revision 1 presented this as a conservative de-risking move; it is in fact a large score change in a direction nobody measured, and it would have made expensive semiconductors score *better*.

Measured production reach. **Re-verified independently 2026-08-31** once the Supabase connection returned; figures below are mine, with the review's in brackets where they differ. The filter is `features->'fundamental'->'sector' ILIKE '%semiconductor%'`` on US rows.

```
metric	verified	review
semiconductor observations	594	342
distinct symbols	16	15
distinct dates	41	21
eligible-long observations	205	209
harshest tier (ratio >= 2.0, **22**)	**150**	148
strongest reward (ratio < 0.7, **18**)	26	39
```

The base counts differ (the review ran two days earlier against a smaller table, and may have scoped to rows where P/E was applied), but the two load-bearing numbers agree closely: **~150** observations sit in the **-22 tier** and **~205** are **eligible-long**. The conclusion is unchanged and confirmed: a blanket removal would have lifted roughly 150 observations by 22 points each.

Full tier distribution (US, semiconductors):

```
tier	ratio	adjustment	rows
harshest	>= 2.0	-22	150
	1.4-2.0	-12	47
	1.0-1.4	-3	35
	0.7-1.0	+8	35
best	< 0.7	+18	26
not applied	-	0	**301**
```

**New finding not in the review:** 301 of 594 semiconductor observations (51%) got NO P/E adjustment at all – `pe_scoring_status <> 'applied'``, i.e. non-positive or beyond `'MAX_SCORABLE_PE = 200'`. Half the sector already receives no P/E signal, which weakens the case that the term is doing meaningful work here and should be measured before any change.

Worked example (AMD, most recent row): `pe_ratio` 123.28` against `pe_sector_norm` 30` via `pe_sector_mapping_status: "crosswalk"` gives `pe_vs_sector_ratio` 4.11` -> the **-22 tier**.

**Also withdrawn:** the blanket `'CYCLICAL_SECTORS'` set. Revision 1 proposed covering energy, materials, autos and shipping. None of those were measured. Energy in particular is a large fraction of the current book, so a change there is high-impact and entirely unevicenced. Semiconductors are the only sector with a measurement behind them.

### Revised F1 – semiconductor-only, MEASURE-ONLY ablation

No live formula change. Build a frozen, read-only counterfactual that:

1. Replays recorded observations using the **exact additive behaviour above** – for each row, recompute `fundamental_score`` with the P/E adjustment removed, and propagate through the recorded `weights_used`` and availability mask.
2. Reports, semiconductors only: score deltas by tier, `entry_eligible`` threshold crossings in both directions, and same-date rank changes.
3. Reports eligible-long benchmark-neutral h5/h10 outcomes for the affected rows.
4. Is scoped to `'semiconductors'` alone. Other cyclical sectors are out of scope until separately measured.

The underlying intuition – that a single year's earnings misprices a cyclical – remains sound, and standard valuation practice is to assess cyclical earnings across a full cycle ([Damodaran, *Valuation\**, ch. 22](<https://pages.stern.nyu.edu/~adamodar/pdfiles/val3ed/c22.pdf>)). But that argues for a **normalised-earnings experiment**, not for deleting P/E. Deleting the term is not the same intervention as normalising the input, and revision 1 conflated them. A normalised-earnings arm is the better long-term experiment and is recorded here as future work, not proposed now.

---

## F2 – macro is constant in VALUE but not in EFFECTIVE WEIGHT

`fetchMacroScore(supabase, market, now)` takes no symbol, and the raw US macro score was constant within every one of 48 production dates inspected.

**\*\*Corrected mechanism.\*\*** Revision 1 concluded from this that macro contributes zero cross-sectional rank information. That is wrong, and the reason is the availability mask.

`computeWeightedAnalystScore` renormalises weights **\*\*per row\*\*** across whichever dimensions are available for that row. Rows differ in which dimensions are missing, so the **\*effective\*** macro weight differs between symbols on the same date. A constant value multiplied by a varying weight is not a constant contribution, so macro **\*\*can and does\*\*** move cross-sectional rank.

**\*\*Re-verified independently 2026-08-31\*\***, and the effect is WIDER than the review reported:

| check                               | verified          | review      |
|-------------------------------------|-------------------|-------------|
| ---                                 | ---               | ---         |
| dates where macro VALUE is constant | **49 of 49**      | 48 of 48    |
| dates where macro WEIGHT varies     | **46 of 49**      | 45 of 48    |
| effective weight range              | **0.0% - 42.86%** | 15% - 37.5% |

The review's 15%-37.5% understated it. Actual production distribution of the macro weight:

| weight | rows  | share |
|--------|-------|-------|
| ---:   | ---:  | ---:  |
| 0.150  | 2,153 | 40.4% |
| 0.1667 | 1,500 | 28.1% |
| 0.250  | 1,379 | 25.9% |
| 0.000  | 99    | 1.9%  |
| 0.375  | 66    | 1.2%  |
| 0.4286 | 33    | 0.6%  |

This confirms the review's correction on the live baseline too: the modal US macro weight is **\*\*15%\*\***, not the 10% revision 1 quoted from `archetypes.ts`. Note the 99 rows at weight **\*\*zero\*\*** – macro unavailable and excluded – sitting alongside rows at 42.86% on the same dates. That spread is precisely the mechanism revision 1 missed.

### Exclusion counterfactual – APPROXIMATE ONLY, NOT EXACT

**\*\*RETRACTED 2026-08-31 (same day):** this replay was described as "exact". It is not.  
Three defects, all confirmed in code:

1. **\*\*Stored `analyst\_score` is ROUNDED.\*\*** `computeWeightedAnalystScore` returns `Math.round(score)` (`lib/scoring/weighted-score.ts:61`). The shortcut below inverts a weighted sum that no longer exists at full precision.
2. **\*\*ETF scores are CAPPED after weighting.\*\*** `research-agent.ts:1702` applies `Math.min(rawAnalystScore, ETF\_SCORE\_CAP)` with `ETF\_SCORE\_CAP = 65`. For a capped ETF the stored score is not the weighted sum at all, so subtracting a macro contribution from it is meaningless.
3. **\*\*The 118 "not modellable" rows ARE reconstructible\*\***, and excluding them hid real behaviour. Base weights are stored in `features.weighting.base\_weights` (`research-agent.ts:2312`). More importantly, removing macro can drop a row to one dimension and trip the thin-evidence abstention gate (`isThinEvidence`, `research-agent.ts:1703`) – a live behavioural consequence, not an artefact.

The shortcut used was:

**new\_score = (analyst\_score - macro\_score \* w\_macro) / (1 - w\_macro) # APPROXIMATION**

An independent current-code replay over deterministic US observations reports:

| metric                                        | code replay                    | this shortcut  |
|-----------------------------------------------|--------------------------------|----------------|
| ---                                           | ---                            | ---            |
| applicable rows                               | 5,242                          | 5,156          |
| rows where the shortcut disagrees numerically | **4,573**                      | –              |
| rows where threshold classification differs   | **98**                         | –              |
| crossings up                                  | **53**                         | 35             |
| crossings down                                | **149**                        | 152            |
| rows becoming thin-evidence                   | **86** (39 currently eligible) | not modelled   |
| dispersion                                    | 14.12 -> **17.82**             | 14.12 -> 17.12 |

The shortcut disagrees numerically on 87% of rows. Its crossing counts landing near the correct ones is luck, not accuracy. **\*\*The `35 up / 152 down` figure must not be cited.\*\***

Superseded figures, retained only as the record of what was claimed:

| metric                       | shortcut (WRONG) | earlier review |
|------------------------------|------------------|----------------|
| ---                          | ---              | ---            |
| rows replayed                | 5,156            | 5,143          |
| mean absolute score change   | 2.83             | 4.83           |
| maximum change               | 12.33            | 35             |
| upward threshold crossings   | 35               | 32             |
| downward threshold crossings | 152              | 146            |
| dispersion                   | 14.124 -> 17.120 | 14.31 -> 18.91 |

Neither column is decision-grade. Use the code replay above.

On the thin rows: 118 rows have fewer than 3 available dimensions, and removing macro leaves one, where `computeWeightedAnalystScore` falls back to the FIXED base split rather than renormalising. Revision 2 called these "not modellable" and dropped them. Wrong on both counts – `base\_weights` is stored, and the abstention they trigger is exactly the behaviour worth measuring. Retained below only to show the sensitivity:

| scope                   | n     | mean abs | max abs | up | down | dispersion       |
|-------------------------|-------|----------|---------|----|------|------------------|
| modellable (dims >= 3)  | 5,156 | 2.83     | 12.33   | 35 | 152  | 14.124 -> 17.120 |
| all rows incl. low-dim  | 5,274 | 2.89     | 27.75   | 35 | 163  | 14.296 -> 17.300 |
| low-dim only (dims < 3) | 118   | 5.38     | 27.75   | 0  | 11   | 20.237 -> 23.871 |

The review's `dispersion before` of 14.31 matches the all-rows figure (14.296) rather than the modellable one (14.124), so it appears to have included the low-dim rows – which accounts for the larger maximum. **The conservative reading is the correct one**: those 118 rows do not renormalise in production, so their contribution is an artefact of the counterfactual, not a property of the change.

**The DIRECTION survives every method; the magnitudes do not.** Under the code replay, crossings are asymmetric (**149 down against 53 up**, ~3:1), dispersion rises ~26%, and 86 rows fall into thin-evidence abstention, 39 of them currently eligible. Excluding macro makes the book materially more selective. That conclusion has now held under three successive methods and is the one durable finding here.

These are not trade flips – downstream evidence and direction gates still apply – but they establish that F2 is a live behavioural change and not a mechanical cleanup.

### Required method for the decision-grade replay (nothing above qualifies)

1. Call the **shared scoring implementation** (`computeWeightedAnalystScore`) with stored component scores, `availability\_mask`, and `features.weighting.base\_weights`. Do not invert the stored score.
2. Apply the ETF cap (`Math.min(raw, ETF\_SCORE\_CAP)`) exactly where the live path does.
3. Include thin-evidence rows, and report **score crossings** separately from direction/eligibility changes – different consequences.
4. Restrict the decision-grade cohort to `score\_source = 'deterministic\_v1'`.
5. **Freeze an observation cutoff** so counts cannot drift between runs. Figures here have already shifted once because the table grew mid-analysis.

**Also withdrawn:** the "US and India must share one objective" argument. Revision 1 treated the divergence (India excludes macro entirely) as a defect. It is not inherently one. Market-local scoring is legitimate; performance is compared through market-specific benchmark-relative returns. Forcing identical formulas could actively harm India if its informative dimensions differ. The claim is retracted.

### Revised F2 – two separate experiments, neither enabled

**F2-i – selection-score macro exclusion (measure-only).** Replay using the **recorded** availability masks and **recorded** `weights\_used` – not assumed weights. Report threshold crossings both directions, same-date rank changes, eligible-long benchmark-neutral h5/h10, turnover, concentration, and benchmark excess return.

**F2-ii – book-level macro risk simulation (measure-only).** Macro as an exposure/timing control rather than a selection input, requiring an **explicit regime-to-exposure policy** declared in advance: caps, stale-data behaviour, and rollback conditions. Moving macro into sizing is a separate risk-policy approval, not a consequence of approving F2-i.

Neither promotes without clearing predeclared replay **and** forward-shadow gates.

---

## F3 – Dead scoring code (approved in principle, pending owner go-ahead)

`buildStockPrompt` (`lib/research-agent.ts:1052`), `buildEtfPrompt`, and – per the review – `buildSynthesisPrompt` are unreachable. The live path is `computeScores` and shares none of their logic.

They contain a complete, plausible-looking scoring specification, including at `:1253` an instruction to use **"general macro knowledge"** that contradicts the doctrine preamble at `:1039` forbidding recalled numbers. This is a documentation hazard, and a demonstrated one: reading it produced a confident wrong claim about macro scoring during the drafting of revision 1.

### Approved scope, behaviour-neutral

1. Delete `buildStockPrompt`.
2. Delete `buildEtfPrompt`.
3. Evaluate and, if confirmed dead, delete `buildSynthesisPrompt`.
4. Remove the stale comment at `:406` claiming momentum flows through `buildStockPrompt`.
5. Verify no call sites via static search; then full test suite, typecheck, and a production-parity build.

No behaviour change by construction. Ship separately from F1/F2.

---

```

What this does NOT propose

- No semiconductor domain model (design wins, foundry allocation, HBM contracts,
 book-to-bill, hyperscaler capex). None of that data is fetched by any provider in this
 stack, and the base scorer has no demonstrated cross-sectional edge to improve on
 (US eligible-long h10 rank IC **0.077** / 21 dates; India **0.008** / 17 dates).
- No blanket cyclical-sector rule.
- No live formula change of any kind. F1 and F2 are now measurement instruments only.
- No change to eligibility thresholds, sizing, stops, targets, or exits.
- No claim that any of this improves returns.

Sequencing

1. ~~F3~~ - DONE, shipped as `07d142ab`.
2. **F1 ablation** - semiconductor-only, measure-only, exact additive replay.
3. **F2-i** - selection-score exclusion replay on recorded masks and weights.
4. **F2-ii** - book-level risk simulation, only with a separately approved regime-to-exposure
 policy.

Nothing in 2-4 changes live scoring. Promotion of any arm requires predeclared replay and
forward-shadow gates.

Open items

- ~~Production figures unverified pending Supabase~~ **RESOLVED 2026-08-31.** F1 and F2
 figures re-verified independently against production; both conclusions hold. Base counts
 differ from the review's in F1 (594 vs 342 observations) and the F2 weight range is wider
 than reported (0-42.86% vs 15-37.5%). Neither difference changes a recommendation.
- ~~F2 exclusion counterfactual unreproduced~~ **RESOLVED 2026-08-31.** Reproduced from the
 rows' own recorded weights and masks. Crossings and dispersion-before match the review;
 the magnitude figures are smaller because 118 low-dimension rows that production would not
 renormalise are excluded here. Direction and asymmetry (152 down / 35 up) confirmed under
 both methods.
- **"Every figure verified" was premature and is retracted.** The F2 counterfactual was
 approximate, and disagrees numerically with a real code replay on 4,573 of 5,242 rows. No
 F2 magnitude here is decision-grade until the replay follows the method above.
- F1's semiconductor counts (594 observations, 150 in the -22 tier) are simple aggregations,
 unaffected by the rounding, ETF-cap and thin-evidence defects. They stand.
- F3 status: **shipped**, commit `07d142ab`, verified on `origin/main` (dead function count
 0 there, 3 at the preceding commit `08e92acc`). An external review reported it unshipped;
 that review read a stale checkout.
- Confirm `buildSynthesisPrompt` is genuinely unreachable before deleting it.
- A normalised-earnings arm for cyclicals is recorded as future work, not proposed.

Source: features\self-healing-agent\FEATURE_ARCHITECTURE.md
Live-Order Caps + Self-Healing / Health-Monitoring Agent

Status: DRAFT - awaiting approval. No implementation code until approved.
Author: Claude (Opus 4.8), reviewed/updated by ChatGPT, 2026-07-08.
Companion to: `features/learning-integrity/FEATURE_ARCHITECTURE.md` (the health
agent surfaces `v_decision_quality` taint rates; the notional caps are an execution
guardrail the agent monitors).
Decision inputs (user, 2026-07-08):
- Per-order live $ cap must be controllable from the app, per market (US/India).
- Self-healing agent: LLM model selectable from Settings (like other agents); its
 health shown on the dashboard alongside the existing agent-LLM list.
- Hard boundary (Claude pushback, accepted): an LLM may **diagnose and suggest**, and
 may **auto-apply only deterministic whitelisted actions**. It must **never**
 autonomously change weights, config money-limits, code, or place/cancel orders.

PART A - Live per-order notional caps (per market)

A1. Current state (verified 2026-07-08)
- `strategy_config.max_order_notional = 50` (single numeric, no currency).
- Enforced only in the US Execution Gateway `app/api/broker/orders/route.ts`
 (`qty * fresh_price > cap` → 403; fail-closed if no cap AND no equity snapshot;
 default fallback = 15% of live equity when the column is null).
- **Gaps:**
 1. No Settings UI to set it - DB-column only.
 2. The Kite route `app/api/kite/order/route.ts` enforces owner-gate + rate-limit +
 long-only + `confirm:true`, but has **no notional cap**. India live orders are
 not $-capped.
 3. Single value is USD-shaped. ■50 would block every India order; $50 is fine for US.
 One column can't serve both currencies.

A2. Design
- **Schema (additive):** add `max_order_notional_usd numeric`, `max_order_notional_inr numeric`
 to `strategy_config`. Backfill `usd` from the existing `50`; set `inr` to an approved
 default (proposal: ■4000 ≈ $48, US-parity). Keep `max_order_notional` as a
 deprecated read-through (usd) for one release, then drop.
- **Enforcement:**

```



- US Gateway reads `max\_order\_notional\_usd`.
- Kite route adds the **same** fresh-quote notional check against `max\_order\_notional\_inr`, same fail-closed semantics (no cap resolvable → refuse).
- Preserve the existing 15%-of-equity fallback per market when the column is null.
- **Settings UI:** owner-gated "Live Order Limits" section with a USD field and an INR field, writing `strategy\_config` via an owner-gated route (extend the existing risk-profile/settings route; never cron-callable).
- **Value shown on the trade-approval screen** so the human sees the ceiling before clicking submit.

**Reviewer corrections to A2 (binding):**

- Keep `max\_order\_notional` as a deprecated USD read-through for at least one release; do **not** drop it in this feature. Dropping is a separate approved cleanup after all deployed code reads `max\_order\_notional\_usd`.
- `max\_order\_notional\_usd` / `max\_order\_notional\_inr` readers must ship only after the additive migration is applied. A live-money route must never treat a missing cap column as uncapped.
- The current US 15%-of-equity fallback may remain only when a fresh live equity snapshot exists. Kite/India must fail closed if `max\_order\_notional\_inr` is null because no trusted Kite equity fallback is defined here.
- Settings writes are owner-human actions only; the health agent/LLM cannot change `strategy\_config` money limits.

**A3. Acceptance criteria**

- A live US order > USD cap → 403; a live India order > INR cap → 403.
- Neither path can place an uncapped live order (fail-closed retained on both).
- Cap editable in-app by owner only; change is audit-logged.
- No cron path can place or uncap an order.
- New positions remain long-only; SELL remains allowed only for held positions. This feature must not weaken either order-side guard.

---

# PART B – Self-healing / health-monitoring agent

**B1. Principle: three tiers, one hard boundary**

| Tier | Actor                          | Allowed                                                  | Risk                                                                            |
|------|--------------------------------|----------------------------------------------------------|---------------------------------------------------------------------------------|
| 1.   | Deterministic auto-remediation | Rule-based code (no LLM)                                 | Bounded whitelist actions (below)   Low – reversible, no judgment               |
| 2.   | LLM triage                     | LLM (model from Settings), <b>read-only</b>              | Diagnose, rank, suggest a fix, mark whether a Tier-1 action can handle it   Low |
| 3.   | Apply                          | Human click, or Tier-1 auto for whitelisted actions only | Execute a suggested fix   Gated                                                 |

**Hard boundary (non-negotiable):** the LLM never autonomously mutates weights, `strategy\_config` money-limits, code, or places/cancels orders. Those remain human-gated regardless of LLM confidence. The LLM's output is advice + a whitelisted-action tag. The deterministic apply route must also reject any action id that would touch orders, code, strategy weights, model selection, broker credentials, secrets, or money limits.

**B2. Tier 1 – deterministic auto-remediation (rules)**

Whitelisted, bounded, already partly present (kill switches, needs-reconcile, AV→FRED fallback). Candidate actions, each individually feature-flagged:

- Re-run research for the N symbols that failed a cron batch.
- Post-enablement: auto-exclude a trade whose linked `v\_decision\_quality` is tainted (sets `excluded\_from\_learning`, never deletes).
- Trigger a one-shot retry using an already-coded fallback path when the primary provider is rate-limited. Do not persistently change provider config, API keys, provider priority, or budgets.
- Auto-resolve a stale `agent\_alerts` row when its condition has cleared.

Each action: idempotent, logged to `agent\_alerts`/`decision\_journal`, and reversible. None touch orders, weights, money-limits, or code.

**B3. Tier 2 – health-triage LLM agent (read-only)**

- **New agent** `health-triage`. Runs on schedule (and/or on a new CRITICAL alert).
- **Model selectable from Settings:** add a row to `agent\_config` (`agent_name='health-triage'`, default `deepseek-v4-flash` – cheap advisory). It then appears in the existing Settings → LLM Config selector automatically, same pattern as `macro-read/mentor`. `getConfiguredModel('health-triage')` with a safe fallback.
- **Inputs (read-only):** open `agent\_alerts`, recent `agent\_runs` errors, `v\_decision\_quality` daily taint/unknown rate, provider budgets (`provider_budget_7d`), broker-token expiry.
- **Output:** one triage record per open issue – `root_cause`, `blast_radius`, `suggested_fix` (plain English), `auto_remediable` (bool → maps to a Tier-1 action id or null), `severity`, `model_used`, tokens. Written to a new table `health_triage` keyed to `agent_alerts.issue_key`.
- **Never** writes to money/code/order/weight paths.
- The LLM prompt/tool contract is read-only. It may receive issue context and emit JSON advice, but it must not receive tools or route access that can write `strategy_config`, `strategy_versions`, `agent_config`, broker/order tables, code, or secrets.

**B4. Tier 3 – apply**

- Dashboard renders each suggestion in the System Health card:
  - Suggested fix: ... [Apply] – the Apply button is enabled **only** when

- `auto\_remediable` maps to a whitelisted Tier-1 action; otherwise it's advice-only text.
- Owner click triggers the deterministic action through an owner-gated, CSRF/Origin protected route. It must not accept cron secrets. Every apply is audit-logged with the triggering triage id.
- Auto-apply without a click is allowed only for explicitly feature-flagged Tier-1 actions that are non-money, non-order, non-code, non-weight, and reversible. It must never call live order routes.

## B5. Dashboard health of the agent itself (user requirement)

- `health-triage` logs to `agent\_runs` (`agent\_type='health-triage'`) → last-run, errors, tokens show in the existing agents list exactly like the other agents.
- The model-freshness checker covers `health-triage` too (deprecated-model → CRITICAL).
- The agent's model + enabled toggle appear in Settings → LLM Config with the rest.
- So it's a first-class citizen: same health surface, same model-config surface as every other agent LLM already on the dashboard.

## B6. Schema

- New table `health-triage(id, ts, issue\_key text, root\_cause text, blast\_radius text, suggested\_fix text, auto\_remediable boolean, remediation\_action text, severity text, applied boolean default false, applied\_at, applied\_by, model\_used text, tokens\_input int, tokens\_output int)`. Links to `agent\_alerts` via `issue\_key`.
- `agent\_config`: one new row `health-triage`.
- Notional-cap columns are the only money-limit schema in Part A, and they are edited only by owner Settings. Part B adds no money/order columns and must not mutate order tables except read-only diagnostics.

## B7. Acceptance criteria

- Changing the `health-triage` model in Settings changes the model used on the next run.
- The agent's health (last run, errors) shows in the dashboard agents list.
- Triage suggestions render on the health card; Apply is live only for whitelisted deterministic actions.
- No code path lets this agent place/cancel an order, change a money-limit, mutate weights, or edit code. (Test: attempt each → blocked.)
- If the LLM is unreachable, Tier-1 remediation and the health card still function (LLM is additive, not load-bearing).
- `app/api/kite/order/route.ts` refuses live India orders over `max\_order\_notional\_inr` and refuses if the INR cap is null/unavailable.
- `app/api/broker/orders/route.ts` reads `max\_order\_notional\_usd`, with the deprecated `max\_order\_notional` only as a one-release compatibility fallback.

## B8. Rollout

1. \*\*A first (small, high-value guardrail):\*\* per-market notional caps + Settings UI + Kite enforcement. Independent of the agent.
2. \*\*B-Tier 1:\*\* deterministic remediation whitelist (feature-flagged, off by default).
3. \*\*B-Tier 2:\*\* `health-triage` agent + `agent\_config` row + `health-triage` table + dashboard surfacing (read-only advice).
4. \*\*B-Tier 3:\*\* one-click apply for whitelisted actions only.

## B9. Open questions

- Triage cadence: every N hours vs only on new CRITICAL? (proposal: on new CRITICAL + a 6h heartbeat.)
- Which Tier-1 actions ship in the first whitelist.
- Default INR cap value (proposal █4000).
- Does `health-triage` also summarize into the daily briefing email?

\*\*Reviewer correction to B9:\*\* Default INR cap remains an open decision until Vaibhav explicitly approves it; proposal is INR 4000.

## B10. Diagram / system-map impact

Adds a `health-triage` node reading `agent\_alerts`/`agent\_runs`/`v\_decision\_quality` and writing `health-triage`. On build, update `public/agent-diagrams/system-map.json` + add the per-agent diagram, and append a history entry per project convention.

## Reviewer changelog (ChatGPT)

- Header: Updated author line to "reviewed/updated by ChatGPT, 2026-07-08" as requested.
- Section A2: Added binding reviewer corrections that keep `max\_order\_notional` as a deprecated USD read-through for at least one release, re
- Section A2: Corrected cap fallback semantics: US may keep the current fresh-equity 15% fallback; Kite/India must fail closed if `max\_order\_
- Section A2/A3: Added explicit owner-human gating for money-limit edits and preserved long-only / sell-only-if-held order-side rules.
- Section B1: Strengthened the hard LLM boundary so the deterministic apply route also rejects action ids touching orders, code, weights, mode
- Section B2: Replaced persistent provider "auto-flip" behavior with one-shot fallback retry only; no automated provider config/API-key/prior
- Section B3: Added a read-only prompt/tool contract for the LLM; it must not receive tools or route access that can write money/config/code/o
- Section B4: Required owner-click routes to be CSRF/Origin protected and not cron-callable; auto-apply is limited to reversible non-money/non
- Section B6: Clarified that Part B adds no money/order schema and must not mutate order tables except read-only diagnostics.
- Section B7: Added acceptance tests for Kite INR notional-cap enforcement and US Gateway use of `max\_order\_notional\_usd`.
- Section B9: Default INR cap is still an open question until Vaibhav explicitly approves it; proposal remains INR 4000.
- Sections reviewed with no additional change: B5, B8, and B10 were consistent after the safety edits above.

---

# Source: features/settings-llm-control\FEATURE\_ARCHITECTURE.md  
# Settings-driven LLM + API-key control for every flow

\*\*Status: SHIPPED (both parts). 2026-07-08.\*\*  
Part 1 – every flow honors `agent\_config`: migration 129 reconciled theme-scout/

briefing/mentor rows to their real current model, then wired those callsites to `getConfiguredModel` (learner + markets-thesis/synthesis + macro-read + backtest-optimize + health-triage + deep-dive already honored it; `research` keeps adaptive routing; deepseek A/B stays deepseek). Part 2 – `lib/llm-keys.ts` (`getProviderKey` vault-first/env-fallback + status/set/clear), owner-gated `app/api/agents/provider-keys`, router + guards + model-check made vault-aware, Providers card in the LLM Config tab (write-only, masked, clear-to-revert-to-env).

<details>

<summary>Historical implementation notes (source-preserved)</summary>

Last updated: 2026-07-08

</details>

## Goal (user ask)

"I should be able to choose the LLM and its API through Settings, so the app can use them for any flow/agent." One place to pick, per agent/flow: (a) which model, (b) which provider API key – and have EVERY flow actually obey it.

## Current state (what exists, what's broken)

- `agent\_config(agent\_name, model, enabled)` table + `getConfiguredModel(svc, name, fallback="deepseek-v4-pro")` ([lib/agent-model-config.ts]) – the intended control point.
- Settings → Agents → \*\*LLM Config\*\* tab + the \*\*Model Freshness\*\* panel ([app/api/models/check]) read `agent\_config`.
- \*\*Gap 1 – not every flow honors it.\*\* Only callsites using `getConfiguredModel` obey the setting (markets-thesis, markets-synthesis, macro-read, backtest-optimize, health-triage). Others HARDCODE and ignore the config:
  - Theme Scout → `llama-3.3-70b-versatile` (but freshness shows it as deepseek – misleading)
  - Briefing → `deepseek-v4-flash` (x2)
  - Mentor-coach → `deepseek-v4-pro`
  - DeepSeek A/B agent → `deepseek-v4-flash`
  - ResearchAgent → computed `screenModel` (claude-haiku on structured path, else deepseek)
 So the panel implies control the code doesn't actually grant for those flows.
- \*\*Gap 2 – API keys are env-only.\*\* `ANTHROPIC\_API\_KEY` / `DEEPSEEK\_API\_KEY` / `GROQ\_API\_KEY` are read from `process.env` inside the router. A user cannot set or rotate a provider key from Settings; adding a provider means a Vercel env change + redeploy.

## Proposed design

### Part 1 – every flow honors `agent\_config` (single source of truth)

- Enumerate every LLM-using flow and give each a stable `agent\_name` (research, screen, briefing-editor, briefing-outlook, theme-scout, mentor, learner, deep-dive, markets-thesis, markets-synthesis, backtest-optimize, macro-read, health-triage, deepseek-ab). Seed one `agent\_config` row per flow.
- Replace each hardcoded `model:` with `await getConfiguredModel(svc, "<agent\_name>")`, keeping the CURRENT model as that row's default (behavior-preserving: nothing changes until the user edits a row). ResearchAgent's structured-vs-thin branch stays, but both branches read their configured model instead of literals.
- The Model Freshness panel then reflects TRUE wiring – every listed agent is one the code actually obeys. (Fixes the Theme-Scout=llama-shown-as-deepseek lie.)

### Part 2 – provider API keys settable from Settings (vault-backed)

- Add a \*\*Providers\*\* section to the LLM Config tab: one row per provider (Anthropic, DeepSeek, Groq, ...) with a masked key field + "Test" button.
- Store keys in the existing \*\*Supabase vault\*\* (same pattern as the Kite/Robinhood tokens + `kairos\_cron\_secret`), NOT in a plain table. Owner-gated write.
- Router key resolution becomes \*\*vault-first, env-fallback\*\*: a small `getProviderKey(provider)` reads the vault secret, falling back to `process.env` so existing env keys keep working with zero migration. `callClaude/callDeepSeek/callGroq` call it instead of reading `process.env` directly.
- \*\*Security constraints (hard):\*\* keys are write-only from the UI (never rendered back – show only ".... last4" + set/rotate), owner-gated, stored encrypted in vault, never logged, never returned by any GET. Adding/rotating a key is an "account-settings"-class action → owner click only, never agent-triggered.

### Part 3 – model dropdown is provider-aware

- The per-agent model dropdown offers the known model ids per provider (from the same list Model Freshness already fetches), plus the tier aliases ("fast"/"reasoning"). Picking a model whose provider has no key surfaces a warning (uses env fallback or fails loudly via the existing System Health funnel).

## Touch list (for approval)

- \*\*DB:\*\* seed `agent\_config` rows for all flows (data, not schema – additive upsert). Vault secrets for provider keys (no new table).
- \*\*lib/llm-router.ts:\*\* `getProviderKey()` vault-first resolution in the three `call\*` fns; no routing-logic change.
- \*\*~8 route/agent files:\*\* swap hardcoded `model:` for `getConfiguredModel(...)`.
- \*\*Settings LLM Config UI:\*\* add Providers (key set/rotate/test) + provider-aware model dropdown; keep per-agent enable toggle.
- \*\*Owner gating + vault read/write helper\*\* (reuse existing vault util).
- \*\*Docs:\*\* SYSTEM\_OVERVIEW.md LLM section + system-map.json only if a flow's provider changes materially (default-preserving, so likely just SYSTEM\_OVERVIEW).

## Non-goals / guardrails

- No autonomous key creation or provider signup – user pastes their own key; the app only stores/uses it. (Account-creation + credential-entry stay owner actions.)
- Default-preserving: seeding rows with today's models means nothing changes until the user edits. No behavior drift on deploy.
- Does not touch money limits, order paths, or the champion-genome live controls.

## Open decisions for you

1. **\*\*Scope now:\*\*** do BOTH parts (full: every-flow-honors-config + vault key management), or Part 1 only first (make every flow obey config; keep keys in env for now)?
2. **\*\*Provider key storage:\*\*** Supabase vault (recommended, matches existing secrets) – confirm, or prefer a different store?
3. Any flow you want to DELIBERATELY pin (e.g. keep Theme Scout on free Groq/llama regardless) rather than expose as user-editable?

No implementation until you approve + answer 1-3.

---

# Source: features\shadow-population\FEATURE\_ARCHITECTURE.md  
# Shadow-Population Strategy Search – Feature Architecture

> Status: **\*\*DRAFT v2 – RESHAPE. P0 IN PROGRESS (Vaibhav approved P0 only, 2026-09-04).\*\***  
> Original author: Claude (Sonnet 5), 2026-09-04.  
> Adversarial review: Codex (GPT-5), 2026-09-04.  
> P0 implementation: Claude (Sonnet 5), 2026-09-04 – see `docs/arch/04-database-schema.md`'s  
> `strategy\_versions` entry for the applied migration. P1/P2/P3 remain unauthorized: no capacity  
> change, automatic eviction, ranking, scoring, sizing, exit, order, or live-capital change.

## 0a. P0 progress (2026-09-04, this session)

Steps 1-2 of §3's P0 list are done and verified live in production; steps 3-5 are partial:

1. ■ `challenger` added to `strategy\_versions\_state\_check` – migration  
`20260904120000\_shadow\_population\_p0\_challenger\_state.sql`, additive, applied.
2. ■ Partial unique index `strategy\_versions\_one\_champion\_per\_market` (market) where  
`is\_champion = true` – same migration, applied. Production held exactly one champion per  
market before the change (verified), so it applied with zero conflict.
3. **\*\*Partial.\*\*** Structural proof done: a rolled-back production transaction confirmed a  
`state='challenger'` row now persists and confirmed the new index correctly rejects a  
synthetic duplicate champion. **\*\*Not yet done:\*\*** verifying the first \*genuine, scheduled\*  
challenger reaches `shadow\_paper` with recorded non-executing observations – that requires  
either the Friday `kairos-validation-sweep` or a real LearnerAgent weight-mutation proposal to  
actually fire against the repaired schema, and is a future event to check for, not something  
forceable synchronously.
4. **\*\*Done, structurally.\*\*** The learner's challenger-insert path already checked `insErr` and  
returned an explicit `challenger\_created: false` error before this change – that half of  
"explicit failure states" predates P0. What P0 adds: `lib/validation/strategy-states.ts` as a  
single shared TS source of truth for the `challenger`/"shadow\_paper" literals, wired into  
all 4 existing call sites (`app/api/agents/learner/route.ts`,  
`app/api/validation/sweep/route.ts`, `lib/research-agent.ts`, `lib/shadows/status.ts`) – this  
closes the actual defect class (independent literals that can silently drift from the DB  
constraint and from each other) rather than only the one symptom already fixed in `2e5021ba`.
5. ■ Capacity left at `max\_active\_shadows = 1` for both markets – unchanged, per P0's own  
instruction not to widen it yet.

**\*\*What "P0 complete" still requires\*\*:** acceptance criterion #1 in §5 – a real scheduled run, not  
a synthetic proof. Check `validation\_experiments` and `strategy\_versions.state='shadow\_paper'`  
counts again after the next Friday sweep or the next LearnerAgent run that proposes a mutation.

## 0. Decision

The product direction is sound: keep a small set of non-executing challengers, measure them  
prospectively, and leave promotion owner-only. The original implementation sequence is not  
safe yet. The existing one-shadow path has never produced a production shadow and currently  
cannot accept the `challenger` rows that feed its weekly sweep.

Proceed in independently approved stages:

1. **\*\*P0 – repair and prove the existing one-shadow pipeline.\*\***
2. **\*\*P1 – raise capacity to three and complete existing UI surfaces.\*\***
3. **\*\*P2 – add matched-window comparative evaluation after prospective data exists.\*\***
4. **\*\*P3 – consider automatic eviction only after P2 has demonstrated stable evidence.\*\***

Do not implement IC-only ranking, automatic eviction, a five-resident population, free-form  
strategy generation, or live multi-strategy execution now.

## 1. Corrections from the review

### 1.1 Learner run 20 was not a correct no-new-data abstention

Production contains four US paper-trade closure rows between the 2026-08-28 and 2026-09-04  
learner runs; the reported closed total rose from 73 to 77. The fallback Mermaid's  
`0 trades closed this run` label was actually displaying the count of orphan outcomes  
reconciled during that invocation. It also said macro was not checked and priors were not  
loaded even though the recorded reasoning chain shows both happened.

The substantive failure was in `loadLabeledDataset`: it put roughly 6,000 observation IDs into one PostgreSQL `.in(...)` URL. The failed label request was converted to an empty cohort, so every score-correlation call fell back to a one-row trade join even though production has 2,327 non-null US h10 benchmark-neutral labels across 31 dates.

The existing-bug repair is separate from this feature: label reads are now bounded in batches, read errors propagate, and fallback Mermaid labels report corpus/reconciliation facts rather than inventing a no-new-trades explanation. This repair does not activate any shadow.

### ### 1.2 The current pipeline is not operationally proven

Verified production state on 2026-09-04:

- both automation policies are enabled with auto-shadow on and capacity one;
- there are zero `shadow_paper` versions, zero policy-version shadow decisions, and zero validation experiments;
- the database `strategy_versions.state` check does not permit `challenger`, while `LearnerAgent` inserts `challenger` and the Friday validation sweep queries `challenger`.

The missing state is a hard P0 prerequisite. Clearing a lifetime closed-trade count justified revisiting the feature; it did not prove this prospective pipeline or supply ranking evidence.

### ### 1.3 The executable genome is narrower than documented

Today the learner proposes weight mutations, validation reads `weights_snapshot`, and `ResearchAgent` replays only `weights_snapshot`. `genomeDiffCount()` has no runtime consumer. Horizon, exit, sizing, and universe fields therefore are not a live coordinate-search space. Expanding those policy fields requires separate replay contracts and must not be represented as already available.

### ### 1.4 Existing visibility must be extended, not duplicated

- `AgentsPage.tsx` already has a Strategy Versions surface and shows state/validation.
- `ValidationAutomationPanel.tsx` already owns the enable/auto-shadow settings, but the API hardcodes capacity one and the UI has no capacity selector.
- Upgrade Path already registers `challenger-validation` and reports shadow count/readiness. All shadow programs must continue to appear there, per the product's established rule.

### ### 1.5 The original comparison and statistics were invalid

Admission `challengerScore` is mean benchmark-neutral log growth; resident IC is a rank correlation. They have different units and cannot be compared for eviction. Shadows admitted at different times also cannot be ranked fairly on their separate lifetime windows.

Both evidence floors apply: at least 20 predictive dates and at least 12 horizon-adjusted effective observations (`n_eff = qualifying_sessions / horizon_days`). At h10 that implies about 120 qualifying sessions, not 12 rows or one week.

Šidák controls family-wise error; FDR is a different procedure. Testing K variants every week also creates repeated-look risk. Any significance claim must predeclare both the variant family and evaluation checkpoints, then apply an explicit correction across both.

## ## 2. Target system boundary

The target remains one champion per market and up to three non-executing shadow residents. Shadows never place orders, receive capital, or self-promote. The owner can promote one version through the existing governed path only after reviewing its evidence.

Identity is the immutable version/genome hash plus a deterministic diff label such as `technical_weight: 0.25→0.30`. LLM-created names are prohibited. Generation initially remains one bounded weight change at a time.

**\*\*Forward reference (2026-09-04).\*\*** `features/robinhood-crypto/FEATURE_ARCHITECTURE.md` raises the question of a genome that differs by instrument family (crypto/ETF/equity), not just by market. This document's position is unchanged: not now. One market's genome population isn't even proven with a single real shadow yet (§0a); splitting further by instrument family multiplies the same sample-starvation problem this whole feature exists to respect. Revisit only after (a) this feature's own P1 has demonstrated a working population and (b) crypto (or another family) has enough real paper history of its own to support a separate genome without starving it.

## ## 3. Staged implementation

### ### P0 – repair and prove one shadow (separate approval required)

1. Add `challenger` to the `strategy_versions.state` database constraint without removing any current state.
2. Add a partial unique constraint that permits at most one champion per market; the current advisory lock protects the promotion RPC but a non-unique index does not protect direct privileged writes.
3. Test the real lifecycle: create challenger → validate → activate as `shadow_paper` → record non-executing decisions. Use a rolled-back transaction for structural proof and then verify the first genuine scheduled run in production.
4. Make the sweep and learner expose explicit failure states; absence of rows must not look like a quiet successful run.
5. Keep capacity at one until a real resident has accumulated prospective decisions.

### ### P1 – bounded population and complete visibility (later approval)

1. Widen the capacity constraint to `0..3`; preserve the stored default/value of one.
2. Let the owner settings API and `ValidationAutomationPanel` accept an integer capacity 0..3.
3. Change ResearchAgent's resident query to a deterministic order and ensure its replay limit equals the schema ceiling of three. Every admitted resident must receive every eligible shadow observation; admission is refused if that invariant cannot be met.
4. Extend the existing Agents Strategy Versions surface with genome diff, admission date, observation count, evidence state, and owner actions.
5. Extend the existing Upgrade Path `challenger-validation` adapter with per-market resident freshness, latest scheduled result, failure reason, and capacity. Do not create a competing dashboard.
6. When full, refuse admission and require manual retirement. No automatic eviction in P1.

### ### P2 – comparative evidence ledger (after prospective data exists)

At each predeclared checkpoint, evaluate champion and residents on the intersection of the same observation IDs and decision dates. Never compare separate lifetime windows.

Primary objective: paired benchmark-neutral log-growth/selection impact aligned with the admission validator. Secondary diagnostics: session-level rank IC, drawdown proxy, turnover, and coverage. A challenger cannot be called superior unless both sample floors clear and the corrected paired comparison passes. Otherwise its state is `collecting` or `inconclusive`.

Add an immutable evaluation ledger with, at minimum:

- market, policy-version ID, as-of date, horizon, and common-window fingerprint;
- row count, qualifying sessions, effective observations, and coverage;
- primary paired objective, delta versus champion, session-level IC, adjusted probability, correction family/checkpoint, and classification;
- unique key over policy version, horizon, as-of date, and common-window fingerprint.

The Friday sweep may compute these records only after its route is explicitly extended; the current sweep validates queued challengers and does not rank residents.

### ### P3 – automatic eviction (deferred)

Only propose eviction after P2 has accumulated enough prospective checkpoints to estimate stability. Any later policy must compare residents on a common window, enforce minimum tenure, use the same-unit primary objective, account for repeated looks, log the decision immutably, and remain independently disableable. Until separately approved, retirement is manual.

### ## 4. Files in scope for a future Builder

P0:

- a new migration for the `challenger` state and champion uniqueness;
- `app/api/agents/learner/route.ts`, `app/api/validation/sweep/route.ts`, and lifecycle tests;
- deployment/runbook verification evidence.

P1:

- a capacity migration;
- `app/api/settings/validation-automation/route.ts`;
- `components/dashboard/ValidationAutomationPanel.tsx`;
- `lib/research-agent.ts`;
- the existing strategy-version UI in `components/dashboard/AgentsPage.tsx`;
- `lib/shadows/registry.ts` and the existing Upgrade Path status adapter;
- API and mutation-detection tests for capacity, replay coverage, and non-execution.

P2:

- a new immutable evaluation-ledger migration;
- a paired common-window evaluator;
- `app/api/validation/sweep/route.ts` scheduling integration;
- existing Agents/Upgrade Path API and UI adapters;
- tests for mismatched windows, both sample floors, repeated-look correction, and immutable outcomes.

### ## 5. Acceptance criteria

1. P0 cannot report success until one real scheduled challenger reaches shadow and records non-executing observations; local tests are not deployed verification.
2. Every active resident is replayed on the same eligible observation stream; missed coverage is a visible failure, not silently ignored.
3. The database prevents more than one champion per market.
4. Capacity is consistent across schema, RPC, settings API, UI, and ResearchAgent replay.
5. `collecting` is shown until both 20 predictive dates and 12 effective observations clear.
6. Comparative claims use a paired common window and commensurate metrics.
7. Multiple-variant and repeated-checkpoint error control is named, implemented, and recorded; FWER and FDR are never conflated.
8. All shadows and their latest results remain visible in Upgrade Path, with the Agents page serving as the detailed management view.
9. No shadow places orders, receives live capital, or promotes itself.
10. Mutation tests must fail when challenger-state support, capacity agreement, replay coverage,

single-champion uniqueness, evidence floors, or non-execution guards are removed.

## 6. Decisions still requiring Vaibhav's approval

1. Approve P0 only: repair and prove the existing one-shadow path.
2. After P0 evidence exists, decide whether to approve P1 with ceiling three.
3. Approve the P2 objective/checkpoint protocol only after the first prospective sample allows its feasibility to be measured.
4. Keep P3 automatic eviction deferred unless a later evidence review explicitly reopens it.

## 7. Explicit non-goals

- no live-capital multi-strategy tournament;
- no automatic promotion or eviction in P0/P1;
- no LLM-invented names or unbounded strategy generation;
- no claim that exit/sizing/horizon/universe genome fields are executable today;
- no retrospective rewriting of evidence and no silent fallback on data-read failure;
- no removal of shadow status from Upgrade Path.

---

# Source: features\shadow-registry\FEATURE\_ARCHITECTURE.md  
# Shadow Registry and Upgrade Path

Status: APPROVED (owner instruction, 2026-07-29)  
Owner: Vaibhav  
Implementation role: Codex

## Problem

Kairos has several measure-only, counterfactual, and paper-validation programs. Their evidence is spread across purpose-built ledgers and cron definitions, so the owner cannot answer five basic questions from the app:

1. What is being measured?
2. What does it cost in provider calls?
3. Has it helped, failed, or merely remained inconclusive?
4. What exact gate remains before it may influence paper or live behavior?
5. How long might that gate take at the observed collection rate?

The absence of one inventory also creates governance drift: an old cron can keep running after its program becomes idle, and a program may be described as "shadow" after paper execution has already been enabled.

## Decision

Build an owner-only `/dashboard/upgrade-path` page backed by a typed, version-controlled registry and live adapters over existing truth ledgers.

Do not create a parallel evidence table. The page reads:

- `provider\_call\_ledger` and `evidence\_policy\_evaluations` for Router evidence;
- `shadow\_decisions` for setup-expert comparisons;
- `edge\_signals`, `edge\_ic\_history`, and `edge\_readiness\_status` for technical calibration;
- `rotation\_config` and `rotation\_events` for capital rotation;
- `earnings\_risk\_observations` for earnings-event risk;
- `international\_allocation\_\*` for allocation evidence;
- `trade\_proposals` and `strategy\_config` for autonomous-live shadow;
- `strategy\_validation\_automation` and `strategy\_versions` for challenger routing;
- a service-only cron-status RPC for schedule truth.

## Product Shape

The left navigation gains `Upgrade Path` under Research.

The shared DashboardShell US/India switch is the sole market authority. The page must not add a second market picker. Every evidence query, count, verdict, blocker, and ETA is scoped to the selected market before aggregation. Programs that do not support the selected market remain visible as `not\_applicable`.

The page contains:

- a compact status bar: total programs, actively collecting, review-ready, blocked/idle, and provider calls recorded over seven days;
- lifecycle filters;
- one un-nested program row/panel per registry entry;
- purpose and concrete value to Kairos;
- market scope and safety boundary;
- actual schedule state;
- evidence completed, target and progress;
- seven-day collection rate;
- tracked provider calls, cache hits and network attempts when authoritative;
- an honest benefit verdict;

- exact blockers and next owner/engineering action;
- estimated days only when a measurable target and non-zero collection rate exist. Otherwise the UI says 'No defensible ETA'.
- immutable mainline provenance for every program: first implementation commit, entry date, implementation scope, and why inert/measure-only code was merged;
- a separately derived production runtime state ('production\_measurement', 'production\_paper', 'scheduled\_idle', 'deployed\_inactive', 'not\_applicable', or 'status\_unavailable') with concrete proof and 'why not next stage' text;
- the current Vercel environment and build SHA when Vercel exposes it. A local response must say local/unverified rather than claim production deployment.

#### ## Registry Contract

'lib/shadows/registry.ts' is the authoritative descriptive catalog. Every entry must include:

- stable 'id';
- display name and category;
- supported markets;
- one-sentence purpose;
- product and trader benefit;
- evidence source;
- current and maximum permitted influence;
- activation gate;
- cron job names, if any;
- provider-call accounting mode;
- owner and architecture reference.
- typed 'mainline' provenance ('commit', 'enteredAt', 'implementationScope', 'reason').

Mainline provenance is version-controlled history, not a runtime flag. Runtime deployment is derived from production config, schedules and evidence ledgers. Neither is allowed to infer that a review-ready program has been promoted.

'lib/shadows/status.ts' owns aggregation and derives a normalized 'ShadowProgramStatus'. It may reference existing ledgers but cannot write them.

The registry also covers the scheduled producer families whose ledgers were added after the original inventory: 'horizon-extension' (the 'horizon\_extension\_shadow' ledger), 'exit-stop-shadow' ('exit\_stop\_shadow\_runs'), 'archetype-ic' ('archetype\_ic\_runs'), and 'alpha-diagnostics' ('backtest\_experiments' rows with 'experiment\_type='alpha\_diagnostic'). Each adapter reports market-local row/date progress and latest write time. A scheduled program with no rows is 'scheduled\_idle', not 'collecting'.

A coverage test pins the known producers/cron names to registry entries. Future shadow work is incomplete until this registry is extended.

#### ## Lifecycle Vocabulary

- 'collecting': schedule/trigger is active and evidence is arriving.
- 'ready\_for\_review': declared evidence gate is met; still no automatic enable.
- 'blocked': evidence or quality gate failed.
- 'armed': automation exists but has no active candidate.
- 'paper\_active': paper behavior is enabled; live remains gated.
- 'idle': scheduled but no evidence has arrived in the observation window.
- 'off': schedule/config is disabled.

These labels describe operational state, not expected profitability.

#### ## Benefit Vocabulary

- 'benefited': realized app-specific evidence supports the change.
- 'promising': directional evidence is positive but the activation gate is not met.
- 'mixed': evidence differs by market or gate.
- 'not\_beneficial': a declared test failed with enough evidence.
- 'insufficient': there is not enough app-specific evidence.
- 'operational\_only': the program improves safety/observability rather than predicting returns.

The API must not infer 'benefited' merely from row count.

#### ## Call Accounting

Provider calls are reported only from authoritative instrumentation:

- Router: 'provider\_call\_ledger', separating cache hits from leased network attempts.
- Reused-input programs: explicitly '0 incremental' when code proves no extra provider fetch occurs.
- Uninstrumented programs: 'not metered'; never display zero.

Future provider-consuming shadows should write to the existing 'provider\_call\_ledger' with a stable run prefix instead of creating a new meter.



```

Read API

`GET /api/upgrade-path`

- confirmed-owner only via `requireOwner()`;
- uses the service client after the owner gate;
- returns normalized aggregates, no raw provider payloads, credentials, cron
 commands, option chains, account IDs, or source bodies;
- fail-soft per adapter: one unavailable ledger does not erase other programs;
- top-level `generatedAt` makes freshness explicit.

Approved Cleanup in the Same Change

1. Owner-gate `/api/options/signal`; it spends an external request.
2. Remove dead ResearchAgent prompt text claiming unusual calls should alter
 conviction. Options are not fetched or scored there.
3. Unschedule `kairos-shadow-us` and `kairos-shadow-india`. They produced no
 shadow proposals in seven days while live-auto is disabled. The routes remain
 available for an explicitly approved future campaign.
4. Do not change scoring, options weights, live-auto settings, or broker/order
 behavior.

Safety

- Page and API are read-only.
- No LLM.
- No scoring or trading consumer.
- US and India metrics stay separate; no currency values are summed.
- A readiness label cannot activate anything.
- Estimates are observational and suppressed when blocked or underidentified.
- Capital rotation must be derived from live config, never hardcoded. It is
 `paper_active` only while `rotation_paper_execute_enabled=true`; after the
 2026-08-11 containment both markets are production shadow measurement with
 paper/live influence disabled.

Acceptance Criteria

1. Unauthorized requests receive 401/403.
2. Every registry entry has purpose, benefit, evidence source, safety boundary,
 gate, owner, and architecture reference.
3. The API reports live production counts without storing duplicate evidence,
 including every scheduled shadow producer in the registry.
4. Provider calls distinguish tracked network attempts, cache hits, zero
 incremental calls, and unmetered calls.
5. Router remains disabled in both markets.
6. Earnings risk remains `policy_mode='shadow'` and `behavior_changed=false`.
7. Live rotation proposals remain disabled.
8. Autonomous-shadow cron jobs are absent after migration and are not expected
 by stale-check.
9. Options remain absent from `analyst_score`.
10. Desktop and 375px mobile views have no overlap or horizontal page overflow;
 wide detail tables may scroll within their own region.
11. Typecheck, focused tests, full tests, production build, schema verification,
 browser verification, and production deployment pass.
12. Every program shows when and why it entered mainline, current runtime use,
 production proof, and why it has not advanced. `ready_for_review` must never
 be rendered as synonymous with scoring/trading activation.
13. Router readiness uses ten distinct fresh passing market sessions, not one
 passing evaluation or ten arbitrary sessions.
14. Event-driven programs use an independent liveness source where zero events
 can be a legitimate result.

Production audit – 2026-08-24

The program-by-program audit and production facts are recorded in
`features/shadow-registry/PRODUCTION_AUDIT_2026-08-24.md`. It found one blocked
implementation defect: the setup-expert batch contract conflicts with migration
163's NULLS-NOT-DISTINCT idempotency index, leaving India with zero setup rows
and making multi-expert US observations incomplete. The dashboard exposes the
blocker; repairing the index remains a separately approved schema change.

Reversal

- Remove the nav/page/API/registry with no evidence loss.
- Reschedule autonomous-shadow jobs from migration `164` only after a new owner
 decision.
- Existing source ledgers remain untouched.

Source: features\shadow-registry\IMPLEMENTATION_RESULT.md
Shadow Registry and Upgrade Path - Implementation Result

Date: 2026-07-29; production-provenance upgrade reviewed 2026-08-24
Status: COMPLETE

```

## ## Shipped

- Owner-only `/dashboard/upgrade-path` page in the Research navigation.
- Typed registry covering 10 data, scoring, trading, risk, portfolio, and learning programs.
- Live adapters over existing evidence ledgers; no parallel evidence table.
- Lifecycle, benefit, market, schedule, progress, collection-rate, provider-call, blocker, safety-boundary, next-action, and defensible-ETA reporting.
- 60-second page refresh plus manual refresh and lifecycle filters.
- DashboardShell's US/India switch is authoritative. Every ledger query and derived metric is scoped to that market; unsupported programs remain visible as `Not applicable`.
- Service-only `get\_shadow\_cron\_status()` RPC.
- Removal of the zero-output `kairos-shadow-us` and `kairos-shadow-india` recurring jobs. Their routes remain available for a future approved campaign.
- Owner authentication and symbol validation on `/api/options/signal`.
- Removal of options-flow content from ResearchAgent prompts. Options remain outside `analyst\_score`; earnings options remain event-risk shadow evidence.
- Per-program mainline date, commit, implementation scope and release rationale.
- Separate production runtime status, proof, and "why not next stage" text; review readiness is no longer visually confusable with activation.
- Current Vercel environment/build SHA in the read response when available.

## ## Production Verification

- Migration `20260729210000\_shadow\_registry\_cron\_status.sql` applied.
- RPC is security-definer with fixed `search\_path`; only `service\_role` can execute it.
- Router shadow/cohort, EdgeScout/IC/readiness, earnings PIT, international allocation, validation sweep, and downside-hedge schedules remain present.
- Autonomous live remains a separate fail-closed path and was not enabled.

## ## Gates

- TypeScript: passed.
- Vitest: 1,421 passed, 7 skipped.
- Next.js production build: passed; page and API included.
- Registry governance tests: 5 passed.

## ## 2026-08-24 audit correction

The full audit is in `PRODUCTION\_AUDIT\_2026-08-24.md`. It corrected Router readiness to ten distinct fresh passing sessions, made degradation liveness independent of whether a rare event fired, made earnings evidence floors reviewable without pretending calibration passed, and exposed a real setup-expert idempotency conflict. No scoring or money-path behavior changed.

## ## Reversal

Remove the read-only page/API/registry and reschedule the two autonomous-shadow jobs only if a new owner-approved evidence campaign declares a target, budget, and review gate. No position, cash, score, strategy, or evidence row requires rollback.

---

# Source: features/shared-viewer-access/FEATURE\_ARCHITECTURE.md  
# Feature Architecture: Shared Viewer Access (Read-Only Friends & Family)

## ## Status

Architecture status: Approved  
Architecture approved: Yes (owner, 2026-09-14)  
Approved scope: Phases 0-3 mechanism. The DECISION to invite any given person stays the owner's, taken in the app.  
Approved date: 2026-09-14  
Implementation allowed: Phases 0-3

**\*\*Phase 0 status: APPLIED AND VERIFIED IN PRODUCTION, 2026-09-14\*\*** —  
`supabase/migrations/20260914140000\_viewer\_phase0\_close\_blanket\_rls.sql`.  
34 blanket-authenticated policies replaced by owner-pinned reads; post-state 0 open, 91 owner-pinned, RLS on 213/213 tables. No viewer role, allowlist, or invitation exists. See Decision 76.

**\*\*Phases 1-3 status: IMPLEMENTED, 2026-09-15\*\*** — `app\_user\_roles` is the sole non-owner authority; middleware and each viewer-safe handler enforce the same allowlist; owner-only `/dashboard/admin/access` can invite, resend a single-use access link, revoke (with a notice), restore, and permanently delete a viewer account. Delete first revokes, then removes the `auth.users` identity (cascading its grant); a typed `DELETE <email>` confirmation is required. Mail delivery is explicitly reported rather than assumed. A delivery failure never undoes a revocation or misreports a sent notice.

The authorization controls, not the UI, are the boundary: the server resolves viewer grants on every request, viewer routes enforce `requireViewerOrOwner`.

and direct PostgREST reads remain owner-pinned by RLS. `tests/viewer-route-sweep.test.ts` asserts both route allowlisting and handler gates so a future route cannot become viewer-reachable merely because its navigation is hidden.

Related designs: `features/per-user-broker-risk/FEATURE\_ARCHITECTURE.md` (draft, unapproved) builds on this one – it gives a guest their own broker connection and private risk analytics, which is the first time a non-owner owns rows. It deliberately does NOT change anything here: viewers stay read-only over the owner's data, and the private plane lives in separate tables.

Relationship to `features/multi-tenant/FEATURE\_ARCHITECTURE.md`: that document designs full per-user tenancy (own book, own broker, own genome, per-user learning). It is **deferred, not cancelled**. This document is the smaller, nearer-term path the owner chose on 2026-09-14 and is the only one proposed for implementation. They are not competing designs: this one ships first and does not preclude the other.

---

## ## Feature Purpose

Let the owner give a small, named circle (friends & family) a **read-only window** into Kairos: how the paper portfolio is doing, and what the research/fundamentals view says. Viewers sign in with their own email. They get no book, no broker connection, no agent, and no ability to change anything.

The design goal is not "multi-user Kairos". It is "a shareable view of the owner's single-user Kairos", with the cheapest possible blast radius.

## ## User/System Questions This Feature Answers

- How does the owner let a specific person see the app without giving them an account that owns data?
- What exactly can a viewer see, and what is provably out of reach?
- How do we guarantee that adding viewers does not multiply API/LLM cost?
- What stops a viewer bypassing the UI and reading the database directly?

## ## Scope

- A two-role model: `owner` (unchanged, full access) and `viewer` (read-only, narrow page set), backed by a server-side allowlist.
- Page and API route gating for the viewer role.
- An RLS lockdown of the tables currently reachable by any authenticated user.
- A viewer-reachable-route audit whose pass condition is: no route a viewer can reach may cause a provider or LLM call.
- Acceptance tests including negative tests issued directly against PostgREST, not through the UI.

## ## Non-Goals (explicitly OUT of scope)

- **Any per-user data.** Viewers own no rows. No viewer book, no viewer watchlist, no viewer settings beyond session.
- **Any broker connection for a viewer**, read-only or otherwise.
- **Any write path for a viewer**, including paper trades, approvals, queue actions, or strategy edits.
- **Any per-user agent, research run, genome, or learning.** Explicitly: no per-user cron fan-out. This is the cost guarantee and it is load-bearing.
- Live portfolio, live risk analytics, broker orders, admin, vault, system health, and cost/LLM telemetry are owner-only and stay owner-only.
- Billing, public signup, self-service invitation.

---

## ## Current Behavior (verified against production + code, 2026-09-14)

Kairos is single-user, gated on one hardcoded email.

| Layer       | Today | Evidence                                                                                                                           |
|-------------|-------|------------------------------------------------------------------------------------------------------------------------------------|
| --- --- --- |       |                                                                                                                                    |
| Identity    |       | OWNER_EMAIL = "<owner-email>"   `lib/auth/owner.ts`                                                                                |
| Page gate   |       | middleware rejects any non-owner session   `middleware.ts` (matches `/dashboard`, `/property`, `/admin` only – <b>not</b> `/api/`) |
| API gate    |       | `requireOwner()` per route   169 of 252 API routes call it; the remainder use cron secrets or inline `ADMIN_EMAIL` checks          |
| Data reads  |       | server-side `createServiceClient()`   266 files; service-role <b>bypasses</b> RLS entirely                                         |
| RLS         |       | enabled on 213/213 public tables   but only 21 tables have any owner column                                                        |

## ### The load-bearing observation

Dashboard pages are **server-rendered** and read through the service-role client (e.g. `app/dashboard/portfolio/page.tsx`). RLS is therefore **not** the delivery mechanism for anything a user sees – the server fetches, the server filters, the page renders.

Two consequences, and they set the whole design:

1. **Viewers need zero direct table access.** Everything they see arrives

server-rendered. So the RLS task is to \*close\* doors, never to open new ones for the viewer role.

2. \*\*RLS is the only thing standing between a viewer and the raw database.\*\*  
 `lib/supabase/client.ts` uses `NEXT\_PUBLIC\_SUPABASE\_ANON\_KEY`, so the anon key ships to the browser. Any holder of a valid session can query PostgREST directly. \*\*The UI is not a security boundary.\*\*

### What a viewer could reach today, if admitted with no other change

RLS policies keyed on `auth.role() = 'authenticated'` or `USING (true)` do not distinguish an owner from a viewer. Verified in production:

**\*\*Read \*and write\* by any authenticated user\*\*** (7 tables) – policy `FOR ALL`:  
 `paper\_portfolio`, `paper\_positions`, `paper\_performance`, `agent\_signals`,  
 `research\_packets`, `learning\_log`, `signal\_weights`.

**\*\*Read by any authenticated user\*\*** (27 tables), including owner-private data:  
 `live\_position\_state`, `agentic\_position\_ledger`, `live\_exit\_ladder\_shadow`,  
 `trade\_log`, `trade\_queue`, `security\_events`, `llm\_call\_log`, `agent\_runs`,  
 `agent\_alerts`, `agent\_memory`, `briefings`, `newsletters`, `readiness\_controls`,  
 `readiness\_runs`, `risk\_analytics\_cache`, `signal\_score\_history`,  
 `strategy\_classifications`, `strategy\_templates`, `paper\_nav\_history`,  
 `price\_cache`, `symbol\_profiles`, `india\_market\_snapshot`, `earnings\_calendar`,  
 `earnings\_consensus\_snapshots`, `instrument\_family\_observations`,  
 `sector\_breadth\_history`, `announcements`.

So a "read-only viewer" would in fact hold **\*\*write access to the paper book and the learner's weights\*\***, and **\*\*read access to live position state\*\*** – none of it visible in, or caused by, the UI.

This is not a live breach today: no non-owner can obtain a session. It is a control with exactly one user's worth of margin, and this feature spends it.

### What is already correct

| Group                                                                                          | Status                                                   |
|------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| 72 tables pinned to the owner's email                                                          | correct – viewers blocked, no change needed              |
| ~10 tables scoped to `auth.uid()`                                                              | correct                                                  |
| `broker_*`, `api_key_vault`, `live_account_snapshots`, `app_settings`, `decision_observations` | RLS on with <b>**no policy**</b> = deny-all to non-owner |

### Browser-side query surface (constrains the lockdown)

Only three components query Supabase from the browser, touching six tables:

| File                                      | Tables                               |
|-------------------------------------------|--------------------------------------|
| `components/dashboard/DashboardShell.tsx` | `strategy_config`, `trade_proposals` |
| `app/dashboard/intelligence/page.tsx`     | `newsletters`, `profiles`            |
| `app/dashboard/settings/page.tsx`         | `profiles`, `strategy_versions`      |

`DashboardShell` wraps every dashboard page including the viewer's. `strategy\_config` is already owner-email-pinned, so a viewer's browser query returns an empty set rather than an error – the shell should degrade, not break. **\*\*This must be verified, not assumed\*\***, and is an acceptance test below.

---

## Proposed Behavior

### 1. Roles and identity

Two roles: `owner` and `viewer`. Owner behaviour is unchanged.

- Identity resolves **\*\*server-side\*\*** from a stored allowlist keyed on the Supabase `auth.users.id`, not on an email string alone and not on any user-editable profile field. Email remains the invitation address; the user ID is the authority once the account exists.
- A viewer cannot change their own role. Role is owner-managed only.
- Revocation must terminate access at the **\*\*API and RLS layers\*\***, not merely hide navigation.
- Invitation uses Supabase's server-side invite so the recipient sets their own password. No owner-chosen password is ever transmitted, stored, or logged.

`OWNER\_EMAIL` stays the definition of owner; the allowlist is additive and only ever grants the strictly weaker `viewer` role.

### 2. What a viewer sees

Exactly two areas:

- **\*\*Paper Portfolio\*\*** – the owner's paper book: NAV, positions, trades, performance vs benchmark. Labelled unambiguously as the owner's book, so no viewer reads it as their own performance.
- **\*\*Research / fundamentals\*\*** – the research view and symbol fundamentals.

Everything else returns 403 for a viewer: live portfolio, risk, admin, vault, settings, agents, system health, learning, upgrade path, trading, property, capital.

### ### 3. RLS lockdown (the core change)

Because viewers need no direct table access, the change is a strict narrowing:

1. **Remove write access** from the 7 `FOR ALL` tables. Writes belong to service-role only. This is correct regardless of this feature – nothing in the app writes to those tables from the browser.
2. **Remove the blanket `authenticated` SELECT** from the 27 read-open tables, leaving service-role (and owner-pinned where one already exists).
3. **Add no new viewer-readable policy.** If a viewer can reach a table directly, that is a design error, not a feature.

Net effect: a viewer's session grants access to nothing in PostgREST. All viewer data arrives server-rendered through role-checked pages.

Ordering note: this step is independently valuable and carries **zero invitation risk**. It should ship and soak **before** any viewer account exists.

### ### 4. Route gating and the cost rule

- Extend `middleware.ts` to resolve role and restrict viewers to the permitted page set.
- `requireOwner()` stays on every route that is not explicitly viewer-safe. A new `requireViewerOrOwner()` guards only the handful of read endpoints backing the two viewer pages.
- **Viewer-reachable-route sweep** (pass condition for launch): **every route a viewer can reach must be read-only over already-persisted tables.** No route a viewer can reach may call a market-data provider, an LLM, or a broker. The 83 routes not currently calling `requireOwner` are the audit population.

### ### 5. Cost model

Adding viewers costs **nothing per viewer** in provider spend:

- No per-user crons. The 96 active pg\_cron jobs stay singular.
- No per-user research, scoring, or LLM calls.
- Viewers read rows that already exist; the tenth viewer costs what the first did.

The cost risk is **not** viewer count – it is a single viewer-reachable endpoint that triggers a provider call. Rule 4 above is the entire mitigation. This matters because Alpha Vantage usage already exceeds the code default (35-71 calls/day observed against a default of 25); a viewer-triggered research path would compound an existing overrun.

---

### ## Phased Rollout

| Phase | Content                                                                                                                                   | Gate to next |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| 0     | <b>DONE 2026-09-14.</b> RLS lockdown (§3) only. No role, no allowlist, no invitation.   Owner's own usage unaffected for one full session |              |
| 1     | <b>DONE 2026-09-14.</b> Role + allowlist + middleware/page gating + viewer-safe API guards   Isolation matrix passes (below)              |              |
| 2     | <b>DONE 2026-09-14.</b> Viewer-reachable-route sweep (§4)   Zero viewer-reachable provider/LLM paths                                      |              |
| 3     | <b>DONE 2026-09-14 (mechanism).</b> Owner-triggered invitation from `/dashboard/admin/access`   –                                         |              |

**Phase 3 is a capability, not an act.** The invite button exists; no invitation has been sent and none will be sent by anyone but the owner, from the app. That placement is deliberate: the open regulatory question below is a judgement about a specific person and what they will do with what they see, so the decision sits with the owner at the moment of inviting, not in a deployment.

Mechanics: the recipient receives a Supabase invitation email and sets their own password – no password is accepted, generated, stored, logged or returned. Access comes from the `app\_user\_roles` row, not from the auth user, so a half-completed invite fails closed: they can sign in and reach nothing. Re-inviting a previously revoked person reuses their account and restores the grant rather than erroring.

No phase may be skipped, and Phase 3 requires Phases 0-2 verified in production, not locally.

### ## Acceptance Tests

Negative tests must be issued **directly against PostgREST with a viewer JWT**, never through the UI – the UI is not the boundary being tested.

1. With a viewer session token, a direct PostgREST `SELECT` on each of the 34 previously-open tables returns empty or 401/403.
2. With a viewer session token, a direct PostgREST `INSERT`/`UPDATE`/`DELETE` on `paper\_portfolio`, `paper\_positions`, `paper\_performance`, `agent\_signals`, `research\_packets`, `learning\_log`, `signal\_weights` fails.
3. A viewer requesting any owner-only page or API route receives 403.

4. A viewer can load Paper Portfolio and the research view, and the rendered data matches the owner's view.
5. `DashboardShell` renders for a viewer without error despite its browser-side `strategy\_config` / `trade\_proposals` queries returning empty.
6. No route reachable by a viewer issues a provider or LLM call – asserted by `llm\_call\_log` and provider-call audit rows being unchanged across a full viewer session.
7. Owner behaviour is byte-identical before and after: paper book, live portfolio, agents, crons all unchanged.
8. A revoked viewer is denied at the API layer, not merely at navigation.
9. Full suite, typecheck, production build.

## Open Decisions (for the owner)

1. **Regulatory.** Showing research and signals to people who may trade their own money on them is materially different from showing them a performance chart. India (SEBI research-analyst rules) is the sharper exposure. This is not resolvable in code and is not a claim this document adjudicates – it is flagged as a gate to clear outside the repository. A paper-performance-only viewer (dropping the research page) materially reduces it.
2. **Does the viewer page set include research at all**, or only Paper Portfolio? Decision 1 may answer this.
3. **Allowlist storage** – a new owner-managed table vs an env var. A table allows revocation without redeploy; an env var is simpler and harder to tamper with.
4. **Attribution labelling** – exact wording that makes clear the book is the owner's, not the viewer's.

## Update triggers (per CLAUDE.md)

If approved and implemented, the SAME change must update:  
 `docs/arch/04-database-schema.md` (RLS policy changes),  
 `docs/arch/07-coding-conventions.md` (role gating + the viewer-route cost rule),  
 `docs/arch/08-risk-and-safety.md` (viewer lockout boundary),  
 and `PROJECT\_DECISIONS.md` (a decision record for the two-role model).  
 `05-crons-and-scheduling.md` is explicitly NOT updated: this feature adds no cron and no fan-out, and that absence is the cost guarantee.

---

## Phase 1-2 as built (2026-09-14)

### Viewer page set, and why the rest are excluded

| Page                                 | Verdict         | Reason                                                                                          |
|--------------------------------------|-----------------|-------------------------------------------------------------------------------------------------|
| --- --- ---                          |                 |                                                                                                 |
| `/dashboard/portfolio`               | included        | Server-rendered from the database only                                                          |
| `/dashboard/research` (Fundamentals) | included        | `/api/research/{chart-data,universe}` are pure DB reads – zero outbound calls                   |
| `/dashboard/symbol/[symbol]`         | included        | Reads `agent_signals` + `paper_trades` only; without it every symbol link on the portfolio page |
| `/dashboard` (Home)                  | excluded        | Reads `live_account_snapshots` – owner's real broker data                                       |
| `/dashboard/markets`                 | excluded        | for now**   Calls Massive. NOT per load – corrected 2026-09-14, see below                       |
| `/dashboard/calendar`                | safe to include | **already has a 24h DB TTL – corrected 2026-09-14, see below                                    |
| `/dashboard/scanner`                 | excluded        | Screener endpoints call providers                                                               |
| everything else                      | excluded        | Owner-private: live portfolio, risk, agents, learning, admin, vault, settings, trading          |

### Correction (2026-09-14): these endpoints are already cached

An earlier revision of this document claimed Markets and Calendar "call a provider on every page load". That was **wrong**, and it is recorded here rather than quietly edited away because it changed an exclusion decision.

| Endpoint                 | Actual caching                                                                                                      |
|--------------------------|---------------------------------------------------------------------------------------------------------------------|
| --- ---                  |                                                                                                                     |
| `/api/calendar/earnings` | **24h DB TTL** against `earnings_calendar.fetched_at` – already once a day                                          |
| `/api/markets/overview`  | `export const revalidate = 300`, a 5-minute in-memory cache, `next: { revalidate: 3600 }` on the grouped fetch, and |
| `/api/markets/quotes`    | `next: { revalidate: 300 }` per symbol                                                                              |

So Calendar meets the once-a-day bar today and may be included. Markets stays excluded for a different and better reason: it serves **daily data** on a five-minute refresh cycle. `/api/markets/quotes` calls Massive's `/prev` endpoint – the previous day's close – and /api/markets/overview` uses grouped daily bars whose closes its own comment calls immutable. Up to ~288 refresh windows a day for values that change once.`

The fix is to match cadence to the data, which India already does (`'kairos-india-markets-fill` warns india_market_snapshot` at 10:15 UTC and the page reads the table). US has no equivalent and computes on demand. Giving US the same treatment removes provider calls from the page load entirely, which also makes Markets viewer-safe. That is separate, approval-gated work – a new cron plus a snapshot table plus a route rewrite, not a list edit.`

### Two owner-private leaks found and closed while building

1. **`/api/research/universe` returned broker_orders` – the owner's real money orders (side, qty, fill price, status), merged into the Fundamentals page. The`**

query is now skipped by role rather than filtered after the fact, so a viewer's response is built without ever reading the table.

2. `**profiles.role` was self-writable.\*\* Its RLS is ``FOR ALL USING (auth.uid() = id)`` with no ``WITH CHECK``, and `middleware.ts` gated ``/admin`` on ``profiles.role in ('admin','superadmin')`` – a self-promotion path to ``/admin`` the moment a second account existed. Now: a DB trigger rejects any client-side change to ``role``, and ``/admin`` gates on owner identity instead.

### ### Authority and revocation

- A viewer's role comes only from ``app_user_roles`` (service-role write only), never from ``profiles``.
- Revocation sets ``revoked_at``; the row is kept so grant history survives. Middleware re-reads the grant on `**every**` request, so revocation stops the session at the edge and at every viewer-safe route – not merely in navigation.
- ``app_user_roles`` has two read policies: owner reads all, and a viewer reads only their own row (``user_id = auth.uid()``). Without the self-read, middleware – which runs on the caller's session, not service-role – resolves no grant and signs the viewer straight back out.

### ### Owner-facing controls

``/dashboard/admin/access`` (owner-only) lists every grant, what each role may view and edit, and provides revoke/restore. Backed by ``/api/admin/access``.

### ### Phase 2 enforcement

``tests/viewer-route-sweep.test.ts`` fails the build if any viewer-reachable route gains an outbound call or a provider/broker/LLM import. It also asserts the calendar and analytics helpers on the viewer read path stay `**synchronous**` – a synchronous function cannot await a network call, which proves the path cannot reach ``fetchMarketStatuses`` inside ``market-calendar.ts``.

---

## Invitation lifecycle correctness (2026-09-15, owner-approved "fix all")

### ### The defect

Three questions the owner asks of the access page had been collapsed into one green `**Active**`, because the only fact the page carried was ``active = !revoked_at``:

1. They accepted the invitation and can sign in.
2. The invitation was sent and is sitting unopened.
3. The invitation `**bounced**`. Nobody received it and nobody ever will.

(3) is why this matters. Nothing in the chain could detect it:

- ``generateLink`` creates an ``auth.users`` row for any address matching the route's syntax check. It does not verify the mailbox exists.
- Resend returns success when it `**accepts**` a message. A dead mailbox bounces asynchronously, minutes later, reported only by webhook – which the app did not consume.

So a mistyped address produced an orphan auth user plus a grant the owner's own screen reported as active access. The access itself was inert (no link, no session), but the page whose entire job is answering `**"who can see my book?"**` was giving the wrong answer.

### ### Three cases, as verified against the code and production

| Case                          | Behaviour                                                                                                                                           | Status |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| Invite the same address twice | <code>`findUserByEmail`</code> → <code>`returning=true`</code> → <code>**recovery**</code> link (Supabase refuses to re-invite an existing account) |        |
| Revoke, then invite again     | Same path; the upsert explicitly writes <code>`revoked_at: null, revoked_by: null`</code> . Their existing password still works                     |        |
| Address does not exist        | Grant written on provider acceptance, rendered "Active" forever.   <code>**Fixed here**</code>                                                      |        |

### ### Design

`**Acceptance and deliverability are separate axes and are deliberately not merged into one enum.**`

- `**Acceptance**` is ``auth.users.confirmed_at``, `*read*` by the GET handler and never mirrored into ``app_user_roles``. Supabase owns that fact; a copy would go stale the moment someone accepts.
- `**Deliverability**` is the new ``undeliverable_`` columns, written only by the Resend webhook.

Collapsing them would introduce a second lie in the opposite direction: a guest who already uses the app daily would look locked out the day their mailbox fills up. ``lib/auth/grant-status.ts`` is the single derivation, shared by the API, the page, and the tests.

`**Deliverability NEVER gates access.**` A bounce is a fact about a mailbox, not a

judgement about a person. Revocation stays the owner's deliberate act.

### The webhook is a public endpoint, so it fails closed

`POST /api/webhooks/resend` must be public – Resend calls it – and what it writes is read by the owner as "can this person be reached". Therefore:

- Every request is Svix signature-verified against `RESEND\_WEBHOOK\_SECRET`, over the **raw bytes**, before the body is parsed. No secret, missing headers, wrong signature, or a timestamp outside  $\pm 5$  minutes  $\rightarrow$  401, nothing written. **There is no verification-skip flag**, because that flag is the bug.
- It can write **only** the three `undeliverable_*` columns. It cannot insert, cannot upsert, cannot touch `role`, `granted_by`, `revoked_at`, or `revoked_by`. An unverified public endpoint that could change who signs in would be handing a stranger the access controls.
- Transient bounces (full mailbox, greylisting) are ignored – they clear on their own, and flagging them would put a red warning on a grant that is fine by morning.
- A recipient with no grant row (the owner's own newsletter) is a silent no-op.

### Residual risk, stated rather than solved

A typo landing on a domain **someone else controls** means that person receives a working set-password link and becomes a viewer. That is inherent to email invitation and is not fixed here. What this change does is make it **visible**: such an invite shows as "Invited – not yet accepted" until used, and the owner can revoke it. Shortening the link's validity is a separate decision, not taken.

### Acceptance tests

`tests/invite-delivery-state.test.ts` – 22 assertions, all 12 mutations in the harness caught, including: pending rendered as active, a bounce rendered as active, a post-acceptance bounce removing access, a dev bypass on the missing secret, dropped replay tolerance, an always-true signature compare, the webhook gaining insert/upsert or touching `revoked_at`, parsing before verifying, and the page reverting to counting un-revoked rows.

## Email links and delivery truth (2026-09-15, owner-approved "fix only the login issues")

### Defect 1 – an invite link changed the owner's password

Observed in production: the owner opened a viewer's invitation in a browser still signed in as the owner, chose a password, and it was written to the OWNER's account ("New password should be different from the old password" on a retry confirmed it). The viewer's account never received a password, so it could not sign in.

Cause, two parts together:

1. The route mailed Supabase's `action_link`, which returns tokens in the URL hash. The PKCE browser client from `@supabase/ssr` does not read hash tokens, so opening the link never replaced the owner's session.
2. `/reset-password` showed its form whenever `getSession()` returned any session and then called `updateUser({ password })` on it.

Fix (`lib/auth/email-link.ts`, `app/reset-password/page.tsx`, `app/auth/confirm/page.tsx`, `app/api/admin/access/route.ts`):

- Invitation and recovery emails link to `/reset-password?token_hash=...&type=...`, built from `generateLink().properties.hashed_token`. The owner's "Resend email" sign-in link goes to `/auth/confirm?token_hash=...&type=magiclink&next=/dashboard/portfolio`.
- Both pages verify the link themselves (`verifyOtp`; also a PKCE `code`, and Supabase's hash tokens via `setSession` so the login page's Supabase-sent "Forgot password" mail keeps working). Verifying replaces whatever session the browser had. No link, an expired link, or a link of the wrong type is an error; an existing session is never a fallback.
- `/reset-password` shows "for <email>", strips the token from the address bar, and re-checks `getUser().id` against the link's account immediately before `updateUser`.
- A sign-in link cannot set a password and a recovery link cannot be used as a plain sign-in link.

Residual: opening a viewer's link in the owner's browser now signs that browser in as the viewer (the correct account), which signs the owner out there.

### Defect 2 – "the notice email sent" when it was only accepted

A deletion notice was accepted by Resend and landed in the recipient's spam folder while the page said it was sent. The message id was discarded, and the grant row that could have held a status is cascade-deleted with the account.

Fix: `access_email_notices` (migration `20260915220000`, applied and verified in production). The route records every accepted invitation, sign-in link, revoke and delete notice by Resend message id; the webhook updates its status by id before looking for a grant (update only, never insert). The access page words



the result as "accepted for delivery" and lists recent access emails with status: accepted, delivered to their mail server, bounced, reported as spam by recipient. It states plainly that a spam-folder placement is invisible and shows as delivered.

Not fixable in code: mail from Resend's shared ``<verified-email-sender>`` sender is often filtered to spam and may only be sent to the Resend account owner. A verified sending domain plus `EMAIL\_FROM` is the owner's decision.

### ### Acceptance tests

`tests/email-link-session.test.ts` (link parsing, session establishment, type restrictions, open-redirect guard, and source guards on both pages and the route) and `tests/access-email-notices.test.ts` (id recorded for all four email kinds, webhook update order, RLS with no policy, page never overclaims).

### ## Phase 3 page set (2026-09-15, owner-approved)

Owner asked to open Earnings Calendar, Research → Daily Funnel, Score Tracker and Markets to viewers, and reported that a viewer clicking a symbol saw "Failed to load CORDSCABLE.NS: Forbidden". Audit and outcome:

|  | Surface            | Verdict | Why / how                                                                                                          |
|--|--------------------|---------|--------------------------------------------------------------------------------------------------------------------|
|  | Deep Dive          | ✓       | ✓                                                                                                                  |
|  | Research           | ✓       | ✓                                                                                                                  |
|  | Score Tracker      | ✓       | ✓                                                                                                                  |
|  | Evolution          | ✓       | ✓                                                                                                                  |
|  | Decision Review    | ✓       | ✓                                                                                                                  |
|  | Pipeline Health    | ✓       | ✓                                                                                                                  |
|  | New Listings       | ✓       | ✓                                                                                                                  |
|  | Learning internals | ✗       | owner-only routes; hidden for                                                                                      |
|  | Markets            | ✗       | Massive / Finnhub / SEC calls, an insider panel that reads the owner's live holdings, and an LLM "refresh macro re |

`VIEWER\_API\_ROUTES` entries can now be `exact`, so allowing a route never admits its children (`research-journal/context` and `/evolution` stay owner-only).  
`lib/auth/use-role.ts` gives pages the role for PRESENTATION only; every route enforces access itself.

Known gap, not changed here: `/api/research/chart-data` (Fundamentals chart, viewer-allowed since Phase 1) calls `fetchPriceHistory`, which can backfill from a provider when `price\_cache` is short. The route sweep checks route files, not the libraries they import, so it did not catch this.

Acceptance: `tests/viewer-research-pages.test.ts` (allowlist boundaries, live-book exclusion in the funnel / trades / price routes, and that viewer pages fire no owner-only or provider request) and `tests/viewer-route-sweep.test.ts` (now honors `exact`).

---

# Source: features\stock-context\FEATURE\_ARCHITECTURE.md  
# Feature Architecture: Stock Context

### ## Status

Architecture status: Draft  
Architecture approved: Yes (owner: "build it"; Codex endorsed peers-context-only)  
Approved scope: Read-only "what is this stock" context layer, off the money path  
Approved date: 2026-07-15  
Implementation allowed: Yes

### ## Feature Purpose

Research and watchlist screens currently show bare tickers (`RELIANCE.NS`, `NVDA`) with no plain-language answer to "what is this company and why am I looking at it?". Stock Context adds a lightweight, cached, display-only profile layer: company name, a one-sentence "what it does", sector/industry, exchange, market-cap tier, next earnings date, and up to a few peer tickers shown as \*\*context chips\*\* (never clickable trades). It also surfaces a short "why revisit + when" reason derived from data the app already holds.

This is orientation only. It never scores, sizes, gates, or orders anything.

### ## User/System Questions This Feature Answers

- What is this company, in one sentence?
- What sector / industry / exchange is it, and roughly how big (mega/large/mid/small/micro)?
- When does it next report earnings?
- Who are a few peers (for context – NOT trade suggestions)?
- Why is this symbol in my queue/watchlist, and when should I look again?

### ## Scope

This feature includes:

- A `symbol\_profiles` cache table (per `(symbol, market)`), RLS-locked.
- `lib/data/symbol-profile.ts` – fetch + cache one symbol's profile (FREE providers only).

- `app/api/agents/symbol-profiles/backfill/route.ts` – owner/cron-gated, bounded backfill for watchlist symbols missing a fresh (<30d) profile
- `lib/revisit-reason.ts` – pure function: short "why revisit + when" string from existing row data.
- `app/api/symbol-profile/route.ts` – owner-gated GET returning the cached profile + revisit reason.
- `components/dashboard/StockContextStrip.tsx` – mobile-first display strip.
- Wiring the strip into the symbol detail page.

## ## Non-Goals

This feature does not include:

- Any LLM call anywhere (the one-liner is composed deterministically).
- Any import by scoring / sizing / gating / order code (off the money path – hard invariant).
- Clickable peers or peer-driven navigation into trades. Peers are inert context chips.
- Live price/quote (that already exists elsewhere on the detail page).
- Paid data providers. FREE tier only (Finnhub free, Yahoo auth-free/crumb).
- Mixing per-market data or per-currency values.

## ## Current Behavior

Research/watchlist rows render the ticker string and, where present, `watchlist.company\_name` (often null). No sector/industry/earnings/peers context.

## ## Proposed Behavior

1. A backfill job (owner-triggered or cron) fills `symbol\_profiles` for watchlist symbols that lack a profile fresher than 30 days, bounded by a wall-clock budget so it never blows Vercel `maxDuration`. It also copies the fetched company name into `watchlist.company\_name` when that column is null.
2. The symbol detail page reads the cached profile + a computed revisit reason via an owner-gated GET route and renders `StockContextStrip`.
3. If no profile is cached yet, the strip renders nothing (graceful empty state) – the page is fully functional without it.

## ## User Journey / System Flow

1. Owner opens `/dashboard/symbol/NVDA`.
2. Page server-fetches signals/trades (existing) and passes `symbol` + `market` to the client.
3. `StockContextStrip` calls `GET /api/symbol-profile?symbol=NVDA&market=us`.
4. Route (owner-gated) reads `symbol\_profiles` cache + computes revisit reason, returns JSON.
5. Strip renders: name · one-liner · sector · exchange · mkt-cap tier · next earnings · peer chips · revisit reason.

## ## Screen / Page / Module Inventory

- Table: `symbol\_profiles` (cache).
- Module: `lib/data/symbol-profile.ts` (fetch+cache; US Finnhub, India Yahoo).
- Module: `lib/revisit-reason.ts` (pure).
- Route: `POST /api/agents/symbol-profiles/backfill` (owner/cron-gated, bounded).
- Route: `GET /api/symbol-profile` (owner-gated read).
- Component: `components/dashboard/StockContextStrip.tsx`.
- Wiring: `app/dashboard/symbol/[symbol]/page.tsx`.

## ## UI Architecture

### ### Layout

A single horizontal strip above the tab bar on the symbol detail page. On mobile it wraps to multiple rows; each datum is a labeled inline chip. Peer tickers render as a small chip row labeled "Peers (context)".

### ### Components

`StockContextStrip` (client). Fetches its own data; renders nothing while loading or when no profile exists. Uses the existing `T` token palette + `clamp()` responsive sizing.

### ### Sheets / Modals / Drawers

None.

### ### Navigation

None changed. Peer chips are NOT links and do not navigate.

### ### Empty State

Renders nothing (null) – the rest of the page is unaffected.

### ### Loading State

Renders nothing (null) until the fetch resolves – avoids layout jitter for a secondary, non-blocking strip.

### ### Error State

On fetch error the strip renders nothing (fail-soft; this is context, not critical).

### ### Success State

Full strip with all available fields; missing fields are omitted, not shown as "-".

## ## System Architecture

### ### Modules

- `symbol-profile.ts` – `getSymbolProfile(symbol, market, svc)` returns the cached row if fresh (<30d), else fetches + upserts + returns. `fetchSymbolProfile()` composes

```

the profile from free providers. `oneLinerFrom()` builds the deterministic sentence.
`capTierFrom()` maps market-cap (USD millions) to a tier.
- `revisit-reason.ts` - `computeRevisitReason(input)` pure; no I/O.

API Contracts
- `GET /api/symbol-profile?symbol=<sym>&market=us|india` → `{ profile: SymbolProfile | null, revisit: { reason, urgency } | null }`. Owner-ga
- `POST /api/agents/symbol-profiles/backfill?market=us|india&limit=<n>` (owner OR `x-cron-secret`) → `{ processed, filled, watchlistNamesBack

Data Models
`symbol_profiles`:
- `symbol text`, `market text` (`us|india`), `company_name text`, `one_liner text`,
 `sector text`, `industry text`, `exchange text`, `market_cap_tier text`
 (`mega|large|mid|small|micro`), `next_earnings_date date`, `peers text[]`,
 `source text`, `updated_at timestampz`. PK `(symbol, market)`.

Auth / Permissions
- Table: RLS ENABLED. Policy: `for select to authenticated using (true)` - anon denied,
 owner (authenticated) can read, service_role writes via bypass. (New public tables
 with RLS off are a Security Advisor ERROR - this table ships RLS-on from migration 1.)
- Read route: `requireOwner()`.
- Backfill route: `requireOwner()` OR `verifyCronSecret()` (service-to-service).

Error Handling
Every external fetch is wrapped and fail-soft: a provider miss yields a partial or
null profile, never a thrown error. India peers/earnings are best-effort (often null).

Data Architecture
- Required data: `symbol`, `market`.
- Optional data: everything else (all nullable; store what the provider returns).
- Mock vs real vs derived: real (Finnhub/Yahoo) + derived (`one_liner`, `market_cap_tier`).
- Persistence: `symbol_profiles`, refreshed when older than 30 days.
- Validation: `market` constrained to `us|india`; `market_cap_tier` constrained to the 5 tiers.

Providers (FREE only)
- **US**: Finnhub `/stock/profile2` (name, finnhubIndustry→sector, exchange,
 marketCapitalization→tier), `/stock/peers` (peers), `/calendar/earnings`
 (next earnings date). Finnhub free = 60/min, no daily cap.
- **India** (`.NS`/`.BO`): Yahoo (auth-free chart + crumbed quoteSummary, reused from
 `lib/india-data.ts`) for name/sector/industry/market-cap/exchange, and
 `fetchIndiaEarningsDate` for next earnings. Peers are unavailable for free on India -
 stored as `[]`/null (best-effort, acceptable).

Off-Money-Path Guarantee
`lib/data/symbol-profile.ts`, `lib/revisit-reason.ts`, `StockContextStrip.tsx`, and both
routes are display-only. They are NEVER imported by scoring/sizing/gate/order code
(`lib/research-agent`, `lib/scoring/*`, `lib/data/scores.ts`, paper-trade/trader routes).
Import direction is one-way: this feature imports existing helpers; nothing on the money
path imports this feature. Deterministic - no LLM is invoked anywhere in this feature.

Per-Market Handling
`market` is a first-class key (`(symbol, market)` PK). US and India never share a row.
India profiles are Yahoo-sourced with best-effort peers/earnings; US profiles are
Finnhub-sourced. No per-currency numeric values are stored (tier is a categorical label),
so no currency mixing is possible.

Peers-Context-Only Rule
Peers are stored as `text[]` and rendered as inert chips under a "Peers (context)" label.
They are NEVER clickable, NEVER navigated to, and NEVER fed into scoring, screening,
candidate generation, or any candidate list. This is a hard boundary Codex endorsed.

Files Likely To Change
- `features/stock-context/FEATURE_ARCHITECTURE.md` (this file)
- `supabase/migrations/20260715150000_symbol_profiles.sql` (new)
- `lib/data/symbol-profile.ts` (new)
- `lib/revisit-reason.ts` (new)
- `tests/revisit-reason.test.ts` (new)
- `app/api/agents/symbol-profiles/backfill/route.ts` (new)
- `app/api/symbol-profile/route.ts` (new)
- `components/dashboard/StockContextStrip.tsx` (new)
- `app/dashboard/symbol/[symbol]/page.tsx` (wire strip)
- `lib/india-data.ts` (additive: expose name/market-cap/exchange from the Yahoo overview)

Files / Behavior That Must Not Change
- Scoring / sizing / gate / order code - must not import this feature.
- `computeScores`, `research-agent`, `paper-trade`, `trader` - untouched.
- Peer navigation - must remain absent.

Acceptance Criteria

```

- `symbol\_profiles` exists in prod with RLS on and an authenticated SELECT policy (verified).
- `getSymbolProfile` returns a cached row and refreshes when >30d old.
- No LLM call in any file of this feature.
- No scoring/order module imports this feature.
- Backfill is idempotent, bounded by a wall-clock budget, and backfills `watchlist.company\_name`.
- Strip is responsive (mobile-first) and renders peers as non-clickable chips.
- `tsc --noEmit`, `npm run build`, `vitest run` all pass.

## Approval

Architecture approved: Yes  
 Approved scope: As above (peers-context-only, off money path)  
 Implementation allowed: Yes

---

# Source: features\strategy-evidence-scorecard\FEATURE\_ARCHITECTURE.md  
 # Strategy Evidence Scorecard

> Status: **\*\*DRAFT** – for owner approval. No code written.\*\*  
 > Date: 2026-09-01. Influence if approved: **\*\*read-only surface.\*\*** It displays  
 > evidence; it activates nothing and no money path reads it.

## The question it answers

> Which strategy – or combination – is currently adding benchmark-relative  
 > return, capturing large winners, limiting losses, and doing so with enough  
 > evidence to trust?

Deliberately **\*\*not\*\*** "which strategy has the highest return", and explicitly  
**\*\*not** a win-rate leaderboard. A leaderboard ranked on win rate would reward the  
 exact pathology this book already exhibits: many small wins and one large loss.  
 Measured on the live ledger, `time\_stop` closes 73.7% of lots at a mean +1.66%  
 while `stop\_hit` loses a mean -4.72%.

## Location

A third tab on the existing Strategies page:

`Fit Scores · Algo Library · Live Evidence`

That page already shows symbol-to-strategy fit scores and reference templates.  
 Governed performance evidence currently lives in Learning, and raw research  
 observations live on Research – neither is the right home for a  
 strategy-versus-strategy comparison.

## The table

| Strategy         | Evidence stage | 12-week alpha | Payoff ratio | Profit factor | Winning weeks | Evidence  | Status       |
|------------------|----------------|---------------|--------------|---------------|---------------|-----------|--------------|
| RSI Pullback     | Forward shadow | +1.8%         | 1.9x         | 1.42          | 7/10          | 31 trades | Promising    |
| Trend Breakout   | Paper          | +0.6%         | 2.6x         | 1.18          | 6/12          | 24 trades | Collecting   |
| Calendar Overlay | Replay only    | +0.3%         | 1.1x         | 1.04          | 5/11          | 11 weeks  | Insufficient |
| Champion + RSI   | Paper          | +1.1%         | 1.7x         | 1.31          | 8/12          | 38 trades | Promising    |
| Current Champion | Paper          | -0.4%         | 0.8x         | 0.91          | 4/12          | 42 trades | Degrading    |

Visible columns, in priority order:

1. **\*\*Alpha versus the correct market benchmark\*\*** – never absolute return. US compares to V00, India to NIFTY 50; the two are never pooled.
2. **\*\*Payoff ratio\*\*** – average winner / average loser. This is the column that answers "win big, lose small", and it is why win rate is demoted.
3. **\*\*Profit factor\*\*** – gross gains / gross losses.
4. **\*\*Winning weeks\*\***, as `7/10` – never a streak (see below).
5. **\*\*Evidence count and stage.\*\***
6. **\*\*Status\*\*** – computed, never hand-set.

**\*\*Win rate is NOT a headline column.\*\*** It appears in the expanded row alongside average win, average loss, max drawdown, Sharpe, costs and a weekly sparkline.

## Winning weeks, not streaks

Show two distinct facts:

- **\*\*Positive in 7 of the last 10 completed weeks\*\***
- **\*\*Current positive-alpha streak: 3 weeks\*\***

A streak resets on a single bad week and encourages recent-winner chasing.  
 Persistence research on fund performance finds apparent short-term winners often reflect common exposures and costs rather than durable skill  
 ([Carhart 1997](https://doi.org/10.1111/j.1540-6261.1997.tb03808.x)) – applied cautiously here, since that work studied funds and not this system.

## Evidence stages – always visible

Every row shows which population produced its numbers:

```
stage	meaning
`replay`	historical simulation only; hypothetical
`shadow`	forward, non-executing, real-time signals
`paper`	executed in the paper book
`live`	real money
```

A replay number and a live number must never sit in the same column without this label. The SEC's guidance on performance claims is explicit that backtests are hypothetical, that past performance does not predict future results, and that cherry-picked periods mislead

([SEC investor bulletin](https://www.investor.gov/introduction-investing/general-resources/news-alerts/alerts-bulletins/investor-bulletins-47))

### ## Statuses – frozen thresholds

These are **financially load-bearing** and must be fixed here, before any strategy is graded against them. Tuning a threshold after seeing which strategy it promotes is the failure this whole document exists to prevent.

```
status	condition
`insufficient_evidence`	below the evidence floor: `nEffective < 12` or `nDates < 60`
`collecting`	floors met, no verdict yet – accruing forward evidence
`promising`	positive net alpha AND payoff ratio > 1 AND floors met, but not yet multiple-testing significant
`working`	ALL of: positive net benchmark alpha; payoff ratio > 1; profit factor > 1; max drawdown no worse than the champion's; multiple-testing significant
`degrading`	previously `working`, now failing any one of those conditions
`failed`	negative net alpha with floors met – measured, and measured badly
`retired`	owner-retired; row remains for the record
```

**`working`** requires more than a positive return. It requires benchmark-relative alpha, acceptable drawdown, positive expectancy, sufficient independent evidence, and no integrity failure. A strategy cannot reach **`working`** on replay evidence alone, ever.

A strategy declared a **hedge** is graded differently and says so: it may carry modest or negative standalone return, but must reduce downside enough to justify its carrying cost. It must not be quietly scored against the easier return-strategy criteria.

### ## Combination rows

Rows such as **`Champion + RSI + Trend`** appear alongside single strategies, sharing the same columns and statuses. The decision object is the champion-relative one defined in **`features/external-strategy-discovery`** section 7 – **`champion`**, **`champion+A`**, **`champion+B`**, **`champion+A+B`** – so a combination that merely duplicates existing exposure is visible as adding nothing.

### ## What exists and what is missing

Already available:

- **`components/dashboard/StrategyGovernancePanel.tsx:33`** surfaces signal count, win rate, average return, Sharpe, drawdown and alpha.
- **`lib/analytics/performance-metrics.ts:105`** computes expectancy and profit factor.

Missing, and therefore the actual build:

- strategy-level **weekly** return history
- **payoff ratio** attached to each strategy run
- benchmark-relative **weekly consistency** (**`7/10`**)
- **evidence-stage** separation
- a **governed status** calculation with the frozen thresholds above
- **combination rows**

### ## What this does NOT do

- It does not activate, promote, size or retire anything. Every action remains an owner decision.
- It does not rank by win rate, or by return alone.
- It does not display a replay number without its stage label.
- It does not pool US and India.

### ## Open question for the owner

The **`working`** threshold set above is my proposal, not a derived optimum. The one I would most expect to argue about is **“max drawdown no worse than the champion’s”** – that is strict, and it will block a strategy that earns more while drawing down slightly deeper. The alternative is a drawdown-adjusted return comparison, which is more permissive and harder to read at a glance. Say which you want before implementation; changing it afterwards, once rows exist, is exactly the tuning this document forbids.

---

# Source: features\strategy-library-shadow\FEATURE\_ARCHITECTURE.md

```

Public strategy library – shadow harness

> Status: **DRAFT – scope and brief. No code written. Not approved.**
> Date: 2026-09-01. Influence if approved: **none while shadow.** Any promotion to
> live is a separate money-path approval.
>
> Written to be actionable by Codex or any other agent without this conversation.

What the owner asked for

1. Parse the strategies at <https://www.quantifiedstrategies.com/trading-strategies-free/>
2. Run them as shadows.
3. When they qualify, start using them.
4. Test whether **combinations** work better or worse.
5. Determine **which strategies suit which stocks/ETFs** – explicitly: "some
 strategies work for some stocks, ETFs and not others, so the app and you
 should be able to customize."
6. Survey other reputable suites / GitHub projects and test any not used before.

All six are buildable. Point 3 and point 5 are where this either becomes a real
instrument or a noise generator, and the difference is entirely statistical
discipline, not engineering.

What is actually on the page (fetched 2026-09-01)

- **~200 strategies**, rules given in plain English, often with Pine Script or
 Python. Reported performance statistics are **frequently withheld or
 paywalled** – so the rules are usable, the claimed edges are not verifiable
 from the page.
- Categories, with examples:
 - **Mean reversion**: RSI(2) on SPY, 5-day low, NR7
 - **Trend**: 200-day MA, Golden Cross
 - **Seasonal/calendar**: Turnaround Tuesday, Turn of the Month, Santa Claus Rally
 - **Volatility**: Bollinger squeeze
 - **Overnight**: close-to-open
 - **Indicator**: MACD histogram, Stochastic
 - **Day trading**: outside day, intraday breaks
 - **Bonds**: TLT trend following

Three filters that remove most of them before any statistics

1. **Intraday and overnight strategies are out by construction.** This system is
 EOD-only with a 2-20 market-day swing mandate. Close-to-open, outside day and
 every day-trading rule cannot be evaluated on the data we persist, and
 pretending otherwise would produce a number with no path to execution.
2. **Index-level calendar rules are market-wide, not cross-sectional.**
 Turnaround Tuesday, Turn of the Month and Santa Claus Rally produce the SAME
 signal for every symbol on a date. That is exactly the defect found in
 `macro_score` on 2026-09-01: a constant within a date contributes nothing to a
 cross-sectional ranking, and can only act as a timing/exposure overlay. They
 belong in a separate exposure track, never in the selection composite.
3. **Rules whose published edge is paywalled cannot be replicated, only
 re-derived.** That is fine, but it means we are testing OUR implementation of a
 described rule, not validating their result. Any write-up must say so.

Realistic survivors: **roughly 20-40** rules that are EOD, cross-sectional or
per-symbol, and fully specified.

THE BINDING CONSTRAINT – read this before designing anything

This is not an engineering problem. The system currently cannot evaluate **one**
hypothesis, let alone hundreds.

Measured on this book, 2026-09-01:

quantity	value
independent decision dates, h10	**26** (US), 26 (India)
effective observations at h10 (`nDates / horizonDays`)	**2.6**
floor required for a verdict (`MIN_EFFECTIVE_OBSERVATIONS`)	**12**
dates needed for ONE h10 hypothesis	~120
US eligible-long h10 rank IC	**~0.0768**
India eligible-long h10 rank IC	**~0.0083**

Now apply multiplicity. Šidák-adjusted `alpha = 1 - (1 - 0.05)^(1/m)`:

| what is searched | trials `m` | adjusted alpha | critical two-sided `|t|` |
|---|---|---|---|
| 1 predeclared hypothesis | 1 | 0.05 | 1.96 |
| the ATR exit arm (already shipped) | 14 | 0.00366 | 2.91 |
| ~200 strategies | 200 | 2.6e-4 | 3.66 |
| 200 strategies x ~100 instruments | 20,000 | 2.6e-6 | **4.71** |
| all PAIRS of 200, per instrument | ~2.0e6 | 2.6e-8 | **5.57** |

**With 26 independent dates, power against a `|t|` threshold of 4.7 is

```

approximately zero.\*\* Every "winner" that emerges from a 20,000-arm search on this much data is overwhelmingly likely to be noise, and the search itself guarantees some arm will look excellent.

This is not an argument against building the harness. It is an argument that \*\*the harness must accrue evidence for months before it is read\*\*, and that its default verdict must be `insufficient\_evidence` – the same posture as `features/atr-exit-stop`, which is expected to say exactly that until ~2027-Q1.

## Per-instrument customization – the owner's point 5, done honestly

The intuition is correct: mean reversion genuinely behaves differently on a low-beta utility than on a leveraged semiconductor ETF. The naive implementation is also the single most dangerous thing in this brief.

\*\*Do NOT run an independent test per (strategy, instrument).\*\* With ~113 US symbols that is a 20,000-arm search, and per-symbol samples are tiny – most symbols have well under 40 observations.

\*\*Do this instead – hierarchical estimation with shrinkage:\*\*

1. Estimate one \*\*global\*\* effect per strategy across the whole universe.
2. Estimate per-instrument deviations, then \*\*shrink them toward the global estimate\*\* in proportion to how little data each instrument has. A symbol with 12 observations should barely move off the global mean; one with 300 can move further. (James-Stein / empirical-Bayes shrinkage; `pymc` or a closed-form normal-normal estimator both work.)
3. Prefer \*\*instrument-CLASS\*\* grouping over per-symbol wherever possible – sector, liquidity bucket, realised-volatility bucket, ETF vs single name. `lib/scoring/instrument-taxonomy.ts` already classifies instruments and is the natural grouping key. A per-class effect has an order of magnitude more data behind it than a per-symbol one and is far less likely to be noise.
4. Report the \*\*shrunk\*\* estimate as the headline. Report the raw per-symbol estimate only alongside its sample size, clearly marked as unstable.

The honest version of "customize per stock" is: \*learn a strategy-by-class effect, and let per-symbol deviate only where the data earns it.\*

## Combinations – the owner's point 4

Test combinations \*\*only\*\* as a predeclared, small, theory-motivated set. Not the full pairwise grid.

Reasonable: one trend rule + one mean-reversion rule (they are known to be negatively correlated, so the combination has a prior); a regime filter (200-day MA) applied to a mean-reversion entry. That is a handful of arms with a stated reason, not 19,900 arms.

Record the combination arms in the registry BEFORE running, and count them in the trial total. A combination discovered by scanning the grid must be labelled as such and carries the grid's multiplicity, not a single test's.

## Proposed build – three stages, only stage 1 is proposed now

### Stage 1 – registry + parser (buildable now)

- `strategy\_library` table: `id`, `source\_url`, `name`, `category`, `rule\_spec` (jsonb), `horizon\_days`, `applies\_to` (`cross\_sectional` | `per\_symbol` | `market\_timing`), `eligible` (bool), `exclusion\_reason`, `added\_at`, `predeclared\_at`.
- A parser producing `rule\_spec` from the page. \*\*Firecrawl is configured but failed to connect in this session (`CONNECT\_TIMEOUT`); use WebFetch or a plain HTTP fetch.\*\* The page is static HTML; JS rendering is not required.
- \*\*Every rule is entered by hand-review before it becomes `eligible`.\*\* An LLM-parsed rule that nobody read is a rule nobody understands, and this system has already been burned by acting on plausible-looking specifications that were never executed (`buildStockPrompt`, deleted 2026-09-01).
- Freeze `predeclared\_at` per strategy. Anything added after the evaluation window opens is a new trial, not a free one.

### Stage 2 – shadow evaluation harness (after stage 1 review)

- Reuse the existing evidence contract rather than inventing a second one: `lib/learning/entry-cohort.ts` (eligible-long only), `effectiveObservations`, `MIN\_EFFECTIVE\_OBSERVATIONS = 12`, `MIN\_REVIEW\_DATES = 60`, and the Sidák helper now in `lib/trading/exit-stop-shadow.ts`.
- Date-clustered, paired against the live champion on the SAME observations – the pattern `archetype-ic` already uses.
- Write to `strategy\_shadow\_runs`, storing `trials\_considered` and `sidak\_alpha` on every row, as `exit\_stop\_shadow\_runs` does.
- Default verdict `insufficient\_evidence`. Expected to stay there for months.

### Stage 3 – promotion (NOT proposed; requires separate approval)

Gates, all of which must hold, per market:

1. `nEffective >= 12` and `nDates >= 60`.
2. Šidák-adjusted significance at the **full** trial count, including every arm ever run, not just the survivors.
3. Positive out-of-sample on a forward window declared before it was seen.
4. Holds at more than one horizon, or the divergence is explained.
5. Mechanism confirmed – the strategy's stated reason for working is visible in the data, not just the return.

Failing any gate leaves the arm in shadow. There is no partial promotion.

## Existing suites worth adopting (survey, 2026-09-01)

We should not write a backtester. Several mature ones exist.

```
project	what it is	fit here
[VectorBT](https://github.com/polakowo/vectorbt)	NumPy/Numba vectorised; built for thousands of parameter combinations	Best fit for the
[Backtesting.py](https://github.com/kernc/backtesting.py)	lightweight, mature, intuitive	Good for single-strategy verification and sanit
[Zipline-Reloaded](https://github.com/stefan-jansen/zipline-reloaded)	Quantopian descendant, pipeline API	Strong for factor/cross-sect
[Backtrader](https://github.com/mementum/backtrader)	event-driven, huge community	Mature but the upstream is quiet; a fork is more activ
[NautilusTrader](https://github.com/nautechsystems/nautilus_trader)	Rust-native, production event-driven engine	Overkill for EOD swing s
[pandas-ta](https://github.com/twopirllc/pandas-ta)	130+ indicators, 60+ candlestick patterns	Adopt. Removes the need to hand-implement
[awesome-systematic-trading](https://github.com/paperswithbacktest/awesome-systematic-trading)	curated index of libraries and papers	Use
[awesome-quant](https://github.com/wilsonfreitas/awesome-quant)	broader curated list	Same
```

**Recommended:** `pandas-ta` for indicators + `VectorBT` for the sweep, both offline and measure-only. Neither touches the live path. A Python worker already exists in this repo (`scripts/python`).

**Caution worth stating:** VectorBT makes it effortless to run a million backtests. The binding constraint is statistical, not computational, and a tool that removes the compute cost removes the only friction that was accidentally protecting us.

## Honest assessment

**Build stage 1.** A registry of predeclared, hand-reviewed rules with frozen timestamps is useful regardless of what the evaluation later says, and the data accrues whether or not anyone is watching.

**Do not expect a usable answer this year.** At the current rate (~26 h10 dates per seven weeks) a single-hypothesis verdict arrives around 2027-Q1. A 200-strategy search needs materially more than that, and a per-symbol search of that size may never be answerable on one book's traffic – which is the real argument for class-level grouping over per-symbol.

**The biggest risk is not a bad strategy. It is a good-looking one.** This book has produced four retracted claims in the past week, every one from reading a number that came from the wrong population or an unrun code path. A 200-arm search on 26 dates will manufacture beautiful numbers. The harness's job is to refuse them.

---

# Source: features\strategy-portfolio-lab\FEATURE\_ARCHITECTURE.md  
# Strategy Portfolio Lab

```
> Status: Approved for P0 implementation. Owner reconfirmed the shared
> paper/live evidence requirement on 2026-09-13; this remains non-executing:
> no score, paper fill, live proposal, exit, sizing, or broker activation.
> Date: 2026-08-02
```

## Decision

Turn a selected Strategy Library template into an immutable, market-local strategy version that can validate and collect non-executing shadow decisions beside the champion. Templates remain reference material until explicitly admitted to this lab.

## Boundaries

- US and India versions, evidence, limits, and comparisons never mix.
- At most **three total** active template shadows per market; a predeclared combination challenger counts as one of the three. This conservative capacity decision prevents sample starvation and supersedes the earlier ambiguous "three plus one" wording.
- A combination is declared before its first validation run and increments the market trial-family count. No search across arbitrary template combinations.
- The canonical scorer, PaperTrader, PositionMonitor, cash, positions, exits, Router, and broker paths never read template-shadow output.
- A passed backtest is eligibility evidence only. Promotion remains governed by the existing dormant OOS gate and owner-only RPC.

## P0 Build

1. Add a service-only `strategy\_template\_shadow\_configs` ledger: market, template



- id(s), immutable config fingerprint, state ('draft|validating|shadow|paused|retired'), declared trial family, and owner audit fields.
2. Compile one selected template into a 'strategy\_versions' challenger with frozen entry/exit rules and source-template lineage; reject any rule not represented by Kairos' deterministic grammar.
3. Reuse existing deterministic validation and 'shadow\_decisions'; do not create a second evaluator or price path.
4. Add a Learning comparison view: champion versus each shadow, decision overlap, would-enter count, turnover proxy, validation result, latest shadow evidence, and explicit "insufficient evidence" until forward outcomes mature.
5. Label Strategies as manual research tools until a template has an active shadow.

## ## Evidence and recommendation

The system may recommend only: continue collecting, pause/retire for a deterministic failure, or submit for owner review. It cannot recommend activation from the best historical result without the declared trial-family multiple-testing adjustment, PIT walk-forward evidence, and forward shadow outcomes.

## ## Build order

P0 schema/config + template compiler; P1 shadow comparison UI; P2 predeclared combination compiler; P3 only after promotion-grade OOS evidence exists.

## ## Acceptance

- A template shadow cannot write a signal, fill, proposal, exit, or order.
- Attempts beyond the per-market limits fail atomically.
- Every candidate and combination has frozen source/config/trial provenance.
- Comparison never sums USD and INR evidence.
- Disabling or retiring a shadow preserves history and immediately stops new rows.

---

# Source: features\symbol-score-history\FEATURE\_ARCHITECTURE.md  
# Feature Architecture: Per-Symbol Score History vs Price

## ## Status

Architecture status: Approved  
Architecture approved: Yes (owner, 2026-09-15 – "Proceed")  
Approved scope: Read-only. Plot the dimension scores already written by every research run against price, for any researched symbol.  
Implementation allowed: Yes – shipped 2026-09-15.

## ## The defect this fixes

`/dashboard/research/<symbol>` → **\*\*Score History\*\*** read `/api/research/trades`, whose scores live as columns on the `paper\_trades` row. So a symbol had a score history **\*\*only if it had produced a trade\*\***.

Research runs on hundreds of symbols that never trade. Every one of those runs already writes the full dimension set to `agent\_signals`. Observed 2026-09-15:

| Symbol | Research runs                | Paper trades | Tab showed                              |
|--------|------------------------------|--------------|-----------------------------------------|
| STM    | 56 (2026-07-17 → 2026-09-15) | 0            | "No scored paper trades found for STM." |

Two months of daily scores – including a fundamental move from 23 → 70 around 2026-09-12 that pulled the analyst score 35 → 56 – were in the database and invisible on the page built to show them.

Coverage is not the constraint. Last 90 days: **\*\*364 symbols, 9,107 runs\*\***, with `fundamental/technical/sentiment/macro/insider` present on all but 5 rows.

## ## Design

### ### Source of truth

`agent\_signals`, keyed by `symbol` + `market`. It is written once per research run per symbol, trade or no trade. `paper\_trades` is no longer consulted for score history; it still supplies execution markers drawn **\*\*over\*\*** the series, so a symbol that did trade keeps both.

### ### One point per day, last run wins

A symbol is sometimes researched twice in a day (STM was, on 2026-08-31 and 2026-09-01). For STM's 365-day window, **\*\*56 runs collapse to 34 chart points\*\***.

- Plotting both puts two points on one x value, which renders as a vertical spike that looks like score volatility that never happened.
- **\*\*Averaging them is rejected\*\***: it invents a score no run ever produced. The last run of the day wins – it is a real observation, and it is the one standing at the close.
- The point carries `runs`, and the UI says "last of N runs that day", so the

collapse is never silent.

`lib/research/score-history.ts` owns this rule so it is testable apart from the route and the chart.

### Gaps are shown as gaps

Scores exist only on days a research run happened. Those series are null on every other day and the lines `**break**` there – ``connectNulls`` is deliberately off for scores (and on for price, which is continuous). Joining the dots across a three-week gap would draw a trend the agent never observed.

### Honest zero

``technical_score`` legitimately reads ``0`` for STM. Checked before assuming a bug: across 30 days technical spans 0-100 with a mean of 61.7, so STM genuinely scores low – it is down 7%. A real ``0`` must survive as ``0`` and never collapse to null, or the chart would report "no data" where the agent had a firm opinion.

### Empty state

The old message blamed missing `*trades*`. A symbol with no runs in the window now reads "No research runs recorded for {symbol} in this window" – a different claim, and the true one.

## Non-goals (explicitly out of scope)

- `**Per-indicator attribution**` (which P/E or RSI reading moved a dimension). Not recorded on the signal row; the UI says so rather than guessing.
- `**`recommendation`**` – null on every STM row, so it is not surfaced.
- Any write path. This feature reads; it changes no score, trade or decision.

## Acceptance tests

``tests/research-score-history.test.ts`` – 16 assertions. All 13 mutations in the harness caught, including: same-day runs averaged, same-day runs both kept, a real zero collapsed to null, the Insider dimension dropped, provenance dropped, points unsorted, the route reverting to ``paper_trades``, the route losing its owner gate or its market scoping, the tab gating on trades again, score lines smoothing over gaps, and the dual axis collapsed to one.

Two of those mutations initially survived and exposed vacuous assertions of the author's own making: a dimension check that looped over ``SCORE_DIMENSIONS`` (so deleting a dimension deleted its own assertion), and an axis check that matched ``yAxisId`` anywhere (the lines carry it too, so it passed with the axis removed). Both tests were rewritten to name what they assert.

## Update triggers (per CLAUDE.md)

Update this file when: the score dimensions change, the same-day collapse rule changes, the source table changes, or per-indicator attribution is added.

---

# Source: features\system-health-model-resilience\FEATURE\_ARCHITECTURE.md  
# Feature Architecture: System Health funnel + Model Resilience

## Status

Architecture status: Implemented (P1 + P2 + P3)  
Architecture approved: Yes ("build", 2026-07-07)  
Approved scope: All three phases  
Approved date: 2026-07-07  
Implementation allowed: Yes

### Implementation notes (2026-07-07)

- Migration 099: ``agent_alerts.issue_key`` + partial unique index (one open row per key).
- ``lib/system-health.ts``: ``reportIssue`` / ``resolveIssue`` / ``reconcileIssues``.
- Reporters wired: model-check (deprecated → critical, reachable-provider-aware auto-resolve; newer-available → info), AV budget exhaustion (self-expiring), kill-switch trips (per market), unreconciled orders (Gateway), Robinhood + Kite token expiry.
- Surfaces: ``SystemHealthCard`` on the dashboard home (persistent, severity-ranked, deep-link fix hints; green when clean) + "Open Issues" band in every daily brief. DashboardShell severity vocabulary extended with ``critical`` (top rank).
- Model resilience (``lib/lm-router.ts``): ``TIER_MODELS`` aliases (fast/reasoning/claude-fast/claude-smart), ``SAME_TIER_FALLBACK`` graceful fallback on an unavailable model (loud ``model-fallback:`` alert, never blind-latest), and ``priceFor`` pricing fallback so cost never silently logs \$0.

## Why this feature exists

Tonight's session exposed the core risk of an autonomous-ish trading system: `**silent failures.**` Every real problem was invisible until stumbled upon – DeepSeek models deprecated (agents would've errored on next call), the vault

`display\_name`/`provider` NOT-NULL break, the `trade\_queue` vs `trade\_proposals` split, the AV budget starving the scorer, the snapshot parse returning nulls. None surfaced anywhere the user would see them.

Two related gaps:

1. **\*\*No unified "open issues until fixed" surface.\*\*** Issues are scattered: `model-check` and a couple of others post to `agent\_alerts`, but most subsystems report nothing, there's no dedup/auto-resolve, and there's no persistent dashboard panel or brief section that shows OPEN issues until they clear.
2. **\*\*Model config is brittle.\*\*** Concrete model names are hardcoded/stored per agent; when a provider renames or deprecates a model, every agent pointed at it breaks and must be hand-updated (as just happened). There is no tier indirection and no graceful fallback – a deprecated model is a hard error, not a flagged degradation.

This feature makes failures **\*\*loud and persistent\*\*** and makes model config **\*\*break-proof but never silently drifting\*\***. It changes no trading logic.

Existing infra to BUILD ON (do not duplicate):

- `agent\_alerts` table: id, created\_at, severity, category, title, detail, resolved, resolved\_at, auto\_expire\_at.
- `/api/alerts`: GET (open alerts), POST (create), PATCH (resolve).
- Current reporters: `models/check`, `research/cron`, `broker/orders/sync` only.

---

## Phase 1 – Issue funnel: standardized reporting + dedup + auto-resolve

### Problem

`agent\_alerts` exists but: (a) most subsystems don't report into it; (b) no stable dedup key, so a recurring condition would spam duplicate rows (or, as today, never report at all); (c) no auto-resolve when the condition clears.

### Proposed behavior

- Add `agent\_alerts.issue\_key` text` (a stable identifier for a condition, e.g. `model-deprecated:deepseek-reasoner`, `av-budget-exhausted`, `kite-token-expired`, `order-needs-reconcile:<orderId>`, `cron-failed:kairos-learner`). Migration adds the column + a partial unique index on `(issue\_key) WHERE resolved = false` so at most one OPEN row per condition.
- A shared helper `reportIssue({ issueKey, severity, category, title, detail, autoExpireAt? })` – upserts by `issue\_key` on open rows (refreshes detail / keeps the existing open row rather than duplicating), and `resolveIssue(issueKey)` – marks the matching open row resolved. Both service-side, used by any subsystem.
- Wire reporters across the app (each reports AND auto-resolves):
  - **\*\*Model deprecation\*\*** (`models/check`): open an issue per deprecated assignment; resolve when the model reappears / is reassigned.
  - **\*\*API budget\*\*** (`av-cache` / a daily check): open when AV daily budget is exhausted; auto-expire next day.
  - **\*\*Broker/connection health\*\***: Kite token expired/near-expiry, Robinhood MCP token missing/refresh-failing.
  - **\*\*Kill switches\*\***: open an issue while a kill switch is tripped; resolve when cleared.
  - **\*\*Unreconciled orders\*\***: `broker\_orders.status = unknown\_needs\_reconcile` opens an issue until the order reconciles.
  - **\*\*Cron failures\*\***: a cron that errors or writes 0 where it shouldn't.
  - **\*\*Schema drift\*\***: a migration named in the repo not applied to the live DB (optional, lower priority).
- Auto-resolve: reporters that run on a schedule resolve their own issue when the condition is gone; time-bounded ones use `auto\_expire\_at`.

### Data model

- `agent\_alerts.issue\_key` text` + `create unique index ... (issue\_key) where resolved = false`. (Verify current indexes first.)

### Acceptance criteria

- A recurring condition produces exactly ONE open `agent\_alerts` row (dedup by issue\_key), not duplicates.
- Resolving the underlying condition auto-clears the issue.
- At least the model-deprecation, budget, broker-token, kill-switch, and needs-reconcile conditions report into the funnel.

---

## Phase 2 – Persistent surfaces: dashboard System Health card + brief section

### Problem

Even the issues that ARE in `agent\_alerts` aren't shown anywhere persistent – only reachable by opening a specific card.

### Proposed behavior

- **\*\*Dashboard "System Health" card\*\*** (top of `/dashboard`, or the status bar): reads OPEN `agent\_alerts`, shows a count badge (red when any CRITICAL/HIGH),

- grouped by severity + category, each with title/detail and (where possible) a one-click "how to fix" hint or deep link (e.g. model-deprecation → Settings → LLM Config). Stays visible until issues resolve. Collapsed/green when clean.
- **\*\*Daily brief "Open Issues" section\*\*** (morning + evening, both markets): lists OPEN issues (severity-ranked) so they're in the user's face every day until fixed – the thing that would have surfaced tonight's silent breakages on day one.
- Owner-only; reads via ``/api/alerts`` (already exists) or a small ``/api/system-health`` aggregator.

#### ### Acceptance criteria

- Open issues appear on the dashboard persistently and in every daily brief until resolved.
- Zero open issues → a clean/green state, no noise.
- Severity ordering: CRITICAL/HIGH first; live-trading-affecting issues (broker token, kill switch, needs-reconcile) surfaced most prominently.

#### ### Operational taxonomy correction (2026-07-20)

``agent_alerts`` remains the transport for both actionable conditions and bounded operational telemetry, but the product must not call both "open issues":

- ``critical``, ``error``, and ``warn`` are **\*\*actions required\*\***, shown expanded and counted in the System Health headline;
- ``info`` is an **\*\*operational notice\*\***, collapsed by default and counted separately;
- successful agent activity (for example, Theme Scout adding symbols) belongs in the agent/watchlist history, not System Health, and must not write an alert;
- expected sparse evidence and free-tier quota exhaustion may remain notices because they explain score coverage, but their wording must not imply a broken provider or more real calls than the enforced budget allowed.

This changes presentation and activity routing only. It does not suppress real broker, cron, data-starvation, kill-switch, or reconciliation faults.

---

#### ## Phase 3 – Model resilience: tier aliases + graceful fallback

##### ### Problem

Model names are concrete and per-agent. A provider rename/deprecation hard-breaks every agent pointed at it (just happened: deepseek-chat/reasoner → V4). "Always use latest" is NOT the answer for a trading system – a new model silently changes reasoning quality, output format, and cost, which must never drift under live financial decisions without review (the model-check's "never auto-switched, upgrades come from reviews" stance is correct).

##### ### Proposed behavior

- **\*\*Tier aliases.\*\*** Introduce roles (``fast``, ``reasoning``) resolved by ONE small mapping (config table or ``lib/llm-router.ts`` constant) → the current concrete model per provider (e.g. fast→deepseek-v4-flash, reasoning→deepseek-v4-pro). ``agent_config`` can reference a tier OR a concrete model. Next provider rename = update ONE mapping line, not every agent row.
- **\*\*Graceful fallback (break-proof, never silent).\*\*** When ``callLLM`` gets a model-not-found / deprecated error (or the model-check has flagged the model), the router falls back to a currently-available model of the SAME tier and ``reportIssue({issueKey:'model-fallback:<agent>', severity:'high', ...})`` so the run completes but the swap is loudly, persistently flagged for review. It does NOT silently ride "latest" – the fallback is same-tier and surfaced.
- **\*\*Pricing safety.\*\*** When a new model is used, if it has no ``PRICING`` entry, cost logging must not silently zero it – fall back to the tier's price and flag "pricing unverified for <model>" so ``llm_call_log`` cost stays meaningful.

##### ### Non-Goals

- No auto-upgrade to the newest model. Model \*upgrades\* remain a reviewed decision (the checker proposes; a human approves), per existing doctrine.

##### ### Acceptance criteria

- A deprecated configured model does NOT hard-break the agent – it falls back to a same-tier available model and raises a persistent issue.
- Changing which concrete model a tier maps to is a one-line change.
- No LLM cost is silently logged as \$0 due to a missing pricing entry.

##### ## Cross-cutting

- Read-only/advisory surfaces; nothing here trades, sizes, or changes agent reasoning beyond swapping a broken model for a same-tier working one.
- Owner-gated APIs; no secrets in alert ``detail``.

##### ## Files (indicative, verify before building)

- ``supabase/migrations/0NN_agent_alerts_issue_key.sql`` (issue\_key + partial unique index)
- ``lib/system-health.ts`` (new – reportIssue/resolveIssue helpers)
- reporters: ``app/api/models/check``, ``lib/av-cache.ts``, ``lib/kite.ts`` / ``lib/robinhood-mcp.ts`` (token health), ``lib/kill-switches.ts``, ``app/api/broker/orders/sync``, cron routes.

```

- `components/dashboard/SystemHealthCard.tsx` (new) + dashboard/StatusBar wiring
- `app/api/briefing/generate/route.ts` (Open Issues section)
- `lib/llm-router.ts` (tier aliases + graceful fallback + pricing fallback)
- `app/api/system-health/route.ts` (optional aggregator)

Approval

Architecture approved: No
Approved scope: None
Implementation allowed: No

Source: features\system-reference\FEATURE_ARCHITECTURE.md
System Reference Architecture

> Status: Approved
> Implementation allowed: Yes
> Last updated: 2026-09-21

Goal

Replace the stale, hardcoded Architecture tab on Dashboard -> Agents with a
maintainable owner-only reference surface. Preserve the live agent diagram as the
topology authority and do not add a new main-navigation documentation area.

Scope

- Curated documents grouped by orientation, architecture chapters, decisions, and
 selected feature designs.
- Owner-gated read/download endpoint backed by a fixed document registry.
- Architecture portal, concise system overview, and chapter-index governance rules.
- Existing Agent & Flow Architecture visualization remains in place below the tabs.

Contract

`GET /api/system-reference/:documentId`

- Requires the confirmed owner through `requireOwner()`.
- `documentId` must resolve from `SYSTEM_REFERENCE_DOCUMENTS`; unknown IDs return 404.
- The server reads only the fixed registry path and returns Markdown with `private`,
 no-store` caching and `nosniff`.
- `?download=1` changes disposition to attachment. No request value is used to form a
 filesystem path.

Non-goals

- No repository/file browser, editor, search index, public docs portal, or markdown
 authoring workflow.
- No architecture state in Supabase and no user-provided documents.
- No provider, data, scoring, paper, live execution, broker, or schema change.
- No duplicate static agent topology; the JSON map remains canonical.

Governance

The portal explains ownership. `docs/arch/` is the detailed operational architecture;
feature documents are micro-architecture; implementation result files record delivery;
project decisions capture approval and rationale. An agent-flow change updates the
diagram source and history, then its relevant chapter. A drift test guards the diagram
contract.

The friend-facing current-state document is `docs/arch/10-current-system-reference.md`.
This feature design specifies the reference surface; chapter 10 explains current
runtime contracts and distinguishes active, paper-only, shadow, collecting, blocked,
and proposed work.

Acceptance criteria

1. The old hardcoded architecture content, including stale schedules and account
 fragments, is removed from the client bundle.
2. The System Reference tab supports opening and downloading only allowlisted files.
3. Anonymous/non-owner calls receive the usual owner-gate response; traversal-like
 identifiers cannot return a repository file.
4. The live topology remains visible and linked from the reference panel.
5. TypeScript, focused tests, and production build pass.

Source: features\systematic-pattern-discovery\FEATURE_ARCHITECTURE.md
Systematic Pattern Discovery – Feature Architecture

> Status: **PARKED – IDEA ONLY. NEEDS MORE RESEARCH AND THOUGHT BEFORE ANY BUILD.**
> Captured: 2026-09-05. Author: Claude (Opus 5), Architect role. Owner: Vaibhav.
> **Do NOT start implementation from this document.** It records a line of thinking and the
> objections against it. Several load-bearing claims below are UNVERIFIED training knowledge

```

> and are marked as such. Every one must be checked against a primary source before any  
> decision rests on it.  
> No migration, no code, no data acquisition, no provider signup is authorized here.

## ## 0. What this is

An automated alpha-research layer: mine long historical price/fundamental data for statistically significant patterns, validate them cross-sectionally and out-of-sample, and graduate only the survivors into Kairos's existing shadow framework as non-executing signals.

Triggered by a 2026-09-05 conversation about the overnight-drift anomaly (buying at close and selling at the next open historically dominating open-to-close returns for some names).

## ## 1. Why it's interesting

The overnight/intraday return split is a real, peer-reviewed effect, not folklore. Cliff, Cooper & Gulen (2008) and Lou, Polk & Skouras (2019) document that a large share of the US equity risk premium has historically accrued overnight (close→open) rather than intraday (open→close). Plausible mechanisms: earnings and analyst actions releasing outside market hours, options-dealer hedging flow at the open, institutional order accumulation during the session executing at/after the close.

■ **\*\*UNVERIFIED\*\***: the two citations above are from training knowledge. Confirm authors, years and actual findings before citing them anywhere that matters.

More important than this one anomaly: Kairos has no systematic way to ask **"which known effects actually work in our universe?"** That question is answerable and currently unanswered.

## ## 2. The two objections that nearly kill it

These are the reasons this is parked rather than scheduled. Neither is resolved.

### ### 2.1 Survivorship bias

The motivating example (one semiconductor name showing an extraordinary cumulative overnight return) is very likely this bug. That company survived; same-sector peers that went to zero are absent from every free dataset. Backtesting today's index constituents over 20 years selects on two decades of survival and makes almost any rule look excellent.

**\*\*Consequence\*\*** the data requirement is not "historical prices." It is "historical prices including securities that died." That is a much shorter and mostly non-free list of sources.

### ### 2.2 Multiple comparisons

200 features × 4 horizons × 6 cross-sectional slices ≈ 4,800 tests. At p<0.05, roughly 240 will appear significant by chance alone. Without Benjamini-Hochberg (or equivalent) correction plus a strict time-based out-of-sample holdout, this system's primary output will be confident noise.

This is the main event, not a footnote. Any build plan that treats correction as a later refinement is wrong.

## ## 3. Why `price\_cache` cannot be the source

An initial version of this plan proposed building the feature library from the existing `price\_cache` table. That is wrong and was corrected in the same conversation.

`price\_cache` holds only symbols Kairos has actually researched, only since Kairos began running. Direct evidence from the 2026-09-05 exit-geometry counterfactual: joining closed `paper\_trades` to `price\_cache` matched only **\*\*55 of 113\*\*** non-tainted India trades and **\*\*40 of 52\*\*** US trades – it does not fully cover even the names we have already traded.

Discovery needs ~15-20 years, thousands of symbols, and multiple regimes (2008, 2020, 2022). That data must come from outside. This is the single largest open work item.

## ## 4. Data sources – ALL UNVERIFIED, verify before use

■ Compiled from training knowledge on 2026-09-05 with web search and firecrawl both unavailable. Treat every row as a hypothesis. Check licence terms, actual history depth, and delisted-symbol coverage directly before relying on any of them.

| Source                      | Claimed history | Delisted included? | Cost | Note                                                                           |
|-----------------------------|-----------------|--------------------|------|--------------------------------------------------------------------------------|
| --- --- --- ---             |                 |                    |      |                                                                                |
| NSE bhavcopy                | Full            | Believed yes       | Free | Daily archive files. Believed the cleanest free option                         |
| Stooq                       | ~20y            | Partial            | Free | Bulk CSV, global                                                               |
| Yahoo (yfinance)            | 20y+            | <b>**No**</b>      | Free | Survivorship-biased. Usable for features on live names; NOT for backtest truth |
| SEC EDGAR                   | Full            | Yes                | Free | Fundamentals/filings incl. bankrupt filers. No prices                          |
| Kaggle dumps                | Varies          | Partial            | Free | Quality varies; verify before trusting                                         |
| Sharadar (Nasdaq Data Link) | ~20y            | Yes                | Paid | Likely the technically correct answer                                          |
| Norgate                     | ~40y            | Yes                | Paid | Retail gold standard                                                           |

**\*\*Constraint conflict, unresolved\*\*** the survivorship-complete sources appear to be the paid ones, which collide with the standing \$0-cloud rule (see `[[feedback-free-cloud-only]]`). The two candidate resolutions – accept and **\*measure\*** US survivorship contamination, or treat India as the clean research market and US as features-only – are open questions for the owner (§8).

## ## 5. Storage sketch (if it ever proceeds)

~5,000 symbols × 20 years × 252 sessions ≈ \*\*25M rows\*\*. Supabase free tier is ~500MB (■■■ unverified current figure). Raw prices must not live in Supabase.

Cloudflare R2 (10GB free tier — unverified) ← raw OHLCV as Parquet, partitioned by year

↓

GitHub Actions (2000 min/mo free — unverified) ← DuckDB over Parquet: features + IC

↓

Supabase ← results ONLY: feature IC, pattern registry, hypotheses

Consistent with the free-cloud rule if the tier figures hold. Verify all three before committing.

## 6. The cross-sectional grid — the actual product

A pattern must survive every slice. This grid, not a list of patterns, is what would make this worth building.

| Slice                | Question                           | What it kills                                    |
|----------------------|------------------------------------|--------------------------------------------------|
| Per stock            | Holds for one name?                | Nothing — single-stock findings are always noise |
| Pooled cross-section | Holds for the median of thousands? | Stock-specific flukes                            |
| By sector            | Tech only, or utilities too?       | A sector factor disguised as a signal            |
| By geo               | US and India both?                 | US-microstructure artifacts                      |
| By regime            | 2008 + 2020 + 2022 + calm?         | "Long beta in a bull market"                     |
| By liquidity tier    | Large and small cap?               | Illiquidity premium in disguise                  |
| By era               | 2005-12 / 2013-20 / 2021-now?      | Decayed, already-arbitraged effects              |

Overnight drift is the natural first test case: if it appears in US large cap AND India AND across 2008/2020/2022 AND in both tech and utilities, it is structural. If it only appears in US semis post-2015, it is a story about one company, not a strategy.

## 7. Role of the LLM — deliberately narrow

The LLM is **not** the alpha source. Treating it as one is the fastest route to a confident wrong answer. Its legitimate jobs:

1. **Hypothesis generation** — propose feature combinations worth testing. Cheap, parallel, fine if most are junk.
2. **Mechanism check** — for a surviving pattern, argue why it *should* persist. A pattern with no economic mechanism is a coincidence with good p-values.
3. **Kill-condition authoring** — name what would break it (spreads widen, regime flips, crowding).

Statistics decide; the LLM proposes and narrates. If it is allowed to validate, this becomes an expensive confirmation-bias machine.

## 8. Open questions — must be answered before any build

1. **Survivorship**: accept measured contamination in a free US dataset, or make India the research market (clean, free) and treat US as features-only? Unresolved, owner decision.
2. Do the free tier figures in \$5 actually hold today (R2 10GB, GH Actions 2000 min, Supabase 500MB)?
3. Does NSE bhavcopy genuinely include delisted names for the full window, and what is its licence?
4. What is the minimum viable feature count that is worth the multiple-comparison cost? Fewer, better motivated features may beat 200 scattergun ones.
5. Does this compete for attention with the *already-identified* problems (exit geometry, US selection having no rank, the just-repaired learner)? Those are known defects with known impact; this is speculative upside. Sequencing matters and currently favours the known defects.
6. What is the realistic maintenance cost of a 25M-row pipeline on free infrastructure?

## 9. Honest expected outcome

Finding a genuinely novel, undiscovered anomaly is unlikely — this ground has been mined by better-resourced quants for forty years. The realistic value, in descending order:

1. **Which known anomalies actually work in Kairos's universe.** Momentum may be alive in the India names and dead in the US ones. Currently unknown; directly actionable.
2. **Regime conditioning** — e.g. "momentum works below VIX 20, inverts above 30." Upgrades existing dimensions rather than adding new ones.
3. **A kill list** — proof that a dimension currently being scored has zero IC. Given the Alpha Diagnostic already reported US selection "has no rank," this may be the highest-value output. Deleting a dead dimension is a real improvement.

That is a good outcome. It is not "find a secret nobody knows," and the feature should never be justified on that basis.

## 10. Proposed sequencing (only if approved later)

- **Phase 0** — prove the pipe. NSE bhavcopy, 2 years, → R2 Parquet, DuckDB, one question: does overnight drift exist cross-sectionally in India? Stop cheaply if the pipe or the answer fails.
- **Phase 1** — history + storage; measure and *document* US survivorship contamination.
- **Phase 2** — feature + IC engine, BH correction on by default.
- **Phase 3** — time-based OOS gate (pre-2022 discover / post-2022 validate). Expect most Phase 2

```

findings to die here.
- **Phase 4** – survivors get an LLM mechanism hypothesis and enter the **existing** shadow
 framework. No new promotion path – champion/challenger already exists.

11. Related

- `[[feedback-free-cloud-only]]` – the $0 constraint this collides with
- `features/shadow-population/FEATURE_ARCHITECTURE.md` – the graduation path any survivor would use
- `features/alpha-diagnostic-lab/FEATURE_ARCHITECTURE.md` – already diagnosed "US selection has no rank"
- `docs/arch/09-learning-loop.md` – champion/challenger machinery this would feed

Source: features\technical-factor-calibration\FEATURE_ARCHITECTURE.md
Technical Factor Calibration

> Status: **APPROVED FOR MEASURE-ONLY IMPLEMENTATION**
> Approved by owner: 2026-07-21
> Decision authority: `PROJECT_DECISIONS.md` Decision 51

1. Decision

Measure whether Kairos's existing technical score and a deliberately small set of
candidate technical factors predict forward returns in the existing Edge evidence
lab. Do not change production scoring from this feature.

The first trial family is frozen to:

1. `kairos_technical_score_v1`: the exact current `computeTechnical` plus
 `scoreTechnical` composite, including its breakdown veto;
2. `rel_strength_6m`: the existing stock-minus-market six-month return;
3. `macd_atr_12_26_9`: MACD(12,26,9) histogram divided by Wilder ATR(14);
4. `signed_adx_14`: ADX(14) strength signed by +DI versus -DI; and
5. the existing price/volume factors already in EdgeScout.

No Fibonacci, oscillator zoo, per-stock optimization, or automatic sector formula
is added. MACD and ADX are challengers, not new score weights.

2. Why

The current technical formula is a hand-tuned prior. Kairos needs a repeatable way
to determine whether the composite, its existing components, or a small challenger
set adds market-local forward-return information. The same path must be reusable
when parameters are reconsidered later and must retain losing trials to prevent
selection-history erasure.

3. Existing Substrate

Reuse the existing measure-only Edge lab:

- `lib/edges/registry.ts`: pure factor definitions over candles frozen at `asOf`;
- `lib/edges/ic.ts`: market-local forward-return rank IC with sampled dates and
 Newey-West standard errors;
- `edge_signals` and `edge_signal_inputs`: immutable prospective observations and
 input lineage;
- `edge_ic_history`: historical diagnostics; and
- `edge_market_status`: advisory market-level lifecycle only.

This feature creates no parallel truth layer and makes no provider call per added
factor. EdgeScout resolves a symbol's candle history once per run and evaluates every factor
over the same logical `asOf` slice.

Current PIT limit: the run stores a dataset fingerprint, but its candle inputs are
retrieved from current providers. Slicing them to `asOf` prevents direct future-bar
look-ahead; it does not prove that historical bars were not restated after `asOf`.
Therefore this evidence is retrospective/measure-only today and must not be labeled
PIT-safe or satisfy a future promotion gate until a Kairos-owned immutable daily
candle-vintage archive is available.

4. Recalibration And Memory

An edge ID is a versioned formula identity. Changing any period, threshold, anchor,
normalization, unavailable-data rule, or veto creates a new edge ID; it never edits
an old ID in place. Every run records market, horizon, window end, segment, history
depth, sampling step, universe size, evidence quality, and provider report.

`edge_ic_history` stores separate rows for:

- `segment_type = market`, `segment_value = all`; and
- `segment_type = sector`, `segment_value = <normalized sector>`.

Old market-wide rows are migrated to the explicit `market/all` identity. A rerun of
the same edge/market/window/horizon/segment against the exact same frozen dataset and
configuration is idempotent. Provider corrections or newly available symbols change
the dataset fingerprint and append a distinct snapshot. Each report records the

```



benchmark source and per-provider symbol counts so the change is explainable. A changed formula requires a new edge ID and therefore appends a distinct trial history.

The Vibe comparator is pinned separately to commit `a5eb30fd00d6ee71cd5099d15f57de5ae47010ff`. Its full package is not admitted: the audited package includes LLM, MCP, broker, web, dynamic-import, and network dependencies unnecessary for calibration. A later sandbox may compare a narrowly vendored MIT validation subset against the same frozen inputs. It cannot replace Kairos evidence or influence scoring.

#### ## 5. Sector Policy

Sector is an evaluation slice, not initially a separate strategy:

- use `symbol\_profiles.sector` when present;
- fall back only to Kairos's deterministic known-symbol map;
- treat missing, `Other`, diversified funds, fixed income, commodities, and digital assets as unclassified and exclude them from equity-sector conclusions;
- require at least five symbols in a sector on an as-of date before calculating its cross-sectional IC; and
- require the existing minimum of 36 independent sampled IC observations before a sector result can be anything other than `measure\_only`.

Sector-specific parameters require a separate future decision. They must improve out of sample across multiple windows and neighboring parameter values after all tested sectors/parameters are included in the trial family. A single strong sector backtest is not sufficient.

#### ## 6. Market Isolation

US and India are evaluated independently. They never share symbols, benchmarks, returns, sectors, observations, IC rows, status, currency, or promotion evidence. The same formula may be useful in one market and benched in the other.

#### ## 7. Safety Boundary

This feature may write only Edge analytics, run metadata, and System Health state. It may not read or write production score weights, `agent\_signals`, entry eligibility, mandates, positions, cash, proposals, broker orders, or exits. No LLM participates.

`shadow\_eligible` remains advisory. No result changes production behavior without a separate architecture, locked holdout, trial-family correction, shadow observation, paper validation, and owner approval.

#### ## 8. Acceptance Criteria

- Existing production `scoreTechnicals()` is represented exactly as a versioned edge.
- MACD is normalized by ATR and returns null on insufficient/invalid data.
- ADX uses Wilder smoothing, is direction-signed, and returns null on insufficient data.
- Market-wide and sector rows cannot collide in persistence.
- Sector rows never change `edge\_market\_status`; only market/all evidence can do so.
- Unknown/thin sectors abstain rather than report zero or fabricate confidence.
- Added factors trigger no additional provider request within a run.
- Formula, as-of slicing, market isolation, segment isolation, and thin-sample tests pass.
- Every persisted report names the benchmark source and per-provider symbol counts.
- Current-provider historical-candle evidence is labeled retrospective, not PIT-safe.
- TypeScript, Vitest, production build, migration verification, and security checks pass.

#### ## 9. Build Order

1. Version and add the three bounded factor definitions.
2. Extend IC evaluation with optional sector maps and explicit segments.
3. Migrate historical IC rows to `market/all` and update the unique key.
4. Load display/profile sector metadata at the measure-only route boundary.
5. Run US and India diagnostics; allow the existing evidence floor to abstain.
6. After enough evidence, compare parameter plateaus and decide whether any native score change deserves a separate proposal.

#### ## 10. Explicit Non-Goals

- No score/weight/threshold change.
- No Vibe app, agent, MCP, provider, or broker installation.
- No per-sector production parameters.
- No automatic selection of the best in-sample trial.
- No claim that retrospective current-universe history is point-in-time or free of survivorship bias.

#### ## 11. Data-Truth Finding (2026-07-31)

Production distributions show the live technical score is heavily saturated: exactly 100 on 19.3% of US observations and 40.2% of India observations over 45 days. Exactly 20 occurred on 19.6% US and 13.6% India. The standalone bottom-quartile weak-close veto was removed after matured cohorts failed to support it as a hard breakdown classifier; ATR-scaled and high-volume breakdown conditions remain.

The 100 ceiling is not changed here. Add it to the versioned measure-only comparison

and require market-local out-of-sample evidence before proposing a new live formula.  
Full production audit: `features/scoring-data-truth/FEATURE\_ARCHITECTURE.md`.

---

# Source: features\theme-tracking\FEATURE\_ARCHITECTURE.md  
# Theme Tracking — Feature Architecture

Status: **\*\*Step 1 approved and shipped 2026-08-04. Steps 2-5 remain proposals.\*\***  
Author: Claude · 2026-08-04  
Related: `app/api/agents/theme-scout/route.ts`, `features/edge-factor-discovery/`,  
`features/india-scorer-discrimination/DIAGNOSIS.md` §14, `R3\_DIMENSION\_FEASIBILITY.md`.

---

### ## 1. Correcting the premise

The question was whether the app can tell which themes and sub-themes are rising or declining. My first answer was that themes are not tracked at all. **\*\*That was wrong\*\*** — `watchlist.theme` exists and is written on every Theme Scout row:

```
rowsBySymbol.set(clean, { source: "llm_theme", theme: t.theme, ... })
```

182 rows across 13 scout runs, **\*\*zero nulls\*\***. The data is there.

The real defect is narrower and more interesting: **\*\*the theme has no stable identity\*\***, so it cannot be counted across runs.

---

### ## 2. The measured defect — identity drift

182 rows produce **\*\*42 distinct theme strings\*\*** over 13 runs, and **\*\*32 of those 42 appear in exactly one run\*\***. Naive normalisation (lowercase, strip spaces and hyphens) collapses 42 → 40, so the drift is semantic, not formatting.

Cybersecurity is the clearest case — **\*\*six strings, one theme\*\***:

| string                    | rows | runs | first → last  |
|---------------------------|------|------|---------------|
| --- ---: ---: ---         |      |      |               |
| `Cybersecurity`           | 18   | 5    | 06-30 → 07-06 |
| `Cyber Security`          | 16   | 5    | 06-30 → 07-06 |
| `Cybersecurity Solutions` | 4    | 2    | 07-02 → 07-06 |
| `Cyber Security Boom`     | 4    | 2    | 07-01 → 07-02 |
| `Cybersecurity Demand`    | 4    | 1    | 06-30         |
| `Cybersecurity Threats`   | 2    | 1    | 06-30         |

48 rows over five-plus weeks, which should read as one persistent theme, and instead reads as six — four of them apparently one-off noise. Same pattern in `Cloud Computing` / `Cloud Computing Expansion`, `Digital Payments` / `Digital Payments Growth`, `Renewable Energy` / `Clean Energy`.

The cause is structural: the theme name is free text from an LLM prompt, minted fresh each run with no vocabulary and no memory of prior runs.

**\*\*Consequence:\*\*** no rise/decline signal is computable today. "Cybersecurity has appeared in 5 of the last 13 runs and is strengthening" is a true statement the data cannot currently express.

### ### A second finding, surfaced by the same query

The three most recent runs (07-27, 08-03) produced `Autonomous Tech`, `Debt Reduction`, `Consumer Expansion`, `Grid Safety`, `Defensive Consumer` — each appearing exactly once, and several of which are not investable themes but generic observations. The recurring themes (`Cloud Computing`, `Cybersecurity`, `Renewable Energy`) all stop at 07-15.

Either the prompt or the news input has drifted toward vaguer output. Worth diagnosing separately; normalising identity will not fix a theme that is genuinely one-off noise, it will only make the noise visible.

---

### ## 3. Scope

**\*\*In scope:\*\*** give a theme a stable identity, record its per-run observations, and display strength on the Markets page.

**\*\*Explicitly out of scope:** a theme or sector momentum SCORING dimension.

This is not caution, it is the rule established two days ago and it applies with full force. `R3\_DIMENSION\_FEASIBILITY.md` rejected NSE FII/DII precisely because its per-date cross-sectional  $\sigma$  is 0.00 by construction — one national number, no symbol dimension. **\*\*Theme and sector momentum have the same shape\*\***: every member of a theme receives the same value on a given day, so within-theme  $\sigma$  is zero. It

would shift the composite level without improving ordering – the exact failure already documented for US `macro\_score` ( $\sigma$  2.26) and `insider\_score` (94.1% at a single value), which together consume 0.225 of US weight and contribute no cross-sectional information.

It would also be a threshold change: DIAGNOSIS.md §14e measured \*\*11.6% of US rows within  $\pm 2$  points of the gate\*\*, so any composite level shift moves admission.

Theme membership may legitimately inform \*\*discovery admission\*\* – which names enter the funnel. That is not scoring and does not carry this objection.

---

## ## 4. Proposal

### ### 4.1 Stable theme identity

Add `theme\_slug` alongside the existing free-text `theme`. Resolution order:

1. \*\*Controlled vocabulary first.\*\* A small curated list of durable themes (``cybersecurity``, ``cloud-computing``, ``renewable-energy``, ``ev``, ``digital-payments``, ``ai-infrastructure``, ...) with alias sets. Deterministic, auditable, and it collapses the six cybersecurity strings on day one.
2. \*\*Unmatched → `theme\_slug` = null\*\* and the raw string retained.

\*\*Do not use an LLM to normalise.\*\* The drift is \*caused\* by free-text LLM output; adding a second LLM to reconcile it makes the vocabulary non-deterministic and unauditably. A theme that does not match the vocabulary is recorded as unmatched, and the vocabulary is extended by an owner-reviewed edit when a genuinely new theme recurs. Unmatched rate is itself the metric that tells you when to extend it.

### ### 4.2 Per-run observation ledger

`theme\_observations`: one row per (run, theme\_slug) – run timestamp, market, member symbols, member count, whether matched or unmatched. Append-only.

`watchlist` rows expire after 7 days, so today the history is only recoverable by accident. A theme's recurrence pattern is the entire signal; it needs its own durable record.

### ### 4.3 Markets page – display only

Once ~8 weeks of observations exist: recurrence (runs present / runs total), first and last seen, member count trend, and aggregate member forward return.

Labelled \*\*observational\*\*, with the run count shown next to every claim. A theme seen twice is not a trend and the surface must not imply it is.

---

## ## 5. What this unlocks

Theme Scout has been running since 2026-06-30 and \*\*its output has never been measured\*\*. It uses an LLM to pick both the themes and the member stocks, and those members enter research at candidate priority 3. That is admission-only, so it clears the no-LLM-on-money-path rule – but it means an unmeasured LLM judgement has been shaping the research funnel for five weeks.

With `theme\_slug` plus the observation ledger, the answerable question becomes: \*\*do theme-sourced candidates outperform watchlist-sourced ones?\*\* That is a discovery-quality measurement, not a scoring change, and it is the first thing that would justify or retire this agent.

It also depends on the provenance fix shipped in `73c398bf` – before that, carried-forward candidates were relabelled `watchlist` and theme attribution was destroyed on the round trip.

---

## ## 6. Sequencing

1. `theme\_slug` + controlled vocabulary + `theme\_observations` – no UI, no scoring. Backfill the 182 existing rows through the vocabulary so the five weeks already collected are not lost.
2. Report unmatched rate and the recurrence table. Diagnose the \$2 drift toward one-off themes before building any surface on top of it.
3. Markets display, only once  $\geq 8$  weeks of observations exist.
4. Theme-vs-watchlist discovery-quality comparison.
5. Theme membership as a discovery-admission input – separate proposal.

Step 1 is small and reversible. Steps 3-5 each need their own approval.

---

## ## 7. Open questions for the owner

1. **\*\*Approve step 1?\*\*** It is additive and touches no scoring path.
2. **\*\*Sub-themes\*\*** – the original question asked about them. Nothing in the data supports a hierarchy yet: with 42 strings over 13 runs and 32 appearing once, there is no evidence of parent/child structure. Recommend deferring until the flat vocabulary has stabilised.
3. **\*\*Is Theme Scout worth keeping at all?\*\*** Five weeks, 182 rows, zero measurement, and recent output drifting toward non-themes. Step 1 makes that answerable rather than a matter of taste. Retiring it is a legitimate outcome.

---

## 8. Step 1 – shipped 2026-08-04

`lib/themes/vocabulary.ts` (10 slugs, aliases seeded from the 42 strings actually observed), `watchlist.theme\_slug`, and the append-only `theme\_observations` ledger. Theme Scout now resolves a slug on write and appends one observation row per (run, theme), reporting `unmatched\_themes` in its response.

**\*\*Backfill result – 32 of 42 distinct themes matched.\*\*** The 10 unmatched are exactly the strings flagged in \$2 as one-off observations rather than investable themes: `Grid Safety`, `Tech Sector Rebound`, `Consumer Expansion`, `Debt Reduction`, `Defensive Consumer`, `Stable Dividend Payers`, `Energy Merger Boom`, `Global Diversification`, `Cash Flow Recovery`, `Industrial Expansion`. They are recorded with a null slug, not guessed into one.

Recurrence is now computable for the first time – **\*\*counted in distinct run dates\*\***, not observation rows, because several raw strings collapse to one slug on the same day:

```
slug	dates present	% of 13 runs	first → last
`cloud-computing`	8	62%	06-30 → 07-15
`cybersecurity`	5	38%	06-30 → 07-06
`electric-vehicles`	5	38%	06-30 → 07-06
`financials`	4	31%	07-06 → 07-27
`renewable-energy`	4	31%	06-30 → 07-06
```

Cybersecurity's six fragments are one theme present in 38% of runs. That statement was not expressible yesterday.

**\*\*It also confirms the \$2 concern.\*\*** Every durable theme's `last\_seen` is 2026-07-15 or earlier; nothing since has recurred. Whatever changed around mid-July is a live question, and step 2 is now to diagnose it rather than build a surface over it.

The backfill was generated *from* the TypeScript resolver rather than reimplemented in SQL, so the vocabulary keeps exactly one implementation.

---

## 9. Step 2 – diagnosis: there was no drift (2026-08-05)

Step 2 was to diagnose why theme output degraded after mid-July. **\*\*It did not.\*\*** \$8's claim that "every durable theme's last\_seen is 2026-07-15 or earlier" was an artifact of the backfill, and is retracted.

### 9a. Why the backfill was wrong

`run\_date` was keyed on `watchlist.created\_at`. Theme Scout **\*\*upserts on (user\_id, symbol)\*\***, so a symbol first added on 06-30 keeps `created\_at = 06-30` forever while later runs overwrite its `theme`. Weeks of themes were attributed to the day a symbol was first seen – which is why 06-30 appeared to carry 15 themes when `MAX\_THEMES = 3` makes that impossible in one run.

`updated\_at` is no better: **\*\*162 of 182 rows are stamped 2026-07-17, a Friday\*\*** – a bulk touch from the 07-16 watchlist market-casing migration, not a run.

**\*\*Neither column identifies the run that wrote a theme\*\***, so prior theme history is not reconstructible from this table at all. The 68 backfilled `theme\_observations` rows were deleted.

### 9b. What actually changed

Cadence. `kairos-theme-scout` runs `0 0 \* \* 1` – **\*\*Mondays, weekly\*\***. The genuine run dates are all Mondays (06-29, 07-06, 07-13, 07-20, 07-27, 08-03); migration 052 had unscheduled the earlier `theme-scout-daily`. With `MAX\_THEMES = 3` × `MAX\_THEME\_STOCKS = 2`, a run caps at **\*\*6 members\*\*** – and the three most recent Mondays produced 2, 6 and 5. Exactly at cap, not degraded.

### 9c. The root cause

**\*\*Theme Scout wrote no `agent\_runs` row.\*\*** Five weeks of weekly runs with no run record, no duration, no error trail – its only trace was watchlist rows whose timestamps are contaminated by upsert and by unrelated migrations. That is why the question was unanswerable, and why an attempt to answer it from those

timestamps produced 68 wrong rows.

Fixed: the scout now opens an `agent\_runs` row and closes it on **all five** terminal paths (no market data, LLM parse failure, no themes, no owner profile, success), reporting the unmatched-vocabulary count in the summary. A row left `running` is indistinguishable from a hang, which is the same class of silence.

**The ledger's real history therefore starts with the next Monday run**, not with the five weeks already collected. Those are gone.

---

# Source: features\time-review-exit\FEATURE\_ARCHITECTURE.md  
# Time-Review Exit Policy

Status: APPROVED FOR P0 MEASUREMENT ONLY on 2026-08-07

## ## Decision

Kairos must not close an otherwise healthy profitable position solely because it has crossed its configured holding horizon. The current horizon remains a hard paper/live exit until a separately governed candidate proves better. The first change is therefore a forward-only, append-only time-review shadow, not an exit change.

## ## Problem

`PositionMonitor` currently closes a paper position when its age is greater than its resolved horizon. A common ten-session plan therefore closes on session eleven even when the position is profitable, has fresh research support, and is still trending. That makes the horizon a liquidation timer rather than a review point. Existing exit shadows compare static stops, targets, trails, and clocks; they cannot reconstruct whether a specific position was healthy at its horizon.

## ## Product Rule

The eventual candidate policy is a deterministic time review:

1. Mechanical protection always wins. A protective stop, confirmed thesis/score exit, target handling, or mandatory risk restriction may close first.
2. At the configured horizon, a profitable alpha position is reviewed, not automatically extended.
3. It can be retained only when every required review input is fresh and the predeclared health rule passes. A one-session dip, a stale score, or missing evidence does not manufacture a positive result.
4. Partial-profit logic remains independent: a target can take a bounded partial and move the remaining stop to breakeven. It is not an excuse to keep adding risk or to remove protection.
5. A fixed maximum extension remains a safety boundary. This design never turns a swing mandate into an indefinite investment.

No LLM can select an extension, exit, threshold, or position quantity. LLMs may only explain a completed deterministic observation outside the money path.

## ## Why A New Forward Ledger Is Required

`decision\_observations` is immutable entry-time research evidence. Price candles can replay a later return, but cannot recover the score freshness, direction, drawdown, earnings state, or eligible replacement set known on the real review date. Recomputing those facts today would introduce look-ahead bias. Historical price-only path simulations remain useful for static geometry, but are not proof for this policy.

P0 must instead write one immutable observation at each real horizon review. Future close outcomes are attached only after their market sessions mature.

## ## P0: Collection Only

### ### Trigger

The existing per-market `PositionMonitor` run observes each open paper alpha position at exactly its resolved horizon, before the incumbent `age > horizon` time-stop can fire. Hedge positions are excluded. The observer must be best-effort and may never delay, suppress, or alter an exit.

### ### Immutable Review Record

Each review record contains:

- position, symbol, market, native currency, entry timestamp and review session;
- entry price, review price, unrealized return, high-water mark and drawdown from that high;
- frozen resolved horizon and candidate extension horizon;
- freshest holding research score, direction, timestamp, score-age status, and

- exit/hold thresholds in force;
- deterministic review classification and every missing/failed input;
- entry mandate/strategy version provenance and an idempotency key.

The record is append-only, market-local, owner-readable, and written by the service role only. US/USD and India/INR records are never combined.

### ### Initial Candidate Family

The trial family is deliberately small and predeclared:

- baseline: incumbent exit on the next session after the configured horizon;
- candidate A: extend five market sessions only when profitable, score is fresh, score is at or above the hold threshold, direction remains long, and drawdown from the stored high-water mark is no greater than one initial stop distance;
- candidate B: the same rule with ten market sessions.

Unknown score, unavailable direction, invalid price, or stale evidence is a recorded `not\_eligible`, never a synthetic healthy state. The candidate never widens a stop, raises a position size, reopens a name, or overrides an existing score/stop/target exit.

### ## P1: Outcome Labels And Evaluation

After five or ten completed sessions, a labeler records native-currency return, benchmark return over exactly the same dates, excess return, maximum favourable and adverse excursion, and whether the candidate would have hit a mechanical stop. It also records whether an already-qualified replacement candidate existed at the review session. Replacement is an attribution field in P1; it is not reconstructed from future signals and does not authorize rotation.

P1 compares each candidate with the incumbent by market only. It reports sample size and distinct review sessions, costs, drawdown, turnover, raw and benchmark-relative returns, and false retention cases. It does not use a per-trade excess average to approve a rule with a different holding period; any activation needs an execution-faithful, market-local portfolio simulation with redeployment.

### ## Activation Gates

No P0/P1 record changes an exit. A paper-only candidate requires all of:

1. at least twenty distinct review sessions per market and a predeclared trial correction across the two extensions;
2. no deterioration in cost-adjusted portfolio return, benchmark-relative return, maximum drawdown, or turnover against the incumbent in sealed replay;
3. a forward shadow whose point estimate and adverse-case review are consistent with the sealed replay;
4. an approved architecture update, a versioned mandate flag defaulting off, and an owner promotion for that market.

Live behavior is a separate approval after paper evidence. It retains all broker, kill-switch, mandate, and protective-order gates.

### ## Explicit Non-Goals

- no immediate modification to the `PositionMonitor` time stop;
- no LLM exit authority;
- no automatic adaptive extension duration;
- no cross-market/currency portfolio comparison;
- no using a current score to backfill a historic review;
- no capital-rotation decision in this feature.

### ## Acceptance Criteria

1. The P0 observer cannot change a paper position, paper trade, cash balance, order event, exit reason, score, mandate, or broker proposal.
2. A malformed or failed P0 write is visible in diagnostics but does not interrupt the incumbent monitor loop.
3. A review observation is idempotent for one position and review session.
4. Candidate health cannot be true with a stale/missing score or invalid price.
5. Tests prove the observer cannot execute an exit and that US/India facts cannot be combined.
6. Upgrade Path exposes collection progress and explicitly says it has no current influence.

### ## Build Order

1. Add the append-only review ledger and owner-only RLS.
2. Add a pure classifier and tests for the predeclared candidate family.
3. Add a best-effort P0 observer to PositionMonitor with no behavior change.
4. Add outcome maturation and Upgrade Path readiness reporting.
5. Build the execution-faithful portfolio simulator only after enough review observations exist; do not activate from static candle evidence.

---

```

Source: features\time-review-exit\IMPLEMENTATION_RESULT.md
Time-Review Exit Shadow – Implementation Result

Implemented: 2026-09-03

Outcome

The approved measure-only time-review pipeline is complete. PositionMonitor now captures one immutable observation for an alpha position at its exact resolved horizon, before the incumbent time stop. The existing horizon-shadow schedule matures matched +5/+10-session outcomes. Neither ledger has a money-path consumer, and the unconditional time stop remains active.

What shipped

- `time-review-v1` deterministic classifier with fail-closed score, direction, profitability, high-water drawdown, initial-stop-distance, active-stop, and target vetoes.
- Exact position/session idempotency and immutable market-local observation and outcome ledgers.
- Matched incumbent-next-session versus +5/+10-session outcomes with benchmark return, MFE/MAE, retained-stop result, and replacement-candidate attribution.
- Upgrade Path counts only exact review sessions with both outcomes matured. Legacy daily one-day-extension rows remain visible context but cannot advance readiness.

Production proof

- Migration `time_review_exit_shadow` applied to FinanceOS Supabase project ``.
- Both tables have RLS, owner-only SELECT policies, narrow grants, and UPDATE / DELETE rejection triggers.
- Rolled-back transaction proved observation insertion, duplicate-key refusal, outcome insertion, and both append-only guards; zero synthetic rows remained.
- Supabase security advisors reported no finding for either new table. The only table-specific performance notices were expected unused-index INFO notices on the still-empty observation ledger.

Verification

- Mutation test: inverting the exact-horizon predicate failed three detectors.
- Focused: 26 tests passed.
- Full suite: 2,449 passed, 7 skipped.
- `tsc --noEmit`: passed.
- Isolated Next.js production build: passed.

Still gated

No time-stop behavior changed. Twenty market sessions with exact reviews and both outcomes only permits the next review. A sealed market-local portfolio simulation with redeployment, costs, drawdown, turnover, multiple-trial control, adverse-case review, and explicit owner approval remains mandatory before a paper-policy change. Live use requires a separate approval.

Source: features\trade-behavior-mirror\FEATURE_ARCHITECTURE.md
Trade Behavior Mirror – Past-Trade Analysis – Feature Architecture

> **STATUS: DRAFT – NOT APPROVED.** Proposal for review. No code until explicit sign-off.
> Owner decision pending. Created 2026-07-10.

1. Intent (owner's ask, restated)

Import all past Robinhood transactions. For each historical buy/sell, reconstruct the **fundamental + technical values *at the time* of the trade**, compare to those same stocks **now**, and have the Mentor characterize the owner's **behavioral / mental pattern** – do you buy strength or dips, sell winners too early, average down on losers, panic in drawdowns – grounded in the actual indicator values at each entry/exit, not vibes.

2. What already exists (partial build)

- **Import**: Robinhood CSV → `trade_decisions` (symbol, action, qty, exec_price, exec_date). `app/api/live-portfolio/import-csv/route.ts`.
- **Enrich**: per trade → macro regime at trade date, **forward** price 1d/1w/1m/3m, `outcome_score` (was the buy/sell right), `pattern_tags` (win/loss/breakeven + regime). `app/api/live-portfolio/enrich/route.ts`.
- **Mentor** reads `trade_decisions` (thesis/evaluate/ask) – comments on outcomes.
- Trades embedded into RAG (`trade_memories`, migrations 043/045).

3. The gap (the core of the ask – NOT built)

- ■ **Point-in-time fundamental + technical values at each trade.** Enrich pulls forward *price* only – never "RSI/P-E/ROE/sentiment when you bought." So it cannot say "you bought when RSI was 82 and P/E was 60."

```

- ■ **\*\*Then-vs-now comparison\*\*** per stock.
- ■ **\*\*Behavioral fingerprint synthesis\*\*** – the mental-pattern mirror itself.

## 4. Why it's non-trivial

- **\*\*Technical PIT\*\*** (RSI/EMA/RS/52w-distance at exec\_date): CHEAP – computable from the daily series enrich already fetches (`TIME\_SERIES\_DAILY\_ADJUSTED`). No new data cost.
- **\*\*Fundamental PIT\*\*** (P/E in 2021): NOT free – AV OVERVIEW is a *\*current\** snapshot only. Must reconstruct: historical price × historical EPS from **\*\*FinancialDatasets** historical income statements\*\* (MCP available). Higher cost, per-symbol historical fetch.

## 5. Proposed design – 3 layers

**\*\*Layer A – Technical PIT fingerprint (cheap, high value).\*\***  
 For each `trade\_decisions` row, compute at `exec\_date` from the daily series: RSI(14), EMA20/50 position, 20d trend, relative strength vs SPY/^NSEI, 52w-high distance, volume state. Store on the row (new columns, additive migration). Also snapshot each symbol's values **\*\*now\*\*** → then-vs-now delta. Reuses `computeTechnicals` (`lib/data/technicalts`) fed a PIT-sliced series.

**\*\*Layer B – Fundamental PIT.\*\***  
 For each trade, fetch historical income statement / EPS at/around `exec\_date` from **FinancialDatasets**; derive P/E (exec\_price × shares / historical earnings), margin, growth. Store PIT + now. Day-cached + budget-guarded. Degrades gracefully when history is missing (Layer A still stands alone).

**\*\*Layer C – Mentor behavioral fingerprint.\*\***  
 A deterministic pass clusters entries/exits by their PIT signatures and computes behavioral metrics: entry-RSI distribution (chase vs dip), win-hold vs loss-hold duration (cut winners early?), add-to-loser rate (averaging down), drawdown-exit timing (panic sells), regime performance. The Mentor LLM then *\*narrates\** these deterministic findings (never invents the numbers) → `mentor\_insights`. This is the mirror.

## 6. Data model (additive, draft)

`trade\_decisions` new columns (nullable, backfilled by re-enrich):  
 `rsi\_at\_trade, ema20\_pos\_at\_trade, ema50\_pos\_at\_trade, rs\_vs\_bench\_at\_trade, dist\_52w\_high\_at\_trade, pe\_at\_trade, roe\_at\_trade, rev\_growth\_at\_trade, tech\_now\_json, fund\_now\_json, fingerprint\_json`.

New: `behavior\_fingerprint` (one row per re-analysis run: entry-style, exit-discipline, averaging, regime-skew metrics + Mentor narrative).

## 7. Guardrails

- Owner-only (all `live-portfolio/\*` already `requireOwner`).
- Read/analysis only – never places or alters live/paper orders.
- PIT reconstruction must not leak future data into the "at-trade" snapshot.
- Mentor narrates deterministic metrics; it does not fabricate indicator values.
- **FinancialDatasets** fetches go through the day-cache + budget guard.

## 8. Phasing

- **\*\*P0\*\*** – Layer A (technical PIT + then-vs-now). Additive migration; re-enrich backfills.
- **\*\*P1\*\*** – Layer C on Layer-A signatures (behavioral fingerprint + Mentor narration).
- **\*\*P2\*\*** – Layer B (fundamental PIT) once A+C prove useful.

## 9. Open questions for review

1. Start with Layer A only (cheap, fast) or commit to A+B+C up front?
2. Benchmark for at-trade relative strength: SPY only, or SPY + sector?
3. How far back does the Robinhood history go (bounds the **FinancialDatasets** historical cost)?
4. Behavioral metrics to prioritize – which patterns matter most to you to see first?

---

# Source: features\trading-mandate\FEATURE\_ARCHITECTURE.md  
 # Feature Architecture: Cross-Market Trading Mandates

## Status

Architecture status: Approved by owner  
 Approved date: 2026-07-12  
 Roles: Codex Architect + Builder  
 Implementation allowed: Yes  
 Implementation status: Complete and migration 168 applied to FinanceOS production

## Purpose

Give the user an honest, enforceable description of how agents trade without pretending that a few exit percentages constitute a different investment strategy. A mandate is configured independently for US and India and separates:

- investment horizon: `short\_swing`, `swing`, or `position`;
- strategy preference: `adaptive`, `momentum`, `balanced`, or `value\_quality`;
- horizon governance: `user` or `agent`;
- existing-position policy: `grandfather` or `apply`.

Day trading and genuine long-term investing are out of scope. The approved



platform remains long-only and uses once-daily decisions over 2-20 market days.

### ## Product Contract

|  | Horizon                                | Market-day range | Default | Entry | Stop | Target |
|--|----------------------------------------|------------------|---------|-------|------|--------|
|  | ---   ---:   ---:   ---:   ---:   ---: |                  |         |       |      |        |
|  | Short swing                            | 2-5              | 5       | 65    | 5%   | 12%    |
|  | Swing                                  | 5-15             | 10      | 60    | 7%   | 20%    |
|  | Position                               | 10-20            | 20      | 58    | 10%  | 30%    |

Risk profile remains orthogonal: it controls sizing, concentration, and kill switches. A mandate controls horizon, evidence emphasis, and exit geometry.

Strategy preferences use bounded deterministic multipliers before weight renormalization. They do not invent evidence and cannot make an unavailable dimension available:

- adaptive: champion/default weights unchanged;
- momentum: technical 1.25, sentiment 1.10, fundamental 0.85;
- balanced: all 1.00;
- value\_quality: fundamental 1.30, insider 1.10, technical 0.85, sentiment 0.85.

The exact effective weights, mandate version, and governance source are stored with every decision observation and signal.

### ## Authority And Precedence

1. Hard safety, liquidity, account, currency, and kill-switch controls always win.
2. The user mandate selects horizon and strategy preference per market.
3. With `horizon\_governance=user` (default), the mandate horizon overrides a champion genome's horizon. The champion still controls its other validated fields.
4. With `horizon\_governance=agent`, a valid promoted champion may select horizon within the mandate's min/max range; out-of-range values are clamped.
5. Changing a mandate never promotes or mutates a champion.
6. LearnerAgent may propose challengers but cannot change the mandate.

### ## Existing Positions

Default `grandfather`: each new paper fill stores a mandate snapshot and resolved horizon. PositionMonitor uses the stored horizon for that position, so changing Settings does not cause surprise exits. `apply` deliberately applies the current mandate to open positions on the next monitor run and is visibly labeled.

### ## Cross-Agent Flow

- Settings: owner-only read/write for US and India mandates.
- ResearchAgent: resolve mandate by symbol market; tilt only applicable+available dimension weights; use the mandate entry threshold; persist provenance.
- PaperTrader: require signal/market match; resolve the same mandate; use mandate stop, target, and horizon; persist snapshot on position/trade/order event.
- PositionMonitor: count market weekdays, not calendar days; use the fill snapshot under grandfather policy; otherwise resolve current mandate/champion precedence.
- TraderAgent/live proposals: include mandate snapshot and horizon in proposal rationale/audit. Existing approval, account, LTCM, and notional gates remain.
- LearnerAgent: receives mandate as immutable context. Challengers outside user horizon bounds are ineligible; it cannot write mandate rows.
- Backtest/validation: run and report results under the same resolved mandate, market calendar, factor tilts, threshold, stop, target, and horizon.
- Explainer/journal: show selected mandate, effective weights, resolved horizon, governance source, and whether an existing position was grandfathered.

### ## Data Contract

New `trading\_mandates` table, one row per market:

- `market` primary key (`us|india`)
- `horizon\_style`, `strategy\_preference`, `horizon\_governance`
- `min\_hold\_days`, `target\_hold\_days`, `max\_hold\_days`
- `score\_threshold`, `stop\_loss\_pct`, `target\_pct`
- `existing\_positions\_policy`
- `version`, timestamps, `updated\_by`

New nullable provenance columns on paper positions and trades:

- `mandate\_version`
- `mandate\_snapshot jsonb`
- `resolved\_horizon\_days`

All writes are owner/service-only. Reads fail closed for trade-affecting paths. Pre-migration code falls back to the behavior-preserving Position/Balanced mandate and reports the source as `default`.

### ## Market Rules

- US and India mandates are independent; no currency or configuration blending.

- Holding age uses each market's weekdays. Exchange-holiday support remains a calendar-service follow-up; weekends are never counted.
- US and India can use different strategies because evidence coverage differs. Missing dimensions are still excluded before renormalization.

#### ## Non-Goals

- Intraday/day trading, leverage, shorting, options, or crypto.
- A cosmetic `long\_term` label for a 20-day strategy.
- Automatic live enablement or order submission.
- Agent mutation of user mandates.
- Retroactive rewriting of historical signals, fills, or outcomes.

#### ## Acceptance Gates

- Per-market settings round-trip independently.
- Research effective weights and thresholds match the selected mandate.
- Unavailable evidence remains unavailable after tilting.
- Paper fills persist a mandate snapshot and resolved horizon.
- Grandfathered positions retain their entry mandate after Settings changes.
- User governance overrides champion horizon; agent governance clamps it.
- US and India tests prove no mandate/currency leakage.
- No live order path is enabled or invoked.
- Tests, typecheck, production build, and dependency audit pass.

#### ## Implementation Result

Implemented 2026-07-12 across Settings, ResearchAgent, PaperTrader, PositionMonitor, LearnerAgent, and the US proposal-building TraderAgent. Migration 168 was applied directly as one reviewed SQL file to the verified linked FinanceOS project (`<production-project-ref>`) because its timestamped migration ledger does not align with the repository's numbered local ledger and a broad CLI push could replay unrelated migrations. The table, US/India seeds, provenance columns, and owner-read RLS policy were verified from the production schema after application.

#### ## Paper Entry Cadence Correction (2026-07-21)

- Research produces signals only; it never invokes PaperTrader inline.
- Exactly one pg\_cron job per market attempts paper entries each regular session.
- PaperTrader independently verifies the market-local session and holiday calendar, so UTC/DST drift or a manual invocation cannot create an off-session fill.
- US runs at 15:15 UTC (inside both EDT and EST sessions); India runs at 04:10 UTC (09:40 IST). Both retain same-session signal freshness and all existing gates.
- A rejected fill RPC is written to `pipeline\_stage\_events` and summarized by reason in `agent\_runs.workload\_metrics`; observability never changes eligibility.
- The per-market 10-name default remains a concentration gate, not a cash-use target. Idle cash does not authorize an eleventh name or any add to an open alpha name. A later fresh score reassesses the holding; it is not a second entry instruction without separately validated add-to-winner architecture.

---

# Source: features\trading-mandate\IMPLEMENTATION\_RESULT.md  
# Trading Mandates Implementation Result

Status: Complete  
Completed: 2026-07-12  
Commit: `5616338` (`feat: add cross-market trading mandates`)  
Production migration: `168\_trading\_mandates.sql` applied and verified on FinanceOS Supabase project `<production-project-ref>`.

#### ## Delivered

- Independent US and India Trading Mandates.
- Separate horizon, strategy preference, horizon governance, and existing-position policy.
- Owner-only Settings API and market-aware Settings panel.
- ResearchAgent applies bounded evidence tilts and persists mandate provenance.
- PaperTrader writes entry-time mandate snapshots and resolved horizons.
- PositionMonitor honors snapshots and counts weekdays rather than calendar days.
- LearnerAgent receives the mandate as immutable context.
- US TraderAgent applies the US mandate threshold and records it in proposal risk context.
- Production database verified: mandate table, US/India seed rows, paper trade/position provenance columns, and owner-read RLS policy.

#### ## Verification

- 311 tests passed; 6 skipped.
- TypeScript passed.
- Production build passed.
- Dependency audit found zero vulnerabilities.
- No broker orders, previews, live enablement, or deployment mutation was performed.

#### ## Follow-Ups (Not Blocking Completion)

- Verify the normal Vercel Git integration deploys commit `5616338`.
- Add exchange-holiday calendars; current holding-age logic excludes weekends only.
- Show mandate provenance more broadly in portfolio and proposal detail views.

- Use mandate defaults in manual backtest controls.

These follow-ups do not block mandate configuration or the core US/India agent behavior.

## ## 2026-07-16 Safety Addendum

- Added owner-configurable `max\_open\_positions` per market (default 10). It is an entry-only gate: lowering it below the current book never l
- Added `max\_signal\_age\_sessions` per market (default 2). PositionMonitor ignores stale score/direction evidence but continues stop, target, l
- The fill RPC reads the canonical mandate cap and treats the application parameter as tighten-only defense in depth.
- Research now unions current paper alpha positions with only the latest live snapshot per account. Held symbols are marked `isHeld`, research
- Migration `20260716013000\_mandate\_capacity\_and\_score\_freshness.sql` was applied to `<production-project-ref>`. Production remained at 11 US
- Correlation-aware P0/P1 was not activated because candidate-to-book pair observations are not persisted today. The corrected prerequisite a

---

# Source: features\upgrade-path-attribution\FEATURE\_ARCHITECTURE.md  
# Upgrade Path Causal Performance Attribution

Status: APPROVED — owner instruction, 2026-09-18  
Owner: Vaibhav  
Scope: Upgrade Path governance and evidence reporting only. No score, sizing, paper, live, broker or execution behavior changes.

## ## Decision

Every Upgrade Path entry must show one of four explicit attribution states:

1. `measured` — a predeclared baseline-versus-variant comparison exists on the same market, common evaluation window, same eligible population, and cost model. It reports incremental portfolio return, benchmark-relative return, uncertainty, drawdown and turnover.
2. `collecting` — the program can affect a portfolio eventually, but its matched outcome window is not mature. It reports the exact missing sample/session gate, never an estimated gain.
3. `not\_attributable` — the program is operational or descriptive only (for example data freshness, diagnostics or health), so a portfolio-return number would be a category error. It states the operational proof instead.
4. `invalid` — a purported result failed provenance, common-window, baseline, cost, independence, or version checks. It is visible as invalid and cannot make a path review-ready.

`ready\_for\_review` continues to mean only that the path's declared evidence gate is met. It is not a return forecast, approval, or deployment action.

## ## Causal contract

One append-only `upgrade\_path\_attribution\_runs` row represents one immutable comparison for one `{program\_id, market, program\_version, baseline\_version, as\_of\_session}`. The row stores:

- comparison type: `matched\_replay`, `paper\_cohort`, `operational\_only`;
- start/end session and non-overlapping evaluation horizon;
- baseline and variant portfolio returns, same-window benchmark return, and incremental return (`variant` - `baseline`);
- net-of-cost incremental return, turnover, maximum drawdown delta, number of independent sessions, confidence interval and t-statistic where meaningful;
- matched population hash, point-in-time input hash, mandate/strategy versions, cost-model version, and a validity/reason field.

No result may combine US and India, use a different candidate population on the two arms, overlap its nominal independent sessions, compare gross variant return to net baseline return, or attribute a result across a mid-window program/mandate version change. Any such mismatch writes `invalid` rather than a number.

## ## Program eligibility

Only programs with a predeclared decision alternative may eventually use `matched\_replay`: exits, entry/selection challengers, capital rotation, allocation, sizing and strategy variants. Their report must compare the same eligible decisions on the same dates before declaring portfolio impact.

Programs that become paper-active use `paper\_cohort` only after a frozen activation date and an otherwise unchanged control/baseline. If simultaneous changes prevent isolation, the result is `invalid`, not attributed.

Purely operational paths are permanently `not\_attributable`. They may report data coverage, outage avoidance or correctness proof, but never a hypothetical portfolio uplift.

## ## UI

Each Upgrade Path card gains an **\*\*Attribution\*\*** section:

- state pill and comparison type;
- "Since" date and baseline/variant versions;

- incremental net return, benchmark-relative incremental return, 95% interval, independent-session count, turnover and drawdown delta when `measured`;
- exact blocker when `collecting`;
- explicit explanation when `not\_attributable` or `invalid`.

The page footer repeats that an observed incremental return is historical evidence, not a forecast. A path cannot be called beneficial from an unpaired P&L total or from aggregate portfolio performance after multiple releases.

## ## Acceptance criteria

1. Every registry entry declares its attribution class; no silent default.
2. The status API returns an attribution object for every entry.
3. UI renders the state and never substitutes a generic benefit label for a causal result.
4. Tests reject mixed markets, version drift, mismatched windows, gross/net comparisons, missing costs, and overlapping-session claims.
5. No writer or consumer may promote/enable a strategy from attribution alone.

---

# Source: features\us-keyless-screener\FEATURE\_ARCHITECTURE.md  
# US Keyless Screener – Feature Architecture

Status: **\*\*Draft – awaiting owner approval. No code written.\*\***  
Author: Claude · 2026-08-02  
Revision 3 – corrects rev 2's field validation and drops the FCF leg (§13).  
Related: `features/feature-pack-validation/`, `docs/arch/03-agents.md`.

---

## ## 1. Why this exists

US candidate discovery is dead and has been for at least ten days.

`decision\_observations`, 2026-07-23 → 2026-08-02, by `discovery\_source`:

| Market | Source                                 | n            |
|--------|----------------------------------------|--------------|
| US     | `holding`                              | 600          |
| US     | `watchlist`                            | 151          |
| US     | `manual`                               | 48           |
| US     | `screener_momentum` / `screener_value` | <b>**0**</b> |
| India  | `india_holding`                        | 140          |
| India  | `india_screener`                       | 87           |
| India  | `manual`                               | 13           |

Cause: the FinancialDatasets key is at `\$0.00` (observed 2026-07-29).  
`runScreener()` returns `[]` and the run continues from other queues – fail-soft, no crash, nothing visibly broken.

The consequence is not "fewer candidates." **\*\*Every US decision for ten days concerned a name the system already held or already watched.\*\*** The evidence record being accumulated right now – the same record gating threshold recalibration, learner promotion, and every feature-pack decision – is a closed loop over prior selections. That bias does not appear in any row count.

India is unaffected; it never used FinancialDatasets.

## ### Why not simply top up the key

1. **\*\*Recurrence.\*\*** The failure mode is a paid provider going quiet while the system reports healthy. Topping up restores discovery until the next exhaustion. The alarm gap (§7) is the real defect; the balance is only the trigger.
2. **\*\*Standing constraint.\*\*** The owner's standing rule is \$0 cloud. A paid dependency on the discovery path is a permanent exception to it.

Topping up remains a legitimate choice; §6 keeps the FinancialDatasets path as a fallback.

---

## ## 2. What is proposed

Replace the FinancialDatasets screener call with Yahoo's **\*\*custom screener endpoint\*\*** – keyless, server-side, and already reachable using the crumb/cookie handshake this codebase implements for India.

POST <https://query2.finance.yahoo.com/v1/finance/screener?crumb=<crumb>>

Two calls per research run: one momentum bucket, one value bucket. Yahoo evaluates the criteria across the whole US listed universe and returns ranked, capped results.

**\*\*Scope boundary: discovery only.\*\*** It changes which symbols enter the research

batch. It does not touch scoring, direction, eligibility, sizing, entries, exits, promotion, or any broker path.

---

## 3. Verification – all of this was probed live, 2026-08-02

Not reasoned from documentation. Actual responses.

**\*\*Endpoint reachable, keyless.\*\*** `GET /v1/test/getcrumb` with a `fc.yahoo.com` cookie yields a crumb; the POST returns `200`.

**\*\*Seven of the eight fields are honoured. One is silently ignored.\*\***

An earlier revision of this section listed all eight as validated, each probed with a `gt 0` filter and a result count. **\*\*That method was wrong and the conclusion it produced was wrong.\*\*** A `gt 0` filter is nearly a no-op whether the criterion is honoured or discarded, so a non-zero count proves only that the request did not error. It cannot distinguish a working filter from an ignored one. The corrected method is an **\*\*absurd-value control\*\***: set a threshold no security can satisfy and confirm the count collapses.

Baseline – exchange + volume + `intradaymarketcap > 1e9`, no other criterion: **\*\*1,831\*\*** names.

```
FD field	Yahoo criterion	sane	absurd	verdict
`revenue_growth`	`quarterlyrevenuegrowth.quarterly`	639	0	honoured
`earnings_growth`	`epsgrowth.lasttwelvemonths`	822	0	honoured
`gross_margin`	`grossprofitmargin.lasttwelvemonths`	1,338	0	honoured
`return_on_equity`	`returnnonequity.lasttwelvemonths`	642	0	honoured
`price_to_earnings_ratio`	`peratio.lasttwelvemonths`	890	385	honoured
`debt_to_equity`	`totaldebtequity.lasttwelvemonths`	1,093	91	honoured
`free_cash_flow_yield`	`freecashflow.lasttwelvemonths`	1,831	**1,831**	**IGNORED**
(unused)	`netincomemargin.lasttwelvemonths`	1,110	0	honoured
```

`freecashflow.lasttwelvemonths > 999,999,999,999,999` – one quadrillion dollars – returns the same 1,831 as no filter at all. The criterion is accepted by the API and discarded. Market-cap banding does not recover it either: four bands with band-relative FCF floors summed to exactly the unbanded total.

The `fields` request parameter is also ignored – the response is a fixed 89-key quote payload with no `freeCashflow`, so the yield cannot be computed client-side from screener output either.

**\*\*Units are percentages, not ratios.\*\*** `returnnonequity.lasttwelvemonths > 15` means 15%, not 1500%. `totaldebtequity` likewise: `< 100` is the ratio `< 1.0`. Every threshold must be multiplied by 100 when ported. A ratio passed unconverted would match everything and silently disable the filter.

**\*\*The full momentum bucket returns real results.\*\*** `revG>15, epsG>10, GM>25, ROE>15, mcap>2e9` → `total: 366`.

### The contamination finding – and why an exchange filter is mandatory

With `region = us`, that momentum bucket's top results were:

GGPSF, NVPTF, IDDTF, IDTVF, NLY, NLY-PF, NLY-PG, KXIAY

Mostly OTC/pink-sheet foreign issues (the `F`/`Y` suffixes) and **\*\*preferred share series\*\*** (`NLY-PF`, `NLY-PG`). The value bucket was worse: `CBAOF`, `LYTHF`, `BBAGF`. These are not tradeable US common stock; Robinhood cannot fill most of them, and a preferred series is not the security the score describes.

Constraining to `exchange in (NMS, NYQ)` plus `dayvolume > 500000`:

NLY, MU, SKHY, IBRX, BE, CRDO, SIMO, IAG, ERO, CORZ (total: 132)

Liquid NASDAQ GS / NYSE common stock. **\*\*The exchange and volume filters are not optional hardening – without them this feature injects untradeable symbols into research.\*\***

---

## 4. Design

### 4.1 `lib/data/yahoo-screener.ts` (new)

POST <https://query2.finance.yahoo.com/v1/finance/screener?crumb=<crumb>>

body: { size, offset, sortField, sortType, quoteType: "EQUITY", query: {...} }

Reuses the existing `getCrumb()` cookie/crumb helper – no new auth surface.

Every query carries a mandatory base clause, applied by the module and not

overridable by callers:

```
- `exchange` in `NMS` | `NYSE`
- `dayvolume > 500_000`
- `quoteType = "EQUITY"``
```

Response symbols are additionally validated against the existing symbol policy before entering research: reject anything containing ``-` or ``.` (preferred series, units, warrants), reject known leveraged/inverse products via `isLeveragedInverseEtf`.

### 4.2 Bucket definitions

Thresholds carried over from the current FD filters, converted to percent:

**\*\*momentum\*\*** – sort `quarterlyrevenuegrowth.quarterly` DESC

quarterlyrevenuegrowth.quarterly > 15

epsgrowth.lasttwelvemonths > 10

grossprofitmargin.lasttwelvemonths > 25

returnnonequity.lasttwelvemonths > 15

intradaymarketcap > 2\_000\_000\_000

**\*\*value\*\*** – sort `peratio.lasttwelvemonths` ASC

peratio.lasttwelvemonths > 0

peratio.lasttwelvemonths < 18

totaldebtequity.lasttwelvemonths < 100 (= ratio 1.0)

intradaymarketcap > 1\_000\_000\_000

No free-cash-flow leg – see §5. It is omitted rather than approximated.

The existing round-robin interleave and cap are unchanged. The locked rule of **\*\*3 screener candidates/day\*\*** is unchanged – this restores supply to a starved funnel, it does not widen the gate.

### 4.3 `runScreener()`

1. Yahoo custom screener, both buckets

2. on failure, and only if a funded FD key exists → existing FinancialDatasets path

3. else → [] and raise the staleness issue (§7)

---

## 5. The free-cash-flow leg is dropped

The current value bucket requires `free\_cash\_flow\_yield > 0.04`. **\*\*No Yahoo criterion can express it, and there is no weaker working substitute.\*\***

Yahoo offers `freecashflow.lasttwelvemonths` – absolute dollars, not a yield – and §3 proves that criterion is silently discarded. So the previously-considered options collapse:

- **\*Accept `freecashflow > 0` as a loosening\*** – not available. The filter does nothing at any threshold.
- **\*Band the market cap and use a band-relative FCF floor\*** – not available. The bands summed to the unfiltered total.
- **\*Compute the yield from screener output\*** – not available. `freeCashflow` is absent from the response and the `fields` parameter is ignored.

The leg is therefore **\*\*omitted\*\***, and the value bucket becomes `0 < P/E < 18`, `debt/equity < 1.0`, `market cap > \$1B`, plus the mandatory exchange/volume base clause.

**\*\*What is genuinely lost:\*\*** the independent cash-flow cross-check on accounting earnings. P/E still carries "cheap" and debt/equity still carries balance-sheet quality, so the bucket remains coherent – but it no longer verifies that reported earnings convert to cash.

**\*\*Why no proxy is substituted:\*\*** `netincomemargin.lasttwelvemonths` is honoured and would look like a reasonable stand-in. It is not: it is earnings-derived, and cross-checking earnings against earnings is circular – precisely the check the FCF leg existed to provide. A proxy that reads like the original while providing none of its independence is worse than an acknowledged gap.

Record in `PROJECT\_DECISIONS.md` as a deliberate omission with this reasoning, so a later reader does not restore it by adding a field that does not work.

---

#### ## 5b. Field-contract control – permanent, not a one-off

The §3 discovery is the strongest argument in this document, and it is not about free cash flow. **\*\*A screener criterion can be accepted by the API, returned without error, and contribute nothing.\*\*** No exception, no warning, no test failure – just a filter that quietly stops filtering.

R1 (undocumented endpoint) is not a hypothetical risk to be accepted with a shrug. It materialised on the first field probed carefully, before any code was written. Every remaining honoured field is one Yahoo deployment away from the same state, and the failure is invisible by construction: the screen keeps returning names, the run keeps succeeding, and the bucket silently widens to whatever the surviving criteria allow.

**\*\*Ship an absurd-value control as a live contract check\*\*, not a one-time validation:**

- for each criterion, issue the bucket query with a threshold no security can satisfy (`gt` → absurdly high, `lt` → absurdly low)
- the count must collapse relative to the same query without that criterion
- if it does not, the criterion is a no-op: raise `screener-field-degraded:<field>` at `critical` and **\*\*fall back to FinancialDatasets\*\*** rather than screening on a bucket that is quietly wider than its definition

Run it on a schedule, not per research run – the probe costs one request per field and the condition changes on Yahoo's deploy cadence, not hourly. Daily, alongside the existing model-freshness check, is sufficient.

This is cheap, it protects all seven working legs rather than the one that failed, and without it the next silent deprecation produces a wider screen that nothing reports. It is the piece of this proposal most worth keeping even if the screener direction is rejected in favour of funding FinancialDatasets – the same class of check applies to any provider whose contract is not versioned.

---

#### ## 6. What is deliberately not changed

- No score, weight, threshold, direction, eligibility, sizing, entry, exit, promotion, or broker behaviour.
- No change to the candidates/day cap.
- FinancialDatasets is **\*\*kept\*\*** as fallback, not deleted.
- Long-only for new entries; SELL capability on holdings untouched.
- India path untouched.

---

#### ## 7. The alarm – the actual defect being fixed

The ten-day outage was silent. `runScreener()` did call `reportIssue()` on a missing key, but nothing watched what matters: **\*\*whether US discovery produced any candidate at all.\*\***

Add a market-local discovery-coverage check:

- raise `discovery-starved:us` at `warn` when a research run completes with zero screener-sourced candidates
- escalate to `critical` after three consecutive such runs
- resolve on the first run that yields one

**\*\*Ship this first, independently.\*\*** It is small, depends on nothing else here, and without it the next discovery outage – from any cause, keyless or paid – is equally invisible.

---

#### ## 8. Honesty constraints

Yahoo's screener evaluates **\*\*current\*\*** listings and current fundamentals. It is not point-in-time and carries survivorship bias by construction.

Acceptable for **\*discovery\*** – choosing what to research now is a present-tense question. **\*\*Not\*\*** acceptable as backtest or IC input. These results must not feed `lib/edges/` OOS folds, the PIT universe resolver, or any evidence pool without a separate PIT contract and separate approval.

The Upgrade Path must describe this as restored discovery breadth, never as an alpha improvement. Widening the funnel changes what gets examined, not what is true.

---

## ## 9. Risks

**\*\*R1 – undocumented endpoint (materialised, mitigated).\*\*** ``/v1/finance/screener`` is not a published API. Yahoo can change the field vocabulary, the crumb flow, or the response shape without notice. **\*\*This is no longer hypothetical:** ``freecashflow.lasttwelvemonths`` is already in the failed state (§3), accepted and discarded, and was caught only because it was probed with an absurd-value control. **\*\*Mitigations:** the §5b contract check turns a silent no-op into a ``critical`` issue with an automatic FD fallback; the §7 alarm catches total discovery failure; the field mapping is a single table in one module. This is the principal risk and §5b is the reason it is survivable.

**\*\*R2 – single-provider concentration.\*\*** After this, both markets' discovery depends on Yahoo. Today India alone dies if Yahoo blocks; after this, both do. Partially mitigated by keeping the FD fallback funded-or-not.

**\*\*R3 – unit errors.\*\*** Percent-vs-ratio (§3) is the specific trap; ``totaldebtequity < 1.0`` instead of ``< 100`` would silently pass every name. Mitigation: unit assertions in tests, one per threshold.

**\*\*R4 – untradeable symbols entering research.\*\*** Proven real (§3). Mitigation: the mandatory base clause and symbol-policy validation in §4.1, with a test asserting that a known OTC and a known preferred series are both rejected.

**\*\*R5 – rate limiting.\*\*** Two POSTs per research run is negligible next to India's 600-symbol nightly rotation. Low.

---

## ## 10. Acceptance gates

1. Both buckets return non-empty from the deployed environment (not just locally – Vercel egress differs).
2. Every returned symbol passes exchange, volume, and symbol-policy validation; tests assert an OTC ticker and a ``-P`` preferred series are rejected.
3. Unit tests assert percent-scaled thresholds for all six numeric criteria.
- 3b. The §5b absurd-value control passes for every criterion in both buckets at ship time, and its scheduled job is registered. A criterion that fails the control must not be in a shipped bucket.
4. ``discovery_source`` shows non-zero ``screener_momentum`` / ``screener_value`` for US on the first post-deploy research run.
5. The §7 alarm fires in a forced-empty test and resolves on recovery.
6. The §5 deviation is recorded in ``PROJECT_DECISIONS.md`` with the owner's choice.
7. ``npx tsc --noEmit``, ``npx vitest run``, ``npm run build`` all clean.

---

## ## 11. Documentation to update on ship

- ``docs/arch/02-tech-stack.md`` – Yahoo screener as a US discovery provider
- ``docs/arch/03-agents.md`` – ResearchAgent discovery inputs
- ``public/agent-diagrams/system-map.json`` – node + ``history`` entry
- ``PROJECT_DECISIONS.md`` – keyless-over-paid discovery, and the §5 deviation

No migration, no new table, no new cron.

---

## ## 12. Open questions for the owner

1. **\*\*Approve the direction?\*** Yahoo keyless screener, or top up FinancialDatasets?
2. **~Which handling of the lost FCF-yield filter?~** **\*\*Resolved by measurement, not by preference.\*\*** No working substitute exists; the leg is dropped (§5). Confirm you accept the value bucket without a cash-flow cross-check.
3. **\*\*Ship §5b's field-contract control regardless of 1?\*** Recommend yes. It applies to any unversioned provider contract, including a re-funded FinancialDatasets.
4. **~Ship §7's alarm first?~** **\*\*Done\*\*** – ``discovery-starved:us`` shipped in ``66b7862e``, independent of everything else here.

---

## ## 13. Revision history

**\*\*Rev 3 (2026-08-02).\*\*** Corrects rev 2's field validation, which was unsound. Rev 2 probed each criterion with a ``gt 0`` filter and treated a non-zero result count as proof the field worked. That test cannot fail: a ``gt 0`` filter is nearly a no-op whether the criterion is honoured or discarded, so it demonstrated only that the request did not error. Re-probed with an absurd-value control, ``freecashflow.lasttwelvemonths`` proved to be silently ignored – accepted by the API and contributing nothing at any threshold, including one quadrillion dollars. Rev 2's §5 offered three ways to handle a "loosened" FCF filter; all three assumed a filter that does not exist. The leg is dropped (§5), and the control test is promoted to permanent infrastructure (§5b) because the same failure can silently disable any of the seven working criteria.



**\*\*Rev 2 (2026-08-02).\*\*** Rewritten after probing Yahoo's custom screener endpoint live. Rev 1 proposed enumerating a US universe from the Nasdaq Trader symbol directory into a new `us_screen_cache` table, refreshed by a nightly cron rotating 600 symbols per run – mirroring India's architecture, which exists because Yahoo has no `*India*` screener coverage worth using.

For US, Yahoo screens server-side. That deletes the universe file, the table, the migration, the cron, and the multi-night rotation, replacing all of it with two POST requests. Rev 1's Nasdaq Trader reachability gate is moot; both files were confirmed reachable (``200``, 345KB and 533KB) but are no longer needed.

Rev 1 also missed the OTC/preferred contamination entirely, because a directory-file universe would have filtered those out structurally. Screening server-side surfaces them, which is why §4.1's base clause exists.

---

# Source: features\us-paper-fractional-shares\FEATURE\_ARCHITECTURE.md  
# US Paper Fractional Shares

> Status: **\*\*APPROVED FOR IMPLEMENTATION\*\***  
> Owner approval: 2026-07-26  
> Scope: US paper book only

#### ## Problem

The US paper pool holds cash that can fund qualified positions, but PaperTrader rounds all quantity down to a whole share. A valid US allocation below one share becomes an ``insufficient_cash_for_1_share`` rejection even though US fractional execution is available. Production on 2026-07-26 had \$5,178.58 cash and multiple such skips.

#### ## Decision

Allow US paper entries, rotations, and target partials to use quantities rounded down to six decimal places. India remains whole-share because this app has not established fractional NSE execution semantics. ``qty`` and all relevant paper RPC arguments are already ``numeric``, so no schema migration is needed.

The US paper mandate increases from 10 to 15 alpha names. It is a per-market setting; it does not modify India, existing positions, stops, targets, score thresholds, or live-order limits.

#### ## Rules

- US entry quantity: ``floor(maxSpend / price, 6 decimals)``; must be finite and  $> 0$ .
- India entry quantity: unchanged ``floor(maxSpend / price)``; must be  $\geq 1$ .
- US partial target: sell half rounded down to six decimals only when both the sell and remaining legs are positive. Otherwise close the full position.
- All cash, notional, caps, name/sector limits, rotation gates, and transactional RPC checks run unchanged and continue to receive the exact numeric quantity.
- No live broker call or live quantity behavior changes.

#### ## Acceptance

- A \$100 US allocation can buy a fractional share priced above \$100.
- An equivalent India allocation remains rejected if it cannot buy one share.
- US partial profit-taking preserves quantity and cash conservation.
- Decimal quantities pass the existing fill/exit/rotation RPC contracts.
- Tests cover rounding, no dust/NaN, US-vs-India isolation, and partial exits.

---

# Source: features\us-selection-and-hold-gaps\FEATURE\_ARCHITECTURE.md  
# Decision-Intent and Holding-Exit Integrity

> Status: **\*\*APPROVED FOR INTEGRITY BUILD; SCORE POLICY REMAINS MEASURE-ONLY.\*\***  
> Last revised: 2026-09-11.  
> Live auto-trading remains disabled. No score weight, eligibility threshold,  
> stop, target, sizing rule, or broker order is changed by this feature.

#### ## Problem

The observation ledger mixed two decisions: an **\*\*entry candidate\*\*** considered for a new position and a **\*\*holding review\*\*** produced because the symbol was already owned. ``entry_eligible=true AND direction='long'`` cannot distinguish them. Most US h10 rows in the disputed selection analysis were holding re-scores, so it answered a hold/exit question and was described incorrectly as selection. Consecutive h10 windows were also treated as independent, inflating t-statistics.

The same ambiguity reached execution: paper and live monitors loaded the latest deterministic symbol score without proving it was a session-validated holding review from the current position episode. A candidate score must never become exit authority.

#### ## Architecture

### ### 1. Immutable decision intent

`decision\_observations.decision\_context` records `entry\_candidate` or `holding\_review` at write time. Historical rows remain NULL. Legacy readers may use a small explicit `discovery\_source` mapping; unknown sources fail closed.

All entry analytics use the shared `isEntryCandidateLong` predicate. Holding analytics use `isHoldingReview`. The low-level score/direction predicate remains only where intent is irrelevant.

### ### 2. Exit authority and persistence

Both paper and live monitors require:

1. `agent\_signals.is\_holding=true`;
2. `session\_validated=true` and `score\_source=deterministic\_v1`;
3. signal timestamp at or after the position opened; and
4. normal freshness checks.

Loss of score conviction is not an immediate sale. The first qualifying holding session arms the exit; only a strictly newer holding session below threshold confirms it. Recovery disarms it. Stop and target protection remain independent. The existing `direction\_flip\_armed\_session` name is retained for compatibility, but stores the broader conviction-exit persistence state.

### ### 3. Score-exit shadow

The daily `score-exit-shadow` route evaluates three predeclared holding-score thresholds: entry threshold, entry threshold minus 10, and minus 20. It uses only matured labels linked to session-validated holding signals. Results are immutable, market-local, fingerprinted, and visible in Upgrade Path.

Each arm reports trigger count, clean trigger count, unresolved competing exits, paired incremental return versus holding, avoided-loss share, foregone-gain share, and worst paired outcome. A row whose MFE/MAE touches stop or target is unresolved because excursions cannot establish event order.

Overlapping windows use `effective observations = distinct sessions / horizon days`. Below 12 effective observations the status is `insufficient\_evidence`; no naïve significance claim or promotion is allowed.

### ### 4. Closed-lot high-water invariant

The exit RPC patch is retained, but a `BEFORE UPDATE OF closed\_at` trigger also enforces the invariant at the table boundary. Full, partial, and residual closes are covered by a transaction-rolled-back SQL verification. Historical NULLs are not reconstructed.

### ## Data contracts

- `20260911193121\_decision\_observation\_intent.sql` adds nullable constrained `decision\_context` without rewriting history.
- `20260911194022\_enforce\_closed\_trade\_high\_water.sql` adds the high-water trigger.
- `20260911194147\_score\_exit\_shadow\_runs.sql` adds an append-only owner-readable evidence ledger and both-market weekday schedules.
- No money-path module may read `score\_exit\_shadow\_runs`.

### ## Acceptance gates

- Candidate signals cannot arm or confirm a paper or live exit.
- A holding score needs two distinct fresh sessions to confirm an exit.
- Entry diagnostics use decision intent; unknown legacy sources fail closed.
- Full, partial, and residual closed lots retain verified high-water.
- Score-exit evidence appears daily in Upgrade Path but cannot affect trading.
- Any future policy change requires 12 effective observations, competing-risk and cost review, execution-faithful replay, and explicit owner approval.

### ## Superseded claims

The earlier `t=-5.42`, "US selector picks the worst names," and related causal claims are withdrawn. They used a mostly holding-review population and naïve independence across overlapping windows. That population is relevant to exits, so it is now measured by the dedicated shadow rather than reused as entry evidence.

---

# Source: features\volatility-scaled-exit-geometry\FEATURE\_ARCHITECTURE.md  
# Volatility-Scaled Exit Geometry

> Status: **\*\*PROPOSED** – architecture only. **\*\*** No scoring, sizing, stop, target, paper or live-order behavior changes are authorized by this document.  
> Last revised: 2026-09-12.  
> Money-path influence: **\*\*none\*\*** until Stage 2 is separately approved.

### ## 0. The problem, measured

The mandate provides one market-level fallback stop and target:

```
market	stop	target	horizon	R:R
US	7%	8%	10d	**1.14 : 1**
India	7%	8%	10d	**1.14 : 1**
```

Source: `trading\_mandates`, one row per market, set by the owner 2026-08-03 (US version 4, India version 3).

`resolveExecutionRiskReward` (`lib/trading/trade-plan.ts`) first tries a market-local, entry-candidate MAE/MFE percentile rule and otherwise returns the mandate; `bindTradePrices` multiplies the selected percentages against the actual fill. The rule is deterministic, but still global to a market/horizon: no volatility, ATR, beta, price level, liquidity or sector term enters.

### Why a flat percentage is the wrong shape

A fixed percentage is a **different bet on every symbol**, because the same percentage is a different number of standard deviations depending on the name:

- a 2%-daily-ATR utility: a 7% stop is ~3.5 ATR – rarely touched by noise, but each loss is large relative to the move that caused it;
- a 6%-daily-ATR semi: a 7% stop is ~1.2 ATR – touched by ordinary intraday noise, so the position is stopped out repeatedly before the thesis resolves.

The system therefore takes systematically **more** risk-of-ruin-per-trade on quiet names and systematically **more** whipsaw on volatile ones, while reporting a single "7% stop" as if it were one policy. It is not one policy; it is an unmanaged spread of policies.

The 1.14:1 reward-to-risk compounds this: it needs a **~47% win rate** before costs just to break even, and that bar is identical for a high-conviction setup and a marginal one.

### What already exists

```
Component	State
`lib/trading/exit-stop-shadow.ts`	Measure-only. Tests H1: 2.8 ATR stop vs the mandate stop, target held identical. Sidak-adjusted for the
`lib/trading/exit-geometry-shadow.ts`	Barrier resolution + counterfactual replay over matured labels.
`resolveExecutionRiskReward` ledger percentile	Overrides the mandate when there are at least 60 labelled entry-candidate observations. It
Anything ATR in a live path	**None.** `atr` appears only in the two shadow modules.
```

So the machinery to evaluate this exists; the evidence does not yet.

## 1. Non-goals

- No change to any live or paper stop, target, horizon or size in Stages 0-1.
- No per-symbol **score** change. This is exit geometry only.
- No new provider dependency: ATR is computed from bars already cached.
- No replacement of the trailing-stop ladder (`lib/trading/exit-ladder.ts`). The ladder consumes whatever initial stop it is given; this proposal changes where that number comes from, not how the trail ratchets.
- No claim that ATR is the right volatility estimator. It is the one already implemented and already under test.

## 2. Proposed staging

### Stage 0 – accumulate honest evidence

The shadow baseline must be explicitly stamped. A current-mandate baseline is useful only as a current-policy control; it must not be labelled as a historical reconstruction of plans that may have used the ledger percentile rule.

Exit criteria:

- `exit\_stop\_shadow\_runs` rows with `matches\_live\_mandate = true` and `status = 'measured'` for **both** markets, proven separately;
- `effective\_observations >= MIN\_EFFECTIVE\_OBSERVATIONS` per market;
- the two pre-existing rows (2026-09-06, `matches\_live\_mandate = false`) are excluded from any conclusion – they measured a 19.2% target that was never deployed.

**Do not raise the shadow's cadence to force this.** It reads matured labels; running it daily recomputes the same cohort and invites peeking at a moving result. Weekly is correct – the constraint is label maturation, not compute.

### Stage 1 – shadow the full geometry, not just the stop

H1 varies only the stop. That was the right first cut, but a 1.14:1 R:R means the **target** is at least as suspect as the stop. Stage 1 adds arms that vary target with volatility, with the stop held at the mandate value, so the two terms are never confounded in one arm.

Every arm is predeclared and counted in the multiple-testing family. The Sidak

adjustment must be recomputed for the \*new\* family size – silently reusing  
`TRIALS\_CONSIDERED = 14` after adding arms would understate the threshold.

### ### Stage 2 – separately proposed switch

Only after Stage 1 has matured labels and no quality regression may a separate architecture propose changing `resolveExecutionRiskReward`. It must state:

- the exact geometry function and its bounds;
- a \*\*frozen, read-only counterfactual\*\* showing which historical decisions flip, per market, before any live formula changes (Scoring Data-Truth Protocol §6);
- US and India proven separately (§7) – India's ATR distribution and tick/circuit-limit rules differ and cannot be validated by a pooled result;
- rollback: the mandate row stays the fallback, so reverting is a config change.

### ## 3. Open questions for the owner

1. Is the 1.14:1 R:R deliberate? A 7%/8% pairing is unusual for a 10-day swing horizon and may simply be a stale setting rather than a chosen policy. This is worth answering \*before\* spending evidence on the stop term.
2. Should volatility scaling bound the \*position size\* instead of, or as well as, the stop? Equal-risk sizing (smaller position on a wider stop) achieves much of the same risk normalization without touching exit geometry, and the flat `position\_size\_pct` cap has the identical one-size-fits-all defect.
3. Minimum-tick and circuit-limit handling for India before any ATR stop is allowed to produce a price there.

### ## 4. Acceptance criteria

- [ ] No live stop, target or size changed by Stages 0-1.
- [ ] Every persisted shadow row stamps the geometry it measured (`baseline\_stop\_pct`, `baseline\_target\_pct`, `candidate\_stop\_atr`, `matches\_live\_mandate`) – shipped 2026-09-10.
- [ ] Conclusions cite only rows with `matches\_live\_mandate = true`.
- [ ] US and India reported separately, never pooled.
- [ ] The multiple-testing family is recounted whenever an arm is added.
- [ ] A frozen counterfactual exists before any Stage 2 formula change.
- [x] New entry/proposal evidence records a versioned execution-plan provenance in its append-only journal/policy snapshot (2026-09-12). This improves reproducibility but does not validate or promote a formula.

---

# Source: features\walk-forward-ic-folds\FEATURE\_ARCHITECTURE.md  
# Feature Architecture: Purged Out-of-Sample IC Validation

### ## Status

Architecture status: \*\*MEASURE-ONLY – promotion dormant\*\*

Architecture approved: Steps 2-3 shipped; US step 4 shipped; India step 4 blocked. Yahoo-first candle routing shipped 2026-08-18 (Annex B1).

Approved scope: diagnostics and evidence hardening only

Approved date: 2026-07-28

Implementation allowed: Measurement work may continue; policy promotion may not.

**\*\*Outcome.\*\*** The stack produced useful negative evidence: `mom\_12\_1` has no useful signal at h5 in this sample, and daily IC is noisy. The study did not prove an effective breadth of 17 or that universe size and sector neutralization cannot help. Temporal IC variance combines sampling noise, genuine factor variation, changing membership, and coverage; the current artifact does not identify those components.

`POST /api/agents/backtest/promote` remains disabled. EdgeScout/EdgeIC continue as advisory diagnostics and must never be cited as promotion evidence.

Next measure-only work must record usable cross-section per date, compare universe sizes on matched dates, measure sector-neutral IC with point-in-time sector provenance, and obtain matched-horizon evidence. None of those results may change scoring or trade authority directly.

**\*\*2026-07-28 correction build:\*\*** OOS report schema v2 now retains the usable cross-section for every successful date plus the resolved universe fingerprint, members, provenance, and dates to which each snapshot was applied. OOS Spearman IC now ranks raw finite edge values; it no longer winsorizes first and creates artificial tail ties. This improves future diagnostics only.

**\*\*Remaining order:\*\***

1. Use per-date membership plus required snapshot persistence for every promotion-grade run; India remains blocked on a PIT membership source.
2. Bind experiments to edge id, formula version, market, horizon, and immutable OOS run fingerprint.
3. Run matched-date  $n=200/n=400$  and point-in-time sector-neutral diagnostics.
4. Run a separately predeclared h20 design with HAC-lag sensitivity and costs.
5. Only then decide whether a promotion statistic and floor are supportable.

Items 1-4 are not interchangeable. Running experiments without per-date

membership and required persistence would spend provider quota on another non-promotion-grade artifact.

**\*\*US step 4 shipped and verified:\*\*** `us\_pit\_adv20\_top400\_v2` computes average dollar volume over exactly 20 market sessions ending at each as-of date, shares session fetches across overlapping windows, fails closed on an incomplete window, and persists a top-400 superset whose deterministic prefixes support matched n=200/n=400 tests. `persist\_edge\_pit\_snapshot()` writes through a service-role-only, advisory-locked RPC; exact reruns are idempotent, differing reruns conflict, and PIT rows are append-only. The orchestrator must set `persistSnapshots=true` and `membershipCadence="per\_date"` for promotion-grade output. Defaults remain diagnostic-only.

#### ## Implementation Status (2026-07-28)

Steps 2 and 3 are SHIPPED. Migration `20260728090000\_promotion\_schema\_repair\_and\_atomic\_rpc.sql`, applied and verified against production while `strategy\_policies` and `backtest\_experiments` were both empty:

```
Repair	State
`dsr` renamed `t_margin_vs_trials` (it was never a Deflated Sharpe Ratio)	done
`walk_forward_pass` renamed `ic_stability_pass`	done
`validation_mode` NOT NULL, CHECK in (`purged_temporal_oos`, `walk_forward`)	done - a legacy rolling-window result has no representable value
`experiment_id` NOT NULL FK to `backtest_experiments`	done
Mutation trigger compares whole rows minus `superseded_at`	done - the hand-listed column set left 7 fields silently mutable
`superseded_at` and experiment `policy_id` are write-once	done
`promote_strategy_policy()` RPC, SECURITY DEFINER, service-role only	done
Promote route calls the RPC instead of supersede-then-insert	done
`experiment_id` required; `variants_run` must be recorded	done - the old fallback chain substituted a flattering trial count when the run failed
```

P0 proven in production 2026-07-28: inside one transaction, two promotions produced `baseline` then `variant` with exactly one active row; a third promotion was then forced to fail at insert AFTER its supersede, and the incumbent survived unchanged. The whole block was rolled back, leaving both tables at zero rows.

Promotion remains DORMANT at the route (`promotion\_evidence\_not\_oos`, 503). Steps 2-3 fixed how a policy would be written. They did nothing about whether the evidence deserves to be written, which is steps 4-10.

#### ## Decision

Build this before any `strategy\_policies` promotion is allowed.

The current `edge\_ic\_history` rows are useful retrospective diagnostics, but they are not promotion evidence. Splitting the same retrospective current-name dataset into date ranges would not repair point-in-time universe bias, experiment-selection bias, or the absence of a frozen train/test protocol.

Until this feature and the atomic promotion RPC are approved and shipped, `POST /api/agents/backtest/promote` must remain operationally dormant. A later approved implementation should make the route fail closed with `promotion\_evidence\_not\_oos` rather than treating three rolling windows as an evidence minimum.

#### ## Purpose

Create reproducible, market-local, point-in-time out-of-sample evidence for an edge before it can become a policy. The system must answer:

1. What formula and trial family were fixed before evaluation?
2. What symbols and inputs were knowable on each as-of date?
3. Which observations trained or calibrated the formula?
4. Which later observations tested it without label overlap?
5. Does the combined out-of-sample evidence survive dependence, costs, and multiple testing?
6. Can promotion append a replacement policy atomically?

#### ## Measured Current State

As of 2026-07-27:

```
Fact	Production value
Rolling history per IC row	1,000 calendar days
US rows per edge/horizon vs distinct end dates	6 vs 4
Span from first to last US end date	16 calendar days
Approximate overlap	98.4%
Best latest US market-wide t-stat	1.73
Best latest India observation	1.57, Financials sector, only 2 windows
Active `strategy_policies`	0
`backtest_experiments`	0
Promotion-grade PIT universe	US infrastructure shipped; India unavailable; no promotion run accepted yet
```

| Regime rows allowed by `edge\_ic\_history` schema | no |

The current universe is documented in migration 132 and the EdgeIC response as a current-liquid snapshot. Replaying today's survivors through old dates is not point-in-time validation.

#### ## Corrections To The Previous Draft

1. **\*\*Disjoint test slices are not automatically walk-forward.\*\*** For a fixed, never-refit formula they are purged temporal out-of-sample folds. The term walk-forward is reserved for a protocol that freezes each fold's formula or parameters using only its earlier training/calibration interval and then evaluates a later test interval.
2. **\*\*Purge is measured in market sessions, not calendar days.\*\*** For horizon `H`, no test origin may have a forward-return label extending beyond its test interval. Training observations whose labels touch the test interval are purged; an embargo is added only when the feature construction requires it.
3. **\*\*A boolean `is\_disjoint` is not proof.\*\*** Boundaries, label end dates, fingerprints, and non-overlap validation are durable facts. The gate recomputes their consistency.
4. **\*\*Fold index is not a global identity.\*\*** It is unique only inside a frozen experiment/validation plan.
5. **\*\*Current-universe history cannot promote.\*\*** Point-in-time membership, delistings, symbol changes, corporate-action adjustment policy, and input availability are required.
6. **\*\*Fold-over-fold endpoint decay is not the primary statistic.\*\*** The primary evidence is the predeclared aggregate out-of-sample date-level rank-IC series with a dependence-robust uncertainty estimate. Fold sign consistency and a worst-fold guard are secondary diagnostics.
7. **\*\*The current `t - expectedMaxT` value is not the Bailey/Lopez de Prado Deflated Sharpe Ratio.\*\*** It is a trial-count-adjusted t margin. DSR also accounts for sample length and non-normality of strategy returns. IC discovery should use a declared multiple-testing procedure; DSR belongs on cost-adjusted strategy-return evidence.

#### ## Statistical Contract

##### ### Unit of observation

- Primary series: one cross-sectional rank IC per eligible market session/as-of date.
- The same-market cross section must meet a predeclared minimum symbol count.
- US and India observations, calendars, universes, currencies, benchmarks, and results are never pooled.
- Sector evidence is a separate plan and must meet stricter sample floors.
- Regime evidence is unavailable until a separate approved, PIT-safe regime label exists. It must not silently fall back to market-wide evidence.

##### ### Frozen plan

Before any result is computed, an immutable experiment records:

- `edge\_id` and exact `formula\_version`
- market, horizon, and exactly one segment dimension
- trial-family identifier and total challengers considered since the current champion was promoted
- point-in-time universe policy/version and snapshot fingerprint
- feature/input availability policy and adjustment policy
- train/test boundaries expressed as market sessions
- step size, purge sessions, optional embargo sessions, fold count
- minimum as-of dates and minimum cross-section size
- primary statistic, secondary diagnostics, and pass thresholds
- data cutoff and code/config fingerprint

Changing any field creates a new experiment. It never edits a completed one.

##### ### Fold construction

For fold `k`:

1. Build or calibrate only from sessions at or before `train\_end[k]`.
2. Purge training origins whose `H`-session labels overlap the test interval.
3. Freeze the selected formula/parameters.
4. Evaluate consecutive later sessions from `test\_start[k]` through `test\_end[k]`.
5. Require every label to mature by `label\_end[k] <= data\_cutoff`.
6. Append immutable date-level IC observations and one fold summary.

Test intervals do not overlap. Their label intervals do not cross fold boundaries. Folds are generated from an anchored plan, never from the cron's wall-clock execution date.

If there is no training or calibration step because the formula was fixed externally before all folds, label the mode `purged\_temporal\_oos`, not `walk\_forward`.

### ### Aggregation and pass criteria

The gate consumes all matured OOS date-level IC observations for one frozen plan:

- minimum 3 completed test folds
- minimum 24 eligible as-of dates per fold, subject to power analysis before approval
- minimum cross-section size per as-of date; sparse dates are excluded and counted, never converted to zero IC
- aggregate mean rank IC above the approved floor
- Newey-West/HAC t-stat on the concatenated chronological OOS IC series above the approved hurdle
- positive fold sign in a predeclared majority and no catastrophic worst fold
- multiple-testing control against the complete frozen trial family
- cost-adjusted long-only bucket test and benchmark comparison before policy promotion
- no unresolved PIT, provider, adjustment, or provenance degradation

No single latest fold, maximum fold t-stat, or pooled standard error from overlapping windows may decide promotion.

For factor discovery, use a predeclared false-discovery method appropriate to the trial dependence. Do not label the current expected-max-t subtraction as DSR. If DSR is later used, compute it from the strategy-return series with the paper's sample-length, skewness, kurtosis, and trial-selection inputs.

### ## Data Architecture

#### ### Reuse the existing lineage

Do not create a third experiment/provenance truth layer.

Extend `backtest\_experiments` to be the immutable plan header with:

- `edge\_id`, `formula\_version`, `horizon`
- `validation\_mode`
- `trial\_family\_id`, `trials\_considered`
- `universe\_policy\_version`, `universe\_fingerprint`
- `dataset\_fingerprint`, `code\_version`
- `validation\_spec` JSONB with a schema/version CHECK
- `data\_cutoff`

The plan references existing `edge\_signal\_inputs`, `edge\_universe\_members`, and evidence/provider fingerprints. It does not copy or reinterpret their provenance.

#### ### New result tables

`edge\_ic\_fold\_results`:

- `experiment\_id`, `fold\_index`
- train, purge, test, and label-end session boundaries
- formula and dataset fingerprints
- `as\_of\_dates`, `min\_cross\_section\_n`, `excluded\_dates`
- mean IC, IC IR, HAC t-stat, confidence interval
- status and machine-readable refusal reasons
- unique `(experiment\_id, fold\_index)`
- service-role only, append-only

`edge\_ic\_oos\_observations`:

- `experiment\_id`, `fold\_index`, `as\_of\_date`
- rank IC, cross-section count, input/universe fingerprint
- unique `(experiment\_id, as\_of\_date)`
- service-role only, append-only

`edge\_ic\_history` remains the retrospective diagnostic ledger. Do not add `is\_disjoint` to it and then treat the same current-universe reconstruction as promotion-grade.

#### ### Strategy-policy schema repair

The promotion migration must:

1. add the correctly named `ic\_stability\_pass` and explicit `validation\_mode`; deprecate `walk\_forward\_pass`
2. stop describing the trial-adjusted t margin as `dsr`
3. harden the mutation trigger so every field except a one-way `superseded\_at: null -> timestamp` transition is immutable
4. bind each policy to its exact experiment and fold result set
5. make experiment `policy\_id` a one-way null-to-value transition

Production currently has zero policy rows, so this is the lowest-risk time to repair the schema semantics.

## ## Atomic Promotion Boundary

Supersede and insert must occur in one database transaction. A partial unique index prevents two active rows but cannot prevent zero active rows after supersede succeeds and insert fails.

Add a service-role-only database RPC modeled on  
`promote\_strategy\_champion`:

1. validate the experiment, segment, formula, market, horizon, trial family, matured folds, and gate result inside the transaction
2. acquire a transaction advisory lock for the exact policy segment
3. lock the incumbent row
4. append the new policy
5. supersede the incumbent
6. bind the experiment to the new policy
7. return both IDs

Revoke execution from `public`, `anon`, and `authenticated`; grant only the service role. The HTTP route remains cron-or-confirmed-owner gated and calls only this RPC.

## ## Module Inventory

```
Module	Approved future change
`lib/edges/ic.ts`	Pure fold evaluator over frozen observations
`lib/edges/validation-plan.ts`	Session-aware plan validation and fingerprints
`app/api/agents/edge-ic/route.ts`	Keep retrospective diagnostics; do not overload it
new validation route/worker	Execute approved frozen plans and append OOS results
`lib/gates/promotion-gate.ts`	Consume aggregate OOS result, not rolling windows
`app/api/agents/backtest/promote/route.ts`	Validate request, require bound experiment, call atomic RPC
`supabase/migrations/*`	Extend lineage, add immutable results, repair policy semantics, add RPC
`docs/arch/04-database-schema.md`	Canonical schema and immutability contract
`docs/arch/09-learning-loop.md`	Discovery -> OOS validation -> promotion sequence
`public/agent-diagrams/system-map.json`	Remove all current walk-forward/DSR overclaims
```

## ## Failure Modes And Required Refusals

```
Failure	Required outcome
Current-day universe replayed historically	refuse `universe_not_point_in_time`
Feature observed/revised after as-of date	refuse `input_not_point_in_time`
Formula/config changed after plan creation	refuse `plan_fingerprint_mismatch`
Label crosses test boundary or is immature	refuse `label_overlap_or_immature`
Fold/test range overlaps another result	refuse `test_fold_overlap`
Too few dates or symbols	refuse `sample_floor_not_met`
Experiment segment differs from request	refuse `experiment_scope_mismatch`
Trial count absent or lower than family ledger	refuse `trial_family_incomplete`
Any India/US mixing	refuse `market_scope_mismatch`
Promotion insert/bind fails	transaction rolls back; incumbent remains active
```

## ## Validation Plan

### ### Pure tests

- US and India session calendars, including holidays
- horizon-label purge at every fold boundary
- no overlap in test origins or label intervals
- immature labels rejected
- exact rerun idempotency
- changed formula/universe/config creates a different plan
- sparse fold rejection
- aggregate HAC statistic uses chronological OOS observations
- no max-t or latest-fold cherry-pick
- malformed/mismatched trial family fails closed

### ### Database tests

- anon/authenticated cannot read or mutate plan/results
- result rows cannot update/delete
- experiment and policy bindings are one-way
- concurrent promotion leaves exactly one active policy
- forced insert failure preserves the incumbent
- null/value segment branches match the unique-index key exactly

### ### Acceptance

1. A synthetic known-positive factor passes only under clean PIT OOS data.
2. The same factor fails when its universe or labels are contaminated.
3. A known-null factor does not pass under repeated trial families.
4. No policy can be created from legacy `edge\_ic\_history` alone.
5. Full typecheck, tests, build, schema verification, RLS checks, and production dry-run pass before activation.

## ## Build Order



1. Approve this architecture and thresholds/power analysis.
2. Disable legacy-window promotion explicitly while the policy table is empty.
3. Repair policy/experiment schema semantics and add the atomic RPC.
4. Establish PIT universe and input eligibility for US and India separately.
5. Extend the existing experiment lineage and add immutable OOS results.
6. Build the session-aware fold engine and synthetic leakage tests.
7. Backfill diagnostics only; never relabel legacy rows as OOS.
8. Run paper-only validation plans and review at least one complete trial family.
9. Wire the promotion route to the atomic RPC.
10. Consider policy consumption only under a separate approved architecture.

#### ## Open Decisions For Approval

1. Minimum dates and symbols per fold, determined by power analysis rather than an arbitrary `fold\_days`.
2. Expanding-window calibration versus externally fixed formula mode.
3. False-discovery procedure for correlated edge families.
4. Required cost-adjusted portfolio test and benchmark.
5. PIT-universe source and delisting/corporate-action policy for each market.

#### ## References

- Bailey and Lopez de Prado, \*The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting and Non-Normality\*: <https://ssrn.com/abstract=2460551>
- Bailey, Borwein, Lopez de Prado, and Zhu, \*The Probability of Backtest Overfitting\*: <https://ssrn.com/abstract=2326253>
- Harvey, Liu, and Zhu, \*... and the Cross-Section of Expected Returns\*: <https://www.nber.org/papers/w20592>

These sources support multiple-testing and backtest-overfitting controls. The specific Kairos schema, market isolation, PIT requirements, and fail-closed design above are architectural decisions derived from the current code and production schema.

---

#### # Annex A – Open Decisions #1 and #5: options with arithmetic (2026-07-28)

Prepared for owner decision. Nothing here is approved or implemented.

#### ## Back-derived diagnostics (not direct sigma measurements)

An effective dispersion proxy backed out of Kairos's own rows, using `sigma = mean\_ic \* sqrt(as\_of\_dates) / t\_stat` on the latest window of each market-wide edge/horizon. Because `t\_stat` uses a HAC standard error, this proxy mixes sample dispersion with serial covariance and is not a direct sample sd:

| Market | Horizon | Universe | As-of dates | Implied sigma_IC | Mean abs IC | Best abs IC |
|--------|---------|----------|-------------|------------------|-------------|-------------|
| US     | 5       | 40       | 96          | 0.318            | 0.0192      | 0.0403      |
| US     | 10      | 40       | 96          | 0.364            | 0.0279      | 0.0625      |
| US     | 20      | 40       | 96          | 0.438            | 0.0308      | 0.0612      |
| India  | 5       | 39       | 120         | 0.222            | 0.0133      | 0.0325      |
| India  | 10      | 39       | 120         | 0.264            | 0.0127      | 0.0461      |
| India  | 20      | 39       | 120         | 0.307            | 0.0203      | 0.0561      |

The theoretical cross-sectional rank-IC standard deviation for `n` independent names is approximately  $1/\sqrt{n-3}$ ; at  $n=40$  that is  $0.164$ . US h20 shows  $0.438$ , an effective inflation of  $\sim 2.7\times$ . Label overlap is a plausible contributor, but the proxy does not identify how much comes from overlap, serial covariance, factor structure, universe construction, or regime concentration.

#### ## Decision #1 – sample floors

Required as-of dates for a target IC at hurdle T:

$$N = (T * \sigma_{IC} / IC_{target})^2$$

Two independent levers, and they are not equally priced.

**\*\*Lever 1 – remove label overlap\*\*** (`step\_days >= horizon`). If sigma fell from the 0.438 HAC-implied proxy to  $0.164$ , N would fall by  $\sim 7.1\times$ . But available dates/year falls from  $\sim 50$  to  $\sim 12.5$ , costing  $4\times$ . This is a sensitivity scenario, not a measured  $1.8\times$  net gain.

**\*\*Lever 2 – enlarge the universe.\*\***  $\sigma \sim 1/\sqrt{n-3}$ , and N scales with  $\sigma^2$ , so N falls *\*linearly\** in n. This costs **\*no calendar time at all\***.

| n   | sigma_IC | vs n=40          |
|-----|----------|------------------|
| 40  | 0.164    | 1.0x             |
| 100 | 0.101    | 2.6x fewer dates |
| 200 | 0.071    | 5.3x fewer dates |

| 400 | 0.050 | 10.7x fewer dates |

Years of history required, non-overlapping (step=horizon=20, ~12.5 dates/year),  
at T=2.0:

| True IC              | n=40    | n=100   | n=200   | n=400   |  |
|----------------------|---------|---------|---------|---------|--|
| --- ---              | --- --- | --- --- | --- --- | --- --- |  |
| 0.03                 | 9.6 yrs | 3.6 yrs | 1.8 yrs | 0.9 yrs |  |
| 0.05                 | 3.4 yrs | 1.3 yrs | 0.7 yrs | 0.3 yrs |  |
| 0.06 (best observed) | 2.4 yrs | 0.9 yrs | 0.5 yrs | 0.2 yrs |  |

At T=3.0 (Harvey/Liu/Zhu for newly proposed factors) multiply every cell by 2.25.

**\*\*The conclusion is blunt: at n=40 nothing in the current registry is provable this decade. Universe size is the dominant lever and it is free.\*\***

### Options

| Option  | Floors                                                           | Consequence        |                                                                                                            |
|---------|------------------------------------------------------------------|--------------------|------------------------------------------------------------------------------------------------------------|
| --- --- | --- ---                                                          | --- ---            |                                                                                                            |
| **1A    | Keep n=40**                                                      | any                | Rejected. Even IC=0.06 needs 2.4 yrs at T=2.0 and 5.4 yrs at T=3.0. Weak edges are permanently unprovable. |
| **1B    | n>=100, 12 dates/fold, 3 folds**                                 | 36 dates, ~2.9 yrs | Detects IC>=0.05 at T=2.0. Marginal at T=3.0.                                                              |
| **1C    | n>=200 US / n>=100 India, 12 dates/fold, 3 folds (RECOMMENDED)** | 36 dates, ~2.9 yrs | Detects IC>=0.035 at T=2.0 and IC>=0.05 at T=3.0.                                                          |
| **1D    | n>=400, 8 dates/fold, 3 folds**                                  | 24 dates, ~1.9 yrs | Strongest, but 400 liquid US names stretches "liquid" and multiplies fetch cost                            |

Recommended floors under 1C: `min\_symbols\_per\_as\_of\_date` 200 US / 100 India;  
`min\_as\_of\_dates\_per\_fold` 12; `min\_folds` 3; `step\_days` = horizon; declared IC  
floor \*\*0.04\*\*, replacing the current 0.02 – which the table shows is below the  
detection limit at any realistic sample.

**\*\*Verify before adopting:\*\*** the system map records Yahoo at "251 bars" while  
`edge\_ic\_history.history\_days` is 1000. 1C needs ~3 years of daily candles for  
200 US names. If the 251-bar cap is real, deeper history must be sourced first  
and 1C is blocked until then.

## Decision #5 – point-in-time universe

The current list is a hand-picked, current-liquid snapshot  
(`lib/edges/universe.ts` says so in its own header). Replaying today's survivors  
through past dates is survivorship bias, and it cannot be fixed by waiting.

| Option  | Survivorship fixed?                     | Cost                 | Notes                                           |                                                                                     |
|---------|-----------------------------------------|----------------------|-------------------------------------------------|-------------------------------------------------------------------------------------|
| --- --- | --- ---                                 | --- ---              | --- ---                                         |                                                                                     |
| **5A    | Massive PIT tickers (RECOMMENDED, US)** | Yes                  | MCP already connected; plan limits unverified   | `GET /v3/reference/tickers` takes `date`                                            |
| **5B    | Reconstructed liquidity screen**        | **No**               | Free                                            | Rank by trailing dollar volume using only data <= as-of date, take top N. Removes * |
| **5C    | Freeze-forward (run in parallel)**      | Yes, by construction | Free, already running                           | `edge_universe_members` has snapshotted since 2026-07-28                            |
| **5D    | Index-membership history**              | Yes                  | Free-ish for NIFTY via NSE archives; S&P harder | Manual/scraped, ongoing maintenance. Fallback for                                   |

Recommended: **\*\*5A + 5B for US\*\*** (PIT membership, then a PIT liquidity rank  
within it), **\*\*5D for India\*\*** if Massive's NSE coverage is thin, and **\*\*5C** running  
in parallel indefinitely\*\* as the unimpeachable ground truth that eventually  
supersedes the reconstruction. Any disagreement between 5C and 5A/5B once they  
overlap is a bug in the reconstruction and must fail closed.

**\*\*Verify before adopting 5A:\*\*** Massive plan limits for `/v3/reference/tickers`  
with `active=false`, and whether NSE/India is covered at all. Both are unknown.

## What these two decisions do NOT settle

Even at 1C + 5A, the binding constraint stays the t-hurdle. Best current  
latest-window t is 1.73 (US) / 1.04 (India). A larger universe and clean PIT  
data change the \*precision\* of the estimate, not its \*sign\* – they make a real  
edge provable and a fake edge refutable. Neither manufactures an edge that is  
not there.

---

# Annex B – Verification of the two unknowns (2026-07-28)

Both open questions from Annex A were tested against the live providers. One  
blocker is real, one dissolved, and a third problem surfaced that Annex A missed.

## B1. Candle depth – RESOLVED, and the fix already exists

**\*\*Massive (Polygon-backed) has a hard 2-year lookback wall on the current plan.\*\***

- `/v2/aggs/ticker/AAPL/range/1/day/2022-07-01/2026-07-27` with `limit=5000`  
returned exactly **\*\*500 bars, 2024-07-29 → 2026-07-27\*\***.
- Requesting only the older window `2022-07-01 → 2023-07-01` returned  
**\*\*HTTP 403 `NOT\_AUTHORIZED` – "Your plan doesn't include this data timeframe."\*\***

This matters because `resolveCandles()` routes **\*\*US\*\*** through Massive first  
(`lib/edges/data.ts`), so US IC history is capped at ~2 years. Net of a 252-day  
feature lookback (12-1 momentum), that leaves only ~12 usable non-overlapping

20-day as-of dates. 1C would have been unbuildable on US.

**\*\*However** – Yahoo's auth-free chart endpoint serves 5 years for BOTH markets, free, and Kairos already has the code.**\*\* Measured:**

```
Symbol	range=5y	range=3y
AAPL	1254 bars, from 2021-07-28	751 bars, from 2023-07-28
RELIANCE.NS	1239 bars, from 2021-07-27	742 bars, from 2023-07-27
```

``fetchIndiaCandles(symbol, range)`` in ``lib/india-data.ts`` is a **\*\*generic Yahoo chart fetcher\*\***. It is India-only by naming accident, not by capability – it takes any symbol and any range, and ``indiaRange()`` already maps `>900d` → ``3y`` and `>1400d` → ``5y``. The 251-bar figure in the system map is simply the 1y default.

**\*\*SHIPPED 2026-08-18 (owner-approved).** ``fetchYahooCandles`` is now the FIRST source for both markets in ``resolveCandles()`` (``lib/edges/data.ts``), lifting the 2-year Massive ceiling on US 1C history. ``fetchUsCandles()`` – the main research path, which has its own local 1y Yahoo fetcher – is untouched, so live scoring inputs did not move. No new provider, no new key, no cost.

Two effects, both intended:

- **\*\*Depth:\*\*** US 1C history goes from ~2y (~12 usable non-overlapping h20 as-of dates, below the 12/fold floor) to 5y (~50), which is what makes 1C buildable.
- **\*\*Budget:\*\*** EdgeScout no longer cascades into EODHD's 20/day free tier. Measured 2026-08-17 before the change: Massive 12 symbols, EODHD 20 (its cap), TwelveData 24, remainder ``unavailable`` – because Massive per-symbol candles are paced at 12.5s (5/min).

**\*\*Discontinuity** – read before comparing 1C across the boundary. ``edge_ic_history`` rows written from 2026-08-18 are computed on Yahoo bars where earlier rows used Massive/EODHD/TwelveData. The per-run ``providerCounts`` (``lib/edges/ic.ts``) records which, so the change is attributable rather than silent, but rows either side are NOT like-for-like. ``lib/gates/promotion-gate.ts`` reads this table over a 1000-day window, so a promotion evaluated across the boundary mixes sources – segment by ``providerCounts`` before treating a change in 1C as signal.

Recomputed 1C feasibility with 5 years (~1254 sessions, minus a 252-day feature lookback, non-overlapping at h20 → ~50 as-of dates):

```
n	Detectable 1C at T=2.0	at T=3.0
100	0.029	0.043
200	0.020	0.030
```

**\*\*1C becomes fully buildable:** 3 folds x 16 dates, comfortably above the 12/fold floor, with the proposed 0.04 1C floor safely above the detection limit.**\*\***

**## B2. Massive PIT coverage – CONFIRMED for US, ABSENT for India**

**\*\*US: works.\*\*** ``GET /v3/reference/tickers`` with ``date=2023-06-30`` and ``active=false`` returned genuine delisted names with ``delisted_utc`` timestamps (e.g. Altaba, delisted 2019-10-07; Advanced Accelerator Applications, 2018-02-12). This is a real point-in-time membership source with survivorship intact, already connected, no new vendor.

**\*\*India: not covered.\*\*** Searching ``RELIANCE`` returned only US-listed and OTC instruments – Reliance Global Group (NASDAQ), Reliance Inc (NYSE), and OTC GDRs. No NSE listings. Massive is US-equities-only for this purpose.

**## B3. Consequence Annex A did not anticipate**

The two markets have **\*\*opposite\*\*** gaps:

```
	PIT universe source	Deep candle history
US	5A Massive – **yes**	Massive capped at 2y; **Yahoo 5y fixes it**
India	Massive – **no**, needs 5D	Yahoo – **yes, 5y**
```

Neither market has both from one provider. The recommended pairing is therefore **\*\*asymmetric by necessity\*\***, not by preference:

- **\*\*US:\*\*** 5A (Massive PIT tickers) for membership + Yahoo 5y for candles + 5B liquidity rank within the PIT set.
- **\*\*India:\*\*** 5D (NSE index-membership archives) for membership + Yahoo 5y for candles. Until 5D exists, India has **\*\*no\*\*** PIT universe and therefore cannot produce promotion-grade evidence at all – it stays diagnostic-only.
- **\*\*Both:\*\*** 5C freeze-forward continues in parallel as ground truth.

This means US reaches promotable evidence materially before India. That is an honest consequence of the data available, not a choice – and it should be recorded rather than papered over by letting India promote on a survivorship-biased universe.

**## Revised recommendation**

Adopt **\*\*1C + 5A/5B (US) + 5D (India, blocked until built)\*\***, with the Yahoo deep-history change as a prerequisite for both. Do not relax the fold floors to accommodate the 2-year Massive wall – the wall is removable for free, and lowering statistical floors to fit a provider limit would be exactly the kind of accommodation this whole feature exists to prevent.

**## B4. Why the brokers (Robinhood / Webull / Kite) do not solve either problem**

Asked during review: can the broker APIs supply what is missing?

**\*\*For #5 (PIT universe) – structurally impossible, not a plan limitation.\*\***  
A broker API lists instruments you can trade *\*today\**. That is the literal definition of a survivorship-biased universe. Robinhood has no reason to serve a name that delisted in 2019, and does not. ``active=false`` on the Massive reference endpoint exists precisely because a *\*market-data\** vendor has an incentive to retain dead tickers, while a *\*broker\** has none. No broker integration can fix survivorship bias, regardless of entitlement or plan.

**\*\*For #1 (candle depth) – possible in principle, unnecessary in practice, and poorly shaped for it.\*\***  
- Broker historicals are entitled and rate-limited for a personal account, not for a 200-symbol nightly backfill across 5 years.  
- Robinhood MCP requires an interactive OAuth session; the token is not available to unattended validation jobs on the same terms as an open HTTP endpoint.  
- Webull's data tools are recorded in this repo as requiring ``category:US_STOCK``, returning quarterly fractions, being cron-entitled, and the adapter is currently dead.  
- Kite is India-only and account-scoped.  
- Yahoo already supplies 5 years for both markets, free, keyless, with existing code (B1). Adding a broker dependency to solve a solved problem would trade a keyless endpoint for an entitled, rate-limited, auth-bound one.

**\*\*Conclusion:\*\*** the brokers stay what they are – execution and live account state. They are not a research-data source, and routing validation evidence through them would couple the evidence layer to trading entitlements. The existing separation is correct and should not be relaxed.

---

**# Annex C – Correction to Annex B, and the step-4 scope boundary (2026-07-28)**

**## C1. Annex B overstated the Massive dependency – corrected**

Annex B said `"`resolveCandles()` routes US through Massive first, so US IC history is capped at ~2 years."` That is true **\*\*only of the edge/IC path\*\***. It is not true of US candles generally, and the difference matters.

There are two separate US candle resolvers:

| Resolver                        | Used by                               | US order           | Depth                                                                           |
|---------------------------------|---------------------------------------|--------------------|---------------------------------------------------------------------------------|
| <code>`resolveCandles()`</code> | in <code>`lib/edges/data.ts`</code>   | edge lab / IC      | Massive → EODHD → TwelveData   <b>**capped at 2y by the Massive plan**</b>      |
| <code>`fetchUsCandles()`</code> | in <code>`lib/data/candles.ts`</code> | main research path | <b>**Yahoo first**</b> , then Massive → EODHD → TwelveData → AV   governed by t |

So the main US path was already Yahoo-first. Only the edge/IC path – the one this feature depends on – goes to Massive first and hits the 2-year wall. The Annex B conclusion for step 4 stands; the general claim about "US candles" did not, and is withdrawn.

**## C2. Correction: the live US candle path is not under-served**

The earlier text described a stale API shape. ``lib/data/candles.ts`` owns a local ``fetchYahooCandles(symbol)`` that explicitly requests ``range=1y``, returning roughly 251 sessions. It is not the market-agnostic deep-history function with a range argument.

The production technical score uses RSI14, EMA20/50, ATR14, 20-day trend, and 20-day volume. One year is sufficient for those inputs. ``mom_12_1`` is an edge lab formula and uses the separate OOS candle resolver with explicitly requested deep history; it is not computed by the live ResearchAgent technical score.

Therefore there is no pending live candle-depth fix and no reason to change production scoring inputs. The system-map ``YAH00`` node now records the actual one-year path and the separate deep-history role.

**## C3. Step 4 is NOT unblocked by the 1C + 5A/5B approval alone**

Owner approved **\*\*1C\*\*** (sample floors) and **\*\*5A/5B\*\*** (PIT universe source) on 2026-07-28. Those answer Open Decisions **\*\*#1\*\*** and **\*\*#5\*\***.

Three remain open, and the approved scope line at the top of this document says steps 4-10 are blocked on **\*\*all five\*\***:

| # | Decision | Blocks |

```

|---|---|---|
| 2 | Expanding-window calibration vs externally fixed formula | Fold construction – whether a fold has a train segment at all, and therefore
| 3 | False-discovery procedure for correlated edge families | Aggregate pass criteria (step 8) |
| 4 | Required cost-adjusted portfolio test and benchmark | Final promotion criteria (step 9) |

```

**\*\*#2 genuinely blocks step 4.\*\*** Whether folds carry a training segment determines the fold boundary layout, the purge rule, and which ``validation_mode`` value the schema will accept – all of which step 4 must fix before any universe snapshot is keyed to a plan.

**\*\*#3 and #4 do not block step 4.\*\*** They gate steps 8-9. Step 4 could proceed once #2 is answered.

Recommendation: answer #2 before step 4 starts; #3 and #4 can be decided while steps 4-7 are built.

---

# Annex D – Open Decision #2: calibration mode (2026-07-28)

Recommendation: **\*\*`purged\_temporal\_oos`\*\***. Not a preference – the codebase already determines it.

## Evidence

**\*\*Nothing in the edge pipeline is fitted from data.\*\***

```

Check	Result
`grep -c 'fit	train
Edge formulas	Closed-form with hardcoded constants – `mom_12_1` reads `c[n-22] / c[n-253] - 1`; MACD is fixed 12/26/9; ADX is fixed 14; 52w
`crossSectionalZ` / `winsorize` (`lib/edges/standardize.ts`)	Computed **per (date, market, edge) across symbols** – purely cross-sectional
`kairos_technical_score_v1` → `scoreTechnicals`	Hand-set integer constants (+15 EMA50, +10 EMA20, ±25 RSI anchors), authored by a human.

```

A fold cannot have a training segment when there is nothing to train. Declaring ``walk_forward`` would mean building train/test boundaries, a purge rule against the train edge, and a refit step that would all be inert – machinery that implies a protocol the system does not actually run. That is the same class of overclaim as the ``walk_forward_pass`` field this feature already renamed.

## What this does NOT excuse

**\*\*Hand-set constants are still researcher degrees of freedom.\*\*** ``+15`` for EMA50 was chosen by a person who had seen this market. That is selection bias – but it is bias in the **\*\*trial family\*\***, which Open Decision #3 (false-discovery procedure) governs, not bias the fold protocol can repair. No train/test split fixes a constant a human tuned by eye before the split existed.

Two consequences to carry into the frozen plan:

1. The trial family must count human tuning history, not just machine variants. ``variants_run`` currently counts engine runs only.
2. **\*\*`kairos\_technical\_score\_v1` is not a priored factor.\*\*** ``mom_12_1``, ``high_52w_proximity`` and ``vol_adj_mom_6m`` carry published citations (Jegadeesh-Titman, George-Hwang); the in-house composite does not. The ``T_HURDLE = 2.0`` comment justifies itself as "pried-factor standard", which is defensible for the published set and **\*\*not\*\*** for the composite. Harvey/Liu/Zhu argue ~3.0 for a newly proposed factor. Recommend a per-edge hurdle: 2.0 for cited priors, 3.0 for in-house constructions.

## Guard required

``purged_temporal_oos`` is correct **\*\*today\*\*** and must not silently persist if that changes. The genome (``entry_threshold``, ``entry.rank_pct_min``, the five dimension weights) **\*\*is\*\*** fitted – `LearnerAgent` mutates it from realized outcomes. It does not touch ``edge_ic_history`` today because IC is computed per raw edge and the genome acts downstream on ``analyst_score`` and `PaperTrader`. If a composite whose parameters come from `LearnerAgent` ever becomes a validated edge, that edge requires ``walk_forward`` and must not be allowed the simpler mode.

Step 4 should therefore record, per experiment, whether the formula carries any data-derived parameter, and the gate must reject ``purged_temporal_oos`` when it does. The ``validation_mode`` CHECK already accepts both values, so this is a plan-level assertion rather than a schema change.

## Decision requested

Approve ``purged_temporal_oos`` as the mode for the current registry, with:

- the per-edge t-hurdle split (2.0 cited / 3.0 in-house), and
- the data-derived-parameter guard forcing ``walk_forward`` when it ever applies.

This unblocks step 4. #3 and #4 remain open and gate steps 8-9 only.

---

# Annex E – Step 4 progress and a new constraint (2026-07-28)

Approved 2026-07-28: 1C sample floors, 5A/5B PIT universe, `purged\_temporal\_oos` mode, the 2.0/3.0 per-edge t-hurdle split, and the fitted-parameter guard.

## Shipped

`lib/edges/pit-universe.ts` – resolves which symbols were tradeable and liquid ON an as-of date, using only information knowable then. 17 tests.

It **fails closed** on every path, with a named reason and never a fallback to the curated current-liquid list:

```
Reason	When
`universe_not_point_in_time`	market ≠ us – Massive carries no NSE listings, so India has no PIT source until 5D
`liquidity_not_available_for_date`	as-of date outside the ~2y aggregate entitlement (checked before any network call)
`membership_incomplete`	the page walk truncated – see below
`liquidity_unavailable`	grouped aggregates returned nothing usable
`universe_below_min_symbols`	fewer eligible liquid names than the 1C floor
`provider_unconfigured`	no API key
```

Migration `20260728120000\_pit\_universe\_provenance.sql` **written, NOT APPLIED** (Supabase MCP unavailable at the time). It extends `edge\_universe\_members` rather than adding a parallel table: `is\_point\_in\_time` (default **false**, so every pre-existing row correctly declares itself non-PIT), `membership\_source`, `pit\_policy\_version`, `active\_on\_as\_of`, `delisted\_at`, `adv\_value`, `adv\_rank`, `snapshot\_fingerprint`. No shipped code reads these columns yet, so nothing is schema-coupled ahead of the migration.

## Defect caught during verification

The first implementation walked pagination and **break**ed on any failed page. A **real 429** was hit while verifying against the live provider – the membership set is ~10k tickers, so a full walk is ~10 requests and the plan rate-limits.

That would have produced a smaller universe that looked entirely valid: fewer names, a different fingerprint, a different IC, and no error anywhere. A truncated walk is not a smaller universe, it is an **unknown** one. `fetchPitMembership` now returns `{ tickers, complete, pages }` and the resolver refuses on `complete: false`. Two tests cover it.

## NEW CONSTRAINT – 1C's fold floors are not reachable yet

Measured entitlement boundary (verified 2026-07-28):

```
As-of date	Grouped daily aggregates
2026-07-24	OK – 12,410 tickers in one call
2024-10-15	OK – 10,704
2023-06-30	NOT_AUTHORIZED
```

Membership resolves at any date; **liquidity does not**. So the usable OOS window is ~2 years ≈ 500 sessions. Non-overlapping at h20 that is **~25 as-of dates**.

1C approved **33 folds × 12 dates = 36**. Short by ~11.

Options – I am **not** picking one, because Annex A explicitly recommended against relaxing statistical floors to fit a provider limit:

- 33 folds × 8 dates = 24**. Meets fold count, breaks the 12/fold floor.
- 32 folds × 12 = 24**. Meets the per-fold floor, breaks the 3-fold minimum.
- Wait**. Each further month adds ~1 as-of date at h20; reaching 36 takes ~11 months.
- Find another whole-market liquidity source** with deeper history. Yahoo is per-symbol, so ranking ~10k names per date is not viable through it; this needs a grouped/bulk endpoint.
- Rank liquidity once per fold** rather than per as-of date. Liquidity is slow-moving, so a fold-start rank is defensible and cuts the calls ~12x – but it does not extend the 2-year entitlement, so it does not add dates.

Note options 1 and 2 both land on 24 usable dates. The honest reading is that the current entitlement supports roughly **one** 1C-shaped experiment, not three folds at full width.

## Not done

- Migration not applied; persistence of snapshots to `edge_universe_members` not written (blocked on the migration).
- Input-eligibility enforcement beyond the universe (feature availability per as-of date) not started.
- Steps 5-10 unchanged. Promotion remains dormant.
- Open Decisions #3 (false-discovery) and #4 (cost-adjusted test) still open, gating steps 8-9.

---

# Annex F – The 25-vs-36 question was the wrong question (2026-07-28)

## The framing error

Annex E presented "~25 available vs 36 required" as a shortfall needing a relaxed floor. \*\*36 was never a statistical requirement.\*\* It was the arithmetic product of a fold partition (3 x 12) that Annex A chose without deriving.

Under the approved `purged\_temporal\_oos` mode there is \*\*no training segment\*\* (Annex D: nothing in the edge pipeline is fitted). So folds do not partition data for leakage control – they exist only as diagnostics: fold sign consistency and a worst-fold guard. The primary statistic is the aggregate HAC t-stat over the \*\*concatenated\*\* OOS date-level IC series.

Statistical power therefore depends on \*\*N total as-of dates\*\*, not on how they are partitioned. Asking "3x12 or 2x12 or 3x8" was asking about the wrong number.

## The number that actually binds

Detectable IC =  $T \cdot \sigma / \sqrt{N}$ , against the approved \*\*0.04\*\* IC floor:

| Hurdle                     | N needed to detect 0.04 | Available (~2y) | Verdict                          |
|----------------------------|-------------------------|-----------------|----------------------------------|
| --- --- ---                | ---                     | ---             | ---                              |
| **T = 2.0** (cited priors) | **12.6 dates**          | ~25             | **2x margin – sufficient today** |
| **T = 3.0** (in-house)     | **28.4 dates**          | ~25             | short by ~4 dates (~4 months)    |

The approved IC floor (0.04) sits \*\*above\*\* the detection limit at N=25 (0.028 at T=2.0). The floor binds first, not the sample size. That is the correct relationship – the floor should be the constraint, since it encodes what counts as economically meaningful rather than merely measurable.

## Recommendation

1. \*\*Fold layout: 3 folds x 8 dates = 24.\*\* Three folds preserves the sign-consistency diagnostic; 8 per fold is fine because per-fold t-stats are diagnostics, never the promotion statistic. No floor is being "relaxed to fit a provider limit" – the 12/fold figure was never derived.
2. \*\*Cited-prior edges proceed now\*\* (`mom\_12\_1`, `high\_52w\_proximity`, `vol\_adj\_mom\_6m`, `rel\_strength\_6m` – the ones carrying published citations).
3. \*\*In-house edges are refused until N >= 29\*\* – `kairos\_technical\_score\_v1` and friends carry the 3.0 hurdle per the approved split, and 25 dates cannot detect a 0.04 IC at that hurdle. ~4 months of accumulation, or they simply never qualify. Both are acceptable outcomes; silently running them at 25 dates is not.
4. \*\*Do not wait 11 months.\*\* That recommendation in Annex E was downstream of the 36-date error.

## THE ASSUMPTION THIS ALL RESTS ON – measure it first

Everything above uses `sigma = 0.071`, the \*theoretical\* cross-sectional rank-IC standard deviation at n=200 ( $1/\sqrt{n-3}$ ). The \*\*measured\*\* sigma on the current windows was previously described as \*\*0.438\*\*. That value is actually a HAC-implied proxy, not a sample sd, and its excess cannot be attributed to overlap alone.

The whole plan assumes that removing overlap (step = horizon) collapses sigma toward theoretical. Sensitivity:

| True sigma                       | N needed for IC=0.04 @ T=2.0 | Years @12.5/yr                       |
|----------------------------------|------------------------------|--------------------------------------|
| --- --- ---                      | ---                          | ---                                  |
| 0.071 (assumed)                  | 12.6                         | 1.0                                  |
| 0.090                            | 20.2                         | 1.6                                  |
| **0.100**                        | **25.0**                     | **2.0 – BREAK-EVEN vs what we have** |
| 0.150                            | 56.2                         | 4.5                                  |
| 0.200                            | 100.0                        | 8.0                                  |
| 0.438 (HAC-implied legacy proxy) | 479.6                        | 38.4                                 |

\*\*The recommendation holds only if realized sigma <= ~0.10.\*\* Above that, N=25 is not enough and the answer changes materially; at anything near 0.438 the entire IC-on-40-to-200-names approach is dead regardless of entitlement.

So the \*\*first output of the fold engine must be the realized sigma of the non-overlapping OOS IC series\*\*, reported before any promotion decision is evaluated. If it lands above ~0.10, stop and re-derive rather than proceeding on these floors. This is cheap to measure and it invalidates or confirms the plan in one run.

## Open Decisions status

#1 answered (1C). #2 answered (Annex D). #5 answered (5A/5B, US only). #3 (false-discovery) and #4 (cost-adjusted test) remain open, gating steps 8-9.

---

# Annex G – Fold engine shipped; migration was NOT applied (2026-07-28)

## Migration verification FAILED

Reported as applied. It is not. Verified through PostgREST with the app's own service-role credentials, with a control:

select=symbol -> [{"symbol":"SOXL"}] (table reachable)

select=is\_point\_in\_time -> 42703 "column ... does not exist" (NOT applied)

Same result for `pit\_policy\_version`, `snapshot\_fingerprint`, `adv\_rank`, `delisted\_at`. Migration `20260728120000\_pit\_universe\_provenance.sql` is in the repo and not in the database – exactly the failure mode the standing rule describes. Snapshot persistence remains **blocked** and no shipped code reads those columns.

## Shipped – `lib/edges/folds.ts`

Pure: no network, no database, no clock. 14 tests.

**buildPurgedFolds** lays out disjoint folds over market SESSIONS. Purge is structural, not advisory: fold `k+1` starts only after fold `k`'s last as-of date has had `horizonSessions` to mature, so no fold's label window can reach into the next fold's features.

Refusals, all fail-closed:

| Reason             | Why                                                                                                                                                                                                                                                             |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| step_below_horizon | step < horizon means consecutive as-of dates share forward-return windows – the exact defect that makes the legacy insufficient_sessions   Refuses rather than emitting a short final fold, which would be a smaller sample masquerading as a peer of the other |
| invalid_plan       | Non-positive horizon/fold/date/step.                                                                                                                                                                                                                            |

**validateFoldDisjointness** independently re-derives non-overlap from the indices. Per the architecture's "a boolean is not proof", the gate must recompute this rather than trust a flag the builder set about itself. It is tested against a hand-built violation the builder cannot produce.

**aggregateOosIc** returns the concatenated-series statistics, reporting `sigmaIc` and `sigmaWithinPlan` alongside `tHac`.

**neweyWestSEofMean** was exported from `lib/edges/ic.ts` rather than reimplemented – one Newey-West implementation, not two.

## The approved layout fits

3 folds x 8 dates at step = horizon = 20 needs **483 sessions**  $\approx$  1.9 years, inside the ~500 sessions the 2-year liquidity entitlement allows. Confirmed by test, not by arithmetic in a document.

Note **neweyWestLag(20, 20) = 1**: with non-overlapping as-of dates there is no mechanical autocorrelation left to correct. That is the point of step = horizon, and it is the mechanism by which sigma is expected to fall from the measured 0.438 toward theoretical.

## The stop condition is now executable

**SIGMA\_PLAN\_CEILING = 0.10** is in code with **sigmaWithinPlan** computed against it. Annex F made every approved floor conditional on realized sigma; that condition is no longer prose. A caller must check **sigmaWithinPlan** before treating **tHac** as meaningful, and both tests for it are present – one series inside the ceiling, one above it.

## Not done

- Snapshot persistence – blocked on the migration.
- The runner that fetches PIT universes and candles per as-of date and produces the per-date IC series the aggregator consumes.
- Steps 7-10. Promotion remains dormant.
- Open Decisions #3 and #4 still open, gating steps 8-9.

---

# Annex H – Migration applied and verified; OOS runner shipped (2026-07-28)

## Migration `20260728120000` – APPLIED, verified independently

Supabase MCP was unavailable, so it was applied through the Management API (`POST /v1/projects/{ref}/database/query`, HTTP 201). `urllib` was blocked by Cloudflare (403/1010); curl succeeded.

Verified afterwards through PostgREST with a control column, the same check that caught the earlier false "applied" report:

| Column                                                                                                                           | Result |
|----------------------------------------------------------------------------------------------------------------------------------|--------|
| symbol (control)                                                                                                                 | OK     |
| is_point_in_time, membership_source, pit_policy_version, active_on_as_of, delisted_at, adv_value, adv_rank, snapshot_fingerprint |        |



The honesty default behaved as designed: \*\*805 of 805 pre-existing rows are `is\_point\_in\_time = false`, zero true.\*\* Every legacy row now declares itself a survivorship-biased snapshot rather than passing as evidence.

## Shipped – `lib/edges/oos-runner.ts`

Turns a fold plan into the per-as-of-date IC series. 12 tests.

The ordering discipline is the entire point, and it is tested rather than asserted:

```
Element	Sees
feature	candles `<= asOf` only
label	`asOf` → `asOf + H`, and only if fully matured
universe	PIT membership resolved AT `asOf`
```

A spy edge confirms it: with monotonically rising series, the values the edge observes at `day(1)` are the SECOND close of each symbol, never the fifth. Had the full series leaked in, every date would have produced the same value.

Refusals, all counted rather than silently dropped:

- `cross\_section\_below\_min` – a sparse date is excluded and recorded. Coercing it to IC = 0 would assert "no predictive power" when the truth is "unmeasurable".
- `ic\_not\_finite` – a degenerate cross-section (zero label variance) is refused for the same reason. Found by a test whose own fixture was accidentally degenerate; the guard was already correct.
- `universe\_unavailable` – `datesEvaluated + datesSkipped` always equals the planned date count, so no date can vanish unaccounted for.

Labels that have not matured by the data cutoff return null rather than a truncated return: a partially matured label is a shorter horizon wearing the same name, and mixing the two silently changes what the IC measures.

## The stop condition is executable end to end

`describeSigma()` renders the Annex F verdict in words, including the dates that \*would\* be required at the realized sigma. At the legacy 0.438 it emits:

```
> STOP: sigma=0.4380 EXCEEDS the 0.10 plan ceiling. Detecting the approved 0.04
> IC floor at t=2.0 would need ~480 as-of dates, not 24.
```

Both branches are tested.

## Not done

- Snapshot persistence to `edge\_universe\_members` (schema is now ready).
- The fetch orchestration: resolve PIT universes across the 24 as-of dates, fetch the union of symbols' 5y candles once each, and call `runOosFolds`. This is the step that finally produces the real sigma number.
- Steps 8-10. Promotion remains dormant.
- Open Decisions #3 and #4 still open, gating steps 8-9.

---

# Annex I – PRELIMINARY SIGMA MEASUREMENT (2026-07-28)

First end-to-end OOS harness run. \*\*Verdict: useful diagnostic, not promotion-grade evidence and not sufficient to void the approved plan.\*\*

## Run configuration

`mom\_12\_1` (Jegadeesh-Titman, a cited prior), US, horizon 20 sessions, step = horizon, 2 folds x 6 dates = \*\*12 as-of dates\*\*, top-200 PIT universe, min cross-section 100, 5y Yahoo candles, as-of dates confined to the 2-year liquidity window. 255 symbols fetched, 5 missing. \*\*Zero universe errors, zero skipped dates\*\* – every planned date produced an IC.

Approximation used: `per\_fold` membership cadence (2 walks instead of 12), so this is not fully point-in-time. The run also ranked liquidity from one session rather than a trailing ADV window and used raw Yahoo closes rather than corporate-action-adjusted prices. Those are material methodology limitations, not footnotes.

## Result

```
Statistic	Value
n (as-of dates)	12
mean IC	**0.0335**
realized sigma	**0.1436**
Newey-West SE	0.0419
t_HAC	**0.80**
```

```
| NW lag | 1 (a configured HAC choice; not justified by overlap alone) |
| fold signs | [+1, +1] – consistently positive |
```

The saved artifact contains only this aggregate, not the 12 per-date IC values. The aggregate therefore cannot be independently recomputed from the artifact. The runner now emits the complete per-date series; this preliminary run must be rerun before its sigma is used.

## What the run does and does not show

The legacy `**0.438**` value was back-derived from ``mean_ic * sqrt(n) / t_stat``. Because that t-stat used a HAC standard error, 0.438 is not a directly measured sample standard deviation and is not comparable one-for-one with 0.1436. The claim that removing overlap produced a 3.1x sigma reduction is withdrawn.

For the 12 preliminary observations, the sample standard deviation is 0.1436. A textbook chi-square interval under independent normal observations is [0.1017, 0.2437], but cross-sectional ICs are not established to be independent normal draws here. Its lower endpoint clears 0.10 by only 0.0017, so that model assumption decides the conclusion. “Confidently above 0.10” is not supported.

The difference from the iid heuristic may reflect factor structure, universe construction, corporate actions, regime concentration, or sampling error. This run does not identify the cause. The earlier factor-structure attribution is withdrawn until it is tested.

## What it costs

Conditional planning arithmetic at sigma = 0.1436:

```
IC floor	T = 2.0	T = 3.0
0.04 (approved)	**51.5**	115.9
0.05	33.0	74.2
0.06	22.9	51.5
0.08	12.9	29.0
```

These values are arithmetic conditional on a stable sigma, independent effective dates, and the stated hurdle. They are not a new sample-size contract.

The preliminary ``mom_12_1`` point estimate does not pass the promotion hurdle: mean IC 0.0335 and `t_HAC` 0.80. That is a diagnostic refusal, not proof that the edge is ineffective or unreachable under a corrected, adequately sized design.

## Honest reading

The fail-closed outcome is right: promotion remains dormant. The stronger claims that the machinery is fully correct, the overlap hypothesis is confirmed, or no realistic promotion path exists are not established by this run.

## Options, none chosen

1. Rerun with adjusted prices, full per-date output, and the latest eligible session window.
2. Replace one-day dollar volume with a trailing PIT liquidity measure.
3. Use per-date membership for any evidence that could reach promotion.
4. Predeclare the HAC lag/sensitivity analysis and minimum number of dates.
5. Only then compare universe sizes, horizons, and edge families under immutable experiment lineage.

## Immediate consequence

The observed point estimate exceeds ``SIGMA_PLAN_CEILING = 0.10``, so it raises a planning warning. It does `**not**` void the approved floors. `**Promotion stays dormant**` because the run is approximate, too small, not independently recomputable, and not fully PIT.

Open Decisions #3 and #4 remain open, but they are moot until the sample-floor question is resolved.

---

# Annex L - Immutable bounded OOS runs (2026-07-29)

The first two runs under the immutable manifest contract completed in production. Both used code ``cb538a77b8572ba76739affc321cec9990ddb27``, adjusted candles bounded at ``2026-07-28``, persisted per-date PIT membership, and no approximations. They are evidence collection only. Promotion remains dormant.

```
Experiment	Variant	Dates	Mean IC	Sigma	HAC t	Universe errors
`d61c4990-f03a-4b0d-919f-a3d4e604428b` h5	n=200	3	-0.1934	0.4404	-0.93	0
same dates/snapshots	n=400	3	-0.1678	0.4323	-0.82	0
`b7a1eba5-70eb-4d48-8f76-6e16582b431b` h20	n=200	3	-0.0902	0.4681	-0.44	0
```

The matched h5 result does not support the claim that increasing the cross-section from 200 to 400 will normalize temporal IC dispersion: sigma changed by only 0.0081 on the same dates. The direct h20 result is also noisy, so h5 dispersion must not be transferred to h20. Three dates are far below a decision-grade sample and the recent negative means do not establish that the factor is permanently dead. The correct next action is to accumulate more predeclared PIT dates and test sector-neutral IC before any strategy-return or promotion work.

Annex K's corrected 80-date raw-rank rerun later measured pooled h5 sigma `0.2639`. This does not conflict with the three-date `0.4404`: the latter is a very wide recent-window estimate. The rerun does not support winsorization as the explanation, although the old artifact lacks enough universe provenance for a clean method-only comparison.

Plan fingerprints:

- h5 matched family:  
`43775a8d831304c5fc3552e3c2d3c3b134e9ad4bd6f0052bc4b4df8eb3bb64ae`
- h20 family:  
`59d2f383e546bd2e27f718a35f0206fc83eec004100679ec843c6ef94b621ae0`

---

# Annex J – 2026-07-28 adversarial review blockers

The following are hard blockers before promotion can be enabled:

1. **\*\*Experiment lineage implementation (completed 2026-07-29).\*\***  
`backtest\_experiments` now binds `edge\_id`, evidence horizon, formula version, trial family, universe policy, cutoff, code version, the canonical immutable plan hash, and write-once realized universe/dataset/run hashes. `runBoundOosVariant()` constructs the provider run from that manifest and enforces the data cutoff. The dormant promotion boundary still needs to independently verify these identities before it may ever be enabled.
2. **\*\*Liquidity is not trailing ADV.\*\*** `resolvePitUniverse()` currently ranks one session's adjusted close times volume. The approved contract says trailing PIT dollar volume. An event-volume spike can therefore enter the universe and alter IC/sigma. Build a cached trailing window or rename and reapprove the selection policy; do not present the current rank as ADV.
3. **\*\*Promotion evidence requires per-date membership.\*\*** `per\_fold` is a quota approximation and must remain diagnostic-only. No promotion writer may accept a report carrying that approximation.
4. **\*\*HAC lag selection is not derived.\*\*** `neweyWestLag(20, 20) = 1` is a policy choice, not a consequence of non-overlapping labels. Predeclare lag selection and report lag sensitivity before setting a statistical gate. Newey and West explicitly treat lag selection as an estimation problem, and note finite sample size distortions in their simulations.
5. **\*\*Fold partition is conditionally irrelevant only.\*\*** Aggregate mean/sigma on a fixed concatenated series do not depend on fold labels, but purge width, entitlement depth, fold consistency, and worst-fold gates change the usable dates and decision. The earlier unconditional wording is withdrawn.

Primary references:

- Newey, W.K. and West, K.D. (1994), "Automatic Lag Selection in Covariance Matrix Estimation," *Review of Economic Studies* 61(4), 631-653, <https://doi.org/10.2307/2297912>
- Bailey, D.H., Borwein, J., López de Prado, M., and Zhu, Q.J. (2015), "The Probability of Backtest Overfitting," *Journal of Computational Finance*, <https://ssrn.com/abstract=2326253>

---

# Annex K – Multi-period h5 dispersion diagnostic (corrected 2026-07-29)

The original run applied 2%-tail winsorization before Spearman. That was unnecessary because Spearman already ranks observations, and clipping creates artificial ties. The design was rerun with raw finite values under immutable diagnostic experiment `2b7cf834-b516-48bd-9533-d2511229e7f6`, fixed cutoff `2026-07-28`, persisted fold-anchor snapshots, and code `88c963f14ef6e8651076055b774b10628b844771`.

The corrected run measured pooled sigma **\*\*0.2639\*\***, close to the old 0.2665. This does not support winsorization as the explanation for Annex L's three-date sigma of 0.4404; the shorter, recent date sample is the material design difference. This is not a clean one-variable comparison because the old artifact omitted its members and its four universe fingerprints differ from the newly persisted trailing-ADV snapshots. The permanent closure and `n\_eff = 17` inference remain superseded in `PROJECT\_DECISIONS.md`.

Design: `mom\_12\_1`, US, **\*\*horizon 5\*\*** (step 5), 4 folds x 20 dates = **\*\*80 as-of dates\*\***, universe **\*\*held at 200\*\***, `per\_fold` cadence, adjusted prices. 320 symbols, **\*\*0 universe errors, 0 skipped dates\*\***.

Horizon 5 rather than 20 because the ~2-year entitlement holds ~100 as-of dates

at h5 versus ~25 at h20 – four \*independent\* periods of 20 instead of four of 6. This measures h5 temporal dispersion. It does not estimate h20 dispersion or decompose the sources of variance.

## Result 1 – h5 dispersion is high and varies across folds

| Fold | Period                   | mean IC | sigma  |
|------|--------------------------|---------|--------|
| 0    | 2024-12-13 .. 2025-05-05 | -0.0124 | 0.2286 |
| 1    | 2025-05-13 .. 2025-09-29 | +0.0331 | 0.2254 |
| 2    | 2025-10-07 .. 2026-02-24 | -0.0069 | 0.2818 |
| 3    | 2026-03-04 .. 2026-07-21 | +0.0423 | 0.3235 |

\*\*Pooled n=80: mean IC 0.0140, sigma 0.2639, HAC t 0.51. Fold sigma spread: 0.2254-0.3235.\*\*

The four estimates provide a useful range, but they rise monotonically and each uses only 20 dates. They do not prove stationarity, and they cannot explain whether the movement comes from market regimes, changing cross-sections, membership cadence, or sampling noise. The earlier h20 estimates remain too small to settle h20 dispersion.

## Result 2 – effective breadth is not identified

Realized pooled sigma \*\*0.2639\*\* against a theoretical \*\*0.0712\*\* at n=200.

That comparison is diagnostic only.  $1/\sqrt{n-3}$  is not an estimator that can turn observed time-series variance of Spearman IC into independent-name count. Observed variance includes cross-sectional sampling error, time-varying true IC, changing membership, and data attrition. The raw artifact also omitted successful-date cross-section counts, so those causes cannot be separated retrospectively.

The previous  $n_{\text{eff}} \approx 17$  claim and the derived 180-date requirement are withdrawn. Test n=200 versus n=400 on the same dates and memberships before claiming how dispersion scales. This is a measure-only diagnostic, not a reason to enable promotion.

Statistical basis for the correction:

- Ornstein and Lyhagen (2016), \*Asymptotic Properties of Spearman's Rank Correlation for Variables with Finite Support\*, derives dependence-specific asymptotic variance rather than treating the independence-null variance as a general effective-sample-size estimator:  
<https://doi.org/10.1371/journal.pone.0145595>
- Moskowitz and Grinblatt (1999), \*Do Industries Explain Momentum?\*, documents a strong industry component in stock momentum. That supports measuring a sector-neutral specification; it does not justify predicting its result without running it: <https://doi.org/10.1111/0022-1082.00146>

## Result 3 – `mom\_12\_1` shows no signal at h5

Fold signs \*\*[-1, +1, -1, +1]\*\* - alternating across four independent periods. Pooled mean IC \*\*0.0140\*\*, t\_HAC \*\*0.51\*\*.

Not weak-but-positive. No consistent sign at all. (The h20 runs were [+1, +1], so this is horizon-specific and does not condemn the factor at 20 sessions – but it does mean h5 carries nothing.)

## What this settles

1. `mom\_12\_1` is not useful at h5 in this sample.
2. Daily h5 IC is much noisier than an independent-name null benchmark.
3. The original fixed sample floors are not validated by this run.
4. Promotion must remain disabled.

## What it does NOT settle

- h20 sigma or power; h5 cannot be transferred to h20.
- Effective breadth or the contribution of correlation versus factor non-stationarity and changing coverage.
- Whether n=400 reduces sampling variance on matched dates.
- Whether point-in-time sector-neutral IC improves stability.
- Whether a \*different\* edge has signal. Only `mom\_12\_1` was run.
- `per\_fold` cadence remains an approximation – this is a planning measurement, \*\*not promotion evidence\*\*.

## Consequence

Promotion stays disabled. Continue only bounded, predeclared measure-only experiments. Record per-date cross-section and provenance, compare matched universes, and do not derive production thresholds from this single edge and horizon.

---

```

Source: features\webull-broker\FEATURE_ARCHITECTURE.md
Feature Architecture – Webull as 3rd Broker (MCP)

> Status: **Phase 1 (read-only, Cloud MCP) BUILT – OAuth connected; live read capture still fail-soft if a configured tool is not offered by
> Decision: **Cloud MCP (OAuth), not OpenAPI** – the OpenAPI REQUIRES an IP
> Whitelist (no free serverless has a static egress IP; would force an Oracle
> Free VM), whereas the MCP's OAuth is not IP-locked and runs from Vercel free.
> Reuses the Robinhood MCP OAuth machinery.
> Last updated: 2026-07-13
> **Refactored 2026-07-13 onto the config-driven MCP broker framework** – Webull
> is now a registry entry, not bespoke code (see "Config-driven framework" below).
> **Reviewer correction 2026-07-13:** the currently connected Codex Webull MCP
> surface exposes account/balance/position/order-history style read tools, but not
> the quote/research/order-placement tools listed in the original architecture.
> Treat quote/research/order tool names as unverified until `tools/list` in the
> deployed app proves them. The order adapter must remain unreachable.

Config-driven framework (2026-07-13 refactor)
The bespoke `lib/webull-mcp.ts` was generalized so future OAuth-MCP read-only
brokers (E*TRADE, other US/India) are added by a CONFIG ENTRY, not new code:
- `lib/brokers/mcp-registry.ts` – `McpBrokerConfig` type + `MCP_BROKERS` registry
 (id/label/market/currency, mcpUrl, oauth endpoints+scopes, vaultProvider+keys,
 tool names, and response field-name candidates). Seeded with the **webull** entry
 (same endpoints/scopes/tools/vault keys – behaviour-neutral).
- `lib/brokers/mcp-driver.ts` – the generic driver: every fn takes an
 `McpBrokerConfig` – `getOrRegisterClient` / `buildAuthUrl` / `exchangeCode` /
 `getValidAccessToken` (CAS refresh) / `hasToken` / `mcpRpc` / `captureAccounts`
 (driven by cfg.tools + cfg.fields → `BrokerAccount[]`) / `checkTokenHealth` /
 `disconnect` / `saveOAuthState` + `consumeOAuthState` (keyed by broker id).
 Reuses `makePkce/makeState/signOAuthCookie/verifyOAuthCookie/mcpToolJson` from
 `lib/robinhood-mcp.ts`.
- Generic routes `app/api/broker-mcp/[broker]/{login,callback,status,disconnect}`
 resolve the config from `MCP_BROKERS[broker]` (404 if unknown). Login/status/
 disconnect owner-gated; callback verifies the single-use server-side state.
- Settings iterates `MCP_BROKERS` → one connect card per broker (future brokers
 auto-render). The legacy `app/api/webull/*` routes are thin 307 redirects to the
 generic ones (preserving the DCR-registered `api/webull/callback` redirect_uri).
- `lib/webull-mcp.ts` was **deleted** – nothing imported it after the repoint.

Phase 1 built (read-only)
- OAuth 2.1 (DCR + PKCE S256, hosted Vercel callback) + CAS token refresh (vault
 keys `WEBULL_MCP_*`, provider `webull_mcp`) + JSON-RPC client + `captureAccounts()`
 (get_account_list → get_account_positions + get_account_balance → get_stock_quotes),
 now via the generic `lib/brokers/mcp-driver.ts` + `MCP_BROKERS.webull`. Scope
 `account:read market:read instrument:read` ONLY. If the connected MCP surface
 omits `get_stock_quotes`, capture fails soft rather than fabricating prices.
 No `order:write` scope is requested by the login route.
- Legacy routes `app/api/webull/{login,callback,status,disconnect}` → 307 redirects
 to `app/api/broker-mcp/webull/{login,callback,status,disconnect}`.
- Settings connect card (generic, driven by the registry). `BrokerName` union + `webull`.
- **Owner action:** already connected once through cloud OAuth. Verify holdings
 appear after the next live snapshot, and investigate only read-tool gaps.
- **Phase 2 (later):** order adapter (`order:write` + preview→place→status→cancel)
 behind an explicit per-account allowlist + all existing gates. Code exists only
 as an inert adapter scaffold and is not reachable while `orderCapable=false`,
 `webull_orders_enabled` is absent/false, and no Webull trading allowlist row is
 configured.

Confirmed integration facts (from public OAuth metadata + Webull MCP docs)

Cloud MCP endpoint: `https://api.webull.com/mcp` (JSON-RPC, MCP spec).
Auth = OAuth 2.1 + PKCE(S256) + dynamic client registration – identical shape
to the Robinhood MCP already implemented in `lib/robinhood-mcp.ts`, so those
helpers are reused with Webull's endpoints:
- authorize: `https://passport.webull.com/oauth2/ai-mcp/login`
- token: `https://uisuserauth.webullfintech.com/api/userauth/oauth/token/token`
- register: `https://uisuserauth.webullfintech.com/api/userauth/oauth/client/register`
- public client (`token_endpoint_auth_method: none`), grants `authorization_code` + `refresh_token`
- scopes: `account:read order:read order:write market:read instrument:read`

Tool schema status: partly verified, order placement unverified.

Current connected tool exposure in Codex on 2026-07-13 included:
`get_account_list`, `get_account_balance`, `get_account_positions`,
`get_order_detail`, `get_instruments`, `get_open_orders`, `get_order_history`.
It did **not** expose `get_stock_quotes`, `get_stock_snapshot`,
`preview_stock_order`, `place_stock_order`, `cancel_order`, or the analyst/
financial research tools. The table below is therefore a desired/previous-doc
mapping, not proof that orders can be placed from this app today.

Need	Webull MCP tool
accounts	`get_account_list`
balance / buying power	`get_account_balance`
holdings	`get_account_positions`

```

```
quotes (pricing)	`get_stock_quotes` / `get_stock_snapshot`
review order	`preview_stock_order`
place order	`place_stock_order`
order status	`get_order_detail`
open orders	`get_open_orders`
cancel	`cancel_order`
```

Only the account/balance/position/order-read portion is currently verified in this Codex connection. Quotes and order placement must stay fail-soft/unreachable until a deployed `tools/list` proves the tools and schemas.

## ## Goal

Add Webull as a third broker – US equities, parallel to Robinhood – using the existing broker abstraction (`BrokerAdapter` + registry). Same safety stack; the adapter only executes, every gate sits above it.

## ## Why it fits cleanly

`lib/brokers/adapters/types.ts`: "adding an execution broker = one new file implementing this interface + one registry entry. Zero route/UI changes." Webull is US (like Robinhood), and it's an MCP – so the **Robinhood-MCP** integration is the template, reused almost verbatim with a different endpoint + tool names.

## ## Build (phased – read-only first)

### ### Phase 0 – Discovery (interactive; blocks the rest)

The exact OAuth endpoints + tool names come from the live server, not guesswork. Standard MCP auth = OAuth 2.1 + dynamic client registration via a

`well-known/oauth-authorization-server` discovery doc – the **same** flow the RH integration already implements\* (`lib/robinhood-mcp.ts`: DCR → PKCE S256 → token in vault → CAS refresh). Steps:

1. Connect the Webull MCP (interactive OAuth) – owner authorizes.
2. Call `tools/list` to capture the real tool names (accounts / positions / quotes / review-order / place-order / order-status / cancel).
3. Pin those into the client. No adapter code is written against guessed names.

### ### Phase 1 – Read-only (dashboard)

- `lib/webull-mcp.ts` – generic `webullRpc(token, method, params, session)` (copy of RH's `mcpRpc`, endpoint `https://api.webull.com/mcp`) + `captureWebullAccounts()` → `BrokerAccount[]`.
- `broker\_accounts` row(s): `broker='webull'`, `market='us'`, `account\_role`, `enabled`, `notional\_cap\_usd`, `agentic\_allowed=false` (reads only).
- Wire into the live account book (like the 6 RH accounts): holdings, risk, the live-vs-V00 chart, `live\_account\_snapshots`.
- OAuth routes `/api/webull/login` + `/api/webull/callback` – reuse the RH OAuth helpers (dynamic client reg, PKCE, signed state cookie, vault token, CAS refresh). Owner-gated login; callback verifies the state nonce.
- Settings: Webull connect card + account list (mirror the RH-MCP card).

### ### Phase 2 – Order-capable (only if opted in)

- `lib/brokers/adapters/webull.ts` is an inert adapter scaffold. It must not be enabled until live tool discovery proves preview/place/status/cancel schemas, Webull order scopes are intentionally requested, and the owner approves.
- An account becomes order-capable **ONLY** when you explicitly allowlist it (`broker\_accounts{broker='webull', market='us', role='trading'}`) and the broker registry marks Webull order-capable.
- Every existing gate applies unchanged: `isTradingEnabled(svc, 'us')`, per-market controls, kill switch, notional caps, human click + confirm.

## ## Safety (unchanged)

- Read-only first; orders are a separate, opt-in phase.
- One allowlisted account for orders; all others read-only.
- Credentials never in code/logs – vault only; owner enters/authorizes.
- Same fail-closed order path as RH (dup-index, durable ACK, reconcile).

## ## Two decisions needed to start

1. **Scope**: read-only (holdings on the dashboard) first – recommended – or go straight to order-capable?
2. **Discovery**: how do I get its tool schema – (a) you add the Webull MCP as a connector to this app so I can `tools/list` it, or (b) you paste the Webull MCP auth + tool docs? (I can't run the interactive OAuth from here.)

---

# Source: features/webull-trading-api/FEATURE\_ARCHITECTURE.md

# Webull Trading API Adapter

> Status: transport implementation approved 2026-07-19; disabled and not approved for activation. US live money path.

## ## 2026-07-19 Transport Restoration Decision

The signed HTTP sender may be implemented now, but remains unreachable from the broker registry and must not make a real request during build or test. The activation flag stays false and the adapter continues to report unconfigured.

The current `api\_key\_vault` schema has no environment column. Environment

isolation therefore uses three distinct key names per environment under ``provider='webull_trade': `WEBULL_TRADE<ENV>_APP_KEY`, `WEBULL_TRADE<ENV>_APP_SECRET`, and `WEBULL_TRADE<ENV>_ACCESS_TOKEN`. This is the existing vault convention and avoids an unapproved schema migration.`

There is one network implementation and two capabilities over it:

- A one-shot `**preflight permit**` reads and requires the false-by-default database feature flag and can call only the official token-check, account-list, or order-preview paths. It cannot place, query, or cancel an order. This resolves the unavoidable ordering dependency: live token status must be known before gate 7 can pass.
- A one-shot `**order permit**` is created only from a full ``GateSnapshot`` for which all nine gates pass. It permits only the fixed order endpoint allowlist and is consumed by the first request, preventing stale gate clearance from becoming a reusable transport handle.

The current hardcoded account gate names the production account only, so order permits are production-only. Sandbox token/account/preview proof is supported, but a sandbox place/cancel proof remains blocked until its exact test account has a separately approved allowlist gate; production account identity must never be reused as a proxy for a sandbox account.

Both capabilities use the same ``liveWebullTransport()`` sender and the same sole ``fetch`` call. Environment and host are pinned to the credential; signing uses the hostname without the URL scheme, as required by Webull. Every request gets a new nonce and timestamp, a 10-second default timeout, the exact signed body bytes, and ``x-access-token``. HTTP and network failures return fixed, redacted ``TransportResult`` errors and never throw credential material.

The official token-status endpoint is ``POST /openapi/auth/token/check``, not the previously proposed ``GET /openapi/trade/token/query``. A successful ``NORMAL`` response produces the fresh ``WebullTokenRecord`` consumed by gate 7. No keepalive schedule is added.

### Sandbox proof sequence (not yet authorized)

1. Provision sandbox-only vault keys and keep the production keys absent.
2. With the database feature flag enabled only for the supervised proof window, call token-check, then ``GET /openapi/account/list``. Pass only if status is ``NORMAL``, exactly the expected test account is returned, and no production account identifier appears.
3. Call ``POST /openapi/trade/order/preview`` with a one-share US equity limit fixture. Pass only if the broker accepts the account/order shape and returns a non-mutating estimate.
4. Add a separately approved exact sandbox-account gate before enabling any sandbox place/cancel capability. The production account constant is not valid proof of sandbox identity.
5. With owner approval, submit one non-marketable one-share limit order that is inside Webull's accepted price bands, query it, cancel it, and query the terminal state. Do not use ``$0.01``: a broker price-band rejection proves only rejection handling, not place/query/cancel reconciliation.
6. Pass only if one client order id maps to one broker order, no fill occurs, the cancel becomes terminal, the audit/reconciliation record is complete, and no secret appears in logs. Any timeout or unknown state stops the sequence for reconciliation; it is never resubmitted.

## Resolved Boundary

Webull exposes two different products:

- `**Cloud MCP**` is an AI-friendly, query-only surface. Kairos may use it for account, position, balance, and quote reads. It has no configured order scopes or write tools and ``orderCapable`` remains false.
- `**Trading API**` is a separately entitled, signed REST/gRPC integration. It is the only candidate for future Webull order placement.

The two transports, credentials, scopes, and registries must never be combined. The existing ``webull`` MCP adapter remains read-only. A future trading adapter gets a distinct id such as ``webull_trade``.

## Verified Trading API Facts

- Individual applications require Webull approval and API credentials.
- Access tokens are reusable; 2FA (when enabled) is verified via the Webull app ONCE at token acquisition, NOT a secret pasted into each order – so autonomous cron execution is technically feasible (owner Q1 RESOLVED).
- The official token lifecycle documents ``INVALID`` after `**15 consecutive days without an API call**`, ``EXPIRED`` when initial verification is not completed within 5 minutes, and reusable ``NORMAL`` tokens. Kairos tracks the last confirmed authenticated call, checks token status before order activity after an idle interval, and alerts before the 15-day boundary. It must not generate meaningless keepalive traffic merely to hide an unused credential. Invalid, expired, or unknown status fails closed before submission.
- Requests use Webull's documented signed authentication protocol, including its canonical request and HMAC-SHA1 signature. Do not substitute generic OAuth or

- a vendor SDK assumption.
- Stock order endpoints support market/limit/stop variants, client order ids, order query/cancel/replace, and documented rate limits. The Trading API also documents richer types – GTC stop-market/stop-limit, OTO/OCO/OTOCO, attached take-profit/stop-loss, fractional, extended/overnight sessions, and TWAP/VWAP/POV algos. **\*\*Documented ≠ in scope.\*\*** These are pinned to fixtures only if/when a later phase needs them; the initial adapter must NOT expose them.
- **\*\*Initial order-type scope (locked small):\*\*** market + limit for manual-approved entries, and **\*\*GTC `STOP\_LOSS` in `CORE` as the disaster floor only\*\*** (the hybrid-stop feature). OCO/OTOCO come only after the basic place/query/cancel/reconcile lifecycle is proven. Trailing stops, shorts, options, algos, and automatic venue routing are excluded outright.
- Trailing-stop stock orders are documented with DAY time-in-force only – they cannot rest multi-day, so they are NOT a substitute for Kairos's exit policy and stay out (adopting broker-native trailing would also make exits vary by venue and contaminate the Learner – see `features/hybrid-stop`).
- Webull fills the conditional `BrokerProtectiveCapabilities` matrix from `features/hybrid-stop`. Initial fixtures declare only US equity `SELL` protection using GTC `STOP\_LOSS` in `CORE`. The generic presence of `ALL`/`NIGHT` sessions elsewhere in the API does not prove a stop order can trigger there.
- Sandbox and production are separate environments and credentials.

Exact fields and limits must be pinned from the current official documentation in contract fixtures immediately before implementation because this is a live broker surface and may change.

## ## Recommendation

Do not enable Webull trading now. Build only after the existing paper and live decision loops, router cutover, and protective-order policy are stable. When built:

- Webull is an additional US live venue, not a new strategy.
- Deterministic Kairos logic remains authoritative.
- Native trailing stops stay out until a separately versioned strategy experiment proves broker-dependent exit semantics are acceptable.
- Long-only is enforced in both the gateway and adapter; `SHORT` is rejected.

## ## Credential and Transport Design

- Store production and sandbox credentials as separate encrypted vault records.
- Never expose app secret, access token, refresh material, account id, signature, or canonical request in logs, errors, UI, or model context.
- Fetch credentials through a server-only accessor with bounded in-memory caching.
- Pin endpoint environment to the credential record. A sandbox credential can never resolve a production host.
- Implement the official signing algorithm directly with golden fixtures from the documentation. Enforce TLS, timestamp skew, and a fresh nonce per request; redact the canonical request before any telemetry boundary because it can contain account and order data even when the secret itself is absent.
- Use a bounded timeout and retry only requests proven idempotent.

## ## Adapter Contract

Create a distinct `webull\_trade` adapter implementing:

- account discovery and exact allowlist match
- position, cash, buying-power, order, and fill reconciliation
- preview when supported by the entitled surface
- place, query, modify, and cancel with a stable `client\_order\_id`
- normalized partial-fill and unknown-state handling
- provider rate-limit classification and health reporting

It writes only through the existing execution kernel and execution ledger. It may not create a parallel trade, cash, position, or P&L store.

The current `lib/brokers/adapters/webull.ts` is an inert scaffold for MCP-derived order tools that do not exist. During implementation it must be deleted or replaced, not enabled and not left as a second Webull order adapter. The read-only Cloud MCP registry may remain under `webull`; signed execution uses only `webull\_trade`.

## ## Mandatory Gate Ladder

Every order requires all gates in this order:

1. Global trading enabled and app not paused.
2. US market control enabled and kill switch clear.
3. Drawdown/circuit breakers clear; exits remain possible when entries halt.
4. Live autonomy/approval mode satisfied.
5. Exactly one enabled Webull trading account allowlisted for US.
6. Explicit `webull\_trade\_orders\_enabled` false-by-default feature flag.
7. Valid `NORMAL` production token when 2FA is enabled, valid signing credential, expected endpoint, and acceptable timestamp skew.
8. Existing buying-power, notional, name, sector, gross, turnover, and duplicate order checks.
9. Deterministic quantity no greater than the reconciled mandate allowance.



Missing schema, config, account, credential, or broker response fails closed.

## ## Order Lifecycle

1. Reserve an execution intent using the existing idempotency mechanism.
2. Reconcile account and held quantity immediately before submission.
3. Build a normalized order and reject short/options/unsupported sessions locally.
4. Sign and submit once with a stable client order id.
5. A timeout or response without an order id becomes `needs\_reconcile`; never report success and never blindly resubmit.
6. Reconcile order status and fills until terminal.
7. Update the existing execution and position truth using actual fills.

## ## Failure and Security Tests

1. Cloud MCP configuration contains no write scope or order tool.
2. Every missing gate blocks before network submission.
3. Two allowlisted accounts fail closed as ambiguous.
4. `SHORT`, options, and unsupported order/session combinations are rejected.
5. Sandbox credentials cannot call production and vice versa.
6. Official signature fixtures match exactly; mutated method/path/body fail.
7. Secret capture assertions cover logs, errors, traces, and health events.
8. Reusing a client order id cannot create a second order.
9. Timeout and malformed response enter reconciliation, not retry-to-place.
10. Partial fills and external cancellations preserve correct held quantity.
11. US pause, kill, drawdown, approval, and notional gates are regression-tested.
12. India state is unreachable from the adapter.
13. A token idle for 15 days, unknown token status, timestamp replay, or nonce reuse blocks before order submission.
14. GTC `STOP\_LOSS` is rejected outside the exact session/product combinations proven by current contract fixtures.

## ## Open Owner Decisions

1. Does Webull replace Robinhood for US live execution or remain an optional second venue? Recommendation: optional manual venue first; no automatic routing.
2. Which approved individual Webull account is allowlisted?
3. Is the required Webull API entitlement active for that account?
4. After sandbox proof, authorize one manually approved minimal live order?

## ## Build Order After Approval

1. Confirm entitlement and capture current official schemas/rates/signing fixtures.
2. Add vault records and secret-hygiene tests.
3. Build read/reconciliation in sandbox with no place permission.
4. Build signed order operations behind all gates, still disabled.
5. Run fault injection and full money-path regression tests.
6. Owner reviews sandbox ledger and explicitly approves one minimal live test.
7. Keep autonomous Webull routing disabled until a separate production sign-off.

## ## Primary Sources

- Authentication and reusable 2FA token overview: <https://developer.webull.com/apis/docs/authentication/overview/>
- Token lifecycle and 15-day inactivity rule: <https://developer.webull.com/apis/docs/authentication/token/>
- Stock order lifecycle, types, TIF, and sessions: <https://developer.webull.com/apis/docs/trade-api/stock/>
- Cloud MCP query-only tool inventory: <https://developer.webull.com/apis/docs/AI-friendly-Resources/mcp/>

---

# Source: features\weekend-research-catchup\FEATURE\_ARCHITECTURE.md  
# Market-Closed-Day Research Catch-up and Agent Capacity

Status: APPROVED by owner on 2026-07-19 ("do it"; holiday extension approved with "ok")

## ## Problem

The US and India research queues can contain more symbols than one market-session ResearchAgent run can finish. Provider quotas reset on weekends and full exchange holidays, but warming alone does not finish scoring, write a safely reusable staged result, or reduce the next-session scoring workload.  
The app also does not show queue depth, throughput, or estimated time to clear.

## ## Decision

Use market-closed-day quota for full deterministic research catch-up, but make every result non-executable until the same market completes a fresh session revalidation. Keep paper and live trading session-only. Display honest backlog and capacity telemetry for queue-backed and workload-backed agents.

## ## Signal State Machine

stateDiagram-v2

[\*] --> weekend\_staged: Closed-day catch-up scores symbol

weekend\_staged --> superseded: Newer closed-day score replaces it

weekend\_staged --> revalidated: Session ResearchAgent successfully re-scores symbol

revalidated --> [\*]

[\*] --> pending: Normal session-validated research score

pending --> paper\_traded: PaperTrader fills

pending --> expired: Trading-day freshness expires

pending --> rank\_rejected: Cross-sectional gate rejects entry

`weekend\_staged` is retained as the legacy database status name. Its semantic meaning is now any supported full market-closure day. Every such row has `session\_validated=false` and an `as\_of\_session` equal to the last completed market session. A normal weekday score has `session\_validated=true`. A successful weekday score marks older staged rows for that market/symbol `revalidated`; the newly inserted `pending` row is the only entry candidate.

#### ## Downstream Behavior

- **PaperTrader**: positive allowlist requires `pending`, `deterministic\_v1`, and `session\_validated=true`. It never fills staged rows.
- **TraderAgent/live proposals**: same positive allowlist. It never creates a proposal from a staged row.
- **AutonomousShadow / AutonomousLive**: their direct recent-signal queries also require `session\_validated=true`; weekday scheduling and the 24-hour lookback are not treated as sufficient proof.
- **PositionMonitor**: mechanical price stops, targets, trailing stops, and time stops remain independent. Score/direction exits may only read `session\_validated=true` signals; staged scores cannot force a sale.
- **LearnerAgent**: staged rows remain measurement evidence, but cannot become filled-trade outcomes. Existing observation-ledger rules remain unchanged.
- **ResearchAgent weekday run**: holdings remain first-class and cap-exempt. A successful re-score produces the fresh decision for that session. A failure leaves the staged row informational only.
- **CapitalRotation**: latest-score lookup requires session validation, so a staged score cannot change weakest-holding selection or a rotation edge.

#### ## Closed-Day Scheduling

Schedule one bounded catch-up trigger per market every day after that market's prewarm window. A shared exchange-local calendar permits work only on weekends or verified full equity-market holidays. Normal trading days self-skip. An unsupported calendar year and special sessions such as NSE Muhurat Trading both abstain. An unsupported year opens one deduplicated System Health warning until the annual calendar is installed. The route uses the existing research wall-clock budget, provider pacing, candidate cap, per-market scope, and idempotency guard. It does not chain PaperTrader or TraderAgent.

The 2026 US calendar is sourced from NYSE. The 2026 India calendar is sourced from NSE Capital Market circular CMTR71775 plus amendment CMTR72260. Settlement, currency, debt, and commodity holiday lists are not interchangeable with the equity-market calendar. Early-close days are trading days, not catch-up days.

Closed-day candidate scores are retained in `research\_queue` for next-session revalidation. Held symbols do not need queue retention because holdings are always gathered. Repeated closed-day runs supersede the prior staged row rather than creating multiple active staged decisions.

#### ## Data Model

Add to `agent\_signals`:

- `session\_validated` boolean not null default true
- `as\_of\_session` date null
- `staged\_at` timestamptz null
- CHECK: `status='weekend\_staged'` implies `session\_validated=false`
- one active `weekend\_staged` row per `(market,symbol)`

No order, position, portfolio, or broker schema changes.

#### ## Agent Capacity View

The Agents dashboard gets a **Capacity** tab, scoped by the global market switcher. Each row declares its workload type so a workload is not mislabeled as a durable queue:

- **ResearchAgent**: `research\_queue` depth, oldest age, staged count, recent median successful throughput/day, and estimated clearing days.
- **PaperTrader**: fresh validated pending longs; configured per-run selection cap.

- PositionMonitor: currently open paper positions; all are expected per run.
- LearnerAgent: report unavailable when a defensible queue cannot be derived; never fabricate a backlog.

Capacity uses the median of recent completed per-market runs where possible. Environment caps are shown as configured ceilings, not achieved throughput. ``estimated_days = ceil(backlog / max(1, observed_daily_capacity))``; null is shown when no defensible estimate exists.

### ## Failure Rules

- Missing migration makes the new positive eligibility query return no entry candidates (fail closed).
- Closed-day scoring failure re-defers the symbol and writes the normal run error.
- Queue telemetry failure shows unavailable; it never blocks any agent.
- A staged row can never be mutated into ``pending``; revalidation writes a fresh row, preserving point-in-time history.
- US and India remain fully separate. No queue, throughput, score, or estimate is cross-summed.

### ## Acceptance Criteria

1. Weekend and verified full-holiday catch-up writes ``weekend_staged`` signals and no trade/proposal.
2. PaperTrader and TraderAgent reject staged/unvalidated signals even at score 100.
3. PositionMonitor ignores staged scores for score/direction exits.
4. Next-session success writes a new validated signal and closes prior staged state.
5. Failed session revalidation cannot make the staged result executable.
6. Closed-day candidate remains queued for session revalidation without increasing its failed-attempt count.
7. Capacity UI is market-scoped and labels queue versus workload honestly.
8. Existing weekday research-to-paper flow remains unchanged for validated rows.

### ## Disable / Rollback

Unschedule the two closed-day catch-up crons. Existing staged rows remain inert and may be marked ``superseded``. The additive columns can stay; their defaults preserve the pre-feature weekday behavior.

---

# Source: features\weekend-research-catchup\IMPLEMENTATION\_RESULT.md  
# Market-Closed-Day Research Catch-up – Implementation Result

Completed: 2026-07-19

### ## Shipped behavior

- US and India catch-up triggers run daily after prewarm and self-skip unless the exchange is closed for a supported weekend or full holiday.
- Catch-up reads only each market's carry-forward ``research_queue``.
- Scores are written under the legacy ``weekend_staged`` status with ``session_validated=false`` and the last completed market session.
- Successfully staged candidates remain queued for next-session revalidation.
- A session success writes a new validated row and retires the staged row.
- PaperTrader and TraderAgent require positive session validation.
- AutonomousShadow, AutonomousLive, and CapitalRotation direct signal reads require the same positive validation proof.
- PositionMonitor ignores staged scores for conviction exits; mechanical exits remain active and unchanged.
- Agents → Capacity reports queue/workload type, pending preparation, staged work, recent median throughput, configured ceiling, oldest age, and estimated clearing days separately for US and India.

### ## Production proof

Supabase project ``<production-project-ref>``:

- ``agent_signals.session_validated`` boolean not null default true
- ``agent_signals.as_of_session`` date
- ``agent_signals.staged_at`` timestampz
- ``agent_runs.workload_metrics`` jsonb
- validated ``agent_signals_weekend_stage_unvalidated`` CHECK
- partial unique ``agent_signals_one_weekend_stage_per_symbol``
- active ``kairos-closed-day-research-us`` at ``10 15 * * *``
- active ``kairos-closed-day-research-india`` at ``10 5 * * *``
- old ``kairos-weekend-research-*`` jobs absent
- migration tracker contains ``market_holiday_research_catchup``
- 2026 calendar sources: NYSE trading calendar; NSE Capital Market circular CMTR71775 plus amendment CMTR72260
- all 538 pre-existing signals remained ``session_validated=true``

Live Sunday run after deployment:

- US: 26 queued candidates processed, 26 staged, 0 failed

```

- India: 4 queued candidates processed, 4 staged, 0 failed
- every staged row: `as_of_session=2026-07-17`,
 `session_validated=false`
- queues remained US 26 / India 4 for the next session revalidation
- zero PaperTrader runs, zero paper trades, zero trade proposals, and zero
 `session_validated=false AND status='pending'` rows after the run

Verification

- `npx tsc --noEmit`: pass
- `npx vitest run`: 1,106 pass, 6 skip
- `npm run build`: pass; fresh `.next/BUILD_ID` generated
- `ggshield secret scan pre-commit`: no secrets
- Vercel deployment for `bc4bcb3a`: Ready before cron migration applied

No live order, paper fill, position mutation, Router activation, provider-policy
change, or cross-market aggregation was performed by this implementation.

Source: features\zerodha-coin-portfolio\FEATURE_ARCHITECTURE.md
Zerodha Coin portfolio sleeve

Status: APPROVED by owner (`go`, 2026-09-03)

Decision record

- Why: Kairos currently omits the owner's Coin mutual-fund assets, understating the visible India portfolio and hiding fund concentration.
- Who: The single owner using the existing Zerodha Kite Connect session.
- ROI: Complete read-only India account visibility without expanding the trading or stock-scoring surface.
- Shipped at: 2026-09-03 (code complete; live account read awaits the owner's next Kite login because the stored daily token is expired).

Scope

Use the existing daily Kite access token to read `GET /mf/holdings`. Normalize the response into a separate Coin contract and return it alongside the existing `GET /api/kite/holdings`.

The existing `nav` field remains equity cash plus equity holdings for compatibility. A new `combined_nav` is returned only when equity NAV exists.

Non-goals

- No mutual-fund purchase, redemption, SIP, or other order endpoint.
- No Coin holding enters ResearchAgent, stock scoring, entry eligibility, stops, targets, ladders, or live/paper equity performance.
- No constituent look-through, category benchmark, historical NAV reconstruction, or fund recommendations in this version.
- No database schema or persisted credential change.
- No claim that the Upstox/NSE equity universe is the Zerodha or Coin instrument master.

Contracts

`lib/brokers/coin.ts` owns deterministic parsing and aggregation. Invalid quantities are rejected. Missing NAV or cost stays `null`, never zero.

`GET /api/kite/portfolio` remains owner-gated and adds:

coin: {
 available: boolean;
 holdings: CoinHolding[];
 holding_count: number;
 valuation_complete: boolean;
 total_invested: number | null;
 total_value: number | null;
 total_pnl: number | null;
 error?: string;
};

combined_nav: number | null;

UI and failure states

The existing full-width India Live Portfolio page gets a separate read-only Coin section. It distinguishes loading, reconnect/unavailable, success, and error states.

The page states that Coin NAV is end-of-day/last available and that existing performance, risk, research, and order controls remain equity-only.

Acceptance criteria

1. The same fresh Kite session reads both equity and Coin holdings.

```

2. Coin failure never blanks equity holdings or changes the legacy equity `nav`.
3. Empty Coin holdings are not reported as a connection failure.
4. Missing NAV prevents combined totals rather than becoming zero.
5. No Coin write endpoint or order control exists.
6. Parsing tests, API contract tests, full test suite, typecheck, and production build pass before shipping. Owner-authenticated live-data ver

---

# Source: SYSTEM\_OVERVIEW.md  
# Kairos System Overview

> Last reviewed: 2026-08-06 (architecture-to-production conformance baseline)  
> Audience: an engineer or operator who needs a quick orientation before reading the  
> canonical architecture chapters.

Kairos is a personal investing research system for separate US/USD and India/INR books. It runs research and paper-trading workflows autonomously within fixed policy boundaries, records outcomes, and uses governed evidence to evaluate future changes. Live broker behavior remains separately controlled and safety-gated.

## The lifecycle

flowchart LR

DATA["Market and broker evidence"] --> RESEARCH["Research and deterministic scoring"]

RESEARCH --> SIGNALS["Market-local signal ledger"]

SIGNALS --> PAPER["PaperTrader"]

PAPER --> MONITOR["PositionMonitor"]

MONITOR --> OUTCOMES["Outcomes and performance truth"]

OUTCOMES --> LEARNING["Learner and validation"]

LEARNING --> RESEARCH

This is an orientation diagram, not the live topology. The rendered agent map in Dashboard -> Agents is the single source for agent-to-agent edges and history.

## What keeps the system safe

- US/USD and India/INR pools remain isolated.
- LLM output is advisory and explainable; deterministic code owns financial state, eligibility, pricing, risk checks, and broker execution.
- Provider degradation, stale evidence, and ambiguous order state are treated as explicit conditions rather than silently fabricated data.
- Shadows and historical replay are observation systems until their own evidence and approval gates are satisfied.
- Research first classifies economic exposure and instrument family. Substitute vehicles count as one idea in evaluation, and family-specific evidence remains shadow-only until separate US/India validation and owner promotion.
- The exogenous-risk foundation records future India domestic and global-spillover evidence only. It has no source adapter or decision consumer; missing evidence is unavailable rather than neutral.
- Live actions are subject to the active autonomy, market-control, kill-switch, mandate, and broker-account controls.

## Where to go next

The complete architecture is not repeated here. Use the [Architecture Portal](ARCHITECTURE.md) for the documentation hierarchy, then begin at [docs/arch/00-index.md](docs/arch/00-index.md). For a particular feature, its `features/<feature>/FEATURE\_ARCHITECTURE.md` is the micro-architecture; [PROJECT\_DECISIONS.md](PROJECT\_DECISIONS.md) records why approved choices were made.

When this overview and a chapter disagree, the chapter wins for subsystem behavior; when a chapter and the live diagram disagree about topology, the diagram source and its history must be reconciled in the same change.

The current conformance baseline is recorded in [docs/audits/2026-08-06-architecture-production-conformance.md](docs/audits/2026-08-06-architecture-production-conformance.md). It is the place to check whether a design is active, dormant, shadow-only, or only a proposal before relying on it operationally.

This overview is updated only when the reader's orientation changes. A provider, schema, schedule, money-path, or feature-state change updates its owning chapter and feature implementation result in the same commit; see the contract in [ARCHITECTURE.md](ARCHITECTURE.md).

---

# Source: ARCHITECTURE.md

# Kairos Architecture Portal

> Last reviewed: 2026-08-06 (architecture-to-production conformance baseline)  
> Purpose: the stable entry point to Kairos architecture. It explains where truth  
> lives; it does not duplicate the implementation detail held by the documents it links.

## What Kairos is

Kairos is a personal US and India investing research system. It keeps USD and INR books separate, researches instruments, executes an autonomous paper loop, observes outcomes, and permits only explicitly governed live actions. LLMs may summarize or propose; deterministic services own eligibility, pricing, risk controls, accounting, and broker execution.

## Non-negotiable system properties

1. The US and India markets are separate pools. Values, returns, risk, and decisions are never cross-summed across currencies.
2. An LLM is never a money-path authority. It cannot create an executable order, override a safety gate, or promote a strategy.
3. A missing, stale, ambiguous, or degraded required input fails closed for the decision it protects. Exits retain their independent safety path.
4. Paper behavior is evidence, not proof. Research, shadow, replay, validation, and promotion are distinct lifecycle states.
5. Broker credentials, account identifiers, and raw provider responses do not belong in client bundles or reference documentation.
6. Daily technical scoring consumes completed regular-market sessions only, and scheduled agent slots are expressed in exchange-local time rather than assumed UTC.

## Documentation hierarchy

| Source | Owns | Do not use it for |  
|---|---|---|  
| `ARCHITECTURE.md` | This portal, source-of-truth rules, cross-cutting principles | Detailed schedules, schemas, or current provider behavior  
| `SYSTEM\_OVERVIEW.md` | Short onboarding explanation and high-level lifecycle | Operational implementation detail |  
| `docs/arch/` | Definitive chapter-by-chapter operational architecture | Feature alternatives and implementation history |  
| `features/<feature>/FEATURE\_ARCHITECTURE.md` | Feature micro-architecture: scope, contracts, states, alternatives, rollout, and non-goals |  
| `features/<feature>/IMPLEMENTATION\_RESULT.md` | What was actually delivered and any approved deviations | Future design authority |  
| `PROJECT\_DECISIONS.md` | Approved decisions, rationale, alternatives, and reversal cost | Unapproved proposals |  
| `public/agent-diagrams/system-map.json` | Rendered live agent-to-agent topology and its history | Narrative safety sequences or data-model

Start with [the chapter index](docs/arch/00-index.md). Read a feature document before changing a scoped feature. Read the decision record before reopening an approved choice.

The current evidence-backed conformance baseline is recorded in  
[docs/audits/2026-08-06-architecture-production-conformance.md](docs/audits/2026-08-06-architecture-production-conformance.md).  
It distinguishes active behavior from dormant scaffolds and design-only proposals.

## Architecture views

- **System orientation:** [SYSTEM\_OVERVIEW.md](SYSTEM\_OVERVIEW.md)  
- **Agent responsibilities:** [docs/arch/03-agents.md](docs/arch/03-agents.md)  
- **Schema and data contracts:** [docs/arch/04-database-schema.md](docs/arch/04-database-schema.md)  
- **Schedules:** [docs/arch/05-crons-and-scheduling.md](docs/arch/05-crons-and-scheduling.md)  
- **Safety and money path:** [docs/arch/08-risk-and-safety.md](docs/arch/08-risk-and-safety.md)  
- **Learning and promotion:** [docs/arch/09-learning-loop.md](docs/arch/09-learning-loop.md)  
- **Live topology:** Dashboard -> Agents -> the "Agent & Flow Architecture" section.

The topology is rendered from the JSON source above. Do not paste a second copy of its edges into a chapter. A chapter diagram is appropriate only when it explains a different altitude, such as a gate sequence, an event timeline, or a subsystem flow.

## Change protocol

Every meaningful change follows this sequence:

1. Record the decision or create/update the feature architecture before implementation.
2. Implement only the approved scope, with tests proportional to risk.
3. Update every affected architecture chapter in the same change.
4. Update the live agent diagram when an agent's inputs, outputs, dependencies, or safety boundary changes. Update its history entry with the reason.
5. Add an implementation result when a feature architecture is shipped; record an approved deviation rather than silently editing history.
6. Verify the rendered diagram and architecture drift tests before shipping.

The top-level overview and index are not rewritten for every implementation detail. They are updated when the reading path, system orientation, or documentation ownership changes. The owning chapter and feature implementation result are mandatory whenever a change affects their contract. A completed `WORK\_LOG.md` entry is not evidence that a feature shipped until those required records and production migration state agree.

For a scoring-dimension change, "update the architecture" specifically means all of the following must agree with production code in the same commit:

- plain-English meaning: what question the dimension is trying to answer;

- source order, market applicability, freshness, and fallback behavior;
- exact formula, constants, thresholds, clamps, and any veto/cap;
- missing-data behavior and whether the dimension is excluded or merely displayed;
- whether the value can affect eligibility, sizing, exits, learning, or only explanation.

The definitive home for those facts is `docs/arch/03-agents.md` under ResearchAgent. Provider mechanics belong in chapter 02, persisted fields in chapter 04, and money-path consequences in chapter 08. A feature document may explain why a change was proposed, but it must not become the only place where the active formula is documented.

Changes to a money path additionally require the applicable risk chapter, schema chapter (when persistence changes), migration verification, and an explicit fail-closed review. A feature is not complete merely because code compiles.

## In-app reference surface

Dashboard -> Agents -> **System Reference** exposes a curated owner-only allowlist of these records for reading or download. It is deliberately not a general repository browser and does not add a main-navigation "Docs" destination. The agent page remains the appropriate contextual home because it already owns the live topology.

---

# Source: PROJECT\_DECISIONS.md  
# Project Decisions â€” Kairos

## Purpose

Records approved product, architecture, UX, technical, and business decisions.

Only approved decisions belong here. Proposed decisions live in PROJECT\_BUILD\_LOG.md until approved.

## Decision Template

## Decision <N>: <Title>

Date:

Status: Approved

Category: Product / UX / Architecture / Technical / Business / Data / Security

Context:

Decision:

Reason:

Alternatives considered:

Impact:

Files/features affected:

Reversal cost: Low / Medium / High